

Oracle® Database

High Availability Overview and Best Practices



19c
F23691-56
July 2026

ORACLE®

Copyright © 2005, 2026, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=docacc>.

Oracle customer access to and use of Oracle support services will be pursuant to the terms and conditions specified in their Oracle order for the applicable services.

Contents

Preface

Audience	i
Related Documents	i
Conventions	ii

Part I Oracle Database High Availability Overview

1 Overview of High Availability

What Is High Availability?	1
Importance of Availability	2
Cost of Downtime	2
Causes of Downtime	3
Chaos Engineering	7
Roadmap to Implementing the Maximum Availability Architecture	8

2 High Availability and Data Protection – Getting From Requirements to Architecture

High Availability Requirements	1
A Methodology for Documenting High Availability Requirements	2
Business Impact Analysis	2
Cost of Downtime	3
Recovery Time Objective	3
Recovery Point Objective	4
Manageability Goal	4
Total Cost of Ownership and Return on Investment	5
Mapping Requirements to Architectures	5
Oracle MAA Reference Architectures	5
Bronze Reference Architecture	7
Silver Reference Architecture	7
Gold Reference Architecture	7
Platinum Reference Architecture	7

Diamond Reference Architecture	8
High Availability and Data Protection Attributes by Tier	8

3 Features for Maximizing Availability

Oracle Data Guard	1
Oracle Active Data Guard	4
Oracle Data Guard Advantages Over Traditional Solutions	6
Data Guard and Planned Maintenance	7
Data Guard Redo Apply and Standby-First Patching	7
Data Guard Transient Logical Rolling Upgrades	8
Rolling Upgrade Using Oracle Active Data Guard	9
Oracle GoldenGate	10
Best Practice: Oracle Active Data Guard and Oracle GoldenGate	11
When to Use Oracle Active Data Guard	12
When to Use Oracle GoldenGate	12
When to Use Oracle Active Data Guard and Oracle GoldenGate Together	13
Recovery Manager	13
Oracle Real Application Clusters and Oracle Clusterware	15
Benefits of Using Oracle Clusterware	16
Benefits of Using Oracle Real Application Clusters and Oracle Clusterware	17
Oracle RAC Advantages Over Traditional Cold Cluster Solutions	18
Oracle RAC One Node	19
Oracle Automatic Storage Management	20
Fast Recovery Area	22
Corruption Prevention, Detection, and Repair	22
Data Recovery Advisor	25
Oracle Flashback Technology	26
Oracle Flashback Query	27
Oracle Flashback Version Query	27
Oracle Flashback Transaction	28
Oracle Flashback Transaction Query	28
Oracle Flashback Table	28
Oracle Flashback Drop	28
Restore Points	29
Oracle Flashback Database	29
Flashback Pluggable Database	30
Block Media Recovery Using Flashback Logs or Physical Standby Database	30
Flashback Data Archive	31
Oracle Data Pump and Data Transport	31
Oracle Replication Technologies for Non-Database Files	31
Oracle ASM Cluster File System	32

	Oracle Database File System	33
	Oracle Solaris ZFS Storage Appliance Replication	34
	Oracle Multitenant	35
	Oracle Sharding	37
	Oracle Restart	38
	Online Reorganization and Redefinition	39
	Zero Data Loss Recovery Appliance	39
	Fleet Patching and Provisioning	40
	Edition-Based Redefinition	40
4	Oracle Database High Availability Solutions for Unplanned Downtime	
	Outage Types and Oracle High Availability Solutions for Unplanned Downtime	1
	Managing Unplanned Outages for MAA Reference Architectures and Multitenant Architectures	6
5	Oracle Database High Availability Solutions for Planned Downtime	
	Oracle High Availability Solutions for Planned Maintenance	1
	High Availability Solutions for Migration	3
6	Enabling Continuous Service for Applications	
7	Operational Prerequisites to Maximizing Availability	
	Understand High Availability and Performance Service-Level Agreements	1
	Implement and Validate a High Availability Architecture That Meets Your SLAs	1
	Establish Test Practices and Environment	1
	Configuring Test and QA Environments	2
	Performing Preproduction Validation Steps	3
	Set Up and Use Security Best Practices	5
	Establish Change Control Procedures	5
	Apply Recommended Software Updates and Security Updates Periodically	5
	Establish Disaster Recovery Environment	6
	Establish and Validate Disaster Recovery Practices	7
	Establish Escalation Management Procedures	7
	Configure Monitoring and Service Request Infrastructure for High Availability	8
	Run Database Health Checks Periodically	8
	Configure Monitoring	8
	Configure Automatic Service Request Infrastructure	10
	Exercise Capacity Planning	10

Part II Oracle Database High Availability Best Practices

8 Overview of Oracle Database High Availability Best Practices

9 Oracle Database Configuration Best Practices

Use a Server Parameter File (SPFILE)	1
Enable Archive Log Mode and Forced Logging	1
Configure an Alternate Local Archiving Destination	1
Use a Fast Recovery Area	2
Enable Flashback Database	3
Set FAST_START_MTTR_TARGET Initialization Parameter	4
Protect Against Data Corruption	4
Set the LOG_BUFFER Initialization Parameter to 128MB or Higher	5
Set USE_LARGE_PAGES=ONLY	5
Use Bigfile Tablespace	5
Use Automatic Shared Memory Management and Avoid Memory Paging	7
Use Oracle Clusterware	8

10 Oracle Flashback Best Practices

Oracle Flashback Performance Observations	1
Oracle Flashback Configuration Best Practices	2
Oracle Flashback Operational Best Practices	4
Oracle Flashback Performance Tuning for Specific Application Use Cases	4

11 Oracle Global Data Services Best Practices

Introduction to Global Data Services	1
Global Data Services Concepts	2
Key Capabilities of Global Data Services	3
Benefits of Global Data Services	4
Application Workload Suitability for Global Data Services	5
Global Data Services in Oracle Maximum Availability Architecture	5
Partial or Full Site Outage with Global Data Services	6
Global Data Services Configuration	7
High-Level Deployment Steps	7
Configuration Example	7
Configuration Best Practices	12

Using FAN ONS with Global Data Services	12
Application-Level Configuration	14
Configuring FAN for OCI Clients	15
Controlling Logon Storms	16
Graceful Application Switchover	16
Using Oracle Active Data Guard with Global Data Services	17
Using Oracle GoldenGate with Global Data Services	19
Global Data Services Failover Across Regions Flow	22
Global Data Services Limitations and Requirements	23

Part III Oracle RAC and Clusterware Best Practices

12 Overview of Oracle RAC and Clusterware Best Practices

13 MAA Platinum and MAA Diamond: Oracle RAC and Exadata

Exadata High Availability at Every Tier	1
High Availability Differentiators	2
Unplanned Outages and Planned Maintenance Features	3
Data Protection Features	4
Quality of Service (QoS) Features	5
Health Checks	7
Application Impact with Exadata and MAA Best Practices	7

Part IV Oracle Data Guard Best Practices

14 Overview of MAA Best Practices for Oracle Data Guard

15 Plan an Oracle Data Guard Deployment

Oracle Data Guard Architectures	1
Application Considerations for Oracle Data Guard Deployments	1
Deciding Between Full Site Failover or Seamless Connection Failover	1
Full Site Failover Best Practices	2
Configuring Seamless Connection Failover	5
Assessing and Optimizing Network Performance	5
Gather Topology Information	7
Understanding Network Usage of Data Guard	7

Understanding Targets and Goals for Instantiation	7
Understanding Throughput Requirements and Average Redo Write Size for Redo Transport	7
Verify Average Redo Write Size	8
Understand Current Network Throughput	9
Optimizing Redo Transport with One and Many Processes	12
Using This Data	17
Determining Oracle Data Guard Protection Mode	17
Offloading Queries to a Read-Only Standby Database	18

16 Configure and Deploy Oracle Data Guard

Oracle Data Guard Configuration Best Practices	1
Apply Oracle Database Configuration Best Practices First	1
Use Recovery Manager to Create Standby Databases	1
Use Oracle Data Guard Broker with Oracle Data Guard	1
Example Broker Installation and Configuration	2
Configure Redo Transport Mode	3
Validate the Broker Configuration	3
Configure Fast Start Failover	5
Fast Start Failover with Multiple Standby Databases	7
Set Log Buffer Optimally	8
Set Send and Receive Buffer Sizes	8
Set SDU Size to 65535 for Synchronous Transport Only	8
Configure Online Redo Logs Appropriately	9
Sizing Redo Logs	9
Use Standby Redo Log Groups	10
Protect Against Data Corruption	11
Use Flashback Database for Reinstatement After Failover	12
Use Force Logging Mode	12
Configure Fast Start Failover to Bound RTO and RPO (MAA Gold Requirement)	13
Configure Standby AWR	16
Configuring Multiple Standby Databases	16
Managing Oracle Data Guard Configurations with Multiple Standby Databases	17
Multiple Standby Databases and Redo Routes	17
Using the RedoRoutes Property for Remote Alternate Destinations	18
Fast Start Failover with Multiple Standby Databases	19
Setting FastStartFailoverTarget	20
Switchover with FastStartFailoverTarget Set	20
Fast-Start Failover Outage Handling	21
Oracle Active Data Guard Far Sync Solution	21
About Far Sync	21

Offloading to a Far Sync Instance	22
Far Sync Deployment Topologies	22
Case 1: Zero Data Loss Protection Following Role Transitions	23
Case 2: Reader Farm Support	23
Case 3: Cloud Deployment With Far Sync Hub	24
Far Sync High Availability Topologies	25
Choosing a Far Sync Deployment Topology	26
Far Sync Configuration Best Practices	27
Configuring the Active Data Guard Far Sync Architecture	28
Configuring the Far Sync Instances	28
Setting Up HA Far Sync Instances	30
Configuring Far Sync Instances with Oracle RAC or Oracle Clusterware	31
Encrypting a Database Using Data Guard and Fast Offline Encryption	31

17 Tune and Troubleshoot Oracle Data Guard

Overview of Oracle Data Guard Tuning and Troubleshooting	1
Redo Transport Troubleshooting and Tuning	1
Gather Topology Information	2
Verify Transport Lag and Understand Redo Transport Configuration	2
Gather Information to Troubleshoot Transport Lag	3
Compare Redo Generation Rate History on the Primary	4
Evaluate the Transport Network and Tune	5
Gather and Monitor System Resources	5
Tune to Meet Data Guard Resource Requirements	6
Advanced Troubleshooting: Determining Network Time with Asynchronous Redo Transport	7
Disable Oracle Net Encryption for Redo Transport Only	10
Tuning and Troubleshooting Synchronous Redo Transport	11
Understanding How Synchronous Transport Ensures Data Integrity	11
Assessing Performance in a Synchronous Redo Transport Environment	12
Why the Log File Sync Wait Event is Misleading	12
Understanding What Causes Outliers	13
Effects of Synchronous Redo Transport Remote Writes	14
Example of Synchronous Redo Transport Performance Troubleshooting	14
Redo Apply Troubleshooting and Tuning	15
Understanding Redo Apply and Redo Apply Performance Expectations	16
Verify Apply Lag	17
Gather Information	18
Compare Redo Generation Rate History on the Primary	21
Tune Single Instance Redo Apply	21
Evaluate System Resource Bottlenecks	21

Tune Redo Apply by Evaluating Database Wait Events	22
Enable Multi-Instance Redo Apply if Required	26
Addressing a Very Large Redo Apply Gap	28
Improving Redo Apply Rates by Sacrificing Data Protection	29
Role Transition, Assessment, and Tuning	29
Prerequisite Data Guard Health Check Before Role Transition	30
Every Quarter	30
One Month Before Switchover	30
Days Before Switchover	33
Data Guard Role Transition	33
Monitor Data Guard Role Transitions	34
Key Switchover Operations and Alert Log Tags	34
Key Failover Operations and Alert Log Tags	35
Post Role Transition Validation	36
Troubleshooting Problems During a Switchover Operation	36
Sources of Diagnostic Information	36
Retry Switchover After Correcting the Initial Problem	37
Rolling Back After Unsuccessful Switchover to Maximize Uptime	37
Data Guard Performance Observations	37
Data Guard Role Transition Duration	37
Data Guard Role Transition Optimizations in Oracle AI Database 26ai	41
Redo Apply Performance	42
Application Throughput and Response Time Impact with Data Guard	44

18 Monitor an Oracle Data Guard Configuration

Monitoring Oracle Data Guard Configuration Health Using the Broker	1
Detecting Transport or Apply Lag Using the Oracle Data Guard Broker	3
Monitoring Oracle Data Guard Configuration Health Using SQL	5
Oracle Data Guard Broker Diagnostic Information	7
Detecting and Monitoring Data Corruption	7

19 Optimizing Automatic Failover in Common Scenarios to Minimize Downtime

Automatic Database Failover for Primary Database Outages	1
Automatic Data Integrity and Avoidance of Split Brain	2
Automatic Reconnect Following Any Outage That Results in Network Timeout	3
Automatic Reconnect Following Resolution of Standby Outage	3
Data Guard Broker Properties That Affect Outage Repair Times	3
Data Guard Standby Database Outage Repair	5
Oracle Active Data Guard Far Sync – Examples and Outage Scenarios	8

Part V MAA Platinum and Oracle GoldenGate Best Practices

20 MAA Platinum Reference Architecture Overview

21 Overview of Oracle GoldenGate Best Practices

22 Cloud Within Region: Configuring Oracle GoldenGate Hub

Overview of MAA GoldenGate Hub	1
Planning GGHUB Placement in the Platinum MAA Architecture	2
Where to Place the MAA Primary GGHUB and Standby GGHUB	2
MAA GGHUBs Placed in the Same OCI Region	3
Task 1: Configure the Source and Target Databases for Oracle GoldenGate	7
Step 1.1 - Configure the Databases	8
Step 1.2 - Create a GoldenGate Database Administrator User	9
Step 1.3 - Create the Database Services	10
Task 2: Deploy Oracle GoldenGate Maximum Availability Hub on Oracle Cloud Marketplace	11
Task 3: Configure the Oracle GoldenGate Environment	12
Step 3.1 - Create Database Credentials	12
Step 3.2 - Create an Oracle Net TNS Alias for Oracle GoldenGate Database Connections	13
Step 3.3 - Set Up Schema Supplemental Logging	15
Step 3.4 - Create the Autostart Profile	15
Step 3.5 - Configure Oracle GoldenGate Processes	16

23 Cloud Across Regions: Configuring Oracle GoldenGate Hub

Overview of MAA GoldenGate Hub	1
Planning GGHUB Placement in the Platinum MAA Architecture	2
Where to Place the MAA Primary GGHUB and Standby GGHUB	2
MAA GGHUBs Placed in Different OCI Regions	3
Task 1: Configure the Source and Target Databases for Oracle GoldenGate	7
Step 1.1 - Configure the Databases	8
Step 1.2 - Create a GoldenGate Database Administrator User	9
Step 1.3 - Create the Database Services	10
Task 2: Prepare a Primary and Standby Base System for GGHUB	11
Step 2.1 - Deploy an Oracle RAC 2-Node Cluster System	11

Step 2.2 - Remove the Standard Database and Rearrange the Disk Group Layout	12
Step 2.3 - Download the Required Software	13
Step 2.4 - Configure Oracle Linux To Use the Oracle Public YUM Repository	14
Task 3: Configure Oracle GoldenGate for the Primary and Standby GGHUB	14
Step 3.1 - Install Oracle GoldenGate Software	14
Step 3.2 - Configure the Cloud Network	18
Step 3.3 - Configure ACFS File System Replication Between GGHUBs in the Same Region	22
Step 3.4 - Create the Oracle GoldenGate Deployment	39
Step 3.5 - Configure Oracle Grid Infrastructure Agent (XAG)	41
Step 3.6 - Configure NGINX Reverse Proxy	45
Step 3.7 - Securing Oracle GoldenGate Microservices to Restrict Non-Secure Direct Access	52
Step 3.8 - Create a Clusterware Resource to Manage NGINX	54
Step 3.9 - Create an Oracle Net TNS Alias for Oracle GoldenGate Database Connections	56
Task 4: Configure the Oracle GoldenGate Environment	58
Step 4.1 - Create Database Credentials	58
Step 4.2 - Set Up Schema Supplemental Logging	58
Step 4.3 - Create the Autostart Profile	59
Step 4.4 - Configure Oracle GoldenGate Processes	59

24 Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices

Overview of Oracle GoldenGate Microservices Architecture Configuration on Oracle Exadata Database Service	1
Task 1 - Before You Begin	2
Task 2 - Configure the Oracle Database for GoldenGate	4
Task 3 - Create a Shared File System to Store the Oracle GoldenGate Deployment	7
Task 4 - Install Oracle GoldenGate	16
Task 5 - Create the Oracle GoldenGate Deployment	20
Task 6 - Configure the Network	22
Task 7 - Configure Oracle Grid Infrastructure Agent	25
Task 8 - Configure NGINX Reverse Proxy	30
Task 9 - Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections	39
Task 10 - Create a New Profile	40
Task 11 - Configure Oracle GoldenGate Processes	41

25 Cloud MAA Platinum: Oracle GoldenGate Microservices Architecture Integrated with Active Data Guard

Overview	1
----------	---

Task 1 - Before You Begin	2
Task 2 - Configure the Oracle Database for GoldenGate	3
Task 3 - Configure Oracle Database File System	4
Task 4 - Install Oracle GoldenGate	8
Task 5 - Create Oracle GoldenGate Deployment Directories	8
Task 6 - Network Configuration	9
Task 7 - Configure Standby NGINX Reverse Proxy	9
Task 8 - Configure Oracle Grid Infrastructure Agent	13
Task 9 - Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections	15
Task 10 - Configure Oracle GoldenGate Processes	16
Example Distribution Path Target Change Script	22

26 On-Premises: Configuring Oracle GoldenGate Hub

Overview of MAA GoldenGate Hub	1
Planning GGHUB Placement in the Platinum MAA Architecture	2
MAA GGHUBs Placed in the Same Data Center	3
MAA GGHUBs Placed in Different Data Centers	7
Security Considerations for GGHUB	12
Task 1: Configure the Source and Target Databases for Oracle GoldenGate	13
Task 2: Prepare GGHUB Clusters and Ansible Orchestration Host	17
Task 3: Configure Primary and Standby GGHUB Clusters Using Ansible Automation	23
Task 4: Configure the Oracle GoldenGate Environment	32
Reference	44
Alternate Method to Generate Ansible Inventory File for Deployment	44
Alternate Deployment Option to Configure Only Primary or Standby GGHUB	45
Validate GGHUB Deployment	45
GGHUB Ansible Playbook oracle.gghub.maa_ha Functions	47
Change GoldenGate Microservices Deployment Password Using Ansible	49
Understanding NGINX Reverse Proxy and Certificate Configuration	49

27 On-Premises: Oracle GoldenGate Microservices Architecture with Oracle Real Application Clusters Configuration Best Practices

Summary of Recommendations when Deploying Oracle GoldenGate on Oracle RAC	1
Task 1: Configure the Oracle Database for Oracle GoldenGate	2
Task 2: Create the Database Replication Administrator User	2
Task 3: Create the Database Services	3
Task 4: Set Up a File System on Oracle RAC	3
Task 5: Install Oracle GoldenGate	8
Task 6: Create the Oracle GoldenGate Deployment	8
Task 7: Oracle Clusterware Configuration	10

Task 8: Configure NGINX Reverse Proxy	15
Task 9: Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections	16
Task 10: Configure Oracle GoldenGate Processes	17
Task 11: Configure Autostart of Extract and Replicat Processes	22

28 On-Premises MAA Platinum: Oracle GoldenGate Microservices Architecture Integrated with Active Data Guard

Prerequisites	1
Task 1: Configure the Standby Database for Oracle GoldenGate	2
Task 2: Modify the Primary Database Service	3
Task 3: Create the Standby Database Service	3
Task 4: Configure DBFS on the Standby Cluster Nodes	3
Task 5: Install Oracle GoldenGate Software	4
Task 6: Create Oracle GoldenGate Deployment Directories	5
Task 7: Configure the Standby NGINX Reverse Proxy	5
Task 8: Configure Oracle Clusterware	8
Task 9: Create Oracle Net TNS Aliases for Oracle GoldenGate Database Connections	9
Task 10: Configure Oracle GoldenGate Processes	10
Example Distribution Path Target Change Script	15

29 Managing Outages for Oracle GoldenGate Hub

Managing Planned Outages	1
Managing Unplanned Outages	3

30 Oracle GoldenGate Active-Active Guidance for Developers and Administrators

Preparing for Conflict Resolution	1
Best Practices for Active-Active Oracle GoldenGate Architecture	2

31 Troubleshooting Oracle GoldenGate

Troubleshooting MAA GoldenGate Hub	1
Oracle GoldenGate Extract Failure or Error Conditions Considerations	1
Troubleshooting ACFS Replication	3
Troubleshooting Oracle GoldenGate	4
Troubleshooting GGHUB Deployment Using Ansible	8
Troubleshooting Oracle GoldenGate on Oracle RAC	13
Example Configuration Problems	16

Part VI Data Corruption Prevention, Detection and Repair

32 Data Corruption Prevention, Detection, and Repair

Data Corruption Types	1
Summary of Oracle Data Corruption Resiliency Capabilities	4
Preventing Data Corruption	5
Detecting Data Corruption	7
Repairing Data Corruption	8
References	10

Part VII Oracle Database Cloud Best Practices

33 Overview of Oracle AI Database Cloud Best Practices

34 Oracle Maximum Availability Architecture and Oracle Autonomous Database

Oracle MAA for Oracle Autonomous Database on Dedicated Exadata Infrastructure	1
Autonomous Database Dedicated with Default High Availability Option (MAA Silver)	1
Autonomous Database Dedicated with Autonomous Data Guard Option (MAA Gold)	3
Updating Autonomous Data Guard Settings	6
Autonomous Data Guard Life Cycle Management	6
MAA Autonomous Data Guard RTO and RPO Observations	7
Autonomous Database with Autonomous Data Guard Option and Oracle GoldenGate (MAA Platinum)	8
Implementing the MAA Platinum Solution	9
Preparing Application for Seamless Application Failover	9
Multicloud Considerations for Autonomous Data Guard	10
Oracle MAA for Oracle Autonomous Database Serverless	10
Autonomous Database Serverless with Default High Availability Option (MAA Silver)	10
Autonomous Database Serverless with Autonomous Data Guard Option (MAA Gold)	12
Adding an Autonomous Standby Database	13
Monitoring Apply Lag	15
Autonomous Data Guard Role Transitions	17
Manual Failover Operations and Determining Data Loss	18
Notifications for Automatic Failover	19
MAA Autonomous Data Guard RTO and RPO Observations	22
Preparing an Application for Seamless Application Failover	23

35 Oracle Maximum Availability Architecture in Oracle Exadata Cloud Systems

Oracle Maximum Availability Architecture Benefits	1
Expected Impact with Unplanned Outages	3
Expected Impact with Planned Maintenance	4
Achieving Continuous Availability For Your Applications	11
Oracle Maximum Availability Architecture Reference Architectures in Oracle Exadata Cloud	15

36 Oracle Maximum Availability Architecture for Multicloud

MAA Multicloud Certification Matrix	1
MAA Evaluations on Multicloud Solutions	2
Oracle Database Multicloud Evaluations by Oracle MAA	3
Network Evaluation	3
MAA Silver Architecture and Evaluation	6
MAA Gold Architecture and Evaluation	7
MAA Platinum Architecture and Evaluation	7
Oracle Maximum Availability Architecture Benefits	8
Expected Impact During Unplanned Outages	9
Expected Impact During Planned Maintenance	10
Achieving Continuous Availability For Your Applications	15
Oracle Maximum Availability Architecture for Oracle Database@Azure	18
Network Results	19
MAA Silver Network Topology and Evaluation	21
Application Network Layer on Azure	22
Backup and Restore Observations	22
MAA Gold and Platinum Network Topology and Evaluation	22
Network Layer	24
Setting Up Networking Across Availability Zones	25
Setting Up Networking Across Regions	28
Oracle Maximum Availability Architecture for Oracle Database@Google Cloud	29
Network Results	30
MAA Silver Network Topology and Evaluation	32
Application Network Layer on Google Cloud	33
Backup and Restore Observations	33
MAA Gold and MAA Platinum Network Topology and Evaluation	33
Oracle Active Data Guard Principles for ExaDB-D on Google Cloud Configuration	35
Networking Between Primary and Standby Clusters	36
Oracle Maximum Availability Architecture for Oracle Database@AWS	38

MAA Silver Network Topology and Evaluation	39
Application Network Layer and Application Failover	41
Backup and Restore Observations	41
MAA Gold and MAA Platinum Network Topology and Evaluation	42
Data Guard Considerations and Network Peering	44

37 Oracle Data Guard Hybrid Cloud Configuration

Benefits Of Hybrid Data Guard in the Oracle Cloud	1
MAA Recommendations for Using Exadata Cloud for Disaster Recovery	2
Security Requirements and Considerations	2
Platform, Database, and Network Prerequisites	3
Cloud Network Prerequisites	4
On-Premises Prerequisites	5
Evaluate Network Using oracdpctest	5
Configuration	5
Implement MAA Best Practice Parameter Settings on the Primary Database	6
Validating Connectivity between On-Premises and Exadata Cloud Hosts	6
Prepare the Primary Database Environment	7
Create an ACFS Mount Point	7
Configure Transparent Data Encryption on the Source Database	7
Check the TDE Master Key Before Instantiation	9
Configure Online Redo Logs	9
Size Redo Logs	10
Enable Flashback Database	11
Configure Redo Transport - Oracle Net Encryption	11
Investigate Log for Errors (TFA)	11
Instantiate the Standby Using Oracle DBaaS Tools	12
Task 1: Install dbcactl in the On-Premises Environment	12
Task 2: Prepare the Cloud Environment for Instantiation of the Standby Database	12
Task 3: Instantiate the Standby Database	14
Configure ACFS	15
Run dbcactl Operation prepareForStandby in the On-Premises Environment	15
Run dbaascli Operation configureStandby in the Cloud Environment	17
Task 4: Validate the Standby Database	20
Task 5: Offline Encrypt the Cloud Standby Database	21
Task 6: Implement Recommended MAA Best Practices	21
Data Guard Life Cycle Operations	23
Health Check and Monitoring	23

38 Migration to Oracle Cloud Best Practices

Part VIII Continuous Availability for Applications

39 About Application High Availability Levels

40 Configuring Continuous Availability for Applications

Configuring Level 1: Basic Application High Availability	1
Step 1: Configure High Availability Database Services	1
Configure High Availability Services	2
Configure High Availability Services for Oracle Active Data Guard or Standby Roles	3
Considerations for Oracle Cloud Database Services	3
Step 2: Configure the Connection String for High Availability	4
Step 3: Use Fast Application Notification	6
Step 4: Developer Determines if the Application Should Implement Reconnection Logic	8
Configuring Level 2: Prepare Applications for Planned Maintenance	8
Recommended Option: Use an Oracle Connection Pool	9
Alternate Options	10
Alternate Option 1: Use Request Boundaries	10
Alternate Option 2: Use Connection Validation or Tests	10
Server-Side Operations for Planned Maintenance	11
Configuring Level 3: Mask Unplanned and Planned Failovers from Applications	12
Configure Services for AC and TAC	13
Return Connections to the Connection Pool	14
Side Effect	14
Restore Original Function Values During Replay	14
Client Configuration	15
Review Monitoring Statistics	16

41 Continuous Availability for Applications Reference

Connection Time Estimates During Data Guard Switchover or Failover	1
Oracle Net TNS String Parameters	2
Connection Retry Logic Examples	3
Server-Side Planned Maintenance Command Examples	6
Configuring FAN for Specific Drivers and Pools	7
ONS FAN Application Requirements	10

Part IX Oracle Multitenant Best Practices

42 Overview of Oracle Multitenant Best Practices

43 PDB Switchover and Failover in a Multitenant Configuration

PDB Switchover Use Case	2
Prerequisites	2
Configuring PDB Switchover	3
PDB Failover Use Case	11
Prerequisites	12
Additional Considerations	12
Configuring PDB Failover	13
Reference	19
Full Example Commands with Output	19
Keyword Definitions	25
Sample Oracle Database Net Services Connect Aliases	26
Resolving Errors	30

Part X Full Site Switch in Oracle Cloud or On-Premises

44 Full Site Switch in Oracle Cloud or On-Premise

Performing Role Transitions Between Regions	2
Best Practices for Full Site Switchover	4
More Information About Full Site Switchover	5

Part XI Planned Maintenance

45 Software Maintenance Best Practices

Software Security Recommendations	1
Software Maintenance Tools and Capabilities	2
Patching and Upgrade Guidance and Recommendations	6
Software Maintenance Lifecycle	7
Detect When Software Needs To Be Updated	7
Define a Software Maintenance Policy	8

Determine Software Health	13
Obtain Recommended Software	15
Database and Grid Infrastructure, On-Premises	15
Exadata (On-Premises)	16
Apply Software Updates	17
Software Update Method Selection: Benefits and Tradeoffs	17

Preface

This book introduces you to Oracle best practices for deploying a highly available database environment, and provides best practices for configuring the Oracle MAA reference architectures.

Part 1 provides an overview of high availability and helps you to determine your high availability requirements. It describes the Oracle Database products and features that are designed to support high availability and describes the primary database architectures that can help your business achieve high availability.

Part 2 describes the best practices for configuring a highly available Oracle database, using features provided with Oracle Database, which lets you achieve MAA Bronze reference architecture service levels

Part 3 describes the best practices for configuring a highly available Oracle database using Oracle Data Guard for replication and data protection, which lets you achieve MAA Gold reference architecture service levels.

This preface contains these topics:

- [Audience](#)
- [Related Documents](#)
- [Conventions](#)

Audience

This book is intended for chief technology officers, information technology architects, database administrators, system administrators, network administrators, and application administrators who perform the following tasks:

- Plan data centers
- Implement data center policies
- Maintain high availability systems
- Plan and build high availability solutions

Related Documents

Knowledge of Oracle Database, Oracle RAC, and Data Guard concepts and terminology is required to understand the configuration and implementation details described in this book. For more information, see the Oracle Database documentation set. These books may be of particular interest:

- *Oracle Database Administrator's Guide*
- *Oracle Clusterware Administration and Deployment Guide*
- *Oracle Real Application Clusters Administration and Deployment Guide*

- *Oracle Automatic Storage Management Administrator's Guide*
- *Oracle Data Guard Concepts and Administration*
- *Oracle Database Backup and Recovery User's Guide*

Many books in the documentation set use the sample schemas of the seed database, which is installed by default when you install Oracle Database. See *Oracle Database Sample Schemas* for information about using these schemas.

Also, you can download the Oracle MAA best practice technical briefs at <http://www.oracle.com/goto/maa>.

Conventions

The following text conventions are used in this document:

Convention	Meaning
boldface	Boldface type indicates graphical user interface elements associated with an action, or terms defined in text or the glossary.
<i>italic</i>	Italic type indicates book titles, emphasis, or placeholder variables for which you supply particular values.
monospace	Monospace type indicates commands within a paragraph, URLs, code in examples, text that appears on the screen, or text that you enter.

Part I

Oracle Database High Availability Overview

- [Overview of High Availability](#)
- [High Availability and Data Protection – Getting From Requirements to Architecture](#)
- [Features for Maximizing Availability](#)
- [Oracle Database High Availability Solutions for Unplanned Downtime](#)
- [Oracle Database High Availability Solutions for Planned Downtime](#)
- [Enabling Continuous Service for Applications](#)
- [Operational Prerequisites to Maximizing Availability](#)

1

Overview of High Availability

See the following topics to learn what high availability and why it is important. Then follow the roadmap to implementing a Maximum Availability Architecture.

What Is High Availability?

Availability is the degree to which an application and database service is available.

Availability is measured by the perception of an application's user. Users experience frustration when their data is unavailable or the computing system is not performing as expected, and they do not understand or care to differentiate between the complex components of an overall solution. Performance failures due to higher than expected usage create the same disruption as the failure of critical components in the architecture. If a user cannot access the application or database service, it is said to be unavailable. Generally, the term downtime is used to refer to periods when a system is unavailable.

Users who want their systems to be always ready to serve them need high availability. A system that is highly available is designed to provide uninterrupted computing services during essential time periods, during most hours of the day, and most days of the week throughout the year; this measurement is often shown as 24x365. Such systems may also need a high availability solution for planned maintenance operations such as upgrading a system's hardware or software.

Reliability, recoverability, timely error detection, and continuous operations are primary characteristics of a highly available solution:

- **Reliability:** Reliable hardware is one component of a high availability solution. Reliable software—including the database, web servers, and applications—is just as critical to implementing a highly available solution. A related characteristic is resilience. For example, low-cost commodity hardware, combined with software such as Oracle Real Application Clusters (Oracle RAC), can be used to implement a very reliable system. The resilience of an Oracle RAC database allows processing to continue even though individual servers may fail. For example, the Oracle RAC database allows processing to continue even though individual servers may fail.
- **Recoverability:** Even though there may be many ways to recover from a failure, it is important to determine what types of failures may occur in your high availability environment and how to recover from those failures quickly in order to meet your business requirements. For example, if a critical table is accidentally deleted from the database, what action should you take to recover it? Does your architecture provide the ability to recover in the time specified in a service-level agreement (SLA)?
- **Timely error detection:** If a component in your architecture fails, then fast detection is essential to recover from the unexpected failure. Although you may be able to recover quickly from an outage, if it takes an additional 90 minutes to discover the problem, then you may not meet your SLA. Monitoring the health of your environment requires reliable software to view it quickly and the ability to notify the database administrator of a problem.
- **Continuous operation:** Providing continuous access to your data is essential when very little or no downtime is acceptable to perform maintenance activities. Activities, such as moving a table to another location in the database or even adding CPUs to your hardware, should be transparent to the user in a high availability architecture.

More specifically, a high availability architecture should have the following traits:

- Tolerate failures such that processing continues with minimal or no interruption
- Be transparent to—or tolerant of—system, data, or application changes
- Provide built-in preventive measures
- Provide active monitoring and fast detection of failures
- Provide fast recoverability
- Automate detection and recovery operations
- Protect the data to minimize or prevent data loss and corruptions
- Implement the operational best practices to manage your environment
- Achieve the goals set in SLAs (for example, recovery time objectives (RTOs) and recovery point objectives (RPOs)) for the lowest possible total cost of ownership

Importance of Availability

The importance of high availability varies among applications. Databases and the internet have enabled worldwide collaboration and information sharing by extending the reach of database applications throughout organizations and communities.

This reach emphasizes the importance of high availability in data management solutions. Both small businesses and global enterprises have users all over the world who require access to data 24 hours a day. Without this data access, operations can stop, and revenue is lost. Users now demand service-level agreements from their information technology (IT) departments and solution providers, reflecting the increasing dependence on these solutions. Increasingly, availability is measured in dollars, euros, and yen, not just in time and convenience.

Enterprises have used their IT infrastructure to provide a competitive advantage, increase productivity, and empower users to make faster and more informed decisions. However, with these benefits has come an increasing dependence on that infrastructure. If a critical application becomes unavailable, then the business can be in jeopardy. The business might lose revenue, incur penalties, and receive bad publicity that has a lasting effect on customers and on the company's stock price.

It is important to examine the factors that determine how your data is protected and maximize availability to your users.

Cost of Downtime

The need to deliver increasing levels of availability continues to accelerate as enterprises reengineer their solutions to gain competitive advantage. Most often, these new solutions rely on immediate access to critical business data.

When data is not available, the operation can cease to function. Downtime can lead to lost productivity, lost revenue, damaged customer relationships, bad publicity, and lawsuits.

It is not always easy to place a direct cost on downtime. Angry customers, idle employees, and bad publicity are all costly, but not directly measured in currency. On the other hand, lost revenue and legal penalties incurred because SLA objectives are not met can easily be quantified. The cost of downtime can quickly grow in industries that are dependent on their solutions to provide service.

Other factors to consider in the cost of downtime are:

- The maximum tolerable length of a single unplanned outage
If the event lasts less than 30 seconds, then it may cause very little impact and may be barely perceptible to users. As the length of the outage grows, the effect may grow exponentially and negatively affect the business.
- The maximum frequency of allowable incidents
Frequent outages, even if short in duration, may similarly disrupt business operations.

When designing a solution, it is important to recognize the true cost of downtime to understand how the business can benefit from availability improvements.

Oracle provides a range of high availability solutions to fit every organization regardless of size. Small workgroups and global enterprises alike are able to extend the reach of their critical business applications. With Oracle and the Internet, applications and data are reliably accessible everywhere, at any time.

Causes of Downtime

One of the challenges in designing a high availability solution is examining and addressing all of the possible causes of downtime.

It is important to consider causes of both unplanned and planned downtime when designing a fault-tolerant and resilient IT infrastructure. Planned downtime can be just as disruptive to operations as unplanned downtime, especially in global enterprises that support users in multiple time zones.

The following table describes unplanned outage types and provides examples of each type.

Table 1-1 Causes of Unplanned Downtime

Type	Description	Examples
Site failure	A site failure may affect all processing at a data center, or a subset of applications supported by a data center. The definition of site varies given the contexts of on-premises and cloud. • Site failure - entire regional failure • Data center - entire data center location • Availability domain - isolated data center within a region with possibly many other availability domains • Fault domain - isolated set of system resources within an Availability Domain or data center Typically, each site, data center, availability domain, and fault domain has its own set of isolated hardware, DB compute, network, storage, and power.	• Extended site-wide power failure • Site-wide network failure • Natural disaster makes a data center inoperable • Terrorist or malicious attack on operations or the site

Table 1-1 (Cont.) Causes of Unplanned Downtime

Type	Description	Examples
Cluster-wide failure	The whole cluster hosting an Oracle RAC database is unavailable or fails. This includes: - Failures of nodes in the cluster - Failure of any other components that result in the cluster being unavailable and the Oracle database and instances on the site being unavailable	- The last surviving node on the Oracle RAC cluster fails and the node or database cannot be restarted - Both redundant cluster interconnections fail or Clusterware failure - Database corruption so severe that continuity is not possible on the current database server - Clusterware and hardware-software defects preventing availability or stability.
Computer failure	A computer failure outage occurs when the system running the database becomes unavailable because it has failed or is no longer available. When the database uses Oracle RAC then a computer failure represents a subset of the system (while retaining full access to the data).	- Database system hardware failure - Operating system failure - Oracle instance failure
Network failure	A network failure outage occurs when a network device stops or reduces network traffic and communication from your application to database, database to storage, or any system to system that is critical to your application service processing.	- Network switch failure - Network interface failure - Network cable failures
Storage failure	A storage failure outage occurs when the storage holding some or all of the database contents becomes unavailable because it has shut down or is no longer available.	- Disk or flash drive failure - Disk controller failure - Storage array failure

Table 1-1 (Cont.) Causes of Unplanned Downtime

Type	Description	Examples
Data corruption	A corrupt block is a block that was changed so that it differs from what Oracle Database expects to find. Block corruptions can be categorized as physical or logical: - In a physical block corruption, which is also called a media corruption, the database does not recognize the block at all; the checksum is invalid or the block contains all zeros. An example of a more sophisticated block corruption is when the block header and footer do not match. - In a logical block corruption, the contents of the block are physically sound and pass the physical block checks; however, the block can be logically inconsistent. Examples of logical block corruption include incorrect block type, incorrect data or redo block sequence number, corruption of a row piece or index entry, or data dictionary corruptions. Block corruptions can also be divided into interblock corruption and intrablock corruption: - In an intrablock corruption, the corruption occurs in the block itself and can be either a physical or a logical block corruption. - In an interblock corruption, the corruption occurs between blocks and can only be a logical block corruption. A data corruption outage occurs when a hardware, software, or network component causes corrupt data to be read or written. The service-level impact of a data corruption outage may vary, from a small portion of the application or database (down to a single database block) to a large portion of the application or database (making it essentially unusable).	- Operating system or storage device driver failure - Faulty host bus adapter - Disk controller failure - Volume manager error causing a bad disk read or write - Software or hardware defects
Human error	A human error outage occurs when unintentional or other actions are committed that cause data in the database to become incorrect or unusable. The service-level impact of a human error outage can vary significantly, depending on the amount and critical nature of the affected data.	- File deletion (at the file system level) - Dropped database object - Inadvertent data changes - Malicious data changes

Table 1-1 (Cont.) Causes of Unplanned Downtime

Type	Description	Examples
Lost or stray writes	A lost or stray write is another form of data corruption, but it is much more difficult to detect and repair quickly. A data block stray or lost write occurs when: - For a lost write, an I/O subsystem acknowledges the completion of the block write even though the write I/O did not occur in the persistent storage. On a subsequent block read on the primary database, the I/O subsystem returns the stale version of the data block, which might be used to update other blocks of the database, thereby corrupting it. - For a stray write, the write I/O completed but it was written somewhere else, and a subsequent read operation returns the stale value. - For an Oracle RAC system, a read I/O from one cluster node returns stale data after a write I/O is completed from another node (lost write). For example, this occurs if a network file system (NFS) is mounted in Oracle RAC without disabling attribute caching (for example, without using the noac option). In this case, the write I/O from one node is not immediately visible to another node because it is cached. Block corruptions caused by stray writes or lost writes can cause havoc to your database availability. The data block may be physically or logically correct but subsequent disk reads will show blocks that are stale or with an incorrect Oracle Database block address.	- Operating system or storage device driver failure - Faulty host bus adapter - Disk controller failure - Volume manager error - Other application software - Lack of network file systems (NFS) write visibility across a cluster - Software or hardware defects
Delay, slowdown, or hangs	A delay or slowdown occurs when the database or the application cannot process transactions because of a resource or lock contention. A perceived delay can be caused by lack of system resources.	- Database or application deadlocks - Runaway processes that consume system resources - Logon storms or system faults - Combination of application peaks with lack of system or database resources. This can occur with one application or many applications in a consolidated database environment without proper resource management. - Archived redo log destination or fast recovery area destination becomes full - Oversubscribed or heavily consolidated database system

The following table describes planned outage types and provides examples of each type.

Table 1-2 Causes of Planned Downtime

Type	Description	Examples
Software changes	- Planned periodic software changes to apply minor fixes for stability and security - Planned annual or bi-annual major upgrades to adopt new features and capabilities	- Software updates, including security updates to operating system, clusterware, or database - Major upgrade of operating system, clusterware, or database - Updating or upgrading application software
System and database changes	- Planned system changes to replace defected hardware - Planned system changes to expand or reduce system resources - Planned database changes to adopt parameter changes - Planned change to migrate to new hardware or architecture	- Adding or removing processors or memory to a server - Adding or removing nodes to or from a cluster - Adding or removing disks drives or storage arrays - Replacing any Field Replaceable Unit (FRU) - Changing configuration parameters - System platform migration - Migrating to cluster architecture - Migrating to new storage
Data changes	Planned data changes to the logical structure or physical organization of Oracle Database objects. The primary objective of these changes is to improve performance or manageability.	- Table definition changes - Adding table partitioning - Creating and rebuilding indexes
Application changes	Planned application changes can include data changes and schema and programmatic changes. The primary objective of these changes is to improve performance, manageability, and functionality.	Application upgrades

Oracle offers high availability solutions to help avoid both unplanned and planned downtime, and recover from failures. [Oracle Database High Availability Solutions for Unplanned Downtime](#) and [Oracle Database High Availability Solutions for Planned Downtime](#) discuss each of these high availability solutions in detail.

Chaos Engineering

Maximum Availability Architecture leverages Chaos Engineering throughout its testing and development life cycles to ensure that end-to-end application and database availability is preserved or at its optimal levels for any fault or maintenance event.

Chaos Engineering is the discipline of experimenting on a system in order to build confidence in the system's capability to withstand turbulent conditions in production. Specifically, MAA injects various faults and planned maintenance events to evaluate application and database impact throughout our development, stress, and testing cycles. With that experimentation, best practices, defects, and lessons learned are derived, and that knowledge is put back in practice to evolve and improve our MAA solutions.

Roadmap to Implementing the Maximum Availability Architecture

Oracle high availability solutions and sound operational practices are the key to successful implementation of an IT infrastructure. However, technology alone is not enough.

Choosing and implementing an architecture that best fits your availability requirements can be a daunting task. Oracle Maximum Availability Architecture (MAA) simplifies the process of choosing and implementing a high availability architecture to fit your business requirements with the following considerations:

- Encompasses redundancy across all components
- Provides protection and tolerance from computer failures, storage failures, human errors, data corruption, lost writes, system delays or slowdowns, and site disasters
- Recovers from outages as quickly and transparently as possible
- Provides solutions to eliminate or reduce planned downtime
- Provides consistent high performance and robust security
- Provides Oracle Engineered System and cloud options to simplify deployment and management and achieve the highest scalability, performance, and availability
- Achieves SLAs at the lowest possible total cost of ownership
- Applies to On-Premise, Oracle Public Cloud, and hybrid architectures consisting of parts on-premise and part in the cloud
- Provides special consideration to Container or Oracle Multitenant, Oracle Database In-Memory, and Oracle Sharding architectures

To build, implement, and maintain this type of architecture, you need to:

1. Analyze your specific high availability requirements, including both the technical and operational aspects of your IT systems and business processes, as described in [High Availability and Data Protection – Getting From Requirements to Architecture](#).
2. Evaluate the various high availability architectures and their benefits and options, as described in [Oracle MAA Reference Architectures](#).
3. Understand the availability impact for each MAA reference architecture, or various high availability features, on businesses and applications, as described in [Oracle Database High Availability Solutions for Unplanned Downtime](#), and [Oracle Database High Availability Solutions for Planned Downtime](#).
4. Familiarize yourself with Oracle high availability features, as described in [Features for Maximizing Availability](#).
5. Use operational best practices to provide a successful MAA implementation, as described in [Operational Prerequisites to Maximizing Availability](#).
6. Implement a high availability architecture using Oracle MAA resources, which provide technical details about the various Oracle MAA high availability technologies, along with best practice recommendations for configuring and using such technologies, such as Oracle MAA best practices technical briefs, customer papers with proof of concepts, customer case studies, recorded web casts, demonstrations, and presentations.

Additional Oracle MAA resources are available at <http://www.oracle.com/goto/maa>.

2

High Availability and Data Protection – Getting From Requirements to Architecture

See the following topics to learn how Oracle Maximum Availability Architecture provides a framework to effectively evaluate the high availability requirements of an enterprise.

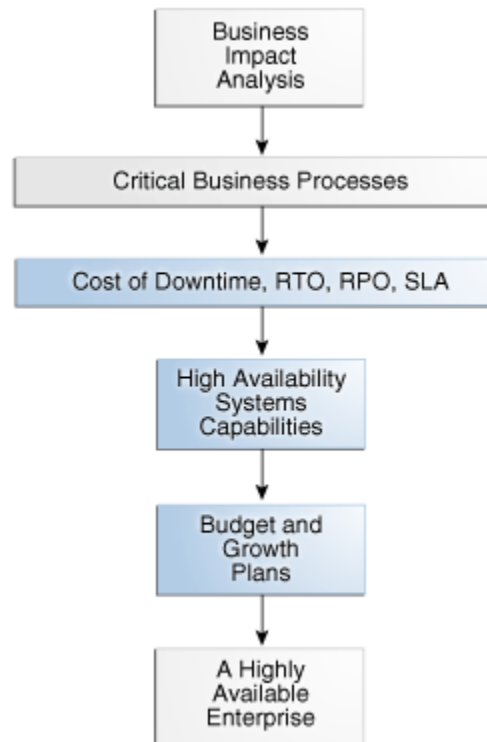
High Availability Requirements

Any effort to design and implement a high availability strategy for Oracle Database begins by performing a thorough business impact analysis to identify the consequences to the enterprise of downtime and data loss, whether caused by unplanned or planned outages.

The term "business impact" is intended to be agnostic of whether the enterprise is a commercial venture, government agency, or not-for-profit institution. In all cases, data loss and downtime can seriously impact the ability of any enterprise to perform its functions. Implementing high availability may involve critical tasks such as:

- Retiring legacy systems
- Investing in more capable and robust systems and facilities
- Redesigning the overall IT architecture and operations to adapt to this high availability model
- Modifying existing applications to take full advantage of high availability infrastructures
- Redesigning business processes
- Hiring and training personnel
- Moving parts or an entire application or database into the Oracle Public Cloud
- Balancing the right level of consolidation, flexibility, and isolation
- Understanding the capabilities and limitations of your existing system and network infrastructure

By combining your business analysis with an understanding of the level of investment required to implement different high availability solutions, you can develop a high availability architecture that achieves both business and technical objectives.

Figure 2-1 Planning and Implementing a Highly Available Enterprise

A Methodology for Documenting High Availability Requirements

The elements of this analysis framework are:

- [Business Impact Analysis](#)
- [Cost of Downtime](#)
- [Recovery Time Objective](#)
- [Recovery Point Objective](#)
- [Manageability Goal](#)
- [Total Cost of Ownership and Return on Investment](#)

Business Impact Analysis

The business impact analysis categorizes the business processes based on the severity of the impact of IT-related outages.

A rigorous business impact analysis:

- Identifies the critical business processes in an organization
- Calculates the quantifiable loss risk for unplanned and planned IT outages affecting each of these business processes
- Outlines the effects of these outages
- Considers essential business functions, people and system resources, government regulations, and internal and external business dependencies

- Is based on objective and subjective data gathered from interviews with knowledgeable and experienced personnel
- Reviews business practice histories, financial reports, IT systems logs, and so on

For example, consider a semiconductor manufacturer with chip fabrication plants located worldwide. Semiconductor manufacturing is an intensely competitive business requiring a huge financial investment that is amortized over high production volumes. The human resource applications used by plant administration are unlikely to be considered as mission-critical as the applications that control the manufacturing process in the plant. Failure of the applications that support manufacturing affects production levels and have a direct impact on the financial results of the company.

As another example, an internal knowledge management system is likely to be considered mission-critical for a management consulting firm, because the business of a client-focused company is based on internal research accessibility for its consultants and knowledge workers. The cost of downtime of such a system is extremely high for this business.

Similarly, an e-commerce company is highly dependent on customer traffic to its website to generate revenue. Any disruption in service and loss of availability can dampen customer experience and drive away customers to the competition. Thus, the company needs to ensure that the existing infrastructure can scale and handle spikes in customer traffic. Sometimes, this is not possible using on-premise hardware and by moving the cloud the company can ensure their systems always remain operational.

Cost of Downtime

A complete business impact analysis provides the insight needed to quantify the cost of unplanned and planned downtime.

Understanding this cost is essential because it helps prioritize your high availability investment and directly influences the high availability technologies that you choose to minimize the downtime risk.

Various reports have been published, documenting the costs of downtime in different industries. Examples include costs that range from millions of dollars for each hour of brokerage operations and credit card sales, to tens of thousands of dollars for each hour of package shipping services.

These numbers are staggering. The Internet and Cloud can connect the business directly to millions of customers. Application downtime can disrupt this connection, cutting off a business from its customers. In addition to lost revenue, downtime can negatively affect customer relationships, competitive advantages, legal obligations, industry reputation, and shareholder confidence.

Recovery Time Objective

The business impact analysis determines your tolerance to downtime, also known as the recovery time objective (RTO).

An RTO is defined as the maximum amount of time that an IT-based business process can be down before the organization starts suffering unacceptable consequences (financial losses, customer dissatisfaction, reputation, and so on). RTO indicates the downtime tolerance of a business process or an organization in general.

RTO requirements are driven by the mission-critical nature of the business. Therefore, for a system running a stock exchange, the RTO is zero or near to zero.

An organization is likely to have varying RTO requirements across its various business processes. A high volume e-commerce website, for which there is an expectation of rapid response times, and for which customer switching costs are very low, the web-based customer interaction system that drives e-commerce sales is likely to have an RTO of zero or close to zero. However, the RTO of the systems that support back-end operations, such as shipping and billing, can be higher. If these back-end systems are down, then the business may resort to manual operations temporarily without a significant visible impact.

Some organizations have varying RTOs based on the probability of failures. One simple class separation is local failures (such as single database compute, disk/flash, network failure) as opposed to disasters (such as a complete cluster, database, data corruptions, or a site failure). Typically, business-critical customers have an RTO of less than 1 minute for local failures, and may have a higher RTO of less than 1 hour for disasters. For mission-critical applications the RTOs may indeed be the same for all unplanned outages.

Recovery Point Objective

The business impact analysis also determines your tolerance to data loss, also known as a recovery point objective (RPO).

The RPO is the maximum amount of data that an IT-based business process can lose without harm to the organization. RPO measures the data-loss tolerance of a business process or an organization in general. This data loss is often measured in terms of time, for example, zero, seconds, hours, or days of data loss.

A stock exchange where millions of dollars worth of transactions occur every minute cannot afford to lose any data. Therefore, its RPO must be zero. The web-based sales system in the e-commerce example does not require an RPO of zero, although a low RPO is essential for customer satisfaction. However, its back-end merchandising and inventory update system can have a higher RPO because lost data can be reentered.

An RPO of zero can be challenging for disasters, but it can be accomplished with various Oracle technologies protecting your database, especially Zero Data Loss Recovery Appliance.

Manageability Goal

A manageability goal is more subjective than either the RPO or the RTO. You must make an objective evaluation of the skill sets, management resources, and tools available in an organization, and the degree to which the organization can successfully manage all elements of a high availability architecture.

Just as RPO and RTO measure an organization's tolerance for downtime and data loss, your manageability goal measures the organization's tolerance for complexity in the IT environment. When less complexity is a requirement, simpler methods of achieving high availability are preferred over methods that may be more complex to manage, even if the latter could attain more aggressive RTO and RPO objectives. Understanding manageability goals helps organizations differentiate between what is possible and what is practical to implement.

Moving Oracle databases to Oracle Cloud can reduce manageability cost and complexity significantly, because Oracle Cloud lets you to choose between various Maximum Availability Architecture architectures with built-in configuration and life cycle operations. With Autonomous Database Cloud, database life cycle operations, such as backup and restore, software updates, and key repair operations are automatic.

Total Cost of Ownership and Return on Investment

Understanding the total cost of ownership (TCO) and objectives for return on investment (ROI) are essential to selecting a high availability architecture that also achieves the business goals of your organization.

TCO includes all costs (such as acquisition, implementation, systems, networks, facilities, staff, training, and support) over the useful life of your chosen high availability solution. Likewise, the ROI calculation captures all of the financial benefits that accrue for a given high availability architecture.

For example, consider a high availability architecture in which IT systems and storage at a remote standby site remain idle, with no other business use that can be served by the standby systems. The only return on investment for the standby site is the costs related to downtime avoided by its use in a failover scenario. Contrast this with a different high availability architecture that enables IT systems and storage at the standby site to be used productively while in the standby role (for example, for reports or for off-loading the overhead of user queries or distributing read-write workload from the primary system). The return on investment of such an architecture includes both the cost of downtime avoided and the financial benefits that accrue to its productive use, while also providing high availability and data protection.

Enterprises can also reduce TCO for growing infrastructure needs by moving workloads to the cloud rather than making an upfront capital investment in building a new data center. The major economic appeal is to convert capital expenditures into operational expenditures, and generate a higher ROI.

Mapping Requirements to Architectures

The business impact analysis will help you document what is already known. The outcome of the business impact analysis provides the insight you need to group databases having similar RTO and RPO objectives together.

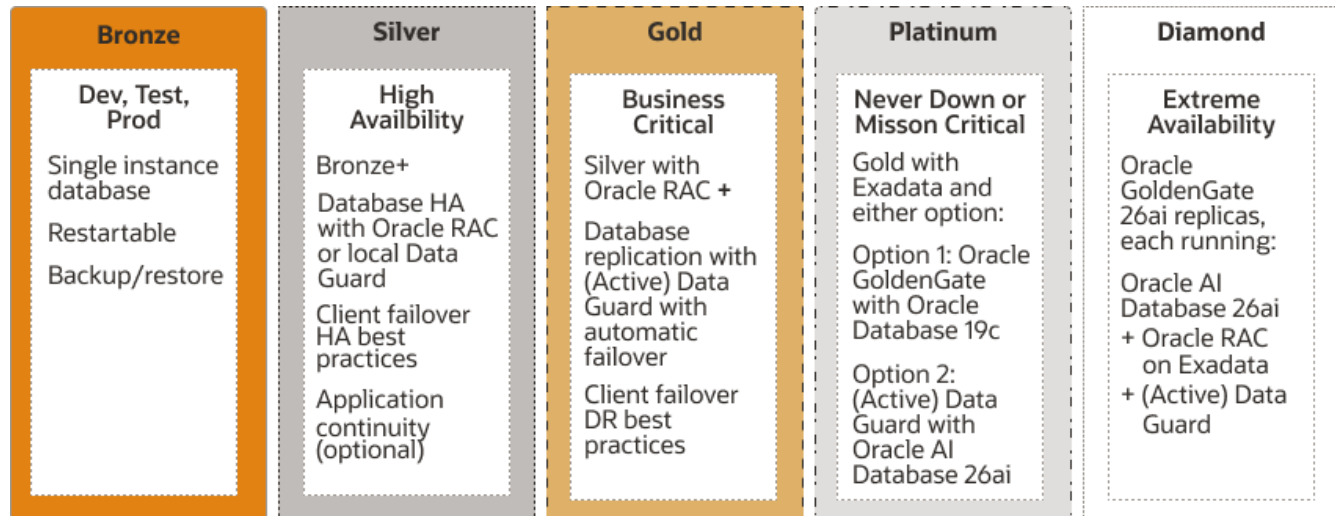
Different applications, and the databases that support them, represent varying degrees of importance to the enterprise. A high level of investment in high availability infrastructure may not make sense for an application that if down, would not have an immediate impact on the enterprise. So where do you start?

Groups of databases by similar RTO and RPO can be mapped to a controlled set of high availability reference architectures that most closely address the required service levels. Note that in the case where there are dependencies between databases, they are grouped with the database having the most stringent high availability requirement.

Oracle MAA Reference Architectures

Oracle MAA best practices define high availability reference architectures that address the complete range of availability and data protection required by enterprises of all sizes and lines of business.

The Platinum, Gold, Silver, and Bronze MAA reference architectures, or tiers, are applicable to on-premises, private and public cloud configurations, and hybrid cloud. They deliver the service levels described in the following figure.

Figure 2-2 Oracle MAA Reference Architectures

Each tier uses a different MAA reference architecture to deploy the optimal set of Oracle high availability capabilities that reliably achieve a given service level at the lowest cost and complexity. The tiers explicitly address all types of unplanned outages, including data corruption, component failure, and system and site outages, as well as planned outages due to maintenance, migrations, or other purposes.

Container databases (CDBs) using Oracle Multitenant can exist in any tier, Bronze through Platinum, providing higher consolidation density and higher TCO. Typically, the consolidation density is higher with Bronze and Silver tiers, and there is less or zero consolidation when deploying a Platinum tier.

Oracle Database In-Memory can also be leveraged in any of the MAA tiers. Because the In-Memory column store is seamlessly integrated into Oracle Database, all of the high availability benefits that come from the MAA tiers are inherited when implementing Oracle Database In-Memory.

Oracle Engineered Systems can also exist in any of the tiers. Integrating Zero Data Loss Recovery Appliance (Recovery Appliance) as the Oracle Database backup and recovery solution for your entire data center reduces RPO and RTO when restoring from backups. Leveraging Oracle Exadata Database Machine as your database platform in the MAA reference architectures provides the best database platform solution with the lowest RTO and brownout, along with additional Exadata MAA quality of service.

① See Also

[High Availability Reference Architectures](#)

[Oracle Exadata Database Machine: Maximum Availability Architecture](#) and [MAA Best Practices for Oracle Exadata Database Machine](#)

<http://www.oracle.com/goto/maa> for MAA technical brief “Oracle Database In-Memory High Availability Best Practices”

Bronze Reference Architecture

The Bronze tier is appropriate for databases where simple restart of a failed component (e.g. listener, database instance, or database) or restore from backup is "HA and DR enough."

The Bronze reference architecture is based on a single instance Oracle Database using MAA best practices that implement the many capabilities for data protection and high availability included with every Oracle Enterprise Edition license. Oracle-optimized backups using Oracle Recovery Manager (RMAN) provide data protection, and are used to restore availability should an outage prevent the database from restarting. The Bronze architecture then uses a redundant system infrastructure enhanced by Oracle's technologies, such as Oracle Restart, Recovery Manager (RMAN), Zero Data Loss Recovery Appliance, Flashback technologies, Online Redefinition, Online Patching, Automatic Storage Management (ASM), Oracle Multitenant, and more.

Silver Reference Architecture

The Silver tier provides an additional level of high availability for databases that require minimal or zero downtime in the event of database instance or server failure, as well as most common planned maintenance events, such as hardware and software updates.

The Silver reference architecture adds a rich set of enterprise capabilities and benefits, including clustering technology using either Oracle RAC or Oracle RAC One Node. Also, Application Continuity provides a reliable replay of in-flight transactions, which masks outages from users and simplifies application failover.

Gold Reference Architecture

The Gold tier raises the stakes substantially for business-critical applications that cannot tolerate high RTO and RPO for any disasters such as database, cluster, corruptions, or site failures. Additionally, major database upgrades or site migrations can be done in seconds.

The Gold tier also reduces costs while improving your return on investment by actively using all of the replicas at all times.

The Gold reference architecture adds database-aware replication technologies, Oracle Data Guard and Oracle Active Data Guard, which synchronize one or more replicas of the production database to provide real time data protection and availability. Database-aware replication substantially enhances high availability and data protection (corruption protection) beyond what is possible with storage replication technologies. Oracle Active Data Guard Far Sync is used for zero data loss protection at any distance.

See also, [Oracle Data Guard Advantages Over Traditional Solutions](#).

Platinum Reference Architecture

The Platinum tier introduces two architecture patterns: Oracle GoldenGate replication with Oracle Database 19c Oracle Active Data Guard and Oracle AI Database 26ai and later releases.

Edition Based Redefinition lets application developers design for zero-downtime application upgrades. You can alternatively design applications for Oracle Sharding, which provides extreme availability by distributing subsets of a database into highly available shards, while the application can access the entire database as one single logical database.

Each of these technologies requires additional effort to implement, but they deliver substantial value for the most critical applications where downtime is not an option.

Diamond Reference Architecture

The Diamond tier introduces builds on MAA Platinum with Oracle GoldenGate, and is ideal for extremely mission critical systems.

The MAA Diamond reference architecture requires the same services as MAA Platinum with GoldeGate, except that it requires:

- Oracle GoldenGate 26ai active/active replication
- Oracle AI Database 26ai on Oracle Exadata
- MAA-hardened GoldenGate Hub configuration

High Availability and Data Protection Attributes by Tier

Each MAA reference architecture delivers known and tested levels of downtime and data protection.

The following table summarizes the high availability and data protection attributes inherent to each architecture. Each architecture includes all of the capabilities of the previous architecture, and builds upon it to handle an expanded set of outages. The various components included and the service levels achieved by each architecture are described in other topics.

Table 2-1 High Availability and Data Protection Attributes By MAA Reference Architecture

MAA Reference Architecture	Unplanned Outages (Local Site)	Planned Maintenance	Data Protection	Unrecoverable Local Outages and Disaster Recovery
Bronze	Single Instance, auto-restart for recoverable instance and server failures. Redundancy for system infrastructure so that single component failures such as disk, flash, and network should not result in downtime.	Some online, most off-line	Basic runtime validation combined with manual checks	Restore from backup, potential to lose data generated since the last backup. Using Zero Data Loss Recovery Appliance reduces the potential to lose data to zero or near zero.
Silver	HA with automatic failover for instance and server failures	Most rolling, some online, few offline	Basic runtime validation combined with manual checks	Restore from backup, potential to lose data generated since the last backup. Using Zero Data Loss Recovery Appliance reduces the potential to lose data to zero or near zero. In-flight transactions are preserved with Application Continuity.

Table 2-1 (Cont.) High Availability and Data Protection Attributes By MAA Reference Architecture

MAA Reference Architecture	Unplanned Outages (Local Site)	Planned Maintenance	Data Protection	Unrecoverable Local Outages and Disaster Recovery
Gold	Comprehensive high availability and disaster recovery	All rolling or online	Comprehensive runtime validation combined with manual checks	Real-time failover, zero or near-zero data loss
Platinum	Zero application outage for Platinum ready applications	Zero application outage	Comprehensive runtime validation combined with manual checks	Zero application outage for Platinum-ready applications, with zero data loss. Oracle RAC, Oracle Active Data Guard, and Oracle GoldenGate complement each other, providing a wide array of solutions to achieve zero database service downtime for unplanned outages. Alternatively, use Oracle Sharding for site failure protection, because impact on the application is only on shards in failed site rather than the entire database. Each shard can be configured with real-time failover, zero or near-zero data loss, or zero application outage for Platinum-ready applications. In-flight transactions are preserved, with zero data loss.

① See Also

<http://www.oracle.com/goto/maa>

3

Features for Maximizing Availability

Familiarize yourself with the following Oracle Database high availability features used in MAA solutions.

Oracle Data Guard

Oracle Data Guard ensures high availability, data protection, and disaster recovery for enterprise data.

Data Guard provides a comprehensive set of services that create, maintain, manage, and monitor one or more standby databases to enable Oracle databases to survive outages of any kind, including natural disasters and data corruptions. A Data Guard standby database is an exact replica of the production database and thus can be transparently utilized in combination with traditional backup, restoration, flashback, and cluster techniques to provide the highest possible level of data protection, data availability and disaster recovery. Data Guard is included in Oracle Enterprise Edition.

A Data Guard configuration consists of one primary database and one or more standby databases. A primary database can be either a single-instance Oracle database or an Oracle RAC database. Similar to a primary database, a standby database can be either a single-instance Oracle database or an Oracle RAC database. Using a backup copy of the primary database, you can create up to 30 standby databases that receive redo directly from the primary database. Optionally you can use a cascaded standby to create Data Guard configurations where the primary transmits redo to a single remote destination, and that destination forwards redo to multiple standby databases. This enables a primary database to efficiently synchronize many more than 30 standby databases if desired.

Note:

Oracle Active Data Guard is an extension of basic Data Guard providing advanced features that off-load various types of processing from a production database, extend zero data loss protection over any distance, and that enhance high availability. Oracle Active Data Guard is licensed separately from Oracle Database Enterprise Edition.

There are several types of standby databases. Data Guard physical standby database is the MAA best practice for data protection and disaster recovery and is the most common type of standby database used. A physical standby database uses Redo Apply (an extension of Oracle media recovery) to maintain an exact, physical replica of the production database. When configured using MAA best practices, Redo Apply uses multiple Oracle-aware validation checks to prevent corruptions that can impact a primary database from impacting the standby. Other types of Data Guard standby databases include: snapshot standby (a standby open read/write for test or other purposes) and logical standby (used to reduce planned downtime).

Benefits of Using Data Guard

- Continuous Oracle-aware validation of all changes using multiple checks for physical and logical consistency of structures within an Oracle data block and redo, before updates are applied to a standby database. This isolates the standby database and prevents it from being impacted by data corruptions that can occur on the primary system.
- Transparent operation: There are no restrictions on the use of Data Guard physical standby for data protection. Redo Apply supports all data and storage types, all DDL

operations, and all applications (custom and packaged applications), and guarantees data consistency across primary and standby databases.

- Highest performance: Fast redo transport for best recovery point objective, fast apply performance for best recovery time objective. Multi-instance redo apply provides Oracle RAC scalability for redo apply, eliminating bottlenecks of a single database server. Redo apply can essentially scale up to available CPU, I/O, and network across your Oracle RAC cluster. An observed redo apply rate of 3500 MB per second (12 TB/hour) on 8 node RAC Exadata.
- Fast failover to a standby database to maintain availability should the primary database fail for any reason. Failover is either a manual or automatic operation depending on how Data Guard is configured.
- Integrated client notification framework to enable application clients to connect to a new primary database after a failover occurs.
- Automatic or automated (depending upon configuration) resynchronization of a failed primary database, quickly converting it to a synchronized standby database after a failover occurs.
- Choice of flexible data protection levels to support all network configurations, availability and performance SLAs, and business requirements.
- Management of a primary and all of its standby databases as a single configuration to simplify management and monitoring using either the Data Guard Broker command-line interface or Oracle Enterprise Manager Cloud Control.
- Data Guard Broker greatly improves manageability with additional features for comprehensive configuration health checks, resumable switchover operations, streamlined role transitions, support for cascaded standby configurations, and user-configurable thresholds for transport and apply lag to automatically monitor the ability of the configuration to support SLAs for recovery point and recovery time objectives at any instant in time.
- Efficient transport to multiple remote destinations using a single redo stream originating from the primary production database and forwarded by a cascading standby database.
- Snapshot Standby enables a physical standby database to be open read/write for testing or any activity that requires a read/write replica of production data. A snapshot standby continues to receive but does not apply updates generated by the primary. When testing is complete, a snapshot standby is converted back into a synchronized physical standby database by first discarding the changes made during the open read/write, and then applying the redo received from the primary database. Primary data is always protected. Snapshot standby is particularly useful when used in conjunction with Oracle Real Application Testing (workload is captured at the production database for replay and subsequent performance analysis at the standby database-an exact replica of production).
- Reduction of planned downtime by using a standby database to perform maintenance in a rolling manner. The only downtime is the time required to perform a Data Guard switchover; applications remain available while the maintenance is being performed.
- Increased flexibility for Data Guard configurations where the primary and standby systems may have different CPU architectures or operating systems subject to limitations defined in [Data Guard Support for Heterogeneous Primary and Physical Standbys in Same Data Guard Configuration \(KB142824\)](#).
- Efficient disaster recovery for a container database (CDB). Data Guard failover and switchover completes using a single command at a CDB level regardless of how many pluggable databases (PDBs) are consolidated within the CDB.
- Enables a specific administration privilege, SYSDG, to handle standard administration duties for Data Guard. This new privilege is based on the least privilege principle, in which

a user is granted only the necessary privileges required to perform a specific function and no more. The SYSDBA privilege continues to work as in previous releases.

- The Oracle Database In-Memory column store is supported on standby databases in an Active Data Guard environment.
- Further improves performance and availability of Data Warehouses in a Data Guard configuration by tracking information from NOLOGGING operations so they can be repaired with the new RMAN command `RECOVER DATABASE NOLOGGING`.
- Improves the impact multiple SYNC transport destinations have on the primary database through the use of a new parameter `DATA_GUARD_SYNC_LATENCY`. This parameter defines the maximum amount of time (in seconds) that the Primary database must wait before disconnecting subsequent destinations after at least one synchronous standby has acknowledged receipt of the redo.
- Data Guard Broker improves manageability by supporting destinations of different Endianness than the primary in addition to enhancing management of alternate destinations.
- Data Guard improves protection and Return To Operations (RTO) and Recovery Point Objectives (RPO) through multiple features including:
 - Multi-Instance Redo Apply (MIRA) provides scalable redo apply performance across Oracle RAC instances reducing RTO for even higher production OLTP or batch workloads
 - Compare primary and standby database blocks using the new `DBMS_DBCOMP` package to help identify lost writes so they can be resolved efficiently.
 - Fast Start Failover (FSFO) has the robustness of highly available zero data loss configurations with support for Maximum Protection mode while giving the flexibility of multiple observers and multiple failover targets for high availability in any configuration. FSFO can also be configured to automatically fail over to the standby with the detection of a lost write on the primary .
 - RPO is improved with no data loss failovers after a storage failure in ASYNC configurations and Data Guard Broker support for Application Continuity, improving the user experience during Data Guard role transitions.
- Oracle Data Guard Broker further improves the management of databases by supporting destinations of different endianness than the primary in addition to enhancing management of alternate archive destinations when the primary destination is unavailable.
- Oracle Data Guard Database Compare tool compares data blocks stored in an Oracle Data Guard primary database and its physical standby databases. Use this tool to find disk errors (such as lost write) that cannot be detected by other tools like the `DBVERIFY` utility.
- Oracle Data Guard Broker supports multiple automatic failover targets in a fast-start failover configuration. Designating multiple failover targets significantly improves the likelihood that there is always a standby suitable for automatic failover when needed.
- Dynamically change Oracle Data Guard Broker Fast-Start Failover target. The fast-start failover target standby database can be changed dynamically, to another standby database in the target list, without disabling fast-start failover. (new in Oracle Database 19c)
- Propagate restore points from primary to standby Site. Restore points created on the primary database are propagated to the standby sites, so that they are available even after a failover operation. (new in Oracle Database 19c)

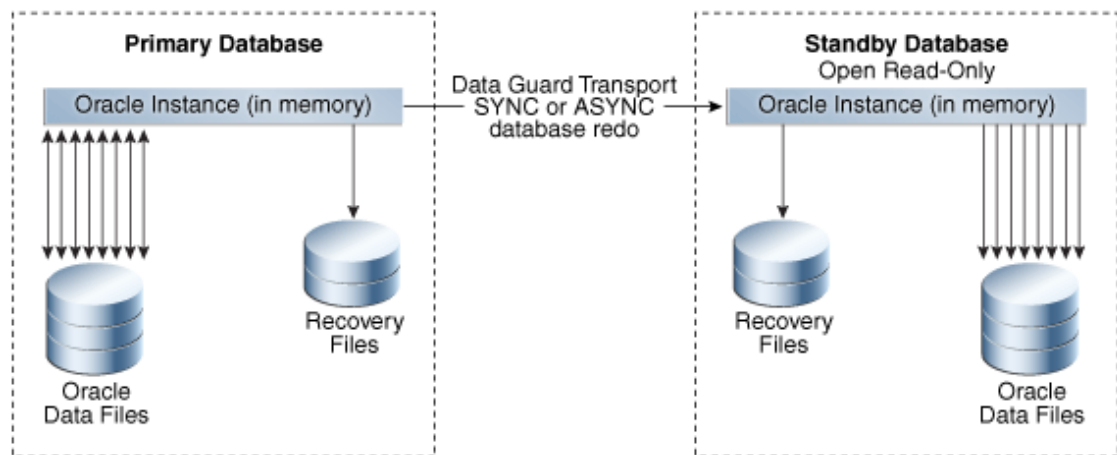
- Oracle Data Guard automatic outage resolution can be tuned to fit your specific needs. Oracle Data Guard has an internal mechanism to detect hung processes and terminate them, allowing the normal outage resolution to occur. (new in Oracle Database 19c)
- Active Data Guard DML redirection helps load balancing between the primary and standby databases. Incidental Data Manipulation Language (DML) operations can be run on Active Data Guard standby databases. This allows more applications to benefit from using an Active Data Guard standby database when some writes are required. When incidental DML is issued on an Active Data Guard standby database, the update is passed to the primary database where it is processed. The resulting redo of the transaction updates the standby database after which control is returned to the application. (new in Oracle Database 19c)

Oracle Active Data Guard

Oracle Active Data Guard is Oracle's strategic solution for real time data protection and disaster recovery for the Oracle database using a physical replication process.

Oracle Active Data Guard also provides high return on investment in disaster recovery systems by enabling a standby database to be open read-only while it applies changes received from the primary database. Oracle Active Data Guard is a separately licensed product that provides advanced features that greatly expand Data Guard capabilities included with Oracle Enterprise Edition.

Figure 3-1 Oracle Active Data Guard Architecture



Oracle Active Data Guard enables administrators to improve performance by offloading processing from the primary database to a physical standby database that is open read-only while it applies updates received from the primary database. Offload capabilities of Oracle Active Data Guard include read-only reporting and ad-hoc queries (including DML to global temporary tables and unique global or session sequences), data extracts, fast incremental backups, redo transport compression, efficient servicing of multiple remote destinations, and the ability to extend zero data loss protection to a remote standby database without impacting primary database performance. Oracle Active Data Guard also increases high availability by performing automatic block repair and enabling High Availability Upgrades automation.

Note:

Oracle Active Data Guard is licensed separately as a database option license for Oracle Database Enterprise Edition. All Oracle Active Data Guard capabilities are also included in an

Oracle Golden Gate license for Oracle Enterprise Edition. This provides customers with the choice of a standalone license for Oracle Active Data Guard, or licensing Oracle GoldenGate to acquire access to all advanced Oracle replication capabilities.

Benefits of Oracle Active Data Guard

Oracle Active Data Guard inherits all of the benefits previously listed for Data Guard, plus the following:

- Improves primary database performance: Production-offload to an Oracle Active Data Guard standby database of read-only applications, reporting, and ad hoc queries. Any application compatible with a read-only database can run on an Oracle Active Data Guard standby. Oracle also provides integration that enables the offloading of many Oracle E-Business Suite Reports, PeopleTools reporting, Oracle Business Intelligence Enterprise Edition (OBIEE), and Oracle TopLink applications to an Oracle Active Data Guard standby database.
- DML global temporary tables and the use of sequences at the standby database significantly expands the number of read-only applications that can be off-loaded from production databases to an Oracle Active Data Guard standby database.
- The unique ability to easily scale read performance using multiple Oracle Active Data Guard standby databases, also referred to as a Reader Farm.
- Production-offload of data extracts using Oracle Data Pump or other methods that read directly from the source database.
- Production-offload of the performance impact from network latency in a synchronous, zero data loss configuration where primary and standby databases are separated by hundreds or thousands of miles. Far sync uses a lightweight instance (control file and archive log files, but no recovery and no data files), deployed on a system independent of the primary database. The far sync instance is ideally located at the maximum distance from the primary system that an application can tolerate the performance impact of synchronous transport to provide optimal protection. Data Guard transmits redo synchronously to the far sync instance and far sync forwards the redo asynchronously to a remote standby database that is the ultimate failover target. If the primary database fails, the same failover command used for any Data Guard configuration, or mouse click using Oracle Enterprise Manager Cloud Control, or automatic failover using Data Guard Fast-Start Failover initiates a zero data loss failover to the remote destination. This transparently extends zero data loss protection to a remote standby database just as if it were receiving redo directly from the primary database, while avoiding the performance impact to the primary database of WAN network latency in a synchronous configuration.
- Production-offload of the overhead of servicing multiple remote standby destinations using far sync. In a far sync configuration, the primary database ships a single stream of redo to a far sync instance using synchronous or asynchronous transport. The far sync instance is able to forward redo asynchronously to as many as 29 remote destinations with zero incremental overhead on the source database.
- Data Guard maximum availability supports the use of the

NOAFFIRM

redo transport attribute. A standby database returns receipt acknowledgment to its primary database as soon as redo is received in memory. The standby database does not wait for the Remote File Server (RFS) to write to a standby redo log file.

This feature provides increased primary database performance in Data Guard configurations using maximum availability and SYNC redo transport. Fast Sync isolates the primary database in a maximum availability configuration from any performance impact

due to slow I/O at a standby database. This new FAST SYNC feature can work with a physical standby target or within a far sync configuration.

- Production-offload of CPU cycles required to perform redo transport compression. Redo transport compression can be performed by the far sync instance if the Data Guard configuration is licensed for Oracle Advanced Compression. This conserves bandwidth with zero incremental overhead on the primary database.
- Production-offload and increased backup performance by moving fast incremental backups off of the primary database and to the standby database by utilizing Oracle Active Data Guard support for RMAN block change tracking.
- Increased high availability using Oracle Active Data Guard automatic block repair to repair block corruptions, including file header corruptions, detected at either the primary or standby, transparent to applications and users.
- Increased high availability by reducing planned downtime for upgrading to new Oracle Database patch sets and database releases using the additional automation provided by high availability Upgrade.
- Connection preservation on an Active Data Guard standby through a role change facilitates improved reporting and improves the user experience. The connections pause while the database role changes to a primary database and resume, improving the user experience.
- The Oracle Enterprise Manager Diagnostic tool can be used with Active Data Guard to capture and send performance data to the Automatic Workload Repository, while the SQL Tuning Advisor allows primary database SQL statement tuning to be offloaded to a standby database.
- Active Data Guard support for the Oracle Database In-Memory option enables reporting to be offloaded to the standby database while reaping the benefits the In-Memory option provides, including tailored column stores for the standby database workload.

Oracle Data Guard Advantages Over Traditional Solutions

Oracle Data Guard provides a number of advantages over traditional solutions.

- Fast, automatic or automated database failover for data corruptions, lost writes, and database and site failures, with recovery times of potentially seconds with Data Guard as opposed to hours with traditional solutions
- Zero data loss over wide area network using Oracle Active Data Guard Far Sync
- Offload processing for redo transport compression and redo transmission to up to 29 remote destinations using Oracle Active Data Guard Far Sync
- Automatic corruption repair automatically replaces a physical block corruption on the primary or physical standby by copying a good block from a physical standby or primary database
- Most comprehensive protection against data corruptions and lost writes on the primary database
- Reduced downtime for storage, Oracle ASM, Oracle RAC, system migrations and some platform migrations, and changes using Data Guard switchover
- Reduced downtime for database upgrades with Data Guard rolling upgrade capabilities
- Ability to off-load primary database activities—such as backups, queries, or reporting—without sacrificing the RTO and RPO ability to use the standby database as a read-only resource using the real-time query apply lag capability, including Database In-Memory column support

- Ability to integrate non-database files using Oracle Database File System (DBFS) or Oracle Automatic Storage Management Cluster File System (Oracle ACFS) as part of the full site failover operations
- No need for instance restart, storage remastering, or application reconnection after site failures
- Transparency to applications
- Transparent and integrated support (application continuity and transaction guard) for application failover
- Effective network utilization
- Database In-Memory support
- Integrated service and client failover that reduces overall application RTO
- Enhanced and integrated Data Guard awareness with existing Oracle technologies such as Oracle RAC, RMAN, Oracle GoldenGate, Enterprise Manager, health check (orachk), DBCA, and Fleet Patch and Provisioning

For data resident in Oracle databases, Data Guard, with its built-in zero-data-loss capability, is more efficient, less expensive, and better optimized for data protection and disaster recovery than traditional remote mirroring solutions. Data Guard provides a compelling set of technical and business reasons that justify its adoption as the disaster recovery and data protection technology of choice, over traditional remote mirroring solutions.

Data Guard and Planned Maintenance

Data Guard standby databases can be used to reduce planned downtime by performing maintenance in a rolling fashion. Changes are implemented first at the standby database. The configuration is allowed to run with the primary at the old version and standby at the new version until there is confidence that the new version is ready for production. A Data Guard switchover can be performed, transitioning production to the new version or same changes can be applied to production in a rolling fashion. The only possible database downtime is the time required to perform the switchover.

There are several approaches to performing maintenance in a rolling fashion using a Data Guard standby. Customer requirements and preferences determine which approach is used.

Data Guard Redo Apply and Standby-First Patching

Beginning with Oracle Database 10g, there has been increased flexibility in cross-platform support using Data Guard Redo Apply.

In certain Data Guard configurations, primary and standby databases are able to run on systems having different operating systems (for example, Windows and Linux), word size (32bit/64bit), different storage, different Exadata hardware and software versions, or different hardware architectures. Redo Apply can also be used to migrate to Oracle Automatic Storage Management (ASM), to move from single instance Oracle databases to Oracle RAC, to perform technology refresh, or to move from one data center to the next.

Beginning with Oracle Database 11g Release 2 (11.2), Standby-First Patch Apply (physical standby using Redo Apply) can support different database software patch levels between a primary database and its physical standby database for the purpose of applying and validating Oracle patches in a rolling fashion. Patches eligible for Standby-First patching include:

- Database Release Updates (RUs) or Release Update Revisions (RURs)
- Database Patch Set Update (PSU)

- Database Critical Patch Update (CPU)
- Database bundled patch

Standby-First Patch Apply is supported for certified database software patches for Oracle Database Enterprise Edition 11g Release 2 (11.2) and later.

In each of the types of planned maintenance previously described, the configuration begins with a primary and physical standby database (in the case of migration to a new platform, or to ASM or Oracle RAC, the standby is created on the new platform). After all changes are implemented at the physical standby database, Redo Apply (physical replication) is used to synchronize the standby with the primary. A Data Guard switchover is used to transfer production to the standby (the new environment).

① See Also

My Oracle Support Note [413484.1](#) for information about mixed platform combinations supported in a Data Guard configuration.

My Oracle Support Note [1265700.1](#) for more information about Standby First Patch Apply and the README for each patch to determine if a target patch is certified as being a Standby-First Patch.

Data Guard Transient Logical Rolling Upgrades

There are numerous types of maintenance tasks that are unable to use Redo Apply (physical replication) to synchronize the original version of a database with the changed or upgraded version. These tasks include:

- Database patches or upgrades that are not Standby-First Patch Apply-eligible. This includes database patch-sets (11.2.0.2 to 11.2.0.4) and upgrade to new Oracle Database releases (18c to 19c).
- Maintenance must be performed that modifies the physical structure of a database that would require downtime (for example, adding partitioning to non-partitioned tables, changing Basicfile LOBs to Securefile LOBs, changing XML-CLOB to Binary XML, or altering a table to be OLTP-compressed).

All of the previous types of maintenance can be performed in a rolling fashion using a Data Guard standby database by using Data Guard SQL Apply (logical replication) to synchronize the old and new versions of the database. Prior to Oracle Database 11g this required creating a logical standby database, performing the maintenance on the logical standby, resynchronizing the standby with the primary, and then switching over. Additionally if a physical standby was being used for disaster recovery, then a new physical standby database would have to be created from a backup of the production database at the new version. This represented a number of logistical and cost challenges when upgrading a multi-terabyte database.

Beginning with Oracle Database 11g, database rolling upgrades can use a new procedure called Transient Logical that begins and ends with a physical standby database. SQL Apply is only used during the phase when Data Guard is synchronizing across old and new versions. A new logical standby database does not need to be created if there is already a physical standby in place. A new physical standby database does not need to be created from a backup of the production database at the new version after the maintenance is complete. Similar to the traditional process of upgrading a Data Guard configuration having an in-place physical standby, the original primary is upgraded or changed using redo from the new primary

database and Redo Apply (a single catalog upgrade migrates both primary and standby databases to the new Oracle release).

Transient Logical upgrades require that the primary database be at Oracle Database 11g release 1 (11.1) or later and that the database meet the prerequisites of SQL Apply.

Oracle provides a Bourne shell script that automates a number of the manual steps required by the Transient Logical rolling upgrade process.

Databases that use Oracle Database Vault can be upgraded to new Oracle Database releases and patch sets by using Oracle Data Guard database rolling upgrades (transient logical standby only).

① See Also

<http://www.oracle.com/goto/maa> for Oracle MAA technical brief “Oracle Database Rolling Upgrades: Using a Data Guard Physical Standby Database”

Rolling Upgrade Using Oracle Active Data Guard

Rolling database upgrade using Oracle Active Data Guard provides a simpler, automated, and easily repeatable method for reducing planned downtime than represented by the manual Transient Logical rolling upgrade procedure.

Rolling upgrade using Oracle Active Data Guard transforms the 42 or more steps required by the manual procedure into several easy-to-use DBMS_ROLLING PL/SQL packages. Rolling upgrades performed using the DBMS_ROLLING PL/SQL package are supported on a multitenant container database (CDB).

A rolling upgrade using Oracle Active Data Guard:

- Generates an upgrade plan with a configuration-specific set of instructions to guide you through the upgrade process.
- Modifies parameters of the rolling upgrade.
- Configures primary and standby databases participating in the upgrade.
- Performs switchover of the production database to the new version. Switchover is the only downtime required.
- Completes the upgrade of the old primary and any additional standby databases in the Data Guard configuration and resynchronizes with the new primary.

Rolling upgrade using Oracle Active Data Guard has the following additional benefits:

- Provides a simple specify-compile-run protocol
 - Catches configuration errors at the compilation step
 - Runtime errors are detected during processing
- The state is kept in the database
 - Enables a reliable, repeatable process
- Runtime steps are constant regardless of how many databases are involved
- Handles failure at the original primary database
- Enables data protection for the upgraded primary at all times

① See Also

<http://www.oracle.com/goto/maa> for Oracle MAA technical brief “Oracle Database Rolling Upgrades: Using a Data Guard Physical Standby Database”

Oracle Data Guard Concepts and Administration

Oracle GoldenGate

Oracle GoldenGate is Oracle's strategic logical replication solution for data distribution and data integration.

Oracle GoldenGate offers a real-time, log-based change data capture and replication software platform. The software provides capture, routing, transformation, and delivery of transactional data across heterogeneous databases in real time.

Unlike replication solutions from other vendors, Oracle GoldenGate is more closely integrated with Oracle Database while also providing an open, modular architecture ideal for replication across heterogeneous database management systems. This combination of attributes eliminates compromise, making Oracle GoldenGate the preferred logical replication solution for addressing requirements that span Oracle Database and non-Oracle Database environments.

A typical environment includes a capture, pump, and delivery process. Each of these processes can run on most of the popular operating systems and databases, including Oracle Database. All or a portion of the data can be replicated, and the data within any of these processes can be manipulated for not only heterogeneous environments but also different database schemas, table names, or table structures. Oracle GoldenGate also supports bidirectional replication with preconfigured conflict detection and resolution handlers to aid in resolving data conflicts.

Oracle GoldenGate logical replication enables all databases in an Oracle GoldenGate configuration, both source and target databases, to be open read-write. This makes it a key component of MAA for addressing a broad range of high availability challenges for zero downtime maintenance, cross platform migration, and continuous data availability, specifically:

- **Zero or near zero downtime maintenance.** In this architecture, Oracle GoldenGate provides greater flexibility than the capabilities provided by Data Guard. Oracle GoldenGate source and target databases can have a different physical and logical structure, can reside on different hardware and operating system architectures, can span wide differences in Oracle Database releases (for example, 12.2 to 19c), or be a mix of Oracle and non-Oracle systems. This allows for the modernization of 24x7 servers and allows new Oracle features to be implemented without impacting the availability of the databases. Maintenance is first performed on a target database while production runs on the source. After the maintenance is complete, production can be moved to the source all at once, similar to a Data Guard switchover. Optionally, bidirectional replication can be used to gradually move users over to the new system to create the perception of zero downtime. In either case, Oracle GoldenGate replication can be enabled in the reverse direction to keep the original source database synchronized during a transition period, making it simple to effect a planned fall-back to the previous version if needed, with minimal downtime and no data loss.
- **Zero or near-zero downtime migrations when a Data Guard solution is not applicable.** Platform or database migrations can be carried out using Oracle GoldenGate as the data synchronization method between the old and new systems. Once the database has been instantiated on another host, Oracle GoldenGate is configured to replicate changes from the production database. A guaranteed restore point can be created on the

migrated database so that after user testing the database can be flashed back, and Oracle GoldenGate can apply any outstanding data changes from the production database before moving the application users to the new database, similar to a snapshot standby database. If desired, bi-directional replication can also be configured from the migrated database back to the production database for use as a fallback solution.

- **Zero or near-zero downtime application upgrades.** Application upgrades that modify back-end database objects typically result in significant planned downtime while maintenance is being performed. Oracle GoldenGate replication enables data transformations that map database objects used by a previous version of an application to objects modified by the new version of an application. This enables database maintenance to be performed on a separate copy of the production database without impacting the availability of the application. After the maintenance is complete and Oracle GoldenGate has finished synchronizing old and new versions, users can be switched to the new version of the application.
- **Read-write access to a replica database while it is being synchronized with its source database.** This is most often used to offload reporting to a copy of a production database when the reporting application requires a read-write connection to database in order to function. This is also relevant to disaster recovery environments where the nature of the technology used for the application tier requires an active read-write connection to the DR database at all times in order to meet recovery time objectives.
- **Active-Active replication.** Oracle GoldenGate supports an active-active multi-directional configuration, where there are two or more systems with identical sets of data that can be changed by application users on either system. Oracle GoldenGate replicates transactional data changes from each database to the others to keep all sets of data current.
- Seamless moves between Oracle Real Application Clusters (RAC) nodes in the event of database instance failure or during applicable maintenance operations. This ability provides high availability with Oracle GoldenGate and it is possible to patch and upgrade the Oracle GoldenGate software on one or more nodes in the cluster without affecting the node where Oracle GoldenGate is currently running. Then at a predetermined time, Oracle GoldenGate can be switched to one of the upgraded nodes. The switch is done without reconfiguring Oracle GoldenGate because configuration information is shared across the Oracle RAC cluster.

① See Also

[Oracle GoldenGate Documentation](#)

<http://www.oracle.com/goto/maa> for Oracle MAA Oracle GoldenGate technical briefs

Best Practice: Oracle Active Data Guard and Oracle GoldenGate

While Oracle Active Data Guard and Oracle GoldenGate are each capable of maintaining a synchronized copy of an Oracle database, each has unique characteristics that result in high availability architectures that can use one technology or the other, or both at the same time, depending upon requirements.

Examples of MAA Best Practice guidelines are as follows:

When to Use Oracle Active Data Guard

Use Oracle Active Data Guard when the emphasis is on simplicity, data protection, and availability.

- Simplest, fastest, one-way replication of a complete Oracle database.
- No restrictions: Data Guard Redo Apply supports all data and storage types and Oracle features; transparent replication of DDL
- Features optimized for data protection: Detects silent corruptions that can occur on source or target; automatically repairs corrupt blocks
- Synchronized standby open read-only provides simple read-only offloading for maximum ROI
- Transparency of backups: A Data Guard primary and standby are physically exact copies of each other; RMAN backups are completely interchangeable
- Zero data loss protection at any distance, without impacting database performance
- Minimizing planned downtime and risk using standby first patching, database rolling upgrades, and select platform migrations
- Reduce risk of introducing change by dual purposing a DR system for testing using Data Guard Snapshot Standby
- Integrated automatic database and client failover
- Integrated management of a complete configuration: Data Guard Broker command line interface or Oracle Enterprise Manager Cloud Control

When to Use Oracle GoldenGate

Use Oracle GoldenGate when the emphasis is on advanced replication requirements not addressed by Oracle Active Data Guard.

- Any requirement where the replica database must be open read/write while synchronizing with the primary database
- Any data replication requirements such as multimaster and bidirectional replication, subset replication, many-to-one replication, and data transformations.
- When data replication is required between endian format platforms or across-database major versions.
- Maintenance and migrations where zero downtime or near zero downtime is required. Oracle GoldenGate can be used to migrate between application versions, for example, from Application 1.0 to Application 2.0 without downtime.
- Database rolling upgrades where it is desired to replicate from new version down to the old version for the purpose of fast fall-back if something is wrong with the upgrade.
- Zero downtime planned maintenance where bidirectional replication is used to gradually migrate users to the new version, creating the perception of zero downtime. Note that bidirectional replication requires avoiding or resolving update conflicts that can occur on disparate databases.

When to Use Oracle Active Data Guard and Oracle GoldenGate Together

Oracle Active Data Guard and Oracle GoldenGate are not mutually exclusive. The following are use cases of high availability architectures that include the simultaneous use of Oracle Active Data Guard and Oracle GoldenGate.

- An Oracle Active Data Guard standby is utilized for disaster protection and database rolling upgrades for a mission critical OLTP database. At the same time, Oracle GoldenGate is used to replicate data from the Data Guard primary database (or from the standby database using Oracle GoldenGate ALO mode) for ETL update of an enterprise data warehouse.
- Oracle GoldenGate subset replication is used to create an operational data store (ODS) that extracts, transforms, and aggregates data from numerous data sources. The ODS supports mission critical application systems that generate significant revenue for the company. An Oracle Active Data Guard standby database is used to protect the ODS, providing optimal data protection and availability.
- Oracle GoldenGate bidirectional replication is utilized to synchronize two databases separated by thousands of miles. User workload is distributed across each database based upon geography, workload, and service level using Global Data Services (GDS). Each Oracle GoldenGate copy has its own local synchronous Data Guard standby database that enables zero data loss failover if an outage occurs. Oracle GoldenGate capture and apply processes are easily restarted on the new primary database following a failover because the primary and standby are an exact, up-to-date replica of each other.
- An Oracle Active Data Guard standby database used for disaster protection is temporarily converted into an Oracle GoldenGate target for the purpose of performing planned maintenance not supported by Data Guard. For example, a Siebel application upgrade requiring modification of back-end database objects which require comprehensive testing before switching users over to the new system.
- Oracle Active Data Guard is used to protect a production environment when a major database version upgrade is required offering zero or near-zero downtime (for example, Oracle 18c to 19c.) A second primary/standby environment is created using the new database version, and Oracle GoldenGate is used to replicate data from the production environment to the copy with one-way or bidirectional replication. When Oracle GoldenGate has completed synchronizing the old and new environments, production is switched to the new environment and the old environment is decommissioned. This provides zero or minimal downtime depending upon configuration, eliminates risk by providing complete isolation between the old and new environment, and avoids any impact to data protection and availability SLAs if problems are encountered during the upgrade process.

📘 See Also

<http://www.oracle.com/goto/maa> for Oracle MAA Best Practices technical brief
“Transparent Role Transitions With Oracle Data Guard and Oracle GoldenGate”

Recovery Manager

Recovery Manager (RMAN) provides a comprehensive foundation for efficiently backing up and recovering the database. RMAN eliminates operational complexity while providing superior performance and availability of the database.

RMAN determines the most efficient method of running the requested backup, restoration, or recovery operation and then submits these operations to the Oracle Database server for processing. RMAN and the server automatically identify modifications to the structure of the database and dynamically adjust the required operation to adapt to the changes.

RMAN is the standard interface to backup and restore from Recovery Appliance, local disk (ZFS storage), tape, and cloud object store.

RMAN provides the following benefits:

- Support for Oracle Sharding - RMAN support for every independent database (shard)
- Enhancement for Sparse Databases - allows backup and restore to operate on

SPARSE

backup sets and or image copies

- Over the Network Standby Database repair of

NONLOGGED

operation - new syntax for validation and repair on Standby -

VALIDATE/RECOVER . . . NONLOGGED BLOCK;

- RMAN DUPLICATE

feature enhanced to support creation of Far Sync from Primary and backup

- RMAN DUPLICATE

Using Encrypted Backups - RMAN enhanced support non Auto-login wallet based encrypted backups with a new

SET

command - enables interrupt-free cloning

- Support for cross-platform backup and restore over the network
- Network-enabled restoration allows the

RESTORE

operations to copy data files directly from one database to another over the network

- Simplified table restoration with the

RECOVER TABLE

command

- Support for Oracle Multitenant, including backup and recovery of individual pluggable databases

- Support for cross-platform Oracle Multitenant, including backup and recovery of individual PDBs
- Automatic channel failover on backup and restore operations
- Automatic failover to a previous backup when the restore operation discovers a missing or corrupt backup
- Automatic creation of new database files and temporary files during recovery
- Automatic recovery through a previous point-in-time recovery—recovery through reset logs
- Block media recovery, which enables the data file to remain online while fixing the block corruption
- Fast incremental backups using block change tracking
- Fast backup and restore operations with intrafile and interfile parallelism
- Enhanced security with a virtual private recovery catalog
- Merger of incremental backups into image copies, providing up-to-date recoverability
- Optimized backup and restoration of required files only
- Retention policy to ensure that relevant backups are retained
- Ability to resume backup and restore operations in case of failure
- Automatic backup of the control file and the server parameter file, ensuring that backup metadata is available in times of database structural changes and media failure and disasters
- Easily reinstantiate a new database from an existing backup or directly from the production database (thus eliminating staging areas) using the

Duplicate

command.

① See Also

Oracle Database Backup and Recovery User's Guide

Oracle Real Application Clusters and Oracle Clusterware

Oracle RAC and Oracle Clusterware enable Oracle Database to run any packaged or custom application across a set of clustered servers.

This capability provides the highest levels of availability and the most flexible scalability. If a clustered server fails, then Oracle Database continues running on the surviving servers. When more processing power is needed, you can add another server without interrupting access to data.

Oracle RAC enables multiple instances that are linked by an interconnect to share access to an Oracle database. In an Oracle RAC environment, Oracle Database runs on two or more systems in a cluster while concurrently accessing a single shared database. The result is a single database system that spans multiple hardware systems, enabling Oracle RAC to provide high availability and redundancy during failures in the cluster. Oracle RAC

accommodates all system types, from read-only data warehouse systems to update-intensive online transaction processing (OLTP) systems.

Oracle Clusterware is software that, when installed on servers running the same operating system, enables the servers to be bound together to operate as if they are one server, and manages the availability of user applications and Oracle databases. Oracle Clusterware also provides all of the features required for cluster management, including node membership, group services, global resource management, and high availability functions:

- For high availability, you can place Oracle databases (single-instance or Oracle RAC databases), and user applications (Oracle and non-Oracle) under the management and protection of Oracle Clusterware so that the databases and applications restart when a process fails or so that a failover to another node occurs after a node failure.
- For cluster management, Oracle Clusterware presents multiple independent servers as if they are a single-system image or one virtual server. This single virtual server is preserved across the cluster for all management operations, enabling administrators to perform installations, configurations, backups, upgrades, and monitoring functions. Then, Oracle Clusterware automatically distributes the processing of these management functions to the appropriate nodes in the cluster.

Oracle Clusterware is a requirement for using Oracle RAC. Oracle Clusterware is the only clusterware that you need for most platforms on which Oracle RAC operates. Although Oracle Database continues to support third-party clusterware products on specified platforms, using Oracle Clusterware provides these main benefits:

- Dispenses with proprietary vendor clusterware
- Uses an integrated software stack from Oracle that provides disk management with local or remote Oracle Automatic Storage Management (Oracle Flex ASM) to data management with Oracle Database and Oracle RAC
- Can be configured in large clusters, called an Oracle Flex Cluster.

In addition, Oracle Database features, such as Oracle services, use the underlying Oracle Clusterware mechanisms to provide their capabilities.

Oracle Clusterware requires two clusterware components: a voting disk to record node membership information and the Oracle Cluster Registry (OCR) to record cluster configuration information. The voting disk and the OCR must reside on shared storage. Oracle Clusterware requires that each node be connected to a private network over a private interconnect.

Benefits of Using Oracle Clusterware

Oracle Clusterware provides the following benefits.

- Tolerates and quickly recovers from computer and instance failures.
- Simplifies management and support by means of using Oracle Clusterware together with Oracle Database. By using fewer vendors and an all Oracle stack you gain better integration compared to using third-party clusterware.
- Performs rolling upgrades for system and hardware changes. For example, you can apply Oracle Clusterware upgrades, patch sets, and interim patches in a rolling fashion.

When you upgrade to Oracle Database 12c, Oracle Clusterware and Oracle ASM binaries are installed as a single binary called the Oracle Grid Infrastructure. You can upgrade Oracle Clusterware in a rolling manner from Oracle Clusterware 10g and Oracle Clusterware 11g; however, you can only upgrade Oracle ASM in a rolling manner from Oracle Database 11g release 1 (11.1).

- Automatically restarts failed Oracle processes.

- Automatically manages the virtual IP (VIP) address. When a node fails, the node's VIP address fails over to another node on which the VIP address can accept connections.
- Automatically restarts resources from failed nodes on surviving nodes.
- Controls Oracle processes as follows:
 - For Oracle RAC databases, Oracle Clusterware controls all Oracle processes by default.
 - For Oracle single-instance databases, Oracle Clusterware enables you to configure the Oracle processes into a resource group that is under the control of Oracle Clusterware.
- Provides an application programming interface (API) for Oracle and non-Oracle applications that enables you to control other Oracle processes with Oracle Clusterware, such as restart or react to failures and certain rules.
- Manages node membership and prevents split-brain syndrome in which two or more instances attempt to control the database.
- Using server weight-based node eviction allows for aligning the choice of which node gets evicted in case of certain failures in the cluster with business requirements, ensuring that the most important workload is kept alive for as long as possible, assuming an equal choice between servers.
- Provides the ability to perform rolling release upgrades of Oracle Clusterware, with no downtime for applications.

Benefits of Using Oracle Real Application Clusters and Oracle Clusterware

Together, Oracle RAC and Oracle Clusterware provide all of the Oracle Clusterware benefits plus the following benefits.

- Provides better integration and support of Oracle Database by using an all Oracle software stack compared to using third-party clusterware.
- Relocate Oracle Service automatically. Plus, when you perform additional fast application notification (FAN) and client configuration, distribute FAN events so that applications can react immediately to achieve fast, automatic, and intelligent connection and failover.
- Detect connection failures fast and automatically, and remove terminated connections for any Java application using Oracle Universal Connection Pool (Oracle UCP) Fast Connection Failover and FAN events.
- Balance work requests using Oracle UCP runtime connection load balancing.
- Use runtime connection load balancing with Oracle UCP, Oracle Call Interface (OCI), and Oracle Data Provider for .NET (ODP.NET).
- Distribute work across all available instances using load balancing advisory.
- You can configure a database so that Oracle Clusterware is aware of the CPU requirements and limits for the given database. Oracle Clusterware uses this information to place the database resource only on servers that have a sufficient number of CPUs, amount of memory, or both.
- Allow the flexibility to increase processing capacity using commodity hardware without downtime or changes to the application.
- Provide comprehensive manageability integrating database and cluster features.
- Provide scalability across database instances.
- Implement Fast Connection Failover for nonpooled connections.

Oracle RAC Advantages Over Traditional Cold Cluster Solutions

Oracle RAC provides many advantages over traditional cold cluster solutions, including the following.

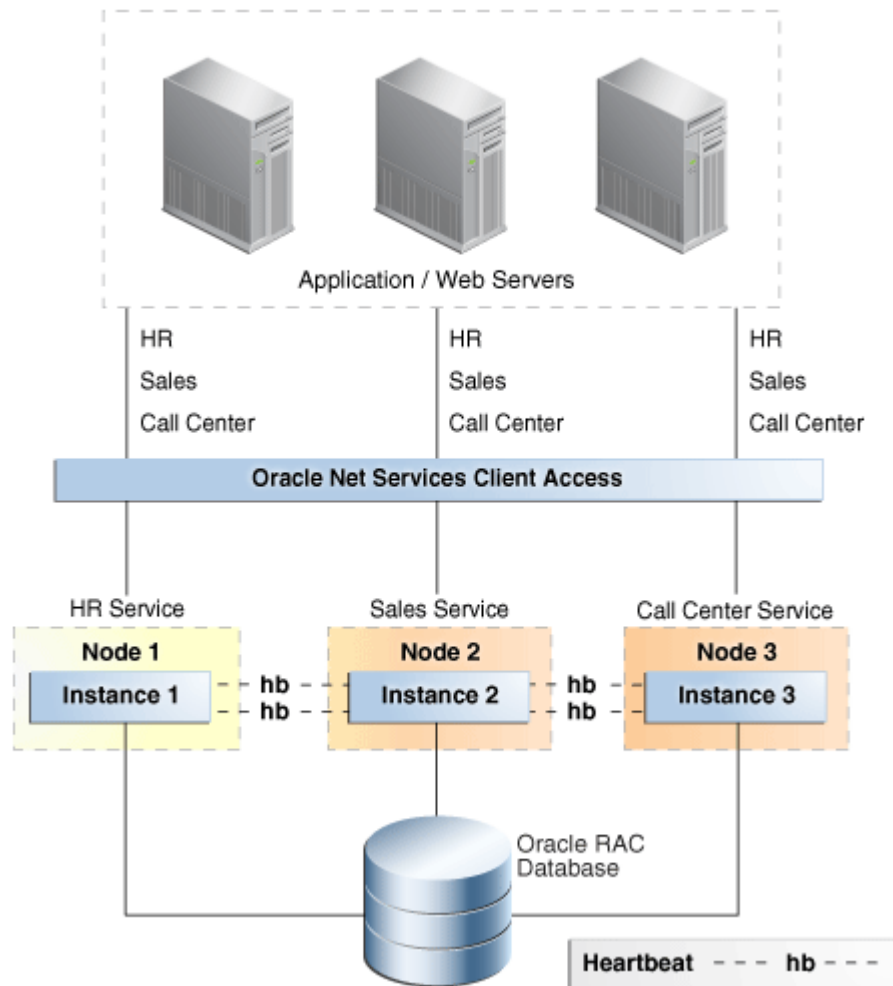
- Scalability across database instances
- Flexibility to increase processing capacity using commodity hardware without downtime or changes to the application
- Ability to tolerate and quickly recover from computer and instance failures (measured in seconds)
- Application brownout can be zero or seconds compared to minutes and hours with cold cluster solutions
- Optimized communication in the cluster over redundant network interfaces, without using bonding or other technologies

Oracle Grid Infrastructure and Oracle RAC make use of Redundant Interconnect Usage that distributes network traffic and ensures optimal communication in the cluster. This functionality is available starting with Oracle Database 11g Release 2 (11.2.0.2). In previous releases, technologies like bonding or trunking were used to make use of redundant networks for the interconnect.

- Rolling upgrades for system and hardware changes
- Rolling patch upgrades for some interim patches, security patches, CPUs, and cluster software
- Fast, automatic, and intelligent connection and service relocation and failover
- Comprehensive manageability integrating database and cluster features with Grid Plug and Play and policy-based cluster and capacity management
- Load balancing advisory and run-time connection load balancing help redirect and balance work across the appropriate resources
- Oracle Quality of Service (QoS) Management for policy-based run-time management of resource allocation to database workloads to ensure service levels are met in order of business need under dynamic conditions. This is accomplished by assigning a service to a server pool where the database is running. Resources from the pool are used to make sure the required capacity is available.
- Oracle Enterprise Management support for Oracle ASM and Oracle ACFS, Grid Plug and Play, Cluster Resource Management, Oracle Clusterware and Oracle RAC Provisioning and patching.
- SCAN (Single Client Access Name) support as a single name to the clients connecting to Oracle RAC that does not change throughout the life of the cluster, even if you add or remove nodes from the cluster.

The following figure shows Oracle Database with Oracle RAC architecture. This figure shows Oracle Database with Oracle RAC architecture for a partitioned three-node database. An Oracle RAC database is connected to three instances on different nodes. Each instance is associated with a service: HR, Sales, and Call Center. The instances monitor each other by checking "heartbeats." Oracle Net Services provide client access to the Application/web server tier at the top of the figure.

Figure 3-2 Oracle Database with Oracle RAC Architecture

**Note**

After Oracle release 11.2, Oracle RAC One Node or Oracle RAC is the preferred solution over Oracle Clusterware (Cold Cluster Failover) because it is a more complete and feature-rich solution.

See Also

Oracle RAC Administration and Deployment Guide

Oracle Clusterware Administration and Deployment Guide

Oracle RAC One Node

Oracle Real Application Clusters One Node (Oracle RAC One Node) is a single instance of an Oracle RAC database that runs on one node in a cluster.

This feature enables you to consolidate many databases into one cluster with minimal overhead, protecting them from both planned and unplanned downtime. The consolidated databases reap the high availability benefits of failover protection, online rolling patch application, and rolling upgrades for the operating system and Oracle Clusterware.

Oracle RAC One Node enables better availability than cold failover for single-instance databases because of the Oracle technology called online database relocation, which intelligently migrates database instances and connections to other cluster nodes for high availability and load balancing. Online database relocation is performed using the Server Control Utility (SRVCTL).

Oracle RAC One Node provides the following:

- Always available single-instance database services
- Built-in cluster failover for high availability
- Live migration of instances across servers
- Online rolling patches and rolling upgrades for single-instance databases
- Online upgrade from single-instance to multiple-instance Oracle RAC
- Better consolidation for database servers
- Enhanced server virtualization
- Lower cost development and test platform for full Oracle RAC
- Relocation of Oracle RAC primary and standby databases configured with Data Guard. This functionality is available starting with Oracle Database 11g Release 2 (11.2.0.2).

Oracle RAC One Node also facilitates the consolidation of database storage, standardizes your database environment, and, when necessary, enables you to transition to a full, multiple-instance Oracle RAC database without downtime or disruption.

Oracle Automatic Storage Management

Oracle ASM provides a vertically integrated file system and volume manager directly in the Oracle Database kernel.

This design provides several benefits, resulting in:

- Significantly less work to provision database storage
- Higher level of availability
- Elimination of the expense, installation, and maintenance of specialized storage products
- Unique capabilities for database applications

For optimal performance, Oracle ASM spreads files across all available storage. To protect against data loss, Oracle ASM extends the concept of SAME (stripe and mirror everything) and adds more flexibility because it can mirror at the database file level rather than at the entire disk level.

More important, Oracle ASM simplifies the processes of setting up mirroring, adding disks, and removing disks. Instead of managing hundreds or possibly thousands of files (as in a large data warehouse), database administrators using Oracle ASM create and administer a larger-grained object called a disk group. The disk group identifies the set of disks that are managed as a logical unit. Automation of file naming and placement of the underlying database files save administrators time and ensure adherence to standard best practices.

The Oracle ASM native mirroring mechanism (two-way or three-way) protects against storage failures. With Oracle ASM mirroring, you can provide an additional level of data protection with the use of failure groups. A failure group is a set of disks sharing a common resource (disk controller or an entire disk array) whose failure can be tolerated. After it is defined, an Oracle ASM failure group intelligently places redundant copies of the data in separate failure groups. This ensures that the data is available and transparently protected against the failure of any component in the storage subsystem.

By using Oracle ASM, you can:

- Mirror and stripe across drives and storage arrays.
- Automatically remirror from a failed drive to remaining drives.
- Automatically rebalance stored data when disks are added or removed while the database remains online.
- Support Oracle database files and non-database files using Oracle Automatic Storage Management Cluster File System (Oracle ACFS).
- Allow for operational simplicity in managing database storage.
- Manage the Oracle Cluster Registry (OCR) and voting disks.
- Provide preferred read capability on disks that are local to the instance, which gives better performance for an extended cluster.
- Support very large databases.
- Support Oracle ASM rolling upgrades.
- Improve availability and reliability using the Oracle ASM disk scrubbing process to find and repair logical data corruptions using mirror disks.
- Support finer granularity in tuning and security.
- Provide fast repair after a temporary disk failure through Oracle ASM Fast Mirror Resync and automatic repair of block corruptions if a good copy exists in one of the mirrors.
- Provide disaster recovery capability for the file system by enabling replication of Oracle ACFS across the network to a remote site.
- Patch the Oracle ASM instance without impacting the clients that are being serviced using Oracle Flex ASM. A database instance can be directed to access Oracle ASM metadata from another location while the current Oracle ASM instance it is connected to is taken offline for planned maintenance.
- Monitor and manage the speed and status of Oracle ASM Disk Resync and Rebalance operations.
- Bring online multiple disks simultaneously and manage performance better by controlling resync parallelism using the Oracle ASM Resync Power Limit. Recover faster after a cell or disk failure, and the instance doing the resync is failing; this is made possible by using a Disk Resync Checkpoint which enables a resync to resume from where it was interrupted or stopped instead of starting from the beginning.
- Automatically connect database instances to another Oracle ASM instance using Oracle Flex ASM. The local database instance can still access the required metadata and data if an Oracle ASM instance fails due to an unplanned outage.
- Use flex diskgroups to prioritize high availability benefits across multiple databases all using the same diskgroup. Some of the key HA benefits are file extent redundancy, rebalance power limit, and rebalance priority. With flex diskgroups, you can set different values for the above features for different databases, resulting in prioritization across multiple databases within one diskgroup.

- Use flex diskgroups to implement `quoto_groups` across multiple databases sharing one diskgroup which helps in space management and protection.
- Use flex diskgroups to create point-in-time database clones using the ASM split mirror feature.
- Use preferred reads with stretch clusters to improve performance by affinitizing reads to a site.

① See Also

Oracle Automatic Storage Management Administrator's Guide

Fast Recovery Area

The fast recovery area is a unified storage location for all recovery-related files and activities in Oracle Database.

After this feature is enabled, all RMAN backups, archived redo log files, control file autobackups, flashback logs, and data file copies are automatically written to a specified file system or Oracle ASM disk group, and the management of this disk space is handled by RMAN and the database server.

Performing a backup to disk is faster because using the fast recovery area eliminates the bottleneck of writing to tape. More important, if database media recovery is required, then data file backups are readily available. Restoration and recovery time is reduced because you do not need to find a tape and a free tape device to restore the needed data files and archived redo log files.

The fast recovery area provides the following benefits:

- Unified storage location of related recovery files
- Management of the disk space allocated for recovery files, which simplifies database administration tasks
- Fast, reliable, disk-based backup and restoration

① See Also

Oracle Database Backup and Recovery User's Guide

Corruption Prevention, Detection, and Repair

Data block corruptions can be very disruptive and challenging to repair. Corruptions can cause serious application and database downtime and data loss when encountered and worse yet it can go undetected for hours, days and even weeks leading to even longer application downtime once detected. Unfortunately, there is not one way to comprehensively prevent, detect, and repair data corruptions within the database because the source and cause of corruptions can be anywhere in memory, hardware, firmware, storage, operating system, software, or user error. Worse yet, third-party solutions that do not understand Oracle data block semantics and how Oracle changes data blocks do not prevent and detect data block corruptions well. Third party remote mirroring technologies can propagate data corruptions to

the database replica (standby) leading to a double failure, data loss, and much longer downtime. Third party backup and restore solutions cannot detect corrupted backups or bad sectors until a restore or validate operation is issued, resulting in longer restore times and once again potential data loss.

Oracle MAA has a comprehensive plan to prevent, detect, and repair all forms of data block corruptions including physical block corruptions, logical block corruptions, stray writes, and lost writes. These additional safeguards provide the most comprehensive Oracle data block corruption prevention, detection, and repair solution. Details of this plan are described in the My Oracle Support note "Best Practices for Corruption Detection, Prevention, and Automatic Repair - in a Data Guard Configuration (Doc ID 1302539.1)."

The following outlines block corruption checks for various manual operational checks and runtime and background corruption checks. Database administrators and the operations team can incorporate manual checks such as running Oracle Recovery Manager (RMAN) backups, RMAN "check logical" validations, or running the `ANALYZE VALIDATE STRUCTURE` command on important objects. Manual checks are especially important to validate data that are rarely updated or queried.

Runtime checks are far superior in that they catch corruptions almost immediately or during runtime for actively queried and updated data. Runtime checks can prevent corruptions or automatically fix corruptions resulting in better data protection and higher application availability. A new background check has been introduced in Exadata to automatically scan and scrub disks intelligently with no application overhead and to automatically fix physically corrupted blocks.

Table 3-1 Summary of Block Corruption Checks

Checks	Capabilities	Physical Block Corruption	Logical Block Corruption
Manual checks	Dbverify, Analyze	Physical block checks	Logical intra-block and inter-object consistency checks
Manual checks	RMAN	Physical block checks during backup and restore operations	Intra-block logical checks
Manual checks	ASM Scrub	Physical block checks	Some logical intra-block checks
Runtime checks	Oracle Active Data Guard	1. Continuous physical block checking at standby during transport and apply 2. Strong database isolation eliminates single point database failure 3. Automatic repair of block corruptions, including file block headers in Oracle Database 12c Release 2 4. Automatic database failover	1. With <code>DB_LOST_WRITE_PROTECT</code> enabled, detection of lost writes (11.2 and higher). With 11.2.0.4 and Data Guard broker, ability to shutdown the primary when lost writes are detected on the primary database. 2. With <code>DB_BLOCK_CHECKING</code> enabled on the standby, additional intra-block logical checks

Table 3-1 (Cont.) Summary of Block Corruption Checks

Checks	Capabilities	Physical Block Corruption	Logical Block Corruption
Runtime checks	Database	With DB_BLOCK_CHECKSUM, in-memory data block and redo checksum validation	With DB_BLOCK_CHECKING, in-memory intra-block check validation Starting in Oracle Database 18c, and with Shadow Lost Write Protection enabled, Oracle tracks system change numbers (SCNs) for tracked data files and enables early lost write detection. When lost writes are detected, an error is returned immediately. See Shadow Lost Write Protection description following this table.
Runtime checks	ASM and ASM software mirroring (inherent in Exadata, Supercluster, and Zero Data Loss Recovery Appliance)	Implicit data corruption detection for reads and writes and automatic repair if good ASM extent block pair is available during writes	.
Runtime checks	DIX + T10 DIF	Checksum validation from operating system to HBA controller to disk (firmware). Validation for reads and writes for certified Linux, HBA and disks.	.
Runtime checks	Hardware and Storage	Limited checks due to lack of Oracle integration. Checksum is most common.	Limited checks due to lack of Oracle integration. Checksum is most common
Runtime checks	Exadata	Comprehensive HARD checks on writes	HARD checks on writes
Background checks	Exadata	Automatic HARD disk scrub and repair. Detects and fixes bad sectors.	.

Shadow Lost Write Protection

New in Oracle Database 18c, shadow lost write protection detects a lost write before it can result in a major data corruption. You can enable shadow lost write protection for a database, a tablespace, or a data file without requiring an Oracle Data Guard standby database. Shadow lost write protection provides fast detection and immediate response to a lost write, thus minimizing the data loss that can occur in a database due to data corruption.

① See Also

Oracle Database Reference for more information about the views and initialization parameters

My Oracle Support Note [1302539.1](#)

Data Recovery Advisor

Data Recovery Advisor automatically diagnoses persistent (on-disk) data failures, presents appropriate repair options, and runs repair operations at your request.

You can use Data Recovery Advisor to troubleshoot primary databases, logical standby databases, physical standby databases, and snapshot standby databases.

Data Recovery Advisor includes the following functionality:

- **Failure diagnosis**
The first symptoms of database failure are usually error messages, alarms, trace files and dumps, and failed health checks. Assessing these symptoms can be complicated, error-prone, and time-consuming. Data Recovery Advisor automatically diagnoses data failures and informs you about them.
- **Failure impact assessment**
After a failure is diagnosed, you must understand its extent and assess its impact on applications before devising a repair strategy. Data Recovery Advisor automatically assesses the impact of a failure and displays it in an easily understood format.
- **Repair generation**
Even if a failure was diagnosed correctly, selecting the correct repair strategy can be error-prone and stressful. Moreover, there is often a high penalty for making poor decisions in terms of increased downtime and loss of data. Data Recovery Advisor automatically determines the best repair for a set of failures and presents it to you.
- **Repair feasibility checks**
Before presenting repair options, Data Recovery Advisor validates them with respect to the specific environment and availability of media components required to complete the proposed repair, including restoring files directly from the primary or standby database to complete the proposed repair.
- **Repair automation**
If you accept the suggested repair option, Data Recovery Advisor automatically performs the repair, verifies that the repair was successful, and closes the appropriate failures.
- **Validation of data consistency and database recoverability**
Data Recovery Advisor can validate the consistency of your data, and backups and redo stream, whenever you choose.
- **Early detection of corruption**
Through Health Monitor, you can schedule periodic runs of Data Recovery Advisor diagnostic checks to detect data failures before a database process running a transaction discovers the corruption and signals an error. Early warnings can limit the damage caused by corruption.

- Integration of data validation and repair

Data Recovery Advisor is a single tool for data validation and repair.

① Note

Data Recovery Advisor only supports single-instance databases. Oracle RAC databases are not supported.

① See Also

Oracle Database Backup and Recovery User's Guide for information about Data Recovery Advisor supported database configurations.

Oracle Flashback Technology

Oracle Flashback technology is a group of Oracle Database features that let you view past states of database, database objects, transactions or rows or to rewind the database, database objects, transactions or rows to a previous state without using point-in-time media recovery.

With flashback features, you can:

- Perform queries to show data as it looked at a previous point in time
- Perform queries that return metadata that shows a detailed history of changes to the database
- Recover tables or rows to a previous point in time
- Automatically track and archive transactional data changes
- Roll back a transaction and its dependent transactions while the database remains online
- Undrop a table
- Recover a database to a point-in-time without a restore operation

Other than the flashback database feature, most Oracle Flashback features use the Automatic Undo Management (AUM) system to obtain metadata and historical data for transactions. They rely on undo data, which are records of the effects of individual transactions. For example, if a user runs an UPDATE statement to change a salary from 1000 to 1100, then Oracle Database stores the value 1000 in the undo data.

Undo data is persistent and survives a database shutdown. By using flashback features, you can use undo data to query past data or recover from logical damage. Besides using it in flashback features, Oracle Database uses undo data to perform these actions:

- Roll back active transactions
- Recover terminated transactions by using database or process recovery
- Provide read consistency for SQL queries

Oracle Flashback can address and rewind data that is compromised due to various human or operator errors that inadvertently or maliciously change data, cause bad installations and upgrades, and result in logical errors in applications. These problems use features such as flashback transaction, flashback drop, flashback table, and flashback database.

① See Also

Oracle Database Development Guide

Performing Flashback and Database Point-in-Time Recovery, Using Flashback Database and Restore Points, and Performing Block Media Recovery in the *Oracle Database Backup and Recovery User's Guide*

Oracle Database PL/SQL Packages and Types Reference

Oracle Database Backup and Recovery Reference

Oracle Flashback Query

Oracle Flashback Query (Flashback Query) provides the ability to view data as it existed in the past by using the Automatic Undo Management system to obtain metadata and historical data for transactions.

Undo data is persistent and survives a database malfunction or shutdown. The unique features of Flashback Query not only provide the ability to query previous versions of tables, they also provide a powerful mechanism to recover from erroneous operations.

Uses of Flashback Query include:

- Recovering lost data or undoing incorrect, committed changes. For example, rows that were deleted or updated can be immediately repaired even after they were committed.
- Comparing current data with the corresponding data at some time in the past. For example, by using a daily report that shows the changes in data from yesterday, it is possible to compare individual rows of table data, or find intersections or unions of sets of rows.
- Checking the state of transactional data at a particular time, such as verifying the account balance on a certain day.
- Simplifying application design by removing the need to store certain types of temporal data. By using Flashback Query, it is possible to retrieve past data directly from the database.
- Applying packaged applications, such as report generation tools, to past data.
- Providing self-service error correction for an application, enabling users to undo and correct their errors.

Oracle Flashback Version Query

Oracle Flashback Version Query is an extension to SQL that you can use to retrieve the versions of rows in a given table that existed at a specific time interval.

Oracle Flashback Version Query returns a row for each version of the row that existed in the specified time interval. For any given table, a new row version is created each time the COMMIT statement is issued.

Oracle Flashback Version Query is a powerful tool that database administrators (database administrators) can use to run analysis to determine the source of problems. Additionally, application developers can use Oracle Flashback Version Query to build customized applications for auditing purposes.

Oracle Flashback Transaction

Oracle Flashback Transaction backs out a transaction and its dependent transactions.

The

`DBMS_FLASHBACK.TRANSACTION_BACKOUT()`

procedure rolls back a transaction and its dependent transactions while the database remains online. This recovery operation uses undo data to create and run the compensating transactions that return the affected data to its original state. You can query the

`DBA_FLASHBACK_TRANSACTION_STATE`

view to see whether the transaction was backed out using dependency rules or forced out by either:

- Backing out nonconflicting rows
- Applying undo SQL

Oracle Flashback Transaction increases availability during logical recovery by quickly backing out a specific transaction or set of transactions and their dependent transactions. You use one command to back out transactions while the database remains online.

Oracle Flashback Transaction Query

Oracle Flashback Transaction Query provides a mechanism to view all of the changes made to the database at the transaction level.

When used in conjunction with Oracle Flashback Version Query, it offers a fast and efficient means to recover from a human or application error. Oracle Flashback Transaction Query increases the ability to perform online diagnosis of problems in the database by returning the database user that changed the row, and performs analysis and audits on transactions.

Oracle Flashback Table

Oracle Flashback Table recovers a table to a previous point in time.

It provides a fast, online solution for recovering a table or set of tables that were changed by a human or application error. In most cases, Oracle Flashback Table alleviates the need for administrators to perform more complicated point-in-time recovery operations. The data in the original table is not lost when you use Oracle Flashback Table because you can return the table to its original state.

Oracle Flashback Drop

Although there is no easy way to recover dropped tables, indexes, constraints, or triggers, Oracle Flashback Drop provides a safety net when you are dropping objects.

When you drop a table, it is automatically placed into the Recycle Bin. The Recycle Bin is a virtual container where all dropped objects reside. You can continue to query data in a dropped table.

Restore Points

When an Oracle Flashback recovery operation is performed on the database, you must determine the point in time—identified by the system change number (SCN) or time stamp—to which you can later flash back the data.

Oracle Flashback restore points are labels that you can define to substitute for the SCN or transaction time used in Flashback Database, Flashback Table, and Oracle Recovery Manager (RMAN) operations. Furthermore, a database can be flashed back through a previous database recovery and opened with an

`OPEN RESETLOGS`

command by using guaranteed restore points. Guaranteed restore points allow major database changes—such as database batch jobs, upgrades, or patches—to be quickly undone by ensuring that the undo required to rewind the database is retained.

Using the restore points feature provides the following benefits:

- The ability to quickly restore to a consistent state, to a time before a planned operation that has gone awry (for example, a failed batch job, an Oracle software upgrade, or an application upgrade)
- The ability to resynchronize a snapshot standby database with the primary database
- A quick mechanism to restore a test or cloned database to its original state

Oracle Flashback Database

Oracle Flashback Database is the equivalent of a fast rewind button, quickly returning a database to a previous point in time without requiring a time consuming restore and roll forward using a backup and archived logs.

The larger the size of the database, the greater the advantage of using Oracle Flashback Database for fast point in time recovery.

Enabling Oracle Flashback Database provides the following benefits:

- Fast point in time recovery to repair logical corruptions, such as those caused by administrative error.
- Useful for iterative testing when used with Oracle restore points. A restore point can be set, database changes implemented, and test workload run to assess impact. Oracle Flashback Database can then be used to discard the changes and return the database to the original starting point, different modifications can be made, and the same test workload run a second time to have a true basis for comparing the impact of the different configuration changes.
- Data Guard uses Oracle Flashback Database to quickly reinstantiate a failed primary database as a new standby (after a failover has occurred), without requiring the failed primary to be restored from a backup.
- Flashback database operates at the CDB level or the PDB level.

Flashback Pluggable Database

You can rewind a PDB to a previous SCN. The `FLASHBACK PLUGGABLE DATABASE` command, which is available through SQL or Recovery Manager, is analogous to `FLASHBACK DATABASE` in a non-CDB.

Flashback PDB protects an individual PDB against data corruption, widespread user errors, and redo corruption. The operation does not rewind data in other PDBs in the CDB.

You can use

```
CREATE RESTORE POINT ... FOR PLUGGABLE
      DATABASE
```

to create a PDB restore point, which is only usable within a specified PDB. As with CDB restore points, PDB restore points can be normal or guaranteed. A guaranteed restore point never ages out of the control file and must be explicitly dropped. If you connect to the root, and if you do not specify the

```
FOR PLUGGABLE
      DATABASE
```

clause, then you create a CDB restore point, which is usable by all PDBs.

A special type of PDB restore point is a clean restore point, which you can only create when a PDB is closed. For PDBs with shared undo, rewinding the PDB to a clean restore point is faster than other options because it does not require restoring backups or creating a temporary database instance.

Block Media Recovery Using Flashback Logs or Physical Standby Database

After attempting to automatically repair corrupted blocks, block media recovery can optionally retrieve a more recent copy of a data block from the flashback logs to reduce recovery time.

Automatic block repair allows corrupt blocks on the primary database to be automatically repaired as soon as they are detected, by using good blocks from a physical standby database.

Furthermore, a corrupted block encountered during instance recovery does not result in instance recovery failure. The block is automatically marked as corrupt and added to the RMAN corruption list in the

```
V$DATABASE_BLOCK_CORRUPTION
```

table. You can subsequently issue the RMAN

```
RECOVER BLOCK
```

command to fix the associated block. In addition, the RMAN

```
RECOVER BLOCK
```

command restores blocks from a physical standby database, if it is available.

Flashback Data Archive

The Flashback Data Archive is stored in a tablespace and contains transactional changes to every record in a table for the duration of the record's lifetime.

The archived data can be retained for a much longer duration than the retention period offered by an undo tablespace, and used to retrieve very old data for analysis and repair.

Oracle Data Pump and Data Transport

Oracle Data Pump technology enables very high-speed movement of data and metadata from one database to another. Data Pump is used to perform the following planned maintenance activities:

- Database migration to a different platform
- Database migration to pluggable databases
- Database upgrade

The Data Pump features that enable the planned maintenance activities listed above are the following:

- Full transportable export/import to move an entire database to a different database instance
- Transportable tablespaces to move a set of tablespaces between databases

① See Also

Transporting Data

Oracle Replication Technologies for Non-Database Files

Oracle ASM Cluster File System, Oracle Database File System, and Oracle Solaris ZFS Storage Appliance Replication are the Oracle replication technologies for non-database files.

Table 3-2 Oracle Replication Technologies for Non-Database Files

Technology	Recommended Usage	Comments
Oracle ASM Cluster File System	Recommended to provide a single-node and cluster-wide file system solution integrated with Oracle ASM, Oracle Clusterware, and Oracle Enterprise Manager technologies. Provides a loosely coupled full stack replication solution when combined with Data Guard or Oracle GoldenGate.	Oracle ACFS establishes and maintains communication with the Oracle ASM instance to participate in Oracle ASM state transitions including Oracle ASM instance and disk group status updates and disk group rebalancing. Supports many database and application files, including executables, database trace files, database alert logs, application reports, BFILEs, and configuration files. Other supported files are video, audio, text, images, engineering drawings, and other general-purpose application file data. Can provide near-time consistency between database changes and file system changes when point-in-time recovery happens Can be exported and accessed by remote clients using standard NAS File Access Protocols such as NFS and CIFS.
Oracle Database File System	Recommended for providing stronger synchronization between database and non-database systems.	Can be integrated with the database to maintain complete consistency between the database changes and the file system changes All data stored in the database and can be used with Oracle Active Data Guard to provide both disaster recovery and read-only access Can take advantage all of the Oracle database features
Oracle Solaris ZFS Storage Appliance Replication	Recommended for disaster recovery protection for non-database files, and specifically for Oracle Fusion Middleware critical files stored outside of the database.	Replicates all non-database objects, including Oracle Fusion Middleware binaries configuration Can provide near time consistency between database changes and file system changes when point-in-time recovery happens

Oracle ASM Cluster File System

Oracle ASM Cluster File System (ACFS) is a multiplatform, scalable file system, and storage management technology that extends Oracle Automatic Storage Management (Oracle ASM) functionality to support customer files maintained outside of Oracle Database.

Oracle ACFS supports many database and application files, including executables, database trace files, database alert logs, application reports, BFILEs, and configuration files. Other supported files are video, audio, text, images, engineering drawings, and other general-purpose application file data.

Oracle ACFS takes advantage of the following Oracle ASM functionality:

- Oracle ACFS dynamic file system resizing
- Maximized performance through direct access to Oracle ASM disk group storage

- Balanced distribution of Oracle ACFS across Oracle ASM disk group storage for increased I/O parallelism
- Data reliability through Oracle ASM mirroring protection mechanisms

Oracle ACFS Replication, similar to Data Guard for the database, enables replication of Oracle ACFS file systems across the network to a remote site, providing disaster recovery capability for the file system. Oracle ACFS replication captures file system changes written to disk for a primary file system and records the changes in files called replication logs. These logs are transported to the site hosting the associated standby file system where background processes read the logs and apply the changes recorded in the logs to the standby file system. After the changes recorded in a replication log are successfully applied to the standby file system, the replication log is deleted from the sites hosting the primary and standby file systems.

An additional feature of Oracle ACFS is that it offers snapshot-based replication for generic and application files, providing an HA solution for disaster recovery and Test/Development environments. Oracle Databases stored in ACFS can leverage Oracle Multitenant and ACFS snapshot technologies to create quick and efficient snapshot clones of pluggable databases.

Oracle Data Guard and Oracle ACFS can be combined to provide a full stack high availability solution with Data Guard protecting the database with a standby database and Oracle ACFS replicating the file system changes to the standby host. For planned outages the file system and the database remain consistent to a point in time with zero data loss.

① See Also

[Oracle ACFS ASM Cluster File System: What is it and How to use it](#)

<http://www.oracle.com/goto/maa> for Oracle MAA technical brief “Full Stack Role Transition - Oracle ACFS and Oracle Data Guard”

Oracle Database File System

Oracle Database File System (DBFS) takes advantage of the features of the database to store files, and the strengths of the database in efficiently managing relational data, to implement a standard file system interface for files stored in the database.

With this interface, storing files in the database is no longer limited to programs specifically written to use BLOB and CLOB programmatic interfaces. Files in the database can now be transparently accessed using any operating system (OS) program that acts on files. For example, extract, transform, and load (ETL) tools can transparently store staging files in the database.

Oracle DBFS provides the following benefits:

- Full stack integration recovery and failover: By storing file system files in a database structure, it is possible to easily perform point-in-time recovery of both database objects and file system data.
- Disaster Recovery System Return on Investment (ROI): All changes to files contained in DBFS are also logged through the Oracle database redo log stream and thus can be passed to a Data Guard physical standby database. Using Oracle Active Data Guard technology, the DBFS file system can be mounted read-only using the physical standby database as the source. Changes made on the primary are propagated to the standby database and are visible once applied to the standby.

- File system backups: Because DBFS is stored in the database as database objects, standard RMAN backup and recovery functionality can be applied to file system data. Any backup, restore, or recovery operation that can be performed on a database or object within a database can also be performed against the DBFS file system.

① See Also

Database File System (DBFS)

Oracle Solaris ZFS Storage Appliance Replication

The Oracle Solaris ZFS Storage Appliance series supports snapshot-based replication of projects and shares from a source appliance to any number of target appliances manually, on a schedule, or continuously.

The Oracle Solaris ZFS Storage Appliance series supports the following use cases:

- Disaster recovery: Replication can be used to mirror an appliance for disaster recovery. In the event of a disaster that impacts the service of the primary appliance (or even an entire data center), administrators activate the service at the disaster recovery site, which takes over using the most recently replicated data. When the primary site is restored, data changed while the disaster recovery site was in service can be migrated back to the primary site, and normal service is restored. Such scenarios are fully testable before a disaster occurs.
- Data distribution: Replication can be used to distribute data (such as virtual machine images or media) to remote systems across the world in situations where clients of the target appliance would not ordinarily be able to reach the source appliance directly, or such a setup would have prohibitively high latency. One example uses this scheme for local caching to improve latency of read-only data (such as documents).
- Disk-to-disk backup: Replication can be used as a backup solution for environments in which tape backups are not feasible. Tape backup might not be feasible, for example, because the available bandwidth is insufficient or because the latency for recovery is too high.
- Data migration: Replication can be used to migrate data and configuration between Oracle Solaris ZFS Storage appliances when upgrading hardware or rebalancing storage. Shadow migration can also be used for this purpose.

The architecture of Oracle Solaris ZFS Storage Appliance also makes it an ideal platform to complement Data Guard for disaster recovery of Oracle Fusion Middleware. Oracle Fusion Middleware has a number of critical files that are stored outside of the database. These binaries, configuration data, metadata, logs and so on also require data protection to ensure availability of the Oracle Fusion Middleware. For these, the built-in replication feature of the ZFS Storage Appliance is used to move this data to a remote disaster recovery site.

Benefits of the Oracle Solaris ZFS Storage Appliance when used with Oracle Fusion Middleware include:

- Leverages remote replication for Oracle Fusion Middleware
- Provides ability to quickly create clones and snapshots of databases to increase ROI of DR sites

① See Also

[Oracle ZFS Storage Appliance Software](#)

Oracle Multitenant

Oracle Multitenant is the optimal database consolidation method. The multitenant architecture combines the best attributes of each of the previous consolidation methods without their accompanying tradeoffs.

Oracle Multitenant helps reduce IT costs by simplifying consolidation, provisioning, upgrades and more. This new architecture allows a container database (CDB) to hold many pluggable databases (PDBs). To applications, these PDBs appear as a standalone database, and no changes are required to the application in order to access the PDB. By consolidating multiple databases as PDBs into a single CDB, you are provided with the ability to manage "many as one". The flexibility remains to operate on PDBs in isolation should your business require it.

Oracle Multitenant is fully compliant with and takes direct advantage of high availability features such as Oracle Real Application Clusters, Oracle Data Guard, and Oracle GoldenGate, just like any non-container database (non-CDB), meaning it can be used in any of the Oracle MAA reference architectures. Grouping multiple PDBs with the same high availability requirements into the same CDB ensures that all of those PDBs and their applications are managed and protected with the same technologies and configurations.

Benefits of Using Oracle Multitenant

- High consolidation density - Many PDBs can be stored in a single CDB. These PDBs share background processes and memory structures letting you run more PDBs than you would non-CDBs, because the overhead for each non-CDB is removed or reduced. You can store up to 4095 PDBs in a CDB. Each PDB can also have a different character set from other PDBs within the same CDB, as long as the CDB root character set is a superset of all of the PDBs' character sets. Logical standby databases also support this mix of character sets to allow rolling upgrades with a transient logical standby database.
- Online provisioning operations, including clones, refreshable clones, and PDB relocation - A PDB can be unplugged from one CDB and plugged into another. A PDB can also be cloned either into the same CDB or into a different CDB. Cloning can be used to create a "gold image" or seed database for DBaaS or SaaS environments. This PDB can then be rapidly cloned to easily set up database environments for new customers.
 - Near Zero Downtime PDB Relocation – This feature significantly reduces the downtime of relocating a PDB from one CDB to another by using clone functionality. The source PDB remains open and functional while the relocation takes place. The application outage is reduced to a very short window while the source PDB is brought to a consistent state, and the destination PDB is synchronized and brought online. This functionality also takes advantage of another new feature, Listener Redirects, which allows you to keep the same connect descriptor for applications and connect to the destination PDB even after it has been relocated.
 - Online provisioning and cloning – Clones of PDBs can be created without requiring the source PDB to be placed in read only-mode. The source PDB can be left in read-write mode and accessible to applications for the duration of the clone operation.
 - Refreshable Clone PDB – Clones of PDBs can be created in such a way as to be refreshed with changes with changes made to the source PDB applied either automatically at set intervals or manually. For a clone to be refreshable it must remain in read-only mode. The clone can be converted into an ordinary PDB by opening it

read-write. Refreshable clones are well suited to be used as test masters for Exadata storage snapshots.

- New patching and upgrade options -When you upgrade or patch a CDB, all of the PDBs in that container are also upgraded or patched. If you need isolation, you can unplug a PDB and plug it into a CDB at a later version.
- Database backup and recovery - By consolidating multiple databases as PDBs, operations such as backup and disaster recovery are performed at the container level. Oracle Multitenant also provides the flexibility to backup and restore individual PDBs with no impact to other running PDBs in the same CDB.
- Operation with Oracle Data Guard - Data Guard configurations are maintained at the CDB level. When a Data Guard role transition (either failover or switchover) is performed, all PDBs are transitioned to the new primary database. There is no need to create or manage multiple Data Guard configurations for each PDB as would be required for single databases. Existing tools such as Data Guard Standby First Patching and Data Guard Transient Logical Rolling Upgrade can still be used to reduce downtime and are performed at the container level, so all PDBs will be maintained in a single operation.
 - PDB Migration with Data Guard Broker – The Data Guard broker has been enhanced to provide automation for migrating PDBs from one CDB, either the primary database or the standby database, to another CDB. This can be used for straight migration of a PDB from one CDB to another running at either at the same version or a CDB running at a higher version to start the upgrade process. This automation can also be used to affect a single PDB failover by using the PDBs files at a standby database to plug into a different CDB at the same version.
 - Subset Standby - A subset standby enables users of Oracle Multitenant to designate a subset of the PDBs in a CDB for replication to a standby database. This provides a finer granularity of designating which standby databases will contain which PDBs.
- Operation with Oracle GoldenGate - All of functionality provided by Oracle GoldenGate also exists for Oracle Multitenant. GoldenGate also provides the flexibility to operate at the PDB level, allowing replication to occur for a subset of the PDBs in a CDB. GoldenGate can be used for minimal to zero downtime upgrades either at the CDB level or at an individual PDB level.
- Resource management - Just as Oracle Resource Manager can control resource utilization between single databases, it can also control resource utilization of individual PDBs in a container. This can ensure that a single PDB does not access more than its assigned share of system resources. You can specify guaranteed minimums and maximums for SGA, buffer cache, shared pool, and PGA memory at the PDB limit.
- Operation with Oracle Flashback Database - If fast point-in-time recovery is required, the initial release of Oracle Multitenant enables using Flashback Database at the CDB level. Oracle Multitenant enables Flashback Database to be used on an individual PDB without impacting the availability of other PDBs. Flashback Database can be performed at the CDB level which will flashback all of the PDBs in the container. Individual PDBs can be flashed back using the Flashback Pluggable Database feature. When flashing back an individual PDB all other PDBs remain unaffected.
- Data Guard Broker PDB Migration or Failover - In multitenant broker configurations, you may need to move a Production PDB from one container database to another container database that resides on the same system. You may also need to failover a PDB from a Data Guard Standby database to a new production container database when the production PDB has failed but the container database and all other PDBs function normally. Using the new Data Guard Broker command, `MIGRATE PLUGGABLE DATABASE`, you can easily move a single PDB from one container database to another, or failover a single PDB from a Data Guard standby to a new production container database. (new in Oracle Database 12c Release 2)

① See Also

- *Oracle Multitenant Administrator's Guide*
- Oracle MAA technical brief "Best Practices for Database Consolidation" at <https://www.oracle.com/database/technologies/high-availability/oracle-database-maa-best-practices.html>

Oracle Sharding

Oracle Sharding is a feature of Oracle Database that lets you automatically distribute and replicate data across a pool of Oracle databases that share no hardware or software. Oracle Sharding provides the best features and capabilities of mature RDBMS and NoSQL databases, such as:

- SQL language used for object creation, strict data consistency, complex joins, ACID transaction properties, distributed transactions, relational data store, security, encryption, robust performance optimizer, backup and recovery, and patching with Oracle Database
- Oracle innovations and enterprise-level features, including Advanced Security, Automatic Storage Management (ASM), Advanced Compression, partitioning, high-performance storage engine, SMP scalability, Oracle RAC, Exadata, in-memory columnar, online redefinition, JSON document store, and so on
- Distributed database-aware Oracle Database tools, such as SQL Developer, Enterprise Manager Cloud Control, Recovery Manager (RMAN), and Data Pump, for distributed database application development and management
- Programmatic interfaces, such as Java Database Connectivity (JDBC), Oracle Call Interface (OCI), Universal Connection Pool (UCP), Oracle Data Provider for .NET (ODP.NET), and PL/SQL, including extensions for distributed database application development
- Extreme availability with Oracle Data Guard and Oracle Active Data Guard. Other replication options include Raft replication, which is built into Oracle Globally Distributed AI Database.
- Support for multi-model data like relational, text, and JSON
- Existing life-cycle management and operational processes can be kept, leveraging in-house and world-wide Oracle database administrator skill sets
- Enterprise-level support
- Extreme scalability and availability of NoSQL databases

Customers choose a distributed database architecture primarily for the following reasons:

- Scalability and agility (hyperscaling)
 - Ability to horizontally scale data, handling increasing data volumes and user loads by adding more shards (nodes) without any application changes
 - Ability to add or drop shards with application transparency
 - Each shard is open and active for the application.
- High availability with always-on continuous database availability
 - Automatic failover and data replication across shards to ensure the database remains operational, even if some shards or nodes go down

- Fault tolerance and isolation across shards, including protection from data center, availability domain (AD), or availability zone (AZ) failures
- Failover in low single-digit seconds or extremely low Recovery Time Objective for database failures
- Zero data loss
- Seamless distributed application integration
 - Distributed application is initially designed with a shard key for data operation, and queries may be adjusted to be shard scoped aware.
 - No application changes are required while scaling or adjusting the number of database shards.
 - From the perspective of an application, a distributed database looks like a single database; the number of shards and the distribution of data across those shards are completely transparent to the application.
- Consistency and transaction support
 - Support for consistent data reads/writes, with required levels of transactional guarantees (strong, eventual, and so on) across shards as needed by the application.
 - External consistency for sharded applications is a property that guarantees transactions appear to execute as if they occurred one at a time, in an order that respects the real-time, or "wall-clock," order of their commits. This is a critical and challenging requirement for sharded databases, ensuring that all observers see the same, globally-consistent transaction history, regardless of which shard they interact with.
- Ease of management and monitoring
 - Simple operations for shard orchestration, monitoring, backup/restore, high availability, software updates, and scaling operations.

For more information, see [Using Oracle Sharding](#).

Oracle Restart

Oracle Restart enhances the availability of a single-instance (nonclustered) Oracle database and its components.

Oracle Restart is used in single-instance environments only. For Oracle Real Application Clusters (Oracle RAC) environments, the functionality to automatically restart components is provided by Oracle Clusterware.

If you install Oracle Restart, it automatically restarts the database, the listener, and other Oracle components after a hardware or software failure or whenever the database's host computer restarts. It also ensures that the Oracle components are restarted in the proper order, in accordance with component dependencies.

Oracle Restart periodically monitors the health of components—such as SQL*Plus, the Listener Control utility (LSNRCTL), ASMCMD, and Oracle Data Guard—that are integrated with Oracle Restart. If the health check fails for a component, Oracle Restart shuts down and restarts the component.

Oracle Restart runs out of the Oracle Grid Infrastructure home, which you install separately from Oracle Database homes.

Integrated client failover applications depend on role based services and Fast Application Notification events, managed by Oracle clusterware, to alert the application to failures. Single instance databases must have Oracle Restart to achieve integrated client failover.

① See Also

Oracle Database Administrator's Guide for information about installing and configuring the Oracle Restart feature

Online Reorganization and Redefinition

One way to enhance availability and manageability is to allow user access to the database during a data reorganization operation.

The Online Reorganization and Redefinition feature in Oracle Database offers administrators significant flexibility to modify the physical attributes of a table and transform both data and table structure while allowing user access to the database. This capability improves data availability, query performance, response time, and disk space usage. All of these are important in a mission-critical environment and make the application upgrade process easier, safer, and faster.

Use Oracle Database online maintenance features to significantly reduce (or eliminate) the application downtime required to make changes to an application's database objects

① See Also

Redefining Tables Online in *Oracle Database Administrator's Guide*

Zero Data Loss Recovery Appliance

The cloud-scale Zero Data Loss Recovery Appliance, commonly known as Recovery Appliance, is an engineered system designed to dramatically reduce data loss and backup overhead for all Oracle databases in the enterprise.

Integrated with Recovery Manager (RMAN), the Recovery Appliance enables a centralized, incremental-forever backup strategy for large numbers of databases, using cloud-scale, fault-tolerant hardware and storage. The Recovery Appliance continuously validates backups for recoverability.

Recovery Appliance is the MAA-preferred backup and recovery appliance because:

- Elimination of data loss when restoring from Recovery Appliance
- Minimal backup overhead
- Improved end-to-end data protection visibility
- Cloud-scale protection
- Integrates very well with all MAA reference architectures including Oracle Sharding tier

① See Also

[Zero Data Loss Recovery Appliance Documentation](#)

Fleet Patching and Provisioning

Fleet Patching and Provisioning maintains a space-efficient repository of software, more precisely "gold images," which are standardized software homes that can be provisioned to any number of target machines.

Any number of homes can be provisioned from a given gold image, and Fleet Patching and Provisioning maintains lineage information so that the provenance of deployed software is always known. Gold images can be organized into series, allowing you to create groupings that track the evolution of a release, with different series for different tailored solutions such as Oracle Database patch bundles for specific applications. A notification system informs interested parties when a new image is available in a given series. Fleet Patching and Provisioning is a feature of Oracle Grid Infrastructure. The components that form the Fleet Patching and Provisioning Server are managed automatically by Oracle Grid Infrastructure.

Fleet Patching and Provisioning can provision databases, clusterware, middleware, and custom software. Fleet Patching and Provisioning offers additional features for creating, configuring, patching and upgrading Oracle Grid Infrastructure and Oracle Database deployments. These capabilities simplify maintenance, reducing its risk and impact, and provide a roll-back option if changes need to be backed out. Additional capabilities include provisioning clusters and databases onto base machines, and simple capacity on demand by growing and shrinking clusters and Oracle RAC databases. All of these operations are performed with single commands which replace the numerous manual steps otherwise required. All commands and their outcomes are recorded in an audit log. All workflows allow customization to support the unique requirements of any environment.

The key benefits of Fleet Patching and Provisioning are:

- Enables and enforces standardization
- Simplifies provisioning, patching and upgrading
- Minimizes the impact and risk of maintenance
- Increases automation and reduces touch points
- Supports large scale deployments

See Also:

Fleet Patching and Provisioning and Maintenance in *Oracle Clusterware Administration and Deployment Guide*

[Oracle Fleet Patching and Provisioning \(FPP\) Introduction and Technical Overview](#)

Edition-Based Redefinition

Planned application changes may include changes to data, schemas, and programs. The primary objective of these changes is to improve performance, manageability, and functionality. An example is an application upgrade.

Edition-based redefinition (EBR) lets you upgrade the database component of an application while it is in use, thereby minimizing or eliminating downtime. To upgrade an application while it is in use, you must copy the database objects that comprise the database component of the

application and redefine the copied objects in isolation. Your changes do not affect users of the application; they can continue to run the unchanged application. When you are sure that your changes are correct, you make the upgraded application available to all users.

 See Also

Using Edition-Based Redefinition in *Oracle Database Development Guide*

4

Oracle Database High Availability Solutions for Unplanned Downtime

Oracle Database offers an integrated suite of high availability solutions that increase availability.

These solutions also **eliminate or minimize** both planned and unplanned downtime, and help enterprises maintain business continuity 24 hours a day, 7 days a week. However, Oracle's high availability solutions not only go beyond reducing downtime, but also help to improve overall performance, scalability, and manageability.

Outage Types and Oracle High Availability Solutions for Unplanned Downtime

Various Oracle MAA high availability solutions for unplanned downtime are described here in an easy to navigate matrix.

The following table shows how the features discussed in the referenced (hyperlinked) sections can be used to address various causes of unplanned downtime. Where several Oracle solutions are listed, the MAA recommended solution is indicated in the Oracle MAA Solution column.

Table 4-1 Outage Types and Oracle High Availability Solutions for Unplanned Downtime

Outage Scope	Oracle MAA Solution	Benefits
Site failures	Oracle Data Guard and Enabling Continuous Service for Applications (MAA recommended)	• Integrated client and application failover • Fastest and simplest database replication • Supports all data types • Zero data loss by eliminating propagation delay • Oracle Active Data Guard – Supports read-only services and DML on global temporary tables and sequences to off-load more work from the primary – Allows small updates to be redirected to the primary enabling read-mostly reports to be offloaded to standby
	Oracle GoldenGate	• Database In-Memory support • Flexible logical replication solution (target is open read/write) • Active-active high availability (with conflict resolution) • Heterogeneous platform and heterogeneous database support • Potential zero downtime with custom application failover

Table 4-1 (Cont.) Outage Types and Oracle High Availability Solutions for Unplanned Downtime

Outage Scope	Oracle MAA Solution	Benefits
Instance or computer failures	Globally Distributed AI Database with Oracle AI Database 26ai using native Raft replication	• Active-active high availability with strong consistency • Single logical database across distributed deployments • Zero data loss through synchronous replication • Automatic failover with integrated application and database • Simple, high-performance logical replication with Raft
	Recovery Manager, Zero Data Loss Recovery Appliance and Oracle Secure Backup	• Fully managed database recovery and integration with Oracle Secure Backup • Recovery Appliance – provides end-to-end data protection for backups – reduces data loss for database restores – Non-real-time recovery
	Oracle Real Application Clusters and Oracle Clusterware and Enabling Continuous Service for Applications (MAA recommended)	• Integrated client and application failover • Automatic recovery of failed nodes and instances • Lowest application brownout with Oracle Real Application Clusters
	Oracle RAC One Node and Enabling Continuous Service for Applications	• Integrated client and application failover • Online database relocation migrates connections and instances to another node • Better database availability than traditional cold failover solutions
	Oracle Data Guard and Enabling Continuous Service for Applications	• Integrated client and application failover • Fastest and simplest database replication • Supports all data types • Zero data loss by eliminating propagation delay • Oracle Active Data Guard – Supports read-only services and DML on global temporary tables and sequences to off-load more work from the primary – Allows small updates to be redirected to the primary enabling read-mostly reports to be offloaded to standby
	Globally Distributed AI Database with Oracle AI Database 26ai using native Raft replication	• Database In-Memory support • Active-active high availability with strong consistency • Single logical database across distributed deployments • Zero data loss through synchronous replication • Automatic failover with integrated application and database • Simple, high-performance logical replication with Raft
	Oracle GoldenGate	• Flexible logical replication solution (target is open read/write) • Active-Active high availability (with conflict resolution) • Heterogeneous platform and heterogeneous database support • Potential zero downtime with custom application failover

Table 4-1 (Cont.) Outage Types and Oracle High Availability Solutions for Unplanned Downtime

Outage Scope	Oracle MAA Solution	Benefits
Storage failures	Oracle Automatic Storage Management (MAA recommended)	Mirroring and online automatic rebalancing places redundant copies of the data in separate failure groups.
	Oracle Data Guard (MAA recommended)	• Integrated client and application failover • Fastest and simplest database replication • Supports all data types • Zero data loss by eliminating propagation delay • Oracle Active Data Guard supports read-only services and DML on global temporary tables and sequences to off-load more work from the primary • Database In-Memory support
	Recovery Manager with Fast Recovery Area , and Zero Data Loss Recovery Appliance (MAA recommended)	Fully managed database recovery and managed disk and tape backups
	Globally Distributed AI Database with Oracle AI Database 26ai using native Raft replication	• Active-active high availability with strong consistency • Single logical database across distributed deployments • Zero data loss through synchronous replication • Automatic failover with integrated application and database • Simple, high-performance logical replication with Raft
	Oracle GoldenGate	• Flexible logical replication solution (target is open read/write) • Active-active high availability (with conflict resolution) • Heterogeneous platform and heterogeneous database support • Potential zero downtime with custom application failover
Data corruption	Corruption Prevention, Detection, and Repair (MAA recommended) Database initialization settings such as DB_BLOCK_CHECKING, DB_BLOCK_CHECKSUM, and DB_LOST_WRITE_PROTECT	Different levels of data and redo block corruption prevention and detection at the database level

Table 4-1 (Cont.) Outage Types and Oracle High Availability Solutions for Unplanned Downtime

Outage Scope	Oracle MAA Solution	Benefits
Data corruption	Oracle Data Guard (MAA recommended) Oracle Active Data Guard Automatic Block Repair DB_LOST_WRITE_PROTECT initialization parameter	• In a Data Guard configuration with an Oracle Active Data Guard standby – Physical block corruptions detected by Oracle at a primary database are automatically repaired using a good copy of the block retrieved from the standby, and vice versa – The repair is transparent to the user and application, and data corruptions can definitely be isolated • With MAA recommended initialization settings, Oracle Active Data Guard and Oracle Exadata Database Machine, achieve most comprehensive full stack corruption protection. • With DB_LOST_WRITE_PROTECT enabled – A lost write that occurred on the primary database is detected either by the physical standby database or during media recovery of the primary database, recovery is stopped to preserve the consistency of the database – Failing over to the standby database using Data Guard will result in some data loss – Data Guard Broker's PrimaryLostWrite property supports SHUTDOWN and CONTINUE, plus FAILOVER and FORCEFAILOVER options, when lost writes are detected on the primary database. See <i>Oracle Data Guard Broker</i> – DB_LOST_WRITE_PROTECT initialization parameter provides lost write detection • Shadow lost write protection detects a lost write before it can result in major data corruption. You can enable shadow lost write protection for a database, a tablespace, or a data file without requiring an Oracle Data Guard standby database. Note the impact on your workload may vary.

Table 4-1 (Cont.) Outage Types and Oracle High Availability Solutions for Unplanned Downtime

Outage Scope	Oracle MAA Solution	Benefits
Data corruption	Dbverify, Analyze, Data Recovery Advisor and Recovery Manager , Zero Data Loss Recovery Appliance , and ASM Scrub with Fast Recovery Area (MAA recommended)	These tools allow the administrator to run manual checks to help detect and potentially repair from various data corruptions. • Dbverify and Analyze conduct physical block and logical intra-block checks. Analyze can conduct inter-object consistency checks. • Data Recovery Advisor automatically detects data corruptions and recommends the best recovery plan. • RMAN operations can – Conduct both physical and inter-block logical checks – Run online block-media recovery using flashback logs, backups, or the standby database to help recover from physical block corruptions • Recovery Appliance – Does periodic backup validation that helps ensure that your backups are valid – Allows you to input your recovery window requirements, and alerts you when those SLAs cannot be met with your existing backups managed by Recovery Appliance • ASM Scrub detects and attempts to repair physical and logical data corruptions with the ASM pair in normal and high redundancy disks groups.
	Oracle Exadata Database Machine and Oracle Automatic Storage Management (MAA recommended)	• If Oracle ASM detects a corruption and has a good mirror, ASM returns the good block and repairs the corruption during a subsequent write I/O. • Exadata provides implicit HARD enabled checks to prevent data corruptions caused by bad or misdirected storage I/O. • Exadata provides automatic HARD disk scrub and repair. Detects and fixes bad sectors. • DIX +T10 DIF Extensions provides end to end data integrity for reads and writes through a checksum validation from a vendor's host adapter to the storage device
	DIX + T10 DIF Extensions (MAA recommended where applicable)	
Human errors	Globally Distributed AI Database with Oracle AI Database 26ai using native Raft replication	• Simple, high-performance logical replication with Raft. Logical replica can be used as a failover target if partner replica is corrupted. • Active-active high availability
	Oracle GoldenGate	• Flexible logical replication solution (target is open read/write). Logical replica can be used as a failover target if partner replica is corrupted. • Active-active high availability (with conflict resolution) • Heterogeneous platform and heterogeneous database support
	Oracle security features (MAA recommended)	Restrict access to prevent human errors

Table 4-1 (Cont.) Outage Types and Oracle High Availability Solutions for Unplanned Downtime

Outage Scope	Oracle MAA Solution	Benefits
Delays or slow downs	Oracle Flashback Technology (MAA recommended)	- Fine-grained error investigation of incorrect results - Fine-grained and database-wide or pluggable database rewind and recovery capabilities
	Oracle Database and Oracle Enterprise Manager Oracle Data Guard (MAA recommended) and Enabling Continuous Service for Applications	Oracle Database automatically monitors for instance and database delays or cluster slow downs and attempts to remove blocking processes or instances to prevent prolonged delays or unnecessary node evictions. Oracle Enterprise Manager or a customized application heartbeat can be configured to detect application or response time slowdown and react to these SLA breaches. For example, you can configure the Enterprise Manager Beacon to monitor and detect application response times. Then, after a certain threshold expires, Enterprise Manager can call the Data Guard
DBMS_DG.INITIALIZE_FS_FAILOVER		
PL/SQL procedure to initiate a failover. See the section about "Managing Fast-Start Failover" in <i>Oracle Data Guard Broker</i> .		
File system data	Oracle Replication Technologies for Non-Database Files	Database In-Memory support
		Enables full stack failover that includes non-database files

Managing Unplanned Outages for MAA Reference Architectures and Multitenant Architectures

High availability solutions in each of the MAA service-level tiers for the MAA reference architectures and multitenant architectures are described in an easy to navigate matrix.

If you are managing many databases in DBaaS, we recommend using the MAA tiers and Oracle Multitenant as described in [Oracle MAA Reference Architectures](#).

The following table identifies various unplanned outages that can impact a database in a multitenant architecture. It also identifies the Oracle high availability solution to address that outage that is available in each of the MAA reference architectures.

Table 4-2 Unplanned Outage Matrix for MAA Reference Architectures and Multitenant Architectures

Event	Solutions by MAA Architecture	Recovery Window (RTO)	Data Loss (RPO)
Instance Failure	BRONZE: Oracle Restart	Minutes if instance can restart	Zero

Table 4-2 (Cont.) Unplanned Outage Matrix for MAA Reference Architectures and Multitenant Architectures

Event	Solutions by MAA Architecture	Recovery Window (RTO)	Data Loss (RPO)
Permanent Node Failure (but storage available)	SILVER: Oracle RAC (see Oracle Real Application Clusters and Oracle Clusterware) or Oracle RAC One Node , and Enabling Continuous Service for Applications	Seconds with Oracle RAC, minutes with Oracle RAC One Node	Zero
	GOLD: Oracle RAC (see Oracle Real Application Clusters and Oracle Clusterware and Enabling Continuous Service for Applications)	Seconds	Zero
	PLATINUM: Oracle RAC (see Oracle Real Application Clusters and Oracle Clusterware) and Enabling Continuous Service for Applications	Zero Application Outage	Zero
	BRONZE: Restore and recover	Hours to Day	Zero
	SILVER: Oracle RAC (see Oracle Real Application Clusters and Oracle Clusterware) and Enabling Continuous Service for Applications	Seconds	Zero
	SILVER: Oracle RAC One Node and Enabling Continuous Service for Applications	Minutes	Zero
	GOLD: Oracle RAC (see Oracle Real Application Clusters and Oracle Clusterware) and Enabling Continuous Service for Applications	Seconds	Zero

Table 4-2 (Cont.) Unplanned Outage Matrix for MAA Reference Architectures and Multitenant Architectures

Event	Solutions by MAA Architecture	Recovery Window (RTO)	Data Loss (RPO)
	PLATINUM: Oracle RAC (see Oracle Real Application Clusters and Oracle Clusterware) and Enabling Continuous Service for Applications	Seconds	Zero
Storage Failure	ALL: Oracle Automatic Storage Management	Zero downtime	Zero
Data corruptions	BRONZE/SILVER: Basic protection Some corruptions require recover restore and recovery of pluggable database (PDB), entire multitenant container database (CDB) or non-container database (non-CDB)	Hour to Days	• Since last backup if unrecoverable • Zero or Near Zero with Recovery Appliance
	GOLD: Comprehensive corruption protection and Auto Block Repair with Oracle Active Data Guard	• Zero with auto block repair • Seconds to minutes if corruption due to lost writes and using Data Guard Fast Start failover.	Zero unless corruption due to lost writes
	PLATINUM: Comprehensive corruption protection and Auto Block Repair with Oracle Active Data Guard Oracle GoldenGate replica with custom application failover	• Zero with auto block repair • Zero with Oracle GoldenGate replica	Zero when using Active Data Guard Fast-Start Failover and Oracle GoldenGate
Human error	ALL: Logical failures resolved by flashback drop, flashback table, flashback transaction, flashback query flashback pluggable database, and undo.	Dependent on detection time but isolated to PDB and applications using those objects.	Dependent on logical failure

Table 4-2 (Cont.) Unplanned Outage Matrix for MAA Reference Architectures and Multitenant Architectures

Event	Solutions by MAA Architecture	Recovery Window (RTO)	Data Loss (RPO)
Database unusable, system, site or storage failures, wide spread corruptions or disasters	All: Comprehensive logical failures impacting an entire database and PDB that requires RMAN point in time recovery (PDB) or flashback pluggable database	Dependent on detection time	Dependent on logical failure time
	BRONZE/SILVER: Restore and recover	Hours to Days	• Since last database and archive backup • Zero or near zero with Recovery Appliance
	GOLD: Active Data Guard Fast-Start Failover and Enabling Continuous Service for Applications	Seconds	Zero to Near Zero
Performance Degradation	PLATINUM: Oracle GoldenGate replica with custom application failover	Zero	Zero when using Active Data Guard Fast-Start Failover and Oracle GoldenGate
	ALL: Oracle Enterprise Manager for monitoring and detection, Database Resource Management for Resource Limits and ongoing Performance Tuning	No downtime but degraded service	Zero

5

Oracle Database High Availability Solutions for Planned Downtime

Planned downtime can be just as disruptive to operations as unplanned downtime. This is especially true for global enterprises that must support users in multiple time zones, or for those that must provide Internet access to customers 24 hours a day, 7 days a week.

See the following topics to learn about keeping your database highly available during planned downtime.

Oracle High Availability Solutions for Planned Maintenance

Oracle provides high availability solutions for all planned maintenance.

The following table describes the various Oracle high availability solutions and their projected downtime for various maintenance activities.

Table 5-1 Oracle High Availability Solutions for Planned Maintenance

Maintenance Event	High Availability Solutions with Target Outage Time
Dynamic and Online Resource Provisioning, or Online reorganization and redefinition	Zero application and database downtime for - Changing initialization parameters dynamically - Renaming and relocating datafiles online - Automatic memory management tuning - Online reorganization and redefinition (managing tables and managing indexes) See the <i>Oracle Database Administrator Guide</i>, <i>Oracle Database Reference</i> (to evaluate which parameters to use on dynamic), and Online Data Reorganization and Redefinition
Operating system software or hardware updates and patches	Zero database downtime with Oracle RAC and Oracle RAC One Node Rolling Seconds to minutes database downtime with Standby-First Patch Apply and subsequent Data Guard Switchover
Oracle Database or Grid Infrastructure interim or diagnostic patches	Zero downtime with Database Online Patching or Zero-Downtime Oracle Grid Infrastructure Patching Zero database downtime with Oracle RAC and Oracle RAC One Node Rolling Zero application downtime with Application Checklist for Continuous Service for MAA Solutions Seconds to minutes database downtime with Standby-First Patch Apply and subsequent Data Guard Switchover

Table 5-1 (Cont.) Oracle High Availability Solutions for Planned Maintenance

Maintenance Event	High Availability Solutions with Target Outage Time
Oracle Database or Grid Infrastructure quarterly updates under the Critical Patch Update (CPU) program, or Oracle Grid Infrastructure release upgrades	Zero database downtime with Oracle RAC and Oracle RAC One Node Rolling. Zero application downtime with Application Checklist for Continuous Service for MAA Solutions Seconds to minutes downtime with Standby-First Patch Apply and subsequent Data Guard Switchover Special consideration is required during rolling database quarterly updates for applications that use database OJVM. See My Oracle Support RAC Rolling Install Process for the "Oracle JavaVM Component Database PSU/RU" (OJVM PSU/RU) Patches (Doc ID 2217053.1) for details.
Oracle Database Release Upgrade (for example, Oracle Database 11g to 12.2 or 12.2 to 19c)	Seconds to minutes downtime with Data Guard transient logical or DBMS_ROLLING solution Zero downtime with Oracle GoldenGate See Automated Database Upgrades using Oracle Active Data Guard and DBMS_ROLLING for 12.2 and higher database releases or Database Rolling Upgrade using Data Guard for older releases.
Exadata database server software updates	Zero database downtime with Oracle RAC Rolling Zero application downtime with Application Checklist for Continuous Service for MAA Solutions Seconds to minutes downtime with Standby-First Patch Apply and subsequent Data Guard Switchover See Updating Exadata Software
Exadata storage server or Exadata switch software updates	Zero downtime using Exadata patchmgr See Updating Exadata Software
Database Server or Oracle RAC cluster changes (add node, drop node, adjust CPU or memory size of the database server)	Some hardware changes like adjusting CPU can be done online without restarting the database server. Refer to the hardware specific documentation. If the change is not online, then Zero database downtime with Oracle RAC and Oracle RAC One Node Rolling. Zero application downtime with Application Checklist for Continuous Service for MAA Solutions Seconds to minutes downtime with Standby-First Patch Apply and subsequent Data Guard Switchover
Application upgrades	Zero downtime with Edition Based Redefinition Zero downtime with Oracle GoldenGate See Edition Based Redefinition and Oracle GoldenGate documentation

Table 5-1 (Cont.) Oracle High Availability Solutions for Planned Maintenance

Maintenance Event	High Availability Solutions with Target Outage Time
Fleet-wide software maintenance events - Oracle Database or Grid Infrastructure interim or diagnostic patches - Oracle Database or Grid Infrastructure quarterly updates under the Critical Patch Update (CPU) program, or Oracle Grid Infrastructure release upgrades - Exadata database server software updates - Exadata storage server or Exadata switch software updates	Use Fleet Patching and Provisioning, which leverages the following high availability solutions to achieve the target outage times for fleet-wide software maintenance events: Zero database downtime with Oracle RAC and Oracle RAC One Node Rolling Zero application downtime with Application Checklist for Continuous Service for MAA Solutions for - Oracle Database or Grid Infrastructure interim or diagnostic patches - Oracle Database or Grid Infrastructure quarterly updates under the Critical Patch Update (CPU) program, or Oracle Grid Infrastructure release upgrades - Exadata database server software updates Zero downtime using Exadata patchmgr for Exadata storage server or Exadata switch software updates

High Availability Solutions for Migration

Oracle MAA recommends several solutions for reducing downtime due to database migration.

The following table describes the high availability solutions for migration at a high level.

Table 5-2 High Availability Solutions for Migration

Maintenance Event	High Availability Solutions with Target Outage Time
Migrate to an on-premises Oracle Exadata Database Machine or any Oracle Database cloud service, including Oracle Exadata Database Service on Cloud@Customer See Zero Downtime Migration for a complete list of supported services and platforms	Use the Zero Downtime Migration tool, which provides - Physical migration with RMAN backup and restore, with an optional low downtime option using Oracle Data Guard. This is the simplest turnkey migration solution, which is ideal when the source and target system platform (for example, Linux to Linux) and database versions (Oracle Database 19c to 19c) are the same. - Logical migration with Oracle Data Pump, with an optional low downtime option using Oracle GoldenGate. This is the only option for migrating a database when the source and target system platform (For example, AIX to Linux) or major database versions (Oracle Database 12c to 19c) are different. To migrate to Oracle Autonomous Database, use the Oracle Cloud Infrastructure Database Migration service (or the Zero Downtime Migration tool), which provide - Offline migration with Data Pump - Online migration with Data Pump and Oracle GoldenGate

Table 5-2 (Cont.) High Availability Solutions for Migration

Maintenance Event	High Availability Solutions with Target Outage Time
Migrate the database to a different server or platform	Seconds to minutes downtime with Oracle Data Guard for certain platform combinations Zero downtime with Oracle GoldenGate Data Guard always supports primary and standby combinations on the same platform. For heterogeneous platforms, Refer to Data Guard Support for Heterogeneous Primary and Physical Standbys in Same Data Guard Configuration (Doc ID 413484.1)
Migrate database to an incompatible character set	Zero downtime with Oracle GoldenGate See Character Set Migration
Migrate to pluggable databases to another container database	Seconds to minutes downtime with Pluggable Database Relocate (PDB Relocate) See Relocating a PDB
Migrate to new storage	Zero Downtime with Oracle Automatic Storage Management if storage is compatible with Oracle Data Guard for certain platform combinations Zero Downtime with Oracle GoldenGate
Migrate database from a single-instance system to an Oracle RAC cluster	Zero Downtime with Oracle RAC when applicable. See Adding Oracle RAC to Nodes with Oracle Clusterware Installed Seconds to minutes downtime with Oracle Data Guard for certain platform combinations Zero Downtime with Oracle GoldenGate

6

Enabling Continuous Service for Applications

Applications achieve continuous service easily when the underlying network, systems, and databases are always available.

To achieve continuous service in the face of unplanned outages and planned maintenance activities can be challenging. An MAA database architecture and its configuration and operational best practices is built upon redundancy and its ability to tolerate, prevent, and at times auto-repair failures.

However, applications can incur downtime whenever a failure hits a database instance, a database node, or the entire cluster or data center. Similarly, some planned maintenance activities may require restarting a database instance, a database node, or an entire database server to be restarted.

In all cases, following a simple checklist, your applications can incur zero or very little downtime whenever the database service that the application is connected to can be moved to another Oracle RAC instance or to another database.

See [Configuring Continuous Availability for Applications](#) for various levels and options to achieve continuous service for your application.

Drain Timeouts for Planned Maintenance Events

For planned maintenance events, some applications require time to complete their in-flight transactions.

The amount of time (DRAIN_TIMEOUT) for any workload to gracefully complete its in-flight transactions and move its sessions vary based on the workload characteristics. For short OLTP transactions, a DRAIN_TIMEOUT of 1 minute may be sufficient, while batch jobs might require 30 minutes. In some cases it might be best to suspend these long transactions to times outside the planned maintenance window.

The trade-off for configuring a longer DRAIN_TIMEOUT is that the planned maintenance window would be extended.

The following table outlines planned maintenance events that will incur Oracle RAC instance rolling restart and the relevant service drain timeout variables that may impact your application.

Table 6-1 Drain Timeout Variables for Planned Maintenance Events

Planned Maintenance Event	Application Drain Timeout Variables
Exadata Database Host (Dom0) software changes	Exadata Host handles operating system (OS) shutdown with maximum timeout of 10 minutes. OS shutdown calls an <code>rhphelper</code>, which has the following drain timeout settings: • <code>DRAIN_TIMEOUT</code>: value used for services that do not have a <code>drain_timeout</code> defined. Default 180 • <code>MAX_DRAIN_TIMEOUT</code>: overrides any higher <code>drain_timeout</code> value defined for a given service. Default 300 Each Clusterware-managed service is also controlled by a <code>drain_timeout</code> attribute that can be lower than the above values. See also: Using RHPHelper to Minimize Downtime During Planned Maintenance on Exadata (Doc ID 2385790.1)
Exadata Database Guest (DomU) software changes	Exadata <code>patchmgr</code> and <code>dbnodeupdate</code> software programs call <code>rhphelper</code>, which has the following drain timeout settings: <code>DRAIN_TIMEOUT</code>: value used for services that do not have a <code>drain_timeout</code> defined. Default 180 <code>MAX_DRAIN_TIMEOUT</code>: overrides any higher <code>drain_timeout</code> value defined for a given service. Default 300 Each Clusterware-managed service is also controlled by a <code>drain_timeout</code> attribute that can be lower than the above values. See also: Using RHPHelper to Minimize Downtime During Planned Maintenance on Exadata (Doc ID 2385790.1)
Oracle Grid Infrastructure (GI) software changes or upgrade	The recommend steps are described in Graceful Application Switchover in RAC with No Application Interruption (Doc ID 1593712.1). Example: <pre>srvctl stop instance -o immediate - drain_timeout 600 -failover -force</pre> Each Clusterware-managed service is also controlled by a <code>drain_timeout</code> attribute that can be lower than the above values.
Oracle Database Software changes	The recommend steps are described in Graceful Application Switchover in RAC with No Application Interruption (Doc ID 1593712.1). Example: <pre>srvctl stop instance -o immediate - drain_timeout 600 -failover -force</pre> Each Clusterware-managed service is also controlled by a <code>drain_timeout</code> attribute that can be lower than the above values.

See also:

[Configuring Continuous Availability for Applications](#)

7

Operational Prerequisites to Maximizing Availability

Use the following operational best practices to provide a successful MAA implementation.

Understand High Availability and Performance Service-Level Agreements

Understand and document your high availability (HA) and performance service-level agreements (SLAs):

- Understand the attributes of High Availability and various causes of downtime as described in [Overview of High Availability](#).
- Establish high availability and performance SLAs from lines of business, upper management, and technical teams as described in [High Availability Requirements](#), and [A Methodology for Documenting High Availability Requirements](#).

Implement and Validate a High Availability Architecture That Meets Your SLAs

When you have agreement on your high availability and performance service level requirements:

- **Map** the requirements to one of the Oracle MAA standard and validated MAA reference architectures, as described in [High Availability and Data Protection – Getting From Requirements to Architecture](#)
- **Evaluate** the outage and planned maintenance matrices relevant to your reference architecture in [Oracle Database High Availability Solutions for Unplanned Downtime](#) and [Oracle Database High Availability Solutions for Planned Downtime](#)
- **Learn** about the database features required to implement your MAA architecture in [High Availability Architectures](#)

Establish Test Practices and Environment

You must **validate** or **automate** the following to ensure that your high availability SLAs are met:

- All software update and upgrade maintenance events
- All repair operations, including those for various types of unplanned outages
- Backup, restore, and recovery operations

If you use Oracle Data Guard for disaster recovery and data protection, Oracle recommends that you:

- Perform periodic switchover operations, or conduct full application and database failover tests
- Validate end-to-end role transition procedures by performing application and Data Guard switchovers periodically

A good test environment and proper test practices are essential prerequisites to achieving the highest stability and availability in your production environment. By validating every change in your test environment thoroughly, you can proactively detect, prevent, and avoid problems before applying the same change on your production systems.

These practices involve the following:

Configuring Test and QA Environments

The test environment should be a replica of the production MAA environment (for example, using the MAA Gold reference architecture.) There will be trade offs if the test system is not identical to the MAA service-level driven standard reference architecture that you plan to implement. It is recommended that you perform functional, performance, and availability tests with a workload that mimics production. Evaluate if availability and performance SLAs are maintained after each change, and ensure that clear fallback or repair procedures are in place if things go awry, while applying the change on the production environment.

With a properly configured test system, many problems can be avoided, because changes are validated with an equivalent production and standby database configuration containing a full data set and using a workload framework to mimic production (for example, using Oracle Real Application Testing.)

Do not try to reduce costs by eliminating the test system, because that decision ultimately affects the stability and the availability of your production applications. Using only a subset of system resources for testing and QA has the tradeoffs shown in the following table, which is an example of the MAA Gold reference architecture.

Table 7-1 Tradeoffs for Different Test and QA Environments

Test Environment	Benefits and Tradeoffs
Full Replica of Production and Standby Systems	Validate: • All software updates and upgrades • All functional tests • Full performance at production scale • Full high availability and disaster recovery testing
Full Replica of Production Systems	Validate: • All software updates and upgrades • All functional tests • Full performance at production scale • Full high availability minus the standby system Cannot Validate: • Disaster recovery testing • Any standby redo apply or read only workload performance testing • Redo transport performance and impact on production system resources due to redo transport • Any use case using the standby database such as Database Rolling Upgrade, Snapshot Standby, and so on.

Table 7-1 (Cont.) Tradeoffs for Different Test and QA Environments

Test Environment	Benefits and Tradeoffs
Standby System	Validate: • Most software update changes • All read-only functional tests • Full performance--if using Data Guard Snapshot Standby, but this can extend recovery time if a failover is required • Resource management and scheduling--required if standby and test databases exist on the same system Cannot Validate: • Role transition and disaster recovery testing • Any use case using the standby database such as Database Rolling Upgrade, Snapshot Standby, and so on.
Shared System Resource	Validate: • Most software update changes • All functional tests Cannot Validate: This environment may be suitable for performance testing if enough system resources can be allocated to mimic production. Typically, however, the environment includes a subset of production system resources, compromising performance validation. Resource management and scheduling is required. Standby or disaster recovery testing may not be possible or limited.
Smaller or Subset of the system resources	Validate: • All software update changes • All functional tests • Limited full-scale high availability evaluations Cannot Validate: • Performance testing at production scale • Standby or disaster recovery testing may not be possible or limited.
Different hardware or platform system resources but same operating system	Validate: • Some software update changes • Limited firmware patching test • All functional tests unless limited by new hardware features • Limited production scale performance tests • Limited full-scale high availability evaluations • Standby or disaster recovery testing may not be possible or limited.

See Also

Oracle Database Testing Guide

Performing Preproduction Validation Steps

Pre-production validation and testing of hardware, software, database, application or any changes is an important way to maintain stability. The high-level pre-production validation steps are:

1. Review the patch or upgrade documentation or any document relevant to that change. Evaluate the possibility of performing a rolling upgrade if your SLAs require zero or minimal downtime. Evaluate any rolling upgrade opportunities to minimize or eliminate planned downtime. Evaluate whether the patch or the change qualifies for Standby-First Patching.

Note:

Standby-First Patch enables you to apply a patch initially to a physical standby database while the primary database remains at the previous software release (this applies to certain types of software updates and does not apply to major release upgrades; use the Data Guard transient logical standby and DBMS_ROLLING method for patch sets and major releases). Once you are satisfied with the change, then perform a switchover to the standby database. The fallback is to switchback if required. Alternatively, you can proceed to the following step and apply the change to your production environment. For more information, see "Oracle Patch Assurance - Data Guard Standby-First Patch Apply" in My Oracle Support Note 1265700.1 at <https://support.oracle.com/CSP/main/article?cmd=show&type=NOT&id=1265700.1>

2. Validate the application in a test environment and ensure the change meets or exceeds your functionality, performance, and availability requirements. Automate the procedure and be sure to also document and test a fallback procedure. This requires comparing metrics captured before and after patch application on the test and against metrics captured on the production system. Real Application Testing may be used to capture the workload on the production system and replay it on the test system. AWR and SQL Performance Analyzer may be used to assess performance improvement or regression resulting from the patch.

Validate the new software on a test system that mimics your production environment, and ensure the change meets or exceeds your functionality, performance, and availability requirements. Automate the patch or upgrade procedure and ensure fallback. Being thorough during this step eliminates most critical issues during and after the patch or upgrade.

3. Use Oracle Real Application Testing and test data management features to comprehensively validate your application while also complying with any security restrictions your line of business may have. Oracle Real Application Testing (a separate database option) enables you to perform real-world testing of Oracle Database. By capturing production workloads and assessing the impact of system changes on these workloads before production deployment, Oracle Real Application Testing minimizes the risk of instabilities associated with system changes. SQL Performance Analyzer and Database Replay are key components of Oracle Real Application Testing. Depending on the nature and impact of the system change being tested, and on the type of system on which the test will be performed, you can use either or both components to perform your testing.

When performing real-world testing there is a risk of exposing sensitive data to non-production users in a test environment. The test data management features of Oracle Database help to minimize this risk by enabling you to perform data masking and data subsetting on the test data.

4. If applicable, perform final pre-production validation of all changes on a Data Guard standby database before applying them to production. Apply the change in a Data Guard environment, if applicable.
5. Apply the change in your production environment.

① See Also

[Data Guard Redo Apply and Standby-First Patching](#) and [Data Guard Transient Logical Rolling Upgrades](#)

[Converting a Physical Standby Database into a Snapshot Standby Database](#) and [Performing a Rolling Upgrade With an Existing Physical Standby Database](#) in Oracle Data Guard Concepts and Administration

[Oracle Database Rolling Upgrades: Using a Data Guard Physical Standby Database](#) on <http://www.oracle.com/goto/maa>

[Oracle Patch Assurance - Data Guard Standby-First Patch Apply \(Doc ID 1265700.1\)](#)

Set Up and Use Security Best Practices

Corporate data can be at grave risk if placed on a system or database that does not have proper security measures in place. A well-defined security policy can help protect your systems from unwanted access and protect sensitive corporate information from sabotage. Proper data protection reduces the chance of outages due to security breaches.

① See Also

Oracle Database Security Guide.

Establish Change Control Procedures

Institute procedures that manage and control changes as a way to maintain the stability of the system and to ensure that no changes are incorporated in the primary database unless they have been rigorously evaluated on your test systems, or any one of the base architectures in the MAA service-level tiers.

Review the changes and get feedback and approval from your change management team.

Apply Recommended Software Updates and Security Updates Periodically

Maintaining software at current or recent versions provides many benefits, such as better software security, improved resource utilization and stability, continued compatibility with newer related software, better support and faster resolution of issues, and the ability to receive fixes for newly discovered issues.

Update all software on a regular basis. Oracle recommends following these practices:

- Learn the release and support timelines for all software that your MAA environment depends upon in order to develop a plan for upgrade to a new major software release and a plan for installing proactive updates for current releases.

For example, Oracle Database release and support timelines is available in My Oracle Support Note 742060.1 "Release Schedule of Current Database Releases".

- Upgrade to a later major software release before proactive software updates for your current release cease.
- Install proactive software updates for your current release as they become available, typically on a monthly or quarterly basis.

However, business requirements may dictate that the adoption of certain proactive updates is delayed or skipped. In such cases Oracle recommends that the currently running software never lags the most recently released proactive update by more than 12 months.
- Install reactive software patches (also known as interim or one-off patches) for critical issues published in My Oracle Support Alerts as soon as feasible.
- Validate the software update process and perform soak testing on a test system before updating software on production systems.
- Use Oracle health check tools, Orachk and Exachk, to provide Oracle software upgrade and proactive update advice, critical issue software update recommendations, patching and upgrading pre-checks, database and system health checks, and MAA recommendations.

Orachk supports non-engineered systems and Oracle Database Appliance. Exachk supports engineered systems Oracle Exadata Database Machine and Oracle Zero Data Loss Recovery Appliance.

See also:

For Oracle Database and Grid Infrastructure:

- "Release Schedule of Current Database Releases" in My Oracle Support Note [742060.1](#)
- "Primary Note for Database Proactive Patch Program" in My Oracle Support Note [888.1](#)
- "Oracle Database 19c Important Recommended One-off Patches" in My Oracle Support Note [555.1](#)

For engineered systems (Exadata Database Machine and Zero Data Loss Recovery Appliance):

- "Exadata Database Machine and Exadata Storage Server Supported Versions" in My Oracle Support Note [888828.1](#)
- "Exadata Critical Issues" in My Oracle Support Note [1270094.1](#)
- "Oracle Exadata: Exadata and Linux Important Recommended Fixes" in My Oracle Support Note [556.1](#)
- "Oracle Exadata Database Machine Exachk" in My Oracle Support Note [1070954.1](#)

For non-engineered systems:

- "Autonomous Health Framework (AHF) - Including TFA and Orachk/Exachk" in My Oracle Support Note [2550798.1](#)

Establish Disaster Recovery Environment

To achieve the same performance and HA characteristics as the source or primary database, the disaster recovery environment or target should be symmetric or similarly configured to the production system.

If the disaster recovery target is a standby database or Oracle GoldenGate replica, symmetric or similar database compute CPU, memory, and throughput is required to match the same performance. Similarly, the storage should be able to handle the same IOPS, throughput, and response time.

When the disaster recovery target is used by other applications or databases for database consolidation and cost efficiency, additional resources will be required to ensure acceptable performance with other concurrent workloads.

Establish and Validate Disaster Recovery Practices

Disaster recovery validation is required to ensure that you meet your disaster recovery service level requirements such as RTO and RPO.

Whether you have a standby database, Oracle GoldenGate replica, or leverage database backups from Zero Data Loss Recovery Appliance (Recovery Appliance), ZFS Storage, or another third party, it is important to ensure that the operations and database administration teams are well prepared to failover or restore the database and application any time the primary database is down or underperforming. The concerned teams should be able to detect and decide to failover or restore as required. Such efficient preparation before disasters will significantly reduce overall downtime.

If you use Data Guard or Oracle GoldenGate for high availability, disaster recovery, and data protection, Oracle recommends that you perform regular application and database switchover operations **every three to six months**, or conduct full application and database failover tests.

Periodic RMAN cross checks, RMAN backup validations, and complete database restore and recovery are required to validate your disaster recovery solution through backups. Inherent backup checks and validations are done automatically with the Recovery Appliance, but periodic restore and recovery tests are still recommended.

See also: [Role Transition, Assessment, and Tuning](#)

Establish Escalation Management Procedures

Establish escalation management procedures so repair is not hindered. Most repair solutions, when conducted properly are automatic and transparent with the MAA solution. The challenges occur when the primary database or system is not meeting availability or performance SLAs and failover procedures are not automatic as in the case with some Data Guard failover scenarios. Downtime can be prolonged if proper escalation policies are not followed and decisions are not made quickly.

If availability is the top priority, perform failover and repair operations first and then proceed with gathering logs and information for Root Cause Analysis (RCA) after the application service has been reestablished. For simple data gathering, use the Trace File Analyzer Collector (TFA).

① See Also

MAA web page at <http://www.oracle.com/goto/maa>

My Oracle Support note 1513912.2 “TFA Collector - Tool for Enhanced Diagnostic Gathering” at [1513912.2](#)

Configure Monitoring and Service Request Infrastructure for High Availability

To maintain your High Availability environment, you should configure the monitoring infrastructure that can detect and react to performance and high availability related thresholds before any downtime has occurred.

Also, where available, Oracle can detect failures, dispatch field engineers, and replace failed hardware components such as disks, flash cards, fans, or power supplies without customer involvement.

Run Database Health Checks Periodically

Oracle Database health checks are designed to evaluate your hardware and software configuration and MAA compliance to best practices.

All of the Oracle health check tools will evaluate Oracle Grid Infrastructure, Oracle Database, and provide an automated MAA scorecard or review that highlights when key architectural and configuration settings are not enabled for tolerance of failures or fast recovery. For Oracle's engineered systems such as Exadata Database Machine, there may be hundreds of additional software, fault and configuration checks.

Oracle recommends periodically (for example, monthly for Exadata Database Machine) downloading the latest database health check, running the health check, and addressing the key FAILURES, WARNINGS, and INFO messages. Use Exachk for Engineered Systems such as Oracle Exadata Database Machine and Oracle Zero Data Loss Recovery Appliance, and use Orachk for non-engineered systems and Oracle Database Appliance.

Furthermore, it is recommended that you run the health check prior to and after any planned maintenance activity.

You must evaluate:

- Existing or new critical health check alerts prior to planned maintenance window
- Existing software or critical software recommendations
- Adding any new recommendations to the planned maintenance window after testing

See Also

My Oracle Support Note 1268927.2 "ORAchK - Health Checks for the Oracle Stack" at <https://support.oracle.com/CSP/main/article?cmd=show&type=NOT&id=1268927.2>

My Oracle Support Note 1070954.1 "Oracle Exadata Database Machine exachk or HealthCheck" at <https://support.oracle.com/CSP/main/article?cmd=show&type=NOT&id=1070954.1>

Configure Monitoring

When deciding on the best route for monitoring your Exadata fleet, you need to consider how the fleet you are monitoring is deployed (On-Premises, Cloud@Customer, Oracle Cloud Infrastructure) and where your monitoring is or can be deployed.

- **On-Premises**

For fleets including on-premises Exadata, Enterprise Manager includes necessary monitoring for responsibilities spanning all three deployment types and is the MAA Best Practice.

- **Cloud**

For fleets only in Cloud@Customer and/or OCI, who do not currently have Enterprise Manager or On-Premises monitoring deployment options, the OCI Observability & Management services provide various options for basic and advanced monitoring and manageability.

Configure Oracle Enterprise Manager Monitoring

If your Exadata fleet includes On-Premises deployment, you should configure and use Enterprise Manager and the monitoring infrastructure that detects and reacts to performance and high availability related thresholds to avoid potential downtime.

The monitoring infrastructure assists you with monitoring for High Availability and enables you to do the following:

- Monitor system, network, application, database and storage statistics
- Monitor performance and service statistics
- Create performance and high availability thresholds as early warning indicators of system or application problems
- Provide performance and availability advice
- Established alerts and tools and database performance
- Receive alerts for engineered systems hardware faults

Enterprise Manager provides monitoring and management for Exadata and Databases deployed on-premises, on Cloud@Customer, and OCI.

- For on-premises Exadata deployments, see [Oracle Exadata Database Machine Getting Started Guide](#).
- For Cloud@Customer and OCI Exadata deployments, see [Oracle Enterprise Manager Cloud Control for Oracle Exadata Cloud](#)

Configure Enterprise Manager for high availability to ensure that the manageability solution is as highly available as the systems that you're monitoring.

For configuration details for HA see [Oracle Enterprise Manager Cloud Control Advanced Installation and Configuration Guide](#). For additional MAA Best Practices for Enterprise Manager see <http://www.oracle.com/goto/maa>.

Oracle Observability and Management Services can be used in conjunction with Enterprise Manager to provide additional Exadata manageability features. For details, see the following:

- Operations Insights [Exadata Insights](#)
- [Harvest Entity Model Data from Enterprise Manager Cloud Control Collect Logs](#)

Configure OCI Observability and Management Services Monitoring

If your Exadata fleet includes only Cloud@Customer and/or OCI deployment, and you do not currently have Enterprise Manager or on-premises monitoring deployment options, you should configure and use the OCI Observability and Management platform of services that work together to provide monitoring and management of Oracle Cloud targets.

Basic default metrics and events for performance, high availability, and health are available in the OCI console. For details see the following documentation:

- Exadata Database Service on Dedicated Infrastructure
 - Metrics for [VM Clusters](#) and [Exadata Database Service on Dedicated Infrastructure](#) available in the Monitoring Service
 - [Events](#)
- Exadata Database Service on Cloud@Customer
 - Metrics for [VM Clusters](#) and [Exadata Database Service on Cloud@Customer](#) available in the Monitoring Service
 - [Events](#)

Advanced metrics and management features are available in the Database Management service:

- Exadata Database Service on Dedicated Infrastructure
 - Metrics available in the [Database Management Service](#)
 - [Diagnose and Troubleshoot Problems with Pluggable Databases](#)

Advanced analytics features are available in the Operations Insights Service:

- Exadata Database Service on Dedicated Infrastructure
 - [Operations Insights for Oracle Databases](#)

See also: [Oracle Cloud Observability and Management Platform](#)

Configure Automatic Service Request Infrastructure

In addition to monitoring infrastructure with Enterprise Manager, Oracle can detect failures, dispatch field engineers, and replace failing hardware without customer involvement.

For example, Oracle Automatic Service Request (ASR) is a secure, scalable, customer-installable software solution available as a feature. The software resolves problems faster by using auto-case generation for Oracle's server and storage systems when specific hardware faults occur.

See also: [Oracle Automatic Service Request \(Doc ID 1185493.1\)](#)

Exercise Capacity Planning

Periodically perform capacity planning exercises to ensure that your current hardware resources can accommodate existing workload and projected growth.

With database consolidation, this exercise should be done before migrating or adding a new database to the existing system.

Note that concurrent workloads can interfere with each other and can cause unpredictable behavior at times, so performance and HA testing may be required.

Using Database multitenant container databases, database resource management, or Exadata consolidation practices can help optimize existing system resources and constrain workload usage to meet expectations.

Check the Latest MAA Best Practices

The MAA solution encompasses the full stack of Oracle technologies, so you can find MAA best practices for Oracle Database, Oracle Cloud, Oracle Exadata, Zero Data Loss Recovery Appliance, Oracle Fusion Middleware, Oracle Applications Unlimited, and Oracle Enterprise Manager on the MAA pages.

MAA solutions and best practices continue to be developed and published on <http://www.oracle.com/goto/maa>.

Part II

Oracle Database High Availability Best Practices

- [Overview of Oracle Database High Availability Best Practices](#)
- [Oracle Database Configuration Best Practices](#)
- [Oracle Flashback Best Practices](#)
- [Oracle Global Data Services Best Practices](#)

8

Overview of Oracle Database High Availability Best Practices

By adopting the Oracle MAA best practices for Oracle Database, you can achieve the service levels of the Oracle MAA Bronze reference architecture.

The Bronze architecture achieves the highest availability for a single-instance database configuration, whether it is a standalone database or part of a consolidated multitenant database, by using the high availability capabilities included in Oracle Database Enterprise Edition.

The Bronze architecture is the base configuration for the other MAA reference architectures. The Oracle Database best practices should also be implemented in the Silver, Gold, and Platinum reference architectures, unless specifically noted in the best practices for that architecture.

For information about the components, service levels, and benefits of the Bronze reference architecture, as well as the MAA architectures that build on the Bronze base, see the "High Availability Reference Architectures" interactive diagram at <https://www.oracle.com/webfolder/technetwork/tutorials/architecture-diagrams/high-availability-overview/high-availability-reference-architectures.html>.

Oracle Database Configuration Best Practices

Adopt the Oracle MAA best practices for configuring all Oracle single-instance databases to reduce or avoid outages, reduce the risk of corruption, and improve recovery performance.

Note that the following Oracle Database best practices are used to configure the Oracle MAA Bronze reference architecture, and they are also the base database base practices for the other MAA reference architectures: Silver (Oracle RAC), Gold (Oracle Data Guard), and Platinum (Oracle GoldenGate).

Use a Server Parameter File (SPFILE)

The server parameter file (SPFILE) enables a single, central parameter file to hold all database initialization parameters associated with all instances of a database. This provides a simple, persistent, and robust environment for managing database parameters. SPFILE is recommended to be placed in the DATA ASM disk group.

Enable Archive Log Mode and Forced Logging

Running the database in ARCHIVELOG mode and using database FORCE LOGGING mode are prerequisites for database recovery operations.

The ARCHIVELOG mode enables online database backup and is necessary to recover the database to a point in time later than what has been restored. Features such as Oracle Data Guard and Flashback Database require that the production database run in ARCHIVELOG mode.

If you can isolate data that never needs to be recovered within specific tablespaces, then you can use tablespace level FORCE LOGGING attributes instead of the database FORCE LOGGING mode.

Configure an Alternate Local Archiving Destination

The local archive destination, usually LOG_ARCHIVE_DEST_1, should have an alternate local destination on a different ASM disk group. This configuration prevents database hanging due to lack of archive log space if DB_RECOVERY_FILE_DEST fills up or is unavailable for any reason.

Table 9-1 Alternate Local Archiving Configuration Parameters

Database Parameter	LOG_ARCHIVE_DEST_n parameter settings for local archive destinations
LOG_ARCHIVE_DEST_n	LOCATION=USE_DB_FILE_RECOVERY_DEST VALID_FOR=(ALL_LOGFILES,ALL_ROLES) MAX_FAILURE=1 REOPEN=5 DB_UNIQUE_NAME=db_unique_name of the database ALTERNATE=some other log archive destination. Must be log_archive_dest_[1-10]
LOG_ARCHIVE_DEST_y	LOCATION=A disk group other than the disk group used for DB_RECOVERY_FILE_DEST. Usually the DATA disk group. VALID_FOR=(ALL_LOGFILES,ALL_ROLES) MAX_FAILURE=1 REOPEN=5 ALTERNATE= the primary local archive log destination: usually LOG_ARCHIVE_DEST_1
DB_RECOVERY_FILE_DEST	Archive destination, for example, a RECO disk group
LOG_ARCHIVE_DEST_STATE_n	ENABLE
LOG_ARCHIVE_DEST_STATE_y	ALTERNATE

Sample parameter settings:

- LOG_ARCHIVE_DEST_1='LOCATION=USE_DB_FILE_RECOVERY_DEST
VALID_FOR=(ALL_LOGFILES,ALL_ROLES) MAX_FAILURE=1 REOPEN=5
DB_UNIQUE_NAME=db_unique_name of the database ALTERNATE=LOG_ARCHIVE_DEST_10'
- LOG_ARCHIVE_DEST_10='LOCATION=+DATA VALID_FOR=(ALL_LOGFILES,ALL_ROLES)
MAX_FAILURE=1 REOPEN=5 DB_UNIQUE_NAME=db_unique_name of the database
ALTERNATE=LOG_ARCHIVE_DEST_1'
- LOG_ARCHIVE_DEST_STATE_1 =enable
- LOG_ARCHIVE_DEST_STATE_10=alternate
- DB_RECOVERY_FILE_DEST=typically the RECO disk group

Use a Fast Recovery Area

The Fast Recovery Area is Oracle-managed disk space that provides a centralized disk location for backup and recovery files.

The Fast Recovery Area is defined by setting the following database initialization parameters:

- DB_RECOVERY_FILE_DEST specifies the default location for the fast recovery area. Set this parameter to the RECO disk group.
- DB_RECOVERY_FILE_DEST_SIZE specifies (in bytes) the hard limit on the total space to be used by database recovery files created in the recovery area location.

Set this parameter to a value large enough to store archived logs, flashback logs and any local database backup files locally. Having the files locally can reduce your recovery time after restoring a backup. RMAN will automatically manage these files according to your RMAN backup and data retention policies. Typically customers store 24 hours of data in the destination

When your system hosts many databases sharing the same DB_RECOVERY_FILE_DEST_SIZE, space needs to manage and monitored holistically. Recommended to alert when RECO disk group for example is 90% full.

Enable Flashback Database

Flashback Database provides an efficient alternative to point-in-time recovery for reversing unwanted database changes.

Flashback Database lets you rewind an entire database backward in time, reversing the effects of database changes within a time window. The effects are similar to database point-in-time recovery. You can flash back a database by running a single RMAN command or a SQL*Plus statement instead of using a complex procedure.

To enable Flashback Database, configure a fast recovery area and set a flashback retention target using the best practices listed below. This retention target specifies how far back you can rewind a database with Flashback Database.

- Know your application performance baseline before you enable flashback database to help determine the overhead and to assess the application workload implications of enabling flashback database.
- Ensure that the fast recovery area space is sufficient to hold the flashback database flashback logs. A general rule of thumb is that the volume of flashback log generation is approximately the same order of magnitude as redo log generation. For example, if you intend to set DB_FLASHBACK_RETENTION_TARGET to 24 hours, and if the database generates 20 GB of redo in a day, then allow 20 GB to 30 GB disk space for the flashback logs.
 - An additional method to determine fast recovery area sizing is to enable flashback database and allow the database to run for a short period of time (2-3 hours). Query V\$FLASHBACK_DATABASE_STAT. ESTIMATED_FLASHBACK_SIZE to retrieve the estimated amount of space required for the fast recovery area.
 - Note that the DB_FLASHBACK_RETENTION_TARGET is a target and there is no guarantee that you can flashback the database that far. In some cases if there is space pressure in the fast recovery area where the flashback logs are stored, then the oldest flashback logs may be deleted. To guarantee a flashback point-in-time you must use guaranteed restore points.
- Ensure that there is sufficient I/O bandwidth to the fast recovery area. Insufficient I/O bandwidth with flashback database on is usually indicated by a high occurrence of the FLASHBACK BUF FREE BY RVWR wait event.
- To monitor the progress of a flashback database operation you can query the V\$SESSION_LONGOPS view. An example query to monitor progress is

```
SELECT sofar, totalwork, units FROM v$session_longops WHERE opname =  
'Flashback Database';
```

- For repetitive tests where you must flashback to the same point, use flashback database guaranteed restore points instead of enabling flashback database. This will minimize space usage.

- Flashback PDB can rewind a pluggable database without affecting other PDBs in the CDB. You can also create PDB restore points.

Set FAST_START_MTTR_TARGET Initialization Parameter

With Fast-Start Fault Recovery, the FAST_START_MTTR_TARGET initialization parameter simplifies the configuration of recovery time from instance or system failure.

The FAST_START_MTTR_TARGET parameter specifies a target for the expected recovery time objective (RTO), which is the time, in seconds, that it should take to start the instance and perform cache recovery. When you set this parameter, the database manages incremental checkpoint writes in an attempt to meet the target. If you have chosen a practical value for this parameter, then you can expect your database to recover, on average, in approximately the number of seconds you have chosen.

Initially, set the FAST_START_MTTR_TARGET initialization parameter to 300 (seconds), or to the value required for your expected recovery time objective (RTO). As you set or lower this value, database writer (DBWR) will become more active to meet your recovery targets.

Make sure that you have sufficient IO bandwidth to handle potential higher load. See the Database Performance Tuning Guide for information about monitoring and tuning FAST_START_MTTR_TARGET.

Outage testing for cases such as node or instance failures during peak loads is recommended.

Protect Against Data Corruption

Oracle Database corruption prevention, detection, and repair capabilities are built on internal knowledge of the data and transactions it protects, and on the intelligent integration of its comprehensive high availability solutions.

A data block is corrupted when it is not in a recognized Oracle Database format, or its contents are not internally consistent. Data block corruption can damage internal Oracle control information or application and user data, leading to crippling loss of critical data and services.

When Oracle Database detects corruption, it offers block media recovery and data file media recovery to recover the data. You can undo database-wide logical corruptions caused by human or application errors with Oracle Flashback Technologies. Tools are also available for proactive validation of logical data structures. For example, the SQL*Plus ANALYZE TABLE statement detects inter-block corruptions.

The following are best practices for protecting your database against corruption.

- Use Oracle Automatic Storage Management (Oracle ASM) to provide disk mirroring to protect against disk failures.
- Use the HIGH redundancy disk type for optimal corruption repair with Oracle ASM.

Using Oracle ASM redundancy for disk groups provides mirrored extents that can be used by the database if an I/O error or corruption is encountered. For continued protection, Oracle ASM redundancy lets you move an extent to a different area on a disk if an I/O error occurs. The Oracle ASM redundancy mechanism is useful if you have bad sectors returning media errors.

- Enable Flashback technologies for fast point-in-time recovery from logical corruptions that are most often caused by human error, and for fast reinstatement of a primary database following failover.

- Implement a backup and recovery strategy with Recovery Manager (RMAN) and periodically use the RMAN BACKUP VALIDATE CHECK LOGICAL scan to detect corruptions.
Use RMAN and Oracle Secure Backup for additional block checks during backup and restore operations. Use Zero Data Loss Recovery Appliance for backup and recovery validation including corruption checks and repairs, central backup validation, reduced production database impact, and Enterprise Cloud backup and recovery solutions.
- Set database initialization parameter DB_BLOCK_CHECKSUM=TYPICAL or FULL.
- Evaluate setting DB_BLOCK_CHECKING=MEDIUM or FULL, but only after a full performance evaluation with the application.

Set the LOG_BUFFER Initialization Parameter to 128MB or Higher

Set the LOG_BUFFER initialization parameter to a minimum of 128 MB for databases with flashback enabled.

Set USE_LARGE_PAGES=ONLY

On Linux, the database's SGA should leverage large pages for consistent performance and stability.

There are two ways to ensure this happens with the USE_LARGE_PAGES parameter:

- USE_LARGE_PAGES=ONLY - Hugepages must be preallocated before instance startup.
- USE_LARGE_PAGES=AUTO_ONLY - Hugepages are dynamically acquired at instance startup time, but this dynamic acquisition can fail if memory is fragmented or if another instance is starting up and dynamically acquiring hugepages at the same time.

The MAA best practice is USE_LARGE_PAGES=ONLY. This recommendation is applicable for Cloud and non-Cloud environments, and all Cloud and Exadata automation tools ensure this configuration is in place.

Note

Oracle RDBMS 19c default for USE_LARGE_PAGES on Exadata is AUTO_ONLY, but this value will be deprecated in the future.

Use Bigfile Tablespace

As databases grow larger more data files are added to smallfile tablespaces, which requires additional administration, monitoring, and maintenance, while negatively impacting database open time and role transition time in Oracle Data Guard environments.

Bigfile tablespaces allow a single large data file per tablespace, up to 32TB for 8k blocksize and 128TB for 32k blocksize. The single data file reduces the number of files in the database thus improving database checkpoint, database open, and role transition time, while improving administration costs.

Recommendations include:

- For **new database** design and deployment, use bigfile tablespaces and partitioning to minimize the number of data files. Partitioning of large tables prevents having an enormous bigfile. A reasonable bigfile should still be 16TB or less.
 - For very large tables that have different retention policies, or have different access requirements, use Oracle Partitioning as part of your database and object design. Oracle Partitioning can also work around any potential bigfile size limitation.
 - For very large tablespaces, use bigfile tablespaces instead of many smallfile data files. Bigfile tablespaces are only supported for locally managed tablespaces with automatic segment space management.
 - There are no negative trade-offs for using bigfile tablespaces, other than understanding the maximum limits for your DB_BLOCK_SIZE. To continue to ensure good database backup and restore performance, you should also use the RMAN SECTION SIZE parameter to parallelize backup and restore operations when there are bigfile tablespaces.
- For **existing databases** with a lot of data files, focus on tablespaces that have the most data files and evaluate if you can use the ALTER TABLE MOVE or online redefinition to migrate tables or partitions to bigfile tablespaces.

The following tables show a recent Data Guard performance test which demonstrates that reducing the number of data files in the database from 9000 data files to ~100 data files improved failover times by 10x and switchover times by 4 times.

Unplanned Outage/DR (Failover)	Initial Configuration	Tuned MAA Configuration
Close to Mount (C2M)	21 secs	1 sec
Terminal Recovery (TR)	154 secs	2 secs
Convert to Primary (C2P)	114 secs	5 secs
Open new Primary (OnP)	98 secs	28 secs
Open PDB and Start Service (OPDB)	146 secs	16 secs
Total App Downtime	533 secs or 8min 53 secs	52 secs (90% drop)

Planned DR Switch (Switchover)	Initial Configuration	Tuned MAA Configuration
Convert Primary to Standby	26 secs	21 sec
Convert Standby to Primary (C2P)	47 secs	7 secs
Open new Primary (OnP)	152 secs	14 secs
Open PDB and Start Service (OPDB)	130 secs	39 secs
Total App Downtime	355 secs or 5 minutes 55 secs	81 secs (78% drop)

For existing databases with a lot of data files, the following table compares the use of ALTER TABLE MOVE or DBMS_REDEFINITION to migrate tables or partitions to bigfile tablespaces.

Areas of Interest or Use Cases	DBMS_REDEFINITION	ALTER TABLE MOVE ONLINE
Application Impact	• No DDL changes allowed during move • Application blackout of seconds during activation	• No DDL changes allowed during move • Application blackout during final switch unknown

Areas of Interest or Use Cases	DBMS_REDEFINITION	ALTER TABLE MOVE ONLINE
Application Functionality Supported	- DML supported - PDML supported - No DDL changes allowed during move	- DML supported - PDML not supported - No DDL changes allowed during move
Impact of indexes	- Available during move - Indexes maintained	- Available during move - Indexes maintained after move (with UPDATE INDEXES clause) - Indexes moved separately (REBUILD ONLINE)
Space Requirements	Double space required (tables+indexes)	Double space required (tables+indexes)
Table Partition Functionality	Move entire partitioned table with one execution	Move partition by partition in order to maintain all indexes
Statistics Management	New statistics can be created before activation	New statistics created after activation
Monitoring Progress	You can query the V\$ONLINE_REDEF view to monitor the progress of an online table redefinition operation.	Query V\$SESSION_LONGOPS?
Resume on failure	Restart-able	Unknown
Rollback	Yes	N/A
Restrictions	- Tables with LONG columns can be redefined online, but those columns must be converted to CLOBs. Also, LONG RAW columns must be converted to BLOBs. Tables with LOB columns are acceptable. - Index-organized table can be moved - Domain indexes can be moved - Parallel DML and direct path INSERT operations are allowed Many 'corner case' restrictions with DBMS_REDEFINITION. See Restrictions for Online Redefinition of Tables in <i>Oracle Database Administrator's Guide</i>	- Cannot move a table with a LONG or RAW column - Cannot move partitioned index-organized table. - Cannot move if a domain index is defined on the table like spatial, XML, or a Text index. - Parallel DML and direct path INSERT operations are not possible during a table move. - Cannot move index-organized tables that contain any LOB, VARRAY, Oracle-supplied type, or user-defined object type columns.
Documentation and References	See DBMS_REDEFINITION in <i>Oracle Database PL/SQL Packages and Types Reference</i>	See ALTER TABLE in <i>Oracle Database SQL Language Reference</i>

Use Automatic Shared Memory Management and Avoid Memory Paging

Enable Automatic Shared Memory Management by setting the SGA_TARGET parameter, and set the USE_LARGE_PAGES database initialization parameter to AUTO_ONLY or ONLY and the USE_LARGE_PAGES ASM initialization parameter to TRUE.

Use the following guidelines in addition to setting SGA_TARGET to enable Automatic Shared Memory Management.

- The sum of SGA and PGA memory allocations on the database server should always be less than your system's physical memory while still accommodating memory required for processes, PGA, and other applications running on the same database server.

- To get an accurate understanding of memory use, monitor PGA memory and host-based memory use by querying V\$PGASTAT for operating systems statistics.
- Avoid memory paging by adjusting the number of databases and applications, or reducing the allocated memory settings.

Set PGA_AGGREGATE_LIMIT to specify a hard limit on PGA memory usage. If the PGA_AGGREGATE_LIMIT value is exceeded, Oracle Database first terminates session calls that are consuming the most untunable PGA memory. Then, if the total PGA memory usage is still over the limit, the sessions that are using the most untunable memory will be terminated.

Set the database initialization parameter USE_LARGE_PAGES=AUTO_ONLY or ONLY, and set the ASM initialization parameter USE_LARGE_PAGES=TRUE.

- Make sure that the entire SGA of a database instance is stored in HugePages by setting the init.ora parameter USE_LARGE_PAGES=ONLY, or set to AUTO_ONLY on Exadata systems.

Setting USE_LARGE_PAGES=ONLY is recommended for database instances, because this parameter ensures that an instance will only start when it can get all of its memory for SGA from HugePages.

- For ASM instances leave the parameter USE_LARGE_PAGES=ONLY (the default value). This setting still ensures that HugePages are used when available, but also ensures that ASM as part of Grid Infrastructure starts when HugePages are not configured, or insufficiently configured.
- Use Automatic Shared Memory Management, because HugePages are not compatible with Automatic Memory Management.

Use Oracle Clusterware

Oracle Clusterware lets servers communicate with each other, so that they appear to function as a collective unit. Oracle Clusterware has high availability options for all Oracle databases including for single instance Oracle databases. Oracle Clusterware is one of minimum requirements in making applications highly available.

Oracle Clusterware provides the infrastructure necessary to run Oracle Real Application Clusters (Oracle RAC), Oracle RAC One Node, and Oracle Restart. Oracle Grid Infrastructure is the software that provides the infrastructure for an enterprise grid architecture. In a cluster, this software includes Oracle Clusterware and Oracle ASM.

For a standalone server, the Grid Infrastructure includes Oracle Restart and Oracle ASM. Oracle Restart provides managed startup and restart of a single-instance (non-clustered) Oracle database, Oracle ASM instance, service, listener, and any other process running on the server. If an interruption of a service occurs after a hardware or software failure, Oracle Restart automatically restarts the component.

Oracle Clusterware manages resources and resource groups to increase their availability, based on how you configure them. You can configure your resources and resource groups so that Oracle Clusterware:

- Starts resources and resource groups during cluster or server start
- Restarts resources and resource groups when failures occur
- Relocates resources and resource groups to other servers, if the servers are available

For more information, see *Oracle Clusterware Administration and Deployment Guide* topics, High Availability Options for Oracle Database and Making Applications Highly Available Using Oracle Clusterware.

Oracle Flashback Best Practices

Oracle Database Flashback Technologies is a unique and rich set of data recovery solutions that let the database reverse human errors by selectively and efficiently undoing the effects of a mistake.

Before Flashback was introduced to Oracle Database, it might have taken minutes to damage a database but hours to recover it. With Flashback, correcting an error takes about as long as it took to make it. In addition, the time required to recover from this error is not dependent on the database size, which is a capability unique to Oracle Database.

Flashback supports database recovery at all levels, including the row, transaction, table, and the entire database. Flashback provides an ever-growing set of features to view and rewind data back and forth in time, and address several critical high availability and disaster recovery use cases. The list of features and use cases, as well as some key examples, can be found in [Oracle Flashback Technology](#).

The Flashback features give you the capability to query historical data, perform change analysis, and perform the self-service repair to recover from logical corruptions while the database is online. With Oracle Flashback Technology, you can indeed undo the past.

Oracle Flashback Performance Observations

After adopting the configuration and operational best practices and applying recommended patches, Oracle has observed the following performance observations when Flashback Database is enabled on the primary or standby databases.

- Flashing back a database or a PDB to the previous hour usually takes seconds and minutes, even with a very high workload. It finishes in a fraction of the time it takes to apply a given amount of redo. Here are some observations:
 - Flashing back a large batch workload consisting of 400 GB of changes completed in less than 5 minutes.
 - Flashing back of a heavy OLTP of 8GB of changes completed in less than 2 minutes.
 - Due to many variables, there is no rule-of-thumb or calculation to estimate the time to complete a flashback. The tests which produced these observations were done on Exadata to remove system bottlenecks such as storage I/O bandwidth.
- The impact on OLTP workload on the primary database is usually less than 5 percent.
- The impact of a large insert (batch inserts, for example) or direct load operations is usually less than 5 percent if Flashback block new optimization is in effect; otherwise, the impact can vary dramatically (2-40% impact), so testing is required.

Refer to the Flashback use cases that mention block new optimization descriptions and exceptions in [Oracle Flashback Performance Tuning for Specific Application Use Cases](#).

- Enabling Flashback database can reduce peak redo apply performance rates on a physical standby database if the standby system cannot handle the additional I/O throughput requirements in the Fast Recovery Area. However, even with Flashback database enabled on the standby, the achievable redo apply rates with Flashback enabled are still very high and can outperform application redo generation rates.

The following lists describe the critical flashback milestones and key performance improvements across different Oracle Database software releases:

Oracle Database 12c Release 2 (12.2)

- Flashback Pluggable Database enables the flashback of individual PDBs without affecting other PDBs.
- PDB Restore Points enable an ease of use method to set an alias to an SCN. This alias can then be used for flashback PDB or Point-In-Time Recovery.

Oracle Database 19c

- Creating a Restore Point on a primary database automatically propagates to a standby database, and creates a corresponding Restore Point on the standby database.
- When Flashback Database is enabled on both the primary and standby databases in an Oracle Data Guard configuration, flashing back the primary database causes the standby database to automatically flash back as well.

Oracle Database 21c

- Migrate Flashback Data Archive-enabled tables between different database releases
- Flashback Database support for data file resizing operations
- PDBs can be recovered to an orphan PDB incarnation within the same CDB incarnation or an ancestor incarnation

Oracle Flashback Configuration Best Practices

The following are Oracle MAA best practices for configuring Flashback technologies in Oracle Database.

Setting DB_FLASHBACK_RETENTION_TARGET

Set the DB_FLASHBACK_RETENTION_TARGET initialization parameter to the largest value prescribed by any of the following conditions that apply:

- To leverage Flashback database to reinstate your failed primary database after Oracle Data Guard failover, set DB_FLASHBACK_RETENTION_TARGET to a minimum of 60 (minutes) to enable reinstatement of a failed primary. When enabling Flashback database, a couple of hours are required to generate sufficient flashback data into the flashback logs before reinstatement is possible. You can query V\$FLASHBACK_DATABASE_LOG to find the oldest flashback time.
- Consider cases where there are multiple outages (for example, a network outage, followed later by a primary database outage) that may result in a transport lag between the primary and standby databases at failover time. For such cases, set DB_FLASHBACK_RETENTION_TARGET to a value equal to the sum of 60 (mins) plus the maximum transport lag that you wish to accommodate. This ensures that the failed primary database can be flashed back to an SCN that precedes the SCN at which the standby became primary. This is a requirement for primary reinstatement.
- If you are using Flashback Database for fast point-in-time recovery from user error or logical corruptions, set DB_FLASHBACK_RETENTION_TARGET to a value equal to the farthest time in the past that you wish to be able to recover from.
- In most cases, DB_FLASHBACK_RETENTION_TARGET should be set to the same value on the primary and standby.

Sizing the Fast Recovery Area

Flashback Database uses its own logging mechanism, creating flashback logs and storing them in the Fast Recovery Area (FRA). Ensure that the FRA has allocated sufficient space to accommodate the flashback logs for the target retention size and for peak batch rates. Sizing the FRA is described in detail in the Oracle Backup and Recovery documentation, but generally the volume of flashback log generation is similar in magnitude to redo log generation. Use the following conservative formula and approach:

Target FRA = Current FRA + DB_FLASHBACK_RETENTION_TARGET x 60 x Peak Redo Rate (MB/sec)

Example:

- Current FRA or DB_RECOVERY_FILE_DEST_SIZE=1000GB
- Target DB_FLASHBACK_RETENTION_TARGET=360 (360 minutes)
- From AWR:
 - The peak redo rate for OLTP workload is 3 MB/sec for the database
 - The peak redo rate for the batch workload is 30 MB/sec for the database, and the longest duration is 4 hours
 - The worst-case redo generation size for a 6 hour window is (240 minutes x 30 MB/sec x 60 secs/min) + (120 minutes x 3 MB/sec x 60 secs/min) = 453,600 MB, or approximately 443 GB
- Proposed FRA or DB_RECOVERY_FILE_DEST_SIZE= 443 GB +1000 GB = 1443 GB

An additional method to determine FRA sizing is to enable Flashback Database and allow the database applications to run for a short period (2-3 hours), and then query V\$FLASHBACK_DATABASE_STAT. ESTIMATED_FLASHBACK_SIZE.

Note that the DB_FLASHBACK_RETENTION_TARGET is a target, and there is no guarantee that you can flash back the database that far. The oldest flashback logs may be deleted if there is space pressure in the FRA where the flashback logs are stored. See *Maintaining the Fast Recovery Area in Oracle Database Backup and Recovery User's Guide* for a detailed explanation of the FRA deletion rules. You must use guaranteed restore points (GRP) to guarantee a flashback point-in-time. The required flashback logs will never be recycled or purged with GRP until GRP is dropped. The database can stop responding if there is a GRP but there is insufficient space, so you must allocate more space in the FRA depending on the intended duration of the GRP.

Configuring sufficient I/O bandwidth for Fast Recovery Area

Insufficient I/O bandwidth with Flashback Database on is usually indicated by a high occurrence of the "FLASHBACK BUF FREE BY RVWR" wait event in an Automatic Workload Repository (AWR) report for OLTP workloads and "FLASHBACK LOG FILE WRITE" latency > 30 ms for large insert operations.

In general, flashback I/Os are 1 MB in size. The overall write throughput would be similar to the redo generation rate if database force logging were enabled, or identical to your load rate for direct load operations. For simplicity, configure one large shared storage GRID and configure DATA on the outer portion of the disks or LUNS and RECO (FRA) on the inner amount of the disks or LUNS. This is done automatically for Exadata systems.

Setting LOG_BUFFER

To give Flashback Database more buffer space in memory, set the initialization parameter LOG_BUFFER=256MB or higher, depending on operating system limits.

Oracle Flashback Operational Best Practices

The following are Oracle MAA recommended operational best practices for Flashback Database.

- Gather database statistics using Automatic Workload Repository (AWR) or Oracle Enterprise Manager before and after enabling Flashback Database, so you can measure the impact of enabling Flashback Database.
- Using Oracle Enterprise Manager, set the Enterprise Manager monitoring metric, "Recovery Area Free Space (%)" for proactive alerts of space issues with the fast recovery area (FRA).
- To monitor the progress of a Flashback Database operation, you can query the V\$SESSION_LONGOPS view. For example,

```
select * from v$session_longops where opname like 'Flashback%';
```

If more detail is required on the Flashback Database operation, generate a detailed trace of the Flashback Database operation in the DIAGNOSTIC_DEST trace directory for the database by setting database parameter _FLASHBACK_VERBOSE_INFO=TRUE.

- When using Flashback Database to perform repeated tests on a test database, it is recommended that you use Guaranteed Restore Points (GRP) only, without explicitly turning on Flashback Database. To minimize space usage and flashback performance overhead, follow this recommended approach:

```
Create Guaranteed Restore Point (GRP)
Execute test
loop
    Flashback database to GRP
    Open resetlogs
    Create new GRP
    Drop old GRP
    Execute
testEnd loop
```

- Follow the Oracle Data Guard redo apply best practices described in [Redo Apply Troubleshooting and Tuning](#).

Oracle Flashback Performance Tuning for Specific Application Use Cases

Performance Tuning for OLTP Workloads

The "flashback buf free by RVWR" wait event only occurs when Flashback Database is enabled. A session waits for the recovery writer (RVWR) to write flashback data to the flashback logs on disk because the buffers are full. The session may need to wait until RVWR can free up the buffers.

Suppose this event becomes one of the top wait events for the database. In that case, it is typically because the file system or storage system for the Fast Recovery Area (FRA) has insufficient I/O bandwidth to accommodate additional I/O from the Flashback writes. Refer to the Flashback Database section in the *Oracle Database Backup and Recovery User's Guide* for tuning considerations and evaluate the corresponding I/O and storage statistics.

Table 10-1 Top 5 Timed Foreground Events

Event	Waits	Times	Average wait (ms)	% database time	Wait class
write complete waits	1,842	23,938	12995	33.68	Configuration
flashback buf free by RVWR	53,916	20,350	377	28.63	Configuration
cell single block physical read	3,029,626	16,348	5	23.00	User I/O
buffer busy waits	6,248	5,513	882	7.76	Concurrency
DB CPU		1,757		2.47	

Performance Tuning for Direct Path Operations

Look at the "flashback log write bytes" and "physical write bytes" system statistics found in v\$sysstat, in your AWR reports, or Oracle Enterprise Manager.

- "flashback log write bytes" = The total size in bytes of Flashback Database data written by RVWR to Flashback Database logs
- "physical write bytes" = The total size in bytes of all disk writes from the database application activity (and not other kinds of instance activity).

If (flashback log write bytes) / (physical write bytes) < 5%, then Flashback is not impacting your performance.

Otherwise, evaluate any operational changes or bug fixes that will allow you to use the Flashback block new optimization feature (refer to performance observation section above). Furthermore, ignore the "flashback log file sync" wait event, even if it's one of the top wait events.

Example of block new optimization in effect

In this example:

- flashback log write bytes = 1,223,442,432
- physical write bytes = 184,412,282,880

The result of (flashback log write bytes) / (physical write bytes) = 0.0066 < 5%, implies that there's only a fraction of flashback data compared to the physical writes within this interval where there are direct load operations. Even in this case, the "flashback log file sync" wait event was the 2nd highest wait event in the database, as shown in the following table.

Table 10-2 Top 5 Timed Foreground Events

Event	Waits	Times	Average wait (ms)	% database time	Wait class
direct path write	136,553	7,875	58	39.12	User I/O
flashback log file sync	91,566	5,887	64	29.25	User I/O
DB CPU		3,092		15.36	
log buffer space	20,545	1,737	85	8.63	Configuration
gc buffer busy release	1,277	487	382	2.42	Cluster

Example of block new optimization not in effect

In this example:

- flashback log write bytes= 184,438,194,176
- physical write bytes =184,405,925,888

The result of (flashback log write bytes) / (physical write bytes) = 100% > 5%, implies that in this case all direct writes also result in flashback log writes. Listed here are the top wait events for this case.

Table 10-3 Top 5 Timed Foreground Events

Event	Waits	Times	Average wait (ms)	% database time	Wait class
flashback log file sync	170,088	22,385	132	52.04	User I/O
direct path write	278,185	8,284	30	19.26	User I/O
flashback buf free by RVWR	38,396	5,048	131	11.74	Configuration
direct path read	220,618	4,242	19	9.86	User I/O
DB CPU		2,788		6.48	

Performance Tuning for Conventional Load Operations

The following examples illustrate two conventional loads, one that uses block new optimization and one that does not.

Example of block new optimization not in effect

The example below does not use the block new optimization because of a truncate just before loading the table. The wait events for a conventional load without block new optimization show a fairly large amount of total wait time spent in "flashback log file sync". This is because of time needed to read them before the image of the block into the buffer cache and well as writing the block to the flashback log.

Table 10-4 Top 5 Timed Foreground Events

Event	Waits	Times	Average wait (ms)	% database time	Wait class
flashback log file sync	235,187	13,728	58	30.82	User I/O
direct path write	558,037	10,818	19	24.29	User I/O
direct path read	459,076	8,419	18	18.90	User I/O
DB CPU		6,171		13.85	
flashback buf free by RVWR	79,463	4,268	54	9.58	Configuration

Looking at the instance statistics below you can see very little increase in the statistics that track block new optimizations.

Statistic	Total	Per second	Per transaction
flashback cache read optimizations for block new	62	0.06	1.13
flashback direct read optimizations for block new	8	0.01	0.15
flashback log write bytes	177,533,280,256	177,245,433.67	3,227,877,822.84

Statistic	Total	Per second	Per transaction
flashback log writes	18,917	18.89	343.95

If the "flashback cache read optimizations for block new" is much smaller than "flashback log writes" then the block new optimization does not have an effect.

The best tuning recommendation for the above load operation would be to increase I/O bandwidth or, perhaps better, change the manner in which the load is performed so that it can take advantage of block new optimizations. You can also wait until you are outside the flashback retention target, or remove the object from recycle bin if it was dropped.

Example of block new optimization not effect

The wait events for a conventional load with block new optimization show a relatively small amount of total time spent in "flashback log file sync" compared to other database waits, as shown here.

Table 10-5 Top 5 Timed Foreground Events

Event	Waits	Times	Average wait (ms)	% database time	Wait class
direct path write	284,115	8,977	32	34.20	User I/O
DB CPU		6,284		23.94	
log buffer space	128,879	5,081	39	19.36	Configuration
flashback log file sync	139,546	3,178	23	12.11	User I/O
latch: redo allocation	95,887	1,511	16	5.76	Other

Looking at the instance statistics you can see that the statistics that track block new operations have significantly increased during the load.

Statistic	Total	Per second	Per transaction
flashback cache read optimizations for block new	329	0.53	9.68
flashback direct read optimizations for block new	698,410	1,116.43	20,541.47
flashback log write bytes	1,197,514,752	1,914,271.66	35,221,022.12
flashback log writes	18,951	30.29	557.38

Oracle Global Data Services Best Practices

Oracle Database Global Data Services (GDS) is a holistic automated workload management feature of Oracle Database.

GDS provides workload routing, load balancing, inter-database service failover, replication lag-based routing, role-based global services, and centralized workload management for a set of replicated databases that are globally distributed or located within the same data center.

You can use GDS to achieve these benefits without the need to integrate with multiple-point solutions or homegrown products. GDS provides optimized hardware and software utilization, better performance, scalability, and availability for application workloads running on replicated databases.

Introduction to Global Data Services

Numerous organizations have multiple copies of their production or standby databases either locally or in different geographical locations. This redundancy fulfills various business needs, such as ensuring uninterrupted availability, preparing for disaster recovery, localizing content and caching, scaling operations, optimizing performance for local users, and complying with local regulations. Oracle Active Data Guard and Oracle GoldenGate, which synchronize these copies, are Oracle's strategic disaster recovery and replication technologies that provide the lowest Recovery Time Objective (RTO) and Recovery Point Objectives (RPO).

Achieving high performance and high availability by distributing workload across multiple database replicas presents challenges that extend beyond the capabilities of the replication technology. The workload must be load-balanced to use resources effectively and achieve the best performance. Outages must be handled intelligently so that applications remain available and deliver the best possible quality of service should a replica go offline. In an ideal world, load balancing and high availability using a pool of replicated databases occurs seamlessly and optimally using real-time information available from the Oracle Database. This ideal is achievable using Oracle Global Data Services (GDS).

GDS extends the concept of database services, a mechanism used for workload management in Oracle Real Application Clusters (RAC), to a globally replicated configuration that includes any combination of Oracle RAC, single instance database, Oracle Active Data Guard, and Oracle GoldenGate. GDS allows services to be deployed anywhere within this globally replicated configuration, supporting load balancing, high availability, regional affinity, and so on. GDS is built on time-tested technological building blocks such as services (Dynamic Workload Management), Oracle Active Data Guard and GoldenGate replication, and Oracle Net Listener.

Even though the feature is called Global Data Services, the databases that constitute the GDS configuration can be globally distributed or located within the same data center. Clients can securely connect by specifying a service name without needing to know where the physical data center assets providing that service are located, enabling a highly flexible deployment for an enterprise data cloud. With GDS, replicated databases appear to the applications as a single data source.

A GDS configuration contains multiple global service managers per region. The global service managers are "global listeners," which understand real-time load characteristics and the user-defined service placement policies on the replicated databases. These global service

managers are instrumental in performing inter-database service failovers and load balancing of GDS. GDS is a shared infrastructure that can govern multiple sets of replicated databases. This documentation describes the configuration and operational practices for GDS. It is intended for enterprise architects, database architects, database administrators, technology managers, solution architects, application architects, and those who are influential in the overall database architecture design.

Global Data Services Concepts

Database services are logical abstractions for managing workloads in an Oracle database. Each service represents a workload with common attributes, service-level thresholds, and priorities. The grouping can be based on work characteristics, including the application function. For example, the Oracle E-Business Suite defines a service for each application module, such as general ledger, accounts receivable, and order entry. Services are built into Oracle Database, providing a single system image for workloads. Services enable administrators to configure a workload, administer it, enable and disable it, and measure the workload as a single entity. Clients connect using a database service name.

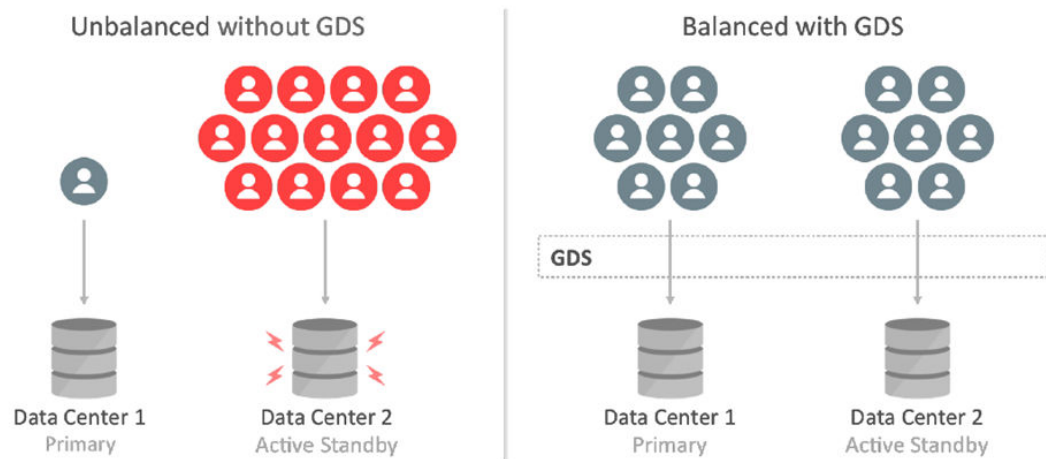
In Oracle RAC, a service can span one or more instances and facilitate workload balancing based on real-time transaction performance. This provides high availability, rolling changes by workload, and complete location transparency. For replicated environments, GDS introduces the concept of a "global service". Global services are provided across a set of databases containing replicated data that belongs to a particular administrative domain known as a GDS pool. Examples of a GDS pool are a SALES pool or an HR pool. A set of databases in a GDS configuration and the database clients are said to be in the same GDS region if they share the network proximity. Examples of GDS regions are the Asian region, European region, and so on.

All of the characteristics of traditional database services are supported by global services. Global services extends traditional database services with additional attributes such as global service placement, replication lag (Oracle Active Data Guard and Oracle GoldenGate from 19c onwards), and region affinity.

Global service placement: When a global service is created, GDS allows the preferred and available databases for that service to be specified. The available databases support a global service if the preferred database fails. In addition, GDS allows you to configure a service to run on all the replicas of a given GDS pool.

Replication lag: Clients can be routed to the Oracle Active Data Guard standbys that are not lagging by the tolerance limit established with the lag attribute of a global service.

Region affinity: A global service allows you to set preferences to which region (for example Asia or Europe) your given applications should connect.

Figure 11-1 Workload Balancing with Global Data Services

Key Capabilities of Global Data Services

Global Data Services (GDS) technology provides the following principal capabilities:

- **Region-based workload routing:** With GDS, you can choose to configure client connections to be routed among a set of replicated databases in a local region. This capability allows you to maximize application performance (avoiding the network latency overhead of accessing databases in remote areas).
- **Connect-time load balancing:** Global service managers use the load statistics from all databases in the GDS pool, inter-region network latency, and the configured connect-time load balancing goal to route the incoming connections to the best database in a GDS pool.
- **Runtime load balancing:** GDS enables runtime load balancing across replicated databases by publishing a real-time load balancing advisory for connection pool-based clients (for example, OCI, JDBC, ODP.NET, WebLogic, and so on.). The connection pool-based clients subscribe to this load-balancing advisory and route database requests in real time across already-established connections.

With GDS's runtime connection load balancing feature, application client work requests are dynamically routed to the database that offers the best performance. In addition, GDS also supports the ability to dynamically re-distribute connections when the database performance changes.

- **Inter-database service failover:** If a database global service crash occurs, GDS, considering the service placement attributes, automatically performs an inter-database service failover to another available database in the pool. GDS sends Fast Application Notification (FAN) events so that the client connection pools can reconnect to the new database where the global service has been started.
- **Replication lag-based workload routing:** With Oracle Active Data Guard, standbys may lag behind the primary database. A global service allows you to choose the acceptable lag tolerance for a given application. GDS routes requests to a standby database whose lag is below the limit. If the lag exceeds the lag limit, the service is relocated to another available standby database that lags below the threshold. New requests are routed to a standby database that satisfies the lag limit. The global service is shut down if there is no available

database. When the lag is resolved or comes within the limit, GDS automatically brings up the service.

With Oracle GoldenGate replication, when the lag exceeds the lag threshold defined for a service (lag defined by `SELECT MAX(INCOMING_LAG FROM GGADMIN.GG_LAG)`), that service is stopped on that database. The service defines the effect of that; it may or may not terminate all the sessions based on the configuration. The service is restarted if the lag comes back within the threshold. After the service has been stopped, the global service manager automatically performs failover processing. Any new connections to this service are directed elsewhere than the lagged database. So, if there are two databases in the pool, and the service is preferred_all with lag=10 initially, the service runs on both databases, and the connections are load-balanced. If the second database goes past the lag threshold, the service is stopped there, and any new connections are directed only to the first database. If the lag comes back within the threshold, the service is restarted, load balancing continues, and new connections can use the second database.

- **Role-based global services:** When a database role transition is performed with Oracle Data Guard Broker, GDS can automatically relocate the global service to the new primary database and the new standby if the role assigned to the service matches the role of the database.
- **Centralized workload management for replicas:** GDS allows more straightforward configuration and management of the replicated databases' resources located anywhere with a single unified framework.

Benefits of Global Data Services

Global Data Services (GDS) allows fault-tolerant database services to be deployed and centrally managed across a set of replicated databases. The GDS framework provides workload balancing across these databases. More specifically, GDS is an Oracle-integrated solution that renders the following benefits:

- Higher availability and global scalability support seamless inter-database service failover among replicated databases in any data center, yielding higher application availability.
- GDS provides application scalability on demand by allowing dynamic addition of databases. It enables replicated databases to be added to the GDS infrastructure dynamically and transparently to obtain additional resource capability to scale application workloads. GDS allows this with no change to the application configuration or client connectivity.

Better Performance and Elasticity

With integrated load balancing across multiple databases, GDS addresses inter-region resource fragmentation. Under-utilized resources in one region can be put to work on the workload of another region's over-utilized resources, achieving optimal resource utilization. GDS sends work requests to less powerful databases in a GDS pool containing replicated databases running on database servers of different processor generations and various resources (CPU, memory, I/O). When more powerful databases are overloaded, the goal should be to equalize the response time.

Improved Manageability

The GDSCTL command-line interface or the Oracle Enterprise Manager Cloud Control graphical user interface can administer a GDS configuration. With centralized administration of global resources, geographically dispersed replicated databases, whether regional or global, can be effectively utilized within the unified framework of GDS.

Centralized workload management of replicated databases, inter-database service failover, and runtime load balancing are unique features of GDS. GDS enables a truly elastic and agile enterprise and extends the benefits of a private data cloud.

Application Workload Suitability for Global Data Services

Global Data Services (GDS) is best for replication-aware application workloads; it is designed to work in replicated environments. Applications that are suitable for GDS adoption possess any of the following characteristics:

- The application can separate its work into read-only, read-mostly, and read-write services to use Oracle Active Data Guard or Oracle GoldenGate replicas. GDS does not distinguish between read-only, read-write, and read-mostly transactions. The application connectivity has to be updated to separate read-only or read-mostly services from read-write services, and the GDS administrator can configure the global services on appropriate databases. For example, a reporting application can function directly with a read-only service at an Oracle Active Data Guard standby database.
- Administrators should be aware of and avoid or resolve multi-master update conflicts to take advantage of Oracle GoldenGate replicas. For example, an internet directory application with built-in conflict resolution enables the read-write workload to be distributed across multiple databases, each open read-write and synchronized using Oracle GoldenGate multi-master replication.
- Ideally, the application is tolerant of replication lag. For example, a web-based package tracking application that allows customers to track the status of their shipments using a read-only replica, where the replica does not lag the source transactional system by more than 10 seconds.

Global Data Services in Oracle Maximum Availability Architecture

The Oracle Maximum Availability Architecture (MAA) is Oracle's best practices blueprint for the integrated suite of Oracle's advanced High Availability (HA) technologies. Enterprises that leverage MAA in their IT infrastructure find they can quickly and efficiently deploy applications that meet their business requirements for high availability.

Without Oracle Global Data Services, administrators were required to deploy 3rd party load balancers or have opted for custom-written connection managers. The 3rd party solutions come with significant integration costs, and the DIY homegrown solutions require high initial cost and time to value as well as an overhead of maintenance and support effort.

Using Global Data Services, you can unify replicated database resources within a single framework, avoiding the need to produce homegrown or 3rd party integration for load balancing. You can minimize the vendor integration touch points in their high availability and disaster recovery stack.

Figure 11-2 Oracle Global Data Services in Maximum Availability Architecture

Global Data Services is a strategic MAA component available within the Oracle Database. GDS is well integrated with the Oracle ecosystem, providing workload routing, load balancing, and service failover across replicated databases located within and across data centers. Simply put, GDS is a database load balancer for replicated databases and provides high availability through the inter-database service failover capability.

Global Data Services lets administrators manage client workloads automatically and transparently across replicated databases that offer common services. A database service is a named representation of one or more database instances. Services let you group database workloads and route a particular work request to an appropriate instance. A global service is provided by multiple databases synchronized through data replication.

Global Data Services provides dynamic load balancing, failover, and centralized service management for replicated databases that offer common services. The set of databases can include Oracle RAC and non-cluster Oracle databases interrelated through Oracle Data Guard, databases consolidated under Oracle Multitenant, Oracle GoldenGate, or any other replication technology.

For detailed information about GDS, see the Global Data Services technical brief at <http://oracle.com/goto/gds>.

Partial or Full Site Outage with Global Data Services

Complete-site failure results in both the application and database tiers being unavailable. To maintain availability, users must be redirected to a secondary site that hosts a redundant application tier and a synchronized copy of the production database. MAA's best practice is to maintain a running application tier at the standby site to avoid startup time and use Oracle Data Guard to manage the synchronized copy of the production database. Upon site failure, a WAN traffic manager performs a DNS failover (manually or automatically) to redirect all users to the application tier at the standby site. In contrast, a failover transitions the standby database to the primary production role.

A partial-site failure is when the application or database stack encounters an outage. Connecting clients to the disaster recovery site application stack can mitigate application stack failure. However, when the primary database becomes unavailable, the application tier connected to the primary database remains intact. All that is required to maintain availability is to redirect the application tier to the new primary database after a Data Guard failover has been completed. In this use case, the standby database is located within a distance from the surviving application tier such that it can deliver acceptable performance using a remote connection after a database failover has occurred.

Global Data Services (GDS) enables service management and load balancing between replicated databases *within* a region. However, the application tier still functions when GDS global service managers can route connections to the best available replica based on load balancing policies and service availability. By contrast, an out-of-region failover requires users to be directed to a remote application tier local to the new production database (served by a different set of in-region global service managers). This document focuses on GDS configuration for failover within a region.

Global Data Services Configuration

High-Level Deployment Steps

The following are the basic steps you would take to implement Global Data Services.

1. Install Global Data Services (GDS) global service manager software on global service manager servers.
 - Minimum of 1 global service manager for each region
 - Recommended 3 global service managers for each region
2. Pre-create the GDS catalog database.
3. Set up GDS Administrator accounts and privileges.
4. Configure GDS.
 - Create the GDS catalog and standby databases.
 - Add global service managers, regions, pools, databases, and global services.
5. Set up client connectivity.

Configuration Example

The following steps describe how to implement Global Data Services.

This example configuration of Global Data Services (GDS) uses an Administrator-managed Oracle RAC database. Administrator-managed deployment means that you configure database services to run on specific instances belonging to a particular database using a preferred and available designation.

Policy-managed deployment is based on server pools, where database services run within a server pool as singletons or uniformly across all of the servers in the server pool. Databases are deployed in one or more server pools, and the size of the server pools determines the number of database instances in the deployment. For detailed information about GDS, see *Global Data Services Concepts and Administration Guide*.

1. Create and prepare a GDS catalog database.

GDS uses a catalog database to store meta-data relating to the layout and status of the GDS configuration. For maximum availability, Oracle recommends that the GDS catalog database be deployed independently and that Oracle's high-availability features, such as Oracle Real Application Clusters (Oracle RAC) and Oracle Data Guard, be used to protect the catalog database against outages.
2. Create the GSM_ADMIN user and assign that user the GSMADMIN_ROLE.

Note that by default, the password for both GSM_ADMIN, GSMUSER, and GSMCATUSER expires after 180 days.

```
SQL> create user gsm_admin identified by password;
```

User created.

```
SQL> grant gsmadmin_role to gsm_admin;
```

Grant succeeded.

```
SQL> exit
```

3. Copy the Oracle Net alias that can be used to access the catalog database and place it in the tnsnames.ora file in the global service manager home.

```
GDSCAT =  
(DESCRIPTION =  
(ADDRESS = (PROTOCOL = TCP)(HOST = <hostname>)(PORT = 1521))  
(CONNECT_DATA =  
(SERVER = DEDICATED)  
(SERVICE_NAME = gdscat)  
))
```

4. With the environment configured for the global service manager home, use GDSCCTL to connect to and create the GDS catalog with Auto VNCR disabled (Auto VNCR can cause problems with Oracle RAC deployments).

```
GDSCCTL>create catalog -database gdscat -user gsm_admin -autovncr OFF
```

"gsm_admin" password:

Catalog is created

5. Connect to the catalog database, unlock the GSMCATUSER user, and set the password.

```
SQL> alter user gsmcatuser account unlock;
```

User altered.

```
SQL> alter user gsmcatuser identified by password;
```

User altered.

6. With the environment configured for the global service manager home, use GDSCCTL to connect to, create, and start the global service manager listeners.

As a best practice, global service manager listeners should reside on hardware separate from that hosting the Oracle Databases in the GDS configuration. The resource requirements for hardware needed to run global service manager listeners are lightweight and can easily be accommodated using virtual machines.

```
GDSCCTL>add gsm -gsm gsm1 -listener 1522 -catalog gdscat
```

"gsmcatuser" password:

```
Create credential oracle.security.client.connect_string1
```

```
GSM successfully added
```

```
GDSCCTL>start gsm -gsm gsm1
```

```
GSM is started successfully
```

```
GDSCCTL>status
```

```
Alias GSM1
```

```
Version 19.17.0.3.0
```

```
Start Date 13-APR-2023 09:40:59
```

```
Trace Level off
```

```
Listener Log File
```

```
/u01/app/oracle/diag/gsm/hostname/gsm1/alert/log.xml
```

```
Listener Trace File
```

```
/u01/app/oracle/diag/gsm/hostname/gsm1/trace/ora_64863_139739749930432.trc
```

```
Endpoint summary
```

```
(ADDRESS=(HOST=hostname.example.com)(PORT=1522)(PROTOCOL=tcp))
```

```
GSMOCI Version 0.6.11
```

```
Mastership Y
```

```
Connected to GDS catalog Y
```

```
Process Id 64883
```

```
Number of reconnections 0
```

```
Pending tasks. Total 0
```

```
Tasks in process. Total 0
```

```
Regional Mastership TRUE
```

```
Total messages published 0
```

```
Time Zone -04:00
```

```
Orphaned Buddy Regions: None
```

```
GDS region regionora
```

7. With the environment configured for the global service manager home, use GDSCCTL to create a default GDS pool and default region.

```
GDSCCTL>add gdspool -gdspool sales
```

```
GDSCCTL>add region -region slc
```

```
GDSCCTL>add region -region sca
```

8. With Auto VNCR disabled during GDS catalog creation to avoid issues, use GDSCCTL to add hosts using the add invitednode command, using the host name or IP address appropriately.

```
GDSCCTL>add invitednode 192.0.2.1
```

```
GDSCCTL>add invitednode host1.example.com
```

9. Unlock the GSMUSER account.

Before adding a database to a pool, the database administrator should unlock the GSMUSER account and give the password to the GDS pool administrator, as shown in the following example.

```
SQL> alter user gsmuser account unlock;
```

User altered.

```
SQL> alter user gsmuser identified by password;
```

User altered.

10. Add databases to the GDS pool.

To be part of a GDS pool, a database must use a server parameter file (SPFILE). An Oracle RAC database should also have SCAN set up.

To add a database, connect to the GDS catalog using the GDS pool or GDS administrator credentials. For example, without Data Guard, the following add database command can be used.

```
GDSCTL>add database -connect mts -region sca -gdspool sales
```

Catalog connection is established

"gsmuser" password:

DB Unique Name: mts

Note

When using Oracle Active Data Guard with GDS, use `add brokerconfig` instead of `add database`, and then use `modify database` to configure the standby database (see `add brokerconfig`). The syntax for these commands would be like the following.

```
GDSCTL>add brokerconfig -connect <primary_db> -gdspool <dbpool> -  
region <dc> -pwd <gsmuser_pwd>
```

```
GDSCTL>modify database -database <standby_db> -connect <dc> -  
gdspool <dbpool> -region <dc> -pwd <gsmuser_pwd>
```

Database instance registration with a global service manager succeeds only when the request originates from a valid node. If a host on which a database resides contains multiple network interfaces, the auto-configuration could register the wrong set of IP addresses, leading to the database registration being rejected.

11. Correct any rejected registration and properly discover all database instances.

If a firewall exists between the global service managers, and the databases and the ports are not opened, the registration fails. From the global service manager alert log, you will see entries similar to the following.

```
Listener(VNCR option 1) rejected Registration request from destination
```

```
192.0.2.2
```

```
Listener(VNCR option 1) rejected Registration request from destination
```

```
192.0.2.3
```

You will find that the database object exists in the GDS catalog, but some or all instances associated with specific hosts are missing.

```
GDSCTL>databases
```

```
Database: "mts" Registered: Y State: Ok ONS: Y. Role: PRIMARY
```

```
Instances: 1 Region: slc
```

```
Registered instances:
```

```
sales%1
```

To correct the rejected registration and properly discover all database instances, run `add invitednode` using the rejected IP address listed in the global service manager alert log.

12. If there is a firewall between the global service managers and the database, then once the ports have been opened and verified using `tnsping`, issue the `add invitenode` command as shown here.

```
GDSCTL>add invitednode 192.0.2.3
```

```
GDSCTL>databases
```

```
Database: "mts" Registered: Y State: Ok ONS: Y. Role: PRIMARY
```

```
Instances: 2 Region: slc
```

```
Registered instances:
```

```
sales%1
```

```
sales%2
```

13. Create a service on the GDS pool databases.

The `GDSCTL add service` command creates a service on the GDS pool databases.

```
GDSCTL>add service -service sales_sb -preferred_all -gdspool sales -  
notification TRUE
```

If this is an Oracle RAC database being added with multiple instances, then you must modify the service to add the database instances.

```
GDSCTL>modify service -gdspool sales -service sales_sb -database mts -  
add_instances -preferred mts1,mts2
```

```
GDSCTL>modify service -gdspool sales -service sales_sb -database stm -  
add_instances -preferred stm1,stm2
```

```
GDSCTL>start service -service sales_sb -gdspool sales
```

14. Verify that the global service is running.

```
GDSCTL>services
```

```
Service "sales_sb.sales.oradbccloud" has 2 instance(s). Affinity: ANYWHERE
```

```
Instance "sales%1", name: "mts1", db: "mts", region: "slc", status: ready.
```

```
Instance "sales%2", name: "mts2", db: "mts", region: "slc", status: ready.
```

Configuration Best Practices

Oracle MAA recommends the following best practices for implementing Global Data Services:

- Each client communicates using an Oracle-integrated connection pool such as UCP, OCI, or ODP.NET. The connection pools will be notified about any service failovers and load balancing advisory notifications using Fast Application Notification Events.
- **Run three global service managers in each region.** Create three global service managers in each region so that if one global service manager goes down, you have two remaining global service managers to provide redundancy. Each global service manager should reside on separate hardware. Global service managers enable connection routing among replicated databases. A global service manager is a stateless, lightweight, and intelligent listener that can repopulate its metadata from the GDS catalog.
- **Protect the GDS catalog database with Oracle Data Guard.** The GDS catalog is a small (< 100 GBs) repository that hosts the metadata of the GDS configuration, regions, global service managers, global services, databases, and so on. MAA recommends that you set up a local Data Guard standby database configured with Maximum Availability database protection mode, Data Guard Fast-Start failover, and a remote physical standby database. All GDS catalog standby databases should use Oracle Active Data Guard for the best data protection and reside on separate hardware and storage.

Using FAN ONS with Global Data Services

Fast Application Notification (FAN) uses the Oracle Notification Service (ONS) for event propagation to all Oracle Database clients, including JDBC, Tuxedo, and listener clients.

ONS is installed as part of Oracle Global Data Services, Oracle Grid Infrastructure on a cluster, in an Oracle Data Guard installation, and when Oracle WebLogic is installed.

ONS propagates FAN events to all other ONS daemons it is registered with. No steps are needed to configure or enable FAN on the database server side, with one exception: OCI FAN and ODP FAN require that notification be set to TRUE for the service by GDSCTL. With FAN

auto-configuration at the client, ONS jar files must be on the CLASSPATH or in the ORACLE_HOME, depending on your client.

General Best Practices for Configuring FCF Clients

Follow these best practices before progressing to driver-specific instructions.

- Use a dynamic database service. Using FAN requires that the application connects to the database using a dynamic global database service. This is a service created using GDSCTL.
- Do not connect using the database service or PDB service. These services are for administration only and are not supported for FAN. The TNSnames entry or URL must use the service name syntax and follow best practices by specifying a dynamic database service name. Refer to the examples later in this document.
- Use the Oracle Notification Service when you use FAN with JDBC thin, Oracle Database OCI, or ODP.Net clients. FAN is received over ONS. Accordingly, in the Oracle Database, ONS FAN auto-configuration is introduced so that FCF clients can discover the server-side ONS networks and self-configure. FAN is automatically enabled when ONS libraries or jars are present.
- Enabling FAN on most FCF clients is still necessary in the Oracle Database. FAN auto-configuration removes the need to list the global service managers an FCF client needs.
- Listing server hosts is incompatible with location transparency and causes issues with updating clients when the server configuration changes. Clients already use a TNS address string or URL to locate the global service manager listeners.
- FAN auto-configuration uses the TNS addresses to locate the global service manager listeners and then asks each server database for the ONS server-side addresses. When there is more than one global service manager FAN auto-configuration contacts each and obtains an ONS configuration for each one.
- The ONS network is discovered from the URL when using the Oracle Database. An ONS node group is automatically obtained for each address list when LOAD_BALANCE is off across the address lists.
- By default, the FCF client maintains three hosts for redundancy in each node group in the ONS configuration.
- Each node group corresponds to each GDS data center. For example, if there is a primary database and several Oracle Data Guard standbys, there are by default three ONS connections maintained at each node group. The node groups are discovered when using FAN auto-configuration.

With node_groups defined by FAN auto-configuration, and load_balance=false (the default), more ONS endpoints are not required. If you want to increase the number of endpoints, you can do this by increasing max connections. This applies to each node group. Increasing to 4 in this example maintains four ONS connections at each node. Increasing this value consumes more sockets.

```
oracle.ons.maxconnections=4 ONS
```

- If the client is to connect to multiple clusters and receive FAN events from them, for example in Oracle RAC with a Data Guard event, then multiple ONS node groups are needed. FAN auto-configuration creates these node groups using the URL or TNS name. If automatic configuration of ONS (Auto-ONS) is not used, specify the node groups in the Oracle Grid Infrastructure or oraaccess.xml configuration files.

Client Side Configuration

As a best practice, multiple global service managers should be highly available. Clients should be configured for multiple connection endpoints where these endpoints are global service managers rather than local, remote, or single client access name (SCAN) listeners. For OCI / ODP .Net clients use the following TNS name structure.

```
(DESCRIPTION=(CONNECT_TIMEOUT=90)(RETRY_COUNT=30)(RETRY_DELAY=3)
(TRANSPORT_CONNECT_TIMEOUT=3)
  (ADDRESS_LIST =
    (LOAD_BALANCE=on)
    (ADDRESS=(PROTOCOL=TCP)(HOST=GSM1)(PORT=1522))
    (ADDRESS=(PROTOCOL=TCP)(HOST=GSM2)(PORT=1522))
    (ADDRESS=(PROTOCOL=TCP)(HOST=GSM3)(PORT=1522)))
  (ADDRESS_LIST=
    (LOAD_BALANCE=on)
    (ADDRESS=(PROTOCOL=TCP)(HOST=GSM2)(PORT=1522)))
(CONNECT_DATA=(SERVICE_NAME=sales)))
```

Always use dynamic global database services created by GDSCTL to connect to the database. Do not use the database service or PDB service, which are for administration only not for application usage and they do not provide FAN and many other features because they are only available at mount.

Use the latest client driver aligned with the latest or older RDBMS for JDBC. Use one DESCRIPTION in the TNS names entry or URL Using more causes long delays connecting when RETRY_COUNT and RETRY_DELAY are used. Set CONNECT_TIMEOUT=90 or higher to prevent logon storms for OCI and ODP clients.

Application-Level Configuration

Configuring FAN for Java Clients Using Universal Connection Pool

The best way to take advantage of FCF with the Oracle Database JDBC thin driver is to use the Universal Connection Pool (UCP) or WebLogic Server Active GridLink.

Setting the pool property FastConnectionFailoverEnabled on the Universal Connection Pool enables Fast Connection Failover (FCF). Active GridLink always has FCF enabled by default. Third-party application servers, including IBM WebSphere and Apache Tomcat, support UCP as a connection pool replacement.

For more information about embedding UCP with other web servers, see the following technical briefs.

- Design and deploy WebSphere applications for planned or unplanned database downtimes and runtime load balancing with UCP (<https://www.oracle.com/docs/tech/database/planned-unplanned-rlb-ucp-websphere.pdf>)
- Design and deploy Tomcat applications for planned or unplanned database downtimes and Runtime Load Balancing with UCP (<https://www.oracle.com/docs/tech/database/planned-unplanned-rlb-ucp-tomcat.pdf>)

Follow these configuration steps to enable Fast Connection Failover.

1. The connection URL must use the service name syntax and follow best practices by specifying a dynamic database service name and the JDBC URL structure (above and below).

All other URL formats are not highly available. The URL may use JDBC thin or JDBC OCI.

2. If wallet authentication has not been established, remote ONS configuration is needed.

Set the pool property `setONSConfiguration` in a property file as shown in the following example. The property file specified must contain an `ons.nodes` property and, optionally, properties for `oracle.ons.walletfile` and `oracle.ons.walletpassword`. An example of an `ons.properties` file is shown here.

```
PoolDataSource pds = PoolDataSourceFactory.getPoolDataSource();
pds.setConnectionPoolName("FCFSamplePool");
pds.setFastConnectionFailoverEnabled(true);
pds.setONSConfiguration("propertiesfile=/usr/ons/ons.properties");
pds.setConnectionFactoryClassName("oracle.jdbc.pool.OracleDataSource");
pds.setURL("jdbc:oracle:thin@((CONNECT_TIMEOUT=4)(RETRY_COUNT=30)
(RETRY_DELAY=3) "+ "
(ADDRESS_LIST = "+ " (LOAD_BALANCE=on) "+ " ( ADDRESS =
(PROTOCOL = TCP)(HOST=GSM1)(PORT=1522))) "+ " (ADDRESS_LIST = "+
" (LOAD_BALANCE=on)
"+ "( ADDRESS = (PROTOCOL = TCP)(HOST=GSM2)(PORT=1522)))"+
"(CONNECT_DATA=(SERVICE_NAME=service_name)))");
```

3. Ensure the pool property `setFastConnectionFailoverEnabled=true` is set.
4. The `CLASSPATH` must contain `ons.jar`, `ucp.jar`, and the JDBC driver jar file.
For example, `ojdbc8.jar`.
5. If you are using JDBC thin with Oracle Database, Application Continuity can be configured to failover the connections after FAN is received.
6. If the database needs different ONS endpoints than those autoconfigured, the ONS endpoints can be enabled.

In a situation where multiple clusters exist with auto-ons enabled, auto-ons would generate node lists with the following guidelines:

For EVERY active nodelist `oracle.ons.maxconnections` is set to 3 by default, so there is no need to set this explicitly. This example will result in the ONS client trying to maintain six total connections.

Configuring FAN for OCI Clients

OCI clients use ONS for FAN transport.

OCI clients embed FAN at the driver level so that all clients can use it regardless of the pooling solution. Ideally, both the server and the client would use release 19c or later.

Configuration for SQL*Plus and PHP

1. Set notification for the service.
2. For PHP clients only, add `oci8.events=0n` to `php.ini`.

Important: If `xml` is present with `events=false` or events are not specified, this disables the usage of FAN. To maintain FAN with SQL*Plus and PHP when `oraccess.xml` is in use, set `events=true`.

3. On the client side, using a client and Oracle Database, enable FAN in `xml`.

Configuration for OCI Clients

1. Tell OCI where to find ONS Listeners Starting.

The client installation comes with ONS linked into the client library. Using auto-config, the ONS endpoints are discovered from the TNS address. This automatic method is the recommended approach. Like ODP.Net, manual ONS configuration is also supported using `oraaccess.xml`.

2. Enable FAN high availability events for the OCI connections.

To enable FAN you edit the OCI file `xml` to specify the global parameter events. This file is located in `$ORACLE_HOME/network/admin`. See [Step 3: Use Fast Application Notification](#) for more information.

3. Tell OCI where to find ONS Listeners.

The client installation comes with ONS linked into the client library. Using auto-config, the ONS endpoints are discovered from the TNS address. This automatic method is the recommended approach. Like ODP.Net, manual ONS configuration is also supported using `oraaccess.xml`.

4. Enable FAN on the server for all OCI clients.

It is still necessary to enable FAN on the database server for all OCI clients (including SQL*Plus).

Controlling Logon Storms

Small connection pools are strongly recommended, but controlling logon storms can be done with tuning when you have many connections.

Oracle MAA recommends the following tuning on servers that host Global Service Managers.

1. Increase the Listen backlog at the OS level.

To have the new value take effect without rebooting the server, perform the following as root.

```
echo 8000 > /proc/sys/net/core/somaxconn
```

2. To persist the value across reboots, add this setting to `/etc/sysctl.conf`.

```
net.core.somaxconn=6000
```

3. Increase `queuesize` for the global service manager listener.

Update `sqlnet.ora` in Oracle home that the listeners are running from to increase the `queuesize` parameter:

```
TCP.QUEUESIZE=6000
```

Graceful Application Switchover

Database services are used to manage workloads during the planned outage properly. Services must be properly created, and the application must obtain connections from a service.

These recommendations assume using a FAN-aware connection pool, such as Oracle Universal Connection Pool (UCP) to gracefully drain connections without application interruption from a service that is stopped. Your applications can use other connection types that don't support FAN-aware connection pools or have long-running transactions. Ideally, these applications will be disconnected before the maintenance window.

The recommendations below describe how to disconnect some sessions when their transaction ends in a timely manner or, ultimately, when the instance is shut down for maintenance.

The recommended and validated approach to understanding and optimizing your application's connection configuration is provided in the following sections; certain applications may have specific guidelines to follow.

Understanding Your Application's Use of Connections

Understanding how your application obtains and releases its connections is critical to determining whether it can gracefully switch to other instances in the cluster.

Find the following information about your application:

- What was the workload during the planned outage (OLTP/short or batch/long transactions)?
 - Short transactions using a connection pool such as UCP or ODP.NET can be quiesced rapidly.
 - Long transactions need additional time to quiesce or must have batch jobs stopped or queued at an appropriate time in advance.
- What type of connection was obtained: Java, OCI, ODP with C#, or ODP with OCI)?
 - UCP, ICC, ODP.NET, and OCI session pools use Fast Application Notification (FAN) to drain the pool quickly; other connections require waiting until the connection is closed (or termination if the application allows)
- What is the amount of time to wait for the connection pool to quiesce before stopping the service?
 - Useful to know the proper amount of time is given before disconnection is performed
- Can the application handle disconnection after the transaction completes (applies to batch workloads)?
 - If the application can't handle being disconnected gracefully, it must be stopped before the planned maintenance, or Application Continuity might be an option to avoid interruption.

Services and Application Configuration Best Practices

You must have properly configured services and application attributes to perform a graceful switchover successfully. See My Oracle Support Doc ID [1617163.1](#) for a matrix of validated drivers and applications clients.

① Note

You must test your configuration to ensure that it is set up and performs switchover properly before relying on it for a production system.

Using Oracle Active Data Guard with Global Data Services

Configure sessions to move in a rolling manner for Oracle Active Data Guard reader farm.

Prerequisites

You must have the following in place for this procedure to work correctly.

- Oracle Active Data Guard configuration using Oracle Database (release 19c or later recommended).
- Global Data Services (GDS) configuration using global service manager (release 19c or later recommended).
- A GDS service has been created to run on all Active Data Guard databases in the configuration.

For example:

```
GDSCTL>add service -service sales_sb -preferred_all -gdspool sales
               -role physical_standby -notification TRUE
GDSCTL>modify service -gdspool sales -service sales_sb -database mts -
add_instances
               -preferred mts1,mts2
GDSCTL>modify service -gdspool sales -service sales_sb -database stm -
add_instances
               -preferred stm1,stm2
GDSCTL>start service -service sales_sb -gdspool sales
```

1. Check the current status of the services and related instances to ensure that services can be moved successfully.

Note that the service should only be available on the source standby database at this point.

```
GDSCTL>services
Service "sales_sb.sales.oradbccloud" has 2 instance(s). Affinity: ANYWHERE
  Instance "sales%1", name: "mts1", db: "mts", region: "slc", status:
ready.
  Instance "sales%2", name: "mts2", db: "mts", region: "slc", status:
ready.
```

2. Stop services typically (not using the FORCE option) on the source database where connections are to be removed.
 - This step will quiesce the FAN-aware connection pools using FAN.
 - New connections are directed to other instances offering that service, and idle sessions are disconnected from the pool using the services.
 - Existing connections can continue until their work is complete and they are returned to the connection pool.

```
GDSCTL>stop service -service sales_sb -database mts -gdspool sales
```

Allow an agreed upon time for the sessions to disconnect and relocate, then continue with the next steps.

Note

If you are performing a rolling upgrade of an Active Data Guard reader farm and the services are not running on other Active Data Guard reader nodes, you can complete the service stop on this database before performing the GDSCTL stop service described in this step.

3. Disconnect long-running sessions after the current query is completed.

Preferably, long-running queries have been scheduled to stop or are queued before the window when connections are to be moved. This step handles long-running sessions that are still running and now need to be stopped (killed) abruptly.

4. Log on to the instance that you intend to shut down.
5. Check V\$SESSION to see if any sessions are still connected to the service.

```
SQL> SELECT service_name, count(1) FROM v$session  
GROUP BY service_name ORDER BY 2;
```

6. Run the DBMS_SERVICE.DISCONNECT_SESSION package for the service you stopped earlier.

For example:

```
SQL> exec  
  
dbms_service.disconnect_session('oltp_work',DBMS_SERVICE.POST_TRANSACTION);
```

7. Check V\$SESSION again to ensure that sessions have logged off from the service.

```
SQL> SELECT service_name, count(1) FROM v$session  
GROUP BY service_name ORDER BY 2;
```

8. Start the GDS service on the target database and allow sessions to connect.

```
GDSCTL>start service -service sales_sb -database stm -gdspool sales
```

9. Log on to the target database and check V\$SESSION to see sessions connected to the service.

```
SQL> SELECT service_name, count(1) FROM v$session  
GROUP BY service_name ORDER BY 2;
```

Using Oracle GoldenGate with Global Data Services

The following Oracle GoldenGate role transition example topology consists of two databases: GG replica1 and GG replica2. Oracle GoldenGate is set up with uni-directional replication, with Extract running initially on GG replica1 and Replicat running initially on GG replica2. The generic steps still apply for bi-directional GoldenGate replicas or downstream mining GoldenGate replicas.

Prerequisites

You must have the following in place for this procedure to work correctly.

- Oracle GoldenGate configuration that uses Oracle Database (19c or higher recommended)
- GoldenGate processes should not connect to the source or target database using the GDS service name, but a dedicated TNS alias. Using the GDS service will cause the database connections to terminate prematurely, causing possible data loss.
- A heartbeat table has been implemented in the GoldenGate source and target databases to track replication latency and ensure the Replicat applied SCN synchronization. The GoldenGate automatic heartbeat table feature should be enabled. Refer to the Oracle

GoldenGate Administration Guide for details on the automatic heartbeat table: <https://docs.oracle.com/en/middleware/goldengate/core/19.1/gclir/add-heartbeattable.html>.

- Global Data Services (GDS) configuration using global service manager (19c or higher recommended)
- GDS service has been created so that it can be run on all databases in the GoldenGate configuration.

For example:

```
GDSCTL>add service -service sales_sb -preferred_all -gdspool sales
GDSCTL>modify service -gdspool sales -service sales_sb -database mts
-add_instances -preferred mts1,mts2
GDSCTL>modify service -gdspool sales -service sales_sb -database stm
-add_instances -preferred stm1,stm2
GDSCTL>start service -service sales_sb -gdspool sales
```

Note

If you are using the lag tolerance option, specify the lag limit for the global service in seconds. Options for add service or modify service are -lag {lag_value | ANY}.

1. Check the current status of the services and related instances to ensure that they can be moved successfully.

At this point, the service should only be available on the source database.

```
GDSCTL>services
Service "sales_sb.sales.oradbcloud" has 2 instance(s). Affinity: ANYWHERE
Instance "sales%1", name: "mts1", db: "mts", region: "slc", status:
ready.
Instance "sales%2", name: "mts2", db: "mts", region: "slc", status:
ready.
```

2. Stop services (not using the FORCE option) on the source database where connections are to be removed.
 - This step will quiesce the FAN-aware connection pools using FAN.
 - New connections are directed to other instances offering that service, and idle sessions are disconnected from the pool using the services.
 - Existing connections can continue until their work is complete and they are returned to the connection pool.

```
GDSCTL>stop service -service sales_sb -database mts -gdspool sales -force
```

Allow an agreed upon time for the sessions to disconnect and relocate, then continue with the next steps. The time to allow for sessions to drain depends on the workload and user transactions for your application.

3. Disconnect long-running sessions after the current transaction is completed.

Preferably, long-running batch jobs are scheduled to be stopped or queued before the maintenance window. This step handles long-running sessions that are still running and must be stopped abruptly (e.g., killed). Check with the application developers if these long-

running batch jobs are idempotent and recoverable before disconnecting long-running sessions.

4. Log on to the instance that you intend to shut down, and check V\$SESSION to see if any sessions are still connected to the service.

```
SQL> SELECT service_name, count(1) FROM v$session
      GROUP BY service_name ORDER BY 2;
```

5. Run the DBMS_SERVICE.DISCONNECT_SESSION package for the service you stopped earlier. For example:

```
SQL> exec
      dbms_service.disconnect_session('oltp_work', DBMS_SERVICE.POST_TRANSACTION);
```

6. Check V\$SESSION again to ensure sessions have logged off from the service.

```
SQL> SELECT service_name, count(1) FROM v$session
      GROUP BY service_name ORDER BY 2;
```

7. When all sessions associated with the GDS service have been disconnected, verify that all data from the GoldenGate source databases have been replicated to the target database.

- Record the current database SCN from the GoldenGate SOURCE database.

```
SQL> SELECT current_scn FROM v$database;
```

- On the GoldenGate TARGET database, continue to monitor the Replicat applied SCN using the following query.

```
SQL> SELECT lwm_position FROM v$gg_apply_coordinator;
```

- When the target LWM_POSITION SCN is greater than the CURRENT_SCN recorded in the first step, it is safe to assume that all transactions have been replicated from the source to the target database. The users can now be switched over to the GoldenGate target database.

The above steps allow for a graceful switchover. However, if this is a failover event where the source database is unavailable, you can estimate the data loss using the steps below.

1. When using the automatic heartbeat table, use the following query to determine the replication latency.

```
SQL> col Lag(secs) format 999.9
SQL> col "Seconds since heartbeat" format 999.9
SQL> col "GG Path" format a32
SQL> col TARGET format a12
SQL> col SOURCE format a12
SQL> set lines 140
SQL> select remote_database "SOURCE", local_database "TARGET",
      incoming_path "GG Path",
      incoming_lag "Lag(secs)", incoming_heartbeat_age "Seconds since
      heartbeat" from gg_lag;
```

SOURCE	TARGET	GG Path
--------	--------	---------

```

Lag(secs) Seconds since heartbeat
-----
-----
              MTS              GDST              EXT_1A ==> DPUMP_1A ==> REP_1A
              7.3              9.0

```

The above example shows a possible 7.3 seconds of data loss between the source and target databases.

2. Start the GDS service on the target database and allow sessions to connect.

Note that if the application workload can accept a certain level of data lag, it is possible to perform this step much earlier than step two listed above.

```
GDSCTL>start service -service sales_sb -database stm -gdspool sales
```

3. Log on to the target database and check V\$SESSION to see sessions connected to the service.

```
SQL> SELECT service_name, count(1) FROM v$session
      GROUP BY service_name ORDER BY 2;
```

Global Data Services Failover Across Regions Flow

1. The administrator has failed over or switched the production database to the secondary site. This is automatic if you are using Data Guard fast-start failover.
2. The administrator starts the middle-tier application servers on the secondary site if they are not already running.
3. The wide-area traffic manager selection of the secondary site can be automatic in the case of an entire site failure. The wide-area traffic manager at the secondary site returns the virtual IP address of a load balancer at the secondary site, and clients are directed automatically on the subsequent reconnect. In this scenario, the site failover is accomplished by an automatic domain name system (DNS) failover.
4. Alternatively, a DNS administrator can manually change the wide-area traffic manager selection to the secondary site for the entire site or specific applications.

The following is an example of a manual DNS failover:

- Change the DNS to point to the secondary site load balancer: The primary (primary) DNS server is updated with the new zone information, and the change is announced with DNS NOTIFY.
- The secondary DNS servers are notified of the zone update with a DNS NOTIFY announcement, and the secondary DNS servers pull the new zone information.
- Clear affected records from caching DNS servers.
A caching DNS server is used primarily for performance and fast response. The caching server obtains information from an authoritative DNS server in response to a host query and then saves (caches) the data locally. On a second or subsequent request for the same data, the caching DNS server responds with its locally stored data (the cache) until the response's time-to-live (TTL) value expires. At this time, the server refreshes the data from the zone master. If the DNS record is changed on the primary DNS server, then the caching DNS server does not pick up the change for cached records until TTL expires. Flushing the cache forces the caching DNS server to go to an authoritative DNS server again for the updated DNS information.

- Flush the cache if the DNS server being used supports such a capability. The following is the flush capability of standard DNS BIND versions:
 - BIND 9.3.0: The command `rndc flushname name` flushes individual entries from the cache.
 - BIND 9.2.0 and 9.2.1: The cache can be flushed with the command `rndc flush`.
 - BIND 8 and BIND 9 up to 9.1.3: Restarting the named server clears the cache.
- 5. Refresh local DNS service caching.
Some operating systems might cache DNS information locally in the local name service cache. If so, this cache must also be cleared to recognize DNS updates quickly.
- 6. The secondary site load balancer directs traffic to the secondary site middle-tier application server.

Global Data Services Limitations and Requirements

A Single GDS Manages	GDS Databases
• 5,000 GDS Pools • 10 GDS Regions • 5 global service managers per Region • 10,000 Database instances • 10,000 Global Services • 1,000 Mid-tier connection pools	• Can be a Single Instance or RAC • Can be CDB or Non-CDB • Can run on commodity or Engineered systems (Oracle Exadata, ODA) • Are managed with GDSCTL CLI or Enterprise Manager DB Plug-in • Must be licensed for Oracle Active Data Guard or Oracle GoldenGate

Requirement	Network Load Balancer	Oracle GDS
Locality-based routing	Yes	Yes
Connect-time database load balancing	Yes	Yes
Publish routing and failover intelligence to clients	No	Yes
Replication lag-based database workload routing	No	Yes
Inter-database global service failover	No	Yes
Automatic role-based global services	No	Yes
Centralized management of database services across replicas	No	Yes
Native integration for Oracle Active Data Guard	No	Yes
Cost effectiveness	Additional expenditure required	Included with Oracle Active Data Guard or Oracle GoldenGate license

Part III

Oracle RAC and Clusterware Best Practices

- [Overview of Oracle RAC and Clusterware Best Practices](#)
- [MAA Platinum and MAA Diamond: Oracle RAC and Exadata](#)

Overview of Oracle RAC and Clusterware Best Practices

Oracle Clusterware and Oracle Real Application Clusters (RAC) are Oracle's strategic high availability and resource management database framework in a cluster environment, and an integral part of the Oracle MAA Silver reference architecture.

Adding Oracle RAC to a Bronze MAA reference architecture elevates it to a Silver MAA reference architecture. The Silver MAA reference architecture is designed for databases that can't afford to wait for a cold restart or a restore from backup, should there be an unrecoverable database instance or server failure.

The Silver reference architecture has the potential to provide zero downtime for node or instance failures, and zero downtime for most database and system software updates, that are not achievable with the Bronze architecture. To learn more about the Silver MAA reference architecture, see [High Availability Reference Architectures](#).

Oracle Clusterware and Oracle RAC provide the following benefits:

- High availability framework and cluster management solution
 - Manages resources, such as Virtual Internet Protocol (VIP) addresses, databases, listeners, and services
 - Provides HA framework for Oracle database resources and non-Oracle database resources, such as third party agents
- Active-active clustering for scalability and availability
 - **High Availability** If a server or database instance fails, connections to surviving instances are not affected; connections to the failed instance quickly failover to surviving instances that are already running and open on other servers in the Oracle RAC cluster
 - **Scalability and Performance** Oracle RAC is ideal for high-volume applications or consolidated environments where scalability and the ability to dynamically add or re-prioritize capacity across more than a single server are required. An individual database may have instances running on one or more nodes of a cluster. Similarly, a database service may be available on one or more database instances. Additional nodes, database instances, and database services can be provisioned online. The ability to easily distribute workload across the cluster makes Oracle RAC the ideal complement for Oracle Multitenant when consolidating many databases.

The following table highlights various Oracle Clusterware and Real Application Cluster configuration best practices.

Table 12-1 Oracle RAC HA Use Cases and Best Practices

Use Case	Best Practices
Certified and validated Clusterware software stack	Use Oracle Clusterware and avoid third-party Clusterware. See <i>Oracle Database Clusterware Administration and Deployment Guide</i>. Clusterware is built-in to all Oracle Exadata Systems.
Certified and validated storage architecture	Use Oracle Automatic Storage Management (Oracle ASM) and Oracle ASM Cluster File System (Oracle ACFS) instead of third party volume managers and cluster file systems for the following MAA benefits: • Eliminate hot spots by distributing work across all disks • Scale and adjust storage capacity by adding and dropping disks and storage online • Reduce complexity by providing a simplified and uniform method (ASMCMD, ASMCA, ExaCLI, or oeadaccli) to manage database storage • Inherent data corruption detection and repair when using ASM diskgroups • Simple management, patching and maintenance with an integrated Oracle Grid Infrastructure (Clusterware +ASM) without any additional drivers When using ASM with external redundancy, ensure that the underlying storage and network is highly available with no single point of failure. When using ASM native redundancy, high redundancy diskgroups are recommended to provide maximum protection for unplanned outages and during storage software updates. By default Exadata deployments use high redundancy for all diskgroups (both for data and recovery destinations). Oracle Automatic Storage Management Cluster File System (Oracle ACFS) is a multi-platform, scalable file system and storage management technology that extends Oracle Automatic Storage Management (Oracle ASM) functionality to support all customer files and can be leveraged for non-database files. These best practices are built-in to all Oracle Exadata Systems. See <i>Introducing Oracle Automatic Storage Management</i>

Table 12-1 (Cont.) Oracle RAC HA Use Cases and Best Practices

Use Case	Best Practices
Certified and validated network architecture	Ensure that the entire database and storage network topology has multiple network paths with no single point of failure. When connecting to the database service, use built-in Virtual Internet Protocol (VIP) addresses, Single Client Access Name (SCAN), and multiple local SCAN listeners configured over a bonded client network. Use a separate high bandwidth, bonded network for backup or Data Guard traffic. For the private network used as the cluster interconnect, Oracle recommends that non-Exadata customers use Oracle HAIP for network redundancy instead of using bonded networks. Bonding configurations have various attributes that behave differently with different network cards and switch settings. This recommendation does not apply to the private cluster interconnect in Exadata environments, because the bond setup has been properly configured and validated. Further, Exadata uses the CLUSTER_INTERCONNECT parameter over the highly available bonded network. Generic systems should NOT use the CLUSTER_INTERCONNECT and bonding but rather use Oracle HAIP.
Cluster configuration checks	Use Cluster Verification Utility (CVU) at monthly intervals to validate a range of cluster and Oracle RAC components such as shared storage devices, networking configurations, system requirements, and Oracle Clusterware. See Cluster Verification Utility Reference To perform a holistic, proactive health check and to evaluate if Oracle RAC or Exadata best practices are being followed, use exachk for Exadata RAC systems, or use orachk for non-Exadata RAC systems, at monthly intervals and before and after any software update. See ORAchk - Health Checks for the Oracle Stack (Doc ID 1268927.2) and Oracle Exadata Database Machine exachk or HealthCheck (Doc ID 1070954.1). Note that both exachk and orachk include CVU checks. Exachk covers software and configuration best practices and critical alerts for Storage, Network, Clusterware, ASM, and Database. Incorporate configuration recommendations from CVU, exachk, or orachk.

Table 12-1 (Cont.) Oracle RAC HA Use Cases and Best Practices

Use Case	Best Practices
Enable RHPHelper for zero application downtime after node shutdown or restart	RHPHelper is an Oracle Clusterware feature for Linux that permits zero application downtime when a node is shut down or restarted. It does the following: - Relocates services (that can be relocated) to other nodes, and stops services that cannot be relocated to other nodes (because they are already running everywhere they can run) - Waits for the sessions of the services relocated or stopped to drain to the available instances - Gracefully shuts down all database instances on a node, which avoids impacts from instance recovery due to aborted instances during Linux shutdown RHPHelper properly stops Oracle RAC database services with a drain timeout so connections move to other nodes in the cluster when those connections are made using connection pools or compatible drivers (using ONS FAN or In-band notifications). See Configuring Level 1: Basic Application High Availability for ensuring your applications are configured to drain properly. RHPHelper has been installed and enabled since Oracle 18.3 on Exadata. As of Oracle 23ai, it is installed on all Linux systems, but is still only enabled automatically on Exadata. In Oracle 23ai, Oracle Clusterware has added new commands to configure and enable RHPHelper: - crsctl set crs drain <pre>crsctl set crs drain enable true draintimeout 120</pre> - crsctl get crs drain <pre>crsctl get crs drain</pre> This command returns output similar to the following: <pre>'Drain timeout': 120 'Maximum drain timeout': 150</pre> See crsctl set crs drain and crsctl get crs drain for command details. For a Linux system with Oracle 23ai Grid Infrastructure installed, the above commands will configure and enable RHPHelper. For Oracle 19c and earlier, see Using RHPHelper to Minimize Downtime During Planned Maintenance on Exadata (Doc ID 2385790.1)

Table 12-1 (Cont.) Oracle RAC HA Use Cases and Best Practices

Use Case	Best Practices
Reduce downtime for database node or instance failures	Typically, the default settings are sufficient for most use cases. If node detection and instance recovery need to be expedited, evaluate lower values for FAST_START_MTTR_TARGET Reducing FAST_START_MTTR_TARGET can increase database writer activity significantly, so additional I/O bandwidth is required. For Exadata systems, Instant Failure Detection capabilities use remote direct memory access (RDMA) to quickly confirm server failures in less than 2 seconds compared to typical 30 seconds detection found in most Oracle RAC clusters.
Eliminate downtime for software updates	Use Oracle RAC rolling updates for Clusterware or database software updates (for example, Release Updates) to avoid downtime. Use out-of-place software updates when possible, so rollback and fallback use cases are simplified. Use software gold images to eliminate the complexity of running database patch utility. For a fleet of databases on a single Oracle RAC cluster or multiple clusters, use Oracle Fleet Patching and Provisioning
Make application and processes highly available on the cluster	When an application, process, or server fails in a cluster, you want the disruption to be as short as possible and transparent to users. For example, when an application fails on a server, that application can be restarted on another server in the cluster, minimizing or negating any disruption in the use of that application. Similarly, if a server in a cluster fails, then all of the applications and processes running on that server must failover to another server to continue providing service to the users. Using the built-in generic_application resource type, Oracle Clusterware can manage all of these entities to ensure high availability, resource types or customizable scripts and application agent programs, and resource attributes that you assign to applications and processes. Use Oracle Clusterware to manage third-party resources and agents that reside on the cluster. See Making Applications Highly Available Using Oracle Clusterware
Reduce application downtime for planned and unplanned outages	Leverage Clusterware-managed services and application best practices to achieve zero application downtime. Use SRVCTL to manage services for your PDB. Never use default service for application connectivity. Always have at least one preferred Oracle RAC instance and at least one additional available Oracle RAC instance for High Availability. Applications should subscribe to HA Fast Application Notifications (FAN) and be configured to respond and failover if required. See Enabling Continuous Service for Applications and Continuous Availability - Application Checklist for Continuous Service for MAA Solutions

Table 12-1 (Cont.) Oracle RAC HA Use Cases and Best Practices

Use Case	Best Practices
Capacity planning	Capacity planning and sizing should be done before deployment, and periodically afterward, to ensure that there are sufficient system resources to meet application performance requirements. Capacity planning needs to accommodate growth or consolidation of databases, additional application workloads, additional processes, or anything that strains existing system resources. Evaluating if performance requirements are still met during an unplanned outage or planned maintenance events is also crucial.

MAA Platinum and MAA Diamond: Oracle RAC and Exadata

Both Oracle Real Application Clusters (RAC) and Oracle Exadata Database Machine deliver high availability; however, only Exadata, as a fully engineered system, uniquely guarantees near-zero brownout times (0-10 seconds) for local failures.

This document contrasts Exadata's superior HA performance with tuned custom Oracle RAC setups on redundant infrastructure. Given its exceptional availability, robust data protection, and performance-preserving Quality of Service, Exadata is a mandatory prerequisite for mission-critical database solutions at the MAA Platinum and MAA Diamond tier and above, both on-premises and in the cloud. While Oracle RAC offers excellent high availability (HA), non-Exadata configurations can experience application brownouts up to 60 seconds during various local failures.

Exadata's unique high-availability capabilities significantly minimize application impact during unplanned outages (compute, network, storage) and seamless infrastructure updates. It also provides enhanced data integrity with automatic corruption repair. Crucially, Exadata manages "gray failures"—underperforming or degraded components—even before full failure. This comprehensive resilience solidifies Exadata as the optimal database platform within the MAA framework and justifies its prerequisite status for the MAA Platinum tier and above.

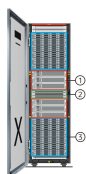
Oracle Exadata's fully engineered design and deep integration across hardware and software tiers make it the superior choice for high availability in mission-critical database environments, in particular in the areas of unplanned outages/planned maintenance, data protection, and Quality of Service. While Oracle RAC offers robust high availability, Exadata uniquely minimizes application brownouts to near-zero during local failures, a significant improvement over the potential brownouts of non-Exadata configurations that can take minutes.

Its comprehensive resilience, encompassing instant failure detection, smart caching, proactive "gray failure" management, and enhanced data protection with automatic corruption repair, ensures unparalleled service levels and consistent performance. These unique capabilities solidify Exadata's position as a mandatory prerequisite for MAA Platinum tier and above database solutions, both on-premise and in the cloud, ultimately delivering the highest levels of availability and data integrity.

Exadata High Availability at Every Tier

Exadata is designed for high availability in every layer, hardware, and software, and has smart software that provides additional benefits.

The image below shows Oracle Exadata Database Machine's built in high availability features.



1. Redundant Database Servers

- Active-Active highly available clustered servers
- Hot-swappable power supplies, fans, and flash cards
- Redundant power distribution units
- Integrated HA software/firmware stack

2. Redundant Network

- Redundant 100Gbit/sec RoCE and switches
- Client access using HA bonded networks
- Integrated HA software/firmware stack

3. Redundant Storage Grid

- Data mirrored across storage servers
- Hot-swappable power supplies, fans, M.2 drives, and flash cards
- Redundant, non-blocking I/O paths
- Integrated HA software/firmware stack

The perfect pairing of hardware, software, and life cycle optimizations, combined with built-in MAA best practices, makes Exadata the best MAA database platform. Every Exadata release is extensively tested, and every major Exadata release or Exadata hardware generation requires even more stringent and long MAA chaos engineering validation that ensures there are no HA regressions, and that any additional HA feature works.

Application throughput and response time are measured, and any deviation from our TPS or response time thresholds is flagged and alerted on. MAA and Exadata development meet weekly on configuration and life cycle best practices, and also discuss any critical issues that need to be reported to our invaluable Exadata community through Exachk, Oracle cloud alerting, or another alerting mechanism.

High Availability Differentiators

Exadata is uniquely qualified to support the most stringent Oracle Real Application Clusters (RAC) system service levels with the following high-level differentiators that exist in all three tiers: compute, network, and storage.

- **Unplanned Outages and Planned Maintenance:** Exadata significantly minimizes application impact during both unplanned outages (compute, network, storage) and planned infrastructure updates. It handles unplanned outages in low single-digit seconds and allows for planned maintenance and software updates with no application impact, often completely online. It also specifically manages "gray failures" (underperforming components) by removing and repairing them.
- **Data Protection:** Exadata provides enhanced data integrity and resilience against rare but highly impactful storage issues. It automatically detects and repairs corruption and handles multiple storage partner failures, all without application impact. Features like Hardware Assisted Resilient Data (HARD) checks and auto-detection/repair mechanisms are highlighted.
- **Quality of Service (QoS):** Exadata ensures applications maintain low latency and consistent performance, even during unplanned outages and planned maintenance. It achieves this by intelligently caching and prioritizing data movement at appropriate tiers, and by having features that detect sick components or outages, and quickly redirect I/O to

healthy partners. This includes automatic tuning of ASM rebalance, I/O latency capping, Smart Flash Log, and I/O Resource Manager (IORM).

- **Health Checks:** Exadata is always deployed with MAA best practices, but it also has many automated health check features to ensure the system continues to run properly, and that divergence from MAA best practices has not occurred.

Unplanned Outages and Planned Maintenance Features

Exadata is engineered to deliver exceptional database availability, both by mitigating the impact of unexpected failures and by enabling seamless, disruption-free planned maintenance.

For efficiency, this topic groups together Exadata features that contribute to high availability during both unexpected outages and scheduled maintenance, given their shared advantages. It explores the key features that ensure minimal downtime and continuous operation, highlighting how Exadata intelligently handles everything from component failures to routine system updates without affecting your critical applications.

Instant Failure Detection

Exadata rapidly detects failures in the database and storage servers using RDMA, which is much faster than traditional TCP timeouts. This immediate detection and node eviction prevent applications from "hanging" or experiencing disruptive blackouts.

RoCE Network Port Redundancy and Monitoring

Exadata's RoCE network ports are configured for active-active redundancy, ensuring continuous cluster and storage traffic, even if a port fails or is taken offline for maintenance. During multi-port maintenance, Exadata coordinates port activation across nodes to prevent application disruptions. Furthermore, ExaPortMon actively monitors port health and can automatically take down a "sick" port (one appearing online but not passing traffic) to redirect traffic to a healthy alternative.

Smart Flash and Hard Disk Replacement

After replacing a flash or hard disk, Exadata intelligently manages I/O by temporarily prioritizing reads from healthy partner storage cells. Simultaneously, it warms up the flash cache on the newly replaced storage. Once the cache is sufficiently populated, the system fully reintegrates the replaced drive, ensuring consistent and low I/O latency for applications throughout the process.

Smart Handshake for Storage Server Shutdown

During planned maintenance, such as a rolling software update on storage servers, Exadata synchronizes the shutdown process with the database servers. This "smart handshake" prevents any application impact and avoids the perception of an unplanned outage.

Do Not Service LED

To prevent accidental outages caused by human error, Exadata illuminates a "do not service" LED on storage servers when a partner storage component is offline. This clear visual indicator warns data center personnel not to perform maintenance on the illuminated server, safeguarding system availability.

Hot-Swappable Storage Components

All storage components within an Exadata system, including hard disks, M.2 boot drives, and Flash drives, can be replaced online without requiring a system shutdown or impacting ongoing operations.

Operating System Upgrades

Exadata simplifies major Linux operating system upgrades, offering a seamless, out-of-the-box experience with no application impact or downtime.

Database Node Live Update

Exadata System Software enables online updates for operating systems, firmware, and Exadata software on database nodes. A Live Update defers any pending work to a scheduled time, ensuring optimal and secure operation without interrupting database services.

Storage and RoCE Switch Rolling Software Updates

Software updates for storage servers and RoCE switches can be performed in a rolling fashion across the Exadata system. This allows updates to be applied sequentially without affecting the continuous availability of the Oracle Database.

Data Protection Features

Maintaining data integrity is paramount for any mission-critical database. This topic details Exadata's comprehensive suite of data protection features, designed to prevent, detect, and automatically repair data corruptions and I/O errors.

From hardware-level checks to intelligent software mechanisms, Exadata ensures your data remains safe, consistent, and continuously available, minimizing the risk of application downtime and safeguarding your most valuable asset.

Corruptions and I/O Errors

Exadata provides robust features to safeguard your data against corruption and I/O errors, which can otherwise compromise application and database availability. Data corruption occurs when a database block's content is unexpectedly altered, either physically (incomplete writes) or logically (lost writes). I/O errors typically arise from damaged or unavailable physical storage media. Exadata is uniquely equipped to reduce, detect, report, and even eliminate these issues, ensuring data integrity and continuous operation.

Hardware Assisted Resilient Data

Exadata incorporates HARD checks to prevent corruption in critical database files, including:

- SPFILE
- Control file
- Redo Log files
- Data files
- Data Guard Broker files

If a HARD check identifies corruption, the flawed data is prevented from being written. This feature is automatically active during ASM rebalance operations when the `DB_BLOCK_CHECKSUM` parameter is enabled.

Auto Detection and Repair

Exadata's intelligent design prevents and resolves data corruption seamlessly. If a network packet in the I/O path is corrupted during a write operation, the storage cell blocks the write, and ASM automatically retries the operation. This ensures that the application is never impacted by the corruption. Should a database update encounter existing corruption, the system automatically reads the correct data from an ASM mirror and repairs the corrupted

copy. All of these actions occur transparently, without impacting the database or user applications.

Physical Scrubbing

Exadata storage cells continuously perform "scrubbing" during idle periods to detect and automatically repair I/O errors on disks. To prevent performance impact, this scrubbing intelligently slows down if it detects that an application is actively accessing the disks being scrubbed.

Logical Scrubbing

Exadata offers tools for detecting logical corruptions:

- For Exadata ASM systems, the ASM scrub command verifies all copies of mirrored data.
- For Exadata Exascale systems, the xsh scrub command performs similar checks. Both commands identify and report any discrepancies found across storage mirrors.

Storage Server Disk Confinement

Beyond standard predictive failure analysis, Exadata constantly monitors disk performance and health. If a disk shows signs of poor performance (a potential precursor to failure), it is "confined," and I/O is automatically redirected to healthy mirrored copies. The storage server then runs a comprehensive health check:

- If the disk is healthy, it is returned to service, and data is resynchronized.
- If the disk is unhealthy, it is automatically taken offline, and data is rebalanced to maintain redundancy. The unhealthy disk can then be safely replaced. This feature not only protects data by proactively addressing problematic drives, but also acts as a Quality of Service enhancement by preventing applications from accessing underperforming storage.

Priority Rebalance

When a storage failure occurs, impacting data redundancy, ASM rebalance operations prioritize restoring redundancy over simply balancing data. Crucially, within redundancy restoration, ASM prioritizes the most critical files first, such as control files and online redo log files. This ensures that the most vital database components regain their redundancy as quickly as possible.

Quality of Service (QoS) Features

For mission-critical applications, maintaining consistent performance and low response times is crucial, even during unplanned outages, degraded component performance (gray failures), or planned maintenance. Exadata provides unique, value-added features that actively detect issues and intelligently redirect operations to healthy components, ensuring consistent application performance and minimizing impact.

Automatic ASM Rebalance Tuning

Exadata dynamically adjusts the speed of ASM (Automatic Storage Management) rebalance operations. This ensures that data redundancy is restored as quickly as possible during periods of low system activity, while automatically slowing down when the system is busy to prevent any negative impact on client I/O latency.

I/O Latency Capping

When any part of the storage system experiences high I/O latency, Exadata can intelligently redirect I/O requests to an alternative location for faster processing. Specifically, read I/Os are

redirected to healthy partner storage cells, and write I/Os are temporarily buffered to local flash storage on the same cell if a primary write path is slow, then written to persistent storage.

Smart Flash Log

Optimizing redo log write latency is vital for OLTP (Online Transaction Processing) database performance. Exadata's Smart Flash Log eliminates write latency outliers by simultaneously writing redo logs to both Flash Cache and a dedicated Flash Log device. The system then acknowledges the write as complete from the fastest device, ensuring maximum commit speed. Online and standby redo log writes are automatically and transparently cached in the write-back Smart Flash Cache, boosting log write throughput by leveraging flash instead of traditional disks.

Smart OLTP Caching

Exadata employs a multi-tiered caching strategy for frequently accessed data (hot data). When the Database Writer (DBWR) needs to clear space from the database buffer cache, the primary mirror of a data block is moved to the ultra-low-latency Data Accelerator (XR Memory), while the secondary mirror is stored in the Flash Cache, and the tertiary mirror remains on disk. If the storage server with the primary mirror fails, I/O is automatically redirected to the cell with the secondary mirror, and the data is read from its Flash Cache into the low-latency Data Accelerator. The tertiary mirror continues to provide data protection. When the failed server comes back online, its flash cache is quickly warmed up to restore optimal read performance.

I/O Resource Manager (IORM)

IORM is a unique Exadata feature that effectively manages and prioritizes storage server I/O resources during contention. It allows administrators to define IORM plans that tag and prioritize I/Os based on various criteria, including the database, Pluggable Database (PDB), Container Database (CDB), non-CDB, and Cluster name, as well as purpose (primary or standby) and priority. This is particularly valuable in consolidated environments and can be combined with Database Resource Manager for comprehensive workload management.

RoCE Network Lane Prioritization

The RDMA (Remote Direct Memory Access) network fabric in Exadata includes dedicated "high-speed" QoS lanes for critical I/O traffic, such as that from the Log Writer (LGWR) process. This ensures that transaction commit latency is never stalled by network congestion or other traffic, directly contributing to faster transaction response times.

Rebalance Caching Tier Preservation

During an ASM rebalance operation, Exadata ensures that data previously cached in high-performance storage tiers (such as XR Memory or Flash Cache) remains in those same tiers at the destination. This continuity guarantees that client applications continue to experience low I/O latency throughout the rebalance process.

ASM Partner Flash Read

When a disk or set of disks changes state (for example, a failed disk being replaced or disks going offline/online during a rolling software update), Exadata can intelligently fulfill client I/O requests from partner storage cells if doing so provides lower latency. Specifically, even after a replaced disk's flash cache is warmed up, Exadata will verify if the primary copy of data is in the local cell's flash cache. If not, it will retrieve the data from a partner cell's flash cache and populate the local cache. This is especially beneficial for applications whose active data set changes during storage events, ensuring consistent low latency.

RDMA Broadcast on Commit

Oracle Real Application Clusters (RAC) leverages RDMA on Exadata for its broadcast-on-commit mechanism. This provides extremely low-latency messaging, leading to faster transaction response times and improved application scalability.

Health Checks

Management Server

The Management Server (MS) runs on both Exadata storage servers and database compute nodes, offering extensive monitoring and alerting capabilities. It not only provides alerts for hardware failures (for example, disk failures) but also for operating system resource oversubscription, such as excessive CPU usage, memory consumption, or swapping, enabling proactive issue resolution.

For more information, see [Understanding Automated Storage Server Maintenance Tasks and Policies](#) in *Oracle Exadata Database Machine User's Guide*.

Exadata AWR Report

The Exadata section of the AWR report contains data about the expected configuration and performance of the Exadata system, and reports misconfiguration and resource oversubscription if it exists.

For more information, see

- [Automatic Workload Repository \(AWR\)](#) in the Exadata System Software User's Guide
- [Generating Automatic Workload Repository Reports](#) in *Oracle AI Database Performance Tuning Guide*

Real-Time Insight

Real-Time Insight provides the infrastructure to automatically stream up-to-the-second metric observations from all of the servers in your Exadata fleet, and feeds the metrics into customizable monitoring dashboards for real-time analysis and problem-solving.

For more information, see [Using Real-Time Insight](#) in the Exadata System Software User's Guide.

Exachk

Exachk is a proactive tool that you can use to identify and address any deviations from Oracle MAA best practices on your Exadata systems. Resolving these identified gaps enhances system stability and contributes to higher availability for both unplanned outages and planned maintenance.

For more information, see the [ORAchk and EXAchk User's Guide](#).

Application Impact with Exadata and MAA Best Practices

The following table summarizes Exadata's differentiating features, described above, into higher-level categories across the three tiers of Exadata.

Comprehensive Exadata MAA Evaluations

The MAA team regularly evaluates high availability use cases running typical customer workloads in properly sized Exadata environments configured with MAA best practices. The evaluations are comprehensive, focusing not just on functional success but on the overall user

experience, and the primary focal point is ensuring that application service levels are not impacted.

One of the customer facing outputs from the MAA team is the "Exadata Extreme Availability Matrix" that extensively details HA use cases, Exadata MAA features supporting them, and application impact expectations. See [MOS Doc ID 3103956.1](#) for the latest Exadata Extreme Availability Matrix.

Key HA Use Cases

The following are some details about key HA use cases from the Exadata Extreme Availability Matrix.

Table 13-1 Planned Maintenance HA Use Cases

HA Use Case	Application Impact
Database Node Rolling Software Update (Exadata, Grid Infrastructure, and Database)	No application delay
Storage Server Rolling Software Update	No application delay
RDMA Network Fabric Switch Rolling Software Update	10 seconds or less of application delay

Table 13-2 Unplanned Outages HA Use Cases

HA Use Case	Application Impact
Hard Disk and Flash Failure	No application delay
Database Node and Instance Failure	10 seconds or less of application delay
RDMA Network Fabric Switch or Port Failure	10 seconds or less of application delay

Gray Failures

A gray failure is a system malfunction that manifests as subtle, partial, or intermittent issues, rather than a complete "hard" failure where the system stops working entirely. Gray failures can severely degrade application service levels, in some cases causing full service level stoppage, and they are very difficult hard to detect and repair quickly.

Some examples of gray failures are resource over-subscription, slow/hung instances, slow/hung database nodes, slow/hung storage, and slow/hung network. Exadata, configured with MAA best practices, provides additional protection against gray failures. More details can be found in the Exadata Extreme Availability Matrix. The examples below highlight how Exadata helps maintain high performance, stability and availability for your applications and databases.

Gray Failure HA Use Case	Exadata Feature and Handling
Oversubscribed resources	Exadata leverages MS alerting, Exadata AWR support, and Real Time Insight to proactively notify and report on resource oversubscription so it can be addressed.
RoCE network port dropped packets	Exadata leverages the Exaportmon feature to detect sick network ports and take them down if traffic is not flowing correctly
Exadata Storage Cell slow/sick flash disk and hard disk	Exadata leverages the cell side IO latency capping features to cancel IO and route to secondary partners when primary partners are very slow
I/O hang	Exadata leverages the I/O hang detection to reboot a cell when the I/Os are not being serviced due to, for example, a sick controller

Part IV

Oracle Data Guard Best Practices

- [Overview of MAA Best Practices for Oracle Data Guard](#)
- [Plan an Oracle Data Guard Deployment](#)
- [Configure and Deploy Oracle Data Guard](#)
- [Tune and Troubleshoot Oracle Data Guard](#)
- [Monitor an Oracle Data Guard Configuration](#)
- [Optimizing Automatic Failover in Common Scenarios to Minimize Downtime](#)

Overview of MAA Best Practices for Oracle Data Guard

By adding a physical standby database with Oracle Active Data Guard, a Silver MAA reference architecture is elevated to a Gold MAA reference architecture. Implement Oracle Data Guard best practices to achieve minimal downtime and potentially zero data loss for all unplanned outages.

Oracle Active Data Guard plays an important role in delivering the high availability and comprehensive data protection that you expect of the Gold MAA reference architecture. The Gold reference architecture, consisting of an Oracle RAC primary database and Oracle RAC standby systems with Oracle Active Data Guard, plus MAA configuration and life cycle operations, provides a comprehensive set of services that create, maintain, manage, and monitor one or more standby databases. Oracle Active Data Guard protects your data during all types of planned maintenance activities, such as software updates and major database upgrades, and unplanned outages, including database failures, site outages, natural disasters, and data corruptions.

The goal of Oracle Data Guard best practices is to help you implement tested and proven MAA best practices to ensure a successful and stable Data Guard deployment. The following steps connect you to the Oracle MAA best practices for planning, implementing, and maintaining this type of architecture.

1. Plan your Oracle Data Guard architecture, and take into account various considerations for the application, network, and so on.
2. Configure and Deploy Data Guard using Oracle MAA best practices.
3. Tune and Troubleshoot your Data Guard deployment.

To learn more about the Gold MAA reference architecture, see [High Availability Reference Architectures](#).

Plan an Oracle Data Guard Deployment

Analyze your specific requirements, including both the technical and operational aspects of your IT systems and business processes, understand the availability impact for the Oracle Data Guard architecture options, and consider the impact of your application and network.

Oracle Data Guard Architectures

The Gold MAA reference architecture provides you with four architecture patterns, using Oracle Active Data Guard to eliminate single point of failure. The patterns vary from a single remote active standby with Fast Start Failover and HA Observer, to including far sync instances, multiple standbys, and reader farms.

When planning your Gold MAA Reference Architecture, see [High Availability Reference Architectures](#) for an overview of each Gold architecture pattern, and choose the elements to incorporate based on your requirements.

Application Considerations for Oracle Data Guard Deployments

As part of planning your Oracle Data Guard deployment, consider the resources required and application availability requirements in a fail over scenario.

Deciding Between Full Site Failover or Seamless Connection Failover

The first step is to evaluate which failover option best meets your business and application requirements when your primary database or primary site is inaccessible or lost due to a disaster.

The following table describes various conditions for each outage type and recommends a failover option in each scenario.

Table 15-1 Recommended Failover Options for Different Outage Scenarios

Outage Type	Condition	Recommended Failover Option
Primary Site Failure (including all application servers)	Primary site contains all existing application servers (or mid-tier servers) that were connected to the failed primary database.	Full site failover is required

Table 15-1 (Cont.) Recommended Failover Options for Different Outage Scenarios

Outage Type	Condition	Recommended Failover Option
Primary Site Failure (with some application servers surviving)	Some or all application servers are not impacted and the surviving application servers can reconnect to new primary database in a secondary disaster recovery site. Application performance and throughput is still acceptable with different network latency between application servers and new primary database in a secondary disaster recovery site. Typically analytical or reporting applications can tolerate higher network latency between client and database without any noticeable performance impact, while OLTP applications performance may suffer more significantly if there is an increase in network latency between the application server and database.	Seamless connection failover is recommended to minimize downtime and enable automatic application and database failover.
Complete Primary Database or Primary Server Failure	Application servers are not impacted and users can reconnect to new primary database in a secondary disaster recovery site. Application performance and throughput is still acceptable with different network latency between application servers and new primary database in a secondary disaster recovery site. Typically analytical or reporting applications can tolerate higher network latency between client and database without any noticeable performance impact, while OLTP applications performance may suffer more significantly if there is an increase in network latency between the application server and database.	If performance is acceptable, seamless connection failover is recommended to minimize downtime and enable automatic application and database failover. Otherwise, full site failover is required.

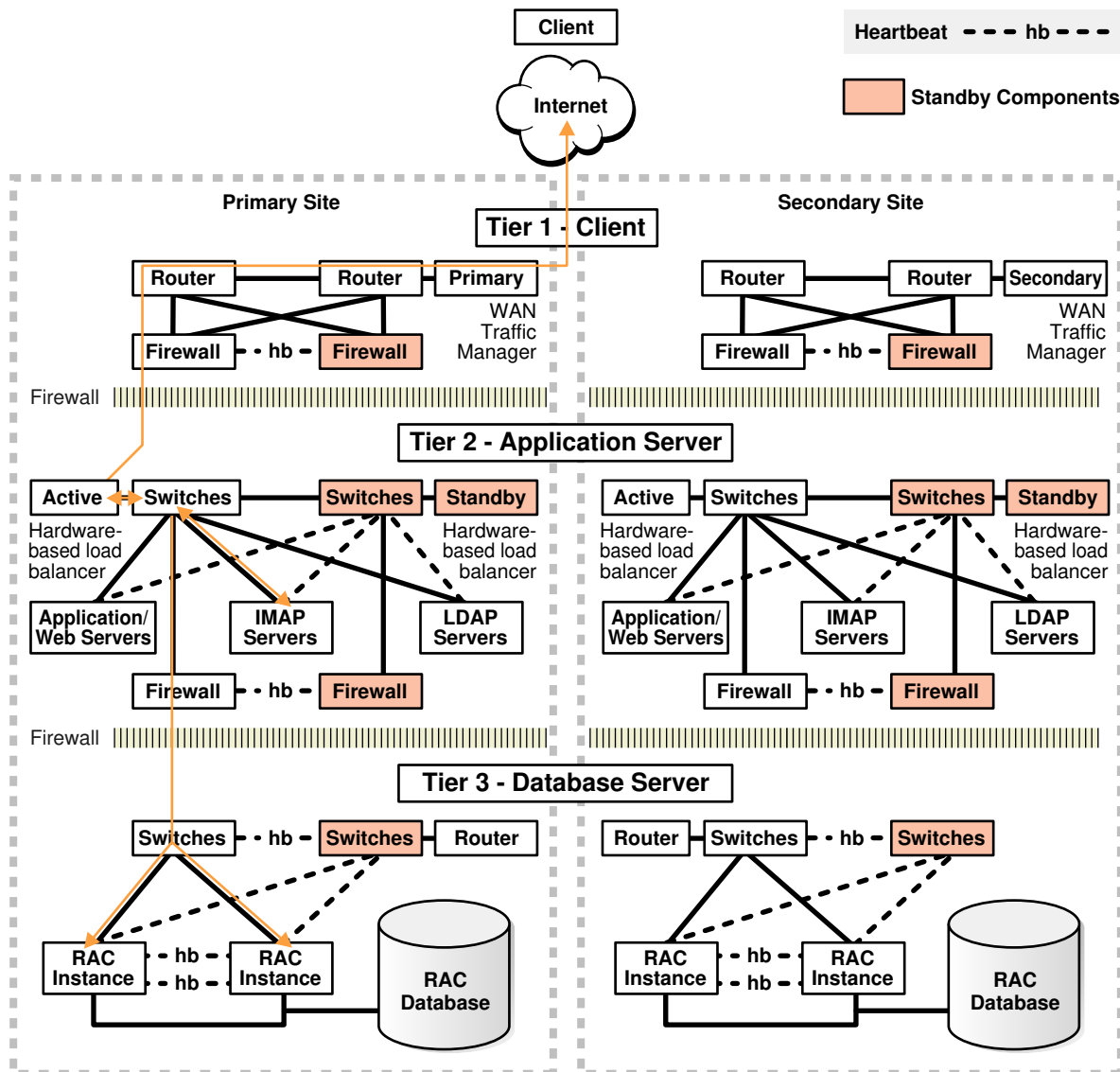
Full Site Failover Best Practices

A **full site failover** means that the complete site fails over to another site with a new set of application tiers and a new primary database.

Complete site failure results in both the application and database tiers becoming unavailable. To maintain availability, application users must be redirected to a secondary site that hosts a redundant application tier and a synchronized copy of the production database.

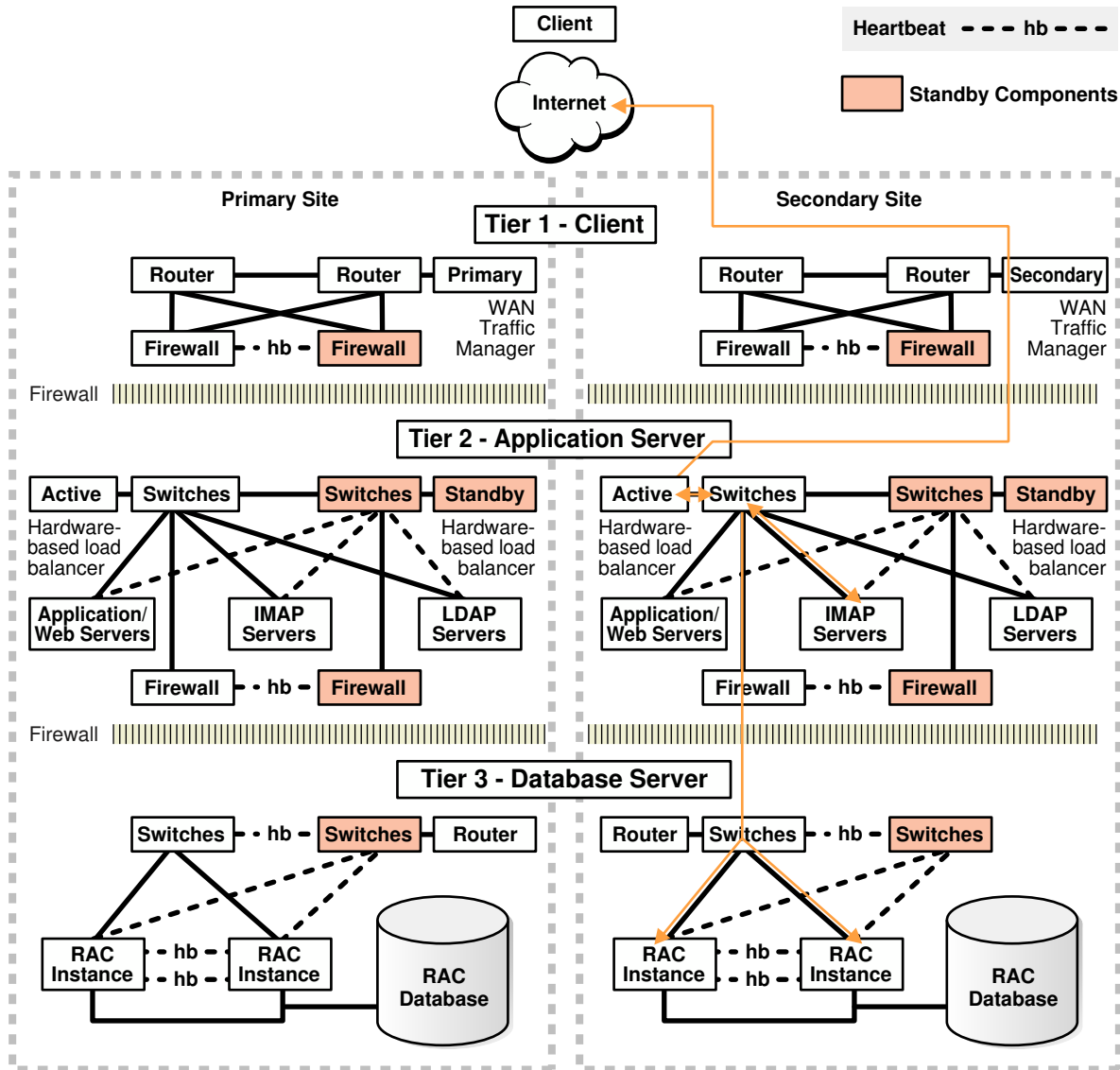
Consider the two figures below. The first figure shows the network routes before failover. Client or application requests enter the Primary site at the client tier, and are routed to the application server and database server tiers on the primary site.

Figure 15-1 Network Routes Before Site Failover



The second figure, below, illustrates the network routes after a complete site failover. Client or application requests enter the Secondary site at the client tier and follow the same path on the secondary site that they followed on the primary site.

Figure 15-2 Network Routes After Site Failover



MAA best practice is to maintain a running application tier at the standby site to avoid incurring start-up time, and to use Oracle Data Guard to maintain a synchronized copy of the production database. Upon site failure, a WAN traffic manager is used to perform a DNS failover (either manually or automatically) to redirect all users to the application tier at standby site while a Data Guard failover transitions the standby database to the primary production role.

Use Oracle Active Data Guard Fast-Start Failover to automate the database failover. Application server and non-database failovers can be automated and coordinated by using Oracle Site Guard. Oracle Site Guard orchestrates and automates any operations, such as starting up application servers on the secondary site, resynchronizing non-database meta data as Data Guard fails over automatically.

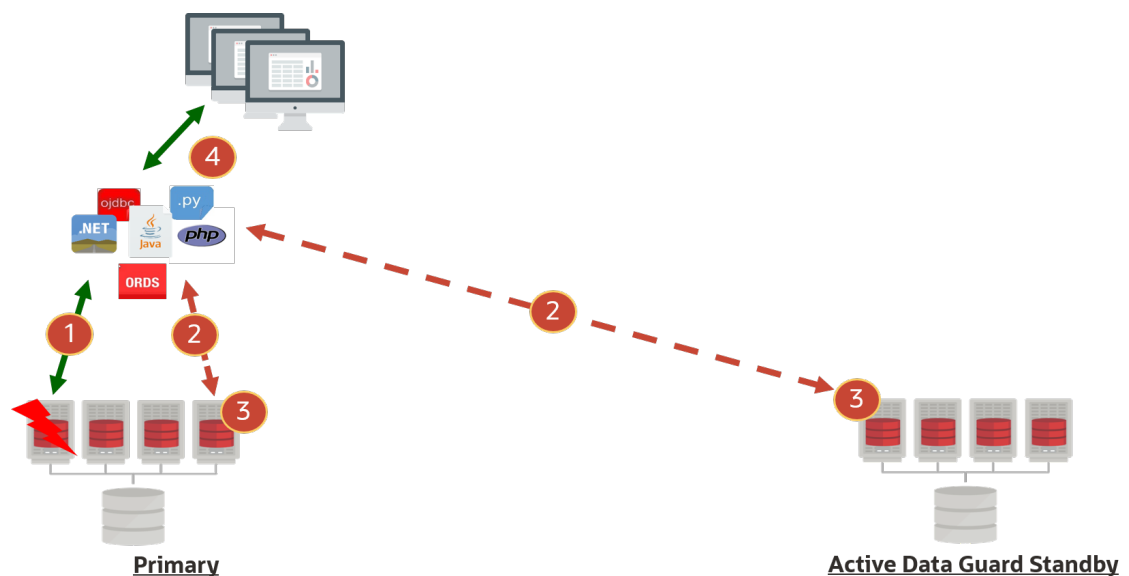
For more information about Oracle Site Guard, see the Oracle Site Guard Administrator's Guide.

Configuring Seamless Connection Failover

Automating seamless client failover in an Oracle Data Guard configuration includes relocating database services to the new primary database as part of a Data Guard failover, notifying clients that a failure has occurred to break them out of TCP timeout, and redirecting clients to the new primary database.

In the following figure, a database request is interrupted by an outage or timeout (1), so the session reconnects to the Oracle RAC cluster (2) (or standby) (2), the database request replays automatically on the alternate node (3), and the result from the database request is returned to the user (4).

Figure 15-3 Seamless Connection Failover



To achieve seamless connection failover, refer to [Configuring Continuous Availability for Applications](#).

Assessing and Optimizing Network Performance

Oracle Data Guard relies on the underlying network to send redo from the primary database to standby databases. Ensuring that the network is healthy and capable of supporting peak redo generation rates helps avoid future transport lags.

A transport lag forms when the primary database cannot ship redo to the standby faster than primary instance's redo generation rate. A transport lag can lead to potential data loss if a primary database failure occurs.

Network assessment consists of evaluating

- Network reliability
- Network bandwidth to accommodate peak redo generation rates

Note

Each instance of the primary database instance generates its own redo and ships redo to the standby database in a single network stream. Therefore, maximizing single process network throughput for each node is critical for redo transport.

Historically there are areas that can reduce network and redo transport throughput resulting in potential transport lags:

1. Network firewalls or network encryption

Network firewalls and network (not Oracle Net) encryption can reduce overall throughput significantly. Verify throughput with the `oratcp` tool (described below), with and without encryption, and tune accordingly.

At times reducing the encryption level can increase throughput significantly. A balance is required to meet security needs with your performance and data loss requirements.

2. Redo transport compression

When database initialization parameter has `LOG_ARCHIVE_DEST_N` attribute `COMPRESSION=ENABLE`, Oracle background processes have to compress the redo before sending network message, and uncompress the redo before processing the redo. This reduces the overall redo and network throughput. Compression is only recommended if network bandwidth is insufficient between the primary and standby destinations.

3. Oracle Net encryption

Depending on the Oracle Net encryption level, this will have varying redo throughput impact, because Oracle Net messages containing redo have to be encrypted before sending and then unencrypted before redo processing.

Note that if database encryption is already enabled with Transparent Data Encryption (TDE), redo is already encrypted, although Oracle Net encryption can also encrypt the message headers.

4. Untuned network for redo transport

- Increasing maximum operating system socket buffer size can increase single process throughput by 2-8 times. Test with different socket buffer sizes to see what value yields positive results, and ensure throughput is greater than the peak redo throughput.
- Compare performance with various MTU settings.

If average redo write size is less than 1500 bytes, then try various MTU settings including MTU=9000 (for example, Jumbo Frames) for network interface that sends or receives redo on your system. This may reduce some unnecessary network round trips which will increase overall throughput.

Also note that for SYNC transport, Oracle's average redo write size (for example, Oracle message send) increases significantly as determined by `v$sysstats` or AWR statistics "redo size / redo writes".

When sending redo across geographical regions, experiments have shown that using MTU=9000 can also benefit in some network topologies. Conduct performance tests with `oratcp` and compare the results with default MTU and MTU=9000 settings.

Gather Topology Information

Understanding the topology of the Oracle Data Guard configuration, and its relevance to Data Guard performance, helps eliminate infrastructure weaknesses that are often incorrectly attributed to the Data Guard architecture.

Oracle recommends that you outline the following high-level architecture information.

- Describe the primary and standby database system (number of nodes in Oracle RAC cluster, CPUs and memory per database node, storage I/O system)
- Describe network topology connecting the primary and standby systems
 - Network switches and firewalls in between primary and standby
 - Network bandwidth and latency

For standby databases with symmetric hardware and configuration, and with a well tuned network configuration that can support peak redo generation rates, the transport lag should be less than 1 second.

Understanding Network Usage of Data Guard

The phases of the Data Guard life cycle which use the network most heavily are:

- Instantiation - During this phase of standby database creation, files can be copied using parallelism from any host. Determining the degree of parallelism which maximizes throughput between nodes helps to optimize the standby instantiation process.
- Redo Transport (Steady State)- Under normal Data Guard operation the primary database ships redo to the standby which is then applied. Each RAC instance of a primary database ships redo in a single stream from the host on which it is running. Understanding the requirements of each primary database instance and ensuring a single process can achieve the throughput requirements is critical to a standby database staying current with the primary database.

Understanding Targets and Goals for Instantiation

Instantiation for large databases can take hours, or in extreme cases days. To allow for planning of an instantiation and also maximize throughput between the primary and standby system to complete the instantiation in as timely a manner as possible, first determine the goal for instantiation time. Then follow the process defined below to maximize per process throughput and identify the optimal degree of parallelism between the primary and standby nodes.

Understanding Throughput Requirements and Average Redo Write Size for Redo Transport

Required network bandwidth of a given Data Guard configuration is determined by the redo generate rate of the primary database.

Note

In cases where the primary database is pre-existing, a baseline for the required network bandwidth can be established. If there is no existing primary database, skip this step and future references to the data further in the process.

While the Automatic Workload Repository (AWR) tool can be used to determine the redo generation rate, the snapshots are often 30 or 60 minutes apart which can dilute the peak rate. Since peak rates often occur for shorter periods of time, it is more accurate to use the following query which calculates the redo generation rate for each log when run on an existing database. (change the timestamps as appropriate)

```
SQL> SELECT THREAD#, SEQUENCE#, BLOCKS*BLOCK_SIZE/1024/1024 MB,
(NEXT_TIME-FIRST_TIME)*86400 SEC,
(BLOCKS*BLOCK_SIZE/1024/1024)/((NEXT_TIME-FIRST_TIME)*86400) "MB/S"
FROM V$ARCHIVED_LOG
WHERE ((NEXT_TIME-FIRST_TIME)*86400<>0)
AND FIRST_TIME BETWEEN TO_DATE('2022/01/15 08:00:00','YYYY/MM/DD HH24:MI:SS')
AND TO_DATE('2022/01/15 11:00:00','YYYY/MM/DD HH24:MI:SS')
AND DEST_ID=1 ORDER BY FIRST_TIME;
```

Example output:

THREAD#	SEQUENCE#	MB	SEC	MB/s
2	2291	29366.1963	831	35.338383
1	2565	29365.6553	781	37.6000708
2	2292	29359.3403	537	54.672887
1	2566	29407.8296	813	36.1719921
2	2293	29389.7012	678	43.3476418
2	2294	29325.2217	1236	23.7259075
1	2567	11407.3379	2658	4.29169973
2	2295	24682.4648	477	51.7452093
2	2296	29359.4458	954	30.7751004
2	2297	29311.3638	586	50.0193921
1	2568	3867.44092	5510	.701894903

Note

To find the peak redo rate, choose times during the highest level of processing, such as peak OLTP periods, End of Quarter batch processing or End of Year batch processing.

In this short example the highest rate was about 52MB/s. Ideally the network will support the maximum rate plus 30% or 68MB/s for this application.

Verify Average Redo Write Size

Using v\$sysstats or looking at your AWR reports for various workload and peak intervals, record the average redo write size based on

Average Redo Write Size = "REDO SIZE" / "REDO WRITES"

Use this average redo write size in your oratcp experiments. If the average redo write size > 1500 bytes, experiment with various MTU settings.

Understand Current Network Throughput

The Oracle utility oratcptest is a general-purpose tool for measuring network bandwidth and latency similar to iperf/qperf which can be run by any OS user.

The oratcptest utility provides options for controlling the network load such as:

- Network message size
- Delay time between messages
- Parallel streams
- Whether or not the oratcptest server should write messages on disk.
- Simulating Data Guard SYNC transport by waiting for acknowledgment (ACK) of a packet or ASYNC transport by not waiting for the ACK.

Note

This tool, like any Oracle network streaming transport, can simulate efficient network packet transfers from the source host to target host similar to Data Guard transport. Throughput can saturate the available network bandwidth between source and target servers. Therefore, Oracle recommends that short duration tests are performed and that consideration is given for any other critical applications sharing the same network.

Measure the Existing Throughput of One and Many Processes

Do the following tasks to measure the existing throughput.

Task 1: Install oratcptest

1. Download the oratcptest.jar file from MOS note 2064368.1
2. Copy the JAR file onto both client (primary) and server (standby)

Note

oratcptest requires JRE 6 or later

3. Verify that the host has JRE 6 or later
4. On all primary and standby hosts, verify that the JVM can run the JAR file by displaying the help

```
# java -jar oratcptest.jar -help
```

Task 2: Determine the Existing Throughput for a Single Process

Data Guard asynchronous redo transport (ASYNC) uses a streaming protocol which does not wait for packet acknowledgment and therefore achieves higher rates than SYNC transport.

1. Start the test server on the receiving (standby) side.

```
java -jar oratcptest.jar -server [IP of standby host or VIP in RAC
configurations] -port=<any available port number>
```

2. Run the test client. (Change the server address and port number to match that of your server started in step 4.)

```
$ java -jar oratcptest.jar [IP of standby host or VIP in RAC
configurations]
-port=<port number> -mode=async -duration=120 -interval=20s
```

```
[Requesting a test]
    Message payload          = 1 Mbyte
    Payload content type    = RANDOM
    Delay between messages  = NO
    Number of connections   = 1
    Socket send buffer      = (system default)
    Transport mode          = ASYNC
    Disk write              = NO
    Statistics interval     = 20 seconds
    Test duration           = 2 minutes
    Test frequency          = NO
    Network Timeout         = NO
    (1 Mbyte = 1024x1024 bytes)
```

```
(17:54:44) The server is ready.
           Throughput
(17:55:04)    20.919 Mbytes/s
(17:55:24)    12.883 Mbytes/s
(17:55:44)    10.457 Mbytes/s
(17:56:04)    10.408 Mbytes/s
(17:56:24)    12.423 Mbytes/s
(17:56:44)    13.701 Mbytes/s
(17:56:44) Test finished.
           Socket send buffer = 2 Mbytes
           Avg. throughput = 13.466 Mbytes/s
```

In this example the average throughput between these two nodes was about 13 MB/s which does not meet the requirements of 68 MB/s from the query.

Note

This process can be scheduled to run at a given frequency using the `-freq` option to determine if the bandwidth varies at different times of the day. For instance setting `-freq=1h/24h` will repeat the test every hour for 24 hours.

Task 3: Determine Existing Throughput for Multiple Processes

1. Repeat the previous test with two (2) connections (using `num_conn` parameter).

```
$ java -jar oratcptest.jar <target IP address> -port=<port number>
-duration=60s -interval=10s -mode=async [-output=<results file>] -
num_conn=2
```

```
[Requesting a test]
    Message payload      = 1 Mbyte
    Payload content type = RANDOM
    Delay between messages = NO
    Number of connections = 2
    Socket send buffer   = (system default)
    Transport mode       = ASYNC
    Disk write           = NO
    Statistics interval  = 20 seconds
    Test duration        = 2 minutes
    Test frequency       = NO
    Network Timeout      = NO
    (1 Mbyte = 1024x1024 bytes)
```

```
(18:08:02) The server is ready.
           Throughput
(18:08:22) 44.894 Mbytes/s
(18:08:42) 23.949 Mbytes/s
(18:09:02) 25.206 Mbytes/s
(18:09:22) 23.051 Mbytes/s
(18:09:42) 24.978 Mbytes/s
(18:10:02) 22.647 Mbytes/s
(18:10:02) Test finished.
           Avg. socket send buffer = 2097152
           Avg. aggregate throughput = 27.454 Mbytes/s
```

2. Re-run step 1 iteratively and increase the value of num_conn by two each time until the aggregate throughput does not increase for three consecutive values. For example if the aggregate throughput is approximately the same for 10, 12 and 14 connections, stop.

Note

RMAN can utilize all nodes in the cluster for instantiation. To find the total aggregate throughput, see My Oracle Support [Creating a Physical Standby database using RMAN restore database from service \(Doc ID 2283978.1\)](#).

3. Run the same test with all nodes in all clusters to find the current total aggregate throughput. Node 1 of primary to node 1 of standby, node 2 to node 2, etc. Sum the throughput found for all nodes.
4. Reverse the roles and repeat the tests.
5. Note the number of connections which achieved the best aggregate throughput.

Use the total size of the database and total aggregate throughput to estimate the amount of time it will take to complete the copy of the database. A full instantiation also needs to apply the redo generated during the copy. Some additional percentage (0%-50%) should be added to this estimated time based on how active the database is.

If the estimated time meets the goal, no additional tuning is required for instantiation.

Optimizing Redo Transport with One and Many Processes

If throughput from the prior single and multiple process tests meet the targets, no additional tuning is required. If higher throughput is required, setting the maximum TCP socket buffers size to a larger value is the primary method to potentially increase throughput.

Setting TCP Socket Buffer Size

The TCP socket buffers are system memory buffers which temporarily store incoming and outgoing data. Outgoing data is stored on the write buffers while incoming data is stored on the read buffers. Read and write socket buffers are allocated separately. When a buffer, generally the read buffer, fills up (often do to the application not pulling data out of the buffer fast enough), a message is sent to the sender to slow down or stop sending data. Allocating a larger buffer often improves redo transport by giving the application time to pull data off the wire without stopping the sender.

Tuning TCP socket buffer size is the primary approach to improving ASYNC transport and can improve SYNC transport as well in some cases.

Note

With larger socket buffer sizes, TCP selective acknowledgment (SACK) is strongly recommended. Often times this is enabled by default but refer to your operating system documentation for details on confirming or enabling TCP selective acknowledgment.

To set TCP Socket Buffer Size do the following tasks.

Task 1: Determine Optimal Maximum Socket Buffer Size

Find the optimal maximum socket buffer size for a single process on the target network link by running a series of tests.

Note

Bandwidth Delay Product is the product of the network link capacity of a channel and the round time, or latency. The minimum recommended value for socket buffer sizes is $3 \times \text{BDP}$, especially for a high-latency, high-bandwidth network. Use `oratcpctest` to tune the socket buffer sizes.

Task 2: Set Maximum Socket Buffer Size Temporarily

On the primary and standby systems follow these steps to set the maximum socket buffer size for requests. This will be done in memory and will not persist if the server is restarted for any reason.

Do the following steps as root.

1. First find the current size of the kernel parameters `net.ipv4.tcp_rmem` and `net.ipv4.tcp_wmem`. The values returned are the minimum, default and maximum size for socket buffers which TCP dynamically allocates. If a process requires more than the

default given when a socket is created, more buffers will be dynamically allocated up to the maximum value.

```
# cat /proc/sys/net/ipv4/tcp_rmem
4096      87380    6291456
```

```
# cat /proc/sys/net/ipv4/tcp_wmem
4096      16384    4194304
```

2. Change the values to 16MB or whatever $3 \times \text{BDP}$ was calculated to be

```
# sysctl -w net.ipv4.tcp_rmem='4096 87380 16777216';
```

```
# sysctl -w net.ipv4.tcp_wmem='4096 16384 16777216';
```

Note

Increasing these values can increase system memory usage of any network socket on the system.

Note

Changes made with `sysctl` are not permanent. Update the `/etc/sysctl.conf` file to persist these changes through machine restarts. There will be a step to change the configuration file at the end of this process once the proper setting is determined.

Task 3: Test Throughput of a Single Process

Re-run the previous tests allowing the socket buffers to dynamically grow to the new maximum set in the previous step

(as oracle)

Server (standby):

```
$ java -jar oratcptest.jar -server [IP of standby host or VIP in RAC
configurations]
-port=<port number>
```

Client (primary):

```
$ java -jar oratcptest.jar <IP of standby host or VIP in RAC configurations>
-port=<port number> -mode=async -duration=120s -interval=20s
```

Note

Do not use the `oratcptest sockbuf` parameter because the kernel parameters which govern explicit requests for socket buffer size are different than those set for this test.

After the test completes the results from the client and server show the value for socket buffers during that test. At the time of this writing, that value is half of the actual socket buffer size and should be doubled to find the actual size used.

Client

```
[Requesting a test]
  Message payload = 1 Mbyte
  Payload content type = RANDOM
  Delay between messages = NO
  Number of connections = 1
  Socket send buffer = 2 Mbytes
  Transport mode = ASYNC
  Disk write = NO
  Statistics interval = 20 seconds
  Test duration = 2 minutes
  Test frequency = NO
  Network Timeout = NO
  (1 Mbyte = 1024x1024 bytes)
(11:39:16) The server is ready.
      Throughput
(11:39:36) 71.322 Mbytes/s
(11:39:56) 71.376 Mbytes/s
(11:40:16) 72.104 Mbytes/s
(11:40:36) 79.332 Mbytes/s
(11:40:56) 76.426 Mbytes/s
(11:41:16) 68.713 Mbytes/s
(11:41:16) Test finished.

      Socket send buffer = 8388608
      Avg. throughput = 73.209 Mbytes/s
```

Server

The test terminated. The socket receive buffer was 8 Mbytes.

Note

oratcptest is reporting half of the buffers allocated to the socket. Double the number reported for the actual socket buffer size used during the test.

Task 4: Test Throughput of Multiple Processes

Now repeat the test using the num_conn value determined in the first tests. for example,. if the peak aggregate throughput was reached with 10 processes set num_conn=10.

Client

```
$ java -jar oratcptest.jar <IP of standby host or VIP in RAC configurations>
  -port=<port number> -mode=async -duration=120s -interval=20s -num_conn=10
```

```
[Requesting a test]
  Message payload      = 1 Mbyte
  Payload content type = RANDOM
```



```
Delay between messages = NO
Number of connections   = 10
Socket send buffer      = (system default)
Transport mode          = ASYNC
Disk write              = NO
Statistics interval     = 20 seconds
Test duration           = 2 minutes
Test frequency          = NO
Network Timeout         = NO
(1 Mbyte = 1024x1024 bytes)
```

```
(19:01:38) The server is ready.
              Throughput
(19:01:58)    266.077 Mbytes/s
(19:02:18)    242.035 Mbytes/s
(19:02:38)    179.574 Mbytes/s
(19:02:58)    189.578 Mbytes/s
(19:03:18)    218.856 Mbytes/s
(19:03:38)    209.130 Mbytes/s
(19:03:38) Test finished.
              Avg. socket send buffer = 8 Mbytes
              Avg. aggregate throughput = 217.537 Mbytes/s
```

Note

oracptest is reporting half of the buffers allocated to the socket. Double the number reported for the actual socket buffer size used during the test.

Server (Each connection will have the receive buffer printed. Double the socket buffer size in each instance)

```
The test terminated. The socket receive buffer was 8 Mbytes.

The test terminated. The socket receive buffer was 8 Mbytes.

The test terminated. The socket receive buffer was 8 Mbytes.

The test terminated. The socket receive buffer was 8 Mbytes.

The test terminated. The socket receive buffer was 8 Mbytes.

The test terminated. The socket receive buffer was 8 Mbytes.

The test terminated. The socket receive buffer was 8 Mbytes.

The test terminated. The socket receive buffer was 8 Mbytes.

The test terminated. The socket receive buffer was 8 Mbytes.

The test terminated. The socket receive buffer was 8 Mbytes.
```

Use the total size of the database and total aggregate throughput to estimate the amount of time it will take to complete the copy of the database. A full instantiation also needs to apply

the redo generated during the copy. Some additional percentage (0%-50%) should be added to this estimated time based on how active the database is.

Task 5: Repeat the Tests

Repeat the previous two tests with higher values for `tcp_rmem` and `tcp_wmem` if more throughput is needed. Understand that these higher values are available for other sockets as well but will be dynamically allocated only if needed. The table shows sample data tracking the different throughput results for different socket buffer sizes.

<code>tcp_rmem</code> maximum	<code>tcp_wmem</code> maximum	Single Process Throughput	Single Node Multi-Process Maximum Aggregate Throughput	Single Node Multi-Process Parallelism
6291456	4194304	13.5 MB/s	203 MB/s	16
8388608	8388608	48 MB/s	523 MB/s	14
16777216	16777216	73 MB/s	700 MB/s	14
33554432	33554432	132 MB/s	823 MB/s	14

Task 6: Set Parameters Permanently

Changes using `sysctl` modify the values in memory which do not persist through a reboot of the host. Once the optimal size for socket buffers is determined, set the kernel parameters so they persist through server restarts by editing the `/etc/sysctl.conf` file.

This must be done on all nodes of the primary and standby systems.

To make these changes persistent, edit the `/etc/sysctl.conf` either modifying the existing values or adding these values to the file if they are absent.

```
net.ipv4.tcp_rmem='4096 87380 16777216'
```

```
net.ipv4.tcp_wmem='4096 16384 16777216'
```

Task 7: Evaluate Larger MTU

Determine the network interfaces that are used by the Data Guard transport.

If Average Redo Write Size > current MTU setting (for example, typically the default 1500), evaluate if jumbo frames (for example, MTU=9000) can reduce the network RTT for these large network packets and improve overall redo throughput.

Shown here is an example of changing the MTU for Data Guard transport network interface for testing purposes on Linux.

```
ifconfig bondeth0 mtu 9000 up
```

Repeat the same `oratcp` performance methodology as described above with the higher MTU size to see if greater throughput is achieved.

If performance gains are noticed, work with system and network engineers to change MTU size for DG transport for both primary and standby databases.

Using This Data

The throughput numbers can be used to determine throughput to aid in Redo Transport and Instantiation situations.

Redo Transport

If the single process throughput does not exceed the single instance redo generation rate for a primary database, the standby will not stay current with the primary during these times. Further evaluation and network tuning by the network engineering team may be required in these cases.

Instantiation

Once the maximum aggregate throughput of all nodes is understood, a rough estimate for instantiation can be developed. As an example, if there is a 100 TB database on a 2-node RAC to be instantiated and each node can achieve 300 MB/s it should take about 50 hours to copy the data files. Additional work to instantiate will add some percentage to that number (~30%).

$300 \text{ MB/s} * 60 \text{ seconds/minute} * 60 \text{ minutes/hour} * 2 \text{ nodes} = \sim 2 \text{ TB/hr aggregate for both nodes}$
 $100\text{TB} / 2\text{TB/hr} = \sim 50 \text{ hours}$

The steps to instantiate a database using large database optimizations such as using multiple nodes is described in [Creating a Physical Standby database using RMAN restore database from service \(Doc ID 2283978.1\)](#).

Determining Oracle Data Guard Protection Mode

Oracle Data Guard can run in three different protection modes, which cater to different performance, availability, and data loss requirements. Use this guide to determine which protection mode fits your business requirements and your potential environmental constraints.

Maximum Protection mode guarantees that no data loss will occur if the primary database fails, even in the case of multiple failures (for example, the network between the primary and standby fails, and then at a later time, the primary fails). This policy is enforced by never signaling commit success for a primary database transaction until at least one synchronous Data Guard standby has acknowledged that redo has been hardened to disk. Without such an acknowledgment the primary database will stall and eventually shut down rather than allow unprotected transactions to commit.

To maintain availability in cases where the primary database is operational but the standby database is not, the best practice is to always have a minimum of **two synchronous standby databases** in a Maximum Protection configuration. Primary database availability is not impacted if it receives acknowledgment from at least one synchronous standby database.

Choose this protection mode if zero data loss is more important than database availability. Workload impact analysis is recommended to measure whether any overhead is acceptable when enabling SYNC transport.

Maximum Availability mode guarantees that no data loss will occur in cases where the primary database experiences the first failure to impact the configuration. Unlike the Maximum Protection mode, Maximum Availability will wait a maximum of NET_TIMEOUT seconds for an acknowledgment from any of the standby databases, after which it will signal commit success to the application and move to the next transaction. Primary database availability (thus the name of the mode) is not impacted by an inability to communicate with the standby (for

example, due to standby or network outages). Data Guard will continue to ping the standby and automatically re-establish connection and resynchronize the standby database when possible, but during the period when primary and standby have diverged there will be data loss should a second failure impact the primary database.

For this reason, it is a best practice to monitor protection level, which is simplest using Enterprise Manager Grid Control, and quickly resolve any disruption in communication between the primary and standby before a second failure can occur. This is the most common zero data loss database protection mode.

Choose this protection mode if zero data loss is very important but you want the primary database to continue to be available even with the unlikely case that all standby databases are not reachable. You can complement this solution by integrating multiple standby databases or using Far Sync instances to implement a zero data loss standby solution across a WAN. Workload impact analysis is recommended to measure whether any overhead is acceptable when enabling SYNC transport.

Maximum Performance mode is the default Data Guard mode, and it provides the highest level of data protection that is possible without affecting the performance or the availability of the primary database. This is accomplished by allowing a transaction to commit as soon as the redo data needed to recover that transaction is written to the local online redo log at the primary database (the same behavior as if there were no standby database). Data Guard transmits redo concurrently to 1) the standby database directly from the primary log buffer and 2) to the local online redo log write asynchronously enabling a very low potential data loss if the primary site is lost. There is never any wait for standby acknowledgment but the potential data loss for this data protection mode can still be near zero..

Similar to Maximum Availability mode, it is a best practice to monitor the protection level using Enterprise Manager Grid Control, and quickly resolve any disruption in communication between primary and standby before a second failure can occur.

Choose this mode if minimum data loss is acceptable and zero performance impact on the primary is required.

Offloading Queries to a Read-Only Standby Database

Offloading queries and reporting workloads to read-only standby databases can free up your primary database system resources, giving you the ability to add more users, workloads, or even databases.

When you leverage both primary and standby database resources, your business and your applications benefit with higher total system usage, and potentially higher application throughput.

Offload appropriate workloads by following these steps.

1. Identify which application modules are read-only or read-mostly.
 - Evaluate whether you have application services or modules that are read-only.
 - Small and short read-only queries are good candidates to offload to the standby database.
 - Short DMLs, especially those that are response-time sensitive, should not be offloaded to the standby.
 - Large reports or analytic reports are good candidates to offload.
 - Reports that are primarily reads, and that may have an infrequent DML, typically at the start or end of a report, may be good candidates to offload.

To enable DML Redirection, see `ADG_REDIRECT_DML`.

2. Gather information about the expected application performance, throughput, response time, or elapsed time service levels for each offload candidate.
 - Once you have determined which queries and reports are good candidates to offload, find out the required expected and maximum response time or elapsed time for each of them. For example some large analytic reports must complete within a 2 hour time span.
 - For short queries, determine the expected response time and throughput expectations.
 - These requirements are sometimes referred to as application performance Service Level Agreements, which you need for the next step.
3. Test the performance of each candidate on the standby, and determine whether it meets your requirements.
 - Even though the primary and standby databases have essentially identical data, they are independent databases, independent machines, independent configurations, and have different workloads. For example, an Active Data Guard read-only standby database has a redo apply workload plus the queries that are offloaded, while the primary database may have OLTP, batch, and query workloads.
 - Reported elapsed times, query response time, and workload performance may vary between the primary and standby due to these system, configuration, and workload differences.
 - Tuning requires that you understand system resources, SQL plans, and individual query CPU and wait profile. The tuning recommendations are applicable for both primary and standby databases. See *Diagnosing and Tuning Database Performance*.
4. Offload a subset of the queries that meet your performance requirements, freeing up resources on the primary database for additional processing capacity.
 - Once you have determined which queries and reports can be offloaded, and the performance of those activities are acceptable, then slowly offload some of the workload and monitor it.
 - Do not oversubscribe and offload too much workload to the standby such that redo apply cannot keep pace after tuning. If the standby falls behind, then you lose that standby as a viable role transition target, and in most cases a standby that lags cannot be used to offload queries.

What if a specific query does not meet your requirements?

1. Consult with a performance engineer and follow the recommendations in *Database Performance Tuning Guide*.
2. A particular query response time or throughput or report elapsed time is not guaranteed to be the same on the standby system as it was on the primary. Analyze the system resources, SQL plans, overall CPU work time and wait times.

For example, you may see standby query `scn advance wait` is contributing to a much longer elapsed time in one of your short queries. This wait increase is attributed to Active Data Guard redo apply. If a query sees a certain row in a data block and needs to roll it back because the transaction has not committed as of the query System Commit Number (SCN), it needs to apply corresponding undo to get a consistent read for that query. If the redo for the corresponding undo change has not been applied by redo apply yet, the query needs to wait. The presence of such wait is itself not an issue, and typically may be a couple of milliseconds, but it will vary by workload and may be higher in Real Application Cluster database systems.

Configure and Deploy Oracle Data Guard

Use the following Oracle MAA best practice recommendations to configure and deploy Oracle Data Guard.

Oracle Data Guard Configuration Best Practices

The following topics describe Oracle MAA best practices for configuring your Oracle Data Guard configuration.

Apply Oracle Database Configuration Best Practices First

Before you implement the Oracle Data Guard best practices that follow, apply the Oracle Database configuration best practices.

The Oracle Data Guard configuration best practices are considered additional to the general Oracle Database configuration best practices, and will help you achieve the services levels you expect of the MAA Gold reference architecture. It is implied that all of the database configuration best practices should be followed in a Data Guard configuration, and that the Data Guard recommendations discussed here supplant the general database recommendation where there are conflicts.

See [Oracle Database Configuration Best Practices](#) for more details.

Use Recovery Manager to Create Standby Databases

There are several methods you can use to create an Oracle Data Guard standby database, but because of its simplicity, the Oracle MAA recommended approach is to create a physical standby database using the `RMAN RESTORE . . . FROM SERVICE` clause.

For information about this approach see [Creating a Physical Standby database using RMAN restore from service \(Doc ID 2283978.1\)](#).

Use Oracle Data Guard Broker with Oracle Data Guard

Use Oracle Data Guard broker to create, manage, and monitor an Oracle Data Guard configuration.

You can perform all Data Guard management operations locally or remotely using the broker interfaces: the Data Guard management pages in Oracle Enterprise Manager, which is the broker's graphical user interface (GUI), and the Data Guard command-line interface, called DGMGRL.

The broker interfaces improve usability and centralize management and monitoring of the Data Guard configuration. Available as a feature of Oracle Database Enterprise Edition and Personal Edition, the broker is also integrated with Oracle Database, Oracle Enterprise Manager, and Oracle Cloud Control Plane.

Example Broker Installation and Configuration

The following is an example broker installation and configuration, which is used in all of the broker configuration best practices examples.

Prerequisites:

- Primary database, standby database, and observers reside on separate servers and hardware to provide fault isolation.
 - Both primary and standby databases must use an SPFILE.
 - Set the DG_BROKER_START initialization parameter to TRUE.
 - If any of the databases in the configuration is an Oracle RAC database, you must set up the DG_BROKER_CONFIG_FILE*n* initialization parameters for that database such that they point to the same shared files for all instances of that database. The shared files could be files on a cluster file system, if available, on raw devices, or stored using Oracle Automatic Storage Management.
1. If they do not already exist, create Oracle Net Services aliases that connect to the primary and the standby databases. These aliases should exist in the database home for each host or member of the Data Guard configuration. For Oracle RAC configurations, the aliases should connect using the SCAN name.

```
chicago =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS=(PROTOCOL= TCP)
        (HOST=prmy-scan)(PORT=1521)))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = chicago)))
```

```
boston =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS=(PROTOCOL= TCP)
        (HOST=stby-scan)(PORT=1521)))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = boston)))
```

2. On a primary host, connect with DGMGRL and create the configuration.

```
$ dgmgrl sys
Enter password: password
DGMGRL> create configuration 'dg_config' as primary database is 'chicago'
connect identifier is chicago;

Configuration "dg_config" created with primary database "chicago"

DGMGRL> add database 'boston' as connect identifier is boston;

Database "boston" added
```



```
DGMGRL> enable configuration;  
Enabled.
```

3. By default the broker sets up a LOG_ARCHIVE_DEST_1 for Maximum Performance database protection mode.

The broker configures the remote archive destinations with the default values for asynchronous transport, as shown here.

```
log_archive_dest_3=service="boston", ASYNC NOAFFIRM delay=0 optional  
compression=disable max_failure=0 reopen=300 db_unique_name="boston"  
net_timeout=30, valid_for=(online_logfile,all_roles)
```

Configure Redo Transport Mode

Configure the redo transport service on each configuration member by setting the LogXptMode property to one of the following modes.

- **ASYNCR** configures redo transport services for this standby database using the ASYNCR and NOAFFIRM attributes of the LOG_ARCHIVE_DEST_1 initialization parameter. This mode, along with standby redo log files, enables minimum data loss data protection of potentially less couple seconds with zero performance impact.
- **FASTSYNCR** configures redo transport services for this standby database using the SYNCR and NOAFFIRM attributes of the LOG_ARCHIVE_DEST_1 initialization parameter. Configure synchronous redo transport mode with the NOAFFIRM attribute (default=AFFIRM) when using maximum availability mode protection mode. This helps to minimize the performance impact of synchronous redo transport by acknowledging the receipt of redo once it has been successfully received and verified within standby memory, but before the redo has been written to the standby redo log. Zero data loss protection is still preserved when only the primary database fails.
- **SYNCR** configures redo transport services for this standby database using the SYNCR and AFFIRM attributes of the LOG_ARCHIVE_DEST_1 initialization parameter. This mode, along with standby redo log files, is required for configurations operating in either maximum protection mode or maximum availability mode. This redo transport service enables zero data loss data protection to the primary database, but also can incur a higher performance impact if the round trip latency between primary and standby is high (for example, more than 2ms). This option is required for maximum protection mode.

Use the EDIT DATABASE SET PROPERTY command to set the transport mode the broker configuration, as shown in these examples.

```
DGMGRL> EDIT DATABASE 'boston' SET PROPERTY LogXptMode=ASYNCR;
```

```
DGMGRL> EDIT DATABASE 'chicago' SET PROPERTY LogXptMode=FASTSYNCR;
```

```
DGMGRL> EDIT DATABASE 'SanFran' SET PROPERTY LogXptMode=SYNCR;
```

Validate the Broker Configuration

To identify any problems with the overall configuration, validate it using the following steps.

1. Show the status of the broker configuration using the SHOW CONFIGURATION command.

```
DGMGRL> show configuration;
```

```
Configuration - dg
```



```
Protection Mode: MaxPerformance
Members:
chicago - Primary database
boston - Physical standby database
```

```
Fast-Start Failover: DISABLED
```

```
Configuration Status:
SUCCESS (status updated 18 seconds ago)
```

If the configuration status is SUCCESS, everything in the broker configuration is working properly. However, if the configuration status is WARNING or ERROR then something is wrong in the configuration. Additional error messages that accompany a WARNING or ERROR status can be used to identify the issues. The next step is to examine each database in the configuration to narrow down what the specific error is related to.

2. To identify warnings on the primary and standby databases, show their statuses using the SHOW DATABASE command.

```
DGMGRL> show database chicago
```

```
Database - chicago
```

```
Role:                PRIMARY
Intended State:      TRANSPORT-ON
Instance(s):
  tin1
  tin2
```

```
Database Status:
SUCCESS
```

If the database status is SUCCESS then the database is working properly. However, if database status is WARNING or ERROR, then something is wrong in the database. Additional error messages accompany the WARNING or ERROR status and can be used to identify current issues.

Repeat the SHOW DATABASE command on the standby database and assess any error messages.

3. Validate the databases on Oracle Database 12.1 and later.

In addition to the above commands, in Oracle Database 12.1 and later, the Data Guard broker features a VALIDATE DATABASE command.

```
DGMGRL> validate database chicago
```

```
Database Role:    Primary database
Ready for Switchover: Yes
```

```
DGMGRL> validate database boston;
```

```
Database Role:    Physical standby database
Primary Database: tin
```

```
Ready for Switchover: No
Ready for Failover:   Yes (Primary Running)
```

Capacity Information:

Database	Instances	Threads
tin	2	2
can	1	2

Warning: the target standby has fewer instances than the primary database, this may impact application performance

Standby Apply-Related Information:

```
Apply State:      Not Running
Apply Lag:        Unknown
Apply Delay:      0 minutes
```

The VALIDATE DATABASE command does not provide a SUCCESS or WARNING status and must be examined to determine if any action needs to be taken.

Configure Fast Start Failover

Fast-start failover allows the broker to automatically fail over to a previously chosen standby database in the event of loss of the primary database. Enabling fast-start failover is requirement to meet stringent RTO requirements in the case of primary database, cluster, or site failure.

Fast-start failover quickly and reliably fails over the target standby database to the primary database role, without requiring you to perform any manual steps to invoke the failover. Fast-start failover can be used only in a broker configuration.

If the primary database has multiple standby databases, then you can specify multiple fast-start failover targets, using the FastStartFailoverTarget property. The targets are referred to as candidate targets. The broker selects a target based on the order in which they are specified on the FastStartFailoverTarget property. If the designated fast-start failover target develops a problem and cannot be the target of a failover, then the broker automatically changes the fast-start failover target to one of the other candidate targets.

You can use any protection mode with fast-start failover. The maximum protection and maximum availability modes provide an automatic failover environment guaranteed to lose no data. Maximum performance mode provides an automatic failover environment guaranteed to lose no more than the amount of data (in seconds) specified by the FastStartFailoverLagLimit configuration property. This property indicates the maximum amount of data loss that is permissible in order for an automatic failover to occur. It is only used when fast-start failover is enabled and the configuration is operating in maximum performance mode.

1. Set the FastStartFailoverThreshold property to specify the number of seconds you want the observer and target standby database to wait, after detecting the primary database is unavailable, before initiating a failover, as shown in this example.

```
DGMGR> EDIT CONFIGURATION SET PROPERTY FastStartFailoverThreshold =
seconds;
```

A fast-start failover occurs when the observer and the standby database both lose contact with the production database for a period of time that exceeds the value set for FastStartFailoverThreshold, and when both parties agree that the state of the

configuration is synchronized (Maximum Availability), or that the lag is not more than the configured `FastStartFailoverLagLimit` (Maximum Performance).

An optimum value for `FastStartFailoverThreshold` weighs the trade-off between the fastest possible failover (minimizing downtime) and unnecessarily triggering failover because of temporary network irregularities or other short-lived events that do not have material impact on availability.

The default value for `FastStartFailoverThreshold` is 30 seconds.

The following table shows the recommended settings for `FastStartFailoverThreshold` in different use cases.

Table 16-1 Minimum Recommended Settings for `FastStartFailoverThreshold`

Configuration	minimum Recommended Setting
Single-instance primary, low latency, and a reliable network	15 seconds
Single-instance primary and a high latency network over WAN	30 seconds
Oracle RAC primary	Oracle RAC miscount + reconfiguration time + 30 seconds

2. Determine where to place the observer in your topology.

In an ideal state fast-start failover is deployed with the primary, standby, and observer, each within their own availability domain (AD) or data center; however, configurations that only use two availability domains, or even a single availability domain, must be supported. The following are observer placement recommendations for two use cases.

Deployment Configuration 1: 2 regions with two ADs in each region.

- Initial primary region has the primary database in AD1, and two high availability observers (one observer in AD2 and second HA observer in AD1)
- Initial standby region has the standby database in AD1, and two high availability observers used after role change (one observer in AD2 and second HA observer in AD1)
- For the observer, MAA recommends at least 2 observer targets in the same primary region but in different ADs

Deployment Configuration 2: 2 regions with only 1 AD in each region

- Initial primary regions have the primary database and two light weight servers to host observers
- Initial standby region has the standby database and two light weight servers to host observers (when there is a role change)

3. Configure observer high availability.

You can register up to three observers to monitor a single Data Guard broker configuration. Each observer is identified by a name that you supply when you issue the `START OBSERVER` command. You can also start the observers as a background process.

```
DGMGRL> sys@boston
Enter password: password
DGMGRL> start observer number_one in background;
```

On the same host or a different host you can start additional observers for high availability:

```
DGMGRL> sys@boston
Enter password: password
DGMGRL> start observer number_two in background;
```

Only the primary observer can coordinate fast-start failover with Data Guard broker. All other registered observers are considered to be backup observers.

If the observer was not placed in the background then the observer is a continuously running process that is created when the `START OBSERVER` command is issued. Therefore, the command-line prompt on the observer computer does not return until you issue the `STOP OBSERVER` command from another DGMGRL session. To issue commands and interact with the broker configuration, you must connect using another DGMGRL client session.

Now that you have correctly configured fast-start failover, the following conditions can trigger a failover.

- Database failure where all database instances are down
- Data files taken offline because of I/O errors
- Both the Observer and the standby database lose their network connection to the production database, and the standby database confirms that it is in a synchronized state
- A user-configurable condition

Optionally, you can specify the following conditions for which a fast-start failover can be invoked. It is recommend that you leave these user-configurable conditions at the default values and not invoke an automatic failover.

- Data file offline (write error)
- Corrupted Dictionary
- Corrupted Control file
- Inaccessible Log file
- Stuck Archiver
- ORA-240 (control file enqueue timeout)

Should one of these conditions be detected, the observer fails over to the standby, and the primary shuts down, regardless of how `FastStartFailoverPmyShutdown` is set. Note that the for user-configurable conditions, the fast-start failover threshold is ignored and the failover proceeds immediately.

Fast Start Failover with Multiple Standby Databases

The `FastStartFailoverTarget` configuration property specifies the `DB_UNIQUE_NAME` of one or more standby databases that can act as target databases in a fast-start failover situation when the database on which the property is set is the primary database. These possible target databases are referred to as candidate fast-start failover targets.

The `FastStartFailoverTarget` configuration property can only be set to the name of physical standbys. It cannot be set to the name of a snapshot standby database, far sync instance, or Zero Data Loss Recovery Appliance.

If only one physical standby database exists, then the broker selects that as the default value for this property on the primary database when fast-start failover is enabled. If more than one

physical standby database exists, then the broker selects one based on the order in which they are specified in the property definition. Targets are verified when fast-start failover is enabled

Set Log Buffer Optimally

Set LOG_BUFFER to a minimum of 256 MB when using Oracle Data Guard with asynchronous redo transport.

Doing so allows the asynchronous redo transport to read redo from the log buffer and avoid disk I/Os to online redo logs. For workloads that have a very high redo generation rate (for example, > 50 MB/sec per database instance) the LOG_BUFFER can be increased up to maximum value allowed for the platform being used.

Note

The maximum LOG_BUFFER setting for Linux platform is 2 GB and for Windows is 1 GB.

Set Send and Receive Buffer Sizes

Redo transport processes, especially the receive/standby side, generally benefit from more TCP socket buffers in high latency network paths. TCP socket buffers can be managed by the TCP stack dynamically.

Setting the maximum values for tcp_rmem and tcp_wmem allows the kernel to dynamically modify the buffers allocated to a socket as needed.

Bandwidth Delay Product (BDP) is the product of the network link capacity of a channel and the round time, or latency. The minimum recommended value for socket buffer sizes is 3*BDP, especially for a high-latency, high-bandwidth network. Use `oracdpctest` to tune the socket buffer sizes.

As root, set the maximum value for tcp_rmem and tcp_wmem to 3*<Bandwidth Delay Product>. In this example BDP is 16MB. The other two values for these parameters should be left as they currently are set on the system.

```
# sysctl -w net.ipv4.tcp_rmem='4096 87380 16777216';
```

```
# sysctl -w net.ipv4.tcp_wmem='4096 16384 16777216';
```

Using `sysctl` to change these values changes them dynamically in memory only and will be lost when the system is rebooted. Additionally set these values in `/etc/sysctl.conf` on linux systems. Add these entries if the values are absent in the file.

```
net.ipv4.tcp_rmem='4096 87380 16777216'
```

```
net.ipv4.tcp_wmem='4096 16384 16777216'
```

Set SDU Size to 65535 for Synchronous Transport Only

With Oracle Net Services you can control data transfer by adjusting the session data unit (SDU) size. Oracle testing has shown that setting the SDU parameter to its maximum value of 65535 improves performance of synchronous transport.

You can set SDU on a per connection basis using the SDU parameter in the local naming configuration file, `tnsnames.ora`, and the listener configuration file, `listener.ora`, or you can set the SDU for all Oracle Net Services connections with the profile parameter `DEFAULT_SDU_SIZE` in the `sqlnet.ora` file.

Configure Online Redo Logs Appropriately

Redo log switching has a significant impact on redo transport and apply performance. Follow these best practices for sizing the online redo logs on the primary and standby databases.

Following these guidelines for online redo logs.

- All online redo log groups should have identically sized logs (to the byte).
- Online redo logs should reside on high performing disks (DATA disk groups).
- Create a minimum of three online redo log groups per thread of redo on Oracle RAC instances.
- Create online redo log groups on shared disks in an Oracle RAC environment.
- Multiplex online redo logs (multiple members per log group) unless they are placed on high redundancy disk groups.
- Size online redo logs to switch no more than 12 times per hour (every ~5 minutes). When log switches occur at a higher frequency, such as less than a minute, database hangs may occur if database checkpointing or archiving rates are not keeping up with peak redo generation rate. An online redo log cannot be reused unless changes have been checkpointed, and the online redo log has been archived if archivelog mode is enabled.

Sizing Redo Logs

Size the redo logs based on the peak redo generation rate of the primary database.

You can determine the peak rate by running the query below for a period of time that includes the peak workload. The peak rate could be seen at month-end, quarter-end, or annually. Size the redo logs to handle the highest rate in order for redo apply to perform consistently during these workloads.

```
SQL> SELECT thread#,sequence#,blocks*block_size/1024/1024 MB,(next_time-
first_time)*86400 sec,
       blocks*block_size/1024/1024)/((next_time-first_time)*86400) "MB/s"
FROM v$archived_log WHERE ((next_time-first_time)*86400<>0) and first_time
between to_date('2015/01/15 08:00:00','YYYY/MM/DD HH24:MI:SS')
and to_date('2015/01/15 11:00:00','YYYY/MM/DD HH24:MI:SS') and dest_id=1
order by first_time;
```

THREAD#	SEQUENCE#	MB	SEC	MB/s
2	2291	29366.1963	831	35.338383
1	2565	29365.6553	781	37.6000708
2	2292	29359.3403	537	54.672887
1	2566	29407.8296	813	36.1719921
2	2293	29389.7012	678	43.3476418
2	2294	29325.2217	1236	23.7259075
1	2567	11407.3379	2658	4.29169973
2	2295	29452.4648	477	61.7452093
2	2296	29359.4458	954	30.7751004

2	2297	29311.3638	586	50.0193921
1	2568	3867.44092	5510	.701894903

Choose the redo log size based on the peak generation rate with the following chart. Repeat this query if workload changes or additional pluggable database is added.

Peak Redo Rate	Recommended Redo Log Size
<= 5 MB/sec	500 MB
<= 25 MB/sec	1 GB
<= 75 MB/sec	4 GB
<= 150 MB/sec	8 GB
<= 300 MB/sec	16 GB
> 300 MB/sec	32 GB

Use Standby Redo Log Groups

Configure the standby redo log groups on all primary and standby databases for improved availability and performance.

For each redo log thread--a thread is associated with an Oracle RAC database instance--the number of standby redo log groups must be greater than or equal to ($\geq$) the number of online redo log groups.

Consider the following additional guidelines when creating standby redo log groups.

- All online redo logs and standby redo log groups should have identically sized logs (to the byte). Standby redo logs are not used if they are not the same size as the online redo logs.
- All standby redo log groups should have identically sized logs (to the byte) on both the primary and standby databases.
- Standby redo logs should reside on high performing disks (DATA disk group).
- Standby redo logs should be multiplexed (multiple members per log group) unless placed on high redundancy disk groups. Multiplexing standby redo logs is optional in all cases because Data Guard can fetch any missing redo.
- In an Oracle RAC environment, create standby redo logs on a shared disk.
- In an Oracle RAC environment, assign a thread to each standby redo log group.

The following example creates three log groups for each redo thread.

```
SQL> ALTER DATABASE ADD STANDBY LOGFILE THREAD 1 GROUP 7 ('+DATA') SIZE
4194304000, GROUP 8 ('+DATA') SIZE 4194304000, GROUP 9 ('+DATA') SIZE
4194304000;
```

```
SQL> ALTER DATABASE ADD STANDBY LOGFILE THREAD 2 GROUP 10 ('+DATA') SIZE
4194304000, GROUP 11 ('+DATA') SIZE 4194304000, GROUP 12 ('+DATA') SIZE
4194304000
```

To check the thread number and group numbers of the online redo logs, query the V\$LOG view.

```
SQL> SELECT * FROM V$LOG;
```

To check the results of the `ALTER DATABASE ADD STANDBY LOGFILE THREAD` statements, query the `V$STANDBY_LOG` view.

```
SQL> SELECT * FROM V$STANDBY_LOG;
```

Protect Against Data Corruption

Oracle Database corruption prevention, detection, and repair capabilities are built on internal knowledge of the data and transactions it protects, and on the intelligent integration of its comprehensive high availability solutions.

When data corruption is detected, Oracle Data Guard, block media recovery, and data file media recovery can recover the data. Database-wide logical corruptions caused by human or application errors can be undone with Oracle Flashback Technologies.

Tools are also available for proactive validation of logical data structures. For example, the `SQL*Plus ANALYZE TABLE` statement detects inter-block corruptions.

Achieve the most comprehensive data corruption prevention and detection with these best practices.

- Use Oracle Data Guard with physical standby databases to prevent widespread block corruption. Oracle Data Guard is the best solution for protecting Oracle data against data loss and corruption, and lost writes.
- Set the Oracle Database block-corruption initialization parameters on the Data Guard primary and standby databases as shown in the following table.

Table 16-2 Block-Corruption Initialization Parameter Settings

On the primary database set...	On the standby databases set...
<code>DB_BLOCK_CHECKSUM=TYPICAL</code>	<code>DB_BLOCK_CHECKSUM=TYPICAL</code>
<code>DB_LOST_WRITE_PROTECT=AUTO</code>	<code>DB_LOST_WRITE_PROTECT=AUTO</code>
<code>DB_BLOCK_CHECKING=FALSE*</code>	<code>DB_BLOCK_CHECKING=MEDIUM</code>

* `DB_BLOCK_CHECKING` on the PRIMARY is recommended to be set to `MEDIUM` or `FULL` but only after a full performance evaluation with the application.

- Performance overhead is incurred on every block change, therefore performance testing is of particular importance when setting the `DB_BLOCK_CHECKING` parameter. Oracle highly recommends the minimum setting of `DB_BLOCK_CHECKING=MEDIUM` (block checks on data blocks but not index blocks) on either the primary or standby database. If the performance overhead of enabling `DB_BLOCK_CHECKING` to `MEDIUM` or `FULL` is unacceptable on your primary database, then set `DB_BLOCK_CHECKING` to `MEDIUM` or `FULL` for your standby databases.
- Setting `DB_LOST_WRITE_PROTECT=AUTO` (available and default after Oracle Database 19c Release 19.26) provides lost writes corruption detection and potentially auto-repair when the standby database's real-time apply is enabled. With `DB_LOST_WRITE_PROTECT=AUTO`, the standby database performs lost write detection while redo apply is reasonably current; otherwise these additional checks are temporarily disabled until apply lag is near current again. `DB_LOST_WRITE_PROTECT=AUTO` temporarily suspends lost write protection when apply lag > min(60 seconds, $\frac{2}{3} \times \text{FSFO ApplyLagThreshold}$). Depending on workloads, enabling lost write protection increases the primary database redo generation and incurs additional redo apply processing and physical reads on the standby.

- See [Best Practices for Corruption Detection, Prevention, and Automatic Repair - in a Data Guard Configuration \(KB143286\)](#) for more information and insight.

The following recommendations also help to protect against data corruptions.

- Use Oracle Automatic Storage Management (Oracle ASM) to provide disk mirroring to protect against disk failures.
- Use Oracle ASM HIGH REDUNDANCY for optimal corruption repair. Using Oracle ASM redundancy for disk groups provides mirrored extents that can be used by the database if an I/O error or corruption is encountered. For continued protection, Oracle ASM redundancy provides the ability to move an extent to a different area on a disk if an I/O error occurs. The Oracle ASM redundancy mechanism is useful if you have bad sectors returning media errors.
- Enable Flashback Technologies for fast point-in-time recovery from logical corruptions most often caused by human error and for fast reinstatement of a primary database following failover.
- Use RMAN for additional block checks during backup and restore operations. Implement a backup and recovery strategy with Recovery Manager (RMAN) and periodically use the RMAN BACKUP VALIDATE CHECK LOGICAL scan to detect corruptions.
- Use Zero Data Loss Recovery Appliance or Autonomous Recovery Service for backup and recovery validation including corruption checks and repairs, central backup validation, reduced production database impact, and Enterprise Cloud backup and recovery solution.

Use Flashback Database for Reinstatement After Failover

Enable Flashback Database on both the primary and standby database, so that if the original primary database has not been damaged, you can reinstate the original primary database as a new standby database following a failover.

If there is a failure during the switchover process, then it can easily be reversed when Flashback Database is enabled.

Set DB_FLASHBACK_RETENTION_TARGET to the same value on the standby database as the primary. Set DB_FLASHBACK_RETENTION_TARGET initialization parameter to the largest value prescribed by any of the following conditions that apply.

- To leverage flashback database to reinstate your failed primary database after Data Guard failover, for most cases set DB_FLASHBACK_RETENTION_TARGET to a minimum of 120 (minutes) to enable reinstatement of a failed primary.
- If using Flashback Database for fast point in time recovery from user error or logical corruptions, set DB_FLASHBACK_RETENTION_TARGET to a value equal to the farthest time in the past to which the database should be recovered. If you can detect and repair from logical corruptions in less than 24 hours, then set DB_FLASHBACK_RETENTION_TARGET to a minimum of 1440 (minutes).

Use Force Logging Mode

When the primary database is in FORCE LOGGING mode, all database data changes are logged. FORCE LOGGING mode ensures that the standby database remains consistent with the primary database.

If it is not possible to use this mode because you require the load performance with NOLOGGING operations, then see [Enable an Appropriate Logging Mode](#) for other options.

You can enable force logging immediately by issuing an `ALTER DATABASE FORCE LOGGING` statement. If you specify `FORCE LOGGING`, then Oracle waits for all ongoing non-logged operations to finish.

Configure Fast Start Failover to Bound RTO and RPO (MAA Gold Requirement)

Enabling fast-start failover is requirement to meet stringent RTO and RPO requirements in the case of primary database, cluster, or site failure. With Data Guard fast-start failover, there's a Data Guard observer to provide quorum of 2 to preserve database consistency and prevent database split brains.

Fast-start failover allows the Data Guard broker to automatically fail over to a previously chosen standby database in the event of loss of the primary database. Fast-start failover quickly and reliably switches the target standby database over to the primary database role, without requiring you to perform any manual steps to invoke the failover. Fast-start failover can be used only in a Data Guard broker configuration.

If the primary database has multiple standby databases, then you can specify multiple fast-start failover targets, using the `FastStartFailoverTarget` property. The targets are referred to as candidate targets. The broker selects a target based on the order in which they are specified on the `FastStartFailoverTarget` property. If the designated fast-start failover target develops a problem and cannot be the target of a failover, then the broker automatically changes the fast-start failover target to one of the other candidate targets.

You can use any Data Guard protection mode with fast-start failover to reduce RTO and bound RPO.

- The maximum protection and maximum availability modes provide an automatic failover environment guaranteed to lose no data or ensure zero data loss automatic failover.
- Maximum performance mode enables minimum or no database and application performance impact and provides an automatic failover environment guaranteed to lose no more than the amount of data (in seconds) specified by the `FastStartFailoverLagLimit` configuration property. The `FastStartFailoverLagLimit` property indicates the maximum amount of data loss that is permissible in order for an automatic failover to occur. It is only used when fast-start failover is enabled and the configuration is operating in maximum performance mode.

Without Data Guard Fast-Start Failover, you cannot bound RPO or data loss, because in a case where the primary is isolated, the primary can continue to generate more redo and transactions unbeknownst to the standby database. With Data Guard Fast-Start Failover, the primary will be `STALLED` or shut down depending on the Data Guard broker `FastStartFailoverPmyShutdown` property.

1. Set the `FastStartFailoverThreshold` property to specify the number of seconds you want the observer and target standby database to wait, after detecting the primary database is unavailable, before initiating a failover, as shown in this example.

```
DGMGRL> EDIT CONFIGURATION SET PROPERTY FastStartFailoverThreshold =  
seconds;
```

A fast-start failover occurs when the observer and the standby database both lose contact with the production database for a period of time that exceeds the value set for `FastStartFailoverThreshold`, and when both parties agree that the state of the

configuration is synchronized (Maximum Availability mode), or that the lag is not more than the configured `FastStartFailoverLagLimit` (Maximum Performance mode).

An optimum value for `FastStartFailoverThreshold` weighs the trade-off between the fastest possible failover (minimizing downtime) and unnecessarily triggering failover because of temporary network irregularities or other short-lived events that do not have material impact on availability.

The default value for `FastStartFailoverThreshold` is 30 seconds.

The following table shows the recommended settings for `FastStartFailoverThreshold` in different use cases.

Configuration	Minimum Recommended Setting
Single-instance primary, low latency, and a reliable network	15 seconds
Single-instance primary and a high latency network over WAN	30 seconds
Oracle RAC primary	Oracle RAC miscount + reconfiguration time + 30 seconds For Exadata systems, minimum setting of 30 seconds

2. Determine where to place the observer in your topology.

The Data Guard broker observer provides a quorum of 2 to preserve database consistency and avoid split brains. Data Guard fast-start failover always guarantees that only one primary database exists, and external consistency is guaranteed by routing transactions to the primary database. In an ideal state, fast-start failover is deployed with the primary, standby, and observer, each within their own availability domain (AD) or data center; however, configurations that only use two availability domains, or even a single availability domain, must be supported. The following are observer placement recommendations for two use cases.

- **Deployment Configuration 1: 2 regions with two ADs in each region.**
 - Initial primary region has the primary database in AD1, and two high availability observers (one observer in AD2 and second HA observer in AD1)
 - Initial standby region has the standby database in AD1, and two high availability observers used after role change (one observer in AD2 and second HA observer in AD1)
 - For the observer, MAA recommends at least 2 observer targets in the same primary region but in different ADs
- **Deployment Configuration 2: 2 regions with only 1 AD in each region**
 - Initial primary regions have the primary database and two light weight servers to host observers
 - Initial standby region has the standby database and two light weight servers to host observers (when there is a role change)

3. Configure observer high availability.

You can register up to three observers to monitor a single Data Guard broker configuration. Each observer is identified by a name that you supply when you issue the `START OBSERVER` command. You can also start the observers as a background process.

```
DGMGRL> sys@boston
Enter password:
DGMGRL> start observer number_one in background;
```

On the same host or a different host you can start additional observers for high availability:

```
DGMGRL> sys@boston
Enter password:
DGMGRL> start observer number_two in background;
```

Only the primary observer can coordinate fast-start failover with Data Guard broker. All other registered observers are considered to be backup observers.

If the observer was not placed in the background, then the observer is a continuously executing process that is created when the `START OBSERVER` command is issued. Therefore, the command-line prompt on the observer computer does not return until you issue the `STOP OBSERVER` command from another DGMGRL session. To issue commands and interact with the broker configuration, you must connect using another DGMGRL client session.

Triggering Fast-Start Failover

Now that you have correctly configured fast-start failover, the following conditions can trigger a failover.

- Database failure where all database instances are down
- Data files taken offline because of I/O errors
- Both the Observer and the standby database lose their network connection to the production database, and the standby database confirms that it is in a synchronized state
- A user-configurable condition

Optionally, you can specify the following conditions for which a fast-start failover can be invoked. It is recommended that you leave these user-configurable conditions at the default values and not invoke an automatic failover.

- Datafile offline (write error)
- Corrupted Dictionary
- Corrupted Controlfile
- Inaccessible Logfile
- Stuck Archiver
- ORA-240 (control file enqueue timeout)

Should one of these conditions be detected, the observer fails over to the standby, and the primary shuts down, regardless of how `FastStartFailoverPmyShutdown` is set. Note that the for user-configurable conditions, the fast-start failover threshold is ignored and the failover proceeds immediately.

Fast Start Failover with Multiple Standby Databases

The `FastStartFailoverTarget` configuration property specifies the `DB_UNIQUE_NAME` of one or more standby databases that can act as target databases in a fast-start failover situation when the database on which the property is set is the primary database. These possible target databases are referred to as candidate fast-start failover targets.

The `FastStartFailoverTarget` configuration property can only be set to the name of physical standbys. It cannot be set to the name of a snapshot standby database, far sync instance, or Zero Data Loss Recovery Appliance.

If only one physical standby database exists, then the broker selects that as the default value for this property on the primary database when fast-start failover is enabled. If more than one physical standby database exists, then the broker selects one based on the order in which they are specified in the property definition. Targets are verified when fast-start failover is enabled.

Configure Standby AWR

Since Oracle Database 12c (12.2), Automatic Workload Repository (AWR) snapshots can be taken of the standby database.

Standby AWR is the best tool for identifying performance issues with recovery and reporting workloads in an Active Data Guard standby database.

See *Managing Automatic Workload Repository in Active Data Guard Standby Databases* for details about configuring and managing standby AWR.

Note

For Oracle Exadata Cloud Data Guard deployments, standby AWR is configured as part of instantiation.

To Create Standby AWR Reports

1. Identify the AWR ID (`NODE_ID`) for the standby database.

```
SQL> select NODE_ID, NODE_NAME from DBA_UMF_REGISTRATION;
```

2. Run the reports from the primary database using the `NODE_ID` for the target database as the `DBID`.

- For instance level reports (for example, assessing redo apply performance bottlenecks) use the `awrrpti` script.

```
SQL> ?/rdbms/admin/awrrpti
```

- For global AWR reports on the standby (for example, assessing query performance) use the `awrg rpti` script.

```
SQL> ?/rdbms/admin/awrg rpti
```

Configuring Multiple Standby Databases

An Oracle Data Guard configuration with multiple standby databases gives you the benefits of both local and remote standby databases.

A local standby database can provide zero data loss failover and application downtime reduced to seconds. If a regional disaster occurs, making the primary and local standby systems inaccessible, the application and database can fail over to the remote standby. See ["Gold: Multiple Standby Databases"](#) for a full discussion of the features and benefits of a multiple standby configuration.

Managing Oracle Data Guard Configurations with Multiple Standby Databases

The Oracle Data Guard broker automates management and operation tasks across multiple databases in an Oracle Data Guard configuration. The broker also monitors all of the systems in a single Oracle Data Guard configuration.

In a multi-member Data Guard configuration the following redo transport destinations are supported:

- Oracle Data Guard standby databases
- Far sync instances (See Using Far Sync Instances for more information)
- Oracle Streams downstream capture databases
- Zero Data Loss Recovery Appliance (Recovery Appliance)

Multiple Standby Databases and Redo Routes

You can use the Oracle Data Guard broker RedoRoutes property to override the default behavior by which a primary database sends the redo that it generates to every other redo transport destination in the configuration.

An example redo transport topology that differs from the default would be one in which a physical standby, or a far sync instance, forwards redo received from the primary database to one or more destinations, or one in which the redo transport mode used for a given destination is dependent on which database is in the primary role.

Consider a configuration that has a primary database (North_Sales) and two physical standby databases (Local_Sales and Remote_Sales). The Local_Sales database is located in the same data center as the primary for high availability purposes and for simpler application and database failover. The Remote_Sales database is located in a remote data center for disaster recovery purposes.

Rather than have North_Sales ship its redo to both databases, you can use the RedoRoutes broker property to configure real-time cascading, in which the local physical standby database forwards to Remote_Sales the redo it receives from North_Sales. To accomplish this, the RedoRoutes property is set on North_Sales and Local_Sales as follows:

- On the North_Sales database, the RedoRoutes property specifies that if North_Sales is in the primary role, then it should ship redo to the Local_Sales database using synchronous transport mode. This rule prevents the primary from shipping redo data directly to the Remote_Sales database.
- On the Local_Sales database, the RedoRoutes property must specify that if North_Sales is in the primary role, then Local_Sales should forward redo it receives from North_Sales on to Remote_Sales.

To see the runtime RedoRoutes configuration, use the `SHOW CONFIGURATION` command. For example:

```
DGMGRL> SHOW CONFIGURATION;
```

Configuration - Sales_Configuration

Protection Mode: MaxAvailability

Members:

North_Sales - Primary database

Local_Sales - Physical standby database

Remote_Sales - Physical standby database (receiving current redo)

Fast-Start Failover: DISABLED

Configuration Status:

SUCCESS

Note that the asynchronous redo transport attribute was explicitly specified in the redo route rule for the Remote_Sales destination to enable real-time cascading of redo to that destination. (Real-time cascading requires a license for the Oracle Active Data Guard option.)

To disable real-time cascading of redo, do not specify the asynchronous redo transport attribute. For example:

```
DGMGRL> EDIT DATABASE 'Local_Sales' SET PROPERTY 'RedoRoutes' =  
'(North_Sales : Remote_Sales)';
```

See RedoRoutes for more information.

Using the RedoRoutes Property for Remote Alternate Destinations

The RedoRoutes property can be used to set up a remote alternate destination, so that a terminal member can still receive redo data even if the member from which it was receiving the redo data fails.

Using the previous example, you can have the primary database, North_Sales, send redo data directly to Remote_Sales if the Local_Sales standby database failed. It is also possible, using the PRIORITY attribute, to specify that once the Local_Sales failure has been resolved it can resume shipping redo to Remote_Sales.

```
DGMGRL> EDIT DATABASE 'North_Sales' SET PROPERTY  
'RedoRoutes' = '(LOCAL : ( Local_Sales ASYNC PRIORITY=1, Remote_Sales ASYNC  
PRIORITY=2 ))';  
Property "RedoRoutes" updated
```

```
DGMGRL> EDIT DATABASE 'Local_Sales'  
SET PROPERTY 'RedoRoutes' = '(North_Sales : Remote_Sales ASYNC)';  
Property "RedoRoutes" updated
```

```
DGMGRL> SHOW CONFIGURATION;
```

Configuration - Sales_Configuration

Protection Mode: MaxPerformance


```

Members:
North_Sales    - Primary database
Local_Sales    - Physical standby database
Remote_Sales   - Physical standby database
Fast-Start Failover: DISABLED

```

```

Configuration Status:
SUCCESS

```

To see the full RedoRoutes configuration, use the `SHOW CONFIGURATION VERBOSE` command. For example:

```
DGMGRL> SHOW CONFIGURATION VERBOSE;
```

```
Configuration - Sales_Configuration
```

```

Protection Mode: MaxPerformance
Members:
North_Sales - Primary database
Local_Sales - Physical standby database
Remote_Sales - Physical standby database
Remote_Sales - Physical standby database (alternate of Local_Sales)

```

```

Properties:
FastStartFailoverThreshold      = '180'
OperationTimeout                = '30'
TraceLevel                     = 'USER'
FastStartFailoverLagLimit      = '300'
CommunicationTimeout           = '180'
ObserverReconnect              = '0'
FastStartFailoverAutoReinstate = 'TRUE'
FastStartFailoverPmyShutdown   = 'TRUE'
BystandersFollowRoleChange     = 'ALL'
ObserverOverride               = 'FALSE'
ExternalDestination1           = ''
ExternalDestination2           = ''
PrimaryLostWriteAction         = 'CONTINUE'
ConfigurationWideServiceName   = 'c0_CFG'

```

```
Fast-Start Failover: DISABLED
```

```

Configuration Status:
SUCCESS

```

Fast Start Failover with Multiple Standby Databases

The Oracle Data Guard `FastStartFailoverTarget` broker configuration property specifies the `DB_UNIQUE_NAME` of one or more standby databases that can act as target databases in a fast-start failover scenario when the database on which the property is set is the primary database.

These possible target databases are referred to as *candidate fast-start failover targets*. The `FastStartFailoverTarget` property can only be set to the name of physical standbys. It cannot be set to the name of a snapshot standby database, far sync instance, or Zero Data Loss Recovery Appliance.

If only one physical standby database exists, then the broker selects that database as the default value for `FastStartFailoverTarget` on the primary database when fast-start failover is enabled. If more than one physical standby database exists, then the broker selects a single standby based on the order in which they are specified in the property definition. The targets are verified when fast-start failover is enabled.

See also, `FastStartFailoverTarget`.

Setting FastStartFailoverTarget

If you have two or more standby databases, set up the `FastStartFailoverTarget` configuration property on the primary database to indicate the desired fast-start failover target standby database.

The Oracle Data Guard broker reciprocally sets this property for the target standby database to indicate the primary database as its future target standby database when fast-start failover is actually enabled. There is no need for you set this property on the target standby as this is done for you automatically. For example:

```
DGMGRL> edit database moe set property ='curly,larry';  
Property "faststartfailovertarget" updated
```

After `FastStartFailoverTarget` is configured, continue with enabling fast-start failover. When fast-start failover is enabled, you cannot change the `FastStartFailoverTarget` configuration property on the primary or target standby databases.

To change the `FastStartFailoverTarget` property to point to a different standby database, disable fast-start failover, set the `FastStartFailoverTarget` property, and reenale fast-start failover. This action does not impact primary or standby database availability or up time.

Switchover with FastStartFailoverTarget Set

If fast-start failover is enabled with `FastStartFailoverTarget` set you can still perform a switchover or a manual failover, as long the role change is directed to the same standby database that was specified for the `FastStartFailoverTarget` database property on the primary database.

Attempting to switch over to a standby that is not the fast-start failover target results in ORA-16655.

```
DGMGRL> switchover to curly  
Performing switchover NOW, please wait...  
Error: ORA-16655: specified standby database not the current fast-start  
failover target standby
```

To switch over to a standby that is not the primary fast-start target:

1. Disable fast-start failover.

```
DGMGRL> DISABLE FAST_START FAILOVER;
```

2. Edit the FastStartFailoverTarget property to list the standby you wish to switch over to first.

```
DGMGRL> edit database moe set property
FastStartFailoverTarget='curly,larry';
Property "faststartfailovertarget" updated
```

3. Enable fast-start failover.

```
DGMGRL> ENABLE FAST_START FAILOVER;
```

4. Perform the switchover operation.

```
DGMGRL> switchover to curly
Performing switchover NOW, please wait...
```

Fast-Start Failover Outage Handling

If the primary database's fast-start failover target standby database becomes unavailable, perhaps because the standby database or instance is down or there's an issue with transporting redo, then the primary's fast-start failover target is automatically switched to the next target configured in the FastStartFailoverTarget property.

Note that it can take several ping cycles to effect the target switch: one ping to recognize that the current target is not viable, and another ping to propose the target switch and finalize it.

If the original fast-start failover target comes back online, a switch back to the original target is not performed automatically. To get the original target back after an outage you must disable and then enable fast-start failover.

Oracle Active Data Guard Far Sync Solution

To support zero data loss, you can deploy between the primary and standby databases an Oracle Data Guard far sync instance, which is a remote Oracle Data Guard destination that accepts redo from the primary database and then ships that redo to other members of the Oracle Data Guard configuration.

About Far Sync

Far Sync is an Oracle Active Data Guard feature that provides increased flexibility in the location of a disaster recovery site for those who wish to implement zero data loss protection.

Even users who have already deployed Oracle Data Guard synchronous transport can benefit from configuring a far sync instance closer to the primary than their current standby to reduce the performance impact on the production database.

Synchronous redo transport over WAN distances or on an under-performing network often has too large an impact on primary database performance to support zero data loss protection. Oracle Active Data Guard Far Sync provides the ability to perform a zero data loss failover to a remote standby database without requiring a second standby database or complex operation.

Far Sync enables this by deploying a far sync instance (a lightweight Oracle instance) at a distance that is within an acceptable range of the primary for synchronous redo transport. A far sync instance receives redo from the primary using synchronous transport and forwards the redo to up to 29 remote standby databases using asynchronous transport.

Far sync instances are part of the Oracle Active Data Guard Far Sync feature, which requires an Oracle Active Data Guard license.

Offloading to a Far Sync Instance

A far sync instance offloads from the primary any overhead of resolving gaps in redo received by the remote standby database (for example, following network or standby database outages) and can conserve WAN bandwidth by performing redo transport compression without impacting primary database performance.

Note that redo compression requires that the Advanced Compression Option be licensed.

Redo Transport Encryption can additionally be offloaded to the far sync instance. Including Advanced Security Option (ASO) encryption during MAA testing showed no impact to the performance of the primary nor currency of the standby databases.

Oracle recommends using ASO for encryption because it is tested and integrated with Oracle Net and Data Guard.

Note that Oracle Advanced Security Option is a licensed option.

Far Sync Deployment Topologies

Oracle Active Data Guard Far Sync provides the ability to perform a zero data loss failover to a remote standby database without requiring a second standby database or complex operation.

Data Guard enables this by deploying a far sync instance (a lightweight Oracle instance that has only a control file, SPFILE, password file and standby log files; there are no database files or online redo logs) at a distance that is within an acceptable range of the primary for synchronous transport. A far sync instance receives redo from the primary through synchronous transport and immediately forwards the redo to up to 29 remote standby databases using asynchronous transport. A far sync instance can also forward redo to the new Oracle Database Backup, Logging, and Recovery Appliance.

Figure 16-1 Far Sync Architecture Overview



The following use cases illustrate the benefits of various architecture choices you can implement with far sync instances.

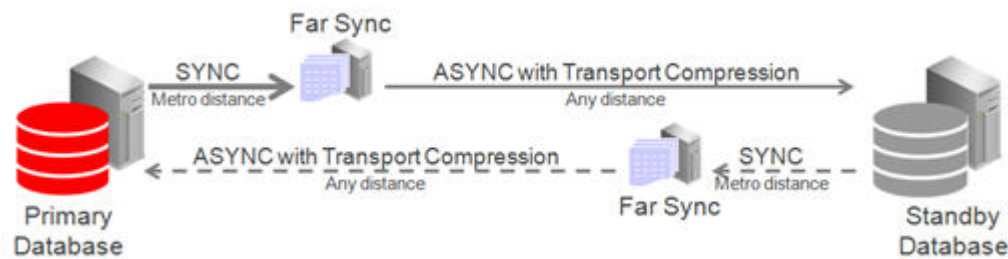
Case 1: Zero Data Loss Protection Following Role Transitions

This is the most basic example in which a primary database uses high availability far sync instances to extend zero data loss failover to a remote standby database.

Ideally the high availability far sync instance is deployed in a location separate from the primary database to isolate it from site failure, but within a metro area distance (network RTT of 5ms or less – subject to performance testing). Even if no separate location is available there is still a benefit to deploying a far sync instance within the same data center to enable fast, zero data loss failover for all unrecoverable outages short of full site failure.

The remote high availability far sync instance is idle while the standby database is in a standby role. It becomes active when the standby database transitions to the primary database role, enabling zero data loss failover to the new standby (old primary). The high availability far sync instance that is local to the original primary database becomes inactive while it is in a standby role.

Figure 16-2 Role Transition Facilitated by Far Sync

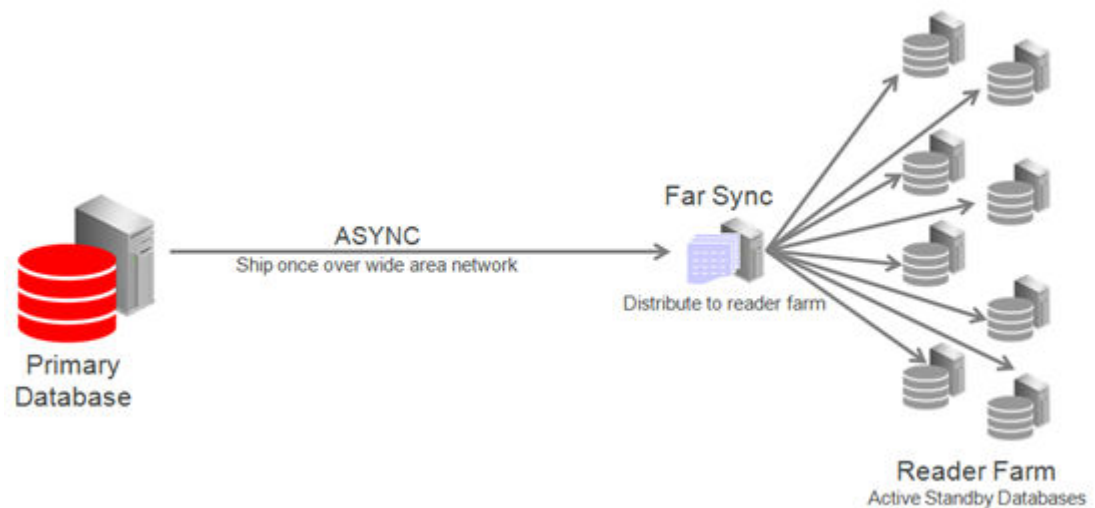


High availability far sync options are described in Far Sync Instance High Availability Typologies.

Case 2: Reader Farm Support

Far Sync can support up to 30 remote destinations, making it a very useful tool for supporting a reader farm – an Active Data Guard configuration having multiple active standby databases to easily scale read performance.

In this example the reader farm is configured in a remote location from the primary database. The primary ships once over the WAN to the far sync instance located in the remote destination and Far Sync distributes redo locally to all active standby databases in the reader farm.

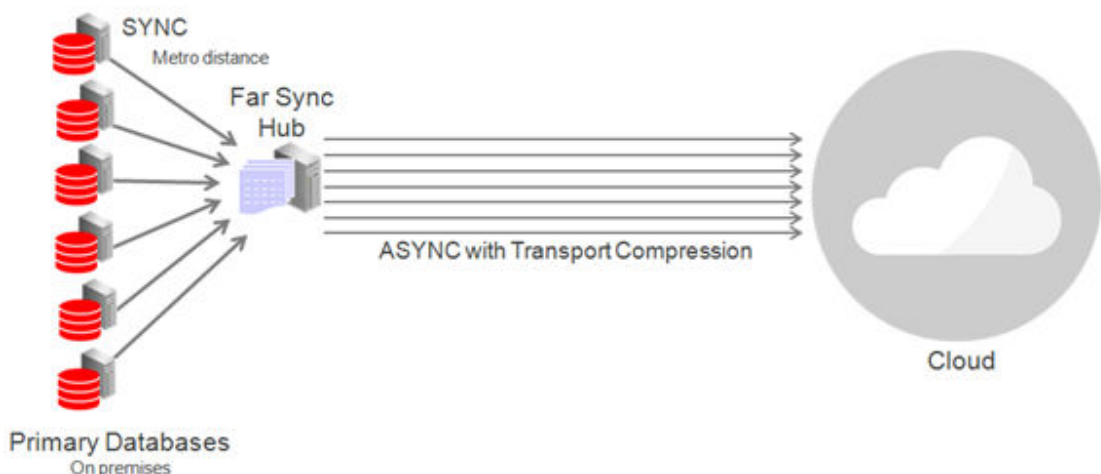
Figure 16-3 Far Sync Ships Redo to Reader Farm

Case 3: Cloud Deployment With Far Sync Hub

Far Sync is a very lightweight process; a single physical server can support multiple far sync instances, each providing zero data loss failover to a remote destination.

The diagram below shows primary databases shipping to a single physical machine operating as a far sync "hub" consisting of multiple far sync instances on a single physical machine. Primaries and the far sync hub are on-premises while standby databases are deployed remotely in the cloud.

Note that all of the systems in this configuration (primary and standby database hosts and far sync instance host) must meet the usual requirements for compatibility in a Data Guard configuration described in [Data Guard Support for Heterogeneous Primary and Physical Standbys in Same Data Guard Configuration \(Doc ID 413484.1\)](#).

Figure 16-4 Far Sync Hub Architecture

Far Sync High Availability Topologies

To keep far sync instances highly available, consider the following deployment topologies.

Deploy Far Sync Instances on Oracle Real Application Clusters

The far sync instance can be placed on an Oracle RAC cluster. In this configuration a far sync instance is only active on one server at a time while other servers provide automatic failover for high availability. The characteristics of this approach include:

- Lowest data loss potential and brown-out when the active far sync instance or node fails.
- The ability to resume zero data loss protection quickly after far sync instance failure.
- By itself, this solution does not address cluster failure.

The most critical applications are well served by a pair of Oracle RAC far sync instances, each configured as an alternate for the other and deployed at different locations. This provides the most robust HA and data protection (during instance, node, cluster and site outages).

Deploy Far Sync Instances on Alternate Destinations and Multiple Far Sync instances

Configuring two separate far sync instances on distinct physical machines, each serving as an alternate destination for the other, provides far sync instance high availability in a non-Oracle RAC environment. Each destination defined on the primary database contains the `ALTERNATE` keyword assigning the other far sync instance as the alternate. When the active far sync instance enters an error state the alternate destination pointing to the alternate far sync instance is enabled automatically. By defining a far sync instance as an alternate destination, Maximum Availability protection will be maintained after a briefly dropping to a resynchronization state while the new destination is prepared.

The characteristics of this approach include:

- Retains zero data loss coverage after far sync instance transport failures (instance or network outages).
- Failure testing has shown
 - During far sync instance failures a performance brownout of approximately 3.5 seconds while `SYNC` redo transport starts (network sync service - NSS).
 - During network failures a short brownout equal to the setting of the destination's `net_timeout` parameter was observed.
- HA for machine outage assuming each far sync instance is on separate hardware.
- HA for site outage assuming far sync instances are deployed in separate sites.
- Higher application brown-out and resynchronization time during far sync instance outages compared with Far Sync with Oracle RAC

Deploy a Far Sync Instance on the Terminal Standby as an Alternate Destination

The simplest approach to maintaining data protection during a far sync instance outage is to create an alternate `LOG_ARCHIVE_DEST_n` pointing directly to the terminal standby (the terminal failover target). Asynchronous transport to the remote destination is the most likely choice in order to avoid the performance impact on the primary caused by WAN network latency.

Asynchronous transport can achieve near-zero data loss protection (as little as sub-seconds to seconds of exposure), but because it never waits for standby acknowledgment, it is unable to provide a zero data loss guarantee. In this configuration the protection level must be dropped to Maximum Performance prior to a switchover (planned event) as the level must be

enforceable on the target in order to perform the transition. Changing protection levels and transport methods is a dynamic operation that does not require downtime.

During a far sync instance outage, redo transport automatically fails over to using the alternate destination. Once the far sync instance is repaired and resumes operation, transport automatically switches back to the far sync instance and zero data loss protection is restored.

The characteristics of this approach include:

- No additional hardware or far sync instances to manage.
- Loss of zero data loss coverage during a far sync instance outage. Data protection level drops to UNSYNCHRONIZED with ASYNC transport until the Far sync instance can resume operation and the standby become fully synchronized.

Choosing a Far Sync Deployment Topology

All configurations for far sync instance high availability perform equally with regard to receiving and sending redo. The choice of configuration should be based on application tolerance to the maximum data loss (RPO) and application brownout period of the different failure scenarios.

- Far sync instances deployed on Oracle RAC provides the lowest brownout and best protection however has no coverage for cluster or site outage. The most critical applications are well served by a pair of Oracle RAC far sync instances configured as alternates for each other and deployed at different locations. This provides the most robust Far Sync high availability (instance, node, cluster, and site failure) protection.
- Alternate far sync instances in a non-RAC environment provide the ability to place each instance on separate physical database servers. This configuration provides protection by deploying the far sync instances in different sites. Applications where data protection is critical but where cost is an important consideration are best served by deploying a pair of single node far sync instances, each as an alternate for the other. There is, however, slightly increased application brownout and longer resynchronization time while transport transitions from one far sync instance to the other. There is also the potential for data loss should a second outage impact the primary database while transport transitions from one far sync instance to the other.
- Terminal standby alternate configurations require that the application accept that there is no zero data loss protection while the far sync instance is not available, but requires no additional hardware to implement. Applications that can tolerate increased data loss potential during a far sync instance outage, and where low cost is the main consideration, are best served by configuring the terminal standby as an alternate location using asynchronous redo transport. Use of the terminal standby as an alternate destination requires accepting that the configuration will run in asynchronous mode during the entire period required to resolve the far sync instance outage. The advantage of this approach is that it requires no additional hardware or software to deploy or manage. Applications that can tolerate increased data loss potential during a far sync instance outage and where low cost is the main consideration are best served by configuring the terminal standby as an alternate location using ASYNC redo transport.
- A Far Sync hub is an efficient way of consolidating far sync instances for multiple Data Guard configurations on a single physical host. Cloud deployments that include a zero data loss service level category can deploy a Far Sync hub to efficiently consolidate far sync instances for multiple zero data loss configuration on a single physical machine or cluster
- Applications where data protection is critical but where cost is an important consideration are best served by deploying a pair of single node far sync instances, each as an alternate for the other.

Far Sync Configuration Best Practices

The following are far sync configuration best practices that are necessary in addition to those best practices that apply to any synchronous redo transport destination.

- The network between the primary database and the far sync instance must:
 - Have round trip latency low enough so that the impact to response time and throughput of the primary database does not exceed business requirements. The degree of impact is very application specific and will require testing to validate. In general, experience shows that there is a higher likelihood of success if the round-trip latency is less than 5ms, though there are successful deployments at higher latencies.
 - Provide enough bandwidth between the primary database and the far sync instance to accommodate peak redo volumes, in addition to any other traffic sharing the network. Redo transport compression can be used to reduce network bandwidth requirements.
 - Ideally have redundant network links that also tolerate network component failure.
- Standard Oracle Data Guard network best practices, such as setting appropriate TCP send and receive buffer sizes equal to three times the bandwidth delay product. See [Configure Online Redo Logs Appropriately](#).
- Standby redo logs for a far sync instance should be placed on storage with sufficient IOPS (writes per second) capacity to exceed the I/O of the LGWR process on the primary database during peak activity, in addition to any IOPS from other activities. This is an important consideration. For example:
 - If the far sync instance has lower performing disks than the primary database, it will not be able to forward redo to remote destinations as fast as it is received, and an archive log gap may form.
 - In redo gap resolution scenarios, due to planned maintenance on the standby or network outages, for example, there will be additional I/O requests for gap resolution on top of peak redo coming in.
 - Lower performing disks at the far sync instance will delay acknowledgment to the primary database, increasing the total round-trip time between primary and standby databases and impacting application response time. This impact can be eliminated by using Fast Sync between the primary database and the far sync instance.
- The far sync instance should follow the same standby redo log best practices as the standby database. See [Configure Online Redo Logs Appropriately](#).
- The standby redo logs of an alternate far sync instance should be manually cleared before use to achieve the fastest return to synchronous transport when the alternate far sync is activated. For example:

```
ALTER DATABASE CLEAR LOGFILE GROUP 4, GROUP 5, GROUP 6;
```

- Oracle MAA performance testing shows that a small far sync instance SGA does not impact the performance of the far sync instance or the primary database. To conserve system resources, you can configure the minimum SGA required for Far Sync to function.
 - Set CPU_COUNT=4. Values of 1 or 2 are possible when neither compression nor encryption are not being used.
 - Reducing the CPU_COUNT during testing has no effect on the performance of the Far sync instance.

- Configure far sync instances for both the primary and standby databases to maintain zero data loss protection following role transitions. The second far sync instance configured in proximity to the standby database is idle until the standby becomes the primary database, enabling synchronous redo transport in the reverse direction.

Note that in a Data Guard Broker configuration, a switchover (planned role transition) cannot occur while in Maximum Availability mode unless the protection mode can be enforced from the target standby site. If the standby database does not have its own far sync instance it will have to be configured to ship asynchronous redo to the original primary database after the roles are reversed. This prevents a switchover from occurring unless the protection mode for the primary database is first dropped from Maximum Availability to Maximum Performance.
- Fast Sync yields a 4% to 12% primary database performance improvement compared to synchronous transport, depending on the network latency and the I/O speed of the far sync instance hardware.
- Provided CPU, I/O, and network requirements are met.
 - Placing the far sync instance on a virtual machine produces no reduction in performance over physical hardware configurations.
 - Multiple far sync instances servicing multiple Data Guard configurations can share the same physical server, cluster, or virtual machine.
- Note that archives may need to be managed on the far sync server.

Configuring the Active Data Guard Far Sync Architecture

The following topics walk you through an example of configuring an Active Data Guard Far Sync architecture.

Configuring the Far Sync Instances

The following examples show you how to add far sync instances to an Oracle Data Guard broker configuration.

The first step is to add a far sync standby instance that is independent or fault isolated from the primary database server, and where the network latency between the primary server and the far sync server is consistently low enough that application performance can tolerate it (for example, < 5 ms).

In the following example, far sync instance FS1 is created for the primary database, North_Sales.

```
DGMGRL> ADD FAR_SYNC FS1 AS CONNECT IDENTIFIER IS FS1.example.com;
Far Sync FS1 added
DGMGRL> ENABLE FAR_SYNC FS1;
Enabled.
DGMGRL> SHOW CONFIGURATION;
```

Configuration - DRSolution

```
Protection Mode: MaxPerformance
Members:
North_Sales - Primary database
FS1         - Far Sync
South_Sales - Physical standby database
```

Fast-Start Failover: DISABLED

Configuration Status:
SUCCESS

After a far sync instance has been added to the configuration, set up redo transport to support maximum availability mode and then upgrade the protection mode, as shown in the following example.

```
DGMGRL> EDIT DATABASE 'North_Sales' SET PROPERTY 'RedoRoutes' = '(LOCAL : FS1
SYNC)';
DGMGRL> EDIT FAR_SYNC 'FS1' SET PROPERTY 'RedoRoutes' = '(North_Sales :
South_Sales ASYNC)';
DGMGRL> EDIT CONFIGURATION SET PROTECTION MODE AS MaxAvailability;
DGMGRL> SHOW CONFIGURATION;
```

Configuration - DRSolution

```
Protection Mode: MaxAvailability
Members:
North_Sales - Primary database
FS1          - Far Sync
South_Sales - Physical standby database
```

Fast-Start Failover: DISABLED
Configuration Status:
SUCCESS

To ensure that maximum availability protection mode can be maintained when the remote standby database, South_Sales, becomes the primary database after a switchover or a failover, add a second far sync instance to the configuration so that South_Sales can send redo in synchronous mode, which in turn will send redo to the new terminal database, North_Sales, after the role transition.

The following example shows you how to add a second far sync instance (FS2) to the broker configuration.

```
DGMGRL> ADD FAR_SYNC FS2 AS CONNECT IDENTIFIER IS FS2.example.com;
Far Sync FS2 added
DGMGRL> EDIT FAR_SYNC 'FS2' SET PROPERTY 'RedoRoutes' = '(South_Sales :
North_Sales ASYNC)';
DGMGRL> ENABLE FAR_SYNC FS2;
Enabled.
DGMGRL> EDIT DATABASE 'South_Sales' SET PROPERTY 'RedoRoutes' = '(LOCAL : FS2
SYNC)';
DGMGRL> SHOW CONFIGURATION;
```

Configuration - DRSolution

```
Protection Mode: MaxAvailability
Members:
North_Sales - Primary database
FS1          - Far Sync
South_Sales - Physical standby database
FS2          - Far Sync (inactive)
```

```
Fast-Start Failover: DISABLED
Configuration Status:
SUCCESS
```

Setting Up HA Far Sync Instances

Alternate HA far sync instances are set up to provide high availability for the far sync instances you created for the primary and remote standby databases.

The following example shows you how to add a second far sync instance (FS1a) to the primary database's far sync instance (FS1) in the Oracle Data Guard broker configuration, so that if the primary far sync instance becomes unavailable, redo transport will use the alternate far sync instance.

```
DGMGRL> ADD FAR_SYNC FS1a AS CONNECT IDENTIFIER IS FS1a.example.com;
Far Sync FS1a added
DGMGRL> EDIT DATABASE 'North_Sales' SET PROPERTY 'RedoRoutes' = ' (LOCAL:(FS1
SYNC PRIORITY=1, FS1a SYNC PRIORITY=2))';
DGMGRL> EDIT FAR_SYNC 'FS1' SET PROPERTY 'RedoRoutes' = '(North_Sales :
South_Sales ASYNC)';
DGMGRL> EDIT FAR_SYNC 'FS1a' SET PROPERTY 'RedoRoutes' = '(North_Sales :
South_Sales ASYNC)';
DGMGRL> EDIT CONFIGURATION SET PROTECTION MODE AS MaxAvailability;
DGMGRL> SHOW CONFIGURATION;
```

Configuration - DRSolution

```
Protection Mode: MaxAvailability
Members:
North_Sales - Primary database
  FS1        - Far Sync
  FS1a       - Far Sync
South_Sales - Physical standby database
```

```
Fast-Start Failover: DISABLED
Configuration Status:
SUCCESS
```

After adding the alternate far sync instance on the primary, use the following example to add an alternate far sync instance (FS2a) on the standby.

```
DGMGRL> ADD FAR_SYNC FS2a AS CONNECT IDENTIFIER IS FS2a.example.com;
Far Sync FS2a added
DGMGRL> EDIT DATABASE 'South_Sales' SET PROPERTY 'RedoRoutes' = ' (LOCAL:(FS2
SYNC PRIORITY=1, FS2a SYNC PRIORITY=2))';
DGMGRL> EDIT FAR_SYNC 'FS2' SET PROPERTY 'RedoRoutes' = '(South_Sales :
North_Sales ASYNC)';
DGMGRL> EDIT FAR_SYNC 'FS2a' SET PROPERTY 'RedoRoutes' = '(South_Sales :
North_Sales ASYNC)';
DGMGRL> EDIT CONFIGURATION SET PROTECTION MODE AS MaxAvailability;
DGMGRL> SHOW CONFIGURATION;
```

Configuration - DRSolution

```
Protection Mode: MaxAvailability
Members:
  North_Sales - Primary database
    FS1 - Far Sync
    FS1a - Far Sync
  South_Sales - Physical standby database
    FS2 - Far Sync (inactive)
    FS2a - Far Sync (inactive)

Fast-Start Failover: DISABLED
Configuration Status:
SUCCESS
```

Configuring Far Sync Instances with Oracle RAC or Oracle Clusterware

If a far sync instance is deployed on a server or cluster with Oracle Clusterware (for example, in an Oracle Restart, Oracle Real Application Clusters (Oracle RAC), or Oracle RAC One Node installation), then use the SRVCTL utility to specify a default open mode of mount.

You can use a command such as the following:

```
srvctl modify database -d db_unique_name -startoption MOUNT
```

Encrypting a Database Using Data Guard and Fast Offline Encryption

Encrypting a database using Transparent Data Encryption (TDE) can be done more quickly, with minimal down time, and no extra space requirements, by using the standby database and offline encryption.

In this two-phase process, the standby database is encrypted offline, followed by a switchover, and then the offline encryption is repeated on the new standby database (formerly the primary).

In more recent Oracle releases online encryption is also available. Online encryption may fit the needs for some, but requires additional storage while a tablespace is converted, and online encryption can be a time-consuming process because each block is read and written to a new encrypted data file. With fast offline encryption, each data file is encrypted directly, in-place, on a mounted standby database.

Step 1: Configure Transparent Data Encryption (TDE)

There are a number of different TDE configuration options. Different Oracle releases have different requirements. It is strongly recommended that you review Introduction to Transparent Data Encryption in the *Oracle Database Advanced Security Guide* for your database release to understand the configuration options and implications of TDE.

Note

This process describes configuring a united, file-based keystore, which means that the wallets are stored on a file system, and all keys for all PDBs are stored in a single wallet.
For more complex configurations such as isolated PDBs, Oracle Key Vault (OKV), or Hardware Security Module (HSM), see [Using Transparent Data Encryption](#) in *Oracle Database Advanced Security Guide* for details.

The following are the basic parameters required to configure a united, file-based keystore. The parameters are configured on the primary and standby databases but may have different values.

Parameter	Configuration Best Practice
WALLET_ROOT	Starting in Oracle Database 18c, configuring the WALLET_ROOT database parameter is the best practice for specifying the root directory for all database wallets. For clustered databases, the location specified in WALLET_ROOT must be a shared location such as an ASM disk. <pre>ALTER SYSTEM SET WALLET_ROOT='+DATA/db_unique_name' SCOPE=SPFILE SID='*';</pre> Note WALLET_ROOT is a static parameter; the database must be restarted for the changes to take effect. TDE_CONFIGURATION cannot be set until the database is restarted with the WALLET_ROOT set. See WALLET_ROOT in the <i>Oracle Database Reference</i> for more details.
TDE_CONFIGURATION	The TDE_CONFIGURATION database parameter sets the type of keystore. TDE_CONFIGURATION is dynamic, but it can only be set after the database is restarted with WALLET_ROOT configured. <pre>ALTER SYSTEM SET TDE_CONFIGURATION='KESTORE_CONFIGURATION=FILE' SCOPE=SPFILE SID='*';</pre> See TDE_CONFIGURATION in the <i>Oracle Database Reference</i> for more details.

Parameter	Configuration Best Practice
TABLESPACE_ENCRYPTION	The database parameter TABLESPACE_ENCRYPTION is available in Oracle 19c (19.16). TABLESPACE_ENCRYPTION is an alternative to ENCRYPT_NEW_TABLESPACES, which specifies whether to encrypt new tablespaces when they are created. If both TABLESPACE_ENCRYPTION and ENCRYPT_NEW_TABLESPACES parameters are set, TABLESPACE_ENCRYPTION takes precedence. The values for TABLESPACE_ENCRYPTION are as follows: - AUTO_ENABLE - All newly created tablespaces will be encrypted. This is the Oracle Cloud default which cannot be overridden in Oracle 19c (19.16) and later. - MANUAL_ENABLE - Manually control whether tablespaces are encrypted with the ENCRYPTION clause on the CREATE statement. - DECRYPT_ONLY - No tablespaces will be encrypted. This setting is used in a hybrid Data Guard configuration where the on-premises database remains unencrypted while the cloud database is encrypted. <pre>ALTER SYSTEM SET TABLESPACE_ENCRYPTION=MANUAL_ENABLE SCOPE=BOTH SID='*';</pre> See TABLESPACE_ENCRYPTION in the <i>Oracle Database Reference</i> for more details.
ENCRYPT_NEW_TABLESPACES	Before Oracle Database 19c (19.16) the ENCRYPT_NEW_TABLESPACES parameter specifies whether to encrypt new tablespaces. The values for ENCRYPT_NEW_TABLESPACES are as follows: - CLOUD_ONLY - When a tablespace is created in the Oracle Cloud, it is transparently encrypted with the default encryption algorithm, whether or not the encryption clause is included. The default encryption algorithm can be changed as shown in Step 2, but AES128 is the default algorithm. - ALWAYS - The new tablespace is transparently encrypted whether or not the database is in the Oracle Cloud. - DDL - Manually control whether tablespaces are encrypted with the ENCRYPTION clause. See ENCRYPT_NEW_TABLESPACES in the <i>Oracle Database Reference</i> for more details.

The following table indicates which TDE parameters to configure based on your Oracle Database release.

Oracle Release	WALLET_ROOT and TDE_CONFIGURATION	TABLESPACE_ENCRYPTION	ENCRYPT_NEW_TABLESPACES
Oracle 19c (19.16) and later	Yes	Yes	No
Oracle 18c to 19c (19.15)	Yes	No	Yes

Step 2: Set the default encryption algorithm

The default encryption algorithm for TDE is AES128. In Oracle 21c and later releases, the algorithm can be set with the `TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM` parameter, but this setting must be configured **before** creating the wallet. Likewise, `_"tablespace_encryption_default_algorithm"` can be used in Oracle 19c and earlier with patch 30398099.

This setting determines the encryption algorithm used on new tablespaces for `TABLESPACE_ENCRYPTION=AUTO_ENABLE`, `ENCRYPT_NEW_TABLESPACES=ALWAYS`, and for offline encryption used in this process.

On the primary and standby databases issue:

```
-- for Oracle 21c and later
ALTER SYSTEM SET "tablespace_encryption_default_algorithm"='AES256'
scope=both;

-- for Oracle 19c and earlier
ALTER SYSTEM SET " _tablespace_encryption_default_algorithm"='AES256'
scope=both;
```

Step 3: Create the encryption wallet and set the master key

The TDE documentation is very thorough in describing creation of the wallet, or keystore, and setting the master encryption key on the primary database.

See [Configuring a Software Keystore and TDE Master Encryption Key for United Mode](#) in *Oracle Database Advanced Security Guide* for details.

Note that even if the primary database is to remain unencrypted after the standby is encrypted, in a hybrid Data Guard use case the master key **must** be set on the primary database. This key is used to encrypt data on the standby during redo apply, and after role transition. The key is be used to decrypt data from the encrypted primary cloud database after role transition.

Step 4: Copy the wallet files to the standby database environment

The standby database must have a copy of the encryption wallet and the auto-login keystore to perform encryption operations on the standby database. Copy the files from the primary database to the standby database accordingly.

From the location defined by `WALLET_ROOT`. If the target directory does not exist on the standby, it should be created manually.

Copy files to each node:

```
ASMCMD> cp +DATA/PRIMARY_ORACLE_UNQNAME/TDE/cwallet.sso /tmp
ASMCMD> cp +DATA/PRIMARY_ORACLE_UNQNAME/TDE/ewallet.p12 /tmp

<primary host>$ scp /tmp/cwallet.sso ewallet.p12 oracle@standby_host:/tmp

<standby host> ASMCMD> cp /tmp/cwallet.sso +DATA/STANDBY_db_unique_name/TDE/
<standby host> ASMCMD> cp /tmp/ewallet.p12 +DATA/STANDBY_db_unique_name/TDE/
```

Alternatively, the files can be copied directly from ASM to ASM.

```
ASMCMD>cp cwallet.sso sys/password@stbyhost1.+ASM1:+DATA/
STANDBY_ORACLE_UNQNAME/TDE/
```

```
ASMCMD>cp ewallet.p12 sys/password@stbyhost1.+ASM1:+DATA/  
STANDBY_ORACLE_UNQNAME/TDE/
```

Note

In Oracle Database 19c and later releases the `kscopy asmcmd` command can be used instead of `cp`. See My Oracle Support Doc ID 2951705.1 How to backup TDE keystore using ASMCMD for additional details.

Step 5: Verify Data Guard health

Before starting the offline encryption process, make sure that the standby database is current with the primary. Managed recovery must be stopped during the encryption process, and so ensuring that the standby database is current with the primary reduces the redo gap that must be applied after the encryption process.

On the primary or standby database, look up redo apply lag, then validate the standby database as shown in the following example. The Data Guard Broker command `VALIDATE DATABASE` lists potential configuration gaps. Address any gaps and verify that the status of "Ready for Switchover" and "Ready for Failover" are both YES.

```
DGMGRL> SHOW CONFIGURATION LAG
```

```
Configuration - dgconfig
```

```
Protection Mode: MaxPerformance  
Members:  
primary_db      - Primary database  
standby_db      - Physical standby database  
                  Transport Lag:      0 seconds (computed 1 second ago)  
                  Apply Lag:          0 seconds (computed 1 second ago)
```

```
Fast-Start Failover: Disabled
```

```
Configuration Status:  
SUCCESS (status updated 11 seconds ago)
```

```
DGMGRL> VALIDATE DATABASE <standby>
```

```
Database Role:      Physical standby database  
Primary Database:   primary  
  
Ready for Switchover: Yes  
Ready for Failover: Yes (Primary Running)
```

Step 6: Place the standby database in a mounted state with recovery stopped

Before you run the offline encryption process directly against the data files, the standby database must be mounted and recovery must be stopped. All instances of the standby can be used during the encryption process to encrypt multiple files simultaneously.

```
$ srvctl stop database -d standby -o immediate
```



```
$ srvctl start database -d standby -o mount
```

```
DGMGRL> EDIT DATABASE standby SET STATE=APPLY-OFF;
```

Redo transport services continue to ship redo to ensure that the archived logs are present at the standby database. This process maintains Recovery Point Objective (RPO) in the event of a failure during the encryption process.

For a database that is very active, the required number of archived logs could be significant, so make sure that there is sufficient space in the recovery area.

Step 7: Encrypt data files in-place and in parallel on the standby database

The encryption properties of TEMP tablespaces cannot be changed after creation. To encrypt a TEMP tablespace it must be created as encrypted.

To use an encrypted TEMP tablespace, create a new TEMP tablespace using the ENCRYPTION clause and make it the default temporary tablespace. Then drop the original TEMP tablespace.

```
SQL> CREATE TEMPORARY TABLESPACE TEMP_ENC ENCRYPTION ENCRYPT;
```

```
SQL> ALTER DATABASE DEFAULT TEMPORARY TABLESPACE TEMP_ENC;
```

The UNDO and TEMP metadata that is generated from sensitive data in an encrypted tablespace is already encrypted; therefore, encrypting UNDO and TEMP tablespaces is optional.

1. Make sure the standby database is mounted and the keystore is open.

```
SQL> select inst_id,database_role,open_mode from gv$database;
```

INST_ID	DATABASE_ROLE	OPEN_MODE
1	PHYSICAL STANDBY	MOUNTED
2	PHYSICAL STANDBY	MOUNTED

```
SQL> col WRL_PARAMETER format a40
SQL> set linesize 120 pagesize 9999
SQL> select * from gv$encryption_wallet;
```

INST_ID	WRL_TYPE	WRL_PARAMETER	STATUS
1	file	+DATA/ORACLE_UNQNAME/TDE	OPEN

2. Encrypt the data files.

The offline encryption command encrypts each data file with a single process; however, multiple data files can be encrypted in parallel with separate sessions. Each session can fully utilize a CPU core. It is recommended that each instance issues a number of sessions less than or equal to the number of cores on the host.

The following query can be used to generate a script to convert the data files. Break the script into multiple scripts and run each smaller script in an individual session. The most efficient process is to encrypt large files individually while placing multiple smaller files in a separate script.

Note

The seed database files do not need to be encrypted.

```

set lines 120
set pages 9999
spool encrypt.sql
select 'alter session set container='||pdb.name||';'||chr(10)||'alter
database datafile '||chr(39)||df.name||chr(39)||' encrypt;' COMMAND
from v$tablespace ts, v$datafile df, v$pdb pdb where ts.ts#=df.ts# and
ts.con_id=df.con_id and df.con_id=pdb.con_id and pdb.name <> 'PDB$SEED';

spool off
COMMAND
-----
-----
alter session set container=ORADBP11;
alter database datafile '+DATA/DB_UNIQUE_NAME/
E73F249E7030C3B8E0537B544664A065/DATAFILE/system.336.1113852973' encrypt;

alter session set container=ORADBP11;
alter database datafile '+DATA/DB_UNIQUE_NAME/
E73F249E7030C3B8E0537B544664A065/DATAFILE/sysaux.335.1113852973' encrypt;

alter session set container=ORADBP11;
alter database datafile '+DATA/DB_UNIQUE_NAME/
E73F249E7030C3B8E0537B544664A065/DATAFILE/undotbs1.337.1113852973' encrypt;

<...>

```

3. TEMP files can be encrypted by dropping and recreating them using the ENCRYPTION clause in the CREATE statement. Identify existing TEMP files using the V\$TEMPFILE view.
4. Validate that all data files are encrypted by querying V\$DATAFILE_HEADER.ENCRYPTED. After file encryption is completed, the ENCRYPTED column indicates whether the file is encrypted (YES) or not (NO). All data files except those belonging to the seed PDB should be encrypted.

Step 8: Restart redo apply and catch up on the standby database

After it is confirmed that all data files are encrypted, the standby database must apply all of the redo from the primary that was generated during the encryption process. The following are recommended ways to catch up redo on the standby database, depending on the amount of redo that needs to be applied.

- If the gap is small, restart managed recovery and apply the redo gap until the apply lag is 0.

On the primary or standby database run

```
DGMGRL> edit database standby set state=apply-on;
```

- If the encryption process took longer, and the primary database was very active, the gap might be large. It is often faster to use an incremental roll forward approach to copy only the blocks which have changed since apply was stopped.

That process is described in My Oracle Support note [How to Roll Forward a Standby Database Using Recover Database From Service \(Doc ID 2850185.1\)](#). Recovery is still needed when the roll forward is complete, but this process can shorten the time significantly to close large gaps.

Step 9: Perform a Data Guard switchover to begin encryption on the primary database

Until you are ready to encrypt the primary database, you can allow the unencrypted primary database to ship unencrypted redo to the standby, where it is encrypted by the standby indefinitely.

When you are ready to encrypt the primary database, and it is convenient to switch the database roles, perform a Data Guard switchover, making the encrypted standby database the new primary and the unencrypted primary database the new standby.

On the original primary database which is now the standby, repeat steps 5-8 to encrypt the data files and catch up on redo.

Step 11: Perform a Data Guard switchover (optional)

If, after both the standby and primary database are encrypted, you prefer to revert to the original primary-standby database roles, you can perform a Data Guard switchover to re-establish their original roles.

Tune and Troubleshoot Oracle Data Guard

When redo transport, redo apply, or role transitions are not meeting your expected requirements, use the following guidelines to help you tune and troubleshoot your deployment.

Overview of Oracle Data Guard Tuning and Troubleshooting

To get the best performance from your Oracle Data Guard configuration, use the following Oracle MAA best practices for monitoring, assessment, and performance tuning.

- Ensure that Oracle Database and Oracle Data Guard configuration best practices are in place.

The assumption when assessing and tuning is that all of the Oracle Database and Data Guard configuration best practices are already integrated in the environment. Evaluate the adherence to those best practices before doing any tuning.
- Assess and tune redo transport services

Oracle Data Guard automatically tunes redo transport to optimize performance. However, if you observe performance issues, you can monitor and tune redo transport services.

Asynchronous redo transport with Maximum Performance data protection mode is the default Oracle Data Guard configuration. Tuning asynchronous redo transport consists mainly of ensuring that the primary, standby, and network resources are sufficient for handling the workload, and that you monitor those resources for bottlenecks.

Synchronous redo transport does sacrifice some performance for zero data loss; however, using sound MAA recommended methods, you can monitor and assess the impact and distribute resources appropriately.
- Assess and tune redo apply

In most cases, the default Oracle settings result in satisfactory performance for media recovery when the standby is always up to date. However, as applications and databases increase in size and throughput, media recovery operations can benefit from additional tuning to further optimize recovery time or redo apply throughput on a standby database
- Assess and tune role transitions

With proper planning and implementation, Oracle Data Guard and Active Data Guard role transitions can effectively minimize downtime and ensure that the database environment is restored with minimal impact on the business. Performance tests using a physical standby database and Oracle Maximum Availability Architecture (MAA) best practices have shown that switchover and failover can be reduced to seconds.

Redo Transport Troubleshooting and Tuning

Oracle Data Guard redo transport performance is directly dependent on the performance of the primary and standby systems, the network that connects them, and the I/O subsystem.

For most Oracle Data Guard configurations, you should be able to achieve zero or minimal data loss by troubleshooting and tuning redo transport.

The guidance presented here assumes that the MAA configuration best practices are followed. As a prerequisite, ensure that the [Oracle Data Guard Configuration Best Practices](#) are implemented.

To improve transport holistically, leverage the data gathering and troubleshooting methodology described in the topics that follow, which guide you through gathering the necessary data to assess whether there is indeed a redo transport problem and what can be tuned to optimize redo transport throughput.

Gather Topology Information

Understanding the topology of the Oracle Data Guard configuration, and its relevance to Data Guard performance, helps eliminate infrastructure weaknesses that are often incorrectly attributed to the Data Guard architecture.

Oracle recommends that you outline the following high-level architecture information.

- Describe the primary and standby database system (number of nodes in Oracle RAC cluster, CPUs and memory per database node, storage I/O system)
- Describe network topology connecting the primary and standby systems
 - Network components/devices in between primary and standby
 - Network bandwidth and latency

For standby databases with hardware and configuration that is symmetric with the primary database, and a reliable, well-tuned network with sufficient bandwidth, the transport lag should be less than 10 seconds and in most cases less than 1 second.

Verify Transport Lag and Understand Redo Transport Configuration

To understand the difference between a transport lag and an apply lag, first recognize that a transport lag is the amount of redo that has not yet been received by the standby database. A transport lag impacts the Recovery Point Objective (RPO).

An apply lag indicates the amount of redo not yet applied to the standby and includes the transport lag. Therefore, an apply lag will be at least equal to the current transport lag and may be higher but never lower. An apply lag can indicate an impact on RPO, when there is a corresponding transport lag, and on Recovery Time Objective (RTO), absent a transport lag, because the unapplied redo is usually applied to the standby before the standby becomes a primary as a result of switchover or failover operation.

To determine if there is any lag on the standby database, and if this is a transport or apply lag, query the V\$DATAGUARD_STATS view.

```
SQL> select name,value,time_computed,datum_time from v$dataguard_stats where  
name='%Lag';
```

The DATUM_TIME column is the local time on the standby database when the datum used to compute the lag metric was received. Therefore, the lag reported is "as of" DATUM_TIME. It is assumed that the primary and standby maintain their connection, but in cases where the primary-standby connection is lost and DATUM_TIME does not change over multiple queries, the potential data loss cannot be calculated precisely, and should be assumed to be the difference between the DATUM_TIME from the query and current time on the standby database.

To obtain a histogram that shows the history of transport or apply lag values since the standby instance was last started, query the V\$STANDBY_EVENT_HISTOGRAM view.

```
SQL> select * from v$standby_event_histogram where name like '%lag' and count >0;
```

To evaluate the transport or apply lag over a time period, take a snapshot of V\$STANDBY_EVENT_HISTOGRAM at the beginning of the time period and compare that snapshot with one taken at the end of the time period.

```
SQL> col NAME format a10
SQL> select NAME, TIME, UNIT, COUNT, LAST_TIME_UPDATED from
V$STANDBY_EVENT_HISTOGRAM where
name like '%lag' and count >0 order by LAST_TIME_UPDATED;
```

NAME	TIME UNIT	COUNT	LAST_TIME_UPDATED
transport lag	41 seconds	3	01/05/2022 16:30:59
transport lag	245 seconds	1	01/05/2022 16:31:02
transport lag	365 seconds	2	01/05/2022 16:31:03
transport lag	451 seconds	2	01/05/2022 16:31:04

If you observe an unacceptable redo transport lag, continue this redo transport investigation with [Gather Information to Troubleshoot Transport Lag](#).

If you see no transport lag but an unacceptable redo apply lag, address the apply lag using the methodology in [Redo Apply Troubleshooting and Tuning](#).

Gather Information to Troubleshoot Transport Lag

Gather the following information and investigate the questions when an unacceptable redo transport lag is observed:

- When did the transport lag occur? Record the V\$DATAGUARD_STATS and V\$STANDBY_EVENT_HISTOGRAM data to show when the lag started and how the lag is changing over time.
- Does the transport lag occur during a certain time period, such as daily at 12 midnight for daily batch operations, monthly during large batch operation, or quarterly during quarter end?
- Check the LOG_ARCHIVE_DEST setting for each enabled destination, and verify whether redo COMPRESSION or ENCRYPTION is enabled. Overall redo transport throughput can be negatively impacted because redo must be compressed or encrypted before sending, and then uncompressed or unencrypted upon receiving it on the standby. Verify if that change was recent, and if you can test disabling these setting attributes.
- Check the Oracle Net settings to evaluate if Oracle Net encryption is enabled. If Oracle Net encryption is enabled, when was it enabled and at what level? Oracle Net encryption can slow down redo throughput significantly because redo is encrypted before sending and unencrypted upon receiving the redo on the standby. Optionally, disable or reduce encryption levels to see if the redo transport lag reduces. See [Configuring Oracle Database Native Network Encryption and Data Integrity](#).

Compare Redo Generation Rate History on the Primary

There are cases where the primary database redo generation rate is exceptionally high for a short period of time, such as during large batch jobs, data loads, data pump operations, create table as select, PDML operations, or end of month, quarter, or year batch updates.

Obtain the redo generation history from the primary database and compare that to when the redo transport or redo apply lag started. Check if the redo generation rate is exceptionally high because of additional workloads, such as adding new pluggable databases or new application services. By doing so, additional tuning may be required to accommodate this additional load.

As part of troubleshooting, gather the following information or address the following questions:

- Gather daily history of primary database's redo generation rate using this query.

```
SQL> select trunc(completion_time) as "DATE", count(*) as "LOG SWITCHES",
       round(sum(blocks*block_size)/1024/1024) as "REDO PER DAY (MB)"
from v$log
where dest_id=1
group by trunc(completion_time) order by 1;
```

- Gather the peak redo rate for each hour.

```
select begin_time,instance_number,metric_name,
       round(value/1024/1024) "Max Rate in MB"
from
(select to_char(begin_time,'DD-MON-YYYY HH24')
 begin_time,instance_number,metric_name,max(value) value
from dba_hist_sysmetric_history
where metric_name = 'Redo Generated Per Sec'
and trunc(begin_time) >=trunc(sysdate-7)
group by to_char(begin_time,'DD-MON-YYYY
HH24'),instance_number,metric_name)
order by 1,3;
```

Optionally change the '7' to some other value to change the number of days in the output.

The preferred query is `dba_hist_sysmetric_history` because it provides more precise information, but the retention is limited by AWR retention. If the dates you are looking for are not available from the previous query, gather per log redo generation rate using `v$log` instead.

```
SQL> alter session set nls_date_format='YYYY/MM/DD HH24:MI:SS';
SQL> select thread#,sequence#,blocks*block_size/1024/1024 MB,(next_time-
first_time)*86400 sec,
       blocks*block_size/1024/1024)/((next_time-first_time)*86400) "MB/s" from
v$log
where ((next_time-first_time)*86400<>0)
and first_time between to_date('2024/10/15 08:00:00','YYYY/MM/DD
HH24:MI:SS')
and to_date('2024/11/15 11:00:00','YYYY/MM/DD HH24:MI:SS')
and dest_id=1 order by first_time;
```

- Gather hourly snapshots of the redo generation rate from the Automatic Workload Repository (AWR) report 6 hours before the start of any redo or transport lag.

By default, Oracle Database automatically generates snapshots once every hour; however, you may want to manually create snapshots to capture statistics at times different from those of the automatically generated snapshots. To view information about an existing snapshot, use the `DBA_HIST_SNAPSHOT` view.

See *Creating Snapshots* in the *Oracle Database Performance Tuning Guide* for complete information about AWR and generating snapshots and AWR reports.

- Is this primary redo generation rate exceptionally high compared to prior history?
- If possible, determine the workload that corresponds to the high redo generation rate and evaluate whether it's transient or if it can be tuned.

For example, for large purge operations, consider truncate or drop partition operations to reduce the redo generation volumes.

Evaluate the Transport Network and Tune

Redo transport consists of the primary database instance background process sending redo to the standby database background process. You can evaluate if the network is optimized for Oracle Data Guard redo transport.

If asynchronous redo transport is configured, redo data is streamed to the standby in large packets asynchronously. To tune asynchronous redo transport over the network, you need to optimize a single process network throughput.

If synchronous redo transport is configured, each redo write must be acknowledged by the primary and standby databases before proceeding to the next redo write. You can optimize standby synchronous transport by using the `FASTSYNC` attribute as part of the `LOG_ARCHIVE_DEST` setting, but higher network latency (for example, more than 5 milliseconds) impacts overall redo transport throughput.

Before you continue, see [Assessing and Optimizing Network Performance](#) first to:

- Assess whether you have sufficient network bandwidth to support the primary's redo generation rate
- Determine optimal TCP socket buffer sizes to tune redo transport
- Tune operating system limits on socket buffer sizes to tune redo transport
- Determine optimal MTU setting for redo write size
- Tune MTU to increase network throughput for redo transport

If network configuration is tuned, evaluate if the transport lag (refer to [Verify Transport Lag and Understand Redo Transport Configuration](#)) is reducing to acceptable levels. If that's the case, you have met your goals and you can stop. Otherwise continue with the rest of the rest of tuning and troubleshooting section.

Gather and Monitor System Resources

Gather Oracle Linux OSWatcher or Oracle Exadata Exawatcher data to analyze system resources.

OSWatcher (oswbb) is a collection of UNIX shell scripts intended to collect and archive operating system and network metrics to aid support in diagnosing performance issues. As a best practice, you should install and run OSWatcher on every node that has a running Oracle

instance. In the case of a performance issue, Oracle support can use this data to help diagnose performance problems which may be outside the database.

You can download OSWatcher from [OSWatcher \(Doc ID 301137.1\)](#).

ExaWatcher is a utility that collects performance data on the storage servers and database servers on an Exadata system. The data collected includes operating system statistics, such as `iostat`, cell statistics (`cellsvstat`), and network statistics.

See *Using ExaWatcher Charts to Monitor Exadata Database Machine Performance* in the *Oracle Exadata Database Machine Maintenance Guide* for more information.

Tune to Meet Data Guard Resource Requirements

Redo transport can be impacted if:

- Primary or standby database is completely CPU bound
- The ASYNC process (TT0n) is CPU bound, that is, using over 70% of a CPU
- Primary or standby database I/O system is saturated
- Network topology can't support the redo generation rates

Evaluate whether the primary database system has:

- Sufficient CPU utilization for Log Writer Process (LGWR) to post foregrounds efficiently
- Sufficient I/O bandwidth so local log writes maintain low I/O latency during peak rates
- Network interfaces that can handle peak redo rate volumes combined with any other network activity across the same interface
- Automatic Workload Repository (AWR), Active Session History (ASH), and OSwatcher or Exawatcher data gathered from the primary database for tuning and troubleshooting

Evaluate whether the standby database system has:

- Sufficient CPU utilization for the remote file server (RFS), the Oracle Data Guard process that receives redo at the standby database, to efficiently write to standby redo logs
- Sufficient I/O bandwidth to enable local log writes to maintain low I/O latency during peak rates
- A network interface that can receive the peak redo rate volumes combined with any other network activity across the same interface
- AWR, ASH, and OSwatcher or Exawatcher data gathered from the standby database for tuning and troubleshooting

Note

The top issue encountered with the standby database is poor standby log write latency because of insufficient I/O bandwidth. This problem can be mitigated by using Data Guard Fast Sync.

If system configuration is tuned and the above resource constraints are removed, evaluate if the transport lag (refer to *Verify Transport Lag and Understand Redo Transport Configuration*) is reducing to acceptable levels. If that's the case, you have met your goals.

Advanced Troubleshooting: Determining Network Time with Asynchronous Redo Transport

Before you proceed, first see [Assessing and Optimizing Network Performance](#).

Given enough resources, especially network bandwidth, asynchronous redo transport can maintain pace with very high workloads. In cases where resources are constrained, asynchronous redo transport can begin to fall behind resulting in a growing transport lag on the standby database.

Asynchronous redo transport (ASYNC) transmits redo data asynchronously with respect to transaction commitment. A transaction can commit without waiting for an acknowledgment that the redo generated by that transaction was successfully transmitted to a remote standby database. With ASYNC transport, the primary database Log Writer Process (LGWR) continues to acknowledge commit success even if limited bandwidth prevents the redo of previous transactions from being sent to the standby database immediately (picture a sink filling with water faster than it can drain).

ASYNC uses a TT0n process to transmit redo directly from the log buffer of the primary database. If the TT0n process is unable to keep pace and the log buffer is recycled before the redo can be transmitted to the standby database, then the TT0n process automatically transitions to reading and sending from the online redo log file (ORL) on disk. Once TT0n transmission has caught up with current redo generation, it automatically transitions back to reading and sending directly from the log buffer.

In cases in which there are two or more log switches before the TT00 has completed sending the original ORL, the TT00 will still transition back to reading the contents of the current online log file. Any ORLs that were archived in between the original ORL and the current ORL are automatically transmitted using Oracle Data Guard's redo gap resolution process.

Sufficient resources, such as network bandwidth, CPU, memory, and log file I/O on both the primary and standby databases are critical to the performance of an asynchronous Data Guard configuration.

To determine which resource is constraining asynchronous transport, use `krsh stats` which can be enabled by setting event 16421 on both the primary and standby databases:

```
alter session set events '16421 trace name context forever, level 3';
```

This event is very lightweight and won't affect performance of the primary or standby database.

This dynamic event should be set on all primary and standby instances, and it will write statistics into the TT00 or remote file server (RFS) trace file when shipping for a given sequence has completed. Looking in the trace file, you will see the `krsh_end` stats at the beginning and end of the file. The stats at the end of the file will provide insight into where asynchronous shipping was spending time.

Note

Identify the async transport process and the tracefile:

```
SELECT DP.NAME, DP.PID OSPID, P.TRACEFILE FROM V$DATAGUARD_PROCESS DP,
V$PROCESS P
WHERE DP.PID=P.SPID AND DP.ROLE LIKE 'async%';
```

For example:

```
krshb_end: Begin stats dump for T-1.S-593
max number of buffers in use          10
Operation elapsed time (micro seconds) 474051333
File transfer time (micro seconds)     474051326
Network Statistics
LOG_ARCHIVE_DEST_2 : OCI REQUEST
  Total count : OCI REQUEST            2748
  Total time  : OCI REQUEST            81374
  Average time : OCI REQUEST           29
LOG_ARCHIVE_DEST_2 : NETWORK SEND
  Total count : NETWORK SEND           2748
  Total time  : NETWORK SEND          286554724
  Average time : NETWORK SEND         104277
  Total data buffers queued             9644
  Total data buffers completed          9644
  Total bytes written                   9885272064
  Total bytes completed synchronously   9885272064
  Average network send size (blocks)    7025
  Average network send buffers          3.51
  Average buffer turnaround time        240889
Throughput (MB/s)                19.89
Total network layer time                286636098
Percentage of time in network    60.47
Disk Statistics
  Total count : DISK READ              11531
  Total time  : DISK READ             12335132
  Average time : DISK READ             1069
  Read-ahead blocks                    14151680
  Log buffer blocks                    266915
  Disk stall blocks                    4888576
  Total count : BUFFER RELEASE         9643
  Total time  : BUFFER RELEASE         7229
  Average time : BUFFER RELEASE         0
Total disk layer time                  12342361
Percentage of time in disk layer  2.60
Data Guard Processing Statistics
  Total count : SLEEP                  198
  Total time  : SLEEP                 172351312
  Average time : SLEEP                 870461
Total DG layer time                    175072867
Percentage of time in DG layer    36.93
Remote Server-Side Network Statistics
LOG_ARCHIVE_DEST_2 : NETWORK GET
```

```

Total count : NETWORK GET          8242
Total bytes : NETWORK GET          9885272064
Total time : NETWORK GET           453233790
Average time : NETWORK GET         54990
Total server-side network layer time 453233790
Percentage of time in network        95.61
Remote Server-Side Disk Statistics
LOG_ARCHIVE_DEST_2 : DISK WRITE
Total count : DISK WRITE           9644
Total time : DISK WRITE            8731303
Average time : DISK WRITE          905
LOG_ARCHIVE_DEST_2 : DISK NOSTALL REAP
Total count : DISK NOSTALL REAP     9644
Total time : DISK NOSTALL REAP     579066
Average time : DISK NOSTALL REAP    60
LOG_ARCHIVE_DEST_2 : BUFFER GET
Total count : BUFFER GET           9644
Total time : BUFFER GET            3607
Average time : BUFFER GET           0
Total server-side disk layer time    9313976
Percentage of time in disk layer      1.96
Remote Server-Side Data Guard Processing Statistics
LOG_ARCHIVE_DEST_2 : PUBLISH RTA BOUNDARY
Total count : PUBLISH RTA BOUNDARY 8948
Total time : PUBLISH RTA BOUNDARY 3665841
Average time : PUBLISH RTA BOUNDARY 409
LOG_ARCHIVE_DEST_2 : VALIDATE BUFFER
Total count : VALIDATE BUFFER       9644
Total time : VALIDATE BUFFER       1403088
Average time : VALIDATE BUFFER      145
Total Server-Side DG layer time      11503560
Percentage of time in DG layer       2.43
krsb_end: End stats dump

```

The above output comes from a test run where a transport lag is just beginning to occur. You can observe a lag due to network congestion increase, and the time waiting on the network layer increases above 50%. If a transport lag is the result of either compression or encryption, the percentage of time spent in the Data Guard layer would become the majority.

When looking at krsb stats (Note all times in the output are in microseconds):

Start by looking where the most time is spent on the client side, be that network, disk or Data Guard.

Network Statistics

- 'Average time : NETWORK SEND' - Higher values here (more than 1 microsecond) could indicate network encryption or redo compression is enabled as it is taking longer to put the packets on the wire.
- Total bytes completed synchronously / Total bytes written - If the quotient is less than 90%, more TCP socket buffers would likely be helpful. This would require changing the `tcp_rmem` and `tcp_wmem` max values and restarting transport.

Disk Statistics

- Log buffer blocks - The best case scenario is when all the blocks are read from the log buffer (that is, memory) rather than disk (for example, read-ahead or disk stalls). If that is

not the case, increasing the size of the `log_buffer` parameter on the primary may help improve transport. Changing the `log_buffer` parameter requires a bounce of the database (Oracle RAC Rolling supported).

Note that a larger log buffer simply improves the *chance* that the next redo to be read by the `async` process will be in the log buffer. Under heavy, sustained workloads eventually reads are likely to go to disk, especially when there is another factor slowing down transport (such as encryption or latency).

- If there are a high percentage of 'Disk stall blocks', increase the value of `_log_archive_buffers` to 256. Changing this value requires a database restart to take effect (Oracle RAC Rolling supported).

Data Guard Statistics

- If most of the time spent is in Data Guard statistics and there is a transport lag, there may be a resource issue on the system as a whole (such as CPU or memory shortage). Review ExaWatcher or OSWatcher for the system for system bottlenecks.

To disable `krsb` stats set event 16421 to level 1:

```
alter session set events '16421 trace name context forever, level 1';
```

Disable Oracle Net Encryption for Redo Transport Only

Oracle Net encryption can have a negative impact on redo transport at higher redo generation rates. Encryption increases the CPU utilization of the send and receive processes which may result in a CPU bound process.

There may be cases where Oracle Net encryption is required for application client connections but it is acceptable to the business to disable encryption for redo transport. For example, when the primary and standby are in the same data center or the full transport path is protected by customer firewalls. In this case it is possible to disable Oracle Net encryption just for redo transport by changing the net service descriptor.

By including the `SECURITY` clause below in the descriptor for the net service name of the standby database Oracle Net encryption is disabled only for connections using this alias. The same setting should be added to the primary database entry on the standby database cluster.

```
(SECURITY=(ENCRYPTION_CLIENT=REJECTED))
```

Example:

```
net_service_name=
(DESCRIPTION=
  (ADDRESS_LIST=
    (ADDRESS=(PROTOCOL=tcp)(HOST=<standby scan listener>)(PORT=1521))
  )
  (CONNECT_DATA=
    (SERVICE_NAME=<standby database service>)
  )
  (SECURITY= (SECURITY=(ENCRYPTION_CLIENT=REJECTED)))
)
```

All other connections to or from the database are governed by the `sqlnet.ora` parameter settings. Only connections using this net service name will have Oracle Net encryption disabled.

Tuning and Troubleshooting Synchronous Redo Transport

Before you proceed, first see [Assessing and Optimizing Network Performance](#).

The following topics describe how to assess synchronous redo transport.

- [Understanding How Synchronous Transport Ensures Data Integrity](#)
- [Assessing Performance in a Synchronous Redo Transport Environment](#)
- [Why the Log File Sync Wait Event is Misleading](#)
- [Understanding What Causes Outliers](#)
- [Effects of Synchronous Redo Transport Remote Writes](#)
- [Example of Synchronous Redo Transport Performance Troubleshooting](#)

Understanding How Synchronous Transport Ensures Data Integrity

The following algorithms ensure data consistency in an Oracle Data Guard synchronous redo transport configuration.

- Log Writer Process (LGWR) redo write on the primary database online redo log and the Data Guard Network Services Server (NSS) redo write to standby redo log are identical.
- The Data Guard Managed Recovery Process (MRP) at the standby database cannot apply redo unless the redo has been written to the primary database online redo log, with the only exception being during a Data Guard failover operation (when the primary is gone).

In addition to shipping redo synchronously, NSS and LGWR exchange information regarding the safe redo block boundary that standby recovery can apply up to from its standby redo logs (SRLs). This prevents the standby from applying redo it may have received, but which the primary has not yet acknowledged as committed to its own online redo logs.

The possible failure scenarios include:

- If primary database LGWR cannot write to online redo log, then LGWR and the instance crash. Instance or crash recovery will recover to the last committed transaction in the online redo log and roll back any uncommitted transactions. The current log will be completed and archived.
- On the standby, the partial standby redo log completes with the correct value for the size to match the corresponding online redo log. If any redo blocks are missing from the standby redo log, those are shipped over (without reshipping the entire redo log).
- If the primary database crashes resulting in an automatic or manual zero data loss failover, then part of the Data Guard failover operation will do "terminal recovery" and read and recover the current standby redo log.

Once recovery finishes applying all of the redo in the standby redo logs, the new primary database comes up and archives the newly completed log group. All new and existing standby databases discard any redo in the online redo logs, flashback to a consistent system change number (SCN), and only apply the archives coming from the new primary database. Once again the Data Guard environment is in sync with the (new) primary database.

Assessing Performance in a Synchronous Redo Transport Environment

When assessing performance in an Oracle Data Guard synchronous redo transport environment (SYNC) it is important that you know how the different wait events relate to each other. The impact of enabling synchronous redo transport varies between applications.

To understand why, consider the following description of work the Log Writer Process (LGWR) performs when a commit is issued.

1. Foreground process posts LGWR for commit ("log file sync" starts). If there are concurrent commit requests queued, LGWR will batch all outstanding commit requests together resulting in a continuous strand of redo.
2. LGWR waits for CPU.
3. LGWR starts redo write ("redo write time" starts).
4. For Oracle RAC database, LGWR broadcasts the current write to other instances.
5. After preprocessing, if there is a SYNC standby, LGWR starts the remote write ("SYNC remote write" starts).
6. LGWR issues local write ("log file parallel write").
7. If there is a SYNC standby, LGWR waits for the remote write to complete.
8. After checking the I/O status, LGWR ends "redo write time / SYNC remote write".
9. For Oracle RAC database, LGWR waits for the broadcast ack.
10. LGWR updates the on-disk SCN.
11. LGWR posts the foregrounds.
12. Foregrounds wait for CPU.
13. Foregrounds ends "log file sync".

Use the following approaches to assess performance.

- For batch loads, the most important factor is to monitor the elapsed time, because most of these processes must be completed in a fixed period of time. The database workloads for these operations are very different than the normal OLTP workloads. For example, the size of the writes can be significantly larger, so using log file sync averages does not give you an accurate view or comparison.
- For OLTP workloads, monitor the volume of transactions per second (from Automatic Workload Repository (AWR)) and the redo rate (redo size per second) from the AWR report. This information gives you a clear picture of the application throughput and how it is impacted by enabling synchronous redo transport.

Why the Log File Sync Wait Event is Misleading

Typically, the "log file sync" wait event on the primary database is the first place administrators look when they want to assess the impact of enabling synchronous redo transport (SYNC).

If the average log file sync wait before enabling SYNC was 3ms, and after enabling SYNC was 6ms, then the assumption is that SYNC impacted performance by one hundred percent. Oracle does not recommend using log file sync wait times to measure the impact of SYNC because the averages can be very deceiving, and the actual impact of SYNC on response time and throughput may be much lower than the event indicates.

When a user session commits, the Log Writer Process (LGWR) will go through the process of getting on the CPU, submitting the I/O, waiting for the I/O to complete, and then getting back on the CPU to post foreground processes that the commit has completed. This whole time period is covered by the log file sync wait event. While LGWR is performing its work there are, in most cases, other sessions committing that must wait for LGWR to finish before processing their commits. The size and number of sessions waiting are determined by how many sessions an application has, and how frequently those sessions commit. This batching up of commits is generally referred to as *application concurrency*.

For example, assume that it normally takes 0.5ms to perform log writes (log file parallel write), 1ms to service commits (log file sync), and on average you are servicing 100 sessions for each commit. If there was an anomaly in the storage tier, and the log write I/O for one commit took 20ms to complete, then you could have up to 2,000 sessions waiting on log file sync, while there would only be 1 long wait attributed to log file parallel write. Having a large number of sessions waiting on one long outlier can greatly skew the log file sync averages.

The output from V\$EVENT_HISTOGRAM for the log file sync wait event for a particular period in time is shown in the following table.

Table 17-1 V\$EVENT_HISTOGRAM Output for the Log File Sync Wait Event

Milliseconds	Number of Waits	Percent of Total Waits
1	17610	21.83%
2	43670	54.14%
4	8394	10.41%
8	4072	5.05%
16	4344	5.39%
32	2109	2.61%
64	460	0.57%
128	6	0.01%

The output shows that 92% of the log file sync wait times are less than 8ms, with the vast majority less than 4ms (86%). Waits over 8ms are outliers and only make up 8% of wait times overall, but because of the number of sessions waiting on those outliers (because of batching of commits) the averages get skewed. The skewed averages are misleading when log file sync average waits times are used as a metric for assessing the impact of SYNC.

Understanding What Causes Outliers

Any disruption to the I/O on the primary or standby databases, or spikes in network latency, can cause high log file sync outliers with synchronous redo transport. You can see this effect when the standby system's I/O subsystem is inferior to that of the primary system.

Often administrators host multiple databases such as dev and test on standby systems, which can impair I/O response. It is important to monitor I/O using iostat to determine if the disks reach maximum IOPS, because this affects the performance of SYNC writes.

Frequent log switches are significant cause of outliers. Consider what occurs on the standby when a log switch on the primary occurs, as follows.

1. Remote file server (RFS) process on the standby must finish updates to the standby redo log header.
2. RFS then switches into a new standby redo log with additional header updates.
3. Switching logs forces a full checkpoint on the standby.

This causes all dirty buffers in the buffer cache to be written to disk, causing a spike in write I/O. In a non-symmetric configuration where the standby storage subsystem does not have the same performance as the primary database, this results in higher I/O latency.

4. The previous standby redo log must be archived, increasing both read and write I/O.

Effects of Synchronous Redo Transport Remote Writes

When you enable synchronous redo transport (SYNC), you introduce a remote write (remote file server (RFS) write to a standby redo log) in addition to the normal local write for commit processing.

This remote write, depending on network latency and remote I/O bandwidth, can make commit processing time increase. Because commit processing takes longer, you observe more sessions waiting on the Log Writer Process (LGWR) to finish its work and begin work on the commit request, that is, application concurrency has increased. You can observe increased application concurrency by analyzing database statistics and wait events.

Consider the example in the following table.

Table 17-2 Affect of Sync Transport Increasing Application Concurrency

SYNC	Redo Rate	Network Latency	TPS from AWR	log file sync average (ms)	log file parallel write average (ms)	RFS random I/O	SYNC remote write average (ms)	Redo write size (KB)	Redo writes
Defer	25MB	0	5,514.94	0.74	0.47	NA	NA	10.58	2,246,356
Yes	25MB	0	5,280.20	2.6	.51	.65	.95	20.50	989,791
Impact	0	-	-4%	+251%	+8.5%	NA	NA	+93.8%	-55.9%

In the above example, enabling SYNC reduced the number of redo writes, but increased the size of each redo write. Because the size of the redo write increased, you can expect the time spent doing the I/O (both local and remote) to increase. The log file sync wait time is higher because there is more work per wait.

However, at the application level, the impact on the transaction rate or the transaction response time might change very little as more sessions are serviced for each commit. This is why it is important to measure the impact of SYNC at the application level, and not depend entirely on database wait events. It is also a perfect example of why log file sync wait event is a misleading indicator of the actual impact SYNC has on the application.

Example of Synchronous Redo Transport Performance Troubleshooting

To look at synchronous redo transport performance, calculate the time spent for local redo writes latency, average redo write size for each write, and overall redo write latency, as shown here.

Use the following wait events to do the calculations.

- local redo write latency = 'log file parallel write'
- remote write latency = 'SYNC remote write'
- average redo write size per write = 'redo size' / 'redo writes'
- average commit latency seen by foregrounds = 'log file sync'

Statistics from an Automatic Work Repository (AWR) report on an Oracle database are provided in the following table. Synchronous redo transport (SYNC) was enabled to a local standby with a 1ms network latency to compare the performance impact to a baseline with SYNC disabled.

Table 17-3 Assessing Synchronous Redo Transport Performance with Oracle Database

Metric	Baseline (No SYNC)	SYNC	Impact
redo rate (MB/s)	25	25	no change
log file sync	0.68	4.60	+576%
log file parallel write average (ms)	0.57	0.62	+8.8%
TPS	7,814.92	6224.03	-20.3%
RFS random I/O	NA	2.89	NA
SYNC remote write average (ms)	NA	3.45	NA
redo writes	2,312,366	897,751	-61.2%
redo write size (KB)	10.58	20.50	+93.8%

In the above example observe that log file sync waits averages increased dramatically after enabling SYNC. While the local writes remained fairly constant, the biggest factor in increasing log file sync was the addition of the SYNC remote write. Of the SYNC remote write the network latency is zero, so focusing on the remote write into the standby redo log shows an average time of 2.89ms. This is an immediate red flag given that the primary and standby were using the same hardware, and the SYNC remote write average time should be similar to the primary's log file parallel write average time.

In the above example, the standby redo logs have multiple members, and they are placed in a slower performing disk group. After reducing the standby redo logs to a single member, and placing them in a fast disk group, you can see results such as those shown in the following table.

Table 17-4 SYNC Performance After Reducing Standby Redo Logs to a Single Member and Placing on a Fast Disk Group

Metric	Baseline (No SYNC)	SYNC	Impact
redo rate (MB/s)	25	25	no change
log file sync	0.67	1.60	+139%
log file parallel write	0.51	0.63	+23.5%
TPS	7714.36	7458.08	-3.3%
RFS random I/O	NA	.89	NA
SYNC remote write average (ms)	NA	1.45	NA
redo writes	2,364,388	996,532	-57.9%
redo write size (KB)	10.61	20.32	+91.5%

Redo Apply Troubleshooting and Tuning

Most Oracle Data Guard configurations should be able to minimize apply lag by troubleshooting and tuning redo apply. Redo apply performance is directly dependent on the performance of the standby systems.

The guidance presented here assumes that the MAA configuration best practices are followed. As a prerequisites, ensure that the [Oracle Data Guard Configuration Best Practices](#) are implemented.

To improve apply performance holistically, leverage the data gathering and troubleshooting methodology described in the topics below.

Understanding Redo Apply and Redo Apply Performance Expectations

Standby database recovery is the process of replaying all DML and DDL operations. The high level process is:

1. Redo is received from the primary database and written into standby redo logs (SRLs). When the database is an Oracle RAC database, each thread (instance) is stored in it's assigned SRLs.
2. The log merger process, sometimes known as the recovery coordinator, merges the threads of redo and places the resulting change vectors into memory buffers.
3. Recovery worker processes identify which data blocks are required and read them into the buffer cache if they are not already present. Then the worker processes apply the change vectors to the blocks in the buffer cache.
4. At checkpoint time, database writer processes write the validated buffer changes to data files, advancing the database's checkpoint time stamp, called the System Commit Number (SCN). Checkpoint can be the most extensive I/O load in the recovery process.

Redo Apply Performance Expectations

Performance, and the resulting apply rate, mainly depend on the type of workload that is being recovered and the system resources allocated to and available for recovery.

Oracle recommends that the primary and standby database systems are symmetric, including equivalent I/O subsystems, memory, and CPU resources. The primary reason for this recommendation is so that the application performs at the same level, no matter which database is the primary database; however, redo apply performance also benefits greatly from symmetric primary and standby databases. Features such as data protection (DB_BLOCK_CHECKING, DB_BLOCK_CHECKSUM, DB_LOST_WRITE_PROTECT) require CPU and I/O resources, as does reporting on the standby database using Oracle Active Data Guard.

For the most part, redo apply performance should keep up with the redo generation rates, resulting in near zero apply lag with system resources are symmetric. During peak workloads, there may be a slight redo apply gap which should naturally reduce to near zero once workloads return to normal levels.

OLTP Workloads

Recovering Online Transaction Processing (OLTP) workloads can be very I/O intensive because an OLTP workload performs small changes to many different blocks. This results in large numbers of small random block reads into the buffer cache during recovery. Subsequently, the database writers run large batches of write I/Os to maintain the buffer cache and to checkpoint the database periodically. Therefore, recovery of OLTP workloads requires the storage subsystem to handle a high number of I/Os Per Second (IOPS) in order to achieve optimal rates. This is another reason for recommending that the primary and standby database systems are symmetric.

Recovery testing of OLTP workloads, generated by swingbench on Oracle Exadata Database Machine quarter rack systems with no resource bottlenecks, achieved approximately 150 MB/sec apply rates. Rates of 200+ MB/s with single instance redo apply have been observed

by customers on larger Exadata systems. These rates are more challenging to achieve in non-Exadata systems since the I/O and network throughput are lower.

Batch Workloads

In contrast to OLTP workload recovery, recovering batch workloads is more efficient because batch workloads consist of large sequential reads and writes. A lot more redo changes are occurring while reading and modifying significantly fewer data blocks, resulting in much faster redo apply rates than OLTP workloads. In addition, batch direct load operation recovery optimizations result in greater efficiency and even higher recovery rates.

Using batch load or parallel DML (PDML) workloads with no impeding system resource bottleneck, internal redo apply testing on small Exadata Database Machine quarter rack systems resulted in approximately 200-300 MB/sec apply rates. Customers have observed 600+ MB/sec apply rates with single instance redo apply for their batch workloads for larger Exadata systems. These rates can be achieved by non-Exadata systems, but system resource capacity and scalable network and I/O subsystems are required to handle these demanding workloads.

Mixed Workloads

The difference between OLTP and batch recovery performance profiles and different system shapes explains why applications with variation in their mixtures of OLTP and batch workloads can have different recovery rates at a standby database, even if the primary database redo generation rates are similar. Customers have achieved 100-1100 MB/sec redo apply rates with various mixed workloads for various Exadata systems. These rates can be achieved by non-Exadata systems, but system resource capacity and scalable database compute, network, and I/O subsystems are required to handle these demanding workloads. These extreme redo apply rates are rarely achieved on non-Exadata systems.

Catch Up Redo Apply Performance Expectations

Compared to real-time redo apply, redo apply during a "catch up" period may require even more system resources. If there is a large redo gap, see [Addressing a Very Large Redo Apply Gap](#) for recommendations.

Verify Apply Lag

Recovery performance can vary with the workload type and the redo generation rate of the primary database. A lower apply rate does not necessarily indicate a recovery performance issue. However, a persistent or increasing apply lag, without an accompanying transport lag, is the best indication of a recovery performance bottleneck.

To identify and quantify apply lags and transport lags, query the V\$DATAGUARD_STATS view in the standby database.

```
SQL> select name, value, time_computed, datum_time from v$dataguard_stats
where name='%lag';
```

The DATUM_TIME column is the local time on the standby database when the datum used to compute the metric was received. The lag metrics are computed based on data that is periodically received from the primary database. An unchanging value in this column across multiple queries indicates that the standby database is not receiving data from the primary database. The potential data loss in this scenario would be from the last datum time from V\$DATAGUARD_STATS to the current time on the standby.

To obtain a histogram that shows the history of transport or apply lag values since the standby instance was last started, query the V\$STANDBY_EVENT_HISTOGRAM view.

```
SQL> select * from v$standby_event_histogram where name like '%lag' and
count >0;
```

To evaluate the transport or apply lag over a time period, take a snapshot of V\$STANDBY_EVENT_HISTOGRAM in the standby database at the beginning of the time period, and compare that snapshot with one taken at the end of the time period.

```
SQL> col NAME format a10
SQL> select NAME, TIME, UNIT, COUNT, LAST_TIME_UPDATED from
V$STANDBY_EVENT_HISTOGRAM
where name like '%lag' and count >0 order by LAST_TIME_UPDATED;
```

Example output:

NAME	TIME	UNIT	COUNT	LAST_TIME_UPDATED
-----	-----	-----	-----	-----
apply lag	23	seconds	3	02/05/2022 16:30:59
apply lag	135	seconds	1	02/05/2022 16:31:02
apply lag	173	seconds	2	02/05/2022 16:32:03
apply lag	295	seconds	2	02/05/2022 16:34:04

A transport lag can cause an apply lag. If a high apply lag is observed with a near zero transport lag, continue with this redo apply investigation in [Gather Information](#).

If a high transport lag is observed, first address the transport lag, using the methodology in [Redo Transport Troubleshooting and Tuning](#).

Gather Information

Gather the following information when an unacceptable apply lag is occurring:

- When did the apply lag occur?

Record the V\$DATAGUARD_STATS and V\$STANDBY_EVENT_HISTOGRAM data every 15 to 30 minutes to identify when the lag started and how lag changed over time in the last 24 hours.

```
SQL>select name, value, time_computed, datum_time from v$dataguard_stats
where name='%lag';
```

```
SQL>select * from v$standby_event_histogram where name like '%lag' and
count >0;
```

- Does the apply lag occur at certain time period, such as daily at 12 midnight for daily batch operations, monthly during large batch operation, quarterly during quarter end?
- Gather data from the standby Automatic Work Repository (AWR) report V\$RECOVERY_PROGRESS, and take multiple standby AWR snapshots at 30 minute intervals before and during the apply lag.

See [How to Generate AWRs in Active Data Guard Standby Databases \(Doc ID 2409808.1\)](#).

For example:

```
SQL> set lines 120 pages 99
SQL> alter session set nls_date_format='YYYY/MM/DD HH24:MI:SS';
SQL> select START_TIME, ITEM, SOFAR, UNITS from gv$recovery_progress;
```

Sample output:

START_TIME	ITEM	SOFAR	UNITS
2022/02/28 23:02:36	Log Files	8	Files
2022/02/28 23:02:36	Active Apply Rate	54385	KB/sec
2022/02/28 23:02:36	Average Apply Rate	12753	KB/sec
2022/02/28 23:02:36	Maximum Apply Rate	65977	KB/sec
2022/02/28 23:02:36	Redo Applied	2092	Megabytes
2022/02/28 23:02:36	Last Applied Redo	0	SCN+Time
2022/02/28 23:02:36	Active Time	41	Seconds
2022/02/28 23:02:36	Apply Time per Log	1	Seconds
2022/02/28 23:02:36	Checkpoint Time per Log	0	Seconds
2022/02/28 23:02:36	Elapsed Time	168	Seconds
2022/02/28 23:02:36	Standby Apply Lag	2	Seconds

The simplest way to determine application throughput in terms of redo volume is to collect Automatic Workload Repository (AWR) reports on the primary database during normal and peak workloads, and determine the number of bytes per second of redo data the production database is producing. Then compare the speed at which redo is being generated with the Active Apply Rate columns in the V\$RECOVERY_PROGRESS view to determine if the standby database is able to maintain the pace.

If the apply lag is above your expectations, then evaluate redo apply performance by querying the V\$RECOVERY_PROGRESS view. This view contains the columns described in the following table.

The most useful statistic is the Active Apply rate because the Average Apply Rate includes idle time spent waiting for redo to arrive making it less indicative of apply performance.

Table 17-5 V\$RECOVERY_PROGRESS View Columns

Column	Description
Average Apply Rate	Redo Applied / Elapsed Time includes time spent actively applying redo and time spent waiting for redo to arrive
Active Apply Rate	Redo Applied / Active Time is a moving average over the last 3 minutes, and the rate does not include time spent waiting for redo to arrive
Maximum Apply Rate	Redo Applied / Active Time is peak measured throughput or maximum rate achieved over a moving average over last 3 minutes; rate does not include time spent waiting for redo to arrive
Redo Applied	Total amount of data in bytes that has been applied

Table 17-5 (Cont.) V\$RECOVERY_PROGRESS View Columns

Column	Description
Last Applied Redo	System change number (SCN) and time stamp of last redo applied. This is the time as stored in the redo stream, so it can be used to compare where the standby database is relative to the primary.
Apply Time per Log	Average time spent actively applying redo in a log file.
Checkpoint Time per Log	Average time spent for a log boundary checkpoint.
Active Time	Total duration applying the redo, but not waiting for redo
Elapsed Time	Total duration applying the redo, including waiting for redo
Standby Apply Lag	Number of seconds that redo apply has not been applied for. Possible standby is behind the primary.
Log Files	Number of log files applied so far.

Active Session History

In cases where standby AWR is not available, or the standby database is not in open read-only mode, the top waits can be gathered using the V\$ACTIVE_SESSION_HISTORY view. Standby AWR is strongly recommended due to the additional information and detail provided but these queries are useful in some cases.

To select the top 10 waits over the last 30 minutes (replace 30 with some other number of minutes ago from current time):

```
select * from (
select a.event_id, e.name, sum(a.time_waited) total_time_waited
from v$active_session_history a, v$event_name e
where a.event_id = e.event_id and a.SAMPLE_TIME >= (sysdate - 30 / (24 * 60))
group by a.event_id, e.name order by 3 desc)
where rownum < 11;
```

To select the waits between two timestamps (example shows a 3 hour period between 2021/01/01 00:00:00 and 2021/01/01 03:00:00) :

```
select * from (
select a.event_id, e.name, sum(a.time_waited) total_time_waited
from v$active_session_history a, v$event_name e
where a.event_id = e.event_id
and a.SAMPLE_TIME
between to_date('2021/01/01 00:00:00', 'YYYY/MM/DD HH24:MI:SS') and
to_date('2021/01/01 03:00:00', 'YYYY/MM/DD HH24:MI:SS')
group by a.event_id, e.name
order by 3 desc)
where rownum < 11
/
```


Compare Redo Generation Rate History on the Primary

There are cases where the primary database's redo generation rate is exceptionally high for a small period of time, such as during large batch jobs, data loads, data pump operations, create table as select or PDML operations or end of month, quarter or year batch updates.

Obtain the redo generation history from the primary database and compare that to when the redo transport or redo apply lag started. Check if the redo generation rate is exceptionally high due to additional workloads, such as adding new pluggable databases (PDBs) or new application services. Additional tuning may be required to accommodate this additional load.

As part of troubleshooting, gather the following information or address the following questions:

- Gather daily history of the primary database's redo generation rate using this query.

```
SQL> select trunc(completion_time) as "DATE", count(*) as "LOG SWITCHES",  
round(sum(blocks*block_size)/1024/1024) as "REDO PER DAY (MB)"  
from v$sarchived_log  
where dest_id=1  
group by trunc(completion_time) order by 1;
```

- Gather the per log redo generation rate, starting 6 hours before the start of any redo or transport lag.

```
SQL> alter session set nls_date_format='YYYY/MM/DD HH24:MI:SS';  
SQL> select thread#, sequence#, blocks*block_size/1024/1024 MB, (next_time-  
first_time)*86400 sec, blocks*block_size/1024/1024)/((next_time-  
first_time)*86400) "MB/s" from v$sarchived_log  
where ((next_time-first_time)*86400<>0)  
and first_time between to_date('2015/01/15 08:00:00', 'YYYY/MM/DD  
HH24:MI:SS')  
and to_date('2015/01/15 11:00:00', 'YYYY/MM/DD HH24:MI:SS')  
and dest_id=1 order by first_time;
```

- Is this primary redo generation rate exceptionally high compared to prior history?
- If possible, determine the workload that corresponds to the high redo generation rate, and evaluate if it's transient or if it can be tuned.

For example, for large purge operations, consider truncate or drop partition operations to reduce the redo generation volumes.

Tune Single Instance Redo Apply

Single instance redo apply (SIRA) tuning is an iterative process and a mandatory prerequisite before even evaluating multi-instance redo apply (MIRA). The iterative process consists of

1. Evaluating and addressing system resource bottlenecks
2. Tuning based on top standby database wait events

Evaluate System Resource Bottlenecks

First, evaluate system resources such as CPU utilization and I/O subsystem. Use utilities such as top and iostat or statistics from OSwatcher or ExaWatcher to determine if there is

contention for those resources. Addressing any resource bottlenecks to free up resources required for redo apply can improve apply performance.

Redo apply can be impacted if:

- The managed recovery node is completely CPU bound
- The standby database's I/O system is saturated
- The standby database SGA, specifically the buffer cache, is not at least the same size (or larger) than that on the primary database

For optimal recovery performance the standby database system requires:

- Sufficient CPU utilization for Recovery Coordinator (PR00) and recovery workers (PRnn)
- Sufficient I/O bandwidth to maintain low I/O latency during peak rates
- A network interface that can receive the peak redo rate volumes, in addition to any other network activity across the same interface
- Sufficient memory to accommodate a symmetric SGA and buffer cache; the size of the log buffer and buffer cache generally have the biggest impact on redo apply performance

What to gather and how?

- Gather standby Automatic Work Repository (AWR) reports with intervals of 30 minutes or less.

See *Managing Automatic Workload Repository in Active Data Guard Standby Databases* in *Oracle Database Performance Tuning Guide*

- Gather Active Session History (ASH) data for more real time granular waits.

See *Generating Active Session History Reports* in *Oracle Database Performance Tuning Guide*

- Gather Oracle Linux OSwatcher or Oracle Exadata ExaWatcher data to analyze system resources.

For Exadata systems, see *Using ExaWatcher Charts to Monitor Exadata Database Machine Performance* in *Oracle Exadata Database Machine Maintenance Guide*

- Gather top process information to check if the recovery coordinator (PR00) is CPU bound by using top or ps commands.

Some common indicators and causes of resource bottlenecks include:

- Low CPU idle time may indicate the system is CPU bound
- Long disk or flash service times or high IOPS may indicate I/O contention or saturation
- Undersized systems and shared systems with many active databases may cause contention for these resources
- Reporting workloads in an Active Data Guard standby can also cause contention

Tune Redo Apply by Evaluating Database Wait Events

Once you have verified that there are no system resource bottlenecks, it is time to assess standby database wait events by looking at the standby Automatic Work Repository (AWR) reports.

Before assessing database wait events, it is important to understand where the waits occur during the process flow involved in recovery.

1. Redo is received on the standby by the Remote File Server (RFS) process.

The RFS process writes newly received redo for each thread into the current standby redo log for that thread. The RFS write operation is tracked by the rfs random I/O wait event.

2. Once redo has been written, the recovery coordinator process (pr00) reads the redo from the standby redo logs (or archived logs) for each thread.

This read I/O is tracked by the log file sequential read wait event.

3. The recovery coordinator then merges redo from all threads together and places the redo into memory buffers for the recovery workers.

The wait events for writing and reading into recovery memory buffers is tracked by the parallel recovery read buffer free and parallel recovery change buffer free wait events.

4. The recovery processes retrieve redo or change vectors from the memory buffers and begin the process of applying the changes to data blocks.

First the recovery workers determine which data blocks need to be recovered and reads those into the buffer cache if it's not already present.

This read I/O by the recovery workers is tracked by the recovery read wait event.

5. When a log is switched on the primary for any thread, the standby coordinates a switch of the standby redo log for that thread at the same time.

In earlier versions a log switch on a standby forces a full checkpoint, which results in flushing all dirty buffers from the buffer cache out to the data files on the standby. Starting with Oracle Database 18c, checkpoints also occur at regular time intervals, thus amortizing checkpoint I/O across all phases.

During checkpoint, multiple database writer processes (DBWR) write the data file blocks down to the data files, with its write time tracked by the db file parallel write wait event. The total time for the checkpoint to complete is covered by the checkpoint complete wait event.

During the apply phase it is normal to observe that the recovery coordinator process (pr00) has high utilization on a single CPU, while during the checkpoint phase there is an increase in DB writer processes (dbwrn) CPU utilization indicating increased write I/O to the data files.

The following table provides a description as well as tuning advice for wait events involved in the recovery process.

Table 17-6 Recovery Process Wait Events

Column	Description	Tuning Recommendations
Logfile sequential read	The parallel recovery coordinator is waiting on I/O from the online redo log, standby redo log, or the archived redo log.	Tune or increase the I/O bandwidth for the ASM disk group or storage subsystem where the archive logs, standby redo logs, or online redo logs reside.
Parallel recovery read buffer free	This event indicates that all read buffers are being used by workers, and usually indicates that the recovery workers lag behind the coordinator.	Increase <code>_log_read_buffers</code> to max 256
Parallel recovery change buffer free	The parallel recovery coordinator is waiting for a buffer to be released by a recovery worker. Again, this is a sign the recovery workers are behind the coordinator.	Tune or increase the I/O bandwidth for the ASM disk group or storage subsystem where data files reside.

Table 17-6 (Cont.) Recovery Process Wait Events

Column	Description	Tuning Recommendations
Data file init write	The parallel recovery coordinator is waiting for a file resize to finish, as would occur with file auto extend.	This is a non-tunable event, but evaluate your primary database workload to discover why there are so many data file resize operations. Optionally, use a larger NEXT size when AUTOEXTEND is enabled
Parallel recovery control message reply	The parallel recovery coordinator is waiting for all recovery workers to respond to a synchronous control message.	N/A. This is an idle event.
Parallel recovery slave next change	The parallel recovery worker is waiting for a change to be shipped from the coordinator. This is in essence an idle event for the recovery worker. To determine the amount of CPU a recovery worker is using, divide the time spent in this event by the number of workers started, and subtract that value from the total elapsed time.	N/A. This is an idle event.
DB File Sequential Read	A parallel recovery worker (or serial recovery process) is waiting for a batch of synchronous data block reads to complete.	Tune or increase the I/O bandwidth for the ASM disk group or storage subsystem where data files reside.
Checkpoint completed	Recovery is waiting for checkpoint to complete, and redo apply is not applying any changes currently.	Tune or increase the I/O bandwidth for the ASM disk group or storage subsystem where data files reside. Also, increase the number of db_writer_processes until the checkpoint completed wait event is lower than the db file parallel write wait event. Also consider increasing the online log file size on the primary and standby to decrease the number of full checkpoints at log switch boundaries.
Recovery read	A parallel recovery worker is waiting for a batched data block I/O.	Tune or increase the I/O bandwidth for the ASM disk group where data files reside.

Table 17-6 (Cont.) Recovery Process Wait Events

Column	Description	Tuning Recommendations
Recovery apply pending and/or recovery receive buffer free (MIRA)	Recovery apply pending = the time the logmerger process waited (in centiseconds) for apply workers to apply all pending changes up to a certain SCN. Recovery receive buffer free = the time (in centiseconds) spent by the receiver process on the instance waiting for apply workers to apply changes from received buffers so that they can be freed for the next change.	Increase <code>_mira_num_local_buffers</code> and <code>_mira_num_receive_buffers</code>. Note that these parameters use space from the shared pool equal to $(_mira_num_local_buffers * 2) + (_mira_num_receive_buffers * instances - 1)$ in each instance of the standby database. For example: In a 2-node RAC standby database when <code>_mira_num_local_buffers = 100</code> and <code>_mira_num_receive_buffers = 100</code> the total SGA space used by these parameters is $(100 * 2) + (100 * 1) = 300\text{MB}$ of the SGA.

Note

The default for `_mira_num_local_buffers` and `_mira_num_receive_buffers` is 25.

See [How to Generate AWRs in Active Data Guard Standby Databases \(Doc ID 2409808.1\)](#) for more information about generating AWRs on the standby database.

Enable Multi-Instance Redo Apply if Required

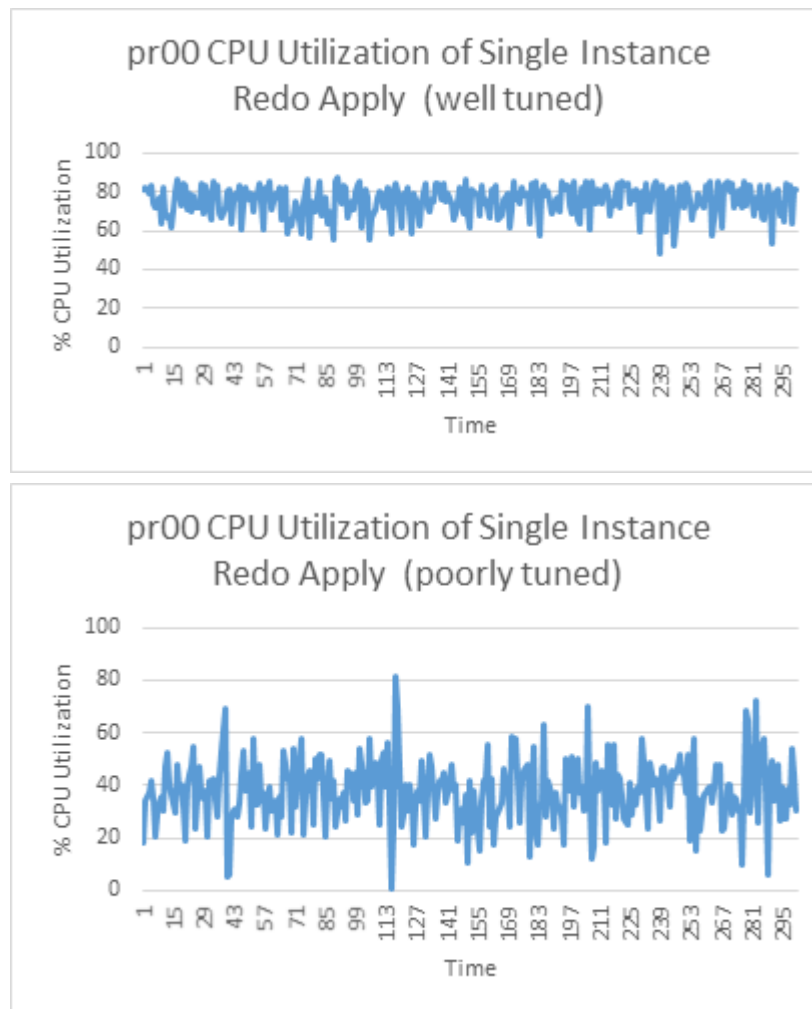
Multi-instance redo apply (MIRA) has the potential to improve redo apply by running multiple recovery coordinators and redo apply (worker) processes across Oracle RAC database instances of the standby database. MIRA is optimized for later Oracle Database releases, and the redo apply benefits vary based on workloads.

Prerequisites for Considering MIRA

- Single-instance redo apply (SIRA) has been completely tuned and is not I/O bound.
- Recovery coordinator (PR00) is CPU bound.

Examine the CPU utilization of the recovery coordinator/log merger process `ora_pr00_<SID>` over a period of an hour. If the coordinator process has a CPU utilization % of over 70% for a majority of that time, this may be the bottleneck, and MIRA may improve recovery performance.

Shown here are two examples of output from the `top` command showing the CPU utilization of the `pr00`.



If the recovery coordinator CPU utilization is largely below 70% with only a few short spikes, it is not CPU bound, and there is likely a resource issue or some additional tuning that will improve performance. If the recovery coordinator is not CPU bound, return to tuning SIRA.

- Most MIRA optimizations are implemented in Oracle Database 19c and are not available in earlier database releases. In fact, Oracle recommends the database release be no earlier than Oracle Database 19.13 because it includes some important fixes, including 29924147, 31290017, 31047740, 31326320, 30559129, 31538891, 29785544, 29715220, 29845691, 30421009, 30412188, 30361070, 32486528, 33821145 and 28389153.
- All Oracle Exadata Database Machine systems based either on InfiniBand network fabric or on RDMA over Converged Ethernet (RoCE) network fabric require an additional step on the primary database, as shown in this table.

Table 17-7 Oracle Exadata Database Machine prerequisites to enable MIRA

Exadata System	Database Release	Steps
Exadata Storage cells with persistent memory (XRMEM)	19.13 and higher	No additional steps
Without XRMEM	19.13 and higher	Set dynamic parameter on all instances _cache_fusion_pipelined_updates_enable=FALSE
Any Exadata System	19.13 and lower	1. Apply Patch 31962730 2. Set dynamic parameter on all instances _cache_fusion_pipelined_updates_enable=FALSE

Note

Only redo generated with the dynamic parameter
_cache_fusion_pipelined_updates_enable or static parameter
_cache_fusion_pipelined_updates set to FALSE can be recovered with MIRA.

Enable Multi-instance Redo Apply and Tune

1. Enable multi-instance redo apply (MIRA) by indicating the number of apply instances.

Leave all previous single-instance redo apply (SIRA) tuning changes in place. The MAA recommendation for MIRA is to use all standby database instances for apply.

2. It is recommended that you start with increased buffer sizes for MIRA.

These parameters provide additional buffer space to pass blocks between instances.

- "_mira_local_buffers"=100 (default 25)
- "_mira_num_receive_buffers"=100 (default 25)
- "_mira_rcv_max_buffers"=10000 (default 500) - does not increase SGA usage, simply sets a cap.

These values will increase SGA usage of MIRA. Ensure that there is sufficient available SGA memory for these new settings.

The additional memory requirements (in MB) for each participating MIRA Oracle RAC instance = ((_mira_local_buffers*2)+(_mira_num_receive_buffers*[#instances-1])) MB

For example: In a 4-node Oracle RAC system, if _mira_num_local_buffers=100 and _mira_num_receive_buffers=100, then (100*2)+(100*3)=500MB from the SGA.

Note

For these parameter to take effect, the standby database must be restarted (A RAC rolling restart is allowed).

3. Enable MIRA using one of these methods.

- Set an Oracle Data Guard Broker property

'ApplyInstances'=<#|ALL>

- Or run

```
SQL> ALTER DATABASE RECOVER MANAGED STANDBY DATABASE DISCONNECT FROM  
SESSION INSTANCES ALL;
```

4. Check for system resource contention after tuning MIRA.

Follow the same practices described in [Evaluate System Resource Bottlenecks](#).

5. Tune MIRA based on wait events described here.

Follow the methodology in [Tune Redo Apply by Evaluating Database Wait Events](#).

If "recovery apply pending" or "recovery receive buffer free" are among the top wait events, increase `_mira_num_receive_buffers` and `_mira_num_local_buffers` incrementally by 100 to reduce this wait event.

These parameters provide additional buffer space to pass blocks between instances. Evaluate whether there is sufficient memory in the SGA to accommodate the additional buffer space.

6. Finding the right values for buffers can be in iterative process.

Monitor the apply rate and Standby AWR reports for a period of time that is representative of the normal workload, including peak workload time periods. If "recovery apply pending" and/or "recovery receive buffer free" are still among the top waits, and are of a significant percentage of the waits, increase the buffer parameters further and repeat.

Addressing a Very Large Redo Apply Gap

If the apply lag is larger than 24 hours, consider using a standby roll forward method to skip over the gap rather than apply all of the redo. See [How to Roll Forward a Standby Database Using Recover Database From Service \(12.2 and higher\) \(Doc ID 2850185.1\)](#)

This approach pulls changed Oracle data blocks directly from the primary database, and can potentially mitigate a large redo gap in half the time required to apply all of the redo.

The disadvantages of this approach are:

- Logical corruption and lost write detection checks and balances that are inherent to redo apply and standby databases are skipped
- Manual intervention is required to issue these commands and restart redo apply once it's completed.

Data blocks are still verified for physical corruptions.

Improving Redo Apply Rates by Sacrificing Data Protection

There are extremely rare conditions where redo apply cannot be tuned to achieve even higher redo apply rates to stay current with the primary. In these cases it may be necessary to turn off recommended data protection settings to help improve redo apply performance.

The following table describes some potential interim changes and their potential gains and trade offs.

Change	Potential Gain	Potential Trade-offs
Stop redo apply and use recover from service See How to Roll Forward a Standby Database Using Recover Database From Service (12.2 and higher) (Doc ID 2850185.1)	Optimized approach to recover from a large redo transport or redo apply gap, such as when the gap exceeds 24 hours	No logical block or lost writes data protection checks No redo block checksum verification
Mount standby instead of Active Data Guard	Potential 5-10% redo apply performance gain, but mostly for batch workloads	No real-time auto block repair of physical corruptions No real-time query on the standby Neither of the above trade-offs may be as relevant when the standby is lagging beyond application threshold
Disable or reduce DB_BLOCK_CHECKING on the standby	Reduces CPU utilization during redo apply If CPU resources are limited, this change can improve redo apply by 10-40%	Potential "rare" logical block corruptions may not be detected and can be propagated to the standby
Disable Flashback Database	Eliminates flashback IOPS requirement on RECO If storage IOPS is the constraining resource, then this change can help redo apply performance	Lose the ability to quickly rewind the standby
Disable DB_LOST_WRITE_PROTECT on the primary and standby	Eliminates additional read IOPS on the standby due to block read redo generated on the primary to detect lost writes This change is an option if IOPS capacity is saturated	Lost writes are not detected early on either primary or standby databases

Role Transition, Assessment, and Tuning

With thorough planning, configuration, and tuning, Oracle Data Guard role transitions can effectively minimize downtime and ensure that the database environment is restored with minimal impact on the business.

Using a physical standby database, Oracle MAA testing has determined that switchover and failover times with Oracle Data Guard have been reduced to seconds. This section describes best practices for both switchover and failover. While following best practices, switchover times of approximately 30 seconds for Oracle RAC and less 10 seconds for a single instance database have been observed. Detection time is separate.

Prerequisite Data Guard Health Check Before Role Transition

Complete the following prerequisites before performing a switchover operation.

Every Quarter

Perform the following steps every quarter.

1. Ensure that your Oracle Data Guard configuration is MAA compliant.
 - a. Refer to [Oracle Database Configuration Best Practices](#) and [Oracle Data Guard Configuration Best Practices](#) to ensure that all recommended Data Guard configuration practices are in place.
 - b. Refer to [Overview of Oracle Multitenant Best Practices](#) for PDB service recommendations.
2. Run a simple application test, which includes:
 - a. Convert existing the standby database to a snapshot standby.
 - b. Validate the application connection to the read-write test database as if this was a disaster recovery test. See [Configuring Continuous Availability for Applications](#) for configuration guidance.
3. Test your end-to-end application failover after a Data Guard role transition.
 - a. Issue a Data Guard switchover.
 - b. Orchestrate the entire application failover.
 - c. Switch back is optional.

One Month Before Switchover

One month before performing a switchover operation, consult the MOS note “Oracle Database 19c Important Recommended One-off Patches (Doc ID 555.1)” to identify any critical issues that might affect your release.

Also consider suspending or shutting down long running reporting or jobs including monitoring, auditing, and database backups that create persistent connections during the target planned maintenance window that contains the Data Guard switchover operation.

Common configuration issues that impact application service availability while performing a Data Guard role transition with Oracle Multitenant database are:

- PDB saved state or triggers are used and fail during Data Guard role transition
- PDB default service is leveraged instead of using Oracle clusterware-managed distinct services for each PDB for your application service
- Wallet/security settings are not the same on the standby

To ensure application service and application failover readiness:

1. Never use PDB default services, nor SAVED STATE (except during relocate operations), nor database triggers to manage role-based services.
2. Use clusterware-managed distinct services on each PDB for your application service, and leverage that application service to connect to the database.

3. When defining a clusterware-managed application service, define which PDB and services will be started, and in which Oracle RAC instance and database role.
4. For Data Guard, always use role-based services by assigning a role to each clusterware-managed service.

Validate Database Switchover and Failover Readiness

You can use the `VALIDATE` command to perform a comprehensive set of database checks before performing a role change. The command checks the following items:

- Whether there is missing redo data on a standby database
- Whether flashback is enabled
- The number of temporary tablespace files configured
- Whether an online data file move is in progress
- Whether online redo logs are cleared for a physical standby database
- Whether standby redo logs are cleared for a primary database
- The online log file configuration
- The standby log file configuration
- Apply-related property settings
- Transport-related property settings
- Whether there are any errors in the Automatic Diagnostic Repository (for example, control file corruptions, system data file problems, user data file problems)

The three main `VALIDATE` commands that should be issued prior to switchover are:

1. `VALIDATE DATABASE VERBOSE standby` - The `VALIDATE DATABASE` command shows a brief summary of the database, and reports any errors or warnings that were detected. `VALIDATE DATABASE VERBOSE` shows everything in the brief summary plus all items that were validated.
2. `VALIDATE DATABASE standby SPFILE` - The `VALIDATE DATABASE SPFILE` command reports any parameter differences between primary and the specified standby databases.
3. `VALIDATE NETWORK CONFIGURATION FOR ALL` - The `VALIDATE NETWORK CONFIGURATION` command performs network connectivity checks between members of a configuration.

To summarize how to evaluate Role Transition readiness, review the following:

- PRIMARY DATABASE Section:
 - `DGMGRL> VALIDATE DATABASE VERBOSE 'Primary_DBName';`
 - Check if there are PDB saved states in the primary database.
 - * `SELECT * FROM dba_pdb_saved_states;`
 - Evaluate health with `exachk` or `orachk`.
- For each STANDBY DATABASE `STANDBY_DB_UNIQUE_NAME` Section:
 - `DGMGRL> VALIDATE DATABASE VERBOSE 'Standby_DBName';`
 - `DGMGRL> VALIDATE DATABASE 'Standby_DBName' SPFILE;`

- Evaluate health with `exachk` or `orachk`.
- Evaluate if the standby cluster and database are symmetric with the primary cluster and database. This ensures identical or similar performance after role transition.
- Evaluate whether the cluster shape and system resources are the same, spfile memory settings are the same, and number of databases sharing the cluster resources are the same. If not, highlight the differences and evaluate if system resources are available by reviewing `exawatcher` or `oswatcher` graphs.
- Network Section:
 - `DGMGRL> VALIDATE NETWORK CONFIGURATION FOR ALL;`
- Redo Rate History Section:
 - `SQL> SELECT thread#,sequence#,blocks*block_size/1024/1024 MB,(next_time-first_time)*86400 sec, blocks*block_size/1024/1024)/((next_time-first_time)*86400) "MB/s" FROM v$sarchived_log WHERE ((next_time-first_time)*86400<>0) and first_time between to_date('2015/01/15 08:00:00','YYYY/MM/DD HH24:MI:SS') and to_date('2015/01/15 11:00:00','YYYY/MM/DD HH24:MI:SS') and dest_id=1 order by first_time;`

Example:

The Oracle Data Guard broker `VALIDATE DATABASE` command gathers information related to switchover and failover readiness.

The validation verifies that the standby and primary database are reachable and the apply lag is less than `ApplyLagThreshold` for the target database. If these data points are favorable, the command output displays "Ready for Failover: Yes" as shown below. In addition, if redo transport is running, the command output displays "Ready for Switchover: Yes".

```
DGMGRL> validate database [verbose] database_name
```

```
Database Role: Physical standby database
Primary Database: standby_db_unique_name
```

```
Ready for Switchover: Yes
Ready for Failover: Yes (Primary Running)
```

`VALIDATE DATABASE` checks additional information that can impact switchover time and database performance, such as whether the online redo logs have been cleared, number of temporary tablespaces, parameter mismatches between primary and standby, and the status of flashback databases.

In most failover cases the primary database has crashed or become unavailable. The Ready for Failover output indicates if the primary database is running when `VALIDATE DATABASE` was issued. This state does not prevent a failover, but it is recommended that you stop the primary database before issuing a failover to avoid a *split-brain* scenario where the configuration has two primary databases. The broker only guarantees split-brain avoidance on failover when Fast-Start Failover is used.

You should also run `VALIDATE DATABASE VERBOSE standby`, `VALIDATE DATABASE standby SPFILE`, and `VALIDATE NETWORK CONFIGURATION FOR ALL` periodically as a configuration monitoring tool.

Days Before Switchover

Perform the following steps days before performing a Data Guard switchover.

1. Set the Data Guard broker trace level.

The Data Guard broker `TraceLevel` configuration property is used to control the amount of tracing performed by the broker for every member in the configuration. Setting the property to `USER` limits the tracing to completed operations and to any warning or error messages resulting from an operation or health check. Setting the property to `SUPPORT` increases the amount of tracing to include lower-level information needed to troubleshoot any issues.

```
DGMGRL> EDIT CONFIGURATION SET PROPERTY TraceLevel = SUPPORT;
```

2. Enable role transition metrics.

The Time Management Interface (TMI) event is a low overhead event which adds a line to the alert log whenever certain calls are executed in Oracle.

These entries in the alert log, or tags, delineate the beginning and end of a call. The tables in the topics below depict the delineation of key switchover and failover operations. This method is the most accurate for determining where time is being spent.

Set the database level event 16453 trace name context forever, level 15 on all databases. There are two methods of enabling this trace, either using the `EVENT` database parameter or setting the `EVENTS` at the system level. The difference is that the `EVENT` parameter is not dynamic but is persistent across restarts. `SET EVENTS` is dynamic but NOT persistent across database restarts. See the following examples.

```
ALTER SYSTEM SET EVENT='16453 trace name contextforever, level 15'  
scope=spfile sid='*';
```

```
ALTER SYSTEM SET EVENTS '16453 trace name context forever, level 15';
```

Data Guard Role Transition

Always use Oracle Data Guard broker or any Oracle UI or utility that ultimately calls the Data Guard broker command.

Suspend or shut down any long running reports or batch jobs including monitoring, auditing, and database backups that have persistent connections.

Use the Oracle Data Guard broker `SWITCHOVER` command to initiate switchover, and the `FAILOVER` command to initiate failover.

As part of a switchover or failover operation the broker does the following.

- Configures redo transport from the new primary database
- Starts redo apply on the new standby database
- Ensures that other standby databases in the broker configuration are viable and receiving redo from the new primary

- Integrates Oracle Clusterware and Global Data Services to ensure that the role-based services are started

Before issuing the Data Guard switchover, suspend or shut down long running reporting or jobs including monitoring, auditing, and database backups that create persistent connections.

To configure broker to initiate switchover, log in as SYS or SYSDBA and issue:

```
DGMGRL> SWITCHOVER TO database_name;
```

To configure broker to initiate failover, run:

```
DGMGRL> FAILOVER TO database_name [IMMEDIATE];
```

By default FAILOVER applies all redo that was received before failing over. The IMMEDIATE clause skips the pending redo and fails over immediately.

The SWITCHOVER and FAILOVER commands are idempotent and can be re-issued in the unlikely event of a failed transition.

Monitor Data Guard Role Transitions

Refer to the Data Guard Broker messages while the Data Guard role transition is happening. To extract detailed role transition status, refer to the primary and standby alert logs and broker logs for Data Guard switchover and failover messages and tags.

Key Switchover Operations and Alert Log Tags

Switchover is broken down into four main steps as follows.

- Convert to Standby** - terminate any existing production sessions, convert the control file into a standby control file, and send a message to the standby to continue the switchover.
The Convert to Standby - these steps are found in the alert log of the original primary. All remaining steps are found in the original standby alert log.
- Cancel Recovery** - apply remaining redo and stop recovery.
- Convert to Primary** - a two-step close (to the mounted state) of instances (one instance, then all others), clear online redo logs, convert control file to primary control file, and data Guard Broker bookkeeping.
- Open New Primary** - parallel open of all instances.

Table 17-8 Alert Log Tags Defining the Steps with Time Management Interface Event Enabled

Step	Stage	Time Management Interface Event Enabled
Convert To Standby(primary alert log)	BEGIN	TMI: dbsdrv switchover to target BEGIN <DATE> <TIMESTAMP>
Convert To Standby(primary alert log)	END	TMI: kcv_switchover_to_target send 'switchover to primary' msg BEGIN <DATE> <TIMESTAMP>

Table 17-8 (Cont.) Alert Log Tags Defining the Steps with Time Management Interface Event Enabled

Step	Stage	Time Management Interface Event Enabled
Cancel Recovery(standby alert log)	BEGIN	TMI: kcv_commit_to_so_to_primary wait for MRP to die BEGIN <DATE> <TIMESTAMP>
Cancel Recovery(standby alert log)	END	TMI: kcv_commit_to_so_to_primary wait for MRP to die END <DATE> <TIMESTAMP>
Convert to Primary (standby alert log)	BEGIN	TMI: kcv_commit_to_so_to_primary BEGIN CTSO to primary <DATE> <TIMESTAMP>
Convert to Primary (standby alert log)	END	TMI: adbdv BEGIN 10 <DATE> <TIMESTAMP>
Open Primary(standby alert log)	BEGIN	TMI: adbdv BEGIN 10 <DATE> <TIMESTAMP>
Open Primary(standby alert log)	END	TMI: adbdv END 10 <DATE> <TIMESTAMP>

Key Failover Operations and Alert Log Tags

All failover steps are documented in the alert log of the target standby where the failover was performed.

1. Cancel Recovery - Stop recovery and close all instances (to mounted) in parallel.
2. Terminal Recovery - Archive standby redo logs and recover any unapplied redo.
3. Convert to Primary - Clear online redo logs and convert control file to standby control file.
4. Open Primary - Open all instances in parallel.

Table 17-9 Failover Alert Log Tags Defining the Steps with Time Management Interface Event Enabled

Step	Stage	Time Management Interface Event Enabled
Cancel Recovery	BEGIN	TMI: adbdv termRecovery BEGIN <DATE> <TIMESTAMP>
Cancel Recovery	END	TMI: adbdv termRecovery END <DATE> <TIMESTAMP>
Terminal Recovery	BEGIN	TMI: krdsr full BEGIN Starting media recovery <DATE> <TIMESTAMP>
Terminal Recovery	END	TMI: krdsr full END end media recovery <DATE> <TIMESTAMP>
Convert to Primary	BEGIN	TMI: kcv_commit_to_so_to_primary BEGIN CTSO to primary <DATE> <TIMESTAMP>

Table 17-9 (Cont.) Failover Alert Log Tags Defining the Steps with Time Management Interface Event Enabled

Step	Stage	Time Management Interface Event Enabled
Convert to Primary	END	TMI: adbdv BEGIN 10 <DATE> <TIMESTAMP>
Open Primary	BEGIN	TMI: adbdv BEGIN 10 <DATE> <TIMESTAMP>
Open Primary	END	TMI: adbdv END 10 <DATE> <TIMESTAMP>

Post Role Transition Validation

Use the `SHOW CONFIGURATION VERBOSE` command to verify that the switchover or failover and standby reinstate was successful.

```
DGMGRL> SHOW CONFIGURATION VERBOSE;
Configuration - DRSolution
Protection Mode: MaxAvailability
Members:
    South_Sales - Primary database
    North_Sales - Physical standby database
Fast-Start Failover: DISABLED
Configuration Status:
    SUCCESS
```

Troubleshooting Problems During a Switchover Operation

The most important goal after a failed Data Guard switchover or failover operation is to resume database and application availability as soon as possible.

Sources of Diagnostic Information

The Oracle Data Guard broker provides information about its activities in several forms.

- Database status information - You can use the `SHOW DATABASE VERBOSE db_unique_name` command to get a brief description of the database (name, role, and so on), database status, and information about any health check problems.


```
DGMGRL> SHOW DATABASE VERBOSE db_unique_name
```
- Oracle alert log files - The broker records key information in the alert log file for each instance of each database in a broker configuration. You can check the alert log files for such information when troubleshooting Oracle Data Guard.
- Oracle Data Guard "broker log files" - For each instance of each database in a broker configuration, the broker DMON process records important behavior and status information in a broker log file, useful in diagnosing Oracle Data Guard failures. The `TraceLevel` configuration property is used to specify the level of diagnostic information reported in the broker log files. The broker log file is created in the same directory as the alert log and is named `drc<$ORACLE_SID>.log`.

Retry Switchover After Correcting the Initial Problem

If the reported problems can be corrected quickly, you can retry the switchover operation.

If the reported problems cannot be corrected or the switchover operation fails even after correcting the reported problems, then you can choose another database for the switchover or restore the configuration to its pre-switchover state and then retry the switchover or refer to Rolling Back After Unsuccessful Switchover to Maximize Uptime.

```
DGMGRL> SWITCHOVER TO database_name;
```

Rolling Back After Unsuccessful Switchover to Maximize Uptime

For physical standby databases in situations where an error occurred, and it is not possible to continue with the switchover in a timely fashion, revert the new physical standby database back to the primary role to minimize database downtime.

Take the following steps.

1. Shut down and mount the new standby database (old primary).
2. Start Redo Apply on the new standby database.
3. Verify that the new standby database is ready to be switched back to the primary role.

Query the SWITCHOVER_STATUS column of the V\$DATABASE view on the new standby database. A value of TO PRIMARY or SESSIONS ACTIVE indicates that the new standby database is ready to be switched to the primary role. Continue to query this column until the value returned is either TO PRIMARY or SESSIONS ACTIVE.

4. Issue the following statement to convert the new standby database back to the primary role:

```
SQL> ALTER DATABASE SWITCHOVER TO target_db_name;
```

If step 4 fails, see Roll Back After Unsuccessful Switchover and Start Over in

Data Guard Performance Observations

Data Guard Role Transition Duration

Oracle Data Guard and Oracle MAA Gold reference architectures provide disaster recovery and high availability solutions when the primary database, cluster, or site fails or is inaccessible.

Each Data Guard environment is different and the time to perform role transitions can vary significantly. Variables including, but not limited to, SGA size, number of Oracle RAC instances, number of PDBs, number of data files, number of Oracle services and connections to the database at the time of role transition impact the length of a given role transition.

Generally, Data Guard switchover (planned maintenance) is slightly longer than Data Guard failover (unplanned outages).

Assuming system resources are not a bottleneck, the standby system is symmetric to the primary, and apply lag is near zero, the biggest factors that increase Data Guard role transition timings for Oracle 19c or earlier database releases are the following:

- Number of Pluggable Databases (PDBs) (for example, more than 25 PDBs)
- Number of Oracle Clusterware managed services (for example, more than 200 Oracle Services)
- Number of data files (more than 500 Oracle data files)

If any of the above exceeds the thresholds, Data Guard switchover or failover timings can increase by minutes. For example, a database with 5000 or more data files can increase role transition timings by 5 or more minutes. A database with 500 Oracle services can increase role transition timings by a minute or more. A database with 50 or more PDBs can increase role transition timings by a minute or more.

Because the number of data files is a big factor, MAA recommends using large data file sizes or BigFiles when creating tablespaces and partitions. If some databases that are protected by standby databases are critical, MAA recommends isolating them in their own separate container database (CDB) or keeping the number of PDBs to a minimum.

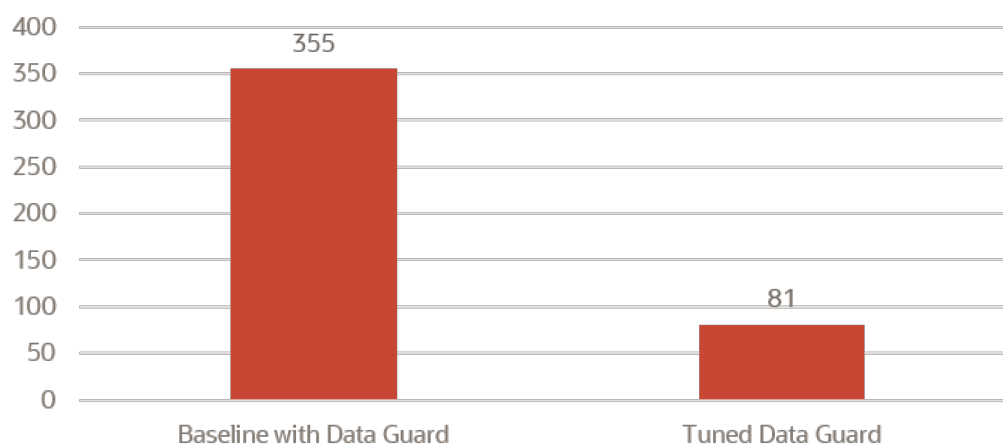
The following information is meant to educate you about ways to optimize role transitions.

Data Guard Switchover Duration

When attempting to minimize application downtime for planned maintenance:

- Before planned maintenance windows, avoid or defer batch jobs or long running reports. Peak processing windows should also be avoided.
- Because Data Guard switchover is graceful, which entails a shutdown of the source primary database, any application drain timeout is respected. See [Enabling Continuous Service for Applications](#) for Oracle Clusterware service drain attributes and settings.
- Data Guard switchover operations on a single instance (non-RAC) can be less than 30 seconds.
- Data Guard switchover operations on Real Application Cluster vary, but can be from 30 seconds to 7 minutes. The duration may increase with more PDBs (for example, > 25 PDBs), more application services (for example, 200 services), and if the database has a large number of data files (for example, 1000s of data files).

The following graph and table show one example of how much switchover operation duration can decrease when MAA tuning recommendations are implemented. Results will vary.

Figure 17-1 Planned maintenance: DR switch duration in seconds

Planned DR Switch (Switchover)	Initial Configuration	Tuned MAA Configuration
Convert Primary to Standby	26 secs	21 sec
Convert Standby to Primary (C2P)	47 secs	7 secs
Open new Primary (OnP)	152 secs	14 secs
Open PDB and Start Service (OPDB)	130 secs	39 secs
Total App Downtime	355 secs or 5 mins 55 secs	81 secs (78% drop)

The Tuned MAA Configuration timings were achieved by implementing the following MAA recommended practices:

- [Use Bigfile Tablespace](#)
- [Oracle Data Guard Configuration Best Practices](#)
- [Role Transition, Assessment, and Tuning](#)

Data Guard Failover Duration

When attempting to minimize application downtime for DR scenarios:

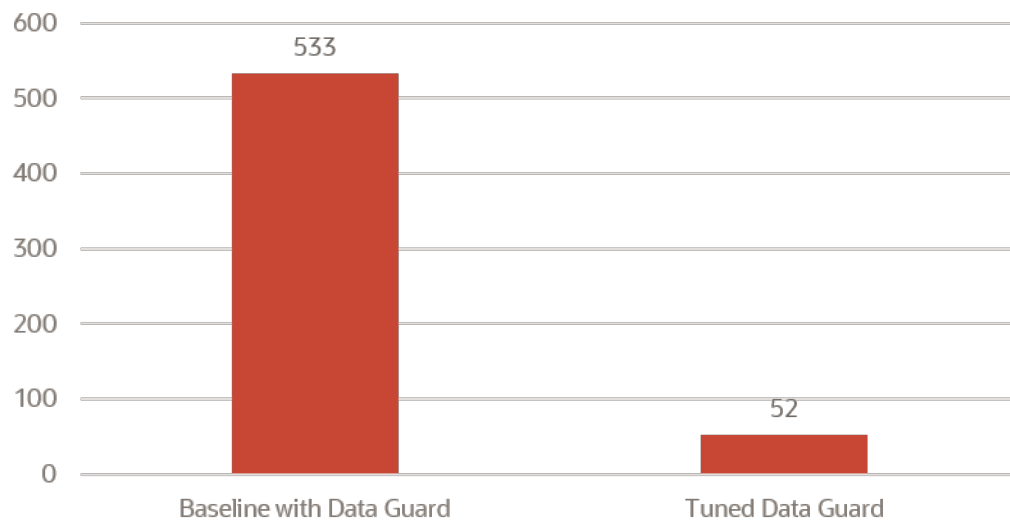
- To limit Recovery Time Objective (RTO or database down time) and Recovery Point Objective (RPO or data loss), automatic detection and fail over is required. See Fast-Start Failover in *Oracle Data Guard Broker*.
- The database administrator can determine the appropriate "detection time" before initiating an automatic failover by setting `FastStartFailoverThreshold`. See Enabling Fast-Start Failover Task 4: Set the `FastStartFailoverThreshold` Configuration Property in *Oracle Data Guard Broker*.

The MAA recommended setting is between 5 seconds and 60 seconds for a reliable network. Oracle RAC restart may also recover from a transient error on the primary. Setting this threshold higher gives the restart a chance to complete and avoid failover, which can be intrusive in some environments. The trade off is that application downtime increases in the event an actual failover is required.

- Data Guard failover operations on a single instance (non-RAC) can be less than 20 seconds.
- Data Guard failover operations on a Real Application Cluster vary but can be from 20 seconds to 7 minutes. The duration may increase with more PDBs (for example, > 25 PDBs), more application services (for example, 200 services) and if the database has a large number of data files (for example, 1000s of data files).

The following graph and table show one example how much failover operation duration can decrease when MAA tuning recommendations are implemented. Results will vary.

Figure 17-2 Unplanned DR failover duration in seconds



Unplanned Outage/DR (Failover)	Initial Configuration	Tuned MAA Configuration
Close to Mount (C2M)	21 secs	1 sec
Terminal Recovery (TR)	154 secs	2 secs
Convert to Primary (C2P)	114 secs	5 secs
Open new Primary (OnP)	98 secs	28 secs
Open PDB and Start Service (OPDB)	146 secs	16 secs
Total App Downtime	533 secs or 8min 53 secs	52 secs (90% drop)

The Tuned MAA Configuration timings were achieved by implementing the following MAA recommended practices:

- Evaluate Data Guard Fast-Start Failover and test with different FastStartFailoverThreshold settings
- [Use Bigfile Tablespace](#)
- [Oracle Data Guard Configuration Best Practices](#)
- [Role Transition, Assessment, and Tuning](#)

Customer Examples

Real-world Data Guard role transition duration observations from Oracle customers are shown in the following table.

Primary and Data Guard Configuration	Observed RTO
Single instance database failover in Database Cloud (DBCS) with heavy OLTP workload. Data Guard threshold is 5 seconds.	20 secs
Large commercial bank POC results on 4 node Oracle RAC with heavy OLTP workload, that consists of only one PDB in a large CDB and >= 25 clusterware managed services	51 secs (unplanned DR) 82 secs (planned DR)
ExaDB-D 2-node RAC MAA testing with heavy OLTP workload	78 secs
ADB-D 2-node RAC MAA testing (25 PDBs, 250 services) with heavy OLTP workload	104 secs
ADB-D 2-node RAC MAA testing (50 PDBs, 600 services) with heavy OLTP workload	180 secs
ADB-D 2-node RAC MAA testing (4 CDBs, 100 PDBs total, 500 services) with heavy OLTP workload	164 secs
Oracle SaaS Fleet (thousands) of 12-node RAC, 400 GB SGA, 4000+ data files (note: reducing number of data files to hundreds can reduce downtime by minutes)	< 6 mins
Third Party SaaS Fleet (thousands) of 7-12 node RACs with quarterly site switch	< 5 mins

Data Guard Role Transition Optimizations in Oracle AI Database 26ai

Data Guard role transition timings can be 400% times faster in Oracle AI Database 26ai because of many Oracle AI Database 26ai optimizations in various areas, including PDB and CDB open and close operations, recovery, and database checkpoint and clusterware service management.

The examples that follow show role transitions are less than 30 seconds, and you will see faster Data Guard role transition timings after upgrading to Oracle AI Database 26ai compared to Oracle Database 19c.

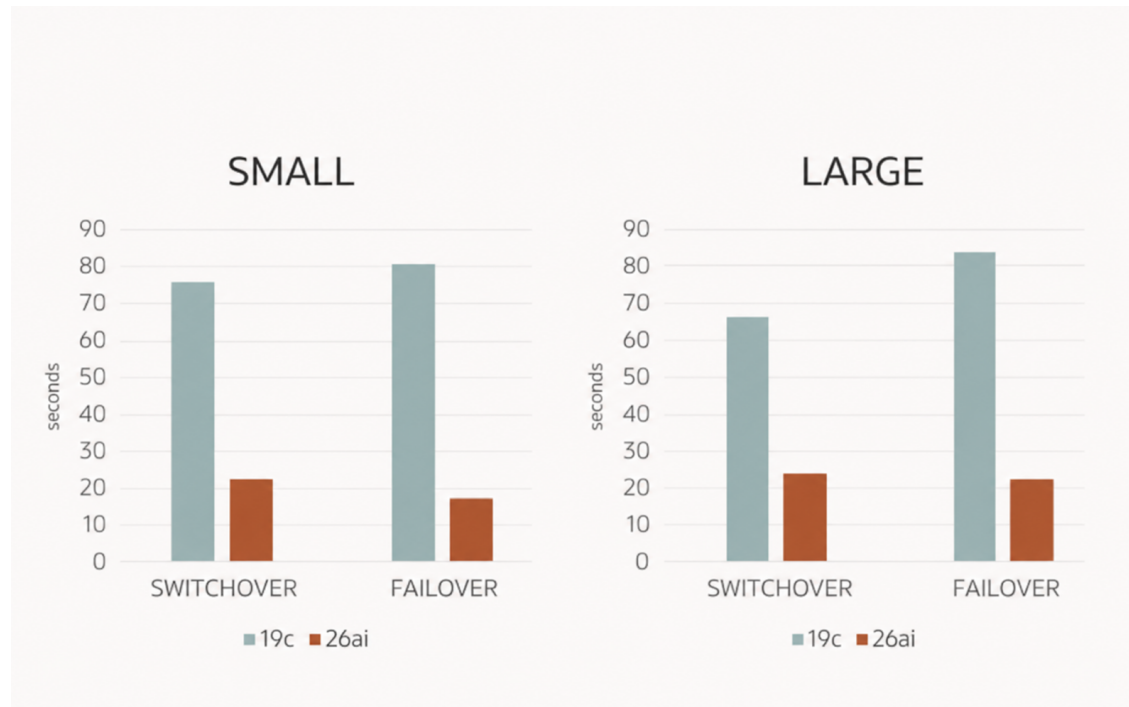
There are cases where the Oracle AI Database 26ai Data Guard role transition timings may exceed 30 seconds. The main factors are:

- Number of data files (for example, more than 2000 data files) and how much outstanding media recovery needed. This is the most significant and most impactful variable.

For example, a database on a CDB that has 5000 data files and a heavy workload, resulting in outstanding redo that needs to be applied, may have an RTO greater than 60 seconds, or even more than a couple of minutes. Observed is an additional 10ms per data file. By using Oracle recommended BigFiles instead of smallfiles on any new database creation, this problem is eliminated for all new databases.

- Number of PDBs and Clusterware managed services. If you have many PDBs (more than 25 PDBs) or hundreds of Clusterware managed services, you may find an increase in RTO timings.

Data Guard switchover and failover timings are reduced significantly in Oracle AI Database and Oracle Grid Infrastructure 26ai, as shown in the following image.



What Was Measured

Oracle MAA measured the time difference between entering the SWITCHOVER or FAILOVER phase in DGMGRL and starting the services, as seen in the alert log.

For that, two different configurations were used, as shown in the following table.

Configuration	System	PDBs	Data Files	Services	Redo Rate
SMALL	2-node Exadata RAC	5	50	10	60 MB/s
LARGE	4-node Exadata RAC	12	500	12	100 MB/s

Redo Apply Performance

Single Instance Redo Apply (SIRA) is the default redo apply mode, and for most cases should be sufficient. Focus on tuning and understanding the bottlenecks of SIRA before even considering Multi-Instance Redo Apply (MIRA). See [Redo Apply Troubleshooting and Tuning](#).

Assuming there is no system resource contention, such as storage bandwidth limitations or non-tuned database wait event due to read-write I/O or checkpoint wait events, Multi-Instance Redo Apply (MIRA) can improve redo apply performance by scaling 30 to 80% for each additional Oracle RAC instance in a MIRA configuration.

There are a number of factors affecting the rate of improvement, such as the type of workload on the primary, additional read-only workload on the standby, and any new top standby wait events.

Figure 17-3 Example apply rates of Swingbench OLTP workloads

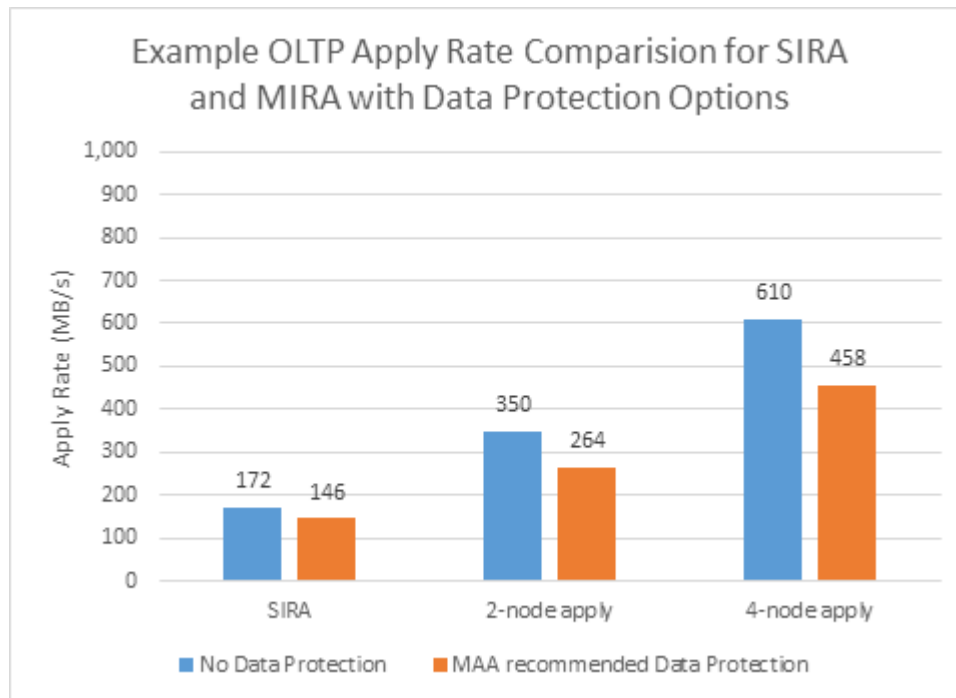
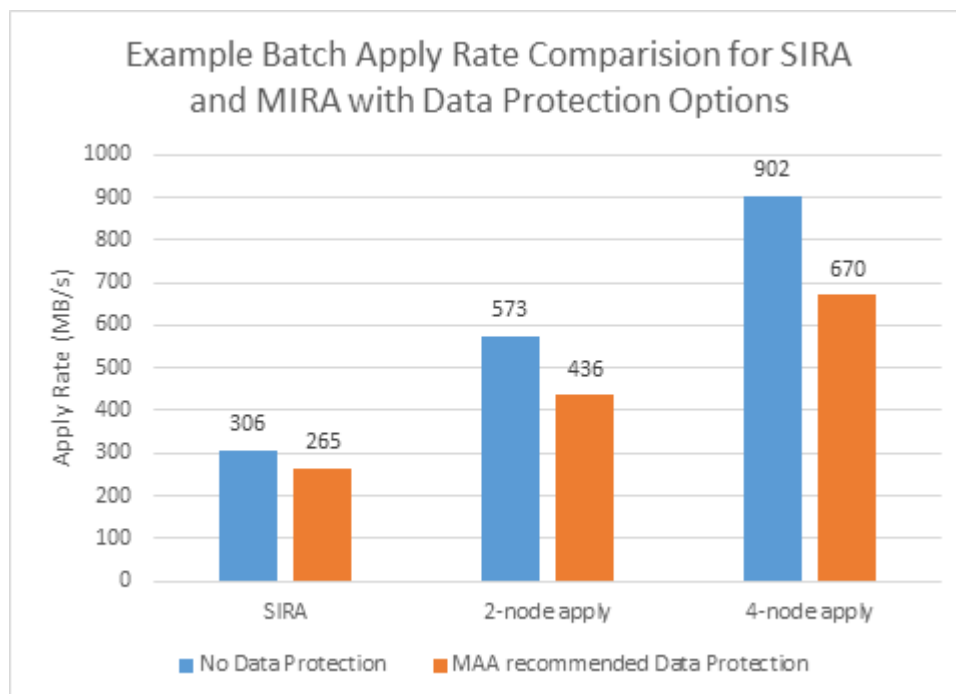


Figure 17-4 Example apply rates of batch insert workloads



For large redo gaps, you can quickly catch up on the gaps by implementing the method described in [Addressing a Very Large Redo Apply Gap](#).

In very rare case you can temporarily disable some MAA data protection settings if apply performance is more important. Refer to [Improving Redo Apply Rates by Sacrificing Data Protection](#).

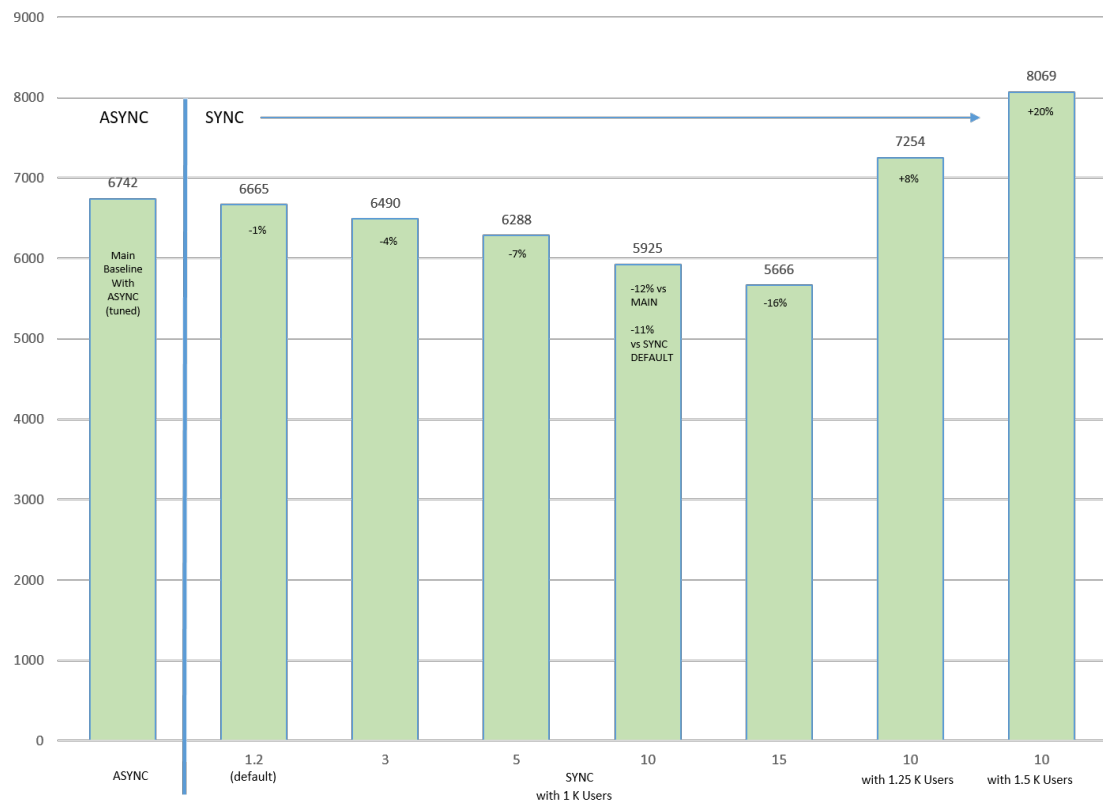
Application Throughput and Response Time Impact with Data Guard

Application throughput and response time impact is near zero when you enable Data Guard Max Performance protection mode or ASYNC transport. Throughput and application response time is typically not impacted at all in those cases.

With Data Guard Max Availability or Max Protection mode or SYNC transport, the application performance impact varies, which is why application performance testing is always recommended before you enable SYNC transport. With a tuned network and low round-trip latency (RTT), the impact can also be negligible, even though every log commit has to be acknowledged to every available SYNC standby database in parallel to preserve a zero data loss solution.

Here's an example of the application throughput impact but application impact varies based on workload:

Figure 17-5 Application impact with MTU=9000



Notice the lower network RTT latency (x axis), the application (TPS or y axis) throughput reduces.

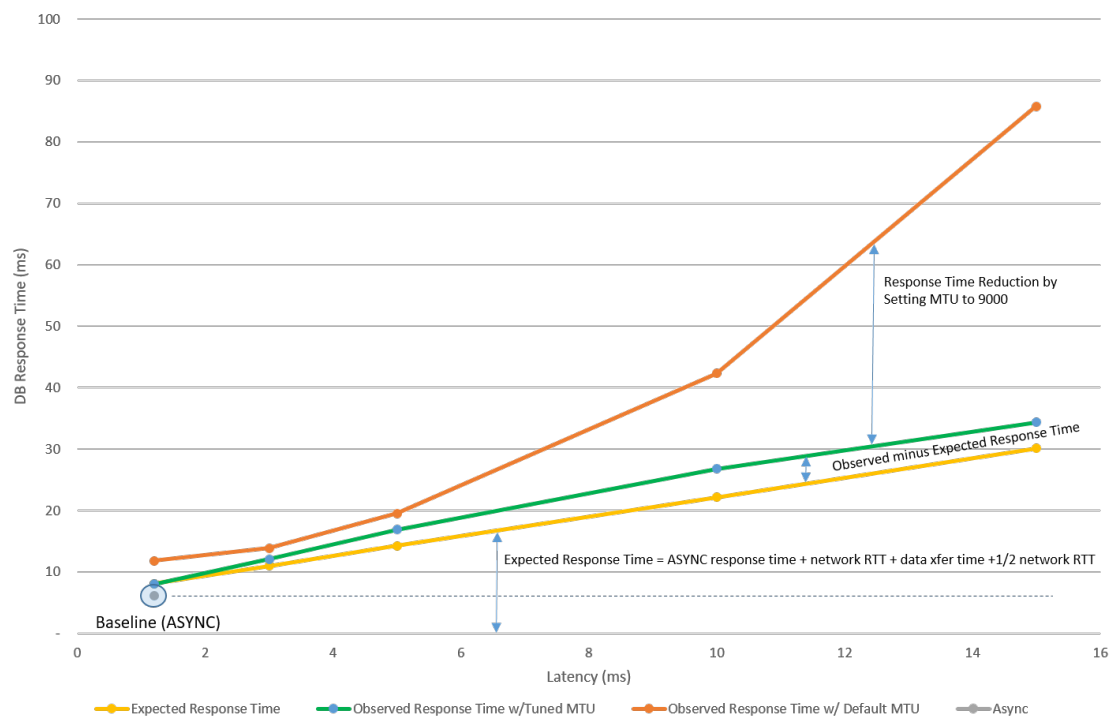
Note that in this network environment we observed that increasing MTU from 1500 (default) to 9000 (for example, jumbo frames) helped significantly since log message size increased significantly with SYNC. With the larger MTU size, the number of network packets per redo send request are reduced.

See [Assessing and Optimizing Network Performance](#) for details about tuning the network including the socket buffer size and MTU.

Even when throughput decreases significantly with higher RTT latency, you can increase TPS if your application can increase the concurrency. In the above chart, the last 2 columns increased the workload concurrency by adding more users.

Application response time with SYNC transport can also increase, but will vary based on each application workload and network tuning. With SYNC transport, all log writes have to wait for standby SYNC acknowledgment. This additional wait result in more foregrounds waiting for commit acknowledgment. Because commits have to be acknowledged by the standby database and more foregrounds are waiting for commits, the average log write size increases which affects the redo/data transfer time, as shown in the following chart.

Figure 17-6 Database response time (ms) vs latency (ms) for tuned and default MTU



In this example, we observed from AWR reports that average redo write size increased significantly, and tuning MTU reduced the response time impact. See [Assessing and Optimizing Network Performance](#) on tuning network including the socket buffer size and MTU.

After tuning the network, the response time impact was very predictable and low. Note that response impact varies per application workload.

To get the best application performance with Data Guard, use the following practices:

- Tune the application without Data Guard first and you should observe similar performance for ASYNC transport
- Implement [Oracle Data Guard Configuration Best Practices](#)
- Use [Redo Transport Troubleshooting and Tuning](#) methods
- Tune the network to improve application performance with SYNC. See [Assessing and Optimizing Network Performance](#)
- Application workload specific changes that can help increase throughput for SYNC Transport are:
 - Evaluate adding more concurrency or users to increase throughput.
 - For non-critical workloads within certain sessions that do not require zero data loss, evaluate advanced COMMIT_WRITE attribute to NOWAIT.

In this case, you can commit before receiving the acknowledgment. Redo is still sent to persistent redo logs but is done asynchronously. Recovery is guaranteed for all persistent committed transactions in the redo that is applied. See COMMIT_WRITE in *Oracle Database Reference*.

Monitor an Oracle Data Guard Configuration

Use the following Oracle MAA best practice recommendations to monitor an Oracle Data Guard configuration.

Monitoring Oracle Data Guard Configuration Health Using the Broker

The Oracle data Guard broker issues a health check once a minute and updates the configuration status. To force a health check to occur immediately, run the command `show configuration verbose`.

On a primary database, the health check determines if the following conditions are met.

- Database is in the state specified by the user, as recorded in the broker configuration file
- Database is in the correct data protection mode
- Database is using a server parameter file (SPFILE)
- Database is in the ARCHIVELOG mode
- Redo transport services do not have any errors
- Database settings match those specified by the broker configurable properties
- Redo transport settings match those specified by the redo transport-related properties of the standby databases
- Current data protection level is consistent with configured data protection mode
- Primary database is able to resolve all gaps for all standby databases

On a standby database, the health check determines whether the following conditions are met.

- Database is in the state specified by the user, as recorded in the broker configuration file
- Database is using a server parameter file (SPFILE)
- Database settings match those specified by the broker configurable properties
- Primary and target standby databases are synchronized or within lag limits if fast-start failover is enabled

To identify any warnings on the overall configuration, show the status using the `SHOW CONFIGURATION` command.

```
DGMGRL> show configuration;
```

Configuration - dg

```
Protection Mode: MaxPerformance
Members:
tin - Primary database
can - Physical standby database
```

Fast-Start Failover: DISABLED

Configuration Status:
SUCCESS (status updated 18 seconds ago)

If the configuration status is SUCCESS, everything in the broker configuration is working properly.

However, if you see a status of WARNING or ERROR, then something is wrong in the configuration. Additional error messages will accompany the WARNING or ERROR status that should be used to identify current issues.

The next step is to examine each database in the configuration to narrow down what the specific error is related to.

To identify the warnings on the primary database, get its status using the SHOW DATABASE command.

```
DGMGRL> show database tin
```

Database - tin

```
Role:                PRIMARY
Intended State:      TRANSPORT-ON
Instance(s):
  tin1
  tin2
```

Database Status:
SUCCESS

If the database status is SUCCESS then the database is working properly.

However, if you see a status of WARNING or ERROR, then something is wrong in the database. Additional error messages will accompany the WARNING or ERROR status that should be used to identify current issues.

Repeat the same SHOW DATABASE command on the standby database and assess any error messages.

In addition to the above commands, the broker features a VALIDATE DATABASE command.

```
DGMGRL> validate database tin
```

```
Database Role:    Primary database
Ready for Switchover: Yes
```

```
DGMGRL> validate database can;
```

```
Database Role:    Physical standby database
Primary Database: tin
```

```
Ready for Switchover: No
Ready for Failover:  Yes (Primary Running)
```

```
Capacity Information:
Database Instances      Threads
```

```

tin          2          2
can          1          2
Warning: the target standby has fewer instances than the
primary database, this may impact application performance

```

Standby Apply-Related Information:

```

Apply State:      Not Running
Apply Lag:        Unknown
Apply Delay:      0 minutes

```

The VALIDATE DATABASE does not provide a SUCCESS or WARNING status and must be examined to determine if any action needs to be taken.

It is recommended that you run the VALIDATE DATABASE command after creating the broker configuration, and before and after any role transition operation.

The VALIDATE DATABASE command performs the following checks.

- Whether there is missing redo data on a standby database
- Whether flashback is enabled
- The number of temporary tablespace files configured
- Whether an online data file move is in progress
- Whether online redo logs are cleared for a physical standby database
- Whether standby redo logs are cleared for a primary database
- The online log file configuration
- The standby log file configuration
- Apply-related property settings
- Transport-related property settings
- Whether there are any errors in the Automatic Diagnostic Repository (for example, control file corruptions, system data file problems, user data file problems)

Detecting Transport or Apply Lag Using the Oracle Data Guard Broker

Given enough resources, in particular network bandwidth, an Oracle Data Guard standby can maintain pace with very high workloads. In cases where resources are constrained, the standby can begin to fall behind, resulting in a transport or apply lag.

A **transport lag** is the amount of data, measured in time, that the standby has not received from the primary.

An **apply lag** is the difference, in elapsed time, between when the last applied change became visible on the standby and when that same change was first visible on the primary.

When using the Data Guard broker, the transport or apply lag can be viewed by using the SHOW DATABASE command and referencing the standby database, as shown here.

```
DGMGRL> show database orclsb
```

```
Database - orclsb
```

```

Role:              PHYSICAL STANDBY
Intended State:    APPLY-ON

```

```
Transport Lag:      0 seconds (computed 0 seconds ago)
Apply Lag:          0 seconds (computed 1 second ago)
Average Apply Rate: 792.00 KByte/s
Real Time Query:    ON
Instance(s):
  orclsb1 (apply instance)
  orclsb2
```

```
Database Status:
SUCCESS
```

The broker `TransportDisconnectedThreshold` database property (default of 0 in Oracle Database 11.2, and 30 seconds for Oracle Database 12.1 and later releases) can be used to generate a warning status for a standby when the last communication from the primary database exceeds the value specified by the property. The property value is expressed in seconds.

The following is an example of the warning when a disconnection has occurred.

```
DGMGRL> show database orclsb;
```

```
Database - orclsb
```

```
Role:                PHYSICAL STANDBY
Intended State:       APPLY-ON
Transport Lag:        0 seconds (computed 981 seconds ago)
Apply Lag:            0 seconds (computed 981 seconds ago)
Average Apply Rate:   12.00 KByte/s
Real Time Query:      OFF
Instance(s):
  orclsb1 (apply instance)
  orclsb2
```

```
Database Warning(s):
ORA-16857: member disconnected from redo source for longer than specified
threshold
```

The broker also has the following configurable database properties that you can use to generate warnings when a transport or apply lag exceed a user defined value.

- The `ApplyLagThreshold` property generates a warning status for a logical or physical standby when the database's apply lag exceeds the value specified by the property.
The property value is expressed in seconds. A value of 0 seconds results in no warnings being generated when an apply lag exists. As a best practice, Oracle recommends setting `ApplyLagThreshold` to at least 15 minutes.
- The `TransportLagThreshold` property can be used to generate a warning status for a logical, physical, or snapshot standby when the database's transport lag exceeds the value specified by the property.
The property value is expressed in seconds. A value of 0 seconds results in no warnings being generated when a transport lag exists. As a best practice, Oracle recommends setting `TransportLagThreshold` to at least 15 minutes.

Monitoring Oracle Data Guard Configuration Health Using SQL

You can use the queries in the following tables to assess the overall Data Guard configuration health on the primary database and the standby database.

Table 18-1 Primary Database Queries

Goal	Query	Expected Results
Check if any remote standby archive destination is getting errors	<pre>select sysdate, status, error from gv\$archive_dest_status where type='PHYSICAL' and status!='VALID' or error is not null;</pre>	Good health = no rows returned If the query returns rows, then raise an alert with the returned data.
Check if all remote standby archive destinations is enabled or VALID		
Check if any NOLOGGING activity occurred on the primary database in the last day	<pre>select file#, name, unrecoverable_change#, unrecoverable_time from v\$datafile where unrecoverable_time > (sysdate - 1);</pre>	Good health = no rows returned If the query returns rows, then the standby database is vulnerable, and the files listed in the output must be refreshed on the standby.
Detect gaps on the standby database	<pre>select sysdate, database_mode, recovery_mode, gap_status from v\$archive_dest_status where type='PHYSICAL' and gap_status != 'NO GAP';</pre>	Good health = no rows returned If the query returns rows, then there's an existing gap between the primary and the standby database, and you must run the same query on the standby database. If the output from the primary and standby is identical, then no action is required. If the output on the standby does not match the output from the primary, then the datafile on the standby should be refreshed.
Assess whether any severe Data Guard event occurred in the last day	<pre>select * from v\$dataguard_status where severity in ('Error', 'Fatal') and timestamp > (sysdate -1);</pre>	Good health = no rows returned If the query returns rows, then raise an alert with the returned output.

Table 18-1 (Cont.) Primary Database Queries

Goal	Query	Expected Results
FOR SYNC ENVIRONMENTS ONLY: Assess if running in Maximum Availability mode and configuration is in sync	<pre> select sysdate, protection_mode, synchronized, synchronization_status from v\$archive_dest_status where type='PHYSICAL' and synchronization_status != 'OK'; </pre>	Good health = no rows returned If the query returns rows, then raise an alert with the returned output.

Table 18-2 Physical Standby Database Queries

Goal	Query	Expected Results
Determine if there is a transport lag	<pre> select name, value, time_computed, datum_time from v\$dataguard_stats where name='transport lag' and value > '+00 00:01:00'; </pre>	Good health = no rows returned If no rows are returned, then this implies that there is no transport lag
Determine if there is an apply lag	<pre> select name, value, time_computed, datum_time from v\$dataguard_stats where name='apply lag' and value > '+00 00:01:00'; </pre>	Good health = no rows returned If no rows are returned, then this implies that there is no apply lag
Standby data file check (offline files or files that are not accessible)	<pre> select * from v\$datafile_header where status='OFFLINE' or ERROR is not null; </pre>	Good health = no rows returned Any rows returned list the files that have I/O or recovery issues
Verify that the Media Recovery Process is currently running	<pre> select * from v\$managed_standby where process like 'MRP%'; </pre>	Good health = rows returned If no rows are returned, then the MRP process is not running

Table 18-2 (Cont.) Physical Standby Database Queries

Goal	Query	Expected Results
Assess whether any severe Data Guard event occurred in the last day	<pre>select * from v\$dataguard_status where severity in ('Error','Fatal') and timestamp > (sysdate -1);</pre>	Good health = no rows returned If the query returns rows, then raise an alert with the returned output

Oracle Data Guard Broker Diagnostic Information

The Oracle Data Guard broker provides information about its activities in several forms.

- Database status information
- Oracle alert log files

The broker records key information in the alert log file for each instance of each database in a broker configuration.
- Oracle Data Guard broker log files

For each instance of each database in a broker configuration, the broker DMON process records important behavior and status information in a broker log file, which are useful for diagnosing Oracle Data Guard failures. The Set the TraceLevel configuration property to specify the level of diagnostic information reported in the broker log files. The broker log file is created in the same directory as the alert log and is named drc<\${ORACLE_SID}>.log.
- Oracle Data Guard command line (DGMGRL) logfile option

If the DGMGRL command-line interface was started with the -logfile optional parameter, then the resulting log file may contain a useful record of past operations and error conditions.

Detecting and Monitoring Data Corruption

If corrupt data is written to disk, or if a component failure causes good data to become corrupt after it is written, then it is critical that you detect the corrupted blocks as soon as possible.

To monitor the database for errors and alerts:

- Query the V\$DATABASE_BLOCK_CORRUPTION view that is automatically updated when block corruption is detected or repaired.
- Configure Data Recovery Advisor to automatically diagnose data failures, determine and present appropriate repair options, and perform repair operations at your request.

Note that Data Recovery Advisor integrates with the Oracle Enterprise Manager Support Workbench (Support Workbench), the Health Monitor, and RMAN.

- Use Data Guard to detect physical corruptions and to detect lost writes.

Data Guard can detect physical corruptions when the apply process stops due to a corrupted block in the redo stream or when it detects a lost write.

Use Enterprise Manager to manage and monitor your Data Guard configuration.

By taking advantage of Automatic Block Media Recovery, a corrupt block found on either a primary database or a physical standby database can be fixed automatically when the Active Data Guard option is used.

- Use SQL*Plus to detect data file corruptions and inter-block corruptions.

Run this SQL*Plus statement:

```
sqlplus> ANALYZE TABLE table_name VALIDATE STRUCTURE CASCADE;
```

After finding the corruptions, the table can be re-created or another action can be taken.

- An Recovery Manager (RMAN) backup and recovery strategy can detect physical block corruptions.

A more intensive RMAN check using the following command can detect logical block corruptions.

```
RMAN> BACKUP VALIDATE CHECK LOGICAL;
```

Optimizing Automatic Failover in Common Scenarios to Minimize Downtime

The following topics provide Oracle Data Guard MAA best practices in relation to common scenarios to avoid or minimize downtime and prevent various problematic scenarios.

Automatic Database Failover for Primary Database Outages

Fast Start Failover (FSFO) is the Oracle Data Guard feature that performs automatic database failover when there is an outage of a primary database. FSFO is controlled by the Data Guard observer process, which is part of the Data Guard broker. When the observer process that controls automatic database failover is started, it creates threads that connect to both the primary and standby databases. Each thread creates a database session and verifies the database health using the `dbms_dr`s package. If a thread doesn't respond, a new thread is created and Data Guard attempts to create a session and use the `dbms_dr`s package. If the threads do not respond by a defined period of time (threshold) then the database is considered to be unavailable.

If the primary database becomes network isolated from the standby and observer at the same time, the primary stalls (no new transactions are able to commit) until it can again make contact with either observer or standby. This is done to explicitly prevent split brain. If the observer and standby agree that neither can see the primary and if FSFO was enabled at the time they lost contact, then automatic failover occurs once the FSFO threshold is exceeded. At this same time, the primary will have also passed the FSFO threshold, causing it to abort so that existing connections are not able to continue reading stale data after a failover has occurred. If the primary is able to connect with either the observer or the standby designated as the failover target before the FSFO threshold expires, then it is allowed to continue to process transactions. If the primary aborts because the FSFO threshold is exceeded, then when the primary is restarted, it must receive permission from the observer or its original standby before it is able to open. When it makes contact, it is told that either a failover occurred while it was away, and it is automatically converted into a standby database or, if no failover occurred, it is allowed to open. This assures there is never a case where there are two active primaries at the same time.

If the primary loses contact with either the observer or the standby, the primary keeps processing transactions because it still has the ability to communicate with one of the two parties. If both the primary and standby have lost contact with the observer they fall into an UNOBSERVED state. If the primary has lost contact with the standby it falls into an UNSYNCHRONIZED state. This is how we ensure that an observer or standby outage, or the outage of both at different times, does not impact the availability of the primary database in an FSFO configuration.

If the primary database appears unavailable to the observer, the observer initiates an automatic failover if the standby is 1) responding 2) perceives the primary to be unavailable and 3) is synchronized as of the last communication.

As a best practice, the observer should ideally be located at a third site or third data center so that the primary, observer, and standby have isolated power, server, storage, and network infrastructure. It is possible to achieve a similar level of isolation using two sites; however,

when Data Guard is used for disaster recovery, the observer should never be deployed at the primary database server or cluster, or on the standby database server or cluster.

An additional best practice is to have the observer use the same path to connect to the production database as is used by the application tier. The intent of this practice is for the observer to have the same access path as application connections so that if application connections can't see the primary, the observer has the same problem and can quickly respond. This best practice comes with a caveat if the application tier is local to the production database while the standby is in a remote data center for disaster recovery. In this case, it is not wise to deploy the observer with the application tier, as it could experience the same issues as the database. Instead, the observer should be deployed at either the standby location or a third data center.

In all recent Oracle Database releases, the Data Guard broker includes a feature that regularly makes a new connection with the primary to simulate what new connections see, providing additional health monitoring of the primary database.

Automatic Data Integrity and Avoidance of Split Brain

An Oracle Data Guard configuration maintains multiple synchronized copies of the production database. Because Data Guard uses simple one-way replication, the primary database is the only database open read-write; all other replicas (standby databases) must be either in mount mode or open read-only (if using Oracle Active Data Guard). The term 'split brain' describes a scenario where there are two copies of the same database open read-write, each operating independently of the other. This is a very undesirable circumstance that results in inconsistent data. The following algorithms ensure data consistency in a Data Guard synchronous configuration and the avoidance of split-brain:

- The primary database Log Writer (LGWR) redo write and the sync redo transport write that transmits to the standby database are identical in redo content and size.
- The Data Guard Managed Recovery Process (MRP) at the standby database cannot apply redo unless the redo has been written to the primary's online redo log, with the only exception being during a Data Guard failover (via the recover managed standby database finish) operation. In addition to shipping redo synchronously, the sync transport process and LGWR also exchange information regarding the safe redo block boundary that standby recovery can apply up to from its standby redo logs. This boundary prevents the standby from applying redo it may have received, but which the primary has not yet acknowledged as committed to its own online redo logs.

Data Guard SYNC maintains data integrity in all of the 3 general outage categories:

- Primary failure and restart (for example, LGWR I/O failure, LGWR crash, instance failure, or database failure and restart): For example, if a primary database's LGWR cannot write to an online redo log, then the instance's LGWR and instance will crash. An Oracle RAC node or single instance database crash recovery will recover to the last committed transaction in the online redo log and roll back any uncommitted transactions. The current log will be completed and archived to any enabled log archive destination in a Data Guard destination.
- Failure resulting in missing redo on the standby: If the sync transport process and the Remote File Server (RFS) process that receives redo at the standby database detects missing current or last bit of redo on the standby regardless the outage cause, RFS requests the missing redo to be resent, and it is written directly into the Standby Redo Log (SRL).
- Any outage resulting in a zero data loss failover operation: If the primary database crashes, resulting in an automatic or manual zero data loss failover, then the Data Guard

failover operation will do "terminal recovery" (using the recover managed standby finish operation) and read and recover the current SRL. Once recovery completes applying all the redo in the SRLs, the new primary database comes up, and it archives the newly completed online log group. All new and existing standby databases discard any redo for the same log group sequence and thread, flashback to consistent SCN, and only apply the archives coming from the new primary database. Once again, the Data Guard environment is in sync with the (new) primary database and there's no deviation or data loss.

Automatic Reconnect Following Any Outage That Results in Network Timeout

Network or cluster failure outage cases are more involved because the heartbeat ARCH can hang on the network and may end up getting killed based on internal timers. The internal timers for detecting hung transport processes are controlled by the underscore parameter `_redo_transport_stall_time`, which is for the short disk/network calls, and the underscore parameter `_redo_transport_stall_time_long`, which is for the longer version of the disk/network calls.

After the heartbeat ARCH is killed, another ARCH becomes the heartbeat ARCH, and, after approximately 'REOPEN + 1 minute + <hang time..before killed >', it attempts to connect to the network. Only when the heartbeat ARCH is able to reconnect to the standby will LGWR attempt to connect to the standby database. This ensures that the application does not incur any unnecessary impact, and LGWR does not suffer a NetTimeout hang on an unhealthy network.

Automatic Reconnect Following Resolution of Standby Outage

The heartbeat ARCH is one of the primary database's ARCH processes that is designated to ping the standby. The heartbeat ARCH will 'ping' the standby every 1 minute when the network is healthy and the standby is reachable. This is done to detect gaps in archive logs received by the standby. If there is a network failure, the heartbeat ARCH does not attempt to connect for up to REOPEN time; however, there are states where Oracle Data Guard will reset the 'remote' destination sooner than 'REOPEN + 1 minute' using a feature called standby announcement. For the standby bounce case (not a network hang), a mounted standby RFS or Fetch Archive Log (FAL) will communicate with the primary database and 'announce' that the standby is now available. In such cases, the primary does not wait for REOPEN - it will connect to the standby immediately after being posted.

With Maximum Availability and LGWR SYNC redo transport, note that it is only the heartbeat ARCH that attempts a reconnect to the standby after a network failure. LGWR never attempts to connect to the standby after a failure unless the heartbeat ARCH has already confirmed that the standby is accessible.

Gap resolution for a reconnect following a standby outage is done automatically. For example, consider a primary in the middle of log sequence 100, the standby database crashes, and then the primary advances through logs 101, 102, and 103. When the standby becomes available, the primary switches logs to 104 and starts either ASYNC or SYNC transport at that point. It then ships the remaining portion of log sequence 100 and uses several archive processes to ship 101, 102, and 103 in parallel.

Data Guard Broker Properties That Affect Outage Repair Times

The following Data Guard broker properties can be configured to reduce outage repair times. Recommended values depend on the environment in which they are being used, and they should be tested to assess application impact.

Data Guard Broker Property: NetTimeout

Default Value: 30 Seconds

Recommended value: 5 to 10 seconds if the network is responsive and reliable

Specifies a maximum number of seconds that the LGWR background process blocks while waiting for a synchronous redo transport destination to acknowledge redo data sent to it. If an acknowledgment is not received within NetTimeout seconds, an error is logged, and the redo transport session to that destination is terminated.

This property is put into effect for all outages that would place the sync transport process in a TCP timeout with the standby processes, such as network failure, standby host failure, and standby cluster failure. NetTimeout should not be set < 5 seconds.

Data Guard Broker Property: ReopenSecs

Default Value: 300 Seconds

Recommended value: 30 seconds

Specifies the minimum number of seconds before redo transport services should try to reopen a failed destination. Once reopen expires and the destination is open it is attempted at the next log switch.

This property is in effect for all outages that place the sync transport in an error state that closes the destination, such as a standby instance/database/node/cluster outage, network outage, or sync transport process death.

Data Guard Broker Property: MaxFailure

Default Value: None

Recommended Value: Depends on application availability and business requirements.

Controls the consecutive number of times redo transport services attempt to reestablish communication and transmit redo data to a failed destination before the primary database gives up on the destination.

This property is in effect for all outages that place the sync transport in an error state that closes the destination, such as a standby instance/database/node/cluster outage, network outage, or sync transport process death.

Data Guard Broker Property: FastStartFailoverThreshold

Default Value: 30 seconds

Recommended value: 6 to 15 seconds if the network is responsive and reliable. For primary Oracle RAC instances, add maximum clusterware heartbeat timeout (CSS miscount default for Linux is 30 seconds. For Exadata, you can use 2 seconds) to FastStartFailoverThreshold.

The FastStartFailoverThreshold configuration property defines the number of seconds the observer attempts to reconnect to the primary database before initiating a fast-start failover to the target standby database. The time interval starts when the observer first loses connection with the primary database. If the observer is unable to regain a connection to the primary database within the specified time, then the observer initiates a fast-start failover.

This property is in effect for all outages that would require a failover to the standby database due to a complete primary database outage.

Database Initialization Parameter: FAST_START_MTTR_TARGET

Default Value: 0

Recommended value: the value required for your expected recovery time objective (RTO)

FAST_START_MTTR_TARGET lets you specify the number of seconds the database takes to perform crash recovery of a single instance. When specified, FAST_START_MTTR_TARGET is overridden by LOG_CHECKPOINT_INTERVAL.

This property is put into effect for all outages that require a database to perform instance recovery, such as instance failure. When you set this database initialization parameter, the database manages incremental checkpoint writes in an attempt to meet the target. The incremental checkpoints may occur more often, increasing the amount of I/O from the database writer (DBWR). It is recommended that you perform testing before reducing this parameter.

Data Guard Standby Database Outage Repair

Kill Apply Instance

How to run a test

1. Start workload on the primary database.
2. Ensure that the primary database is in Maximum Availability mode.


```
select protection_mode,protection_level from v$database;
```
3. Abort instance running managed recovery process (MRP).

```
shutdown abort
```

What to look for

- Notification of primary becoming unsynchronized


```
LGWR: Attempting destination LOG_ARCHIVE_DEST_2 network reconnect (3135)
LGWR: Error 1041 disconnecting from destination LOG_ARCHIVE_DEST_2 standby
host
```
- Notification of primary becoming synchronized


```
Destination LOG_ARCHIVE_DEST_2 is SYNCHRONIZED
```

After the first log switch, the PROTECTION_LEVEL drops to RESYNCHRONIZATION.

```
SQL> select inst_id,protection_mode,protection_level from gv$database;
```

INST_ID	PROTECTION_MODE	PROTECTION_LEVEL
1	MAXIMUM AVAILABILITY	RESYNCHRONIZATION
2	MAXIMUM AVAILABILITY	RESYNCHRONIZATION

The PROTECTION_LEVEL returns to MAXIMUM AVAILABILITY after the second log switch.

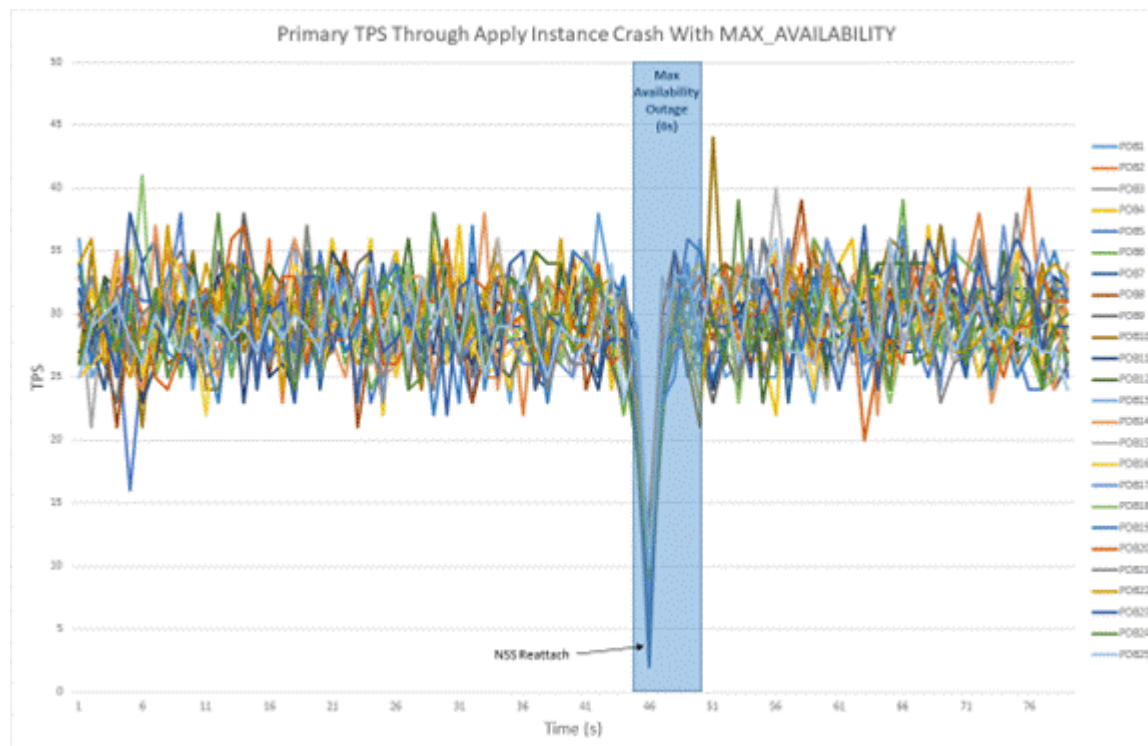
```
SQL> select inst_id,protection_mode,protection_level from gv$database;
```

INST_ID	PROTECTION_MODE	PROTECTION_LEVEL
1	MAXIMUM AVAILABILITY	MAXIMUM AVAILABILITY
2	MAXIMUM AVAILABILITY	MAXIMUM AVAILABILITY

Expected impact on primary

With the loss of the apply instance, the primary database immediately switches a log to demote the destination to RESYNCHRONIZATION MODE. After the primary is able to reconnect to the standby and ship the missing redo, a second log switch occurs to promote the destination to SYNCHRONOUS, restoring MAXIMUM AVAILABILITY.

Figure 19-1 Primary Database Workload with Apply Instance Crash



If the standby instances are open read-only and an apply instance is killed, all standby instances must be brought down to the mount state. This is done using alter database close normal. If this close operation takes a long time due to a large number of read-only connections, set `_ABORT_ON_MRP_CRASH=TRUE`. With the database initialization parameter `ABORT_ON_MRP_CRASH=TRUE`, the standby aborts and restarts all other instances and brings them to the mount state.

Kill Standby Database (All Instances)

How to run a test

1. Start workload on the primary database.
2. Ensure that the primary database is in Maximum Availability mode.

```
select protection_mode,protection_level from v$database;
```

3. Abort all standby database instances at the same time using shutdown abort.

```
srvctl stop database -d <standby> -o abort
```

What to look for

- Notification of primary becoming unsynchronized

```
Destination LOG_ARCHIVE_DEST_2 is UNSYNCHRONIZED
```

- Notification of primary becoming synchronized

```
Destination LOG_ARCHIVE_DEST_2 is SYNCHRONIZED
```

Note

This notification only occurs after the standby has been restarted.

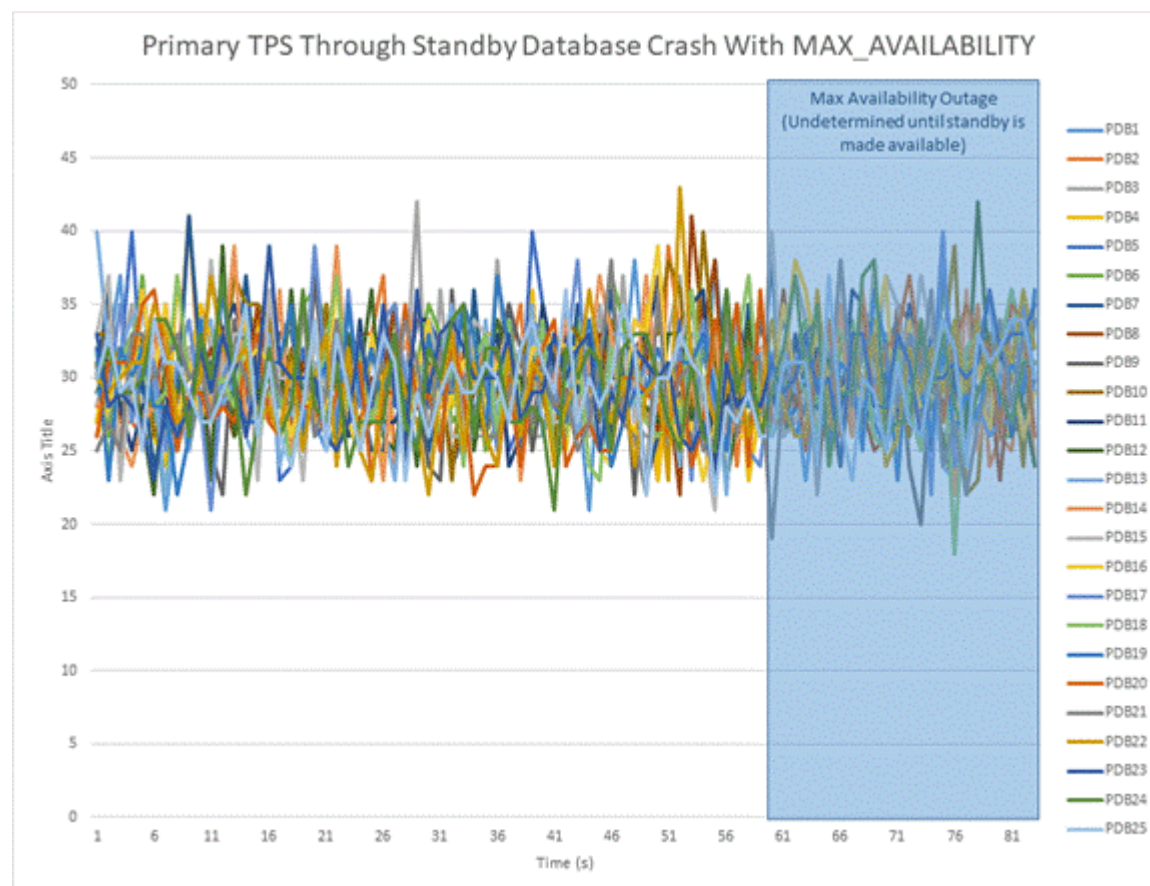
After the log switch the PROTECTION_LEVEL drops to RESYNCHRONIZATION.

```
SQL> select inst_id,protection_mode,protection_level from gv$database;
```

INST_ID	PROTECTION_MODE	PROTECTION_LEVEL
1	MAXIMUM AVAILABILITY	RESYNCHRONIZATION
2	MAXIMUM AVAILABILITY	RESYNCHRONIZATION

Expected impact on primary

Sync transport detects the instance crashes immediately, and there should not be any additional brownout for the application. With the loss of the entire standby database, the primary database immediately switches a log in order to demote the destination to RESYNCHRONIZATION MODE. The primary does not return to MAXIMUM AVAILABILITY until the standby is brought back to at least a mounted state, at which point the gap in redo will be resolved and log shipping may resume. A second brief brownout occurs when the standby is again made available to the primary due to the necessary log switch to put the configuration back into MAXIMUM AVAILABILITY.

Figure 19-2 Primary Database Workload with Standby Database Crash

Oracle Active Data Guard Far Sync – Examples and Outage Scenarios

Oracle Active Data Guard Far Sync provides the ability to perform a zero data loss failover to a remote standby database without requiring a second standby database or complex operation. Far Sync enables this by deploying a Far Sync instance (a lightweight Oracle instance that has only a control file, server parameter file (SPFILE), password file, and standby log files-- there are no database files or online redo logs-- at a distance that is within an acceptable range of the primary for SYNC transport. A Far Sync instance receives redo from the primary via SYNC transport and immediately forwards the redo to up to 29 remote standby databases via ASYNC transport. A Far Sync instance can also forward redo to the Oracle Zero Data Loss Recovery Appliance.

An outage of a Far Sync instance running in Maximum Availability mode has no impact on the availability of the primary database other than a transitory brownout while the primary database receives an error notification. The primary resumes processing database transactions after notification is received. In most cases, notification is immediate, though there are certain fault conditions that suspend fault notification until the threshold for `net_timeout` expires (a user-configurable Data Guard broker property with a default of 30 seconds). This is standard operation for any Data Guard configuration that uses Maximum Availability mode. Data Guard's self-healing mechanisms automatically reconnect and resynchronize a standby

database once the problem that caused it to disconnect is resolved. These same automatic mechanisms also apply to Far Sync.

High availability (HA) in the Far Sync context addresses the ability to eliminate or minimize the potential for data loss should there be a double failure scenario, for example, a primary database outage immediately following a Far Sync outage or vice versa. HA in this context can be achieved in multiple ways. Each approach has its own trade-offs and considerations, which are described in the sections that follow. Combinations of the options listed are also feasible.

High Availability Using the Terminal Standby as an Alternate Destination

The simplest approach to maintaining data protection during a Far Sync outage is to create an alternate log archive destination pointing directly to the terminal standby (the ultimate failover target). Asynchronous transport (ASYNC) to the remote destination is the most likely choice in order to avoid the performance impact caused by WAN network latency. ASYNC can achieve near-zero data loss protection (as little as sub-seconds of exposure), but because it never waits for standby acknowledgment, it is unable to provide a zero data loss guarantee. During a Far Sync outage, redo transport automatically fails over to using the alternate destination. Once the Far Sync instance has been repaired and resumes operation, transport will automatically switch back to the Far Sync instance and zero data loss protection is restored.

Note that during a switchover to the terminal standby (a planned role transition), the configuration's protection mode must be reduced to Maximum Performance so that the mode is enforceable on the role transition target. Changing protection modes and transport methods is a dynamic operation with zero downtime.

The characteristics of this approach include:

- No additional Far Sync hardware or instances to manage.
- Loss of zero data loss coverage during a far sync instance outage. Data protection level drops to UNSYNCHRONIZED with ASYNC transport until the Far Sync instance is able to resume operation, synchronous communication is re-established, and the standby database becomes fully synchronized.

Relevant Data Guard broker properties for this example include:

- Primary (primary):
RedoRoutes='(LOCAL : farsyncA FASTSYNC ALT=(standby ASYNC FALLBACK))';
MaxFailure=1
NetTimeout=15
reopensecs=10
- Primary Far Sync "A" (farsyncA)
RedoRoutes='(primary:standby ASYNC)';
MaxFailure=1
NetTimeout=15
reopensecs=10
- Standby (standby):
RedoRoutes='(LOCAL : primary ASYNC)';
MaxFailure=1
NetTimeout=15
reopensecs=10

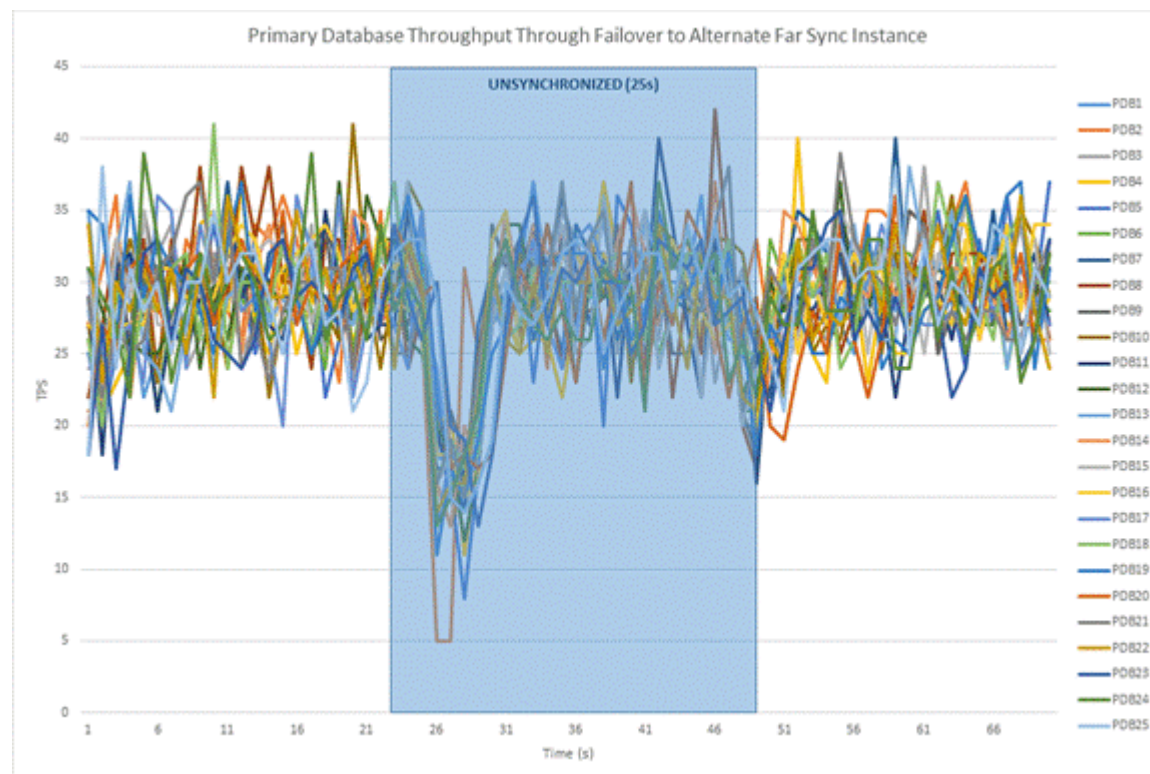
High Availability Using an Alternate Far Sync Instance

A more robust approach to maintaining data protection in the event of a Far Sync outage is to deploy a second Far Sync instance as an alternative destination. Unlike the previous example, the alternate Far Sync instance prevents the data protection level from being degraded to Maximum Performance (ASYNCR) while the first Far Sync instance is repaired.

The following image shows what occurs during an outage when the active Far Sync instance in this example has failed but the server on which it was running is still operating.

- There is a brief application brownout of 3 to 4 seconds while the primary database connects to the alternate Far Sync instance.
 - In this failure state, an error is immediately returned to the primary database without waiting for NetTimeout. The brownout can be calculated based on property settings $((\text{MaxFailure}-1)*\text{ReopenSecs}) + 4\text{s}$.
- The maximum potential data loss, should there be a second outage that impacts the primary database before the configuration is resynchronized, is a function of the amount of redo generated while the primary reconnects to the alternate Far Sync instance and the configuration is resynchronized.

Figure 19-3 Primary Database Workload with Failover to Alternate Far Sync Instance



Behavior is slightly different in a failure state where the server on which the Far Sync instance is running crashes. The application brownout is extended by an additional NetTimeout seconds, and the potential exposure to data loss should a second outage impact the primary is increased due to the additional processing required before resynchronization is complete.

The characteristics of this approach include:

- Reduced data loss exposure should there be a second outage that impacts the primary database before a Far Sync instance is repaired and returned to service.
- A symmetrical configuration that is able to maintain zero data loss protection following a role transition (planned switchover) that promotes the standby database to the primary role (the original primary becomes a zero data loss failover target). Please note that an SQL*Plus configuration with an alternate destination will automatically fall back to the initial log archive destination when it becomes available. Within a Data Guard broker configuration, the FALLBACK keyword in the RedoRoutes property is used to enable this behavior.

Relevant configuration Data Guard broker properties for this example include:

- Primary (primary):
RedoRoutes='(LOCAL : farsyncA FASTSYNC ALT=(farsyncB ASYNC FALLBACK))';
MaxFailure=1
NetTimeout=15
reopensecs=10
- Primary Far Sync "A" (farsyncA)
RedoRoutes='(primary:standby ASYNC)';
MaxFailure=1
NetTimeout=15
reopensecs=10
- Far Sync "B" (farsyncB)
RedoRoutes='(primary:standby ASYNC)';
MaxFailure=1
NetTimeout=15
reopensecs=10
- Standby (standby):
RedoRoutes='(LOCAL : SBfarsyncA FASTSYNC ALT=(SBfarsyncB ASYNC FALLBACK))';
MaxFailure=1
NetTimeout=15
reopensecs=10
- Standby Far Sync "A" (SBfarsyncA)
RedoRoutes='(standby:primary ASYNC)';
MaxFailure=1
NetTimeout=15
reopensecs=10
- Standby Far Sync "B" (SBfarsyncB)
RedoRoutes='(standby:primary ASYNC)';
MaxFailure=1
NetTimeout=15
reopensecs=10

Far Sync Using Oracle Real Application Clusters

A Far Sync instance can also be placed on an Oracle RAC. In this configuration, Far Sync is only active on one server at a time while other servers provide automatic failover for HA.

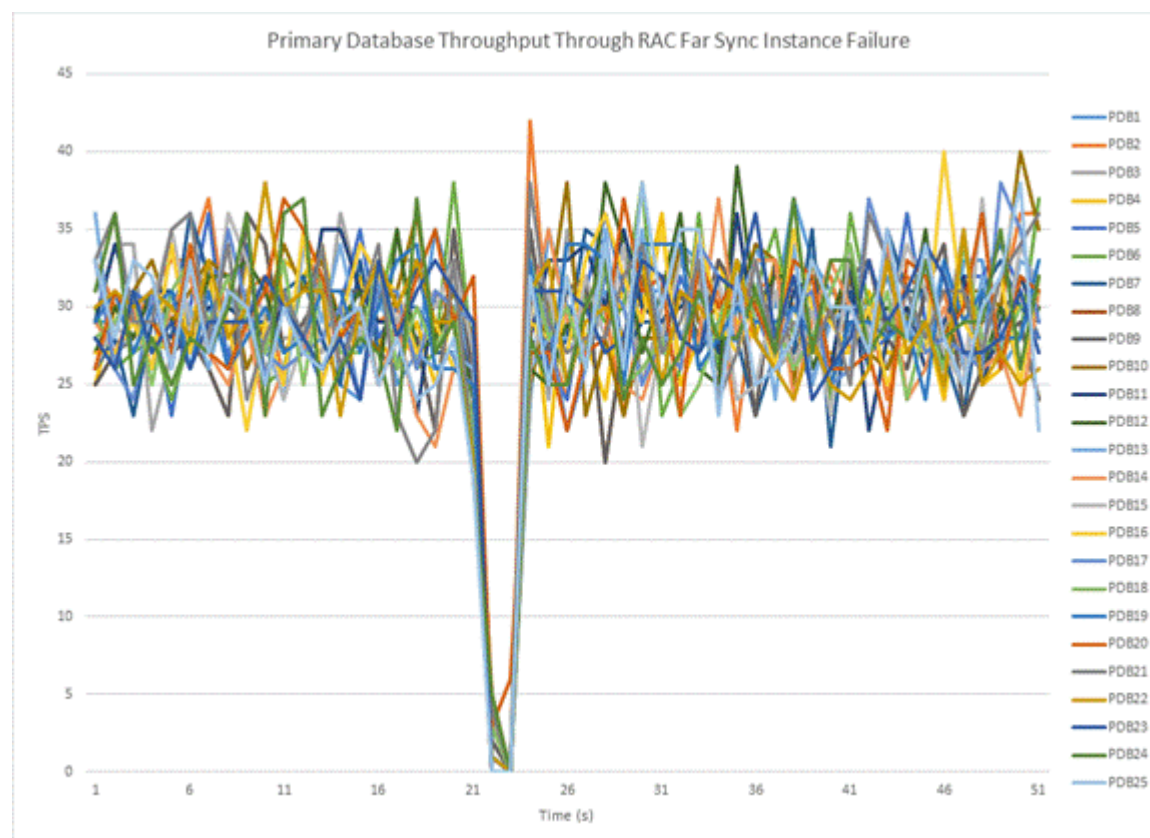
The characteristics of this approach include:

- Lowest application brownout when a Far Sync instance fails.
- Fastest resumption of zero data loss protection after Far Sync instance failure.
- This solution does not address cluster failure by itself; if the cluster becomes unavailable, configuring an alternate destination is still required to maintain data protection.

Far Sync Instance Failure in an Oracle RAC Cluster

The following image shows the impact on application throughput and data protection should there be a failure of the Far Sync instance receiving redo when all Oracle RAC nodes are still functional.

Figure 19-4 Primary Database Workload with RAC Far Sync Instance Failure



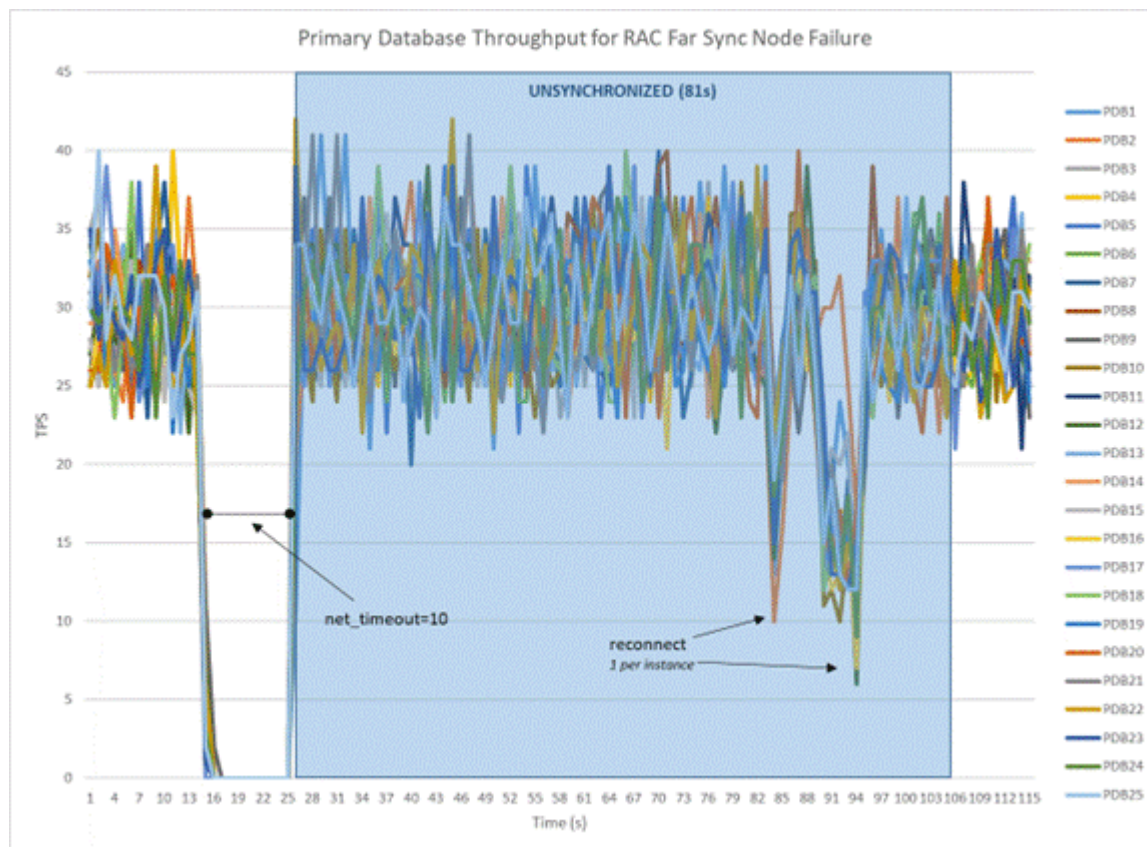
In this use case, when the active Far Sync instance fails, Oracle RAC immediately detects the outage and automatically fails over redo transport to an already running Far Sync instance on one of the surviving Oracle RAC nodes (no alternate destination needs to be defined for this to occur). There is a very brief application brownout of less than one second to acknowledge and process the error notification during instance failover (`net_timeout` does not apply to this error state). The configuration remains at the Maximum Availability protection level, maintaining zero data loss protection throughout the transition from one node of the cluster to the next – no re-

transmission is necessary due to the new Far Sync instance being able to access existing SRLs on the cluster's shared storage. Fast detection and no interruption of zero data loss protection are substantial benefits of using Oracle RAC to host the Far Sync instance.

Node (Server) Failure in an Oracle RAC Cluster

The following image shows the impact of an Oracle RAC node failure--an outage of the server on which the active Far Sync instance is running. Node failure incurs a brief brownout equal to the Data Guard NetTimeout property causing the first dip in application throughput. Then a second brownout occurs while Data Guard redo transport re-establishes connection with the surviving node. Data loss potential is greater in this case because the node failure results in Data Guard entering a resynchronization state. Data Guard quickly resynchronizes the primary and standby and returns the configuration to a zero data loss protection level after the new connection is established. The time required to resynchronize the configuration will vary depending on how much redo needs to be transmitted to the surviving instance.

Figure 19-5 Primary Database Workload with RAC Far Sync Node Failure



Primary Database Outage Repair

The following topic describes outage repair for various outages that could occur on the primary side of an Oracle Data Guard configuration.

Sync Transport Process Death

How to run a test

1. Start workload on the primary database.
2. Ensure that the primary database is in Maximum Availability mode.

```
select protection_mode,protection_level from v$database;
```

3. Find the NSS process for one instance.

```
ps -aef | grep ora_nss
```

4. Kill the NSS process.

```
kill -9 <pid>
```

What to look for

- Notification of primary becoming unsynchronized

```
Destination LOG_ARCHIVE_DEST_2 is UNSYNCHRONIZED
```

- Notification of primary becoming synchronized

```
Destination LOG_ARCHIVE_DEST_2 is SYNCHRONIZED
```

After the first log switch, the PROTECTION_LEVEL drops to RESYNCHRONIZATION.

```
SQL> select inst_id,protection_mode,protection_level from gv$database;
```

INST_ID	PROTECTION_MODE	PROTECTION_LEVEL
1	MAXIMUM AVAILABILITY	RESYNCHRONIZATION
2	MAXIMUM AVAILABILITY	RESYNCHRONIZATION

The PROTECTION_LEVEL returns to MAXIMUM AVAILABILITY after the second log switch.

```
SQL> select inst_id,protection_mode,protection_level from gv$database;
```

INST_ID	PROTECTION_MODE	PROTECTION_LEVEL
1	MAXIMUM AVAILABILITY	MAXIMUM AVAILABILITY
2	MAXIMUM AVAILABILITY	MAXIMUM AVAILABILITY

Expected impact on primary

With the loss of a sync transport process on the primary, LGWR detects sync transport death in approximately 10 seconds, after which a log switch occurs in order to demote the destination to RESYNCHRONIZATION mode. Up to 60 seconds later, heartbeat ARCH initiates the respawn of the sync transport process followed by a log switch to promote the destination to MAXIMUM AVAILABILITY.

Loss of Network Between Primary and Standby Databases

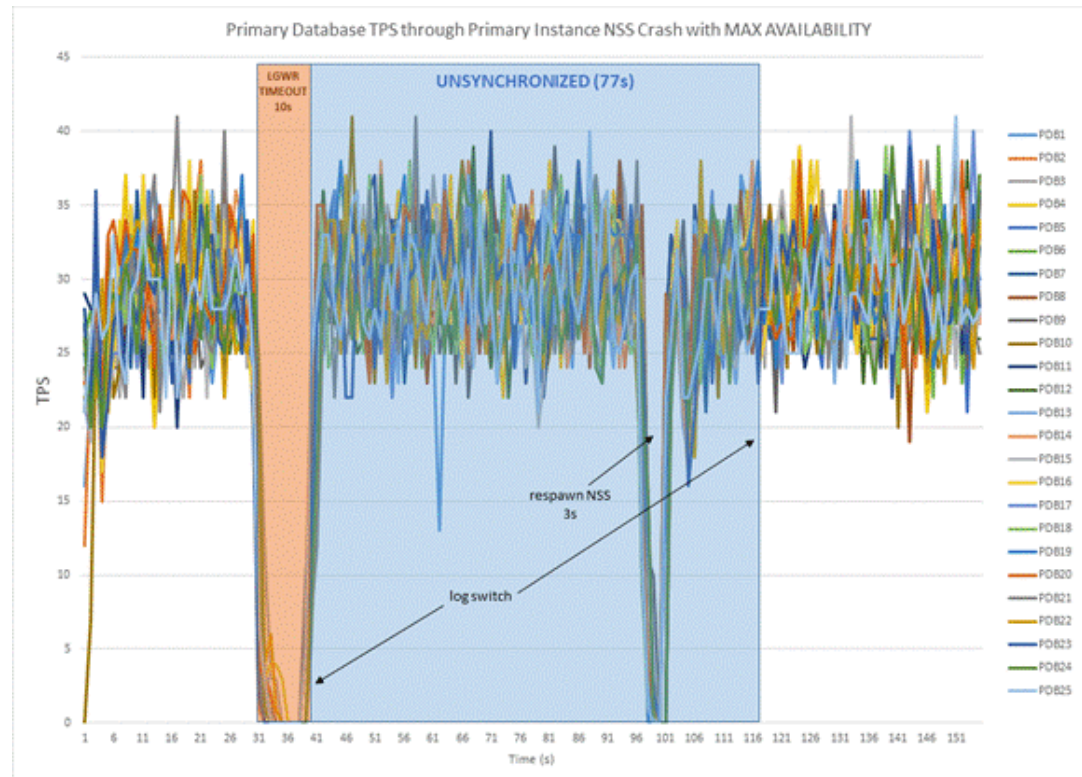
How to run a test

1. Set the standby properties NetTimeout=30 or 5 and MaxFailure=1.

2. Start workload on the primary database.
3. Ensure that the primary database is in Maximum Availability mode.

```
select protection_mode, protection_level from v$database;
```

Figure 19-6 Primary Database Workload with Loss of Sync Transport Process



4. Down the standby interfaces used by the primary as root.

```
ifdown eth0
```

What to look for

- Notification of primary becoming unsynchronized

Destination LOG_ARCHIVE_DEST_2 is UNSYNCHRONIZED

Note

The destination will not become resynchronized until the network is restored, the destination FAILURE_COUNT is reset, and the primary reconnects with the standby.

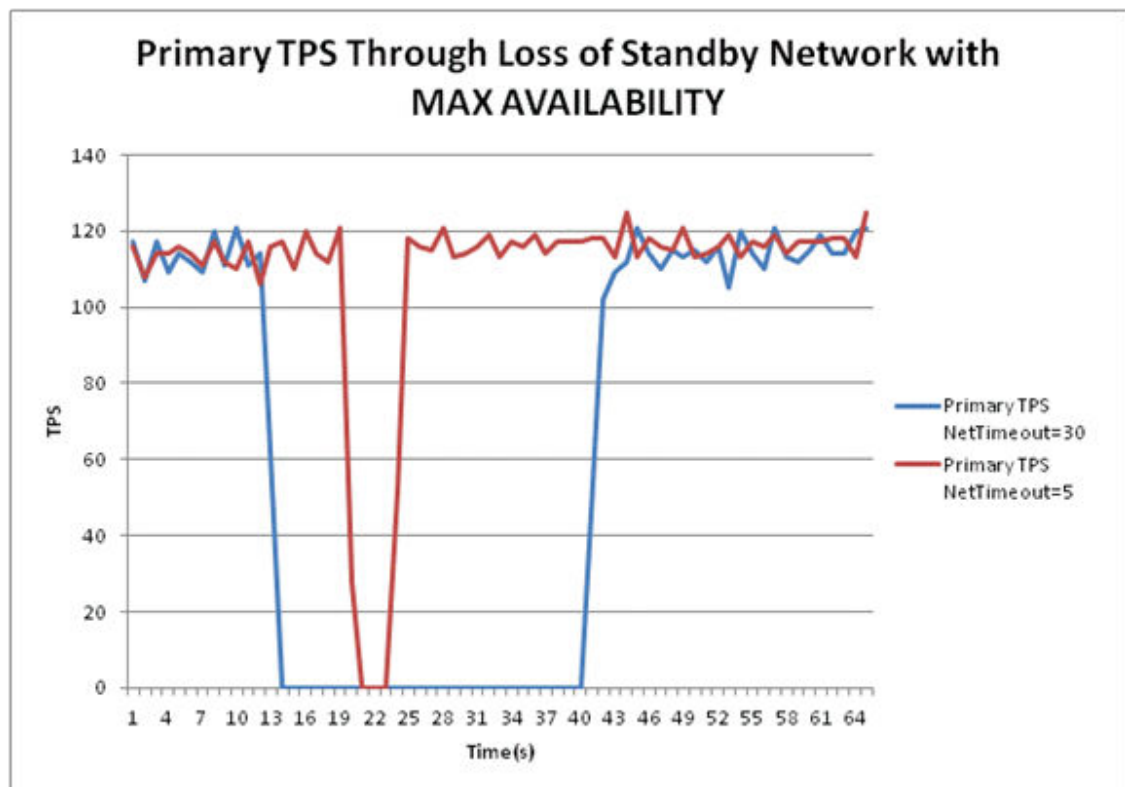
Note

Once the failure count for the destination reaches the specified MaxFailure property value, the only way to reuse the destination is to modify the MaxFailure property value (setting it to the current value qualifies) or any property. This has the effect of resetting the failure count to zero (0).

Expected impact to primary

After the loss of the network the primary freezes all processing for NetTimeout seconds waiting for the network. Provided MaxFailure=1, once NetTimeout expires the destination drops to RESYNCHRONIZATION and processing continues unprotected. NetTimeout should not be set lower than 5 and should only be set as low as 5 on a low latency reliable network.

Figure 19-7 Primary Database workload with Loss of Standby Network

**Primary Instance Death****How to run a test**

1. Start workload on the primary database.
2. Ensure that the primary database is in Maximum Availability mode.

```
select protection_mode, protection_level from v$database;
```

3. Abort one instance of the primary.

```
shutdown abort
```

What to look for

- There is no explicit warning in the alert log of the drop to resync however it is implied due to the loss of instance and necessary instance recovery.
- Notification of reconfiguration and instance recovery

```
Reconfiguration started (old inc 3, new inc 5)
<...>
Instance recovery: looking for dead threads
<...>
Completed instance recovery at
```

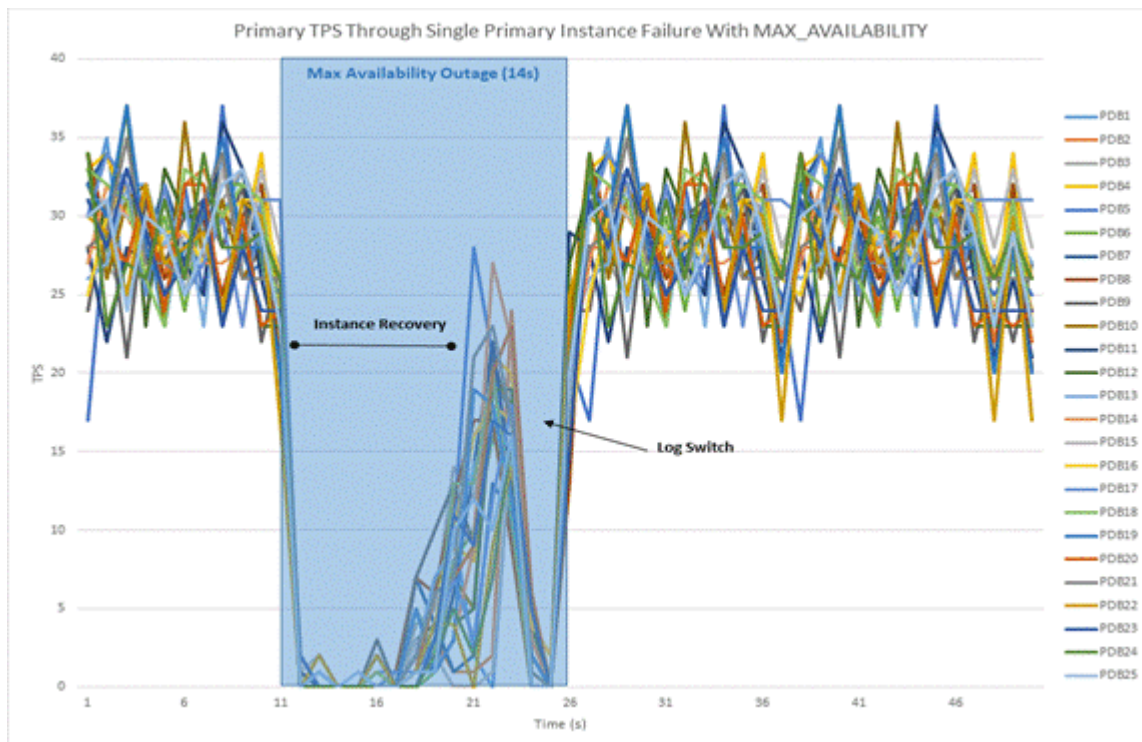
- Notification of primary becoming synchronized

```
Client pid [#####] attached to RFS pid [#####] at remote instance number
[#] at dest '<standby>'
```

Expected Impact on Primary

After the instance is lost, one of the surviving instances performs instance recovery, and the remote destination must be re-initialized followed by a log switch, which puts the configuration back into MAXIMUM AVAILABILITY protection.

Figure 19-8 RAC Primary Database Workload with Primary instance Failure



Primary Database Outage and Failover

How to run a test

1. Start workload on the primary database.
2. Ensure that the primary database is in Maximum Availability mode.

```
select protection_mode, protection_level from v$database;
```

3. Abort the primary database.

```
shutdown abort
```

4. Initiate failover to standby, using manual or Fast-start Failover (FSFO).

What to look for

- Beginning of the failover in the initial standby alert log

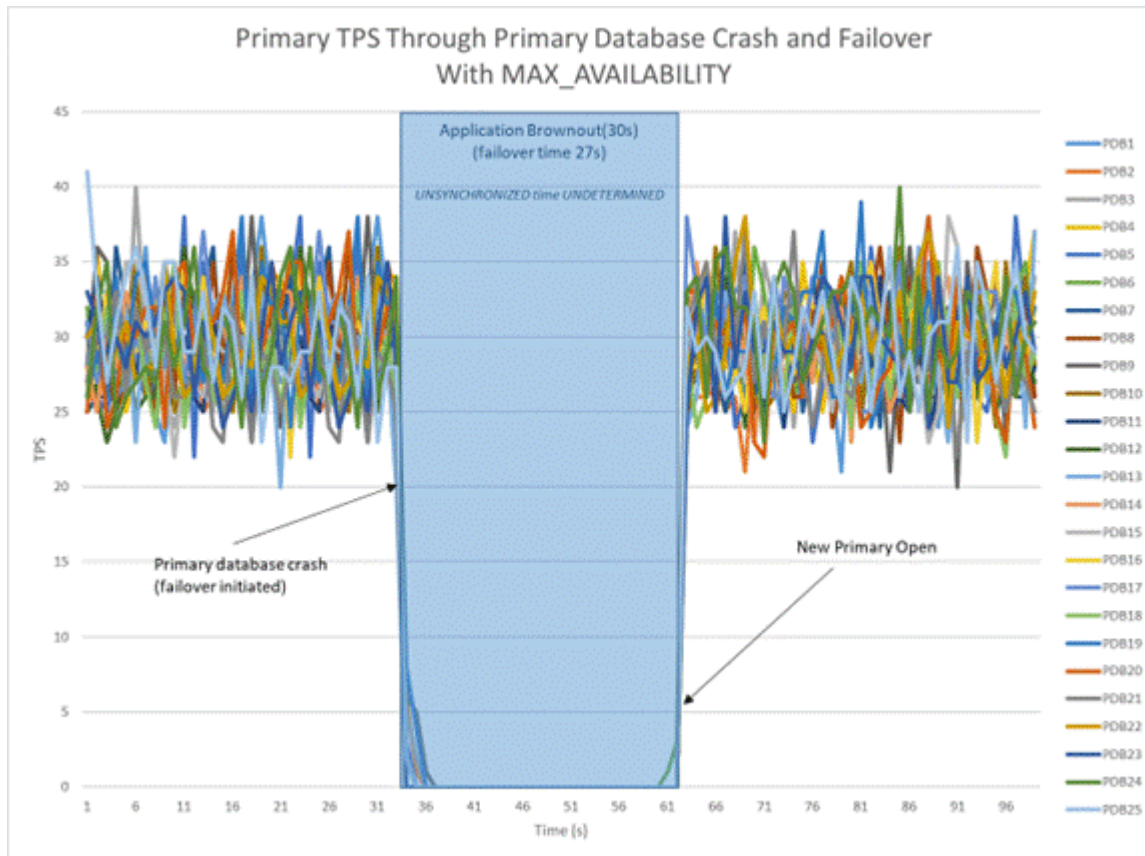
```
Data Guard Broker: Beginning failover
```

- Completion of the failover in the initial standby alert log

```
Failover succeeded. The primary database is now '<standby>'
```

Expected impact on primary

After the failover, the primary database opens, providing database availability to the application. MAXIMUM AVAILABILITY protection is not restored until the old primary database is reinstated and resynchronized.

Figure 19-9 Primary Database Workload with Primary Database Crash and Failover

Part V

MAA Platinum and Oracle GoldenGate Best Practices

- [MAA Platinum Reference Architecture Overview](#)
- [Overview of Oracle GoldenGate Best Practices](#)
- [Cloud Within Region: Configuring Oracle GoldenGate Hub](#)
- [Cloud Across Regions: Configuring Oracle GoldenGate Hub](#)
- [Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices](#)
- [Cloud MAA Platinum: Oracle GoldenGate Microservices Architecture Integrated with Active Data Guard](#)
- [On-Premises: Configuring Oracle GoldenGate Hub](#)
- [On-Premises: Oracle GoldenGate Microservices Architecture with Oracle Real Application Clusters Configuration Best Practices](#)
- [On-Premises MAA Platinum: Oracle GoldenGate Microservices Architecture Integrated with Active Data Guard](#)
- [Managing Outages for Oracle GoldenGate Hub](#)
- [Oracle GoldenGate Active-Active Guidance for Developers and Administrators](#)
- [Troubleshooting Oracle GoldenGate](#)

MAA Platinum Reference Architecture Overview

MAA Platinum or Never-Down Architecture, delivers near-zero Recovery Time Objective (RTO, or downtime incurred during an outage) and potentially zero or near zero Recover Point Objective (RPO, or data loss potential).

The MAA Platinum reference architecture ensures:

- RTO = zero or near-zero for all local failures using the Oracle Exadata Database Machine platform with its inherent Oracle RAC, full-stack redundancy, and failover capabilities
- RTO = zero or near-zero for disasters, such as database, cluster, or site failures, achieved by redirecting the application to an active Oracle GoldenGate source or target
- Zero downtime maintenance for software and hardware updates using Oracle RAC and Exadata Database Machine platform
- Zero downtime database upgrade or application upgrade by redirecting the application to an upgraded Oracle GoldenGate source or target database
- RPO = zero or near-zero data loss, depending on the Oracle Data Guard protection mode setting, which dictates the redo transport (SYNC, FAR SYNC, or ASYNC)
- Fast re-synchronization and zero or near-zero RPO between Oracle GoldenGate source and target databases after a disaster.

After any database failure, automatic failover to its standby database occurs automatically. Subsequently, automatic re-synchronization between Oracle GoldenGate source and target databases will resume. For SYNC transport, this leads to eventual zero data loss.

Table 20-1 MAA Platinum Outage Matrix

Event	RTO/RPO Service Level Objective ¹
Unplanned Outage	
Recoverable node or instance failure	Zero or single digit seconds ^{2,3}
Disasters including corruptions and site failures	Zero ³
Planned Maintenance	
Most common software and hardware updates	Zero ²
Major database upgrade or application upgrade	Zero ³

¹RPO=0 unless explicitly specified

²To achieve zero downtime or lowest impact for online processing, apply MAA application high availability best practices (also known as The Checklist). For long running transactions, such as batch operations, it's recommended that you defer them outside the planned maintenance window.

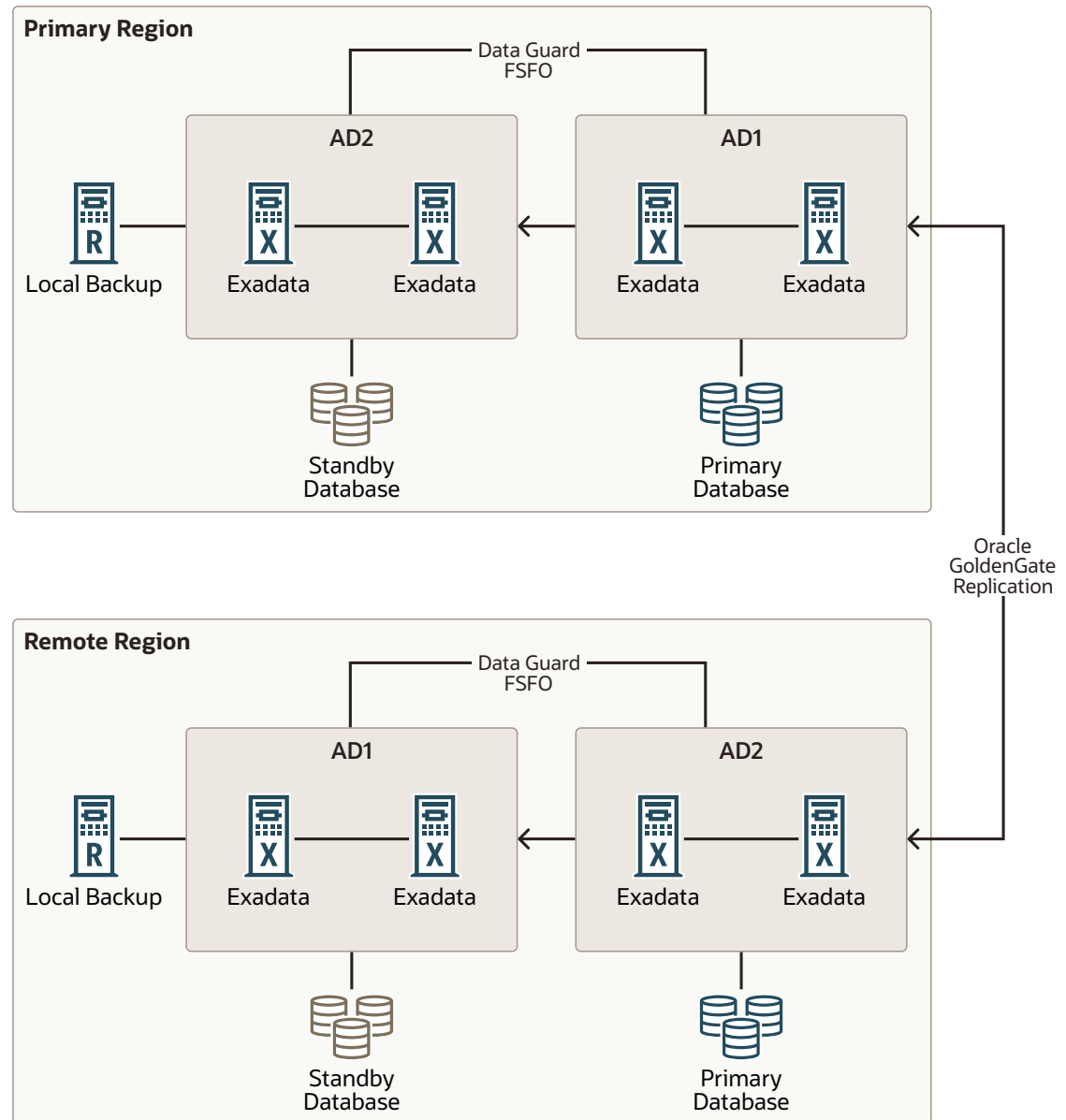
³Application failover is customized or managed with Global Data Services.

Enabling features of the Platinum MAA solution include:

- **Oracle Real Application Clusters** with recommended **Exadata** Database Machine Platform, Exadata Database Service on Dedicated Infrastructure (ExaDB-D), or Oracle Exadata Database Service on Cloud@Customer (ExaDB-C@C) for the source and target databases
- **Oracle Active Data Guard** with Fast-Start Failover to bound data loss and automatically fail over to standby in case of database, cluster, or data center failures. The standby databases are typically in separate Fault Domains (FDs) with separate power supplies, residing in separate data centers, Availability Domains (ADs), or Availability Zones (AZs), with independent power and network. The standbys can also reside across regions with typically the greatest fault isolation.
- **Oracle GoldenGate** enables two active read-write database systems that can be leveraged for applications to fail over immediately after database, cluster, site failure, or planned outages such as database or application upgrades. The source and target databases on which Oracle GoldenGate replication is occurring can reside in the same region, across ADs, or across regions.

MAA Platinum architecture is illustrated in the image below, where two "active read-write" primary databases, or source or target databases, reside in separate regions. Oracle GoldenGate replication occurs between the source and target databases between primary and remote regions. Each primary database is protected by a standby database in another AD within the same region.

With Data Guard Fast-Start Failover (FSFO), the standby database becomes the new primary database automatically after primary database, cluster, or AD failure. With MAA Oracle GoldenGate configuration best practices implemented, replication resumes automatically between the source and target databases after any Data Guard role transition. Each database resides on an Exadata platform with its inherent built-in Real Application Cluster, system and storage redundancy, and low brownout failover capabilities.

Figure 20-1 MAA Platinum Reference Architecture

MAA Platinum Architecture Variants

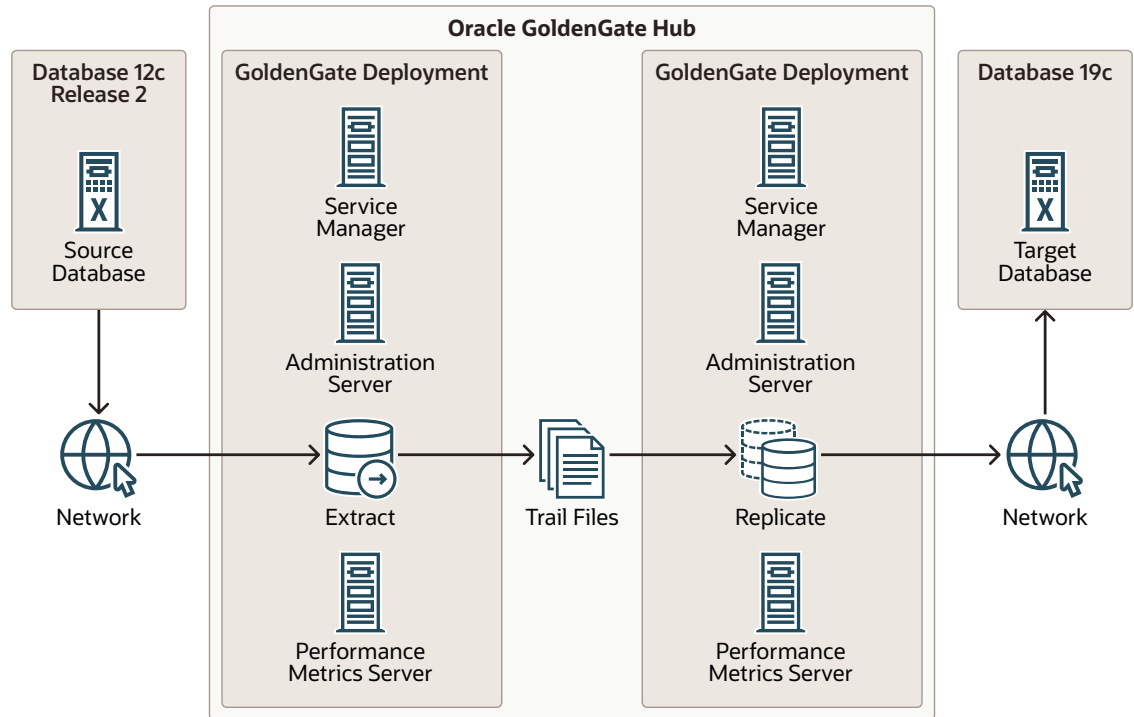
When setting up MAA Platinum architecture, the administrator can decide between setting up Oracle GoldenGate on each potential source or target database, or creating an Oracle GoldenGate hub independent from the database servers.

The MAA Oracle GoldenGate hub, shown in the following image, provides the following advantages:

- Offloads Oracle GoldenGate software installation, configuration, and life cycle management from source and target Exadata database systems.
- Reduces Oracle GoldenGate resource impact on the source and target database systems.

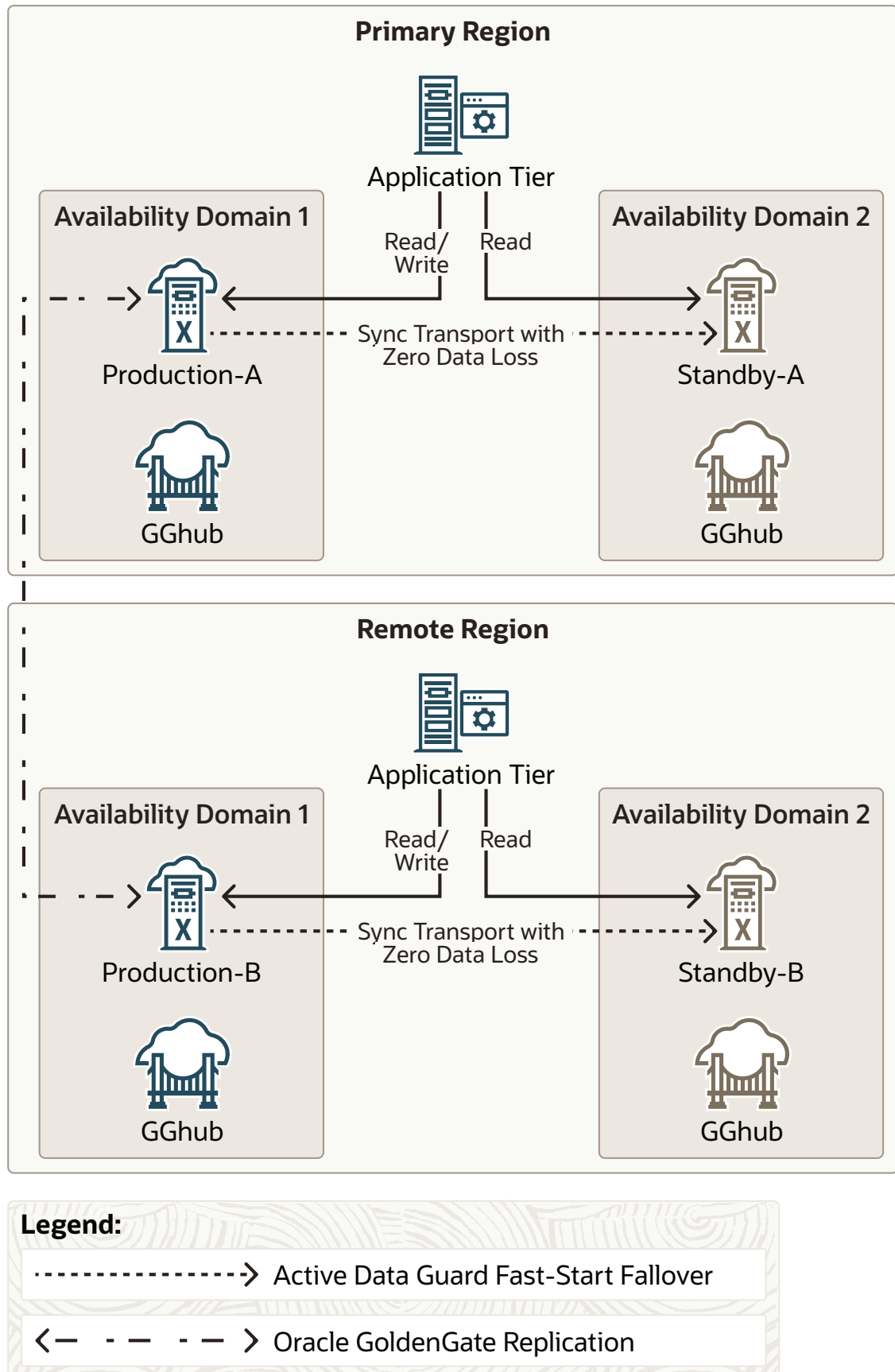
- Provides high availability by configuring a 2-node cluster server for fast and simple failover, and disaster recovery by leveraging ACFS replication to another identical GoldenGate hub server on a separate 2-node cluster server.
- Consolidates Oracle GoldenGate configurations and software deployment for multiple independent MAA Platinum or Oracle GoldenGate architectures.

Figure 20-2 MAA Oracle GoldenGate Hub



An example of MAA Oracle GoldenGate hub with MAA Platinum architecture is shown in the image below. Each hub is a 2-node cluster providing local high availability, and for additional protection uses ACFS replication to another hub, typically deployed across Availability Domains (ADs) or across regions.

Figure 20-3 MAA Platinum with Oracle GoldenGate Hub



How to Implement the MAA Platinum Solution

To achieve an MAA Platinum solution, review and leverage the technical papers and documentation referenced in the following steps.

Step 1 - Choose Oracle GoldenGate Deployment

Decide between implementing the recommended MAA Oracle GoldenGate Hub solution, or setting up Oracle GoldenGate directly on Exadata database servers.

Oracle GoldenGate Deployment	Guidance
MAA Oracle GoldenGate Hub	This is the recommended deployment configuration Deploy MAA Oracle GoldenGate Hub on Exadata Database Service in Oracle Cloud • Cloud Within Region: Configuring Oracle GoldenGate Hub • Cloud Across Regions: Configuring Oracle GoldenGate Hub Deploy MAA Oracle GoldenGate Hub on Exadata On-Premises • On-Premises: Configuring Oracle GoldenGate Hub
Oracle GoldenGate	Deploy Oracle GoldenGate on Exadata Database Service in Oracle Cloud • Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices • Cloud MAA Platinum: Oracle GoldenGate Microservices Architecture Integrated with Active Data Guard Deploy Oracle GoldenGate on Exadata On-Premises • On-Premises: Oracle GoldenGate Microservices Architecture with Oracle Real Application Clusters Configuration Best Practices • On-Premises MAA Platinum: Oracle GoldenGate Microservices Architecture Integrated with Active Data Guard

Step 2 - Configure Automatic Conflict Detection and Resolution

Bidirectional replication requires conflict detection and resolution. Oracle MAA provides guidance for developers and administrators when designing an active-active distributed database architecture using Oracle GoldenGate, where data conflicts may occur between GoldenGate replicas. See [Oracle GoldenGate Active-Active Guidance for Developers and Administrators](#) for additional details.

Step 3 - Implement Application Failover

Oracle Global Data Services allows fault-tolerant database services to be deployed and centrally managed across a set of replicated databases, supporting seamless inter-database service failover among replicated databases in any data center, and providing higher application availability. See [Oracle Global Data Services Best Practices](#) for additional details.

Alternatively, use your own custom application routing to redirect after database or site failure.

Overview of Oracle GoldenGate Best Practices

Configure Oracle GoldenGate using Oracle MAA best practices to get the highest availability and performance out of your Oracle GoldenGate deployment.

Oracle GoldenGate provides the following benefits:

- Uni-directional or bi-directional replication, allowing reads and updates in any replicated database.
- Data movement is in real-time, reducing latency.
- Replicated databases can run on different hardware platforms, database versions, and different database or application configurations, allowing for online migration. This flexibility also allows online database and application upgrades.
- Source and target replicated databases are online, ensuring zero downtime application switchover during outages and planned maintenance activities is possible. Note that application switchover must be customized, rather than using a built-in feature, such as Transparent Application Continuity.
- MAA Oracle GoldenGate Hub (MAA GGHUB) can reduce the complexity and system resource utilization by moving GoldenGate software and processes off of the Exadata database servers.
- Dependable disaster recovery solutions by replicating data across different geographic locations. This allows for failover during regional outages or data center problems, minimizing recovery time objectives (RTO) and fulfilling recovery point objectives (RPO).

The following table highlights various Oracle GoldenGate configuration best practices and MAA Platinum best practices.

Table 21-1 Oracle GoldenGate Use Cases and Best Practices

Use Case	Oracle GoldenGate Best Practices
Database migration to Oracle Cloud or Exadata Platform	Zero Downtime Migration (ZDM) with GoldenGate (logical migration) Oracle Zero Downtime Migration – Logical Migration Performance Guidelines
Database migration requiring minimal or zero downtime Database migration involving cross platform or different database versions	Oracle Database Migration with an Oracle GoldenGate Hub Configuration
Deploy Oracle GoldenGate off of the database server in a Hub configuration	Cloud Within Region: Configuring Oracle GoldenGate Hub Cloud Across Regions: Configuring Oracle GoldenGate Hub On-Premises: Configuring Oracle GoldenGate Hub

Table 21-1 (Cont.) Oracle GoldenGate Use Cases and Best Practices

Use Case	Oracle GoldenGate Best Practices
Install and configure Oracle GoldenGate directly on the Oracle RAC database server or Exadata database system	Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices On-Premises: Oracle GoldenGate Microservices Architecture with Oracle Real Application Clusters Configuration Best Practices
Implement MAA Platinum or install and configure Oracle GoldenGate directly on Oracle RAC database servers with Oracle Active Data Guard	Oracle Cloud: 1. Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices and 2. Cloud MAA Platinum: Oracle GoldenGate Microservices Architecture Integrated with Active Data Guard On-premises systems: 1. On-Premises: Oracle GoldenGate Microservices Architecture with Oracle Real Application Clusters Configuration Best Practices and 2. On-Premises MAA Platinum: Oracle GoldenGate Microservices Architecture Integrated with Active Data Guard
Application failover options for Oracle GoldenGate	Oracle Global Data Services Best Practices and Configuring Continuous Availability for Applications
Oracle GoldenGate bidirectional replication and automatic conflict detection and resolution	For Oracle Cloud Service, Oracle Cloud Infrastructure GoldenGate documentation For on-premises, the latest Oracle GoldenGate documentation For these Oracle GoldenGate or Platinum MAA architectures, active-active replication requires conflict detection and resolution, which includes Automatic Conflict Detection and Resolution (Auto-CDR) or the manual CompareCols / ResolveConflict (CC/RC) method of conflict detection and resolution. Also see Manual Conflict Detection and Resolution in Oracle GoldenGate documentation.

Also see Oracle GoldenGate documentation at: <https://docs.oracle.com/en/middleware/goldengate/core/21.1/>

Overview of Oracle GoldenGate and Supporting Technologies

The technologies that are required to replicate data between databases are Oracle GoldenGate and supporting technologies (such as Oracle Grid Infrastructure Agents, ACFS or DBFS, or GoldenGate hub) to ensure that replication will resume after various failures. A brief overview of Oracle GoldenGate and supporting technologies are described here.

Oracle GoldenGate

Oracle GoldenGate provides real-time, log-based change data capture and delivery between homogenous and heterogeneous systems. This technology lets you construct a cost-effective and low-impact real-time data integration and continuous availability solution.

Oracle GoldenGate replicates data from committed transactions with transaction integrity and minimal overhead on your existing infrastructure. The architecture supports multiple data replication topologies, such as one-to-many, many-to-many, cascading, and bidirectional. Its wide variety of use cases includes real-time business intelligence; query offloading; zero-downtime upgrades and migrations; and active-active databases for data distribution, data synchronization, and high availability.

Oracle GoldenGate Microservices Architecture provides REST-enabled services. The REST-enabled services provide remote configuration, administration, and monitoring through HTML5 web pages, command line interfaces, and APIs.

Recommended Oracle GoldenGate 21c (and higher releases) introduces unified build support, so that a single software installation supports capturing and applying replicated data to multiple major Oracle Database versions (11g Release 2 to 21c). This is possible because an Oracle GoldenGate installation includes the required Oracle Database client libraries without requiring a separate database ORACLE_HOME installation.

Oracle Grid Infrastructure Agents

Oracle Grid Infrastructure Agents (XAG) are Oracle Grid Infrastructure components that provide the high availability (HA) framework to application resources and resource types managed through the agent management interface, AGCTL. This framework provides a complete, ready-to-use solution that contains pre-defined Oracle Grid Infrastructure resource configurations and agents to integrate applications for complete application HA.

The Oracle Grid Infrastructure Agents provide pre-defined Oracle Clusterware resources for Oracle GoldenGate, Siebel, Oracle PeopleSoft, JD Edwards, and Oracle WebLogic Server, as well as Apache and MySQL applications. Using the agent for Oracle GoldenGate simplifies the creation of dependencies on the source and target databases, the application VIP, and the file system (ACFS or DBFS) mount point. The agent command line utility (AGCTL) is used to start and stop Oracle GoldenGate, and can also be used to relocate Oracle GoldenGate between the nodes in the cluster.

Oracle Database File System (DBFS)

Oracle DBFS can be used to store Oracle GoldenGate files.

The Oracle Database File System (DBFS) creates a file system interface to files stored in the database. DBFS is similar to NFS in that it provides a shared network file system that looks like a local file system. Because the data is stored in the database, the file system inherits all the high availability and disaster recovery capabilities provided by Oracle Database.

With DBFS, the server is the Oracle Database. Files are stored as SecureFiles LOBs. PL/SQL procedures implement file system access primitives such as create, open, read, write, and list directory. The implementation of the file system in the database is called the DBFS SecureFiles Store. The DBFS SecureFiles Store allows users to create file systems that can be mounted by clients. Each file system has its own dedicated tables that hold the file system content.

Oracle Advanced Cluster File System (ACFS)

Oracle ACFS can be used to store Oracle GoldenGate files.

Oracle Advanced Cluster File System (Oracle ACFS) is a multi-platform, scalable file system, and storage management technology that extends Oracle Automatic Storage Management (Oracle ASM) functionality to support all customer files.

Oracle ACFS leverages Oracle Clusterware for cluster membership state transitions and resource-based high availability. Oracle ACFS is bundled into the Oracle Grid Infrastructure (GI) allowing for integrated optimized management of databases, resources, volumes, and file systems.

Cloud Within Region: Configuring Oracle GoldenGate Hub

Configure and deploy MAA Oracle GoldenGate Hub architecture within one region on Oracle Cloud using the provided planning considerations, tasks, management, and troubleshooting information.

See the following topics:

- [Overview of MAA GoldenGate Hub](#)
- [Planning GGHub Placement in the Platinum MAA Architecture](#)
- [Task 1: Configure the Source and Target Databases for Oracle GoldenGate](#)
- [Task 2: Deploy Oracle GoldenGate Maximum Availability Hub on Oracle Cloud Marketplace](#)
- [Task 3: Configure the Oracle GoldenGate Environment](#)

Overview of MAA GoldenGate Hub

To achieve the highest levels of availability, resulting in zero or near-zero downtime for both unplanned outages and planned maintenance activities, you can use the combination of Oracle Real Application Clusters (Oracle RAC), Oracle Active Data Guard, and Oracle GoldenGate.

This architecture, typically referred as MAA Platinum, or Never Down Architecture, delivers near zero Recovery Time Objective (RTO--downtime incurred during outage) and potentially zero or near zero Recovery Point Objective (RPO--data loss potential).

Traditionally, Oracle GoldenGate is installed and run locally on the database server that the GoldenGate processes connect to. When used with Oracle Grid Infrastructure Standalone Agent (XAG), Oracle GoldenGate processes can be configured to seamlessly relocate or failover between Oracle RAC nodes and follow Oracle Active Data Guard switchover and failovers.

Using MAA Oracle GoldenGate Hub (MAA GGHub) moves the GoldenGate software and processes off of the Exadata database servers, reducing complexity and system resource utilization. MAA GGHub centralizes Oracle GoldenGate management and offloads the majority of the Oracle GoldenGate processing and associated CPU and storage resource utilization from Exadata system resources. Connectivity between the GoldenGate processes and the databases they operate against is managed with Oracle Net Services.

To achieve an MAA Platinum solution in the Oracle Cloud, you follow these high level steps:

1. Review [MAA Platinum and MAA Diamond: Oracle RAC and Exadata](#) to understand Platinum MAA benefits and use cases.
2. Deploy or migrate your database onto Exadata Cloud Service, Base Database Service, or Autonomous Database on Dedicated Infrastructure Service.
3. Add symmetric standby databases in the Oracle Cloud using Oracle Cloud Control Plan or Cloud automation.

4. Configure and deploy Oracle Data Guard Fast Start Failover using the Oracle MAA best practice recommendations in [Configure Fast Start Failover](#). For Exadata Cloud Service and Base Database Service, this is manual step.
5. Set up MAA GGHUB within a single OCI region, which is detailed in the topics that follow. For an MAA GGHUB setup across OCI regions, see [Cloud Across Regions: Configuring Oracle GoldenGate Hub](#).
6. Configure Bidirectional Replication and Automatic Conflict Detection and Resolution. See [Oracle Cloud Infrastructure GoldenGate documentation](#) for information.
7. Decide on Application Failover Options such as Global Data Services (see [Oracle Global Data Services Best Practices](#)), or use your own customized application failover.

Planning GGHUB Placement in the Platinum MAA Architecture

Extreme availability that delivers zero downtime (RTO=0 or near zero) and zero or near zero data loss (RPO=0 or near zero) typically requires the following Platinum MAA architecture.

1. You have the source and target database in an Oracle GoldenGate architecture to allow your application to fail over immediately in the case of disaster (database, cluster, or site failure) or switch over in the case of a database or application upgrade. This architecture enables the potential RTO of zero or near zero for disaster scenarios and database and application upgrade maintenance.
2. Each source and target database is deployed in Exadata cloud systems so any local failures are tolerated or recovered almost instantly.
3. Each source and target database is configured with a standby database with Data Guard Fast-Start Failover so any failure of the database results in activating a new primary database in seconds to minutes. If SYNC transport is leveraged with Max Availability protection mode, zero data loss Data Guard failover is achieved.
4. Configured with GoldenGate replication using MAA GGHUB between the source and target databases.
5. Configured so that any standby becoming a primary database due to Data Guard switchover or failover will automatically resynchronize with its target GoldenGate database. If zero data loss Data Guard switchover or failover occurs, GoldenGate resynchronization ensures zero data loss across the distributed database environment.
6. Configured with GoldenGate Automatic Conflict Detection and Resolution, which is required after any Data Guard failover operation occurs.

Where to Place the MAA Primary GGHUB and Standby GGHUB

1. The GGHUB Pair (Primary and Standby GGHUB) must reside in the same OCI regions as each primary and standby database. For example:
 - a. If the primary database is in AD1, Region A, and the standby database is in AD2, Region A, then the GGHUB pair will reside in Region A. For this configuration, continue reading the topics in this chapter.
 - b. If the primary database is in Region A and the standby database is in Region B, then the GGHUB pair will split between Region A and B. The primary, or active, GGHUB must be co-located in the same OCI region as the target primary database. For this configuration, see [Cloud Across Regions: Configuring Oracle GoldenGate Hub](#).
2. Performance implications:

- a. Primary or active GGHUB must reside in the same data center as the target database to ensure round trip latency of 4ms or less. (Replicat performance)
 - b. Primary or active GGHUB should be < 90 ms from the source database without incurring GoldenGate performance degradation. (Extract performance)
3. GoldenGate distribution path:
 - a. A GoldenGate distribution path is required if the source and target GGHUBs are in different regions and latency between the OCI regions is > 90 ms.
 - b. In Oracle Cloud, when your Oracle GoldenGate source and target databases reside in the same region, or in different regions in the same country, you never need to set up a GoldenGate distribution path because the latency is always < 90 ms.

MAA GGHUBs Placed in the Same OCI Region

In this scenario, the primary and standby database are located in the same OCI region, and so the primary (active) GGHUB and the standby GGHUB are also located in the same region.

The following architectural components comprise the GGHUBs, as shown in the image below:

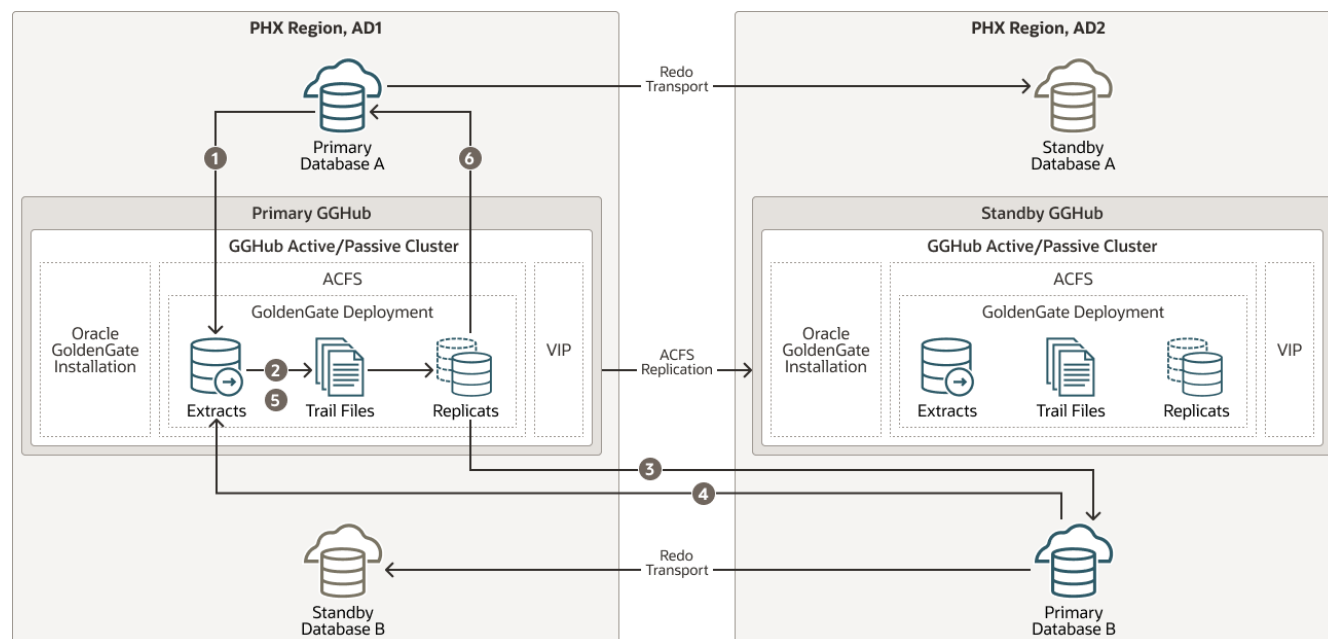
1. Primary database and associated standby database are configured with Oracle Active Data Guard Fast Start Failover (FSFO). FSFO can be configured with any Data Guard protection mode, with ASYNC or SYNC redo transport, depending on your maximum data loss tolerance.
2. Primary GGHUB Active/Passive Cluster: Only one GGHUB software deployment and configuration on the 2-node cluster. This cluster contains the 21c Oracle GoldenGate software deployment that can support Oracle Database 11g (11.2.0.4) and later releases.

This GGHUB can support many primary databases and encapsulates the GoldenGate processes. GoldenGate Extract mines transactions from the source database and GoldenGate Replicat applies the same changes to target database. GoldenGate trail and checkpoint files also reside in the GGHUB ACFS file system.

The HA failover solution is built in to the GGHUB, which includes automatic failover to the passive node in the same cluster, and restarts GoldenGate processes and activity after a node failure.

3. Standby GGHUB Active/Passive Cluster: A Symmetric standby GGHUB is configured. ACFS replication is set up between the primary and standby GGHUBs to preserve all GoldenGate files.

Manual GGHUB failover, which includes ACFS failover, can be performed in the rare case that you lose the entire primary GGHUB.

Figure 22-1 Primary and Standby GGHubs in the Same OCI Region

The figure above depicts data replicated from Primary Database A to Primary Database B and Primary B back to Primary A with the following steps:

1. Primary Database A: Primary A's Logminer server sends redo changes to a Primary GGHUB Extract process.
2. Primary GGHUB: An Extract process writes changes to trail files.
3. Primary GGHUB to Primary Database B: A Primary GGHUB Replicat process applies those changes to the target database (Primary B).
4. Primary Database B: Primary B's Logminer server sends redo to a Primary GGHUB Extract process.
5. Primary GGHUB: A Primary GGHUB Extract process writes changes to trail files.
6. Primary GGHUB to Primary Database A: A Primary GGHUB Replicat process applies those changes to the target database (Primary A).

Note that one GGHUB can support multiple source and target databases, even when the source and target databases are different Oracle Database releases.

Table 22-1 Outage Scenarios, Repair, and Restoring Redundancy for GGHubs in the Same OCI Region

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Primary Database A (or Database B) failure	Impact: Near-zero application downtime. GoldenGate replication resumes when a new primary database starts. 1. One primary database is still available. All activity is routed to the existing available primary database to achieve zero application downtime. Refer to Global Data Service Global Services Failover solution. For example, application services A-F are routed to Database A and application services G-J are routed to Database B. If Database A fails, all application services temporarily go to Database B. 2. The standby becomes the new primary automatically with Data Guard FSFO. Oracle GoldenGate replication resumes and the primary databases resynchronize. Data loss is bounded by the Data Guard protection level. If Maximum Availability or Maximum Protection is configured, zero data loss is achieved. All committed transactions are in one or both databases. Workload can be “rebalanced” when Primary Database A and Database B are available and in sync. For example, when Database A is up and running and in sync, services A-F can go back to Database A.	1. The old primary database is reinstated as the new standby database to restore redundancy. 2. Optionally performing a Data Guard switchover to switch back to the original configuration ensures that at least one primary database resides in an independent AD.
Primary or standby GGHUB single node failure	Impact: No application impact. GoldenGate replication resumes automatically after a couple of minutes. No action is required. The HA failover solution built in to the GGHUB includes automatic failover and restart of GoldenGate processes and activity. Replication activity is blocked until GoldenGate processes are active again. GoldenGate replication blackout could last a couple of minutes.	Once the node restarts, active/passive configuration is re-established.

Table 22-1 (Cont.) Outage Scenarios, Repair, and Restoring Redundancy for GGHUBs in the Same OCI Region

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Primary GGHUB cluster crashes and is not recoverable	Impact: No application impact. GoldenGate replication resumes after restarting the existing GGHUB or performing a manual GGHUB failover operation. 1. If the GGHUB cluster can be restarted, then that's the simplest solution. 2. If the primary GGHUB is not recoverable, then perform a manual GGHUB failover to the standby GGHUB, which includes ACFS failover. This typically takes several minutes. 3. GoldenGate replication stops until the new primary GGHUB is available, so performing step 1 or step 2 should take little time.	If the previous GGHUB eventually restarts, ACFS replication resumes in the other direction automatically. If the GGHUB cluster is lost or unrecoverable, you need to rebuild a new standby GGHUB.
Standby GGHUB cluster crashes and is not recoverable	Impact: No application or replication impact. 1. If the GGHUB cluster can be restarted, then that is the simplest solution, and ACFS replication can resume. 2. If the standby GGHUB is not recoverable, you can rebuild a new standby GGHUB.	N/A

Table 22-1 (Cont.) Outage Scenarios, Repair, and Restoring Redundancy for GGHubs in the Same OCI Region

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Complete Data Center or Availability Domain (AD1 or AD2) failure	Impact: Near-zero application downtime. GoldenGate replication resumes when the new primary database starts. - One primary database is still available. All activity is routed to the existing available primary database to achieve zero application downtime. Refer to Global Data Service Global Services Failover solution. For example, application services A-F are routed to Database A and application services G-J are routed to Database B. If Database A fails, all services temporarily go to Database B. - If the primary GGHub is still functional, GoldenGate replication continues. If the primary GGHub is lost due to availability domain (AD) failure, then a manual GGHub failover is required. GoldenGate replication resumes and the primary databases resynchronize. Data loss is bounded by the Data Guard protection level. If Maximum Availability or Maximum Protection is configured, zero data loss is achieved. All committed transactions are in one or both databases. Workload can be rebalanced when Primary Database A and Database B are available and in sync. When Database A is up and running and in sync, services A-F can go back to Database A.	- When the data center/AD returns, re-establish the configuration, such as reinstate standby. If the previous GGHub eventually restarts, ACFS replication resumes in the other direction automatically. - When possible, perform a Data Guard switchover (back) to get back to the original state where one primary database exists in each AD.

Task 1: Configure the Source and Target Databases for Oracle GoldenGate

The source and target Oracle GoldenGate databases should be configured using the recommendations that follow.

Step 1.1 - Configure the Databases

For Oracle Autonomous Database (ADB-D), Oracle Database 19c (19.20) or a later release is required to support parallel integrated Replicat and conflict resolution. For Oracle Exadata Database Service (ExaDB) or BaseDB, you can use any supported Oracle Database release available in the Oracle cloud.

The database configuration steps that follow are applicable for each Database Cloud Service that supports Platinum MAA solution.

The source and target Oracle GoldenGate databases should be configured using the following recommendations:

For Oracle Autonomous Database (ADB-D), you only need to add supplemental logging:

```
PDB: ALTER PLUGGABLE DATABASE ADD SUPPLEMENTAL LOG DATA;
```

For Oracle Exadata Database Service (ExaDB) or BaseDB do the following steps:

1. Enable Archivelog Mode.

```
SQL> ARCHIVE LOG LIST
Database log mode                Archive Mode
Automatic archival                Enabled
Archive destination               USE_DB_RECOVERY_FILE_DEST
Oldest online log sequence       110
Next log sequence to archive     113
Current log sequence              113
```

2. Enable Force Logging.

```
ALTER DATABASE FORCE LOGGING;
```

3. Enable GoldenGate replication.

```
ALTER SYSTEM SET ENABLE_GOLDENGATE_REPLICATION=TRUE SCOPE=BOTH SID='*';
```

4. Add supplemental logging.

```
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA;
```

5. Configure STREAMS_POOL_SIZE larger to accommodate GoldenGate.

Use this formula to calculate the appropriate STREAMS_POOL_SIZE value:

$$\text{STREAMS_POOL_SIZE} = (((\# \text{Extracts} + \# \text{Integrated Replicats}) * 1\text{GB}) * 1.25)$$

For example, in a database with 2 Extracts and 2 integrated Replicats:

$$\text{STREAMS_POOL_SIZE} = 4\text{GB} * 1.25 = 5\text{GB}$$

And the parameter is set:

```
ALTER SYSTEM SET STREAMS_POOL_SIZE=5G SCOPE=BOTH SID='*';
```

For the steps to prepare the database for Oracle GoldenGate, see [Preparing the Database for Oracle GoldenGate](#).

Step 1.2 - Create a GoldenGate Database Administrator User

For ADB-D

For ADB-D deployments, you only need to issue

```
ALTER USER ggadmin IDENTIFIED BY <password> ACCOUNT UNLOCK;
```

For Oracle Exadata Database Service (ExaDB) or BaseDB

The following steps are only applicable for Oracle Exadata Database Service (ExaDB) or BaseDB.

The source and target databases need a GoldenGate administrator user created, with appropriate privileges assigned as follows:

- For the multitenant container database (CDB):
 - Source database - GoldenGate Extract must be configured to connect to a user in the root container database, using a c##
 - Target database - a separate GoldenGate administrator user is needed for each pluggable database (PDB).
 - For details about creating a GoldenGate administrator in an Oracle Multitenant Database, see [Configuring Oracle GoldenGate in a Multitenant Container Database](#).
- For non-CDB databases, see [Establishing Oracle GoldenGate Credentials](#)
- 1. As the oracle OS user on the source database system, execute the following SQL instructions to create the database user for Oracle GoldenGate and assign the required privileges:

```
[opc@exadb1_node1 ~]$ sudo su - oracle
[oracle@exadb1_node1 ~]$ source dbName.env
[oracle@exadb1_node1 ~]$ sqlplus / as sysdba

# Source CDB
SQL>
alter session set container=cdb$root;
create user c##ggadmin identified by "ggadmin_password" container=all
default tablespace USERS temporary tablespace temp;
alter user c##ggadmin quota unlimited on users;
grant set container to c##ggadmin container=all;
grant alter system to c##ggadmin container=all;
grant create session to c##ggadmin container=all;
grant alter any table to c##ggadmin container=all;
grant resource to c##ggadmin container=all;
exec
dbms_goldengate_auth.grant_admin_privilege('c##ggadmin',container=>'all');

# Source PDB
SQL>
alter session set container=pdbName;
create user ggadmin identified by "ggadmin_password" container=current;
```



```
grant create session to ggadmin container=current;
grant alter any table to ggadmin container=current;
grant resource to ggadmin container=current;
exec dbms_goldengate_auth.grant_admin_privilege('ggadmin');
```

2. As the oracle OS user on the target system, execute the following SQL instructions to create the database user for Oracle GoldenGate and assign the required privileges:

```
[opc@exadb2_node1 ~]$ sudo su - oracle
[oracle@exadb2_node1 ~]$ source dbName.env
[oracle@exadb2_node1 ~]$ sqlplus / as sysdba

# Target PDB
SQL>
alter session set container=pdbName;
create user ggadmin identified by "ggadmin_password" container=current;
grant alter system to ggadmin container=current;
grant create session to ggadmin container=current;
grant alter any table to ggadmin container=current;
grant resource to ggadmin container=current;
grant dv_goldengate_admin, dv_goldengate_redo_access to ggadmin
container=current;
exec dbms_goldengate_auth.grant_admin_privilege('ggadmin');
```

Step 1.3 - Create the Database Services

Note

This step is not required for ADB-D deployments.

If the source and target databases are running the recommended configuration on an Oracle RAC cluster with Oracle Data Guard, a role-based service must be created that allows the Extract or Replicat processes to connect to the correct Data Guard primary database instance.

When using a source multitenant database, a separate service is required for the root container database (CDB) and the pluggable database (PDB) that contains the schema being replicated. For a target multitenant database, a single service is required for the PDB.

1. As the oracle OS user on the primary database system, use `dbaascli` to find the CDB and PDB name, as shown here:

```
[opc@exadb1_node1 ~]$ sudo su - oracle
[oracle@exadb1_node1 ~]$ source dbName.env
[oracle@exadb1_node1 ~]$ dbaascli database getDetails
--dbname dbName |egrep 'dbName|pdbName'

"dbName" : "dbName",
"pdbName" : "pdbName",
```

2. As the oracle OS user on the primary and standby database systems, create and start the CDB database service using the following command:

```
[opc@exadb1_node1 ~]$ sudo su - oracle
[oracle@exadb1_node1 ~]$ source dbName.env
[oracle@exadb1_node1 ~]$ srvctl add service -db $ORACLE_UNQNAME
-service dbName.goldengate.com -preferred ORACLE_SID1
-available ORACLE_SID2 -role PRIMARY
```

3. As the oracle OS user on the primary and standby database systems, create and start the PDB database service using the following command:

```
[oracle@exadb1_node1 ~]$ srvctl add service -db $ORACLE_UNQNAME
-service dbName.pdbName.goldengate.com -preferred ORACLE_SID1
-available ORACLE_SID2 -pdb dbName -role PRIMARY
```

4. As the oracle OS user on the primary and standby database systems, start and verify that the services are running, as shown here:

```
[oracle@exadb1_node1 ~]$ srvctl start service -db $ORACLE_UNQNAME -role
[oracle@exadb1_node1 ~]$ srvctl status service -d $ORACLE_UNQNAME |grep
goldengate
```

```
Service dbName.goldengate.com is running on instance(s) SID1
Service dbName.pdbName.goldengate.com is running on instance(s) SID1
```

Note

Repeat Step 1.3 in the source and target database system.

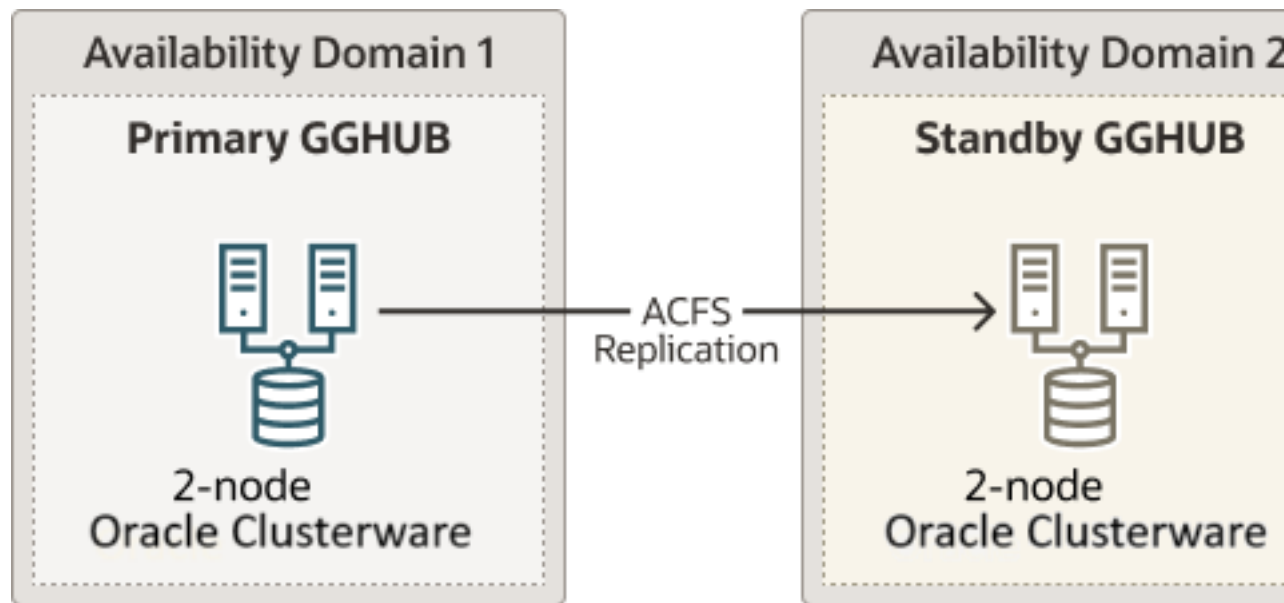
Task 2: Deploy Oracle GoldenGate Maximum Availability Hub on Oracle Cloud Marketplace

Oracle GoldenGate Maximum Availability Hub was designed to save you time in setting up and configuring your Oracle GoldenGate high availability solution.

It provides high availability by configuring a 2-node cluster server for fast and simple failover, and disaster recovery by leveraging Oracle Advanced Cluster File System (ACFS) replication to another identical GoldenGate hub server on a separate 2-node cluster server.

Follow the steps in [Using Oracle GoldenGate Maximum Availability Hub on Oracle Cloud Marketplace](#) to deploy the system, and come back to this topic to continue with the GGHUB configuration.

Figure 22-2 Oracle GoldenGate Maximum Availability Hub Hardware Architecture



Task 3: Configure the Oracle GoldenGate Environment

Step 3.1 - Create Database Credentials

With the Oracle GoldenGate deployment created, use the Oracle GoldenGate Administration Service home page to create the database credentials using the above TNS alias names. See figure 4 below for an example of the database credential creation using the TNS alias.

From a client machine with access to the GGHUB, create a ssh tunnel to connect to the Oracle GoldenGate Administration Service:

```
$ ssh -N -L <local_port>:<vip>:443 -p 22 <gghub-node>
```

As the oggadmin user, create the database credentials:

1. Log in into the Administration Service: https://localhost:<localPort>/<instance_name>/adminsrvr.
2. Click **Configuration** under **Administration Service**.
3. Click the plus button to **Add Credentials** under the **Database** tab.
4. Add the required information for the source and target CDB and PDB as shown in the table:

Region	Container	Domain	Alias	User ID
Region 1	CDB	GoldenGate	Reg1_CDB	c##ggadmin@<tns_alias>
Region 1	PDB	GoldenGate	Reg1_PDB	ggadmin@<tns_alias>

Region	Container	Domain	Alias	User ID
Region 2	CDB	GoldenGate	Reg2_CDB	c##ggadmin@<tns_alias>
Region 2	PDB	GoldenGate	Reg2_PDB	ggadmin@<tns_alias>

Step 3.2 - Create an Oracle Net TNS Alias for Oracle GoldenGate Database Connections

To provide local database connections for the Oracle GoldenGate processes when switching between nodes, create a TNS alias on **all** nodes of the cluster where Oracle GoldenGate may be started. Create the TNS alias in the `tnsnames.ora` file in the `TNS_ADMIN` directory specified in the deployment creation.

If the source database is a multitenant database, two TNS alias entries are required, one for the container database (CDB) and one for the pluggable database (PDB) that is being replicated. For a target Multitenant database, the TNS alias connects the PDB to where replicated data is being applied. The pluggable database `SERVICE_NAME` should be set to the database service created in an earlier step (refer to Step 2.3: Create the Database Services in [Task 2: Prepare a Primary and Standby Base System for GGHUB](#)).

As the oracle OS user on any database node of the primary and the standby database systems, use `dbaascli` to find the database domain name and the SCAN name:

```
# Primary DB
[opc@exadb1_node1]$ sudo su - oracle
[oracle@exadb1_node1]$ source db_name.env
[oracle@exadb1_node1]$ dbaascli database getDetails --dbname <db_name>
|grep 'connectString'

      "connectString" : "<primary_scan_name>:1521/<service_name>"

# Standby DB
[opc@exadb2_node1]$ sudo su - oracle
[oracle@exadb2_node1]$ source db_name.env
[oracle@exadb2_node1]$ dbaascli database getDetails --dbname <db_name>
|grep 'connectString'

      "connectString" : "<standby_scan_name>:1521/<service_name>"
```

As the oracle OS user on all nodes of the primary and standby GGHUB, add the recommended parameters for Oracle GoldenGate in the `sqlnet.ora` file:

```
[opc@gghub_prim1]$ sudo su - oracle
[oracle@gghub_prim1]$ mkdir -p /u01/app/oracle/goldengate/network/admin
[oracle@gghub_prim1]$
cat > /u01/app/oracle/goldengate/network/admin/sqlnet.ora <<EOF
DEFAULT_SDU_SIZE = 2097152
EOF
```

As the oracle OS user on all nodes of the primary and standby GGHUB, follow the steps to create the TNS alias definitions:

```
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghub_prim1 ~]$

cat > /u01/app/oracle/goldengate/network/admin/tnsnames.ora <<EOF

# Source
<source_cbd_service_name>=
  (DESCRIPTION =
    (CONNECT_TIMEOUT=3)(RETRY_COUNT=2)(LOAD_BALANCE=off)(FAILOVER=on)
  (RECV_TIMEOUT=30)
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<primary_scan_name>)
      (PORT=1521)))
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<standby_scan_name>)
      (PORT=1521)))
    (CONNECT_DATA=(SERVICE_NAME =
<source_cbd_service_name>.goldengate.com)))

<source_pdb_service_name>=
  (DESCRIPTION =
    (CONNECT_TIMEOUT=3)(RETRY_COUNT=2)(LOAD_BALANCE=off)(FAILOVER=on)
  (RECV_TIMEOUT=30)
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<primary_scan_name>)(PORT=1521)))
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<standby_scan_name>)(PORT=1521)))
    (CONNECT_DATA=(SERVICE_NAME =
<source_pdb_service_name>.goldengate.com)))

# Target
<target_pdb_service_name>=
  (DESCRIPTION =
    (CONNECT_TIMEOUT=3)(RETRY_COUNT=2)(LOAD_BALANCE=off)(FAILOVER=on)
  (RECV_TIMEOUT=30)
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<primary_scan_name>)(PORT=1521)))
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<standby_scan_name>)(PORT=1521)))
    (CONNECT_DATA=(SERVICE_NAME =
<target_pdb_service_name>.goldengate.com)))

EOF

[oracle@gghub_prim1 ~]$ scp /u01/app/oracle/goldengate/network/admin/*.ora
gghub_prim2:/u01/app/oracle/goldengate/network/admin
```

Note

When the `tnsnames.ora` or `sqlnet.ora` (located in the `TNS_ADMIN` directory for the Oracle GoldenGate deployment) are modified, the deployment needs to be restarted to pick up the changes.

Step 3.3 - Set Up Schema Supplemental Logging

1. Log in to the Oracle GoldenGate Administration Server.
2. Click **Configuration** under **Administration Service**.
3. Click the **Connect to database** button under **Actions** for the **Source Database (Reg_CDB)**.
4. Click the plus button (Add TRANDATA) to **Add TRANDATA** for the **Schema** or **Tables**.

Step 3.4 - Create the Autostart Profile

Create a new profile to automatically start the Extract and Replicat processes when the Oracle GoldenGate Administration Server is started. Then, restart if any Extract or Replicat processes are abandoned. With GoldenGate Microservices, auto start and restart is managed by Profiles.

Using the Oracle GoldenGate Administration Server GUI, create a new profile that can be assigned to each of the Oracle GoldenGate processes:

1. Log in to the **Administration Service** on the Source and Target GoldenGate.
2. Click on **Profile** under **Administration Service**.
3. Click the **plus (+)** sign next to Profiles on the Managed Process Settings home page.
4. Enter the details as follows:
 - Profile Name: Start_Default
 - Description: Default auto-start/restart profile
 - Default Profile: Yes
 - Auto Start: Yes
 - Auto Start Options
 - Startup Delay: 1 min
 - Auto Restart: Yes
 - Auto Restart Options
 - Max Retries: 5
 - Retry Delay: 30 sec
 - Retries Window: 30 min
 - Restart on Failure only: Yes
 - Disable Task After Retries Exhausted: Yes
5. Click **Submit**

Step 3.5 - Configure Oracle GoldenGate Processes

When creating Extract, Distribution Paths, and Replicat processes with Oracle GoldenGate Microservices Architecture, all files that need to be shared between the GGHub nodes are already shared with the deployment files stored on a shared file system.

Listed below are essential configuration details recommended for running Oracle GoldenGate Microservices on GGHub for Extract, Distribution Paths, and Replicat processes.

Perform the following sub-steps to complete this step:

- Step 4.4.1 - Extract Configuration
- Step 4.4.2 - Replicat Configuration
- Step 4.4.3 - Distribution Path Configuration
- Step 4.4.4 - Set up a Heartbeat Table for Monitoring Lag Times

The main goal is to prevent data divergence between GoldenGate replicas and their associated standby databases. This section focuses on configuring Extract so that GoldenGate Extract never gets ahead of the standby database which can result in data divergence.

GoldenGate Parameter	Description	Recommendations
TRANLOGOPTIONS HANDLEDLFAILOVER	This is mandatory setting for Data Guard configurations that have Oracle GoldenGate to ensure GoldenGate Extract never extract data that has not been received by standby database. The HANDLEDLFAILOVER stands for handle DATA LOSS for Data Guard failover. The following parameter must be added to the Extract process parameter file to avoid losing transactions and resulting in logical data inconsistencies after data loss Data Guard failover event. When the two primary tried to reconcile, this parameter ensures that all transactions can be reconciled since the new primary (old standby) is not further behind as expected. Prevents Extract from extracting redo data from the source database, and writing to the trail file data that has not yet been applied to the Oracle Data Guard standby database. If this parameter is not specified, after a data loss failover, it is possible to have data in the target database that is not present in the source database, leading to data divergence and logical data inconsistencies.	MANDATORY when the source database is configured with Data Guard in Max Availability or Max Performance mode.

GoldenGate Parameter	Description	Recommendations
TRANLOGOPTIONS FAILOVERTARGETDESTID <i>n</i>	For multiple standby configurations or cases when Data Guard Fast-Start failover is not enabled, set FAILOVERTARGETDESTID to standby demarcated by LOG_ARCHIVE_DEST to ensure GoldenGate Extract never extract data that has not been received by target standby database. To determine the correct value for FAILOVERTARGETDESTID, use the LOG_ARCHIVE_DEST_N parameter from the GoldenGate source database which is used for sending redo to the source standby database. For example, if LOG_ARCHIVE_DEST_2 points to the standby database, then use a value of 2. When not using Data Guard Fast Start Failover (FSFO) in the source database, this parameter identifies which standby database the Extract process must remain behind, with regard to not extracting redo data that has not yet been applied to the Oracle Data Guard standby database.	MANDATORY when not using FSFO in the source database. To determine the correct value for FAILOVERTARGETDESTID, use the LOG_ARCHIVE_DEST_N parameter from the GoldenGate source database which is used for sending redo to the source standby database. For example, if LOG_ARCHIVE_DEST_2 points to the standby database, then use a value of 2.
TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_WARNING <i>value</i>	The amount of time before a warning message is written to the Extract report file, if Extract is stalled, due to being unable to query the source database standby apply progress. This can occur after a Data Guard failover when the old primary database is not currently available. The default is 60 seconds.	OPTIONAL if want to adjust the timing of when the warning message is written to the Extract report file. Add STANDBY_WARNING <i>value</i> to the TRANLOGOPTIONS HANDLEDLFAILOVER parameter.
TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_ABEND <i>value</i>	The amount of time before Extract abends, if Extract is stalled, due to being unable to query the standby apply progress. The default is 30 minutes.	OPTIONAL if want to adjust the amount of time it takes Extract to abend, when the source database standby is not accessible to enforce the HANDLEDLFAILOVER parameter. Add STANDBY_ABEND <i>value</i> to the TRANLOGOPTIONS HANDLEDLFAILOVER parameter.

GoldenGate Parameter	Description	Recommendations
TRANLOGOPTIONS DLFAILOVER_TIMEOUT <i>value</i>	The amount of time Extract will run on the new source primary database, after a Data Guard role transition, before it will check the status of the standby database. If standby database is not available after the DLFAILOVER_TIMEOUT, Extract will abend. The default is 300 seconds. NOTE: If during normal operations of the source Oracle Data Guard configuration, the standby database becomes unavailable, Extract will stop extracting data from the source database to prevent possible data divergence with the GoldenGate target database due to the HANDLEDLFAILOVER parameter. The DLFAILOVER_TIMEOUT parameter does not take effect when a Data Guard failover has not occurred, and there are no messages output to the Extract report file.	OPTIONAL. If you want to adjust the amount of time an Extract can run on a new primary source database, after a role transition, when the standby is not yet available to honor the TRANLOGOPTIONS HANDLEDLFAILOVER parameter.

Refer to the [Oracle GoldenGate Parameters and Functions Reference Guide](#) for more information about the Extract TRANLOGOPTIONS parameters.

When creating an Extract using the Oracle GoldenGate Administration Service GUI interface, leave the Trail SubDirectory parameter blank so that the trail files are automatically created in the deployment directories stored on the shared file system. The default location for trail files is the /<deployment directory>/var/lib/data directory.

Note

To capture from a multitenant database, you must use an Extract configured at the root level using a c## account. To apply data into a multitenant database, a separate Replicat is needed for each PDB because a Replicat connects at the PDB level and doesn't have access to objects outside of that PDB.

Step 4.4.1 - Extract Configuration

Create the Extract:

1. Log in to the Oracle GoldenGate Administration Server.
2. Click **Overview** under **Administration Service**.
3. Click the **plus** button to **Add Extract**.
4. Select **Integrated Extract**.
5. Add the required information as follows:
 - Process Name: EXT_1
 - Description: Extract for Region 1 CDB
 - Intent: Unidirection

- Begin: Now
 - Trail Name: aa
 - Credential Domain: GoldenGate
 - Credential Alias: Reg1_CDB
 - Register to PDBs: PDB Name
6. Click **Next** and set parameters.

```
EXTRACT ext_1
USERIDALIAS Reg1_CDB DOMAIN GoldenGate
EXTTRAIL aaTRANLOGOPTIONS HANDLEDLFAILOVER
TRANLOGOPTIONS FAILOVERTARGETDESTID 2
SOURCECATALOG PDB_NAME
TABLE OWNER.*;
```

7. Click **Next**.
8. If using CDB Root Capture from PDB, add the SOURCECATALOG parameter with the PDB Name.
9. Click **Create and Run**.

Note

For ADB-D deployments, the extract requires a connection to the PDB rather than the CDB.

See [Oracle GoldenGate Extract Failure or Error Conditions Considerations](#) for more information.

Step 4.4.2 - Replicat Configuration

Oracle generally recommends using integrated parallel Replicat which offers better apply performance for most workloads when the GGHub is in the same region as the target Oracle GoldenGate database.

The best apply performance can be achieved when the network latency between the GGHub and the target database is as low as possible. The following configuration is recommended for the remote Replicat running on the Oracle GGHub.

- APPLY_PARALLELISM – Disables automatic parallelism, instead of using MAX_APPLY_PARALLELISM and MIN_APPLY_PARALLELISM, and allows the highest amount of concurrency to the target database. It is recommended to set this as high as possible based on available CPU of the hub and the target database server.
- MAP_PARALLELISM – Should be set with a value of 2 to 5. With a larger number of appliers, increasing the Mappers increases the ability to hand work to the appliers.
- BATCHSQL – applies DML using array processing which reduces the amount network overheads with a higher latency network. Be aware that if there are many data conflicts, BATCHSQL results in reduced performance, as rollback of the batch operations followed by a re-read from trail file to apply in non-batch mode.

Add a Replicat:

After you've set up your database connections and verified them, you can add a Replicat for the deployment by following these steps:

1. Log in to the Oracle GoldenGate Administration Server.
2. Click the plus (+) sign next to **Replicats** on the Administration Service home page. The Add Replicat page is displayed.
3. Select a Replicat type and click **Next**.
4. Enter the details as follows:
 - Process Name: REP_1
 - Description: Replicat for Region 2 PDB
 - Intent: Unidirectional
 - Credential Domain: GoldenGate
 - Credential Alias: Reg2_PDB
 - Source: Trail
 - Trail Name: aa
 - Begin: Position in Log
 - Checkpoint Table: "GGADMIN"."CHKP_TABLE"
5. Click **Next**.
6. From the **Action Menu**, click **Details** to edit the Replicat **Parameters**:

```

REPLICAT REP_1
USERIDALIAS Reg2_PDB DOMAIN GoldenGate
MAP <SOURCE_PDB_NAME>.<OWNER>.*, TARGET <OWNER>.*;

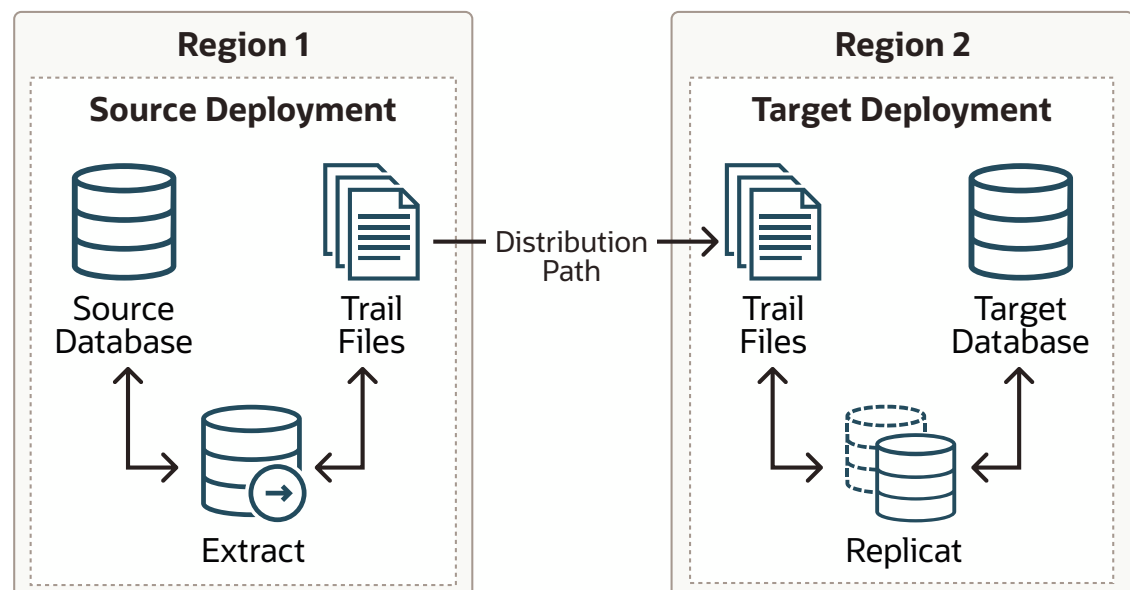
```

7. From the **Action Menu**, click **Start**.

Step 4.4.3 - Distribution Path Configuration

Distribution paths are only necessary when trail files need to be sent to an additional Oracle GoldenGate Hub in a different, or even the same, region as described in the following figure.

Figure 22-3 Oracle GoldenGate Distribution Path



When using Oracle GoldenGate Distribution paths with the NGINX Reverse Proxy, additional steps must be carried out to ensure the path client and server certificates are configured.

More instructions about creating distribution paths are available in [Add a Distribution Path](#). A step-by-step example is in the following video, "[Connect an on-premises Oracle GoldenGate to OCI GoldenGate using NGINX](#)," to correctly configure the certificates.

Here are the steps performed in this sub-step:

- Step 4.4.3.1 - Download the Target Server's Root Certificate, and then upload it to the source Oracle GoldenGate
- Step 4.4.3.2 - Create a user in the Target Deployment for the Source Oracle GoldenGate to use
- Step 4.4.3.3 - Create a Credential in the Source Oracle GoldenGate
- Step 4.4.3.4 - Create a Distribution Path on the Source Oracle GoldenGate to the Target Deployment

Step 4.4.3.1 - Download the Target Server's Root Certificate, and then upload it to the source Oracle GoldenGate

Download the target deployment server's root certificate and add the CA certificate to the source deployment Service Manager.

1. Log in to the **Administration Service** on the Target GoldenGate.
2. Follow "Step 2 - Download the target server's root certificate" in the video "[Connect an on-premises Oracle GoldenGate to OCI GoldenGate using NGINX](#)."

Step 4.4.3.2 - Create a user in the Target Deployment for the Source Oracle GoldenGate to use

Create a user in the target deployment for the distribution path to connect to:

1. Log in to the **Administration Service** on the Target GoldenGate.
2. Click on Administrator under Administration Service.
3. Click the plus (+) sign next to Users.
4. Enter the details as follows:
 - Username: ggnet
 - Role: Operator
 - Type: Password
5. Click **Submit**

Step 4.4.3.3 - Create a Credential in the Source Oracle GoldenGate

Create a credential in the source deployment connecting the target deployment with the user created in the previous step. For example, a domain of OP2C and an alias of WSSNET.

1. Log in to the **Administration Service** on the Source Oracle GoldenGate.
2. Click in **Configuration** under **Administration Service**.
3. Click the **plus (+)** sign next to Credentials on the Database home page.
4. Enter the details as follows:
 - Credential Domain: OP2C
 - Credential Alias: wssnet

- User ID: ggnet

5. Click **Submit**

Step 4.4.3.4 - Create a Distribution Path on the Source Oracle GoldenGate to the Target Deployment

A path is created to send trail files from the Distribution Server to the Receiver Server. You can create a path from the Distribution Service. To add a path for the source deployment:

1. Log in to the **Distribution Service** on the Source Oracle Goldengate.
2. Click the plus (+) sign next to Path on the Distribution Service home page. The Add Path page is displayed.
3. Enter the details as follows:

Option	Description
Path Name	Select a name for the path.
Source: <i>Trail Name</i>	Select the Extract name from the drop-down list, which populates the trail name automatically. If it doesn't, enter the trail name you provided while adding the Extract.
Generated Source URI	Specify localhost for the server's name; this allows the distribution path to be started on any of the Oracle RAC nodes.
Target Authentication Method	Use 'UserID Alias'
Target	Set the Target transfer protocol to wss (secure web socket). Set the Target Host to the target hostname/VIP that will be used for connecting to the target system along with the Port Number that NGINX was configured with (default is 443).
Domain	Set the Domain to the credential domain created above in Step 11.3.3, for example, OP2C.
Alias	The Alias is set to the credential alias wssnet, also created in Step 11.3.3.
Auto Restart Options	Set the distribution path to restart when the Distribution Server starts automatically. This is required, so that manual intervention is not required after a RAC node relocation of the Distribution Server. It is recommended to set the number of Retries to 10. Set the Delay , which is the time in minutes to pause between restart attempts, to 1.

4. Click **Create Path**.
5. From the Action Menu, click **Start**.

Step 4.4.4 - Set up a Heartbeat Table to Monitor Lag Times

Follow [Steps to add Heartbeat Table in OCI GoldenGate](#) to implement the best practices for creating a heartbeat process that can be used to determine where and when lag are developing between a source and target system.

This document walks you through the step-by-step process of creating the necessary tables and added table mapping statements needed to keep track of processing times between a source and target database. Once the information is added into the data flow, the information is

then stored in a target table that can be analyzed to determine when and where the lag is being introduced between the source and target systems.

Cloud Across Regions: Configuring Oracle GoldenGate Hub

Configure and deploy MAA Oracle GoldenGate Hub architecture on Oracle Cloud using the provided planning considerations, tasks, management, and troubleshooting information.

See the following topics:

- [Overview of MAA GoldenGate Hub](#)
- [Planning GGHUB Placement in the Platinum MAA Architecture](#)
- [Task 1: Configure the Source and Target Databases for Oracle GoldenGate](#)
- [Task 2: Prepare a Primary and Standby Base System for GGHUB](#)
- [Task 3: Configure Oracle GoldenGate for the Primary and Standby GGHUB](#)
- [Task 4: Configure the Oracle GoldenGate Environment](#)

Overview of MAA GoldenGate Hub

To achieve the highest levels of availability, resulting in zero or near-zero downtime for both unplanned outages and planned maintenance activities, you can use the combination of Oracle Real Application Clusters (Oracle RAC), Oracle Active Data Guard, and Oracle GoldenGate.

This architecture, typically referred as MAA Platinum, or Never Down Architecture, delivers near zero Recovery Time Objective (RTO--downtime incurred during outage) and potentially zero or near zero Recovery Point Objective (RPO--data loss potential).

Traditionally, Oracle GoldenGate is installed and run locally on the database server that the GoldenGate processes connect to. When used with Oracle Grid Infrastructure Standalone Agent (XAG), Oracle GoldenGate processes can be configured to seamlessly relocate or failover between Oracle RAC nodes and follow Oracle Active Data Guard switchover and failovers.

Using MAA Oracle GoldenGate Hub (MAA GGHUB) moves the GoldenGate software and processes off of the Exadata database servers, reducing complexity and system resource utilization. MAA GGHUB centralizes Oracle GoldenGate management and offloads the majority of the Oracle GoldenGate processing and associated CPU and storage resource utilization from Exadata system resources. Connectivity between the GoldenGate processes and the databases they operate against is managed with Oracle Net Services.

To achieve an MAA Platinum solution in the Oracle Cloud, you follow these high level steps:

1. Review [MAA Platinum and MAA Diamond: Oracle RAC and Exadata](#) to understand Platinum MAA benefits and use cases.
2. Deploy or migrate your database onto Exadata Cloud Service, Base Database Service, or Autonomous Database on Dedicated Infrastructure Service.
3. Add symmetric standby databases in the Oracle Cloud using Oracle Cloud Control Plan or Cloud automation.

4. Configure and deploy Oracle Data Guard Fast Start Failover using the Oracle MAA best practice recommendations in [Configure Fast Start Failover](#). For Exadata Cloud Service and Base Database Service, this is manual step.
5. Set up MAA GGHUB across different OCI regions, which is detailed in the topics that follow. For an MAA GGHUB within one OCI region, see [Cloud Within Region: Configuring Oracle GoldenGate Hub](#).
6. Configure Bidirectional Replication and Automatic Conflict Detection and Resolution. See [Oracle Cloud Infrastructure GoldenGate documentation](#) for information.
7. Decide on Application Failover Options such as Global Data Services (see [Oracle Global Data Services Best Practices](#)), or use your own customized application failover.

Planning GGHUB Placement in the Platinum MAA Architecture

Extreme availability that delivers zero downtime (RTO=0 or near zero) and zero or near zero data loss (RPO=0 or near zero) typically requires the following Platinum MAA architecture.

1. You have the source and target database in an Oracle GoldenGate architecture to allow your application to fail over immediately in the case of disaster (database, cluster, or site failure) or switch over in the case of a database or application upgrade. This architecture enables the potential RTO of zero or near zero for disaster scenarios and database and application upgrade maintenance.
2. Each source and target database is deployed in Exadata cloud systems so any local failures are tolerated or recovered almost instantly.
3. Each source and target database is configured with a standby database with Data Guard Fast-Start Failover so any failure of the database results in activating a new primary database in seconds to minutes. If SYNC transport is leveraged with Max Availability protection mode, zero data loss Data Guard failover is achieved.
4. Configured with GoldenGate replication using MAA GGHUB between the source and target databases.
5. Configured so that any standby becoming a primary database due to Data Guard switchover or failover will automatically resynchronize with its target GoldenGate database. If zero data loss Data Guard switchover or failover occurs, GoldenGate resynchronization ensures zero data loss across the distributed database environment.
6. Configured with GoldenGate Automatic Conflict Detection and Resolution, which is required after any Data Guard failover operation occurs.

Where to Place the MAA Primary GGHUB and Standby GGHUB

1. The GGHUB pair (Primary and Standby GGHUB) must reside in the same OCI regions as each primary and standby database. For example:
 - a. If the primary database is in AD1, Region A, and the standby database is in AD2, Region A, then the GGHUB pair will reside in Region A. For this configuration, see [Cloud Within Region: Configuring Oracle GoldenGate Hub](#).
 - b. If the primary database is in Region A and the standby database is in Region B, then the GGHUB pair will split between Region A and B. The primary, or active, GGHUB must be co-located in the same OCI region as the target primary database. For this configuration, continue reading the topics in this chapter.
2. Performance implications:

- a. Primary or active GGHUB must reside in the same data center as the target database to ensure round trip latency of 4ms or less. (Replicat performance)
 - b. Primary or active GGHUB should be < 90 ms from the source database without incurring GoldenGate performance degradation. (Extract performance)
3. GoldenGate distribution path:
 - a. A GoldenGate distribution path is required if the source and target GGHUBs are in different regions and latency between the OCI regions is > 90 ms.
 - b. In Oracle Cloud, when your Oracle GoldenGate source and target databases reside in the same region, or in different regions in the same country, you never need to set up a GoldenGate distribution path because the latency is always < 90 ms.

MAA GGHUBs Placed in Different OCI Regions

In this scenario, the primary database and its standby database are located in different OCI regions, so the primary (active) GGHUB is located in the same region as the primary database, and the standby GGHUB is located in the same region as the standby database.

The following architectural components comprise the GGHUBs, as shown in the image below:

1. The primary database and associated standby database are configured with Oracle Active Data Guard Fast Start Failover (FSFO). FSFO can be configured with any Data Guard protection mode, with ASYNC or SYNC redo transport, depending on your maximum data loss tolerance.
2. Primary GGHUB Active/Passive Cluster: In this configuration, there's a 2-node cluster with two Oracle GoldenGate software configurations. Because the primary GGHUB needs to be <= 4 ms from the target database, and the two regions (PHX and ASH) network latency > 5 ms, two GGHUB configurations are created for each GGHUB cluster. Essentially, a primary GGHUB configuration will always be in the same region as the target database.

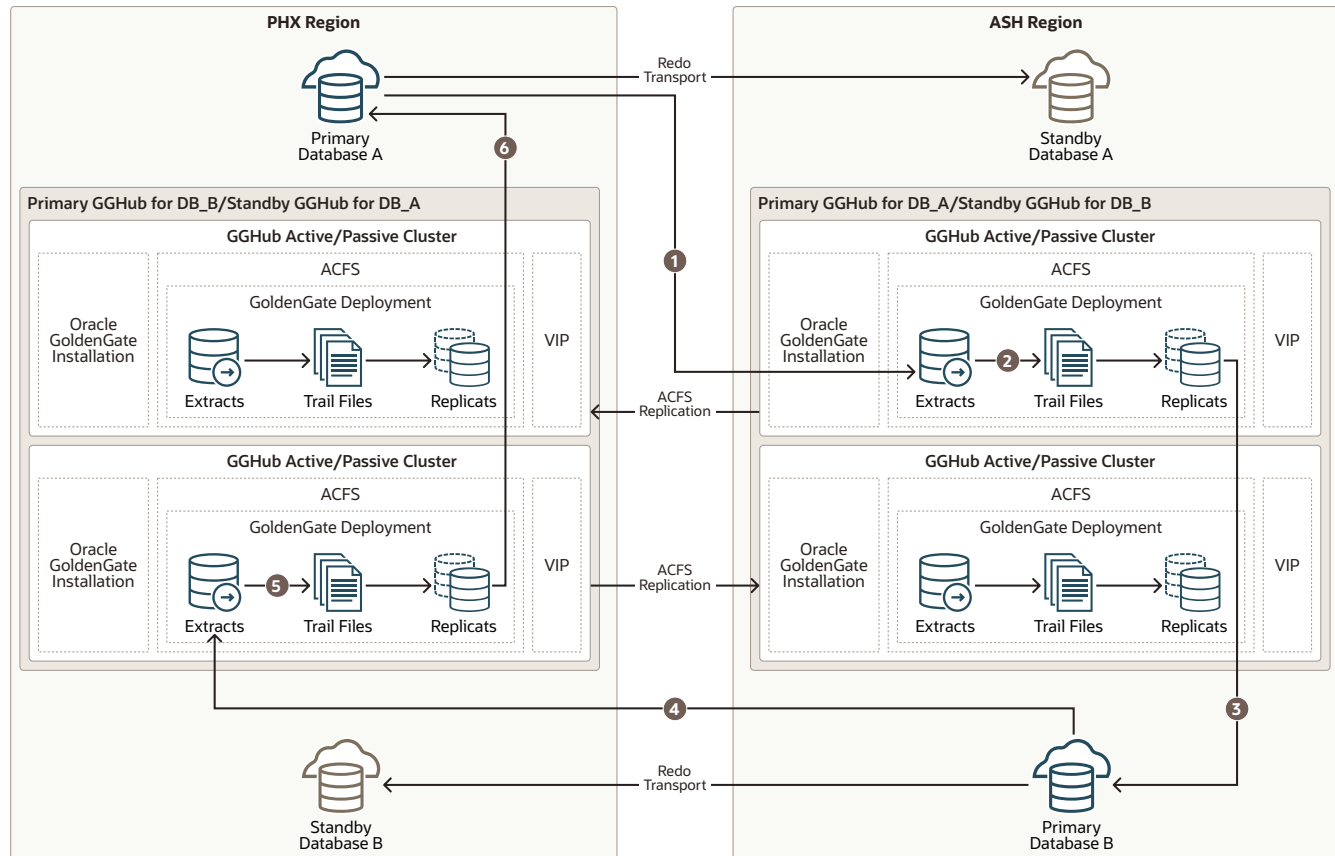
GGHUB is configured with the Oracle GoldenGate 21c software deployment that can support Oracle Database 11g and later releases. This GGHUB can support many primary databases and encapsulates the GoldenGate processes. Extract mines transactions from the source database, and Replicat applies those changes to the target database. GoldenGate trail and checkpoint files also reside in the ACFS file system.

An HA failover solution is built in to the GGHUB cluster, which includes automatic failover and restart of GoldenGate processes and activity after a node failure.

Each GGHUB configuration contains a GoldenGate service manager and deployment, ACFS file system with ACFS replication, and separate application VIP.

3. Standby GGHUB Active/Passive Cluster: A symmetric standby GGHUB is configured. ACFS replication is set up between the primary and standby GGHUBs to preserve all GoldenGate files.

Manual GGHUB failover, which includes ACFS failover, can be performed if you lose the entire primary GGHUB.

Figure 23-1 Primary and Standby GGHubs in Different OCI Regions

The figure above depicts replicating data from Primary Database A to Primary Database B, and Primary B back to Primary A with the following steps:

1. Primary Database A: Primary A's Logminer server sends redo changes to an ASH region GGHub Extract process, which is on the Primary GGHub for Database A.
2. Primary GGHub: The Extract process writes changes to trail files.
3. Primary GGHub to Primary Database B: An ASH region GoldenGate Replicat process applies those changes to the target database (Primary B).
4. Primary Database B: Primary B's Logminer server sends redo to a PHX region GGHub Extract process, which is on the Primary GGHub for Database B.
5. Primary GGHub: The Extract process writes changes to trail files.
6. Primary GGHub to Primary Database A: A PHX region GoldenGate Replicat process applies those changes to the target database (Primary A).

Table 23-1 Outage Scenarios, Repair, and Restoring Redundancy for GGHubs in Different OCI Regions

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Primary Database A (or Database B) failure	Impact: Near-zero application downtime. GoldenGate replication resumes when the new primary database starts. - One primary database is still available. All activity is routed to the existing available primary database to achieve zero application downtime. Refer to the Global Data Services Global Services Failover solution. For example, application services A-F are routed to Database A, and application services G-J are routed to Database B. If Database A fails, all services temporarily go to Database B. - The standby becomes the new primary automatically with Data Guard FSFO. GoldenGate replication resumes and the primary databases resynchronize. Data loss is bounded by the Data Guard protection level. If Maximum Availability or Maximum Protection is configured, zero data loss is achieved. All committed transactions are in one or both databases. Workload can be rebalanced when primary Database A and Database B are available and in sync. For example, when Database A is up and running and in sync, services A-F can go back to Database A. - Replicat performance will be degraded if the primary GGHUB is not in the same region as the target database. Schedule a GGHUB switchover with ACFS replication switchover to resume optimal Replicat performance to the target database. You may then experience two active GGHUB configurations on the same GGHUB cluster.	- The old primary database is reinstated as the new standby database to restore redundancy. - Optionally performing a Data Guard switchover, to switch back to the original configuration, ensures that at least one primary database resides in an independent AD. Schedule a GGHUB switchover with ACFS replication switchover to resume optimal Replicat performance to the target database.

Table 23-1 (Cont.) Outage Scenarios, Repair, and Restoring Redundancy for GGHubs in Different OCI Regions

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Primary or standby GGHUB single node failure	Impact: No application impact. GoldenGate replication resumes automatically after a couple of minutes. No action is required. An HA failover solution is built in to the GGHUB that includes automatic failover and restart of GoldenGate processes and activity. Replication activity is blocked until GoldenGate processes are active again. GoldenGate Replication blackout could last a couple of minutes.	Once the node restarts, active/passive configuration is re-established.
Primary GGHUB cluster crashes and is not recoverable	Impact: No application impact. GoldenGate replication resumes after the existing primary GGHUB restarts or manual GGHUB failover completes. 1. If the GGHUB cluster can be restarted, then that's the simplest solution. 2. If the primary GGHUB is not recoverable, then perform a manual GGHUB failover to the standby GGHUB, which includes ACFS failover. This typically takes several minutes. 3. Replication stops until the new primary GGHUB is started, so performing step 1 or step 2 should take little time. If there's any orchestration, this should be automated.	1. If the previous GGHUB eventually restarts, ACFS replication resumes in the other direction automatically. If the GGHUB cluster is lost or unrecoverable, you need to rebuild a new standby GGHUB. 2. Replicat performance is degraded if the primary GGHUB is not in the same region as the target database. Schedule a GGHUB switchover with ACFS replication switchover to resume optimal Replicat performance to the target database.
Standby GGHUB cluster crashes and is not recoverable	Impact: No application or replication impact. 1. If the GGHUB cluster can be restarted, then that's the simplest solution, and ACFS replication will resume. 2. If the standby GGHUB is not recoverable, you can rebuild a new standby GGHUB.	N/A

Table 23-1 (Cont.) Outage Scenarios, Repair, and Restoring Redundancy for GGHubs in Different OCI Regions

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Complete Regional failure	Impact: Near Zero Application Downtime. GoldenGate replication resumes when the new primary database starts. - One primary database is still available. All activity is routed to the existing available primary database to achieve zero application downtime. Refer to the Global Data Services Global Services Failover solution. For example, application services A-F routed to Database A and application services G-J routed to Database B. If Database A fails, all services will temporarily go to Database B. - If the primary GGHub is still functional, GoldenGate replication will continue. If the primary GGHub is lost due to regional failure, then a manual GGHub failover is required. GoldenGate replication resumes and the primary databases resynchronize. Data loss is bounded by the Data Guard protection level. If Maximum availability or protection is configured, zero data loss is achieved. All committed transactions are in one or both databases. Workload can be rebalanced when Primary Database A and Database B are available and in sync. When Database A is up and running and in sync, services A-F can go back to Database A.	- When the OCI region returns, re-establish configuration such as reinstate standby. If the previous GGHub eventually restarts, ACFS replication resumes in the other direction automatically. - When possible, perform a Data Guard switchover (failback) to get back to the original state where one primary database exists in each region. - Replicat performance is degraded if the primary GGHub is not in the same region as the target database. Schedule a GGHub switchover with ACFS replication switchover to resume optimal Replicat performance to the target database.

Task 1: Configure the Source and Target Databases for Oracle GoldenGate

The source and target Oracle GoldenGate databases should be configured using the recommendations that follow.

Step 1.1 - Configure the Databases

For Oracle Autonomous Database (ADB-D), Oracle Database 19c (19.20) or a later release is required to support parallel integrated Replicat and conflict resolution. For Oracle Exadata Database Service (ExaDB) or BaseDB, you can use any supported Oracle Database release available in the Oracle cloud.

The database configuration steps that follow are applicable for each Database Cloud Service that supports Platinum MAA solution.

The source and target Oracle GoldenGate databases should be configured using the following recommendations:

For Oracle Autonomous Database (ADB-D), you only need to add supplemental logging:

```
PDB: ALTER PLUGGABLE DATABASE ADD SUPPLEMENTAL LOG DATA;
```

For Oracle Exadata Database Service (ExaDB) or BaseDB do the following steps:

1. Enable Archivelog Mode.

```
SQL> ARCHIVE LOG LIST
Database log mode                Archive Mode
Automatic archival               Enabled
Archive destination              USE_DB_RECOVERY_FILE_DEST
Oldest online log sequence       110
Next log sequence to archive     113
Current log sequence             113
```

2. Enable Force Logging.

```
ALTER DATABASE FORCE LOGGING;
```

3. Enable GoldenGate replication.

```
ALTER SYSTEM SET ENABLE_GOLDENGATE_REPLICATION=TRUE SCOPE=BOTH SID='*';
```

4. Add supplemental logging.

```
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA;
```

5. Configure STREAMS_POOL_SIZE larger to accommodate GoldenGate.

Use this formula to calculate the appropriate STREAMS_POOL_SIZE value:

$$\text{STREAMS_POOL_SIZE} = (((\# \text{Extracts} + \# \text{Integrated Replicats}) * 1\text{GB}) * 1.25)$$

For example, in a database with 2 Extracts and 2 integrated Replicats:

$$\text{STREAMS_POOL_SIZE} = 4\text{GB} * 1.25 = 5\text{GB}$$

And the parameter is set:

```
ALTER SYSTEM SET STREAMS_POOL_SIZE=5G SCOPE=BOTH SID='*';
```

For the steps to prepare the database for Oracle GoldenGate, see [Preparing the Database for Oracle GoldenGate](#).

Step 1.2 - Create a GoldenGate Database Administrator User

For ADB-D

For ADB-D deployments, you only need to issue

```
ALTER USER ggadmin IDENTIFIED BY <password> ACCOUNT UNLOCK;
```

For Oracle Exadata Database Service (ExaDB) or BaseDB

The following steps are only applicable for Oracle Exadata Database Service (ExaDB) or BaseDB.

The source and target databases need a GoldenGate administrator user created, with appropriate privileges assigned as follows:

- For the multitenant container database (CDB):
 - Source database - GoldenGate Extract must be configured to connect to a user in the root container database, using a c##
 - Target database - a separate GoldenGate administrator user is needed for each pluggable database (PDB).
 - For details about creating a GoldenGate administrator in an Oracle Multitenant Database, see [Configuring Oracle GoldenGate in a Multitenant Container Database](#).
- For non-CDB databases, see [Establishing Oracle GoldenGate Credentials](#)
- 1. As the oracle OS user on the source database system, execute the following SQL instructions to create the database user for Oracle GoldenGate and assign the required privileges:

```
[opc@exadb1_node1 ~]$ sudo su - oracle
[oracle@exadb1_node1 ~]$ source dbName.env
[oracle@exadb1_node1 ~]$ sqlplus / as sysdba

# Source CDB
SQL>
alter session set container=cdb$root;
create user c##ggadmin identified by "ggadmin_password" container=all
default tablespace USERS temporary tablespace temp;
alter user c##ggadmin quota unlimited on users;
grant set container to c##ggadmin container=all;
grant alter system to c##ggadmin container=all;
grant create session to c##ggadmin container=all;
grant alter any table to c##ggadmin container=all;
grant resource to c##ggadmin container=all;
exec
dbms_goldengate_auth.grant_admin_privilege('c##ggadmin',container=>'all');

# Source PDB
SQL>
alter session set container=pdbName;
create user ggadmin identified by "ggadmin_password" container=current;
```

```
grant create session to ggadmin container=current;
grant alter any table to ggadmin container=current;
grant resource to ggadmin container=current;
exec dbms_goldengate_auth.grant_admin_privilege('ggadmin');
```

2. As the oracle OS user on the target system, execute the following SQL instructions to create the database user for Oracle GoldenGate and assign the required privileges:

```
[opc@exadb2_node1 ~]$ sudo su - oracle
[oracle@exadb2_node1 ~]$ source dbName.env
[oracle@exadb2_node1 ~]$ sqlplus / as sysdba

# Target PDB
SQL>
alter session set container=pdbName;
create user ggadmin identified by "ggadmin_password" container=current;
grant alter system to ggadmin container=current;
grant create session to ggadmin container=current;
grant alter any table to ggadmin container=current;
grant resource to ggadmin container=current;
grant dv_goldengate_admin, dv_goldengate_redo_access to ggadmin
container=current;
exec dbms_goldengate_auth.grant_admin_privilege('ggadmin');
```

Step 1.3 - Create the Database Services

Note

This step is not required for ADB-D deployments.

If the source and target databases are running the recommended configuration on an Oracle RAC cluster with Oracle Data Guard, a role-based service must be created that allows the Extract or Replicat processes to connect to the correct Data Guard primary database instance.

When using a source multitenant database, a separate service is required for the root container database (CDB) and the pluggable database (PDB) that contains the schema being replicated. For a target multitenant database, a single service is required for the PDB.

1. As the oracle OS user on the primary database system, use dbaascli to find the CDB and PDB name, as shown here:

```
[opc@exadb1_node1 ~]$ sudo su - oracle
[oracle@exadb1_node1 ~]$ source dbName.env
[oracle@exadb1_node1 ~]$ dbaascli database getDetails
--dbname dbName |egrep 'dbName|pdbName'

"dbName" : "dbName",
"pdbName" : "pdbName",
```


2. As the oracle OS user on the primary and standby database systems, create and start the CDB database service using the following command:

```
[opc@exadb1_node1 ~]$ sudo su - oracle
[oracle@exadb1_node1 ~]$ source dbName.env
[oracle@exadb1_node1 ~]$ srvctl add service -db $ORACLE_UNQNAME
    -service dbName.goldengate.com -preferred ORACLE_SID1
    -available ORACLE_SID2 -role PRIMARY
```

3. As the oracle OS user on the primary and standby database systems, create and start the PDB database service using the following command:

```
[oracle@exadb1_node1 ~]$ srvctl add service -db $ORACLE_UNQNAME
    -service dbName.pdbName.goldengate.com -preferred ORACLE_SID1
    -available ORACLE_SID2 -pdb dbName -role PRIMARY
```

4. As the oracle OS user on the primary and standby database systems, start and verify that the services are running, as shown here:

```
[oracle@exadb1_node1 ~]$ srvctl start service -db $ORACLE_UNQNAME -role
[oracle@exadb1_node1 ~]$ srvctl status service -d $ORACLE_UNQNAME |grep
goldengate
```

```
Service dbName.goldengate.com is running on instance(s) SID1
Service dbName.pdbName.goldengate.com is running on instance(s) SID1
```

Note

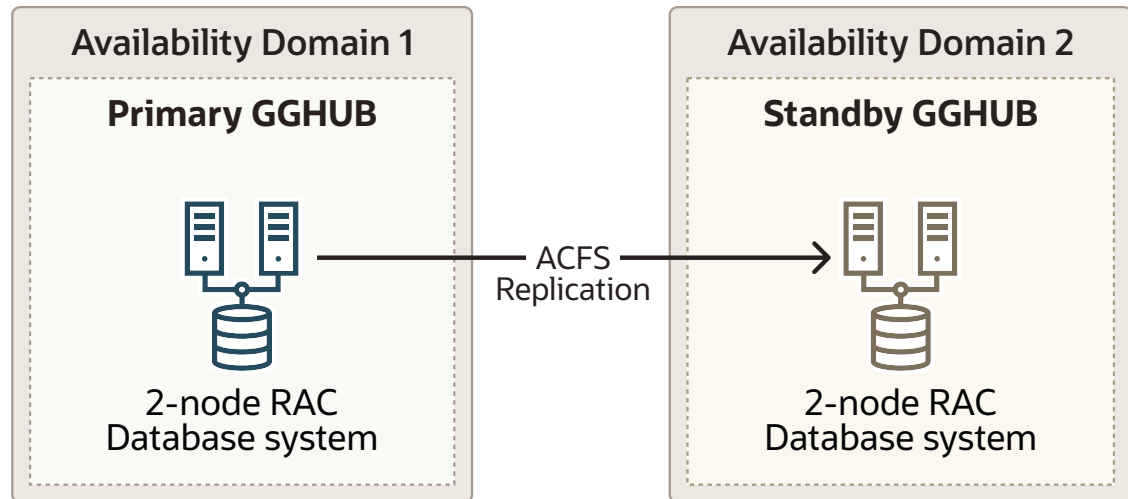
Repeat Step 1.3 in the source and target database system.

Task 2: Prepare a Primary and Standby Base System for GGHUB

Step 2.1 - Deploy an Oracle RAC 2-Node Cluster System

Deploy a minimum of two GGHUBs (primary and standby). Each GGHUB must be deployed as a 2-node Oracle RAC database system as described in [Oracle Base Database Service](#).

Figure 23-2 Oracle GoldenGate Hub Hardware Architecture



Step 2.2 - Remove the Standard Database and Rearrange the Disk Group Layout

1. As the oracle OS user on the first GGHUB node, remove the standard database:

```
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghubN-node1 ~]$ dbca -deleteDatabase -silent -
sourceDB $ORACLE_UNQNAME
Enter SYS user password: #####

[WARNING] [DBT-19202] The Database Configuration Assistant will delete the
Oracle instances and datafiles for your database. All information in the
database will be destroyed.
Prepare for db operation
32% complete
Connecting to database
39% complete
...
100% complete
Database deletion completed.
Look at the log file "/u01/app/oracle/cfgtoollogs/dbca/DB0502_fra2pr/
DB0502_fra2pr.log" for further details.
```

2. As the grid OS user on the second GGHUB node, dismount the RECO diskgroup:

```
[opc@gghub_prim2 ~]$ sudo su - grid
[grid@gghub_prim2 ~]$ sqlplus / as sysasm

SQL> alter diskgroup RECO dismount;
```

3. As the grid OS user on the first **gghub** node, drop the RECO diskgroup and assign the disks to the DATA diskgroup:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ sqlplus / as sysasm

SQL>
drop diskgroup RECO INCLUDING CONTENTS;
alter diskgroup DATA add disk '/dev/RECODISK1';
alter diskgroup DATA add disk '/dev/RECODISK2';
alter diskgroup DATA add disk '/dev/RECODISK3';
alter diskgroup DATA add disk '/dev/RECODISK4';
```

4. As the root OS user on all GGHUB nodes, reboot the node:

```
[opc@gghub_prim1 ~]$ sudo reboot
```

Note

Repeat this step in the primary and standby GGHUBs.

Step 2.3 - Download the Required Software

1. As the opc OS user on all GGHUB nodes, create the staging and scripts directories:

```
[opc@gghub_prim1 ~]$
sudo mkdir -p /u01/oracle/stage
sudo mkdir /u01/oracle/scripts
sudo chown -R oracle:oinstall /u01/oracle
sudo chmod -R g+w /u01/oracle
sudo chmod -R o+w /u01/oracle/stage
```

2. As the opc OS user on all GGHUB nodes, download the following software in the directory /u01/oracle/stage:
 - Download the latest Oracle GoldenGate 21c (or later release) Microservices software from [Oracle GoldenGate Downloads](#).
 - Download subsequent patches to the base release from the **Patches and Updates** tab of [My Oracle Support](#).
 - See [Installing Patches for Oracle GoldenGate Microservices Architecture](#) for more information.
 - Minimum required version is Patch 35214851: Oracle GoldenGate 21.9.0.0.2 Microservices for Oracle
 - Download the latest OPatch release, Patch 6880880, for Oracle Database 21c (21.0.0.0.0) from My Oracle Support Document [2542082.1](#).
 - Download the Oracle Grid Infrastructure Standalone Agents for Oracle Clusterware 19c, release 10.2 or later, from [Oracle Grid Infrastructure Standalone Agents for Oracle Clusterware](#).
 - Download the python script (secureServices.py) from My Oracle Support [Document 2826001.1](#)

- Download the Oracle GGHUB Scripts from My Oracle Support [Document 2951572.1](#)
- 3. As the grid OS user on all GGHUB nodes, unzip the GGHUB scripts file downloaded from My Oracle Support [Document 2951572.1](#) into the directory /u01/oracle/scripts.

Place the script in the same location on all primary and standby GGHUB nodes

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ unzip -q /u01/oracle/stage/
gghub_scripts_YYYYMMDD.zip -d /u01/oracle/scripts/
```

Step 2.4 - Configure Oracle Linux To Use the Oracle Public YUM Repository

The Oracle Linux yum server hosts software for Oracle Linux and compatible distributions. These instructions help you get started configuring your Linux system for Oracle Linux yum server and installing software through yum.

- As the root OS user in all GGHUB systems, create the file /etc/yum.repos.d/oracle-public-yum-ol7.repo with the following contents:

```
[opc@gghub_prim1 ~]$ sudo su -
[root@gghub_prim1 ~]#
cat > /etc/yum.repos.d/oracle-public-yum-ol7.repo <<EOF
[ol7_latest]
name=Oracle Linux $releasever Latest ($basearch)
baseurl=http://yum$ociregion.oracle.com/repo/OracleLinux/OL7/latest/
\ $basearch/
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-oracle
gpgcheck=1
enabled=1
EOF
```

Task 3: Configure Oracle GoldenGate for the Primary and Standby GGHUB

Step 3.1 - Install Oracle GoldenGate Software

Install Oracle GoldenGate software locally on all nodes of the primary and standby GGHUB configuration that will be part of the GoldenGate configuration. Make sure the installation directory is identical on all nodes.

Perform the following sub-steps to complete this step:

- Step 3.1.1 Unzip the Software and Create the Response File for the Installation
- Step 3.1.2 Install Oracle GoldenGate Software
- Step 3.1.3 Installing Patches for Oracle GoldenGate Microservices Architecture

Step 3.1.1 Unzip the Software and Create the Response File for the Installation

As the oracle OS user on all GGHUB nodes, unzip the Oracle GoldenGate software:

```
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghub_prim1 ~]$ unzip -q
/u01/oracle/stage/p36175132_21130000GGRU_Linux-x86-64.zip -d
/u01/oracle/stage
```

The software includes an example response file for Oracle Database 21c and earlier supported versions. Copy the response file to a shared file system, so the same file can be used to install Oracle GoldenGate on all database nodes, and edit the following parameters:

- `INSTALL_OPTION=ora21c`
- `SOFTWARE_LOCATION=/u01/app/oracle/goldengate/gg21c` (recommended location)

As the oracle OS user on all GGHUB nodes, copy and edit the response file for the installation:

```
[oracle@gghub_prim1 ~]$ cp
/u01/oracle/stage/fbo_ggs_Linux_x64_Oracle_services_shiphome/Disk1/response/
oggcore.rsp
/u01/oracle/stage
[oracle@gghub_prim1 ~]$ vi /u01/oracle/stage/oggcore.rsp
```

```
# Before
INSTALL_OPTION=
SOFTWARE_LOCATION=
```

```
# After
INSTALL_OPTION=ora21c
SOFTWARE_LOCATION=/u01/app/oracle/goldengate/gg21c
```

Step 3.1.2 Install Oracle GoldenGate Software

As the oracle OS user on all GGHUB nodes, run `runInstaller` to install Oracle GoldenGate:

```
[oracle@gghub_prim1 ~]$ cd
/u01/oracle/stage/fbo_ggs_Linux_x64_Oracle_services_shiphome/Disk1/
[oracle@gghub_prim1 ~]$ ./runInstaller -silent -nowait
-responseFile /u01/oracle/stage/oggcore.rsp
```

Starting Oracle Universal Installer...

```
Checking Temp space: must be greater than 120 MB.   Actual 32755 MB   Passed
Checking swap space: must be greater than 150 MB.   Actual 16383 MB   Passed
Preparing to launch Oracle Universal Installer from
/tmp/OraInstall2022-07-08_02-54-51PM.
Please wait ...
You can find the log of this install session at:
/u01/app/oraInventory/logs/installActions2022-07-08_02-54-51PM.log
Successfully Setup Software.
The installation of Oracle GoldenGate Services was successful.
Please check
'/u01/app/oraInventory/logs/silentInstall2022-07-08_02-54-51PM.log'
for more details.
```

```
[oracle@gghub_prim1 ~]$ cat
```

```
/u01/app/oraInventory/logs/silentInstall2022-07-08_02-54-51PM.log
```

The installation of Oracle GoldenGate Services was successful.

Step 3.1.3 Installing Patches for Oracle GoldenGate Microservices Architecture

As the oracle OS user on all GGHUB nodes, install the latest OPatch:

```
[oracle@ggghub_prim1 ~]$ unzip -oq -d
/u01/app/oracle/goldengate/gg21c
/u01/oracle/stage/p6880880_210000_Linux-x86-64.zip
[oracle@ggghub_prim1 ~]$ cat >> ~/.bashrc <<EOF
export ORACLE_HOME=/u01/app/oracle/goldengate/gg21c
export PATH=$ORACLE_HOME/OPatch:$PATH
EOF
[oracle@ggghub_prim1 ~]$ . ~/.bashrc
[oracle@ggghub_prim1 ~]$ opatch lsinventory |grep
'Oracle GoldenGate Services'
```

Oracle GoldenGate Services

21.1.0.0.0

```
[oracle@ggghub_prim1 Disk1]$ opatch version
OPatch Version: 12.2.0.1.37
```

OPatch succeeded.

As the oracle OS user on all GGHUB nodes, run OPatch prereq to validate any conflict before applying the patch:

```
[oracle@ggghub_prim1 ~]$ unzip -oq -d /u01/oracle/stage/
/u01/oracle/stage/p35214851_2190000GGRU_Linux-x86-64.zip
[oracle@ggghub_prim1 ~]$ cd /u01/oracle/stage/35214851/
[oracle@ggghub_prim1 35214851]$ opatch prereq
CheckConflictAgainstOHWithDetail -ph ./
```

Oracle Interim Patch Installer version 12.2.0.1.26
Copyright (c) 2023, Oracle Corporation. All rights reserved.

PREREQ session

```
Oracle Home      : /u01/app/oracle/goldengate/gg21c
Central Inventory : /u01/app/oraInventory
   from           : /u01/app/oracle/goldengate/gg21c/oraInst.loc
OPatch version    : 12.2.0.1.26
OUI version       : 12.2.0.9.0
Log file location :
/u01/app/oracle/goldengate/gg21c/cfgtoollogs/patch/
patch2023-04-21_13-44-16PM_1.log
```

Invoking prereq "checkconflictagainstohwithdetail"

Prereq "checkConflictAgainstOHWithDetail" passed.

OPatch succeeded.

As the oracle OS user on all GGHUB nodes, patch Oracle GoldenGate Microservices Architecture using OPatch:

```
[oracle@gghub_prim1 35214851]$ opatch apply
```

```
Oracle Interim Patch Installer version 12.2.0.1.37
Copyright (c) 2023, Oracle Corporation. All rights reserved.
```

```
Oracle Home      : /u01/app/oracle/goldengate/gg21c
Central Inventory : /u01/app/oraInventory
   from           : /u01/app/oracle/goldengate/gg21c/oraInst.loc
OPatch version    : 12.2.0.1.37
OUI version       : 12.2.0.9.0
Log file location :
   /u01/app/oracle/goldengate/gg21c/cfgtoollogs/patch/
   patch2023-04-21_19-40-41PM_1.log
Verifying environment and performing prerequisite checks...
OPatch continues with these patches: 35214851
```

```
Do you want to proceed? [y|n]
```

```
y
```

```
User Responded with: Y
```

```
All checks passed.
```

```
Please shutdown Oracle instances running out of this ORACLE_HOME on
the local system.
```

```
(Oracle Home = '/u01/app/oracle/goldengate/gg21c')
```

```
Is the local system ready for patching? [y|n]
```

```
y
```

```
User Responded with: Y
```

```
Backing up files...
```

```
Applying interim patch '35214851' to OH '/u01/app/oracle/goldengate/gg21c'
```

```
Patching component oracle.oggcore.services.ora21c, 21.1.0.0.0...
```

```
Patch 35214851 successfully applied.
```

```
Log file location:
```

```
   /u01/app/oracle/goldengate/gg21c/cfgtoollogs/patch/
   patch2023-04-21_19-40-41PM_1.log
```

OPatch succeeded.

```
[oracle@gghub_prim1 35214851]$ opatch lspatches
35214851;
```

OPatch succeeded.

Note

Repeat all of the steps in step 3.1 for the primary and standby GGHUB systems.

Step 3.2 - Configure the Cloud Network

You must configure virtual cloud network (VCN) components such as private DNS zones, VIP, bastion, security lists, and firewalls for Oracle GoldenGate to function correctly.

To learn more about VCNs and security lists, including instructions for creating them, see the [Oracle Cloud Infrastructure Networking documentation](#).

Perform the following sub-steps to complete this step:

- Step 3.2.1 - Create an Application Virtual IP Address (VIP) for GGHUB
- Step 3.2.2 - Add an Ingress Rule for port 443
- Step 3.2.3 - Open Port 443 in the GGHUB Firewall
- Step 3.2.4 - Configure Network Connectivity Between the Primary and Standby GGHUB Systems
- Step 3.2.5 - Configure Private DNS Zones Views and Resolvers

Step 3.2.1 - Create an Application Virtual IP Address (VIP) for GGHUB

A dedicated application VIP is required to allow access to the GoldenGate Microservices using the same host name, regardless of which node of the cluster is hosting the services. The VIP is assigned to the GGHUB system and is automatically migrated to another node in the event of a node failure. Two VIPs are required, one for the primary and another one for the standby GGHUBS.

As the grid OS user on all GGHUB nodes, run the following commands to get the vnicId of the Private Endpoint in the same subnet at resource ora.net1.network:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ crsctl status resource -p -attr NAME,USR_ORA_SUBNET
-w "TYPE = ora.network.type" |sort | uniq
```

```
NAME=ora.net1.network
USR_ORA_SUBNET=10.60.2.0
```

```
[grid@gghub_prim1 ~]$ curl 169.254.169.254/opc/v1/vnics
```

```
[
  {
    "macAddr": "02:00:17:04:70:AF",
    "privateIp": "10.60.2.120",
    "subnetCidrBlock": "10.60.2.0/24",
    "virtualRouterIp": "10.60.2.1",
    "vlanTag": 3085,
    "vnicId": "ocid1.vnic.oc1.eu-frankfurt-1.ocid_value"
  },
  {
    "macAddr": "02:00:17:08:69:6E",
    "privateIp": "192.168.16.18",
    "subnetCidrBlock": "192.168.16.16/28",
    "virtualRouterIp": "192.168.16.17",
    "vlanTag": 879,
    "vnicId": "ocid1.vnic.oc1.eu-frankfurt-1.ocid_value"
  }
]
```



```
[grid@ggghub_prim2 ~]$ curl 169.254.169.254/opc/v1/vnics
[
  {
    "macAddr": "00:00:17:00:C9:19",
    "privateIp": "10.60.2.148",
    "subnetCidrBlock": "10.60.2.0/24",
    "virtualRouterIp": "10.60.2.1",
    "vlanTag": 572,
    "vnicId": "ocid1.vnic.oc1.eu-frankfurt-1.ocid_value"
  },
  {
    "macAddr": "02:00:17:00:84:B5",
    "privateIp": "192.168.16.19",
    "subnetCidrBlock": "192.168.16.16/28",
    "virtualRouterIp": "192.168.16.17",
    "vlanTag": 3352,
    "vnicId": "ocid1.vnic.oc1.eu-frankfurt-1.ocid_value"
  }
]
```

Note

For the next step, you will need to use the Cloud Shell to assign the private IP to the GGHUB nodes. See [Using Cloud Shell](#) for more information.

As your user on the cloud shell, run the following commands to assign the private IP to the GGHUB nodes:

```
username@cloudshell:~ (eu-frankfurt-1)$ export node1_vnic=
'ocid1.vnic.oc1.eu-frankfurt-1.abtheljrl5udtgryrscypy5btm1fncawqkjl3kcpj64e2lb5xbmbrehkq'
username@cloudshell:~ (eu-frankfurt-1)$ export node2_vnic=
'ocid1.vnic.oc1.eu-frankfurt-1.abtheljre6rf3xoxtgl2gam3lav4vcyftz5fppm2ciin4wzjxucal2j7b2bq'
username@cloudshell:~ (eu-frankfurt-1)$ export ip_address='10.60.2.65'
username@cloudshell:~ (eu-frankfurt-1)$ oci network vnic assign-private-ip
--unassign-if-already-assigned --vnic-id $node1_vnic --ip-address $ip_address
username@cloudshell:~ (eu-frankfurt-1)$ oci network vnic assign-private-ip
--unassign-if-already-assigned --vnic-id $node2_vnic --ip-address $ip_address
```

Example of the output:

```
{
  "data": {
    "availability-domain": null,
    "compartment-id": "ocid1.compartment.oc1..ocid_value",
    "defined-tags": {},
    "display-name": "privateip20230292502117",
    "freeform-tags": {},
    "hostname-label": null,
    "id": "ocid1.privateip.oc1.eu-frankfurt-1.ocid_value",
    "ip-address": "10.60.2.65",
    "is-primary": false,
    "subnet-id": "ocid1.subnet.oc1.eu-frankfurt-1.ocid_value",
    "time-created": "2023-07-27T10:21:17.851000+00:00",
  }
}
```

```
"vlan-id": null,  
"vnic-id": "ocid1.vnic.oc1.eu-frankfurt-1.ocid_value"  
},  
"etag": "da972988"  
}
```

As the root OS user on the first GGHUB node, run the following command to create the application VIP managed by Oracle Clusterware:

```
[opc@gghub_prim1 ~]$ sudo su -  
[root@gghub_prim1 ~]# sh /u01/oracle/scripts/add_appvip.sh
```

```
Application VIP Name: gghub_prim_vip  
Application VIP Address: 10.60.2.65  
Using configuration parameter file:  
/u01/app/19.0.0.0/grid/crs/install/crsconfig_params  
The log of current session can be found at:  
/u01/app/grid/crsdata/gghublb1/scripts/appvipcfg.log
```

 Note

Repeat all the steps in step 3.2.1 for the primary and standby GGHUB systems.

Step 3.2.2 - Add the Ingress Security List Rules

Using the Cloud Console, add two ingress security list rules in the Virtual Cloud Network (VCN) assigned to the GGHUB.

One ingress rule is for TCP traffic on destination port 443 from authorized source IP addresses and any source port to connect to the Oracle GoldenGate service using NGINX as a reverse proxy, and the other is for allowing ICMP TYPE 8 (ECHO) between the primary and standby GGHUBs required to enable ACFS replication. For more information, see [Working with Security Lists](#) and My Oracle Support [Document 2584309.1](#).

After you update the security list, it will have an entry with values similar to the following ones:

1. NGINX - TCP 443
 - Source Type: CIDR
 - Source CIDR: 0.0.0.0/0
 - IP Protocol: TCP
 - Source Port Range: All
 - Destination Port Range: 443
 - Allows: TCP traffic for ports: 443 HTTPS
 - Description: Oracle GoldenGate 443
2. ACFS - ICMP TYPE 8 (ECHO)
 - Source Type: CIDR
 - Source CIDR: 0.0.0.0/0
 - IP Protocol: ICMP
 - Allows: ICMP traffic for: 8 Echo

- Description: Required for ACFS replication

Step 3.2.3 - Open Port 443 in the GGHub Firewall

As the opc OS user on all GGHub nodes of the primary and standby system, add the required rules to IPTables:

```
[opc@gghub_prim1 ~]$ sudo vi /etc/sysconfig/iptables

-A INPUT -p tcp -m state --state NEW -m tcp --dport 443 -j ACCEPT
-m comment --comment "Required for access to GoldenGate, Do not remove
or modify. "
-A INPUT -p tcp -m state --state NEW -m tcp --match multiport
--dports 9100:9105 -j ACCEPT -m comment --comment "Required for access
to GoldenGate, Do not remove or modify. "

[opc@gghub_prim1 ~]$ sudo systemctl restart iptables
```

Note

See [Implementing Oracle Linux Security](#) for more information.

Step 3.2.4 - Configure Network Connectivity Between the Primary and Standby GGHub Systems

Oracle ACFS snapshot-based replication uses ssh as the transport between the primary and standby clusters. To support ACFS replication, ssh must be usable in either direction between the clusters — from the primary cluster to the standby cluster and from the standby to the primary. See *Configuring ssh for Use With Oracle ACFS Replication* in *Oracle Automatic Storage Management Administrator's Guide*.

To learn more about whether subnets are public or private, including instructions for creating the connection, see section [Connectivity Choices](#) in the Oracle Cloud Infrastructure Networking documentation.

Step 3.2.5 - Configure Private DNS Zones Views and Resolvers

You must create a private DNS zone view and records for each application VIP. This is required for the primary GGHub to reach the standby GGHub deployment VIP host name.

Follow the steps in [Configure private DNS zones views and resolvers](#) to create your private DNS zone and a record entry for each dedicated GGHub application virtual IP address (VIP) created in Step 3.2.1.

As the opc OS user on any GGHub node, validate that all application VIPs can be resolved:

```
[opc@gghub_prim1 ~]$ nslookup
gghub_prim_vip.frankfurt.goldengate.com |tail -2

Address: 10.60.2.120

[opc@gghub_prim1 ~]$ nslookup
gghub_stby_vip.frankfurt.goldengate.com |tail -2

Address: 10.60.0.185
```

Step 3.3 - Configure ACFS File System Replication Between GGHUBs in the Same Region

Oracle GoldenGate Microservices Architecture is designed with a simplified installation and deployment directory structure. The installation directory: should be placed on local storage on each database node to minimize downtime during software patching. The deployment directory: which is created during deployment creation using the Oracle GoldenGate Configuration Assistant (oggca.sh), must be placed on a shared file system. The deployment directory contains configuration, security, log, parameter, trail, and checkpoint files. Placing the deployment in Oracle Automatic Storage Management Cluster File system (ACFS) provides the best recoverability and failover capabilities in the event of a system failure. Ensuring the availability of the checkpoint files cluster-wide is essential so that the GoldenGate processes can continue running from their last known position after a failure occurs.

It is recommended that you allocate enough trail file disk space for a minimum of 12 hours of trail files. Doing this will give sufficient space for trail file generation should a problem occur with the target environment that prevents it from receiving new trail files. The amount of space needed for 12 hours can only be determined by testing trail file generation rates with real production data. If you want to build contingency for a long planned maintenance event of one of the GoldenGate Primary Database or systems, you can allocate sufficient ACFS space for 2 days. Monitoring space utilization is always recommended regardless of how much space is allocated.

Note

If the GoldenGate hub will support multiple service manager deployments using separate ACFS file systems, the following steps should be repeated for each file ACFS file system.

Perform the following sub-steps to complete this step:

- Step 3.3.1 - Create the ASM File system
- Step 3.3.2 - Create the Cluster Ready Services (CRS) Resource
- Step 3.3.3 - Verify the Currently Configured ACFS File System
- Step 3.3.4 - Start and Check the Status of the ACFS Resource
- Step 3.3.5 – Create CRS Dependencies Between ACFS and an Application VIP
- Step 3.3.6 – Create the SSH Daemon CRS Resource
- Step 3.3.7 – Enable ACFS Replication
- Step 3.3.8 – Create the ACFS Replication CRS Action Scripts

Step 3.3.1 - Create the ASM File system

As the grid OS user on the first GGHUB node, use `asmcmd` to create the ACFS volume:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ asmcmd volcreate -G DATA -s 120G ACFS_GG1
```

Note

Modify the file system size according to the determined size requirements.

As the grid OS user on the first GGHUB node, use `asmcmd` to confirm the “Volume Device”:

```
[grid@gghub_prim1 ~]$ asmcmd volinfo -G DATA ACFS_GG1
```

```
Diskgroup Name: DATA
Volume Name: ACFS_GG1
Volume Device: /dev/asm/acfs_gg1-256
State: ENABLED
Size (MB): 1228800
Resize Unit (MB): 64
Redundancy: UNPROT
Stripe Columns: 8
Stripe Width (K): 1024
Usage:
Mountpath:
```

As the grid OS user on the first GGHUB node, format the partition with the following `mkfs` command:

```
[grid@gghub_prim1 ~]$ /sbin/mkfs -t acfs /dev/asm/acfs_gg1-256
```

```
mkfs.acfs: version           = 19.0.0.0.0
mkfs.acfs: on-disk version    = 46.0
mkfs.acfs: volume            = /dev/asm/acfs_gg1-256
mkfs.acfs: volume size       = 128849018880 ( 120.00 GB )
mkfs.acfs: Format complete.
```

Step 3.3.2 - Create the Cluster Ready Services (CRS) Resource

As the `opc` OS user on all GGHUB nodes, create the ACFS mount point:

```
[opc@gghub_prim1 ~]$ sudo mkdir -p /mnt/acfs_gg1
[opc@gghub_prim1 ~]$ sudo chown oracle:oinstall /mnt/acfs_gg1
```

Create the file system resource as the root user. Due to the implementation of distributed file locking on ACFS, unlike DBFS, it is acceptable to mount ACFS on more than one GGHUB node at any one time.

As the root OS user on the first GGHUB node, create the CRS resource for the new ACFS file system:

```
[opc@gghub_prim1 ~]$ sudo su -
[root@gghub_prim1 ~]#

cat > /u01/oracle/scripts/add_asm_filesystem.sh <<EOF
# Run as R00T
$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/bin/srvctl
add filesystem \
-device /dev/asm/<acfs_volume> \
```

```
-volume ACFS_GG1 \  
-diskgroup DATA \  
-path /mnt/acfs_gg1 -user oracle \  
-node gghub_prim1,gghub_prim2 \  
-autostart NEVER \  
-mountowner oracle \  
-mountgroup oinstall \  
-mountperm 755  
EOF  
[root@gghub_prim1 ~]# sh /u01/oracle/scripts/add_asm_filesystem.sh
```

Step 3.3.3 - Verify the Currently Configured ACFS File System

As the grid OS user on the first GGHUB node, use the following command to validate the file system details:

```
[opc@gghub_prim1 ~]$ sudo su - grid  
[grid@gghub_prim1 ~]$ srvctl config filesystem -volume ACFS_GG1  
-diskgroup DATA
```

```
Volume device: /dev/asm/acfs_gg1-256  
Diskgroup name: data  
Volume name: acfs_gg1  
Canonical volume device: /dev/asm/acfs_gg1-256  
Accelerator volume devices:  
Mountpoint path: /mnt/acfs_gg1  
Mount point owner: oracle  
Mount point group: oinstall  
Mount permissions: owner:oracle:rwx,pgrp:oinstall:r-x,other::r-x  
Mount users: grid  
Type: ACFS  
Mount options:  
Description:  
Nodes: gghub_prim1 gghub_prim2  
Server pools: *  
Application ID:  
ACFS file system is enabled  
ACFS file system is individually enabled on nodes:  
ACFS file system is individually disabled on nodes:
```

Step 3.3.4 - Start and Check the Status of the ACFS Resource

As the grid OS user on the first gghub node, use the following command to start and check the file system:

```
[grid@gghub_prim1 ~]$ srvctl start filesystem -volume ACFS_GG1  
-diskgroup DATA -node `hostname`  
[grid@gghub_prim1 ~]$ srvctl status filesystem -volume ACFS_GG1  
-diskgroup DATA  
ACFS file system /mnt/acfs_gg1 is mounted on nodes gghub_prim1
```

The CRS resource created is named using the format `ora.diskgroup_name.volume_name.acfs`. Using the above file system example, the CRS resource is called `ora.data.acfs_gg.acfs`.

As the grid OS user on the first gghub node, use the following command to see the ACFS resource in CRS:

```
[grid@gghub_prim1 ~]$ crsctl stat res ora.data.acfs_gg1.acfs

NAME=ora.data.acfs_gg1.acfs
TYPE=ora.acfs_cluster.type
TARGET=ONLINE
STATE=ONLINE on gghub_prim1
```

Step 3.3.5 – Create CRS Dependencies Between ACFS and an Application VIP

To ensure that the file system is mounted on the same Oracle GGHUB node as the VIP, add the VIP CRS resource as a dependency to the ACFS resource, using the following example commands. Each separate replicated ACFS file system will have its own dedicated VIP.

1. As the root OS user on the first GGHUB node, use the following command to determine the current start and stop dependencies of the VIP resource:

```
[opc@gghub_prim1 ~]$ sudo su -
[root@gghub_prim1 ~]# $(grep ^crs_home /etc/oracle/olr.loc | cut -d=
-f2)/bin/crsctl stat res -w "TYPE co appvip" |grep NAME | cut -f2 -d"="
gghub_prim_vip1

[root@gghub_prim1 ~]# export APPVIP=gghub_prim_vip1
[root@gghub_prim1 ~]# $(grep ^crs_home /etc/oracle/olr.loc | cut -d=
-f2)/bin/crsctl stat res $APPVIP -f|grep _DEPENDENCIES

START_DEPENDENCIES=hard(ora.net1.network) pullup(ora.net1.network)
STOP_DEPENDENCIES=hard(intermediate:ora.net1.network)
```

2. As the root OS user on the first GGHUB node, determine the ACFS file system name:

```
[root@gghub_prim1 ~]# $(grep ^crs_home /etc/oracle/olr.loc | cut -d=
-f2)/bin/crsctl stat res -w "NAME co acfs_gg1" |grep NAME

NAME=ora.data.acfs_gg.acfs

[root@gghub_prim1 ~]# export ACFS_NAME='ora.data.acfs_gg1.acfs'
```

3. As the root OS user on the first GGHUB node, modify the start and stop dependencies of the VIP resource:

```
[root@gghub_prim1 ~]# $(grep ^crs_home /etc/oracle/olr.loc | cut -d=
-f2)/bin/crsctl modify res $APPVIP -attr
"START_DEPENDENCIES='hard(ora.net1.network,$ACFS_NAME)
pullup(ora.net1.network)

pullup:always($ACFS_NAME)',STOP_DEPENDENCIES='hard(intermediate:ora.net1.ne
twork,$ACFS_NAME)',HOSTING_MEMBERS=,PLACEMENT=balanced"
```

4. As the grid OS user on the first GGHUB node, start the VIP resource:

```
[grid@gghub_prim1 ~]$ $(grep ^crs_home /etc/oracle/olr.loc | cut -d=
-f2)/bin/crsctl stat res -w "TYPE co appvip" |grep NAME | cut -f2 -d"="
```

```
gghub_prim_vip1
```

```
[grid@gghub_prim1 ~]$ export APPVIP=gghub_prim_vip1
```

```
[grid@gghub_prim1 ~]$ crsctl start resource $APPVIP
```

```
CRS-2672: Attempting to start 'gghub_prim_vip1' on 'gghub_prim1'
```

```
CRS-2676: Start of 'gghub_prim_vip1' on 'gghub_prim1' succeeded
```

Note

Before moving to the next step, it is important to ensure the VIP can be mounted on both GGHUB nodes.

5. As the grid OS user on the first GGHUB node, relocate the VIP resource:

```
[grid@gghub_prim1 ~]$ crsctl relocate resource $APPVIP -f
```

```
CRS-2673: Attempting to stop 'gghub_prim_vip1' on 'gghub_prim1'
```

```
CRS-2677: Stop of 'gghub_prim_vip1' on 'gghub_prim1' succeeded
```

```
CRS-2673: Attempting to stop 'ora.data.acfs_gg1.acfs' on 'gghub_prim1'
```

```
CRS-2677: Stop of 'ora.data.acfs_gg1.acfs' on 'gghub_prim1' succeeded
```

```
CRS-2672: Attempting to start 'ora.data.acfs_gg1.acfs' on 'gghub_prim2'
```

```
CRS-2676: Start of 'ora.data.acfs_gg1.acfs' on 'gghub_prim2' succeeded
```

```
CRS-2672: Attempting to start 'gghub_prim_vip1' on 'gghub_prim2'
```

```
CRS-2676: Start of 'gghub_prim_vip1' on 'gghub_prim2' succeeded
```

```
[grid@gghub_prim1 ~]$ crsctl status resource $APPVIP
```

```
NAME=gghub_prim_vip1
```

```
TYPE=app.appvitypex2.type
```

```
TARGET=ONLINE
```

```
STATE=ONLINE on gghub_prim2
```

```
[grid@gghub_prim1 ~]$ crsctl relocate resource $APPVIP -f
```

```
CRS-2673: Attempting to stop 'gghub_prim_vip1' on 'gghub_prim2'
```

```
CRS-2677: Stop of 'gghub_prim_vip1' on 'gghub_prim2' succeeded
```

```
CRS-2673: Attempting to stop 'ora.data.acfs_gg1.acfs' on 'gghub_prim2'
```

```
CRS-2677: Stop of 'ora.data.acfs_gg1.acfs' on 'gghub_prim2' succeeded
```

```
CRS-2672: Attempting to start 'ora.data.acfs_gg1.acfs' on 'gghub_prim1'
```

```
CRS-2676: Start of 'ora.data.acfs_gg1.acfs' on 'gghub_prim1' succeeded
```

```
CRS-2672: Attempting to start 'gghub_prim_vip1' on 'gghub_prim1'
```

```
CRS-2676: Start of 'gghub_prim_vip1' on 'gghub_prim1' succeeded
```

6. As the grid OS user on the first GGHUB node, check the status of the ACFS file system:

```
[grid@gghub_prim1 ~]$ srvctl status filesystem -volume ACFS_GG1 -diskgroup DATA
```

```
ACFS file system /mnt/acfs_gg1 is mounted on nodes gghub_prim1
```


Step 3.3.6 – Create the SSH Daemon CRS Resource

ACFS replication uses secure shell (ssh) to communicate between the primary and standby file systems using the virtual IP addresses that were previously created. When a server is rebooted, the ssh daemon is started before the VIP CRS resource, preventing access to the cluster using VIP. The following instructions create an ssh restart CRS resource that will restart the ssh daemon after the virtual IP resource is started. A separate ssh restart CRS resource is needed for each replicated file system.

As the grid OS user on all GGHUB nodes, copy the CRS action script to restart the ssh daemon. Place the script in the same location on all primary and standby GGHUB nodes:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ unzip /u01/oracle/stage/gghub_scripts_<YYYYMMDD>.zip
-d /u01/oracle/scripts/
```

```
Archive: /u01/oracle/stage/gghub_scripts_<YYYYMMDD>.zip
  inflating: /u01/oracle/scripts/acfs_primary.scr
  inflating: /u01/oracle/scripts/acfs_standby.scr
  inflating: /u01/oracle/scripts/sshd_restart.scr
  inflating: /u01/oracle/scripts/add_acfs_primary.sh
  inflating: /u01/oracle/scripts/add_acfs_standby.sh
  inflating: /u01/oracle/scripts/add_nginx.sh
  inflating: /u01/oracle/scripts/add_sshd_restart.sh
  inflating: /u01/oracle/scripts/reverse_proxy_settings.sh
  inflating: /u01/oracle/scripts/secureServices.py
```

As the root OS user on the first GGHUB node, create the CRS resource using the following command:

```
[opc@gghub_prim1 ~]$ sudo su -
[root@gghub_prim1 ~]# sh /u01/oracle/scripts/add_sshd_restart.sh
```

Application VIP Name: gghub_prim_vip

As the grid OS user on the first GGHUB node, start and test the CRS resource:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ crsctl stat res sshd_restart
NAME=sshd_restart
TYPE=cluster_resource
TARGET=OFFLINE
STATE=OFFLINE

[grid@gghub_prim1 ~]$ crsctl start res sshd_restart

CRS-2672: Attempting to start 'sshd_restart' on 'gghub_prim1'
CRS-2676: Start of 'sshd_restart' on 'gghub_prim1' succeeded

[grid@gghub_prim1 ~]$ cat /tmp/sshd_restarted
STARTED

[grid@gghubtest1 ~]$ crsctl stop res sshd_restart
```

```
CRS-2673: Attempting to stop 'sshd_restart' on 'gghub_prim1'
CRS-2677: Stop of 'sshd_restart' on 'gghub_prim1' succeeded
```

```
[grid@gghub1 ~]$ cat /tmp/sshd_restarted
STOPPED
```

```
[grid@gghub1 ~]$ crsctl start res sshd_restart
```

```
CRS-2672: Attempting to start 'sshd_restart' on 'gghub_prim1'
CRS-2676: Start of 'sshd_restart' on 'gghub_prim1' succeeded
```

```
[grid@gghub1 ~]$ crsctl stat res sshd_restart
```

```
NAME=sshd_restart
TYPE=cluster_resource
TARGET=ONLINE
STATE=ONLINE on gghub_prim1
```

Step 3.3.7 – Enable ACFS Replication

ACFS snapshot-based replication uses openssh to transfer the snapshots from between the primary and standby hosts using the designated replication user, which is commonly the grid user.

1. As the grid OS user in the primary and standby hub systems, follow the instructions provided in Configuring ssh for Use With Oracle ACFS Replication to configure the ssh connectivity between the primary and standby nodes.
2. As the grid OS user on all primary and standby GGHUB nodes, use ssh to test connectivity between all primary to standby nodes, and in the reverse direction using ssh as the replication user:

```
# On the Primary GGHUB
```

```
[grid@gghub_prim1 ~]$ ssh gghub_stby_vip1.frankfurt.goldengate.com hostname
gghub_stby1
```

```
[grid@gghub_prim2 ~]$ ssh gghub_stby_vip1.frankfurt.goldengate.com hostname
gghub_stby1
```

```
# On the Standby GGHUB
```

```
[grid@gghub_stby1 ~]$ ssh gghub_prim_vip1.frankfurt.goldengate.com hostname
gghub_prim1
```

```
[grid@gghub_stby2 ~]$ ssh gghub_prim_vip1.frankfurt.goldengate.com hostname
gghub_prim1
```

3. As the grid OS user on the primary and standby GGHUB nodes where ACFS is mounted, use acfsutil to test connectivity between the primary and the standby nodes:

```
# On the Primary GGHUB
```

```
[grid@gghub_prim1 ~]$ srvctl status filesystem -volume ACFS_GG1 -diskgroup
DATA
ACFS file system /mnt/acfs_gg1 is mounted on nodes gghub_prim1
```

```
[grid@gghub_prim1 ~]$ acfsutil repl info -c -u
grid gghub_prim_vip1.frankfurt.goldengate.com
gghub_stby_vip1.frankfurt.goldengate.com
/mnt/acfs_gg1
A valid 'ssh' connection was detected for standby node
gghub_prim_vip1.frankfurt.goldengate.com as user grid.
A valid 'ssh' connection was detected for standby node
gghub_stby_vip1.frankfurt.goldengate.com as user grid.
```

On the Standby GGHUB

```
[grid@gghub_stby1 ~]$ srvctl status filesystem -volume ACFS_GG1 -diskgroup
DATA
ACFS file system /mnt/acfs_gg1 is mounted on nodes gghub_stby1
```

```
[grid@gghub_stby1 ~]$ acfsutil repl info -c -u grid
gghub_prim_vip1.frankfurt.goldengate.com
gghub_stby_vip1.frankfurt.goldengate.com
/mnt/acfs_gg1
A valid 'ssh' connection was detected for standby node
gghub_prim_vip1.frankfurt.goldengate.com as user grid.
A valid 'ssh' connection was detected for standby node
gghub_stby_vip1.frankfurt.goldengate.com as user grid.
```

4. If the `acfsutil` command is executed from a GGHUB node where ACFS is not mounted, the error ACFS-05518 will be shown as expected. Use `srvctl status filesystem` to find the GGHUB where ACFS is mounted and re-execute the command:

```
[grid@gghub_prim1 ~]$ acfsutil repl info -c -u grid
gghub_stby_vip1.frankfurt.goldengate.com
gghub_stby_vip1.frankfurt.goldengate.com
/mnt/acfs_gg1
acfsutil repl info: ACFS-05518: /mnt/acfs_gg1 is not an ACFS mount point
```

```
[grid@gghub_prim1 ~]$ srvctl status filesystem -volume ACFS_GG1 -diskgroup
DATA
ACFS file system /mnt/acfs_gg1 is mounted on nodes gghub_prim2
```

```
[grid@gghub_prim1 ~]$ ssh gghub_prim2
[grid@gghub_prim2 ~]$ acfsutil repl info -c -u grid
gghub_prim_vip1.frankfurt.goldengate.com
gghub_stby_vip1.frankfurt.goldengate.com
/mnt/acfs_gg1
A valid 'ssh' connection was detected for standby node
gghub_prim_vip1.frankfurt.goldengate.com as user grid.
A valid 'ssh' connection was detected for standby node
gghub_stby_vip1.frankfurt.goldengate.com as user grid.
```

Note

Make sure the connectivity is verified between all primary nodes to all standby nodes, as well as in the opposite direction. Only continue when there are no errors with any of the connection tests.

5. As the grid OS user on the standby GGHUB node where ACFS is currently mounted, initialize ACFS replication:

```
[grid@gghub_stby1 ~]$ srvctl status filesystem -volume ACFS_GG1 -diskgroup DATA
```

ACFS file system /mnt/acfs_gg1 is mounted on nodes gghub_stby1

```
[grid@gghub_stby1 ~]$ /sbin/acfsutil repl init standby -u grid /mnt/acfs_gg1
```

6. As the grid OS user on the primary GGHUB node where ACFS is currently mounted, initialize ACFS replication:

```
[grid@gghub_prim1 ~]$ srvctl status filesystem -volume ACFS_GG1 -diskgroup DATA
```

ACFS file system /mnt/acfs_gg is mounted on nodes gghub_prim1

```
[grid@gghub_prim1 ~]$ /sbin/acfsutil repl init primary -C -p
grid@gghub_prim_vip1.frankfurt.goldengate.com -s
grid@gghub_stby_vip1.frankfurt.goldengate.com -m /mnt/acfs_gg1 /mnt/acfs_gg1
```

7. As the grid OS user on the primary and standby GGHUB nodes, monitor the initialization progress, when the status changes to "Send Completed" it means the initial primary file system copy has finished and the primary file system is now being replicated to the standby host:

On the Primary GGHUB

```
[grid@gghub_prim1 ~]$ /sbin/acfsutil repl info -c -v /mnt/acfs_gg1 | grep -i Status
```

Status: Send Completed

On the Standby GGHUB

```
[grid@gghub_prim1 ~]$ /sbin/acfsutil repl info -c -v /mnt/acfs_gg1 | grep -i Status
```

Status: Receive Completed

8. As the grid OS user on the primary and standby GGHUB nodes, verify and monitor the ACFS replicated file system:

On the Primary GGHUB

```
[grid@gghub_prim1 ~]$ acfsutil repl util verifystandby /mnt/acfs_gg1
```

verifystandby returned: 0

On the Standby GGHUB

```
[grid@gghubtest31 ~]$ acfsutil repl util verifyprimary /mnt/acfs_gg1
```

```
verifyprimary returned: 0
```

Note

Both commands will return a value of 0 (zero) if there are no problems detected. If a non-zero value is returned, refer to [Troubleshooting ACFS Replication](#) for monitoring, diagnosing, and resolving common issues with ACFS Replication before continuing.

- As the grid OS user on the primary GGHUB node, use the following command to monitor the status of the ACFS replication:

```
[grid@gghub_prim1 ~]$ /sbin/acfsutil repl info -c -v /mnt/acfs_gg1
```

```
Site: Primary
Primary hostname:
gghub_prim_vip1.frankfurt.goldengate.com
Primary path: /mnt/acfs_gg1
Primary status: Running
Background Resources: Active

Standby connect string:
grid@gghub_stby_vip1.frankfurt.goldengate.com
Standby path: /mnt/acfs_gg1
Replication interval: 0 days, 0 hours, 0 minutes, 0 seconds
Sending primary as of: Fri May 05 12:37:02 2023
Status: Send Completed
Lag Time: 00:00:00
Retries made: 0
Last send started at: Fri May 05 12:37:02 2023
Last send completed at: Fri May 05 12:37:12 2023
Elapsed time for last send: 0 days, 0 hours, 0 minutes, 10 seconds
Next send starts at: now
Replicated tags:
Data transfer compression: Off
ssh strict host key checking: On
Debug log level: 3
Replication ID: 0x4d7d34a
```

- As the grid OS user on the standby GGHUB node where ACFS is currently mounted, use the following command to monitor the status of the ACFS replication:

```
[grid@gghub_stby1 ~]$ /sbin/acfsutil repl info -c -v /mnt/acfs_gg1
```

```
Site: Standby
Primary hostname:
gghub_prim_vip1.frankfurt.goldengate.com
Primary path: /mnt/acfs_gg1

Standby connect string:
```

```

grid@gghub_stby_vip1.frankfurt.goldengate.com
Standby path:                /mnt/acfs_gg1
Replication interval:        0 days, 0 hours, 0 minutes, 0 seconds
Last sync time with primary: Fri May 05 12:37:02 2023
Receiving primary as of:     Fri May 05 12:37:02 2023
Status:                      Receive Completed
Last receive started at:     Fri May 05 12:37:02 2023
Last receive completed at:   Fri May 05 12:37:07 2023
Elapsed time for last receive: 0 days, 0 hours, 0 minutes, 5 seconds
Data transfer compression:   Off
ssh strict host key checking: On
Debug log level:             3
Replication ID:              0x4d7d34a

```

Step 3.3.8 – Create the ACFS Replication CRS Action Scripts

To determine the health of the ACFS primary and standby file systems, CRS action scripts are used. At predefined intervals the action scripts report the health of the file systems into the CRS trace file `crsd_scriptagent_grid.trc`, located in the Grid Infrastructure trace file directory `/u01/app/grid/diag/crs/<node_name>/crs/trace` on each of the primary and standby file system of the GGHUB nodes.

On both, the primary and standby file system clusters, there are two scripts required. One to monitor the local primary file system, and if the remote standby file system is available, and one to monitor the local standby file system and check remote primary file systems' availability. Example scripts are provided to implement the ACFS monitoring, but you must edit them to suit your environment.

Each replicated file system will need its own `acfs_primary` and `acfs_standby` action scripts.

Step 3.3.8.1 - Action Script `acfs_primary.scr`

The `acfs_primary` CRS resource checks whether the current ACFS mount is a primary file system and confirms that the standby file system is accessible and receiving replicated data. The resource is used to automatically determine if Oracle GoldenGate can start processes on the primary Oracle GoldenGate hub. If the standby file system is not accessible by the primary, the example script makes multiple attempts to verify the standby file system.

The `acfs_primary` CRS resource runs on both, the primary and standby hosts, but only returns success when the current file system is the primary file system, and the standby file system is accessible. The script must be placed in the same location on all primary and standby file system nodes.

The following parameters use suggested default settings, which should be tested before changing their values:

- `MOUNT_POINT=/mnt/acfs_gg1`
The replicated ACFS mount point
- `PATH_NAME=$MOUNT_POINT/status/acfs_primary`
Must be unique from other mount files
- `ATTEMPTS=3`
Number of attempts to check the remote standby file system
- `INTERVAL=10`
Number of seconds between each attempt

As the grid OS user on all primary and standby GGHUB nodes, edit the `acfs_primary.scr` script to match the environment:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ vi /u01/oracle/scripts/acfs_primary.scr
```

As the oracle OS user on the primary GGHUB node where ACFS is currently mounted, run the following commands to create the status directory:

```
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghub_prim1 ~]$ mkdir /mnt/acfs_gg1/status
[oracle@gghub_prim1 ~]$ chmod g+w /mnt/acfs_gg1/status
```

As the grid OS user on the primary and standby GGHUB node where ACFS is currently mounted, run the following command to register the `acfs_primary` action script for monitoring the primary and standby file system:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ sh /u01/oracle/scripts/add_acfs_primary.sh

#####
##
List of ACFS resources:
ora.data.acfs_gg1.acfs
#####
##
ACFS resource name: <ora.data.acfs_gg1.acfs>
```

As the grid OS user on the primary GGHUB node where ACFS is currently mounted, start and check the status of the `acfs_primary` resource:

```
[grid@gghub_prim1 ~]$ crsctl start resource acfs_primary
```

```
CRS-2672: Attempting to start 'acfs_primary' on 'gghub_prim1'
CRS-2676: Start of 'acfs_primary' on 'gghub_prim1' succeeded
```

```
[grid@gghub_prim1 ~]$ crsctl stat resource acfs_primary
```

```
NAME=acfs_primary
TYPE=cluster_resource
TARGET=ONLINE
STATE=ONLINE on gghub_prim1
```

```
[grid@gghub_prim1 ~]$ grep acfs_primary
/u01/app/grid/diag/crs/'hostname'/crs/trace/crsd_scriptagent_grid.trc
|grep check
```

```
2023-05-05 12:57:40.372 :CLSDYNAM:2725328640: [acfs_primary]{1:33562:34377}
[check] Executing action script:
/u01/oracle/scripts/acfs_primary.scr[check]
```

```
2023-05-05 12:57:42.376 :CLSDYNAM:2725328640: [acfs_primary]{1:33562:34377}
[check] SUCCESS: STANDBY file system /mnt/acfs_gg1 is ONLINE
```

As the grid OS user on the standby GGHUB node where ACFS is currently mounted, start and check the status of the acfs_primary resource. This step should fail because acfs_primary should **ONLY** be online on the primary GGHUB:

```
[grid@ggghub_stby1 ~]$ crsctl start res acfs_primary -n `hostname`
```

```
CRS-2672: Attempting to start 'acfs_primary' on 'ggghub_stby1'
CRS-2674: Start of 'acfs_primary' on 'ggghub_stby1' succeeded
CRS-2679: Attempting to clean 'acfs_primary' on 'ggghub_stby1'
CRS-2681: Clean of 'acfs_primary' on 'ggghub_stby1' succeeded
CRS-4000: Command Start failed, or completed with errors.
```

```
[grid@ggghub_stby1 ~]$ crsctl stat res acfs_primary
```

```
NAME=acfs_primary
TYPE=cluster_resource
TARGET=ONLINE
STATE=OFFLINE
```

```
[grid@ggghub_stby1 trace]$ grep acfs_primary
/u01/app/grid/diag/crs/`hostname`/crs/trace/crsd_scriptagent_grid.trc
|grep check
```

```
2023-05-05 13:09:53.343 :CLSDYNAM:3598239488: [acfs_primary]{1:8532:2106}
[check] Executing action script: /u01/oracle/scripts/acfs_primary.scr[check]
2023-05-05 13:09:53.394 :CLSDYNAM:3598239488: [acfs_primary]{1:8532:2106}
[check] Detected local standby file system
2023-05-05 13:09:53.493 :CLSDYNAM:1626130176: [acfs_primary]{1:8532:2106}
[clean] Clean/Abort -- Stopping ACFS file system type checking...
```

Note

The status of the acfs_primary resources will only be ONLINE if the ACFS file system is the primary file system. When starting the resource on a node which is not currently on the primary cluster an error will be reported because the resource fails due to being the standby file system. This error can be ignored. The resource will be in OFFLINE status on the ACFS standby cluster.

Step 3.3.8.2 - Action Script acfs_standby.scr

The acfs_standby resource checks that the local file system is a standby file system and verifies the remote primary file system status. If the primary file system fails verification multiple times (controlled by the action script variables), a warning is output to the CRS trace file crsd_scriptagent_grid.trc located in the Grid Infrastructure trace file directory /u01/app/grid/diag/crs/<node_name>/crs/trace.

This resource runs on both the primary and standby hosts, but only returns success when the current file system is the standby file system, and the primary file system is accessible.

The following parameters use suggested default settings, which should be tested before changing their values.

- MOUNT_POINT=/mnt/acfs_gg
This is the replicated ACFS mount point
- ATTEMPTS=3
Number of tries to check the remote primary file system
- INTERVAL=10
Number of seconds between each attempt

As the grid OS user on all primary and standby GGHUB nodes, edit the `acfs_standby.scr` script to match the environment:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ vi /u01/oracle/scripts/acfs_standby.scr
```

As the grid OS user on the primary GGHUB node where ACFS is currently mounted, run the following command to register the `acfs_standby` action script for monitoring the primary and standby file system:

```
[grid@gghub_prim1 ~]$ crsctl stat res -w "TYPE co appvip"
|grep NAME
```

```
NAME=gghub_prim_vip
```

```
[grid@gghub_prim1 ~]$ vi /u01/oracle/scripts/add_acfs_standby.sh
```

```
crsctl add resource acfs_standby \
-type cluster_resource \
-attr "ACTION_SCRIPT=/u01/oracle/scripts/acfs_standby.scr, \
CHECK_INTERVAL=150, \
CHECK_TIMEOUT=140, \
START_DEPENDENCIES='hard(ora.data.acfs_gg1.acfs,gghub_prim_vip)
pullup:always(ora.data.acfs_gg1.acfs,gghub_prim_vip)', \
STOP_DEPENDENCIES='hard(ora.data.acfs_gg1.acfs,gghub_prim_vip)' \
OFFLINE_CHECK_INTERVAL=300, \
RESTART_ATTEMPTS=0, \
INSTANCE_FAILOVER=0"
```

```
[grid@gghub_prim1 ~]$ sh /u01/oracle/scripts/add_acfs_standby.sh
```

As the grid OS user on the primary GGHUB node where ACFS is currently mounted, start and check the status of the `acfs_standby` resource:

```
[grid@gghub_prim1 ~]$ crsctl start res acfs_standby
```

```
CRS-2672: Attempting to start 'acfs_standby' on 'gghub_prim1'
CRS-2676: Start of 'acfs_standby' on 'gghub_prim1' succeeded
```

```
[grid@gghub_prim1 ~]$ grep acfs_standby
/u01/app/grid/diag/crs/'hostname'/crs/trace/crsd_scriptagent_grid.trc
|egrep 'check|INFO'
```

```
2023-05-05 13:22:09.612 :CLSDYNAM:2725328640: [acfs_standby]{1:33562:34709}
```

```
[start] acfs_standby.scr starting to check ACFS remote primary at
/mnt/acfs_gg1
2023-05-05 13:22:09.612 :CLSDYNAM:2725328640: [acfs_standby]{1:33562:34709}
[check] Executing action script: /u01/oracle/scripts/acfs_standby.scr[check]
2023-05-05 13:22:09.663 :CLSDYNAM:2725328640: [acfs_standby]{1:33562:34709}
[check] Local PRIMARY file system /mnt/acfs_gg1
```

As the grid OS user on the standby GGHUB node where ACFS is currently mounted, run the following command to register the acfs_standby action script for monitoring the primary and standby file system:

```
[grid@gghub_stby1 ~]$ crsctl stat res -w "TYPE co appvip"
|grep NAME
```

```
NAME=gghub_stby_vip
```

```
[grid@gghub_stby1 ~]$ vi /u01/oracle/scripts/add_acfs_standby.sh
```

```
crsctl add resource acfs_standby \
-type cluster_resource \
-attr "ACTION_SCRIPT=/u01/oracle/scripts/acfs_standby.scr, \
CHECK_INTERVAL=150, \
CHECK_TIMEOUT=140, \
START_DEPENDENCIES='hard(ora.data.acfs_gg1.acfs,gghub_stby_vip)
pullup:always(ora.data.acfs_gg1.acfs,gghub_stby_vip)', \
STOP_DEPENDENCIES='hard(ora.data.acfs_gg1.acfs,gghub_stby_vip)' \
OFFLINE_CHECK_INTERVAL=300, \
RESTART_ATTEMPTS=0, \
INSTANCE_FAILOVER=0"
```

```
[grid@gghub_stby1 ~]$ sh /u01/oracle/scripts/add_acfs_standby.sh
```

As the grid OS user on the primary GGHUB node where ACFS is currently mounted, start and check the status of the acfs_standby resource:

```
[grid@gghub_stby1 ~]$ crsctl start res acfs_standby
```

```
CRS-2672: Attempting to start 'acfs_standby' on 'gghub_stby1'
CRS-2676: Start of 'acfs_standby' on 'gghub_stby1' succeeded
```

```
[grid@gghub_stby1 ~]$ grep acfs_standby
/u01/app/grid/diag/crs/`hostname`/crs/trace/crsd_scriptagent_grid.trc
|egrep 'check|INFO'
2023-05-05 13:25:20.699 :CLSDYNAM:1427187456: [acfs_standby]{1:8532:2281}
[check] SUCCESS: PRIMARY file system /mnt/acfs_gg1 is ONLINE
2023-05-05 13:25:20.699 : AGFW:1425086208: [ INFO] {1:8532:2281}
acfs_standby 1 1 state changed from: STARTING to: ONLINE
2023-05-05 13:25:20.699 : AGFW:1425086208: [ INFO] {1:8532:2281}
Started implicit monitor for [acfs_standby 1 1]
interval=150000 delay=150000
2023-05-05 13:25:20.699 : AGFW:1425086208: [ INFO] {1:8532:2281}
Agent sending last reply for: RESOURCE_START[acfs_standby 1 1]
ID 4098:8346
```

Step 3.3.9 – Test ACFS GGhub Node Relocation

It is very important to test planned and unplanned ACFS GGhub node relocations and server role transitions before configuring Oracle GoldenGate.

As the grid OS user on the primary and standby GGHUB nodes, run the following command to relocate ACFS between the GGhub nodes:

```
[grid@gghub_prim1 ~]$ srvctl status filesystem -volume ACFS_GG1
-diskgroup DATA
```

ACFS file system /mnt/acfs_gg1 is mounted on nodes gghub_prim1

```
[grid@gghub_prim1 ~]$ srvctl relocate filesystem -diskgroup DATA
-volume acfs_gg1 -force
```

```
[grid@gghub_prim1 ~]$ srvctl status filesystem -volume ACFS_GG1
-diskgroup DATA
```

ACFS file system /mnt/acfs_gg1 is mounted on nodes gghub_prim2

As the grid OS user on the primary and standby GGHUB nodes, verify that the file system is mounted on another node, along with the VIP, sshd_restart, and the two ACFS resources (acfs_primary and acfs_standby) using the following example command:

```
[grid@gghub_prim1 ~]$ crsctl stat res sshd_restart acfs_primary
acfs_standby ora.data.acfs_gg1.acfs sshd_restart -t
```

```
-----
--
Name                Target  State        Server                State details
-----
--
Cluster Resources
-----
--
acfs_primary
   1                ONLINE  ONLINE      gghub_prim2          STABLE
acfs_standby
   1                ONLINE  ONLINE      gghub_prim2          STABLE
gghubfad2
   1                ONLINE  ONLINE      gghub_prim2          STABLE
ora.data.acfs_gg1.acfs
   1                ONLINE  ONLINE      gghub_prim2          mounted on /mnt/
acfs                                                         _gg1, STABLE
sshd_restart
   1                ONLINE  ONLINE      gghub_prim2          STABLE
-----
--
```

```
[grid@gghub_stby1 ~]$ crsctl stat res sshd_restart acfs_primary acfs_standby
ora.data.acfs_gg1.acfs sshd_restart -t
```

```

-----
--
Name          Target  State      Server          State details
-----
--
Cluster Resources
-----
--
acfs_primary
  1          ONLINE  OFFLINE
acfs_standby
  1          ONLINE  ONLINE      gghub_stby2     STABLE
ora.data.acfs_gg1.acfs
  1          ONLINE  ONLINE      gghub_stby2     mounted on /mnt/
acfs
                                     _gg1, STABLE

sshd_restart
  1          ONLINE  ONLINE      gghub_stby2     STABLE
-----
--

```

Step 3.3.10 – Test ACFS Switchover Between the Primary and Standby GGHUB

As the grid OS user on the standby GGHUB node, run the following command to issue an ACFS switchover (role reversal) between the primary and standby GGHUB:

```
[grid@gghub_stby2 ~]$ crsctl stat res ora.data.acfs_gg1.acfs
```

```

NAME=ora.data.acfs_gg1.acfs
TYPE=ora.acfs_cluster.type
TARGET=ONLINE
STATE=ONLINE on gghub_stby2

```

```
[grid@gghub_stby2 ~]$ acfsutil repl failover /mnt/acfs_gg1
```

```
[grid@gghub_stby2 ~]$ /sbin/acfsutil repl info -c -v /mnt/acfs_gg1
```

```

Site:                               Primary
Primary hostname:                   gghub_stby_vip.frankfurt.goldengate.com
Primary path:                        /mnt/acfs_gg1
Primary status:                      Running
Background Resources:               Active

Standby connect string:              gghub_prim_vip.frankfurt.goldengate.com
Standby path:                        /mnt/acfs_gg1
Replication interval:                0 days, 0 hours, 0 minutes, 0 seconds
Sending primary as of:               Fri May 05 13:51:37 2023
Status:                             Send Completed
Lag Time:                           00:00:00
Retries made:                        0
Last send started at:                Fri May 05 13:51:37 2023
Last send completed at:              Fri May 05 13:51:48 2023
Elapsed time for last send:          0 days, 0 hours, 0 minutes, 11 seconds
Next send starts at:                 now

```

```
Replicated tags:
Data transfer compression:      Off
ssh strict host key checking:    On
Debug log level:                3
Replication ID:                 0x4d7d34a
```

As the grid OS user on the new standby GGHUB node (old primary), run the following command to issue an ACFS switchover (role reversal) between the primary and standby GGHUB. This step is optional but recommended to return the sites to the original role:

```
[grid@gghub_prim2 ~]$ crsctl stat res ora.data.acfs_gg1.acfs
```

```
NAME=ora.data.acfs_gg1.acfs
TYPE=ora.acfs_cluster.type
TARGET=ONLINE
STATE=ONLINE on gghub_prim2
```

```
[grid@gghub_prim2 ~]$ /sbin/acfsutil repl info -c -v /mnt/acfs_gg1 |grep Site
Site:                               Standby
```

```
[grid@gghub_prim2 ~]$ acfsutil repl failover /mnt/acfs_gg1
```

```
[grid@gghub_prim2 ~]$ /sbin/acfsutil repl info -c -v /mnt/acfs_gg1 |grep Site
Site:                               Primary
```

Step 3.4 - Create the Oracle GoldenGate Deployment

Once the Oracle GoldenGate software has been installed in GGHUB, the next step is to create a response file to create the GoldenGate deployment using the Oracle GoldenGate Configuration Assistant.

Due the unified build feature introduced in Oracle GoldenGate 21c, a single deployment can now manage Extract and Replicat processes that attach to different Oracle Database versions. Each deployment is created with an Administration Server and (optionally) Performance Metrics Server. If the GoldenGate trail files don't need to be transferred to another hub or GoldenGate environment, there is no need to create a Distribution or Receiver Server.

There are two limitations that currently exist with Oracle GoldenGate and XAG:

- A Service Manager that is registered with XAG can only manage a single deployment. If multiple deployments are required, each deployment must use their own Service Manager. Oracle GoldenGate release 21c simplifies this requirement because it uses a single deployment to support Extract and Replicat processes connecting to different versions of the Oracle Database.
- Each Service Manager registered with XAG must belong to separate OGG_HOME software installation directories. Instead of installing Oracle GoldenGate multiple times, the recommended approach is to install Oracle GoldenGate one time, and then create a symbolic link for each Service Manager OGG_HOME. The symbolic link and OGG_HOME environment variable must be configured **before** running the Oracle GoldenGate Configuration Assistant on all Oracle RAC nodes.

1. Create a Response File

For a silent configuration, please copy the following example file and paste it into any location the oracle user can access. Edit the following values appropriately:

- CONFIGURATION_OPTION
- DEPLOYMENT_NAME
- ADMINISTRATOR_USER
- SERVICEMANAGER_DEPLOYMENT_HOME
- OGG_SOFTWARE_HOME
- OGG_DEPLOYMENT_HOME
- ENV_TNS_ADMIN
- OGG_SCHEMA

Example Response File (oggca.rsp):

As the oracle OS user on the primary GGHUB node where ACFS is currently mounted, create and edit the response file oggca.rsp to create the Oracle GoldenGate deployment:

```
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghub_prim1 ~]$ vi /u01/oracle/stage/oggca.rsp

oracle.install.responseFileVersion=/oracle/install/
rspfmt_oggca_response_schema_v21_1_0
CONFIGURATION_OPTION=ADD
DEPLOYMENT_NAME=gghub1
ADMINISTRATOR_USER=oggadmin
ADMINISTRATOR_PASSWORD=<password_for_oggadmin>
SERVICEMANAGER_DEPLOYMENT_HOME=/mnt/acfs_gg1/deployments/ggsm01
HOST_SERVICEMANAGER=localhost
PORT_SERVICEMANAGER=9100
SECURITY_ENABLED=false
STRONG_PWD_POLICY_ENABLED=true
CREATE_NEW_SERVICEMANAGER=true
REGISTER_SERVICEMANAGER_AS_A_SERVICE=false
INTEGRATE_SERVICEMANAGER_WITH_XAG=true
EXISTING_SERVICEMANAGER_IS_XAG_ENABLED=false
OGG_SOFTWARE_HOME=/u01/app/oracle/goldengate/gg21c
OGG_DEPLOYMENT_HOME=/mnt/acfs_gg1/deployments/gg01
ENV_LD_LIBRARY_PATH=${OGG_HOME}/lib/instantclient:${OGG_HOME}/lib
ENV_TNS_ADMIN=/u01/app/oracle/goldengate/network/admin
FIPS_ENABLED=false
SHARDING_ENABLED=false
ADMINISTRATION_SERVER_ENABLED=true
PORT_ADMINSRVR=9101
DISTRIBUTION_SERVER_ENABLED=true
PORT_DISTSRVR=9102
NON_SECURE_DISTSRVR_CONNECTS_TO_SECURE_RCVRSRVR=false
RECEIVER_SERVER_ENABLED=true
PORT_RCVRSRVR=9103
METRICS_SERVER_ENABLED=true
METRICS_SERVER_IS_CRITICAL=false
PORT_PMSRVR=9104
UDP_PORT_PMSRVR=9105
PMSRVR_DATASTORE_TYPE=BDB
```

```
PMSRVR_DATASTORE_HOME=/u01/app/oracle/goldengate/datastores/gghub1
OGG_SCHEMA=ggadmin
```

2. Create the Oracle GoldenGate Deployment

As the oracle OS user on the primary GGHUB node where ACFS is currently mounted, run oggca.sh to create the GoldenGate deployment:

```
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghub_prim1 ~]$ export OGG_HOME=/u01/app/oracle/goldengate/gg21c
[oracle@gghub_prim1 ~]$ $OGG_HOME/bin/oggca.sh -silent
-responseFile /u01/oracle/stage/oggca.rsp
```

Successfully Setup Software.

3. Create the Oracle GoldenGate Datastores and TNS_ADMIN Directories

As the oracle OS user on all GGHUB nodes of the primary and standby systems, run the following commands to create the Oracle GoldenGate Datastores and TNS_ADMIN directories:

```
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghub_prim1 ~]$ mkdir -p /u01/app/oracle/goldengate/network/admin
[oracle@gghub_prim1 ~]$ mkdir -p /u01/app/oracle/goldengate/datastores/
gghub1
```

Step 3.5 - Configure Oracle Grid Infrastructure Agent (XAG)

The following step-by-step procedure shows how to configure Oracle Clusterware to manage GoldenGate using the Oracle Grid Infrastructure Standalone Agent (XAG). Using XAG automates the ACFS file system mounting, as well as the stopping and starting of the GoldenGate deployment when relocating between Oracle GGHUB nodes.

Step 3.5.1 - Install the Oracle Grid Infrastructure Standalone Agent

It is recommended to install the XAG software as a standalone agent outside the Grid Infrastructure ORACLE_HOME. This way, you can use the latest XAG release available, and the software can be updated without impact to the Grid Infrastructure.

Install the XAG standalone agent outside of the Oracle Grid Infrastructure home directory. XAG must be installed in the same directory on all GGHUB nodes in the system where GoldenGate is installed.

As the grid OS user on the first GGHUB node of the primary and standby systems, unzip the software and run xagsetup.sh:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ unzip /u01/oracle/stage/p31215432_190000_Generic.zip
-d /u01/oracle/stage
[grid@gghub_prim1 ~]$ /u01/oracle/stage/xag/xagsetup.sh --install
--directory /u01/app/grid/xag --all_nodes
```

```
Installing Oracle Grid Infrastructure Agents on: gghub_prim1
Installing Oracle Grid Infrastructure Agents on: gghub_prim2
Updating XAG resources.
Successfully updated XAG resources.
```

As the grid OS user on all GGHUB nodes of the primary and standby systems, add the location of the newly installed XAG software to the PATH variable so that the location of `agctl` is known when the grid user logs on to the machine.

```
[grid@gghub_prim1 ~]$ vi ~/.bashrc
```

```
PATH=/u01/app/grid/xag/bin:$PATH:/u01/app/19.0.0.0/grid/bin; export PATH
```

Note

It is essential to ensure that the XAG bin directory is specified BEFORE the Grid Infrastructure bin directory to ensure the correct `agctl` binary is found. This should be set in the grid user environment to take effect when logging on, such as in the `.bashrc` file when the Bash shell is in use.

Step 3.5.2 - Register Oracle Grid Infrastructure Agent on the Primary and Standby GGhubs

The following procedure shows how to configure Oracle Clusterware to manage Oracle GoldenGate using the Oracle Grid Infrastructure Standalone Agent (XAG). Using XAG automates the mounting of the shared file system as well as the stopping and starting of the Oracle GoldenGate deployment when relocating between Oracle GGHUB nodes.

Oracle GoldenGate must be registered with XAG so that the deployment is started and stopped automatically when the database is started, and the file system is mounted.

To register Oracle GoldenGate Microservices Architecture with XAG, use the following command format.

```
agctl add goldengate <instance_name>
--gg_home <GoldenGate_Home>
--service_manager
--config_home <GoldenGate_SvcMgr_Config>
--var_home <GoldenGate_SvcMgr_Var_Dir>
--port <port number>
--oracle_home <$OGG_HOME/lib/instantclient>
--adminuser <OGG admin user>
--user <GG instance user>
--group <GG instance group>
--file_systems <CRS_resource_name>
--db_services <service_name>
--use_local_services
--attribute START_TIMEOUT=60
```

Where:

- `--gg_home` specifies the location of the GoldenGate software.
- `--service_manager` indicates this is an GoldenGate Microservices instance.
- `--config_home` specifies the GoldenGate deployment configuration home directory.
- `--var_home` specifies the GoldenGate deployment variable home directory.
- `--oracle_home` specifies the Oracle Instant Client home
- `--port` specifies the deployment Service Manager port number.

- `--adminuser` specifies the GoldenGate Microservices administrator account name.
- `--user` specifies the name of the operating system user that owns the GoldenGate deployment.
- `--group` specifies the name of the operating system group that owns the GoldenGate deployment.
- `--filesystems` specifies the CRS file system resource that must be ONLINE before the deployment is started. This will be the `acfs_primary` resource created in a previous step.
- `--filesystem_verify` specifies if XAG should check the existence of the directories specified by the `config_home` and `var_home` parameters. This should be set to `yes` for the active ACFS primary file system. When adding the GoldenGate instance on the standby cluster, specify `no`.
- `--filesystems_always` specifies that XAG will start the GoldenGate Service Manager on the same GGHUB node as the file system CRS resources, specified by the `--filesystems` parameter.
- `--attributes` specifies that the target status of the resource is online. This is required to automatically start the GoldenGate deployment when the `acfs_primary` resource starts.

The GoldenGate deployment must be registered on the primary and standby GGHUBs where ACFS is mounted in either read-write or read-only mode.

As the grid OS user on the first GGHUB node of the primary and standby systems, run the following command to determine which node of the cluster the file system is mounted on:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ crsctl stat res acfs_standby |grep STATE
STATE=ONLINE on gghub_prim1
```

Step 3.5.2.1 - Register the Primary Oracle GoldenGate Microservices Architecture with XAG

As the root OS user on the first node of the primary GGHUB, register Oracle GoldenGate Microservices Architecture with XAG using the following command format:

```
[opc@gghub_prim1 ~]$ sudo su - root
[root@gghub_prim1 ~]# vi /u01/oracle/scripts/add_xag_goldengate.sh

# Run as ROOT:
/u01/app/grid/xag/bin/agctl add goldengate gghub1 \
--gg_home /u01/app/oracle/goldengate/gg21c \
--service_manager \
--config_home /mnt/acfs_gg1/deployments/ggsm01/etc/conf \
--var_home /mnt/acfs_gg1/deployments/ggsm01/var \
--oracle_home /u01/app/oracle/goldengate/gg21c/lib/instantclient \
--port 9100 \
--adminuser oggadmin \
--user oracle \
--group oinstall \
--filesystems acfs_primary \
--filesystems_always yes \
--filesystem_verify yes \
--attribute TARGET_DEFAULT=online
```

```
[root@gghub_prim1 ~]# sh /u01/oracle/scripts/add_xag_goldengate.sh
Enter password for 'oggadmin' : #####
```

As the grid OS user on the first node of the primary GGHUB, verify that Oracle GoldenGate Microservices Architecture is registered with XAG:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ agctl status goldengate
```

```
Goldengate instance 'gghub1' is not running
```

Step 3.5.2.2 - Register the Standby Oracle GoldenGate Microservices Architecture with XAG

As the root OS user on the first node of the standby GGHUB, register Oracle GoldenGate Microservices Architecture with XAG using the following command format:

```
[opc@gghub_stby1 ~]$ sudo su - root
[root@gghub_stby1 ~]# vi /u01/oracle/scripts/add_xag_goldengate.sh
```

```
# Run as ROOT:
/u01/app/grid/xag/bin/agctl add goldengate gghub1 \
--gg_home /u01/app/oracle/goldengate/gg21c \
--service_manager \
--config_home /mnt/acfs_gg1/deployments/ggsm01/etc/conf \
--var_home /mnt/acfs_gg1/deployments/ggsm01/var \
--oracle_home /u01/app/oracle/goldengate/gg21c/lib/instantclient \
--port 9100 --adminuser oggadmin --user oracle --group oinstall \
--filesystems acfs_primary \
--filesystems_always yes \
--filesystem_verify no \
--attribute TARGET_DEFAULT=online
```

```
[root@gghub_stby1 ~]# sh /u01/oracle/scripts/add_xag_goldengate.sh
Enter password for 'oggadmin' : #####
```

Note

When adding the GoldenGate instance on the standby cluster, specify --filesystem_verify no.

As the grid OS user on the first node of the standby GGHUB, verify that Oracle GoldenGate Microservices Architecture is registered with XAG:

```
[opc@gghub_stby1 ~]$ sudo su - grid
[grid@gghub_stby1 ~]$ agctl status goldengate
```

```
Goldengate instance 'gghub1' is not running
```

Step 3.5.3 - Start the Oracle GoldenGate Deployment

Below is some example agctl commands used to manage the GoldenGate deployment with XAG.

As the grid OS user on the first node of the primary GGHUB, run the following command to start and check Oracle GoldenGate deployment:

```
[opc@gghub_prim1 ~]$ sudo su - grid

[grid@gghub_prim1 ~]$ agctl start goldengate gghub1

[grid@gghub_prim1 ~]$ agctl status goldengate
Goldengate instance 'gghub1' is running on gghub_prim1
```

As the grid OS user on the first GGHUB node, run the following command to validate the configuration parameters for the Oracle GoldenGate resource:

```
[grid@gghub_prim1 ~]$ agctl config goldengate gghub1

Instance name: gghub1
Application GoldenGate location is: /u01/app/oracle/goldengate/gg21c
Goldengate MicroServices Architecture environment: yes
Goldengate Service Manager configuration directory:
  /mnt/acfs_gg1/deployments/ggsm01/etc/conf
Goldengate Service Manager var directory:
  /mnt/acfs_gg1/deployments/ggsm01/var
Service Manager Port: 9100
Goldengate Administration User: oggadmin
Autostart on DataGuard role transition to PRIMARY: no
ORACLE_HOME location is:
  /u01/app/oracle/goldengate/gg21c/lib/instantclient
File System resources needed: acfs_primary
CRS additional attributes set: TARGET_DEFAULT=online
```

For more information see [Oracle Grid Infrastructure Bundled Agent](#).

Step 3.6 - Configure NGINX Reverse Proxy

The GoldenGate reverse proxy feature allows a single point of contact for all the GoldenGate microservices associated with a GoldenGate deployment. Without a reverse proxy, the GoldenGate deployment microservices are contacted using a URL consisting of a hostname or IP address and separate port numbers, one for each of the services. For example, to contact the Service Manager, you could use `http://gghub.example.com:9100`, then the Administration Server is `http://gghub.example.com:9101`, the second Service Manager may be accessed using `http://gghub.example.com:9110`, and so on.

When running Oracle GoldenGate in a High Availability (HA) configuration on Oracle Exadata Database Service with the Grid Infrastructure agent (XAG), there is a limitation preventing more than one deployment from being managed by a GoldenGate Service Manager. Because of this limitation, creating a separate virtual IP address (VIP) for each Service Manager/ deployment pair is recommended. This way, the microservices can be accessed directly using the VIP.

With a reverse proxy, port numbers are not required to connect to the microservices because they are replaced with the deployment name and the host name's VIP. For example, to connect to the console via a web browser, use the URLs:

GoldenGate Services	URL
Service Manager	<code>https://localhost:localPort</code>
Administration Server	<code>https://localhost:localPort/instance_name/adminsrvr</code>
Distribution Server	<code>https://localhost:localPort/instance_name/distsrvr</code>
Performance Metric Server	<code>https://localhost:localPort/instance_name/pmsrvr</code>
Receiver Server	<code>https://localhost:localPort/instance_name/recvsrvr</code>

Note

To connect to Oracle GoldenGate in OCI, you must create a bastion (see Step 3.2) and an SSH port forwarding session (see Step 4.1). After this, you can connect to the Oracle GoldenGate Services using `https://localhost:localPort`.

A reverse proxy is mandatory to ensure easy access to microservices and enhance security and manageability.

When running multiple Service Managers, the following instructions will provide configuration using a separate VIP for each Service Manager. NGINX uses the VIP to determine which Service Manager an HTTPS connection request is routed to.

An SSL certificate is required for clients to authenticate the server they connect to through NGINX. Contact your systems administrator to follow your corporate standards to create or obtain the server certificate before proceeding. A separate certificate is required for each VIP and Service Manager pair.

Note

The common name in the CA-signed certificate must match the target hostname/VIP used by NGINX.

Follow the instructions to install and configure NGINX Reverse Proxy with an SSL connection and ensure all external communication is secure.

Step 3.6.1 - Secure Deployments Requirements (Certificates)

A secure deployment involves making RESTful API calls and conveying trail data between the Distribution Server and Receiver Server, over SSL/TLS. You can use your own existing business certificate from your Certificate Authority (CA) or you might create your own certificates. Contact your systems administrator to follow your corporate standards to create or obtain the server certificate before proceeding. A separate certificate is required for each VIP and Service Manager pair.

Step 3.6.2 - Install NGINX Reverse Proxy Server

As the root OS user on all GGHUB nodes, set up the yum repository by creating the file /etc/yum.repos.d/nginx.repo with the following contents:

```
[opc@gghub_prim1 ~]$ sudo su -
[root@gghub_prim1 ~]# cat > /etc/yum.repos.d/nginx.repo <<EOF
[nginx-stable]
name=nginx stable repo
baseurl=http://nginx.org/packages/rhel/7/\$basearch/
gpgcheck=1
enabled=1
gpgkey=https://nginx.org/keys/nginx_signing.key
module_hotfixes=true
EOF
```

As the root OS user on all GGHUB nodes, run the following commands to install, enable, and start NGINX:

```
[root@gghub_prim1 ~]# yum install -y python-requests python-urllib3 nginx
[root@gghub_prim1 ~]# systemctl enable nginx
```

As the root OS user on all GGHUB node, disable the NGINX repository after the software has been installed:

```
[root@gghub_prim1 ~]# yum-config-manager --disable nginx-stable
```

Step 3.6.3 - Create the NGINX Configuration File

You can configure Oracle GoldenGate Microservices Architecture to use a reverse proxy. Oracle GoldenGate MA includes a script called ReverseProxySettings that generates a configuration file for only the NGINX reverse proxy server.

The script requires the following parameters:

- The --user parameter should mirror the GoldenGate administrator account specified with the initial deployment creation.
- The GoldenGate administrator password will be prompted.
- The reverse proxy port number specified by the --port parameter should be the default HTTPS port number (443) unless you are running multiple GoldenGate Service Managers using the same --host. In this case, specify an HTTPS port number that does not conflict with previous Service Manager reverse proxy configurations. For example, if running two Service Managers using the same hostname/VIP, the first reverse proxy configuration is created with '--port 443 --host hostvip01', and the second is created with '--port 444 --host hostvip01'. If using separate hostnames/VIPs, the two Service Manager reverse proxy configurations would be created with '--port 443 --host hostvip01' and '--port 443 --host hostvip02'.
- Lastly, the HTTP port number (9100) should match the Service Manager port number specified during the deployment creation.

Repeat this step for each additional GoldenGate Service Manager.

As the oracle OS user on the first GGHUB node, use the following command to create the Oracle GoldenGate NGINX configuration file:

```
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghub_prim1 ~]$ export OGG_HOME=/u01/app/oracle/goldengate/gg21c
[oracle@gghub_prim1 ~]$ export PATH=$PATH:$OGG_HOME/bin
[oracle@gghub_prim1 ~]$ cd /u01/oracle/scripts
[oracle@gghub_prim1 ~]$ $OGG_HOME/lib/utl/reverseproxy/ReverseProxySettings
--user oggadmin --port 443 --output ogg_<gghub1>.conf http://localhost:9100
--host <VIP hostname>
Password: <oggadmin_password>
```

Step 3.6.4 - Modify NGINX Configuration Files

When multiple GoldenGate Service Managers are configured to use their IP/VIPs with the same HTTPS 443 port, some small changes are required to the NGINX reverse proxy configuration files generated in the previous step. With all Service Managers sharing the same port number, they are independently accessed using their VIP/IP specified by the --host parameter.

As the oracle OS user on the first GGHUB node, determine the deployment name managed by this Service Manager listed in the reverse proxy configuration file and change all occurrences of “_ServiceManager” by prepending the deployment name before the underscore:

```
[oracle@gghub_prim1 ~]$ cd /u01/oracle/scripts
[oracle@gghub_prim1 ~]$ grep "Upstream Servers" ogg_<gghub1>.conf

## Upstream Servers for Deployment 'gghub1'

[oracle@gghub_prim1 ~]$ sed -i 's/_ServiceManager/<gghub1>_ServiceManager/'
ogg_<gghub1>.conf
```

Step 3.6.5 - Install the Server Certificates for NGINX

As the root OS user on the first GGHUB node, copy the server certificates and key files in the /etc/nginx/ssl directory, owned by root with file permissions 400 (-r-----):

```
[opc@gghub_prim1 ~]$ sudo su -
[root@gghub_prim1 ~]# mkdir /etc/nginx/ssl
[root@gghub_prim1 ~]# cp <ssl_keys> /etc/nginx/ssl/.
[root@gghub_prim1 ~]# chmod 400 /etc/nginx/ssl
[root@gghub_prim1 ~]# ll /etc/nginx/ssl

-r----- 1 root root 2750 May 17 06:12 gghub1.chained.crt
-r----- 1 root root 1675 May 17 06:12 gghub1.key
```

As the oracle OS user on the first GGHUB node, set the correct file names for the certificate and key files for each reverse proxy configuration file:

```
[root@gghub_prim1 ~]$ vi /u01/oracle/scripts/ogg_<gghub1>.conf

# Before
ssl_certificate          /etc/nginx/ogg.pem;
```

```

        ssl_certificate_key    /etc/nginx/ogg.pem;

# After
        ssl_certificate        /etc/nginx/ssl/gghub1.chained.crt;
        ssl_certificate_key    /etc/nginx/ssl/gghub1.key;

```

When using CA-signed certificates, the certificate named with the `ssl_certificate` NGINX parameter must include the 1) CA signed, 2) intermediate, and 3) root certificates in a single file. The order is significant; otherwise, NGINX fails to start and displays the error message:

```

(SSL: error:0B080074:x509 certificate routines:
X509_check_private_key:key values mismatch)

```

The root and intermediate certificates can be downloaded from the CA-signed certificate provider.

As the root OS user on the first GGHUB node, generate the SSL certificate single file by using the following example command:

```

[root@gghub_prim1 ~]# cd /etc/nginx/ssl
[root@gghub_prim1 ~]# cat CA_signed_cert.crt
intermediate.crt root.crt > gghub1.chained.crt

```

The `ssl_certificate_key` file is generated when creating the Certificate Signing Request (CSR), which is required when requesting a CA-signed certificate.

Step 3.6.6 - Install the NGINX Configuration File

As the root OS user on the first GGHUB node, copy the deployment configuration file to `/etc/nginx/conf.d` directory and remove the default configuration file:

```

[root@gghub_prim1 ~]# cp /u01/oracle/scripts/ogg_<gghub1>.conf
/etc/nginx/conf.d
[root@gghub_prim1 ~]# rm /etc/nginx/conf.d/default.conf

```

As the root OS user on the first GGHUB node, validate the NGINX configuration file. If there are errors in the file, they will be reported with the following command:

```

[root@gghub_prim1 ~]# nginx -t

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

```

As the root OS user on the first GGHUB node, restart NGINX to load the new configuration:

```

[root@gghub_prim1 ~]# systemctl restart nginx

```

Step 3.6.7 - Test GoldenGate Microservices Connectivity

As the root OS user on the first GGHUB node, create a curl configuration file (`access.cfg`) that contains the deployment user name and password:

```

[root@gghub_prim1 ~]# vi access.cfg
user = "oggadmin:<password>"

```

```
[root@gghub_prim1 ~]# curl -svf
-K access.cfg https://<VIP hostname>:<port#>/services/v2/config/health
-XGET && echo -e "\n*** Success"
```

Sample output:

```
* About to connect() to gghub_prim_vip.frankfurt.goldengate.com port 443 (#0)
* Trying 10.40.0.75...
* Connected to gghub_prim_vip.frankfurt.goldengate.com (10.40.0.75) port 443
(#0)
* Initializing NSS with certpath: sql:/etc/pki/nssdb
* CAfile: /etc/pki/tls/certs/ca-bundle.crt
  Cpath: none
* skipping SSL peer certificate verification
* NSS: client certificate not found (nickname not specified)
* SSL connection using TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
* Server certificate:
*   subject: CN=gghub_prim_vip.frankfurt.goldengate.com,OU=Oracle MAA,
O=Oracle,L=Frankfurt,ST=Frankfurt,C=GE
*   start date: Jul 27 15:59:00 2023 GMT
*   expire date: Jul 26 15:59:00 2024 GMT
*   common name: gghub_prim_vip.frankfurt.goldengate.com
*   issuer: OID.2.5.29.19=CA:true,
CN=gghub_prim_vip.frankfurt.goldengate.com,OU=Oracle
MAA,O=Oracle,L=Frankfurt,C=EU
* Server auth using Basic with user 'oggadmin'
> GET /services/v2/config/health HTTP/1.1
> Authorization: Basic b2dnYWRTaW46V0VsY29tZTEyM19fXw==
> User-Agent: curl/7.29.0
> Host: gghub_prim_vip.frankfurt.goldengate.com
> Accept: */*
>
< HTTP/1.1 200 OK
< Server: nginx/1.24.0
< Date: Thu, 27 Jul 2023 16:25:26 GMT
< Content-Type: application/json
< Content-Length: 941
< Connection: keep-alive
< Set-Cookie:
ogg.sca.mS+pRfBERzqE+RTFZPPoVw=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJv
Z2cuc2NhIiwiaXhwIjozNjAwLCJ0eXAiOiJ4LVNDQS1BdXRob3JpemF0aW9uIiwic3ViIjoib2dnYW
RtaW4iLCJhdWQiOiJvZ2cuc2NhIiwiaWF0IjoxNjkwNDc1MTI2LCJob3N0Ijoiz2dodWJsYV92aXAubG9
uZG9uLmdvbGRlbmdhdGUuY29tIiwicm9sZSI6IlNlY3VyaXR5IiwiaXV0aFR5cGUiOiJCYXNpYyIsImNy
ZWQiOiJFd3VqV0h0dzlGWDNHai9FN1RYU3A1N1dVRjBheUd4OFpCUTdiZDlKOU9RPSIsInNlcnZlcklEI
joiZmFkNWVkn2MtZThlYi00YmE2LTg4Y2EtNmQxYjZjdiMGQ3IiwiaXV0aFR5cGUuZGVudElEIjoioTkyZm
E5NDUtZjA0NC00NzNhLTg0ZjktMTRjNTY0ZjNlODU3In0=.knACABXPmZE4BEyux7lZQ5GnrSCCh4x1zB
VBL
aX3Flo=; Domain=gghub_prim_vip.frankfurt.goldengate.com; Path=/; HttpOnly;
Secure;
```



```

SameSite=strict
< Set-Cookie:

ogg.csrf.ms+pRfBERzqE+RTFZPPoVw=1ae439e625798ee02f8f7498438f27c7bad036b270d6bf
c9
5aee60fcee111d35ea7e8dc5fb5d61a38d49cac51ca53ed9307f9cbe08fab812181cf163a743bf
c7;
Domain=gghub_prim_vip.frankfurt.goldengate.com; Path=/; Secure;
SameSite=strict
< Cache-Control: max-age=0, no-cache, no-store, must-revalidate
< Expires: 0
< Pragma: no-cache
< Content-Security-Policy: default-src 'self' 'unsafe-eval'
'unsafe-inline';img-src 'self' data:;frame-ancestors
https://gghub_prim_vip.frankfurt.goldengate.com;child-src
https://gghub_prim_vip.frankfurt.goldengate.com blob;;
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 1; mode=block
< X-OGG-Proxy-Version: v1
< Strict-Transport-Security: max-age=31536000 ; includeSubDomains
<
* Connection #0 to host gghub_prim_vip.frankfurt.goldengate.com left intact
{"$schema":"api:standardResponse","links":[{"rel":"canonical",
"href":"https://gghub_prim_vip.frankfurt.goldengate.com/services/v2/config/
health",
"mediaType":"application/json"}, {"rel":"self",
"href":"https://gghub_prim_vip.frankfurt.goldengate.com/services/v2/config/
health",
"mediaType":"application/json"}, {"rel":"describedby",
"href":"https://gghub_prim_vip.frankfurt.goldengate.com/services/
ServiceManager/v2/metadata-catalog/health",
"mediaType":"application/schema+json"}], "messages": [],
"response": {"$schema":"ogg:health", "deploymentName":"ServiceManager",
"serviceName":"ServiceManager", "started":"2023-07-27T15:39:41.867Z", "healthy":
true,
"criticalResources":
[{"deploymentName":"gghub1", "name":"adminsrvr", "type":"service",
"status":"running", "healthy":true},
{"deploymentName":"gghub1", "name":"distsrvr",
"type":"service", "status":"running", "healthy":true},
{"deploymentName":"gghub1",
"name":"recvsrvr", "type":"service", "status":"running", "healthy":true}]]}
*** Success

[root@gghub_prim1 ~]# rm access.cfg

```

Note

If the environment is using self-signed SSL certificates, add the flag `--insecure` to the curl command to avoid the error "NSS error -8172 (SEC_ERROR_UNTRUSTED_ISSUER)".

Step 3.6.8 - Remove NGINX default.conf Configuration File

As the root OS user on all GGHUB GGHUB, remove the default configuration file (default.conf) created in /etc/nginx/conf.d:

```
[opc@gghub_prim1 ~]$ sudo rm -f /etc/nginx/conf.d/default.conf
[opc@gghub_prim1 ~]$ sudo nginx -s reload
```

Step 3.6.9 - Distribute the GoldenGate NGINX Configuration Files

Once all the reverse proxy configuration files have been created for the GoldenGate Service Managers, they must be copied to the second GoldenGate Hub node.

As the opc OS user on the first GGHUB node, distribute the NGINX configuration files to all database nodes:

```
[opc@gghub_prim1 ~]$ sudo tar fczP /tmp/nginx_conf.tar /etc/nginx/conf.d/
/etc/nginx/ssl/
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ scp /tmp/nginx_conf.tar gghub_prim2:/tmp/.
```

As the opc OS user on the second GGHUB node, extract the NGINX configuration files and remove the default configuration file:

```
[opc@gghub_prim2 ~]$ sudo tar fxzP /tmp/nginx_conf.tar
[opc@gghub_prim2 ~]$ sudo rm /etc/nginx/conf.d/default.conf
```

As the opc OS user on the second GGHUB node, restart NGINX:

```
[opc@gghub_prim2 ~]$ sudo nginx -t
```

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

```
[root@gghub_prim2 ~]$ sudo systemctl restart nginx
```

Note

Repeat all the steps in section 3.6 for the primary and standby GGHUB systems.

Step 3.7 - Securing Oracle GoldenGate Microservices to Restrict Non-Secure Direct Access

After configuring the NGINX reverse proxy with an unsecured Oracle GoldenGate Microservices deployment, the microservices can continue accessing HTTP (non-secure) using the configured microservices port numbers. For example, the following non-secure URL could be used to access the Administration Server: `http://vip-name:9101`.

Oracle GoldenGate Microservices' default behavior for each server (Service Manager, adminserver, pmsrvr, distsrvr, and recsrvr) is to listen using a configured port number on all network interfaces. This is undesirable for more secure installations, where direct access using HTTP to the Microservices needs to be disabled and only permitted using NGINX HTTPS.

Use the following commands to alter the Service Manager and deployment services listener address to use only the localhost address. Access to the Oracle GoldenGate Microservices will only be permitted from the localhost, and any access outside of the localhost will only succeed using the NGINX HTTPS port.

Step 3.7.1 - Stop the Service Manager

As the grid OS user on the first GGHUB node, stop the GoldenGate deployment:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ agctl stop goldengate gghub1
[grid@gghub_prim1 ~]$ agctl status goldengate
```

Goldengate instance 'gghub1' is not running

Step 3.7.2 - Modify the Service Manager Listener Address

As the oracle OS user on the first GGHUB node, modify the listener address with the following commands. Use the correct port number for the Service Manager being altered:

```
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghub_prim1 ~]$ export OGG_HOME=/u01/app/oracle/goldengate/gg21c
[oracle@gghub_prim1 ~]$ export OGG_VAR_HOME=/mnt/acfs_gg1/deployments/
ggsm01/var
[oracle@gghub_prim1 ~]$ export OGG_ETC_HOME=/mnt/acfs_gg1/deployments/
ggsm01/etc
[oracle@gghub_prim1 ~]$ $OGG_HOME/bin/ServiceManager
--prop=/config/network/serviceListeningPort
--value='{ "port":9100,"address":"127.0.0.1"}' --type=array --persist --exit
```

Step 3.7.3 - Restart the Service Manager and Deployment

As the grid OS user on the first GGHUB node, restart the GoldenGate deployment:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ agctl start goldengate gghub1
[grid@gghub_prim1 ~]$ agctl status goldengate
```

Goldengate instance 'gghub1' is running on exadb-node1

Step 3.7.4 - Modify the GoldenGate Microservices listener address

As the oracle OS user on the first GGHUB node, modify all the GoldenGate microservices (adminsrvr, pmsrvr, distsrvr, recvsrvr) listening address to localhost for the deployments managed by the Service Manager using the following command:

```
[opc@gghub_prim1 ~]$ sudo chmod g+x /u01/oracle/scripts/secureServices.py
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghub_prim1 ~]$ /u01/oracle/scripts/secureServices.py http://
localhost:9100
--user oggadmin
```

Password for 'oggadmin': <oggadmin_password>

*** Securing deployment - gghub1

```
Current value of "/network/serviceListeningPort" for "gghub1/adminsrvr" is
9101
Setting new value and restarting service.
New value of "/network/serviceListeningPort" for "gghub1/adminsrvr" is
{
  "address": "127.0.0.1",
  "port": 9101
}.
Current value of "/network/serviceListeningPort" for "gghub1/distsrvr" is 9102
Setting new value and restarting service.
New value of "/network/serviceListeningPort" for "gghub1/distsrvr" is
{
  "address": "127.0.0.1",
  "port": 9102
}.
Current value of "/network/serviceListeningPort" for "gghub1/pmsrvr" is 9104
Setting new value and restarting service.
New value of "/network/serviceListeningPort" for "gghub1/pmsrvr" is
{
  "address": "127.0.0.1",
  "port": 9104
}.
Current value of "/network/serviceListeningPort" for "gghub1/recvsrvr" is 9103
Setting new value and restarting service.
New value of "/network/serviceListeningPort" for "gghub1/recvsrvr" is
{
  "address": "127.0.0.1",
  "port": 9103
}.
```

Note

To modify a single deployment (adminsrvr, pmsrvr, distsrvr, recvsrvr), add the flag `--deployment instance_name`

Step 3.8 - Create a Clusterware Resource to Manage NGINX

Oracle Clusterware needs to have control over starting the NGINX reverse proxy so that it can be started automatically before the GoldenGate deployments are started.

As the grid OS user on the first GGHUB node, use the following command to get the application VIP resource name required to create the NGINX resource with a dependency on the underlying network CRS resource:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ crsctl stat res -w "TYPE = app.appviptypex2.type" |grep
NAME
NAME=gghub_prim_vip
```

As the root OS user on the first GGHUB node, use the following command to create a Clusterware resource to manage NGINX. Replace the HOSTING_MEMBERS and CARDINALITY values to match your environment:

```
[opc@gghub_prim1 ~]$ sudo su -
[root@gghub_prim1 ~]# vi /u01/oracle/scripts/add_nginx.sh

# Run as ROOT
$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/bin/crsctl add resource
nginx
  -type generic_application
  -attr "ACL='owner:root:rw, pgrp:root:rw, other::r--, group:oinstall:r-x,
user:oracle:rw', EXECUTABLE_NAMES=nginx, START_PROGRAM='/bin/systemctl
start -f nginx', STOP_PROGRAM='/bin/systemctl stop
-f nginx', CHECK_PROGRAMS='/bin/systemctl status nginx'
, START_DEPENDENCIES='hard(<gghub_prim_vip>)
pullup(<gghub_prim_vip>)',
STOP_DEPENDENCIES='hard(intermediate:<gghub_prim_vip>)',
RESTART_ATTEMPTS=0, HOSTING_MEMBERS='<gghub_prim1>,<gghub_prim2>',
CARDINALITY=2"

[root@gghub_prim1 ~]# sh /u01/oracle/scripts/add_nginx.sh
```

The NGINX resource created in this example will run on the named database nodes simultaneously, specified by HOSTING_MEMBERS. This is recommended when multiple GoldenGate Service Manager deployments are configured and can independently move between database nodes.

Once the NGINX Clusterware resource is created, the GoldenGate XAG resources need to be altered so that NGINX must be started before the GoldenGate deployments are started.

As the root OS user on the first GGHUB node, modify the XAG resources using the following example commands.

Determine the current --file systems parameter:

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ agctl config goldengate gghub1 |grep -i "file system"
```

File System resources needed: acfs_primary

Modify the --file systems parameter:

```
[opc@gghub_prim1 ~]$ sudo su -
[root@gghub_prim1 ~]# /u01/app/grid/xag/bin/agctl modify goldengate gghub1
--filesystems acfs_primary,nginx
```

```
[opc@gghub_prim1 ~]$ sudo su - grid
[grid@gghub_prim1 ~]$ agctl config goldengate gghub1 |grep -i "File system"
```

File System resources needed: acfs_primary,nginx

Note

Repeat the above commands for each XAG GoldenGate registration relying on NGINX.

Repeat all the steps in section 3.8 for the primary and standby GGHUB systems.

Step 3.9 - Create an Oracle Net TNS Alias for Oracle GoldenGate Database Connections

To provide local database connections for the Oracle GoldenGate processes when switching between nodes, create a TNS alias on **all** nodes of the cluster where Oracle GoldenGate may be started. Create the TNS alias in the `tnsnames.ora` file in the `TNS_ADMIN` directory specified in the deployment creation.

If the source database is a multitenant database, two TNS alias entries are required, one for the container database (CDB) and one for the pluggable database (PDB) that is being replicated. For a target Multitenant database, the TNS alias connects the PDB to where replicated data is being applied. The pluggable database `SERVICE_NAME` should be set to the database service created in an earlier step (refer to Step 2.3: Create the Database Services in [Task 2: Prepare a Primary and Standby Base System for GGHUB](#)).

As the oracle OS user on any database node of the primary and the standby database systems, use `dbaascli` to find the database domain name and the SCAN name:

```
# Primary DB
[opc@exadb1_node1]$ sudo su - oracle
[oracle@exadb1_node1]$ source db_name.env
[oracle@exadb1_node1]$ dbaascli database getDetails --dbname <db_name>
|grep 'connectString'

"connectString" : "<primary_scan_name>:1521/<service_name>"

# Standby DB
[opc@exadb2_node1]$ sudo su - oracle
[oracle@exadb2_node1]$ source db_name.env
[oracle@exadb2_node1]$ dbaascli database getDetails --dbname <db_name>
|grep 'connectString'

"connectString" : "<standby_scan_name>:1521/<service_name>"
```

As the oracle OS user on all nodes of the primary and standby GGHUB, add the recommended parameters for Oracle GoldenGate in the `sqlnet.ora` file:

```
[opc@gghub_prim1]$ sudo su - oracle
[oracle@gghub_prim1]$ mkdir -p /u01/app/oracle/goldengate/network/admin
[oracle@gghub_prim1]$
cat > /u01/app/oracle/goldengate/network/admin/sqlnet.ora <<EOF
DEFAULT_SDU_SIZE = 2097152
EOF
```

As the oracle OS user on all nodes of the primary and standby GGHUB, follow the steps to create the TNS alias definitions:

```
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghub_prim1 ~]$

cat > /u01/app/oracle/goldengate/network/admin/tnsnames.ora <<EOF

# Source
<source_cbd_service_name>=
  (DESCRIPTION =
    (CONNECT_TIMEOUT=3)(RETRY_COUNT=2)(LOAD_BALANCE=off)(FAILOVER=on)
  (RECV_TIMEOUT=30)
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<primary_scan_name>)
      (PORT=1521)))
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<standby_scan_name>)
      (PORT=1521)))
    (CONNECT_DATA=(SERVICE_NAME =
<source_cbd_service_name>.goldengate.com)))

<source_pdb_service_name>=
  (DESCRIPTION =
    (CONNECT_TIMEOUT=3)(RETRY_COUNT=2)(LOAD_BALANCE=off)(FAILOVER=on)
  (RECV_TIMEOUT=30)
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<primary_scan_name>)(PORT=1521)))
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<standby_scan_name>)(PORT=1521)))
    (CONNECT_DATA=(SERVICE_NAME =
<source_pdb_service_name>.goldengate.com)))

# Target
<target_pdb_service_name>=
  (DESCRIPTION =
    (CONNECT_TIMEOUT=3)(RETRY_COUNT=2)(LOAD_BALANCE=off)(FAILOVER=on)
  (RECV_TIMEOUT=30)
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<primary_scan_name>)(PORT=1521)))
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<standby_scan_name>)(PORT=1521)))
    (CONNECT_DATA=(SERVICE_NAME =
<target_pdb_service_name>.goldengate.com)))

EOF

[oracle@gghub_prim1 ~]$ scp /u01/app/oracle/goldengate/network/admin/*.ora
gghub_prim2:/u01/app/oracle/goldengate/network/admin
```

Note

When the `tnsnames.ora` or `sqlnet.ora` (located in the `TNS_ADMIN` directory for the Oracle GoldenGate deployment) are modified, the deployment needs to be restarted to pick up the changes.

Task 4: Configure the Oracle GoldenGate Environment

Step 4.1 - Create Database Credentials

With the Oracle GoldenGate deployment created, use the Oracle GoldenGate Administration Service home page to create the database credentials using the above TNS alias names. See figure 4 below for an example of the database credential creation using the TNS alias.

From a client machine with access to the GGHUB, create a ssh tunnel to connect to the Oracle GoldenGate Administration Service:

```
$ ssh -N -L <local_port>:<vip>:443 -p 22 <gghub-node>
```

As the oggadmin user, create the database credentials:

1. Log in into the Administration Service: `https://localhost:<localPort>/<instance_name>/adminsrvr`.
2. Click **Configuration** under **Administration Service**.
3. Click the plus button to **Add Credentials** under the **Database** tab.
4. Add the required information for the source and target CDB and PDB as shown in the table:

Region	Container	Domain	Alias	User ID
Region 1	CDB	GoldenGate	Reg1_CDB	c##ggadmin@<tns_alias>
Region 1	PDB	GoldenGate	Reg1_PDB	ggadmin@<tns_alias>
Region 2	CDB	GoldenGate	Reg2_CDB	c##ggadmin@<tns_alias>
Region 2	PDB	GoldenGate	Reg2_PDB	ggadmin@<tns_alias>

Step 4.2 - Set Up Schema Supplemental Logging

1. Log in to the Oracle GoldenGate Administration Server.
2. Click **Configuration** under **Administration Service**.
3. Click the **Connect to database** button under **Actions** for the **Source Database (Reg_CDB)**.
4. Click the plus button (Add TRANDATA) to **Add TRANDATA** for the **Schema** or **Tables**.

Step 4.3 - Create the Autostart Profile

Create a new profile to automatically start the Extract and Replicat processes when the Oracle GoldenGate Administration Server is started. Then, restart if any Extract or Replicat processes are abandoned. With GoldenGate Microservices, auto start and restart is managed by Profiles.

Using the Oracle GoldenGate Administration Server GUI, create a new profile that can be assigned to each of the Oracle GoldenGate processes:

1. Log in to the **Administration Service** on the Source and Target GoldenGate.
2. Click on **Profile** under **Administration Service**.
3. Click the **plus (+)** sign next to Profiles on the Managed Process Settings home page.
4. Enter the details as follows:
 - Profile Name: Start_Default
 - Description: Default auto-start/restart profile
 - Default Profile: Yes
 - Auto Start: Yes
 - Auto Start Options
 - Startup Delay: 1 min
 - Auto Restart: Yes
 - Auto Restart Options
 - Max Retries: 5
 - Retry Delay: 30 sec
 - Retries Window: 30 min
 - Restart on Failure only: Yes
 - Disable Task After Retries Exhausted: Yes
5. Click **Submit**

Step 4.4 - Configure Oracle GoldenGate Processes

When creating Extract, Distribution Paths, and Replicat processes with Oracle GoldenGate Microservices Architecture, all files that need to be shared between the GGHub nodes are already shared with the deployment files stored on a shared file system.

Listed below are essential configuration details recommended for running Oracle GoldenGate Microservices on GGhub for Extract, Distribution Paths, and Replicat processes.

Perform the following sub-steps to complete this step:

- Step 4.4.1 - Extract Configuration
- Step 4.4.2 - Replicat Configuration
- Step 4.4.3 - Distribution Path Configuration
- Step 4.4.4 - Set up a Heartbeat Table for Monitoring Lag Times

The main goal is to prevent data divergence between GoldenGate replicas and their associated standby databases. This section focuses on configuring Extract so that GoldenGate Extract never gets ahead of the standby database which can result in data divergence.

GoldenGate Parameter	Description	Recommendations
TRANLOGOPTIONS HANDLEDLFAILOVER	This is mandatory setting for Data Guard configurations that have Oracle GoldenGate to ensure GoldenGate Extract never extract data that has not been received by standby database. The HANDLEDLFAILOVER stands for handle DATA LOSS for Data Guard failover. The following parameter must be added to the Extract process parameter file to avoid losing transactions and resulting in logical data inconsistencies after data loss Data Guard failover event. When the two primary tried to reconcile, this parameter ensures that all transactions can be reconciled since the new primary (old standby) is not further behind as expected. Prevents Extract from extracting redo data from the source database, and writing to the trail file data that has not yet been applied to the Oracle Data Guard standby database. If this parameter is not specified, after a data loss failover, it is possible to have data in the target database that is not present in the source database, leading to data divergence and logical data inconsistencies.	MANDATORY when the source database is configured with Data Guard in Max Availability or Max Performance mode.
TRANLOGOPTIONS FAILOVERTARGETDESTID n	For multiple standby configurations or cases when Data Guard Fast-Start failover is not enabled, set FAILOVERTARGETDESTID to standby demarcated by LOG_ARCHIVE_DEST to ensure GoldenGate Extract never extract data that has not been received by target standby database. To determine the correct value for FAILOVERTARGETDESTID, use the LOG_ARCHIVE_DEST_N parameter from the GoldenGate source database which is used for sending redo to the source standby database. For example, if LOG_ARCHIVE_DEST_2 points to the standby database, then use a value of 2. When not using Data Guard Fast Start Failover (FSFO) in the source database, this parameter Identifies which standby database the Extract process must remain behind, with regard to not extracting redo data that has not yet been applied to the Oracle Data Guard standby database.	MANDATORY when not using FSFO in the source database. To determine the correct value for FAILOVERTARGETDESTID, use the LOG_ARCHIVE_DEST_N parameter from the GoldenGate source database which is used for sending redo to the source standby database. For example, if LOG_ARCHIVE_DEST_2 points to the standby database, then use a value of 2.

GoldenGate Parameter	Description	Recommendations
TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_WARNING <i>value</i>	The amount of time before a warning message is written to the Extract report file, if Extract is stalled, due to being unable to query the source database standby apply progress. This can occur after a Data Guard failover when the old primary database is not currently available. The default is 60 seconds.	OPTIONAL if want to adjust the timing of when the warning message is written to the Extract report file. Add STANDBY_WARNING <i>value</i> to the TRANLOGOPTIONS HANDLEDLFAILOVER parameter.
TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_ABEND <i>value</i>	The amount of time before Extract abends, if Extract is stalled, due to being unable to query the standby apply progress. The default is 30 minutes.	OPTIONAL if want to adjust the amount of time it takes Extract to abend, when the source database standby is not accessible to enforce the HANDLEDLFAILOVER parameter. Add STANDBY_ABEND <i>value</i> to the TRANLOGOPTIONS HANDLEDLFAILOVER parameter.
TRANLOGOPTIONS DLFAILOVER_TIMEOUT <i>value</i>	The amount of time Extract will run on the new source primary database, after a Data Guard role transition, before it will check the status of the standby database. If standby database is not available after the DLFAILOVER_TIMEOUT, Extract will abend. The default is 300 seconds. NOTE: If during normal operations of the source Oracle Data Guard configuration, the standby database becomes unavailable, Extract will stop extracting data from the source database to prevent possible data divergence with the GoldenGate target database due to the HANDLEDLFAILOVER parameter. The DLFAILOVER_TIMEOUT parameter does not take effect when a Data Guard failover has not occurred, and there are no messages output to the Extract report file.	OPTIONAL. if you want to adjust the amount of time an Extract can run on a new primary source database, after a role transition, when the standby is not yet available to honor the TRANLOGOPTIONS HANDLEDLFAILOVER parameter.

Refer to the [Oracle GoldenGate Parameters and Functions Reference Guide](#) for more information about the Extract TRANLOGOPTIONS parameters.

When creating an Extract using the Oracle GoldenGate Administration Service GUI interface, leave the Trail SubDirectory parameter blank so that the trail files are automatically created in the deployment directories stored on the shared file system. The default location for trail files is the /<deployment directory>/var/lib/data directory.

Note

To capture from a multitenant database, you must use an Extract configured at the root level using a c## account. To apply data into a multitenant database, a separate Replicat is needed for each PDB because a Replicat connects at the PDB level and doesn't have access to objects outside of that PDB.

Step 4.4.1 - Extract Configuration

Create the Extract:

1. Log in to the Oracle GoldenGate Administration Server.
2. Click **Overview** under **Administration Service**.
3. Click the **plus** button to **Add Extract**.
4. Select **Integrated Extract**.
5. Add the required information as follows:
 - Process Name: EXT_1
 - Description: Extract for Region 1 CDB
 - Intent: Unidirection
 - Begin: Now
 - Trail Name: aa
 - Credential Domain: GoldenGate
 - Credential Alias: Reg1_CDB
 - Register to PDBs: PDB Name
6. Click **Next** and set parameters.

```
EXTRACT ext_1
USERIDALIAS Reg1_CDB DOMAIN GoldenGate
EXTTRAIL aaTRANLOGOPTIONS HANDLEDLFAILOVER
TRANLOGOPTIONS FAILOVERTARGETDESTID 2
SOURCECATALOG PDB_NAME
TABLE OWNER.*;
```

7. Click **Next**.
8. If using CDB Root Capture from PDB, add the SOURCECATALOG parameter with the PDB Name.
9. Click **Create and Run**.

Note

For ADB-D deployments, the extract requires a connection to the PDB rather than the CDB.

See [Oracle GoldenGate Extract Failure or Error Conditions Considerations](#) for more information.

Step 4.4.2 - Replicat Configuration

Oracle generally recommends using integrated parallel Replicat which offers better apply performance for most workloads when the GGHUB is in the same region as the target Oracle GoldenGate database.

The best apply performance can be achieved when the network latency between the GGHUB and the target database is as low as possible. The following configuration is recommended for the remote Replicat running on the Oracle GGHUB.

- **APPLY_PARALLELISM** – Disables automatic parallelism, instead of using **MAX_APPLY_PARALLELISM** and **MIN_APPLY_PARALLELISM**, and allows the highest amount of concurrency to the target database. It is recommended to set this as high as possible based on available CPU of the hub and the target database server.
- **MAP_PARALLELISM** – Should be set with a value of 2 to 5. With a larger number of appliers, increasing the Mappers increases the ability to hand work to the appliers.
- **BATCHSQL** – applies DML using array processing which reduces the amount network overheads with a higher latency network. Be aware that if there are many data conflicts, **BATCHSQL** results in reduced performance, as rollback of the batch operations followed by a re-read from trail file to apply in non-batch mode.

Add a Replicat:

After you've set up your database connections and verified them, you can add a Replicat for the deployment by following these steps:

1. Log in to the Oracle GoldenGate Administration Server.
2. Click the plus (+) sign next to **Replicats** on the Administration Service home page. The Add Replicat page is displayed.
3. Select a Replicat type and click **Next**.
4. Enter the details as follows:
 - Process Name: REP_1
 - Description: Replicat for Region 2 PDB
 - Intent: Unidirectional
 - Credential Domain: GoldenGate
 - Credential Alias: Reg2_PDB
 - Source: Trail
 - Trail Name: aa
 - Begin: Position in Log
 - Checkpoint Table: "GGADMIN"."CHKP_TABLE"
5. Click **Next**.
6. From the **Action Menu**, click **Details** to edit the Replicat **Parameters**:

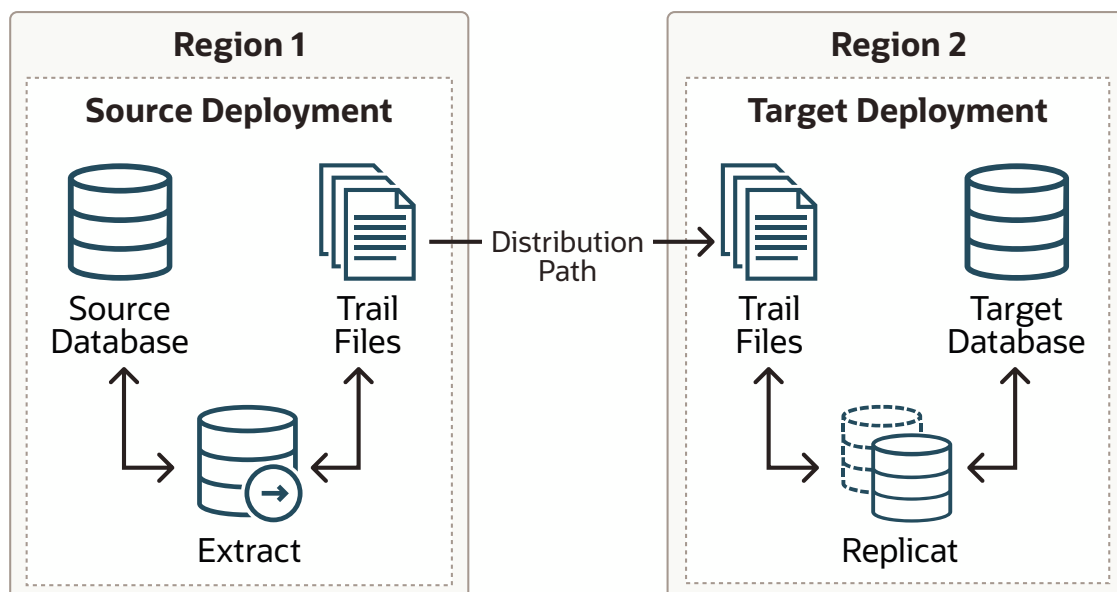
```
REPLICAT REP_1
USERIDALIAS Reg2_PDB DOMAIN GoldenGate
MAP <SOURCE_PDB_NAME>.<OWNER>.*, TARGET <OWNER>.*;
```

7. From the **Action Menu**, click **Start**.

Step 4.4.3 - Distribution Path Configuration

Distribution paths are only necessary when trail files need to be sent to an additional Oracle GoldenGate Hub in a different, or even the same, region as described in the following figure.

Figure 23-3 Oracle GoldenGate Distribution Path



When using Oracle GoldenGate Distribution paths with the NGINX Reverse Proxy, additional steps must be carried out to ensure the path client and server certificates are configured.

More instructions about creating distribution paths are available in [Add a Distribution Path](#). A step-by-step example is in the following video, "[Connect an on-premises Oracle GoldenGate to OCI GoldenGate using NGINX](#)," to correctly configure the certificates.

Here are the steps performed in this sub-step:

- Step 4.4.3.1 - Download the Target Server's Root Certificate, and then upload it to the source Oracle GoldenGate
- Step 4.4.3.2 - Create a user in the Target Deployment for the Source Oracle GoldenGate to use
- Step 4.4.3.3 - Create a Credential in the Source Oracle GoldenGate
- Step 4.4.3.4 - Create a Distribution Path on the Source Oracle GoldenGate to the Target Deployment

Step 4.4.3.1 - Download the Target Server's Root Certificate, and then upload it to the source Oracle GoldenGate

Download the target deployment server's root certificate and add the CA certificate to the source deployment Service Manager.

1. Log in to the **Administration Service** on the Target GoldenGate.
2. Follow "Step 2 - Download the target server's root certificate" in the video "[Connect an on-premises Oracle GoldenGate to OCI GoldenGate using NGINX](#)."

Step 4.4.3.2 - Create a user in the Target Deployment for the Source Oracle GoldenGate to use

Create a user in the target deployment for the distribution path to connect to:

1. Log in to the **Administration Service** on the Target GoldenGate.
2. Click on Administrator under Administration Service.

3. Click the plus (+) sign next to Users.
4. Enter the details as follows:
 - Username: ggnet
 - Role: Operator
 - Type: Password

5. Click **Submit**

Step 4.4.3.3 - Create a Credential in the Source Oracle GoldenGate

Create a credential in the source deployment connecting the target deployment with the user created in the previous step. For example, a domain of OP2C and an alias of WSSNET.

1. Log in to the **Administration Service** on the Source Oracle GoldenGate.
2. Click in **Configuration** under **Administration Service**.
3. Click the **plus (+)** sign next to Credentials on the Database home page.
4. Enter the details as follows:
 - Credential Domain: OP2C
 - Credential Alias: wssnet
 - User ID: ggnet
5. Click **Submit**

Step 4.4.3.4 - Create a Distribution Path on the Source Oracle GoldenGate to the Target Deployment

A path is created to send trail files from the Distribution Server to the Receiver Server. You can create a path from the Distribution Service. To add a path for the source deployment:

1. Log in to the **Distribution Service** on the Source Oracle GoldenGate.
2. Click the plus (+) sign next to Path on the Distribution Service home page. The Add Path page is displayed.
3. Enter the details as follows:

Option	Description
Path Name	Select a name for the path.
Source: <i>Trail Name</i>	Select the Extract name from the drop-down list, which populates the trail name automatically. If it doesn't, enter the trail name you provided while adding the Extract.
Generated Source URI	Specify localhost for the server's name; this allows the distribution path to be started on any of the Oracle RAC nodes.
Target Authentication Method	Use 'UserID Alias'
Target	Set the Target transfer protocol to wss (secure web socket). Set the Target Host to the target hostname/VIP that will be used for connecting to the target system along with the Port Number that NGINX was configured with (default is 443).
Domain	Set the Domain to the credential domain created above in Step 11.3.3, for example, OP2C.

Option	Description
Alias	The Alias is set to the credential alias wssnet, also created in Step 11.3.3.
Auto Restart Options	Set the distribution path to restart when the Distribution Server starts automatically. This is required, so that manual intervention is not required after a RAC node relocation of the Distribution Server. It is recommended to set the number of Retries to 10. Set the Delay , which is the time in minutes to pause between restart attempts, to 1.

4. Click **Create Path**.
5. From the Action Menu, click **Start**.

Step 4.4.4 - Set up a Heartbeat Table to Monitor Lag Times

Follow [Steps to add Heartbeat Table in OCI GoldenGate](#) to implement the best practices for creating a heartbeat process that can be used to determine where and when lag are developing between a source and target system.

This document walks you through the step-by-step process of creating the necessary tables and added table mapping statements needed to keep track of processing times between a source and target database. Once the information is added into the data flow, the information is then stored in a target table that can be analyzed to determine when and where the lag is being introduced between the source and target systems.

Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices

Use these best practices for configuring Oracle GoldenGate Microservices Architecture to work with Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D) or Oracle Exadata Database Service on Cloud@Customer (ExaDB-C@C), and with Oracle Database File System (DBFS) or Oracle ASM Cluster File System (ACFS).

See the following topics:

- [Overview of Oracle GoldenGate Microservices Architecture Configuration on Oracle Exadata Database Service](#)
- [Task 1 - Before You Begin](#)
- [Task 2 - Configure the Oracle Database for GoldenGate](#)
- [Task 3 - Create a Shared File System to Store the Oracle GoldenGate Deployment](#)
- [Task 4 - Install Oracle GoldenGate](#)
- [Task 5 - Create the Oracle GoldenGate Deployment](#)
- [Task 6 - Configure the Network](#)
- [Task 7 - Configure Oracle Grid Infrastructure Agent](#)
- [Task 8 - Configure NGINX Reverse Proxy](#)
- [Task 9 - Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections](#)
- [Task 10 - Create a New Profile](#)
- [Task 11 - Configure Oracle GoldenGate Processes](#)
- [Troubleshooting Oracle GoldenGate on Oracle RAC](#)
- [Example Configuration Problems](#)

Overview of Oracle GoldenGate Microservices Architecture Configuration on Oracle Exadata Database Service

The target Oracle Exadata Database Service that hosts Oracle GoldenGate Microservices Architecture can act as the source database, the target database, or in some cases, as both source and target databases for Oracle GoldenGate. These best practices are applicable for configuring Oracle GoldenGate Microservices Architecture with Oracle Exadata Database Service on Dedicated Infrastructure or Cloud@Customer.

Follow this roadmap to configure Oracle GoldenGate on Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D) or Oracle Exadata Database Service on Cloud@Customer.

- [Task 1 - Before You Begin](#): To configure Oracle GoldenGate on Oracle Exadata Cloud Infrastructure or Cloud@Customer, you need an ExaDB-D or ExaDB-C@C system, CA certificates, and configure some extra software.
- [Task 2 - Configure the Oracle Database for GoldenGate](#): Use best practices to configure the source and target databases in an Oracle GoldenGate replicated environment.
- [Task 3 - Create a Shared File System to Store the Oracle GoldenGate Deployment](#): Set up either Oracle DBFS or Oracle ACFS for configuring HA on Oracle Cloud Infrastructure with Oracle GoldenGate. If your architecture has a GoldenGate replica database protected by a cloud physical standby database (Oracle Data Guard), use Oracle DBFS; otherwise use ACFS.
- [Task 4 - Install Oracle GoldenGate](#): Use best practices to install and configure Oracle GoldenGate components on Oracle Cloud Infrastructure.
- [Task 5 - Create the Oracle GoldenGate Deployment](#): Create a response file to create the GoldenGate deployment using the Oracle GoldenGate Configuration Assistant.
- [Task 6 - Configure the Network](#): Configure virtual cloud network (VCN) components such as private DNS zones, VIP, bastion, security lists and firewalls for Oracle GoldenGate to function properly.
- [Task 7 - Configure Oracle Grid Infrastructure Agent](#): Configure Oracle GoldenGate for HA on Oracle Cloud Infrastructure.
- [Task 8 - Configure NGINX Reverse Proxy](#): Configure reverse proxy and HA by using Nginx.
- [Task 9 - Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections](#): Create a TNS alias to simplify database connectivity for the Oracle GoldenGate processes when switching between Oracle RAC nodes.
- [Task 10 - Create a New Profile](#): Create a new profile to automatically start the Extract and Replicat processes when the Oracle GoldenGate Administration Server is started.
- [Task 11 - Configure Oracle GoldenGate Processes](#): Create and configure Oracle GoldenGate Extract, Replicat, and Path processes need for data replication

Task 1 - Before You Begin

Perform the following steps to complete this task:

- Step 1.1 - Set Up the Oracle Cloud Infrastructure DB System
- Step 1.2 - Download the Required Software
- Step 1.3 - Configure Your System to Install Software from Oracle Linux Yum Server
- Step 1.4 - Secure Deployments Requirements (Certificates)

Step 1.1 - Set Up the Oracle Cloud Infrastructure DB System

To get started, you need an Oracle Exadata Database Service on Dedicated Infrastructure or Cloud@Customer for Oracle GoldenGate deployment.

You can deploy Oracle GoldenGate with an existing ExaDB-D/ExaDB-C@C system or launch a new system, according to your business needs.

For instructions on launching and managing an ExaDB-D system, see [Oracle Exadata Database Service on Dedicated Infrastructure](#) or for ExaDB-C@C see [Oracle Exadata Database Service on Cloud@Customer](#).

Step 1.2 - Download the Required Software

1. Create the staging directory to download all the required software.

```
[opc@exadb-node1 ~]$ sudo su -
[root@exadb-node1 ~]# mkdir /u02/app_acfs/goldengate
[root@exadb-node1 ~]# chown oracle:oinstall /u02/app_acfs/goldengate
[root@exadb-node1 ~]# chmod g+w /u02/app_acfs/goldengate
```

2. Download subsequent patches to the base release, go to the Patches and Updates tab of [My Oracle Support](#).
 - See [Installing Patches for Oracle GoldenGate](#) Microservices Architecture for more information.
 - The minimum required version is Patch 35214851: Oracle GoldenGate 21.9.0.0.2 Microservices for Oracle
3. Download the latest OPatch release, Patch 6880880, for Oracle Database 21c (21.0.0.0.0) from My Oracle Support Document [2542082.1](#).
4. Download the Oracle GoldenGate 21c Microservices software, or higher, from [Oracle GoldenGate Downloads](#).
5. Download the Oracle Grid Infrastructure Standalone Agents for Oracle Clusterware 19c, version 10.2 or higher, from [Oracle Grid Infrastructure Standalone Agents for Oracle Clusterware](#).
6. Download the mount-dbfs-*version*.zip file with mount-dbfs.sh and mount-dbfs.conf from My Oracle Support [Document 1054431.1](#).
7. Download the python script (secureServices.py) from My Oracle Support [Document 2826001.1](#).

Step 1.3 - Configure Your System to Install Software from Oracle Linux Yum Server

Oracle Linux yum server hosts software for Oracle Linux and compatible distributions. These instructions help you get started configuring your Linux system for Oracle Linux yum server and installing software via yum.

1. As the root OS user, create the file /etc/yum.repos.d/oracle-public-yum-ol7.repo with the following contents:

```
[opc@exadb-node1 ~]$ sudo su -
[root@exadb-node1 ~]#
cat > /etc/yum.repos.d/oracle-public-yum-ol7.repo <<EOF
[ol7_latest]
name=Oracle Linux $releasever Latest ($basearch)
baseurl=http://yum$ociregion.oracle.com/repo/OracleLinux/OL7/latest/
$basearch/
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-oracle
gpgcheck=1
enabled=1
EOF
```

2. As the root OS user, follow [Doc ID 2397264.1](#) to modify the configuration file /etc/yum.conf and validate the software repositories are enabled:

```
[root@exadb-node1 ~]# yum repolist
repo id                repo name                status
```

```
!public_ol7_latest Oracle Linux 7.9-6.0.1.el7_9 Latest
(x86_64) 19,712+4,957
repolist: 19,992
```

Step 1.4 - Secure Deployments Requirements (Certificates)

A secure deployment involves making RESTful API calls and conveying trail data between the Distribution Server and Receiver Server, over SSL or TLS.

You can use your own existing business certificate from your Certificate Authority (CA) or you might create your own certificates.

Contact your systems administrator to follow your corporate standards to create or obtain the server certificate before proceeding. A separate certificate is required for each VIP and Service Manager pair.

Task 2 - Configure the Oracle Database for GoldenGate

The source and target Oracle GoldenGate databases should be configured using the following recommendations.

Perform the following steps to complete this task:

- Step 2.1 - Database Configuration
- Step 2.2 - Create the Database Replication Administrator User
- Step 2.3 - Create the Database Services

Step 2.1 - Database Configuration

The source and target Oracle GoldenGate databases should be configured using the following recommendations.

1. Enable Oracle GoldenGate replication by setting the database initialization parameter.
2. Source Oracle GoldenGate Database:
 - Run the database in ARCHIVELOG mode
 - Enable FORCE LOGGING mode
 - Enable minimal supplemental logging
 - Additionally, add schema or table level logging for all replicated objects
3. Configure the streams pool in the System Global Area (SGA) on the source database using the STREAMS_POOL_SIZE initialization parameter. The streams pool is only needed on the target database if integrated Replicat will be used.

For the steps on preparing the database for Oracle GoldenGate, refer to [Using Oracle GoldenGate Classic Architecture with Oracle Database](#).

1. As the oracle OS user on the source and target systems, issue the following SQL instructions to configure the database:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ source <db_name>.env
[oracle@exadb-node1 ~]$ sqlplus / as sysdba
SQL> alter system set ENABLE_GOLDENGATE_REPLICATION=true scope=both
sid='*';
SQL> alter system set STREAMS_POOL_SIZE=<SIZE_IN_GB> scope=both sid='*';
```

2. As the oracle OS user on the source system, issue the following SQL instructions to configure the database:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ source <db_name>.env
[oracle@exadb-node1 ~]$ sqlplus / as sysdba
SQL> ALTER DATABASE FORCE LOGGING;
SQL> ALTER DATABASE ADD SUPPLEMENTAL LOG DATA;
SQL> ARCHIVE LOG LIST
Database log mode          Archive Mode
Automatic archival         Enabled
Archive destination        USE_DB_RECOVERY_FILE_DEST
Oldest online log sequence 110
Next log sequence to archive 113
Current log sequence        113
```

Step 2.2 - Create the Database Replication Administrator User

The source and target Oracle databases need a GoldenGate Administrator user created, with appropriate privileges assigned:

- For multitenant container database (CDB):
 - Source database, GoldenGate Extract must be configured to connect to a user in the root container database, using a c##
 - Target database, a separate GoldenGate administrator user is needed for each pluggable database (PDB). For details about creating a GoldenGate Administrator in an Oracle Multitenant Database, see [Configuring Oracle GoldenGate in a Multitenant Container Database](#).
 - For non-CDB databases, see [Establishing Oracle GoldenGate Credentials](#).
1. As the oracle OS user on the source system, issue the following SQL instructions to create the database user for Oracle GoldenGate and assign the required privileges:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ source <db_name>.env
[oracle@exadb-node1 ~]$ sqlplus / as sysdba

# CDB
alter session set container=cdb$root;
create user c##ggadmin identified by "<ggadmin_password>" container=all
default
tablespace USERS temporary tablespace temp;
alter user c##ggadmin quota unlimited on users;
grant set container to c##ggadmin container=all;
grant alter system to c##ggadmin container=all;
grant create session to c##ggadmin container=all;
grant alter any table to c##ggadmin container=all;
grant resource to c##ggadmin container=all;
exec
dbms_goldengate_auth.grant_admin_privilege('c##ggadmin',container=>'all');

# Source PDB
alter session set container=<PDB_name>;
create user ggadmin identified by "<ggadmin_password>" container=current;
grant create session to ggadmin container=current;
```

```
grant alter any table to ggadmin container=current;
grant resource to ggadmin container=current;
exec dbms_goldengate_auth.grant_admin_privilege('ggadmin');
```

2. As the oracle OS user on the target system, issue the following SQL instructions to create the database user for Oracle GoldenGate and assign the required privileges:

```
# Target PDB
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ source <db_name>.env
[oracle@exadb-node1 ~]$ sqlplus / as sysdba
alter session set container=<PDB_name>;
create user ggadmin identified by "<ggadmin_password>" container=current;
grant alter system to ggadmin container=current;
grant create session to ggadmin container=current;
grant alter any table to ggadmin container=current;
grant resource to ggadmin container=current;
grant dv_goldengate_admin, dv_goldengate_redo_access to ggadmin
container=current;
exec dbms_goldengate_auth.grant_admin_privilege('ggadmin');
```

Step 2.3 - Create the Database Services

A database service is required so that the Oracle Grid Infrastructure Agent will automatically start the Oracle GoldenGate deployment when the database is opened. When DBFS is used for the shared file system, the database service is also used to mount DBFS to the correct RAC instance.

When using a source multitenant database, a separate service is required for the root container database (CDB) and the pluggable database (PDB) that contains the schema being replicated. For a target multitenant database, a single service is required for the PDB.

1. As the oracle OS user, create and start the CDB database service using the following command:

```
[oracle@exadb-node1 ~]$ source <db_name>.env
[oracle@exadb-node1 ~]$ srvctl add service -db $ORACLE_UNQNAME
-svcname `echo $ORACLE_UNQNAME`_ogg -preferred <SID1> -available <SID2>
-role PRIMARY
[oracle@exadb-node1 ~]$ srvctl start service -db $ORACLE_UNQNAME
-svcname `echo $ORACLE_UNQNAME`_ogg
```

If your database is part of a multitenant environment, remember to create the service at the pluggable database (PDB).

2. As the oracle OS user, create and start the PDB database service using the following command:

```
[oracle@exadb-node1 ~]$ dbaascli database getDetails
--dbname <db_name> |grep pdbName
      "pdbName" : "<PDB_NAME>",
[oracle@exadb-node1 ~]$ srvctl add service -db $ORACLE_UNQNAME
-svcname <PDB_NAME>_ogg -preferred <SID1>,<SID2> -pdb <PDB_NAME> -role
PRIMARY
[oracle@exadb-node1 ~]$ srvctl start service -db $ORACLE_UNQNAME
-svcname <PDB_NAME>_ogg
```

3. As the oracle OS user, verify that the services are running:

```
[oracle@exadb-node1 ~]$ srvctl status service -d $ORACLE_UNQNAME |grep _ogg  
Service <ORACLE_UNQNAME>_ogg is running on instance(s) <SID1>  
Service <PDB_NAME>_ogg is running on instance(s) <SID1>
```

See Server Control Utility Reference in *Oracle Real Application Clusters Administration and Deployment Guide* for details about creating database services.

Task 3 - Create a Shared File System to Store the Oracle GoldenGate Deployment

Oracle GoldenGate Microservices Architecture is designed with a simplified installation and deployment directory structure.

- **The installation directory** should be placed on local storage on each database node to minimize downtime during software patching.
- **The deployment directory** which is created during deployment creation using the Oracle GoldenGate Configuration Assistant (oggca.sh), must be placed on a shared file system. The deployment directory contains configuration, security, log, parameter, trail, and checkpoint files.

Placing the deployment in DBFS or Oracle Automatic Storage Management Cluster File System (ACFS) provides the best recoverability and failover capabilities in the event of a system failure. Ensuring the availability of the checkpoint files cluster-wide is essential so that the GoldenGate processes can continue running from their last known position after a failure occurs.

If Oracle GoldenGate will be configured along with Oracle Data Guard, the recommended file system is DBFS. DBFS is contained in the database protected by Data Guard and can be fully integrated with XAG. In the event of a Data Guard role transition, the file system can be automatically mounted on the new primary server, followed by the automated start-up Oracle GoldenGate. This is currently not possible with ACFS since it is not part of the Oracle Data Guard configuration.

Note

This document does not include steps to configure Oracle GoldenGate with Oracle Data Guard.

If Oracle Data Guard is not present, the recommended file system is ACFS. ACFS is a multi-platform, scalable file system and storage management technology that extends Oracle Automatic Storage Management (Oracle ASM) functionality to support customer files maintained outside the Oracle Database.

Perform one of the following steps to complete this task, based on your file system requirements:

- Step 3a - Oracle Database File System (DBFS)
- Step 3b - Oracle ASM Cluster File System (ACFS)

Step 3a - Oracle Database File System (DBFS)

You must create the DBFS tablespace inside the same database to which the Oracle GoldenGate processes are connected. For example, if an Oracle GoldenGate integrated Extract process is extracted from a database called GGDB, the DBFS tablespace would be located in the same GGDB database.

Create a file system for storing the Oracle GoldenGate deployment files. You should allocate enough trail file disk space to permit storage of up to 12 hours of trail files. Doing this will give sufficient space for trail file generation should a problem occur with the target environment that prevents it from receiving new trail files. The amount of space needed for 12 hours can only be determined by testing trail file generation rates with real production data.

Perform the following sub-steps to complete this step:

- Step 3a.1 - Configuring DBFS on Oracle Exadata Database Service
- Step 3a.2 - Create the DBFS Repository
- Step 3a.3 - (Only for CDB) Create an Entry in TNSNAMES
- Step 3a.4 - Download and Edit the mount-dbfs Scripts
- Step 3a.5 - Register the DBFS Resource with Oracle Clusterware
- Step 3a.6 - Start the DBFS Resource

Step 3a.1 - Configuring DBFS on Oracle Exadata Database Service

1. As the opc OS user, add the grid user to the fuse group:

```
[opc@exadb-node1]$ sudo -u grid $(grep ^crs_home /etc/oracle/olr.loc | cut
-d= -f2)/bin/olsnodes > ~/dbs_group
[opc@exadb-node1]$ dcli -g ~/dbs_group -l opc sudo usermod -a -G fuse grid
```

2. As the opc OS user, validate that the file /etc/fuse.conf exists and contains the user_allow_other option:

```
[opc@exadb-node1]$ cat /etc/fuse.conf
# mount_max = 1000
# user_allow_other
```

3. Skip this step if the option user_allow_other is already in the /etc/fuse.conf file. Otherwise, run the following commands as the opc OS user to add the option:

```
[opc@exadb-node1]$ dcli -g ~/dbs_group -l opc "echo user_allow_other |
sudo tee -a /etc/fuse.conf"
```

4. As the opc OS user, create an empty directory that will be used as the mount point for the DBFS file system:

```
[opc@exadb-node1]$ dcli -g ~/dbs_group -l opc sudo mkdir -p /mnt/dbfs
```

5. As the opc OS user, change ownership on the mount point directory so the grid OS user can access it:

```
[opc@exadb-node1]$ dcli -g ~/dbs_group -l opc sudo chown
oracle:oinstall /mnt/dbfs
```

Step 3a.2 - Create the DBFS Repository

Create the DBFS repository inside the target database. To create the repository, create a new tablespace within the target PDB to hold the DBFS objects and a database user that will own the objects.

① Note

When using an Oracle Multitenant Database, the DBFS tablespace **MUST** be created in a Pluggable Database (PDB). It is recommended that you use the same PDB that the GoldenGate Extract or Replicat processes connect to, allowing DBFS to use the same database service created above for its database dependency.

1. As the oracle OS user, create the tablespace in the database:

```
[opc@exadb-node1]$ sudo su - oracle
[oracle@exadb-node1]$ source DB_NAME.env
[oracle@exadb-node1]$ sqlplus / as sysdba
SQL> alter session set container=<pdb_name>;
SQL> create bigfile tablespace dbfstb1 datafile size 32g autoextend on
next 8g
maxsize 300g NOLOGGING EXTENT MANAGEMENT LOCAL AUTOALLOCATE SEGMENT SPACE
MANAGEMENT AUTO;
SQL> create user dbfs_user identified by "<dbfs_user_password>"
default tablespace dbfstb1 quota unlimited on dbfstb1;
SQL> grant connect, create table, create view, create procedure,
dbfs_role to dbfs_user;
```

2. As the oracle OS user, create the database objects that will hold DBFS. This script takes two arguments:

- dbfstb1: tablespace for the DBFS database objects
- goldengate: file system name - this can be any string and will appear as a directory under the mount point

```
[oracle@exadb-node1]$ sqlplus
dbfs_user/"<dbfs_user_password>"@<db_name>_dbfs
SQL> start $ORACLE_HOME/rdbms/admin/dbfs_create_filesystem dbfstb1
goldengate
```

Step 3a.3 - (Only for CDB) Create an Entry in TNSNAMES

1. As the oracle OS user, find the database domain name:

```
[opc@exadb-node1]$ sudo su - oracle
[oracle@exadb-node1]$ source DB_NAME.env
[oracle@exadb-node1]$ sqlplus / as sysdba
SQL> show parameter db_domain
```

NAME	TYPE	VALUE
db_domain	string	<db_domain_name>

2. As the oracle OS user, add a connect entry in \$TNS_ADMIN/tnsnames.ora file:

```
[oracle@exadb-node1]$ vi $TNS_ADMIN/tnsnames.ora
dbfs =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = IPC)(KEY=LISTENER))
    (CONNECT_DATA =
      (SERVICE_NAME = <pdb_service_name>.<db_domain_name> )
    )
  )
```

3. As the oracle OS user, distribute the \$TNS_ADMIN/tnsnames.ora file to the rest of the nodes:

```
[oracle@exadb-node1 ~]$ /usr/local/bin/dcli -l oracle -g ~/dbs_group
-f $TNS_ADMIN/tnsnames.ora -d $TNS_ADMIN/
```

Step 3a.4 - Edit the mount-dbfs Scripts

1. Unzip the zip file and edit the variable settings in the file mount-dbfs.conf for your environment.

Comments in the file will help you to confirm the values for these variables:

- DBNAME: echo \$ORACLE_UNQNAME
- MOUNT_POINT: /mnt/dbfs/goldengate
- ORACLE_HOME (RDBMS ORACLE_HOME directory): echo \$ORACLE_HOME
- GRID_HOME (GRID INFRASTRUCTURE HOME directory): echo \$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)
- DBFS_PASSWD (used only if WALLET=false)
- DBFS_PWDFILE_BASE (used only if WALLET=false)
- WALLET (must be true or false)
- TNS_ADMIN (used only if WALLET=true or PDB): echo \$TNS_ADMIN
- DBFS_LOCAL_TNSALIAS (used only if WALLET=true)
- IS_PDB (set to true if using PDB)
- PDB (PDB name, if applicable): PDB name
- PDB_SERVICE (the database service created in step 2.3, if applicable):
PDB_SERVICE_NAME
- MOUNT_OPTIONS: allow_other,direct_io,failover,nolock
 - The failover option forces all file writes to be committed to the DBFS database in an IMMEDIATE WAIT mode. This prevents data from getting lost when it has been written into the dbfs_client cache, but not yet written to the database at the time of a database or node failure.
 - The nolock mount option is required if you use Oracle Database 18c or later versions because of a change in the DBFS file locking, which can cause issues for GoldenGate processes after an Oracle RAC node failure when a file is currently locked.

- As the grid OS user, unzip the mount-dbfs-<version>.zip and edit the configuration file mount-dbfs.conf:

```
[opc@exadb-node1]$ sudo su - grid
[grid@exadb-node1]$ cd /u02/app_acfs/goldengate
[grid@exadb-node1]$ unzip mount-dbfs-<version>.zip
[grid@exadb-node1]$ vi mount-dbfs.conf
```

Example of mount-dbfs.conf:

```
DBNAME=<DB_UNIQUE_NAME>
MOUNT_POINT=/mnt/dbfs/goldengate
DBFS_USER=dbfs_user
GRID_HOME=$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)
if [ -z "${GRID_HOME}" ]; then
    echo "GRID_HOME is unset or set to the empty string"
fi
ORACLE_HOME=$(($GRID_HOME/bin/srvctl config database -d $DBNAME |grep
'Oracle home:' | cut -d: -f2 |sed 's/ //g')
if [ -z "${ORACLE_HOME}" ]; then
    echo "ORACLE_HOME is unset or set to the empty string"
fi
LOGGER_FACILITY=user
MOUNT_OPTIONS=allow_other,direct_io,failover,nolock
PERL_ALARM_TIMEOUT=14
DBFS_PASSWD=<DBFS_USER_PASSWORD>
DBFS_PWDFILE_BASE=/tmp/.dbfs-passwd.txt
WALLET=false
TNS_ADMIN=$ORACLE_HOME/network/admin/<DB_NAME>
IS_PDB=true
PDB=<PDB_NAME>
PDB_SERVICE=<PDB_SERVICE_NAME>
```

- As the grid OS user, modify the mount-dbfs.sh script to force unmounting of DBFS when the CRS resource is stopped:

```
[grid@exadb-node1]$ vi /u02/app_acfs/goldengate/mount-dbfs.sh

# Change two occurrences of:
$FUSERMOUNT -u $MOUNT_POINT
# To the following:
$FUSERMOUNT -uz $MOUNT_POINT
```

- As the opc OS user, copy mount-dbfs.conf (rename it if desired or needed) to the directory /etc/oracle on database nodes and set proper permissions on it:

```
[opc@exadb-node1]$ sudo -u grid $(grep ^crs_home /etc/oracle/olr.loc | cut
-d= -f2)/bin/olsnodes > ~/dbs_group
[opc@exadb-node1]$ /usr/local/bin/dcli -g ~/dbs_group -l opc -d /tmp
-f /u02/app_acfs/goldengate/mount-dbfs.conf
[opc@exadb-node1]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo
cp /u02/app_acfs/goldengate/mount-dbfs.conf /etc/oracle
[opc@exadb-node1]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo
chown grid:oinstall /etc/oracle/mount-dbfs.conf
```

```
[opc@exadb-node1]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo
chmod 660 /etc/oracle/mount-dbfs.conf
```

5. As the opc OS user, copy mount-dbfs.sh (rename it if desired or needed) to the proper directory (\$GI_HOME/crs/script) on database nodes and set proper permissions on it:

```
[opc@exadb-node1]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo
mkdir $(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/crs/script
[opc@exadb-node1]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo chown
grid:oinstall $(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/crs/
script
[opc@exadb-node1]$ /usr/local/bin/dcli -g ~/dbs_group -l grid
-d $(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/crs/script
-f /u02/app_acfs/goldengate/mount-dbfs.sh
[opc@exadb-node1]$ /usr/local/bin/dcli -g ~/dbs_group -l grid chmod 770
$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/crs/script/mount-
dbfs.sh
```

Step 3a.5 - Register the DBFS Resource with Oracle Clusterware

When registering the resource with Oracle Clusterware, create it as a cluster_resource.

The reason for using cluster_resource is so the file system can only be mounted on a single node at one time, preventing mounting of DBFS from concurrent nodes creating the potential of concurrent file writes, and causing file corruption problems.

1. As the grid OS user, find the resource name for the database service created in a previous step for the DBFS service dependency:

```
[opc@exadb-node1]$ sudo su - grid
[grid@exadb-node1]$ crsctl stat res |grep <PDB_NAME>
NAME=ora.<DB_UNIQUE_NAME>.<SERVICE_NAME>.svc
```

2. As the oracle OS user, register the Clusterware resource by running the following script:

```
[opc@exadb-node1]$ sudo su - oracle
[oracle@exadb-node1]$ vi /u02/app_acfs/goldengate/add-dbfs-resource.sh

##### start script add-dbfs-resource.sh
#!/bin/bash
ACTION_SCRIPT=$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/crs/
script/mount-dbfs.sh
RESNAME=dbfs_mount
DEPNAME=ora.<DB_UNIQUE_NAME>.<SERVICE_NAME>.svc
ORACLE_HOME=$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)
PATH=$ORACLE_HOME/bin:$PATH
export PATH ORACLE_HOME
crsctl add resource $RESNAME \
    -type cluster_resource \
    -attr "ACTION_SCRIPT=$ACTION_SCRIPT, \
        CHECK_INTERVAL=30,RESTART_ATTEMPTS=10, \
        START_DEPENDENCIES='hard($DEPNAME)pullup($DEPNAME)', \
        STOP_DEPENDENCIES='hard($DEPNAME)', \
        SCRIPT_TIMEOUT=300"
##### end script add-dbfs-resource.sh
```

```
[oracle@exadb-node1]$ sh /u02/app_acfs/goldengate/add-dbfs-resource.sh
```

① Note

After creating the \$RESNAME resource, to stop the \$DBNAME database when the \$RESNAME resource is ONLINE, you specify the force flag when using `srvctl`.

For example: `srvctl stop database -d fsdb -f`

Step 3a.6 - Start the DBFS Resource

As the grid OS user, start the resource:

```
[opc@exadb-node1]$ sudo su - grid
[grid@exadb-node1]$ crsctl start res dbfs_mount -n `hostname`
CRS-2672: Attempting to start 'dbfs_mount' on 'exadb-node1'
CRS-2676: Start of 'dbfs_mount' on 'exadb-node1' succeeded
```

```
[grid@exadb-node1]$ crsctl stat res dbfs_mount -t
```

```
-----
--
Name          Target  State        Server          State
details
-----
--
Cluster Resources
-----
--
dbfs_mount
   1          ONLINE  ONLINE      exadb-node1     STABLE
-----
--
```

① Note

Leave the shared file system mounted. It is required for creating the Oracle GoldenGate deployment in a later step.

Step 3b - Oracle ASM Cluster File System (ACFS)

Oracle ACFS is an alternative to DBFS for the shared Oracle GoldenGate files in an Oracle RAC configuration. Create a single ACFS file system for storing the Oracle deployment files.

It is recommended that you allocate enough trail file disk space to permit the storage of up to 12 hours of trail files. Doing this will give sufficient space for trail file generation should a problem occur with the target environment that prevents it from receiving new trail files. The amount of space needed for 12 hours can only be determined by testing trail file generation rates with real production data.

Perform the following sub-steps to complete this step:

- Step 3b.1 - Create the ASM File System
- Step 3b.2 - Make the File System
- Step 3b.3 - Create the Cluster Ready Services (CRS) Resource
- Step 3b.4 - Verify the Currently Configured ACFS File Systems
- Step 3b.5 - Start and Check the Status of the ACFS Resource
- Step 3b.6 - Create GoldenGate ACFS Directory

Step 3b.1 - Create the ASM File System

As the grid OS user, use `asmcmd` to create the volume:

```
[opc@exadb-node1 ~]$ sudo su - grid
[grid@exadb-node1 ~]$ asmcmd volcreate -G DATA1 -s 1200G ACFS_GG
```

① Note

Modify the file system size according to the determined size requirements.

Step 3b.2 - Make the File System

1. As the grid OS user, use `asmcmd` to confirm the “Volume Device”:

```
[grid@exadb-node1 ~]$ asmcmd volinfo -G DATA1 ACFS_GG
```

Following is an example of the ACFS volume device output:

```
Diskgroup Name: DATA1
Volume Name: ACFS_GG
Volume Device: /dev/asm/acfs_gg-151
State: ENABLED
Size (MB): 1228800
Resize Unit (MB): 64
Redundancy: MIRROR
Stripe Columns: 8
Stripe Width (K): 1024
Usage:
Mountpath:
```

2. As the grid OS user, make the file system with the following `mkfs` command:

```
[grid@exadb-node1 ~]$ /sbin/mkfs -t acfs /dev/asm/acfs_gg-151
```

Step 3b.3 - Create the Cluster Ready Services (CRS) Resource

1. As the opc OS user, create the ACFS mount point:

```
[opc@exadb-node1 ~]$ dcli -l opc -g ~/dbs_group sudo mkdir -p /mnt/acfs_gg
[opc@exadb-node1 ~]$ dcli -l opc -g ~/dbs_group sudo chown
oracle:oinstall /mnt/acfs_gg
```

2. Create the file system resource as the root user.

Because the implementation of distributed file locking on ACFS, unlike DBFS, it is acceptable to mount ACFS on more than one Oracle RAC node at any one time.

3. As the root OS user, create the ACFS resource for the new ACFS file system:

```
[opc@exadb-node1 ~]$ sudo su -
[root@exadb-node1 ~]# $(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/bin/srvctl
add filesystem -device /dev/asm/acfs_gg-151 -volume ACFS_GG -diskgroup
DATAC1
-path /mnt/acfs_gg -user oracle
```

Step 3b.4 - Verify the Currently Configured ACFS File Systems

As the grid OS user, use the following command to view the file system details:

```
[opc@exadb-node1 ~]$ sudo su - grid
[grid@exadb-node1 ~]$ srvctl config filesystem -volume ACFS_GG -diskgroup
DATAC1
```

```
Volume device: /dev/asm/acfs_gg-151
Diskgroup name: datac1
Volume name: acfs_gg
Canonical volume device: /dev/asm/acfs_gg-151
Accelerator volume devices:
Mountpoint path: /mnt/acfs_gg
Mount point owner: oracle
Mount point group: oinstall
Mount permissions: owner:oracle:rwx,pgrp:oinstall:r-x,other::r-x
Mount users: grid
Type: ACFS
Mount options:
Description:
ACFS file system is enabled
ACFS file system is individually enabled on nodes:
ACFS file system is individually disabled on nodes:
```

Step 3b.5 - Start and Check the Status of the ACFS Resource

As the grid OS user, use the following command to start and check the file system:

```
[grid@exadb-node1 ~]$ srvctl start filesystem -volume ACFS_GG
-diskgroup DATAC1 -node `hostname`
[grid@exadb-node1 ~]$ srvctl status filesystem -volume ACFS_GG -diskgroup
DATAC1
```

ACFS file system /mnt/acfs_gg is mounted on nodes exadb-node1

The CRS resource created is named using the format `ora.diskgroup_name.volume_name.acfs`. Using the above file system example, the CRS resource is called `ora.datac1.acfs_gg.acfs`.

To see all ACFS file system CRS resources that currently exist, use the following command.

```
[grid@exadb-node1 ~]$ crsctl stat res -w "((TYPE = ora.acfs.type) OR (TYPE =
ora.acfs_cluster.type))"
```

```
NAME=ora.dataac1.acfs_gg.acfs
TYPE=ora.acfs.type
TARGET=ONLINE , OFFLINE
STATE=ONLINE on exadb-node1, OFFLINE
NAME=ora.dataac1.acfsvol01.acfs
TYPE=ora.acfs.type
TARGET=ONLINE , ONLINE
STATE=ONLINE on exadb-node1, ONLINE on exadb-node2
```

Step 3b.6- Create GoldenGate ACFS Directory

As the grid OS user, create the directory for storing the Oracle GoldenGate deployments.

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ mkdir -p /mnt/acfs_gg/deployments
```

Refer to the [Oracle Automatic Storage Management Cluster File System Administrator's Guide](#) for more information about ACFS.

① Note

Leave the shared file system mounted. It is required for creating the Oracle GoldenGate deployment in a later step.

Task 4 - Install Oracle GoldenGate

Install the Oracle GoldenGate software **locally** on all nodes in the Oracle Exadata Database Service configuration that will be part of the Oracle GoldenGate configuration. Make sure the installation directory is **identical** on all nodes.

Perform the following steps to complete this task:

- Step 4.1 - Unzip the Software and Create the Response File for the Installation
- Step 4.2 - Install Oracle GoldenGate
- Step 4-3 - Patch Oracle GoldenGate

Step 4.1 - Unzip the Software and Create the Response File for the Installation

1. As the oracle OS user on the first database node, unzip the software:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ unzip
/u02/app_acfs/goldengate/
213000_fbo_ggs_linux_x64_Oracle_services_shiphome.zip
-d /u02/app_acfs/goldengate
```

The software includes an example response file for Oracle Database release 21c and earlier supported releases. Copy the response file to a shared file system, so the same file can be used to install Oracle GoldenGate on all database nodes, and edit the following parameters:

- `INSTALL_OPTION=ora21c`
 - `SOFTWARE_LOCATION=/u02/app/oracle/goldengate/gg21c` (recommended location)
2. As the oracle OS user on the first database node, copy and edit the response file for the installation.

```
[oracle@exadb-node1 ~]$ cp
/u02/app_acfs/goldengate/fbo_ggs_Linux_x64_Oracle_services_shiphome/Disk1/
response/oggcore.rsp
/u02/app_acfs/goldengate
[oracle@exadb-node1 ~]$ vi /u02/app_acfs/goldengate/oggcore.rsp

# Before edit
INSTALL_OPTION=
SOFTWARE_LOCATION=

# After edit
INSTALL_OPTION=ora21c
SOFTWARE_LOCATION=/u02/app/oracle/goldengate/gg21c
```

Step 4.2 - Install Oracle GoldenGate

As the oracle OS user on all database nodes install Oracle GoldenGate:

```
[oracle@exadb-node1 ~]$ cd
/u02/app_acfs/goldengate/fbo_ggs_Linux_x64_Oracle_services_shiphome/Disk1/
[oracle@exadb-node1 ~]$ ./runInstaller -silent -nowait
-responseFile /u02/app_acfs/goldengate/oggcore.rsp
```

Starting Oracle Universal Installer...

```
Checking Temp space: must be greater than 120 MB.   Actual 32755 MB   Passed
Checking swap space: must be greater than 150 MB.   Actual 16383 MB   Passed
Preparing to launch Oracle Universal Installer from
/tmp/OraInstall2022-07-08_02-54-51PM. Please wait ...
You can find the log of this install session at:
/u01/app/oraInventory/logs/installActions2022-07-08_02-54-51PM.log
Successfully Setup Software.
The installation of Oracle GoldenGate Services was successful.
Please check '/u01/app/oraInventory/logs/
silentInstall2022-07-08_02-54-51PM.log'
for more details.
```

```
[oracle@exadb-node1 ~]$ cat
/u01/app/oraInventory/logs/silentInstall2022-07-08_02-54-51PM.log
The installation of Oracle GoldenGate Services was successful.
```

```
[oracle@exadb-node1 ~]$ ssh exadb-node2
[oracle@exadb-node2 ~]$ cd
/u02/app_acfs/goldengate/fbo_ggs_Linux_x64_Oracle_services_shiphome/Disk1
[oracle@exadb-node2 ~]$ ./runInstaller -silent -nowait
-responseFile /u02/app_acfs/goldengate/oggcore.rsp
```

Starting Oracle Universal Installer...

```
Checking Temp space: must be greater than 120 MB.   Actual 32755 MB   Passed
```

```

Checking swap space: must be greater than 150 MB.   Actual 16383 MB   Passed
Preparing to launch Oracle Universal Installer from
/tmp/OraInstall2022-07-08_03-54-51PM. Please wait ...
You can find the log of this install session at:
/u01/app/oraInventory/logs/installActions2022-07-08_03-54-51PM.log
Successfully Setup Software.
The installation of Oracle GoldenGate Services was successful.
Please check '/u01/app/oraInventory/logs/
silentInstall2022-07-08_03-54-51PM.log'
for more details.

```

```

[oracle@exadb-node1 ~]$ cat
/u01/app/oraInventory/logs/silentInstall2022-07-08_03-54-51PM.log
The installation of Oracle GoldenGate Services was successful.

```

Step 4.3 - Patch Oracle Goldengate

As the oracle OS user on all database nodes, install the latest OPatch:

```

[oracle@exadb-node1 ~]$ unzip -oq -d /u01/app/oracle/goldengate/gg21c
/u02/app_acfs/goldengate /p6880880_210000_Linux-x86-64.zip
[oracle@exadb-node1 ~]$ cat >> ~/.bashrc <<EOF
export ORACLE_HOME=/u01/app/oracle/goldengate/gg21c
export PATH=$ORACLE_HOME/OPatch:$PATH
EOF
[oracle@exadb-node1 ~]$. ~/.bashrc
[oracle@exadb-node1 ~]$ opatch lsinventory |grep 'Oracle GoldenGate Services'

Oracle GoldenGate Services
21.1.0.0.0

[oracle@gghub_prim1 Disk1]$ opatch version

OPatch Version: 12.2.0.1.37

```

As the oracle OS user, run OPatch prereq to validate any conflict before applying the patch:

```

[oracle@exadb-node1 ~]$ unzip -oq -d /u02/app_acfs/goldengate
/u02/app_acfs/goldengate /p35214851_2190000GGRU_Linux-x86-64.zip
[oracle@exadb-node1 ~]$ cd /u02/app_acfs/goldengate/35214851/
[oracle@exadb-node1 35214851]$ opatch prereq CheckConflictAgainstOHWithDetail
-ph ./

```

```

Oracle Interim Patch Installer version 12.2.0.1.26
Copyright (c) 2023, Oracle Corporation. All rights reserved.

```

```

PREREQ session
Oracle Home      : /u01/app/oracle/goldengate/gg21c
Central Inventory : /u01/app/oraInventory
   from           : /u01/app/oracle/goldengate/gg21c/oraInst.loc
OPatch version   : 12.2.0.1.26
OUI version      : 12.2.0.9.0
Log file location : /u01/app/oracle/goldengate/gg21c/cfgtoollogs/patch/
opatch2023-04-21_13-44-16PM_1.log

```

Invoking prereq "checkconflictagainsthwithdetail"

Prereq "checkConflictAgainstOHWithDetail" passed.

As the oracle OS user on all database nodes, patch Oracle GoldenGate Microservices Architecture using OPatch:

```
[oracle@exadb-node1 ~]$ cd /u02/app/acfs/goldengate/35214851/
[oracle@exadb-node1 35214851]$ opatch apply
```

Oracle Interim Patch Installer version 12.2.0.1.37
Copyright (c) 2023, Oracle Corporation. All rights reserved.

Oracle Home : /u01/app/oracle/goldengate/gg21c
Central Inventory : /u01/app/oraInventory
 from : /u01/app/oracle/goldengate/gg21c/oraInst.loc
OPatch version : 12.2.0.1.37
OUI version : 12.2.0.9.0
Log file location : /u01/app/oracle/goldengate/gg21c/cfgtoollogs/patch/
opatch2023-04-21_19-40-41PM_1.log
Verifying environment and performing prerequisite checks...
OPatch continues with these patches: 35214851

Do you want to proceed? [y|n]

y

User Responded with: Y

All checks passed.

Please shutdown Oracle instances running out of this ORACLE_HOME on the local system.

(Oracle Home = '/u01/app/oracle/goldengate/gg21c')

Is the local system ready for patching? [y|n]

y

User Responded with: Y

Backing up files...

Applying interim patch '35214851' to OH '/u01/app/oracle/goldengate/gg21c'

Patching component oracle.oggcore.services.ora21c, 21.1.0.0.0...

Patch 35214851 successfully applied.

Log file location: /u01/app/oracle/goldengate/gg21c/cfgtoollogs/patch/
opatch2023-04-21_19-40-41PM_1.log

OPatch succeeded.

```
[oracle@exadb-node1 35214851]$ opatch lspatches
```

35214851;

Task 5 - Create the Oracle GoldenGate Deployment

When the Oracle GoldenGate software has been installed, your next step is to create a response file to create the Oracle GoldenGate deployment using the Oracle GoldenGate Configuration Assistant.

Perform the following steps to complete this task:

- Step 5.1 - Create a Response File
- Step 5.2 - Create the GoldenGate Deployment
- Step 5.3 - (only if using DBFS) Move the GoldenGate Deployment Temp Directory

Step 5.1 - Create a Response File

For a silent configuration, as the oracle OS user, create and edit the response file `oggca.rsp` to create the Oracle GoldenGate deployment:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ vi /u02/app_acfs/goldengate/oggca.rsp
oracle.install.responseFileVersion=/oracle/install/
rspfmt_oggca_response_schema_v21_1_0

CONFIGURATION_OPTION=ADD
DEPLOYMENT_NAME=<ggNN>
ADMINISTRATOR_USER=oggadmin
ADMINISTRATOR_PASSWORD=<password_for_oggadmin>
SERVICEMANAGER_DEPLOYMENT_HOME=<ACFS or DBFS mount point>/deployments/<ggsmNN>
HOST_SERVICEMANAGER=localhost
PORT_SERVICEMANAGER=9100
SECURITY_ENABLED=false
STRONG_PWD_POLICY_ENABLED=true
CREATE_NEW_SERVICEMANAGER=true
REGISTER_SERVICEMANAGER_AS_A_SERVICE=false
INTEGRATE_SERVICEMANAGER_WITH_XAG=true
EXISTING_SERVICEMANAGER_IS_XAG_ENABLED=false
OGG_SOFTWARE_HOME=/u02/app/oracle/goldengate/gg21c
OGG_DEPLOYMENT_HOME=<ACFS or DBFS mount point>/deployments/<ggNN>
ENV_LD_LIBRARY_PATH=${OGG_HOME}/lib/instantclient:${OGG_HOME}/lib
ENV_TNS_ADMIN=/u02/app/oracle/goldengate/network/admin
FIPS_ENABLED=false
SHARDING_ENABLED=false
ADMINISTRATION_SERVER_ENABLED=true
PORT_ADMINSRVR=9101
DISTRIBUTION_SERVER_ENABLED=true
PORT_DISTSRVR=9102
NON_SECURE_DISTSRVR_CONNECTS_TO_SECURE_RCVRSVR=false
RECEIVER_SERVER_ENABLED=true
PORT_RCVRSVR=9103
METRICS_SERVER_ENABLED=true
METRICS_SERVER_IS_CRITICAL=false
PORT_PMSRVR=9104
UDP_PORT_PMSRVR=9105
PMSRVR_DATASTORE_TYPE=BDB
```

```
PMSVR_DATASTORE_HOME=/u02/app/oracle/goldengate/datastores/<instance_name>
OGG_SCHEMA=<goldengate_database_schema>
```

In the response file, edit the following values appropriately:

- CONFIGURATION_OPTION
- DEPLOYMENT_NAME
- ADMINISTRATOR_USER
- SERVICEMANAGER_DEPLOYMENT_HOME
- OGG_SOFTWARE_HOME
- OGG_DEPLOYMENT_HOME
- ENV_TNS_ADMIN
- OGG_SCHEMA

Step 5.2 - Create the GoldenGate Deployment

As the oracle OS user on the first database node, run `oggca.sh` to create the Oracle GoldenGate deployment:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ export OGG_HOME=/u02/app/oracle/goldengate/gg21c
[oracle@exadb-node1 ~]$ $OGG_HOME/bin/oggca.sh -silent
  -responseFile /u02/app_acfs/goldengate/oggca.rsp
Successfully Setup Software.
```

Step 5.3 - (only if using DBFS) Move the GoldenGate Deployment Temp Directory

After the deployment has been created, if you use DBFS for the shared file system, run the following commands to move the GoldenGate deployment temp directory from DBFS to local storage.

1. As the oracle OS user on the first database node, move the GoldenGate deployment temporary directory to the local storage:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ dcli -l oracle -g ~/dbs_group mkdir
  -p /u02/app/oracle/goldengate/deployments/<instance_name>
[oracle@exadb-node1 ~]$ mv
  /mnt/dbfs/goldengate/deployments/<instance_name>/var/temp
  /u02/app/oracle/goldengate/datastores/<instance_name>
[oracle@exadb-node1 ~]$ ln -s
  /u02/app/oracle/goldengate/deployments/<instance_name>/temp
  /mnt/dbfs/goldengate/deployments/<instance_name>/var/temp
```

2. As the oracle OS user on the rest of the database nodes, create a directory on the local storage:

```
[oracle@exadb-node2 ~]$ mkdir
  /u02/app/oracle/goldengate/deployments/<instance_name>
```

Task 6 - Configure the Network

The way you configure the network depends on your Exadata platform. The first method described in Step 6a applies to ExaDB-D only, and the second method described in Step 6b applies to ExaDB-C@C only.

Perform one of the following steps to complete this task:

- Step 6a - (ExaDB-D only) Configure Oracle Cloud Infrastructure Networking
- Step 6b - (ExaDB-C@C only) Prepare for Application Virtual IP Address Creation

Step 6a - (ExaDB-D only) Configure Oracle Cloud Infrastructure Networking

You must configure virtual cloud network (VCN) components such as private DNS zones, VIP, bastion, security lists, and firewalls for Oracle GoldenGate to function correctly.

To learn more about VCNs and security lists, including instructions for creating them, see [Oracle Cloud Infrastructure Networking](#).

Perform the following sub-steps to complete this step:

- Step 6a.1 - Connect to GoldenGate Microservices Web Interface Using a Private IP
- Step 6a.2 - Create an Application Virtual IP Address (VIP)
- Step 6a.3 - Add Ingress Rule
- Step 6a.4 - Open Port 443 in the Firewall
- Step 6a.5 - Connecting your Source and Target VIP
- Step 6a.5 - Configuring Network Connectivity Between GoldenGate Source and Target
- Step 6a.6 - Configure Private DNS Zones Views and Resolvers

Step 6a.1 - Connect to GoldenGate Microservices Web Interface Using a Private IP

GoldenGate Microservices web interface is only accessible using a private endpoint from within the OCI network or through a bastion host that secures access to OCI resources.

If OCI Bastion service is unavailable in your region, you can use your OCI Compute Instance as a bastion. Follow the steps in [OCI Bastion As A Service](#) to create your bastion. You will need one bastion for each region where Oracle GoldenGate Microservices is running.

Note

After creating a bastion or using a compute instance as a bastion, you need to create an SSH port forwarding session to use `https://localhost:local_port` to connect to Oracle GoldenGate Microservices.

Step 6a.2 - Create an Application Virtual IP Address (VIP)

A dedicated application VIP is required to allow access to the Oracle GoldenGate Microservices using the same host name, regardless of which Oracle RAC node is hosting the services. An application VIP will also ensure the Oracle GoldenGate Distribution Server can communicate with the Distribution Receiver running the current Oracle RAC node.

The VIP is a cluster resource that Oracle Clusterware manages. The VIP is assigned to a database node and is automatically migrated to another node in the event of a node failure.

Using the Console, assign the VIP to the Oracle Exadata Database Service:

1. Open the navigation menu. Click **Oracle Database**, then click **Exadata on Oracle Public Cloud**.
2. Choose your compartment.
3. Click **Exadata VM Cluster** under Oracle Exadata Database Service on Dedicated Infrastructure.
4. Navigate to the Exadata VM Cluster you want to create the new VIP.
5. Under Resources, click **Virtual IP Address**.
6. Click **Attach Virtual IP Address**.
7. In the Attach Virtual IP Address dialog, enter the following mandatory information:
 - **Subnet:** The client subnet
 - **Virtual IP address hostname:** Use the SCAN DNS Name and replace the SCAN word for Oracle GoldenGate (Example: exadb-xxxx-ggN)
8. Click **Create**.

When the Virtual IP Address creation is complete, the status changes from Provisioning to Available, and the assigned IP will be shown in the Virtual IP Address. Make a note of the fully qualified domain name; this is the host name required to connect the source with the target Oracle GoldenGate deployment.

Note

Adding a new VIP is available in most tenancies; log a Service Request if you have any issues.

Step 6a.3 - Add an Ingress Rule

Using the Console, open ingress port 443 to connect the Oracle GoldenGate service using NGINX as a reverse proxy. For more information, see [Working with Security Lists](#).

After you update the security list, it will have an entry with values similar to the following:

- Source Type: CIDR
- Source CIDR: 0.0.0.0/0
- IP Protocol: TCP
- Source Port Range: All
- Destination Port Range: 443
- Allows: TCP traffic for ports: 443 HTTPS
- Description: Oracle GoldenGate 443

Step 6a.4 - Open Port 443 in the Firewall

As the opc OS user, validate if the chains are currently figured to accept traffic:

```
[opc@exadb-node1 ~]$ sudo iptables --list |grep policy
```

```
Chain INPUT (policy ACCEPT)
```

```
Chain FORWARD (policy ACCEPT)
Chain OUTPUT (policy ACCEPT)
```

If the policy is ACCEPT, you can skip this step and proceed with Task 7. Otherwise, contact your network administrator to update the firewall to open port 443 for ingress activity.

Step 6a.5 - Configuring Network Connectivity Between the GoldenGate Source and Target

You can set up your VCN to access the internet if you like. You can also privately connect your VCN to public Oracle Cloud Infrastructure services such as Object Storage, your on-premises network, or another VCN.

To learn more about whether subnets are public or private, including instructions for creating the connection, see [Connectivity Choices](#) in the Oracle Cloud Infrastructure Networking documentation.

Step 6a.6 - Configure Private DNS Zones Views and Resolvers

If the source and target Oracle GoldenGate deployments are in different regions, you must create a private DNS view in the source region with a private zone. This is required for the source Oracle GoldenGate Distribution Path to reach the target Oracle GoldenGate deployment VIP host name.

Follow the steps in [Configure private DNS zones views and resolvers](#) to create your private DNS view and zone.

As the opc OS user on the source system, use the command nslookup to resolve the **Fully qualified domain name** (from Step 6.2) of the target Oracle GoldenGate deployment:

```
[opc@exadb-node1 ~]$ nslookup <target_vip_fully_qualified_domain_name>
Server:                <DNS_IP>
Address:                <DNS_IP>#53
```

```
Non-authoritative answer:
Name:   <target_vip_fully_qualified_domain_name>
Address: <target_vip_ip>
```

Step 6b - (ExaDB-C@C only) Prepare for Application Virtual IP Address Creation

A dedicated application VIP is required to allow access to the Oracle GoldenGate Microservices using the same host name, regardless of which Oracle RAC node is hosting the services. An application VIP will also ensure that the Oracle GoldenGate Distribution Server can communicate with the Distribution Receiver running the current Oracle RAC node.

The VIP is a cluster resource that Oracle Clusterware manages. The VIP is assigned to a database node and is automatically migrated to another node in the event of a node failure.

Your system administrator must provide the IP address for the new Application VIP. This IP address must be in the same subnet as the system environment as determined above.

The VIP will be created in the next Task when you configure the Oracle Grid Infrastructure Agent.

Task 7 - Configure Oracle Grid Infrastructure Agent

The following procedure shows you how to configure Oracle Clusterware to manage Oracle GoldenGate using the Oracle Grid Infrastructure Standalone Agent (XAG).

Using XAG automates the mounting of the shared file system (DBFS or ACFS) as well as the stopping and starting of the Oracle GoldenGate deployment when relocating between Oracle RAC nodes.

Perform the following steps to complete this task:

- Step 7.1 - Install the Oracle Grid Infrastructure Standalone Agent
- Step 7.2 - Configure Oracle Grid Infrastructure Agent
- Step 7.2 - Start the Oracle GoldenGate Deployment

Step 7.1 - Install the Oracle Grid Infrastructure Standalone Agent

It is recommended that you install the XAG software as a standalone agent outside the Grid Infrastructure ORACLE_HOME. This way, you can use the latest XAG release available, and the software can be updated without impact to the Grid Infrastructure.

XAG must be installed in the same directory on all Oracle RAC database nodes in the system where Oracle GoldenGate is installed.

1. As the grid OS user on the first database node, unzip the software and run xagsetup.sh:

```
[opc@exadb-node1 ~]$ sudo su - grid
[grid@exadb-node1 ~]$ unzip /u02/app_acfs/goldengate/
p31215432_190000_Generic.zip
-d /u02/app_acfs/goldengate
[grid@exadb-node1 ~]$ /u02/app_acfs/goldengate/xag/xagsetup.sh --install
--directory /u01/app/grid/xag --all_nodes
Installing Oracle Grid Infrastructure Agents on: exadb-node1
Installing Oracle Grid Infrastructure Agents on: exadb-node2
Updating XAG resources.
Successfully updated XAG resources.
```

2. Add the location of the newly installed XAG software to the PATH variable so that the location of agctl is known when the grid user logs on to the machine.

```
[grid@exadb-node1 ~]$ grep PATH ~/.bashrc
PATH=
/u01/app/grid/xag/bin:/sbin:/bin:/usr/sbin:/usr/bin:/u01/app/19.0.0.0/grid/
bin:/u01/app/19.0.0.0/grid/OPatch;
export PATH
```

Note

It is essential that you ensure that the XAG bin directory is specified **before** the Grid Infrastructure bin directory to ensure that the correct agctl binary is found. This should be set in the grid user environment to take effect when logging on, such as in the .bashrc file when the Bash shell is in use.

Step 7.2 - Configure Oracle Grid Infrastructure Agent

The following procedure shows you how to configure Oracle Clusterware to manage Oracle GoldenGate using the Oracle Grid Infrastructure Standalone Agent (XAG).

Using XAG automates the mounting of the shared file system (DBFS or ACFS) as well as the stopping and starting of the Oracle GoldenGate deployment when relocating between Oracle RAC nodes.

Oracle GoldenGate must be registered with XAG so that the deployment is started and stopped automatically when the database is started and the file system is mounted.

To register Oracle GoldenGate Microservices Architecture with XAG, use the following command format:

```
agctl add goldengate <instance_name>
--gg_home <GoldenGate_Home>
--service_manager
--config_home <GoldenGate_SvcMgr_Config>
--var_home <GoldenGate_SvcMgr_Var_Dir>
--port <port number>
--oracle_home <$OGG_HOME/lib/instantclient>
--adminuser <OGG admin user>
--user <GG instance user>
--group <GG instance group>
--network <network_number>
--ip <ip_address>
--vip_name <vip_name>
--filesystems <CRS_resource_name>
--db_services <service_name>
--use_local_services
--attribute START_TIMEOUT=60
--nodes <node1, node2, ... ,nodeN>
```

Where:

- --gg_home specifies the location of the Oracle GoldenGate software.
- --service_manager indicates this is a GoldenGate Microservices instance.
- --config_home specifies the GoldenGate Service Manager deployment configuration home directory.
- --var_home specifies the GoldenGate Service Manager deployment variable home directory.
- --port specifies the deployment Service Manager port number.
- --oracle_home specifies the location of the Oracle database libraries that are included as part of Oracle GoldenGate 21c and later releases.
Example: \$OGG_HOME/lib/instantclient
- --adminuser specifies the Oracle GoldenGate Microservices administrator account name.
- --user specifies the name of the operating system user that owns the Oracle GoldenGate deployment.
- --group specifies the name of the operating system group that owns the Oracle GoldenGate deployment.
- --network specifies the network subnet for the VIP.

- `--ip` specifies the IP address for the VIP.
If you have already created a VIP, specify it using the `--vip_name vip_name` parameter in place of `--network` and `--ip`.
- `--vip_name` specifies a CRS resource name for an application VIP previously created.
This parameter replaces `--network` and `--ip` (optional).
- `--filesystems` specifies the DBFS or ACFS CRS file system resource that must be mounted before the deployment is started.
- `--db_services` specifies the `ora.<database>.<service_name>.svc` service name created in the previous step.
If you are using Oracle Multitenant Database, specify the PDB database service for Replicat or the CDB database service for an Extract. If using Replicat and Extract, specify both service names, separated by a comma.
- `--use_local_services` specifies that the Oracle GoldenGate instance must be co-located on the same Oracle RAC node where the `db_services` service is running.
- `--attribute name=value` specifies attributes that can be applied.
It is recommended that you modify the attribute `START_TIMEOUT=60` to optimize the blackout after a database crash and restart.
- `--nodes` specifies which of the Oracle RAC nodes this GoldenGate instance can run on.
If Oracle GoldenGate is configured to run on any of the Oracle RAC nodes in the cluster, this parameter should still be used to determine the preferred order of nodes to run Oracle GoldenGate.

Perform one of the following steps to complete this task:

- Step 7.2a - GoldenGate Deployments on DBFS
- Step 7.2b - GoldenGate Deployments on ACFS

Step 7.2a - GoldenGate Deployments on DBFS

1. As the grid OS user on the first database node, run the following command to identify the network number:

```
[opc@exadb-node1 ~]$ sudo su - grid
[grid@exadb-node1 ~]$ srvctl config network
Network 1 exists
Subnet IPv4: 10.1.0.0/255.255.255.0/bondeth0, static
Subnet IPv6:
Ping Targets: 10.1.0.1
Network is enabled
Network is individually enabled on nodes:
Network is individually disabled on nodes:
```

2. As the root OS user on the first database node, register Oracle GoldenGate Microservices Architecture with XAG using the following command format:

```
[opc@exadb-node1 ~]$ sudo su -

[root@exadb-node1 ~]# /u01/app/grid/xag/bin/agctl add goldengate
<instance_name> \
--gg_home /u02/app/oracle/goldengate/gg21c \
--service_manager \
```

```

--config_home /mnt/dbfs/deployments/ggsm01/etc/conf
\
--var_home /mnt/dbfs/deployments/ggsm01/var
\
--port 9100
\
--oracle_home /u02/app/oracle/goldengate/gg21c/lib/instantclient
\
--adminuser oggadmin
\
--user oracle
\
--group oinstall
\
--network 1 --ip <virtual_IP_address>
\
--filesystems <dbfs_mount_name>
\
--db_services ora.<db_service_name>.svc , ora.<pdb_service_name>.svc
\
--use_local_services
\
--attribute START_TIMEOUT=60
\
--nodes <exadb-node1>, <exadb-node2>

Enter password for 'oggadmin' : <oggadmin_password>

```

Step 7.2b - GoldenGate Deployments on ACFS

1. As the grid OS user on the first database node, run the following command to identify the network number:

```

[opc@exadb-node1 ~]$ sudo su - grid
[grid@exadb-node1 ~]$ srvctl config network
Network 1 exists
Subnet IPv4: 10.1.0.0/255.255.255.0/bondeth0, static
Subnet IPv6:
Ping Targets: 10.1.0.1
Network is enabled
Network is individually enabled on nodes:
Network is individually disabled on nodes:

```

2. As the root OS user on the first database node, register Oracle GoldenGate Microservices Architecture with XAG using the following command format:

```

[root@exadb-node1 ~]# /u01/app/grid/xag/bin/agctl add goldengate
<instance_name> \
--gg_home /u02/app/oracle/goldengate/gg21c
\
--service_manager
\
--config_home /mnt/acfs_gg/deployments/ggsm01/etc/conf
\
--var_home /mnt/acfs_gg/deployments/ggsm01/var
\

```

```

--port 9100
--oracle_home /u02/app/oracle/goldengate/gg21c/lib/instantclient
--adminuser oggadmin
--user oracle
--group oinstall
--network 1 --ip <virtual_IP_address>
--filesystems ora.<acfs_name>.acfs
--db_services ora.<db_service_name>.svc
--use_local_services
--attribute START_TIMEOUT=60
--nodes <exadb-node1>,<exadb-node2>

```

Step 7.3 - Start the Oracle GoldenGate Deployment

Below are some example agctl commands used to manage the Oracle GoldenGate deployment with XAG.

1. As the grid OS user, run the following command to start the Oracle GoldenGate deployment:

```

[opc@exadb-node1 ~]$ sudo su - grid
[grid@exadb-node1 ~]$ agctl start goldengate <instance_name>

```

2. As the grid OS user, run the following command to check the status of the Oracle GoldenGate:

```

[grid@exadb-node1 ~]$ agctl status goldengate
Goldengate instance <instance_name> is running on exadb-node1

```

3. As the grid OS user, run the following command to view the configuration parameters for the Oracle GoldenGate resource:

```

[grid@exadb-node1 ~]$ agctl config goldengate <instance_name>
Instance name: <instance_name>
Application GoldenGate location is: /u02/app/oracle/goldengate/gg21c_MS
Goldengate MicroServices Architecture environment: yes
Goldengate Service Manager configuration directory:
/mnt/acfs_gg/deployments/ggsm01/etc/conf
Goldengate Service Manager var directory: /mnt/acfs_gg/deployments/
ggsm01/var
Service Manager Port: 9100
Goldengate Administration User: oggadmin
Autostart on DataGuard role transition to PRIMARY: no
Configured to run on Nodes: exadb-node1 exadb-node2
ORACLE_HOME location is: /u02/app/oracle/goldengate/gg21c/lib/instantclient
Database Services needed: ora.<db_unique_name>.<service_name>.svc

```

```
[use_local_services]  
File System resources needed: ora.datac1.acfs_gg.acfs  
Network: 1, IP:NN.NN.NN.NN, User:oracle, Group:oinstall
```

See [Oracle Grid Infrastructure Standalone Agents for Oracle Clusterware 11g Rel. 2, 12c, 18c and 19c](#) for more information about Oracle Grid Infrastructure Bundled Agent.

Task 8 - Configure NGINX Reverse Proxy

The Oracle GoldenGate reverse proxy feature allows a single point of contact for all of the Oracle GoldenGate Microservices associated with an Oracle GoldenGate deployment.

Without a reverse proxy, the Oracle GoldenGate deployment microservices are contacted using a URL consisting of a host name or IP address and separate port numbers, one for each of the services.

For example, to contact the Service Manager, you could use `http://gghub.example.com:9100`, then the Administration Server is `http://gghub.example.com:9101`, the second Service Manager may be accessed using `http://gghub.example.com:9110`, and so on.

When running Oracle GoldenGate in a High Availability (HA) configuration on Oracle Exadata Database Service with the Grid Infrastructure agent (XAG), there is a limitation preventing more than one deployment from being managed by a GoldenGate Service Manager. Because of this limitation, creating a separate virtual IP address (VIP) for each Service Manager and deployment pair is recommended. This way, the microservices can be accessed directly using the VIP.

With a reverse proxy, port numbers are not required to connect to the microservices because they are replaced with the deployment name and the host name's VIP. For example, to connect to the console with a web browser, use the URLs.

Service	URL
Service Manager	<code>https://localhost:localPort</code>
Administration Server	<code>https://localhost:localPort/instance_name/adminsrvr</code>
Distribution Server	<code>https://localhost:localPort/instance_name/distsrvr</code>
Performance Metric Server	<code>https://localhost:localPort/instance_name/pmsrvr</code>
Receiver Server	<code>https://localhost:localPort/instance_name/recvsrvr</code>

Note

To connect to Oracle GoldenGate in OCI, you must create a bastion and an SSH port forwarding session (see Step 6.1). After this, you can connect to the Oracle GoldenGate Services using `https://localhost:<localPort>`.

A reverse proxy is mandatory to ensure easy access to microservices and enhance security and manageability.

Follow the instructions to install and configure NGINX Reverse Proxy with an SSL connection and ensure all external communication is secure.

Note

When using CA Signed Certificates with NGINX, make sure the NGINX `ssl_certificate` parameter points to a certificate file that contains the certificates in the correct order of CA signed certificate, intermediate certificate, and root certificate.

Perform the following steps to complete this task:

- Step 8.1 - Install NGINX
- Step 8.2 - Configure NGINX Reverse Proxy
- Step 8.3 - Securing GoldenGate Microservices to Restrict Non-secure Direct Access
- Step 8.4 - Create a Clusterware Resource to Manage NGINX

Step 8.1 - Install NGINX Reverse Proxy Server

1. As the root OS user on all nodes, set up the YUM repository by creating the file `/etc/yum.repos.d/nginx.repo` with the following contents:

```
[opc@exadb-node1 ~]$ sudo su -
[root@exadb-node1 ~]# cat > /etc/yum.repos.d/nginx.repo <<EOF
[nginx-stable]
name=nginx stable repo
baseurl=http://nginx.org/packages/rhel/7/\$basearch/
gpgcheck=1
enabled=1
gpgkey=https://nginx.org/keys/nginx_signing.key
module_hotfixes=true
EOF
```

2. As the root OS user, run the following commands to install, enable, and start NGINX:

```
[root@exadb-node1 ~]# yum install -y python-requests python-urllib3 nginx
[root@exadb-node1 ~]# systemctl enable nginx
```

3. As the root OS user, disable the NGINX repository after the software has been installed:

```
[root@exadb-node1 ~]# yum-config-manager --disable nginx-stable
```

Step 8.2 - Configure NGINX Reverse Proxy

A separate reverse proxy configuration is required for each Oracle GoldenGate Home.

When running multiple Service Managers, the following instructions will provide configuration using a separate VIP for each Service Manager. NGINX uses the VIP to determine which Service Manager an HTTPS connection request is routed to.

An SSL certificate is required for clients to authenticate the server they connect to through NGINX. Contact your systems administrator to follow your corporate standards to create or obtain the server certificate before proceeding. A separate certificate is required for each VIP and Service Manager pair.

Note

The common name in the CA-signed certificate must match the target hostname/VIP used by NGINX.

Perform the following sub-steps to complete this step:

- Step 8.2.1 - Create the NGINX Configuration File
- Step 8.2.2 - Modify NGINX Configuration Files
- Step 8.2.3 - Install Server Certificates for NGINX
- Step 8.2.4 - Install the NGINX Configuration File
- Step 8.2.5 - Test the New NGINX Configuration
- Step 8.2.6 - Reload NGINX and the New Configuration
- Step 8.2.7 - Test GoldenGate Microservices Connectivity
- Step 8.2.8 - Distribute the GoldenGate NGINX Configuration Files

Step 8.2.1 - Create the NGINX Configuration File

You can configure Oracle GoldenGate Microservices Architecture to use a reverse proxy. Oracle GoldenGate Microservices Architecture includes a script called `ReverseProxySettings` that generates a configuration file for only the NGINX reverse proxy server.

The script requires the following parameters:

- The `--user` parameter should mirror the GoldenGate administrator account specified with the initial deployment creation.
- The GoldenGate administrator password will be prompted.
- The reverse proxy port number specified by the `--port` parameter should be the default HTTPS port number (443) unless you are running multiple GoldenGate Service Managers using the same `--host`. In this case, specify an HTTPS port number that does not conflict with previous Service Manager reverse proxy configurations. For example, if you are running two Service Managers using the same hostname/VIP, the first reverse proxy configuration is created with `--port 443 --host hostvip01`, and the second is created with `--port 444 --host hostvip01`.

If you are using separate hostnames/VIPs, the two Service Manager reverse proxy configurations would be created with `--port 443 --host hostvip01` and `--port 443 --host hostvip02`.
- Lastly, the HTTP port number (9100) should match the Service Manager port number specified during the deployment creation.

Repeat this step for each additional GoldenGate Service Manager.

As the `oracle` OS user, use the following command to create the Oracle GoldenGate NGINX configuration file:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ export OGG_HOME=/u02/app/oracle/goldengate/gg21c
[oracle@exadb-node1 ~]$ export PATH=$PATH:$OGG_HOME/bin
[oracle@exadb-node1 ~]$ cd /u02/app_acfs/goldengate
[oracle@exadb-node1 ~]$ $OGG_HOME/lib/utl/reverseproxy/ReverseProxySettings
--user oggadmin --port 443 --output ogg_<instance_name>.conf http://
```



```
localhost:9100
--host <VIP hostname/IP>
Password: <oggadmin_password>
```

Step 8.2.2 - Modify NGINX Configuration Files

When multiple GoldenGate Service Managers are configured to use their IP/VIPs with the same HTTPS 443 port, some small changes are required to the NGINX reverse proxy configuration files generated in the previous step.

With all Service Managers sharing the same port number, they are independently accessed using their VIP/IP specified by the --host parameter.

1. As the oracle OS user, determine the deployment name managed by this Service Manager. If not already known, the deployment name is listed in the reverse proxy configuration file:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ cd /u02/app_acfs/goldengate
[oracle@exadb-node1 ~]$ grep "Upstream Servers" ogg_<instance_name>.conf
## Upstream Servers for Deployment '<instance_name>'
```

In this example, the deployment is called SOURCE.

2. As the oracle OS user, change all occurrences of _ServiceManager by prepending the deployment name before the underscore:

```
$ sed -i 's/_ServiceManager/<instance_name>_ServiceManager/'
ogg_<instance_name>.conf
```

Step 8.2.3 - Install Server Certificates for NGINX

1. As the root OS user, copy the server certificates and key files in the /etc/nginx/ssl directory, owned by root with file permissions 400 (-r-----):

```
[opc@exadb-node1 ~]$ sudo su -
[root@exadb-node1 ~]# mkdir /etc/nginx/ssl
[root@exadb-node1 ~]# chmod 400 /etc/nginx/ssl
```

2. As the root OS user, set the correct filenames for the certificate and key files for each reverse proxy configuration file generated in Step 8.2.1:

```
[oracle@exadb-node1 ~]$ vi /u02/app_acfs/goldengate/
ogg_<instance_name>.conf

# Before
ssl_certificate      /etc/nginx/ogg.pem;
ssl_certificate_key  /etc/nginx/ogg.pem;

# After
ssl_certificate      /etc/nginx/ssl/server.chained.crt;
ssl_certificate_key  /etc/nginx/ssl/server.key;
```

When using CA-signed certificates, the certificate named with the ssl_certificate NGINX parameter must include the 1) CA signed, 2) intermediate, and 3) root certificates in a single file. The order is significant; otherwise, NGINX fails to start and displays the error message:

(SSL: error:0B080074:x509 certificate routines: X509_check_private_key:key values mismatch)

The root and intermediate certificates can be downloaded from the CA-signed certificate provider.

The SSL certificate single file can be generated using the following example command:

```
[root@exadb-node1 ~]# cat  
CA_signed_cert.crt intermediate.crt root.crt > server.chained.crt
```

The `ssl_certificate_key` file is generated when creating the Certificate Signing Request (CSR), which is required when requesting a CA-signed certificate.

Step 8.2.4 - Install the NGINX Configuration File

As the root OS user, copy the deployment configuration file (or files if multiple files were created in Step 8.2.1) to `/etc/nginx/conf.d` directory:

```
[root@exadb-node1 ~]# mv /u02/app_acfs/goldengate/ogg_<instance_name>.conf  
/etc/nginx/conf.d
```

Step 8.2.5 - Test the New NGINX Configuration

As the root OS user, validate the NGINX configuration file.

If there are errors in the file, they will be reported with the following command:

```
[root@exadb-node1 ~]# nginx -t  
  
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok  
nginx: configuration file /etc/nginx/nginxconf test is successful
```

Step 8.2.6 - Reload NGINX and the New Configuration

As the root OS user, restart NGINX to load the new configuration:

```
[root@exadb-node1 ~]# systemctl restart nginx
```

Step 8.2.7 - Test GoldenGate Microservices Connectivity

1. As the root OS user, create a curl configuration file (`access.cfg`) that contains the deployment username and password:

```
[root@exadb-node1 ~]# vi access.cfg  
user = "oggadmin:<password>"
```

2. As the root OS user, query the health of the deployments using the following command:

```
[root@exadb-node1 ~]# curl -svf  
-K access.cfg https://<VIP hostname/IP>:<port>/services/v2/config/health  
-XGET && echo -e "\n*** Success"
```

Sample output:

```
{ "$schema": "api:standardResponse", "links":
[{"rel": "canonical", "href": "https://gg-prmy-vip1/services/v2/config/
health",
"mediaType": "application/json"},
{"rel": "self", "href": "https://gg-prmy-vip1/services/v2/config/health",
"mediaType": "application/json"}, {"rel": "describedby",
"href": "https://gg-prmy-vip1/services/ServiceManager/v2/metadata-catalog/
health",
"mediaType": "application/schema+json"}], "messages": [],
"response": { "$schema": "ogg:health", "deploymentName": "ServiceManager",
"serviceName": "ServiceManager", "started": "2021-12-09T23:33:03.425Z", "health
y": true,
"criticalResources":
[{"deploymentName": "SOURCE", "name": "adminsrvr", "type": "service",
"status": "running", "healthy": true},
{"deploymentName": "SOURCE", "name": "distsrvr",
"type": "service", "status": "running", "healthy": true},
{"deploymentName": "SOURCE", "name": "recvsrvr", "type": "service", "status": "run
ning",
"healthy": true}]}]}
*** Success ***
```

3. As the root OS user, remove the curl configuration file (access.cfg) that contains the deployment username and password:

```
[root@exadb-node1 ~]# rm access.cfg
rm: remove regular file 'access.cfg'? y
```

Step 8.2.8 - Distribute the GoldenGate NGINX Configuration Files

When all of the reverse proxy configuration files have been created for the GoldenGate Service Managers, they must be copied to all the database nodes.

1. As the opc OS user, distribute the NGINX configuration files to all database nodes:

```
[opc@exadb-node1 ~]$ sudo tar fczP nginx_conf.tar
/etc/nginx/conf.d/ /etc/nginx/ssl/
[opc@exadb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc -d /tmp
-f nginx_conf.tar
[opc@exadb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo tar
fzxP
/tmp/nginx_conf.tar
```

2. As the opc OS user, test the new NGINX configuration on all nodes the new configuration files were copied to:

```
[opc@exadb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo nginx -
t

exadb-node1: nginx: the configuration file /etc/nginx/nginx.conf syntax is
ok
exadb-node1: nginx: configuration file /etc/nginx/nginx.conf test is
successful
exadb-node2: nginx: the configuration file /etc/nginx/nginx.conf syntax is
ok
```

```
exadb-node2: nginx: configuration file /etc/nginx/nginx.conf test is
successful
```

3. As the opc OS user, restart NGINX to load the new configuration on all nodes:

```
[opc@exadb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo
systemctl
restart nginx
```

Step 8.3 - Securing GoldenGate Microservices to Restrict Non-secure Direct Access

After configuring the NGINX reverse proxy with an unsecured Oracle GoldenGate Microservices deployment, the microservices can continue accessing HTTP (non-secure) using the configured microservices port numbers.

For example, the following non-secure URL could be used to access the Administration Server: `http://<vip-name>:9101`.

Oracle GoldenGate Microservices' default behavior for each server (Service Manager, adminserver, pmsrvr, distsrvr, and recsrvr) is to listen using a configured port number on all network interfaces. This is undesirable for more secure installations, where direct access using HTTP to the microservices needs to be disabled and only permitted using NGINX HTTPS.

Use the following commands to alter the Service Manager and deployment services listener address to use only the localhost address. Access to the Oracle GoldenGate Microservices will only be permitted from the localhost, and any access outside of the localhost will only succeed using the NGINX HTTPS port.

Step 8.3.1 - Stop the Service Manager

As the grid OS user, stop the service manager:

```
[opc@exadb-node1 ~]$ sudo su - grid
[grid@exadb-node1 ~]$ agctl stop goldengate <instance_name>
[grid@exadb-node1 ~]$ agctl status goldengate
Goldengate instance '<instance_name>' is not running
```

Step 8.3.2 - Modify the Service Manager Listener Address

As the oracle OS user, modify the listener address with the following commands.

Use the correct port number for the Service Manager being altered. The server will fail to start, ignore the error, and proceed with the next step:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ export OGG_HOME=/u02/app/oracle/goldengate/gg21c
[oracle@exadb-node1 ~]$ export
OGG_VAR_HOME=<acfs or dbfs mount point>/deployments/ggsm01/var
[oracle@exadb-node1 ~]$ export OGG_ETC_HOME=<acfs or
dbfs mount point>/deployments/ggsm01/etc
[oracle@exadb-node1 ~]$ $OGG_HOME/bin/ServiceManager
--prop=/config/network/serviceListeningPort
--value='{"port":9100,"address":"127.0.0.1"}'
--type=array --persist --exit
[oracle@exadb-node1 ~]$
```

Step 8.3.3 - Restart the Service Manager and Deployment

As the grid OS user, restart the Service Manager and deployment:

```
[opc@exadb-node1 ~]$ sudo su - grid
[grid@exadb-node1 ~]$ agctl start goldengate <instance_name>
[grid@exadb-node1 ~]$ agctl status goldengate
Goldengate instance '<instance_name>' is running on exadb-node1
```

Step 8.3.4 - Modify the GoldenGate Microservices listener address

As the oracle OS user, modify all the GoldenGate microservices (adminsrvr, pmsrvr, distsrvr, recvsrvr) listening address to localhost for the deployments managed by the Service Manager using the following command:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ cd /u02/app_acfs/goldengate
[oracle@exadb-node1 ~]$ chmod u+x secureServices.py
[oracle@exadb-node1 ~]$ ./secureServices.py http://localhost:9100 --user
oggadmin
Password for 'oggadmin': <oggadmin_password>

*** Securing deployment - ogg_deployment

Current value of "/network/serviceListeningPort" for "<instance_name>/
adminsrvr" is
{
  "address": "127.0.0.1",
  "port": 9101
}
Current value of "/network/serviceListeningPort" for "<instance_name>/
distsrvr" is
{
  "address": "127.0.0.1",
  "port": 9102
}
Current value of "/network/serviceListeningPort" for "<instance_name>/pmsrvr"
is
{
  "address": "127.0.0.1",
  "port": 9104
}
Current value of "/network/serviceListeningPort" for "<instance_name>/
recvsrvr" is
{
  "address": "127.0.0.1",
  "port": 9103
}
}
```

Note

To modify a single deployment (adminsrvr, pmsrvr, distsrvr, recvsrvr), add the flag `--deployment instance_name`

Step 8.3.5 - Remove NGINX default.conf Configuration File

As the opc OS user, remove the default configuration file (default.conf) created in /etc/nginx/conf.d:

```
[opc@exadb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo rm
-f /etc/nginx/conf.d/default.conf
[opc@exadb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo nginx -s
reload
```

Step 8.4 - Create a Clusterware Resource to Manage NGINX

Oracle Clusterware needs to have control over starting the NGINX reverse proxy so that it can be started automatically before the Oracle GoldenGate deployments are started.

1. As the grid OS user, use the following command to get the network CRS resource name required to create the NGINX resource with a dependency on the underlying network CRS resource:

```
[opc@exadb-node1 ~]$ sudo su - grid
[grid@exadb-node1 ~]$ crsctl stat res -w "TYPE == ora.network.type"|grep
NAME
```

```
NAME=ora.net1.network
```

2. As the root OS user, use the following example command to create a Clusterware resource to manage NGINX. Replace the HOSTING_MEMBERS and CARDINALITY to match your environment:

```
[opc@exadb-node1 ~]$ sudo su -

[root@exadb-node1 ~]# $(grep ^crs_home /etc/oracle/olr.loc | cut -d= -
f2)/bin/crsctl
add resource nginx -type generic_application -attr
"ACL='owner:root:rwx,pgrp:root:rwx,other::r--,group:oinstall:r-
x,user:oracle:rwx',
EXECUTABLE_NAMES=nginx,START_PROGRAM='/bin/systemctl start
-f nginx',STOP_PROGRAM='/bin/systemctl stop
-f nginx',CHECK_PROGRAMS='/bin/systemctl status nginx',
,START_DEPENDENCIES='hard(ora.net1.network) pullup(ora.net1.network)',
STOP_DEPENDENCIES='hard(intermediate:ora.net1.network)',
RESTART_ATTEMPTS=0, HOSTING_MEMBERS='<exadb-node1, exadb-node2>',
CARDINALITY=2"
```

The NGINX resource created in this example will run on the named database nodes simultaneously, specified by HOSTING_MEMBERS. This is recommended when multiple GoldenGate Service Manager deployments are configured and can independently move between database nodes.

Once the NGINX Clusterware resource is created, the GoldenGate XAG resources need to be altered so that NGINX must be started before the GoldenGate deployments are started.

3. As the root OS user, modify the XAG resources using the following example commands.

```
# Determine the current --filesystems parameter:
[opc@exadb-node1 ~]$ sudo su - grid
[grid@exadb-node1 ~]$ agctl config goldengate <instance_name> |grep "File
System"
File System resources needed: <file_system_resource_name>
```

```
# Modify the --filesystems parameter:
[opc@exadb-node1 ~]$ sudo su -
[root@exadb-node1 ~]# /u01/app/grid/xag/bin/agctl modify goldengate
<instance_name>
--filesystems <file_system_resource_name>,nginx
```

4. Repeat the above commands for each XAG GoldenGate registration relying on NGINX.

Task 9 - Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections

To provide local database connections for the Oracle GoldenGate processes when switching between Oracle RAC nodes, create a TNS alias on **all** of the Oracle RAC nodes where Oracle GoldenGate may be started.

Create the TNS alias in the `tnsnames.ora` file in the `TNS_ADMIN` directory specified in the deployment creation.

Perform the following steps to complete this task:

- Step 9.1 - Create the TNS Alias Definitions
- Step 9.2 - Create the Database Credentials

Step 9.1 - Create the TNS Alias Definitions

If the source database is a multitenant database, two TNS alias entries are required, one for the container database (CDB) and one for the pluggable database (PDB) that is being replicated.

For a target Multitenant database, the TNS alias connects the PDB to where replicated data is being applied. The pluggable database `SERVICE_NAME` should be set to the database service created in an earlier step (Step 2.3: Create the Database Services).

1. As the oracle OS user, find the database domain name:

```
[opc@exadb-node1]$ sudo su - oracle
[oracle@exadb-node1]$ source DB_NAME.env
[oracle@exadb-node1]$ sqlplus / as sysdba
SQL> show parameter db_domain
```

NAME	TYPE	VALUE
db_domain	string	<db_domain_name>

2. As the oracle OS user on the first database node, follow the steps to create the TNS alias definitions and distribute them to all database nodes:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ dcli -l oracle -g ~/dbs_group mkdir -p /u02/app/
oracle/goldengate/network/admin
[oracle@exadb-node1 ~]$ vi /u02/app/oracle/goldengate/network/admin/
tnsnames.ora
```

```
OGGSRV_CDB =
```

```

        (DESCRIPTION =
          (ADDRESS = (PROTOCOL=IPC)(KEY=LISTENER))
          (CONNECT_DATA =
            (SERVICE_NAME = <cdb_service_name>.<db_domain_name>)
          )
        )
OGGSRV_<PDB_NAME> =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL=IPC)(KEY=LISTENER))
    (CONNECT_DATA =

      (SERVICE_NAME = <pdb_service_name>.<db_domain_name>)
    )
  )
[oracle@exadb-node1 ~]$ /usr/local/bin/dcli -l oracle -g ~/dbs_group -
f /u02/app/oracle/goldengate/network/admin/*.ora -d /u02/app/oracle/
goldengate/network/admin

```

Note

When the `tnsnames.ora` or `sqlnet.ora` located in the `TNS_ADMIN` directory for the Oracle GoldenGate deployment are modified, the deployment needs to be restarted to pick up the changes.

Step 9.2 - Create the Database Credentials

With the Oracle GoldenGate deployment created, use the Oracle GoldenGate Administration Service home page to create the database credentials using the above TNS alias names.

As the `oggadmin` user, create the database credentials:

1. Log in into the Administration Service: `https://localhost:<localPort>/<instance_name>/adminsrvr`
2. Click on **Configuration** under Administration Service.
3. Click the plus button to **Add Credentials**.
4. Add the required information.
If the source database is a Multitenant Database, create database credentials for the CDB and PDB. If the target database is a Multitenant Database, create a single credential for the PDB.

Task 10 - Create a New Profile

Create a new profile to automatically start the Extract and Replicat processes when the Oracle GoldenGate Administration Server is started.

Then, restart if any Extract or Replicat processes are abandoned. With GoldenGate Microservices, auto start and restart is managed by Profiles.

Using the Oracle GoldenGate Administration Server GUI, create a new profile that can be assigned to each of the Oracle GoldenGate processes:

1. Log in to the **Administration Service** on the Source and Target GoldenGate.
2. Click on **Profile** under Administration Service.

3. Click the plus (+) sign next to **Profiles** on the **Managed Process Settings** home page. The **Add Profile** page is displayed.
4. Enter the details.
5. Click **Submit**.

Task 11 - Configure Oracle GoldenGate Processes

When creating Extract, Distribution Paths, and Replicat processes with Oracle GoldenGate Microservices Architecture, all files that need to be shared between Oracle RAC nodes are already shared with the deployment files stored on a shared file system (DBFS or ACFS).

Listed below are the essential configuration details recommended for running Oracle GoldenGate Microservices on Oracle RAC for Extract, Distribution Paths, and Replicat processes.

Perform the following steps to complete this task:

- Step 11.1 - Extract Configuration
- Step 11.2 - (DBFS only) Place the Temporary Cache Files on the Shared Storage
- Step 11.3 - Distribution Path Configuration
- Step 11.4 - Replicat Configuration

Step 11.1 - Extract Configuration

When creating an Extract using the Oracle GoldenGate Administration Service GUI interface, leave the `Trail SubDirectory` parameter blank so that the trail files are automatically created in the deployment directories stored on the shared file system. The default location for trail files is the `/<deployment directory>/var/lib/data` directory.

① Note

To capture from a multitenant database, you must use an Extract configured at the root level using a `c##` account. To apply data into a multitenant database, a separate Replicat is needed for each PDB because a Replicat connects at the PDB level and doesn't have access to objects outside of that PDB

Create the database credentials:

1. Log in to the Oracle GoldenGate **Administration Server** in the Source Oracle GoldenGate.
2. Click in **Overview** under **Administration Service**.
3. Click the plus button to **Add Extract**.
4. Select **Integrated Extract**.
5. Add the required information.
6. Click **Next**.
7. If using **CDB Root Capture from PDB**, add the `SOURCATALOG` parameter with the PDB Name.
8. Click **Create**.

Step 11.2 - (DBFS only) Place the Temporary Cache Files on the Shared Storage

If you are using DBFS for shared storage, and the deployment var/temp directory was moved to local storage as described in [Task 5 - Create the Oracle GoldenGate Deployment](#), it is recommended that you use the Extract CACHEMGR parameter to place the temporary cache files on the shared storage.

1. As the oracle OS user, create a new directory under the DBFS deployment mount point.:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ mkdir
/mnt/dbfs/goldengate/deployments/<instance_name>/temp_cache
```

2. Set the Extract parameter to the new directory:

```
CACHEMGR CACHEDIRECTORY
/mnt/dbfs/goldengate/deployments/<instance_name>/temp_cache
```

More instructions about creating an Extract process are available in [Using Oracle GoldenGate Classic Architecture with Oracle Database](#).

Step 11.3 - Distribution Path Configuration

When using Oracle GoldenGate Distribution paths with the NGINX Reverse Proxy, additional steps must be carried out to ensure the path client and server certificates are configured.

More instructions about creating distribution paths are available in [Oracle GoldenGate Microservices Documentation](#). A step-by-step example is in the following video, "[Connect an on-premises Oracle GoldenGate to OCI GoldenGate using NGINX.](#)" to correctly configure the certificates.

Perform the following sub-steps to complete this step:

- Step 11.3.1 - Download the Target Server's Root Certificate, and then upload it to the source Oracle GoldenGate
- Step 11.3.2 - Create a user in the Target Deployment for the Source Oracle GoldenGate to use
- Step 11.3.3 - Create a Credential in the Source Oracle GoldenGate
- Step 11.3.4 - Create a Distribution Path on the Source Oracle GoldenGate to the Target Deployment
- Step 11.3.5 - Verify the Connection in the Target Deployment Console Receiver Service

Step 11.3.1 - Download the Target Server's Root Certificate, and then upload it to the source Oracle GoldenGate

Download the target deployment server's root certificate and add the CA certificate to the source deployment Service Manager.

1. Log in to the **Administration Service** on the Target GoldenGate.
2. Follow "Step 2 - Download the target server's root certificate" in the video "[Connect an on-premises Oracle GoldenGate to OCI GoldenGate using NGINX.](#)"

Step 11.3.2 - Create a user in the Target Deployment for the Source Oracle GoldenGate to use

Create a user in the target deployment for the distribution path to connect to:

1. Log in to the **Administration Service** on the Target GoldenGate.
2. Click **Administrator** under Administration Service.

3. Click the plus (+) sign next to **Users**.
4. Enter the details.

Step 11.3.3 - Create a Credential in the Source Oracle GoldenGate

Create a credential in the source deployment connecting the target deployment with the user created in the previous step. For example, a domain of OP2C and an alias of WSSNET.

1. Log in to the **Administration Service** on the Source Oracle GoldenGate.
2. Click **Configuration** under Administration Service.
3. Click the plus (+) sign next to **Credentials** on the Database home page. The Add Credentials page is displayed.
4. Enter the details.

Step 11.3.4 - Create a Distribution Path on the Source Oracle GoldenGate to the Target Deployment

A path is created to send trail files from the Distribution Server to the Receiver Server. You can create a path from the Distribution Service.

To add a path for the source deployment:

1. Log in to the **Distribution Service** on the Source Oracle Goldengate.
2. Click the plus (+) sign next to **Path** on the Distribution Service home page. The Add Path page is displayed.
3. Enter the details as follows:

Option	Description
Path Name	Select a name for the path.
Source: <i>Trail Name</i>	Select the Extract name from the drop-down list, which populates the trail name automatically. If it doesn't, enter the trail name you provided while adding the Extract.
Generated Source URI	Specify localhost for the server's name; this allows the distribution path to be started on any of the Oracle RAC nodes.
Target Authentication Method	Use UserID Alias
Target	Set the Target transfer protocol to wss (secure web socket). Set the Target Host to the target hostname/VIP that will be used for connecting to the target system along with the Port Number that NGINX was configured with (default is 443).
Domain	Set the Domain to the credential domain created above in Step 11.3.3, for example, OP2C.
Alias	The Alias is set to the credential alias wssnet, also created in Step 11.3.3.
Auto Restart Options	Set the distribution path to restart when the Distribution Server starts automatically. This is required, so that manual intervention is not required after an Oracle RAC node relocation of the Distribution Server. It is recommended to set the number of Retries to 10. Set the Delay , which is the time in minutes to pause between restart attempts, to 1.

4. Click **Create Path**.

5. From the Action Menu, click **Start**.
6. Verify that the Distribution Service is running.

Step 11.3.5 - Verify the Connection in the Target Deployment Console Receiver Service

1. Log in to the **Administration Service** on the Target Deployment Console.
2. Click on **Receiver Service**.

Step 11.4 - Replicat Configuration

The Replicat process receives the trail data and applies it to the database.

Perform the following sub-steps to complete this step:

- Step 11.4.1 - Create the Checkpoint Table
- Step 11.4.2 - Add a Replicat

Step 11.4.1 - Create the Checkpoint Table

The checkpoint table is a required component for Oracle GoldenGate Replicat processes. After connecting to the database from the Credentials page of the Administration Service, you can create the checkpoint table.

Create the checkpoint table in the target deployment:

1. Log in to the **Administration Service** on the Target GoldenGate.
2. Click **Configuration** under Administration Service.
3. Click **Database** and Connect to the target database or PDB.
4. Click the plus (+) sign next to **Checkpoint**. The Add Checkpoint page is displayed.
5. Enter the details.

See [About Checkpoint Table](#) for more information about the checkpoint table.

Step 11.4.2 - Add a Replicat

After you set up your database connections and verified them, you can add a Replicat for the deployment by following these steps:

1. Log in to the **Administration Service** on the Target GoldenGate.
2. Click the plus (+) sign next to **Replicats** on the Administration Service home page. The Add Replicat page is displayed.
3. Select a Replicat type and click **Next**.
4. Enter the details as follows:

Option	Description
Process Name	The name of the Replicat process
Credential Domain	Credential domain created in Step 9.2. In our example is GoldenGate
Credential Alias	Credential alias created in Step 9.2. Our example is Target_PDB
Source	Select the source to use Trail.
Trail Name	A two-character trail name.
Checkpoint Table	Set the use of an existing checkpoint table.

5. Click **Create Path**.

6. From the Action Menu, click **Start**.
7. Verify that the Replicat is running.

Cloud MAA Platinum: Oracle GoldenGate Microservices Architecture Integrated with Active Data Guard

The combination and integration of Oracle GoldenGate Microservices and Oracle Data Guard enables you to achieve an MAA Platinum service-level configuration that achieves zero or near zero downtime for all planned and unplanned outages.

See the following topics:

- [Overview](#)
- [Task 1 - Before You Begin](#)
- [Task 2 - Configure the Oracle Database for GoldenGate](#)
- [Task 3 - Configure Oracle Database File System](#)
- [Task 4 - Install Oracle GoldenGate](#)
- [Task 5 - Create Oracle GoldenGate Deployment Directories](#)
- [Task 6 - Network Configuration](#)
- [Task 7 - Configure Standby NGINX Reverse Proxy](#)
- [Task 8 - Configure Oracle Grid Infrastructure Agent](#)
- [Task 9 - Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections](#)
- [Task 10 - Configure Oracle GoldenGate Processes](#)
- [Example Distribution Path Target Change Script](#)

Overview

With these configuration and operational best practices, Oracle GoldenGate can be configured to work seamlessly with Oracle Data Guard after any zero data loss or data loss role transition.

By using Database File System (DBFS) as the file system for the Oracle GoldenGate Microservices deployment files, Oracle GoldenGate Extract, Distribution Paths, and Replicat processes continue to stay synchronized with the database after a role transition.

Implement these best practices for configuring Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D), or Oracle Exadata Database Service on Cloud@Customer (ExaDB-C@C), to work seamlessly with Oracle Data Guard, using Oracle Real Application Clusters (Oracle RAC), Oracle Clusterware, and Oracle Database File System (DBFS).

These best practices enable Oracle GoldenGate Microservices replication using a database that is protected by a Data Guard standby, to work following an Oracle Data Guard role transition transparently and seamlessly, no matter which Data Guard protection mode is configured (Maximum Performance, Maximum Availability, or Maximum Protection).

There are several key software requirements:

- Oracle Grid Infrastructure 19c or later
Oracle Grid Infrastructure provides the necessary components needed to manage high availability for any business-critical applications. Using Oracle Clusterware (a component of Oracle Grid Infrastructure) network, database, and Oracle GoldenGate resources can be managed to provide availability in the event of a failure.
- Oracle Grid Infrastructure Agent version 10.2 or later
The Oracle Grid Infrastructure Agent leverages the Oracle Grid Infrastructure components to provide integration between Oracle GoldenGate and its dependent resources, such as the database, network, and file system. The agent also integrates Oracle GoldenGate with Oracle Data Guard so that Oracle GoldenGate is restarted on the new primary database following a role transition.
- Oracle Database 19c or later
Refer to [My Oracle Support note 2193391.1](#) for a full list of recommended Oracle Database patches when using Oracle GoldenGate.
- Oracle GoldenGate Microservices version 21c or later
Oracle GoldenGate 21c introduces unified build support so a single software installation supports capturing and applying replicated data to multiple major Oracle Database versions (11g Release 2 to 21c). This is possible because an Oracle GoldenGate installation includes the required Oracle Database client libraries without requiring a separate database ORACLE_HOME installation.
- Oracle DBFS to protect and replicate critical Oracle GoldenGate files
The Oracle Database File System (DBFS) is the only MAA-validated and recommended file system for an Oracle Data Guard and Oracle GoldenGate configuration, because it allows the storage of the required Oracle GoldenGate files, such as the checkpoint and trail files, to be located inside the same database that is protected with Oracle Data Guard, ensuring consistency between the Oracle GoldenGate files and the database in a seamless fashion.

Task 1 - Before You Begin

To get started, complete the following prerequisites:

- Procure Oracle Exadata Database Service on Dedicated Infrastructure or Cloud@Customer for the Oracle GoldenGate deployment.

You can deploy Oracle GoldenGate with an existing ExaDB-D or ExaDB-C@C system or launch a new system, according to your business needs. For instructions on launching and managing an ExaDB-D system, see [Oracle Exadata Database Service on Dedicated Infrastructure](#) or for ExaDB-C@C see [Oracle Exadata Database Service on Cloud@Customer](#).
- Have Oracle GoldenGate configured as detailed in [Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices](#).

DBFS is required for critical Oracle GoldenGate files when integrating with Data Guard.
- The Oracle Data Guard standby database should also be configured and operational before continuing.

For more information about Oracle Data Guard see Getting Started with Oracle Data Guard.
- A secure deployment involves making RESTful API calls and conveying trail data between the Distribution Server and Receiver Server, over SSL/TLS. You can use your own existing

business certificate from your Certificate Authority (CA) or you might create your own certificates.

Contact your systems administrator to follow your corporate standards to create or obtain the server certificate before proceeding. A separate certificate is required for each VIP and Service Manager pair.

Task 2 - Configure the Oracle Database for GoldenGate

Perform the following steps to complete this task:

- Step 2.1 - Configure the Standby Database for Oracle GoldenGate
- Step 2.2 - Modify the Primary Database Service
- Step 2.3 - Create the Standby Database Service

Step 2.1 - Configure the Standby Database for Oracle GoldenGate

The standby database initialization parameters should match those of the primary database, as specified in [Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices](#).

This includes the following parameters:

- `ENABLE_GOLDENGATE_REPLICATION=TRUE`
- For Oracle GoldenGate source databases, enable `FORCE LOGGING` mode and enable minimal supplemental logging.
- If an Oracle GoldenGate source database or running integrated Replicat (parallel or non-parallel), configure the `STREAMS_POOL_SIZE`.

Step 2.2 - Modify the Primary Database Service

On the primary database server, validate the existing database services that were created as part of the original Oracle GoldenGate on Oracle Exadata Database Service configuration.

By default, the service role is defined as `PRIMARY`, so that the service is only started when the database becomes the Data Guard primary database role after a role transition.

As the oracle OS user on the primary system, validate the service role using the following command:

```
[opc@exapri-node1 ~]$ sudo su - oracle
[oracle@exapri-node1 ~]$ source <db_name>.env
[oracle@exapri-node1 ~]$ srvctl config service -db $ORACLE_UNQNAME |
egrep 'Service name|role|Pluggable database name'
Service name: <CDB_SERVICE_NAME>
Service role: PRIMARY
Pluggable database name:
Service name: <PDB_SERVICE_NAME>
Service role: PRIMARY
Pluggable database name: <PDB_NAME>
```

If the role is not `PRIMARY`, modify the service using the following command:

```
[oracle@exapri-node1 ~]$ srvctl modify service -db $ORACLE_UNQNAME
-service <service_name> -role PRIMARY
```


If your database is part of a multitenant environment, remember to modify both the multitenant container database (CDB) and pluggable database (PDB) services.

Step 2.3 - Create the Standby Database Service

On the standby Oracle Exadata Database Service, a database service is required for the standby database so that the Oracle Grid Infrastructure Agent will automatically start the Oracle GoldenGate deployment when the database is opened with the primary role.

When a source database is in a multitenant environment, a separate service is required for the root container database (CDB) and the pluggable database (PDB) that contains the schema being replicated. For a multitenant environment target database, a single service is required for the PDB.

Create the service in the standby database as it was created on the primary database. It is recommended that you use the same service name as was specified on the primary system. The service must be created as a singleton service, using the `-preferred` option, because the application Virtual IP address (VIP), DBFS, and Oracle GoldenGate will run on the system node where the service is running.

1. As the oracle OS user, get the Fully Qualified Domain Name (FQDN):

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ hostname -f
exadb-node1.<FQDN>
```

2. As the oracle OS user on the standby system, create the service using the following command:

```
[opc@exastb-node1 ~]$ sudo su - oracle
[oracle@exastb-node1 ~]$ source <db_name>.env
[oracle@exastb-node1 ~]$ srvctl add service -db $ORACLE_UNQNAME
    -service <CDB_SERVICE_NAME>.<FQDN> -preferred <SID1> -available <SID2>
    -role PRIMARY
[oracle@exastb-node1 ~]$ srvctl add service -db $ORACLE_UNQNAME
    -service <PDB_SERVICE_NAME>.<FQDN> -preferred <SID1> -available <SID2>
    -pdb <PDB name> -role PRIMARY
```

Task 3 - Configure Oracle Database File System

The Database File System (DBFS) is the only recommended solution when configuring Oracle GoldenGate with Oracle Data Guard.

The DBFS user, tablespace, and file system in the database was previously created in the primary database, as detailed in [Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices](#).

Perform the following steps to complete this task:

- Step 3.1 - Configuring DBFS on Oracle Exadata Database Service
- Step 3.2 - (PDB Only) Create an Entry in TNSNAMES
- Step 3.3 - Copy and Edit the mount-dbfs Scripts from the Primary System
- Step 3.4 - Register the DBFS Resource with Oracle Clusterware

Step 3.1 - Configuring DBFS on Oracle Exadata Database Service

1. As the opc OS user on the standby system, add the grid user to the fuse group:

```
[opc@exastb-node1 ~]$ sudo -u grid
$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/bin/olsnodes > ~/
dbs_group
[opc@exadb-node1 ~]$ dcli -g ~/dbs_group -l opc sudo usermod -a -G fuse
grid
```

2. As the opc OS user on the standby system, validate that the file /etc/fuse.conf exists and contains the user_allow_other option:

```
[opc@exastb-node1 ~]$ cat /etc/fuse.conf
# mount_max = 1000
user_allow_other
```

3. As the opc OS user on the standby system, skip this step if the option user_allow_other is already in the /etc/fuse.conf file. Otherwise run the following commands to add the option:

```
[opc@exastb-node1 ~]$ dcli -g ~/dbs_group -l opc "echo user_allow_other |
sudo tee -a /etc/fuse.conf"
```

4. As the opc OS user on the standby system, create an empty directory that will be used as the mount point for the DBFS filesystem.

Note

It is important that the mount point is identical as the one in the primary system, because the physical location of the Oracle GoldenGate deployment is included within the deployment configuration files.

```
[opc@exastb-node1 ~]$ dcli -g ~/dbs_group -l opc sudo mkdir -p /mnt/dbfs
```

5. As the opc OS user on the standby system, change ownership on the mount point directory so the grid OS user can access it:

```
[opc@exastb-node1 ~]$ dcli -g ~/dbs_group -l opc
sudo chown oracle:oinstall /mnt/dbfs
```

Step 3.2 - (PDB Only) Create an Entry in TNSNAMES

1. As the oracle OS user on the standby system, add a connect entry in \$TNS_ADMIN/tnsnames.ora file. Use the PDB service name created in Step 2.3:

```
[oracle@exadb-node1 ~]$ vi $TNS_ADMIN/tnsnames.ora
dbfs =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = IPC)(KEY=LISTENER))
    (CONNECT_DATA =
      (SERVICE_NAME = <PDB_SERVICE_NAME> )
    )
  )
```

2. As the oracle OS user, distribute the \$TNS_ADMIN/tnsnames.ora file to the rest of the nodes:

```
[oracle@exadb-node1 ~]$ /usr/local/bin/dcli -l oracle -g ~/dbs_group
-f $TNS_ADMIN/tnsnames.ora -d $TNS_ADMIN/
```

Step 3.3 - Copy and Edit the mount-dbfs Scripts from the Primary System

1. As the root OS user on the primary system, create a zip file with the files mount-dbfs.conf and mount-dbfs.sh:

```
[opc@exapri-node1 ~]$ sudo su -
[root@exapri-node1 ~]# zip -j /tmp/mount-dbfs.zip
$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/crs/script/mount-
dbfs.sh
/etc/oracle/mount-dbfs.conf
adding: mount-dbfs.sh (deflated 67%)
adding: mount-dbfs.conf (deflated 58%)
```

2. As the opc OS user on the standby system, copy the mount-dbfs.zip file from the primary system to the standby system:

```
[opc@exastb-node1 ~]$ scp exapri-node1.oracle.com:/tmp/mount-dbfs.zip /tmp
```

3. As the opc OS user on the standby system, unzip the mount-dbfs.zip file and edit the configuration file mount-dbfs.conf:

```
[opc@exastb-node1 ~]$ unzip /tmp/mount-dbfs.zip -d /tmp
Archive: /tmp/mount-dbfs.zip
  inflating: /tmp/mount-dbfs.sh
  inflating: /tmp/mount-dbfs.conf
[opc@exastb-node1 ~]$ vi /tmp/mount-dbfs.conf
```

It is recommended that you place them in the same directory as the primary system. You will need to modify the following parameters in the mount-dbfs.conf file to match the standby database:

- DBNAME
- TNS_ADMIN
- PDB_SERVICE

4. As the opc OS user on the standby system, copy mount-dbfs.conf to the directory /etc/oracle on database nodes and set proper permissions on it:

```
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc -d /tmp
-f /tmp/mount-dbfs.conf
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo
cp /tmp/mount-dbfs.conf /etc/oracle
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo
chown oracle:oinstall /etc/oracle/mount-dbfs.conf
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo
chmod 660 /etc/oracle/mount-dbfs.conf
```

- As the opc OS user on the standby system, copy mount-dbfs.sh to the directory \$GI_HOME/crs/script on database nodes and set proper permissions on it:

```
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo mkdir
$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/crs/script
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo chown
grid:oinstall $(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/crs/
script
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l grid
-d $(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/crs/script
-f /tmp/mount-dbfs.sh
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l grid chmod 770
$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/crs/script/mount-
dbfs.sh
```

Step 3.3 - Register the DBFS Resource with Oracle Clusterware

When registering the resource with Oracle Clusterware, be sure to create it as a cluster_resource. The reason for using cluster_resource is so the file system can only be mounted on a single node at one time, preventing mounting of DBFS from concurrent nodes creating the potential of concurrent file writes, causing file corruption problems.

If using Oracle Multitenant, make sure to use the service name for the same PDB that contains the DBFS repository as was created in the primary database.

- As the grid OS user on the standby system, find the resource name for the database service created in a previous step for the DBFS service dependency:

```
[opc@exastb-node1 ~]$ sudo su - grid
[grid@exastb-node1 ~]$ crsctl stat res |grep <PDB_NAME>
NAME=ora.<DB_UNIQUE_NAME>.<PDB_SERVICE_NAME>.svc
```

- As the oracle OS user on the standby system, register the Clusterware resource by executing the following script:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ vi add-dbfs-resource.sh
##### start script add-dbfs-resource.sh
#!/bin/bash
ACTION_SCRIPT=$(grep ^crs_home /etc/oracle/olr.loc | cut -d=
-f2)/crs/script/mount-dbfs.sh
RESNAME=dbfs_mount
DEPNAME=ora.<DB_UNIQUE_NAME>.<PDB_SERVICE_NAME>.svc
ORACLE_HOME=$(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)
PATH=$ORACLE_HOME/bin:$PATH
export PATH ORACLE_HOME
crsctl add resource $RESNAME \
-type cluster_resource \
-attr "ACTION_SCRIPT=$ACTION_SCRIPT, \
CHECK_INTERVAL=30,RESTART_ATTEMPTS=10, \
START_DEPENDENCIES='hard($DEPNAME)pullup($DEPNAME)',\
STOP_DEPENDENCIES='hard($DEPNAME)',\
SCRIPT_TIMEOUT=300"
##### end script add-dbfs-resource.sh
[oracle@exadb-node1 ~]$ sh add-dbfs-resource.sh
```

Note

After creating the \$RESNAME resource, in order to stop the \$DBNAME database when the \$RESNAME resource is ONLINE, you will have to specify the force flag when using `srvctl`.

For example: `srvctl stop database -d $ORACLE_UNQNAME -f`

Task 4 - Install Oracle GoldenGate

Install the Oracle GoldenGate software locally on all nodes in the Oracle Exadata Database Service configuration that will be part of the GoldenGate configuration.

Note

Make sure the installation directory is the identical on all nodes to match the primary system installation directory.

1. As the `opc` OS user on the standby system, copy the `oggcore.rsp` response file from the primary system to the standby system:

```
[opc@standby_node_1 ~]$ scp
primary_node_1:/u02/app_acfs/goldengate/oggcore.rsp
/u02/app_acfs/goldengate
```

2. On the standby system, follow “Step 4.2 - Install Oracle GoldenGate” as detailed in [Task 4 - Install Oracle GoldenGate](#).

Task 5 - Create Oracle GoldenGate Deployment Directories

The Oracle GoldenGate Service Manager and deployment were already created on the primary system, but certain directories and symbolic links need to be configured on the standby system nodes. These directories and symbolic links were created on the primary system, as detailed in [Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices](#).

1. As the `oracle` OS user on the primary system, determine the datastore directory:

```
[opc@exapri-node1 ~]$ sudo su - oracle
[oracle@exapri-node1 ~]$ grep RepoDatastorePath /mnt/dbfs/goldengate
/deployments/<instance_name>/var/log/pmsrvr.log|uniq
"RepoDatastorePath": "",
    "RepoDatastorePath": "/u02/app/oracle/goldengate/datastores/
<instance_name>",
```

2. As the `oracle` OS user on the standby system, create the directory on all database nodes:

```
[oracle@exastb-node1 ~]$ /usr/local/bin/dcli -l oracle -g ~/dbs_group mkdir
-p /u02/app/oracle/goldengate/datastores/<gg_deployment_name>
```

Create the Oracle GoldenGate deployment temp directory local storage to match the symbolic link created on the primary system.

3. As the oracle OS user on the primary system, determine the datastore directory:

```
[oracle@exapri-node1 ~]$ ls -l  
/mnt/dbfs/goldengate/deployments/<instance_name>/var |grep temp  
lrwxrwxrwx 1 oracle oinstall 49 Oct 3 10:20 temp ->  
/u02/app/oracle/goldengate/deployments/<instance_name>/temp
```

4. As the oracle OS user on the standby system, create the same directory on the standby database nodes:

```
[oracle@exastb-node1 ~]$ /usr/local/bin/dcli -l oracle -g ~/dbs_group mkdir  
-p /u02/app/oracle/goldengate/deployments/<instance_name>/temp
```

Task 6 - Network Configuration

On the standby system, follow the instructions in [Task 6 - Configure the Network](#) from Chapter [Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices](#).

Task 7 - Configure Standby NGINX Reverse Proxy

The Oracle GoldenGate reverse proxy feature allows a single point of contact for all of the Oracle GoldenGate microservices associated with an Oracle GoldenGate deployment.

Without reverse proxy, the GoldenGate deployment microservices are contacted using a URL consisting of a host name or IP address and separate port numbers, one for each of the services. Reverse proxy is mandatory to ensure easy access to microservices and provide enhanced security and manageability.

Perform the following steps to complete this task:

- Step 7.1 - Install NGINX Reverse Proxy
- Step 7.2 - Copy NGINX Configuration Files from the Primary System
- Step 7.3 - Install Server Certificates for NGINX
- Step 7.4 - Test and Reload the New NGINX Configuration
- Step 7.5 - Distribute the GoldenGate NGINX Configuration Files
- Step 7.6 - Create a Clusterware Resource to Manage NGINX

Step 7.1 - Install NGINX Reverse Proxy

On the standby system, follow the instructions in “Step 8.1 - Install NGINX Reverse Proxy Server” of [Task 8 - Configure NGINX Reverse Proxy](#) to install NGINX.

Step 7.2 - Copy NGINX Configuration Files from the Primary System

1. As the opc user on the standby system, copy the NGINX configuration file from the primary to the standby database system:

```
[opc@standby_node_1 ~]$ scp
primary_node_1.oracle.com:/etc/nginx/conf.d/ogg_<deployment_name>.conf
/tmp
```

2. As the root user on the standby system, copy the NGINX configuration file from the directory /tmp to the directory /etc/nginx/conf.d:

```
[opc@exastb-node1 ~]$ sudo su -
[root@exastb-node1 ~]# cp /tmp/ogg_<deployment_name>.conf /etc/nginx/conf.d
```

Step 7.3 - Install Server Certificates for NGINX

The standby system will need a different CA signed certificate due to using a different VIP name/address than the primary system. Contact your systems administrator to follow your corporate standards to create or obtain the server certificate before proceeding. A separate certificate is required for each VIP and Service Manager pair.

1. As the root user on the standby system, copy the server CA certificates and key files in the /etc/nginx/ssl directory, owned by root with file permissions 400 (-r-----):

```
[opc@exastb-node1 ~]$ sudo su -
[root@exastb-node1 ~]# mkdir /etc/nginx/ssl
[root@exastb-node1 ~]# chmod 400 /etc/nginx/ssl
```

2. As the root user on the standby system, set the correct filenames for the certificate and key file to match the same the filenames in the NGINX configuration file:

```
[root@exastb-node1 ~]# grep
ssl_certificate /etc/nginx/conf.d/ogg_<deployment_name>.conf
ssl_certificate /etc/nginx/ssl/server.chained.crt;
ssl_certificate_key /etc/nginx/ssl/server.key;
```

Note

If you have copied multiple reverse proxy configuration files copied from the primary system, you will need to repeat this process for each file.

When using CA signed certificates, the certificate named with the `ssl_certificate` NGINX parameter must include the 1) CA signed, 2) intermediate and 3) root certificates in a single file. The order is very important, otherwise NGINX fails to start and displays the error message:

(SSL: error:0B080074:x509 certificate routines: X509_check_private_key:key values mismatch).

The root and intermediate certificates can be downloaded from the CA signed certificate provider.

The single file can be generated using the following example command:

```
[root@exastb-node1 ~]# cat CA_signed_cert.crt intermediate.crt root.crt >
/etc/nginx/ssl/server.chained.crt
```

The `ssl_certificate_key` file is the key file generated when creating the Certificate Signing Request (CSR), which is required when requesting a CA signed certificate.

3. As the root user on the standby system, change the `server_name` parameter to the correct VIP name in the reverse proxy configuration file copied from the primary system:

```
[root@exastb-node1 ~]# vi /etc/nginx/conf.d/ogg_<deployment_name>.conf
# Before:
server_name          exapri-vip.oracle.com;
# After:
server_name          exastb-vip.oracle.com;
```

Step 7.4 - Test and Reload the New NGINX Configuration

Because the VIP will not be running on the standby system until the primary database service is running, there is a parameter that needs to be set in the `/etc/sysctl.conf` file.

1. As the `opc` user on the standby system, add the following parameter to the file `/etc/sysctl.conf`:

```
[opc@exastb-node1 ~]$ sudo vi /etc/sysctl.conf
# allow processes to bind to the non-local address
net.ipv4.ip_nonlocal_bind = 1
```

2. As the `opc` user on the standby system, distribute the `/etc/sysctl.conf` file:

```
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc
-d /tmp -f /etc/sysctl.conf
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo
cp /tmp/sysctl.conf /etc/sysctl.conf
```

3. As the `opc` user on the standby system, reload the modified configuration:

```
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo sysctl
-p /etc/sysctl.conf
```

4. As the `opc` user on the standby system, validate the NGINX configuration file to detect any errors in the configuration. If there are errors in the file, they will be reported by the following command:

```
[opc@exastb-node1 ~]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginxconf test is successful
```

5. As the `opc` user on the standby system, restart NGINX with the new configuration:

```
[opc@exastb-node1 ~]$ sudo systemctl restart nginx
```

Step 7.5 - Distribute the GoldenGate NGINX Configuration Files

Once all the reverse proxy configuration files have been created for the GoldenGate Service Managers, they need to be copied to all the database nodes.

1. As the opc OS user on the standby system, distribute the NGINX configuration files to all the database nodes:

```
[opc@exastb-node1 ~]$ sudo tar fczP nginx_conf.tar
/etc/nginx/conf.d/ /etc/nginx/ssl/
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc
-d /tmp -f nginx_conf.tar
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc
sudo tar fxzP /tmp/nginx_conf.tar
```

2. As the opc OS user on the standby system, test the new NGINX configuration on all nodes the new configuration files were copied to:

```
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc sudo nginx
-t
```

```
exastb-node1: nginx: the configuration file /etc/nginx/nginx.conf syntax
is ok
exastb-node1: nginx: configuration file /etc/nginx/nginx.conf test is
successful
exastb-node2: nginx: the configuration file /etc/nginx/nginx.conf syntax
is ok
exastb-node2: nginx: configuration file /etc/nginx/nginx.conf test is
successful
```

3. As the opc OS user on the standby system, restart NGINX to load the new configuration on all nodes:

```
[opc@exastb-node1 ~]$ /usr/local/bin/dcli -g ~/dbs_group -l opc
sudo systemctl restart nginx
```

Step 7.6 - Create a Clusterware Resource to Manage NGINX

Oracle Clusterware needs to have control over starting the NGINX reverse proxy so that it can be started automatically before the GoldenGate deployments are started.

1. As the grid OS user on the standby system, use the following command to get the network CRS resource name required to create the NGINX resource with a dependency on the underlying network CRS resource:

```
[opc@exastb-node1 ~]$ sudo su - grid
[grid@exastb-node1 ~]$ crsctl stat res -w "TYPE == ora.network.type"|grep
NAME
NAME=ora.net1.network
```

2. As the root user on the standby system, use the following example command to create a Clusterware resource to manage NGINX. Replace HOSTING_MEMBERS and CARDINALITY to match your environment:

```
[opc@exastb-node1 ~]$ sudo su -
[root@exastb-node1 ~]# $(grep ^crs_home /etc/oracle/olr.loc | cut -d=
-f2)/bin/crsctl add resource nginx -type generic_application -attr
```

```
"ACL='owner:root:rw,grp:root:rw,other::r--,group:oinstall:r-
x,user:oracle:rw',
EXECUTABLE_NAMES=nginx,START_PROGRAM='/bin/systemctl start -f nginx',
STOP_PROGRAM='/bin/systemctl stop -f nginx',
CHECK_PROGRAMS='/bin/systemctl status nginx' ,
START_DEPENDENCIES='hard(ora.net1.network) pullup(ora.net1.network)',
STOP_DEPENDENCIES='hard(intermediate:ora.net1.network)',
RESTART_ATTEMPTS=0,
HOSTING_MEMBERS='<exastb-node1, exastb-node2>', CARDINALITY=2"
```

The NGINX resource created in this example will run on the named database nodes at the same time, specified by HOSTING_MEMBERS. This is recommended when multiple GoldenGate Service Manager deployments are configured, and they can independently move between database nodes.

Task 8 - Configure Oracle Grid Infrastructure Agent

The following procedure shows you how to configure Oracle Clusterware to manage Oracle GoldenGate using the Oracle Grid Infrastructure Standalone Agent (XAG).

Using XAG automates the mounting of the shared file system (DBFS) as well as the stopping and starting of the Oracle GoldenGate deployment when relocating between Oracle RAC nodes.

Perform the following steps to complete this task:

- Step 8.1 - Modify the Primary Cluster XAG GoldenGate Instance
- Step 8.2 - Install Oracle Grid Infrastructure Agent
- Step 8.3 - Configure Oracle Grid Infrastructure Agent

Step 8.1 - Modify the Primary Cluster XAG GoldenGate Instance

The Oracle Grid Infrastructure Standalone Agent (XAG) GoldenGate instance on the primary system, must be modified to identify that it is part of an Oracle Data Guard configuration.

As the root user on the primary system, use the following command to modify the Oracle Data Guard autostart flag:

```
[opc@exapri-node1 ~]$ sudo su -
[root@exapri-node1 ~]# /u01/app/grid/xag/bin/agctl modify goldengate
<instance_name>
--dataguard_autostart yes
```

Step 8.2 - Install Oracle Grid Infrastructure Agent

On the standby system, follow the instructions in “Step 7.1 - Install the Oracle Grid Infrastructure Standalone Agent” from [Task 7 - Configure Oracle Grid Infrastructure Agent](#).

Step 8.3 - Configure Oracle Grid Infrastructure Agent

The parameters used to register Oracle GoldenGate Microservices with XAG are similar to those used when registering with the primary system.

1. As the grid user on the primary system, use the following command to determine the current parameters in the primary system:

```
[grid@exapri-node1 ~]$ agctl config goldengate <instance_name>
Instance name: <instance_name>
```

```

Application GoldenGate location is: /u02/app/oracle/goldengate/gg21c
Goldengate MicroServices Architecture environment: yes
Goldengate Service Manager configuration directory:
/mnt/dbfs/goldengate/deployments/<instance_name>/etc/conf
Goldengate Service Manager var directory:
/mnt/dbfs/goldengate/deployments/<instance_name>/var
Service Manager Port: 9100
Goldengate Administration User: oggadmin
Autostart on DataGuard role transition to PRIMARY: yes
Configured to run on Nodes: exapri-node1 exapri-node2
ORACLE_HOME location is: /u02/app/oracle/goldengate/gg21c/lib/instantclient
Database Services needed: ora.<DB_UNIQUE_NAME>.<SERVICE_NAME>.<FQDN>.svc
File System resources needed: dbfs_mount,nginx
Network: 1, IP:<VIP>, User:oracle, Group:oinstall

```

In addition, the XAG parameter `--filesystem_verify no` must be specified to prevent XAG from checking the existence of the DBFS deployment directory when registering the Oracle GoldenGate instance. Without setting this parameter, the XAG registration will fail, because DBFS is not mounted on the standby system.

Note

It is recommended to use the same GoldenGate instance name when registering GoldenGate with XAG as was used in the primary system.

2. As the root user on the standby system, register Oracle GoldenGate Microservices Architecture with XAG use the following command format:

<https://support.oracle.com/rs?type=doc&id=2193391.1>

<http://www.oracle.com/pls/topic/lookup?ctx=db19&id=SBYDB>

```

[root@exastb-node1 ~]# /u01/app/grid/xag/bin/agctl add goldengate
<instance_name> \
--gg_home /u02/app/oracle/goldengate/gg21c \
--service_manager \
--config_home /mnt/dbfs/goldengate/deployments/<ggsm1>/etc/conf \
--var_home /mnt/dbfs/goldengate/deployments/<ggsm1>/var \
--port 9100 \
--oracle_home /u02/app/goldengate/gg21c/lib/instantclient \
--adminuser oggadmin \
--user oracle \
--group oinstall \
--network 1 --ip <virtual_IP_address> \
--filesystems dbfs_mount,nginx \
--db_services ora.<DB_UNIQUE_NAME>.<SERVICE_NAME>.<FQDN>.svc \
--use_local_services \
--nodes <exastb-node1>,<exastb-node2> \
--filesystem_verify no \
--dataguard_autostart yes

```

Task 9 - Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections

The same TNS aliases created on the primary system for the primary database using the IPC protocol must be created with the **same** alias names on each database node of the standby system, using the IPC communication protocol as specified in [Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices](#).

The location of the `tnsnames.ora` used by the Oracle GoldenGate deployment **must** be identical on the standby system nodes as it is on the primary system.

1. As the `oracle` user on the primary system, use the following query REST API call to query the `TNS_ADMIN` location:

```
[opc@exapri-node1 ~]$ sudo su - oracle
[oracle@exapri-node1 ~]$ grep -1 TNS_ADMIN
/mnt/dbfs/goldengate/deployments/ggsm1/etc/conf/deploymentRegistry.dat
{
    "name": "TNS_ADMIN",
    "value": "/u02/app/oracle/goldengate/network/admin"
```

Make sure the `tnsnames.ora` is in this **same** directory on **all** standby database nodes.

2. As the `oracle` OS user on the standby system, follow the steps to create the TNS alias definitions and distribute to all database nodes:

```
[opc@exastb-node1 ~]$ sudo su - oracle
[oracle@exastb-node1 ~]$ dcli -l oracle -g ~/dbs_group mkdir
-p /u02/app/oracle/goldengate/network/admin
[oracle@exastb-node1 ~]$ vi /u02/app/oracle/goldengate/network/admin/
tnsnames.ora
```

```
OGGSRV_CDB =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL=IPC)(KEY=LISTENER))
    (CONNECT_DATA =
      (SERVICE_NAME = <CDB_SERVICE_NAME>)
    )
  )
```

```
OGGSRV_<PDB_NAME> =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL=IPC)(KEY=LISTENER))
    (CONNECT_DATA =
      (SERVICE_NAME = <PDB_SERVICE_NAME>)
    )
  )
```

```
[oracle@exastb-node1 ~]$ /usr/local/bin/dcli -l oracle -g ~/dbs_group
-f /u02/app/oracle/goldengate/network/admin/*.ora
-d /u02/app/oracle/goldengate/network/admin
```

Note

When the `tnsnames.ora` or `sqlnet.ora` located in the `TNS_ADMIN` directory for the Oracle GoldenGate deployment are modified, the deployment needs to be restarted in order to pick up the changes.

Task 10 - Configure Oracle GoldenGate Processes

In addition to the advice provided in [Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices](#), follow the recommendations provided below for Extract, Distribution Paths, and Replicats.

Perform the following steps to complete this task:

- Step 10.1 - Modify the Extract Configuration on the Primary System
- Step 10.2 - Modify the Distribution Path Configuration on the Primary and Standby Systems

Step 10.1 - Modify the Extract Configuration on the Primary System

For Extract processes using Data Guard configurations that are using redo transport Maximum Performance or Maximum Availability modes, the following parameter must be added to the Extract process parameter file **on the primary system** to avoid losing transactions and resulting in logical data inconsistencies:

```
TRANLOGOPTIONS HANDLEDLFAILOVER
```

This parameter prevents Extract from extracting transaction data from redo that has not yet been applied to the Data Guard standby database. This is crucial to preventing Oracle GoldenGate from replicating data to a target database that does not exist in the source standby database.

If this parameter is not specified, after a data loss failover of the source database it is possible to have data in the target database that is not present in the source database, leading to logical data inconsistencies.

By default, after 60 seconds, a warning message will be written to the Extract report file when the Extract is stalled due to not being able to query the standby database applied SCN information. For example:

```
WARNING OGG-02721 Extract has been waiting for the standby database for 60 seconds.
```

The amount of time before the warning message is written to Extract report file can be adjusted using the Extract parameter `TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_WARNING`.

If the Extract is still not able to query the standby database applied SCN information after 30 minutes (default), the Extract process will abend, logging the following message in the Extract report file:

```
ERROR OGG-02722 Extract abended waiting for 1,800 seconds for the standby database to be accessible or caught up with the primary database.
```

If the standby database becomes available before the default 30 timeout expires, Extract continues mining data from the source database and reports the following message to the report file:

```
INFO OGG-02723 Extract resumed from stalled state and started processing LCRs.
```

The timeout value of 30 minutes can be adjusted using the Extract parameter `TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_ABEND <value>`, where value is the number of seconds the standby is unavailable before abending.

If the standby database will be unavailable for a prolonged duration, such as during a planned maintenance outage, and you wish Extract to continue extracting data from the primary database, remove the `TRANLOGOPTIONS HANDLEDLFAILOVER` parameter from the Extract parameter file and restart Extract. Remember to set the parameter after the standby becomes available.

Note

If extracting from a primary database continues while the standby is unavailable, a data loss failover could result after the standby becomes available, and not all the primary redo was applied before a failover. The Oracle GoldenGate target database will contain data that does not exist in the source database.

If the Extract process has been assigned an auto restart profile, as documented in [Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices](#), after a Data Guard role transition, the Extract process will automatically restart. Extract will continue to mine redo data from the new primary database, ignoring the current state of the new standby database, until a default 5-minute timeout period expires. After this time, if the standby is not available Extract will abend with the following errors:

```
INFO OGG-25053 Timeout waiting for 300 seconds for standby database
reinstatement. Now enforcing HANDLEDLFAILOVER.
```

```
ERROR OGG-06219 Unable to extract data from the Logmining server OGG$CAP_XXXXX.
```

```
ERROR OGG-02078 Extract encountered a fatal error in a processing thread and is
abending.
```

Extract will continue to automatically restart, based on the Oracle GoldenGate Microservices auto restart profile, and failing due to reaching the `HANDLEDLFAILOVER` timeout, until the number retries is reached or the new standby database becomes available.

During the timeout period following a database role transition, the `HANDLEDLFAILOVER` parameter is automatically suspended, so data will be replicated to the Oracle GoldenGate replica database without consideration of the source standby database not being kept up to date. The timeout period for the standby database to start up before Extract abends can be adjusted using the Extract parameter `TRANLOGOPTIONS DLFAILOVER_TIMEOUT`.

It is recommended that you leave `DLFAILOVER_TIMEOUT` at the default of 5 minutes, to allow the old primary to convert to a standby. If the new standby database will be unavailable for an extended period of time or completely gone, then in order for Extract to start and remain running, you must remove the `HANDLEDLFAILOVER` parameter from the Extract parameter file. After removing the parameter, Extract no longer waits until redo has been applied to the standby database before extracting the data.

During the time it takes for the standby database to come back online and apply all the redo from the primary database, there will be data divergence between it and the Oracle GoldenGate replica database. This will be resolved once the standby database is up to date. At which point, add the `HANDLEDLFAILOVER` parameter back into the integrated Extract process parameter file, and then stop and restart the Extract.

When Oracle Data Guard Fast-Start Failover is disabled, such that the broker can automatically fail over to a standby database in the event of loss of the primary database, you must specify an additional integrated Extract parameter shown below.

```
TRANLOGOPTIONS FAILOVERTARGETDESTID n
```

This parameter identifies which standby database the Oracle GoldenGate Extract process must remain behind, with regards to not extracting redo data that has not yet been applied to the standby database.

If Oracle Data Guard Fast-Start Failover is disabled, and you don't specify the additional integrated Extract parameter `FAILOVERTARGETDESTID`, the extract will abend with the following errors:

```
ERROR OGG-06219 Unable to extract data from the Logmining server OGG$CAP_XXXXX.
```

```
ERROR OGG-02078 Extract encountered a fatal error in a processing thread and is
abending.
```

To determine the correct value for `FAILOVERTARGETDESTID`, use the `LOG_ARCHIVE_DEST_N` parameter from the Oracle GoldenGate source database which is used for sending redo to the source standby database. For example, if `LOG_ARCHIVE_DEST_2` points to the standby database, then use a value of 2.

As the oracle user on the primary system, execute the following command:

```
[opc@exapri-node1 ~]$ sudo su - oracle
[oracle@exapri-node1 ~]$ source <db_name>.env
[oracle@exapri-node1 ~]$ sqlplus / as sysdba
```

```
SQL> show parameters log_archive_dest
NAME                                TYPE                                VALUE
-----                                -                                -
log_archive_dest_1                  string                             location=USE_DB_RECOVERY_FILE_DEST,
valid_for=(ALL_LOGFILES, ALL_ROLES)
log_archive_dest_2                  string                             service="<db_name>", SYNC AFFIRM delay=0
optional compression=disable max_failure=0
reopen=300
db_unique_name="<db_name>" net_timeout=30,
valid_for=(online_logfile,all_roles)
```

In this example, the Extract parameter would be set to the following:

```
TRANLOGOPTIONS FAILOVERTARGETDESTID 2
```

To add the parameters to the Extract parameter file:

1. Log in into the Oracle GoldenGate **Administration Server** in the Source Oracle GoldenGate.
2. Click in **Overview** under **Administration Service**.

3. Click the **Action** button next to the **Extract** that you want to modify.
4. Select **Details**.
5. Select the **Parameters** tab, and then select the pencil icon to edit the current parameter file.
6. Add the TRANLOGOPTIONS parameters and select **Apply** to save the changes.

For the new parameters to take effect, the Extract process needs to be stopped and restarted, which can be done using the Administration Server.

See [Oracle GoldenGate Parameters and Functions Reference Guide](#) for further information about the Extract TRANLOGOPTIONS parameters.

Step 10.2 - Modify the Distribution Path Configuration on the Primary and Standby Systems

When the target database of an Oracle GoldenGate environment, where the Receiver Server runs, is protected with Oracle Data Guard, there is an important consideration that must be given to any Distribution Path that are sending trail files to the Receiver Server. When the Receiver Server moves to a different system after an Oracle Data Guard role transition, any Distribution Path must be altered to reflect the new target system address.

You can automatically change the Distribution Paths using a database role transition trigger **in the target database** on the Receiver Server system.

If the primary and standby system VIPs use different root CA certificates, the standby certificate will need to be added to the source deployment Service Manager, as detailed in the "Step 11.3 - Distribution Path Configuration" of [Task 11 - Configure Oracle GoldenGate Processes](#)

Follow the instructions below to create a database role transition trigger to modify the Distribution Path target address when the receiver server moves between the primary and standby system, during target database Data Guard role transitions.

Perform the following sub-steps to complete this step:

- Step 10.2.1 - Create a Shell Script to Modify the Distribution Paths
- Step 10.2.2 - Create a DBMS_SCHEDULER job
- Step 10.2.3 - Create the Deployment Config File
- Step 10.2.4 - Create the Database Role Transition Trigger

Step 10.2.1 - Create a Shell Script to Modify the Distribution Paths

[Example Distribution Path Target Change Script](#) contains an example shell script that can be used to modify a distribution path target address. Refer to the example script comments for setting appropriate variable values.

Note

The script should be placed in the same local directory on **all** Oracle RAC nodes of the **TARGETprimary and standby** database systems. Set the script file permissions to 6751.

As the oracle OS user on the TARGET primary and standby systems, follow the steps to create and distribute the script `change_path_target.sh`:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$ /usr/local/bin/dcli -l oracle -g ~/dbs_group mkdir
-p /u02/app/oracle/goldengate/scripts
[oracle@exadb-node1 ~]$ vi /u02/app/oracle/goldengate/scripts/
change_path_target.sh
# Paste the script from Example Distribution Path Target Change Script
[oracle@exadb-node1 ~]$ /usr/local/bin/dcli -l oracle -g ~/dbs_group
-f /u02/app/oracle/goldengate/scripts/change_path_target.sh
-d /u02/app/oracle/goldengate/scripts
```

Step 10.2.2 - Create a DBMS_SCHEDULER job

Creating a DBMS_SCHEDULER job is required to run an operating system shell script from within PL/SQL.

1. As the oracle OS user on the TARGET primary system, create the scheduler job as a SYSDBA user in the root container database (CDB):

```
[opc@exapri-node1 ~]$ sudo su - oracle
[oracle@exapri-node1 ~]$ source <db_name>.env
[oracle@exapri-node1 ~]$ sqlplus / as sysdba
SQL> exec dbms_scheduler.create_job(job_name=>'gg_change_path_target',
job_type=>'EXECUTABLE', number_of_arguments => 6,
job_action=>'/u02/app/oracle/goldengate/scripts/change_path_target.sh',
enabled=>FALSE);
```

To run an external job, you must set the `run_user` and `run_group` parameters in the `$ORACLE_HOME/rdbms/admin/externaljob.ora` file to the Oracle database operating system user and group.

2. As the root OS user on the TARGET primary and standby systems, create file `externaljob.ora`:

```
[opc@exadb-node1 ~]$ sudo su -
[root@exadb-node1 ~]# export DB_NAME=<database_name>
[root@exadb-node1 ~]# dbaascli database getDetails
--dbname $DB_NAME |grep homePath |uniq
    "homePath" : "/u02/app/oracle/product/19.0.0.0/dbhome_1",

[root@exadb-node1 ~]# vi
/u02/app/oracle/product/19.0.0.0/dbhome_1/rdbms/admin/externaljob.ora
# Before
run_user = nobody
run_group = nobody
# After
run_user = oracle
run_group = oinstall
```

3. Repeat this step on all nodes on the primary and standby systems.

Note

The `extrernaljob.ora` must be configured on all Oracle RAC nodes of the primary and standby database systems.

Step 10.2.3 - Create the Deployment Config File

The example shell script uses REST API calls to access the Oracle GoldenGate distribution path. In order to make the REST API calls secure, it is recommended that you include the user name and password in a configuration file, which is read by `curl`.

As the oracle OS user on the TARGET primary and standby systems, create the configuration file containing the deployment credentials:

```
[opc@exadb-node1 ~]$ sudo su - oracle
[oracle@exadb-node1 ~]$
cat > /u02/app/oracle/goldengate/scripts/<INSTANCE_NAME>.cfg << EOF
    user = "oggadmin:<password>"
EOF
[oracle@exadb-node1 ~]$ chmod 600 /u02/app/oracle/goldengate/scripts/
<INSTANCE_NAME>.cfg
[oracle@exadb-node1 ~]$ /usr/local/bin/dcli -l oracle -g ~/dbs_group
-f /u02/app/oracle/goldengate/scripts/<INSTANCE_NAME>.cfg
-d /u02/app/oracle/goldengate/scripts
```

Step 10.2.4 - Create the Database Role Transition Trigger

Create a role transition trigger on the Oracle GoldenGate source database that will be fire when a standby database becomes a primary database, changing the distribution path target address.

As the oracle OS user on the TARGET primary system, execute the following SQL sentence to create the role transition trigger:

```
[opc@exapri-node1 ~]$ sudo su - oracle
[oracle@exapri-node1 ~]$ source <db_name>.env
[oracle@exapri-node1 ~]$ sqlplus / as sysdba
CREATE OR REPLACE TRIGGER gg_change_path
AFTER db_role_change ON DATABASE
declare
    role varchar2(30);
    hostname varchar2(64);
begin
    select database_role into role from v$database;
    select host_name into hostname from v$instance;
    DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',1,'<PRIMARY
Source VIP');
    DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',2,'<STANDBY
Source VIP');

    DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',4,'<Distribution
path name');
    DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',5,'<Instance
name>');
    DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',6,'<Config
file containing the deployment credentials>');
```

```

if role = 'PRIMARY' and hostname like '<primary target cluster name>%'
then
    DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',3,'<PRIMARY
Target VIP>:443');
elseif role = 'PRIMARY'
then
    DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',3,'<STANDBY
Target VIP>:443');
end if;
DBMS_SCHEDULER.RUN_JOB(job_name=>'gg_change_path_target');
end;
/

```

Step 10.3 - Replicat Configuration on the Primary System

As documented in “Step 11.4 - Replicat Configuration” of [Task 11 - Configure Oracle GoldenGate Processes](#), a checkpoint table in the target database is required for all Oracle GoldenGate Replicat processes. There are no other configuration requirements for Replicat when configured with Oracle Data Guard.

Example Distribution Path Target Change Script

The following example script can be used to change a source Oracle GoldenGate deployment distribution path target address to reflect the new location of the receiver server after an Oracle Data Guard role transition. This example assumes the source Oracle GoldenGate deployment is configured in an MAA architecture with Data Guard, such that the distribution server can relocate between a primary and standby systems.

```

#!/bin/bash

# change_path_target.sh - changes the target host of a GG Distribution Path
# when the target moves between primary/standby systems.
# Example usage:
# ./change_path_target.sh <primary source VIP>:443 <standby source VIP>:443
# <path target VIP> <path name> <deployment name> <credentials file>

SOURCE1=$1      # PRIMARY Distribution Server VIP
SOURCE2=$2      # STANDBY Distribution Server VIP
TARGET=$3       # Distribution path target VIP
DPATH=$4        # Distribution path name
DEP=$5          # Deployment name
ACCESS=$6       # Config file containing the deployment credentials.
                # Example contents:
                # user = "oggadmin:<password>"

CONNECT=0

#echo "#${i} - `date`:"
LOGFILE=/tmp/ogg_dpatch_change.txt

result=$(curl -si -K
$ACCESS https://$SOURCE1/$DEP/distsrvr/services/v2/sources/$DPATH
-X GET| grep HTTP | awk '{print $2}')

```

```
# Will return NULL of nginx not running, 502 if cannot contact server, 200 if
# contact to server good, and others (404) for other bad reasons:

if [[ -z $result || $result -ne 200 ]]; then # Managed to access the Distr
Server
    echo "`date` - Couldn't contact Distribution Server at $SOURCE1
Deployment $DEP *****" >> $LOGFILE
else # Try the other source host:
    echo "`date` - Got status of Distribution Server at $SOURCE1 Deployment
$DEP *****" >> $LOGFILE
    SOURCE=$SOURCE1
    CONNECT=1
fi

if [ $CONNECT -eq 1 ]; then
# For secure NGINX patch destination (wss)
    PAYLOAD='{ "target": { "uri": "wss://"${TARGET}"/services/ggnorth/v2/targets?
trail=bb" } }'
    curl -s -K $ACCESS https://$SOURCE/$DEP/distsrvr/services/v2/sources/$DPATH
-X PATCH --data '{"status": "stopped"}'

# Set new target for path:
    curl -s -K $ACCESS https://$SOURCE/$DEP/distsrvr/services/v2/sources/$DPATH
-X PATCH --data "$PAYLOAD"
    echo "`date` - Set path $DPATH on $SOURCE deployment $DEP:" >> $LOGFILE

    curl -s -K $ACCESS https://$SOURCE/$DEP/distsrvr/services/v2/sources/$DPATH
-X GET | python -m json.tool | grep uri >> $LOGFILE
    curl -s -K $ACCESS https://$SOURCE/$DEP/distsrvr/services/v2/sources/$DPATH
-X PATCH --data '{"status": "running"}'
    exit 0
else
    echo "`date` - ERROR: COULDN'T CHANGE DISTRIBUTION PATH ($DPATH) in
Deployment
$DEP at $SOURCE! *****" >> $LOGFILE
fi

# If here, means we couldn't connect to either Distribution Servers
exit 1
```

On-Premises: Configuring Oracle GoldenGate Hub

Configure and deploy the MAA Oracle GoldenGate Hub architecture using the provided planning considerations, tasks, management, and troubleshooting information.

Topics:

- [Overview of MAA GoldenGate Hub](#)
- [Planning GGHub Placement in the Platinum MAA Architecture](#)
- [Security Considerations for GGHub](#)
- [Task 1: Configure the Source and Target Databases for Oracle GoldenGate](#)
- [Task 2: Prepare GGHub Clusters and Ansible Orchestration Host](#)
- [Task 3: Configure Primary and Standby GGHub Clusters Using Ansible Automation](#)
- [Task 4: Configure the Oracle GoldenGate Environment](#)
- [Reference](#)

Overview of MAA GoldenGate Hub

To achieve the highest levels of availability, resulting in zero or near-zero downtime for both unplanned outages and planned maintenance activities, customers frequently use the combination of Oracle Real Application Clusters (Oracle RAC), Oracle Active Data Guard, and Oracle GoldenGate.

This architecture, typically referred as Platinum MAA or Never Down Architecture, delivers near zero Recovery Time Objective (RTO, or downtime incurred during outage) and potentially zero or near zero Recovery Point Objective (RPO, or data loss potential).

Traditionally, Oracle GoldenGate is installed and run locally on the database server that the GoldenGate processes connect to. When used with Oracle Grid Infrastructure Standalone Agent (XAG), Oracle GoldenGate processes can be configured to seamlessly relocate or fail over between Oracle RAC nodes and follow Oracle Active Data Guard switchover and failovers.

Using MAA Oracle GoldenGate Hub (MAA GGHub) moves the GoldenGate software and processes off of the Exadata database servers, reducing complexity and system resource utilization. MAA GGHub centralizes Oracle GoldenGate management and offloads the majority of the Oracle GoldenGate processing and associated CPU and storage resource utilization from Exadata system resources. Connectivity between the GoldenGate processes and the databases they operate against is managed with Oracle Net Services.

Extreme availability that delivers zero or near zero downtime (RTO=0 or near zero), and zero or near zero data loss (RPO=0 or near zero), typically requires the following Platinum MAA architecture where:

1. A source database and target database in an Oracle GoldenGate architecture that allows your application to fail over immediately in cases of disaster (database, cluster, or site

failure) or switch over in cases of database or application upgrade. This architecture enables the potential RTO of zero or near zero for disaster scenarios and database and application upgrade maintenance.

2. Each source and target database is deployed on Exadata systems so that any local failures are tolerated or recovered almost instantly.
3. Each source and target database is configured with a standby database with Data Guard Fast-Start Failover so that any failure of the database results in activating a new primary database in seconds to minutes. If synchronous (SYNC) transport is leveraged with Maximum Availability Protection mode, zero data loss Data Guard failover is achieved.
4. Oracle GoldenGate replication is configured using MAA GGHUB between the source and target databases. Bi-directional replication is commonly configured where the target database replicates other data back to the source using a second GoldenGate configuration.
5. Any standby that becomes a primary database because of Data Guard switchover or failover automatically resynchronizes with its target GoldenGate database. If zero data loss Data Guard switchover or failover occurs, GoldenGate resynchronization ensures zero data loss across the distributed database environment.
6. GoldenGate is configured with Automatic Conflict Detection and Resolution, which is required after any Data Guard failover operation occurs.

To achieve an MAA Platinum solution on-premises, you follow these high level steps:

1. Deploy or migrate your database onto Exadata Database Machine
2. Add symmetric standby databases.
3. Configure and deploy Oracle Data Guard Fast Start Failover using the Oracle MAA best practice recommendations in [Configure Fast Start Failover](#).
4. Set up MAA GGHUB, which is detailed in the topics that follow.
5. Configure Bidirectional Replication and Automatic Conflict Detection and Resolution. See [Bi-Directional Replication](#) and [Automatic Conflict Detection and Resolution](#).
6. Decide on application failover options such as Global Data Services (see [Oracle Global Data Services Best Practices](#)), or use your own customized application failover.

Planning GGHUB Placement in the Platinum MAA Architecture

MAA GGHUB consists of two GGHUB clusters: a primary GGHUB cluster and a standby GGHUB cluster.

Meet the following guidelines when deciding where to place primary and standby GGHUB clusters in relation to the source primary, source standby, target primary, and target standby databases:

- Locate the primary active GGHUB near the target primary database. Network round trip latency between the primary active GGHUB and the target primary database must be 4ms or less to ensure acceptable GoldenGate Replicat performance.
- Network round trip latency between the primary active GGHUB and the source primary database must be 90ms or less to ensure acceptable GoldenGate Extract performance. If network round trip latency exceeds 90ms then GoldenGate distribution path is required.
- When configuring bi-directional replication the guidelines above must still be met. Therefore, if the network round trip latency between the source and target databases exceeds 4ms then it is necessary to configure two active GGHUB configurations in the same GGHUB cluster.

The table below shows you the number of GoldenGate configurations required, based on latency between sites and GoldenGate replication directionality.

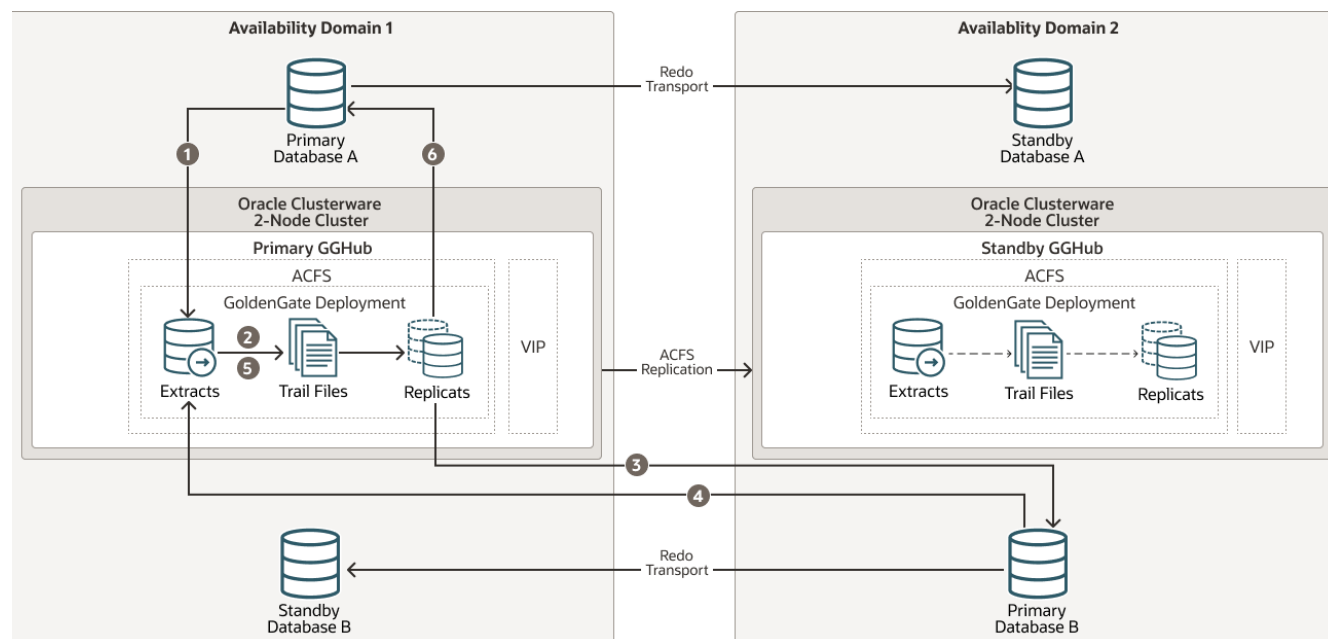
Latency between GGHUB clusters	Uni-directional GoldenGate replication	Bi-directional GoldenGate replication
<=4 milliseconds	1 GoldenGate configuration in 1 GGHUB	1 GoldenGate configuration in 1 GGHUB
>4 milliseconds	1 GoldenGate configuration in 1 GGHUB	2 GoldenGate configurations in 1 GGHUB

MAA GGHUBs Placed in the Same Data Center

In this scenario, the primary and standby database are located such that latency between GGHUB clusters is less than or equal to 4 milliseconds, so the primary (active) GGHUB and the standby GGHUB are typically located in the same data center, in separate availability domains (ADs), as shown in the image below.

As shown below, you have the following architectural components:

1. Primary database and associated standby database are configured with Oracle Active Data Guard Fast Start Failover (FSFO). FSFO can be configured with Data Guard protection mode with ASYNC or SYNC redo transport depending on your maximum data loss tolerance.
2. Primary GGHUB on Active/Passive Cluster: Only one GGHUB software deployment and configuration on the 2-node cluster. This cluster contains the Oracle GoldenGate software deployment at release 23ai or higher, that can support Oracle Database 11.2.0.4 and later database versions.
This GGHUB can support many primary databases and encapsulates the GoldenGate processes: GoldenGate Extract mines transactions from the source database and GoldenGate Replicat applies the same changes to target database. GoldenGate trail and checkpoint files also reside in the GGHUB ACFS file system. The HA failover solution is built in to the GGHUB, which includes automatic failover to the passive node in the same cluster, and restarts GoldenGate processes and activity after a node failure.
3. Standby GGHUB on Active/Passive Cluster: A symmetric standby GGHUB is configured. ACFS replication is set up between the primary and standby GGHUBs to preserve all GoldenGate files. Manual GGHUB failover, which includes ACFS failover, can be executed in the rare case that you lose the entire primary GGHUB.

Figure 26-1 Primary and Standby GGHubs in the Same Data Center

The figure above depicts replicating data from Primary Database A to Primary Database B and Primary B back to Primary A with the following steps:

1. Primary Database A: Primary A's Logminer server sends redo changes to a Primary GGHub Extract process.
2. Primary GGHub: An Extract process writes changes to trail files.
3. Primary GGHub to Primary Database B: A Primary GGHub Replicat process applies those changes to the target database (Primary B).
4. Primary Database B: Primary B's Logminer server sends redo to a Primary GGHub Extract process.
5. Primary GGHub: A Primary GGHub Extract process writes changes to trail files.
6. Primary GGHub to Primary Database A: A Primary GGHub Replicat process applies those changes to the target database (Primary A).

Note that one GGHub can support multiple source and target databases, even when the source and target databases are different Oracle Database releases.

Table 26-1 Outage Scenarios, Repair, and Restoring Redundancy for GGHubs in the Same Data Center

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Primary Database A (or Database B) failure	Impact: Near-zero application downtime. GoldenGate replication resumes when a new primary database starts. 1. One primary database is still available. All activity is routed to the existing available primary database to achieve zero application downtime. Refer to the Global Data Services Global Services Failover solution. For example, application services A-F are routed to Database A and application services G-J are routed to Database B. If Database A fails, all application services temporarily go to Database B. 2. The standby becomes the new primary automatically with Data Guard FSFO. Oracle GoldenGate replication resumes and the primary databases resynchronize. Data loss is bounded by the Data Guard protection level. If Maximum Availability or Maximum Protection is configured, zero data loss is achieved. All committed transactions are in one or both databases. Workload can be “rebalanced” when Primary Database A and Database B are available and in sync. For example, when Database A is up and running and in sync, services A-F can go back to Database A.	1. The old primary database is reinstated as the new standby database to restore redundancy. 2. Optionally performing a Data Guard switchover to switch back to the original configuration ensures that at least one primary database resides in an independent AD.
Primary or standby GGHUB single node failure	Impact: No application impact. GoldenGate replication resumes automatically after a couple of minutes. No action is required. The HA failover solution built in to the GGHUB includes automatic failover and restart of GoldenGate processes and activity. Replication activity is blocked until GoldenGate processes are active again. GoldenGate replication blackout could last a couple of minutes.	Once the node restarts, active/passive configuration is re-established.

Table 26-1 (Cont.) Outage Scenarios, Repair, and Restoring Redundancy for GGHUBs in the Same Data Center

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Primary GGHUB cluster crashes and is not recoverable	Impact: No application impact. GoldenGate replication resumes after restarting the existing GGHUB or executing a manual GGHUB failover operation. 1. If the GGHUB cluster can be restarted, then that's the simplest solution. 2. If the primary GGHUB is not recoverable, then execute a manual GGHUB failover to the standby GGHUB, which includes ACFS failover. This typically takes several minutes. 3. GoldenGate replication stops until the new primary GGHUB is available, so executing step 1 or step 2 should be quick.	If the previous GGHUB eventually restarts, ACFS replication resumes in the other direction automatically. If the GGHUB cluster is lost or unrecoverable, you need to rebuild a new standby GGHUB.
Standby GGHUB cluster crashes and is not recoverable	Impact: No application or replication impact. 1. If the GGHUB cluster can be restarted, then that is the simplest solution, and ACFS replication can resume. 2. If the standby GGHUB is not recoverable, you can rebuild a new standby GGHUB.	N/A

Table 26-1 (Cont.) Outage Scenarios, Repair, and Restoring Redundancy for GGHubs in the Same Data Center

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Availability Domain (AD1 or AD2) failure	Impact: Near-zero application downtime. GoldenGate replication resumes when the new primary database starts. - One primary database is still available. All activity is routed to the existing available primary database to achieve zero application downtime. Refer to the Global Services Failover solution. For example, application services A-F are routed to Database A and application services G-J are routed to Database B. If Database A fails, all services temporarily go to Database B. - If the primary GGHUB is still functional, GoldenGate replication continues. If the primary GGHUB is lost due to AD failure, then a manual GGHUB failover is required. GoldenGate replication resumes and the primary databases resynchronize. Data loss is bounded by the Data Guard protection level. If Maximum Availability or Maximum Protection is configured, zero data loss is achieved. All committed transactions are in one or both databases. Workload can be rebalanced when Primary Database A and Database B are available and in sync. When Database A is up and running and in sync, services A-F can go back to Database A.	- When AD returns, re-establish configuration such as reinstate standby. If the previous GGHUB eventually restarts, ACFS replication resumes in the other direction automatically. - When possible, perform a Data Guard switchover (failback) to get back to the original state where one primary database exists in each AD.

MAA GGHubs Placed in Different Data Centers

In this scenario, the primary and standby databases are located in different data centers (latency greater than 4 milliseconds), so the primary (active) GGHUB is located in the same data center as the primary database, and the standby GGHUB is located in the same data center as the standby database.

As shown in the following image, you have the following architectural components:

- The primary database and associated standby database are configured with Oracle Active Data Guard Fast Start Failover (FSFO). FSFO can be configured with Data Guard protection mode with ASYNC or SYNC redo transport depending on your maximum data loss tolerance.

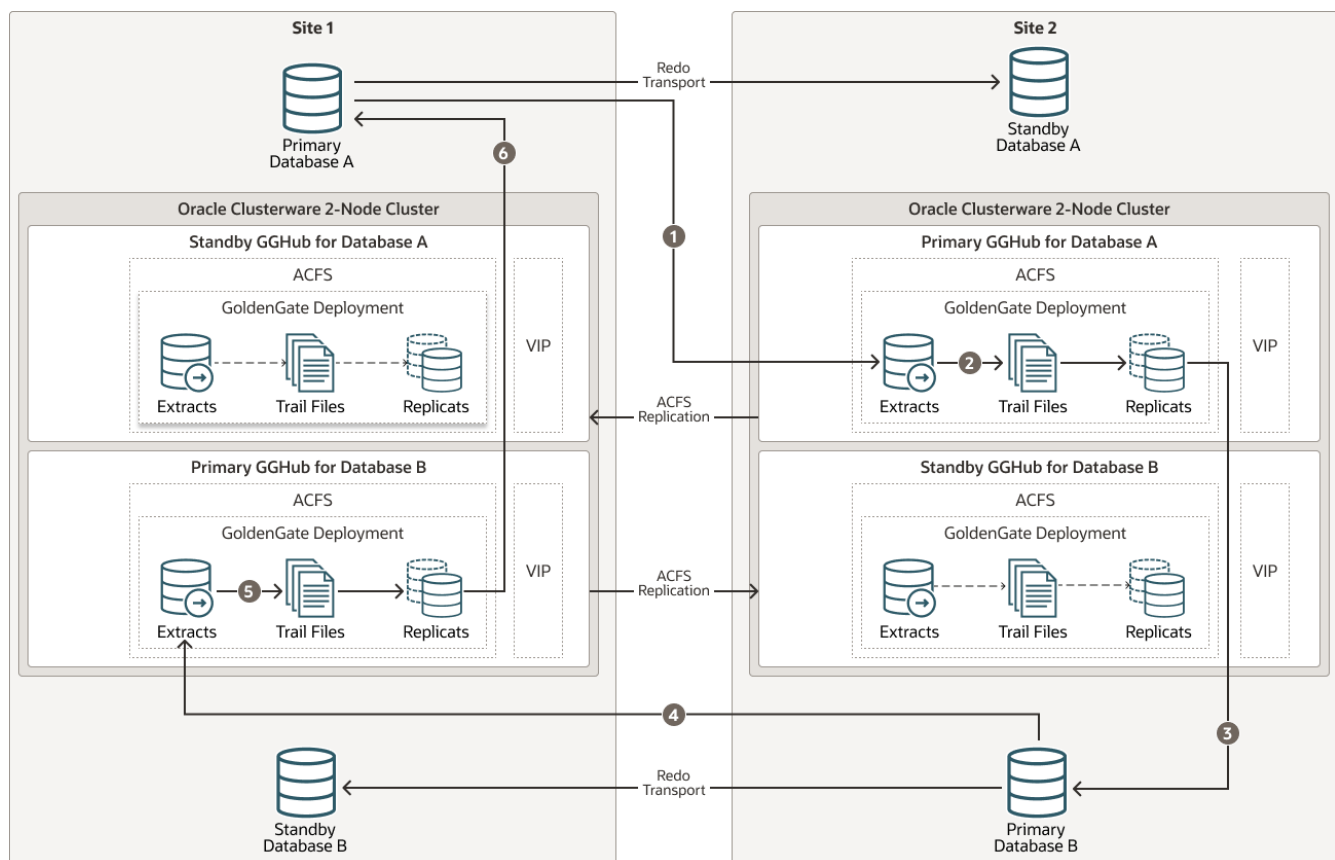
2. **Primary GGHubs on Active/Passive Clusters:** In this configuration, there's a 2-node cluster with two Oracle GoldenGate software configurations. Because a primary GGHub needs to be 4 milliseconds or less from the target database, and the network latency between the sites is greater than 5 milliseconds, two GGHub configurations are created for each GGHub cluster. Essentially, a primary GGHub configuration is always in the same data center as the target database.

The GGHubs are configured with an Oracle GoldenGate 26ai or higher software deployment that can support Oracle Database 11g and later releases. These GGHubs can support many primary databases and encapsulate the GoldenGate processes: Extract mines transactions from the source database, and Replicat applies those changes to the target database. GoldenGate trail and checkpoint files also reside in the ACFS file system. An HA failover solution is built in to the GGHub cluster, which includes automatic failover and restart of GoldenGate processes and activity after a node failure.

Note: Each GGHub configuration contains a GoldenGate Service Manager and deployment, ACFS file system with ACFS replication, and a separate application VIP.

3. **Standby GGHubs on Active/Passive Clusters:** A symmetric standby GGHub is configured in each site. ACFS replication is set up between the primary and standby GGHubs to preserve all GoldenGate files. Manual GGHub failover, which includes ACFS failover, can be executed if you lose the entire primary GGHub.

Figure 26-2 Primary and Standby GGHubs in Different Data Centers



The figure above depicts replicating data from Primary Database A to Primary Database B and Primary B back to Primary A with the following steps:

1. Primary Database A: Primary A's Logminer server sends redo changes to the GGHUB Extract process on Site 2, which is on the Primary GGHUB for Database A.
2. Primary GGHUB: The Extract process writes changes to trail files.
3. Primary GGHUB to Primary Database B: A GoldenGate Replicat process on Site 2 applies those changes to the target database (Primary Database B).
4. Primary Database B: Primary B's Logminer server sends redo to a GGHUB Extract process on Site 1, which is on the Primary GGHUB for Database B.
5. Primary GGHUB: The Extract process writes changes to trail files.
6. Primary GGHUB to Primary Database A: A GoldenGate Replicat process on Site 1 applies those changes to the target database (Primary Database A).

Table 26-2 Outage Scenarios, Repair, and Restoring Redundancy for GGHubs in Different Data Centers

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Primary Database A (or Database B) failure	Impact: Near-zero application downtime. GoldenGate replication resumes when the new primary database starts. - One primary database is still available. All activity is routed to the existing available primary database to achieve zero application downtime. Refer to the Global Data Services Global Services Failover solution. For example, application services A-F are routed to Database A and application services G-J are routed to Database B. If Database A fails, all services temporarily go to Database B. - The standby becomes the new primary automatically with Data Guard FSFO. GoldenGate replication resumes and the primary databases resynchronize. Data loss is bounded by the Data Guard protection level. If Maximum Availability or Maximum Protection is configured, zero data loss is achieved. All committed transactions are in one or both databases. Workload can be rebalanced when primary Database A and Database B are available and in sync. For example, when Database A is up and running and in sync, services A-F can go back to Database A. - Replicat performance will be degraded if the primary GGHUB is not in the same site as the target database. Schedule a GGHUB switchover with ACFS replication switchover to resume optimal Replicat performance to the target database. You may then experience two active GGHUB configurations on the same GGHUB cluster.	- The old primary database is reinstated as the new standby database to restore redundancy. - Optionally performing a Data Guard switchover, to switch back to the original configuration, ensures that at least one primary database resides in an independent AD. Schedule a GGHUB switchover with ACFS replication switchover to resume optimal Replicat performance to the target database.

Table 26-2 (Cont.) Outage Scenarios, Repair, and Restoring Redundancy for GGHUBs in Different Data Centers

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Primary or standby GGHUB single node failure	Impact: No application impact. GoldenGate replication resumes automatically after a couple of minutes. No action is required. An HA failover solution is built in to the GGHUB that includes automatic failover and restart of GoldenGate processes and activity. Replication activity is blocked until GoldenGate processes are active again. GoldenGate Replication blackout could last a couple of minutes.	Once the node restarts, active/passive configuration is re-established.
Primary GGHUB cluster crashes and is not recoverable	Impact: No application impact. GoldenGate replication resumes after the existing primary GGHUB restarts or manual GGHUB failover completes. 1. If the GGHUB cluster can be restarted, then that's the simplest solution. 2. If the primary GGHUB is not recoverable, then execute a manual GGHUB failover to the standby GGHUB, which includes ACFS failover. This typically takes several minutes. 3. Replication stops until the new primary GGHUB is started, so executing step 1 or step 2 should be quick. If there's any orchestration, this should be automated.	1. If the previous GGHUB eventually restarts, ACFS replication resumes in the other direction automatically. If the GGHUB cluster is lost or unrecoverable, you need to rebuild a new standby GGHUB. 2. Replicat performance is degraded if the primary GGHUB is not in the same data center as the target database. Schedule a GGHUB switchover with ACFS replication switchover to resume optimal Replicat performance to the target database.
Standby GGHUB cluster crashes and is not recoverable	Impact: No application or replication impact. 1. If the GGHUB cluster can be restarted, then that's the simplest solution, and ACFS replication will resume. 2. If the standby GGHUB is not recoverable, you can rebuild a new standby GGHUB.	N/A

Table 26-2 (Cont.) Outage Scenarios, Repair, and Restoring Redundancy for GGHUBs in Different Data Centers

Outage Scenario	Application Availability and Repair	Restoring Redundancy and Pristine State
Complete site failure	Impact: Near Zero Application Downtime. GoldenGate replication resumes once new primary database starts. - One primary database is still available. All activity is routed to the existing available primary database to achieve zero application downtime. Refer to the Global Data Services Global Services Failover solution. For example, application services A-F routed to Database A and application services G-J routed to Database B. If Database A fails, all services will temporarily go to Database B. - If the primary GGHUB is still functional, GoldenGate replication will continue. If the primary GGHUB is lost due to site failure, then a manual GGHUB failover is required. GoldenGate replication resumes and the primary databases resynchronize. Data loss is bounded by the Data Guard protection level. If Maximum availability or protection is configured, zero data loss is achieved. All committed transactions are in one or both databases. Workload can be rebalanced when Primary Database A and Database B are available and in sync. When Database A is up and running and in sync, services A-F can go back to Database A.	- When the site returns, re-establish configuration such as reinstate standby. If the previous GGHUB eventually restarts, ACFS replication resumes in the other direction automatically. - When possible, execute a Data Guard switchover (failback) to get back to the original state where one primary database exists in each site. - Replicat performance is degraded if the primary GGHUB is not in the same site as the target database. Schedule a GGHUB switchover with ACFS replication switchover to resume optimal Replicat performance to the target database.

Security Considerations for GGHUB

For detailed guidance on implementing security features and best practices in Oracle GoldenGate Microservices Architecture, see [Oracle GoldenGate Security Feature: Implementation](#).

When configuring GGHUB using the Ansible automation steps in the topics that follow, the security guidelines described at the link above are implemented as follows:

- Security guideline 1 is met by providing the latest Oracle GoldenGate software version in Step 2.3 - "Download the required Oracle GoldenGate Software to the Ansible Orchestration Host" for Ansible automation to install.

- Security guideline 2 is met when Ansible automation configures NGINX reverse proxy with SSL to ensure all external communication is secure. The Ansible automation initially uses a self signed certificate for NGINX configuration. The certificate and key file can be replaced as per your security standards following the process in Step 3.5 - "Replace NGINX SSL Certificate".
- Security guideline 3 is met by providing a strong password for Ansible automation in Step 3.1 - "Create GGHUB Ansible Inventory File".
- The remaining security guidelines can be implemented after Ansible automation during GoldenGate configuration.

Task 1: Configure the Source and Target Databases for Oracle GoldenGate

The source and target Oracle GoldenGate databases should be configured using the following recommendations.

Perform the following steps to complete this task:

- Step 1.1 - Database Configuration
- Step 1.2 - Create the Database Replication Administrator User
- Step 1.3 - Create the Database Services

Step 1.1 - Database Configuration

The source and target Oracle GoldenGate databases should be configured using the following recommendations:

Configuration	Scope	Example
Enable Archivelog Mode	Source and Target	<pre>SQL> ARCHIVE LOG LIST Database log mode Archive Mode Automatic archival Enabled Archive destination USE_DB_RECOVERY_FILE_DEST Oldest online log sequence 110 Next log sequence to archive 113 Current log sequence 113</pre>
Enable FORCE LOGGING	Source and Target	<pre>ALTER DATABASE FORCE LOGGING;</pre>

Configuration	Scope	Example
ENABLE_GOLDENGATE_REPLICATION	Source, Target, and Standbys	<pre>ALTER SYSTEM SET ENABLE_GOLDENGATE_REPLICATION=TRUE SCOPE=BOTH SID='*';</pre>
Enable Subset Database Replication Logging	Source Required on Target for cases when replication reverses	To enable subset database replication logging follow the steps in the Oracle GoldenGate Microservices Architecture documentation: Enable Subset Database Replication Logging

Configuration	Scope	Example
Set STREAMS_POOL_SIZE larger to accommodate GoldenGate only if CDB Root-Level integrated Extract and integrated Replicat is used	Source Required on Target for cases when replication reverses	The value of STREAMS_POOL_SIZE should be set to the following value: STREAMS_POOL_SIZE = (((#Extracts + #Integrated Replicats) * 1GB) * 1.25) For example, in a database with 2 Extracts and 2 integrated Replicats: STREAMS_POOL_SIZE = 4GB * 1.25 = 5GB ALTER SYSTEM SET STREAMS_POOL_SIZE=5G SCOPE=BOTH SID='*';

Note

Note:
It is recommended that you use per-PDB Extract and parallel Replicat which uses Shared Pool, and there is no need to explicitly set the Streams Pool. Refer to MOS document 2998659.1 for clarification.

Step 1.2 - Create the Database Replication Administrator User

A GoldenGate administrator user account is required in both source and target databases, where GoldenGate Extract and Replicat processes connect to.

Configure the Extract process to be a per-PDB Extract. The Extract process can be configured as a per-PDB Extract or a root CDB Extract. It is recommended that you use a per-PDB Extract, which does not require a c## common database user. A per-PDB Extract requires a local GoldenGate administrator user (for example, ggadmin) in the PDB it is extracting from. This can be the same GoldenGate administrator account that is used by Replicat, with additional privileges that Extract requires.

Create the GoldenGate administrator database user, and assign the appropriate privileges as detailed in [Prepare Database Users and Privileges for Oracle](#).

Step 1.3 - Create the Database Services

If the source and target databases are running the recommended configuration on an Oracle RAC cluster with Oracle Data Guard, a role-based service must be created that allows the Extract or Replicat processes to connect to the correct Data Guard primary database instance.

When using a source multitenant database with a root level Extract, separate services are required for the root container database (CDB) and the pluggable database (PDB) that contains the schema being replicated. For a target multitenant database, a single service is required for the PDB. A root container database service is not required for a per-PDB Extract.

1. As the oracle OS user on the primary and standby database systems, create and start the CDB database service using the following command:

```
[oracle@exadb1_node1 ~]$ srvctl add service -db <dbName>
  -service <dbName>_goldengate -preferred <ORACLE_SID1> -available
<ORACLE_SID2>
  -role PRIMARY
```

2. As the oracle OS user on the primary and standby database systems, create and start the PDB database service using the following command:

```
[oracle@exadb1_node1 ~]$ srvctl add service -db <dbName>
  -service <dbName>_<pdbName>_goldengate -preferred <ORACLE_SID1>
  -available <ORACLE_SID2> -pdb <pdbName> -role PRIMARY
```

3. As the oracle OS user on the primary and standby database systems, start and verify that the services are running, as shown here:

```
[oracle@exadb1_node1 ~]$ srvctl start service -db <dbName> -role
[oracle@exadb1_node1 ~]$ srvctl status service -db <dbName> |grep
goldengate
```

```
Service <dbName>_goldengate is running on instance(s) <SID1>
Service <dbName>_<pdbName>_goldengate is running on instance(s) <SID1>
```

Note

Repeat step 1.3 in the source and target database system.

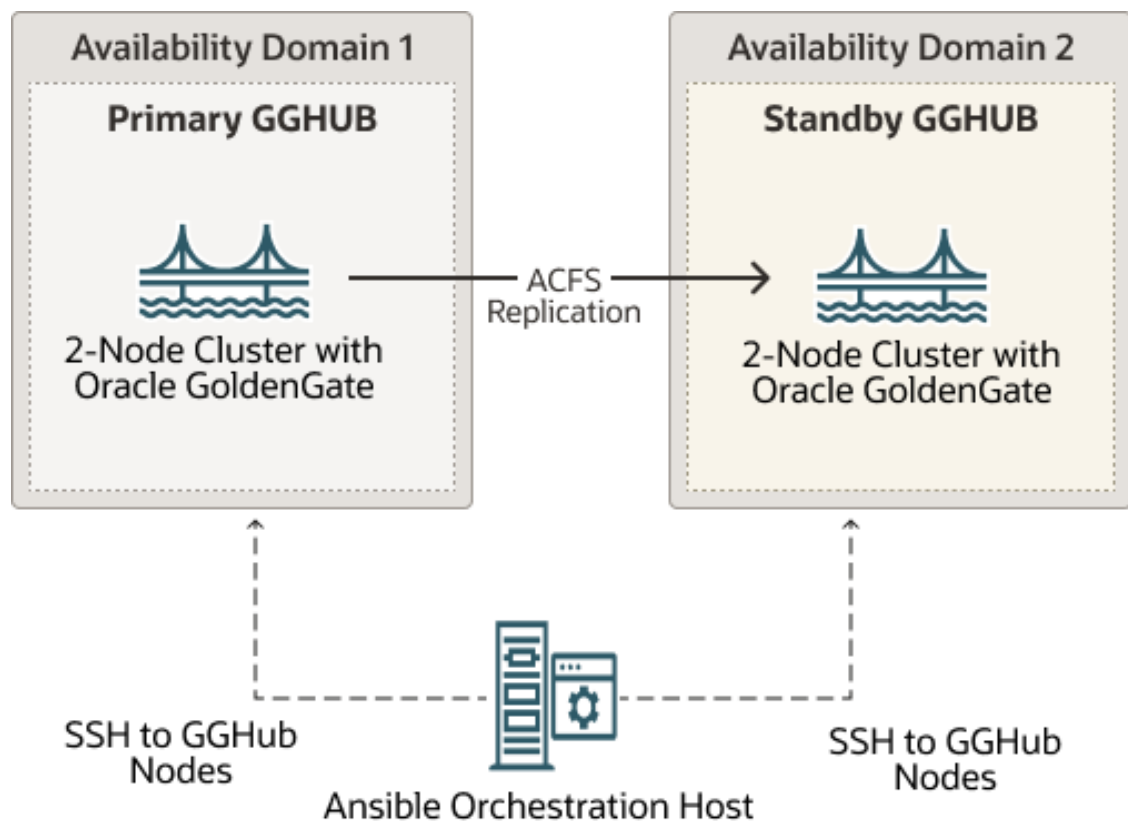
Task 2: Prepare GGHUB Clusters and Ansible Orchestration Host

Perform the following steps to complete this task:

- Step 2.1 - Deploy Primary and Standby Clusters for GGHUB
- Step 2.2 - Deploy Ansible Orchestration Host
- Step 2.3 - Download the Required Oracle GoldenGate Software to the Ansible Orchestration Host

Step 2.1 - Deploy Primary and Standby Clusters for GGHUB

Each GGHUB cluster must be deployed as a 2-node Oracle Grid Infrastructure system with job role separation as described in [Grid Infrastructure Installation and Upgrade Guide for Linux](#).



Complete Requirements for GGHUB Clusters

Primary and standby GGHUB clusters must meet the following requirements:

1. Deploy GGHUB clusters with Oracle Linux 8 operating system and 19c Grid Infrastructure software with the latest release update.
2. Configure required OS users on the GGHUB cluster nodes as follows:
 - Oracle Clusterware software owner, typically `grid` for job role separation installations, which will exist as a result of the Clusterware installation.
 - Oracle GoldenGate software owner, typically `oracle`, which is used by Ansible automation to install GoldenGate software.

- GGHUB orchestration user, such as `opc`, which has privileged access (`su/sudo`) to root OS user.
 - The Ansible orchestration user from the Ansible orchestration host will login to each GGHUB node as this OS user using passwordless ssh key configuration to run commands as `root`, `grid`, or `oracle`, as required.
 - This passwordless ssh key configuration is performed in a later step.
- 3. Allocate an unused application virtual IP address (APP VIP) on each GGHUB cluster on the clusterware public network.
 - The APP VIP is a cluster resource that Oracle Clusterware manages, and is migrated to another cluster node in the event of a node failure or planned maintenance.
 - The APP VIP is used to ensure that the primary GGHUB ACFS replication process sends GoldenGate trail files to the correct standby GGHUB node where the file system is currently mounted. This is accomplished by co-locating the APP VIP and the ACFS CRS resources on the same node.
- 4. Set up passwordless ssh key configuration for the `grid` OS user between the primary and standby GGHUB clusters to support ACFS replication. Specifically this requires the following:

SSH From	SSH To	Purpose
grid OS user on each primary GGHUB node	grid OS user on each standby GGHUB cluster node using the APP VIP name Note: In the <code>grid</code> user's <code>known_hosts</code> file on all standby GGHUB cluster nodes, add the primary cluster APP VIP name or address to each primary cluster node entry	ACFS snapshot based replication
grid OS user on each standby GGHUB node	grid OS user on each primary GGHUB cluster node using the APP VIP name Note: In the <code>grid</code> user's <code>known_hosts</code> file on all primary GGHUB cluster nodes, add the standby cluster APP VIP name or address to each standby cluster node entry.	ACFS snapshot based replication if the ACFS replication direction is reversed or GGHUB standby is converted to primary.

For additional guidance follow the instructions in [Configuring ssh for Use With Oracle ACFS Replication](#) to configure ssh connectivity between the primary and standby nodes.

- 5. Allocate 10GB free disk space to support staging installation files. The software staging directory is supplied during GGHUB deployment. Ansible automation copies the software files to the designated staging directory, but it temporarily uses the GGHUB orchestration user's home directory. Therefore, in addition to the staging directory, at least 10 GB of free space is required in the user's home directory.
- 6. Install the required packages and python modules as listed in the table below:

Required modules and libraries	Description
<code>iptables-services</code>	Required by Ansible automation for <code>nginx</code> configuration

Required modules and libraries	Description
python3.12	Ansible automation requires minimum python version 3.12 for compatibility. Python 3.12 interpreter path is required in a later step.
python3.12-pip	pip package manager associated with a python 3.12 installation.
lxml, packaging, pexpect	Python modules required by Ansible automation. Install using the command: \$ python3.12 -m pip install lxml packaging pexpect

For more details refer to Oracle Linux documentation <https://public-yum.oracle.com/oracle-linux-python.html#InstallFromAppStream>

7. Configure python3 to use Python 3.12 as the default version on the cluster nodes by running the command below as a privileged user:

```
# update-alternatives --config python3
```

8. Install NGINX version 1.24 or higher. Follow the steps provided in the NGINX documentation https://nginx.org/en/linux_packages.html#RHEL to install NGINX. NGINX configuration is performed by Ansible automation during the GGHUB deployment.
9. For compatibility and software requirements if SELinux is planned to be enabled on the GGHUB cluster nodes, refer to My Oracle Support document 3102722.1 [Oracle Automatic Storage Management Cluster File System \(ACFS\) Replication and Security-Enhanced Linux \(SELinux\)](#).

Step 2.2 - Deploy Ansible Orchestration Host and Configure GGHUB Ansible Collection

Provision a separate Oracle Linux 8 host to act as the Ansible orchestration host. This lightweight orchestration host is used to run Ansible playbooks or scripts and login to GGHUB nodes to execute specific tasks to help automate the GGHUB deployment.

Perform the following steps on the Ansible orchestration host:

- Step 2.2.1 - Install Packages Required for Ansible
- Step 2.2.2 - Configure Ansible Environment and Install GGHUB Ansible Collection
- Step 2.2.3 - Download the Required Oracle GoldenGate Software to the Ansible Orchestration Host

Step 2.2.1 - Install Packages Required for Ansible

Install the required packages listed in the table below. To install the required packages subscribe to the Oracle Linux 8 channel on the Unbreakable Linux Network, or configure a yum repository from the Oracle Linux yum server website.

Follow [Configuring a System to Use Oracle Linux Yum Server](#) to define the required repositories on the Ansible orchestration host.

Packages for Oracle Linux 8 Ansible orchestration host	Repository
python3.12	ol8_appstream
python3.12-pip	ol8_appstream

Packages for Oracle Linux 8 Ansible orchestration host	Repository
bzip2	ol8_baseos_latest

After defining the required yum repositories, install the required packages as a privileged user on the Ansible orchestration host.

```
$ sudo dnf install python3.12 python3.12-pip bzip2
```

Additional python packages and libraries are installed in a later step using pip package manager, which, by default, uses the python package index (PyPI) repository. Allow Ansible orchestration host to access python package index repository from <https://pypi.org> or use an approved internal package repository to install the required packages in your python environment.

Step 2.2.2 - Configure Ansible Environment and Install GGHUB Ansible Collection

Perform the following sub-steps to complete this task:

- Step 2.2.2.1 - Create or Identify Ansible Orchestration User
- Step 2.2.2.2 - Obtain GGHUB Ansible Collection
- Step 2.2.2.3 - Update the Default Python Version on the Ansible Orchestration Host
- Step 2.2.2.4 - Ensure That the pip Package Manager is Installed
- Step 2.2.2.5 - Execute the Script `python_venv_activate.sh` From the GGHUB Collection Directory
- Step 2.2.2.6 - Activate Python Virtual Environment

Step 2.2.2.1 - Create or Identify Ansible Orchestration User

Create or identify a non-privileged OS user on the Ansible orchestration host, such as `ansible`, that owns the Ansible installation and can be used to execute all Ansible orchestration tasks.

The Ansible orchestration user will login to the GGHUB orchestration user on each primary and standby GGHUB node using passwordless ssh to run commands as `root`, `grid`, or `oracle`, as required.

All subsequent steps on the Ansible orchestration host are performed as the Ansible orchestration user.

Step 2.2.2.2 - Obtain GGHUB Ansible Collection

GGHUB Ansible collection (Bundled Ansible automation assets such as playbooks, roles, scripts, plugins, in a single, versioned unit) can be downloaded from MOS article [KB858669 - Oracle GoldenGate MicroServices Architecture High Availability with zero or near-zero downtime](#).

In this example `/u01/maagghub/` is used as the top level directory for staging all of the files on the Ansible orchestration host.

Extract the downloaded Ansible collection under the `/u01/maagghub/gghub-ansible-collection/` directory on the Ansible orchestration host.

```
$ mkdir -p /u01/maagghub
```

```
$ cd /u01/maagghub
```


e.g.
`unzip gghub-onprem-yyyyymmdd.zip -d /u01/maagghub/gghub-ansible-collection`

Step 2.2.2.3 - Update the Default Python Version on the Ansible Orchestration Host

The required python version is python3.12 or higher. Update the default python version on the Ansible orchestration host if required.

```
$ sudo update-alternatives --config python3
```

There are 2 programs which provide 'python3'.

Selection	Command

* 1	/usr/bin/python3.6
+ 2	/usr/bin/python3.12

Enter to keep the current selection[+], or type selection number: 2

Validate the python version.

```
$ python3 --version
Python 3.12.11
```

Step 2.2.2.4 - Ensure That the pip Package Manager is Installed

Check if the pip package manager is installed using the following python command. The command should display the pip package manager version without errors.

```
$ python3.12 -m pip -V
pip 23.2.1 from /usr/lib/python3.12/site-packages/pip (python 3.12)
```

Step 2.2.2.5 - Execute the Script `python_venv_activate.sh` From the GGHUB Collection Directory

The `python_venv_activate.sh` script performs the following operations on the Ansible orchestration host:

- Creates `venv`, `ansible` and `collections` subdirectories under the `/u01/maagghub/` directory, which is used as the Ansible base directory for creating the python virtual environment in this example.
- Creates the Python virtual environment named '`venv`'.
If a virtual environment with the same name already exists, the script attempts to update the MAA GGHUB collection in the virtual environment while skipping other steps.
- Copies the `ansible.cfg` environment configuration file under the directory `/u01/maagghub/` with recommended parameter settings.
- Activates the python virtual environment.
- Installs GGHUB required Ansible packages and libraries using pip package manager.
By default the pip package manager uses the python package index (PyPI) repository. Allow Ansible orchestration host to access python package index repository from <https://pypi.org> or use an approved internal package repository.
- Updates python pip package installer.

- Installs the Ansible MAA GGHUB collection under the /u01/maagghub/collections/ directory.
- Lists the GGHUB collection version.

```
$ cd /u01/maagghub
$ sh gghub-ansible-collection/python_venv_activate.sh
```

Step 2.2.2.6 - Activate Python Virtual Environment

Python virtual environment needs to be activated before running any Ansible commands on the Ansible orchestration host.

```
$ source /u01/maagghub/venv/bin/activate
```

After activation of virtual environment the prompt changes as shown here.

```
(gghub) [~]$
```

Verify that python and ansible executables are being used from the virtual environment home.

```
$ which python ansible
/u01/maagghub/venv/bin/python
/u01/maagghub/venv/bin/ansible
```

Update GGHUB Ansible collection to newer version

To upgrade an already installed GGHUB collection to the latest available version, extract the new collection in directory /u01/maagghub/gghub-ansible-collection-new/ and run the following command:

```
$ cd /u01/maagghub
(ggghub) [~]$ ansible-galaxy collection install /u01/maagghub/gghub-ansible-
collection-new --upgrade
```

To install a specific version of the GGHUB collection, extract the collection in directory, for example /u01/maagghub/gghub-ansible-collection-version/, and use the following command:

```
(gghub) [~]$ ansible-galaxy collection install /u01/maagghub/gghub-ansible-
collection-version:==X.Y.Z
```

Step 2.3 - Download the Required Oracle GoldenGate Software to the Ansible Orchestration Host

Create a software repository directory (for example, /u01/maagghub/stage) on the Ansible orchestration host and stage the following software on the Ansible orchestration host:

1. Oracle GoldenGate 23ai software:
 - a. Review [Doc ID 1645495.1](#) to identify the latest Oracle GoldenGate 23ai Patch Set Availability document in the OGG v23ai Patch Set Availability Notes section.
 - b. From the latest Patch Set Availability document, download the patch with description "Oracle GoldenGate <OGGRU> for Oracle (Complete Install)".
2. Oracle Grid Infrastructure Standalone Agents for Oracle Clusterware 19c:

- a. Download the agent software version 10.2 or higher, using the "Download" section in document [Oracle Grid Infrastructure Standalone Agents for Oracle Clusterware](#)

Make a note of the software repository directory path as it is used in a later step.

Task 3: Configure Primary and Standby GGHUB Clusters Using Ansible Automation

Perform the following steps to complete this task:

- Step 3.1 - Create GGHUB Ansible Inventory File
- Step 3.2 - Validate SSH Connectivity and sudo Access for GGHUB Nodes Using Ansible
- Step 3.3 - Deploy and Configure GGHUB Using Ansible
- Step 3.4 - Validate GoldenGate Microservices Connectivity
- Step 3.5 - Replace NGINX SSL Certificate

Step 3.1 - Create GGHUB Ansible Inventory File

GGHUB deployment using Ansible automation requires an inventory file that contains all of the target GGHUB node names and required parameters for the GGHUB configuration.

Ansible playbook `oracle.gghub.onboarding` is executed to generate the Ansible inventory file.

The `oracle.gghub.onboarding` playbook configures an Ansible vault to securely store the encrypted GoldenGate admin user password, so that the password can be used securely and seamlessly during Ansible orchestration steps, as needed.

- The `oracle.gghub.onboarding` playbook requires and prompts for the Ansible vault password and the GoldenGate admin user password.
- The Ansible deployment orchestration uses a system-generated GoldenGate admin password for initial GoldenGate installation, and then changes the GoldenGate admin password to the user-supplied password at the end of the deployment.
- Both system-generated and user-supplied GoldenGate admin passwords are stored in the Ansible vault in an encrypted format.
- The Ansible vault is accessed using the Ansible vault password.

Table 3.1-1 Explanation of Ansible inventory file parameters and values

Table 26-3 Ansible inventory file parameters and values

Ansible inventory file prompts and parameters	Description
GoldenGate deployment name	Deployment name which is also used in ACFS and XAG clusterware resource names. Name must begin with a character and cannot be the GoldenGate Service Manager name. Special characters that are allowed in the name include [- _ / .].
GoldenGate oracle home	GoldenGate software home on the GGHUB cluster nodes.
GoldenGate primary group	Ansible group name to identify the primary GGHUB cluster nodes and associated variables. This name will be part of the primary App VIP clusterware resource name.
GoldenGate primary group node 1	Primary cluster node 1 host name. Ansible orchestration host uses this name to login.

Table 26-3 (Cont.) Ansible inventory file parameters and values

Ansible inventory file prompts and parameters	Description
GoldenGate primary group node 2	Primary cluster node 2 host name. Ansible orchestration host uses this name to login.
GoldenGate primary AppVIP IP address	App VIP for the primary cluster. App VIP needs to be on the public network and registered in DNS.
Primary ASM diskgroup name	Disk group name for the primary cluster which is used to create ACFS file system. File system size is defined by Ansible inventory file parameter ASM_vol_size (see ASM_vol_size row at the bottom of this table).
GoldenGate standby group	Ansible group name to identify the standby GGHUB cluster nodes and associated variables. This name will be part of the standby App VIP clusterware resource name.
GoldenGate standby group node 1	Standby cluster node 1 host name. Ansible orchestration host uses this name to login.
GoldenGate standby group node 2	Standby cluster node 2 host name. Ansible orchestration host uses this name to login.
GoldenGate standby AppVIP IP address	App VIP for the standby cluster. App VIP needs to be on the public network and registered in DNS.
Standby ASM diskgroup name	Disk group name for the GGHUB cluster which will be used to create ACFS file system.
ACFS mount point	Full path of the ACFS mount point. This directory is created if does not exist. The same path is used on both primary and standby GGHUB clusters.
Local software repository directory	Full path of the software repository directory on the Ansible orchestration host.
Remote directory in target machines	Full path of the software staging directory on the GGHUB cluster nodes. Ansible automation creates this directory if it does not exist during deployment process, and copies the required software.
GoldenGate patch file name	GoldenGate software complete install patch file name which is copied to the Ansible orchestration host software repository directory.
XAG patch file name	Oracle Grid Infrastructure standalone agent patch file name which is copied to the Ansible orchestration host software repository directory.
Login user for the target machines	GGHUB orchestration OS user name on the GGHUB cluster nodes with sudo privileges. Ansible orchestration host logs in to GGHUB cluster nodes using this OS user.
ansible_python_interpreter for the target machines	Full path of the python interpreter that exists on the GGHUB cluster nodes (for example, /usr/bin/python3.12 or /u01/app/oracle.ahf/common/venv/bin/python3.12). GGHUB target nodes python interpreter must be 3.12 or higher version. Python version 3.12 is recommended.
Ansible inventory full path	Full path of the Ansible inventory directory on the Ansible orchestration host. Ansible vault files and Ansible inventory file are created in this directory during onboarding Ansible playbook execution.
Password for Ansible Vault	Password which will be used to configure and access Ansible vault. Password needs to be between 8 and 30 characters with at least one lowercase character, one uppercase character and one number. See the description of Ansible vault at the beginning of this section for details.

Table 26-3 (Cont.) Ansible inventory file parameters and values

Ansible inventory file prompts and parameters	Description
Password for GoldenGate deployment	GoldenGate admin password set after initial GoldenGate installation and deployment. Password needs to be between 8 and 30 characters. Password needs to have at least one lowercase character, one uppercase character, one number and one special character from the list [- ! @ % & * . # _].
GG_deployment_number	Deployment number used if more than one GGHUB deployments are managed by Ansible Default: 1
GG_deployment_service_manager_port	The port for the Oracle GoldenGate Service Manager, which is used for accessing the Service Manager in a web browser Default: 9100
GG_deployment_admin_user	Admin user for the GGHUB deployment Default: oggadmin
ASM_vol_size	Size of the ASM volume which will be used for ACFS file system Default: 100GB

There are two methods to generate the Ansible inventory file using the Ansible playbook `oracle.gghub.onboarding`. The recommended method is to run the Ansible onboarding playbook and answer prompts which are explained in this section.

The alternate method--to use a response file as input--is discussed in [Alternate Method to Generate Ansible Inventory File for Deployment](#).

Execute the `oracle.gghub.onboarding` playbook using the supplied localhost Ansible inventory file from the collections directory on the Ansible orchestration host, as shown in the following example.

```
$ source /u01/maagghub/venv/bin/activate
(gghub) [~]$ cd /u01/maagghub
(gghub) [~]$ ansible-playbook oracle.gghub.onboarding
-i /u01/maagghub/collections/ansible_collections/oracle/gghub/inventory/
localhost
```

Example output:

```
GoldenGate - Enter the GoldenGate deployment name: maahub
GoldenGate - Enter the GoldenGate ORACLE_HOME: /u01/app/oracle/goldengate/
gg23ai
Primary - Enter the GoldenGate primary group: maahuba
Primary - Enter the GoldenGate primary group node 1 (Ansible will ssh using
this name):
maahuba1
Primary - Enter the GoldenGate primary group node 2 (Ansible will ssh using
this name):
maahuba2
Primary - Enter the GoldenGate primary APPVIP IP address: 10.53.240.11
Primary - Enter the ASM DiskGroup name: DATA
Standby - Enter the GoldenGate standby group: maahubb
Standby - Enter the GoldenGate standby group node 1 (Ansible will ssh using
```

```

this name):
maahubb1
Standby - Enter the GoldenGate standby group node 2 (Ansible will ssh using
this name):
maahubb2
Standby - Enter the GoldenGate standby APPVIP IP address: 10.53.240.12
Standby - Enter the ASM DiskGroup name: DATA
GGHUB - Enter the path for acfs mount point: /opt/oracle/ggghub/maahub
Patch File - Enter the local software repository directory that stores
software patches:
/u01/maagghub/stage
Patch File - Enter the remote directory in target machines for staging files:
/u01/oracle/stage
Patch File - Enter the GoldenGate patch file name:
p37777817_238025040GGRU_Linux-x86-64.zip
Patch File - Enter the XAG patch file name: p31215432_190000_Generic.zip
Ansible - Sets the login user for the target machines: opc
Ansible - Sets the ansible_python_interpreter for the target
machines: /usr/bin/python3.12
Ansible - Enter the Ansible inventory full path directory: /u01/maagghub/
inventory
Password - Enter the password for Ansible Vault ([8=>length<=30][A-Z][a-z]
[0-9]):
confirm Password - Enter the password for Ansible Vault ([8=>length<=30][A-Z]
[a-z][0-9]):
Password - Enter the password for GoldenGate deployment ''
([8=>length<=30][A-Z][a-z][0-9][-!@%&*.#]):
confirm Password - Enter the password for GoldenGate deployment ''
([8=>length<=30][A-Z][a-z][0-9][-!@%&*.#]):

```

The onboarding playbook configures the Ansible vault, creates the inventory directory and Ansible inventory file when executed using either of the two methods specified above.

The Ansible inventory file can be updated to adjust any parameter values before executing the GGHUB deployment using Ansible.

Validate the Ansible vault and deployment Ansible inventory files in the /u01/maagghub/inventory/ directory.

```
(ggghub) [~]$ ls /u01/maagghub/inventory/
```

```

maahub.yml
vault_pass
maahub.key

```

GGHUB inventory configuration file example:

```
(ggghub) [~]$ cat /u01/maagghub/inventory/maahub.yml
```

```

gghservers:
  children:
    maahuba:
      hosts:
        maahuba1:
        maahuba2:

```

```

vars:
  appvip_ip_address: 10.53.240.11
  asm_diskgroup: DATA
  ansible_python_interpreter: /u01/app/oracle.ahf/common/venv/bin/
python3.12
  ansible_user: opc
maahubb:
  hosts:
    maahubb1:
    maahubb2:
  vars:
    appvip_ip_address: 10.53.240.12
    asm_diskgroup: DATA
    ansible_python_interpreter: /u01/app/oracle.ahf/common/venv/bin/
python3.12
  ansible_user: opc
vars:
  gg_deployment_name: maahub
  gg_deployment_number: 1
  gg_deployment_service_manager_port: 9100
  gg_deployment_admin_user: oggadmin
  gg_hub_groups: maahuba,maahubb
  acfs_name: maahub
  acfs_mount_point: /opt/oracle/ggghub/maahub
  asm_vol_size: 100G
  gg_oracle_home: /u01/app/oracle/goldengate/gg23ai
  gg_patch_file: p37777817_238025040GGRU_Linux-x86-64.zip
  xag_patch_file: p31215432_190000_Generic.zip
  stage_dir_local: /u01/maagghub/stage
  stage_dir_remote: /u01/oracle/stage
  ansible_inventory_dir: /u01/maagghub/inventory

```

Step 3.2 - Configure and Validate SSH Connectivity and sudo Access for GGHUB Cluster Nodes Using Ansible

Configure the following passwordless ssh access:

SSH from	SSH to	Purpose
Ansible orchestration user (for example, ansible) on Ansible orchestration host	GGHub orchestration user (for example, opc) on each GGHUB cluster node This user must have privileged access (su/sudo) to root user. It is specified in the Ansible inventory file as parameter ansible_user.	For Ansible to connect to GGHUB nodes to perform installation, setup, and administration commands run as root, grid, or oracle, as required.

Execute the below Ansible command to test the ssh connectivity from the Ansible orchestration host to GGHUB nodes, and also validate sudo privileges on the GGHUB nodes.

```

(ggghub) [~]$ ansible ggghubservers -i /u01/maagghub/inventory/maahub.yml -m
command
-a 'sudo date'

```

The above Ansible command should return date output and return code 0 from all of the GGHUB hosts specified under the gghservers group in the inventory file `maahub.yml` in this example.

Step 3.3 - Deploy GGHUB on the Primary and Standby Clusters Using Ansible

To configure and deploy the MAA GoldenGate Hub on the primary and standby clusters execute the `oracle.gghub.deploy` Ansible playbook. The Ansible deploy playbook configures one VIP and service manager pair on each GGHUB cluster.

The `oracle.gghub.deploy` playbook performs the following tasks on the primary and standby GGHUB clusters:

- Executes prerequisite checks
- Installs Oracle GoldenGate software
- Configures ACFS file system and ACFS replication between primary and standby GGHUB clusters
- Creates clusterware resources for APP VIPs
- Creates the Oracle GoldenGate Deployment
- Configures Oracle Grid Infrastructure Agent (XAG)
- Configures NGINX Reverse Proxy and creates NGINX clusterware resource
- Secures GoldenGate Microservices to restrict non-secure direct access

Execute the `oracle.gghub.deploy` playbook using the Ansible inventory file as follows:

```
(gghub) [~]$ ansible-playbook oracle.gghub.deploy
-i /u01/maagghub/inventory/maahub.yml -e @/u01/maagghub/inventory/maahub.key
--ask-vault-pass
```

The `oracle.gghub.deploy` playbook prompts for the Ansible vault password, displays all of the configuration information, and then asks for confirmation to continue with the deployment.

GGHUB deployment playbook example output

Vault password:

```
PLAY [Play for deploying GGHUB]
*****

TASK [Gather server default minimum amount of facts]
*****
ok: [maahuba1]

TASK [oracle.gghub.oracle_meta : Assert GoldenGate deployment password]
*****
ok: [maahuba1] => {
  "changed": false,
  "msg": "All assertions passed"
}

TASK [Show GoldenGate Deployment]
*****
<snip>
```


[Confirm GGHUB deployment variables]

Are you sure you want to continue with the GGHUB deployment (yes/no):

<snip>

PLAY RECAP

```
*****
*****
maahuba1      : ok=233  changed=121  unreachable=0
failed=0      skipped=65  rescued=0    ignored=0
maahuba2      : ok=139  changed=47  unreachable=0
failed=0      skipped=145 rescued=0    ignored=0
maahubb1      : ok=180  changed=85  unreachable=0
failed=0      skipped=104 rescued=0    ignored=0
maahubb2      : ok=139  changed=48  unreachable=0
failed=0      skipped=145 rescued=0    ignored=0
```

GGHub deployment playbook execution should not have any "failed" tasks in the Ansible PLAY RECAP summary, which is displayed at the end of the playbook execution. The "ok, changed" and "skipped" number of tasks could vary based on the environment and retry attempts.

After deployment is complete, the `oracle.gghub.verify` playbook runs automatically to display the GGHUB deployment configuration information.

If issues occur during deployment, or if the configuration needs to be removed, see [Troubleshooting GGHUB Deployment Using Ansible](#).

Step 3.4 - Validate GoldenGate Microservices Connectivity

As the root OS user on the first GGHUB node, use the below curl command to validate the Microservices connectivity. The command will prompt for the oggadmin deployment password.

```
[root@gghub_prim1 ~]# vi access.cfg
user = "oggadmin"

[root@gghub_prim1 ~]# curl --insecure --user oggadmin -svf
-K access.cfg https://<vip_name.FQDN>:<port#>/services/v2/config/health
-XGET && echo -e "\n*** Success"
```

Sample output:

Enter host password for user 'oggadmin':

```
* Connected to maahuba-app-vip1.clientsubnet.vcnfraans1.oraclevcn.com
(10.53.240.11)
port 443 (#0)
* ALPN, offering h2
* ALPN, offering http/1.1
* successfully set certificate verify locations:
...
* Server certificate:
*  subject: C=GE; ST=San Francisco; L=California; O=Oracle; OU=Oracle MAA;
CN=maahuba-app-vip1.clientsubnet.vcnfraans1.oraclevcn.com
*   start date: Nov 16 14:04:18 2025 GMT
*   expire date: Nov 16 14:04:18 2026 GMT
*   issuer: C=US; L=San Francisco; O=Oracle; OU=Oracle MAA;
CN=maahuba-app-vip1.clientsubnet.vcnfraans1.oraclevcn.com;
```

```

basicConstraints=CA:true
* SSL certificate verify result: self signed certificate in certificate
chain (19),
  continuing anyway.
* Server auth using Basic with user 'oggadmin'
* TLSv1.3 (OUT), TLS app data, [no content] (0):
> GET /services/v2/config/health HTTP/1.1
> Host: maahuba-app-vip1.clientsubnet.vcnfraans1.oraclevcn.com
> Authorization: Basic b2dnYWRtaW46V0VsY29tZTEyM19f
> User-Agent: curl/7.61.1
> Accept: */*
...
<
{"$schema":"api:standardResponse","links":[{"rel":"canonical","href":
"https://maahuba-app-vip1.clientsubnet.vcnfraans1.oraclevcn.com/services/v2/
config/health",
"mediaType":"application/json"}, {"rel":"self","href":
"https://maahuba-app-vip1.clientsubnet.vcnfraans1.oraclevcn.com/services/v2/
config/health",
"mediaType":"application/json"}, {"rel":"describedby","href":
"https://maahuba-app-vip1.clientsubnet.vcnfraans1.oraclevcn.com/services/
ServiceManager/v2/metadata-catalog/health",
"mediaType":"application/schema+json"}], "messages": [], "response":
{"$schema":"ogg:health",
"deploymentName":"ServiceManager", "serviceName":"ServiceManager", "started":
"2025-11-13T20:25:30.321Z", "healthy":true, "criticalResources":
[{"deploymentName":
"ServiceManager", "name":"ServiceManager", "type":"service", "status":"running",
"healthy":true},
{"deploymentName":"ServiceManager", "name":"pluginsrvr", "type":"service",
"status":"stopped", "healthy":true},
{"deploymentName":"maahub", "name":"adminsrvr", "type":
"service", "status":"running", "healthy":true}, {"* Connection #0 to host
maahuba-app-vip1.clientsubnet.vcnfraans1.oraclevcn.com left intact
"deploymentName":"maahub", "name":"distsrvr", "type":"service", "status":"running",
"healthy":
true},
{"deploymentName":"maahub", "name":"recvsrvr", "type":"service", "status":"runnin
g",
"healthy":true}}]}
*** Success

```

① Note

If the environment is using self-signed SSL certificates, add the flag `--insecure` to the curl command to avoid the error "NSS error -8172 (SEC_ERROR_UNTRUSTED_ISSUER)".

Step 3.5 - Replace NGINX SSL Certificate

Follow the below sub-steps to replace the NGINX SSL certificate and key file for your deployment orchestrated by Ansible automation.

- Step 3.5.1 - Validate NGINX configuration
- Step 3.5.2 - Identify the NGINX configuration file for the deployment

- Step 3.5.3 - Copy the certificate and key files
- Step 3.5.4 - Update the NGINX configuration file
- Step 3.5.5 - Reload and validate NGINX configuration

Note

NGINX commands need to be run as a privileged user on the GGHUB cluster nodes.

Step 3.5.1 - Validate NGINX configuration

Validate the NGINX configuration using the below command:

```
# nginx -t

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Step 3.5.2 - Identify the NGINX configuration file for the deployment

For GGHUB deployment the NGINX configuration files are located under the directory /etc/nginx/conf.d/ and the configuration file name follows the standard ogg_<GG_DEPLOYMENT_NAME>.conf.

The location of the NGINX configuration files is also specified in the /etc/nginx/nginx.conf file.

In this example the GGHUB deployment NGINX configuration file name is ogg_maahub.conf.

```
# ls /etc/nginx/conf.d/

ogg_maahub.conf
```

Step 3.5.3 - Copy the certificate and key files

View the GGHUB NGINX configuration file and note the ssl_certificate and ssl_certificate_key parameter values in the server block:

```
# view /etc/nginx/conf.d/ogg_maahub.conf

server {
    listen          443 ssl;
    listen          [::]:443 ssl;
    proxy_read_timeout    600s;
    proxy_buffer_size    16k;
    proxy_buffers       8 16k;
    server_name       maahuba-app-
vip1.clientsubnet.vcnfraans1.oraclevcn.com;
    ssl_certificate    /etc/nginx/ssl/maahuba-app-vip1.chained.pem;
    ssl_certificate_key /etc/nginx/ssl/maahuba-app-vip1.key;
```

You should receive a certificate file and corresponding private key file from a Certificate Authority (CA) (for example: your_domain.crt and your_domain.key). If your CA provided a CA bundle or intermediate certificates make sure to create a chained certificate following the below guideline.

When using CA-signed certificates, the certificate named with the `ssl_certificate` NGINX parameter must include the 1) CA signed, 2) intermediate, and 3) root certificates in a single file. The order is significant; otherwise, NGINX fails to start and displays the error message:

(SSL: error:0B080074:x509 certificate routines: X509_check_private_key:key values mismatch)

When using chained certificate use `.pem` as the file extension. This extension is commonly used when adding multiple certificates into a single file and referenced in the GoldenGate user guide.

Copy the certificate (chained certificate) and private key file to the `/etc/nginx/ssl/` directory identified from the NGINX configuration file.

Step 3.5.4 - Update the NGINX configuration file

Edit the NGINX configuration file and update the `ssl_certificate` and `ssl_certificate_key` parameter values to use the new certificate and key files.

Step 3.5.5 - Validate NGINX configuration and reload service

Validate the NGINX configuration and reload the NGINX service on the GGHUB cluster nodes.

```
# nginx -t

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

# systemctl reload nginx
# systemctl status nginx.service
```

Repeat the steps on the second node of the GGHUB cluster.

Do Step 3.4 to validate GoldenGate Microservices connectivity after the change.

Task 4: Configure the Oracle GoldenGate Environment

Perform the following steps to complete this task:

- Step 4.1 - Create the Database Credentials
- Step 4.2 - Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections
- Step 4.3 - Set Up Schema Supplemental Logging
- Step 4.4 - Create the Autostart Profile
- Step 4.5 - Configure Oracle GoldenGate Processes

Step 4.1 - Create the Database Credentials

With the Oracle GoldenGate deployment created, use the Oracle GoldenGate Administration Service home page to create the database credentials using the above TNS alias names. See the figure below for an example of the database credential creation using the TNS alias.

As the `oggadmin` user, create the database credentials:

1. Log in into the Administration Service: `https://gghub.example.com:443/deployment_name/adminsrvr`
2. Click **Configuration** under **Administration Service**.
3. Click the **plus (+)** to **Add Credentials** under the **Database** tab.

4. Add the required information for the source and target CDB and PDB:

Data Center	Container	Domain	Alias	User ID
DC 1	CDB	GoldenGate	DC1_CDB	c##ggadmin@<tns_alias>
DC 1	PDB	GoldenGate	DC1_PDB	ggadmin@<tns_alias>
DC 2	CDB	GoldenGate	DC2_CDB	c##ggadmin@<tns_alias>
DC 2	PDB	GoldenGate	DC2_PDB	ggadmin@<tns_alias>

Step 4.2 - Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections

To provide local database connections for the Oracle GoldenGate processes when switching between nodes, create a TNS alias on **all** nodes of the cluster where Oracle GoldenGate may be started. Create the TNS alias in the `tnsnames.ora` file in the `TNS_ADMIN` directory specified in the deployment creation.

Ansible automation uses the `<GoldenGate_Oracle_Home>/network/admin` as the `TNS_ADMIN` directory for the deployment. GoldenGate oracle home is an input variable in the onboarding step 3.1 specified in the Ansible inventory file.

If the source database is a multitenant database, two TNS alias entries are required, one for the container database (CDB) and one for the pluggable database (PDB) that is being replicated. For a target Multitenant database, the TNS alias connects the PDB to where replicated data is being applied. The pluggable database `SERVICE_NAME` should be set to the database service created in an earlier step (refer to Step 1.3 - "Create the Database Services" in [Task 1: Configure the Source and Target Databases for Oracle GoldenGate](#)).

As the oracle OS user on any database node of the primary and the standby database systems, use `dbaascli` to find the database domain name and the SCAN name:

```
# Primary DB
[opc@exadb1_node1]$ sudo su - oracle
[oracle@exadb1_node1]$ source <dbName>.env
[oracle@exadb1_node1]$ dbaascli database getDetails --dbname <dbName> |grep
'connectString'
```

```
"connectString" : "<primary_scan_name>:1521/<service_name>"
```

```
# Standby DB
[opc@exadb2_node1]$ sudo su - oracle
[oracle@exadb2_node1]$ source dbName.env
[oracle@exadb2_node1]$ dbaascli database getDetails --dbname <dbName> |grep
'connectString'
```

```
"connectString" : "<standby_scan_name>:1521/<service_name>"
```

As the oracle OS user on all nodes of the primary and standby GGHUB, add the recommended parameters for Oracle GoldenGate in the `sqlnet.ora` file:

```
[opc@gghub_prim1]$ sudo su - oracle
[oracle@gghub_prim1]$ mkdir -p /u01/app/oracle/goldengate/network/admin
[oracle@gghub_prim1]$
cat > /u01/app/oracle/goldengate/network/admin/sqlnet.ora <<EOF
```

```
DEFAULT_SDU_SIZE = 2097152
EOF
```

As the oracle OS user on all nodes of the primary and standby GGHub, follow the steps to create the TNS alias definitions:

```
[opc@gghub_prim1 ~]$ sudo su - oracle
[oracle@gghub_prim1 ~]$
```

```
cat > /u01/app/oracle/goldengate/network/admin/tnsnames.ora <<EOF
```

```
# Source
<source_cbd_service_name>=
  (DESCRIPTION =
    (CONNECT_TIMEOUT=3)(RETRY_COUNT=2)(LOAD_BALANCE=off)(FAILOVER=on)
  (RECV_TIMEOUT=30)
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<primary_scan_name>)
        (PORT=1521)))
      (ADDRESS_LIST =
        (ADDRESS = (PROTOCOL = TCP)(HOST=<standby_scan_name>)
          (PORT=1521)))
    (CONNECT_DATA=(SERVICE_NAME =
<source_cbd_service_name>.goldengate.com)))

<source_pdb_service_name>=
  (DESCRIPTION =
    (CONNECT_TIMEOUT=3)(RETRY_COUNT=2)(LOAD_BALANCE=off)(FAILOVER=on)
  (RECV_TIMEOUT=30)
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<primary_scan_name>)(PORT=1521)))
      (ADDRESS_LIST =
        (ADDRESS = (PROTOCOL = TCP)(HOST=<standby_scan_name>)(PORT=1521)))
    (CONNECT_DATA=(SERVICE_NAME =
<source_pdb_service_name>.goldengate.com)))

# Target
<target_pdb_service_name>=
  (DESCRIPTION =
    (CONNECT_TIMEOUT=3)(RETRY_COUNT=2)(LOAD_BALANCE=off)(FAILOVER=on)
  (RECV_TIMEOUT=30)
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST=<primary_scan_name>)(PORT=1521)))
      (ADDRESS_LIST =
        (ADDRESS = (PROTOCOL = TCP)(HOST=<standby_scan_name>)(PORT=1521)))
    (CONNECT_DATA=(SERVICE_NAME =
<target_pdb_service_name>.goldengate.com)))

EOF
```

```
[oracle@gghub_prim1 ~]$ scp /u01/app/oracle/goldengate/network/admin/*.ora
gghub_prim2:/u01/app/oracle/goldengate/network/admin
```

Note

When the `tnsnames.ora` or `sqlnet.ora` located in the `TNS_ADMIN` directory for the Oracle GoldenGate deployment are modified, the deployment needs to be restarted to pick up the changes.

Step 4.3 - Setup Schema Supplemental Logging

- Log in to the Oracle GoldenGate Administration Server.
- Click **Configuration** under **Administration Service**.
- Click the **Connect to database** button under **Actions** for the **Source Database (Reg_CDB)**.
- Click the plus button (Add TRANDATA) to **Add TRANDATA** for the **Schema** or **Tables**.

Step 4.4 - Create the Autostart Profile

Create a new profile to automatically start the Extract and Replicat processes when the Oracle GoldenGate Administration Server is started. Then, restart if any Extract or Replicat processes are abandoned. With GoldenGate Microservices, auto start and restart is managed by Profiles.

Using the Oracle GoldenGate Administration Server GUI, create a new profile that can be assigned to each of the Oracle GoldenGate processes:

1. Log in to the **Administration Service** on the Source and Target GoldenGate.
2. Click on **Profile** under **Administration Service**.
3. Click the **plus (+)** sign next to Profiles on the Managed Process Settings home page.
4. Enter the details as follows:
 - Profile Name: `Start_Default`
 - Description: Default auto-start/restart profile
 - Default Profile: Yes
 - Auto Start: Yes
 - Auto Start Options
 - Startup Delay: 1 min
 - Auto Restart: Yes
 - Auto Restart Options
 - Max Retries: 5
 - Retry Delay: 30 sec
 - Retries Window: 30 min
 - Restart on Failure only: Yes
 - Disable Task After Retries Exhausted: Yes
5. Click **Submit**

Step 4.5 - Configure Oracle GoldenGate Processes

When creating Extract, Distribution Paths, and Replicat processes with Oracle GoldenGate Microservices Architecture, all files that need to be shared between the GGHUB nodes are already shared with the deployment files stored on a shared file system.

Below are essential configuration details recommended for running Oracle GoldenGate Microservices on GGHUB for Extract, Distribution Paths, and Replicat processes.

Perform the following sub-steps to complete this step:

- Step 4.5.1 - Extract Configuration
- Step 4.5.2 - Replicat Configuration
- Step 4.5.3 - Distribution Path Configuration
- Step 4.5.4 - Set Up a Heartbeat Table for Monitoring Lag Times

The main goal is to prevent data divergence between GoldenGate replicas and their associated standby databases. This section focuses on configuring Extract so that GoldenGate Extract never gets ahead of the standby database which can result in data divergence.

GoldenGate Parameter	Description	Recommendations
TRANLOGOPTIONS HANDLEDLFAILOVER	This is a mandatory setting for Data Guard configurations that have Oracle GoldenGate to ensure that GoldenGate Extract never extracts data that has not been received by standby database. The HANDLEDLFAILOVER stands for handle DATA LOSS for Data Guard failover. This parameter must be added to the Extract process parameter file to avoid losing transactions and resulting in logical data inconsistencies after data loss Data Guard failover event. When the two primaries try to reconcile, this parameter ensures that all transactions can be reconciled because the new primary (old standby) is not further behind as expected. This parameter prevents Extract from extracting redo data from the source database, and writing to the trail file data that has not yet been applied to the Oracle Data Guard standby database. If this parameter is not specified, after a data loss failover, it is possible to have data in the target database that is not present in the source database, leading to data divergence and logical data inconsistencies.	MANDATORY when the source database is configured with Data Guard in Maximum Availability or Maximum Performance mode.

GoldenGate Parameter	Description	Recommendations
TRANLOGOPTIONS FAILOVERTARGETDESTID <i>n</i>	For multiple standby configurations or cases when Data Guard Fast-Start failover is not enabled, set FAILOVERTARGETDESTID to standby demarcated by LOG_ARCHIVE_DEST to ensure that GoldenGate Extract never extracts data that has not been received by target standby database. When not using Data Guard Fast Start Failover (FSFO) in the source database, this parameter Identifies which standby database the Extract process must remain behind, with regard to not extracting redo data that has not yet been applied to the Oracle Data Guard standby database.	MANDATORY when not using FSFO in the source database. To determine the correct value for FAILOVERTARGETDESTID, use the LOG_ARCHIVE_DEST_<i>N</i> parameter from the GoldenGate source database which is used for sending redo to the source standby database. For example, if LOG_ARCHIVE_DEST_2 points to the standby database, then use a value of 2.
TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_WARNING <i>value</i>	The amount of time before a warning message is written to the Extract report file, if Extract is stalled because it is unable to query the source database standby apply progress. This can occur after a Data Guard failover when the old primary database is not currently available. Default is 60 seconds.	OPTIONAL if you want to adjust the timing of when the warning message is written to the Extract report file. Add STANDBY_WARNING <value> to the TRANLOGOPTIONS HANDLEDLFAILOVER parameter.
TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_ABEND <i>value</i>	The amount of time before Extract abends, if Extract is stalled because it is unable to query the standby apply progress. Default is 30 minutes.	OPTIONAL if you want to adjust the amount of time it takes Extract to abend, when the source database standby is not accessible to enforce the HANDLEDLFAILOVER parameter. Add STANDBY_ABEND <value> to the TRANLOGOPTIONS HANDLEDLFAILOVER parameter.

GoldenGate Parameter	Description	Recommendations
TRANLOGOPTIONS DLFAILOVER_TIMEOUT <i>value</i>	The amount of time Extract runs on the new source primary database after a Data Guard role transition, before it checks the status of the standby database. If standby database is not available after the DLFAILOVER_TIMEOUT, Extract abends. Default is 300 seconds.	OPTIONAL if you want to adjust the amount of time an Extract can run on a new primary source database, after a role transition, when the standby is not yet available to honor the TRANLOGOPTIONS HANDLEDLFAILOVER parameter.

 Note

If, during normal operations of the source Oracle Data Guard configuration, the standby database becomes unavailable, Extract stops extracting data from the source database to prevent possible data divergence with the GoldenGate target database due to the HANDLEDLFAILOVER parameter. The DLFAILOVER_TIMEOUT parameter does not take effect when a Data Guard failover has not occurred, and there are no

GoldenGate Parameter	Description	Recommendations
		messages output to the Extract report file.

See the [Oracle GoldenGate Parameters and Functions Reference Guide](#) for more information about Extract TRANLOGOPTIONS parameters.

When creating an Extract using the Oracle GoldenGate Administration Service GUI, leave the Trail SubDirectory parameter blank so that the trail files are automatically created in the deployment directories stored on the shared file system. The default location for trail files is the /<deployment-directory>/var/lib/data directory.

Note

To capture from a multitenant database, you must use an Extract configured at the root level using a c## account. To apply data into a multitenant database, a separate Replicat is needed for each PDB because a Replicat connects at the PDB level and doesn't have access to objects outside of that PDB.

Step 4.5.1 - Extract Configuration

1. Log in to the Oracle GoldenGate **Administration Server**
2. Click in **Overview** under **Administration Service**
3. Click the **plus (+)** button to **Add Extract**
4. Select **Integrated Extract**
5. Add the required information as follows:
 - Process Name: EXT_1
 - Description: Extract for DC 1 CDB
 - Intent: Unidirectional
 - Begin: Now
 - Trail Name: aa
 - Credential Domain: GoldenGate
 - Credential Alias: DC1_CDB
 - Register to PDBs: PDB Name
6. Click **Next** and set parameters:

```
EXTRACT ext_1
USERIDALIAS DC1_CDB DOMAIN GoldenGate
EXTTRAIL aa
TRANLOGOPTIONS HANDLEDLFAILOVER
TRANLOGOPTIONS FAILOVERTARGETDESTID 2
```

```
SOURCECATALOG <PDB_NAME>  
TABLE <OWNER>.*;
```

7. Click **Next**.
8. If using CDB Root Capture from PDB, add the SOURCECATALOG parameter with the PDB Name
9. For Oracle Data Guard configurations, add the TRANLOGOPTIONS parameter, if required, as explained earlier in this step:
 - Add the parameter TRANLOGOPTIONS HANDLEDLFAILOVER
 - Add the parameter TRANLOGOPTIONS FAILOVERTARGETDESTID <log_archive_dest_number> only if Oracle Data Guard Fast-Start Failover (FSFO) is NOT in use.

10. Click **Create and Run**.

See [Oracle GoldenGate Extract Failure or Error Conditions Considerations](#) for more information.

Step 4.5.2 - Replicat Configuration

Oracle generally recommends using parallel Replicat in non-integrated mode for better performance for most workloads when the GGHub is in the same region as the target database.

Parallel Replicat in integrated mode should only be used when you specifically need features like DML handlers, procedural replication, or automatic conflict detection and resolution (Auto-CDR).

The best apply performance can be achieved when the network latency between the GGHub and the target database is as low as possible.

The following configuration is recommended for the remote Replicat running on the Oracle GGHub.

- APPLY_PARALLELISM – Disables automatic parallelism, instead of using MAX_APPLY_PARALLELISM and MIN_APPLY_PARALLELISM, and allows the highest amount of concurrency to the target database. It is recommended to set this as high as possible based on available CPU of the hub and the target database server.
- MAP_PARALLELISM – Should be set with a value of 2 to 5. With a larger number of appliers, increasing the Mappers increases the ability to hand work to the appliers.
- BATCHSQL – applies DML using array processing which reduces the amount network overheads with a higher latency network. Be aware that if there are many data conflicts, BATCHSQL results in reduced performance, as rollback of the batch operations is followed by a re-read from trail file to apply in non-batch mode.

After you set up database connections and verify them, you can add a Replicat for the deployment by following these steps:

1. Log in to the Oracle GoldenGate **Administration Server**
2. Click the plus (+) sign next to **Replicats** on the Administration Service home page. The Add Replicat page is displayed.
3. Select a Replicat type and click **Next**.
4. Enter the details as follows:
 - Process Name: REP_1

- Description: Replicat for DC 2 PDB
 - Intent: Unidirectional
 - Credential Domain: GoldenGate
 - Credential Alias: DC2_PDB
 - Source: Trail
 - Trail Name: aa
 - Begin: Position in Log
 - Checkpoint Table: "GGADMIN"."CHKP_TABLE"
5. Click **Next**
 6. From the **Action Menu**, click **Details** to edit the Replicat **Parameters**:

```

REPLICAT REP_1
USERIDALIAS Reg2_PDB DOMAIN GoldenGate
MAP <SOURCE_PDB_NAME>.<OWNER>.*, TARGET <OWNER>.*;

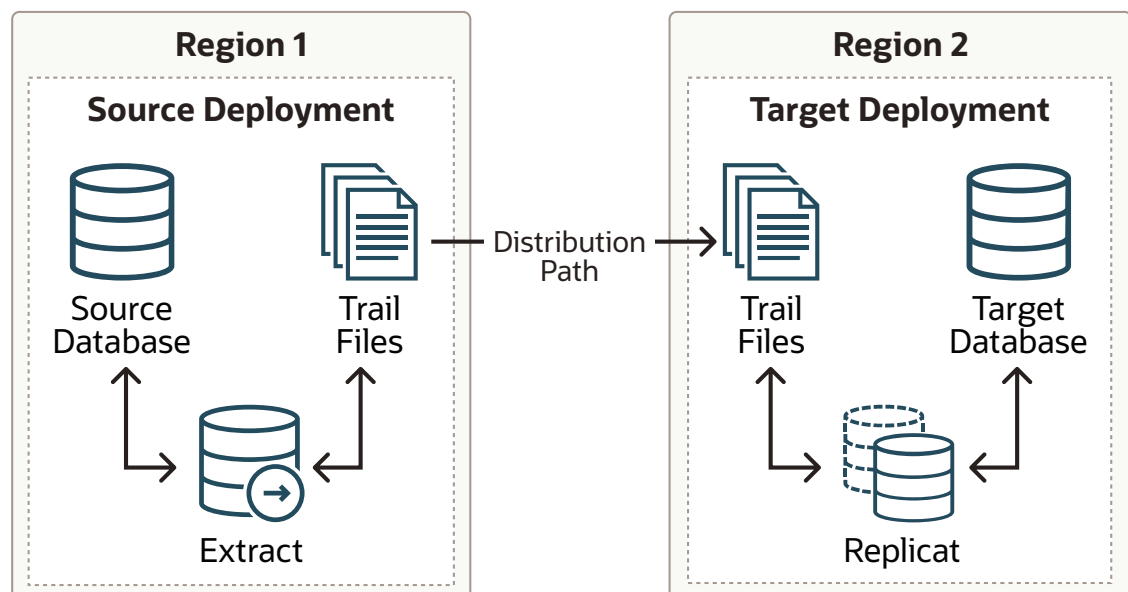
```

7. From the **Action Menu**, click **Start**.

Step 4.5.3 - Distribution Path Configuration

Distribution paths are only necessary when trail files need to be sent to an additional Oracle GoldenGate Hub in a different, or even the same, data center as described in the following figure.

Figure 26-3 Oracle GoldenGate Distribution Path



When using Oracle GoldenGate Distribution paths with the NGINX Reverse Proxy, additional steps must be carried out to ensure the path client and server certificates are configured.

More instructions about creating distribution paths are available in [Add a Distribution Path](#). A step-by-step example is in the following video, "[Connect an on-premises Oracle GoldenGate to OCI GoldenGate using NGINX](#)," to correctly configure the certificates.

Here are the steps performed in this sub-step:

- Step 4.5.3.1 - Download the Target Server's Root Certificate, and then upload it to the source Oracle GoldenGate
- Step 4.5.3.2 - Create a user in the Target Deployment for the Source Oracle GoldenGate to use
- Step 4.5.3.3 - Create a Credential in the Source Oracle GoldenGate
- Step 4.5.3.4 - Create a Distribution Path on the Source Oracle GoldenGate to the Target Deployment
- Step 4.5.3.5 - Distribution Path Recommendations

Step 4.5.3.1 - Download the Target Server's Root Certificate, and then upload it to the source Oracle GoldenGate

Download the target deployment server's root certificate and add the CA certificate to the source deployment Service Manager.

1. Log in to the **Administration Service** on the Target GoldenGate deployment.
2. Follow "Step 2 - Download the target server's root certificate" in the video "[Connect an on-premises Oracle GoldenGate to OCI GoldenGate using NGINX.](#)"

Step 4.5.3.2 - Create a user in the Target Deployment for the Source Oracle GoldenGate to use

Create a user in the target deployment for the distribution path to connect to:

1. Log in to the **Administration Service** on the Target GoldenGate.
2. Click on **Administrator** under **Administration Service**.
3. Click the plus (+) sign next to **Users**.
4. Enter the details as follows:
 - Username: ggnet
 - Role: Operator
 - Type: Password
5. Click **Submit**

Step 4.5.3.3 - Create a Credential in the Source Oracle GoldenGate Deployment

Create a credential in the source deployment connecting the target deployment with the user created in the previous step. For example, a domain of OP2C and an alias of WSSNET.

1. Log in to the **Administration Service** on the Source Oracle GoldenGate.
2. Click in **Configuration** under **Administration Service**.
3. Click the **plus (+)** sign next to **Credentials** on the Database home page.
4. Enter the details as follows:
 - Credential Domain: OP2C
 - Credential Alias: wssnet
 - User ID: ggnet
5. Click **Submit**

Step 4.5.3.4 - Create a Distribution Path on the Source Oracle GoldenGate to the Target Deployment

A path is created to send trail files from the Distribution Server to the Receiver Server. You can create a path from the Distribution Service. To add a path for the source deployment:

1. Log in to the **Distribution Service** on the Source Oracle Goldengate.
2. Click the plus (+) sign next to **Path** on the Distribution Service home page. The Add Path page is displayed.
3. Enter the details as follows:

Option	Description
Path Name	Select a name for the path.
Source: <i>Trail Name</i>	Select the Extract name from the drop-down list, which populates the trail name automatically. If it doesn't, enter the trail name you provided while adding the Extract.
Generated Source URI	Specify localhost for the server's name; this allows the distribution path to be started on any of the Oracle RAC nodes.
Target Authentication Method	Use 'UserID Alias'
Target	Set the Target transfer protocol to wss (secure web socket). Set the Target Host to the target hostname/VIP that will be used for connecting to the target system along with the Port Number that NGINX was configured with (default is 443).
Domain	Set the Domain to the credential domain created above, for example, OP2C.
Alias	The Alias is set to the credential alias wssnet.
Auto Restart Options	Set the distribution path to restart when the Distribution Server starts automatically. This is required, so that manual intervention is not required after a RAC node relocation of the Distribution Server. It is recommended to set the number of Retries to 10. Set the Delay , which is the time in minutes to pause between restart attempts, to 1.

4. Click **Create Path**.
5. From the Action Menu, click **Start**.

Step 4.5.3.5 - Distribution Path Recommendations

If there are any GoldenGate distribution paths sending trail files to the GGHub, after a role transition of the GGHub, the paths will need to be altered to send the trail files to the new primary GGHub system. This can be done using the following example REST call:

```
curl -s -K src_access.cfg
https://Source_VIP/Source_Deployment_Name/distsrvr/services/v2/sources/
Distribution_Path_Name
-X PATCH --data '{"target":{"uri":"ogg://Target_VIP:9103/services/v2/targets?
trail=dd"}}' | python
-m json.tool
```

You can automate changing the source distribution path target address after a hub role transition using the sample shell script shown in [Managing Outages for Oracle GoldenGate Hub](#) which is called by the acfs_standby CRS action script when a file system switchover/failover occurs.

The source distribution paths must be configured to restart automatically after it has failed so that if the target GoldenGate deployment relocates between Oracle RAC nodes or to the standby hub, the distribution paths will restart. If a distribution path was created without automatic restart enabled, it can be enabled through the distribution server web UI or a REST call. For example:

```
$ curl -s -K
  access.cfg https://<Source VIP>/<Source Deployment Name>/distsrvr/
services/v2/sources/ggs_to_gghub
-X PATCH --data '{"options":{"autoRestart":{"delay": 2,"retries": 10}}}' |
python -m json.tool
```

To check the current configuration of a distribution path, use the following example:

```
$ curl -s -K
  access.cfg https://<Source VIP>/<Source Deployment Name>/distsrvr/
services/v2/sources/ggs_to_gghub
-X GET | python -m json.tool
```

```
# Sample output:
"name": "scam_to_gghub",
"options": {
  "autoRestart": {
    "delay": 2,
    "retries": 10
  },
```

Step 4.5.4 - Set up a Heartbeat Table for Monitoring Lag Times

Use the instructions in [Steps to add Heartbeat Table in OCI GoldenGate](#) to implement the best practices for creating a heartbeat process that can be used to determine where and when lag is developing between a source and target system.

This document guides you through the step-by-step process of creating the necessary tables and added table mapping statements needed to keep track of processing times between a source and target database. Once the information is added into the data flow, the information is then stored into a target tables that can be analyzed to determine when and when the lag is introduced between the source and target systems.

Reference

Alternate Method to Generate Ansible Inventory File for Deployment

The Ansible playbook `oracle.gghub.onboarding` can generate the Ansible inventory file for deployment using an alternate method.

The alternate method uses a response file instead of user prompts for all the required values. A template response file is provided in the GGHUB collection directory which must be updated and used with the playbook `oracle.gghub.onboarding`.

Make a copy of the provided template response file from the collection directory and fill in all the parameter values.

Note

A space is required before the parameter value in the response file following the YAML syntax. As per YAML specification, the key-value pairs are marked with a colon and space. For example, `gg_deployment_name: maahub`

```
(gghub) [~]$ cd /u01/maagghub
(ggghub) [~]$ cp gghub-ansible-collection/ggghub_deploy.yml
gghub-ansible-collection/ggghub_maahub.yml
(ggghub) [~]$ vi gghub-ansible-collection/ggghub_maahub.yml
```

Execute the playbook `oracle.gghub.onboarding` using the response file as follows to generate the Ansible inventory file.

```
(gghub) [~]$ ansible-playbook oracle.gghub.onboarding
-i /u01/maagghub/collections/ansible_collections/oracle/ggghub/inventory/
localhost
-e @/u01/maagghub/ggghub-ansible-collection/ggghub_maahub.yml
```

When the playbook `oracle.gghub.onboarding` is executed using a response file it will only prompt for the Ansible vault and GoldenGate admin user passwords.

Alternate Deployment Option to Configure Only Primary or Standby GGHUB

If there is a requirement to deploy only the primary GGHUB or the standby GGHUB initially, use the `limit` option and specify the primary or standby group name from the Ansible inventory file. The `limit` option restricts the execution of a playbook to a specific subset of hosts within the inventory.

In the following example, Ansible automation only deploys GGHUB resources on the primary GGHUB cluster using the `limit` option. ACFS file system will be created.

```
(gghub) [~]$ ansible-playbook oracle.gghub.deploy
-i /u01/maagghub/inventory/maahub.yml --limit maahuba -e @/u01/maagghub/
inventory/maahub.key
--ask-vault-pass
```

Continuing with the example above where just the primary GGHUB is deployed, to proceed with standby GGHUB deployment and ACFS replication setup, run the deploy playbook without the `limit` option. Ansible skips the actions already performed for the primary GGHUB and configures only the standby GGHUB and AFCS replication.

```
(gghub) [~]$ ansible-playbook oracle.gghub.deploy
-i /u01/maagghub/inventory/maahub.yml -e @/u01/maagghub/inventory/maahub.key
--ask-vault-pass
```

Validate GGHUB Deployment

After the deployment is complete, playbook `oracle.gghub.verify` can be executed to display the GGHUB configuration information and status of some of the GGHUB clusterware resources.

```
(gghub) [~] $ ansible-playbook oracle.gghub.verify
-i /u01/maagghub/inventory/maahub.yml --ask-vault-password
```

Example output:

```
TASK [Display GoldeGate status]
*****
ok: [maahuba1] => {
  "msg": [
    {
      "grid": {
        "activeVersion": "19.0.0.0",
        "cluster_nodes": "maahuba1,maahuba2",
        "homePath": "/u01/app/19.0.0.0/grid",
        "master_node": "maahuba1",
        "orabase": "/u01/app/grid",
        "patchVersion": "19.26.0.0",
        "scanListenerTCPPorts": "1521",
        "state": "NORMAL",
        "user": "grid"
      },
      "system": {
        "crs": true,
        "inst_group": "oinstall",
        "inventory_loc": "/u01/app/oraInventory",
        "oracle_ahf_loc": "/u01/app/oracle.ahf",
        "scan_name": "maahuba-
scan.clientsubnet.vcnfraans1.oraclevcn.com"
      }
    },
    {
      "acfs": {
        "device": "/dev/asm/maahub-207",
        "diskgroup": "data",
        "primary_hostname": "maahuba-app-
vip1.clientsubnet.vcnfraans1.oraclevcn.com",
        "primary_path": "/opt/oracle/gghub/maahub",
        "retries_made": "0",
        "site": "primary",
        "standby_connect_string": "grid@maahubb-app-
vip1.clientsubnet.vcnfraans1.oraclevcn.com",
        "standby_path": "/opt/oracle/gghub/maahub",
        "status": "running",
        "volume": "maahub"
      },
      "appvip": {
        "name": "maahuba-app-vip1"
      },
      "crs_status": {
        "acfs_primary": true,
        "acfs_standby": true,
        "ora_acfs": true,
```

```

        "sshd_restart": true,
        "xag_goldengate": true
    },
    "goldengate": {
        "admin_user": "oggadmin",
        "file_system": "acfs_primary,nginx",
        "homePath": "/u01/app/oracle/goldengate/gg23ai",
        "name": "maahub",
        "service_manager_port": "9100",
        "xag_home": " /u01/app/grid/xag"
    }
},
"",
""
]
}

```

GGHub Ansible Playbook oracle.gghub.maa_ha Functions

Playbook `oracle.gghub.maa_ha` provides multiple functions to perform different tasks.

These additional functions are available in the form of tags in the playbook `oracle.gghub.maa_ha`. List the available tags in the playbook as shown below.

```
(gghub) [~] $ ansible-playbook oracle.gghub.maa_ha
-i /u01/maagghub/inventory/maahub.yml --list-tags
```

The following section describes some of the functions in the playbook `oracle.gghub.maa_ha` which are invoked using the specific tag names.

Relocate GGHUB Clusterware Resources

It is highly recommended that you validate the GGHUB clusterware resources high availability and successful role reversal functionality before configuring the GoldenGate extract and replicat processes.

Playbook `oracle.gghub.maa_ha` provides the functionality to relocate GGHUB clusterware resources from the active node of the GGHUB cluster to the available node.

The relocate feature of the playbook `oracle.gghub.maa_ha` can also facilitate the rolling planned maintenance of GGHUB cluster nodes.

Perform the following steps to relocate the GGHUB clusterware resources from the active node to the available node in the cluster:

Step 1. Relocate Primary GGHUB Clusterware Resources

Validate the status of the primary GGHUB clusterware resources before relocating to the available node using the example command in the "Check the Primary and Standby GGHUB Clusterware Resources Status" section of [Troubleshooting GGHUB Deployment Using Ansible](#).

Execute the Ansible playbook `oracle.gghub.maa_ha` with tag `acfs_relocate_primary` to relocate the primary GGHUB clusterware resources from the active node to the available node in the primary cluster.

```
(gghub) [~]$ ansible-playbook oracle.gghub.maa_ha  
-i /u01/maagghub/inventory/maahub.yml --tags acfs_relocate_primary
```

Check the status of the GGHUB clusterware resources after the relocate task is complete. Notice that all the resources should be online on the available node.

Step 2. Relocate Standby GGHUB Clusterware Resources

Validate the status of the standby GGHUB clusterware resources before relocating to the available node using the example command in the "Standby GGHUB Clusterware Resources Status" section of [Troubleshooting GGHUB Deployment Using Ansible](#).

Execute the Ansible playbook `oracle.gghub.maa_ha` with tag `acfs_relocate_standby` to relocate the standby GGHUB clusterware resources from the active node to the available node in the standby cluster.

```
(gghub) [~]$ ansible-playbook oracle.gghub.maa_ha  
-i /u01/maagghub/inventory/maahub.yml --tags acfs_relocate_standby
```

Check the status of the GGHUB clusterware resources after the relocate task is complete. Notice that all of the resources should be online on the available node.

GGHUB Role Reversal

The role of Primary GGHUB can be reversed to a standby GGHUB for planned or unplanned outages and disaster recovery. ACFS replication direction is reversed during this process. Ansible playbook `oracle.gghub.maa_ha` with tag `acfs_role_reversal` can be used to perform GGHUB role reversal.

To perform a role reversal execute the playbook `oracle.gghub.maa_ha` as follows:

```
(gghub) [~]$ ansible-playbook oracle.gghub.maa_ha  
-i /u01/maagghub/inventory/maahub.yml --tags acfs_role_reversal
```

Validate the status of the resources on the new primary and standby GGHUB clusters using the example commands in the "Check the Primary and Standby GGHUB Clusterware Resources Status" section of [Troubleshooting GGHUB Deployment Using Ansible](#).

The new primary GGHUB should show the `acfs_primary` CRS resource state as ONLINE after the role reversal is complete.

Validate the ACFS replication status on the new primary and standby GGHUB cluster using the example commands in the "Check the GGHUB ACFS Mount Replication Status" section of [Troubleshooting GGHUB Deployment Using Ansible](#).

Stopping and Starting GGHUB Clusterware Resources

GGHUB clusterware resources can be stopped and started on the primary and standby GGHUB clusters using specific tags provided in Ansible playbook `oracle.gghub.maa_ha`.

Execute the playbook `oracle.gghub.maa_ha` to accomplish this task as below:

```
(gghub) [~]$ ansible-playbook oracle.gghub.maa_ha
-i /u01/maagghub/inventory/maahub.yml --tags gghub_stop

(ggghub) [~]$ ansible-playbook oracle.gghub.maa_ha
-i /u01/maagghub/inventory/maahub.yml --tags gghub_start
```

To stop just the primary or the standby GGHUB cluster resources use the below tags.

```
(gghub) [~]$ ansible-playbook oracle.gghub.maa_ha
-i /u01/maagghub/inventory/maahub.yml --tags gghub_stop_primary

(ggghub) [~]$ ansible-playbook oracle.gghub.maa_ha
-i /u01/maagghub/inventory/maahub.yml --tags gghub_stop_standby
```

If one of the GGHUB cluster resources are stopped use the `gghub_start` tag to restart. Ansible automation will detect and only start the GGHUB clusterware resources where it is stopped.

Change GoldenGate Microservices Deployment Password Using Ansible

To update the GoldenGate Microservices deployment password execute the Ansible playbook `oracle.gghub.change_password` from the GGHUB Ansible collection.

The playbook will prompt for the Ansible vault password and then prompt for the new GoldenGate Microservices deployment password.

The playbook uses the existing GoldenGate Microservices deployment password from the Ansible vault, and after successful change the GoldenGate Microservices deployment password will be updated in the Ansible vault.

```
(gghub) [~]$ ansible-playbook oracle.gghub.change_password
-i /u01/maagghub/inventory/maahub.yml --ask-vault-password
```

Understanding NGINX Reverse Proxy and Certificate Configuration

This section explains the NGINX reverse proxy configuration and certificate usage in GGHUB clusters.

The GoldenGate reverse proxy feature allows a single point of contact for all the GoldenGate microservices associated with a GoldenGate deployment. Without a reverse proxy, the GoldenGate deployment microservices are contacted using a URL consisting of a hostname or IP address and separate port numbers, one for each of the services. For example, to contact the Service Manager, you could use `http://gghub.example.com:9100`, then the Administration Server is `http://gghub.example.com:9101`, the second Service Manager may be accessed using `http://gghub.example.com:9110`, and so on.

When running Oracle GoldenGate in a High Availability (HA) configuration with the Grid Infrastructure agent (XAG), there is a limitation preventing more than one deployment from being managed by a GoldenGate Service Manager. Because of this limitation, creating a separate virtual IP address (VIP) for each Service Manager/deployment pair is recommended. This way, the microservices can be accessed directly using this VIP.

With a reverse proxy, port numbers are not required to connect to the microservices because they are replaced with the deployment name and the hostname's VIP. For example, to connect to the console via a web browser, use the URLs:

GoldenGate Service	URL
Service Manager	https://gghub.example.com:443
Administration Server	https://gghub.example.com:443/ <deployment_name>/adminsrvr
Distribution Server	https://gghub.example.com:443/ <deployment_name>/distsrvr
Performance Metric Server	https://gghub.example.com:443/ <deployment_name>/pmsrvr
Receiver Server	https://gghub.example.com:443/ <deployment_name>/recvsrvr

When running multiple Service Managers, the following instructions will provide configuration using a separate VIP for each Service Manager. NGINX uses the VIP to determine which Service Manager an HTTPS connection request is routed to.

An SSL certificate is required for clients to authenticate the server they connect to through NGINX. Contact your systems administrator to follow your corporate standards to create or obtain the server certificate before proceeding. A separate certificate is required for each VIP and Service Manager pair.

The Ansible automation configures NGINX reverse proxy with an SSL connection and ensures all external communication is secure. The Ansible automation initially uses a self signed certificate for NGINX configuration. The certificate and key file can be replaced as per corporate security standards for production deployments following the process in

For more details on the NGINX HTTPS configuration that the Ansible automation will deploy refer to NGINX documentation at https://nginx.org/en/docs/http/configuring_https_servers.html.

Note

The common name in the CA-signed certificate must match the target hostname/VIP used by NGINX.

On-Premises: Oracle GoldenGate Microservices Architecture with Oracle Real Application Clusters Configuration Best Practices

Use these best practices for configuring Oracle GoldenGate Microservices Architecture for on-premises systems, including Oracle Exadata, to work with Oracle Real Application Clusters (RAC), Oracle Clusterware, and Oracle Database File System (DBFS) or Oracle Advanced Cluster File System (ACFS).

The target Oracle RAC system that hosts Oracle GoldenGate Microservices Architecture can act as the source database, as the target database, or in some cases as both source and target databases, for Oracle GoldenGate operations.

See the following topics:

- [Summary of Recommendations when Deploying Oracle GoldenGate on Oracle RAC](#)
- [Task 1: Configure the Oracle Database for Oracle GoldenGate](#)
- [Task 2: Create the Database Replication Administrator User](#)
- [Task 3: Create the Database Services](#)
- [Task 4: Set Up a File System on Oracle RAC](#)
- [Task 5: Install Oracle GoldenGate](#)
- [Task 6: Create the Oracle GoldenGate Deployment](#)
- [Task 7: Oracle Clusterware Configuration](#)
- [Task 8: Configure NGINX Reverse Proxy](#)
- [Task 9: Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections](#)
- [Task 10: Configure Oracle GoldenGate Processes](#)
- [Task 11: Configure Autostart of Extract and Replicat Processes](#)

Summary of Recommendations when Deploying Oracle GoldenGate on Oracle RAC

When configuring Oracle GoldenGate in an Oracle RAC environment, follow these recommendations.

- Install the latest version of Oracle GoldenGate software locally on each Oracle RAC node, making sure that the software location is the same on all Oracle RAC nodes.
- Use the Oracle Database File System (DBFS) or Oracle Advanced Cluster File System (ACFS) for the file system where the Oracle GoldenGate files are stored (trail, checkpoint, temporary, report, and parameter files).

- Use the same DBFS or ACFS mount point on all of the Oracle RAC nodes that may run Oracle GoldenGate.
- When creating the GoldenGate deployment, specify either DBFS or ACFS for the deployment location.
- Install Grid Infrastructure agent (XAG) version 10 or later on all Oracle RAC nodes that will run Oracle GoldenGate.
- Configure the GoldenGate processes to automatically start and restart when the deployment is started.

Task 1: Configure the Oracle Database for Oracle GoldenGate

The source and target Oracle GoldenGate databases should be configured using the following recommendations.

- Enable Oracle GoldenGate replication by setting the database initialization parameter `ENABLE_GOLDENGATE_REPLICATION=TRUE`.
- Run the Oracle GoldenGate source database in `ARCHIVELOG` mode.
- Enable `FORCE LOGGING` mode in the Oracle GoldenGate source database.
- Enable minimal supplemental logging in the source database. Additionally, add schema or table level logging for all replicated objects.
- If the Replicat process will be used, configure the streams pool in the System Global Area (SGA) on the source database using the `STREAMS_P00L_SIZE` initialization parameter.

Note that the streams pool is only needed on the target database if integrated Replicat will be used.

Use the following equation to determine the value for `STREAMS_P00L_SIZE`:

`STREAMS_P00L_SIZE = (#Extracts and #Integrated Replicats * 1GB) * 1.25`

For example, in a database with 2 Extracts and 2 integrated Replicats:

`STREAMS_P00L_SIZE = 4GB * 1.25 = 5GB`

When adding Extract or Replicat processes, it is important to recalculate and configure the new streams pool size requirement.

For more information about preparing the database for Oracle GoldenGate, see [Preparing the Database for Oracle GoldenGate](#).

Task 2: Create the Database Replication Administrator User

The source and target Oracle databases need a GoldenGate Administrator user with appropriate privileges assigned.

For single tenant (non-CDB architecture) databases, see [Establishing Oracle GoldenGate Credentials](#)

For a multitenant source database, GoldenGate Extract must be configured to connect to a user in the root container database, using a c## account. For a multitenant target database, a separate GoldenGate administrator user is needed for each PDB that a Replicat applies data to.

For more details about creating a GoldenGate Administrator in an Oracle Multitenant Database, see [Configuring Oracle GoldenGate in a Multitenant Container Database](#)

Task 3: Create the Database Services

A database service is required so that the Oracle Grid Infrastructure Agent automatically starts the GoldenGate deployment when the database is opened. When DBFS is used for the shared file system, the database service is also used to mount DBFS to the correct Oracle RAC instance.

When using a source multitenant database, a separate service is required for the root container database (CDB) and the pluggable database (PDB) that contains the schema being replicated. For a target multitenant database, a single service is required for the PDB.

Create the service using the following command, as the oracle user.

```
$ srvctl add service -db db_name -service service_name  
-preferred instance_1 -available instance_2, instance_3 etc.  
-pdb PDB_name
```

For example:

```
$ srvctl add service -db ggdb -service oggserv_pdb -preferred ggdb1  
-available ggdb2 -pdb GGPDB01
```

If you are not using Oracle Multitenant Database, omit the -pdb parameter.

Task 4: Set Up a File System on Oracle RAC

Oracle GoldenGate Microservices Architecture is designed with a simplified installation and deployment directory structure. The installation directory should be placed on local storage on each Oracle RAC node to provide minimized downtime during software patching.

The deployment directory, which is created during deployment creation using the Oracle GoldenGate Configuration Assistant (oggca.sh), must be placed on a shared file system. The deployment directory contains configuration, security, log, parameter, trail, and checkpoint files.

Placing the deployment in DBFS or ACFS provides the best recovery and failover capabilities in the event of a system failure. Ensuring the availability of the checkpoint files cluster-wide is essential so that after a failure occurs the GoldenGate processes can continue running from their last known position.

If Oracle GoldenGate will be configured along with Oracle Data Guard, the recommended file system is DBFS. DBFS is contained in the database protected by Data Guard, and can be fully integrated with XAG. In the event of a Data Guard role transition, the file system can be automatically mounted on the new primary server, followed by automated start-up of Oracle GoldenGate. This is currently not possible with ACFS, because it is not part of the Oracle Data Guard configuration.

Follow the instructions in the appropriate section below to configure the file system for either DBFS or ACFS.

Oracle Database File System (DBFS)

It is required that you create the DBFS tablespace inside the same database that the Oracle GoldenGate processes are connected to. For example, if a GoldenGate integrated Extract

process is extracting from a database called GGDB, the DBFS tablespace would be located in the same GGDB database.

Follow instructions in [My Oracle Support note 869822.1](#) to install the required FUSE libraries if they are not already installed.

Use the instructions in [My Oracle Support note 1054431.1](#) to configure the database, tablespace, database user, tnsnames.ora Oracle Net connection alias, and permissions on source or target GoldenGate environments required for DBFS.

Note

When using an Oracle Multitenant Database, the DBFS tablespace **MUST** be created in a Pluggable Database (PDB). It is recommended that you use the same PDB that the GoldenGate Extract or Replicat processes are connecting to, allowing DBFS to use the same database service, created above in Task 2, for its database dependency.

When you create a file system for storing the GoldenGate deployment files, it is recommended that you allocate enough trail file disk space to permit storage of up to 12 hours of trail files. Doing this provides sufficient space for trail file generation should a problem occur with the target environment that prevents it from receiving new trail files. The amount of space needed for 12 hours can only be determined by testing trail file generation rates with real production data.

Example DBFS creation:

```
$ cd $ORACLE_HOME/rdbms/admin
$ sqlplus dbfs_user/dbfs_password@database_tns_alias
SQL> start dbfs_create_filesystem dbfs_gg_tbs goldengate
```

Follow the instructions in [My Oracle Support note 1054431.1](#) to configure the newly created DBFS file system so that the DBFS instance and mount point resources are automatically started by Cluster Ready Services (CRS) after a node failure, with the following DBFS configuration and script file modifications.

1. Change the mount-dbfs.conf parameters to reflect your database environment.

Modify the MOUNT_OPTIONS parameter to the following:

```
MOUNT_OPTIONS=allow_other,direct_io,failover,nolock
```

The failover option forces all file writes to be committed to the DBFS database in an IMMEDIATE WAIT mode. This prevents data getting lost when it has been written into the dbfs_client cache but not yet written to the database at the time of a database or node failure.

The nolock mount option is required if you are using Oracle Database 18c or a later release, due to a change in the DBFS file locking which can cause issues for GoldenGate processes after an Oracle RAC node failure when a file is currently locked.

If you are using a dbfs_client from Oracle Database 12c Release 2 (12.2), make sure you have applied the latest release update that includes the fix for bug 27056711. Once the fix has been applied, the MOUNT_OPTIONS should also include the nolock option.

2. Modify the `mount-dbfs.sh` script to force unmounting of DBFS when the CRS resource is stopped.

Change two occurrences of:

```
$FUSERMOUNT -u $MOUNT_POINT
```

To the following:

```
$FUSERMOUNT -uz $MOUNT_POINT
```

3. When registering the resource with Oracle Clusterware, be sure to create it as a `cluster_resource` instead of a `local_resource`, as specified in the My Oracle Support note.

The reason for using `cluster_resource` is so that the file system can only be mounted on a single node at a time, preventing mounting of DBFS from concurrent nodes, which creates the potential for concurrent file writes, causing file corruption problems.

Make sure to use the database service name created in a previous step for the DBFS service dependency.

For example:

```
DBNAME=ggdb
```

```
DEPNAME=ora.$DBNAME.oggserv.svc
```

```
crsctl add resource $RESNAME \  
-type cluster_resource \  
-attr "ACTION_SCRIPT=$ACTION_SCRIPT, \  
CHECK_INTERVAL=30,RESTART_ATTEMPTS=10, \  
START_DEPENDENCIES='hard($DEPNAME)pullup($DEPNAME)',\  
STOP_DEPENDENCIES='hard($DEPNAME)',\  
SCRIPT_TIMEOUT=300"
```

Once the DBFS resource has been created, the file system should be mounted and tested.

```
$ crsctl start res dbfs_mount  
$ crsctl stat res dbfs_mount
```

After the file system is mounted, create the directory for storing the GoldenGate files.

```
$ cd /mnt/dbfs/goldengate  
$ mkdir deployments
```

Note

Leave the shared file system mounted. It is required for creating the GoldenGate deployment in a later step.

Oracle Advanced Cluster File System (ACFS)

Oracle ACFS is an alternative to DBFS for the shared GoldenGate files in an Oracle RAC configuration.

Refer to [My Oracle Support note 1929629.1](#) for more information about ACFS configuration requirements for Oracle Exadata Database Machine.

Create a single ACFS file system for storing the Oracle deployment files.

It is recommended that you allocate enough trail file disk space to permit storage of up to 12 hours of trail files. Doing this provides sufficient space for trail file generation should a problem occur with the target environment that prevents it from receiving new trail files. The amount of space needed for 12 hours can only be determined by testing trail file generation rates with real production data.

1. Create the file system using ASMCMD as the Oracle ASM administrator user.

```
ASMCMD [+] > volcreate -G datac1 -s 1200G ACFS_GG
```

Note

Modify the file system size according to the determined size requirements.

```
ASMCMD> volinfo -G datac1 acfs_gg
```

```
Diskgroup Name: DATA1
Volume Name: ACFS_GG
Volume Device: /dev/asm/acfs_gg-151
State: ENABLED
Size (MB): 1228800
Resize Unit (MB): 64
Redundancy: MIRROR
Stripe Columns: 8
Stripe Width (K): 1024
Usage:
Mountpath:
```

Make the file system with the following mkfs command.

```
$ /sbin/mkfs -t acfs /dev/asm/acfs-gg-151
```

2. Create the CRS resource for the newly created ACFS file system, if not already created. Check to see if the file system resource was already created.

```
$ srvctl status filesystem -volume ACFS_GG -diskgroup DATA1
```

```
ACFS file system /mnt/acfs_gg is mounted on nodes oggadm07, oggadm08
```

If not already created, create the ACFS mount point on all of the Oracle RAC nodes.

```
# mkdir -p /mnt/acfs_gg
```

Create the file system resource as the root user. Due to the implementation of distributed file locking on ACFS, unlike DBFS, it is acceptable to mount ACFS on more than one RAC node at any one time.

Create the ACFS resource using `srvctl` from the Oracle Grid Infrastructure `ORACLE_HOME`.

```
# srvctl add filesystem -device /dev/asm/acfs_gg-151 -volume ACFS_GG  
-diskgroup DATA1 -path /mnt/acfs_gg -user oracle -autostart RESTORE
```

To verify the currently configured ACFS file systems, use the following command to view the file system details.

```
$ srvctl config filesystem  
  
Volume device: /dev/asm/acfs_gg-151  
Diskgroup name: data1  
Volume name: ACFS_GG  
Canonical volume device: /dev/asm/acfs_gg-151  
Accelerator volume devices:  
Mountpoint path: /mnt/acfs_gg  
Mount point owner: oracle
```

Check the status of the ACFS resource and mount it.

```
$ srvctl status filesystem -volume ACFS_GG -diskgroup DATA1
```

ACFS file system /mnt/acfs is not mounted

```
$ srvctl start filesystem -volume ACFS_GG -diskgroup DATA1 -node  
dc1north01
```

The CRS resource that is created is named using the format `ora.diskgroup_name.volume_name.acfs`. Using the above file system example, the CRS resource is called `ora.data1.acfs_gg.acfs`.

To see all ACFS file system CRS resources that currently exist, use the following command.

```
$ crsctl stat res -w "((TYPE = ora.acfs.type) OR (TYPE =  
ora.acfs_cluster.type))"
```

```
NAME=ora.data1.acfs_gg.acfs  
TYPE=ora.acfs.type  
TARGET=ONLINE , OFFLINE  
STATE=ONLINE on dc1north01, OFFLINE
```

3. Create a GoldenGate deployment directory on ACFS.

After the file system is mounted, create the directory for storing the GoldenGate deployments.

```
$ cd /mnt/acfs_gg  
$ mkdir deployments
```

Note

Leave the shared file system mounted. It is required for creating the GoldenGate deployment in a later Task.

Task 5: Install Oracle GoldenGate

Download and install the Oracle GoldenGate 21c Microservices software, or later release.

Download the software at <https://www.oracle.com/middleware/technologies/goldengate-downloads.html>.

Install the Oracle GoldenGate software **locally** on all nodes in the Oracle RAC configuration that will be part of the GoldenGate configuration. Make sure the installation directory is **identical** on all nodes.

Follow the generic installation instructions detailed in [Oracle GoldenGate Microservices Documentation](#).

Task 6: Create the Oracle GoldenGate Deployment

Once the Oracle GoldenGate software has been installed, the next step is to create a deployment using the Oracle GoldenGate Configuration Assistant (oggca).

There are two limitations that currently exist with Oracle GoldenGate and XAG:

1. A Service Manager that is registered with XAG can only manage a single deployment. If multiple deployments are required, each deployment must use their own Service Manager. Oracle GoldenGate release 21c simplifies this requirement because it uses a single deployment to support Extract and Replicat processes connecting to different versions of the Oracle Database.
2. Each Service Manager registered with XAG must belong to separate OGG_HOME software installation directories. Instead of installing Oracle GoldenGate multiple times, the recommended approach is to install Oracle GoldenGate one time, and then create a symbolic link for each Service Manager OGG_HOME.

For example:

```
$ echo $OGG_HOME  
/u01/oracle/goldengate/gg21c_MS  
  
$ ln -s /u01/oracle/goldengate/gg21c_MS /u01/oracle/goldengate/  
gg21c_MS_ggnorth  
  
$ export OGG_HOME=/u01/oracle/goldengate/gg21c_MS_ggnorth  
$ $OGG_HOME/bin/oggca.sh
```

The symbolic link and OGG_HOME environment variable must be configured **before** running the Oracle GoldenGate Configuration Assistant on all Oracle RAC nodes.

Recommendations for creating the GoldenGate deployment in the Oracle GoldenGate Configuration Assistant are as follows.

1. In Service Manager Options, specify the following for the creation of a new Service Manager.
 - a. In the Service Manager Details pane, select **Create New Service Manager**.
 - b. Enter the **Service Manager Deployment Home** location on the shared DBFS or ACFS file system.
 - c. Select to **Integrate with XAG**.
 - d. In the Service Manager Connection Details pane, specify localhost in the **Listening hostname/address** field.

Using localhost allows the deployment to be started on all of the Oracle RAC nodes without the need for a Virtual IP address (VIP).
 - e. Enter the port number in **Listening port**.

2. In **Deployment Directories**, specify the Deployment home directory on the shared DBFS or ACFS file system.
3. In **Environment Variables**, specify a correct **TNS_ADMIN** directory.

Starting with Oracle GoldenGate release 21.3, a database ORACLE_HOME is no longer required because the required database libraries are installed as part of the Oracle GoldenGate installation. It is recommended that you use TNS_ADMIN directory outside of any existing ORACLE_HOME directories.

4. In Security Options, do **not** select **SSL/TLS Security**.

External access to the Oracle GoldenGate Microservices server is achieved by using NGINX Reverse Proxy SSL-termination. Secure access and communication to the GoldenGate deployments will be exclusively through the SSL port 443. Internal connectivity within the same local host between NGINX and GoldenGate does not require SSL.

5. In **Port Settings**, if the Management Pack for Oracle GoldenGate has been licensed, select **Enable Monitoring** to use the performance metric server using either Berkeley Database (BDB) or Lightning Memory Database (LMDB).

For both BDB and LMDB Metrics Service DataStore types, set the **Metrics Service DataStore home** directory to a local directory that exists on all Oracle RAC nodes. For example: `/u01/oracle/goldengate/datastores/deployment_name`

6. Continue through the Oracle GoldenGate Configuration Assistant until the deployment is created.
7. After the deployment has been created, if you are using DBFS for the shared file system and the database version is a release earlier than Oracle Database Release 21c (21.3), run the following commands to move the Oracle GoldenGate deployment temp directory from DBFS to local storage.

On the first node:

```
$ cd <DBFS GoldenGate deployment home directory/var
$ mkdir -p local_storage_directory/deployment_name
$ mv temp local_storage_directory/deployment_name
$ ln -s local_storage_directory/deployment_name/temp temp
```

On all other nodes:

```
$ mkdir local_storage_directory/deployment_name/temp
```

First node example:

```
$ cd /mnt/dbfs/goldengate/deployments/ggnorth/var
$ mkdir -p /u01/oracle/goldengate/deployments/ggnorth
$ mv temp /u01/oracle/goldengate/deployments/ggnorth
$ ln -s /u01/oracle/goldengate/deployments/ggnorth/temp temp
```

On all other nodes:

```
$ mkdir /u01/oracle/goldengate/deployments/ggnorth/temp
```

Task 7: Oracle Clusterware Configuration

The following procedure shows you how to configure Oracle Clusterware to manage Oracle GoldenGate using the Oracle Grid Infrastructure Standalone Agent (XAG).

Using XAG automates the mounting of the shared file system (DBFS or ACFS) and the stopping and starting of the GoldenGate deployment when relocating between Oracle RAC nodes.

1. Install the Oracle Grid Infrastructure Standalone Agent.

It is recommended that you install the XAG software as a standalone agent outside of the Grid Infrastructure ORACLE_HOME. This allows you to use the latest XAG release available, and the software can be updated without impact to the Grid Infrastructure.

When using Oracle GoldenGate Microservices Architecture you **MUST** use XAG version 10.2 or later.

The latest agent software is available for download from the following location:

<http://www.oracle.com/technetwork/database/database-technologies/clusterware/downloads/xag-agents-downloads-3636484.html>

Install the XAG standalone agent outside of the Oracle Grid Infrastructure home directory. XAG must be installed in the same directory on all RAC nodes in the cluster where Oracle GoldenGate is installed.

For example, as the Oracle Grid Infrastructure user, the default of oracle:

```
$ ./xagsetup.sh --install --directory /u01/oracle/xag --all_nodes
```

Add the location of the newly installed XAG software to the PATH variable so that the location of agctl is known when the oracle user logs on to the machine.

```
$ cat .bashrc
export PATH=/u01/oracle/xag/bin:$PATH
```

Note

It is important to make sure that the XAG bin directory is specified BEFORE the Grid Infrastructure bin directory, to ensure the correct agctl binary is found. Set this location in the oracle user environment to take effect at time of logging on, such as in the .bashrc file when the Bash shell is in use.

2. Prepare for Application Virtual IP Address (VIP) Creation.

A dedicated application VIP is required to allow access to the GoldenGate Microservices using the same host name, regardless of which Oracle RAC node is hosting the services. An application VIP also ensures that the GoldenGate Distribution Server can communicate with the Distribution Receiver running the current Oracle RAC node.

The VIP is a cluster resource that Oracle Clusterware manages. The VIP is assigned to a cluster node and is automatically migrated to another node in the event of a node failure.

There are two pieces of information needed before creating the application VIP:

- The network number, which can be identified using the following command.

```
$ crsctl status resource -p -attr ADDRESS_TYPE,NAME,USR_ORA_SUBNET -w
"TYPE = ora.network.type" |sort | uniq
```

```
ADDRESS_TYPE=IPV4
NAME=ora.net1.network
```

```
USR_ORA_SUBNET=10.133.16.0
```

The net1 in NAME=ora.net1.network indicates that this is network 1, and it is of type IPV4.

- The IP address for the new Application VIP, provided by your system administrator. This IP address must be in the same subnet of the cluster environment as determined above.

The VIP will be created in the next step, when configuring the Oracle Grid Infrastructure Agent.

3. Configure Oracle Grid Infrastructure Agent (XAG).

Oracle GoldenGate must be registered with XAG so that the deployment is started and stopped automatically when the database is started and the file system is mounted.

To register Oracle GoldenGate Microservices Architecture with XAG use the following command format.

```
agctl add goldengate instance_name
--gg_home GoldenGate_Home
--service_manager
--config_home GoldenGate_SvcMgr_Config
--var_home GoldenGate_SvcMgr_Var_Dir
--port port_number
--oracle_home $OGG_HOME/lib/instantclient
--adminuser OGG_admin_user
--user GG_instance_user
--group GG_instance_group
--network network_number
--ip ip_address
--vip_name vip_name
--filesystems CRS_resource_name
--db_services service_name
--use_local_services
--nodes node1, node2, ... ,nodeN
```

Where:

--gg_home specifies the location of the Oracle GoldenGate software. Specify the OGG_HOME symbolic link for the OGG_HOME if registering multiple Service Managers (see Task 6: Create the Oracle GoldenGate Deployment).

--service_manager indicates this is a GoldenGate Microservices instance.

--config_home specifies the GoldenGate Service Manager deployment configuration home directory.

--var_home specifies the GoldenGate Service Manager deployment variable home directory.

--port specifies the deployment Service Manager port number.

--oracle_home specifies the location of the Oracle database libraries that are included as part of Oracle GoldenGate 21c and later releases. Example: \$OGG_HOME/lib/instantclient

--adminuser specifies the Oracle GoldenGate Microservices administrator account name.

--user specifies the name of the operating system user that owns the GoldenGate deployment.

--group specifies the name of the operating system group that owns the GoldenGate deployment.

--network specifies the network subnet for the VIP, determined above.

--ip specifies the IP address for the VIP, which was determined above. If you have already created a VIP, then specify it using the --vip_name parameter in place of --network and --ip.

--vip_name specifies a CRS resource name for an application VIP that has previously been created. This parameter replaces --network and --ip (optional).

--filesystems specifies the DBFS or ACFS CRS file system resource that must be mounted before the deployment is started.

--db_services specifies the ora.database.service_name.svc service name that was created in the previous step. If using Oracle Multitenant Database, specify the PDB database service for Replicat, or the CDB database service for an Extract. If using both Replicat and Extract, specify both services names, separated by a comma.

--use_local_services specifies that the GoldenGate instance must be co-located on the same Oracle RAC node where the db_services service is running.

--nodes specifies which of the Oracle RAC nodes this GoldenGate instance can run on. If GoldenGate is configured to run on any of the Oracle RAC nodes in the cluster, this parameter should still be used to determine the preferred order of nodes to run Oracle GoldenGate.

Notes:

- The GoldenGate instance registration with XAG **MUST** be run as the root user.
- The user and group parameters are mandatory because the GoldenGate registration with XAG is run as the root user.

Below are some examples of registering Oracle GoldenGate with XAG.

Example 1: Oracle RAC cluster using DBFS, using an already created application VIP

```
# agctl add goldengate GGNORTH \  
--gg_home /u01/oracle/goldengate/gg21c_MS \  
--service_manager \  

```

```
--config_home /mnt/dbfs/goldengate/deployments/ggsm01/etc/conf \
--var_home /mnt/dbfs/goldengate/deployments/ggsm01/var \
--port 9100 \
--oracle_home /u01/oracle/goldengate/gg21c_MS/lib/instantclient
--adminuser oggadmin
--user oracle \
--group oinstall \
--vip_name gg_vip_prmy \
--filesystems dbfs_mount \
--db_services ora.ds19c.oggserv.svc \
--use_local_services \
--nodes dc1north01,dc1north02
```

Where:

- GoldenGate instance is GGNORTH
- GoldenGate home directory is /u01/oracle/goldengate/gg21c_MS
- This is an Oracle GoldenGate Microservices Architecture instance (--service_manager)
- GoldenGate deployment configuration home directory is /mnt/dbfs/goldengate/deployments/ggsm01/etc/conf
- GoldenGate deployment variable home directory is /mnt/dbfs/goldengate/deployments/ggsm01/var
- Deployment Service Manager port number is 9100
- Oracle GoldenGate Microservices administrator account name is oggadmin
- The GoldenGate user is oracle in the group oinstall
- Application VIP name, managed by CRS, is called gg_vip_prmy
- The CRS resource name for the file system the deployment depends on is dbfs_mount
- The GoldenGate instance will be started on the same Oracle RAC node as the CRS service called ora.ds19c.oraserv.svc will be co-located on the same node as this GoldenGate instance.

Example 2: Oracle RAC cluster, using ACFS, with an application VIP running on a subset of the nodes in the cluster.

```
# agctl add goldengate GGNORTH \
--gg_home /u01/oracle/goldengate/gg21c_MS \
--service_manager \
--config_home /mnt/acfs/goldengate/deployments/ggsm01/etc/conf \
--var_home /mnt/acfs/goldengate/deployments/ggsm01/var \
--port 9100 \
--oracle_home /u01/oracle/goldengate/gg21c_MS/lib/instantclient
--adminuser admin \
--user oracle \
--group oinstall \
--network 1 --ip 10.13.11.203 \
--filesystems ora.datac1.acfs_gg.acfs \
--db_services ora.ds19c.oraserv.svc \
--use_local_services \
--nodes dc1north01,dc1north02
```

Where:

- GoldenGate instance is GGNORTH
- GoldenGate home directory is /u01/oracle/goldengate/gg21c_MS
- This is an Oracle GoldenGate Microservices Architecture instance (--service_manager)
- GoldenGate deployment configuration home directory is /mnt/acfs/goldengate/deployments/ggsm02/etc/conf
- GoldenGate deployment variable home directory is /mnt/acfs/goldengate/deployments/ggsm02/var
- Deployment Service Manager port number is 9100
- Oracle GoldenGate Microservices administrator account name is admin
- GoldenGate user is oracle in the group oinstall
- The network is the default ora.net1.network and the VIP is 10.13.11.203
- The CRS resource name for the file system the deployment depends on is ora.datac1.acfs_gg.acfs
- This GoldenGate instance will be started on the same Oracle RAC node as the CRS service called ora.ds19c.oraserv.svc will be co-located on the same node as this GoldenGate instance
- Oracle GoldenGate will only run on Oracle RAC nodes dc1north01 and dc1north02, listed in priority order.

Example AGCTL Commands

Below are some example agctl commands that are used to manage the Oracle GoldenGate deployment with XAG.

To check the status of Oracle GoldenGate:

```
% agctl status goldengate
Goldengate instance 'GGNORTH' is running on dc1north01
```

To start the GoldenGate deployment, and all Extract/Replicat processes that have been configured to autostart (instructions in a later step):

```
% agctl start goldengate GGNORTH --node dc1north02
```

To stop the GoldenGate deployment:

```
% agctl stop goldengate GGNORTH
```

To manually relocate the GoldenGate deployment to another node:

```
% agctl relocate goldengate GGNORTH --node dc1north02
```

To view the configuration parameters for the GoldenGate resource:

```
% agctl config goldengate GGNORTH
```

Instance name: GGNORTH

```
Application GoldenGate location is: /u01/oracle/goldengate/gg21c_MS
Goldengate MicroServices Architecture environment: yes
Goldengate Service Manager configuration directory: /mnt/dbfs/goldengate/
deployments/ggsm01/etc/conf

Goldengate Service Manager var directory: /mnt/dbfs/goldengate/deployments/
ggsm01/var

Service Manager Port: 9100
Goldengate Administration User: oggadmin
Autostart on DataGuard role transition to PRIMARY: no
Configured to run on Nodes: dc1north01 dc1north02
ORACLE_HOME location is: /u01/oracle/goldengate/gg21c_MS/lib/instantclient
Database Services needed: ora.cdb1.oggcdb.svc [use_local_services]
File System resources needed: ora.dataac1.acfs_gg.acfs
Network: 1, IP: 10.13.11.203, User:oracle, Group:oinstall
```

To delete the GoldenGate XAG resource:

```
$ agctl stop goldengate GGNORTH
# agctl remove goldengate GGNORTH
```

For more information about the Oracle Grid Infrastructure Bundled Agent see [Oracle Grid Infrastructure Standalone Agents for Oracle Clusterware 11g Rel. 2, 12c, 18c and 19c](#).

Task 8: Configure NGINX Reverse Proxy

Follow the instructions provided in [My Oracle Support note 2826001.1](#) to install and configure NGINX Reverse Proxy with SSL connection, and to ensure all external communication is completely secure.

Note

When using CA Signed Certificates with NGINX, make sure the NGINX `ssl_certificate` parameter points to a certificate file that contains the certificates in the correct order of CA signed certificate, intermediate certificate and root certificate.

Oracle Clusterware needs to have control over starting the NGINX reverse proxy so that it can be started automatically before the GoldenGate deployments are started.

The NGINX resource is created with a dependency on the underlying network CRS resource, the name of which can be determined using the following command:

```
$ $GRID_HOME/bin/crsctl stat res -w "TYPE == ora.network.type"|grep NAME

NAME=ora.net1.network
```

As the root user, use the following example command to create a Clusterware resource to manage NGINX.

```
# $GRID_HOME/bin/crsctl add resource nginx -type generic_application -attr
  "ACL='owner:root:rwx,pgrp:root:rwx,other::r--,group:oinstall:r-
x,user:oracle:rwx',
EXECUTABLE_NAMES=nginx,START_PROGRAM='/bin/systemctl
  start -f nginx',STOP_PROGRAM='/bin/systemctl
  stop -f nginx',CHECK_PROGRAMS='/bin/systemctl
  status nginx',START_DEPENDENCIES='hard(ora.net1.network)
  pullup(ora.net1.network)',
STOP_DEPENDENCIES='hard(intermediate:ora.net1.network)',
  RESTART_ATTEMPTS=0, HOSTING_MEMBERS='dc1north01,dc1north02', CARDINALITY=2"
```

The NGINX resource created in this example run on the named cluster nodes at the same time, specified by HOSTING_MEMBERS. This is recommended when multiple GoldenGate Service Manager deployments are configured, and they can independently move between cluster nodes.

Once the NGINX Clusterware resource is created, alter the GoldenGate XAG resources so that NGINX must be started before the GoldenGate deployments are started.

As the oracle user, modify the XAG resources using the following example commands.

Determine the current --filesystems parameter:

```
$ agctl config goldengate SOURCE|grep "File System"
```

File System resources needed: ora.datac1.acfs_gg.acfs

Modify the --filesystems parameter:

```
$ agctl modify goldengate SOURCE --filesystems ora.datac1.acfs_gg.acfs,nginx
```

Repeat the above commands for each of the XAG GoldenGate registrations relying on NGINX.

Task 9: Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections

Create a TNS alias on **all** of the Oracle RAC nodes where Oracle GoldenGate may be started to provide local database connections for the GoldenGate processes when switching between Oracle RAC nodes. Create the TNS alias in the tnsnames.ora file in the TNS_ADMIN directory specified in the deployment creation.

If the source database is a multitenant database, two TNS alias entries are required: one for the container database (CDB) and one for the pluggable database (PDB) that is being replicated. For a target multitenant database, the TNS alias connects the PDB where replicated data is being applied to. The pluggable database SERVICE_NAME should be set to the database service created in an earlier step (refer to [Task 3: Create the Database Services](#)).

Below are some example source database TNS alias definitions using the IPC protocol, which must be defined locally on all RAC nodes.

```
OGGSOURCE_CDB =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL=IPC)(KEY=LISTENER))
    (CONNECT_DATA =
      (SERVICE_NAME = oggserv_cdb)
    )
  )

OGGSOURCE_PDB =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL=IPC)(KEY=LISTENER))
    (CONNECT_DATA =

      (SERVICE_NAME = oggserv_pdb)
    )
  )
```

Note

When the `tnsnames.ora` or `sqlnet.ora`, located in the `TNS_ADMIN` directory for the GoldenGate deployment, are modified, the deployment needs to be restarted in order to pick up the changes.

With the GoldenGate deployment created, use the Administration Server home page to create the database credentials using the above TNS alias names. See Figure 6 below for an example of the database credential creation using the TNS alias appended to the database user name in the 'User ID' field.

If the source database is a multitenant database, create database credentials for the CDB and PDB. If the target database is a multitenant database, create a single credential for the PDB.

Task 10: Configure Oracle GoldenGate Processes

When creating Extract, Distribution Paths, and Replicat processes with Oracle GoldenGate Microservices Architecture, all files that need to be shared between Oracle RAC nodes are already shared with the deployment files stored on a shared file system (DBFS or ACFS).

Listed below are important configuration details that are recommended for running Oracle GoldenGate Microservices on Oracle RAC for Extract, Distribution Paths and Replicat processes.

Extract Configuration

1. When creating an Extract using the Oracle GoldenGate Administration Server GUI interface, leave the **Trail SubDirectory** parameter blank, so that the trail files are automatically created in the deployment directories stored on the shared file system.
The default location for trail files is the `</deployment directory>/var/lib/data`
2. If you are using DBFS for shared storage, and the deployment `var/temp` directory was moved to local storage as described in [Task 6: Create the Oracle GoldenGate Deployment](#),

it is recommended that you use the Extract CACHEDIRECTORY parameter to place the temporary cache files on the shared storage.

Create a new directory under the DBFS deployment mount point. For example:

```
$ mkdir -p /mnt/dbfs/goldengate/deployments/ggnorth/temp_cache
```

Set the Extract parameter to the new directory:

```
CACHEMGR CACHEDIRECTORY /mnt/dbfs/goldengate/deployments/ggnorth/temp_cache
```

Shown below is an example of how the parameters specified for an integrated Extract with the Oracle GoldenGate Administration Server GUI looks in the UI.

Figure 27-1 Extract parameters for defining the temporary cache files

Oracle GoldenGate Services 21.5.0.0.2 for Oracle (gg01)

Administration Service Distribution Service Performance Metrics Service Receiver Service

Overview > Add Extract

Add Extract

Extract Type Extract Options **3** Parameter File

Parameter File

```
EXTRACT EXT_1
USERIDALIAS Source_CDB DOMAIN GoldenGate
EXTRAIL aa

CACHEMGR CACHEDIRECTORY /mnt/dbfs/goldengate/deployments/gg01/temp_cache
```

Back Create Create and Run

Distribution Path Configuration

When using Oracle GoldenGate distribution paths with the NGINX Reverse Proxy, there are additional steps that must be performed to ensure that the path server certificates are configured.

Follow the instructions provided in the following video to correctly configure the certificates:
https://apexapps.oracle.com/pls/apex/f?p=44785:112:0::::P112_CONTENT_ID:31380

Configuration highlights presented in this video:

1. Create a client certificate for the source deployment and add the client certificate to the source deployment Service Manager. (This is not required when using Oracle GoldenGate 21c or later releases.)

2. Download the target deployment server's root certificate and add the CA certificate to the source deployment Service Manager.
3. Create a user in the target deployment for the distribution path to connect to.
4. Create a credential in the source deployment connecting to the target deployment with the user created in the previous step.

For example, a domain of GGNORTH_to_GGSOUTH and an alias of PathReceiver.

After configuring the client and server certificates, the following configuration options need to be set. Refer to the figures below to see where these options are set in the UI.

1. Change the **Generated Source URI** specifying localhost for the server name.

This allows the distribution path to be started on any of the Oracle RAC nodes.

2. Set the **Target Authentication Method** to UserID Alias and the **Target** transfer protocol to wss (secure web socket).

Set the **Target Host** to the target host name/VIP that will be used for connecting to the target system along with the **Port Number** that NGINX was configured with (default is 443).

The target host name/VIP should match the common name in the CA signed certificate used by NGINX.

3. Set the **Domain** to the credential domain created above in step 4 and presented in the video, for example GGNORTH_to_GGSOUTH.

The **Alias** is set to the credential alias, also created in step 4 in the video.

4. Set the distribution path to automatically restart when the Distribution Server starts.

This is required so that manual intervention is not required after an Oracle RAC node relocation of the Distribution Server. It is recommended that you set the number of **Retries** to 10. Set the **Delay**, which is the amount of time in minutes to pause between restart attempts, to 1.

Figure 27-2 Distribution Path Creation steps 1-3

The screenshot shows the Oracle GoldenGate Services 21.5.0.0.2 (gg01) web interface. The 'Distribution Service' tab is selected. The 'Add Path' section is visible, with the following configuration details:

- Path Name:** gg01_to_gg02
- Description:** (empty)
- Reverse proxy enabled?** (toggle off)
- Source:** EXT1
- aa** (input field)
- 1. Generated Source URI:** trail://localhost/services/gg01/distsrvr/v2/sources?trail=aa
- 2. Target Authentication Method:** UserID Alias
- Target:** wss
- TARGET_HOST_VIP** (input field)
- 443** (input field)
- Trail Subdirectory** (input field)
- aa** (input field)
- 3. GG01_to_GG02** (input field)
- PathReceiver** (input field)
- Generated Target URI:** wss://TARGET_HOST_VIP:443/services/v2/targets?trail=aa

Figure 27-3 Distribution Path Creation step 4

Auto Restart Options

4.

Replicat Configuration

1. The checkpoint table is a required component for GoldenGate Replicat processes. Make sure that a checkpoint table has been created in the database GoldenGate administrator (GGADMIN) schema.

The checkpoint table can be created using the Oracle GoldenGate Administration Server GUI, clicking on the '+' button and entering the checkpoint table name in the format of schema.tablename. This is shown in the image below

Figure 27-4 Creating the checkpoint table for Replicat processes

ORACLE® Oracle GoldenGate Services 21.5.0.0.2 for Oracle (gg02)

Administration Service Distribution Service Performance Metrics Service Receiver Service

Database Key Management Parameter Files Tasks

Credentials + ↻ GoldenGateDB : target_gg02

Search in Credentials Table

Domain	Alias	User ID	Action
GoldenGateDB	target_gg02	ggadmin@gg02_target	  

Checkpoint +

Search in Checkpoint Table

Checkpoint Table	Action
No data to display.	

1. * Checkpoint Table: ggadmin.gg_ckpt

Cancel Submit

See [About Checkpoint Table](#) for more information about creating a checkpoint table.

2. When creating a Replicat using the Oracle GoldenGate Administration Server GUI interface, set the **Trail SubDirectory** parameter to the location where the distribution path or local Extract are creating the trail files.
3. If a checkpoint table was created previously, select the table name from the **Checkpoint Table** pulldown list.

Figure 27-5 Replicat creation with Trail SubDirectory and Checkpoint Table

Oracle GoldenGate Services 21.5.0.0.2 for Oracle (gg02)

Administration Service Distribution Service Performance Metrics Service Receiver Service

Overview > Add Replicat

Add Replicat

Replicat Type Replicat Options Parameter File

Basic Information

Process Name: REP_1

Description:

Intent: Unidirectional

Create new credential

Credential Domain: GoldenGateDB

Credential Alias: target_gg02

Source: Trail

Trail Name: aa

Trail Subdirectory:

Begin: Position in Log

Transaction Log Sequence Number: 0

Transaction Log RBA Offset: 0

Checkpoint Table: "GGADMIN"."GG_CKPT"

Task 11: Configure Autostart of Extract and Replicat Processes

Configure the Extract and Replicat processes to automatically start when the Oracle GoldenGate Administration Server is started, and then to restart if any Extract or Replicat processes abend. With GoldenGate Microservices auto start and restart is managed by Profiles.

Using the Oracle GoldenGate Administration Server GUI, create a new profile which can be assigned to each of the Oracle GoldenGate processes.

Profile Configuration Option	Recommended Setting
Default Profile	Enabled
Auto Start	Enabled
Startup Delay	1 minute
Auto Restart	Enabled
Max Retries	5
Retry Delay	30 seconds
Retries Window	30 minutes
Restart on Failure only	Enabled

Profile Configuration Option	Recommended Setting
Disable Task After Retries Exhausted	Enabled

After the profile has been created, and set as the default profile, all new GoldenGate processes created are assigned this profile. For all existing processes, the profile must be assigned to each process.

In the Overview pane, on the Process Information tab, select the **Profile Name** under **Managed Options**.

Note

When using Oracle GoldenGate Microservices with XAG, it is strongly recommended not to enable the 'Critical to deployment health' flag for any Extract or Replicat processes. Doing so can cause an entire GoldenGate deployment outage from a single Extract or Replicat failure, and also prevents XAG from being able to restart GoldenGate. Refer to [Troubleshooting Oracle GoldenGate on Oracle RAC](#) for an example of troubleshooting an outage caused by setting a Replicat to critical.

On-Premises MAA Platinum: Oracle GoldenGate Microservices Architecture Integrated with Active Data Guard

The combination and integration of Oracle GoldenGate Microservices and Oracle Data Guard enables you to achieve an MAA Platinum service-level configuration that achieves zero or near zero downtime for all planned and unplanned outages.

Follow these configuration best practices to enable Oracle GoldenGate Microservices replication using a database that is protected by a Data Guard standby, to transparently and seamlessly work following an Oracle Data Guard role transition, no matter which Data Guard protection mode is configured (Maximum Performance, Maximum Availability, or Maximum Protection).

Topics:

- [Prerequisites](#)
- [Task 1: Configure the Standby Database for Oracle GoldenGate](#)
- [Task 2: Modify the Primary Database Service](#)
- [Task 3: Create the Standby Database Service](#)
- [Task 4: Configure DBFS on the Standby Cluster Nodes](#)
- [Task 5: Install Oracle GoldenGate Software](#)
- [Task 6: Create Oracle GoldenGate Deployment Directories](#)
- [Task 7: Configure the Standby NGINX Reverse Proxy](#)
- [Task 8: Configure Oracle Clusterware](#)
- [Task 9: Create Oracle Net TNS Aliases for Oracle GoldenGate Database Connections](#)
- [Task 10: Configure Oracle GoldenGate Processes](#)
- [Example Distribution Path Target Change Script](#)

Prerequisites

Be sure to complete the following prerequisites before performing any tasks for on-premises MAA Platinum architecture configuration.

- As a prerequisite for MAA Platinum on-premises, have Oracle GoldenGate configured as detailed in [On-Premises: Oracle GoldenGate Microservices Architecture with Oracle Real Application Clusters Configuration Best Practices](#).
- The Database File System (DBFS) is required for critical Oracle GoldenGate files when integrating with Data Guard.
- The Oracle Data Guard standby database should also be configured and operational before continuing.

The following are software requirements that the MAA Platinum configuration is based on:

- Oracle Grid Infrastructure 19c or later

Oracle Grid Infrastructure provides the necessary components needed to manage high availability for any business-critical applications. Using Oracle Clusterware (a component of Oracle Grid Infrastructure) network, database, and Oracle GoldenGate resources can be managed to provide availability in the event of a failure.

- Oracle Grid Infrastructure Agent version 10.2 or later

The Oracle Grid Infrastructure Agent leverages the Oracle Grid Infrastructure components to provide integration between Oracle GoldenGate and its dependent resources, such as the database, network, and file system. The agent also integrates Oracle GoldenGate with Oracle Data Guard so that Oracle GoldenGate is restarted on the new primary database following a role transition.

- Oracle Database 19c or later

See [My Oracle Support Document 2193391.1](#) for a full list of recommended Oracle Database patches when using Oracle GoldenGate.

- Oracle GoldenGate Microservices version 21c or later

Oracle GoldenGate 21c introduces unified build support so a single software installation supports capturing and applying replicated data to multiple major Oracle Database versions (11g Release 2 to 21c). This is possible because an Oracle GoldenGate installation includes the required Oracle Database client libraries without requiring a separate database ORACLE_HOME installation.

- Oracle DBFS to protect and replicate critical Oracle GoldenGate files

The Oracle Database File System (DBFS) is the only MAA-validated and recommended file system for an Oracle Data Guard and Oracle GoldenGate configuration, because it allows the storage of the required Oracle GoldenGate files, such as the checkpoint and trail files, to be located inside the same database that is protected with Oracle Data Guard, ensuring consistency between the Oracle GoldenGate files and the database in a seamless fashion.

When the prerequisites are met, follow the configuration best practices in the Tasks that follow. These tasks should be performed to ensure the seamless integration of Oracle GoldenGate Microservices with Oracle Data Guard, which in turn ensures that GoldenGate continues running after any Data Guard role transition.

Task 1: Configure the Standby Database for Oracle GoldenGate

The standby database initialization parameters should match those of the primary database.

See [Task 1: Configure the Oracle Database for Oracle GoldenGate](#) for details. This includes the following parameters:

- `ENABLE_GOLDENGATE_REPLICATION=TRUE`
- For Oracle GoldenGate source databases, enable `FORCE LOGGING` mode and enable minimal supplemental logging.
- If a GoldenGate source database, or running integrated Replicat (parallel or non-parallel), configure the `STREAMS_POOL_SIZE`.

Task 2: Modify the Primary Database Service

On the primary database server, modify the existing database service that was created as part of the original Oracle GoldenGate on Oracle RAC configuration.

Set the service role to PRIMARY, so that the service is only be started when the database becomes the Data Guard primary database role after a role transition.

As the oracle user, modify the service using the following command:

```
$ srvctl modify service -db dbName -service service_name  
-role PRIMARY
```

If your database is part of a multitenant environment, remember to modify both the multitenant container database (CDB) and pluggable database (PDB) services.

Task 3: Create the Standby Database Service

On the standby cluster, a database service is required for the standby database so that the Oracle Grid Infrastructure Agent automatically starts the Oracle GoldenGate deployment when the database is opened with the primary role.

When a source database is in a multitenant environment, a separate service is required for the root container database (CDB) and the pluggable database (PDB) that contains the schema being replicated. For a multitenant environment target database, a single service is required for the PDB.

Create the service using the following command, as the oracle user, the same way the service was created on the primary cluster.

```
$ srvctl add service -db dbName -service service_name  
-preferred instance_1 -available instance_2, instance_3 etc.  
-pdb pdbName -role PRIMARY
```

It is recommended that you use the same service name as was specified on the primary cluster. The service must be created as a singleton service, using the `-preferred` option, because the application Virtual IP address (VIP), DBFS, and Oracle GoldenGate run on the cluster node where the service is running.

If the database is not in a multitenant environment, or the database is a target database for Oracle GoldenGate, omit the `-pdb` parameter.

Task 4: Configure DBFS on the Standby Cluster Nodes

The Database File System (DBFS) is the only recommended solution when configuring Oracle GoldenGate with Oracle Data Guard.

The DBFS user, tablespace, and file system in the database was previously created in the primary database, as detailed in [Task 4: Set Up a File System on Oracle RAC](#).

The remaining configuration steps are required on all nodes of the standby cluster where Oracle GoldenGate may run.

1. Install the required FUSE libraries, if they are not already installed, by following the instructions in [My Oracle Support Document 869822.1](#).
2. Create the `tnsnames.ora` Oracle Net connection alias using the IPC protocol, similar to the one created on the primary cluster.

```
dbfs =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = IPC)(KEY=LISTENER))
    (CONNECT_DATA =
      (SERVICE_NAME = NAME)
    )
  )
```

3. Create the **same** mount point for DBFS that is used on the primary cluster.

It is important that the mount point is identical, because the physical location of the Oracle GoldenGate deployment is included in the deployment configuration files.

For example:

```
# mkdir /mnt/dbfs
```

4. Copy the `mount-dbfs.conf` and `mount-dbfs.sh` files from the primary cluster to the standby cluster nodes.

It is recommended that you place them in the same directory as the primary cluster.

5. Register the DBFS resource with Oracle Clusterware, using the following example command.

If you are using Oracle Multitenant, make sure to use the service name for the same PDB that contains the DBFS repository as was created in the primary database.

```
DBNAME=dbName
DEPNAME=ora.$DBNAME.oggserv_pdb.svc

crsctl add resource $RESNAME \
  -type cluster_resource \
  -attr "ACTION_SCRIPT=$ACTION_SCRIPT, \
        CHECK_INTERVAL=30,RESTART_ATTEMPTS=10, \
        START_DEPENDENCIES='hard($DEPNAME)pullup($DEPNAME)', \
        STOP_DEPENDENCIES='hard($DEPNAME)', \
        SCRIPT_TIMEOUT=300"
```

Task 5: Install Oracle GoldenGate Software

Install the Oracle GoldenGate software locally on all nodes in the standby cluster that will be part of the Oracle GoldenGate configuration.

Make sure the installation directory is **identical** on all nodes to match the primary cluster installation directory.

Download the Oracle GoldenGate 21c software, or later version, at this location:

<http://www.oracle.com/technetwork/middleware/goldengate/downloads/index.html>

Task 6: Create Oracle GoldenGate Deployment Directories

The Oracle GoldenGate Service Manager and deployment are already created on the primary cluster, as required by the prerequisites, but certain directories and symbolic links need to be configured on the standby cluster nodes.

These directories and symbolic links were created on the primary cluster, in the tasks you performed as part of [On-Premises: Oracle GoldenGate Microservices Architecture with Oracle Real Application Clusters Configuration Best Practices](#).

Now you create the following directories and symbolic links on the all Oracle RAC nodes on the standby cluster as follows.

1. If there are multiple GoldenGate Service Managers configured on the primary cluster, each with their own deployment, and individually registered with XAG, they must belong to separate OGG_HOME software installation directories.

The same directories and symbolic links for the OGG_HOME directories that were configured on primary cluster, must match on the standby cluster.

2. If the GoldenGate deployment was created with the Performance Metric Server enabled, the metric datastore home directory must be created on the standby Oracle RAC nodes.

For example, determine the datastore directory on the primary cluster nodes:

```
$ grep RepoDatastorePath <deployment directory>/var/log/pmsrvr.log|uniq

"RepoDatastorePath": "",
"RepoDatastorePath": "/u01/oracle/goldengate/datastores/ggnorth",
```

Then create the directory on all standby cluster nodes:

```
$ mkdir -p /u01/oracle/goldengate/datastores/ggnorth
```

3. If the database release is earlier than Oracle Database 21c (21.3), create the Oracle GoldenGate deployment temp directory local storage to match the symbolic link created on the primary cluster.

For example, on the primary cluster if you have:

```
$ ls -lrt DBFS_GoldenGate_deployment_home_directory/var/temp

lrwxrwxrwx 1 oracle oinstall 32 Aug 31 12:27 temp
-> /u01/oracle/goldengate/deployments/ggnorth/temp
```

Then create the same directory on the standby cluster nodes:

```
$ mkdir -p /u01/oracle/goldengate/deployments/ggnorth/temp
```

Task 7: Configure the Standby NGINX Reverse Proxy

Follow these steps to configure the standby NGINX reverse proxy.

1. Install NGINX Reverse Proxy.

If NGINX Reverse Proxy has not already been installed, follow the installation instructions at https://nginx.org/en/linux_packages.html.

As the root user, copy the Oracle GoldenGate deployment NGINX configuration files from a primary cluster node to a single standby node directory `/etc/nginx/conf.d`.

For example:

```
[root@dc2north01]# scp dc1north01:/etc/nginx/conf.d/ogg_north.conf
/etc/nginx/conf.d
```

The standby cluster will need a different CA signed certificate due to using a different VIP name/address than the primary cluster. Contact your systems administrator to follow your corporate standards to create or obtain the server certificate before proceeding. A separate certificate is required for each VIP and Service Manager pair.

2. Install server certificates for NGINX.

Install the server CA certificates and key files in the `/etc/nginx/ssl` directory, owned by root with file permissions 400 (-r-----):

```
# mkdir /etc/nginx/ssl
# chmod 400 /etc/nginx/ssl
```

For each reverse proxy configuration file copied from the primary cluster, set the correct file names for the certificate and key file using the following example:

```
ssl_certificate /etc/nginx/ssl/gg-stby-vip1.pem;
ssl_certificate_key /etc/nginx/ssl/gg-stby-vip1.key;
```

When using CA signed certificates, the certificate named with the `ssl_certificate` NGINX parameter must include the root, intermediate, and CA signed certificates in a single file. The order is very important, otherwise NGINX fails to start and displays the error message

(SSL: error:0B080074:x509 certificate routines: X509_check_private_key:key values mismatch).

The root and intermediate certificates can be downloaded from the CA signed certificate provider.

The single file can be generated using the following example command:

```
# cat CA_signed_cert.crt intermediate.crt root.crt
> gg-stby-vip1.pem
```

The `ssl_certificate_key` file is the key file generated when creating the Certificate Signing Request (CSR), which is required when requesting a CA signed certificate.

Change the `server_name` parameter in the reverse proxy configuration file copied from the primary cluster, setting to the correct VIP name. For example:

Before:

```
server_name dc1north-vip1.example.com;
```

After:

```
server_name dc2north-vip1.example.com;
```

3. Validate and restart NGINX.

Because the VIP will not be running on the standby cluster until the primary database service is running, there is a parameter that needs to be set in the `/etc/sysctl.conf` file.

- a. As the root user, make the following modifications to `/etc/sysctl.conf`.

```
# vi /etc/sysctl.conf
```

- b. Add the following parameter:

```
# allow processes to bind to the non-local address

net.ipv4.ip_nonlocal_bind = 1
```

- c. Reload the modified configuration:

```
# sysctl -p /etc/sysctl.conf
```

- d. Validate the NGINX configuration file to detect any errors in the configuration. If there are errors in the file, they will be reported by the following command.

```
# nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginxconf test is successful
```

- e. Restart NGINX with the new configuration:

```
# systemctl restart nginx
```

When the NGINX configuration is complete, copy the configuration file and certificates to matching directories on the other standby cluster nodes.

4. Create an NGINX Clusterware resource.

Oracle Clusterware needs to have control over starting the NGINX reverse proxy so that it can be started automatically before the GoldenGate deployments are started.

The NGINX resource is created with a dependency on the underlying network CRS resource, the name of which can be determined using the following command:

```
$ $GRID_HOME/bin/crsctl stat res -w "TYPE == ora.network.type"|grep NAME
NAME=ora.net1.network
```

- a. As the root user, use the following example command to create a Clusterware resource to manage NGINX.

```
# $GRID_HOME/bin/crsctl add resource nginx -type generic_application
-attr "ACL='owner:root:rw,group:oinstall:r-x,user:oracle:rw',EXECUTABLE_NAMES=nginx,START_PROGRAM='/bin/systemctl
start -f nginx',STOP_PROGRAM='/bin/systemctl
stop -f nginx',CHECK_PROGRAMS='/bin/systemctl
status nginx',START_DEPENDENCIES='hard(ora.net1.network)
pullup(ora.net1.network)',
STOP_DEPENDENCIES='hard(intermediate:ora.net1.network)',
RESTART_ATTEMPTS=0, HOSTING_MEMBERS='dc1north01,dc1north02',
CARDINALITY=2"
```

The NGINX resource created in this example runs on the named cluster nodes at the same time, specified by `HOSTING_MEMBERS`. This is recommended when multiple GoldenGate Service Manager deployments are configured, and they can independently move between cluster nodes.

- b. When the NGINX Clusterware resource is created, alter the GoldenGate XAG resources so that NGINX must be started before the GoldenGate deployments are started.

As the root user, modify the XAG resources using the following example commands.

Determine the current `--filesystems` parameter:

```
# agctl config goldengate GGNORTH | grep "File System"
```

File System resources needed: dbfsgg

Task 8: Configure Oracle Clusterware

1. Modify the primary cluster XAG GoldenGate instance.

The Oracle Grid Infrastructure Standalone Agent (XAG) GoldenGate instance on the primary cluster must be modified as the root user, to identify that it is part of an Oracle Data Guard configuration using the following example command.

```
# agctl modify goldengate instance_name --dataguard_autostart yes
```

2. On the standby cluster, follow the instructions in [Task 7: Oracle Clusterware Configuration](#) to do steps 3-5 below.
3. Install the XAG software on each standby cluster node.

It is recommended that you install the XAG software into the same directory as the primary cluster.

4. Prepare for the XAG application VIP creation.

It is assumed that the VIP and VIP name will be different from that of the primary cluster, so the VIP address will need to be allocated by your systems administrator for the standby cluster.

5. Register Oracle GoldenGate Microservices with XAG.

The parameters used to register Oracle GoldenGate Microservices with XAG are similar to those used when registering with the primary cluster.

- a. Determine the current parameters in the primary cluster using the following command:

```
$ agctl config goldengate GoldenGate_instance_name
```

```
Instance name: GoldenGate_instance_name
Application GoldenGate location is: /u01/oracle/goldengate/gg21c_MS
Goldengate MicroServices Architecture environment: yes
Goldengate Service Manager configuration directory:
/mnt/dbfs/goldengate/deployments/ggnorth_sm/etc/conf
Goldengate Service Manager var directory:
/mnt/dbfs/goldengate//deployments/ggnorth_sm/var
Service Manager Port: 9100
Goldengate Administration User: oggadmin
```

```

Autostart on DataGuard role transition to PRIMARY: yes
Configured to run on Nodes: dc1north01,dc1north02
ORACLE_HOME location is: /u01/oracle/goldengate/gg21c_MS/lib/
instantclient
Database Services needed:
ora.ggdg.oggserv_cdb.svc,ora.ggdg.oggserv_pdb.svc
File System resources needed: dbfsgg,nginx
VIP name: gg_vip_prmy

```

In addition, the XAG parameter `--filesystem_verify no` must be specified to prevent XAG from checking the existence of the DBFS deployment directory when registering the GoldenGate instance. Without setting this parameter, the XAG registration will fail, because DBFS is not mounted on the standby cluster.

Note

It is recommended that you use the same GoldenGate instance name when registering GoldenGate with XAG as was used in the primary cluster.

- b. Register GoldenGate with XAG on the standby cluster, as the root user:

```

# agctl add goldengate GoldenGate_instance_name \
--gg_home /u01/oracle/goldengate/gg21c_MS \
--service_manager \
--config_home /mnt/dbfs/goldengate/deployments/ggnorth_sm/etc/conf \
--var_home /mnt/dbfs/goldengate/deployments/ggnorth_sm/var \
--port 9100 \
--oracle_home /u01/goldengate/gg21c_MS/lib/instantclient \
--adminuser oggadmin \
--user oracle \
--group oinstall \
--vip_name gg_vip_stby \
--filesystems dbfsgg,nginx \
--db_services ora.ggdgs.oggserv_cdb.svc,ora.ggdgs.oggserv_pdb.svc \
--use_local_services \
--nodes dc2north01,dc2north02 \
--filesystem_verify no \
--dataguard_autostart yes

```

For more information about the Oracle Grid Infrastructure Bundled Agent, see <http://www.oracle.com/technetwork/database/database-technologies/clusterware/downloads/xag-agents-downloads-3636484.html>

Task 9: Create Oracle Net TNS Aliases for Oracle GoldenGate Database Connections

The same TNS aliases created on the primary cluster for the primary database using the IPC protocol must be created with the **same** alias names on each node of the standby cluster, using the IPC communication protocol as specified in [Task 9: Create Oracle Net TNS Alias for Oracle GoldenGate Database Connections](#).

The location of `tnsnames.ora` used by the Oracle GoldenGate deployment **must** be the **same** on the standby cluster nodes as it is on the primary cluster.

Use the following query REST API call to query the `TNS_ADMIN` location on the primary cluster.

```
$ curl -s -u OGG_admin_username
https://vip_name/services/v2/deployments/deployment_name
-XGET|python -m json.tool|grep TNS_ADMIN -A1
```

You will be prompted to enter the Oracle GoldenGate Service Manager administrator user password.

For example:

```
$ curl -s -u oggadmin https://dc1north01-vip1/services/v2/deployments/ggnorth
-XGET|python -m json.tool|grep TNS_ADMIN -A1
```

```
"name": "TNS_ADMIN",
"value": "/u01/goldengate/network/admin"
```

Make sure the `tnsnames.ora` is located in this **same** directory on **all** standby cluster nodes.

Example TNS alias for the GoldenGate database:

```
ggnorth_pdb =
  (DESCRIPTION =
    (SDU = 2097152)
    (ADDRESS = (PROTOCOL = IPC)(KEY=LISTENER))
    (CONNECT_DATA =
      (SERVICE_NAME = oggserv_pdb.example.com)
    )
  )
```

Task 10: Configure Oracle GoldenGate Processes

In addition to the guidance provided in [Task 10: Configure Oracle GoldenGate Processes](#), follow the recommendations provided below for Extract, Distribution Paths, and Replicats.

Extract Configuration on the Primary Cluster

For GoldenGate Extract processes using Data Guard configurations that are using redo transport Maximum Performance or Maximum Availability modes, the following parameter must be added to the Extract process parameter file **on the primary cluster** to avoid losing transactions and resulting in logical data inconsistencies:

```
TRANLOGOPTIONS HANDLEDLFAILOVER
```

This parameter prevents Extract from extracting transaction data from redo that has not yet been applied to the Data Guard standby database. This is crucial to preventing Oracle GoldenGate from replicating data to a target database that does not exist in the source standby database.

If this parameter is not specified, after a data loss failover of the source database it is possible to have data in the target database that is not present in the source database, leading to logical data inconsistencies.

By default, after 60 seconds, a warning message will be written to the Extract report file when the Extract is stalled due to not being able to query the standby database applied SCN information. For example:

```
WARNING OGG-02721 Extract has been waiting for the standby database for 60 seconds.
```

The amount of time before the warning message is written to Extract report file can be adjusted using the Extract parameter `TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_WARNING`.

If the Extract is still not able to query the standby database applied SCN information after 30 minutes (default), the Extract process will abend, logging the following message in the Extract report file:

```
ERROR OGG-02722 Extract abended waiting for 1,800 seconds for the standby database to be accessible or caught up with the primary database.
```

If the standby database becomes available before the 30 default timeout expires, Extract continues mining data from the source database and reports the following message to the report file:

```
INFO OGG-02723 Extract resumed from stalled state and started processing LCRs.
```

The timeout value of 30 minutes can be adjusted using the Extract parameter `TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_ABEND value`, where *value* is the number of seconds the standby is unavailable before abending.

If the standby database will be unavailable for a prolonged duration, such as during a planned maintenance outage, and you wish Extract to continue extracting data from the primary database, remove the `TRANLOGOPTIONS HANDLEDLFAILOVER` parameter from the Extract parameter file and restart Extract. Remember to set the parameter after the standby becomes available.

Note

If extracting from a primary database continues while the standby is unavailable, a data loss failover could result after the standby becomes available, and not all the primary redo was applied before a failover. The GoldenGate target database will contain data that does not exist in the source database.

See [TRANLOGOPTIONS](#) for more information about the `TRANLOGOPTIONS HANDLEDLFAILOVER` parameters.

If the Extract process has been assigned an auto restart profile, as documented in [Task 11: Configure Autostart of Extract and Replicat Processes](#), after a Data Guard role transition, the Extract process will automatically restart. Extract will continue to mine redo data from the new primary database, ignoring the current state of the new standby database, until a default 5 minute timeout period expires. After this time, if the standby is not available Extract will abend with the following errors:

```
INFO OGG-25053 Timeout waiting for 300 seconds for standby database reinstatement. Now enforcing HANDLEDLFAILOVER.
```


ERROR OGG-06219 Unable to extract data from the Logmining server OGG\$CAP_EXT1.

ERROR OGG-02078 Extract encountered a fatal error in a processing thread and is abending.

Extract will continue to automatically restart, based on the Oracle GoldenGate Microservices auto restart profile, and failing due to reaching the HANDLEDLFAILOVER timeout, until the number retries is reached or the new standby database becomes available.

During the timeout period following a database role transition, the HANDLEDLFAILOVER parameter is automatically suspended, so data will be replicated to the Oracle GoldenGate replica database without consideration of the source standby database not being kept up to date. The timeout period for the standby database to start up before Extract abends can be adjusted using the Extract parameter TRANLOGOPTIONS DLFAILOVER_TIMEOUT.

It is recommended that you leave DLFAILOVER_TIMEOUT at the default of 5 minutes, to allow the old primary to convert to a standby. If the new standby database will be unavailable for an extended period of time or completely gone, then in order for Extract to start and remain running, you must remove the HANDLEDLFAILOVER parameter from the Extract parameter file. After removing the parameter, Extract no longer waits until redo has been applied to the standby database before extracting the data.

During the time it takes for the standby database to come back online and apply all the redo from the primary

database, there will be data divergence between it and the Oracle GoldenGate replica database. This will be resolved once the standby database is up to date. At which point, add the HANDLEDLFAILOVER parameter back into the integrated Extract process parameter file, and then stop and restart the Extract.

When Oracle Data Guard is configured with fast-start failover, such that the broker can automatically fail over to a standby database in the event of loss of the primary database, you must specify an additional integrated Extract parameter shown below.

TRANLOGOPTIONS FAILOVERTARGETDESTID *n*

This parameter identifies which standby database the Oracle GoldenGate Extract process must remain behind, with regards to not extracting redo data that has not yet been applied to the standby database.

To determine the correct value for FAILOVERTARGETDESTID, use the LOG_ARCHIVE_DEST_*N* parameter from the GoldenGate source database which is used for sending redo to the source standby database. For example, if LOG_ARCHIVE_DEST_2 points to the standby database, then use a value of 2.

For example:

SQL> show parameters log_archive_dest

NAME	TYPE	VALUE
log_archive_dest_1	string	location=USE_DB_RECOVERY_FILE_DEST, valid_for=(ALL_LOGFILES, ALL_ROLES)
log_archive_dest_2	string	service="ggnorths", SYNC AFFIRM delay=0 optional compression=disable max_failure=0 reopen=300

```
db_unique_name="GGNORTHS" net_timeout=30,  
valid_for=(online_logfile,all_roles)
```

In this example, the Extract parameter would be set to the following:

```
TRANLOGOPTIONS FAILOVERTARGETDESTID 2
```

To add the parameters to the Extract parameter file, use the Oracle GoldenGate Administration Server to select display the Extract details

1. On the **Administration Service** tab, select the **Actions** menu for the Extract and choose **Details**.
2. In the Extract details view select the **Parameters** tab, and then select the pencil icon to edit the current parameter file,
3. Add the TRANLOGOPTIONS parameters and select **Apply** to save the changes.

For the new parameters to take effect, the Extract process needs to be stopped and restarted, which can be done using the Administration Server.

More information about the Extract TRANLOGOPTIONS parameters mentioned above, can be found in [TRANLOGOPTIONS](#).

Distribution Path Configuration on the Primary and Standby Cluster

When the target database of an Oracle GoldenGate environment, where the Receiver Server runs, is protected with Oracle Data Guard, there is an important consideration that must be given to any Distribution Paths that are sending trail files to the Receiver Server. When the Receiver Server moves to a different cluster after an Oracle Data Guard role transition, any distribution paths must be altered to reflect the new target cluster address.

You can automatically change the distribution paths using a database role transition trigger in the target database on the Receiver Server cluster.

If the primary and standby cluster VIPs use different root CA certificates, the standby certificate will need to be added to the source deployment Service Manager, as detailed in [On-Premises: Oracle GoldenGate Microservices Architecture with Oracle Real Application Clusters Configuration Best Practices](#).

Follow the instructions below to create a database role transition trigger to modify the distribution path target address when the receiver server moves between the primary and standby cluster, during target database Data Guard role transitions.

1. Create a shell script to modify the distribution paths.

[Example Distribution Path Target Change Script](#) contains an example shell script that can be used to modify a distribution path target address. Refer to the example script comments for setting appropriate variable values.

The script should be placed in the same local directory on **all** Oracle RAC nodes of the primary **and** standby database clusters. Set the script file permissions to 6751.

For example:

```
$ chmod 6751 /u01/oracle/goldengate/scripts/change_path_target.sh
```

The example shell script uses REST API calls to access the GoldenGate distribution path. In order to make the REST API calls secure, it is recommended that you include the

GoldenGate deployment administrator user name and password in a configuration file (access.cfg), as shown here.

```
$ cat /u01/oracle/goldengate/scripts/access.cfg

user = "oggadmin:<password>"
```

The access.cfg file is also referenced in the database role transition trigger below.

2. Create a DBMS_SCHEDULER job.

Creating a DBMS_SCHEDULER job is required to run an operating system shell script from within PL/SQL. Create the scheduler job as a SYSDBA user in the root container database (CDB).

For example:

```
SQL> exec dbms_scheduler.drop_job('gg_change_path_target');
SQL> exec dbms_scheduler.create_job(job_name=>'gg_change_path_target',
    job_type=>'EXECUTABLE', number_of_arguments => 6,
    job_action=>'/u01/oracle/goldengate/scripts/change_path_target.sh',
    enabled=>FALSE);
```

To run an external job, you must set the run_user and run_group parameters in the \$ORACLE_HOME/rdbms/admin/externaljob.ora file to the Oracle database operating system user and group.

For example:

```
run_user = oracle
run_group = oinstall
```

The externaljob.ora must be configured on **all** Oracle RAC nodes of the primary **and** standby database clusters.

3. Create the database role transition trigger.

Create a role transition trigger on the GoldenGate target database that will fire when a standby database becomes a primary database, changing the distribution path target address, using the following example.

```
CREATE OR REPLACE TRIGGER gg_change_path
AFTER db_role_change ON DATABASE
declare
    role varchar2(30);
    hostname varchar2(64);
begin
    select database_role into role from v$database;
    select host_name into hostname from v$instance;
```

```
DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',1,'source_primary_cluster_VIP');
```

```
DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',2,'source_standby_cluster_VIP');
```

```

DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',4,'dist_path_
name');

DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',5,'deployment
_name');
  DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',6, '<dir/
access.cfg>');

  if role = 'PRIMARY' and hostname like 'primary_target_cluster_name%'
  then

DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',3,'primary_ta
rget_cluster_VIP:443');
  elsif role = 'PRIMARY'
  then

DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE('gg_change_path_target',3,'standby_ta
rget_cluster_VIP:443');
  end if;
  DBMS_SCHEDULER.RUN_JOB(job_name=>'gg_change_path_target');
end;
/

```

After creating the database trigger, switch the log file on the primary database to ensure the code is propagated to the standby database using the following command:

```
SQL> alter system switch all logfile;
```

Replicat Configuration on the Primary Cluster

As documented in [On-Premises: Oracle GoldenGate Microservices Architecture with Oracle Real Application Clusters Configuration Best Practices](#), a checkpoint table in the target database is required for all Oracle GoldenGate Replicat processes. There are no other configuration requirements for Replicat when configured with Oracle Data Guard.

Example Distribution Path Target Change Script

The following example script can be used to change a source GoldenGate deployment distribution path target address to reflect the new location of the receiver server after a Data Guard role transition. This example assumes the source GoldenGate deployment is configured in an MAA architecture with Data Guard, such that the distribution server can relocate between a primary and standby cluster.

```

#!/bin/bash

# change_path_target.sh - changes the target host of a GG Distribution Path
when the target
#                               moves between primary/standby clusters.
# Example usage:
# ./change_path_target.sh <primary source VIP>:443 <standby source VIP>:443
<path target VIP> <path name> <deployment name> <credentials file>

SOURCE1=$1      # PRIMARY Distribution Server VIP

```

```

SOURCE2=$2      # STANDBY Distribution Server VIP
TARGET=$3       # Distribution path target VIP
DPATH=$4        # Distribution path name
DEP=$5          # Deployment name
ACCESS=$6       # access.cfg file containing the deployment credentials.
Example contents:
                # user = "oggadmin:<password>"

CONNECT=0

#echo "#{i} - `date`:"
LOGFILE=/tmp/ogg_dpatch_change.txt

result=$(curl -si -K $ACCESS https://$SOURCE1/$DEP/distsrvr/services/v2/
sources/$DPATH -X GET| grep HTTP | awk '{print $2}')

# Will return NULL of nginx not running, 502 if cannot contact server, 200 if
contact to server good, and others (404) for other bad reasons:

if [[ -z $result || $result -ne 200 ]]; then # Managed to access the Distr
Server
    echo "`date` - Couldn't contact Distribution Server at $SOURCE1
Deployment $DEP *****" >> $LOGFILE
else # Try the other source host:
    echo "`date` - Got status of Distribution Server at $SOURCE1
Deployment $DEP *****" >> $LOGFILE
    SOURCE=$SOURCE1
    CONNECT=1
fi

if [ $CONNECT -eq 1 ]; then
# For secure NGINX patch destination (wss)
    PAYLOAD='{ "target": { "uri": "wss://'{TARGET}'/services/ggnorth/v2/targets?
trail=bb" } }'
    curl -s -K $ACCESS https://$SOURCE/$DEP/distsrvr/services/v2/sources/$DPATH
-X PATCH --data '{"status": "stopped"}'

# Set new target for path:
    curl -s -K $ACCESS https://$SOURCE/$DEP/distsrvr/services/v2/sources/$DPATH
-X PATCH --data "$PAYLOAD"
    echo "`date` - Set path $DPATH on $SOURCE deployment $DEP:" >> $LOGFILE

    curl -s -K $ACCESS https://$SOURCE/$DEP/distsrvr/services/v2/sources/$DPATH
-X GET | python -m json.tool | grep uri >> $LOGFILE
    curl -s -K $ACCESS https://$SOURCE/$DEP/distsrvr/services/v2/sources/$DPATH
-X PATCH --data '{"status": "running"}'

exit 0
else
    echo "`date` - ERROR: COULDN'T CHANGE DISTRIBUTION PATH ($DPATH) in
Deployment $DEP at $SOURCE! *****" >> $LOGFILE
fi

# If here, means we couldn't connect to either Distribution Servers
exit 1

```

Managing Outages for Oracle GoldenGate Hub

There are a number of considerations that must be taken into account when the hub undergoes a planned or unplanned outage of either the primary or standby file system clusters.

Topics:

- [Managing Planned Outages](#)
- [Managing Unplanned Outages](#)

Managing Planned Outages

When there is a requirement to perform planned maintenance on the GGHUB, some of the CRS resources should be stopped and disabled to prevent them from restarting, or from causing undesirable results when incorrectly instigating a file system failover, or stopping GoldenGate from running.

Use the following recommendations in the event of a planned outage of the primary or standby hub clusters.

For all planned maintenance events:

- Operating system software or hardware updates and patches
- Oracle Grid Infrastructure interim or diagnostic patches
- Oracle Grid Infrastructure quarterly updates under the Critical Patch Update (CPU) program, or Oracle Grid Infrastructure release upgrades
- GGHUB software life cycle, including:
 - Oracle GoldenGate
 - Oracle Grid Infrastructure Agent
 - NGINX

High Availability Solutions with Target Outage Time:

Seconds to minutes where GoldenGate replication is temporarily suspended

Step 1: Software update of idle GGHUB node

Step 2: GGHUB Node Relocate

Step 3: Software update of the remaining inactive GGHUB node

GGHUB Node Relocate

As the grid OS user on the primary GGHUB system, relocate the Oracle GoldenGate Instance:

```
[grid@gghub_prim1 ~]$ agctl status goldengate
```

```
Goldengate instance 'gghub' is running on gghub_prim1
```

```
[grid@gghub_prim1 ~]$ time agctl relocate goldengate gghub
```

```
real    0m43.984s
user    0m0.156s
sys     0m0.049s
```

As the grid OS user on the primary GGHUB system, check the status of the Oracle GoldenGate Instance:

```
[grid@gghub_prim1 ~]$ agctl status goldengate
```

```
Goldengate instance 'gghub' is running on gghub_prim2
```

GGHub Role Reversal for DR events or to move GGHUB in the same region as the target database

GGHub role reversal performs an ACFS role reversal so that the standby becomes the new primary. With both the primary and standby file systems online, the `acfsutil repl failover` command ensures that all outstanding primary file system changes are transferred and applied to the standby before the role reversal completes.

When to use GGHUB role reversal:

- To move the GGHUB deployment close to the target database for replication performance
- To support site outage
- To support site maintenance

As the grid OS user on the current standby GGHUB node, create the script to perform the ACFS role reversal:

```
[grid@gghub_stby1]$ export ACFS_MOUNT_POINT=/mnt/acfs_gg1
[grid@gghub_stby1]$ export GG_DEPLOYMENT_NAME=gghub
[grid@gghub_stby1]$ ssh ` /sbin/acfsutil repl info -c -v $ACFS_MOUNT_POINT |
grep
'Primary hostname' | awk '{print $3}' | cut -d "@" -f2 `
"agctl stop goldengate $GG_DEPLOYMENT_NAME"
[grid@gghub_stby1]$ /sbin/acfsutil repl failover $ACFS_MOUNT_POINT
[grid@gghub_stby1]$ agctl start goldengate $GG_DEPLOYMENT_NAME
[grid@gghub_stby1]$ agctl status goldengate $GG_DEPLOYMENT_NAME
Goldengate instance 'gghub' is running on gghub_stby1
```

Alternatively, as the grid OS user on any GGHUB node, run the script `acfs_role_reversal.sh` to perform the ACFS role reversal:

```
[grid@gghub_stby1]$ sh /u01/oracle/scripts/acfs_role_reversal.sh
/mnt/acfs_gg1 gghub
```

```
#####
##
ACFS Primary Site: gghub_prim_vip1.frankfurt.goldengate.com
ACFS Standby Site: gghub_stby_vip1.frankfurt.goldengate.com
#####
##
```

```

Thu Nov 30 17:28:37 UTC 2023 - Begin Stop GoldenGate gghub
Thu Nov 30 17:28:38 UTC 2023 - End Stop GoldenGate gghub
#####
##
Thu Nov 30 17:28:38 UTC 2023 - Begin ACFS replication sync /mnt/acfs_gg1
Thu Nov 30 17:28:59 UTC 2023 - End ACFS replication sync /mnt/acfs_gg1
#####
##
Site: Primary
Primary status: Running
Status: Send Completed
Lag Time: 00:00:00
Retries made: 0
Last send started at: Thu Nov 30 17:28:45 2023
Last send completed at: Thu Nov 30 17:28:55 2023
#####
##
Site: Standby
Last sync time with primary: Thu Nov 30 17:28:45 2023
Status: Receive Completed
Last receive started at: Thu Nov 30 17:28:46 2023
Last receive completed at: Thu Nov 30 17:28:52 2023
#####
##
Thu Nov 30 17:29:00 UTC 2023 - Begin Role Reversal
Thu Nov 30 17:30:02 UTC 2023 - End Role Reversal
#####
##
ACFS Primary Site: gghub_stby_vip1.frankfurt.goldengate.com
ACFS Standby Site: gghub_prim_vip1.frankfurt.goldengate.com
#####
##
Site: Primary
Primary status: Running
Status: Send Completed
Lag Time: 00:00:00
Retries made: 0
Last send started at: Thu Nov 30 17:29:45 2023
Last send completed at: Thu Nov 30 17:29:56 2023
#####
##
Site: Standby
Last sync time with primary: Thu Nov 30 17:29:45 2023
Status: Receive Completed
Last receive started at: Thu Nov 30 17:29:50 2023
Last receive completed at: Thu Nov 30 17:29:50 2023
#####
##
Thu Nov 30 17:30:03 UTC 2023 - Begin Start GoldenGate gghub
Thu Nov 30 17:30:10 UTC 2023 - End Start GoldenGate gghub
#####
##

```

Managing Unplanned Outages

Expected Impact with Unplanned Outages

When an unplanned outage occurs on either the primary or standby GGHUB clusters, there are some instructions to ensure the continuous operation of GoldenGate. Use the following GGHUB failure use cases to guide you in the event of an unplanned outage of the primary and standby GGHUB systems.

Use case #1 – Standby Hub Failure or Primary GGHUB Cannot Communicate with the Standby GGHUB

If the primary GGHUB cannot communicate with the standby GGHUB, the following messages will be output into the primary CRS trace file (crsd_scriptagent_grid.trc) on the active cluster node:

```
2023-06-21 12:06:59.506 :CLSDYNAM:1427187456: [acfs_primary]{1:8532:12141}
[check] Executing action script: /u01/oracle/scripts/acfs_primary.scr[check]
2023-06-21 12:07:05.666 :CLSDYNAM:1427187456: [acfs_primary]{1:8532:12141}
[check] WARNING: STANDBY not accessible (attempt 1 of 3))
2023-06-21 12:07:18.683 :CLSDYNAM:1427187456: [acfs_primary]{1:8532:12141}
[check] WARNING: STANDBY not accessible (attempt 2 of 3))
2023-06-21 12:07:31.751 :CLSDYNAM:1427187456: [acfs_primary]{1:8532:12141}
[check] WARNING: STANDBY not accessible (attempt 3 of 3))
2023-06-21 12:07:31.751 :CLSDYNAM:1427187456: [acfs_primary]{1:8532:12141}
[check] WARNING: Problem with STANDBY file system (error: 222)
```

At this time, the standby file system is no longer receiving the primary file system changes. The primary file system and Oracle GoldenGate will continue to function unimpeded.

Use the following action plan with this scenario.

- Check the standby file system, using the command 'acfsutil repl util verifystandby /mnt/acfs_gg -v' to determine why the standby hub is inaccessible.
- After fixing the cause of the communication errors, the standby will automatically catch up applying the outstanding primary file system changes. The warning messages will no longer be reported into the CRS trace file, being replaced with the following message:

```
2023-06-21 12:15:01.720 :CLSDYNAM:1427187456: [acfs_primary]{1:8532:12141}
[check] SUCCESS: STANDBY file system /mnt/acfs_gg is ONLINE
```

Use case #2 – Primary GGHUB Failure or Standby GGHUB Cannot Communicate with the Primary GGHUB

If the standby GGHUB cannot communicate with the primary GGHUB, the the following messages will be output into the standby CRS trace file (crsd_scriptagent_grid.trc) on the active cluster node:

```
2023-06-21 12:24:03.823 :CLSDYNAM:4156544768: [acfs_standby]{1:10141:2}
[check] Executing action script: /u01/oracle/scripts/acfs_standby.scr[check]
2023-06-21 12:24:06.928 :CLSDYNAM:4156544768: [acfs_standby]{1:10141:2}
[check] WARNING: PRIMARY not accessible (attempt 1 of 3)
2023-06-21 12:24:19.945 :CLSDYNAM:4156544768: [acfs_standby]{1:10141:2}
[check] WARNING: PRIMARY not accessible (attempt 2 of 3)
2023-06-21 12:24:32.962 :CLSDYNAM:4156544768: [acfs_standby]{1:10141:2}
[check] WARNING: PRIMARY not accessible (attempt 3 of 3)
2023-06-21 12:24:32.962 :CLSDYNAM:4156544768: [acfs_standby]{1:10141:2}
[check] WARNING: Problem with PRIMARY file system (error: 222)
```

At this time, it is unlikely that the standby file system is receiving file system changes from the primary file system.

Use the following action plan with this scenario.

- Check the primary file system, using the command 'acfsutil repl util verifyprimary /mnt/acfs_gg -v' to determine why the primary hub is inaccessible.
- If the primary file system cluster is down and cannot be restarted, issue an ACFS failover on the standby GGHUB:

```
[grid@gghub_stby1]$ /sbin/acfsutil repl failover /mnt/acfs_gg      #
Specify the correct mount point
```

```
[grid@gghub_stby1]$ acfsutil repl info -c -v /mnt/acfs_gg |egrep 'Site:|
Primary status|Background Resources:'
```

```
Site:                                Primary
Primary status:                      Running
Background Resources:                Active
```

- Run the following commands to prepare the acfs_primary resource to start on the new primary hub, and then restart GoldenGate:

```
[grid@gghub_stby1]$ echo "RESTART" > /mnt/acfs_gg/status/acfs_primary
```

```
[grid@gghub_stby1]$ agctl start goldengate <instance_name>      #
Specify the GoldenGate instance name
```

```
[grid@gghub_stby1]$ agctl status goldengate
```

```
Goldengate instance '<instance_name>' is running on gghubstby-node1
```

- When the old primary file system comes back online, if connectivity is resumed between the new primary and old primary, the old primary file system will automatically convert to the standby.
- If the old primary file system comes back online, but connectivity cannot be established between the primary and standby file systems the acfs_primary resource will detect that node had crashed, and because connectivity to the standby cannot be confirmed, GoldenGate will not be started. This avoids a 'split-brain' where two file systems think they are both the primary because they cannot communicate with each other.

Use case #3 – Double Failure Case: Primary GGHUB Failure and Standby GGHUB Connectivity Failure

If the primary GGHUB crashes and communication cannot be established with the standby file system when it comes back online, the following messages will be output into the primary CRS trace file (crsd_scriptagent_grid.trc) on the active cluster node:

```
2023-06-21 17:08:52.621:[acfs_primary]{1:40360:36312} [start] WARNING:
PRIMARY file system /mnt/acfs_gg previously crashed
2023-06-21 17:08:55.678:[acfs_primary]{1:40360:36312} [start] WARNING:
STANDBY not accessible - disabling acfs_primary
```

If an attempt is made to manually restart the primary file system, an additional message will be output into the CRS trace file:

```
2023-06-21 17:25:54.224:[acfs_primary]{1:40360:37687} [start] WARNING:
PRIMARY /mnt/acfs_gg disabled to prevent split brain
```

Use the following action plan with this scenario.

- Check the standby file system, using the command `'acfsutil repl util verifystandby /mnt/acfs_gg -v'` to determine why the standby hub is inaccessible.
- If communication with the the standby file system can re-established, restart GoldenGate on the primary hub:

```
[grid@gghub_prim1]$ agctl start goldengate <instance_name> # Specify the
GoldenGate instance name
```

```
[grid@gghub_prim1]$ agctl status goldengate
```

```
Goldengate instance '<instance_name>' is running on gghub_prim1
```

- If communication with the standby file system cannot be re-established, use the following commands to restart GoldenGate on the primary hub:

```
[grid@gghub_prim1]$ echo "RESTART" > /mnt/acfs_gg/status/acfs_primary
```

```
[grid@gghub_prim1]$ agctl start goldengate <instance_name> #
Specify the GoldenGate instance name
```

```
[grid@gghub_prim1]$ agctl status goldengate
```

```
Goldengate instance '<instance_name>' is running on gghub_prim1
```

- When communication with the standby file system is restored, ACFS Replication will continue to replicate primary file system changes.

Oracle GoldenGate Active-Active Guidance for Developers and Administrators

Oracle MAA provides guidance for developers and administrators when designing an active-active distributed database architecture using Oracle GoldenGate architecture, where data conflicts may occur between GoldenGate replicas.

For these Oracle GoldenGate or MAA Platinum architectures, active-active replication requires conflict detection and resolution, which includes Automatic Conflict Detection and Resolution (Auto-CDR), or the manual method of conflict detection and resolution using the GoldenGate parameters COMPARECOLS and RESOLVECONFLICT (referred to as the COMPARECOLS/RESOLVECONFLICTS method in this topic). Also see [Automatic Conflict Detection and Resolution](#) and [Manual Conflict Detection and Resolution](#) in Oracle GoldenGate documentation.

For more information about GoldenGate in general, refer to [Oracle GoldenGate documentation](#). These instructions are specific for Oracle-to-Oracle replication. For non-Oracle/heterogeneous active-active refer to COMPARECOLS and RESOLVECONFLICT parameters in the [Oracle GoldenGate Parameters and Functions Reference Guide](#).

Preparing for Conflict Resolution

Active-active replication requires tighter integration between Oracle Database and Oracle GoldenGate, and there are some additional requirements that must be met from an application and database structure perspective to ensure a smooth experience.

These requirements only pertain to tables that could have a conflict. If there are tables in the application that will be updated on one server only (for example, a lookup table that contains zip codes to city relationships that is updated once a year as part of a batch process) then those tables do not need to meet these requirements.

To understand why these requirements exist, here's a quick refresher on how a conflict is detected and how it's resolved:

To detect a conflict, Oracle GoldenGate constructs an UPDATE or DELETE DML statement with a WHERE clause that includes the *before* image of every column that the GoldenGate Replicat can use in a WHERE clause. If that DML operation returns a "row not found" exception, that means that there is a conflict, and additional steps are then taken by the Replicat to determine if some other value in that row was changed, or if the row was deleted. This determination is done by querying the target table to see if the row with that primary key exists or not. The resolution portion will provide eventual consistency so that the two rows are the same between two distinct databases. This resolution typically involves ignoring the current DML operation the Replicat is applying, or overwriting the existing data in the target row.

The Oracle GoldenGate documentation covers the different ways to resolve conflicts for both Auto-CDR and COMPARECOLS/RESOLVECONFLICTS methods as well as examples on how each of them work. See [Automatic Conflict Detection and Resolution](#) and [Manual Conflict Detection and Resolution](#) for details.

Requirements

- Primary key requirements

- The table must have a primary key that is made up of one or more columns.
- The uniqueness of a primary key must be generated so that they are unique across all databases. In two-way active-active, you could have sequences assigned to the primary key that are odd (1,3,5,...) in one database and even (2,4,6,...) in the other. This could guarantee that two databases don't have the same primary key for different rows and you'll never have an insert conflict.
- Table structure requirements
 - If you are using manual conflict detection and resolution and defining your own COMPARECOLS/RESOLVECONFLICTS, which is time-stamp-based resolution, the table must have a time stamp column that uses the same time zone across all databases. Auto-CDR does not require a time stamp column on the existing table.
 - Conflicts within Large Objects (JSON, BLOB, CLOB, for example) and user-defined objects (Abstract data types, collections, arrays) cannot be detected by Oracle GoldenGate, because these columns are not used in the WHERE clause by Oracle GoldenGate.
 - If there is a requirement for uniqueness in non-primary key columns this must be enforced using a trigger on the table or through the application.
 - Supplemental logging of all columns is required for tables that you are replicating, and will result in higher redo log generation for those targeted table changes.
- Required GoldenGate settings
 - For Auto-CDR you must include MAPINVISIBLECOLUMNS and LOGALLSUPCOLS in the Extract. The Replicat must be integrated Replicat or parallel Replicat in integrated mode.
 - For COMPARECOLS/RESOLVECONFLICTS, you must use LOGALLSUPCOLS in the Extract.

Best Practices for Active-Active Oracle GoldenGate Architecture

To ensure that your active-active configuration runs smoothly these are some best-practice recommendations to consider.

Application Recommendations

- If conflicts can be avoided, do so. This could include splitting users by geographical region, like routing users East of the Mississippi to use the server in Ashburn, VA, and routing users West of the Mississippi to use the server located in Phoenix, AZ. This geographical split can often avoid conflicts where two users would be modifying the same row. Users could also be split by account numbers (even on one server, odd on another), or names (A-M or N-Z).
- Connection stickiness or geographical affinity is also important. If you are using a load balancer to route traffic to the databases, it should include some type of connection persistence, so that if a user's connection is lost and they reconnect very quickly they are sent to the same server as before. This prevents situations where a user makes changes on server A, disconnects, and then reconnects. If the user is connected to server B before the data is replicated to it, that could cause a poor user experience.
- While Auto-CDR does support the conflict detection and resolution of primary key updates (which it does through the use of the tombstone table), the COMPARECOLS/RESOLVECONFLICTS method does not. So if you are using the COMPARECOLS/RESOLVECONFLICTS method, ensure that primary key updates are performed in a manner that would prevent conflicts from occurring.

- In some cases, it may be necessary to communicate back to a user that something they did in the database was reversed or overwritten due to the conflict resolution policies in place. Auto-CDR has a view (DBA_APPLY_ERROR_MESSAGES) that can be used to track when a conflict has occurred and what resolution was performed. Monitor this table on all databases.

Database Recommendations

- Edition Based Redefinition can be used to help ensure zero downtime for schema changes, and in Oracle Database 23ai, DDL operations on tables with supplemental logging no longer require an exclusive lock on the table. In Oracle Database 19c, Edition Based Redefinition was not leveraged with active GoldenGate replication because of this exclusive lock requirement.
- DDL should only be performed on one node at a time, and only replicated uni-directionally. GoldenGate doesn't provide support for conflicts within DDL operations.

Oracle GoldenGate Recommendations

- Procedural replication should be enabled, so that when you use the DBMS_GOLDENGATE_ADM calls to enable or disable Auto-CDR on a table, that command is replicated to the target database automatically.
`TRANLOGOPTIONS INTEGRATEDPARAMS (ENABLE_PROCEDURAL_REPLICATION Y)`
- Oracle recommends using Integrated Parallel Replicat. Auto-CDR is not supported with Parallel non-integrated Replicat, Classic Replicat, or Coordinated Replicat.
- DDL replication should be enabled for all tables in replication (syntax `DDL INCLUDE ALL`) in both the Extract and Replicat parameter files.

Troubleshooting Oracle GoldenGate

Topics:

- [Troubleshooting MAA GoldenGate Hub](#)
- [Troubleshooting Oracle GoldenGate on Oracle RAC](#)

Troubleshooting MAA GoldenGate Hub

Oracle GoldenGate Extract Failure or Error Conditions Considerations

For Oracle GoldenGate Extract processes using Oracle Data Guard configurations that are using redo transport Maximum Performance or Maximum Availability modes, the following parameter must be added to the Extract process parameter file **on the primary database system** to avoid losing transactions and resulting in logical data inconsistencies:

```
TRANLOGOPTIONS HANDLEDLFAILOVER
```

This parameter prevents Extract from extracting transaction data from redo that has not yet been applied to the Data Guard standby database. This is crucial to preventing Oracle GoldenGate from replicating data to a target database that does not exist in the source standby database.

If this parameter is not specified, after a data loss failover of the source database it is possible to have data in the target database that is not present in the source database, leading to logical data inconsistencies.

By default, after 60 seconds, a warning message is written to the Extract report file when the Extract is stalled because it can't query the standby database applied SCN information. For example:

```
WARNING OGG-02721 Extract has been waiting for the standby database for 60 seconds.
```

The amount of time before the warning message is written to the Extract report file can be adjusted using the Extract parameter `TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_WARNING`.

If the Extract is still unable to query the standby database applied SCN information after 30 minutes (default), the Extract process abend, logging the following message in the Extract report file:

```
ERROR OGG-02722 Extract abended waiting for 1,800 seconds for the standby database to be accessible or caught up with the primary database.
```

If the standby database becomes available before the default 30 timeout expires, Extract continues mining data from the source database and reports the following message to the report file:

INFO OGG-02723 Extract resumed from stalled state and started processing LCRs.

The timeout value of 30 minutes can be adjusted using the Extract parameter `TRANLOGOPTIONS HANDLEDLFAILOVER STANDBY_ABEND <value>`, where value is the number of seconds the standby is unavailable before abending.

If the standby database will be unavailable for a prolonged duration, such as during a planned maintenance outage, and you wish Extract to continue extracting data from the primary database, remove the `TRANLOGOPTIONS HANDLEDLFAILOVER` parameter from the Extract parameter file and restart Extract. Remember to set the parameter after the standby becomes available.

① Note

If extracting from a primary database continues while the standby is unavailable, a data loss failover could result after the standby becomes available, and not all the primary redo was applied before a failover. The GoldenGate target database will contain data that does not exist in the source database.

If the Extract process has been assigned an auto restart profile, as documented in [Cloud: Oracle GoldenGate Microservices Architecture on Oracle Exadata Database Service Configuration Best Practices](#), after a Data Guard role transition, the Extract process automatically restarts. Extract continues to mine redo data from the new primary database, ignoring the current state of the new standby database, until a default 5 minute timeout period expires. After this time, if the standby is not available Extract will abend with the following errors:

```
INFO OGG-25053 Timeout waiting for 300 seconds for standby database
reinstatement. Now enforcing HANDLEDLFAILOVER.
```

```
ERROR OGG-06219 Unable to extract data from the Logmining server OGG$CAP_XXXXX.
```

```
ERROR OGG-02078 Extract encountered a fatal error in a processing thread and is
abending.
```

Extract continues attempting to automatically restart, based on the Oracle GoldenGate Microservices auto restart profile, and fails because it reaches the `HANDLEDLFAILOVER` timeout, until the number of retries is reached or the new standby database becomes available.

During the timeout period following a database role transition, the `HANDLEDLFAILOVER` parameter is automatically suspended, so data is replicated to the Oracle GoldenGate replica database without consideration of the source standby database not being kept up to date. The timeout period for the standby database to start up before Extract abends can be adjusted using the Extract parameter `TRANLOGOPTIONS DLFAILOVER_TIMEOUT`.

It is recommended that you leave `DLFAILOVER_TIMEOUT` at the default of 5 minutes to allow the old primary to convert to a standby. If the new standby database will be unavailable for an extended period of time or completely gone, then to ensure that Extract starts and remains running, you must remove the `HANDLEDLFAILOVER` parameter from the Extract parameter file. After removing the parameter, Extract no longer waits until redo has been applied to the standby database before extracting the data.

During the time it takes for the standby database to come back online and apply all of the redo from the primary database, there is data divergence between it and the Oracle GoldenGate replica database. This divergence is resolved when the standby database is up to date. At this point you can add the `HANDLEDLFAILOVER` parameter back into the integrated Extract process parameter file, and then stop and restart the Extract.

When Oracle Data Guard Fast Start Failover is disabled, such that the broker can automatically fail over to a standby database in the event of loss of the primary database, you must specify an additional integrated Extract parameter:

```
TRANLOGOPTIONS FAILOVERTARGETDESTID n
```

This parameter identifies which standby database the Oracle GoldenGate Extract process must remain behind, with regards to not extracting redo data that has not yet been applied to the standby database.

If Oracle Data Guard Fast Start Failover is disabled, and you don't specify the additional integrated Extract parameter FAILOVERTARGETDESTID, the extract will abend with the following errors:

```
ERROR OGG-06219 Unable to extract data from the Logmining server OGG$CAP_XXXXX.
```

```
ERROR OGG-02078 Extract encountered a fatal error in a processing thread and is
abending.
```

Troubleshooting ACFS Replication

The health of ACFS replication is determined by the `acfsutil repl util verifyprimary/verifystandby` commands. These commands are called by the example CRS action scripts `acfs_primary.scr` and `acfs_standby.scr`, but they are also implicitly called during a file system role transition.

Both commands will return a value of '0' if there are no problems detected. If a non-zero value is returned, run the same command with verbose flag to see comprehensive output of the verification tests.

As the grid user on the standby GGHUB system, verify the ACFS replication with the primary GGHUB:

```
[grid@ggstub1]$ acfsutil repl util verifyprimary /mnt/acfs_gg -v
- Attempting to ping clust1-vip1
- ping successful
- Attempting to ssh
  '/usr/bin/ssh -o BatchMode=true -o Ciphers=aes128-ctr -o ConnectTimeout=3
-x oracle@clust1-vip1 true 2>&1'
- ssh output: Host key verification failed.
- ssh output: Host key verification failed.
- ssh attempt failed, ret=255
verifyprimary return code: 255
```

The errors reported by the verify command, Host key verification failed, clearly showing why it failed. In this example, there is a problem with the ssh configuration between the standby and the primary file system GGHUBs. Once the problem has been resolved, rerun the verify commands to ensure there are no further problems.

After a failover has completed, it is recommended to check the `acfsutil` trace files for the reason behind the failover. The `acfsutil` trace files are located in the CRS trace file directory, which defaults to `/u01/app/grid/diag/crs/`hostname`/crs/trace/crsd_scriptagent_grid.trc`.

Below are some common failures that can occur with incorrect ACFS replication configuration.

SSH daemon is shutdown or not configured to run on the VIP

When using an application VIP on the ACFS primary and standby GGHub, the ssh daemon must be configured to listen for incoming connections on the VIP address. If this configuration is not done, or the ssh daemon is not running on either of the current primary/standby hosts the `verifyprimary` or `verifystandby` commands will fail with the following error.

As the `grid` user on the primary GGHub system, verify the ACFS replication with the standby GGHub:

```
[grid@gghub_prim1]$ acfsutil repl util verifystandby /mnt/acfs_gg -v
```

```
- Attempting to ping gghubstby.goldengate.com
- ping successful
- Attempting to ssh
  '/usr/bin/ssh -o BatchMode=true -o Ciphers=aes128-ctr -o
ConnectTimeout=3 -x oracle@gghub_stby-avip true 2>&1'

- ssh output: ssh: connect to host gghub_stby1 port 22: Connection refused
- ssh output: ssh: connect to host gghub_stby2 port 22: Connection refused

- ssh attempt failed, ret=255
verifystandby return code: 255
```

As the `grid` user on the standby GGHub system, check that the resource application VIP and `sshd_restart` are running and restart them if not:

```
[grid@gghub_stby1 ~]$ crsctl stat res -w "TYPE co app.appviptypex2"
```

```
NAME=gghubstby
TYPE=app.appviptypex2.type
TARGET=OFFLINE
STATE=OFFLINE
```

```
[grid@gghub_stby1 ~]$ crsctl start res gghubstby
```

```
CRS-2672: Attempting to start 'gghubstby' on 'gghub_stby1'
CRS-2676: Start of 'gghubstby' on 'gghub_stby1' succeeded
CRS-2672: Attempting to start 'sshd_restart' on 'gghub_stby1'
CRS-2676: Start of 'sshd_restart' on 'gghub_stby1' succeeded
```

Check that `acfsutil repl verifystandby/verifyprimary` returns a result of '0' from both the primary and standby host.

Primary ACFS background resources are not running

1. The primary or standby ACFS servers are not accessible
2. ACFS Replication ssh user problem
3. SSH Host key verification failed

Troubleshooting Oracle GoldenGate

There may be occasions when GoldenGate processes are not successfully started on an Oracle RAC node. There are number of files generated by GoldenGate, XAG, and CRS that should be reviewed to determine the cause of the problem.

Below is a list of important log and trace files, along with their example locations and some example output.

XAG log file

Location: <XAG installation directory>/log/<hostname>

Example location: /u01/app/grid/xag/log/`hostname`

File name: agctl_goldengate_grid.trc

Contains all commands executed with agctl along with the output from the commands, including those that CRS executes.

```
2022-04-18 11:52:21: stop resource success
2022-04-18 11:52:38: agctl start goldengate <instance_name>
2022-04-18 11:52:38: executing cmd: /u01/app/19.0.0.0/grid/bin/crsctl status
res xag.<INSTANCE_NAME>.goldengate
2022-04-18 11:52:38: executing cmd: /u01/app/19.0.0.0/grid/bin/crsctl status
res xag.<INSTANCE_NAME>.goldengate -f
2022-04-18 11:52:38: executing cmd: /u01/app/19.0.0.0/grid/bin/crsctl start
resource xag.<INSTANCE_NAME>.goldengate -f
2022-04-18 11:52:45: Command output:
> CRS-2672: Attempting to start 'xag.<INSTANCE_NAME>.goldengate' on 'exadb-
node1'
> CRS-2676: Start of 'xag.<INSTANCE_NAME>.goldengate' on 'exadb-node1'
succeeded
>End Command output
2022-04-18 11:52:45: start resource success
```

XAG GoldenGate instance trace file

Location: <XAG installation directory>/log/<hostname>

Example location: /u01/app/grid/xag/log/`hostname`

File name: <GoldenGate_instance_name>_agent_goldengate.trc

It contains the output from the commands executed by agctl, the environment variables used, and any debug output enabled for the underlying commands.

```
2022-04-18 12:14:46: Exported ORACLE_SID ggdg1
2022-04-18 12:14:46: Exported GGS_HOME /u01/oracle/goldengate/gg21c_MS
2022-04-18 12:14:46: Exported OGG_CONF_HOME /mnt/dbfs/goldengate/deployments/
ggsm01/etc/conf
2022-04-18 12:14:46: Exported LD_LIBRARY_PATH
/u01/oracle/goldengate/gg21c_MS:/u01/app/19.0.0.0/grid/lib:/etc/
ORCLcluster/lib
2022-04-18 12:14:46: Exported LD_LIBRARY_PATH_64 /u01/oracle/goldengate/
gg21c_MS
2022-04-18 12:14:46: Exported LIBPATH /u01/oracle/goldengate/gg21c_MS
2022-04-18 12:14:46: ogg input = {"oggHome":"/u01/oracle/goldengate/
gg21c_MS", "serviceManager":{"oggConfHome":"/mnt/dbfs/goldengate/deployments/
ggsm01/etc/
conf", "portNumber":9100}, "username":"<username>", "credential":"*****"}
```

```
2022-04-18 12:14:46: About to exec /u01/oracle/goldengate/gg21c_MS/bin/
XAGTask HealthCheck
2022-04-18 12:14:47: XAGTask retcode = 0
```

CRS trace file

Location: /u01/app/grid/diag/crs/<hostname>/crs/trace

Example location: /u01/app/grid/diag/crs/`hostname`/crs/trace

File name: crsd_scriptagent_oracle.trc

Contains the output created by any CRS resource action scripts, like XAG or dbfs_mount. This trace file is crucial to determining why DBFS or GoldenGate did not start on a RAC node.

```
2022-04-18 11:52:38.634 : AGFW:549631744: {1:30281:59063} Agent received
the message: RESOURCE_START[xag.<INSTANCE_NAME>.goldengate 1 1] ID
4098:4125749
2022-04-18 11:52:38.634 : AGFW:549631744: {1:30281:59063} Preparing START
command for: xag.<INSTANCE_NAME>.goldengate 1 1
2022-04-18 11:52:38.634 : AGFW:549631744: {1:30281:59063}
xag.<INSTANCE_NAME>.goldengate 1 1 state changed from: OFFLINE to: STARTING
2022-04-18 11:52:38.634 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] Executing action script: /u01/oracle/XAG_MA/bin/
aggoldengatescaas[start]
2022-04-18 11:52:38.786 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] GG agent running command 'start' on
xag.<INSTANCE_NAME>.goldengate
2022-04-18 11:52:42.140 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] ServiceManager fork pid = 265747
2022-04-18 11:52:42.140 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] Waiting for /mnt/dbfs/goldengate/deployments/
ggsm01/var/run/ServiceManager.pid
2022-04-18 11:52:42.140 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] Waiting for SM to start
2022-04-18 11:52:42.140 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] ServiceManager PID = 265749
2022-04-18 11:52:43.643 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] XAGTask retcode = 0
2022-04-18 11:52:43.643 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] XAG HealthCheck after start returned 0
2022-04-18 11:52:43.643 : AGFW:558036736: {1:30281:59063} Command: start
for resource: xag.<INSTANCE_NAME>.goldengate 1 1 completed with status:
SUCCESS
2022-04-18 11:52:43.643 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [check] Executing action script: /u01/oracle/XAG_MA/bin/
aggoldengatescaas[check]
2022-04-18 11:52:43.644 : AGFW:549631744: {1:30281:59063} Agent sending
reply for: RESOURCE_START[xag.<INSTANCE_NAME>.goldengate 1 1] ID 4098:4125749
2022-04-18 11:52:43.795 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [check] GG agent running command 'check' on
xag.<INSTANCE_NAME>.goldengate
2022-04-18 11:52:45.548 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [check] XAGTask retcode = 0
2022-04-18 11:52:45.548 : AGFW:549631744: {1:30281:59063}
xag.<INSTANCE_NAME>.goldengate 1 1 state changed from: STARTING to: ONLINE
```

GoldenGate deployment log files

Location: <Goldengate_deployment_directory>/<instance_name>/var/log

Example location: /mnt/dbfs/goldengate/deployments/<instance_name>/var/log

File names: adminsvr.log, recvsrvr.log, pmsrvr.log, distsrvr.log

Contains the output of start, stop, and status checks of the Oracle GoldenGate deployment processes (Administration Server, Distribution Server, Receiver Server, and Performance Metrics Server).

```
2022-04-18T11:52:42.645-0400 INFO | Setting deploymentName to
'<instance_name>'. (main)
2022-04-18T11:52:42.665-0400 INFO | Read SharedContext from store for length
19 of file '/mnt/dbfs/goldengate/deployments/<instance_name>/var/lib/conf/
adminsvr-resources.dat'. (main)
2022-04-18T11:52:42.723-0400 INFO | XAG Integration enabled (main)
2022-04-18T11:52:42.723-0400 INFO | Configuring security. (main)
2022-04-18T11:52:42.723-0400 INFO | Configuring user authorization secure
store path as '/mnt/dbfs/goldengate/deployments/<instance_name>/var/lib/
credential/secureStore/'. (main)
2022-04-18T11:52:42.731-0400 INFO | Configuring user authorization as
ENABLED. (main)
2022-04-18T11:52:42.749-0400 INFO | Set network configuration. (main)
2022-04-18T11:52:42.749-0400 INFO | Asynchronous operations are enabled with
default synchronous wait time of 30 seconds (main)
2022-04-18T11:52:42.749-0400 INFO | HttpServer configuration complete. (main)
2022-04-18T11:52:42.805-0400 INFO | SIGHUP handler installed. (main)
2022-04-18T11:52:42.813-0400 INFO | SIGINT handler installed. (main)
2022-04-18T11:52:42.815-0400 INFO | SIGTERM handler installed. (main)
2022-04-18T11:52:42.817-0400 WARN | Security is configured as 'disabled'.
(main)
2022-04-18T11:52:42.818-0400 INFO | Starting service listener... (main)
2022-04-18T11:52:42.819-0400 INFO | Mapped 'ALL' interface to address
'ANY:9101' with default IPV4/IPV6 options identified by 'exadb-node1.domain'.
(main)
2022-04-18T11:52:42.821-0400 INFO | Captured 1 interface host names: 'exadb-
node1.domain' (main)
2022-04-18T11:52:42.824-0400 INFO | The Network ipACL specification is empty.
Accepting ANY address on ALL interfaces. (main)
2022-04-18T11:52:42.826-0400 INFO | Server started at
2022-04-18T11:52:42.827-05:00 (2022-04-18T15:52:42.827Z GMT) (main)
```

GoldenGate report files

Location: <Goldengate_deployment_directory>/<instance_name>/var/lib/report

Example location: /mnt/dbfs/goldengate/deployments/<instance_name>/var/lib/report

The GoldenGate report files contain important information, warning messages, and errors for all GoldenGate processes, including the Manager processes. If any of the GoldenGate processes fail to start or abend when running, the process report file will contain important information that can be used to determine the cause of the failure.

```
2022-04-23 13:01:50 ERROR OGG-00446 Unable to lock file " /mnt/acfs_gg/
deployments/<instance_name>/var/lib/checkpt/EXT_1A.cpe" (error 95, Operation
```

not supported).
2022-04-23 13:01:50 ERROR OGG-01668 PROCESS ABENDING.

Troubleshooting GGHUB Deployment Using Ansible

Check the Primary and Standby GGHUB Clusterware Resources Status

Ansible automation creates the GGHUB clusterware resources with names that include the GGHUB deployment name, primary GGHUB group name, and standby GGHUB group name provided in the inventory file.

In this example, the GGHUB deployment name is `maahub`, the primary GGHUB group name is `maahuba`, and the standby GGHUB group name is `maahubb`.

The following table explains the GGHUB clusterware resources using example names.

Example GGHUB clusterware resource name	Purpose
<code>acfs_primary</code>	ACFS clusterware resource that exists on both primary and standby GGHUB clusters, but state is ONLINE only on the primary GGHUB cluster with primary ACFS file system. The <code>acfs_primary</code> clusterware resource is used to automatically determine if Oracle GoldenGate can start processes on the primary GGHUB.
<code>acfs_standby</code>	ACFS clusterware resource that exists on both primary and standby GGHUB clusters. The <code>acfs_standby</code> clusterware resource checks that the local file system is a standby file system and verifies the remote primary file system status.
<code>maahuba-app-vip1</code>	APP VIP resource for the primary GGHUB cluster using the primary GGHUB group name.
<code>maahubb-app-vip1</code>	APP VIP resource for the standby GGHUB cluster using the standby GGHUB group name.
<code>nginx</code>	NGINX clusterware resource. Resource exists on both primary and standby GGHUB clusters.
<code>ora.data.maahub.acfs</code>	ACFS clusterware resource that exists on both primary and standby GGHUB clusters, and is named using the format <code>ora.diskgroup_name.volume_name.acfs</code> . The <code>ora.data.maahub.acfs</code> clusterware resource ensures that the ACFS file system is available before starting GoldenGate Microservices.
<code>ora.repl.dupd.data.maahub.acfs</code>	ACFS replication clusterware resource using the GGHUB deployment name. Resource starts on the primary GGHUB cluster which is the source of ACFS replication. After role reversal this resource starts on the new primary GGHUB cluster.
<code>xag.maahub.goldengate</code>	XAG clusterware resource. Resource exists on both primary and standby GGHUB clusters, but state is ONLINE only on the primary GGHUB cluster.

Primary GGHUB Clusterware resources status

Validate the primary GGHUB clusterware resources on the first node of the primary GGHUB cluster using the `crsctl` utility as the clusterware software owner OS user `grid`.

The deployment name used in this example is `maahub` and the primary group name is `maahuba`.

```
$ crsctl status res acfs_primary acfs_standby nginx
<primary_group>-app-vip1 xag.<gg_deployment_name>.goldengate
ora.data.<gg_deployment_name>.acfs
ora.repl.dupd.data.<gg_deployment_name>.acfs -t
OR
$ crsctl status res acfs_primary acfs_standby nginx maahuba-app-vip1
xag.maahub.goldengate
ora.data.maahub.acfs ora.repl.dupd.data.maahub.acfs -t
```

Example output

```
-----
--
Name                Target  State        Server                State details
-----
--
Cluster Resources
-----
--
acfs_primary
   1                ONLINE  ONLINE       maahuba1              STABLE
acfs_standby
   1                ONLINE  ONLINE       maahuba1              STABLE
maahuba-app-vip1
   1                ONLINE  ONLINE       maahuba1              STABLE
nginx
   1                ONLINE  ONLINE       maahuba1              STABLE
ora.data.maahub.acfs
   1                ONLINE  ONLINE       maahuba1              mounted on /opt/
oracle/gghub/maahub, STABLE
ora.repl.dupd.data.maahub.acfs
   1                ONLINE  ONLINE       maahuba1              STABLE
xag.maahub.goldengate
   1                ONLINE  ONLINE       maahuba1              STABLE
```

Standby GGHUB Clusterware resources status

Validate the standby GGHUB clusterware resources status on the first node of the standby GGHUB cluster.

The standby group name in this example is `maahubb`.

Notice that the `acfs_primary` resource state is `OFFLINE` on the standby cluster as expected.

```
$ crsctl status res acfs_primary acfs_standby ora.data.maahub.acfs
maahubb-app-vip1 maahubb-app-vip1 xag.maahub.goldengate nginx -t
```

Example output

expected.

```

-----
--
Name          Target State      Server          State details
-----
--
Cluster Resources
-----
--
acfs_primary
  1          OFFLINE OFFLINE          STABLE
acfs_standby
  1          ONLINE  ONLINE          maahubb1        STABLE
maahubb-app-vip1
  1          ONLINE  ONLINE          maahubb1        STABLE
nginx
  1          OFFLINE OFFLINE          maahubb1        STABLE
ora.data.maahub.acfs
  1          ONLINE  ONLINE          maahubb1        mounted on /opt/
oracle/ggghub/maahub, STABLE
xag.maahub.goldengate
  1          ONLINE  OFFLINE          STABLE

```

Check the GGHub ACFS Mount Replication Status

Check the ACFS replication initialization status

As the grid OS user on the primary and standby GGHub nodes, monitor the initialization progress. When the status changes to "Send Completed" it means the initial primary file system copy has finished, and the primary file system is now being replicated to the standby host, as shown here.

On the Primary GGHub

```
[grid@ggghub_prim1 ~]$ /sbin/acfsutil repl info -c -v /opt/oracle/ggghub/maahub
| grep Status
```

Status: Send Completed

On the Standby GGHub

```
[grid@ggghub_prim1 ~]$ /sbin/acfsutil repl info -c -v /opt/oracle/ggghub/maahub
| grep Status
```

Status: Receive Completed OR Receiving
incremental differences

Verify the ACFS replicated file system

As the grid OS user on the primary and standby GGHub nodes, verify and monitor the ACFS replicated file system.

On the Primary GGHub


```
[grid@gghub_prim1 ~]$ acfsutil repl util verifystandby /opt/oracle/gghub/maahub
```

```
verifystandby returned: 0
```

```
# On the Standby GGHUB
```

```
[grid@gghubtest31 ~]$ acfsutil repl util verifyprimary /opt/oracle/gghub/maahub
```

```
verifyprimary returned: 0
```

① Note

Both commands return a value of 0 (zero) if there are no problems detected. If a non-zero value is returned, refer to [Troubleshooting ACFS Replication](#) for information about monitoring, diagnosing, and resolving common issues with ACFS Replication before continuing.

Check the ACFS replication status

For a healthy ACFS replication the primary GGHUB ACFS replication status should be Running and retries made number should be 0 on the primary GGHUB cluster.

Use the following command to check the ACFS replication information on the primary GGHUB cluster.

```
$ /usr/sbin/acfsutil repl info -v -c /opt/oracle/gghub/maahub
```

Example output

```
Site: Primary
Primary hostname: maahuba-app-
vip1.clientsubnet.vcnfraans1.oraclevcn.com
Primary path: /opt/oracle/gghub/maahub
Primary status: Running
Background Resources: Active

Standby connect string: grid@maahubb-app-
vip1.clientsubnet.vcnfraans1.oraclevcn.com
Standby path: /opt/oracle/gghub/maahub
Replication interval: 0 days, 0 hours, 0 minutes, 0 seconds
Sending primary as of: Fri Jul 18 21:10:39 2025
Status: Sending incremental differences
Lag Time: 00:00:37 (Constant mode)
Retries made: 0
Standby Backup Snapshot: Up to Date
Last send started at: Fri Jul 18 21:10:39 2025
Last send completed at: In progress
Next send starts at: On send completion
Replicated tags:
Data transfer compression: Off
ssh strict host key checking: On
```

```

Debug log level:          3
Replication ID:          0xa1012b70

```

Standby GGHUB ACFS replication status

Use the following command to check the ACFS replication information on the standby GGHUB cluster.

The status should not have any errors and elapsed time for last receive should not be higher than few seconds.

```
$ /usr/sbin/acfsutil repl info -v -c /opt/oracle/gghub/maahub
```

Example output

```

Site:                      Standby
Primary hostname:          maahuba-app-
vip1.clientsubnet.vcnfraans1.oraclevcn.com
Primary path:              /opt/oracle/gghub/maahub

Standby connect string:    grid@maahubb-app-
vip1.clientsubnet.vcnfraans1.oraclevcn.com
Standby path:              /opt/oracle/gghub/maahub
Replication interval:      0 days, 0 hours, 0 minutes, 0 seconds
Last sync time with primary: Fri Jul 18 21:11:44 2025
Receiving primary as of:   Fri Jul 18 21:11:44 2025
Status:                    Receive Completed
Standby Backup Snapshot:   Up to Date
Last receive started at:   Fri Jul 18 21:11:44 2025
Last receive completed at: Fri Jul 18 21:11:50 2025
Elapsed time for last receive: 0 days, 0 hours, 0 minutes, 6 seconds
Data transfer compression: Off
ssh strict host key checking: On
Debug log level:          3
Replication ID:          0xa1012b70

```

Removal of GGHUB Clusterware Resources and GoldenGate Replication Processes

To remove the GGHUB configuration from both the primary and standby GGHUB clusters including the ACFS mount point and GoldenGate Extract and Replicat processes execute the Ansible playbook `oracle.gghub.remove`.

Playbook `oracle.gghub.remove` asks for confirmation before proceeding with the removal steps.

Note

Use the `oracle.gghub.remove` playbook with caution, because it removes all of the GoldenGate Extract and Replicat processes managed by GGHUB, along with GGHUB clusterware resources and ACFS mounts from both primary and standby GGHUB clusters.

Playbook `oracle.gghub.remove` prompts for the Ansible vault password. Use the GoldenGate admin password from the vault for removal.

Execute the `oracle.gghub.remove` playbook as follows to remove the GGHUB deployment and GoldenGate replication processes.

```
(gghub) [~]$ ansible-playbook oracle.gghub.remove
-i /u01/maagghub/inventory/maahub.yml -e @/u01/maagghub/inventory/maahub.key
--ask-vault-pass
```

GGHub ACFS Replication Resync Using Ansible

GGHub ACFS replication resync can be performed using MAA Ansible collection playbook `oracle.gghub.maa_ha` with tag `acfs_resync`.

ACFS replication resync initializes the replication between the primary and standby GGHUB clusters.

A resync may be required if ACFS replication is in error status and cannot be recovered.

```
(gghub) [~]$ ansible-playbook oracle.gghub.maa_ha
-i /u01/maagghub/inventory/maahub.yml --tags acfs_resync
```

After resync is complete, validate the ACFS replication status on the primary and standby GGHUB clusters using the commands provided in "Check the ACFS replication status" above.

Troubleshooting Oracle GoldenGate on Oracle RAC

There may be occasions when Oracle GoldenGate processes are not successfully started on an Oracle RAC node. Several files generated by Oracle GoldenGate, XAG, and CRS should be reviewed to determine the cause of the problem.

Below is a list of important log and trace files, their example locations, and some examples of output.

XAG log file

Location: <XAG installation directory>/log/<hostname>

Example location: /u01/app/grid/xag/log/`hostname`

File name: `agctl_goldengate_grid.trc`

Contains all commands executed with `agctl` along with the output from the commands, including those that CRS executes.

Example:

```
2022-04-18 11:52:21: stop resource success
2022-04-18 11:52:38: agctl start goldengate <instance_name>
2022-04-18 11:52:38: executing cmd: /u01/app/19.0.0.0/grid/bin/crsctl status
res xag.<INSTANCE_NAME>.goldengate
2022-04-18 11:52:38: executing cmd: /u01/app/19.0.0.0/grid/bin/crsctl status
res xag.<INSTANCE_NAME>.goldengate -f
2022-04-18 11:52:38: executing cmd: /u01/app/19.0.0.0/grid/bin/crsctl start
resource xag.<INSTANCE_NAME>.goldengate -f
2022-04-18 11:52:45: Command output:
> CRS-2672: Attempting to start 'xag.<INSTANCE_NAME>.goldengate' on 'exadb-
node1'
> CRS-2676: Start of 'xag.<INSTANCE_NAME>.goldengate' on 'exadb-node1'
```

succeeded
 >End Command output
 2022-04-18 11:52:45: start resource success

XAG GoldenGate instance trace file

Location: <XAG installation directory>/log/<hostname>

Example location: /u01/app/grid/xag/log/`hostname`

File name: <GoldenGate_instance_name>_agent_goldengate.trc

It contains the output from the commands executed by agctl, the environment variables used, and any debug output enabled for the underlying commands.

Example:

```
2022-04-18 12:14:46: Exported ORACLE_SID ggdg1
2022-04-18 12:14:46: Exported GGS_HOME /u01/oracle/goldengate/gg21c_MS
2022-04-18 12:14:46: Exported OGG_CONF_HOME /mnt/dbfs/goldengate/deployments/
ggsm01/etc/conf
2022-04-18 12:14:46: Exported LD_LIBRARY_PATH
/u01/oracle/goldengate/gg21c_MS:/u01/app/19.0.0.0/grid/lib:/etc/
ORCLcluster/lib
2022-04-18 12:14:46: Exported LD_LIBRARY_PATH_64 /u01/oracle/goldengate/
gg21c_MS
2022-04-18 12:14:46: Exported LIBPATH /u01/oracle/goldengate/gg21c_MS
2022-04-18 12:14:46: ogg input = {"oggHome":"/u01/oracle/goldengate/
gg21c_MS", "serviceManager":{"oggConfHome":"/mnt/dbfs/goldengate/deployments/
ggsm01/etc/conf", "portNumber":9100}, "username":"admin", "credential":"xyz"}
2022-04-18 12:14:46: About to exec /u01/oracle/goldengate/gg21c_MS/bin/
XAGTask HealthCheck
2022-04-18 12:14:47: XAGTask retcode = 0
```

CRS trace file

Location: /u01/app/grid/diag/crs/<hostname>/crs/trace

Example location: /u01/app/grid/diag/crs/`hostname`/crs/trace

File name: crsd_scriptagent_oracle.trc

Contains the output created by any CRS resource action scripts, like XAG or dbfs_mount. This trace file is crucial to determining why DBFS or GoldenGate did not start on a RAC node.

Example:

```
2022-04-18 11:52:38.634 : AGFW:549631744: {1:30281:59063} Agent received
the message: RESOURCE_START[xag.<INSTANCE_NAME>.goldengate 1 1] ID
4098:4125749
2022-04-18 11:52:38.634 : AGFW:549631744: {1:30281:59063} Preparing START
command for: xag.<INSTANCE_NAME>.goldengate 1 1
2022-04-18 11:52:38.634 : AGFW:549631744: {1:30281:59063}
xag.<INSTANCE_NAME>.goldengate 1 1 state changed from: OFFLINE to: STARTING
2022-04-18 11:52:38.634 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] Executing action script: /u01/oracle/XAG_MA/bin/
agggoldengatescaas[start]
2022-04-18 11:52:38.786 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
```

```

{1:30281:59063} [start] GG agent running command 'start' on
xag.<INSTANCE_NAME>.goldengate
2022-04-18 11:52:42.140 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] ServiceManager fork pid = 265747
2022-04-18 11:52:42.140 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] Waiting for /mnt/dbfs/goldengate/deployments/
ggs01/var/run/ServiceManager.pid
2022-04-18 11:52:42.140 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] Waiting for SM to start
2022-04-18 11:52:42.140 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] ServiceManager PID = 265749
2022-04-18 11:52:43.643 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] XAGTask retcode = 0
2022-04-18 11:52:43.643 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [start] XAG HealthCheck after start returned 0
2022-04-18 11:52:43.643 : AGFW:558036736: {1:30281:59063} Command: start
for resource: xag.<INSTANCE_NAME>.goldengate 1 1 completed with status:
SUCCESS
2022-04-18 11:52:43.643 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [check] Executing action script: /u01/oracle/XAG_MA/bin/
aggoldengatescaas[check]
2022-04-18 11:52:43.644 : AGFW:549631744: {1:30281:59063} Agent sending
reply for: RESOURCE_START[xag.<INSTANCE_NAME>.goldengate 1 1] ID 4098:4125749
2022-04-18 11:52:43.795 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [check] GG agent running command 'check' on
xag.<INSTANCE_NAME>.goldengate
2022-04-18 11:52:45.548 :CLSDYNAM:558036736: [xag.<INSTANCE_NAME>.goldengate]
{1:30281:59063} [check] XAGTask retcode = 0
2022-04-18 11:52:45.548 : AGFW:549631744: {1:30281:59063}
xag.<INSTANCE_NAME>.goldengate 1 1 state changed from: STARTING to: ONLINE

```

GoldenGate deployment log files

Location: <Goldengate_deployment_directory>/<instance_name>/var/log

Example location: /mnt/dbfs/goldengate/deployments/<instance_name>/var/log

File names: adminsvr.log, recvsrvr.log, pmsrvr.log, distsvr.log

Contains the output of start, stop, and status checks of the Oracle GoldenGate deployment processes (Administration Server, Distribution Server, Receiver Server, and Performance Metrics Server).

Example:

```

2022-04-18T11:52:42.645-0400 INFO | Setting deploymentName to
'<instance_name>'. (main)
2022-04-18T11:52:42.665-0400 INFO | Read SharedContext from store for length
19 of file '/mnt/dbfs/goldengate/deployments/<instance_name>/var/lib/conf/
adminsvr-resources.dat'. (main)
2022-04-18T11:52:42.723-0400 INFO | XAG Integration enabled (main)
2022-04-18T11:52:42.723-0400 INFO | Configuring security. (main)
2022-04-18T11:52:42.723-0400 INFO | Configuring user authorization secure
store path as '/mnt/dbfs/goldengate/deployments/<instance_name>/var/lib/
credential/secureStore/'. (main)
2022-04-18T11:52:42.731-0400 INFO | Configuring user authorization as
ENABLED. (main)

```

```

2022-04-18T11:52:42.749-0400 INFO | Set network configuration. (main)
2022-04-18T11:52:42.749-0400 INFO | Asynchronous operations are enabled with
default synchronous wait time of 30 seconds (main)
2022-04-18T11:52:42.749-0400 INFO | HttpServer configuration complete. (main)
2022-04-18T11:52:42.805-0400 INFO | SIGHUP handler installed. (main)
2022-04-18T11:52:42.813-0400 INFO | SIGINT handler installed. (main)
2022-04-18T11:52:42.815-0400 INFO | SIGTERM handler installed. (main)
2022-04-18T11:52:42.817-0400 WARN | Security is configured as 'disabled'.
(main)
2022-04-18T11:52:42.818-0400 INFO | Starting service listener... (main)
2022-04-18T11:52:42.819-0400 INFO | Mapped 'ALL' interface to address
'ANY:9101' with default IPV4/IPV6 options identified by 'exadb-node1.domain'.
(main)
2022-04-18T11:52:42.821-0400 INFO | Captured 1 interface host names: 'exadb-
node1.domain' (main)
2022-04-18T11:52:42.824-0400 INFO | The Network ipACL specification is empty.
Accepting ANY address on ALL interfaces. (main)
2022-04-18T11:52:42.826-0400 INFO | Server started at
2022-04-18T11:52:42.827-05:00 (2022-04-18T15:52:42.827Z GMT) (main)

```

GoldenGate report files

Location: <Goldengate_deployment_directory>/<instance_name>/var/lib/report

Example location: /mnt/dbfs/goldengate/deployments/<instance_name>/var/lib/report

The GoldenGate report files contain important information, warning messages, and errors for all GoldenGate processes, including the Manager processes. If any of the GoldenGate processes fail to start or abend when running, the process report file will contain important information that can be used to determine the cause of the failure.

Example errors from an Extract report file:

```

2022-04-23 13:01:50 ERROR OGG-00446 Unable to lock file " /mnt/acfs_gg/
deployments/<instance_name>/var/lib/checkpt/EXT_1A.cpe" (error 95, Operation
not supported).
2022-04-23 13:01:50 ERROR OGG-01668 PROCESS ABENDING.

```

Example Configuration Problems

Below are some configuration problems that can be encountered with GoldenGate in a RAC environment and how to diagnose and resolve them.

Incorrect parameter settings in the mount-dbfs.conf file

When XAG fails to mount DBFS, the failure will be reported either on the command line (if you are running the manual agctl command) or in the XAG log file:

```
$ agctl start goldengate <instance_name> --node exadb-node1
```

```

CRS-2672: Attempting to start 'dbfs_mount' on 'exadb-node1'
CRS-2674: Start of 'dbfs_mount' on 'exadb-node1' failed
CRS-2679: Attempting to clean 'dbfs_mount' on 'exadb-node1'
CRS-2681: Clean of 'dbfs_mount' on 'exadb-node1' succeeded
CRS-4000: Command Start failed, or completed with errors.

```

The XAG log file (agctl_goldengate_grid.trc) has the advantage that it shows timestamps that can be used when looking at other log or trace files:

```
2022-04-19 15:32:16: executing cmd: /u01/app/19.0.0.0/grid/bin/crsctl start
resource xag.<INSTANCE_NAME>.goldengate -f -n exadb-node1
2022-04-19 15:32:19: Command output:
> CRS-2672: Attempting to start 'dbfs_mount' on 'exadb-node1'
> CRS-2674: Start of 'dbfs_mount' on 'exadb-node1' failed
> CRS-2679: Attempting to clean 'dbfs_mount' on 'exadb-node1'
> CRS-2681: Clean of 'dbfs_mount' on 'exadb-node1' succeeded
> CRS-4000: Command Start failed, or completed with errors.
>End Command output
2022-04-19 15:32:19: start resource failed rc=1
```

Next, check the CRS trace file (crsd_scriptagent_oracle.trc), which shows why DBFS failed to mount. Below are some example errors caused by incorrect parameter settings in the mount-dbfs.conf file.

- Incorrect DBNAME

```
2022-04-19 15:32:16.679 : AGFW:1190405888: {1:30281:17383} dbfs_mount
1 1 state changed from: UNKNOWN to: STARTING
2022-04-19 15:32:16.680 :CLSDYNAM:1192507136: [dbfs_mount]{1:30281:17383}
[start]
Executing action script: /u01/oracle/scripts/mount-dbfs.sh[start]
2022-04-19 15:32:16.732 :CLSDYNAM:1192507136: [dbfs_mount]{1:30281:17383}
[start]
mount-dbfs.sh mounting DBFS at /mnt/dbfs from database ggdg
2022-04-19 15:32:17.883 :CLSDYNAM:1192507136: [dbfs_mount]{1:30281:17383}
[start]
ORACLE_SID is
2022-04-19 15:32:17.883 :CLSDYNAM:1192507136: [dbfs_mount]{1:30281:17383}
[start]
No running ORACLE_SID available on this host, exiting
2022-04-19 15:32:17.883 : AGFW:1192507136: {1:30281:17383} Command:
start for
resource: dbfs_mount 1 1 completed with invalid status: 2
```

- Incorrect MOUNT_POINT

```
2022-04-19 16:45:14.534 : AGFW:1734321920: {1:30281:17604} dbfs_mount
1 1 state changed from: UNKNOWN to: STARTING
2022-04-19 16:45:14.535 :CLSDYNAM:1736423168: [dbfs_mount]{1:30281:17604}
[start]
Executing action script: /u01/oracle/scripts/mount-dbfs.sh[start]
2022-04-19 16:45:14.586 :CLSDYNAM:1736423168: [dbfs_mount]{1:30281:17604}
[start]
mount-dbfs.sh mounting DBFS at /mnt/dbfs from database ggdg
2022-04-19 16:45:15.638 :CLSDYNAM:1736423168: [dbfs_mount]{1:30281:17604}
[start]
ORACLE_SID is ggdg1
2022-04-19 16:45:15.738 :CLSDYNAM:1736423168: [dbfs_mount]{1:30281:17604}
[start]
spawning dbfs_client command using SID ggdg1
2022-04-19 16:45:20.745 :CLSDYNAM:1736423168: [dbfs_mount]{1:30281:17604}
```

```
[start]
fuse: bad mount point `/mnt/dbfs': No such file or directory
2022-04-19 16:45:21.747 :CLSDYNAM:1736423168: [dbfs_mount]{1:30281:17604}
[start]
Start - OFFLINE
2022-04-19 16:45:21.747 : AGFW:1736423168: {1:30281:17604} Command:
start for
resource: dbfs_mount 1 1 completed with status: FAIL
```

- Incorrect DBFS_USER or DBFS_PASSWD

```
2022-04-19 16:47:47.855 : AGFW:1384478464: {1:30281:17671} dbfs_mount
1 1 state changed from: UNKNOWN to: STARTING
2022-04-19 16:47:47.856 :CLSDYNAM:1386579712: [dbfs_mount]{1:30281:17671}
[start]
Executing action script: /u01/oracle/scripts/mount-dbfs.sh[start]
2022-04-19 16:47:47.908 :CLSDYNAM:1386579712: [dbfs_mount]{1:30281:17671}
[start]
mount-dbfs.sh mounting DBFS at /mnt/dbfs from database ggdgs
2022-04-19 16:47:48.959 :CLSDYNAM:1386579712: [dbfs_mount]{1:30281:17671}
[start]
ORACLE_SID is ggdg1
2022-04-19 16:47:49.010 :CLSDYNAM:1386579712: [dbfs_mount]{1:30281:17671}
[start]
spawning dbfs_client command using SID ggdg1
2022-04-19 16:47:55.118 :CLSDYNAM:1386579712: [dbfs_mount]{1:30281:17671}
[start]
Fail to connect to database server. Error: ORA-01017: invalid username/
password;
logon denied
2022-04-19 16:47:55.118 :CLSDYNAM:1386579712: [dbfs_mount]{1:30281:17671}
[start]
2022-04-19 16:47:56.219 :CLSDYNAM:1386579712: [dbfs_mount]{1:30281:17671}
[start]
Start - OFFLINE
2022-04-19 16:47:56.220 : AGFW:1386579712: {1:30281:17671} Command:
start for
resource: dbfs_mount 1 1 completed with status: FAIL
```

- Incorrect ORACLE_HOME

```
2022-04-19 16:50:38.952 : AGFW:567502592: {1:30281:17739} dbfs_mount
1 1 state changed from: UNKNOWN to: STARTING
2022-04-19 16:50:38.953 :CLSDYNAM:569603840: [dbfs_mount]{1:30281:17739}
[start]
Executing action script: /u01/oracle/scripts/mount-dbfs.sh[start]
2022-04-19 16:50:39.004 :CLSDYNAM:569603840: [dbfs_mount]{1:30281:17739}
[start]
mount-dbfs.sh mounting DBFS at /mnt/dbfs from database ggdgs
2022-04-19 16:50:39.004 :CLSDYNAM:569603840: [dbfs_mount]{1:30281:17739}
[start]
/u01/oracle/scripts/mount-dbfs.sh: line 136:
/u01/app/oracle/product/19.0.0/rdbms/bin/srvctl: No such file or
directory
2022-04-19 16:50:39.004 :CLSDYNAM:569603840: [dbfs_mount]{1:30281:17739}
[start]
```



```

/u01/oracle/scripts/mount-dbfs.sh: line 139:
/u01/app/oracle/product/19.0.0.0/rdbms/bin/srvctl: No such file or
directory
2022-04-19 16:50:39.004 :CLSDYNAM:569603840: [dbfs_mount]{1:30281:17739}
[start]
ORACLE_SID is
2022-04-19 16:50:39.004 :CLSDYNAM:569603840: [dbfs_mount]{1:30281:17739}
[start]
No running ORACLE_SID available on this host, exiting
2022-04-19 16:50:39.004 : AGFW:569603840: {1:30281:17739} Command:
start for
resource: dbfs_mount 1 1 completed with invalid status: 2

```

To resolve these configuration issues, set the correct parameter values in mount-dbfs.conf.

Problems with file locking on DBFS

If using Oracle Database 12c Release 2 (12.2) and the nolock DBFS mount option is not used, there can be problems with GoldenGate processes trying to lock checkpoint or trail files. The same problem will be encountered if using Oracle Database 11g Release 2 (11.2.0.4) or 12c Release 1 (12.1) with a patch for bug 22646150 applied. This patch changes how DBFS handles file locking to match Oracle Database 12c Release 2 (12.2). To add the nolock DBFS mount option, a patch for bug 27056711 must be applied to the database. If the patch for bug 22646150 has not been applied to the database, the patch for bug 27056711 and the nolock mount option is not required.

Below is an example of diagnosing a GoldenGate Microservices Architecture locking problem.

When starting a deployment with XAG, one or more processes may not start due to detecting a locking conflict on one or more files. This will often occur after a RAC node failover where the deployment did not get a chance to shut down cleanly.

When one of the deployment server processes fails to start (Administration Server, Performance Metrics Server, Distribution Server, Receiver Server, or Service Manager), check the log file for the particular server located in the deployment var/log directory.

For example, the log file /mnt/dbfs/goldengate/deployments/<INSTANCE_NAME>/var/log/pmsrvr.log shows the following error on startup:

```

2022-04-11T12:41:57.619-0700 ERROR| SecureStore failed on open after
retrying due to extended file lock. (main)
2022-04-11T12:41:57.619-0700 ERROR| SecureStore failed to close (28771).
(main)
2022-04-11T12:41:57.619-0700 INFO | Set network configuration. (main)
2022-04-11T12:41:57.619-0700 INFO | Asynchronous operations are enabled with
default
synchronous wait time of 30 seconds (main)
2022-04-11T12:41:57.619-0700 INFO | HttpServer configuration complete. (main)
2022-04-11T12:42:07.674-0700 ERROR| Unable to lock process file, Error is
[1454]
- OGG-01454 (main)
2022-04-11T12:42:07.675-0700 ERROR| Another Instance of PM Server is Already
Running
(main)

```

An Extract process will report start-up failures in the ER-events.log logfile located in the deployment log file directory.

For example, /mnt/dbfs/goldengate/deployments/<instance_name>/var/log/ER-events.log shows the following error:

```
2022-04-11T00:14:56.845-0700 ERROR OGG-01454 Oracle GoldenGate Capture for
Oracle, EXT1.prm: Unable to lock file
"/mnt/dbfs/goldengate/deployments/<instance_name>/var/run/EXT1.pce" (error
11, Resource
temporarily unavailable). Lock currently held by process id (PID) 237495.
2022-04-11T00:14:56.861-0700 ERROR OGG-01668 Oracle GoldenGate Capture
for Oracle,
EXT1.prm: PROCESS ABENDING.
```

Next, check to ensure the process failing to start up is not running on any of the RAC nodes.

Example:

```
$ ps -ef|grep EXT1|grep -v grep
```

Once it has been determined that the process is not running, the deployment must be shutdown cleanly, the file system unmounted, and the correct DBFS patch applied.

Example:

```
$ agctl stop goldengate <INSTANCE_NAME>
$ crsctl stop resource dbfs_mount
```

Check the DBFS mount options:

```
$ ps -ef|grep dbfs_client

oracle  204017      1  0 14:37 ?
          00:00:00 /u01/app/oracle/product/19.1.0.0/dbhome_1/bin/dbfs_client
dbfs@dbfs.local
-o allow_other,failover,direct_io /mnt/dbfs
```

It is clear the nolock mount option was not used, which leads to the locking errors.

Use the guidelines above to determine if a DBFS patch is required. After which, add the nolock mount option to the mount-dbfs.conf file on all Oracle RAC nodes that are part of the deployment.

Example:

```
MOUNT_OPTIONS=allow_other,direct_io,failover,nolock
```

Finally, restart the deployment:

```
$ agctl start goldengate <INSTANCE_NAME>
```

Critical Extract/Replicat process preventing GoldenGate start-up

When an Extract or Replicat process has been set as critical to the health of the deployment in the Administration Server, if the process abends, the Service Manager and deployment will be stopped.

When restarting GoldenGate using XAG (`agctl start goldengate`) it will fail with the following error:

```
$ agctl start goldengate GGNORTH --node dc1north01
CRS-2672: Attempting to start 'xag.GGNORTH.goldengate' on 'dc1north01'
CRS-2674: Start of 'xag.GGNORTH.goldengate' on 'dc1north01' failed
CRS-2679: Attempting to clean 'xag.GGNORTH.goldengate' on 'dc1north01'
CRS-2681: Clean of 'xag.GGNORTH.goldengate' on 'dc1north01' succeeded
CRS-2632: There are no more servers to try to place resource
'xag.GGNORTH.goldengate' on
that would satisfy its placement policy
CRS-4000: Command Start failed, or completed with errors.
```

The CRS trace file (`crsd_scriptagent_oracle.trc`) does not provide enough information to determine the reason behind the startup failure.

Example:

```
2022-06-30 20:20:28.675 : AGFW:521639680: [ INFO] {1:29336:52291}
Command: start for resource: xag.GGNORTH.goldengate 1 1 completed with
status: SUCCESS
2022-06-30 20:20:28.675 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52291}
[check] Executing action script: /u01/oracle/XAG/bin/aggoldengatescaas[check]
2022-06-30 20:20:28.677 : AGFW:517437184: [ INFO] {1:29336:52291}
Agent sending
reply for: RESOURCE_START[xag.GGNORTH.goldengate 1 1] ID 4098:178187
2022-06-30 20:20:28.827 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52291}
[check] GG agent running command 'check' on xag.GGNORTH.goldengate
2022-06-30 20:20:28.978 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52291}
[check] execute XAGTask HealthCheck
2022-06-30 20:20:29.779 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52291}
[check] XAGTask retcode = 3
2022-06-30 20:20:29.780 : AGFW:517437184: [ INFO] {1:29336:52291}
xag.GGNORTH.goldengate 1 1 state changed from: STARTING to: FAILED
```

Starting with Oracle GoldenGate version 21.11, when using XAG, additional GoldenGate status information is logged into the Service Manager `$OGG_VAR_HOME/log/XAGTask.log` file.

For earlier releases of Oracle GoldenGate, enable the CRS debug logging for XAG using the following commands.

```
$ crsctl set log res xag.GGNORTH.goldengate=5
$ agctl start goldengate GGNORTH --node dc1north01
$ crsctl set log res xag.GGNORTH.goldengate=1
```

Check the Service Manager `$OGG_VAR_HOME/log/XAGTask.log` or the CRS trace file (`crsd_scriptagent_oracle.trc`) for more information on the failure.

Example:

```

2022-06-30 20:24:36.389 :CLSDYNAM:521639680:
[xag.GGNORTH.goldengate]{1:29336:52435} [check] Deployment: GGNORTH
2022-06-30 20:24:36.389 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52435}
[check] Service: adminsrvr
2022-06-30 20:24:36.389 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52435}
[check] Status: running
2022-06-30 20:24:36.389 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52435}
[check] Started: 2022-07-01T03:24:35.911Z
2022-06-30 20:24:36.389 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52435}
[check] Health: Failed
2022-06-30 20:24:36.389 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52435}
[check] Resources: Name Type
Status Health
2022-06-30 20:24:36.389 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52435}
[check] -----
-----
2022-06-30 20:24:36.389 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52435}
[check] REP1 replicat
abended Failed
2022-06-30 20:24:36.389 :CLSDYNAM:521639680: [xag.GGNORTH.goldengate]
{1:29336:52435}
[check] HealthCheck failed

```

When Extract or Replicat processes are not set to critical, they will not appear in the CRS trace file. Because Replicat REP1 is shown as failed, this indicates that REP1 is set to critical and is preventing GoldenGate from starting.

To disable the critical setting for Replicat, the GoldenGate Service Manager and deployment must be started manually.

Example:

```

$ export OGG_ETC_HOME=/mnt/acfs/goldengate/deployments/ggsm01/etc
$ export OGG_VAR_HOME=/mnt/acfs/goldengate/deployments/ggsm01/var
$ export OGG_HOME=/u01/oracle/goldengate/ggMS_21c
$ $OGG_HOME/bin/ServiceManager --xagEnabled

```

Using the Administration Server GUI, select the Replicat or Extract details, and unset the critical flag.

Once the critical setting has been disabled, XAG can be used to start and stop Oracle GoldenGate Microservices.

Example:

```
$ agctl start goldengate GGNORTH --node dc1north01  
$ agctl status goldengate GGNORTH
```

Part VI

Data Corruption Prevention, Detection and Repair

- [Data Corruption Prevention, Detection, and Repair](#)

Data Corruption Prevention, Detection, and Repair

Data corruption, defined as the unintentional alteration of data caused by hardware, software, or human errors, poses a critical threat to the integrity of Oracle Database environments. To combat this pervasive concern, Oracle provides robust mechanisms built on three fundamental pillars: Prevention, Detection, and Repair.

Data Corruption Types

Summary of Data Corruption Types

Type	Typical Source/Cause	Detected by	Symptom
Physical corruption	Hardware/storage	Oracle block reads and meta data checks and checksum	Read errors, corrupted structure
Logical corruption	Software/memory	Oracle consistency checks including checking internal data structures	Data exists, but is inconsistent/logically invalid
Lost write corruption	Storage, I/O path	Data Guard real time apply or recovery and Oracle sophisticated block versioning checks	The block version is older than the redo expectations

Physical Block Corruption

These errors occur when a database block's structure is physically damaged or rendered unreadable. Such corruption is typically a result of hardware failures (for example: disk errors, controller faults, faulty cables).

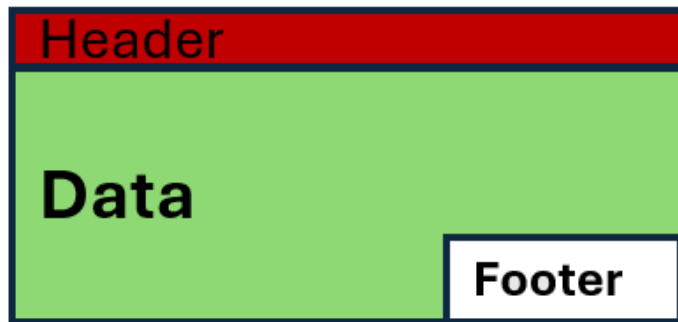
Examples:

- Incorrect or missing block header/footer
- Mismatch in data checksums or internal "magic numbers"
- Blocks containing entirely zeros or random, unrecognizable data patterns

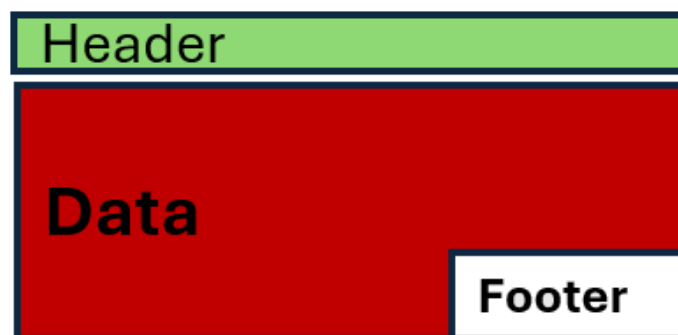
Characteristics:

- Detected by Oracle when attempting to read the corrupted block from disk
- Manifests as severe errors such as ORA-01578: ORACLE data block corrupted
- Can be repaired using block media recovery or data restoration from a valid backup

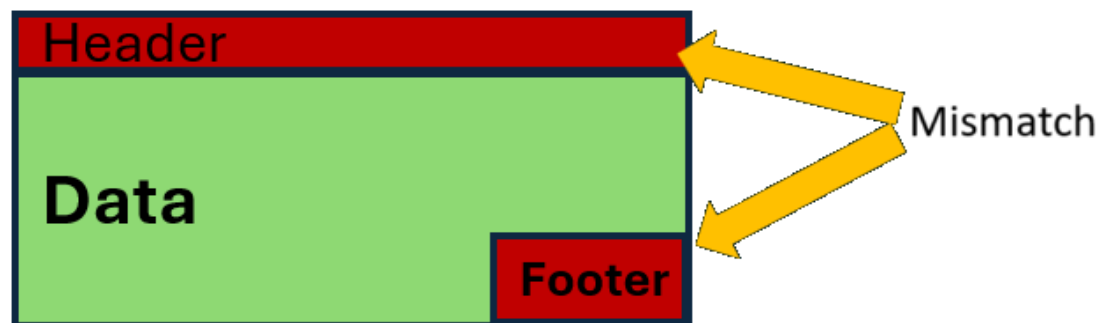
Invalid Header



Invalid Checksum



Fractured - Incomplete



Logical Block Corruption

In contrast to physical corruption, logical corruption arises when a block's physical structure is intact and readable, but its internal contents are inconsistent with Oracle's expected logical rules or metadata. These are commonly indicative of software bugs or memory-related issues. Logical block corruptions can be **intra-block** logical corruptions where the logical corruption is self-contained in single data block, and **inter-block** logical corruptions where data inconsistencies between 2 or more independent data blocks.

Examples:

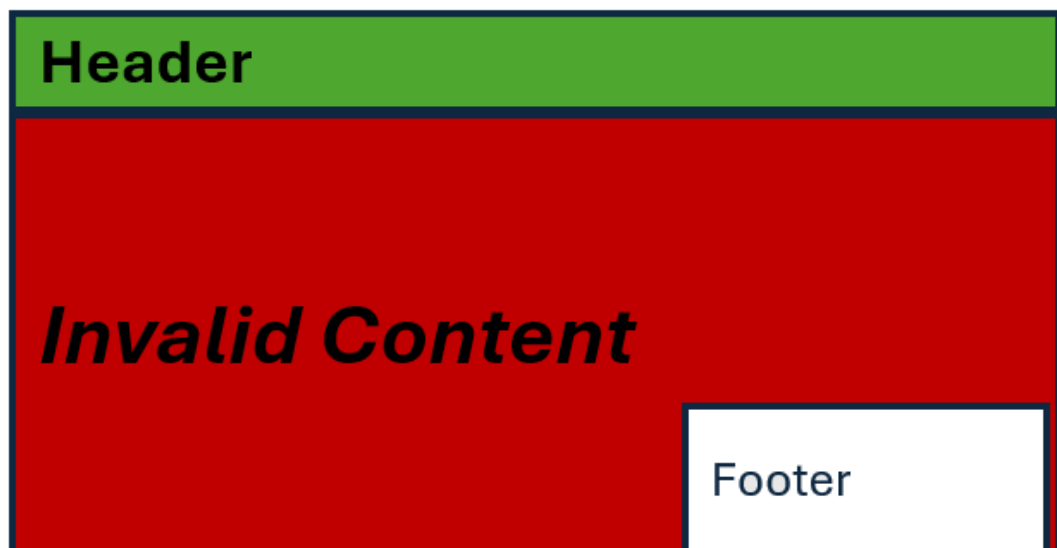
- Index entries referencing non-existent rows is an example of inter-block logical corruption.

- Table rows exhibiting inconsistencies with their associated metadata within the same data block is an example of intra-block logical corruption.
- Incorrect internal pointers, such as those within row piece chains can be intra-block or inter-block logical corruption depending on how many data blocks are involved.

Characteristics:

- The block is physically readable, and its checksums and structure are typically valid.
- Detection requires logical consistency checks, often performed with `RMAN VALIDATE CHECK LOGICAL`, SQL command `ANALYZE ... VALIDATE STRUCTURE`, or exposed during application queries.
- In some cases, repair is possible using block media recovery or without full restoration from backup, for example, by rebuilding affected indexes or using `DBMS_REPAIR` to work around the corruption so the remaining data can be accessed.

Corrupt block with valid checksum

**Lost Write Corruption**

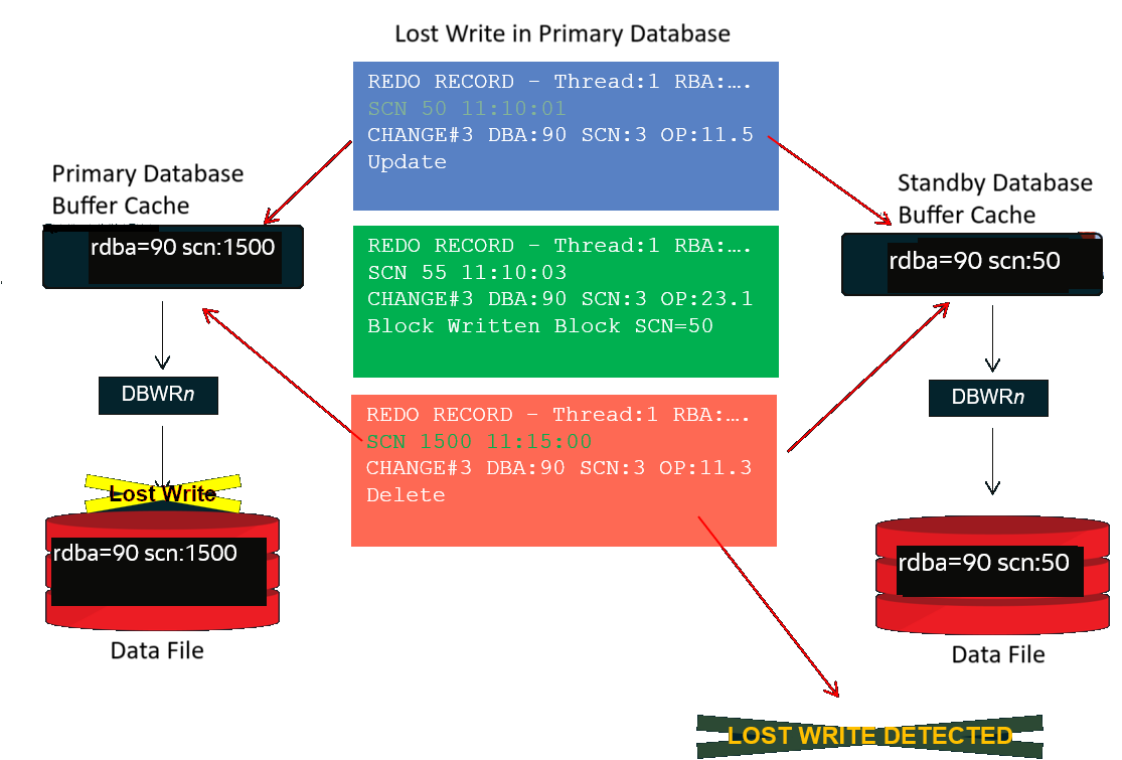
A lost write corruption occurs when a database block modification is acknowledged as successfully written by the storage sub-system, but the actual data is never persistently stored on disk or I/O being "lost". This results in the on-disk block image being older than the database expects, creating a crucial consistency mismatch.

Examples:

- A defect in the I/O sub-system or system firmware resulting in false acknowledged I/Os
- Following a system crash or any hardware failure, a data block on disk does not reflect recently committed changes that were believed to be durable.
- The redo log contains entries expecting a newer block version than what is physically present on disk.

Characteristics:

- This is a data consistency issue, where the data block is stale or older version than expected.
- Primarily detected by advanced features like Oracle Data Guard or recovery using advanced metrics found in block read redo and comparing with data block metadata
- Leads to data divergence, where the primary and standby databases become out of sync, or where the redo stream does not match the actual state of data files



Summary of Oracle Data Corruption Resiliency Capabilities

Proactive prevention, vigilant detection, and proper use of Oracle's built-in repair features form a comprehensive defense against data corruption in Oracle databases.

Pillar	Features, Tools, and Best Practices	Purpose
Prevention	Software Update Practices, Testing, Use Recommended Database Parameters Oracle Active Data Guard, Exadata, Recovery Appliance For details see Preventing Data Corruption	Avoid corruption from happening or avoid corruption from impacting your application

Pillar	Features, Tools, and Best Practices	Purpose
Detection	Recommended Database Parameters, Database and System Alerting, RMAN Oracle Active Data Guard, Recovery Appliance, Exadata For details see Detecting Data Corruption	Find corruption early, notify, and take action
Repair	Oracle Active Data Guard, or GoldenGate, ASM and Exadata, RMAN For details see Repairing Data Corruption	Minimize impact; restore data and automatically repair (that is, Automatic Block Repair), aligning the solution with the lowest downtime and data loss

Preventing Data Corruption

Prevention aims to proactively stop corruption before it can occur.

Best Practices and Oracle Features

- **Maintain up-to-date software**
 - Regularly apply Oracle Release Updates (RUs) and relevant patches. These updates frequently include prioritized corruption prevention, detection, and repair fixes. Customers gain these benefits by adhering to regular software updates and applying critical fixes detailed in [Exadata Critical Issues \(KB623062\)](#) and [Oracle Database 19c and Oracle AI Database 26ai Important Recommended One-off Patches \(KB188772\)](#).
 - Leverage Exachk and Orachk tools for comprehensive software health guidance, including recommendations for updates that prevent data corruptions.
- **Use dedicated test systems**
 - Thoroughly evaluate all software, hardware, database, or application changes in a test environment before deploying to production. This practice helps detect potential new data corruptions and verifies the effectiveness of existing corruption fixes.
 - Employing a representative workload that accurately mimics production is crucial, because some corruptions manifest only under specific or extreme application conditions.
- **Employ reliable hardware**
 - Implement enterprise-grade storage solutions equipped with advanced error-correcting features.
 - Exadata and Oracle ASM offer numerous integrated hardware and software features specifically designed to prevent data corruption from impacting applications and databases.
- **Implement storage redundancy**
 - Use RAID configurations and redundant SAN/NAS solutions for physical storage resilience.
 - Oracle ASM and normal or high redundancy protects from data corruptions. If a block on one ASM disk is physically corrupted (for example, due to hardware failure), ASM can:
 - * Serve an uncorrupted mirrored copy from another disk, shielding the database from encountering corrupted data.

- * Automatically recover the corrupted copy by reading the valid mirrored copy and rewriting it to replace the corrupt one if possible.
 - Exadata inherently provides built-in redundancy, along with advanced corruption prevention and automated repair capabilities. Exadata does regular data validation (scrubbing) to detect and repair corrupted blocks using mirrored copies.
- **Run regular backups**
 - Schedule frequent, automated RMAN backups and ensure continuous archiving of redo logs.
 - Zero Data Loss Recovery Appliance (ZDLRA) offers automatic backup validation and leverages Exadata's corruption prevention and repair benefits.

Key Database Configuration Parameters for Data Corruptions Protection

- **DB_BLOCK_CHECKSUM**
 - Purpose: Detects physical block corruptions by enabling checksumming for every database and redo block written to disk.
 - Recommendation: TYPICAL or higher for both primary and standby databases.
 - Default: TYPICAL
- **DB_BLOCK_CHECKING:**
 - Purpose: Detects logical corruptions and prevents them from being written to disk.
 - Recommendation: MEDIUM or higher for primary or standby databases.
 - Default: FALSE
 - **Note:** Performance overhead can vary. A common compromise is to enable DB_BLOCK_CHECKING on the standby database to ensure logical corruptions are not propagated.
- **DB_LOST_WRITE_PROTECT**
 - Purpose: Detects lost writes on standby databases and actively prevents/detects lost writes on primary databases.
 - Recommendation: AUTO for both primary and standby databases.
 - Default: TYPICAL, NONE, or AUTO depending on the database release. Starting in Oracle Database 19c (19.26), the default changed to AUTO for Data Guard configuration
 - While TYPICAL provides strong lost-write protection, it can increase redo generation on the primary database and add redo apply overhead on the standby database, including additional CPU and physical I/O. For some workloads, this overhead may contribute to increased standby apply lag.
 - With AUTO, lost-write protection is enabled when a physical standby database is current and real-time apply is present. If standby apply lag becomes too high, lost-write checks are temporarily suspended to help preserve standby readiness and role-transition performance.
 - Starting with Oracle Database 19c, automatic block repair of lost writes using a current block from either primary or standby can occur.

Oracle Integrity Tools and Features

- **RMAN backups:** Ensure the integrity of your backups and facilitate recovery.

- **Avoid unreliable storage:** Never place Oracle database files on improperly configured NFS mounts or other unreliable storage devices.
- **Use Oracle Active Data Guard:** When configured with a physical standby database and real-time apply, Oracle Active Data Guard automatically detects all types of data corruption. It also provides automatic block repair for physical data block corruptions and some lost write corruptions on both primary and standby databases, preventing application impact. Contact Oracle Support to investigate the root cause of any detected corruption.
- **Enable Flashback Database:** Flashback database logs can be leveraged for automatic block repair and RMAN block repair of data corruptions.
- **Use ASM redundancy and Exadata** for additional corruption prevention, detection, and repair.

Detecting Data Corruption

Corruption detection focuses on identifying corruption as soon as it occurs or before it can cause widespread damage.

The following are best practices and Oracle features to detect data corruption.

- **Configure database parameters**
 - Ensure that DB_BLOCK_CHECKSUM, DB_BLOCK_CHECKING, and DB_LOST_WRITE_PROTECT are configured for proactive block and write verification. Recommendations for these parameters are described in detail in Preventing Data Corruption.
- **Monitor database and system alerts**
 - Regularly monitor Oracle Database, Oracle Clusterware, or Exadata alert logs and system logs for any corruption-related errors or warnings. Exadata provides additional specialized alerts if corruptions are detected within its storage layer.
- **Query database views**
 - Refer to V\$DATABASE_BLOCK_CORRUPTION for a comprehensive list of known corrupted blocks within the database.
- **Leverage RMAN validation**
 - Perform RMAN backups, which include checks for physical corruption by default.
 - Use BACKUP VALIDATE or VALIDATE DATABASE commands to periodically check for corruption without creating a full backup.
 - Use BACKUP VALIDATE CHECK LOGICAL DATABASE to detect logical corruptions while performing your backups.
 - Refer to [RMAN Continuous Block Corruption Checker \(KB145527\)](#).
- **Use Oracle Active Data Guard and recommended database parameters for data corruptions protection**
 - With Active Data Guard and a physical standby database operating with real-time apply, Data Guard automatically detects all types of data corruption from physical, logical and lost write data corruptions.
 - It further facilitates auto block repair for physical data block corruptions and some lost write corruptions on either primary or standby databases, effectively preventing corruption from impacting applications.
 - Always contact Oracle Support to understand the root cause of any data corruption.
- **Use Zero Data Loss Recovery Appliance (ZDLRA)**

- The Recovery Appliance provides automatic and continuous backup validation, detects and alerts you about backup integrity issues, and offers additional corruption prevention and repair benefits.
- **Detect LOB segment corruption**
 - Some LOB segment corruptions may not be detected by standard database configuration parameters. A new PL/SQL detection script is available for this purpose.
 - Refer to [Handling LOB Segment Corruptions \(KB176066\)](#).
- **Run ANALYZE ... VALIDATE STRUCTURE CASCADE ONLINE**
 - The ANALYZE command can be used to check for structural and referential inconsistencies in tables or indexes.
 - The specific clause for checking for inconsistencies is VALIDATE STRUCTURE CASCADE ONLINE.
 - These types of inter-logical block corruptions may not be detected by recommended database parameters.
- **Use ASM redundancy and Exadata** for additional corruption prevention, detection and repair.
 - Any detected corruptions are in relevant log files, and both ASM and Exadata automatically recover the corrupted copy by reading the valid mirrored copy and rewriting it to replace the corrupt one (if possible). This may not work for logical corruption or even all mirrored copies that have been corrupted or compromised.
- **Use Oracle Enterprise Manager (OEM)**
 - Provides automated monitoring capabilities for corruption events and overall backup health

Repairing Data Corruption

Repair strategies focus on restoring or correcting corrupted data after it has been identified, minimizing impact or working around the corrupted data, and attempting to re-establish data integrity.

The following are best practices and Oracle features to repair data corruption.

- **RMAN block media recovery**
 - This highly efficient method restores only the corrupted blocks from backup, significantly minimizing recovery time and application downtime.
 - Primarily effective for physical data corruption but can help in some logical data block and lost block corruptions when following our recommended practices
 - See [References](#) for articles and documentation topics.
- **RMAN full/incremental restore and recover**
 - Provides comprehensive recovery options to restore entire data files, tablespaces, or the entire database to a consistent, uncorrupted state
- **Oracle Active Data Guard automatic repair and failover**
 - **Automatic Block Repair:** Oracle Active Data Guard can automatically repair physical data corruptions by fetching a good block from its synchronized standby database (or vice versa).

- * Starting with Oracle Database 19c (19.27) and Oracle AI Database 26ai, detected standby lost write corruptions use the Automatic Block Repair feature for transparent repair.
 - * For physical block corruptions (such as ORA-1578), primary database auto-block repairs (that is, a corrupt block is seen on the primary database and Oracle internally fixes it by fetching good block from the standby database) requires standby container database with a minimum apply lag to be open read-only regardless of the standby PDB state. For standby auto-BMR (that is, a corrupt block is seen on the standby database and Oracle internally fixes it by fetching good block from the primary) requires at least a mounted or open standby with a minimum apply lag.
 - * If standby lost write corruption is detected on the standby database, the standby lost write results in an ORA-753 error. Oracle automatically marks this as a soft corrupted data block which triggers Automatic Block Repair. If the block cannot be repaired, it remains soft corrupted, and once the database is open, reads on that block will raise ORA-01578.
 - * For Oracle Cloud Data Guard environments and for primary database lost write on index block, Automatic Block Repair addresses this scenario automatically without the need to create the index.
- **Fast-Start Failover (FSFO) with lost write protection:** To enable automatic failover when a standby detects a lost write corruption, configure Data Guard Broker with FSFO enabled, set `DB_LOST_WRITE_PROTECT` to `TYPICAL` or higher, and set Data Guard broker property `PrimaryLostWriteAction`.
- Options for Data Guard broker property `PrimaryLostWriteAction`:
- * **FAILOVER:** If FSFO is enabled in Maximum Availability or Maximum Performance mode, and the lag is either zero with Maximum Availability or within `FastStartFailoverLagLimit` for Maximum Performance, the observer initiates a failover.
 - * **FORCEFAILOVER:** If FSFO is enabled in Maximum Performance, Maximum Availability, or Maximum Protection mode), the observer initiates a failover, which may result in data loss.
- **Flashback Database**
 - Enables rapid restoration of the database to a point in time before the corruption, significantly reducing recovery time objectives (RTO)
 - Flashback logs can also contribute to RMAN block media recovery.
 - **Targeted table/partition recovery**
 - For specific object-level corruption, RMAN table or tablespace point-in-time recovery, or Data Pump export/import (to extract and load healthy data) can be used.
 - **Index rebuild**
 - If an index block is corrupted, rebuilding the affected index can often resolve the issue without requiring full database or tablespace recovery.
 - **LOB segment corruption repair**
 - New repair mechanisms have been developed specifically for LOB corruptions. Refer to [Handling LOB Segment Corruptions \(KB176066\)](#) for detailed instructions.
 - **Failover to standby database or GoldenGate replica (Lowest RTO/RPO)**
 - To achieve the lowest possible Recovery Time Objective (RTO) and Recovery Point Objective (RPO) when the primary database is experiencing corruptions, initiate a

failover to a healthy Data Guard standby database or a GoldenGate replica. This ensures business continuity by transitioning to a database without any data corruptions.

- **DBMS_REPAIR package**
 - The DBMS_REPAIR package in Oracle is designed to identify and address corruptions in database tables and indexes, especially when physical or logical corruption prevents normal queries or operations from completing. It does not physically fix the corrupt blocks, but instead helps you to isolate and work around them, so that the rest of your data can be accessed or managed.

References

- [Best Practices for Corruption Detection, Prevention, and Automatic Repair - in a Data Guard Configuration \(KB143286\)](#)
- [DBMS_DICTIONARY_CHECK: PL/SQL package procedure that helps you identify Oracle Database dictionary inconsistencies \(KI37544\)](#)
- [Backup and Recovery User's Guide: Performing Block Media Recovery](#)
- [Physical and Logical Block Corruptions. All you wanted to know about it. \(KB135785\)](#)
- [Primary Note for Handling Oracle Database Corruption Issues \(KB139082\)](#)
- [Resolving Logical Block Corruption Errors in a Physical Standby Database \(KB125725\)](#)
- [Resolving ORA-00752 or ORA-600 \[3020\] During Standby Recovery \(KB138737\)](#)
- [Identify the Corruption Extension for Block Corruption, Table/Index Inconsistency, Data Dictionary and Lost Writes \(KB143189\)](#)
- [Handling LOB Segment Corruptions \(KB176066\)](#)
- [How to use OERR to Find the Cause and Resolution for Oracle Errors \(KB142372\)](#)
- [OERR: ORA-8102 "index key not found, obj# %s, file %s, block %s \(%s\)" \(KB860829\)](#)
- [ORA-600 \[kddummy_blkchk\] \(KB185094\)](#)
- [ORA-600 \[kdBlkCheckError\] \(KB185177\)](#)
- [How to clear a block corruption in a TEMPORARY segment \(KB142373\)](#)

Part VII

Oracle Database Cloud Best Practices

- [Overview of Oracle AI Database Cloud Best Practices](#)
- [Oracle Maximum Availability Architecture and Oracle Autonomous Database](#)
- [Oracle Maximum Availability Architecture in Oracle Exadata Cloud Systems](#)
- [Oracle Maximum Availability Architecture for Multicloud](#)
- [Oracle Data Guard Hybrid Cloud Configuration](#)
- [Migration to Oracle Cloud Best Practices](#)

Overview of Oracle AI Database Cloud Best Practices

Oracle Cloud and MAA collaborate closely to enable customer success using the various Oracle AI Database Cloud services.

This partnership ensures

- Databases, Grid Infrastructure, Exadata, and infrastructure are deployed and configured with MAA best practices in [Oracle Cloud Infrastructure](#), including public cloud, [OCI Dedicated Region](#), [Oracle Database@Azure](#), [Oracle Database@Google Cloud](#), [Oracle Database@AWS](#), and [Exadata Cloud@Customer](#).
- Cloud life cycle practices such as patching, backup and restore, disaster recovery, and pluggable database management incorporate MAA optimizations as new features and capabilities are introduced into the Oracle Cloud.
- Oracle Cloud owns and manages network, system, and Exadata infrastructure using MAA practices and optimizations.
The only exception is in Oracle Multicloud where you can choose to use AWS, Azure, or Google's network between application and database tiers, or network between database tiers for Oracle Data Guard and Oracle GoldenGate architectures. Oracle typically recommends using Oracle "OCI" network between Data Guard configurations to reduce additional network hops.
- Oracle high availability and disaster recovery (HA/DR) solutions meet our Enterprise customer standards for Gold and Platinum MAA solutions.

For the latest details, see [Oracle Cloud Maximum Availability Architecture](#).

MAA Cloud Certification Matrix

The following table indicates which MAA Reference Architectures are certified on each Oracle Database cloud service.

Oracle AI Database Cloud Service	MAA Bronze	MAA Silver	MAA Gold - Cross-AD	MAA Gold - Cross-Region	MAA Platinum	MAA Diamond
Oracle ExaDB-D	Certified	Certified	Certified	Certified	Certified	Certified
Oracle ExaDB-C@C	Certified	Certified	Certified	Certified	Certified	Planned
Oracle ExaDB-XS	Certified	Certified	Certified	Certified	Planned	Planned
Oracle ADB-S	Certified	Certified	Certified	Planned	Planned	Planned
Oracle ADB-D	Certified	Certified	Certified	Certified	Certified with Oracle Active Data Guard on Oracle AI Database 26ai	Planned

Oracle AI Database Cloud Service	MAA Bronze	MAA Silver	MAA Gold - Cross-AD	MAA Gold - Cross-Region	MAA Platinum	MAA Diamond
Oracle BaseDB without Oracle RAC	Reevaluation Planned	Reevaluation Planned	Reevaluation Planned	Reevaluation Planned	N/A	N/A
Oracle DB@C-C with Oracle RAC	Certified	Certified	Certified	Planned	N/A	N/A

For multicloud, see [MAA Multicloud Certification Matrix](#).

MAA Validated Solutions

The following topics outline MAA validated solutions and guidance for Oracle Database Cloud services.

Oracle Base Database Service (BaseDB)

MAA Tier	Validated Solution
Bronze	BaseDB Single Instance - Use Oracle AI Database cloud automation to configure the network and create DB Systems and databases - Use Oracle AI Database cloud automation for system and database life cycle operations, including software updates, upgrades, monitoring, alerting, and database administration and management. - Use a cloud-managed backup service. Recommended: Use (Zero Data Loss) Autonomous Recovery Service and real-time redo to reduce data loss in case of disasters - Customer responsibilities: - Create test systems to evaluate application, configuration, or software changes - Execute System and database sizing, resource management, and monitoring
Silver	Option 1: BaseDB 2-Node Oracle RAC (where available) Bronze, plus the following: - Use Oracle AI Database cloud automation to create multi-node Oracle RAC database systems on Oracle Cloud Infrastructure (OCI) - Customer responsibilities: - Maximize application failover and uptime with Achieving Continuous Availability For Your Applications Option 2: BaseDB Single Instance with Data Guard Bronze, plus the following: - Use Oracle AI Database cloud automation to create single instance database systems on Oracle Cloud Infrastructure (OCI) - Use cloud automation to create and manage a standby database. The location can be in another Availability Domain, Availability Zone, or Region for better fault isolation. - Customer responsibilities: - Maximize application failover and uptime with Achieving Continuous Availability For Your Applications Bronze, plus the following: - Customer responsibilities: (Highly Recommended) Set up Data Guard Fast Start Failover (automatic failover) with MAA practices to bound RTO after database, cluster, potentially Availability Domain or regional failure - Tune Data Guard if lagging (see Overview of Oracle Data Guard Tuning and Troubleshooting) - Extend application failover to standby (Achieving Continuous Availability For Your Applications) or use Full Stack Disaster Recovery Service. - Maximize application failover and uptime with Achieving Continuous Availability For Your Applications

MAA Tier	Validated Solution
Gold	Base DB 2-Nodes Oracle RAC (where available) with Oracle Active Data Guard Silver, plus the following: • Recommended: primary and standby database systems are symmetric in shape and system resources. • Use cloud automation to create and manage a standby database. The location can be in another Availability Domain, Availability Zone, or Region for better fault isolation. • Customer responsibilities: – (Highly Recommended) Set up Data Guard Fast Start Failover (automatic failover) with MAA practices to bound RTO after database, cluster, potentially Availability Domain or regional failure – Tune Data Guard if lagging (see Overview of Oracle Data Guard Tuning and Troubleshooting) – Extend application failover to standby (Achieving Continuous Availability For Your Applications) or use Full Stack Disaster Recovery Service.
Platinum	N/A
Diamond	N/A

Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D)

MAA Tier	Validated Solutions
Bronze	N/A
Silver	ExaDB-D (Default) • Use Oracle AI Database cloud automation to create Exadata infrastructure, VM cluster, and Oracle RAC databases. • Use Oracle AI Database cloud automation for system and database life cycle operations, including software updates, upgrades, monitoring, service events, health alerts, and database administration and management. • Use a cloud-managed backup service. Recommended: Use (Zero Data Loss) Autonomous Recovery Service and real-time redo to reduce data loss in case of disasters • Customer responsibilities: – Create test systems to evaluate application, configuration, or software changes. – Execute system and database sizing, resource management and monitoring, and reviewing exachk (holistic health) – Maximize application failover and uptime with Achieving Continuous Availability For Your Applications
Gold	ExaDB-XS with Oracle Active Data Guard Silver, plus the following: • Recommended: primary and standby database systems are symmetric in shape and system resources. • Use cloud automation to create and manage a standby database. The location can be in another Availability Domain, Availability Zone, or Region for better fault isolation. • Customer responsibilities: – Set up Data Guard Fast Start Failover (automatic failover) with MAA practices to bound RTO after database, cluster, potentially Availability Domain or regional failure – Tune Data Guard if lagging (see Overview of Oracle Data Guard Tuning and Troubleshooting) – Extend application failover to standby (Achieving Continuous Availability For Your Applications) or use Full Stack Disaster Recovery Service.

MAA Tier	Validated Solutions
Platinum	Option 1: Upgrade MAA Gold to Oracle AI Database 26ai Option 2: ExaDB-D with Oracle Active Data Guard and Oracle GoldenGate Gold, plus the following customer responsibilities: - Follow Oracle GoldenGate and MAA Platinum architectural solution for Oracle GoldenGate management and tuning (MAA Platinum Reference Architecture Overview). - Extend application failover to Oracle GoldenGate replica with Achieving Continuous Availability For Your Applications - Optionally use Global Data Services for smart workload management between GoldenGate replicas and standby databases
Diamond	Platinum Option 2 with Oracle GoldenGate, and: - Upgrade to Oracle AI Database 26ai - Upgrade to Oracle GoldenGate 26ai

Oracle Exadata Database Service on Exascale Infrastructure (ExaDB-XS)

MAA Tier	Validated Solution
Bronze	N/A
Silver	ExaDB-XS (Default) - Use Oracle AI Database cloud automation to create Exadata infrastructure, VM cluster, and Oracle RAC databases. - Use Oracle AI Database cloud automation for system and database life cycle operations, including software updates, upgrades, monitoring, service events, and health alerts, and database administration and management. - Use a cloud-managed backup service. Recommended: Use (Zero Data Loss) Autonomous Recovery Service and real-time redo to reduce data loss in case of disasters. - Customer responsibilities: - Create test systems to evaluate application, configuration, or software changes. - Execute system and database sizing, resource management and monitoring, and reviewing exachk (holistic health) - Maximize application failover and uptime with Achieving Continuous Availability For Your Applications
Gold	ExaDB-XS with Oracle Active Data Guard Silver, plus the following: - Recommended: primary and standby database systems are symmetric in shape and system resources. - Use cloud automation to create and manage a standby database. The location can be in another Availability Domain, Availability Zone, or Region for better fault isolation. - Customer responsibilities: - Set up Data Guard Fast Start Failover (automatic failover) with MAA practices to bound RTO after database, cluster, potentially Availability Domain or regional failure - Tune Data Guard if lagging (see Overview of Oracle Data Guard Tuning and Troubleshooting) - Extend application failover to standby (Achieving Continuous Availability For Your Applications) or use Full Stack Disaster Recovery Service.
Platinum	Planned
Diamond	Planned

Oracle Exadata Database Service on Cloud@Customer (ExaDB-C@C)

MAA Tier	Validated Solutions
Bronze	N/A

MAA Tier	Validated Solutions
Silver	ExaDB-C@C (Default) - Use Oracle AI Database cloud automation to create Exadata infrastructure, VM cluster, and Oracle RAC databases. - Use Oracle AI Database cloud automation for system and database life cycle operations, including software updates, upgrades, monitoring, service events, and health alerts, and database administration and management. - Use a cloud-managed backup service. Recommended: Use Zero Data Loss Recovery Server and real-time redo to reduce data loss in case of disasters. - Customer responsibilities: - Create test systems to evaluate application, configuration, or software changes. - Execute system and database sizing, resource management and monitoring, and reviewing exachk (holistic health) - Maximize application failover and uptime with Achieving Continuous Availability For Your Applications
Gold	ExaDB-C@C with Oracle Active Data Guard Silver, plus the following: - Recommended: primary and standby database systems are symmetric in shape and system resources. - Use cloud automation to create and manage a standby database. The location can be in another Availability Domain, Availability Zone, or Region for better fault isolation. - Customer responsibilities: - Set up Data Guard Fast Start Failover (automatic failover) with MAA practices to bound RTO after database, cluster, potentially Availability Domain or regional failure - Tune Data Guard if lagging (see Overview of Oracle Data Guard Tuning and Troubleshooting) - Extend application failover to standby (Achieving Continuous Availability For Your Applications) or use Full Stack Disaster Recovery Service.
Platinum	Option 1: Upgrade MAA Gold with Oracle AI Database 26ai Option 2: ExaDB-C@C with Oracle Active Data Guard and Oracle GoldenGate Gold, plus the following: - Extend application failover to GoldenGate replica (Achieving Continuous Availability For Your Applications) - Optionally use Global Data Services for smart workload management between GoldenGate replicas and standby databases - Customer's responsibilities: - Follow Oracle GoldenGate and MAA Platinum architectural solution for Oracle GoldenGate management and tuning (MAA Platinum Reference Architecture Overview).
Diamond	Planned

Autonomous Database Serverless (ADB-S)

MAA Tier	Validated Solutions
Bronze	N/A
Silver	ADB-S (Default) - Use Oracle AI Database cloud automation life cycle operations including software updates, upgrades, monitoring, and database administration and management. - Auto scaling CPU and storage resizing is possible - Customer Responsibilities: - Create test systems to evaluate application changes - Maximize application failover and uptime with Achieving Continuous Availability For Your Applications - Update compute and storage limits when required (See Update Compute and storage limits)

MAA Tier	Validated Solutions
Gold	ADB-S with Autonomous Data Guard Silver, plus the following: • Enable Autonomous Data Guard, choose automatic failover, and data loss tolerance to bound and meet RTO and RPO SLAs. The location can be in another Availability Domain for better fault isolation. Automatic standby failover is only available for a standby within the same region and is required for meeting Gold MAA solution. • Customer responsibilities: – Extend application failover to local standby (Achieving Continuous Availability For Your Applications) or use Full Stack Disaster Recovery Service, for cross region standby. MAA Gold certification includes • Option 1: Standby database within the same region (preferably in a different Availability Domain) • Option 2: Standby database within the same region with an additional cross-region Autonomous Data Guard standby.
Platinum	Planned
Diamond	Planned

Autonomous Database on Dedicated Infrastructure (ADB-D)

MAA Tier	Validated Solutions
Bronze	N/A
Silver	ADB-D (Default) • Use Oracle AI Database cloud automation life cycle operations, including software updates, upgrades, monitoring, and database administration and management. • Customer responsibilities: – Create test systems to evaluate application changes – Maximize application failover and uptime with Achieving Continuous Availability For Your Applications
Gold	ADB-D with Autonomous Data Guard Silver, plus the following: • Enable Autonomous Data Guard, choose protection mode, data loss tolerance, and enable automatic failover. The location can be in another Availability Domain or Region for better fault isolation. • Customer responsibilities: – Extend application failover to standby with Achieving Continuous Availability For Your Applications or use Full Stack Disaster Recovery Service.
Platinum	Option 1: Upgrade MAA Gold with Oracle AI Database 26ai Option 2: ADB-D with Autonomous Data Guard and Oracle GoldenGate Gold, plus the following customer responsibilities: • Set up GoldenGate Cloud Within Region: Configuring Oracle GoldenGate Hub. GoldenGate management and tuning. • Extend application failover to Oracle GoldenGate replica with Achieving Continuous Availability For Your Applications
Diamond	Planned

Oracle Maximum Availability Architecture and Oracle Autonomous Database

Oracle Maximum Availability Architecture (MAA) represents Oracle's comprehensive framework of best practices, reference blueprints, and technologies for ensuring the highest levels of Oracle Database availability, data protection, and disaster recovery.

For users of Oracle Autonomous Database, whether leveraging the elastic, self-service Autonomous Database Serverless or the powerful, dedicated resources of Autonomous Database on Exadata Dedicated Infrastructure, the principles and capabilities of MAA are not a separate set of components to configure; they are the fundamental architecture upon which the service is built and automatically managed.

This fully automated, standardized Autonomous Database infrastructure means that achieving robust availability and disaster recovery for your mission-critical data is dramatically simplified and automated with Autonomous Database. The complexities typically associated with designing, implementing, and maintaining MAA configurations, such as setting up Real Application Clusters and infrastructure redundancy for fast failover for local failures, and setting up Autonomous Data Guard for additional high availability within a region or cross-region disaster recovery, are primarily handled by the Autonomous Database service.

Autonomous Database follows MAA best practices, using integrated technologies to provide built-in resilience, automated failover, and streamlined data protection. This self-managing approach reduces operational overhead and ensures your database benefits from Oracle's highest availability standards with minimal manual intervention. This includes built-in managed services that handle software updates and hardware maintenance, database backup and recovery, system and database monitoring and alerting, and zero downtime for elastic system changes such as increasing CPU and storage requirements.

The following topics provide the latest MAA certifications and observations for each of Oracle Autonomous Database Cloud services.

- [Oracle MAA for Oracle Autonomous Database on Dedicated Exadata Infrastructure](#)
- [Oracle MAA for Oracle Autonomous Database Serverless](#)

Oracle MAA for Oracle Autonomous Database on Dedicated Exadata Infrastructure

Autonomous Database Dedicated with Default High Availability Option (MAA Silver)

High availability is suitable for all development, test, and production databases that have high uptime requirements and zero or low data loss tolerance.

By default, Autonomous Database is highly available, incorporating a multi-node configuration to protect against localized software and hardware failures.

Each Autonomous Database application service resides in at least one Oracle Real Application Clusters (Oracle RAC) instance, with the option to fail over to another available Oracle RAC instance for unplanned outages or planned maintenance activities, enabling zero or near-zero downtime.

Autonomous Database automatic backups are stored in Oracle Cloud Infrastructure Object Storage, and are replicated to another available domain if available. For Autonomous Database with Exadata Cloud at Customer, you have the option to back up to NFS, Oracle Cloud Infrastructure Object Storage, or Zero Data Loss Recovery Appliance (ZDLRA); however, replication of those backups in NFS or ZDLRA is the responsibility of the customer.

You can set up Automatic Backups with varying backup retention policy/period or on-demand manual backups including long-term backups. These backups can be used to restore the database in the event of a disaster. See [Backup and Restore Autonomous Database on Dedicated Exadata Infrastructure](#)

To achieve the application uptime SLOs where most months would be zero downtime, see [Preparing Application for Seamless Application Failover](#).

The following table describes the recovery-time objectives and recovery-point objectives (data loss tolerance) for different outages.

Table 34-1 Default High Availability Policy Recovery Time (RTO) and Recovery Point (RPO) Service-level Objectives for Autonomous Database Dedicated

Failure and Maintenance Events	Database Downtime	Service-level Downtime (RTO)	Potential Service-level Data Loss (RPO)
Localized events, including: - Exadata cluster network topology failures - Storage (disk and flash) failures - Database instance failures - Database server failures - Periodic software and hardware maintenance updates	Zero	Near-zero	Zero
Events that require restoring from backup because a standby database does not exist: - Data corruptions - Human error - Full database failures - Complete storage failures - Availability domain (AD) for multi-AD regions	Minutes to hours (without Autonomous Data Guard)	Minutes to hours (without Autonomous Data Guard)	15 minutes for Autonomous Database on Dedicated Infrastructure (without Autonomous Data Guard)
Events that require non-rolling software updates or database upgrades	Minutes to an hour for Autonomous Database on Dedicated Infrastructure (without Autonomous Data Guard)	Minutes to an hour for Autonomous Database on Dedicated Infrastructure (without Autonomous Data Guard)	Zero

In the table above, the amount of downtime for events that require restoring from a backup varies depending on the nature of the failure. In the most optimistic case, physical block

corruption is detected, and the block is repaired with block media recovery in minutes. In this case, only a small portion of the database is affected with zero data loss. In a more pessimistic case, the entire database or cluster fails, then the database is restored and recovered using the latest database backup, including all archives.

Data loss is limited by the last successful archive log backup, the frequency of which is every 15 minutes for Autonomous Database on Dedicated Exadata Infrastructure. Archive or redo logs are backed up to Oracle Cloud Infrastructure Object Storage, or any supported Autonomous Database on Dedicated Exadata Infrastructure backup destination. Data loss can be seconds, or, at worst minutes of data loss, around the last successful archive log and remaining redo in the online redo logs that were not archived to backup destination.

All local failures incur zero database downtime because of built-in Exadata HA benefit and Oracle Cloud Infrastructure redundancy, and application brownouts are near zero or less 10 seconds. Software and hardware maintenance updates can incur zero database downtime and possible zero application impact because of online updates, Oracle RAC rolling updates, and application failover best practices. See [Preparing Application for Seamless Application Failover](#).

Autonomous Database Dedicated with Autonomous Data Guard Option (MAA Gold)

Enable Autonomous Data Guard for mission-critical production databases that require better uptime requirements for disasters from data corruptions, and database or site failures, while still reaping the Autonomous Database High Availability Option benefits. Additionally, a read-only standby database provides expanded application services to offload reporting, queries, and some updates.

You can also convert your physical standby database to a snapshot standby, but that impacts your Recovery Time Objective (RTO) if you have to switchover or failover to that specific standby.

Enabling Autonomous Data Guard adds one symmetric standby database to an Exadata rack that is located in the same availability domain, another availability domain, or in another region. The primary and standby database systems are configured symmetrically to ensure that performance service levels are maintained after Data Guard role transitions. Up to two standby databases are supported.

For additional protection, a typical MAA Gold architecture consists of a local standby with automatic failover enabled, and a cross-region standby for disaster recovery. For more information about managing an Autonomous Data Guard configuration, see [View the Status of an Autonomous Data Guard Configuration](#)

To meet bound Recovery Time Objectives (RTO) and Recovery Point Objectives (RPO) service-level objectives for database, cluster, or even site failures, enable Automatic Failover in the OCI Console Autonomous Data Guard Settings (see [Updating Autonomous Data Guard Settings](#)). You can choose which standby you want as the automatic failover target by updating Autonomous Data Guard Settings.

If RPO=0 or zero data loss is required, MAA customers typically deploy a local standby where the standby database is placed within the same availability domain or in a different availability domain in the same region. MAA recommends placing the standby in a separate availability domain if available. The round-trip latency between availability domains is typically less 1 millisecond, which in most cases has minimum application performance impact.

MAA recommends evaluating zero data loss configuration or Maximum Availability protection mode with Data Guard synchronous transport mode to ensure acceptable performance. If your

RPO is near zero or minimum data loss is acceptable, you can choose any standby database to enable automatic failover, and configure a fast start failover lag limit to bound maximum data loss before initiating automatic failover.

If the target is a cross-region standby and cross-region disaster recovery orchestration is required, MAA recommends enabling OCI Full Stack Disaster Recovery service where applicable to orchestrate database, application, and possibly network DNS failover and switchover operations. See [Use OCI Full Stack Disaster Recovery on Autonomous Database on Dedicated Exadata Infrastructure](#).

Backups

Backups are scheduled automatically for the standby database, and they are stored in Oracle Cloud Infrastructure Object Storage.

Autonomous Database with Exadata Cloud at Customer provides you with an option to backup to NFS, Oracle Cloud Infrastructure Object Storage, or Zero Data Loss Recovery Appliance; however, replication of those backups is your responsibility. Those backups can be used to restore databases in the event of a double disaster, where both primary and standby databases are lost.

You can set up Automatic Backups with varying backup retention policy/period or on-demand manual backups including long-term backups. See [Backup and Restore Autonomous Database on Dedicated Exadata Infrastructure](#).

Autonomous Data Guard Recovery Time (RTO) and Recovery Point (RPO) Service-level Objectives

The uptime recovery time objectives (downtime) and recovery point objectives (data loss) are low, as described in the table below.

To achieve the application uptime SLAs where most months would be zero downtime, see [Preparing Application for Seamless Application Failover](#). The target uptime RTO applies to container databases with less than 25 pluggable databases and does not include 30 seconds default detection time for automatic failover to standby.

Failure and Maintenance Events	Service-level Downtime (RTO) ¹	Potential Service-level Data Loss (RPO)
Localized events, including: - Exadata cluster network fabric failures - Storage (disk and flash) failures - Database instance failures - Database server failures - Periodic software and hardware maintenance updates on primary or standby	Zero or Near Zero	Zero
 Note Maintenance is typically applied on the standby first. For database software updates, the standby software is applied first, and after approximately a week the primary database software is updated.		
Events that require failover to the standby database using Autonomous Data Guard, including: - Data corruptions (because Active Data Guard has automatic block repair for physical corruptions², a failover operation is required only for logical corruptions or extensive data corruptions) - Full database failures - Complete storage failures - Availability domain or region failures³	A few seconds to two minutes ⁴	Zero with maximum availability protection mode (uses synchronous redo transport). Most commonly used for intra-region standby databases. Near zero for maximum performance protection mode (uses asynchronous redo transport). Most commonly used for cross-region standby databases. Also used for intra-regional standby databases and to ensure zero application impact. RPO is typically less than 10 seconds. RPO can be impacted by network bandwidth and workload throughput between primary and standby clusters.

¹ Service-Level Downtime (RTO) excludes detection time that includes multiple heartbeats to ensure the source is indeed inaccessible before initiating an automatic failover.

² The Active Data Guard automatic block repair for physical corruptions feature is only available for Autonomous Data Guard on Dedicated Infrastructure.

³ Regional failure protection is only available if the standby is located in another region.

⁴ The back-end Autonomous Data Guard role transition timings are much faster than indicated by the Cloud Console refresh rates. Automatic Failover must be enabled to bound RTO.

Autonomous Database on Dedicated Infrastructure has been MAA Gold validated and certified. Autonomous Database on Dedicated Infrastructure was validated with a standby database in the same region, and also with a standby database in a different region, and the above RTO and RPOs were met when the standby target was symmetric to the primary. RTO and RPOs were met with redo rates of up to 1100 MB/sec.

Depending on workload, you may have to scale your Autonomous Exadata VM Cluster system resource on either the primary or standby clusters. Follow the instructions in [Manage Autonomous Exadata VM Cluster Resources](#).

Updating Autonomous Data Guard Settings

You can update the settings of an Autonomous Data Guard standby from the details page of the primary Autonomous Container Database in the configuration.

Required IAM Policies:

use autonomous-container-databases

1. Go to the details page of the primary Autonomous Container Database in the Autonomous Data Guard configuration.

For instructions, see [View Details of an Autonomous Container Database](#).

2. Click **Update Autonomous Data Guard** from **More actions**.

The Update Autonomous Data Guard dialog displays the current settings for Protection Mode and Automatic Failover.

3. You can make the following updates from this dialog:

- a. **Protection mode:** Select Maximum performance or Maximum availability from the drop-down list.
- b. **Automatic failover:** To bound RTO and RPO, automatic failover needs to be enabled.

If automatic failover is not enabled already, you can enable it by selecting **Enable automatic failover**. Similarly, you can deselect **Enable automatic failover** to disable automatic failover for this Autonomous Data Guard setup.

If one of your standby databases is in the same region as the primary database, and the second is in a different region, the local standby database is prioritized over the remote standby as the automatic failover target. When you enable automatic failover, any of the standby databases is considered for the automatic failover target.

Note

You can not enable Automatic Failover for databases with cross-region Autonomous Data Guard setup on Exadata Cloud@Customer deployments.

- c. **Fast start failover lag limit:** If automatic failover is enabled and the protection mode is Maximum Performance, the Fast Start Failover lag limit value is displayed in seconds. By default, this value is set to 30 seconds, but you can change it to any value between 5 and 3600 seconds.
4. Save your changes.

Autonomous Data Guard Life Cycle Management

1. Setting up Standby or Adding Standby with Automatic Failover: see [Manage Autonomous Data Guard Configuration](#)
2. Monitoring Transport and Apply Lags

- a. You can view the Autonomous Data Guard details by selecting Autonomous Data Guard groups or Autonomous Data Guard associations under Resources. The Autonomous Data Guard table displays information about the peer container database, the current apply lag and transport lag, state, and last role change and creation dates. See [Manage Primary and Standby Databases in an Autonomous Data Guard Configuration](#)
- b. Set alarms for ApplyLag and TransportLag to ensure the standby is in sync and protecting your primary database. See [Using the Console](#)
3. Data Guard Role Transitions or Reinstate Standby: see Switchover or Failover topics in [Manage Autonomous Data Guard Configuration](#)
4. Automatic Failover Notifications can be set up using information in [Events for Autonomous Database on Dedicated Exadata Infrastructure](#)

MAA Autonomous Data Guard RTO and RPO Observations

The following table illustrates several configurations that MAA testing and evaluation show achieve Gold MAA tier RTO and RPOs. This was achieved after hundreds of role transitions with various database and cluster outages.

Primary Cluster	Standby Cluster	PDBs	Data Files	Services	Timing (minutes)
2 Node	2 Node	1	14	2	1:18
2 Node	2 Node	5	50	10	1:20
2 Node	2 Node	25	300	250	1:44
4 Node	4 Node	1	500	12	1:17

Autonomous Data Guard Failover Improvements

The tables below illustrate how role transition times have improved in Oracle 23ai.

Large CDB with Single PDB: Data Guard Failover Improvements

- Data Guard configured with between primary Exadata (4 node RAC) and standby Exadata (4 node RAC) X9
- CDB version 23.4 with one PDB, 500 data files, and 12 services
- Workload running: OLTP Swingbench against PDB during role transition
- Redo rate 100MB/second at CDB level with no lag

Oracle release	Close to mount	Term recovery	Media recovery	Convert to primary	CDB open	PDB open + srv start	Total
Oracle 19c (19.22)	00:14	00:17	00:05	00:01	00:06	00:50	01:17
Oracle 23ai (23.4) Release Label	00:04	00:07	00:08	00:01	00:06	00:07	00:29 (62% drop)

CBD with 5 PDBs: Data Guard Failover Improvements

- Data Guard configured with between primary Exadata (2 node RAC) and standby Exadata (2 node RAC)
- CDB version 23.4 with 5x PDB

- 50 total data files spread across the 5 PDBs
- 10 services - 2 per PDB
- Workload running: OLTP Swingbench against each PDB during role transition
- Redo rate ~60MB/second on the CDB
- All improvements in timing have been the result of code fixes

Oracle release	Close to mount	Term recovery	Media recovery	Convert to primary	CDB open	PDB open + srv start	Total
ExaCS Oracle 19c (19.18 + fixes)	00:19	00:25	00:05	00:01	00:15	00:24	)1:20
Oracle 23ai (23.4) Release Label	00:02	00:03	00:01	00:01	00:04	00:04	00:15 (81% drop)

Autonomous Database with Autonomous Data Guard Option and Oracle GoldenGate (MAA Platinum)

MAA Platinum with Autonomous Database on Dedicated Infrastructure is configurable. No guaranteed SLAs are provided since the GoldenGate and application failover configuration is manual.

MAA Platinum or Never-Down Architecture, delivers near-zero recovery time objective (RTO, or downtime incurred during an outage) and potentially zero or near zero recover point objective (RPO, or data loss potential).

The MAA Platinum with Autonomous Database on Dedicated Infrastructure ensures:

- RTO = zero or near-zero for all local failures
- RTO = zero or near-zero for disasters, such as database, cluster, or site failures, achieved by redirecting the application to an Autonomous Database with Autonomous Data Guard or Oracle GoldenGate replica
- Zero downtime maintenance for software and hardware updates
- Zero downtime database upgrade or application upgrade by redirecting the application to an upgraded Oracle GoldenGate replica residing in a separate Autonomous Database on Dedicated Infrastructure
- RPO = zero or near-zero data loss, depending on selecting the Oracle Data Guard Maximum Availability or Maximum Performance protection modes with synchronous redo transport in Autonomous Database with Autonomous Data Guard
- Fast re-synchronization and zero or near-zero RPO between Oracle GoldenGate source and target databases after a disaster using Cloud MAA GoldenGate Hub and Oracle GoldenGate best practices
- After any database failure, automatic failover to its standby database occurs automatically using integrated Data Guard Fast-start Failover (FSFO). Subsequently, automatic re-synchronization between Oracle GoldenGate source and target databases resumes from the new primary after a role transition. For synchronous transport, this leads to eventual zero data loss.

Prerequisites:

- Autonomous Database on Dedicated Infrastructure must be running Oracle Database software release 19.20 or later for GoldenGate conflict resolution support
- Autonomous Database with Autonomous Data Guard and automatic failover needs to be configured for fast GoldenGate resynchronization after a disaster
- GoldenGate setup must be done manually according to Cloud MAA best practices
- Application failover to an available GoldenGate replica or a new primary database must be configured. Currently, Global Data Services (GDS) cannot be used with an Autonomous Database in this architecture.

Implementing the MAA Platinum Solution

To achieve an MAA Platinum solution, review and leverage the technical briefs and documentation referenced in the following steps.

1. Review [MAA Platinum and MAA Diamond: Oracle RAC and Exadata](#) to understand MAA Platinum benefits and use cases.
 - a. Decide primary database locations based on application needs. The primary database will reside in Autonomous Database on Dedicated Infrastructure.
 - b. Decide standby database location based on fault isolation requirements.
 - c. Enable Autonomous Data Guard.
 - d. Choose Autonomous Data Guard protection mode based on RPO tolerance, and set up automatic failover.
2. Set up MAA GoldenGate Hub in Oracle cloud.
 - a. If all primary databases (GoldenGate replicas) are in the same region, see [Cloud Within Region: Configuring Oracle GoldenGate Hub](#).
If primary databases (GoldenGate replicas) are spread across 2 regions, see [Cloud Across Regions: Configuring Oracle GoldenGate Hub](#).
 - b. Configure Bidirectional Replication and Automatic Conflict Detection and Resolution. See [Set Up Bidirectional Replication for Oracle GoldenGate Microservices Architecture](#) or the latest [Oracle GoldenGate 21c documentation](#).
3. Configure custom application failover options so that your application can fail over automatically in the case of database, cluster, or site failure.

Preparing Application for Seamless Application Failover

Ensure that network connectivity to Oracle Cloud Infrastructure is reliable so that you can access your tenancy's Autonomous Database resources.

Follow the guidelines to connect to your Autonomous Database (see [Autonomous Database on Dedicated Exadata Infrastructure](#)). Applications must connect to the predefined service name and download client credentials that include the proper `tnsnames.ora` and `sqlnet.ora` files. You can also change your specific application service's `drain_timeout` attribute to fit your requirements.

For more details about enabling continuous application service through planned and unplanned outages, see [Configuring Continuous Availability for Applications](#).

For Exadata cloud planned maintenance events that require restarting database instances, Oracle automatically relocates services and drain sessions to another available Oracle RAC instance before stopping any Oracle RAC instance. For OLTP applications that follow the MAA checklist, draining and relocating services results in zero application downtime.

Some applications, such as long-running batch jobs or reports, may not be able to drain and relocate gracefully, even with a longer drain timeout. For those applications, Oracle recommends that you schedule the software planned maintenance window excluding these types of activities, or stop these activities before the planned maintenance window. For example, you can reschedule a planned maintenance window so that it is outside your batch windows, or stop batch jobs before a planned maintenance window.

Multicloud Considerations for Autonomous Data Guard

Autonomous Database on Dedicated Exadata Infrastructure is configured identically in Oracle's multicloud partners.

In Oracle's multicloud architecture, infrastructure is organized using a parent-child site model. Autonomous Data Guard in Oracle Cloud Infrastructure (OCI) operates within this framework. In Oracle's multicloud partnerships, the OCI-managed environment functions as the parent site, while the peered environment in the partner's data center is the child site.

Autonomous Data Guard behaves the same in a multicloud deployment as it does in native OCI, for both local and cross-region standby configurations. In this model, all databases, including standby databases, reside in the multicloud partner's data center or region (for example, Amazon Web Services (AWS)), while OCI provides orchestration and operational coordination.

With the databases residing in the multicloud partner's site, there are two distinct differences that may impact RTO and RPO:

- The multicloud partner's network peering or OCI network peering is used, and RPO can be impacted if network bandwidth is not sufficient to handle workload or redo generation.
- Data Guard observer (quorum), required for automatic failover, resides on the OCI site for additional fault isolation from primary and standby databases.

MAA Silver and Gold solutions for Autonomous Database on Dedicated Exadata Infrastructure are generally applicable in all of Oracle's multicloud partners that host Oracle Autonomous Database on Dedicated Exadata Infrastructure.

No additional MAA certification on Autonomous Database on Dedicated Exadata Infrastructure was conducted in the multicloud partner's ecosystems. Given the similarities, Oracle MAA and Autonomous Database on Dedicated Exadata Infrastructure engineers expect similar MAA benefits and certifications.

Oracle MAA for Oracle Autonomous Database Serverless

Autonomous Database Serverless with Default High Availability Option (MAA Silver)

High availability for Oracle Autonomous Database Serverless is recommended for all database environments (development, test, production) with demanding uptime requirements and low tolerance for data loss.

Autonomous Databases are provisioned with high availability enabled by default, employing a multi-node configuration to protect against localized failures. Specifically, Autonomous Database Serverless integrates Exadata Maximum Availability Architecture (MAA) best practices, supports online Oracle RAC rolling software updates, includes integrated backup

and recovery, and offers flexible resource scaling, providing a robust, available, and highly scalable service foundation.

The service architecture places each Autonomous Database application service within at least one Oracle RAC instance. This design facilitates automatic failover to other available RAC instances during unplanned outages or planned maintenance, ensuring zero or near-zero downtime. The underlying Exadata platform contributes significantly, providing inherent data protection, low brownout impact, performance QoS (MAA qualities), and Exadata Smart performance benefits.

Automated backups are stored externally in Oracle Cloud Infrastructure (OCI) Object Storage and are replicated to an alternate availability domain if available. These backups are essential for database recovery and disaster preparedness.

The integrated software lifecycle framework automates major database upgrades, greatly minimizing required downtime for Autonomous Database Serverless.

The following table describes the recovery-time objectives and recovery-point objectives (data loss tolerance) for different outages. For strategies to achieve higher application uptime, see [Configuring Continuous Availability for Applications](#).

Table 34-2 Default High Availability Policy Recovery Time (RTO) and Recovery Point (RPO) Service-level Objectives for Autonomous Database Serverless

Failure and Maintenance Events	Database Downtime	Service-level Downtime (RTO)	Potential Service-level Data Loss (RPO)
Localized events, including: - Exadata cluster network topology failures - Storage (disk and flash) failures - Database instance failures - Database server failures - Periodic software and hardware maintenance updates	Zero	Near-zero	Zero
Events that require restoring from backup when an Autonomous Data Guard standby database is not configured: - Data corruptions - Human error - Full database failures - Complete storage failures - Availability domain (AD) for multi-AD regions	Minutes to hours (without Autonomous Data Guard)	Minutes to hours (without Autonomous Data Guard)	1 minute for Autonomous Database Serverless (without Autonomous Data Guard)
Events that require non-rolling software updates or database upgrades	Less than 10 minutes for Autonomous Database Serverless (without Autonomous Data Guard)	Less than 10 minutes for Autonomous Database Serverless (without Autonomous Data Guard)	Zero

In the table above, the amount of downtime for events that require restoring from a backup varies depending on the nature of the failure. In the most optimistic case, limited physical block

corruption is detected, and individual objects can be restored. In this case, only a small portion of the database is affected, with zero data loss. In a more pessimistic case, the entire database or cluster fails, then the database is restored and recovered using the latest database backup, including all archives.

Data loss is limited by the last successful archive log backup, the frequency of which is 1 minute for Autonomous Database Serverless. Archives and redo from online redo logs are sent to the File Storage Service in different fault domains, if available, in real time, and further transferred remotely to additional regions in Oracle Cloud Infrastructure Object Storage or File Storage Service for future recovery purposes. Data loss can be seconds or, at worst, minutes of data loss, around the last successful archive log and remaining redo in the online redo logs that were not transferred to external storage.

Autonomous Database Serverless with Autonomous Data Guard Option (MAA Gold)

Enable Autonomous Data Guard for mission-critical production databases that require more strict uptime requirements for disasters from data corruptions, and database or site failures, while still receiving the Autonomous Database High Availability Option benefits.

Autonomous Database Serverless provides protection at the pluggable database (PDB) level. Enabling Autonomous Data Guard adds one symmetric standby database to an Exadata rack that is located in the same region or in another region. The primary and standby database systems are configured symmetrically to ensure that performance service levels are maintained after Data Guard role transitions.

Autonomous Database Serverless supports configuring multiple standby databases. A multiple standby configuration consists of a local standby database in the same region and one or more cross-region standby databases. Cross-regional standby databases are restricted to one per region. The MAA Gold Certification for Autonomous Database Serverless architecture includes a primary database and a local standby database in the same region, configured with automatic failover to meet the RTO and RPO Service Level Objectives. An alternative MAA Gold Architecture for Autonomous Database Services is a primary database and a local standby database in the same region, configured with automatic failover and one or more cross-region standby databases for disaster recovery.

Oracle Autonomous Data Guard features asynchronous redo transport (in maximum performance mode) by default to ensure zero application performance impact.

Backups are scheduled automatically for the Autonomous Databases and stored in Oracle Cloud Infrastructure Object Storage. Those backups can be used to restore databases in the event of a combined disaster, where both primary and standby databases are lost.

Automatic Data Guard failover with Autonomous Database Serverless supports a maximum data loss value that needs to be met before automatic failover to the standby. Zero data loss failover is not guaranteed for Autonomous Database Serverless, but it is possible when the primary database fails while the primary system container and infrastructure are still available. This allows the remaining redo to be sent and applied to the same region or local standby database. Automatic failover to the cross-region standby database is not available through the OCI Console.

In all cases, automatic Autonomous Data Guard failover occurs for primary database, cluster, or data center failures when those data loss service levels can be guaranteed. The target standby becomes the new primary database, and all application services are enabled automatically. A manual Data Failover option is provided in the OCI Console. For the manual Data Guard failover option, the calculated downtime for the uptime SLA starts with the time to execute the Data Guard failover operation and ends when the new primary service is enabled.

Depending on your application or business requirements, you can choose whether your database failover site is located in the same region or in a different region.

Autonomous Database Serverless was validated and met the SLOs in the table below. RTO and RPO SLOs were met with redo rates up to 300 MB/sec for the entire Container Database (CDB) where the target Autonomous Data Guard pluggable primary database resides.

Table 34-3 Autonomous Data Guard Recovery Time (RTO) and Recovery Point (RPO) Service-level Objectives

Failure and Maintenance Events	Service-level Downtime (RTO)	Potential Service-level Data Loss (RPO)
Localized events, including: • Exadata cluster network fabric failures • Storage (disk and flash) failures • Database instance failures • Database server failures • Periodic software and hardware maintenance updates	Zero or Near Zero	Zero
Events that require failover to the standby database using Autonomous Data Guard with automatic failover, including: • Data corruptions • Full database failures • Complete storage failures • Availability domain or region failures Note that the automatic failover feature is only available for a standby database within the same region. For cross-region failover or switchover operations, timings are calculated after the command is issued.	Less than 10 minutes MAA observations are typically < 400 seconds	Less than 60 seconds MAA observations are typically < 10 seconds Near zero due to the use of asynchronous redo transport. RPO is typically less than 10 seconds. RPO can be impacted by network bandwidth and throughput between primary and standby clusters.

Adding an Autonomous Standby Database

Autonomous Database Serverless supports, at most, one Autonomous Data Guard local standby database in the same region, and multiple cross-region standby databases for each available region with at most one per region. These additional regions are determined by Oracle. You can have any mix of local and cross-region standby databases for each primary.

When you create a standby database in the same region as your primary database, you can specify a maximum data loss criteria to enable automatic failover for non-zero data loss situations. To achieve MAA Gold, in all configurations you must have at least a local standby database configured with automatic failover so you can meet a low RTO and RPO value in the case of disaster. Automatic failover is not currently provided for cross-region standby databases.

Adding a Local Standby Database

1. In the primary database's OCI Console details page, in the **Autonomous Database information** tab, go to the **Disaster recovery** section, and select the Action menu (3 dots) on the **Local** line, and select **Upgrade to Autonomous Data Guard**.

Autonomous Database information

Tool configuration

Monitoring

Backups

Key history

Disaster recovery

Disaster recovery provides a peer database instance in a different availability domain or in a peer database instance in a different region. With a peer database in the local region, if the primary database becomes unavailable, disaster recovery switches to role of the peer database to primary and begins recreating a new peer database. With a cross-region peer database, you can promote the peer database to primary database if the local region fails. [Learn more](#)

Role	Primary
Local	Backup-based

Upgrade to Autonomous Data Guard

Switchover

...

2. In the **Update disaster recovery** panel, verify that the local region and **Autonomous Data Guard** are selected, then **Submit**.

You can optionally enable a maximum data loss limit in **Automatic Failover with data loss limit in seconds**. If you lose the primary cluster or site, there is some data loss because redo is sent asynchronously, so entering a non-zero value is reasonable if you want to reduce application and database downtime in case of most primary database, primary cluster, or complete data center failures. By setting the value greater than zero, automatic failover occurs only if the calculated data loss is less than or equal to the limit. If zero data loss failover is possible by accessing all primary database redo, it is done automatically. If data loss is greater than the setting, no automatic action occurs.

Adding a Cross-region Standby Database

1. In the primary database's OCI Console details page, select the **Disaster recovery** tab, then select **Add peer database**.

maa001 Available Primary
Autonomous Database

Autonomous Database information Tool configuration Monitoring Backups Key history **Disaster recovery**

Local peer: 1, Cross-region peer: 1.
You may create up to 1 local peer and 1 cross-region peer for every remote paired region that your tenancy is subscribed to.

Add peer database

2. In the **Add peer database** panel, select the location for the standby database from the **Region** list, then **Add**.

Add peer database

Adding disaster recovery creates a peer database, and impacts billing. Do you want to add a disaster recovery peer?

Region

Canada Southeast (Toronto)

Canada Southeast (Toronto)

US East (Ashburn)

Select region for Autonomous Data Guard. If a peer exists in a region, then that region will not appear in the list

There is no option for automatic failover for a cross-region standby database.

After adding the cross-region standby, you can select the standby database name to view its characteristics in the remote region.

Listing Standby Databases

Local standby databases are not directly accessible using either the OCI Console or clients, but they are visible in the **Disaster recovery** sections of the primary database's details page.

A cross-region standby can be managed in its own console page, which is accessible by selecting the database name. Cross-region standbys also appear in the **Disaster recovery** sections of the primary database details page with the region abbreviation appended to the database name.

maa001 Available Primary

Database actions Database connection Performance Hub More actions

Autonomous Database

Autonomous Database Information Tool configuration Monitoring Backups Key history Disaster recovery Refreshable clones Work requests Guided console tour Security Tags

Local peer: 1, Cross-region peer: 1.
You may create up to 1 local peer and 1 cross-region peer for every remote paired region that your tenancy is subscribed to.

Search and Filter

Add peer database

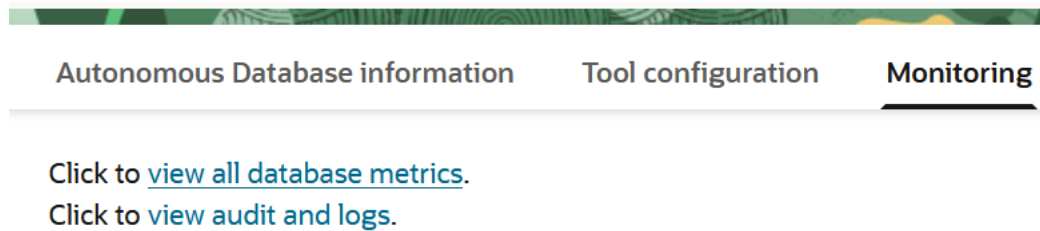
Peer Autonomous Database	Peer role	State	Region	DR type	Remote backup replication	Automatic Failover data loss...	Role changed on	Created	
maa001_YYZ	Standby	Standby	Canada Southeast (Toronto)	Autonomous Data Guard	Disabled	—	Tue, Mar 4, 2025, 21:19:31 UTC	Thu, Jun 13, 2024, 19:49:27 UTC	...
maa001	Standby	Standby	Canada Southeast (Montreal)	Autonomous Data Guard	Disabled	10 seconds	Wed, Mar 19, 2025, 21:28:25 UTC	Mon, Jun 3, 2024, 15:25:11 UTC	...

The original region of the primary database is called the "home region". Each home region has one or more remote regions ("buddy regions") associated with it to contain cross-region standby databases.

Monitoring Apply Lag

You can monitor Autonomous Data Guard apply lag for redo for both a local standby and a cross-region standby. The data is provided in near real time; there is a short delay between gathering and uploading to the control plane for display, and the data applies to the entire POD, not just your specific Autonomous database.

To access the lag information, in the OCI Console Autonomous Database details page, on the **Monitoring** tab, select **view all database metrics**.



On the Service Metrics page, scroll down until you see the **Peer Lag** metric display.

The database being viewed determines the data in the metric. If you are viewing the primary database, the metric data display is for the local standby; if you are viewing the cross-region standby, the metric data display is for the cross-region standby.

Transport lag is typically less than 10 seconds for a local standby and less than 60 seconds for cross-region standby, depending on workloads and peaks.

Figure 34-1 Peer Lag for Local Standby

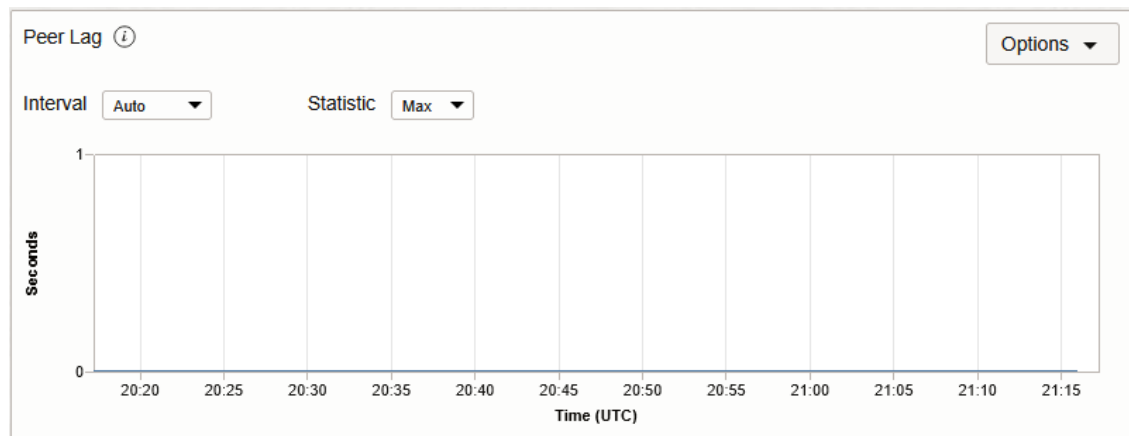
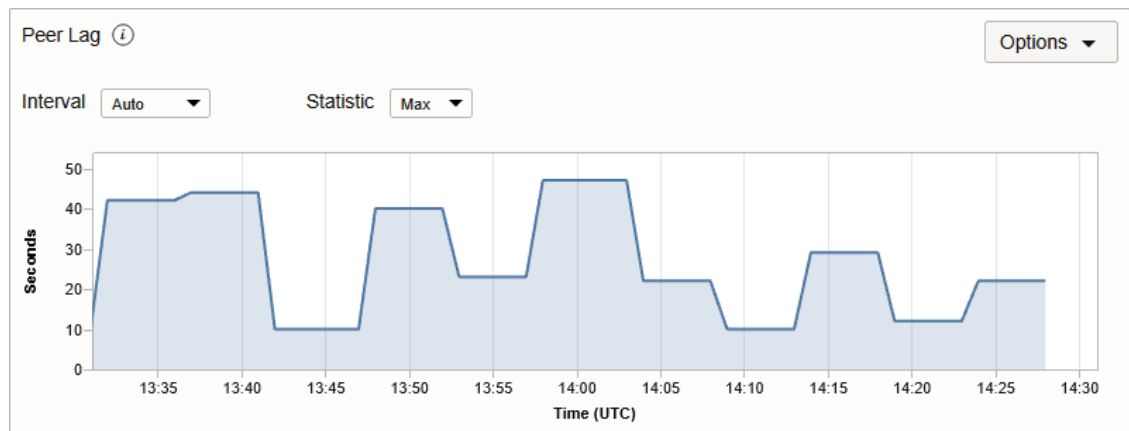


Figure 34-2 Peer Lag for Cross-Region Standby



Autonomous Data Guard Role Transitions

Either a local or cross-region standby database can be used as the target destination for a role transition.

The local standby should be the primary target for switchover operations, ensuring that there is no additional latency in connections and network operations after role transitions complete.

To perform a switchover to the local standby database, in the OCI Console, in the primary database's details page (in the home region), in the **Disaster recovery** section, select **Switchover** on the **Local** line, as shown in the following image.

Figure 34-3 Switchover to Local Standby

Disaster recovery

Disaster recovery provides a peer database instance in a different availability domain or in a peer database instance in a different region. With a peer database in the local region, if the primary database becomes unavailable, disaster recovery switches to role of the peer database to primary and begins recreating a new peer database. With a cross-region peer database, you can perform a manual failover to the peer database if the local region fails. [Learn more](#)

Role	Primary	
	Last failover: Wed, Mar 19, 2025, 20:47:05 UTC.	
Local	Autonomous Data Guard	Switchover
Cross-region	1 Peer	Switchover

If the local standby switchover fails, perform a switchover to the cross-region standby.

Note

The cross-region switchover must be initiated from the remote region cross-region standby's details page in the OCI Console, not from the primary database page.

To perform a switchover to the remote (cross-region) standby database, in the OCI Console, in the cross-region database's details page (in the remote region), in the **Disaster recovery** section, select **Switchover** on the **Role** line, as shown in the following image.

Figure 34-4 Switchover to Cross-region Standby

Disaster recovery

Disaster recovery provides a peer database instance in a different availability domain or in a peer database instance in a different region. With a peer database in the local region, if the primary database becomes unavailable, disaster recovery switches to role of the peer database to primary and begins recreating a new peer database. With a cross-region peer database, you can perform a manual failover to the peer database if the local region fails. [Learn more](#)

Role	Standby	... Switchover Update disaster recovery Disconnect peer
Cross-region backup replication	Not enabled	
Full Stack DR	Configure The list of DR protection groups that have this database as a member. This list may be incomplete due to insufficient policy permissions to access.	

If the switchover is successful, the **Role** line status changes from **Standby** to **Primary** when the primary becomes accessible.

Manual Failover Operations and Determining Data Loss

Because your database is in a shared environment and you only have access to your autonomous databases, you will not know if issues are impacting only your databases or the entire POD. It is always best to react under the premise that the issues are local to only your database.

If you do not have automatic failover enabled, always attempt a switchover using the OCI Console first. A successful switchover guarantees zero data loss. If switchover fails, the role transition option in the OCI Console will change to **Failover**.

The following image shows the **Disaster recovery** section of the cross-region (remote) standby details page, showing that because the **Switchover** action was not successful, the choice is now a **Failover**.

Disaster recovery	
Disaster recovery provides a peer database instance in a different availability domain or in a peer database instance in a different region. With a peer database in the local region, if the primary database becomes unavailable, disaster recovery switches to role of the peer database to primary and begins recreating a new peer database. With a cross-region peer database, you can perform a manual failover to the peer database if the local region fails. Learn more.	
Role	Standby
Cross-region backup replication	Not enabled
Full Stack DR	Configure The list of DR protection groups that have this database as a member. This list may be incomplete due to insufficient policy permissions to access.

...

Failover

Update disaster recovery

Disconnect peer

In this case, the failover operation ensures that all available redo will be applied, though it does not guarantee zero data loss. This applies to both local and cross-region failover operations.

Determining Data Loss

Each manual failover job generates a work request visible in the console.

To determine the amount of data loss after a failover to the local standby, go to the **Work requests** tab on the Autonomous Database details page. For a cross-region standby, look at the **Work requests** in the cross-region new primary.

Select the work request for the failover and view the **Log messages**, which report the data loss incurred during the failover.

Failover Autonomous Database

Succeeded

Work request information

Log messages

Error messages

Associated resources

Q Search and Filter

Message

Timestamp (UTC)

Initializing standby database failover.
Standby database failover started.
Standby database failover completed.Data loss of 6 seconds observed

Tue, Mar 4, 2025, 20:51:40 UTC

Failover Autonomous Database

Tue, Mar 4, 2025, 20:48:18 UTC

Notifications for Automatic Failover

Automatic failover is only provided for the local standby database, and activates when connection loss is detected against the primary database.

Oracle scans the Connection Manager (CMAN) connection log, searching for connection failures. When failures are detected for a period of time (typically 60 to 90 seconds) with no successful connections found for the duration, automatic failover is triggered. Detection of a successful connection resets the check and timing.

Regardless of any Automatic Failover settings, if a failure is detected and if Oracle can guarantee zero data loss, failover to your local region standby occurs automatically.

If a non-zero data loss situation is encountered, Oracle checks if automatic failover is enabled and compares the potential data loss to the Automatic Failover loss limit defined. If the potential data loss is determined to be less than the specified loss limit, automatic failover occurs.

Local standby automatic failover jobs do not create work requests that can be reviewed; however, you can create events to receive notification of the beginning and end of automatic failover operations, in addition to displaying the data loss associated with the failover. Information about creating events can be found in [Get Notified of Autonomous Database Events](#). Notifications require rules to establish the conditions under which a notification is sent.

To enable notifications for local standby automatic failover:

1. Open the Notifications service.
In the OCI Console, select the hamburger menu in the upper left-hand corner, and select **Developer Services**, then select **Notifications** under **Application Integration**.
2. Select **Create Topic** and provide a name and description.
3. Select the new topic and select **Create Subscription**
4. In the Create Subscription panel, select a notification type (email, Slack, pager, etc) and select **Create**.
5. Open the Events service **Rules** page.

In the OCI Console, select the hamburger menu in the upper left hand corner, and select **Observability & Management**, then select **Rules** under **Events Service**.

6. On the Rules page, select **Create Rule**.

A single rule can generate notifications for multiple events.

7. Provide a display name and description, and in the **Rule Conditions** section, enter the following conditions for notifying on Automatic Failover Begin and End events.

- a. To establish the type of event for the rule, configure the first Condition with these values:
 - **Condition:** Event Type
 - **Service Name:** Database
 - **Event Type:** Autonomous Container Database - Critical
- b. To limit the rule to just your tenancy, add a Condition with the following values:
 - **Condition:** Attribute
 - **Service Name:** compartmentId
 - **Attribute Values:** my.tenancy.id
- c. To cause the rule to send notifications for Automatic Failure Begin and End events, add another Condition with the following values:
 - **Condition:** Attribute
 - **Attribute Name:** eventName
 - **Attribute Values:** AutomaticFailoverBegin and AutomaticFailoverEnd

Rule Conditions

Limit the events that trigger actions by defining conditions based on event types, attributes, and filter tags. [Learn more](#)

Condition	Service Name	Event Type
Event Type	Database	Autonomous Container Database - Critical

Condition	Attribute Name	Attribute Values
Attribute	compartmentId	my.tenancy.id

Condition	Attribute Name	Attribute Values
Attribute	eventName	AutomaticFailoverBegin AutomaticFailoverEnd

+ Another Condition

8. In the **Actions** section, below the **Rule Conditions** section in the same dialog, associate the rule with an action to trigger notification, using the following settings:

- **Action Type:** Notifications.
- **Notifications Compartment:** Your tenancy name from the drop down list.
- **Topic:** choose the topic you created in Step 2 above.

9. Click **Create Rule** to save the rule.

After the rules and notification creation are completed, whenever an automatic failover operation begins and ends you receive a notification based on the rule you set up. For example, if you chose an email subscription, you would receive a begin email message with a subject like "OCI Event

Notification :com.oraclecloud.databaseservice.autonomous.database.critical" containing messaging including something like the following:

```
{
  "eventType" :
    "com.oraclecloud.databaseservice.autonomous.database.critical",
  "cloudEventsVersion" : "0.1",
  "eventTypeVersion" : "2.0",
  "source" : "DatabaseService",
  "eventTime" : "2025-03-18T20:21:24Z",
  "contentType" : "application/json",
  "data" : {
    "compartmentId" : "<OCID of your Tenancy>",
    "compartmentName" : "<Tenancy Name>",
    "resourceName" : "<Autonomous Database name>",
    "resourceId" : "<OCID of the Autonomous Database involved>",
    "additionalDetails" : {
      "dbName" : "<Autonomous Database name>",
      "eventName" : "AutomaticFailoverBegin",
      "description" : "Automatic failover for database <Autonomous Database
name> has begun.",
      "autonomousDataType" : "Serverless",
      "workloadType" : "Data Warehouse"
    }
  },
  "eventID" : "<Event ID>",
  "extensions" : {
    "compartmentId" : "<OCID of your Tenancy>"
  }
}
```

Then you would receive an end email message with the same subject containing something like the following. The end email message also contains the calculated data loss from the operation.

```
{
  "eventType" :
    "com.oraclecloud.databaseservice.autonomous.database.critical",
  "cloudEventsVersion" : "0.1",
  "eventTypeVersion" : "2.0",
  "source" : "DatabaseService",
  "eventTime" : "2025-03-19T20:47:13Z",
  "contentType" : "application/json",
  "data" : {
    "compartmentId" : "<OCID of your Tenancy>",
    "compartmentName" : "<Tenancy Name>",
    "resourceName" : "<Autonomous Database name>",
    "resourceId" : "<OCID of the Autonomous Database>",
    "additionalDetails" : {
      "dbName" : "<Autonomous Database name>",
      "eventName" : "AutomaticFailoverEnd",
      "description" : "Automatic failover completed with 4 seconds data loss
and <Autonomous Database name> is AVAILABLE and ready for user operations.",
      "autonomousDataType" : "Serverless",
      "workloadType" : "Data Warehouse"
    }
  }
}
```

```

    }
  },
  "eventID" : "<Event ID>",
  "extensions" : {
    "compartmentId" : "<OCID of your Tenancy>"
  }
}

```

MAA Autonomous Data Guard RTO and RPO Observations

The following table provides MAA observations of hundreds of switchover and failover operations with heavy workload.

Note that the redo rates listed are for the entire POD, not for just the Autonomous Databases involved in the role transition. In Autonomous Database Serverless you have no control over the activity in Autonomous Databases not owned by you. Manual failovers require human detection time and attempts to switchover before submitting the failover operation. The Return to Operation (RTO) times are from the time the failover was submitted. For automatic failover, the detection time is typically 60 to 90 seconds. In our MAA chaos lab, up to 6 concurrent switchover or failover operations will occur concurrently after a major outage.

Table 34-4 Autonomous Data Guard Local Standby Observations

Use Cases	Operation Type	POD Redo Rate	Maximum Application RTO	Maximum RPO
CDB Failure	Manual	250-270 MB/sec	Detection time + 85 seconds	2 seconds
CDB Failure	Automatic	250-265 MB/sec	Detection time +57 seconds	2 seconds
Cluster Failure	Manual	250-280 MB/sec	Detection time +72 seconds	2 seconds
Cluster Failure	Automatic	250-280 MB/sec	Detection time +63 seconds	1 seconds
Switchover	Manual	225-240 MB/sec	118 seconds	0
Availability Domain Failure	Manual or Automatic	250-285 MB/sec	Same as cluster failure	Same as cluster failure

Table 34-5 Autonomous Data Guard Cross-Region Standby Observations

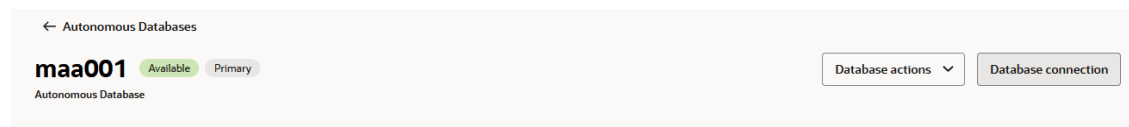
Use Cases	Operation Type	POD Redo Rate	Maximum Application RTO	Maximum RPO
CDB Failure	Manual	175-200MB/sec	Based on reaction to outage by user + potential 5 minutes for failed switchover in UI 106 seconds	1-3 seconds
Cluster Failure	Manual	175-200MB/sec	Based on reaction to outage by user + potential 5 minutes for failed switchover in UI 126 seconds	13 seconds
Switchover	Manual	175-200 MB/sec	407 seconds	0

Table 34-5 (Cont.) Autonomous Data Guard Cross-Region Standby Observations

Use Cases	Operation Type	POD Redo Rate	Maximum Application RTO	Maximum RPO
Site Failure	Manual	175-200 MB/sec	Same as cluster failure	Typically, 30-60 seconds Cross-region redo push is every 60 seconds

Preparing an Application for Seamless Application Failover

To retrieve sample connection strings for the database, in the OCI Console, on the Autonomous Database home region details page select **Database connection**.



You have two options:

- The **Regional wallet** provides connect strings for all local databases in the local region for your tenancy. For example, if you have 10 databases in Ashburn region and you are viewing one of them, the OCI Console generates TNS aliases for all 10, with an address list consisting of only the required host names to connect to the Ashburn copies of the databases.
- When running in the remote region there are two options for the connection string:

- Connect to the cross-region standby displayed in the remote region in the OCI console and retrieve the **Regional wallet** under **Wallet Type** from the **Database connection** section.

The download consists of a ZIP file including a number of files required for connection. The `tnsnames.ora` in the ZIP file contains only the host for the CMAN local to that region, and provides connect strings for all local databases for the local region for your tenancy.

For example, if you have 10 databases in Ashburn region, and you are viewing one of them, the OCI Console generates TNS aliases for all 10 with an address list consisting of only the required host names to connect to the Ashburn copies of the databases.

- Connect to primary database in the home region and retrieve the **Instance wallet** under **Wallet type** from the **Database Connection** section.

The download consists of a ZIP file including a number of files required for connection. The `tnsnames.ora` in the ZIP file contains the host names for both the home and remote region in the address list, with the home region listed first and the remote region listed second.

Using the instance wallet as is, the connection to the initial host in the address list must fail before it tries to connect to the remote region. You can modify the connect strings to follow MAA recommendations and avoid the delay caused by waiting for the connection to the first host in the address list to timeout. For more information see [Step 2: Configure the Connection String for High Availability](#).

Note

For the local standby, the same connect string works regardless of where in the region the database resides internally. The host address connects to the local Connection Manager (CMAN), which in turn connects to the appropriate primary database for the connection string.

MAA has recommendations for connection strings and methods to ensure the highest availability and smoothest transitioning. The connection strings provided here should be used as the starting point. See [Configuring Continuous Availability for Applications](#) for more information.

Multicloud Considerations for Autonomous Data Guard

Autonomous Database Serverless is configured identically in Oracle's multicloud partners.

In Oracle's multicloud architecture, infrastructure is organized using a parent-child site model. Autonomous Data Guard in Oracle Cloud Infrastructure (OCI) operates within this framework. In Oracle's multicloud partnerships, the OCI-managed environment functions as the parent site, while the peered environment in the partner's data center is the child site.

Autonomous Data Guard behaves the same in a multicloud deployment as it does in native OCI, for both local and cross-region standby configurations. In this model, all databases, including standby databases, reside in the multicloud partner's data center or region (for example, Google Cloud Platform or Microsoft Azure), while OCI provides orchestration and operational coordination.

With the databases residing in the multicloud partner's site, there is one distinct difference that may impact RTO and RPO: the multicloud partner's network and local storage are used as part of the redo storage and redo transport life cycle. Oracle manages this automatically as part of the service. In cases of high peer lag, the RTO and RPO may be impacted.

MAA Silver and Gold solutions for Autonomous Database Serverless are generally applicable in all of Oracle's multicloud partners that host Oracle Autonomous Database Serverless.

Oracle Maximum Availability Architecture in Oracle Exadata Cloud Systems

Oracle Maximum Availability Architecture in Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D), Oracle Exadata Database Service on Cloud@Customer (ExaDB-C@C), and Oracle Exadata Database Service on Exascale Infrastructure (ExaDB-XS) provides inherent high availability, data protection, and disaster recovery protection integrated with both cloud automation and life cycle operations, enabling Oracle Exadata Cloud systems to be the best cloud solution for enterprise databases and applications.

See [Oracle Cloud: Maximum Availability Architecture](#) for detailed walk-through of Oracle Cloud MAA architectures and features.

Oracle Maximum Availability Architecture Benefits

- **Deployment:** Oracle Exadata Cloud systems (ExaDB-D and ExaDB-C@C) are deployed using Oracle Maximum Availability Architecture best practices, including configuration best practices for storage, network, operating system, Oracle Grid Infrastructure, and Oracle Database. ExaDB-D is optimized to run enterprise Oracle databases with extreme scalability, availability, and elasticity.
- **Oracle Maximum Availability Architecture database templates:** All Oracle Cloud databases created with Oracle Cloud automation use Oracle Maximum Availability Architecture default settings, which are optimized for Exadata Database Services.

Oracle does not recommend that you use custom scripts to create cloud databases. Other than adjusting memory and system resource settings, avoid migrating previous database parameter settings, especially undocumented parameters. One beneficial database data protection parameter, `DB_BLOCK_CHECKING`, is not enabled by default due to its potential overhead. MAA recommends evaluating the performance impact for your application and enabling this setting if performance impact is reasonable.

- **Backup and restore automation:** When you configure automatic backup to recommended Zero Data Loss Autonomous Recovery Service (Recovery Service) or Oracle Cloud Infrastructure Object Storage, backup copies provide additional protection when multiple availability domains exist in your region, and RMAN validates cloud database backups for any physical corruptions.

Database backups occur daily, with a full backup occurring once during onboarding and incremental backups occurring subsequently with Recovery Service. Archive log backups occur frequently to reduce potential data loss in case of disaster. The archive log frequency is typically 15 or 30 minutes depending on the database service. By enabling real time data protection with Recovery Service, redo is streamed directly from the database, reducing data loss exposure to near zero.

- **Oracle Exadata Database Machine inherent benefits:** Oracle Exadata Database Machine is the best Oracle Maximum Availability Architecture platform that Oracle offers. Exadata is engineered with hardware, software, database, and availability innovations that support the most mission-critical enterprise applications.

Specifically, Exadata provides unique high availability, data protection, and quality-of-service capabilities that set Oracle apart from any other platform or cloud vendor. Sizing

Exadata cloud systems to meet your application and database system resource needs (for example, sufficient CPU, memory, and I/O resources) is very important to maintain the highest availability, stability, and performance. Proper sizing is especially important when consolidating many databases on the same cluster.

For a comprehensive list of Oracle Maximum Availability Architecture benefits for Oracle Exadata Database Machine systems, see [Exadata Database Machine: Maximum Availability Architecture Best Practices](#) and [MAA Platinum and MAA Diamond: Oracle RAC and Exadata](#).

Examples of these benefits include:

- **High availability and low brownout:** Fully-redundant, fault-tolerant hardware exists in the storage, network, and database servers. Resilient, highly-available software, such as Oracle Real Application Clusters (Oracle RAC), Oracle Clusterware, Oracle Database, Oracle Automatic Storage Management, Oracle Linux, and Oracle Exadata Storage Server enable applications to maintain application service levels through unplanned outages and planned maintenance events.

For example, Exadata has instant failure detection that can detect and repair database node, storage server, and network failures in less than two seconds, and resume application and database service uptime and performance. Other platforms can experience 30 seconds, or even minutes, of blackout and extended application brownouts for the same type of failures. Only the Exadata platform offers a wide range of unplanned outage and planned maintenance tests to evaluate end-to-end application and database brownouts and blackouts.

- **Data protection:** Exadata provides Oracle Database with physical and logical block corruption prevention, detection, and, in some cases, automatic remediation.

The Exadata Hardware Assisted Resilient Data (HARD) checks include support for server parameter files, control files, log files, Oracle data files, and Oracle Data Guard broker files, when those files are stored in Exadata storage. This intelligent Exadata storage validation stops corrupted data from being written to disk when a HARD check fails, which eliminates a large class of failures that the database industry had previously been unable to prevent.

Examples of the Exadata HARD checks include:

- Redo and block checksum
- Correct log sequence
- Block type validation
- Block number validation
- Oracle data structures, such as block magic number, block size, sequence number, and block header and tail data structures

Exadata HARD checks are initiated from Exadata storage software (cell services) and work transparently after enabling a database `DB_BLOCK_CHECKSUM` parameter, which is enabled by default in the cloud. Exadata is the only platform that currently supports the HARD initiative.

Furthermore, Oracle Exadata Storage Server provides non-intrusive, automatic hard disk scrub and repair. This feature periodically inspects and repairs hard disks during idle time. If bad sectors are detected on a hard disk, then Oracle Exadata Storage Server automatically sends a request to Oracle Automatic Storage Management (ASM) to repair the bad sectors by reading the data from another mirror copy.

Finally, Exadata and Oracle ASM can detect corruptions as data blocks are read into the buffer cache, and automatically repair data corruption with a good copy of the data block on a subsequent database write. This inherent intelligent data protection makes Exadata

Database Machine and ExaDB-D the best data protection storage platform for Oracle databases.

For comprehensive data protection, a Maximum Availability Architecture best practice is to use a standby database on a separate Exadata instance to detect, prevent, and automatically repair corruptions that cannot be addressed by Exadata alone. The standby database also minimizes downtime and data loss for disasters that result from site, cluster, and database failures.

- **Response time quality of service:** Only Exadata has end-to-end quality-of-service capabilities to ensure that response time remains low and optimum. Database server I/O latency capping and Exadata storage I/O latency capping ensure that read or write I/O can be redirected to partnered cells when response time exceeds a certain threshold.

If storage becomes unreliable (but not failed) because of poor and unpredictable performance, then the disk or flash cache can be confined offline, and later brought back online if heuristics show that I/O performance is back to acceptable levels. Resource management can help prioritize key database network or I/O functionality, so that your application and database perform at an optimized level.

For example, database log writes get priority over backup requests on Exadata network and storage. Furthermore, rapid response time is maintained during storage software updates by ensuring that partner flash cache is warmed so flash misses are minimized.

- **End-to-end testing and holistic health checks:** Because Oracle owns the entire Oracle Exadata Cloud Infrastructure, end-to-end testing and optimizations benefit every Exadata customer around the world, whether hosted on-premises or in the cloud. Validated optimizations and fixes required to run any mission-critical system are uniformly applied after rigorous testing. Health checks are designed to evaluate the entire stack.

The Exadata health check utility EXACHK is Exadata cloud-aware and highlights any configuration and software alerts that may have occurred because of customer changes. No other cloud platform currently has this kind of end-to-end health check available. For Oracle Autonomous Database, EXACHK runs automatically to evaluate Maximum Availability Architecture compliance. For non-autonomous databases, Oracle recommends running EXACHK at least once a month, and before and after any software updates, to evaluate any new best practices and alerts.

- **Higher Uptime:** The uptime service-level agreement per month is 99.95% (a maximum of 22 minutes of downtime per month), but when you use MAA best practices for continuous service, most months would have zero downtime.

Full list of Exadata features and benefits: Whats New in Oracle Exadata Database Machine

Oracle Maximum Availability Architecture best practices paper: Oracle Maximum Availability Architecture (MAA) engineering collaborates with Oracle Cloud teams to integrate Oracle MAA practices that are optimized for Oracle Cloud Infrastructure and security. See [MAA Best Practices for the Oracle Cloud](#) for additional information about continuous availability, Oracle Data Guard, Hybrid Data Guard, Oracle GoldenGate, and other Maximum Availability Architecture-related topics.

Expected Impact with Unplanned Outages

The following table lists various unplanned outages and the associated potential database downtime, application-level Recovery Time Objective (RTO), and data loss potential or recovery point objective (RPO). For Oracle Data Guard architectures, the database downtime or service level downtime does not include detection time or the time it takes before a customer initiates the Cloud Console Data Guard failover operation.

The following table shows the availability and performance impact for unplanned outages.

Failure and Maintenance Events	Database Downtime	Service-Level Downtime (RTO)	Potential Service-Level Data Loss (RPO)
Localized events, including: Exadata cluster network topology failures Storage (disk and flash) failures Database instance failures Database server failures	Zero	Near-zero	Zero
Events that require restoring from backup because a standby database does not exist: Data corruptions Full database failures Complete storage failures Availability domain	Minutes to hours (without Data Guard)	Minutes to hours (without Data Guard)	30 minutes (without Data Guard)
Events using Data Guard to fail over: Data corruptions Full database failures Complete storage failures Availability domain or region failures	Seconds to minutes ¹ Zero downtime for physical corruptions due to auto-block repair feature	Seconds to minutes ¹ The foreground process that detects the physical corruption pauses while auto block repair completes	Zero for Maximum Availability (SYNC) Near Zero for Maximum Performance (ASYNC)

¹ To protect from regional failure, you will need a standby database in a different region than the primary database. To achieve low bounded RTO with Data Guard, you can manually set up automatic failover by configuring Data Guard Fast-Start Failover.

Expected Impact with Planned Maintenance

The following table lists various software updates and the associated database and application impact. This is applicable for all Oracle Exadata Cloud infrastructures, including Oracle Exadata Cloud@Customer (ExaDB-C@C), Oracle Exadata Cloud Infrastructure (ExaDB-D) Gen2, and Oracle Autonomous Database (ADB).

Table 35-1 Availability and Performance Impact for Oracle Exadata Cloud Software Updates

Software Update	Database Impact	Application Impact	Scheduled By	Performed By
Exadata Network Fabric Switches	Zero downtime with No Database Restart	Zero to single-digit seconds brownout	Oracle schedules based on customer preferences and customer can reschedule	Oracle Cloud for both ADB and non-ADB

Table 35-1 (Cont.) Availability and Performance Impact for Oracle Exadata Cloud Software Updates

Software Update	Database Impact	Application Impact	Scheduled By	Performed By
Exadata Storage Servers	Zero downtime with No Database Restart	Zero to single-digit seconds brownout Exadata storage servers are updated in rolling manner maintaining redundancy Oracle Exadata System Software pre-fetches the secondary mirrors of the OLTP data that is most frequently accessed into the flash cache, maintaining application performance during storage server restarts Exadata smart flash for database buffers is maintained across storage server restart With Exadata 21.2 software, Persistent Storage Index and Persistent Columnar Cache features enable consistent query performance after a storage server software update	Oracle schedules based on customer preferences and customer can reschedule	Oracle Cloud for both ADB and non-ADB
Exadata Database Host - Monthly Infrastructure Security Maintenance	Zero downtime with No Host or Database Restart	Zero downtime	Oracle schedules and customer can reschedule	Oracle Cloud for both ADB and non-ADB
Exadata Database Host - Quarterly Infrastructure Maintenance	Zero downtime with Oracle RAC rolling updates	Zero downtime Exadata Database compute resources are reduced until planned maintenance completes	Oracle schedules based on customer preferences and customer can reschedule	Oracle Cloud for both ADB and non-ADB
Exadata Database Guest	Zero downtime with Oracle RAC rolling updates	Zero downtime Exadata Database compute resources are reduced until planned maintenance completes	Customer for ADB	Oracle Cloud for ADB Customer using Oracle Cloud Console/APIs for non-ADB

Table 35-1 (Cont.) Availability and Performance Impact for Oracle Exadata Cloud Software Updates

Software Update	Database Impact	Application Impact	Scheduled By	Performed By
Oracle Database quarterly update or custom image update	Zero downtime with Oracle RAC rolling updates	Zero downtime Exadata Database compute resources are reduced until planned maintenance completes Special consideration is required during rolling database quarterly updates for applications that use database OJVM. See KB137197 for details.	Customer for ADB	Oracle Cloud for ADB. For ADB-D, standby-first patch practices are automatically applied. Customer using Oracle Cloud Console/APIs or dbaascli utility for non-ADB. In-place via database home patch, and out-of-place via database move, software updates exist. Works for Data Guard and standby databases (See KB50142)
Oracle Grid Infrastructure quarterly update or upgrade	Zero downtime with Oracle RAC rolling updates	Zero downtime Exadata Database compute resources are reduced until planned maintenance completes	Customer for ADB	Oracle Cloud for ADB Customer using Oracle Cloud Console/APIs or dbaascli utility for non-ADB
Oracle Database upgrade with downtime	Minutes to Hour(s) downtime	Minutes to Hour(s) downtime	Customer for ADB	Oracle Cloud for ADB Customer using Oracle Cloud Console/APIs or dbaascli utility for non-ADB Works for Data Guard and standby databases (see KB70571)
Oracle Database upgrade with near zero downtime	Minimal downtime with DBMS_ROLLING, Oracle GoldenGate replication, or with pluggable database relocate	Minimal downtime with DBMS_ROLLING, Oracle GoldenGate replication, or with pluggable database relocate	Customer for non-ADB	Oracle Cloud for ADB on Shared Exadata Infrastructure (ADB-S) can run pluggable database relocate for upgrade use cases Customer using dbaascli for non-autonomous leveraging DBMS_ROLLING. See KB52602 Customer using generic Maximum Availability Architecture best practices for non-ADB

Exadata cloud systems have many elastic capabilities that can be used to adjust database and application performance needs. By rearranging resources on need, you can maximize system resources to targeted databases and applications and you can minimize costs. The following table lists elastic Oracle Exadata Cloud Infrastructure and VM Cluster updates, and the impacts associated with those updates on databases and applications. All of these operations can be performed using Oracle Cloud Console or APIs unless specified otherwise.

Table 35-2 Availability and Performance Impact for Exadata Elastic Operations

VM Cluster Changes	Database Impact	Application Impact
Scale Up or Down VM Cluster Memory	Zero downtime with Oracle RAC rolling updates	Zero to single-digit seconds brownout
Scale Up or Down VM Cluster CPU	Zero downtime with No Database Restart	Zero downtime Application performance and throughput can be impacted by available CPU resources
Scale Up or Down (resize) ASM Storage for Database usage	Zero downtime with No Database Restart	Zero downtime Application performance might be minimally impacted.
Scale Up VM Local /u02 File System Size (Exadata X8M and later systems)	Zero downtime with No Database Restart	Zero downtime
Scale Up VM Local /u02 File System Size (Exadata X8 and earlier systems)	Zero downtime with Oracle RAC rolling updates	Zero to single-digit seconds brownout
Scale Down VM Local /u02 File System Size	Zero downtime with Oracle RAC rolling updates for scaling down	Zero to single-digit seconds brownout
Adding Exadata Storage Cells	Zero downtime with No Database Restart	Zero to single-digit seconds brownout Application performance might be minimally impacted
Adding Exadata Database Servers	Zero downtime with No Database Restart	Zero to single-digit seconds brownout Application performance and throughput may increase by adding Oracle RAC instances and CPU resources
Adding/Dropping Database Nodes in Virtual Machines (VMs) Cluster	Zero downtime with No Database Restart	Zero to single-digit seconds brownout Application performance and throughput may increase or decrease by adding or dropping Oracle RAC instances and CPU resources

Because some of these elastic changes may take significant time, and may impact available resources for your application, some planning is required.

Note that “scale down” and “drop” changes will decrease available resources. Care must be taken to not reduce resources below the amount required for database and application stability and to meet application performance targets. Refer to the following table for estimated timings and planning recommendations.

Table 35-3 Customer Planning Recommendations for Exadata Elastic Operations

VM Cluster Changes	Estimated Timings	Customer Planning Recommendations
Scale Up or Down VM Cluster Memory	Time to drain services and Oracle RAC rolling restart Typically 15-30 minutes per node, but may vary depending on application draining	Understanding application draining. See Achieving Continuous Availability For Your Applications Before scaling down memory, ensure that database SGAs can still be stored in hugepages, and that application performance is still acceptable. To preserve predictable application performance and stability: • Monitor and scale up before important high workload patterns require the memory resources • Avoid memory scale down unless all your Databases' SGA and PGA memory fit into the new memory size and that all SGAs are accommodated by system's hugepages.
Scale Up or Down VM Cluster CPU	Online operation, typically less than 5 minutes per VM cluster. Scaling up from a very low value to very high value (10+ oCPU increase) may take 10 minutes.	To preserve predictable application performance and stability: • Monitor and scale up before important high workload patterns require the CPU resources or when consistently reaching an OCPU threshold for tolerated amount of time. • Only scale down if the load average is below a threshold for at least 30 minutes or scale down based on fixed workload schedules (e.g. business hours with 60 OCPUs, non-business hours with 10 OCPUs and batch with 100 oCPUs) • Avoid more than one scale down requests within 2 hours period

Table 35-3 (Cont.) Customer Planning Recommendations for Exadata Elastic Operations

VM Cluster Changes	Estimated Timings	Customer Planning Recommendations
Scale Up or Down (resize) ASM Storage for Database usage	Time varies based on utilized database storage capacity and database activity. The higher percentage of utilized database storage, the longer the resize operation (which includes ASM rebalance) will take. Typically minutes to hours.	Oracle ASM rebalance is initiated automatically. Storage redundancy is retained. Due to inherent best practices of using non-intrusive ASM power limit, application workload impact is minimal. Choose a non-peak window so resize and rebalance operations can be optimized. Since the time may vary significantly, plan for the operation to complete in hours. To estimate the time that an existing resize or rebalance operation per VM cluster, query GV\$ASM_OPERATION. For example, a customer can run the following query every 30 minutes to evaluate how much work (EST_WORK) and how much more time (EST_MINUTES) potentially is required: select operation, pass, state, sofar, est_work, est_minutes from gv\$asm_operation where operation='REBAL'; Note the estimated statistics tend to become more accurate as the rebalance progresses but can vary based on the concurrent workload.
Scale Up VM Local /u02 File System Size (Exadata X8M and later)	Online operation, typically less than 5 minutes per VM cluster	VM local file system space is allocated on local database host disks, which is shared by all VM guests for all VM clusters provisioned on that database host. Do not scale up space for Local /u02 File System unnecessarily on one VM cluster such that no space remains to scale up on other VM clusters on the same Exadata Infrastructure because Local /u02 File System scale down must be performed in a RAC rolling manner, which may cause application disruption.
Scale Up VM Local /u02 File System Size (Exadata X8 and earlier)	Time to drain services and Oracle RAC rolling restart. Typically 15-30 minutes per node, but may vary depending on application draining settings.	Understanding application draining. See Achieving Continuous Availability For Your Applications
Scale Down VM Local /u02 File System Size	Time to drain services and Oracle RAC rolling restart. Typically 15-30 minutes per node, but may vary depending on application draining settings.	Understanding application draining See Achieving Continuous Availability For Your Applications

Table 35-3 (Cont.) Customer Planning Recommendations for Exadata Elastic Operations

VM Cluster Changes	Estimated Timings	Customer Planning Recommendations
Adding Exadata Storage Cells	Online operation to create more available space for administrator to choose how to distribute. Typically 3-72 hours per operation depending number of VM clusters, database storage usage and storage activity. With very active database and heavy storage activity, this can take up to take 72 hours. As part of the add storage cell operation, there are two parts to this operation. 1) storage is added to the system as part the add storage, 2) administrator needs to decide which VM cluster to expand its ASM disk groups as a separate operation.	Plan to add storage when your storage capacity utilization will hit 80% within a month's time since the operation may complete in days. Oracle ASM rebalance is initiated automatically. Storage redundancy is retained. Due to inherent best practices of using non-intrusive ASM power limit, application workload impact is minimal. Since the time may vary significantly, plan for the operation to complete in days before the storage is available. To estimate the time that an existing resize or rebalance operation per VM cluster, query GV\$ASM_OPERATION. For example a customer can run the following query every 30 minutes to evaluate how much work (EST_WORK) and how much more time (EST_MINUTES) potentially is required: <pre>select operation, pass, state, sofar, est_work, est_minutes from gv\$asm_operation where operation='REBAL';</pre> Note the estimated statistics tend to become more accurate as the rebalance progresses, but can vary based on the concurrent workload.
Adding Exadata Database Servers	Online operation to expand your VM cluster. One step process to add the Database Compute to the ExaDB-D and then expand the VM cluster. Approximately 1 to 6 hours per Exadata Database Server	Plan to add Database Compute when your Database resource utilization will hit 80% within a month's time. Be aware and plan for this operation to take many hours to a day. Choose a non-peak window so that the add Database Compute operation can complete faster Each Oracle RAC database registered by Oracle Clusterware and visible in the Oracle Cloud Console is extended. If a database was configured outside the Oracle Cloud Console or without dbaascli, then those databases will not be extended.
Adding/Dropping Database Nodes in Virtual Machines (VMs) Cluster	Zero database downtime when adding Database Nodes in VM cluster typically takes 3-6 hours, depending on the number of databases in the VM cluster Zero database downtime with dropping Database Nodes in VM cluster typically takes 1-2 hours, depending on number of databases in the VM cluster	Understand that the add/drop operation is not instantaneous, and operation may take several hours to complete Drop operation reduces Database compute, OCPU and memory resources, so application performance can be impacted

Achieving Continuous Availability For Your Applications

As part of Oracle Exadata Database Service (ExaDB-D and ExaDB-C@C) all software updates (except for non-rolling database upgrades or non-rolling patches) can be done online or with Oracle RAC rolling updates to achieve continuous database up time. Furthermore, any local failures of storage, Exadata network, or Exadata database server are managed automatically, and database up time is maintained.

To achieve continuous application up time during Oracle RAC switchover or failover events, follow these application-configuration best practices:

- Use Oracle Clusterware-managed database services to connect your application. For Oracle Data Guard environments, use role based services.
- Use recommended connection string with built-in timeouts, retries, and delays, so that incoming connections do not see errors during outages.
- Configure your connections with Fast Application Notification.
- Drain and relocate services. Refer to the table below and use recommended best practices that support draining, such as test connections, when borrowing or starting batches of work, and return connections to pools between uses.
- Leverage Application Continuity or Transparent Application Continuity to replay in-flight uncommitted transactions transparently after failures.

For more details on the above checklist, see [Continuous Availability for Applications](#). Oracle recommends testing your application readiness by following [Validating Application Failover Readiness \(KB59878\)](#).

Depending on the Oracle Exadata Database Service planned maintenance event, Oracle attempts to automatically drain and relocate database services before stopping any Oracle RAC instance. For OLTP applications, draining and relocating services typically work very well and result in zero application downtime.

Some applications, such as long running batch jobs or reports, may not be able to drain and relocate gracefully within the maximum draining time. For those applications, Oracle recommends scheduling the software planned maintenance window around these types of activities or stopping these activities before the planned maintenance window. For example, you can reschedule a planned maintenance window to run outside your batch windows, or stop batch jobs before a planned maintenance window.

Special consideration is required during rolling database quarterly updates for applications that use database OJVM. See [RAC Rolling Install Process for the "Oracle JavaVM Component Database PSU/RU" \(OJVM PSU/RU\) Patches \(KB137197\)](#) for details.

The following table lists planned maintenance events that perform Oracle RAC instance rolling restart, and the relevant service drain timeout variables that may impact your application.

Table 35-4 Application Drain Attributes for Exadata Cloud Software Updates and Elastic Operations

Oracle Exadata Database Service Software Updates or Elastic Operations	Drain Timeout Variables
Oracle DBHOME patch apply and database MOVE	Oracle Cloud software automation stops/relocates database services while honoring drain_timeout settings defined by database service configuration (for example, srvctl).¹ You can override drain_timeout defined on services by using option – drainTimeoutInSeconds with command line operation dbaascli dbHome patch or dbaascli database move. The Oracle Cloud internal maximum draining time supported is 2 hours.
Oracle Grid Infrastructure (GI) patch apply and upgrade	Oracle Cloud software automation stops/relocates database services while honoring drain_timeout settings defined by database service configuration (for example, srvctl).¹ You can override drain_timeout defined on services by using option – drainTimeoutInSeconds with command line operation dbaascli grid patch or dbaascli grid upgrade. The Oracle cloud internal maximum draining time supported is 2 hours.
Virtual machine operating system software update (Exadata Database Guest)	Exadata patchmgr/dbnodeupdate software program calls drain orchestration (rhphep). Drain orchestration has the following drain timeout settings (See Using RHPHelper to Minimize Downtime During Planned Maintenance on Exadata (KB146644) for details): • DRAIN_TIMEOUT – if a service does not have drain_timeout defined, then this value is used. Default value is 180 seconds. • MAX_DRAIN_TIMEOUT - overrides any higher drain_timeout value defined by database service configuration. Default value is 300 seconds. There is no maximum value. DRAIN_TIMEOUT settings defined by database service configuration are honored during service stop/relocate.

Table 35-4 (Cont.) Application Drain Attributes for Exadata Cloud Software Updates and Elastic Operations

Oracle Exadata Database Service Software Updates or Elastic Operations	Drain Timeout Variables
Exadata X8 and earlier systems - Scale up and down VM local /u02 file system size - Scale up or down VM cluster memory	Exadata X8 and earlier systems local file system resize operation calls drain orchestration (rhphe lper). Drain orchestration has the following drain timeout settings (See Using RHPHelper to Minimize Downtime During Planned Maintenance on Exadata (KB146644) for details): - DRAIN_TIMEOUT – if a service does not have drain_timeout defined, then this value is used. Default value is 180 seconds. - MAX_DRAIN_TIMEOUT - overrides any higher drain_timeout value defined by database service configuration. Default value is 300 seconds. DRAIN_TIMEOUT settings defined by database service configuration are honored during service stop/relocate. The Oracle Cloud internal maximum draining time supported for this operation is 300 seconds.
Exadata X8M and later systems Scale down VM local file system size	Exadata X8M and later systems call drain orchestration (rhphe lper). Drain orchestration has the following drain timeout settings (See Using RHPHelper to Minimize Downtime During Planned Maintenance on Exadata (KB146644) for details): - DRAIN_TIMEOUT – if a service does not have drain_timeout defined, then this value is used. Default value is 180 seconds. - MAX_DRAIN_TIMEOUT - overrides any higher drain_timeout value defined by database service configuration. Default value is 300 seconds. DRAIN_TIMEOUT settings defined by database service configuration are honored during service stop/relocate. The Oracle Cloud internal maximum draining time supported for this operation is 300 seconds.

Table 35-4 (Cont.) Application Drain Attributes for Exadata Cloud Software Updates and Elastic Operations

Oracle Exadata Database Service Software Updates or Elastic Operations	Drain Timeout Variables
Exadata X8M and later systems Scale up or down VM cluster memory	Exadata X8M and later systems call drain orchestration (rhpelper). Drain orchestration has the following drain timeout settings (See Using RHPelper to Minimize Downtime During Planned Maintenance on Exadata (KB146644) for details): - DRAIN_TIMEOUT – if a service does not have drain_timeout defined, then this value is used. Default value is 180 seconds. - MAX_DRAIN_TIMEOUT - overrides any higher drain_timeout value defined for a given service, default 300. DRAIN_TIMEOUT settings defined by database service configuration are honored during service stop/relocate. The Oracle Cloud internal maximum draining time supported for this operation is 300 seconds.

Table 35-4 (Cont.) Application Drain Attributes for Exadata Cloud Software Updates and Elastic Operations

Oracle Exadata Database Service Software Updates or Elastic Operations	Drain Timeout Variables
Oracle Exadata Cloud Infrastructure (ExaDB-D) software update	The ExaDB-D database host calls drain orchestration (rhphe1per). Drain orchestration has the following drain timeout settings (See Using RHPHelper to Minimize Downtime During Planned Maintenance on Exadata (KB146644) for details): • DRAIN_TIMEOUT – if a service does not have drain_timeout defined, then this value is used. Default value is 180 seconds. • MAX_DRAIN_TIMEOUT - overrides any higher drain_timeout value defined by database service configuration. Default value is 300 seconds. DRAIN_TIMEOUT settings defined by database service configuration are honored during service stop/relocate. The Oracle Cloud internal maximum draining time supported for this operation is • For Exadata X8 and earlier systems, the timeout is 300 seconds. • For Exadata X8M and later systems, the timeout is 500 seconds. Enhanced Infrastructure Maintenance Controls feature: To achieve draining time longer than the Oracle Cloud internal maximum, leverage the custom action capability of the Enhanced Infrastructure Maintenance Controls feature, which allows you to suspend infrastructure maintenance before the next database server update starts, then directly stop/relocate database services running on the database server, and then resume infrastructure maintenance to proceed to the next database server. This feature is also currently available for Oracle Exadata Cloud@Customer (ExaDB-C@C). See Configure Oracle-Managed Infrastructure Maintenance in Oracle Cloud Infrastructure Documentation for details.

¹ Minimum software requirements to achieve this service drain capability is 1) Oracle Database 12c (12.2) and later and 2) the latest Oracle Cloud DBaaS tooling software.

Oracle Maximum Availability Architecture Reference Architectures in Oracle Exadata Cloud

Oracle Exadata Database Service (ExaDB-D, ExaDB-C@C, and ExaDB-XS) supports all Oracle Maximum Availability Architecture reference architectures, providing support for all Oracle databases, regardless of their specific high availability, data protection, and disaster

recovery service-level agreements. See [MAA Best Practices for the Oracle Cloud](#) for more information about Oracle Maximum Availability Architecture in the Oracle Exadata Cloud.

Oracle Maximum Availability Architecture for Multicloud

Oracle Maximum Availability Architecture (MAA) in Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D) running within Oracle Multicloud data centers ensures inherent high availability, data protection, and scalability, including zero database downtime for software updates and elastic operations.

Oracle Multicloud partners include Oracle Database@Azure, Oracle Database@Google Cloud, and Oracle Database@AWS.

MAA Multicloud Certification Matrix

The following table indicates which MAA Reference Architectures are certified on each Oracle Database cloud service with Oracle Multicloud partners.

Multicloud Database Service	MAA Bronze	MAA Silver	MAA Gold - Cross-AZ	MAA Gold - Cross Region	MAA Platinum	MAA Diamond
Oracle ExaDB-D@Azure	Certified	Certified	Certified	Certified	Certified with Oracle Active Data Guard on Oracle AI Database 26ai	Planned
Oracle ExaDB-XS@Azure	Certified	Certified	Certified	Certified	Planned	Planned
Oracle ADB-S@Azure	Certified	Certified	Certified	Planned	Planned	Planned
Oracle ADB-D@Azure	Certified	Certified	Certified	Certified	Certified with Oracle Active Data Guard on Oracle AI Database 26ai	Planned
Oracle ExaDB-D@Google Cloud	Certified	Certified	Certified	Certified	Certified with Oracle Active Data Guard on Oracle AI Database 26ai	Planned
Oracle ExaDB-XS@Google Cloud	Certified	Certified	Certified	Certified	Planned	Planned
Oracle ADB-S@Google Cloud	Certified	Certified	Certified	Planned	Planned	Planned

Multicloud Database Service	MAA Bronze	MAA Silver	MAA Gold - Cross-AZ	MAA Gold - Cross Region	MAA Platinum	MAA Diamond
Oracle ExaDB-D@AWS	Certified	Certified	Certified	Certified	Certified with Oracle Active Data Guard on Oracle AI Database 26ai	Planned
Oracle ADB-D@AWS	Certified	Certified	Certified	Certified	Certified with Oracle Active Data Guard on Oracle AI Database 26ai	Planned

MAA Evaluations on Multicloud Solutions

Oracle Maximum Availability Architecture (MAA) validates the MAA Silver, MAA Gold, and MAA Platinum reference architectures with its Oracle Multicloud partners.

- **MAA Silver** - The validated MAA Silver solution encompasses the best high availability (HA) database platform, which includes configuration and lifecycle best practices covering Oracle database creation, elastic operations, software updates, and backup and restore. All applications can leverage this solution without any changes.
- **MAA Gold** - The validated MAA Gold solution achieves comprehensive data protection and disaster recovery (DR) when augmented with an Oracle Cloud standby database configured with Oracle Active Data Guard. MAA Gold is the best and simplest cloud solution for mission-critical enterprise databases and applications to achieve maximum resiliency in the case of a local or widespread unexpected outage, without requiring application changes.
- **MAA Platinum Option 1 (Oracle AI Database 26ai and Oracle Active Data Guard)**: This validated MAA Platinum solution builds on top of MAA Gold, but requires Oracle AI Database 26ai, as opposed to Oracle Database 19c, providing comprehensive data protection and disaster recovery (DR) when augmented with an Oracle Cloud standby database configured with Oracle Active Data Guard. Another key requirement is using Exadata Database Service on Dedicated Infrastructure (ExaDB-D) to meet the highest platform HA and potentially lowest application impact for local failures. This MAA Platinum solution is the optimal and simplest cloud solution for mission-critical enterprise databases and applications to achieve maximum resiliency in the case of a local or widespread unexpected outage, without requiring application changes.
- **MAA Platinum Option 2 (Oracle GoldenGate)**: When augmented with Oracle GoldenGate or OCI GoldenGate Marketplace and MAA life cycle best practices, this validated MAA Platinum solution achieves a Never Down architecture, because you can direct the application to any one of the available logical GoldenGate replicas for any outage scenario. This MAA Platinum solution can be implemented with minimum application changes other than routing work to avoid logical transaction conflicts and make logical conflict resolution a rare case.

See [Oracle Cloud: Maximum Availability Architecture](#) for a detailed walk-through of Oracle Cloud MAA architectures and features.

Oracle Database Multicloud Evaluations by Oracle MAA

To ensure success and consistency for Oracle customers, the Oracle MAA solution team evaluates MAA reference architectures and key life cycle operations on the actual Oracle Database in Multicloud environments. The goal is to provide assurances using chaos engineering validation methods.

- Expected RTO and RPO are achieved with 100s of different unplanned outages (gray and complex failures) and planned maintenance activities
- Expected performance impact brownout observed with software updates and elastic operations
- Configuration best practices are in place at deployment, and configuration health checks continue to work as expected
- Key life cycle practices work as expected

When the validation of the MAA solution in our Multicloud environments meets the minimum requirements, Oracle MAA documents the MAA architecture, RTO and RPO observations, and network and performance considerations.

Network Evaluation

An MAA network evaluation of Oracle Database Multicloud solutions consists of the following:

Network Considerations

Network considerations such as network bandwidth and network latency using OCI peered network or Multicloud peered network

- Application to database VM
- Database VM to database VM within the same Availability Domains (ADs) or Availability Zones (AZs)
- Database VM to database VM across ADs or AZs
- Database VM to database VM across regions
- Database VM to database VM across Availability Domain or Availability Zone or cross-region

Network Test Prerequisites

Latency and throughput are measured by `iperf3` and `qperf` (installed on Exadata by default).

- Ensure that the ingress and egress security rules on each OCI VCN allow for port 5201, the default port for `iperf3`, which will also be used for `qperf` in these tests. See [Updating a Security List](#) for more information about security rules.
- Record the Virtual IP addresses (VIP) for the database server VMs that will be tested.

On each database server VM being tested run:

```
as grid (sudo su - grid from opc user):
```

```
$ srvctl config vip -n $(hostname -s)
VIP exists: network number 1, hosting node <hostname>
VIP Name: <VIP name>
```

VIP IPv4 Address: <VIP> ← record this IP address for each host involved

in the tests

VIP IPv6 Address:

VIP is enabled.

VIP is individually enabled on nodes:

VIP is individually disabled on nodes:

Network Tests

- Run the following tests multiple times to ensure consistency.
- It is recommended that the tests be run at different times of the day to ensure consistency throughout.
- For database server VMs, these tests should minimally be performed between one VM of the primary cluster and one VM of the standby database cluster. Additional tests can be performed on all database servers to ensure consistency.
- All iperf3 and qperf commands should be run as root (sudo su - from opc user).
- For iperf3 tests, you can optionally use the -f M parameter to display bitrate results in MB/s.

Single Process Throughput Tests

1. On the standby database server VM run:

```
# iperf3 -s
```

2. On the primary database server VM run the following test multiple times to ensure consistency.

```
# iperf3 -c <remote VIP>
```

3. Record the SUM of the sender at the end of each run.

Sample output:

```
Connecting to host <remote VIP>, port 5201
[ 5] local <local IP> port 49230 connected to <remote VIP> port 5201
[ ID] Interval          Transfer      Bitrate      Retr  Cwnd
[ 5]  0.00-1.00    sec    474 MBytes  3.97 Gbits/sec  1656   2.70 MBytes

[ 5]  1.00-2.00    sec    235 MBytes  1.97 Gbits/sec    0   3.06 MBytes
[ 5]  2.00-3.00    sec    266 MBytes  2.23 Gbits/sec    0   3.44 MBytes
[ 5]  3.00-4.00    sec    264 MBytes  2.21 Gbits/sec   61   2.73 MBytes
[ 5]  4.00-5.00    sec    240 MBytes  2.01 Gbits/sec    0   3.09 MBytes
[ 5]  5.00-6.00    sec    268 MBytes  2.24 Gbits/sec    0   3.43 MBytes
[ 5]  6.00-7.00    sec    266 MBytes  2.23 Gbits/sec   76   1.87 MBytes
[ 5]  7.00-8.00    sec    169 MBytes  1.42 Gbits/sec    0   2.25 MBytes
[ 5]  8.00-9.00    sec    199 MBytes  1.67 Gbits/sec    0   2.61 MBytes
[ 5]  9.00-10.00   sec    229 MBytes  1.92 Gbits/sec    0   2.99 MBytes
- - - - -
[ ID] Interval          Transfer      Bitrate      Retr
[ 5]  0.00-10.00   sec    2.55 GBytes  2.19 Gbits/sec  1793
sender <---- USE THIS VALUE
[ 5]  0.00-10.01   sec    2.54 GBytes  2.18 Gbits/sec                      receiver
```

Multi-Process Throughput Tests

For multiple processes, evaluate different degrees of parallelism 4, 10, and 16.

1. On the standby database server VM run:

```
# iperf3 -s
```

2. On the primary database server VM run each of the following tests multiple times to ensure consistency.

```
# iperf3 -c <remote VIP> -P 4
```

```
# iperf3 -c <remote VIP> -P 10
```

```
# iperf3 -c <remote VIP> -P 16
```

3. Record the SUM of the sender at the end of each run. For example:

```
<...>
```

```
[SUM] 0.00-10.00 sec 10.1 GBytes 8.71 Gbits/sec 2159  
sender
```

4. Repeat these tests in both directions (server on primary VM) to ensure throughput.

Note

High degrees of parallelism (for example, above 16) do not show a representative measure of throughput with iperf3. It is not recommended that you use higher parallelism in the tests.

Latency Tests

1. On the standby database server VM run:

```
# qperf -lp 5201
```

2. On the primary database server VM run the following test multiple times to ensure consistency.

```
# qperf 10.255.0.118 -lp 5201 tcp_lat
```

Note

qperf measures latency in only one direction. Run this test in both directions and sum the values for round trip time (RTT).

Network Results Impact on Application and Database Workload and Architecture

- **Application performance and network round trip latency:** If your application is OLTP or dependent on low latency between the application VM and the database VM, co-locate

your application VM with the database VM and ensure that RTT latency meets your response time requirements.

- **Local standby database and network round trip latency:** If you are considering a zero data loss SYNC transport configuration between the primary and standby in the same region, round trip (RTT) network latency over 3 milliseconds (<1ms preferred) can impact the response time and throughput of the application.

Most standby databases using SYNC transport have RTT network latency < 1 ms. A Globally Distributed Database (sharded database) also benefits from low RTT latency between database shards.

- **Sufficient single process network throughput to support peak database change rate for standby:** A single process iperf throughput test must exceed the maximum throughput of each database instance's peak redo rate, preferably by 20% or more, to handle growth.

If this throughput is not met, once the standby is configured, it may be unable to keep up with the primary database during peak times. Single process throughput is also critical for Oracle Golden Gate and Globally Distributed Database configurations.

Single process throughput should be at least 2.4 Gb/sec (300 MB/sec). If not, log a service request with the multi-vendor partner. With OCI peering, the expectation is that throughput will be much higher than the aforementioned minimum.

- **Sufficient multiple-process network throughput to high data volume transfers:** The parallel iperf test indicates approximately how fast the database can be instantiated with the tested number of processes.

For example, for a 50 TB database, if the test with a degree of parallelism of 8 achieves 1GB/s, a database with eight processes will take over 14 hours to instantiate. The default instantiation uses four channels; a service request can be opened with Oracle Support to increase the degree of parallelism for instantiation.

Multiple-process throughput should be at least 8 Gb/sec (1000 MB/sec). If not, log a service request with the multi-vendor partner. With OCI peering, the expectation is that throughput will be much higher than the aforementioned minimum. Higher latency means a longer delay in data transmission, which can result in lower throughput even if the network speed is high. Other factors resulting in lower throughput include packet loss and congestion.

MAA Silver Architecture and Evaluation

MAA Silver on Oracle Database in a Multicloud solution consists of the following architecture:

- The Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D) cluster residing in the Multicloud partner's data center
- High Availability (HA) and redundant application tier spread across multiple AZs
- Key Management Service, Autonomous Recovery Service (in Multicloud data center or OCI), and Object Storage Service (in OCI) for backup and restore
- Pre-configured redundant and HA network topology

MAA Silver evaluation consists of:

- Network tests from the application VM to database VM, then database VM to backup target solution

- Testing backup and restore performance, throughput, and key use cases using Oracle Cloud Infrastructure (OCI) Object Storage Service (in OCI), or Autonomous Recovery Service in Multicloud data center or OCI
- Setting up an application workload and MAA framework to simulate 100s of different unplanned (blackout and brownout) outages
- Ensuring RTO and RPO are met for local failures
- Ensuring zero downtime or expected brief application brownout for system and database elastic changes or software updates can be met
- Ensuring that the exachk or configuration health checks work as expected, and that the deployed system is MAA compliant

MAA Gold Architecture and Evaluation

MAA Gold on Oracle Database in a Multicloud solution consists of the following architecture:

- Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D) VM clusters (primary and standby databases) reside in the same or separate availability domains (AD) or availability zones (AZ) or separate Multicloud regions

Note that all primary and standby databases and their data reside in Oracle's Multicloud partner's data centers. Assuming primary and standby databases reside in the same AD or AZ, the MAA Gold architecture still provides inherent HA benefits plus DR failover options for database and cluster failures. However, this configuration lacks DR protection for a complete AD or AZ site failure, such as a regional power outage. The MAA architecture only provides regional failure protection if the standby database resides in a separate region.

- HA and redundant application tiers are spread across multiple AZs or regions
- Key Management Service, Autonomous Recovery Service (in Multicloud data center or OCI), and Object Storage Service (in OCI) for backup and restore
- Pre-configured redundant and HA network topology

MAA Gold evaluation builds upon the MAA Silver evaluation, plus:

- Network tests between primary and standby database clusters using OCI peered or Multicloud peered networks to evaluate round-trip latency and bandwidth
- Oracle Data Guard role transition performance and timings for disaster recovery use cases
- Oracle database rolling upgrade with Data Guard

MAA Platinum Architecture and Evaluation

MAA Platinum on Oracle AI Database 26ai with Oracle Active Data Guard with in a Multicloud solution consists of the following architecture:

- Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D) VM clusters (primary and standby databases) residing in the same or separate availability domains (AD) or availability zones (AZ), or separate Multicloud regions

Note that all primary and standby databases and their data reside in Oracle's Multicloud partner's data centers. Assuming primary and standby databases reside in the same AD or AZ, the MAA Platinum architecture still provides inherent HA benefits plus DR failover options for database and cluster failures. However, this configuration lacks DR protection for a complete AD or AZ site failure, such as a regional power outage. The MAA

architecture only provides regional failure protection if the standby database or additional standby database resides in a separate region.

- HA and redundant application tiers spread across multiple AZs or regions
- Key Management Service, Autonomous Recovery Service (in Multicloud data center or OCI), and Object Storage Service (in OCI) for backup and restore
- Pre-configured redundant and HA network topology
- Oracle AI Database 26ai required

MAA Platinum on Oracle AI Database 26ai with Oracle Active Data Guard evaluation builds upon the MAA Gold evaluation, plus:

- Network tests between primary and standby database clusters using OCI peered, or Multicloud peered networks, to evaluate round-trip latency and bandwidth
- Oracle Data Guard role transition performance and timings for disaster recovery use cases
- Database rolling upgrade with Active Data Guard

Oracle Maximum Availability Architecture Benefits

Once certified, the following are some benefits of implementing Oracle MAA reference architectures for Oracle Multicloud.

For a comprehensive list of Oracle Maximum Availability Architecture benefits for Oracle Exadata Database Machine systems, see [Exadata Database Machine: Maximum Availability Architecture](#).

Deployment

Oracle Database in a Multicloud solution running Oracle Exadata Database Service on Dedicated Infrastructure is deployed using Oracle Maximum Availability Architecture best practices, including configuration best practices for storage, network, operating system, Oracle Grid Infrastructure, and Oracle Database. Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D) is optimized to run enterprise Oracle databases with extreme scalability, availability, and elasticity.

Oracle MAA Database Templates

All Oracle Cloud databases created with Oracle Cloud automation use Oracle Maximum Availability Architecture default settings, which are optimized for Oracle Database in Multicloud environments. Oracle does not recommend that you use custom scripts to create cloud databases. To migrate Oracle databases to Oracle Database in a Multicloud solution, use Oracle Zero Downtime Migration (ZDM). See [Introduction to Zero Downtime Migration \(oracle.com\)](#).

Other than adjusting memory and system resource settings, avoid migrating previous database parameter settings, especially undocumented parameters. One beneficial primary database data protection parameter, `DB_BLOCK_CHECKING`, is not enabled by default because of its potential performance overhead. Any standby database configured with cloud automation enables `DB_BLOCK_CHECKING` on the standby automatically to maximize data protection and detection on the standby database. MAA recommends evaluating the performance impact of your application and allowing this parameter to be set on the primary database to maximize logical data corruption prevention and detection if the performance impact is reasonable. In Oracle Database releases 19c and later, the Data Guard broker maintains the data protection settings through MAA best practices.

Backup and Restore Automation

Backup copies provide additional protection when you configure automatic backup to Autonomous Recovery Service in Azure, Google Cloud, or OCI, or to Object Storage Service in OCI. Oracle Recovery Manager (RMAN) validates cloud database backups for physical corruptions.

Database backups occur daily, with a full backup occurring once a week, and incremental backups occurring on all other days. Archived log backups occur frequently to reduce potential data loss in case a complete database restoration and recovery is required. The archived log backup frequency is 30 minutes by default; however, the possible data loss will be zero or near zero with Oracle Data Guard.

With Autonomous Recovery Service, weekly full backups are eliminated, reducing backup windows and impact with its unique incremental backup forever benefit. When Real-Time Data Protection is enabled, data loss can be near zero when restoring from backups. Backups can occur on primary or standby databases, and various retention options exist.

Oracle Exadata Database Machine Inherent Benefits

Oracle Exadata Database Machine is the best Oracle Maximum Availability Architecture database platform that Oracle offers. Exadata is engineered with hardware, software, database, availability, and extreme performance for all workloads and scalability innovations supporting mission-critical enterprise applications.

Specifically, Exadata provides unique high availability, data protection, and quality-of-service capabilities that set Oracle apart from any other platform or cloud vendor. Sizing Exadata cloud systems to meet your application and database system resource needs (for example, sufficient CPU, memory, and I/O resources) is crucial to maintaining the highest availability, stability, and performance. Proper sizing and resource management are critical when consolidating many databases on the same cluster. Database consolidation is a widespread benefit when leveraging Exadata.

See [Oracle Maximum Availability Architecture in Oracle Exadata Cloud Systems](#) for details

Expected Impact During Unplanned Outages

The following table lists various unplanned outage events and the associated potential database downtime, application Recovery Time Objective (RTO), and data loss potential or recovery point objective (RPO).

For Oracle Data Guard architectures (MAA Gold), the database or service level downtime does not include detection time or the time it takes before a customer initiates the Cloud Console Data Guard failover operation.

Outage Event	Database Downtime	Service-Level Downtime (RTO)	Potential Service-Level Data Loss (RPO)
Localized events, including: Exadata cluster network topology failures Storage (disk, flash, and storage cell) failures Database instance failures Database server failures	Zero	Near-zero	Zero

Outage Event	Database Downtime	Service-Level Downtime (RTO)	Potential Service-Level Data Loss (RPO)
Events that require restoration from backup because a standby database does not exist: Data corruptions Full database failures Complete storage failures Availability Zone failures	Minutes to hours (without Data Guard)	Minutes to hours (without Data Guard)	Near Zero with Autonomous Recovery Service and Real-Time Data Protection enabled 30 minutes with Autonomous Recovery Service and Real-Time Data Protection disabled, or with Object Storage Service for cloud backups (without Data Guard)
Events using Data Guard to fail over: Data corruptions Full database failures Complete storage failures Availability Zone failures Complete Region failures	Seconds to minutes ¹ Zero downtime for physical corruptions due to the auto-block repair feature	Seconds to minutes ¹ The foreground process that detects the physical corruption pauses while auto-block repair completes	Zero for Maximum Availability (SYNC redo transport) Near Zero for Maximum Performance (ASYNCRedo transport)

¹For MAA Gold, to protect your database from regional failure, instantiate the standby database in a region different from the primary database. For this MAA evaluation, the standby database was in a different AZ. Also, Data Guard Fast-Start Failover and its Data Guard observers must be set up manually to perform automatic database failover. Application workloads as high as 300 MB/second per Oracle Real Application Cluster instance were validated. The standby database was up-to-date with near-zero lag. Depending on the workload, standby database tuning may be required for extreme workloads (see [Tune and Troubleshoot Oracle Data Guard](#)).

Expected Impact During Planned Maintenance

The following tables describe the impact of various planned maintenance events for Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D) on Oracle's multicloud solutions: Oracle Database@Azure, Oracle Database@Google Cloud, and Oracle Database@AWS.

Impact of Exadata Cloud Software Updates

The following table lists various software updates and their impact on the associated database and application. This applies to ExaDB-D on Oracle's multicloud solutions.

Software Update	Database Impact	Application Impact	Scheduled By	Performed By
Exadata Network Fabric Switches	Zero downtime with no database restart	Zero to single-digit seconds brownout	Oracle schedules based on customer preferences, and customers can reschedule	Oracle Cloud Operation

Software Update	Database Impact	Application Impact	Scheduled By	Performed By
Exadata Storage Servers	Zero downtime with no database restart	Zero to single-digit seconds brownout Exadata storage servers are updated in a rolling manner, maintaining redundancy. Oracle Exadata System Software pre-fetches the secondary mirrors of the OLTP data most frequently accessed into the flash cache, maintaining application performance during storage server restarts. Exadata smart flash for database buffers is maintained across a storage server restart. With Exadata 21.2 software, Persistent Storage Index and Persistent Columnar Cache features enable consistent query performance after a storage server software update.	Oracle schedules based on customer preferences, and customers can reschedule	Oracle Cloud Operation
Exadata Database Host Monthly Infrastructure Security Maintenance	Zero downtime with no host or database restart	Zero downtime	Oracle schedules, and customers can reschedule	Oracle Cloud Operation
Exadata Database Host Quarterly Infrastructure Maintenance	Zero downtime with Oracle RAC rolling updates	Zero downtime Exadata Database compute resources are reduced until planned maintenance is completed.	Oracle schedules based on customer preferences, and customers can reschedule	Oracle Cloud Operation
Exadata Database Guest	Zero downtime with Oracle RAC rolling updates	Zero downtime Exadata Database compute resources are reduced until planned maintenance is completed.	Customer	Customers, using Oracle Cloud Console or APIs

Software Update	Database Impact	Application Impact	Scheduled By	Performed By
Oracle Database quarterly update or custom image update	Zero downtime with Oracle RAC rolling updates	Zero downtime Exadata Database compute resources are reduced until planned maintenance is completed. Applications that use database OJVM require special consideration during rolling database quarterly updates (see My Oracle Support Doc ID 2217053.1).	Customer	Customers using Oracle Cloud Console, APIs, or <code>dbaascli</code> utility In-place, with database home patch, and out-of-place with database move (recommended) Works for Data Guard and standby databases (see My Oracle Support Doc ID 2701789.1)
Oracle Grid Infrastructure quarterly update or upgrade	Zero downtime with Oracle RAC rolling updates	Zero downtime Exadata Database compute resources are reduced until planned maintenance is completed.	Customer	Customers, using Oracle Cloud Console, APIs, or <code>dbaascli</code> utility
Oracle Database upgrade with downtime	Minutes to Hour(s) downtime	Minutes to Hour(s) downtime	Customer	Customers, using Oracle Cloud Console, APIs, or <code>dbaascli</code> utility Works for Data Guard and standby databases (see My Oracle Support Doc ID 2628228.1)
Oracle Database upgrade with near-zero downtime	Minimal downtime with DBMS_ROLLING, Oracle GoldenGate replication, or with pluggable database relocate	Minimal downtime with DBMS_ROLLING, Oracle GoldenGate replication, or with pluggable database relocate	Customer	Customers, using <code>dbaascli</code> leveraging DBMS_ROLLING (see My Oracle Support Doc ID 2832235.1) Customers, using generic Maximum Availability Architecture best practices

Impact of Exadata Elastic Operations

Exadata cloud systems have many elastic capabilities that can be used to adjust database and application performance needs. By rearranging resources on need, you can maximize system resources to targeted databases and applications and minimize costs.

The following table lists elastic Oracle Exadata Cloud Infrastructure and VM Cluster updates and the impacts associated with those updates on databases and applications. Unless specified otherwise, these operations can be performed using Oracle Cloud Console or APIs.

VM Cluster Change	Database Impact	Application Impact
Scale Up or Down VM Cluster Memory	Zero downtime with Oracle RAC rolling updates	Zero to single-digit seconds brownout

VM Cluster Change	Database Impact	Application Impact
Scale Up or Down VM Cluster CPU	Zero downtime with no database restart	Zero downtime Available CPU resources can impact application performance and throughput
Scale Up or Down (resize) ASM Storage for Database usage	Zero downtime with no database restart	Zero downtime Application performance might be minimally impacted
Scale Up VM Local /u02 File System Size (Exadata X9M and later systems)	Zero downtime with no database restart	Zero downtime
Scale Down VM Local /u02 File System Size	Zero downtime with Oracle RAC rolling updates for scaling down	Zero to single-digit seconds brownout
Adding Exadata Storage Cells	Zero downtime with no database restart	Zero to single-digit seconds brownout Application performance might be minimally impacted
Adding Exadata Database Servers	Zero downtime with no database restart	Zero to single-digit seconds brownout Adding Oracle RAC instances and CPU resources may improve application performance and throughput
Adding Database Nodes in Virtual Machines (VMs) Cluster	Zero downtime with no database restart	Zero to single-digit seconds brownout Application performance and throughput may increase or decrease by adding or dropping Oracle RAC instances and CPU resources

Planning for the Impact of Exadata Elastic Operations

Because some of the above elastic changes may take significant time and impact your application's available resources, some planning is required.

Note that “scale down” and “drop” changes will decrease available resources. Care must be taken to not reduce resources below the amount required for database and application stability and to meet application performance targets. The following table provides the estimated duration and planning recommendations for these changes.

VM Cluster Change	Database Impact	Application Impact
Scale Up or Down VM Cluster Memory	Time to drain services and Oracle RAC rolling restart Typically 15-30 minutes per node, but may vary depending on application draining	Understanding application draining See Configuring Continuous Availability for Applications before scaling down memory, ensure that database SGAs can still be stored in hugepages, and that application performance is still acceptable. To preserve predictable application performance and stability: • Monitor and scale up before important high workload patterns require the memory resources • Avoid memory scale down unless all of the database SGA and PGA memory fits into the new memory size, and the system's hugepages accommodate all SGAs

VM Cluster Change	Database Impact	Application Impact
Scale Up or Down VM Cluster CPU	Online operation, typically less than 5 minutes for each VM cluster Scaling up from a very low value to a very high value (10+ OCPUs increase) may take 10 minutes.	To preserve predictable application performance and stability: • Monitor and scale up before important high workload patterns require the CPU resources, or when consistently reaching an OCPU threshold for a tolerated amount of time • Only scale down if the load average is below the threshold for at least 30 minutes, or scale down based on fixed workload schedules (such as business hours with 60 OCPUs, non-business hours with 10 OCPUs, and batch with 100 OCPUs) • Avoid more than one scale-down request within a 2 hour period
Scale Up or Down (resize) ASM Storage for Database usage	Typically minutes to hours Time varies based on utilized database storage capacity and database activity. The higher the percentage of utilized database storage, the longer the resize operation (which includes ASM rebalance) will take.	Oracle ASM rebalance is initiated automatically. Storage redundancy is retained. Because of the inherent best practice of using a non-intrusive ASM power limit, application workload impact is minimal. Choose a non-peak window so resize and rebalance operations can be optimized. Because the time may vary significantly, plan for the operation to be completed in hours. To estimate the time that an existing resize or rebalance operation per VM cluster requires, query GV\$ASM_OPERATION. For example, you can run the following query every 30 minutes to evaluate how much work (EST_WORK) and how much more time (EST_MINUTES) potentially is required: <pre>select operation, pass, state, sofar, est_work, est_minutes from gv\$asm_operation where operation='REBAL';</pre> Note that the estimated statistics tend to become more accurate as the rebalance progresses, but can vary based on the concurrent workload.
Scale Up VM Local /u02 File System Size (Exadata X9M and later)	Online operation, typically less than 5 minutes for each VM cluster	VM local file system space is allocated on local database host disks, which are shared by all VM guests for all VM clusters provisioned on that database host. Do not scale up space for Local /u02 File System unnecessarily on one VM cluster, such that no space remains to scale up on other VM clusters on the same Exadata Infrastructure, because a Local /u02 File System scale down must be performed in an Oracle RAC rolling manner, which may cause application disruption.
Scale Down VM Local /u02 File System Size	Time to drain services and Oracle RAC rolling restart Typically 15-30 minutes for each node, but may vary depending on application draining settings.	To plan, learn about application draining at Configuring Continuous Availability for Applications

VM Cluster Change	Database Impact	Application Impact
Adding Exadata Storage Cells	The online operation creates more available space for administrators to choose how to distribute. Typically, 3-72 hours per operation, depending on the number of VM clusters, database storage usage, and storage activity. With a very active database and heavy storage activity, this can take up to 72 hours. As part of the add storage cell operation, there are two parts to this operation: 1. Storage is added to the Exadata system as part of the add storage operation. 2. The administrator must decide which VM cluster to expand its ASM disk groups as a separate operation.	Plan to add storage when your storage capacity utilization hits 80% within a month, because this operation may be completed in days. Oracle ASM rebalance is initiated automatically. Storage redundancy is retained. Because of inherent best practices in using non-intrusive ASM power limits, the impact of application workload is minimal. Because the time duration may vary significantly, plan to complete the operation days before the storage is available. To estimate the time that an existing resize or rebalance operation will take on each VM cluster, query GV\$ASM_OPERATION. For example, you can run the following query every 30 minutes to evaluate how much work (EST_WORK) and how much more time (EST_MINUTES) is potentially required: <pre>Select operation, pass, state, sofar, est_work, est_minutes from gv\$asm_operation where operation='REBAL';</pre> Note that the estimated statistics tend to become more accurate as the rebalance progresses, but can vary based on the concurrent workload.
Adding Exadata Database Servers	Online operation to expand your VM cluster One-step process to add the Database Compute to the Exadata infrastructure and then expand the VM cluster Approximately 1 to 6 hours for each Exadata Database Server	Plan to add Database Compute when your database resource utilization reaches 80% within a month. Be aware, and plan for this operation to take many hours to a day. Choose a non-peak window so that the add Database Compute operation can be completed faster. Each Oracle RAC database registered by Oracle Clusterware and visible in the Oracle Cloud Console is extended. If a database was configured outside the Oracle Cloud Console, or without dbaascli, it will not be extended.
Adding or Dropping Database Nodes in Virtual Machines (VMs) Cluster	Zero database downtime when adding Database Nodes in the VM cluster. Typically takes 3-6 hours, depending on the number of databases in the VM cluster. Zero database downtime when dropping Database Nodes in the VM cluster. Typically takes 1-2 hours, depending on the number of databases in the VM cluster.	Understand that the add/drop operation is not instantaneous, and the operation may take several hours to complete. The drop operation reduces database computing, OCPU, and memory resources so that application performance can be impacted.

Achieving Continuous Availability For Your Applications

As part of Oracle Exadata Database Service on Dedicated Infrastructure on Oracle Database in a Multicloud solution, all software updates (except for non-rolling database upgrades or non-rolling patches) can be done online or with Oracle RAC rolling updates to achieve continuous database uptime.

Furthermore, any local failures of storage, Exadata network, or Exadata database server are managed automatically, and database uptime is maintained.

To achieve continuous application uptime during Oracle RAC switchover or failover events, follow these application-configuration best practices:

- Use Oracle Clusterware-managed database services to connect your application. For Oracle Data Guard environments, use role-based services.
- Use the recommended connection string with built-in timeouts, retries, and delays so that incoming connections do not see errors during outages.
- Configure your connections with Fast Application Notification.
- Drain and relocate services. Use the recommended best practices in the table below that support draining, such as test connections, when borrowing or starting batches of work, and return connections to pools between uses.
- Leverage Application Continuity or Transparent Application Continuity to replay in-flight uncommitted transactions transparently after failures.

For more details, see [Configuring Continuous Availability for Applications](#). Oracle recommends testing your application readiness by following [Validating Application Failover Readiness \(My Oracle Support Doc ID 2758734.1\)](#).

Depending on the Oracle Exadata Database Service planned maintenance event, Oracle attempts to automatically drain and relocate database services before stopping any Oracle RAC instance. For OLTP applications, draining and relocating services typically work very well and result in zero application downtime.

Some applications, such as long-running batch jobs or reports, may not be able to drain and relocate gracefully within the maximum draining time. For those applications, Oracle recommends scheduling the software planned maintenance window around these types of activities or stopping these activities before the planned maintenance window. For example, you can reschedule a planned maintenance window to run outside your batch windows or stop batch jobs before a planned maintenance window.

Special consideration is required during rolling database quarterly updates for applications that use database OJVM. See [My Oracle Support Doc ID 2217053.1](#) for details.

The following table lists planned maintenance events that perform Oracle RAC instance rolling restart, as well as the relevant service drain timeout variables that may impact your application.

Exadata Cloud Software Updates or Elastic Operation	Drain Timeout Variables
Oracle DBHOME patch apply and database MOVE	Cloud software automation stops/relocates database services while honoring DRAIN_TIMEOUT settings defined by database service configuration (such as <code>srvctl</code>).¹ You can override DRAIN_TIMEOUT defined on services using the option <code>drainTimeoutInSeconds</code> with command line operation <code>dbaascli dbHome patch</code> or <code>dbaascli database move</code>. The Oracle Cloud internal maximum draining time supported is 2 hours.

Exadata Cloud Software Updates or Elastic Operation	Drain Timeout Variables
Oracle Grid Infrastructure (GI) patch apply and upgrade	Cloud software automation stops/relocates database services while honoring DRAIN_TIMEOUT settings defined by database service configuration (such as srvctl).¹ You can override the DRAIN_TIMEOUT defined on services using the option <code>drainTimeoutInSeconds</code> with command line operation <code>dbaascli grid patch</code> or <code>dbaascli grid upgrade</code>. The Oracle Cloud internal maximum draining time supported is 2 hours.
Virtual Machine Operating System Software Update (Exadata Database Guest)	Exadata <code>patchmgr/dbnodeupdate</code> software program calls drain orchestration (<code>rhphelper</code>). Drain orchestration has the following drain timeout settings (see My Oracle Support Doc ID 2385790.1 for details): • DRAIN_TIMEOUT – If a service does not have DRAIN_TIMEOUT defined, then the default value of 180 seconds is used. • MAX_DRAIN_TIMEOUT - Overrides any higher DRAIN_TIMEOUT value defined by database service configuration. The default value is 300 seconds. There is no maximum value. The DRAIN_TIMEOUT settings defined by the database service configuration are honored during service stop/relocate.
Exadata X9M and later systems Scale Down VM Local File System Size	Exadata X9M and later systems call drain orchestration (<code>rhphelper</code>). Drain orchestration has the following drain timeout settings (see My Oracle Support Doc ID 2385790.1 for details): • DRAIN_TIMEOUT – If a service does not have DRAIN_TIMEOUT defined, then the default value of 180 seconds is used. • MAX_DRAIN_TIMEOUT - Overrides any higher DRAIN_TIMEOUT value defined by database service configuration. The default value is 300 seconds. The DRAIN_TIMEOUT settings defined by the database service configuration are honored during service stop/relocate. The Oracle Cloud internal maximum draining time supported for this operation is 300 seconds.

Exadata Cloud Software Updates or Elastic Operation	Drain Timeout Variables
Exadata X9M and later systems Scale Up or Down VM Cluster Memory	Exadata X9M and later systems call drain orchestration (rhphe1per). Drain orchestration has the following drain timeout settings (see My Oracle Support Doc ID 2385790.1 for details): • DRAIN_TIMEOUT – If a service does not have DRAIN_TIMEOUT defined, then the default value of 180 seconds is used. • MAX_DRAIN_TIMEOUT - Overrides any higher DRAIN_TIMEOUT value defined by database service configuration. The default value is 300 seconds. The DRAIN_TIMEOUT settings defined by the database service configuration are honored during service stop/relocate. The Oracle Cloud internal maximum draining time supported for this operation is 900 seconds.
Oracle Exadata Cloud Infrastructure (ExaDB) software update	The ExaDB-D database host calls drain orchestration (rhphe1per). Drain orchestration has the following drain timeout settings (see My Oracle Support Doc ID 2385790.1 for details): • DRAIN_TIMEOUT – If a service does not have DRAIN_TIMEOUT defined, then the default value of 180 seconds is used. • MAX_DRAIN_TIMEOUT - Overrides any higher DRAIN_TIMEOUT value defined by database service configuration. The default value is 300 seconds. The DRAIN_TIMEOUT settings defined by the database service configuration are honored during service stop/relocate. The Oracle Cloud internal maximum draining time supported for this operation is 500 seconds. Enhanced Infrastructure Maintenance Controls: To achieve draining time longer than the Oracle Cloud internal maximum, leverage the custom action capability of Enhanced Infrastructure Maintenance Controls, which allows you to suspend infrastructure maintenance before the next database server update starts, directly stop/relocate database services running on the database server, and then resume infrastructure maintenance to proceed to the next database server. See Configure Oracle-Managed Infrastructure Maintenance in Oracle Cloud Infrastructure documentation for details.

¹Minimum software requirements to achieve this service drain capability are: Oracle Database release 12.2 and later and the latest cloud DBaaS tooling software.

Oracle Maximum Availability Architecture for Oracle Database@Azure

Microsoft Azure is a strategic Oracle Multicloud partner. Oracle Maximum Availability Architecture (MAA) evaluates MAA reference architectures on Oracle Database@Azure, the results of which are shown here.

To learn more about MAA Silver, MAA Gold, and MAA Platinum evaluations and their benefits after certification, see [MAA Evaluations on Multicloud Solutions](#).

Oracle MAA evaluated Oracle's solution in Azure. Oracle MAA continues to re-evaluate periodically to ensure the solution meets all the expected benefits and to highlight any new MAA for Oracle Database@Azure benefits and capabilities. Certification is only given after the MAA evaluation meets the requirements.

Oracle MAA has evaluated and endorsed Oracle Database@Azure for:

- MAA Silver reference architecture on Exadata Database Service on Dedicated Infrastructure (ExaDB-D)
- MAA Gold and MAA Platinum reference architectures when the standby database resides on another ExaDB-D in the same or separate Availability Zones (AZ) or different regions

Evaluation environments:

- Cross-region between Ashburn, Virginia, USA, and London, England regions
- Cross-availability zone (AZ) within the Ashburn, Virginia, USA region

Network Results

The results shown in the following use cases are based on network tests described in [Oracle Database Multicloud Evaluations by Oracle MAA](#).

Use Case: Application VMs to Exadata VM Cluster (Same Region)

RTT Latency Observed: Varies based on placement

- 300 microseconds (Cluster in same Availability Zone (AZ))
- 500 microseconds (Cluster cross-AZ)

Network Throughput Observed: Variable based on placement and VM size

MAA Recommendations:

- Ensure your required RTT latency meets your application requirements.
- Test thoroughly with your implementation. Variables such as VM size and placement can impact results.
- Refer to [Application Network Layer on Azure](#)
- See also, [Proximity placement groups](#) and [Sizes for virtual machines in Azure](#)

Use Case: Between Exadata VM Clusters Across AZs (London, England)

Evaluated for:

- Redo Transport
- Database migration or standby database instantiation

RTT Latency Observed: 1.2 milliseconds (for both peering options)

Network Single Process Throughput Observed:

- 13 Gb/sec (OCI VCN Peering (recommended))
- 13 Gb/sec (Azure Global Virtual Network)

Maximum Parallel Throughput:

- 70 Gb/sec (X11M) (OCI VCN Peering (recommended))
- 70 Gb/sec (X11M) (Azure Global Virtual Network)

MAA Recommendations:

- Repeat MAA-recommended network tests in your environment. See [Network Evaluation](#) for examples.
- Recommended to use OCI peering for higher bandwidth (up to 600% in some examples)
- For OCI peering, see
 - Same region: [Local VCN Peering using Local Peering Gateways](#)
 - Cross-region: [Perform Cross-Regional Disaster Recovery for Exadata Database on Oracle Database@Azure](#)
- For Azure peering, see [Virtual network peering](#)
- Variables may impact bandwidth; see [Optimize network throughput for Azure virtual machines](#)

Use Case: Between Exadata VM Clusters Across Regions (Ashburn, Virginia, USA and London, England)

Evaluated for:

- Redo Transport
- Database migration or standby database instantiation

RTT Latency Observed: ~76 milliseconds (for both peering options)

Network Single Process Throughput Observed:

- 1.57 Gb/sec (OCI VCN Peering (recommended))
- 0.85 Gb/sec (Azure Global Virtual Network)

Maximum Parallel Throughput:

- 60 Gb/sec (X11M) (OCI VCN Peering (recommended))
- 45 Gb/sec (X11M) (Azure Global Virtual Network)

MAA Recommendations:

- Repeat MAA-recommended network tests in your environment. See [Network Evaluation](#) test examples
- Use OCI peering for higher bandwidth (up to 600% in some examples)
- For OCI peering, see
 - Same region: [Local VCN Peering using Local Peering Gateways](#)
 - Cross-region: [Perform Cross-Regional Disaster Recovery for Exadata Database on Oracle Database@Azure](#)
- For Azure peering, see [Virtual network peering](#).
- Variables may impact bandwidth; see [Optimize network throughput for Azure virtual machines](#).

Azure Network Recommendations and Considerations

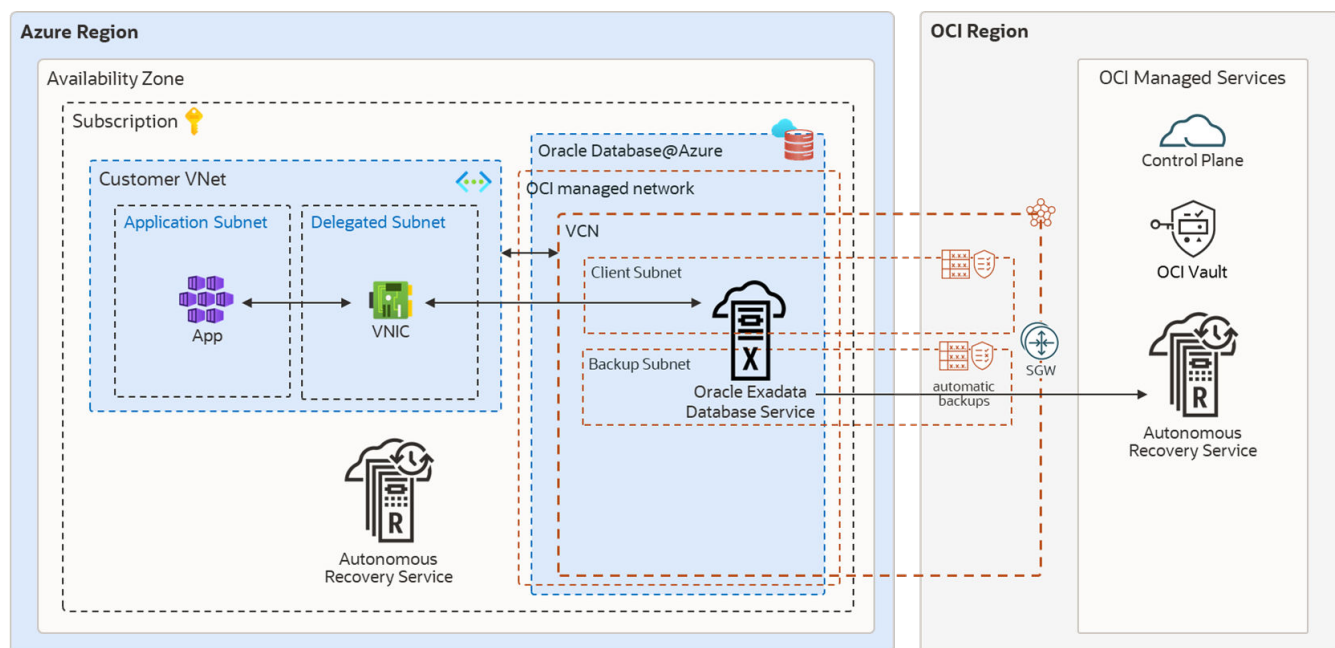
- Network Topology and Connectivity for Oracle Database

- [Get started with Oracle Database@Azure](#)
- [Network planning for Oracle Database@Azure](#)
- Cross Region Azure Peering for Data Guard (OCI Peering Recommended) will be influenced by aggregate throughput.
 - [Edit virtual hub capacity](#)

MAA Silver Network Topology and Evaluation

The following MAA Silver HA Diagram illustrates Oracle RAC on ExaDB-D with backup to Autonomous Recovery Service running in Azure or OCI.

Figure 36-1 MAA Silver Architecture for Oracle Database@Azure



Oracle MAA evaluated and endorsed Oracle Database@Azure for the MAA Silver reference architecture, and observed the following results.

- Application latency may be impacted by the application VM's proximity to the database server target. For the lowest latency, place the application VM in the same Availability Zone (AZ) as the database and use direct communication (not Azure network virtual appliances, firewalls, and so on). For high availability, evaluate placing application VMs in multiple AZs when possible. See Application Network Layer on Azure below and [Network topology and connectivity for Oracle Database@Azure - Getting started](#).
- The backup and restore performance with Autonomous Recovery Service@Azure, Autonomous Recovery Service on OCI, and OSS on OCI meets expectations. See Backup and Restore Observations below.
- Application and database uptime is adhered to as expected while injecting unplanned local outages, updating database and system software, and during system and database elastic changes (for example, increasing CPU, storage, etc). See [Oracle Database Multicloud Evaluations by Oracle MAA](#) for tests evaluated and [Oracle Maximum Availability Architecture in Oracle Exadata Cloud Systems](#) for expected results and benefits.

- Configuration health checks, such as Exachk runs, help ensure MAA compliance. Enabling Database Service "Health" Events and reviewing Exachk output monthly is recommended.

Application Network Layer on Azure

The proximity of the application tier to the database cluster affects application response time.

If you require a very low latency response time (200-400 microseconds), deploy the application VMs in the same Availability Zone (AZ) as the database cluster. Latency increases to 1 millisecond or more when application and database servers are configured across VNets or AZs.

Deploy the application tier over at least two AZs to ensure high availability. The deployment process and solution over multiple AZs varies depending on the application's components, Azure services, and resources involved. For example, with Azure Kubernetes Services (AKS), you can deploy the worker nodes in different AZs. Kubernetes control plane maintains and synchronizes the pods and the workload.

Backup and Restore Observations

RMAN nettest was used for backups, and the expected results were met. See [KB115145](#) for details about nettest.

Oracle Database backup and restore throughput to Oracle's Autonomous Recovery Service or Oracle's Object Storage Service were within performance expectations. For example, an ExaDB-D 2 node cluster (using 16+ OCPUs) and three storage cells may observe a 4 TB/hour backup rate and approximately 8.7 TB/hour restore rate with no other workloads.

By increasing the RMAN channels, you can leverage available network bandwidth or storage bandwidth and achieve as much as 42 TB/hour backup rate and 8.7 TB/hour restore rate for 3 Exadata storage cells. The restore rates can increase as you add more Exadata storage cells. The performance varies based on existing workloads and network traffic on the shared infrastructure.

The Autonomous Recovery Service provides the following additional benefits:

- Leverage real-time data protection capabilities to eliminate data loss
- With a unique "incremental forever" backup benefit, you can significantly reduce backup processing overhead and time for your production databases.
- Implement a policy-driven backup life-cycle management.
- Additional malware protection

MAA Gold and Platinum Network Topology and Evaluation

The recommended MAA Gold and MAA Platinum architectures in Azure consist of:

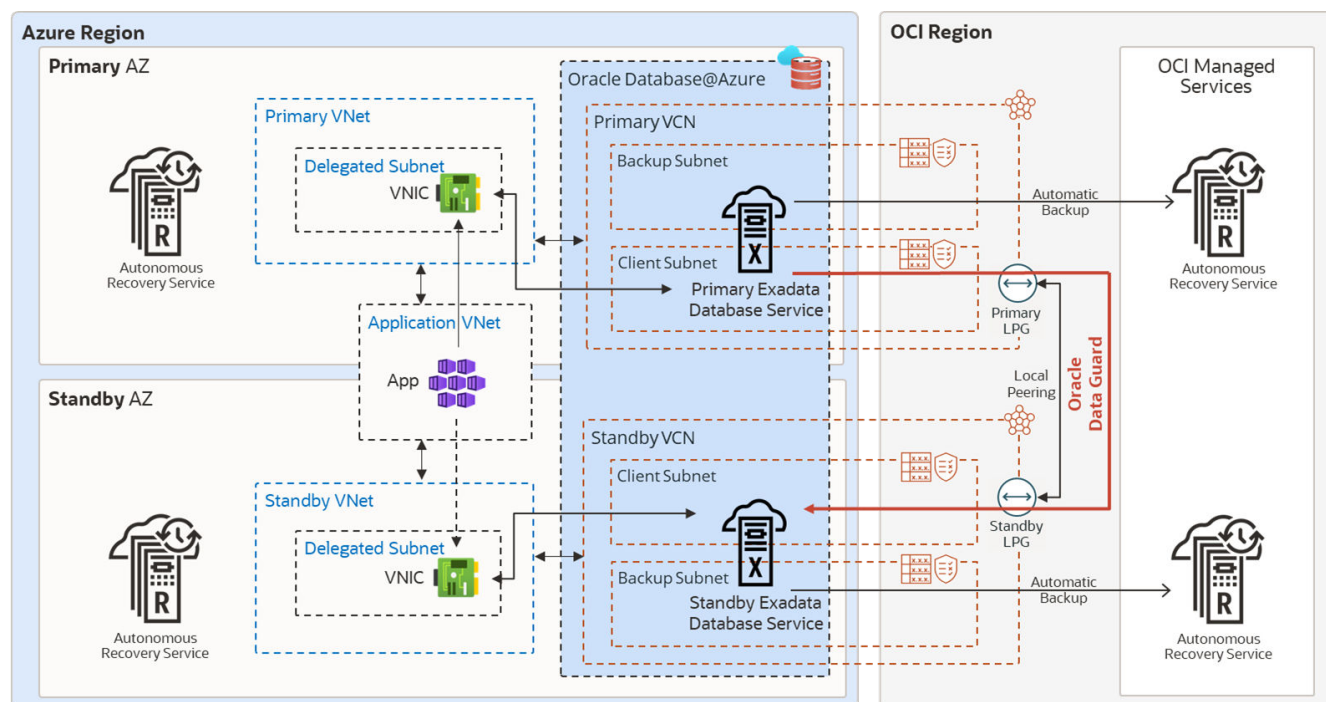
- When using Oracle Data Guard, Oracle Exadata infrastructures in an MAA Gold or MAA Platinum Option 1 (Oracle AI Database 26ai or higher and Oracle Active Data Guard) are provisioned in two different Availability Zones (AZs) or Regions using separate VNets that do not have overlapping IP CIDR ranges.
- Backup network subnets assigned to the primary and standby clusters do not have overlapping IP CIDR ranges.
- The application tier spans at least two AZs and the VNet peers, with each VNet consisting of primary and standby VM Clusters.

- Database backups and restore operations use a high bandwidth network for Autonomous Recovery Service@Azure, Autonomous Recovery Service on OCI, or OCI Object Storage.
- MAA Platinum Option 1 requires Oracle AI Database 26ai or higher and Oracle Active Data Guard.

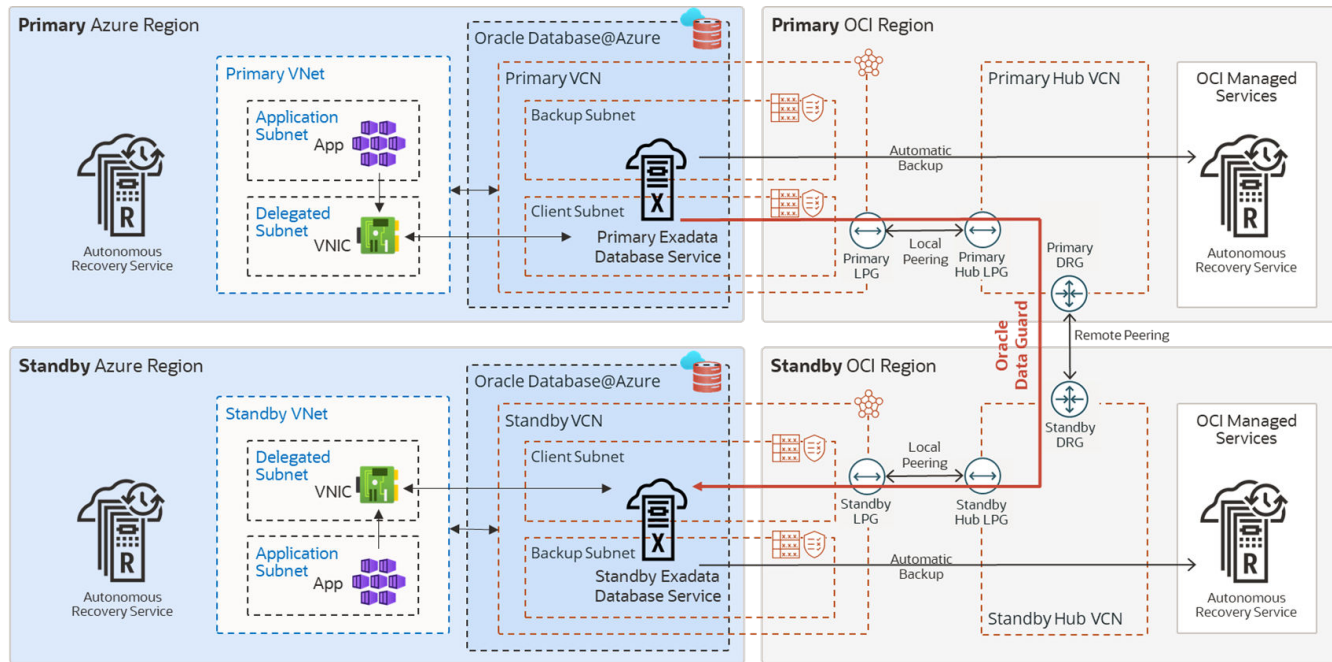
MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher and Oracle Active Data Guard) evaluation builds upon the Silver MAA evaluation, plus:

- Network tests between primary and standby DB clusters using OCI peered or multicloud peered networks to evaluate round-trip latency and bandwidth
- Oracle Data Guard role transition performance and timings for disaster recovery use cases
- Oracle database rolling upgrade with Data Guard
- MAA Platinum Option 1 requires Oracle AI Database 26ai or higher

The following diagram shows MAA Gold and MAA Platinum DR architecture in the same Azure Region.



The following diagram shows MAA Gold and MAA Platinum DR architecture across two Azure Regions.



Network Layer

Oracle Data Guard maintains an exact physical copy of the primary database by transmitting (and applying) all data changes (redo) to the standby database across the network, making network throughput, and in some cases latency, critical to the implementation's success. Use Data Guard switchover for planned maintenance or disaster recovery tests. If the primary database becomes unavailable, use Data Guard failover to resume service.

Peering Networks Between Primary and Standby

This is the most critical decision to ensure your standby can keep up. If the network does not have sufficient bandwidth to support single-process redo throughput, the standby will have a growing transport lag.

The primary and standby Exadata Clusters are deployed in separate networks. Oracle Database@Azure Exadata Clusters are always deployed using separate Virtual Cloud Networks (VCN) in OCI. These separate VCNs must be connected to allow traffic to pass between them, that is they must be "peered" before enabling Data Guard with Oracle cloud automation. For this reason, the networks must use separate, non-overlapping IP CIDR ranges.

Peering across Availability Zones (AZs) can be done using the OCI or Azure networks. Both options provide reasonable throughput; however, Azure peering comes with a chargeback for cross-VNet traffic. OCI provides up to 10TB of data egress per month at no charge, and lower rates when that limit is exceeded. Note that Azure VNets are virtual networks similar to an OCI VCN (Virtual Cloud Network).

For cross-region network peering, OCI is the recommended option for any Enterprise application and databases due to the higher throughput observed during testing. OCI VCN peering provides higher single-process network throughput when compared to Azure VNet peering between the same regions. The latency between database clusters will vary based on physical location and network topology. There is no chargeback for cross-region traffic for the first 10TB per month with OCI VCN peering. Note that single process throughput, maximum network throughput, and latency may vary based on data center locations.

Oracle Database@Azure service network is connected to the Exadata client subnet by a Dynamic Routing Gateway (DRG) managed by Oracle. A DRG is also required to peer VCNs between regions, and only one DRG is allowed for each VCN in OCI. Therefore, to connect the primary and standby VCNs, the communication requires transit over a second VCN with its own DRG in each region.

Recommended OCI VCN Peering for Data Guard

- For a cross availability zone configuration of Data Guard at Azure, follow the instructions in [Setting Up Networking Across Availability Zones](#).
- For a cross-region configuration of Data Guard at Azure, follow the instructions in [Setting Up Networking Across Regions](#).
 - Azure Virtual WAN Hub Model and Network Recommendations: <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/scenarios/oracle-iaas/oracle-network-topology-odaa#design-recommendations>
 - Network Bandwidth based Virtual Hub Settings: <https://learn.microsoft.com/en-us/azure/virtual-wan/hub-settings#edit-virtual-hub-capacity>

Alternative Option of Azure VNet Peering for Data Guard

Choose this option only if the database workload and redo generation rate are much lower than the single process network bandwidth results. To peer the Azure VNets for Data Guard redo traffic, see [Virtual network peering](#).

When networks are peered through Azure, single-process network throughput can be significantly lower than Oracle OCI peered networks.. This is relevant because Data Guard redo transport is a single process for each database instance; therefore, if a single instance produces redo at a higher rate, transport lag may increase and accumulate. There is an additional cost for ingress and egress network traffic in each VNet when networks are peered through Azure.

Enabling Data Guard

After the network is peered by one of the above options, you can Enable Data Guard (see [Use Oracle Data Guard with Exadata Cloud Infrastructure](#)).

Data Guard Role Transitions

The timings of the Data Guard switchover and failover were within expectations compared to a similar setup in Oracle OCI. Application downtime when performing a Data Guard switchover and failover can range from 30 seconds to a few minutes. For guidance on tuning Data Guard role transition timings or examples of role transition timings, see [Tune and Troubleshoot Oracle Data Guard](#).

Setting Up Networking Across Availability Zones

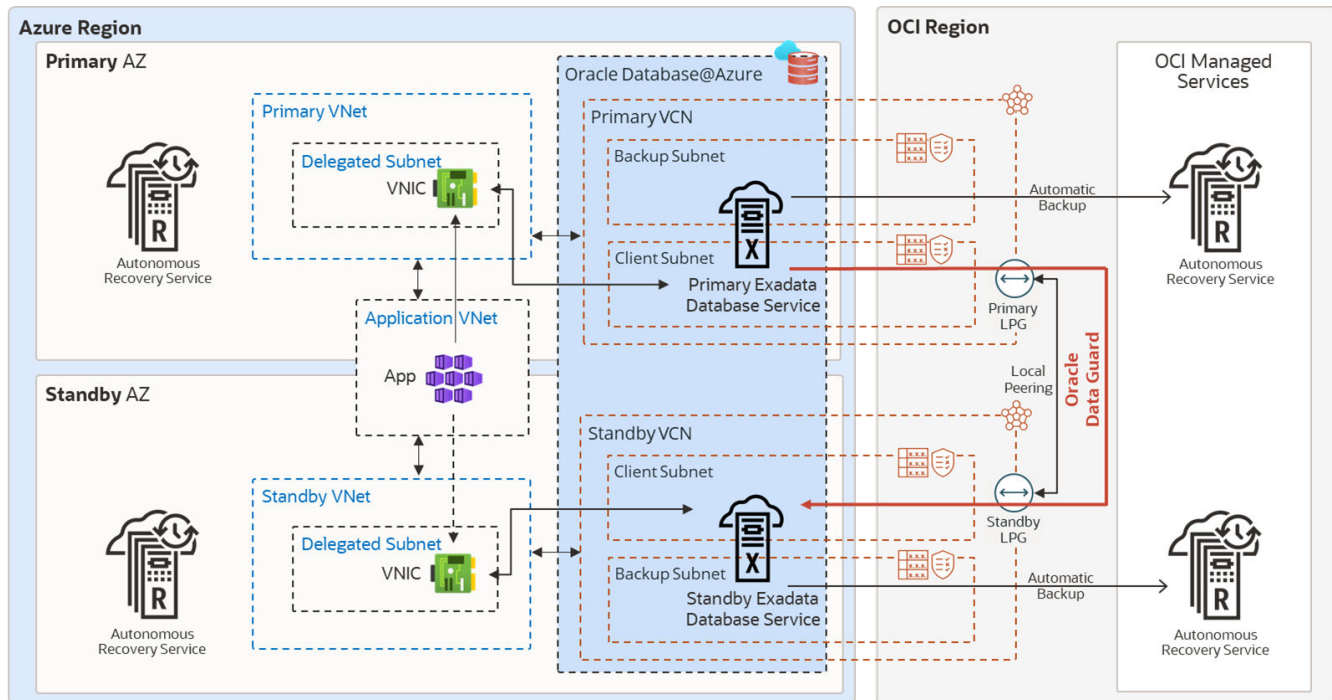
Oracle MAA recommends best practices for networking across availability zones in a disaster recovery setup.

Ensure the following

- Oracle Exadata infrastructures (ExaDB-D) are provisioned in two Availability Zones (AZs). Each Exadata infrastructure deployed at least one Exadata VM Cluster in a separate Virtual Network (VNet) to form the primary and standby environments.

- Ensure primary and standby client and backup subnets are separated VNets without overlapping IP CIDR ranges.
- The Azure Kubernetes Services spans at least two AZs, and the VNet is peered with each VNet of primary and standby VM Clusters.
- Database backups and restore operations are done across a high bandwidth network to OCI Object Storage.

Figure 36-2 MAA Gold DR Architecture in the Same Azure Region



Configuring the Application Tier on Azure

1. Deploy the application tier over at least two AZs.

The process and solution for deploying over multiple AZs varies depending on the application's components, Azure services, and resources involved. For example, with Azure Kubernetes Services (AKS), you can deploy the worker nodes in different AZs. The Kubernetes control plane maintains and synchronizes the pods and the workload.

2. Follow the steps described in [Configuring Continuous Availability for Applications](#).

Configuring the Database Tier on OCI

When Exadata Clusters are created in Azure, each cluster is in a different Virtual Cloud Network (VCN) in OCI. Connectivity between VCNs is required for Data Guard redo transport. This connectivity, or peering, must be configured before enabling Data Guard in Azure. For resources in different VCNs to communicate with each other, as is required by Data Guard, additional steps are required to peer the VCNs and allow the IP address ranges access to each other.

Oracle Data Guard maintains a standby database by transmitting and applying redo data from the primary database. Use Data Guard switchover for planned maintenance or disaster recovery tests. If the primary database becomes unavailable, use Data Guard failover to resume service.

OCI is the preferred network for performance (latency, throughput), and no egress or ingress costs are incurred. When Exadata Clusters are created in Azure, each cluster will be configured in a different OCI Virtual Cloud Network (VCN). As Data Guard requires, you must perform additional steps to peer the VCNs and allow the IP address ranges access to each other for resources in different VCNs to communicate with each other.

The following steps describe the process to enable Data Guard across AZs using the OCI-managed network.

1. Log in to the OCI Console and create a Local Peering Gateway (LPG) in the VCNs of the primary and standby Exadata VM Clusters.

For details see [Creating a Local Peering Gateway](#).

2. Establish a Peer Connection between primary and standby LPG, and select the Unpeered Peer Gateway in the standby VCN.

For details see [Connecting to Another LPG](#).

Each VCN can have only one Local Peering Gateway (LPG), if there are multiple databases on a given Exadata Cluster which will have standby databases on different Exadata Clusters, a Hub VCN will need to be configured. See [Transit Routing inside a hub VCN](#)

3. Update the default route table to route the traffic between the primary and standby databases over the OCI network without incurring any inbound and outbound data transfer costs.

Note

To update the default route table, you currently need to create a support ticket SR providing the tenancy name and dynamic routing gateway (DRG) OCID. If you encounter the error "Authorization failed or requested resource not found", open a service ticket with the following information:

- Title of ticket: "Required VCN Route Table Update permission"
- Include information for each VCN and its DRG attachment: Region, Tenancy OCID, VCN OCID, DRG OCID

4. Update the primary and standby Network Security Group to create a security rule to allow primary and standby client subnet ingress for TCP port 1521.

Optionally you can add SSH port 22 for direct SSH access to the database servers.

5. Enable Data Guard or Oracle Active Data Guard for the primary database.

From the Oracle Database details page, click the Data Guard Associations link, then click Enable Data Guard.

On the Enable Data Guard page:

- a. Select the standby Availability Domain mapped to Azure AZ.
- b. Select the standby Exadata Infrastructure.
- c. Select the desired standby VM Cluster.
- d. Choose Data Guard or Active Data Guard (MAA recommends Active Data Guard for auto block repair of data corruptions and ability to offload reporting).
- e. Choose a protection mode and redo transport type that satisfies your RTO and RPO.
- f. Select an existing database home or create a new one.

It is recommended that you use the same custom database software image of the primary database for the standby database home, so that both have the same patches available.

- g. Enter the password for the SYS user and Enable Data Guard.

Optionally, to reduce the recovery time in case of failures, enable automatic failover (Fast-Start Failover) by installing Data Guard Observer or a separate VM, preferably in a separate location or in the application network. For more information, see Fast-Start Failover in the Oracle Data Guard Broker guide and [Configure Fast Start Failover to Bound RTO and RPO \(MAA Gold Requirement\)](#). Currently these steps are not part of cloud automation and are manual.

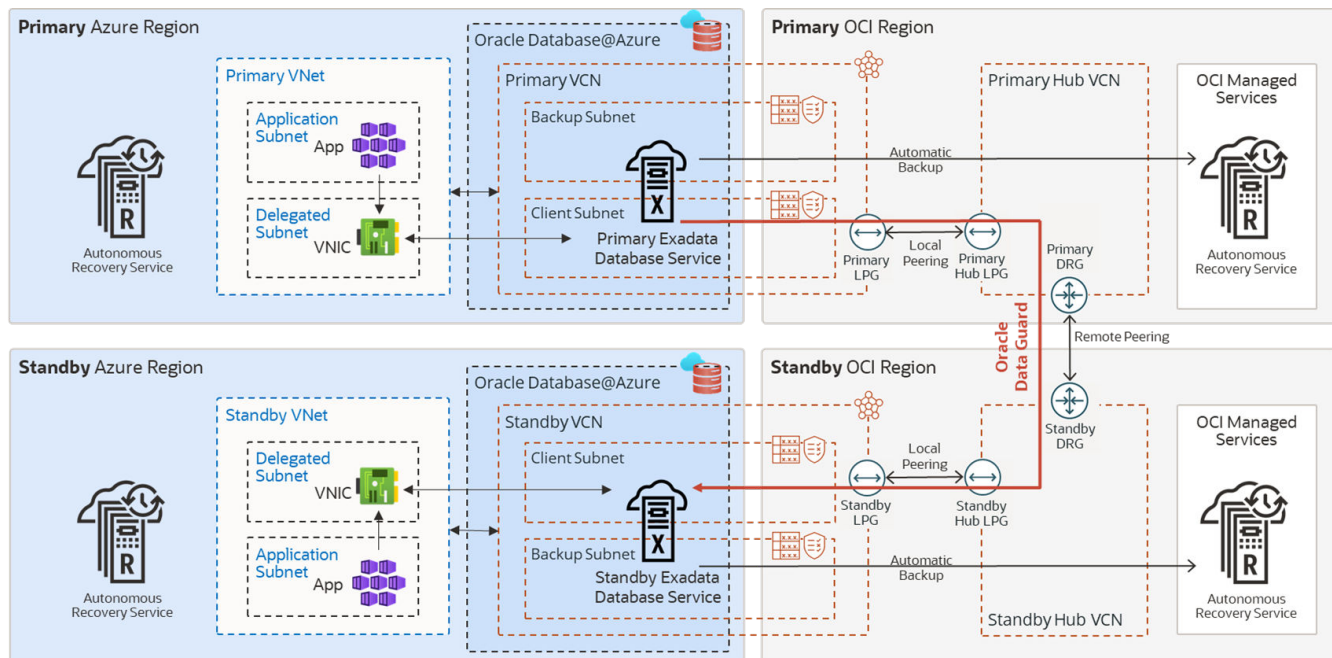
After Data Guard is enabled, the standby database is listed in the Data Guard Associations section.

Setting Up Networking Across Regions

Oracle MAA recommends these best practices for networking across Azure regions in an Oracle Database@Azure DR configuration.

The architecture diagram below shows a containerized Azure Kubernetes Service (AKS) application in two Azure regions. The container images are stored in the Azure container registry and replicated between primary and standby regions. Users access the application externally through a public load balancer. The database is running in an Exadata VM Cluster in primary/standby mode with Oracle Data Guard for data protection. The database TDE encryption keys are stored in OCI Private Vault and are replicated between the regions. The automatic backups are in OCI object storage in each region.

The following diagram shows MAA Gold and MAA Platinum DR architecture across two Azure Regions.



Network Principles for Oracle Database@Azure Across Regions

Oracle MAA recommends:

- Deploy Exadata Infrastructure in each region, primary and standby.
- On each Exadata infrastructure, deploy an Exadata VM Cluster in the delegated subnet of an Azure Virtual Network (VNet).
- The Oracle Real Application Clusters (RAC) database can be instantiated on the cluster. In the same VNet, deploy AKS in a separate subnet.
- Configure Oracle Data Guard to replicate data from one Oracle Database to the other across regions.
- When Exadata VM clusters are created in the Oracle OCI child site, each is created within its own OCI Virtual Cloud Network (VCN). Data Guard requires that the databases communicate with each other to ship redo and perform role transitions. The VCNs must be peered to enable this communication.
- OCI is the preferred network for higher network bandwidth, better network throughput, and reduced cost (the First 10 TB / Month is Free).

For additional details and setup instructions see [Perform Cross-Regional Disaster Recovery for Exadata Database on Oracle Database@Azure](#).

Note

Once configured, it is possible to enable automatic failover (Fast-Start Failover) to reduce recovery time in case of failure by installing Data Guard Observer on a separate VM, preferably in a separate location or in the application network. For more information, see the documentation for Fast-Start Failover, and Configure and Deploy Oracle Data Guard. (These are currently manual steps and not part of cloud automation.)

Oracle Maximum Availability Architecture for Oracle Database@Google Cloud

Google is a strategic Oracle Multicloud hyperscaler partner. Oracle Maximum Availability Architecture (MAA) evaluates MAA reference architectures for Oracle Exadata Database Service on Dedicated Infrastructure (ExaDB-D) on Oracle Database@Google Cloud, the results of which are shown here.

To learn more about MAA Silver, MAA Gold, and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) evaluations and their benefits after certification, see [MAA Evaluations on Multicloud Solutions](#).

Oracle MAA evaluated Oracle's solution in Google Cloud. The goal is to re-evaluate at least annually to ensure the solution meets all the expected benefits and to highlight any new MAA for Oracle Database@Google Cloud benefits and capabilities. Certification is only given after the MAA for Oracle Database@Google Cloud evaluation meets the requirements.

Oracle MAA has evaluated and endorsed the MAA Silver, MAA Gold, and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) reference architectures for Exadata Database Service on Oracle Database@Google Cloud when the standby database resides on an Exadata infrastructure within a single region, across zones within the same region, and across regions.

- The environment tested was across zones and across regions between Frankfurt, Germany and London, England.
- Oracle Active Data Guard configurations on Oracle Database@Google Cloud involving different zones or regions will have different results for throughput and latency.

Network Results

Common abbreviations:

- OCI - Oracle Cloud Infrastructure (Oracle Cloud)
- VCN - Virtual Cloud Network (Oracle)
- VPC - Virtual Private Cloud (Google Cloud)
- VLAN - Virtual Local Area Network
- RCV - Oracle Database Autonomous Recovery Service

The results shown in the following use cases are based on network tests described in [MAA Evaluations on Multicloud Solutions](#).

Use Case: Application VMs to Exadata VM Cluster (Same Region)

RTT Latency Observed: ~300 microseconds (same region/zone)

- Varies based on placement. For improved application VM latency requirements, a Google support ticket should be filed.

Network Throughput Observed: Variable based on VM shape.

- See a machine series comparison at https://cloud.google.com/compute/docs/machine-resource#machine_type_comparison
- Example: n2-standard-4 (4 vCPUs, 16 GB Memory) 10 Gbps.

MAA Recommendations:

- Ensure that your required RTT latency meets your application requirements.
- Test thoroughly with your implementation. Variables such as VM size.
- Refer to [Application Network Layer on Google Cloud](#)

Use Case: Between Exadata VM Clusters in the Same Region or Across Zones

Tested:

- Redo Transport
- Database migration or standby database instantiation
- Example data is for a configuration between zones in Ashburn, Virginia (IAD).

RTT Latency Observed: 1.3 ms

Network Throughput Observed in IAD Cross-zones Example:

# Processes	OCI VCN Peering	Google Cloud VPC Networking
1	34 Gb/s (4250 MB/s)	10 Gb/s (1250 MB/s)

# Processes	OCI VCN Peering	Google Cloud VPC Networking
4	90 Gbits/sec (11250 MB/s)	36 Gb/s (4500 MB/s)
10	90 Gbits/sec (11250 MB/s)	60 Gb/s (7500 MB/s)

MAA Recommendations:

- Repeat MAA-recommended network tests in your environment. Refer to [Network Evaluation](#).
- Test both networking options as described in "Testing Throughput" in [Networking Between Primary and Standby Clusters](#) and ensure that the network throughput exceeds minimum requirements and your peak redo rates.
- For Google networking, both Exadata VM Clusters must be:
 - Created in the same VPC Network (this requires no extra networking)
 - The VPC Network must have MTU set to jumbo frames (8896).
 - The Cloud Interconnect VLAN attachments in both regions, which must have 4 per Exadata Infrastructure, must be set to 10 Gbps network bandwidth or higher to achieve observed results for Active Data Guard redo transport and standby database instantiation. File a request with Google cloud service to request this increase in bandwidth.

Use Case: Between Exadata VM Clusters Across Regions

Tested:

- Redo Transport
- Database migration or standby database instantiation
- Example data is for a configuration between London, England (LHR) and Frankfurt Germany (FRA). Results will vary between different regions.

RTT Latency Observed: 12 ms (FRA-LHR)

Network Throughput Observed in FRA-LHR Example:

# Processes	OCI VCN Peering	Google Cloud VPC Networking*
1 (min 1.4 Gb/sec)	2.1 Gb/s (250 MB/s)	2.7 Gb/s (300 MB/s)
4	9.1 Gb/s (1137 MB/s)	8.3 Gb/s (1037 MB/s)
10 (min 8 Gb/sec)	22.6 Gb/s (2825 MB/s)	11.4 Gb/s (1425 MB/s)

*Google Cloud VPC networking throughput results are heavily dependent upon the Cloud Interconnect VLAN attachments in both regions having 10Gbps bandwidth (default 2Gbps). File a Google support ticket to request the increase or observed throughput is significantly less.

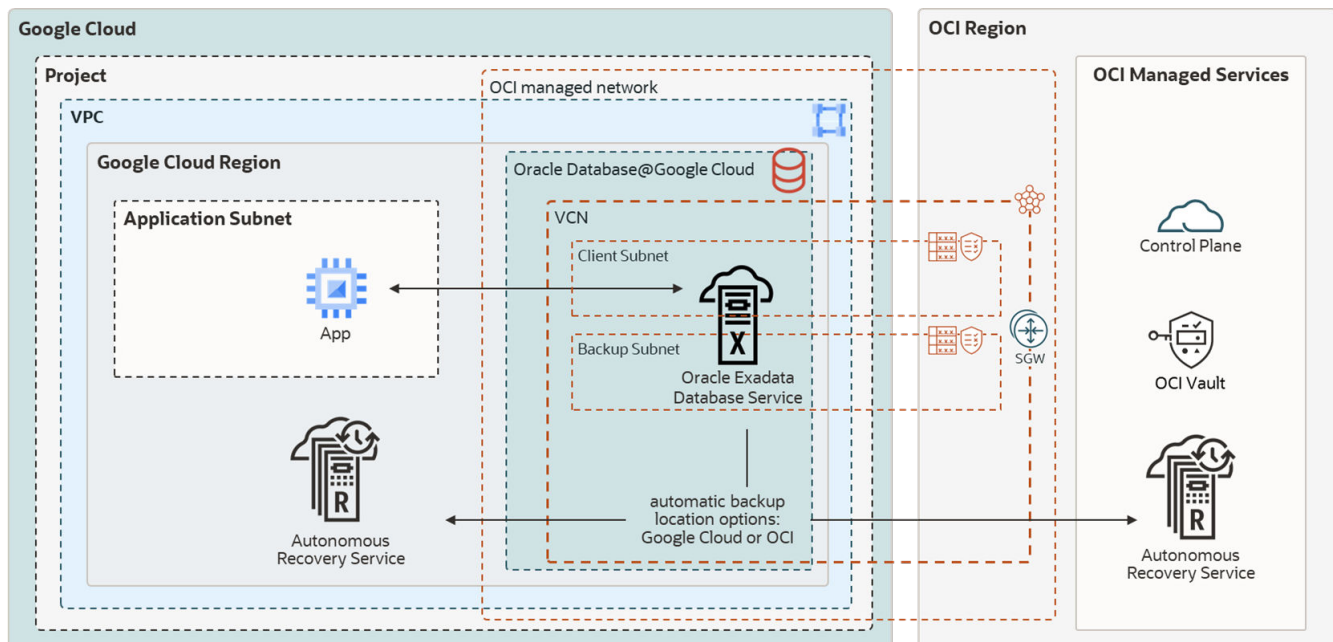
MAA Recommendations:

- Repeat MAA-recommended network tests in your environment. Refer to [Network Evaluation](#).
- Test both networking options as described in "Testing Throughput" in [Networking Between Primary and Standby Clusters](#) and ensure that the network throughput exceeds minimum requirements and your peak redo rates.
- For cross-region OCI peering, see [Perform cross-region disaster recovery for Exadata Database Service on Google Cloud](#).
- For Google networking, both Exadata VM Clusters must be:
 - Created in the same VPC Network (this requires no extra networking)
 - The VPC Network must have MTU set to jumbo frames (8896).
 - The Cloud Interconnect VLAN attachments in both regions, which must have 4 per Exadata infrastructure, must be set to 10Gbps network bandwidth or higher to achieve acceptable results for Active Data Guard redo transport and standby database instantiation.
- Variables may impact bandwidth; refer to Google doc <https://cloud.google.com/compute/docs/network-bandwidth>.

MAA Silver Network Topology and Evaluation

The MAA Silver Reference Architecture provides database high availability in the case of hardware or software failures. Refer to the documentation for a full description of [MAA Silver](#).

Figure 36-3 MAA Silver Architecture for Oracle Database@Google



Oracle MAA has evaluated and endorsed ExaDB-D on Oracle Database@Google Cloud for MAA Silver with the following observations:

- Application latency may be impacted by the application VM's proximity to the target database server. A Google support ticket should be opened to request close proximity for

applications which require the lowest latency to the database. For high availability of the application itself, evaluate multiple, fault-isolated application VMs.

- The backup and restore performance with RCV in Google Cloud (default), RCV in OCI, and Object Storage Service in OCI meets MAA Silver expectations. See "Backup/Restore" <add link> below.
- Application and database availability meet MAA Silver expectations while injecting unplanned local outages, updating database and system software, and during elastic changes for the system and database (for example, increasing CPU, storage, and so on). See [MAA Evaluations on Multicloud Solutions](#) for the relevant tests and [Oracle Maximum Availability Architecture in Oracle Exadata Cloud Systems](#) for expected results and benefits.
- Configuration health checks, such as Exachk, help ensure MAA compliance. MAA recommends enabling [Database Service Health Events](#) and reviewing Exachk output monthly.

Application Network Layer on Google Cloud

The proximity of the application tier to the database cluster affects application response time. If you require a very low latency response time (300 microseconds), deploy the application VMs in the VPC Network in close proximity to the database cluster. [Log a Google support ticket](#) to ensure close proximity of the application VMs to the database cluster.

Deploy the application tier with multiple application VMs with sufficient fault isolation for high availability. The deployment process and solution depend on the application's components, Google services, and the resources involved. For example, with [Google Kubernetes Engine \(GKE\)](#), you can deploy the worker nodes in different locations. Kubernetes control plane maintains and synchronizes the pods and the workload.

Backup and Restore Observations

RMAN net test results met expectations. See [KB115145](#) for details about net test.

Oracle database backup and restore throughput to Oracle Database Autonomous Recovery Service or OCI Object Storage Service were within performance expectations. For example, an ExaDB-D 2-node cluster (using 16+ OCPUs) and 3 storage cells may observe a 4 TB/hour backup rate and approximately 8.7 TB/hour restore rate with no other workloads. By increasing the RMAN channels, you can leverage available network bandwidth or storage bandwidth and achieve as much as 42 TB/hour backup rate and 8.7 TB/hour restore rate for 3 Exadata storage cells. The restore rates can increase as you add more Exadata storage cells. The performance varies based on existing workloads and network traffic on the shared infrastructure.

The [Oracle Database Autonomous Recovery Service](#) provides the following additional benefits:

- Leverage real-time data protection capabilities to eliminate data loss.
- With a unique "incremental forever" backup benefit, you can significantly reduce backup processing overhead and time for your production databases.
- Implement a policy-driven backup life-cycle management.
- Additional malware protection

MAA Gold and MAA Platinum Network Topology and Evaluation

The MAA Gold and MAA Platinum reference architectures protect mission-critical databases with a remote synchronized copy of the database, which can be activated in the event of a disaster. Refer to the documentation for a full description of [MAA Gold and Platinum](#).

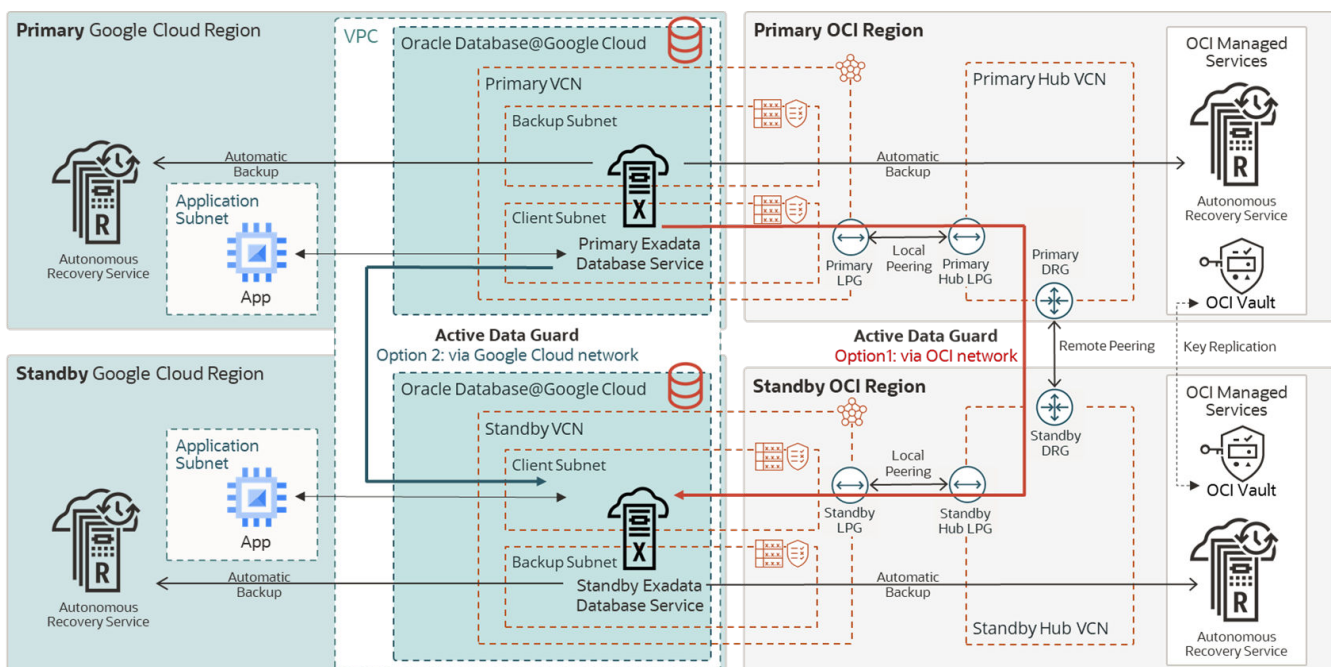
The recommended MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Active Data Guard) architectures in Google Cloud consist of:

- When using Oracle Active Data Guard, Oracle Exadata infrastructures (ExaDB-D) are provisioned in two different regions.
- Network subnets assigned to the primary and standby clusters do not have overlapping IP CIDR ranges.
- The application tier spans multiple App Servers consisting of primary and standby App VM Clusters.
- Database backups and restore operations use a high bandwidth network for Oracle Database Autonomous Recovery Service (in Google Cloud or OCI) or OCI Object Storage.
- MAA Platinum Option 1 requires Oracle AI Database 26ai or higher with Oracle Active Data Guard.

MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) evaluations build upon the Silver MAA evaluation, with:

- Network tests between primary and standby database clusters using OCI peered or multicloud partner peered networks to evaluate round-trip latency and bandwidth.
- Oracle Active Data Guard role transition performance and timings for disaster recovery use cases.
- Oracle Database rolling upgrade with Active Data Guard.

The following architecture diagram for MAA Gold and MAA Platinum Option 1 shows an application in two Google regions. The database is running in an Exadata VM Cluster in primary/standby mode with Oracle Active Data Guard for data protection. The database Transparent Data Encryption (TDE) keys are stored in OCI Vault and are replicated between the regions. The automatic backups are in Oracle Database Autonomous Recovery Service (in Google Cloud or OCI).



Oracle MAA has evaluated and endorsed Exadata Database Service on Oracle Database@Google Cloud for MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) with the following observations:

- Regions chosen for the primary and standby databases are critical to supporting the Recovery Point Objective (RPO). Network throughput must be sufficient as described below in [Networking Between Primary and Standby Clusters](#).
 - Single node throughput from London to Frankfurt for a single process was 2 Gb/sec with a maximum multi-process throughput of ~47 Gb/sec. These throughputs can support most database workloads.
- The latency between an application VM and the database server within the same region was observed to be less than 300 microseconds, which meets MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) expectations.
- Application VM throughput to the database server is dependent on the size of the VM. Refer to [Google Cloud documentation](#) for details about the bandwidth allocated per VM shape.
- Oracle Active Data Guard Life Cycle Operations, switchover, failover, and reinstate are initiated from the OCI Console (accessible from the Google Cloud Console). The performance of these operations is in line with MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) expectations.
- OCI VCN Peering was evaluated, and bandwidth and latency meet MAA criteria. For Google Cloud VPC Networking between the regions, the Google VPC Attachment bandwidth impacts the throughput of the network connections significantly. MAA Recommends at least 10 Gb/sec bandwidth for these attachments of which there are 4 per region (default 2 Gbps). A Google support ticket can be opened to increase the values from the default or when throughput is significantly below expectations. Refer to <https://cloud.google.com/oracle/database/docs/get-support>.
- The VPC network should have an MTU setting of 8896 (default 1460). Refer to [Google Cloud documentation](#) to change this value.

Oracle Active Data Guard Principles for ExaDB-D on Google Cloud Configuration

Oracle MAA recommends:

1. Deploy Exadata Infrastructure in the selected regions for primary and standby.
2. From the Google Cloud Console, on each Exadata infrastructure, deploy an Exadata VM Cluster in the appropriate VPC.
 - For Google Cloud VPC networking option, both VM Clusters should be in the same VPC. For OCI VCN peering the VPC used for each VM Cluster can be different.
 - CIDR ranges for the primary and standby client and backup networks **cannot** overlap.
3. Using the OCI Console, create an Oracle Real Application Clusters (RAC) database on the VM cluster.
 - a. The OCI Console can be reached from the Exadata Infrastructure details page in the Google Cloud Console by clicking **MANAGE IN OCI**.
 - b. Click the appropriate VM cluster.
 - c. Click **Create Database** and complete the wizard.
4. For the application tier, deploy Google Kubernetes Engine (GKE) in the same VPC network as the VM Clusters, on a separate subnet.

- **Note:** the CIDR range for the VM Clusters are not shown in the Google Console but should not overlap the GKE subnet CIDRs.
5. Using the OCI Console, configure Oracle Active Data Guard to replicate data across regions from one Oracle Database to the other.
 - a. From the database page for the targeted database, click **Data Guard group** in the left-hand column.
 - b. Click the **Add Standby** button and complete the wizard.
 - c. Depending on the size of the database, standby creation may take 30 minutes to hours. Progress can be monitored using the Work Requests resource.
 6. When Exadata VM clusters are created in the Oracle OCI child site, each is created within its own OCI Virtual Cloud Network (VCN). Active Data Guard requires that the databases communicate with each other to ship redo and perform role transitions.
 - See the [Networking Between Primary and Standby Clusters](#) section below for additional details about connecting the networks.
 - For VPC networking, VM Clusters created in the same VPC will be able to communicate with each other over the Google Cloud backbone.
 - For OCI VCN peering, follow the steps in "OCI Peering Setup" in the [Networking Between Primary and Standby Clusters](#) section below.
 7. When considering full stack disaster recovery failover, you need to consider DNS failover and coordinate application failover to the new primary database (after Data Guard switchover or failover)

For additional details and setup instructions, see [Perform Cross-Regional Disaster Recovery for Exadata Database Service on Google Cloud](#).

Networking Between Primary and Standby Clusters

Oracle Active Data Guard maintains an exact physical copy of the primary database by transmitting (and applying) all data changes (redo) to the standby database across the network. This makes network throughput and, in some cases, latency critical to the implementation's success.

There are two networking options to connect the primary and standby database clusters. OCI VCN peering and Google Cloud VPC networking. Regardless of the networking option chosen, the CIDR blocks for each cluster must not overlap.

OCI VCN Peering

Each Exadata VM cluster will be deployed on its own OCI Virtual Cloud Network (VCN) within the Google Cloud region, regardless of the Google Cloud VPC used for these infrastructures. To route traffic through the OCI backbone (MAA recommended for consistent throughput results) the VCNs are peered through hub VCNs and associated gateways. See the "OCI Peering Steps" below.

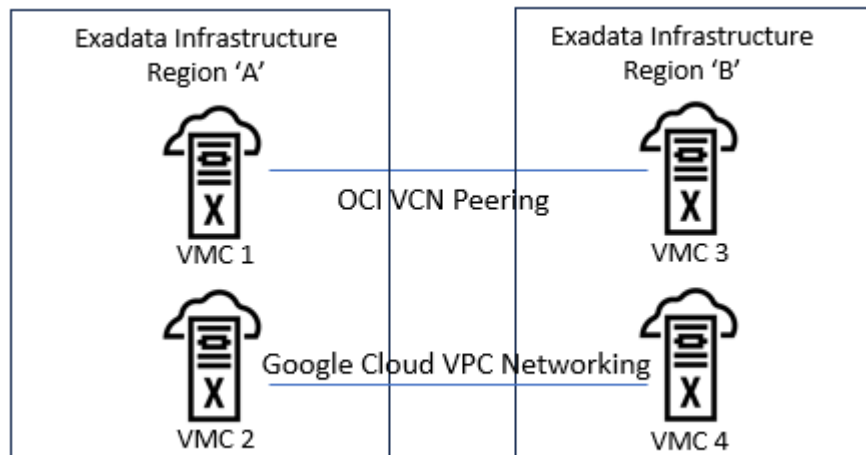
Google Cloud VPC Networking

When connecting the Exadata VM clusters with Google Cloud networking, ensure that each cluster is created in the same VPC network (but in different regions for cross-region configurations). There are no additional steps to allow these clusters to communicate.

Testing Throughput

It is recommended that you test your configuration with both options (see [MAA Evaluations on Multicloud Solutions](#)). Multiple VM clusters can be created on a single infrastructure, while the networking between the two pairs of VM Clusters can be configured differently for testing purposes. Each VM cluster should be the same size.

Figure 36-4 Testing scenario using two Exadata infrastructures with multiple VM clusters peered in two different ways



Comparing the Observed Network Throughput For Frankfurt and London

Two measurements of throughput are critical for a successful implementation.

1. Parallel process throughput is an indicator of how long it will take to instantiate (create) the standby database. By default, parallelism is set to 4 processes per primary database node.
2. Single process throughput is an indicator of how fast redo transport can ship redo to the standby database. Each primary database instance creates its own thread of redo and ships it to the standby by a single process. If the redo generation rate of any instance exceeds the single process throughput capability of a single database node, a transport lag will develop and potentially impact the RPO of the database.

Table 36-1 Network Throughput Example (FRA-LHR)

# Processes	OCI VCN Peering	Google Cloud VPC Networking	Minimum Expectations
1	2.1 Gb/sec (260 MB/sec)	2.7 Gb/sec (300 MB/sec)	2 Gb/sec (250 MB/sec)
4	9.1 Gb/sec (1137 MB/sec)	8.3 Gb/sec (1037 MB/sec)	
10	22.6 Gb/sec (2825 MB/sec)	11.4 Gb/sec (1425 MB/sec)	8 Gb/sec (1000 MB/sec)

Note

Each pair of regions will have different throughput. It is important that testing is performed before Active Data Guard configuration to understand the capabilities of the network option chosen for the selected regions.

OCI Peering Setup

See the detailed steps for deploying and configuring Data Guard using OCI VCN peering in [Perform Cross-Region Disaster Recovery for Exadata Database Service on Google Cloud](#).

Google Cloud VPC Networking Setup

Create the related Oracle Database@Google Cloud Exadata Infrastructures in the same VPC network, in different subnets.

Enabling Active Data Guard

Once the network is configured and tested using one of the above options, you can enable Active Data Guard. See [Use Oracle Data Guard with Exadata Cloud Infrastructure](#) for details.

Active Data Guard Role Transitions

The timings of the Active Data Guard switchover and failover were within expectations compared to a similar setup in Oracle OCI. Application downtime when performing an Active Data Guard switchover and failover can range from 30 seconds to a few minutes. For guidance on tuning Active Data Guard role transition timings or examples of role transition timings, see [Role Transition, Assessment, and Tuning](#).

Automatic Failover

Once configured, it is possible to enable automatic failover (Fast-Start Failover), to reduce recovery time in case of failure, by installing Data Guard Observer on a separate VM, preferably in a separate location or the application network. For more information, see [Configure Fast Start Failover to Bound RTO and RPO \(MAA Gold Requirement\)](#). (These are currently manual steps and not part of cloud automation.)

Oracle Maximum Availability Architecture for Oracle Database@AWS

Amazon Web Services (AWS) is a strategic partner of Oracle Multicloud. Oracle Maximum Availability Architecture (MAA) evaluates MAA reference architectures on Oracle Database@AWS, the results of which are shown here.

To learn more about MAA Silver, MAA Gold, and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) evaluations and their benefits after certification, see [MAA Evaluations on Multicloud Solutions](#).

Oracle MAA evaluated Oracle's solution in AWS. Oracle MAA continues to be periodically re-evaluated to ensure the solution meets all expected benefits, and to highlight any new MAA benefits and capabilities for Oracle Database@AWS. Certification is only given after the MAA evaluation meets the requirements.

Oracle MAA has evaluated and endorsed Oracle Database@AWS for:

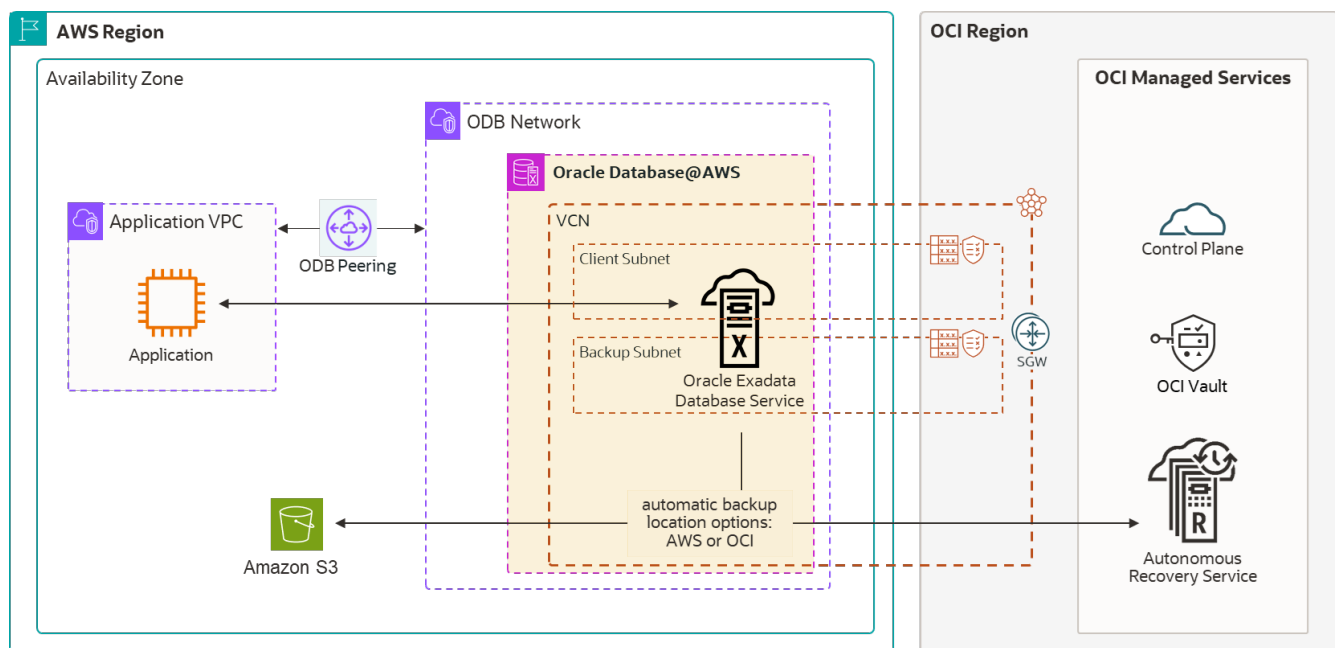
- MAA Silver reference architecture for Oracle Exadata Database Service on Dedicated Infrastructure and Oracle Autonomous Database on Dedicated Exadata Infrastructure
- MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) reference architectures for Oracle Exadata Database Service on Dedicated Infrastructure and Oracle Autonomous Database on Dedicated Exadata Infrastructure, when the standby database resides on another Exadata infrastructure in the same or separate Availability Zones (AZ) or different regions. Note that network latency and network bandwidth may vary depending on source and target locations.

Refer to [Multicloud Considerations for Autonomous Data Guard](#) for information about the MAA Silver and MAA Gold certifications for Oracle Autonomous Database Dedicated.

MAA Silver Network Topology and Evaluation

The following image shows the MAA Silver reference architecture, with Oracle RAC on Exadata Database Service, and showing backup options to Oracle Database Autonomous Recovery Service running in OCI or Amazon S3 in AWS.

Figure 36-5 Oracle Database@AWS in an MAA Silver Reference Architecture



Oracle MAA evaluated and endorsed Exadata Database Service on Oracle Database@AWS for the MAA Silver reference architecture, and observed the following results.

- Application and database uptime is adhered to as expected while injecting unplanned local outages, updating database and system software, and during system and database elastic changes (for example, increasing CPU, storage, and so on). See [MAA Evaluations on Multicloud Solutions](#) for tests evaluated.
- [Oracle Maximum Availability Architecture in Oracle Exadata Cloud Systems](#) describes all of the benefits, including cloud MAA templates, high availability and low brownout, data protection, response time Quality-of-Service, low downtime for unplanned outages, minimum application impact for planned maintenance, and elastic system operations.

Additional MAA Best Practices Specific to AWS

- Take careful consideration of application VM placement and high availability. Application latency may be impacted by the application VM's proximity to the database server target.

- For the lowest latency, place the application VM in the same Availability Zone (AZ) as the database, and use placement groups (not AWS network virtual appliances, firewalls, and the like).

See [High Performance Networking for Oracle Database@AWS](#) for placement group details.

- Ensure that your required RTT latency meets your application requirements.

See [MAA Evaluations on Multicloud Solutions](#) for information about how to evaluate RTT latency. For high availability, evaluate the placement of multiple application VMs in the same AZ or across multiple AZs, depending on your application response time requirements. Test thoroughly with your implementation. Variables such as VM size and placement can impact results.

- To enable transparent application failover, follow the steps in [Configuring Continuous Availability for Applications](#).

- Use Autonomous Recovery Service on OCI for the best data protection, backup, and recovery benefits as discussed later.

The MAA evaluation observed that both Autonomous Recovery Service and Object Storage Service in OCI met expectations. MAA evaluation with S3 in AWS is still underway. See topic "Backup and Restore Observations" below.

- Allow TCP port 6200 ingress into the Exadata VM Cluster client subnet from the networks where your application VMs reside.

TCP port 6200 access lets the Oracle Notification Services (ONS) to communicate about Fast Application Notification (FAN) events, which are required to provide immediate notification of cluster and service events, enabling applications to respond quickly to changes that occur during planned maintenance and unplanned outages. The recommended way to allow TCP port 6200 ingress is to log in to the Oracle Cloud Console, navigate to VM Cluster Information, edit the Client network security group `exa_1521_adjustable_nsg`, and add a security rule where:

- Direction is Ingress
- Source Type is CIDR
- Source CIDR is your application network CIDR
- Protocol is TCP
- Destination Port Range is 6200

- Allow ICMP Echo Request within the Exadata VM Cluster Client subnet.

ICMP Echo Request enables the `ping(8)` command between Exadata virtual machines, which is required during Virtual Machine OS Update by the `patchmgr` utility. The recommended way to allow ICMP Echo Request is to log in to the Oracle Cloud Console, navigate to VM Cluster Information, edit the Client network security group `exa_static_nsg`, and add a security rule where:

- Direction is Ingress
- Source Type is NSG
- Source NSG is `exa_static_nsg`

- Protocol is ICMP
- Type is 8
- See [Security Rules for the Oracle Exadata Database Service on Dedicated Infrastructure](#).

Application Network Layer and Application Failover

Factors that impact the latency and throughput between the application and database include: placement groups, VM size (application), OS kernel (application), networking services (for example, gateways), and data security inspection.

For response time-sensitive applications, pay attention to the observed response time results in the following use case.

For steps to improve the overall latency review, see [Improve network latency for Linux-based EC2 instances](#) and [High Performance Networking for Oracle Database@AWS](#).

See [MAA Evaluations on Multicloud Solutions](#) for the tests that were conducted.

Use Case: Application VMs to Exadata VM Cluster in the Same Region

Observed RTT Latency (US-East-2): Varies based on placement.

- ~165 μ s in the same availability zone (AZ) using Amazon Linux2023 kernel 6.1 AMI
- 1 ms to 1.2 ms across AZs

Observed Network Throughput (US-East-2): Affected by VM size. "Micro" images specifically will be throttled on throughput.

- 5 Gbps single process throughput
- 12-25 Gbps parallel process throughput

MAA Recommendations:

- Ensure your required RTT latency meets your application requirements.
- Have multiple application VMs for high availability, and ensure placement of application VMs continues to meet your application RTT latency requirements.
- Test thoroughly with your implementation. Variables such as VM size and placement can impact results.
- To enable transparent application failover, follow the steps described in [Configuring Continuous Availability for Applications](#).

Backup and Restore Observations

Oracle Database backup and restore throughput to Oracle's Autonomous Recovery Service or Oracle's Object Storage Service was within performance expectations, and all expected built-in backup and restore configuration and life cycle best practices were incorporated into cloud automation.

Using RMAN net test and actual database backup and restore operations, backup and restore performance were within expectations. For example, an Exadata Database Service 2-node VM Cluster (using 16+ OCPUs) and three storage cells may observe a 1.5 TB/hour backup rate and approximately 2.2 TB/hour restore rate with no other workloads.

Note that the default backup and restore RMAN channels vary based on the number of ECPUs per database node. You can increase the number of RMAN channels for higher backup and

restore throughput by using `dbaascli` and to change the `channelsPerNode` value (maximum value is 32 per node for Autonomous Recovery Service or higher for other target services). An average throughput of 40 MB/second per Oracle RMAN channel was observed. The RMAN channel throughput contributes to your overall backup or restore throughput until you max out another resource limitation.

By increasing the RMAN channels, you can leverage available network bandwidth or storage bandwidth, and achieve as much as a 5 TB/hour backup rate and 7 TB/hour restore rate with three Exadata storage cells. The restore rates can increase as you add more Exadata storage cells, with approximately 2GB/second per storage cell. The performance varies based on existing workloads and network traffic on the shared infrastructure.

The Autonomous Recovery Service provides the following additional benefits:

- Leverage real-time data protection capabilities to eliminate data loss.
- With a unique "incremental forever" backup benefit, you can significantly reduce backup processing overhead and time for your production databases. The effective backup rate can be increased to 20+ TB/hour depending on the shape and size of the VM Cluster, database size, and the number of data files.
- Implement a policy-driven backup life-cycle management.
- Additional malware protection
- Data protection health statuses
- Long-term backup retention.
- Built-in high availability for Autonomous Recovery Service deployments

With all of the above benefits, Oracle MAA always recommends Autonomous Recovery Service. To enable Autonomous Recovery Service, see the [Autonomous Recovery Service documentation](#), enable the security rules required, and use the Cloud Console and choose the Autonomous Recovery Service option.

See also:

- [KB115145](#) for details about net test
- Autonomous Recovery Service checklist outlining the prerequisites to use Autonomous Recovery Service, see [Using Oracle Database Autonomous Recovery Service](#)

MAA Gold and MAA Platinum Network Topology and Evaluation

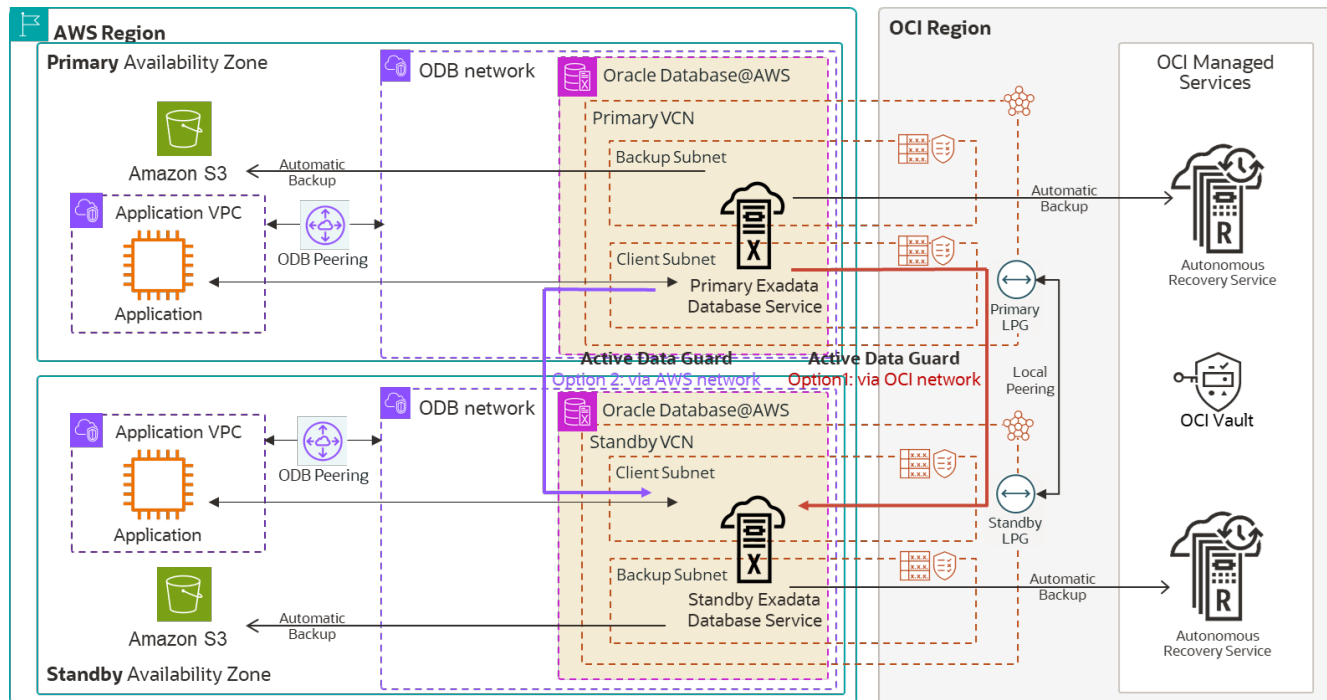
The recommended MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) reference architectures in AWS consists of:

- When using Oracle Data Guard or Oracle Active Data Guard, Oracle Exadata VM Clusters are provisioned in two different, independent Exadata Infrastructures in the same region across Availability Zones (AZs), or across Regions using separate networks that do not have overlapping IP CIDR ranges.
- Backup network subnets assigned to the primary and standby Exadata VM Clusters do not have overlapping IP CIDR ranges.
- Database backup and restore operations use a high-bandwidth network for Autonomous Recovery Service or Object Storage Service in OCI or Amazon S3 Object Storage.

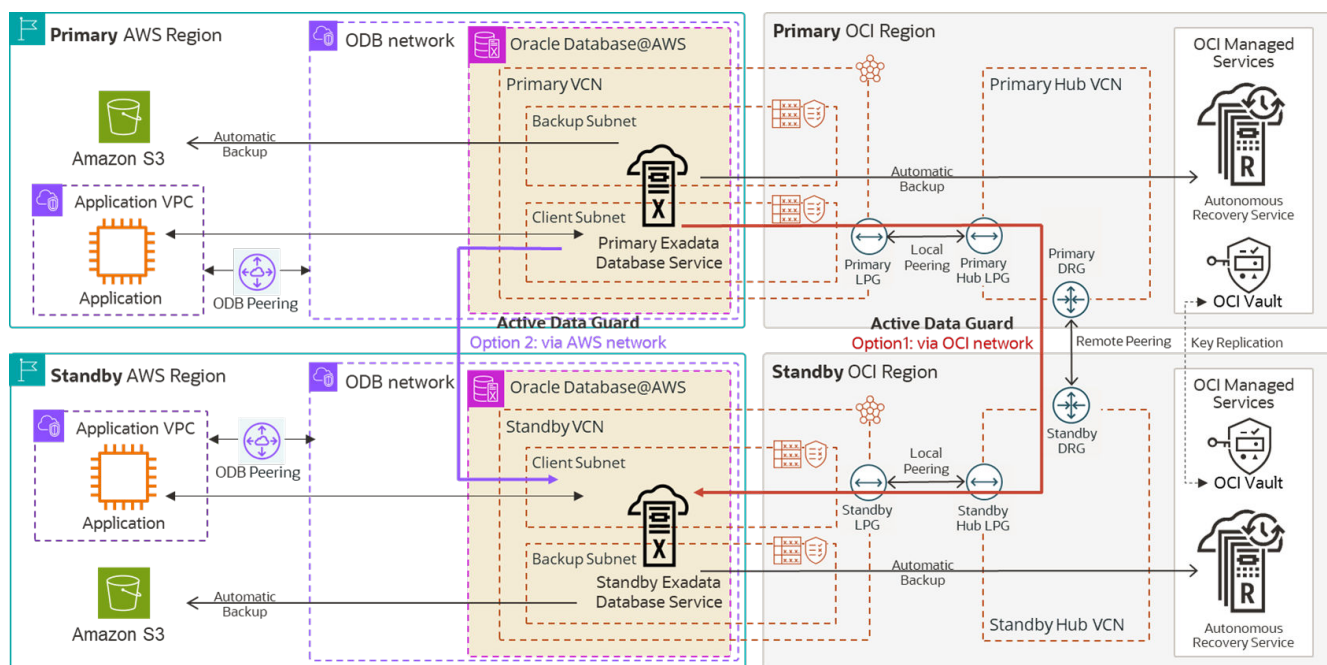
The following images show the MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) reference architectures, with Oracle RAC on Exadata Database Service, and shows backups to Oracle Database Autonomous Recovery

Service running in OCI and Amazon S3 in AWS. The first image shows the architecture implemented in a single AWS region across two Availability Zones, and the second image shows the architecture deployed across two AWS regions.

The following diagram shows MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) Architectures in one AWS Region with two AZs.



The following diagram shows MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) architectures in two AWS Regions.



MAA Gold and MAA Platinum Option 1 (Oracle AI Database 26ai or higher with Oracle Active Data Guard) evaluations build upon the Silver MAA evaluation, plus:

- Network tests between primary and standby Exadata VM Clusters using OCI peered or AWS peered networks to evaluate round-trip latency and bandwidth
- Oracle Data Guard role transition performance and timings for disaster recovery use cases
- Database rolling upgrade with Data Guard

Data Guard Considerations and Network Peering

Oracle Data Guard maintains an exact physical copy of the primary database by transmitting (and applying) all data changes (redo) to the standby database across the network, making network throughput, and in some cases, latency, critical to the implementation's success.

Use Data Guard switchover for planned maintenance or disaster recovery tests. If the primary database becomes unavailable, use Data Guard failover to resume service.

Peering Networks Between Primary and Standby

Choosing your network peering option is an important decision to ensure your standby can keep up with the primary database. If the network does not have sufficient bandwidth to support single-process redo throughput, the standby will have a growing transport lag.

The primary and standby Exadata VM Clusters are deployed in separate networks. Oracle Database@AWS Exadata VM Clusters are always deployed using separate Virtual Cloud Networks (VCN) in OCI. These separate VCNs must be connected to allow traffic to pass between them; that is, they must be "peered" before enabling Data Guard with Oracle Cloud Automation. For this reason, the networks must use separate, non-overlapping IP CIDR ranges.

When creating a standby database across Availability Zones (AZs), consider

- If zero data loss is a business requirement, a lower RTT latency can reduce application impact.
- To reduce standby transport lags and potential data loss, sufficient network bandwidth is required to handle peak application and database workloads. Evaluate your peak change rate (for example, database redo rate).

When creating a standby database across regions, consider

- Due to higher RTT latency across regions, most Data Guard configurations use asynchronous redo transport for their cross region standby database, which essentially eliminates any performance overhead with sending redo to the standby.
- To reduce standby transport lags and data loss, sufficient network bandwidth is required to handle peak application and database workloads. Evaluate your peak change rate (for example, database redo rate) for each database instance and compare that to the single process throughput of a tool like iperf.

You can leverage the recommendations in the following use cases regarding expected network performance when peering with OCI before setting up your Data Guard and standby database. You can also run your own network experiments after peering and before you configure your standby. Refer to [MAA Evaluations on Multicloud Solutions](#) for the tests that were conducted as part of this certification.

Use Case: Cross-AZ Data Guard Between Exadata VM Clusters

This use case was evaluated for:

- Redo Transport
- Database migration or standby database instantiation

RTT Latency Observed:

- 1700 μ s with OCI peering (US-East)
- 1100 μ s with AWS peering (US-East-2)

OCI Peering Network Throughput Observed (US-East-2):

- ~9 Gbps single process throughput.
- Maximum 18 Gbps per node with parallelism.

AWS Peering Network Throughput Observed (US-East-2):

- ~5 Gbps single process throughput
- Maximum ~95 Gbps per node with parallelism

MAA Recommendations:

- For OCI peering, see [Local VCN Peering using Local Peering Gateways](#).
- For cross AZs, see [Implement Disaster Recovery with Cross-Zone Data Guard on Oracle Database@AWS](#).

Note

Different results may be observed in different regions and configurations. Always test to ensure your specific requirements can be achieved.

Use Case: Cross-Region Data Guard Between Exadata VM Clusters

This use case was evaluated for:

- Redo Transport
- Database migration or standby database instantiation

RTT Latency Observed (US-East-1 to US-West-2): Varies between different regions. Regions used Ashburn, VA, to Boardman, OR.

- 93 milliseconds observed with OCI peering

Network Throughput Observed (US-East-1 to US-West-2): Varies based on placement and VM size

- 1.3 Gbps for single process throughput with OCI peering
- 35 Gbps for multiple (64) parallel processes throughput with OCI peering

MAA Recommendations:

- For OCI peering, see [Remote VCN Peering using Dynamic Routing Gateways](#). OCI peering is recommended and provides optimized network bandwidth, better network throughput, and reduced cost (the first 10 TB/month is free).
- For cross-region placement, see [Implement disaster recovery with cross-regional Active Data Guard on Oracle Database@AWS](#).

Note

Different results may be observed in different regions and configurations. Always test to ensure your specific requirements can be achieved.

Path to Gold MAA Architecture

To achieve a Gold Maximum Availability Architecture (MAA), Oracle recommends the following principles for your cross-region deployment:

1. **Deploy Infrastructure:** In both your primary and standby regions, deploy an Exadata Infrastructure.
2. **Create VM Clusters:** On each infrastructure, create an Exadata VM Cluster within the ODB Network.
3. **Instantiate Databases:** Deploy your Oracle Real Application Clusters (RAC) database on the Exadata VM Cluster.
4. **Configure Data Replication:** Set up Oracle Data Guard to replicate data between the two databases across the regions.

Networking for Data Guard

When Exadata VM Clusters are created in AWS, each resides within its own Oracle Cloud Infrastructure (OCI) Virtual Cloud Network (VCN). To enable Data Guard communication, which is essential for shipping redo logs and performing role transitions, these VCNs must be peered. OCI peering is recommended for the most optimized Data Guard networking as it has been optimized for Data Guard redo throughput with minimal latency.

Enabling Data Guard

Once the network is peered using one of the above options, you can enable Data Guard (see [Use Oracle Data Guard with Exadata Cloud Infrastructure](#)). All Data Guard and MAA best Practices are built in. Data Guard adds to the comprehensive data protection and provides minimum data loss and downtime for the database, cluster, or even regional failures.

Data Guard Role Transitions

Data Guard switchover and failover performance was within expectations compared to a similar setup in Oracle OCI. Application downtime when performing a Data Guard switchover and failover can range from 30 seconds to a few minutes.

For guidance on tuning Data Guard role transition timings or examples of role transition timings, see [Tune and Troubleshoot Oracle Data Guard](#). Note that Oracle Cloud console timings do not reflect actual database downtime. To minimize downtime and achieve the low RTO of seconds to minutes, automatic Data Guard failover is required. Setting Data Guard Fast-Start Failover is a manual step. Refer to the Fast-Start Failover and [Configure and Deploy Oracle Data Guard](#) documentation.

Enhancing Availability and Reducing RTO for Disasters

To minimize downtime and achieve the low RTO of seconds to minutes, automatic Data Guard failover is required.

The Fast Start Failover (FSFO) feature automatically initiates a failover and requires you to install a Data Guard Observer on a separate VM. For optimal results, place the Observer VM in a different location from the primary and standby databases. If no additional location is available, place the Observer on the primary database site, preferably in the application network.

Note

This is a manual configuration step and is not part of the standard cloud automation.

For detailed setup instructions, see [Implement disaster recovery with cross-regional Active Data Guard on Oracle Database@AWS](#).

For additional details, refer to the Fast-Start Failover and [Configure and Deploy Oracle Data Guard](#) documentation.

Oracle Data Guard Hybrid Cloud Configuration

A hybrid Oracle Data Guard configuration consists of a primary database and one or more standby databases residing partially on-premises and partially in the Oracle OCI cloud or Oracle multicloud (for example, Oracle Database@Azure, Oracle Database@Google, or Oracle Database@AWS).

Note that the term **Oracle Cloud** refers to both Oracle OCI and Oracle multicloud going forward.

The process detailed in this topic uses the Oracle Database as a Service (DBaaS) tools to create a cloud standby database from an existing on-premises primary database. Oracle DBaaS tools streamline and simplify the process of creating the standby database in Oracle Cloud while incorporating MAA best practices.

After establishing the cloud standby database as described here, you can perform a role transition so that the primary database runs in the cloud instead of on-premises, and Oracle Data Guard life-cycle operations (switchover, failover, and reinstate) are available using the `dbactl/dbaascli` commands.

Note

Additional manual steps may be required during configuration if there are standby databases residing on-premises.

Benefits Of Hybrid Data Guard in the Oracle Cloud

The following are the primary benefits to using a hybrid Data Guard configuration in the Oracle Cloud.

- Flexibility to use Oracle Cloud Infrastructure data centers to supplement an on-premises data center footprint for disaster recovery or other purposes
- Oracle manages the cloud data center and infrastructure
- Ability to switch over (planned events) or fail over (unplanned events) production to the standby database in the cloud during scheduled maintenance or unplanned outages.

Once a failed on-premises database is repaired, it can be synchronized with the current production database in the cloud. Then, production can be switched back to the on-premises database if required.

- Use the same Oracle MAA best practices as the on-premises deployment. Additional Oracle MAA best practices specific to hybrid Data Guard deployments are specified in the topics that follow. When configured with MAA best practices, a hybrid Data Guard configuration provides:
 - Recovery Time Objective (RTO) of seconds when configured with Data Guard fast-start failover and its automatic failover benefits
 - Recovery Point Objective (RPO) less than a second for Data Guard with ASYNC transport

- RPO zero for Data Guard in a Maximum Availability protection mode that leverages a SYNC or FAR SYNC configuration

MAA Recommendations for Using Exadata Cloud for Disaster Recovery

When deploying Exadata Cloud for Disaster Recovery, Oracle MAA recommends:

- Create a cloud database system target that is symmetric or similar to the on-premises primary database to ensure performance SLAs can be met after a role transition. For example, create an Oracle RAC target for an Oracle RAC source, Exadata for Exadata, and so on.
- Ensure that network bandwidth can handle peak redo rates in addition to existing network traffic.

My Oracle Support document [Assessing and Tuning Network Performance for Data Guard and RMAN \(Doc ID 2064368.1\)](#) provides additional network bandwidth troubleshooting guidance for assessing and tuning network performance for Data Guard and RMAN.

- Ensure network reliability and security between on-premises and the Cloud environment.
- Use Oracle Active Data Guard for additional automatic block repair, data protection, and offloading benefits.
- Use Oracle Transparent Data Encryption (TDE) for both primary and standby databases.

My Oracle Support document [Oracle Database Tablespace Encryption Behavior in Oracle Cloud \(Doc ID 2359020.1\)](#) has additional details on TDE behavior in cloud configurations.

- Configure backups to Autonomous Recovery Service or Object Storage for the Oracle databases, in primary or standby role. See [Manage Database Backup and Recovery on Oracle Exadata Database Service on Dedicated Infrastructure](#) and [Database Autonomous Recovery Service](#).

Security Requirements and Considerations

Oracle MAA best practices recommend using Oracle Transparent Data Encryption (TDE) to encrypt the primary and standby databases to ensure that data is encrypted at-rest.

Using TDE to protect data is an essential part of improving the security of the system. Note the following variables that need to be considered when evaluating TDE.

- **CPU overhead** - Encryption requires additional CPU cycles to calculate encrypted and decrypted values. TDE, however, is optimized to minimize the overhead by taking advantage of database caching capabilities and leveraging hardware acceleration within Exadata. Most TDE users see little performance impact on their production systems after enabling TDE.
- **Lower data compression** - Encrypted data compresses less efficiently than non-encrypted data, so any compression applied to data encrypted with TDE results in lower compression ratios. When TDE encryption is used, redo transport compression is not recommended or typically required since data in the redo is already encrypted; however, when TDE is used in conjunction with Oracle Database compression technologies such as Advanced Compression or Hybrid Columnar Compression, compression is performed before the encryption occurs, and the benefits of compression and encryption are both achieved.

- **Key management** - Encryption is only as strong as the encryption key used and the loss of the encryption key is tantamount to losing all data protected by that key.

If encryption is enabled on a few databases, keeping track of the key and its life cycle is relatively easy. As the number of encrypted databases grows, managing keys becomes an increasingly difficult problem. If you are managing a large number of encrypted databases, it is recommended that Oracle Key Vault be used on-premises to store and manage TDE master keys.

Data can be converted during the migration process, but it is recommended that TDE be enabled before beginning the migration to provide the most secure Oracle Data Guard environment. A VPN connection or Oracle Net encryption is also required for inflight encryption for any other database payload that is not encrypted by TDE, such as data file or redo headers for example. See My Oracle Support document [Oracle Database Tablespace Encryption Behavior in Oracle Cloud \(Doc ID 2359020.1\)](#) for more information.

If the on-premises database is not already enabled with TDE, see My Oracle Support document [Primary Note For Transparent Data Encryption \(TDE \) \(Doc ID 1228046.1\)](#) to enable TDE and create wallet files.

If TDE cannot be enabled for the on-premises database, see Encryption of Tablespaces in an Oracle Data Guard Environment in *Oracle Database Advanced Security Guide* for information about decrypting redo operations in hybrid cloud disaster recovery configurations where the Cloud database is encrypted with TDE and the on-premises database is not.

Platform, Database, and Network Prerequisites

The following requirements must be met to ensure a successful migration to a Cloud standby database.

Requirement Type	On-Premises Requirements	Oracle Cloud Requirements
Operating System	Linux, Windows or Solaris X86 (My Oracle Support Note 413484.1 for Data Guard cross-platform compatibility)	Oracle Enterprise Linux (64-bit)
Oracle Database Version*	Extreme performance / BYOL* See Supported Database Editions and Versions for information about database service options in Oracle Cloud.	Extreme performance / BYOL* See Supported Database Editions and Versions for information about database service options in Oracle Cloud.
Oracle Database Architecture	Oracle RAC or single-instance	Oracle RAC or single-instance
Oracle Multitenant	For Oracle 12.1 and above, the primary database must be a multitenant container database (CDB)	Multitenant container database (CDB) or non-CDB
Physical or Virtual Host	Physical or Virtual	Exadata Virtual
Database Size	Any size	Any size. For shape limits please consult Exadata Cloud documentation
TDE Encryption	Recommended	Mandatory for Cloud databases

* The Oracle Database release on the primary and standby databases should be the same database major release and database release update (RU) during initial standby instantiation. For database software updates that are standby-first compatible, the primary and standby database Oracle Home software can be different (for example, 19RU vs 19 RU+1). For the standby instantiation in the Oracle cloud, the standby database Oracle Home software must be

the same or a later RU. See [Oracle Patch Assurance - Data Guard Standby-First Patch Apply \(Doc ID 1265700.1\)](#).

Cloud Network Prerequisites

Data transfers from on-premises to Oracle Cloud use the public network, VPN, and/or the high bandwidth option provided by Oracle FastConnect.

In an Oracle Data Guard configuration, the primary and standby databases must be able to communicate bi-directionally. This requires additional network configuration to allow access to ports between the systems.

Note

Network connectivity configuration is not required for Oracle Exadata Database Service on Cloud@Customer because it is deployed on the on-premises network. Skip to [On-Premises Prerequisites](#) if you are using ExaDB-C@C.

Secure Connectivity

For Oracle Exadata Database Service (not required for ExaDB-C@C) there are two options to privately connect the virtual cloud network to the on-premises network: FastConnect and IPSec VPN. Both methods require a Dynamic Routing Gateway (DRG) to connect to the private Virtual Cloud Network (VCN).

See [Access to Your On-Premises Network](#) for details about creating a DRG.

- **OCI FastConnect** - Provides an easy way to create a dedicated, private connection between the data center and OCI. FastConnect provides higher bandwidth options and a more reliable and consistent networking experience compared to internet-based connections. See [FastConnect Overview](#). (link <https://docs.oracle.com/en-us/iaas/Content/Network/Concepts/fastconnectoverview.htm>) for details.
- **IPSec VPN** - Internet Protocol Security or IP Security (IPSec) is a protocol suite that encrypts the entire IP traffic before the packets are transferred from the source to the destination. See [Site-to-Site VPN Overview](#) for an overview of IPSec in OCI.

Public Internet Connectivity

Connectivity between OCI and on-premises can also be achieved using the public internet.

This method is not secure by default; additional steps must be taken to secure transmissions. The steps for hybrid Data Guard configuration assume public internet connectivity.

By default, cloud security for port 1521 is disabled. Also, this default pre-configured port in the cloud for either a Virtual Machine (VM) or Bare Metal (BM) has open access from the public internet.

1. If a Virtual Cloud Network (VCN) for the standby database doesn't have an Internet Gateway, one must be added.
To create an internet gateway see [Internet Gateway](#).
2. Ingress and egress rules must be configured in the VCN security list to connect from and to the on-premises database.

See [Security Lists](#) for additional information.

On-Premises Prerequisites

The following prerequisites must be met before instantiating the standby database.

Evaluate Network Using `oracdpctest`

In an Oracle Data Guard configuration, the primary and standby databases transmit information in both directions. This requires basic configuration, network tuning, and opening of ports at both the primary and standby databases.

It is vital that the bandwidth exists to support the redo generation rate of the primary database.

Follow instructions in [Assessing and Tuning Network Performance for Data Guard and RMAN \(Doc ID 2064368.1\)](#) to assess and tune the network link between the on-premises and cloud environments.

Configuration

- Name resolution
 - For ExaDB-C@C, because the clusters reside on the on-premises network, the on-premises DNS should resolve each cluster, and no further configuration should be necessary.
 - For Oracle Exadata Database Service, name resolution between the clusters must be configured.

This can be done either using a static file like `/etc/hosts`, or by configuring the on-premises DNS to properly resolve the public IP address of the OCI instance. In addition, the on-premises firewall must have Access Control Lists configured to allow SSH and Oracle Net to be accessed from the on-premises system to OCI.
- Oracle Data Guard in a DR configuration requires access from the Cloud instance to the on-premises database; the primary database listener port must be opened with restricted access from the Cloud IP addresses using features like `iptables`.

Because every corporation has different network security policies, the network administrator must perform operations like the cloud-side network configuration shown in [Cloud Network Prerequisites](#).
- Prompt-less SSH from Oracle Cloud to the on-premises machine. This is configured both for on-premises to Cloud during the provisioning process and from the Cloud to on-premises.
- The configuration of the on-premises firewall to allow inbound SSH connectivity from the Cloud to the on-premises machine.
- It is strongly recommended that you complete the network assessment described above in Evaluate Network Using `oracdpctest`. Setting the appropriate TCP socket buffers setting is especially important for ASYNC redo transport.
- It is recommended that the RDBMS software be the same on the primary and standby database for instantiation. If the current on-premises Oracle Database release is not available in Oracle Exadata Database Service, then the primary database must be on the same major database release and the same or lower Release Update (RU).

Implement MAA Best Practice Parameter Settings on the Primary Database

Most MAA best practices for Data Guard are part of the process described here; however, the Standby Redo Log should be created on the primary database before starting this process.

See [Oracle Data Guard Configuration Best Practices](#) for information.

Validating Connectivity between On-Premises and Exadata Cloud Hosts

After the networking steps are implemented successfully, run the command below to validate that the connection is successful between all sources and all targets in both directions.

1. As the oracle OS user on all nodes of the on-premises environment, use `curl` to test connectivity between all on-premises and all cloud nodes.

```
[oracle@on-prem1]$ curl -v telnet://<FQDN-CLOUD-HOST1>:22 (or other
available port)
* Rebuilt URL to: telnet://<FQDN-CLOUD-HOST1>:22/
* Trying <CLOUD-HOST1-IP>...
* TCP_NODELAY set
* Connected to <FQDN-CLOUD-HOST1> (<CLOUD-HOST1-IP>) port 22 (#0)
SSH-2.0-OpenSSH_8.0
^C
[oracle@on-prem1]$
```

2. As the oracle OS user on all nodes of the cloud environment, use `curl` to test connectivity between all cloud and all on-premises nodes.

```
[oracle@cloud-exadata1]$ curl -v telnet://<FQDN-ON-PREM-HOST1>:22 (or
other available port)
* Rebuilt URL to: telnet://<FQDN-CLOUD-HOST1>:22/
* Trying <ON-PREM-HOST1-IP>...
* TCP_NODELAY set
* Connected to <FQDN-ON-PREM-HOST1> (<ON-PREM-HOST1-IP>) port 22 (#0)
SSH-2.0-OpenSSH_8.0
^C
[oracle@cloud-exadata1]$
```

- 3.

If `curl` connection is successful, proceed to the next step.

Note

Successful connections will get the message "Connected to <FQDN-HOST1> (<HOST-IP>) port 22", while unsuccessful connections will get the message "Failed to connect to <FQDN-HOST1> port 22: Connection refused"

Prepare the Primary Database Environment

The Hybrid Data Guard configuration process uses the Oracle Cloud DBaaS tools and automation to ensure the process used to create the standby database is the same process as is used in the Oracle Cloud.

This will ensure that the cloud database is visible in the cloud user interface.

Because the on-premises database is not a cloud created database, some steps are required to ensure successful completion.

Create an ACFS Mount Point

Hybrid Data Guard configurations require an ACFS mount point for sharing files between instances, for example `tnsnames.ora` and TDE wallets.

This mount point exists in cloud platforms in `/var/opt/oracle/dbaas_acfs`. If there is no existing ACFS mount point for the primary cluster create one using the steps in [Creating an Oracle ACFS File System](#). (An existing ACFS mount point can be used)

Configure Transparent Data Encryption on the Source Database

Transparent Data Encryption (TDE) is required on Oracle Cloud databases, including any standby database which is part of a hybrid Data Guard configuration.

While it is strongly recommended that the on-premises database also be encrypted, leaving the primary database unencrypted as part of a hybrid Data Guard configuration can be configured and is better supported by new parameters in Oracle Database 19c (19.16) and later releases.

For all TDE configurations with Oracle Data Guard, the encryption wallet must be created on the primary database and the master key must be set whether or not the primary database will be encrypted with TDE.

The parameters required for TDE configuration differ depending with Oracle Database releases. The values may be different for each database in the Data Guard configuration.

- In Oracle Database release 19c (19.16) and later, the parameters `TABSPACE_ENCRYPTION`, `WALLET_ROOT`, and `TDE_CONFIGURATION` are required to properly configure TDE.
- For Oracle Database 19c releases before 19.16, set parameters `WALLET_ROOT`, `TDE_CONFIGURATION`, and `ENCRYPT_NEW_TABLESPACES`.
- For releases earlier than Oracle Database 19c, set parameters `ENCRYPTION_WALLET_LOCATION` and `ENCRYPT_NEW_TABLESPACES`.

Note

Unless otherwise specified by the `TABSPACE_ENCRYPTION=DECRYPT_ONLY` parameter, a new tablespace's encryption on the standby database will be the same as that of the primary.

In the following table use the links to find references for setting the primary and standby database parameters.

Parameter	Definition	All Oracle Database releases before 19c	Oracle Database release 19.15 and earlier	Oracle Database release 19.16 and later
ENCRYPTION_WALLET_LOCATION	Defines the location of the wallet	RECOMMENDED	DEPRECATED	DEPRECATED
WALLET_ROOT and TDE_CONFIGURATION	WALLET_ROOT sets the location of the root of the directory for wallet storage for each PDB in a CDB. TDE_CONFIGURATION defines the type of keystore. For example, FILE for a wallet keystore. The keystore type must be set to the same value on the primary and standby database.	N/A	RECOMMENDED	RECOMMENDED
ENCRYPT_NEW_TABLESPACES	Indicates whether a new tablespace on the primary database should be encrypted The ENCRYPT_NEW_TABLESPACES parameter can be set as follows: • CLOUD_ONLY - Default setting. Any new tablespaces created are transparently encrypted with the AES128 algorithm, unless a different algorithm is specified in the ENCRYPTION clause in the CREATE TABLESPACE statement. For on-premises databases, tablespaces are only encrypted if the CREATE TABLESPACE . . . ENCRYPTION clause is specified. • ALWAYS - Any new tablespace created in a primary database, on-premises or in the cloud, will be transparently encrypted with the AES128 algorithm, unless a different encryption algorithm is specified in the CREATE TABLESPACE ENCRYPTION clause. • DDL - Allows you to create tablespaces with or without encryption following the CREATE TABLESPACE command, and also lets you change the encryption algorithm. Note: This value is not applicable for cloud primary databases with releases from Oracle Database 19c (19.16) and later because tablespace encryption is enforced.	RECOMMENDED	RECOMMENDED	NOT RECOMMENDED Override with recommended setting for TABLESPACE_ENCRYPTION

Parameter	Definition	All Oracle Database releases before 19c	Oracle Database release 19.15 and earlier	Oracle Database release 19.16 and later
TABSPACE_ENCRYPTION (see note above)	Oracle Database 19c (19.16) and later releases - indicates whether a new tablespace should be encrypted. Available options are AUTO_ENABLE, MANUAL_ENABLE, and DECRYPT_ONLY. Starting with Oracle Database 19c (19.16), Oracle Cloud forces encryption for all tablespaces in the cloud database. This cannot be overridden. To prevent encrypted tablespaces on an on-premises database (primary or standby) set the TABSPACE_ENCRYPTION parameter to DECRYPT_ONLY. DECRYPT_ONLY is only valid in an on-premises database.	N/A	N/A	RECOMMENDED

To configure Transparent Data Encryption using a TDE wallet follow the steps in Configuring Transparent Data Encryption.

Check the TDE Master Key Before Instantiation

Even in cases where the primary database remains unencrypted, TDE must be configured on the primary database. This configuration includes creating the encryption wallet and setting the master key.

During the process the wallet is copied to the standby database. The master key stored in the wallet will be used by the standby database for encryption.

In the event of a switchover where the cloud standby database becomes the primary database, the key is used by the unencrypted on-premises database to decrypt the encrypted redo from the cloud database.

Failure to set the master key will result in failure of Data Guard managed recovery.

To confirm the master key is set properly:

- Verify that the MASTERKEYID column in V\$DATABASE_KEY_INFO matches a key existing in V\$ENCRYPTION_KEYS on the source database.

In a multitenant container database (CDB) environment, check CDB\$R00T and all the PDBs except PDB\$SEED.

Configure Online Redo Logs

Redo log switches can have a significant impact on redo transport and apply performance.

Follow these best practices for sizing the online redo logs on the primary database before instantiation.

- All online redo log groups should have identically sized logs (to the byte).
- Online redo logs should reside on high performing disks (DATA disk groups) and on high redundancy disk groups if possible.

- Create a minimum of three online redo log groups per thread of redo on Oracle RAC instances.
- Create online redo log groups on shared disks in an Oracle RAC environment.
- Do not multiplex online redo logs (multiple members per log group) unless they are not placed on high redundancy disk groups.
- Size online redo logs to switch no more than 12 times per hour (every ~5 minutes) during peak workloads.

Size Redo Logs

Size redo logs based on the peak redo generation rate of the primary database.

You can determine the peak rate by running the query below for a time period that includes the peak workload.

The peak rate could be seen at month-end, quarter-end, or annually. Using the guidance above, size the redo logs to handle the highest rate +10% in order for redo apply to perform consistently during these workloads.

```
SQL> SELECT thread#,sequence#,blocks*block_size/1024/1024 MB,
(next_time-first_time)*86400 sec,
blocks*block_size/1024/1024)/((next_time-first_time)*86400) "MB/s"
FROM v$sarchived_log
WHERE ((next_time-first_time)*86400<>0)
and first_time between to_date('2015/01/15 08:00:00','YYYY/MM/DD HH24:MI:SS')
and to_date('2015/01/15 11:00:00','YYYY/MM/DD HH24:MI:SS')
and dest_id=1 order by first_time;
```

THREAD#	SEQUENCE#	MB	SEC	MB/s
2	2291	29366.1963	831	35.338383
1	2565	29365.6553	781	37.6000708
2	2292	29359.3403	537	54.672887
1	2566	29407.8296	813	36.1719921
2	2293	29389.7012	678	43.3476418
2	2294	29325.2217	1236	23.7259075
1	2567	11407.3379	2658	4.29169973
2	2295	29452.4648	477	61.7452093
2	2296	29359.4458	954	30.7751004
2	2297	29311.3638	586	50.0193921
1	2568	3867.44092	5510	.701894903

Choose the redo log size based on the peak generation rate with the following chart.

Peak Redo Rate	Recommended Redo Log Size
<= 1 MB/s	500 MB
<= 5 MB/s	1 GB
<= 25 MB/s	8 GB
<= 50 MB/s	16 GB
> 50 MB/s	32 GB

Enable Flashback Database

Flashback Database allows reinstatement of the old primary database as a standby database after a failover.

Without Flashback Database enabled, the old primary database would have to be recreated as a standby after a failover. If flashback database has not already been enabled, enable it now.

To enable flashback database, make sure you have sufficient space and I/O throughput in your Fast Recovery Area or RECO disk group, and evaluate any performance impact.

As the oracle OS user on the first node of the on-premises environment, run the command below to enable flashback database on the primary if it is not already enabled.

```
[oracle@on-prem1]$ echo "alter database flashback on;" | sqlplus -SILENT "/"
as sysdba"
```

Database altered.

Configure Redo Transport - Oracle Net Encryption

To protect against plain text or unencrypted tablespace redo from being visible on the WAN, place the following entries in the sqlnet.ora file on all on-premises database nodes.

These values should already be set by the deployment tool in cloud configurations.

```
#SQLNET.ORA ON ON-PREMISES HOST(S)
SQLNET.CRYPTO_CHECKSUM_CLIENT=ACCEPTED
SQLNET.CRYPTO_CHECKSUM_SERVER=ACCEPTED
SQLNET.CRYPTO_CHECKSUM_TYPES_CLIENT=(SHA256,SHA384,SHA512,SHA1)
SQLNET.CRYPTO_CHECKSUM_TYPES_SERVER=(SHA256,SHA384,SHA512)
SQLNET.ENCRYPTION_CLIENT=REQUESTED
SQLNET.ENCRYPTION_SERVER=REQUESTED
SQLNET.ENCRYPTION_TYPES_CLIENT=(AES256,AES192,AES128)
SQLNET.ENCRYPTION_TYPES_SERVER=(AES256,AES192,AES128)
SQLNET.IGNORE_ANO_ENCRYPTION_FOR_TCPS=TRUE
SSL_CIPHER_SUITES=(SSL_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,SSL_ECDHE_ECDSA_WITH_
AES_256_GCM_SHA384,SSL_ECDHE_RSA_WITH_AES_128_GCM_SHA256,SSL_ECDHE_RSA_WITH_
AES_256_GCM_SHA384)
SSL_CLIENT_AUTHENTICATION=FALSE
SSL_VERSION=1.2
```

Investigate Log for Errors (TFA)

As the root OS user on the first node of the on-premises environment, use Oracle Trace File Analyzer to analyze all of your logs across your cluster to identify recent database errors.

The Event Summary gives you a real-time report the database status.

```
[root@on-prem1]# tfactl events -component RDBMS -database [DBname] -last 7d
```

Output from host : on-prem1

Event Summary:

INFO :0

ERROR :0

WARNING :0

Event Timeline:

No Events Found

Output from host : on-prem2

Event Summary:

INFO :0

ERROR :0

WARNING :0

Event Timeline:

No Events Found

Note

Review the result and investigate any failures before moving forward. Refer to [Collecting and Analyzing Oracle Database Diagnostic Data](#) for more information.

Instantiate the Standby Using Oracle DBaaS Tools

Prepare the on-premises environment and instantiate the standby database using the DBaaS Tools `prepareForStandby` and `configureStandby` work flows.

Task 1: Install dbcactl in the On-Premises Environment

Database Configuration Assistant Utility (dbcactl) is a standalone software package which is part of Oracle Database product suite. It is designed to assist in the database creation, configuration and lifecycle management of Oracle Database.

Follow instructions in My Oracle Support document [Database Configuration Assistant Utility \(dbcactl\) - Standalone Software Package \(Doc ID 3099785.1\)](#) to download the utility used during this process.

Task 2: Prepare the Cloud Environment for Instantiation of the Standby Database

The target `ORACLE_HOME` for the standby database must already exist on the standby system. Use the following steps to create the home if one does not already exist.

It is recommended that the home for the standby database be the same version, RU and include one-off patches. If the same RU is not available, a more recent RU of the same major version can be used.

Install an Oracle Home for the Standby Database

1. As the oracle OS user on the first node of the on-premises environment, use `opatch` to list the database release.

```
[oracle@on-prem1]$ $ORACLE_HOME/OPatch/opatch lspatches | grep 'Database Release Update'
```

```
36912597;Database Release Update : 19.25.0.0.241015 (36912597)
```

2. As the root OS user on the first node of the cloud environment, use `dbaascli` to list the available images matching the on-premises release.

```
[root@cloud-exadata1 ~]# dbaascli cswLib showImages | grep -A 1 '19.25'
```

```
2.IMAGE_TAG=19.25.0.0.0
  VERSION=19.25.0.0.0
  DESCRIPTION=19c OCT 2024 DB Image
```

3. As the root OS user on the first node of the cloud environment, use `dbaascli` to download the available image matching the on-premises release.

```
[root@cloud-exadata1 ~]# dbaascli cswlib download --version 19.25.0.0.0
...
dbaascli execution completed
```

4. As the root OS user on the first node of the cloud environment, use `dbaascli` to create the target `ORACLE_HOME` matching the on-premises release.

```
[root@cloud-exadata ~]# dbaascli dbHome create --version 19.25.0.0.0

-----
Running Plugin_initialization job
...
----- START OF PLUGIN RESULT -----
{"ORACLE_HOME_NAME":"OraHome4","ORACLE_HOME":"/u02/app/oracle/product/
19.0.0.0/dbhome_3"}
----- END OF PLUGIN RESULT -----

dbaascli execution completed
```

Note

Take note of the `ORACLE_HOME_NAME`, you will need it for the migration.

The Oracle Home on the cloud system can also be created using the OCI User Interface.

If Necessary, Modify Instantiation Parallelism

The default settings for instantiation will use four (4) RMAN channels on each standby database instance for instantiation (for example, a 2-node standby cluster = 8 total channels). This default balances impact on the systems involved with speed of data file copy.

However, for very large databases where the system resource impact is not a concern, more channels can be configured to help the file copy complete faster.

This control is in a configuration file on the standby system which can be updated with `dbaascli`.

On the first standby node as root:

1. Confirm the current setting.

```
[root@cloud-exadata ~]# dbaascli dataguard showDGConfigAttributes |  
grep restore_channels_count "restore_channels_count" : "4",
```

2. Update to the desired PER NODE parallelism using `dbaascli`.

```
[root@cloud-exadata ~]# dbaascli dataguard updateDGConfigAttributes  
--attributes restore_channels_count=6  
<...>  
The following Data Guard configuration attributes were updated  
successfully:  
{  
  "dgConfigurationAttributes" : {  
    "restore_channels_count" : "6"  
  }  
}
```

3. Verify the new setting.

```
[root@cloud-exadata ~]# dbaascli dataguard showDGConfigAttributes |  
grep restore_channels_count "restore_channels_count" : "6",
```

Note

This change impacts all future instantiations, whether for Hybrid Data Guard or OCI standby database creation. It is recommended that you reset the value to the default after this instantiation completes.

Task 3: Instantiate the Standby Database

After the preparations are complete you can run the Oracle DBaaS Tools `prepareForStandby` and `configureStandby` work flows to instantiate the cloud standby database.

You will actually run two jobs using the Oracle DBaaS Tools:

- Run `dbctl` operation `prepareForStandby` in the on-premises environment.
- Run `dbaascli` operation `configureStandby` in the cloud environment.

Configure ACFS

As the root OS user on the first node of the on-premises environment, create the ACFS mount point required to execute prepareForStandby.

```
[root@on-prem1]# $(grep ^crs_home /etc/oracle/olr.loc | cut -d=
-f2)/bin/crsctl stat res -w "TYPE == ora.acfs.type" |grep NAME
NAME=ora.datac1.acfsvol01.acfs
```

```
[root@on-prem1]# $(grep ^crs_home /etc/oracle/olr.loc | cut -d= -f2)/bin/
crsctl
stat res ora.datac1.acfsvol01.acfs -f |grep ^MOUNTPOINT_PATH
MOUNTPOINT_PATH=/acfs01
```

Create a directory for the dbname (oradb1 for this example):

```
[root@on-prem1]# mkdir /acfs01/oradb1
[root@on-prem1]# chown oracle:oinstall /acfs01/oradb1
```

Run dbctl Operation prepareForStandby in the On-Premises Environment

Now you are prepared to start the process. First, you will prepare the primary database to become part of a Data Guard configuration. Gather the following information which will be passed to the workflow.

1. As the root OS user on the first node of the cloud environment, collect the standbyScanName and standbyScanPort required to execute dbctl.

```
[root@cloud-exadata1 ~]# $(grep ^crs_home /etc/oracle/olr.loc | cut
-d= -f2)/bin/srvctl config scan |grep 'SCAN name:' | awk '{print $3}'
cloud-exadata-scan.clientnet.default.oraclevcn.com,
```

```
[root@cloud-exadata1 ~]# dbaascli grid getDetails | grep
scanListenerTCPPorts | cut
-d "[" -f2 | cut -d "]" -f1
1521
```

Make note of these pieces of information to be passed to the workflow.

2. As the oracle OS user on the first node of the on-premises environment, collect the primary database unique name required to execute dbctl.

```
[oracle@on-prem1]$ echo "select DB_UNIQUE_NAME from v\$database;" |
sqlplus -SILENT "/" as sysdba"
```

```
DB_UNIQUE_NAME
-----
oradb1_onprem
```

- As the oracle OS user on the first node of the on-premises environment, set the environment variable ORACLE_HOME pointing to the on-premises database Oracle Home.

```
[oracle@on-prem1]$ srvctl config database -home
oradb1      /u01/app/oracle/product/19.0.0.0/dbhome_1      19.0.0.0.0

[oracle@on-prem1]$ export ORACLE_HOME=/u01/app/oracle/product/19.0.0.0/
dbhome_1
```

- With that information gathered, run the prepareForStandby work flow with the appropriate replacements.

As the oracle OS user on the first node of the on-premises environment, run dbcsctl operation prepareForStandby with corresponding arguments.

```
[oracle@on-prem1]$

<dbcsctl_folder>/dbcsctl \
-silent \
-oui_internal \
-configureDatabase \
-prepareForStandby \
-dgTNSNamesoraFilePath /acfs01/oradb1 \
-sourceDB oradb1_onprem \
-standbyDBUniqueName oradb1_cloud \
-standbyScanName cloud-exadata-scan.clientnet.default.oraclevcn.com \
-standbyScanPort 1521 \
-standbyDBDomain clientnet.default.oraclevcn.com \
-blobFileLocation /tmp
Loading DBCA...

SYS_PASSWORD_PROMPT
<ENTER_SYS_PASSWORD>

Session ID of the current execution is: 53
-----
Running Create_dg_services job
Completed Create_dg_services job
25% complete
-----
Running Update_tnsnames_ora_file job
Completed Update_tnsnames_ora_file job
50% complete
-----
Running Update_ifile_entry job
Completed Update_ifile_entry job
75% complete
-----
Running Prepare_blob_file job
Completed Prepare_blob_file job
100% complete
----- PLUGIN NOTES -----
Successfully created blob file: /tmp/
oradb1_2024-11-21_10-59-53AM_137640.tar
----- END OF PLUGIN NOTES -----
Look at the log file
```

```
"/u01/app/oracle/cfgtoollogs/dbca/oradb1_onprem/oradb1_onprem.log"
for further details.
```

The workflow creates a zip file with required files to instantiate the standby database. For example, tnsnames.ora, TDE wallet, and so on. This TAR file is listed in the output of the command and must be copied from the to a location on the standby system.

5. As the oracle OS user on the first node of the on-premises environment, copy the generated blob file to the first node of the cloud environment.

```
[oracle@on-prem1]$ scp /tmp/oradb1_2024-11-21_10-59-53AM_137640.tar
opc@cloud-exadata1:/tmp
```

Run dbaascli Operation configureStandby in the Cloud Environment

1. First, gather some information that has to be passed to the configureStandby work flow.

As the oracle OS user on the first node of the on-premises environment, collect the primaryScanIPAddresses, primaryScanPort, and primaryServiceName required to execute dbaascli.

```
[oracle@on-prem1]$ $(grep ^crs_home /etc/oracle/olr.loc |
  cut -d= -f2)/bin/srvctl config scan |grep 'SCAN name:' | awk '{print $3}'
maaфра2vm01-oe6ab-scan.clientnet.maaфраdefault.oraclevcn.com
```

```
[oracle@on-prem1]$ srvctl config scan_listener -scannumber 1 |grep
Endpoints
Endpoints: TCP:1521/TCP:2484
```

```
[oracle@on-prem1]$ lsnrctl status | grep oradb1_onprem | cut -d'"' -f 2
oradb1_onprem.companyxyz.region.com
```

2. As the root OS user on the first node of the cloud environment, run dbaascli operation configureStandby with corresponding arguments.

```
[root@cloud-exadata1]#
dbaascli dataguard configureStandby \
--dbname oradb1 \
--oracleHome /u02/app/oracle/product/19.0.0.0/dbhome_3 \
--standbyDBUniqueName oradb1_cloud \
--primaryScanIPAddresses companyxyz.region.com \
--primaryScanPort 1521 \
--primaryServiceName oradb1_onprem.companyxyz.region.com \
--protectionMode MAX_PERFORMANCE \
--transportType ASYNC \
--activeDG true \
--standbyScanIPAddresses cloud-exadata-
scan.clientnet.default.oraclevcn.com \
--standbyScanPort 1521 \
--standbyBlobFromPrimary /tmp/oradb1_2024-11-21_10-59-53AM_137640.tar
```

Enter PRIMARY_DB_SYS_PASSWORD:

```

<ENTER_PRIMARY_DB_SYS_PASSWORD>
Enter PRIMARY_DB_TDE_PASSWORD:
<ENTER_PRIMARY_DB_SYS_PASSWORD>
Enter AWR_ADMIN_PASSWORD:
<ENTER_AWR_ADMIN_PASSWORD>
Enter AWR_ADMIN_PASSWORD (reconfirmation):
<ENTER_AWR_ADMIN_PASSWORD>

Loading PILOT...
Session ID of the current execution is: 10015
Log file location:
/var/opt/oracle/log/oradb1/dataguard/configureStandby/
pilot_2024-11-21_11-28-21-AM_270773
-----
Running Plugin_initialization job
Enter PRIMARY_DB_SYS_PASSWORD
*****
Enter PRIMARY_DB_TDE_PASSWORD
*****
Enter AWR_ADMIN_PASSWORD
*****
Completed Plugin_initialization job
-----
Running Validate_plugin_inputs job
Completed Validate_plugin_inputs job
-----
Running Default_database_values_initialization job
Completed Default_database_values_initialization job
-----
...
-----
Running Generate_dgconfig_details job
Acquiring native write lock: global_dgsystem_details_generation
Releasing native lock: global_dgsystem_details_generation
Completed Generate_dgconfig_details job
Releasing lock: oradb1
Releasing lock: _u02_app_oracle_product_19.0.0.0_dbhome_3
-----
Running Cleanup job
Completed Cleanup job

dbaascli execution completed

```

Note

The Oracle MAA best practice is for the standby to be open read-only to enable Automatic Block Media Recovery; Use the flag `--activeDG true`

3. As the root OS user on the first node of the cloud environment, run the following commands to monitor the progress of the `dbaascli` operation `configureStandby`.

```

[root@cloud-exadata1]# export dbName=oradb1
[root@cloud-exadata1]# export dbUniqueName=oradb1_cloud

```



```
[root@cloud-exadata1]# tail -20f `ls -t
/var/opt/oracle/log/$dbName/dataguard/configureStandby/pilot_* | head -1`

FINE: [2024-11-21 11:43:38.908 UTC][pool-334-thread-1]
[DBCAExecutor$DBCAExecutionProcessHandler.logOutput:96]
-----
FINE: [2024-11-21 11:43:38.910 UTC][pool-334-thread-1]
[DBCAExecutor$DBCAExecutionProcessHandler.logOutput:96]
Running Open_pdb job
FINE: [2024-11-21 11:43:42.661 UTC][pool-334-thread-1]
[DBCAExecutor$DBCAExecutionProcessHandler.logOutput:96]
Completed Open_pdb job
FINE: [2024-11-21 11:43:42.668 UTC][pool-334-thread-1]
[DBCAExecutor$DBCAExecutionProcessHandler.logOutput:96]
90% complete
FINE: [2024-11-21 11:43:42.692 UTC][pool-334-thread-1]
[DBCAExecutor$DBCAExecutionProcessHandler.logOutput:96]
-----
FINE: [2024-11-21 11:43:42.697 UTC][pool-334-thread-1]
[DBCAExecutor$DBCAExecutionProcessHandler.logOutput:96]
Running Create_services job

[root@cloud-exadata1]# tail -20f `ls -t
/var/opt/oracle/log/$dbName/dataguard/configureStandby/$dbNameUniqueName/
trace* | head -1`

[pool-3-thread-4] [ 2024-11-21 12:03:55.015 UTC ]
[CRSCache.getAttributesFromCRS:155]
CRS: name: ora.oradb1_cloud.oradb1_oradb1p3.paas.oracle.com.svc, type 1,
node: null
[pool-3-thread-4] [ 2024-11-21 12:03:55.015 UTC ]
[CRSCache.getAttributesFromCRS:156]
attrs: [GLOBAL]
[pool-3-thread-4] [ 2024-11-21 12:03:55.016 UTC ]
[CRSCache.getAttributesFromCRS:163]
CRS: [<GLOBAL:false>]
[pool-3-thread-4] [ 2024-11-21 12:03:55.016 UTC ]
[CRSNative.genericStartEntity:668]
[MAJOR EVENT] About to start resource:
Name: ora.oradb1_cloud.oradb1_oradb1p3.paas.oracle.com.svc, force:false
node: null,
options: 0, filter null
[pool-3-thread-4] [ 2024-11-21 12:03:55.016 UTC ]
[CRSNative.genericStartEntity:678]
filter = null
[pool-3-thread-4] [ 2024-11-21 12:03:55.016 UTC ]
[CRSNative.genericStartEntity:679]
node name = null

[root@cloud-exadata1]# tail -20f /u01/app/grid/diag/crs/`hostname`/crs/
trace/alert.log

2024-11-21 12:13:59.362 [CRSD(398369)]
CRS-2772: Server 'maafra3vm02-qbi0z1' has been assigned to pool
'ora.oradb1_oradb1_oradb1p3_ro.paas.oracle.com'.
2024-11-21 12:13:59.363 [CRSD(398369)]
```

```
CRS-2772: Server 'maafra3vm02-qbi0z2' has been assigned to pool
'ora.oradb1_oradb1_oradb1p3_ro.paas.oracle.com'.
2024-11-21 12:14:00.420 [CRSD(398369)]
CRS-2772: Server 'maafra3vm02-qbi0z1' has been assigned to pool
'ora.oradb1_oradb1_oradb1p4.paas.oracle.com'.
2024-11-21 12:14:00.421 [CRSD(398369)]
CRS-2772: Server 'maafra3vm02-qbi0z2' has been assigned to pool
'ora.oradb1_oradb1_oradb1p4.paas.oracle.com'.
```

Task 4: Validate the Standby Database

1. As the root OS user on the first node of the cloud environment, check the Oracle database configuration.

```
[root@cloud-exadata1]# dbaascli database getDetails --dbname oradb1 |
egrep 'dbName|dbRole|dbType|patchVersion'
"dbName" : "oradb1",
"dbRole" : "PHYSICAL_STANDBY",
"dbType" : "RAC",
"patchVersion" : "19.25.0.0.0",
"pdbName" : "ORADB1P1",
"pdbName" : "ORADB1P2",
"pdbName" : "ORADB1P3",
"pdbName" : "ORADB1P4",
"pdbName" : "ORADB1P5",
```

2. As the root OS user on the first node of the cloud environment, check the Oracle Data Guard Broker configuration.

```
[root@cloud-exadata1]# dbaascli dataguard getDetails --dbName oradb1 |
egrep 'protectionMode|status|dbUniqueName|dgRole|standbyType|
transportType|switchoverReadiness|failoverReadiness|redoTransportState|
databaseStatus'

"protectionMode" : "MAX_PERFORMANCE",
"status" : "SUCCESS",
"dbUniqueName" : "oradb1_onprem",
"dgRole" : "PRIMARY",
"standbyType" : null,
"transportType" : "ASYNC",
"switchoverReadiness" : "HEALTHY",
"failoverReadiness" : "HEALTHY",
"redoTransportState" : "ON",
"databaseStatus" : "SUCCESS",
"dbUniqueName" : "oradb1_cloud",
"dgRole" : "STANDBY",
"standbyType" : "PHYSICAL_STANDBY",
"transportType" : "ASYNC",
"switchoverReadiness" : "HEALTHY",
"failoverReadiness" : "HEALTHY",
"redoTransportState" : null,
"databaseStatus" : "SUCCESS",
```

'status' should be SUCCESS. If any other status is shown, re-run the command after waiting 2 minutes to give the Broker time to update. If issues persist, see the Oracle Data Guard Broker documentation to diagnose and correct any issues.

3. As the oracle OS user on the first node of the cloud environment, validate the standby database.

```
[oracle@cloud-exadata1]$ dgmgrl / 'validate database oradb1_cloud'
```

```
DGMGRL for Linux: Release 19.0.0.0.0 - Production on Thu Nov 21 15:26:55
2024
Version 19.25.0.0.0
```

```
Copyright (c) 1982, 2019, Oracle and/or its affiliates. All rights
reserved.
```

```
Welcome to DGMGRL, type "help" for information.
Connected to "oradb1_cloud"
Connected as SYSDB.
```

```
Database Role:      Physical standby database
Primary Database:   oradb1_onprem
```

```
Ready for Switchover: Yes
Ready for Failover:  Yes (Primary Running)
```

```
Managed by Clusterware:
  oradb1_onprem   : YES
  oradb1_cloud:   YES
```

Task 5: Offline Encrypt the Cloud Standby Database

If the primary database was encrypted with TDE before instantiation, skip this task.

If the primary database was not encrypted with TDE prior to instantiation, the standby in Oracle Cloud must be encrypted using fast offline encryption.

See [KB127996 Implementing Transparent Data Encryption \(TDE\) Using Standby-First Offline Encryption](#) to complete the encryption for the standby database only. That is, do not perform a switchover and encrypt the on-premises database. The on-premises database will remain unencrypted.

For complete details about mixed encryption Data Guard configurations see [KB18578 Oracle Database Tablespace Encryption Behavior in Oracle Cloud](#).

Task 6: Implement Recommended MAA Best Practices

After standby instantiation, evaluate implementing the following Oracle MAA best practices to achieve better data protection and availability.

While many of the recommended best practices are implemented as part of the workflows, some additional recommendations are not implemented as they may have a performance impact.

The additional key best practices are listed below. Also see [Oracle Data Guard Configuration Best Practices](#) for details about Oracle MAA recommended best practices for Oracle Data Guard.

Set Data Protection Parameters

MAA best practice recommendations include the following settings on the primary and standby databases.

```
db_block_checksum=TYPICAL
```

```
db_lost_write_protect=TYPICAL
```

```
db_block_checking=MEDIUM
```

```
SQL> show parameter db_block_checksum
```

NAME	TYPE	VALUE
db_block_checksum	string	TYPICAL

```
SQL> alter system set db_block_checksum=TYPICAL scope=both sid='*';
```

```
SQL> show parameter db_lost_write_protect
```

NAME	TYPE	VALUE
db_lost_write_protect	string	typical

```
SQL> alter system set db_lost_write_protect=TYPICAL scope=both sid='*';
```

```
SQL> show parameter db_block_checking
```

NAME	TYPE	VALUE
db_block_checking	string	OFF

```
SQL> alter system set db_block_checking=MEDIUM scope=both sid='*';
```

Note that the `db_block_checking` setting has an impact on primary database performance and should be thoroughly tested with a production workload in a lower, production-like environment.

If the performance impact is determined to be unacceptable on the primary database, the standby database should set `db_block_checking=MEDIUM` with the primary set to `NONE` and set the cloudautomation Data Guard Broker property to '1' for both databases so that the value will be automatically changed accordingly after a role transition swapping values of the primary and standby databases.

```
DGMGRL> edit database primary-unique-name set property cloudautomation=1;
Property "cloudautomation" updated
```

```
DGMGRL> edit database standby-unique-name set property cloudautomation=1;
Property "cloudautomation" updated
```

Note that the `cloudautomation` property must be set on both databases to work properly.

Configure Backups

Optionally, configure automatic backups for the Oracle cloud database, for primary or standby role, using Autonomous Recovery Service or object storage.

Autonomous Recovery Service allows you to offload backup processing and storage in addition to the following benefits:

- Significantly reduce dependencies on backup infrastructure
- Develop a centralized backup management strategy for all the supported OCI database services
- Retain backups with Recovery Service for a maximum period of 95 days
- Leverage real-time data protection capabilities to eliminate data loss
- Significantly reduce backup processing overhead for your production databases
- Implement a dedicated network for Recovery Service operations in each virtual cloud network (VCN)
- Automate backup validation to ensure recoverability
- Implement a policy-driven backup life-cycle management

See [Manage Database Backup and Recovery on Oracle Exadata Database Service on Dedicated Infrastructure](#) and [Database Autonomous Recovery Service](#) for more information.

Data Guard Life Cycle Operations

When the configuration is complete, Data Guard life-cycle operations, failover, switchover, and reinstate can be performed by `dbaascli` commands. See the [OCI Documentation](#) for details.

Health Check and Monitoring

After instantiating the standby database, a health check should be performed to ensure that the Oracle Data Guard databases (primary and standby) are compliant with Oracle MAA best practices.

It is also recommended that you perform the health check monthly, and before and after database maintenance. Oracle Autonomous Health Framework and automated tools including an Oracle MAA Scorecard using OraChk or ExaChk are recommended for checking the health of a Data Guard configuration. See [Oracle Autonomous Health Framework User's Guide](#) and [Oracle ORAchk and Oracle EXAchk documentation](#).

Regular monitoring of the Oracle Data Guard configuration is not provided in a hybrid Data Guard configuration and must be done manually. See [Monitor an Oracle Data Guard Configuration](#) for more information.

Migration to Oracle Cloud Best Practices

The following is a summary of cloud migration best practices, and provides links to resources where details can be found.

Use Zero Downtime Migration (ZDM)

Oracle MAA's simple, automated, one-button approach solution for moving your Oracle Databases into the Oracle Cloud

Use the latest ZDM version. See [Zero Downtime Migration](#) for detailed product information and software kit download.

For cross-platform migration cases, such as migrating from AIX, Sparc, or Windows-based platforms to Linux platforms, ZDM uses data pump, RMAN, Transportable Tablespace (XTTS), and Oracle GoldenGate instantiation and migration options. ZDM supports online migration with Data Pump and Oracle GoldenGate, or offline migration with RMAN XTTS for data and Data Pump for metadata migration.

For cross-platform migration cases using Transportable Tablespace and RMAN Incremental Backups, see [KB144840](#). ZDM does not support Full Transportable Data Pump with RMAN Incremental Backup migrations.

Evaluate network and use high-bandwidth network

- See [Network Evaluation](#)
- See [Assessing and Optimizing Network Performance](#)
- Network bandwidth impacts migration time. Migration downtime is impacted by switchover timings.

Plan and take the opportunity to refresh and optimize

Planning:

- Understand your HA and DR Service Level Agreements, and make sure you pick the correct cloud MAA reference architecture to meet those needs (see [Oracle Cloud Maximum Availability Architecture](#)). Also, see [Overview of Oracle AI Database Cloud Best Practices](#).
- Understand and document your application performance Key Performance Indicators (KPIs).
- Understand if the target cluster will have one or many databases, and size it accordingly as described below.
- Plan to take advantage of new benefits when using Exadata, Exascale, other Oracle Database Cloud services or Autonomous Database.
- Understand what the maximum tolerated downtime for data migration is for your application. Note that migration options are significantly simpler when you can tolerate some downtime. The options depend on if you can tolerate hours (for example, using Backup/Restore, RMAN XTTS or Data Pump), seconds to low minutes (using Data Guard, for example) or near zero (using Oracle GoldenGate). Also, if possible, physical migration is a simpler solution requiring less operational and application investment.

Refresh and optimize:

- With logical database migration, you have an opportunity to optimize by reducing the number of data files by using BigFiles, restructure tables and indexes, or even drop indexes and make any schema redesigns. Any logical migration or optimization will add to migration planning and testing investments.
- Use Oracle cloud-supplied system configuration, server parameter file (SPFILE), and automation. Only change database initialization parameters that are relevant for resource performance sizing. MAA works with various database cloud services to ensure MAA best practice compliance to achieve the highest stability and availability.
- Do NOT migrate your undocumented parameters.
- Use cloud automation such as Cloud Control Plan or Oracle-supplied cloud automation instead of your on-premises custom scripts.

Size for success

Assess current workload and resource utilization

- Collect usage metrics: Use tools like Oracle Automatic Workload Repository (AWR), PerfHub, or Enterprise Manager to collect at least 7-30 days of actual workload data (CPU, Memory, Storage, IOPS, Throughput).
- Peak workload analysis: Identify peak periods to size for your maximum required capacity—not just the average.

Inventory and profile applications

- Database characteristics: List database count, size, growth rate, redundancy, high availability, and disaster recovery requirements.
- Workload type: OLTP, OLAP, mixed workloads, and any special database features or options in use (such as partitioning or compression).

Calculate core requirements

- CPU: Size based on peak CPU usage, considering future growth (typically add 20-30% headroom).
- Memory: Assess SGA and PGA memory usage, plus OS requirements.
- Consolidation: Factor additional resources if you are consolidating multiple databases or instances.

Storage sizing

- Capacity: Calculate size needed for data, indexes, online redo logs, and possibly standby redo logs, temp tablespace, undo, system, and sysaux space for DATA disk group RECO disk group contains archive logs (typically 24 hours), flashback logs (typically 24 hours, or same as archive log space), and in some cases database backups. For simplicity use Cloud Defaults of DATA 80% and RECO 20% when database backups are offloaded to Oracle cloud managed services, or DATA 40% and RECO 60% when database backups are local to the VM cluster.
- Performance: Consider flash IOPS for low latency, disk IOPS, storage throughput, and latency needs. Exadata storage provides high performance, so try to model storage needs for both capacity and speed. Incorporate ASM redundancy level in your calculations. Oracle Cloud uses ASM High Redundancy as a default for all database services using Exadata because of its superior data protection.

High availability and disaster recovery

- Redundancy: Choose correct Exadata or database shapes to meet your high availability SLAs.
- Data Guard needs: If you are using Oracle Data Guard, consider sizing your standby environments symmetrically so that performance KPIs are preserved after Data Guard role transitions.

Choose the right Exadata shape

- Match your computed requirements to the available Exadata Database Service shapes (for example, number of database/compute servers, storage servers, vCPU allocations).

Helpful Oracle tools and documentation

- [Oracle Cloud Capacity Analytics \(OCCA\)](#): Internal and partner-accessible tools for detailed sizing (reach out to your Oracle account team)
- Oracle Exadata Cloud Service Documentation: [Exadata Database Service on Dedicated Infrastructure](#), [Exadata Database Service on Exascale Infrastructure](#), [Exadata Database Service on Cloud@Customer](#), or [Oracle Base Database Service](#)
- *Oracle Database Performance Tuning Guide*: Measuring Database Performance and Gathering Database Statistics

Prepare for high availability and disaster recovery

- Plan for application high availability (HA) and the best way to take most advantage of Oracle Real Application Clusters (RAC) benefits. See [Continuous Availability for Applications](#) and Design and Deployment Techniques to enable applications and database consolidation in an Oracle RAC environment.
- Plan for disaster recovery (DR).
 - If DR is important, then add a Cloud standby to the target before migration.
 - Use Cloud automation and MAA best practices to create the standby database.
 - Understand how best to failover or switchover the application. Evaluate [Oracle Cloud Infrastructure Full Stack Disaster Recovery](#) service.

Test and ensure you meet your performance and requirements

- Use ZDM -eval mode to create a test environment
- Evaluate application performance to ensure Key Performance Indicators (KPIs) and performance Service Level Agreements (SLAs) are met
- Evaluate simple outages to ensure application and HA SLAs are met
- Practice migration multiple times to ensure readiness

Cut over during low workload

- Pick a non-peak application window to do the migration.
- Establish a game plan to stay behind if issues arise.
- When possible, establish a fallback plan. For logical migration using Oracle GoldenGate, you will need to set up bi-directional replication. For physical migration using Data Guard, you need to use the ZDM parameter ZDM_SKIP_DG_CONFIG_CLEANUP so Data Guard transport can continue after migration.

Part VIII

Continuous Availability for Applications

- [About Application High Availability Levels](#)
- [Configuring Continuous Availability for Applications](#)
- Reference:
 - [Connection Time Estimates During Data Guard Switchover or Failover](#)
 - [Oracle Net TNS String Parameters](#)
 - [Connection Retry Logic Examples](#)
 - [Server-Side Planned Maintenance Command Examples](#)
 - [Configuring FAN for Specific Drivers and Pools](#)
 - [ONS FAN Application Requirements](#)
 - [In-Band Draining Notification Application Requirements](#)

About Application High Availability Levels

Ensure that your applications are configured to quickly and automatically shift workload to available Oracle RAC instances or standby databases during planned maintenance and unplanned outages.

Depending on your application's high availability requirements, you can implement the level of high availability (HA) protection that you need.

Application up time is maximized by following these recommendations when there are outages.

The primary audience for this document is application developers and application owners. Operational examples are included for database administrators and PDB administrators.

HA protection levels are defined in the table below, and each increase in level builds upon the previous level.

HA Level	Configuration	Experience	Benefits
Level 1: Basic Application High Availability See Configuring Level 1: Basic Application High Availability	Database or Security Administrator: - Configure role-based database services - Leverage recommended database connection string, and optionally configure LDAP and wallets - Enable Oracle Notification Service (ONS)/Fast Application Notification (FAN) Application Developer: - Use MAA recommended connect string - Use Basic exception handling Implementation effort: Minimal - estimated 1 hour for administrator, and less than 1 hour for developers (assuming the application has connection failure exception logic already coded)	Implementing Level 1 protection provides significant benefits compared to third party application failover solutions due to application + Oracle integration and intelligence to reduce application impact. - Reduced application downtime - Applications see errors during planned maintenance and unplanned outages; the application may trap the connection error and, if it is appropriate for the application, reconnect to another Oracle RAC instance or database with the target service - Applicable for unplanned outages and planned maintenance. In some cases, long running transactions should be deferred or suspended during planned maintenance.	High availability with application automatically failing over and reconnecting (if appropriate for the application) - Quick timeouts and automated connection retry with database connect string - Location transparency with services: role-based services for standby and read-only databases so that applications are automatically routed to the proper instance with the correct role - ONS/FAN auto-configured with database connect string - Immediate interrupt on outages when using FAN (no need to tune timeouts and wait for them) - Clusterware is aware of RAC and VIP health, so there is no waiting on downed end points thanks to FAN

HA Level	Configuration	Experience	Benefits
Level 2: Prepare Applications for Planned Maintenance See Configuring Level 2: Prepare Applications for Planned Maintenance	Level 1 configuration + Application Developer: Use Oracle connection pools or connection tests and return your connection to the pool between uses Additional implementation effort for developers: A few hours with minimal effort with Oracle Connection pools, including those in application servers - up to days when not using an application server, depending on application complexity. Developer may need to identify connection tests used in the application and possibly create new ones in the database	- Avoids errors during planned maintenance (errors still possible for unplanned outages) - Ability to drain and move workload gracefully without application interruption - Applicable for unplanned outages and planned maintenance events. For some cases, long running transactions should be staged with connection tests between batches. If this is not possible, the batch job should be deferred during planned maintenance.	Workload moves gracefully across instances with a slight delay and no errors during planned maintenance
Level 3: Mask Unplanned and Planned Failovers from Applications See Configuring Level 3: Mask Unplanned and Planned Failovers from Applications	Level 1 and 2 configuration + "Application Continuity" Solution Database or Security Administrator: - Additional security and privileges required for PL/SQL aspects of the application Application Developers: - External actions (for example, side effects) outside the database need to be considered Additional implementation effort: Days to weeks of collaboration between developers and database administrators to review protection coverage (depending on application complexity)	- In-flight transactions automatically acknowledge the commit or replay without application code changes - Database administrators and application developers coordinate to ensure readiness using AWR statistics to assess protection coverage, and use ACCHK (application continuity health check) to identify coverage or exceptions when transactions can or cannot be replayed	Masks unplanned and planned fail overs from applications - Applications avoid seeing errors during planned maintenance and outages - In-flight uncommitted transactions are replayed; committed transactions are acknowledged and not replayed

All of the HA Levels described in the table above are superior to connection management approaches using load balancers as single connection VIP endpoints for the following reasons:

- **Smart Service Health and Intelligent Reconnect:** Oracle Clusterware and Oracle Data Guard Broker closely monitor the health and state of the clusters and databases to ensure connections are routed to the database service that is opened on a primary.
- **Transparent and Automatic Failover:** There is no need to query the health of databases and decide which is the proper one to move a VIP; everything is transparent in the high availability approaches described in the table.

- **Fast Notification and Automatic Connection Retries:** The disconnection of already connected sessions is immediate, and happens intelligently when Oracle Clusterware and Data Guard Broker detect outages or role changes on the primary and standby databases.

Terms

The following terms are used throughout this document:

- **Application Continuity:** Application Continuity is an Oracle Database feature that masks interruptions at the database level for planned and unplanned events, improving the fault tolerance of applications that rely on Oracle Database.
- **Transparent Application Continuity (TAC):** Introduced with Oracle Database 18c, and enhanced in Oracle Database 19c and 23ai, extends support to application configurations that do not use Oracle connection pools (though they are still the preferred solution).
- **Draining:** Move a connection from one Oracle Real Application Clusters (RAC) instance to another available Oracle RAC instance.

Draining to move sessions gracefully from one instance to another is used during planned maintenance and load rebalancing. The connection is moved when the application returns the connection to a pool and then obtains a new connection or another rule is satisfied.

- **Fail over:** Reestablish an equivalent session at a new instance that offers the service.
Fail over occurs during unplanned outages and during planned maintenance when sessions do not drain within an allotted period of time. The application should not receive errors when Application Continuity is configured.
- **Fast Application Notification (FAN):** FAN is essential to break clients out of TCP/IP timeouts immediately following failures. FAN notifies clients immediately when resources become available, and initiates draining of database sessions so clients experience no outages during planned maintenance. FAN also includes notifying configuration-level and service-level information that includes changes in service status.
- **Oracle Notification Service (ONS):** A facility that creates a bridge with middle-tier servers or applications to transport cluster events to application logic for handling or reaction.

Software Recommendations

The following software is recommended for HA Level configurations:

- Oracle Real Application Clusters (Oracle RAC) and Oracle Clusterware (which provides services and infrastructure to efficiently manage outages), preferably with Oracle Grid Infrastructure (GI) release 19c or later
- Oracle Active Data Guard is recommended for protection from database, cluster, storage or site failures
- Oracle Database 19c client and database or a later long-term support version, with the most recent patch level

Configuring Continuous Availability for Applications

Depending on your application's high availability requirements, you can implement the level of high availability (HA) protection that you need.

- [Configuring Level 1: Basic Application High Availability](#)
- [Configuring Level 2: Prepare Applications for Planned Maintenance](#)
- [Configuring Level 3: Mask Unplanned and Planned Failovers from Applications](#)

The primary audience for this document is application developers and application owners. Operational examples are included for database administrators and PDB administrators.

In [About Application High Availability Levels](#), the three HA protection levels and terms are defined, and software recommendations are listed.

Configuring Level 1: Basic Application High Availability

Implement a level of high availability that allows applications to immediately react to instance, node, or database failures, and quickly establish new connections to surviving database instances.

With application HA Level 1, downtime is minimized for unplanned and planned outages. You get these benefits by ensuring that the application configuration implements these recommendations. No code changes are required.

At a high level, the steps to implement Level 1 are:

- [Step 1: Configure High Availability Database Services](#)
- [Step 2: Configure the Connection String for High Availability](#)
- [Step 3: Use Fast Application Notification](#)
- [Step 4: Developer Determines if the Application Should Implement Reconnection Logic](#)

Step 1: Configure High Availability Database Services

Create a non-default, role-based database service to use high-availability features.

A database service is a logical abstraction for managing workloads or a group of applications sharing similar SLAs or types of workloads (for example, OLTP vs. batch). Database services provide location transparency and hide complex aspects of the underlying system from the client.

Your application must connect to a non-default database service to use high-availability features. You must explicitly create a service (or several services as needed for different application workloads) instead of using the default database service or the default PDB service (that is, the service with the same name as the database or PDB).

On Oracle Autonomous Database, services are created for you using recommended attributes.

About Server-Side Configuration for Services

These services are configured by a database administrator to set up services through Oracle Clusterware.

When using Oracle Data Guard and standby databases, create services using the primary role to ensure that applications connect to the primary database for read/write operations, and standby role for services to optionally offload read-only and small infrequent writes to the standby database.

Services start and stop automatically after a Data Guard role transition (for example, switchover or failover) based on their roles.

Configure your services according to your architecture in one of the following sections:

- [Configure High Availability Services](#)
- [Configure High Availability Services for Oracle Active Data Guard or Standby Roles](#)

Note

Services must be started so that they can be used after creating them. Use a command like this:

```
$ srvctl start service -db mydb -service my_service
```

See also:

Using Oracle Services in *Oracle Real Application Clusters Administration and Deployment Guide*

Configure High Availability Services

Create a non-default, role-based database service to use high-availability features.

A service may be configured to direct connections to a single preferred instance, or alternatively, if the preferred instance is down, to an available instance. When a service is available only on one instance, it is called a *singleton service*. This allows you to isolate workloads among instances in a cluster.

You could also configure a service to put connections on multiple instances of a cluster, to spread work across all instances. Also, if one instance is down, connections can be made on the surviving instances.

There are other combinations where you can configure a subset of instances as "preferred" and another subset of instances as "available". These subsets provide for spreading load across some instances while isolating work from others (and still have instances available in case of a failure).

See [Considerations for Oracle Cloud Database Services](#) if your database is running on the Oracle Cloud.

Example 1: Singleton Service

This example creates a singleton service called `my_service` for the primary role, where the connections are made on instance `inst1`, unless that instance is not available. If the instance is not available, connections are made on `inst2`.

The example also configures a default drain timeout of 300 seconds to wait for sessions to drain; at the end of that time any remaining sessions are terminated because of the IMMEDIATE option.

```
$ srvctl add service -db mydb -service my_service -pdb mypdb -preferred inst1  
-available inst2 -notification TRUE -drain_timeout 300 -stopoption IMMEDIATE  
-role PRIMARY
```

If you want your application to gracefully switch to another Oracle RAC instance with no application blackout, set the drain_timeout interval to a sufficient timeout that allows your applications to close their connections between transactions and gracefully stop or move to another instance. The drain_timeout interval is best leveraged for short OLTP applications. For large batch operations, it's best defer or suspend these operations before a planned maintenance window.

Example 2: Service with Multiple Instances

This example creates a service that is similar to the singleton above but spreads connections across multiple instances in this cluster:

```
$ srvctl add service -db mydb -service my_service -pdb mypdb  
-preferred inst1,inst2 -notification TRUE -drain_timeout 300 -stopoption  
IMMEDIATE  
-clbgoal LONG -rlbgoal SERVICE_TIME -role PRIMARY
```

Configure High Availability Services for Oracle Active Data Guard or Standby Roles

A service used to connect to a standby database (read-only physical standby) is created as follows:

```
$ srvctl add service -db mydb -service my_standby_service -pdb mypdb  
-preferred inst1 -available inst2 -notification TRUE -drain_timeout 300  
-stopoption IMMEDIATE -clbgoal LONG -rlbgoal SERVICE_TIME -role  
PHYSICAL_STANDBY
```

Considerations for Oracle Cloud Database Services

A default service is created along with every PDB that is provisioned in the Oracle cloud.

Note the following considerations depending on the type of cloud service you have.

Autonomous Database Serverless

For Autonomous Database Serverless, database services are pre-configured to support different performance and concurrency characteristics. You can modify the services to enable certain availability features using the DBMS_APP_CONT_ADMIN package.

Specifically:

- To set the drain timeout attributes, use DBMS_APP_CONT_ADMIN.SET_DRAINING. See DBMS_APP_CONT_ADMIN for details.
- To set TAC or AC attributes, use DBMS_APP_CONT_ADMIN.ENABLE_TAC or DBMS_APP_CONT_ADMIN.ENABLE_AC

This sets the associated service attributes to their recommended values to support TAC or AC

- To set specific attributes, use `DBMS_APP_CONT_ADMIN.MODIFY_SERVICE`

The extent to which the `DBMS_APP_CONT_ADMIN` package supports changes to the service depends on the database version.

Availability aspects of services in Autonomous Database Serverless, such as preferred and available instances are configured automatically. See [Database Service Names for Autonomous Database](#) for more details and other options.

Autonomous Database on Dedicated Exadata Infrastructure

For Autonomous Database on Dedicated Exadata Infrastructure, database services are pre-configured with TAC, AC, and support for various degrees of parallelism. Use the service name that meets the needs of your application.

Availability aspects of services in Autonomous Database, such as preferred and available instances, are configured automatically. See [Predefined Database Service Names for Autonomous Databases](#) for more details and other options.

Oracle Exadata Database Service on Dedicated Infrastructure and Oracle Base Database

Do not use the default PDB or CDB service.

The service in the connect string provided by the UI is for administrative purposes, not for client application connections. You must create a database service for your applications by following the recommendations in [Configure High Availability Services](#).

" You must create a database service for your applications by following the recommendations in the above section, "Configure High Availability Services".

Step 2: Configure the Connection String for High Availability

Oracle recommends that your application use the connection string configuration shown here to connect successfully during various scenarios including database switchover and failover to other sites.

Example 1: Connect string with Oracle RAC primary database and no standby

```
Alias = (DESCRIPTION =
(CONNECT_TIMEOUT= 90)(RETRY_COUNT=20)(RETRY_DELAY=3)
(TRANSPORT_CONNECT_TIMEOUT=1000ms)
  (ADDRESS_LIST =
    (LOAD_BALANCE=on)
    (ADDRESS = (PROTOCOL = TCP)(HOST=clu_site1-scan)(PORT=1521)))
  (CONNECT_DATA=(SERVICE_NAME = my_service)))
```

Example 2: Connect string with Oracle RAC primary and standby databases

This example makes connections to an Oracle RAC primary database or a standby database, depending on which one is available.

```
Alias = (DESCRIPTION =
(CONNECT_TIMEOUT= 90)(RETRY_COUNT=100)(RETRY_DELAY=3)
(TRANSPORT_CONNECT_TIMEOUT=1000ms)
  (ADDRESS_LIST =
    (LOAD_BALANCE=on)
```



```
(ADDRESS = (PROTOCOL = TCP)(HOST=clu_site1-scan)(PORT=1521)))
(ADDRESS_LIST =
  (LOAD_BALANCE=on)
  (ADDRESS = (PROTOCOL = TCP)(HOST=clu_site2-scan)(PORT=1521)))
(CONNECT_DATA=(SERVICE_NAME = my_service)))
```

① Note

clu_site1-scan and clu_site2-scan refer to SCAN listeners in a cluster on site1 and site2, respectively.

It's recommended that you use the most recent drivers, but all Oracle drivers from release 12.2 and later should use the example connection strings above. Specific values can be tuned, but the values shown in this example are reasonable starting points, and so usable for almost all cases.

It is highly recommended that you maintain your connect string or URL in a central location, such as LDAP or `tnsnames.ora`. Do not scatter the connect string or URL in property files or private locations, as doing so makes it extremely difficult to maintain. Using a centralized location helps you preserve standard format, tuning, and service settings. Oracle's solution for this is to use LDAP with the Oracle Unified Directory product.

For JDBC, the connection strings listed above would be implemented as shown in these examples.

Example 1. Oracle RAC with no standby

```
jdbc:oracle:thin:@(DESCRIPTION =(CONNECT_TIMEOUT= 90)(RETRY_COUNT=20)
(RETRY_DELAY=3)
(TRANSPORT_CONNECT_TIMEOUT=1000ms)(ADDRESS_LIST=(LOAD_BALANCE=on)(ADDRESS =
(PROTOCOL = TCP)
(HOST=clu_site1-scan)(PORT=1521)))(CONNECT_DATA=(SERVICE_NAME = my_service)))
```

Example 2. Oracle RAC with standby

```
jdbc:oracle:thin:@(DESCRIPTION =(CONNECT_TIMEOUT= 90)(RETRY_COUNT=100)
(RETRY_DELAY=3)
(TRANSPORT_CONNECT_TIMEOUT=1000ms)(ADDRESS_LIST=(LOAD_BALANCE=on)(ADDRESS =
(PROTOCOL = TCP)
(HOST=clu_site1-scan)(PORT=1521)))(ADDRESS_LIST=(LOAD_BALANCE=on)(ADDRESS =
(PROTOCOL = TCP)
(HOST=clu_site2-scan)(PORT=1521)))(CONNECT_DATA=(SERVICE_NAME = my_service)))
```

For other clients, refer to your client's documentation.

See also:

- [Connection Time Estimates During Data Guard Switchover or Failover](#)
- [Oracle Net TNS String Parameters](#)
- [Oracle Unified Directory](#) in *Administering Oracle Unified Directory*
- Overview of Local Naming Parameters in *Oracle Database Net Services Reference*

Oracle Cloud Considerations for Connection Strings

Oracle Cloud provides sample connection strings for CDBs and PDBs that are provisioned.

For Cloud services such as Exadata Database Services on Dedicated Infrastructure or Oracle Base Database Service, you must change the connection string that is provided to include the new service you added in step 1, instead of the default service in the provided connect string.

If your database has a Data Guard association, then you may want to use the connection string format in step 2 that includes the primary and standby entries, if you require your existing application to fail over to the standby automatically after a primary database failure.

For a standby in the same region, you should typically add the standby in the connect string, but for a cross-region standby, evaluate the OCI Full Stack Disaster Recovery service to orchestrate application, database, and client failover. See [Full Stack Disaster Recovery on Oracle.com](#).

Step 3: Use Fast Application Notification

When a service needs to drain for routine maintenance, or experiences unplanned failures (such as node or network outages), the application needs to be informed in real time so that it can quickly move connections to another instance or site. This is accomplished using Oracle's Fast Application Notification (FAN) feature, which enables applications and connection pools to receive event notifications from one or more clusters.

The ability to receive FAN events is enabled automatically when using the recommended service and connect string in steps 1 and 2 above, along with Oracle JDBC drivers (the latest version is recommended, but not earlier than Oracle release 12.2).

The ONS port (by default, 6200) needs to be opened on all of your database servers, the firewall, and Oracle Active Data Guard nodes.

The use of FAN is not mandatory, but is highly desirable because it can detect many types of unplanned outage scenarios and enable applications to handle these scenarios gracefully to maintain high availability.

FAN uses Oracle Clusterware's Oracle Notification Service (ONS) to receive events from the cluster. ONS requires ports to be available between the client and the servers, and in some cases this requires a firewall port to be opened (6200 by default) on all of your database servers, the firewall, and Oracle Active Data Guard nodes.

Alternative if you can't use ONS FAN: In-Band Notification

When port 6200 cannot be opened or is not available, Oracle's connection drivers will enable "in-band" notifications automatically using the database connections themselves. In-band notifications are received on the next round-trip to the database.

This notification simply tells the driver that the service is draining and the client should close the connection. Clients will not receive events for an instance or node failure to advise a client to disconnect immediately, because those kinds of failures will terminate connections ungracefully, removing the ability to see any notification because the connection would be gone.

In-band notification is for planned maintenance and does not apply to unplanned outages.

Oracle cloud normally has port 6200 enabled, but if you have blocked port 6200, then in-band notification is automatically used to initiate draining of connections to other nodes during planned maintenance.

See [In-Band Draining Notification Application Requirements](#) to ensure that your client is properly configured to use in-band FAN.

Oracle Autonomous Database Serverless

In-band notifications are used with Autonomous Database Serverless so clients can drain their connections during planned maintenance operations.

To ensure your clients are ready to use In-band notifications, see specific recommendations for Autonomous Database Serverless in [Best Practices to Maintain Application Availability During Maintenance Windows](#).

Enabling ONS/FAN for Clients

There are no application code changes to use FAN. FAN only requires an Oracle driver and the recommended database connect strings in step 2 above.

By default, starting with 19c, ONS is auto-configured by leveraging the recommended database connect strings (in step 2 above), when those strings are used to connect to a cluster using the cluster's SCAN listeners. ONS will automatically determine which nodes it should establish connections to, including nodes in standby clusters (as long as the standby clusters are in the connect string).

It is important to use the TNS formats shown in Step 2 for auto-configuration of FAN; using a different format syntax can prevent FAN from being auto-configured.

If you are not able to use the recommended connect URL/string (in step 2), configure your clients to subscribe to ONS manually by setting the list of ONS nodes and ports.

For example, in UCP, ONS endpoints could be configured like the following example (other pools would use something similar; check your pool's documentation):

```
pds.setONSConfiguration("nodes=racnode1:4200,racnode2:4200\nwalletfile=/  
oracle11/onswalletfile");
```

This example shows an ONS configuration using a wallet file, which is typically required with Oracle Cloud but should not be used in other environments.

If you need to configure ONS explicitly as shown above, it is recommended that you create a property file and reference that file to obtain ONS configuration values instead of hard-coding values. See Remote Configuration of ONS for details.

Ensure that your application server and drivers are compatible with ONS/FAN by reviewing [ONS FAN Application Requirements](#) and [Configuring FAN for Specific Drivers and Pools](#).

Developer Actions to Enable High Availability

There are simple configuration changes for your application developers to make to enable application failover HA and draining. See [Configuring FAN for Specific Drivers and Pools](#) for details.

See also:

Overview of Oracle Integrated Clients and FAN in *Oracle Real Application Clusters Administration and Deployment Guide*

Step 4: Developer Determines if the Application Should Implement Reconnection Logic

Applications can be written to catch connection failure exceptions and errors during database calls so that they can obtain new connections and continue with new work, if it is reasonable to continue.

There are many factors to take into account to determine if it is reasonable for an application to continue and how it should proceed after losing connections. In [Configuring Level 3: Mask Unplanned and Planned Failovers from Applications](#), a robust solution is presented for masking failures from your application transparently using AC and TAC.

For JDBC-based applications, the `SQLRecoverableException` can be caught to distinguish connection errors from typical application or SQL errors. If a connection error is caught, then a new connection should be obtained. This is simpler and more robust than checking for individual Oracle errors (which can adjust by Oracle Database release) in the `SQLException` class.

See also:

[Connection Retry Logic Examples](#)

Configuring Level 2: Prepare Applications for Planned Maintenance

Building on Level 1, Basic Application High Availability, Level 2 adds the ability to drain sessions for minimal application impact during planned maintenance.

After implementing Level 1, you are ready to implement a planned maintenance solution appropriate to your application from one of the choices below. You can use planned operations to relocate or stop services, or to switch over, allowing for graceful completion of the users' work.

The recommended approach to avoid impacting applications is drain work in an Oracle RAC rolling fashion. Typically a period of time is allocated to perform the draining. Our recommended choice is to use Oracle connection pools that are integrated with FAN to initiate draining.

If you are unable to drain automatically using the options listed below, an alternative approach is to ensure the application manages to stop and defer work before maintenance starts.

Employ the following practices to increase your application high availability to level 2:

- Use an Oracle Connection Pool and return your connection to the pool between requests. See [Recommended Option: Use an Oracle Connection Pool](#).
Alternatively, use a third-party connection pool that uses request boundaries.
- If you cannot use an Oracle connection pool or third-party pool with request boundaries, you can use the following alternatives with existing applications:
 - [Alternate Option 1: Use Request Boundaries](#)
 - [Alternate Option 2: Use Connection Validation or Tests](#)
- Leverage server-side operations for planned maintenance. See [Server-Side Operations for Planned Maintenance](#)

- Ensure that sufficient node capacity is available so that the load from one instance can be spread to other available instances without impacting the workload during a maintenance period.

Recommended Option: Use an Oracle Connection Pool

Using a FAN-aware Oracle connection pool is the recommended solution for managing planned maintenance.

Oracle pools provide full lifecycle management: draining, reconnecting, and rebalancing across nodes and sites. When performing rolling maintenance across a cluster, as the maintenance progresses and completes (for each instance or node), sessions are moved and rebalanced across instances. There is no impact to users when your application uses an Oracle Pool with FAN and returns connections to the pool between requests since these connections will drain and move to other instances.

Supported Oracle Pools include:

- Java; Universal Connection Pool (UCP)
 - UCP documentation: [Introduction to UCP](#)
 - Springboot with UCP: [UCP Best Practices for Oracle Database 19c and Spring Boot](#)
- Java; WebLogic Active GridLink
- Tuxedo
- OCI Session Pool
- ODP.NET core, managed, and unmanaged providers
- Oracle Python driver connection pool
- Node.js Oracle driver connection pool
- Hikari Connection Pools: [HikariCP Best Practices for Oracle Database and Spring Boot](#)

When using these pools, no application changes are needed other than ensuring that your connections are returned to the pool between requests.

It is a best practice that an application obtains a connection only for the time that it needs it, and then returns the connection to the pool as soon as it is finished making its database calls. Holding a connection instead of returning it to the pool prevents the pool from gracefully moving sessions to available instances, and it uses resources inefficiently, requiring many more connections than would otherwise be used. An application should, therefore, obtain a connection and then return that connection immediately after the work is complete. The connections are then available for later use by other threads, or your thread when needed again. Returning connections to a connection pool is a general recommendation regardless of how draining is implemented.

Note

The syntax for obtaining and returning a connection varies by pool implementation. For example, in UCP you use the `getConnection()` method of the `PoolDataSource` object to obtain a connection and mark the beginning of the database request, and the `close()` method to return it, marking the end of the database request after you've done some work in the database.

Oracle Connection Pools validate a connection whenever a connection is borrowed to ensure that the connection can be used without any errors.

Alternate Options

Only look at these alternate options if you cannot use the recommended option. The alternate options are listed in priority order.

Alternate Option 1: Use Request Boundaries

For existing applications, retrofit them by using session/connection pools or by wrapping operations within a `requestBegin` and `requestEnd` boundary. For example, for the Oracle Call Interface: `OCIRequestBegin` and `OCIRequestEnd`.

Request boundaries are visible to the database, and they enable functionality such as draining for planned maintenance, load balancing, and multiplexing to be isolated at the database layer. Sessions can be re-established with no visible disruption to the application layers above.

Alternate Option 2: Use Connection Validation or Tests

If you cannot use an Oracle Pool, or do not wish to implement request boundaries, then the Oracle client drivers (19c or later) or Oracle Database (19c or later) can drain the sessions for you.

When services are relocated or stopped, or there is a switchover to a standby site via Oracle Data Guard, the Oracle Database and Oracle client drivers are notified to look for safe places to release connections according to the following:

- Standard connection tests for connection validity (for example `isValid()` in JDBC)
- Custom SQL tests for connection validity

For custom batch applications, test the connection between batches. When the connection test fails, create or borrow another connection.

For third-party connection pools, enable connection tests offered by the vendor. When the connection test fails, the third-party pool will close the connection and allow you to borrow another one.

See [In-Band Draining Notification Application Requirements](#) for implementing connection validation and tests for your specific application or driver.

Note

- When you use a connection test, the outcome of the connection test applies to that session only. Do not use connection tests to make general decisions about the instance and to make a determination to stop more than the session to which the test applies.
- Disable connection pool properties for flushing and destroying the pool on connection test failure when using Oracle WebLogic Server data sources.
- A monitor is functionality that makes a decision about the health of an instance. With FAN and Runtime Load Balancing such monitors are no longer needed and not susceptible to incorrect decisions. If you do want a monitor, SQL in that monitor must NOT be misinterpreted as a connection test for draining the application. There are a few ways to avoid this misinterpretation:

- Disable a monitor's specific health query using the `dbms_app_cont_admin` package:

```
dbms_app_cont_admin.disable_connection_test(dbms_app_cont_admin.s  
ql_test, 'SELECT COUNT(1) FROM DUAL');
```

Here, the query used by the monitor, 'SELECT COUNT(1) FROM DUAL', is not considered a connection test. If there are any connection tests that also use this query, then they would be disabled and a different query would be needed.

- Embed a comment into the monitor query to distinguish it from any of the registered connection tests:

```
SELECT /* My Health monitor query */ COUNT(1) monitor FROM DUAL
```

- You may disable connection tests by using the `DBMS_APP_CONT_ADMIN.DISABLE_CONNECTION_TEST` procedure. You can also add, modify, or delete connection tests using the `DBMS_APP_CONT_ADMIN` package.

Server-Side Operations for Planned Maintenance

Server-side operations are required to manage connections for planned maintenance.

The server side operations for planned maintenance are implicitly executed if you are using Oracle cloud software automation, or are in a cloud maintenance window that requires restarting the database instance.

Note that services connected to the Oracle Database are configured with connection tests and a drain timeout specifying how long to allow for draining, and the stopoption (typically IMMEDIATE), that applies after the drain timeout expires. The stop, relocate, and switchover commands managed by SRVCTL include a `drain_timeout` and `stopoption` switch to override values set on the service if needed.

Oracle recommends configuring services with the required drain timeout applicable to that service, so they are used automatically during maintenance operations.

Maintenance commands are similar to the commands described in the examples in [Server-Side Planned Maintenance Command Examples](#). You can use these commands to start draining. Include additional options, if needed, as described in My Oracle Support (MOS) Note:

Doc ID 1593712.1. Oracle tools, such as Fleet Patching and Provisioning (FPP) use these commands as well.

Oracle Clusterware can start instances that are not currently running, but can run a service that requires that instance. Services that cannot be relocated or do not need relocation, are stopped. If a singleton service is defined with no other available instances, then it may incur complete downtime, which is expected behavior. It is better to have preferred instances and at least one available instance always defined.

After the maintenance is complete and the instance is restarted, no additional SRVCTL action is required because the Oracle Clusterware service attribute automatically determines where services will end up.

See also:

Server Draining Ahead of Planned Maintenance in *Oracle Real Application Clusters Administration and Deployment Guide*

Configuring Level 3: Mask Unplanned and Planned Failovers from Applications

Building on Level 1 and Level 2, the features presented in Level 3 are recommended to achieve continuous availability for applications, regardless of database interruptions, outages, timeouts, or when application workload won't drain.

Application Continuity

Application Continuity (AC) hides unplanned outages, starting with Oracle Database 12.1 for thin Java-based applications, and Oracle Database 12.2.0.1 for OCI and ODP.NET based applications with support for open-source drivers, such as Node.js, and Python, beginning with Oracle Database 19c.

Application Continuity rebuilds the session by recovering the session from a known point which includes session states and transactional states. Application Continuity rebuilds all in-flight work. The application continues as it was, seeing a slightly delayed execution time when a failover occurs.

The standard mode for Application Continuity is for OLTP applications using an Oracle connection pool or a third party connection pool with request boundaries.

Transparent Application Continuity

Starting with Oracle Database 19c, Transparent Application Continuity (TAC) transparently tracks and records session and transactional state so the database session can be recovered following recoverable outages. This is done with no reliance on application knowledge or application code changes, allowing Transparent Application Continuity to be enabled for applications by default.

Application transparency and failover are achieved by consuming the state-tracking information that captures and categorizes the session state usage as the application issues user calls.

Choose AC or TAC

If you have an OLTP application that uses an Oracle connection pool (or RedHat JBOSS EAP), you have a choice between Application Continuity and Transparent Application Continuity.

To decide which feature to use, you can run the application with each and choose the one with higher value for cumulative user calls protected by Application Continuity.

If you are not using an Oracle connection pool (as with SQL*Plus 19c or SQLcl 23), or you do not have knowledge about the application, then use TAC.

Planned Failover with AC and TAC

Planned failover is failover that is invoked by the Oracle Database at points where the database decides that a session is replayable and is expected not to drain.

Planned failover is enabled by default when using AC or TAC. It improves situations where other draining methods are not active, for example, because FAN or connection tests are not configured.

Planned failover expedites maintenance by failing over early when replay is enabled.

For example, planned failover with TAC is the maintenance solution used with SQL*Plus.

See also:

- Ensuring Application Continuity in *Oracle Real Application Clusters Administration and Deployment Guide*
- [Blog: database-heartbeat Application Continuity](#)
- Restrictions and Other Considerations for Application Continuity

Configure Services for AC and TAC

Set COMMIT_OUTCOME = TRUE

Determines whether transaction COMMIT outcome is accessible after the COMMIT has executed. While the database guarantees that COMMIT is durable, this setting ensures that the outcome of the COMMIT is durable. Applications use this feature to probe the status of the commit last executed after an outage, and is available to applications to determine an outcome

Set FAILOVER_TYPE

Set FAILOVER_TYPE to AUTO when using TAC.

Alternatively, set the database service attribute FAILOVER_TYPE to TRANSACTION to use Application Continuity.

Set FAILOVER_RESTORE

An application can be written to change the database session state (using ALTER SESSION commands typically), and these states need to be in place if you want the work to be replayed after failover.

To restore your session state at failover, set the attribute FAILOVER_RESTORE on your database service. Use LEVEL1 for AC or use AUTO for TAC.

The use of wallets is highly recommended. AC and TAC leverage wallets to ensure all modifiable database parameters are restored automatically with FAILOVER_RESTORE. Wallets are enabled for Autonomous Database and are the same as those used for database links.

See also:

Configuring a Keystore for FAILOVER_RESTORE in *Oracle Real Application Clusters Administration and Deployment Guide* to learn how to set up wallets for databases.

Examples

Example service configuration for TAC:

```
$ srvctl add service -db mydb -service my_service -pdb mypdb
  -preferred inst1 -available inst2 -commit_outcome TRUE -failover_type AUTO
  -failover_restore AUTO -notification TRUE -drain_timeout 300 -stopoption
IMMEDIATE
  -role PRIMARY
```

Example service configuration for AC:

```
$ srvctl add service -db mydb -service my_service -pdb mypdb -preferred inst1
  -available inst2 -commit_outcome TRUE -failover_type TRANSACTION
  -failover_restore LEVEL1 -notification TRUE -drain_timeout 300 -stopoption
IMMEDIATE
  -role PRIMARY
```

Return Connections to the Connection Pool

Request boundaries are required for Application Continuity (AC) and are recommended for Transparent Application Continuity (TAC).

Request boundaries are automatically embedded in the session for you when you use an Oracle connection pool, such as Universal Connection Pool (UCP) or OCI Session Pool. The application should return the connection to the Oracle connection pool when the unit of work, the database request, is completed, to insert the end of request boundary. This also applies to using ODP.Net Unmanaged Provider, WebLogic Active GridLink, and RedHat.

Side Effect

When a database request includes an external call from the database, such as sending mail or transferring a file, this is called a **side effect**.

When replay occurs, there is a choice as to whether side effects should be replayed. Many applications want to repeat side effects such as creating journal entries, sending mail, and performing file writes. For Application Continuity, side effects are replayed, but can be programmatically avoided. Conversely, Transparent Application Continuity does not replay side effects.

Starting with Oracle 23ai, there are PL/SQL procedures to set rules for how replay handles side effects. See the `REPLAY` related procedures in the `DBMS_APP_CONT` in Summary of `DBMS_APP_CONT` Subprograms.

Restore Original Function Values During Replay

Oracle Database 19c keeps the values of `SYSDATE`, `SYSTIMESTAMP`, `SYS_GUID`, and `sequence.NEXTVAL`, `CURRENT_TIMESTAMP`, and `LOCALTIMESTAMP` for SQL during replay.

If you are using PL/SQL, then GRANT KEEP for application users, and use the KEEP clause for a sequence owner. When the KEEP privilege is granted, replay applies the original function result at replay.

```
SQL> GRANT KEEP DATE TIME to scott;  
SQL> GRANT KEEP SYSGUID to scott;  
SQL> GRANT KEEP SEQUENCE mySequence on mysequence.myobject to scott;
```

Client Configuration

JDBC Configuration

Ensure your JDBC configuration uses:

- The recommended JDBC data source for standalone JDBC, or configure it as connection factory class for a Java connection pool (such as UCP) or a WebLogic AGL Server connection pool or RedHat JBOSS EAP.

See *Configuring the Data Source for Application Continuity in the Oracle Universal Connection Pool Developer's Guide* for information about enabling AC and TAC on UCP. You configure the JDBC driver data source class `oracle.jdbc.replay.OracleDataSourceImpl` as the connection factory class on the UCP data source `PoolDataSourceImpl`.

For 19c.x.x.x and older drivers, use `oracle.jdbc.replay.OracleDataSourceImpl`

For 21ai.x.x.x and newer drivers, use `oracle.jdbc.datasource.impl.OracleDataSource`

Note that the exact data source and connection pool configuration are always specific to a particular vendor's product, such as a 3rd-party connection pool, framework, application server, or container.

- The JDBC driver statement cache in place of an application server statement cache.

This allows the driver to know that statements are closed and memory is to be freed at the end of requests. To use the JDBC statement cache, use the connection property `oracle.jdbc.implicitStatementCacheSize` (`OracleConnection.CONNECTION_PROPERTY_IMPLICIT_STATEMENT_CACHE_SIZE`). The value for the cache size matches your number of open_cursors. For example:

`oracle.jdbc.implicitStatementCacheSize=nnn` where nnn is typically between 10 and 100, and is equal to the number of open cursors your application maintains.

ODP.NET Configuration

ODP.NET AC and TAC are enabled by default on the client side.

To use AC or TAC with managed ODP.NET or ODP.NET Core, install the Oracle 23ai version of the driver. The Oracle Database version must be Oracle 19c or higher.

Unmanaged ODP.NET supports AC starting with Oracle 12.2, and TAC starting with Oracle 19c.

See [ODP.NET Application Continuity](#).

Review Monitoring Statistics

Application Continuity collects statistics to monitor your protection levels. These statistics are saved in the Automatic Workload Repository (AWR) and are available in Automatic Workload Repository reports.

Review the statistics to determine the extent of protected calls or if the protected call count or protected time decreases. See Protection Level Statistics for details about using database statistics to analyze AC protection.

Use the ACCHK utility to discover details about the cause.

See also:

Application Continuity Protection Check in *Oracle Real Application Clusters Administration and Deployment Guide*.

Continuous Availability for Applications Reference

Reference topics for Configuring Continuous Availability for Applications.

Topics:

- [Connection Time Estimates During Data Guard Switchover or Failover](#)
- [Oracle Net TNS String Parameters](#)
- [Connection Retry Logic Examples](#)
- [Server-Side Planned Maintenance Command Examples](#)
- [Configuring FAN for Specific Drivers and Pools](#)
- [ONS FAN Application Requirements](#)
- [In-Band Draining Notification Application Requirements](#)

Connection Time Estimates During Data Guard Switchover or Failover

The settings in the connect string allow for the following maximum times to connect during switchover or failover.

- Data Guard Switchover:

$$\text{RETRY_COUNT} \times \text{RETRY_DELAY} = 100 \times 3 \text{ sec} = 300 \text{ sec.}$$
- Data Guard Failover:

$$(3 \text{ SCANS} \times \text{TRANSPORT_CONNECT_TIMEOUT}) + (\text{RETRY_COUNT} \times (\text{RETRY_DELAY} + (3 \text{ SCANS} \times \text{TRANSPORT_CONNECT_TIMEOUT}))) = (3 \times 1) + (100 \times (3 + (3 \times 1))) = 3 + 600 = 603 \text{ sec}$$

After Data Guard switchover / Data Guard failover to clu-site2, initial connections to clu-site2 take 3 seconds when clu-site1 is down (use of a connection pool helps mitigate this delay). When clu-site1 is reachable again (when it becomes a standby) connections are nearly instantaneous because the listener on the standby will answer immediately that the service is not there, prompting the client to connect to the other ADDRESS_LIST.

- If the switchover or failover completes much earlier than the maximum time, the application will experience less impact.
- Increase RETRY_COUNT if your system might take longer than 300 sec to complete a switchover or failover. If you need more time to complete a Data Guard switchover, then change RETRY_COUNT greater than 100.
- If you aren't using Oracle Clusterware, then your HOST address will not reference a SCAN VIP but a single VIP. This means that TRANSPORT_CONNECT_TIMEOUT must be set to higher or lower values to account for network latency.

Oracle Net TNS String Parameters

The parameters used in the connect string are explained here.

CONNECT_TIMEOUT

Applies when a connection to a listener address is attempted.

This setting represents the maximum time in which a connection using a specific ADDRESS endpoint has to complete. It includes the transport connection time and any other actions that must happen (redirection from SCAN VIP to listener VIP and finally to the foreground spawned process).

CONNECT_TIMEOUT should be larger than TRANSPORT_CONNECT_TIMEOUT, otherwise TRANSPORT_CONNECT_TIMEOUT is effectively capped by CONNECT_TIMEOUT. When TRANSPORT_CONNECT_TIMEOUT is not specified, then CONNECT_TIMEOUT acts as the timeout for the entire connection attempt to an ADDRESS endpoint, both transport and final connection to the database foreground.

Oracle recommends the value for CONNECT_TIMEOUT be large enough to account for the value of TRANSPORT_CONNECT_TIMEOUT, in addition to potential delays that may occur when connecting to busy listeners and hosts. The value of 90 seconds in the example connect string is very generous and might need to be shortened in some cases. But, if it is too short, then the setting could be counter-productive because it causes additional attempts that can also fail, and can introduce more unproductive workload on the servers to handle connection requests that might be prematurely abandoned.

RETRY_COUNT

If a connection attempt fails across all ADDRESS_LISTS, then additional attempts to connect beginning with the first ADDRESS_LIST will be made RETRY_COUNT times.

This is useful when a switchover or failover to a standby is in progress and the connection needs to keep trying until the operation is complete.

RETRY_DELAY

Seconds in between retry attempts.

A short amount of time is given to allow the new primary database time to open. This parameter is used with RETRY_COUNT to wait a reasonable amount of time to connect to a newly opened database.

It is better to have short retry delays with many retry counts so the connection can complete close to the time the primary database opened.

TRANSPORT_CONNECT_TIMEOUT=1000ms

Allow up to 1000 milliseconds to connect to the listener using TCP hosts in the ADDRESS. If no connection is made, then try the next ADDRESS.

When an Oracle RAC SCAN host name is used, each IP in the SCAN address is expanded internally into a separate ADDRESS string. Each ADDRESS is then attempted if a connection attempt fails.

Adjust this parameter for your environment to minimize the time spent waiting for the listener endpoint connection to succeed. If the time is too short, you might give up on a connection attempt too soon, causing more delays and potentially a failure to connect. If the time is too

long, then if the listener endpoint is unreachable, you might spend too much time waiting before giving up and trying another endpoint.

The host names specify SCAN VIPs. They are always available when using a cluster. This means that if a node or network is down, the VIP sends an instant reply, so that when connecting, the next address is used immediately if the service is not offered at the VIP address.

LOAD_BALANCE=ON within ADDRESS_LIST

When a HOST inside an ADDRESS resolves to multiple addresses for Oracle RAC SCAN, then all of the addresses are tried in a random order.

If you set LOAD_BALANCE=OFF, then the order is the same every time, which might overburden one of the SCAN listeners, so its recommended to set it to ON.

EXPIRE_TIME

With Oracle Client 19c, EXPIRE_TIME can be used in tnsnames.ora connect descriptors to prevent firewalls from terminating idle connections and to adjust keepalive timeouts.

The general recommendation for EXPIRE_TIME is to use a value that is slightly less than half of the termination period. In older versions of Oracle Client, a tnsnames.ora connect descriptor option ENABLE=BROKEN can be used instead of EXPIRE_TIME. These settings can also aid detection of a terminated remote database server.

Connection Retry Logic Examples

Reference code examples for reconnection logic.

Simple Retry

```

Connection jdbcConnection = getConnection();
int iterationCount = 0;
int maxIterations = 10;
for (int i = 0; i < maxIterations, i++) {
    try {
        // apply the raise (DML + commit):
        giveRaiseToAllEmployees(jdbcConnection, i * 5);
        // no exception, the procedure completed:
        iterationCount++;
        Thread.sleep(1000);
    } catch (SQLRecoverableException recoverableException) {
        // Get a new connection only if the error was recoverable.
        System.out.println("SQLRecoverableException on iteration " +
iterationCount )
        System.out.println("DB Connection lost - will attempt to get a new
connection to continue with the other iterations")

        // IF its OK to lose this work and move onto the next
        // iteration you could now try to get a new connection
        // This depends on what the code is doing; in many use
        // cases you must stop working, in others you can proceed
        // after logging a message to a log file
        // In our example, we assume we can proceed with the rest
        // of the loop if possible.
        // Using Transaction Guard, we can know if the work

```

```

        // committed and move on safely (covered in another example).
        try {
            jdbcConnection.close(); // close old connection:
            System.out.println("Connection closed - getting a new one")
            jdbcConnection = getConnection(); // reconnect to continue with
other iterations
        } catch (Exception ex) {
            System.out.println("Unable to close or get a new connection -
giving up")
            throw ex;
        }
    } catch (SQLException nonRecoverableException) {
        // This is not a recoverable exception, so give up
        System.out.println("SQL UN-recoverable exception...give up the rest of
the iterations")
        throw nonRecoverableException;
    }
}

```

Connection Retry Logic with Transaction Guard

```

Connection jdbcConnection = getConnection();
boolean isJobDone = false;
while (!isJobDone) {
    try {
        // apply the raise (DML + commit):
        giveRaiseToAllEmployees(jdbcConnection, 5);
        // no exception, the procedure completed:
        isJobDone = true;
    } catch (SQLRecoverableException recoverableException) {
        // Retry only if the error was recoverable.
        try {
            jdbcConnection.close(); // close old connection:
        } catch (Exception ex) {} // pass through other exceptions
        Connection newJDBCCConnection = getConnection(); // reconnect to allow
retry
        // Use Transaction Guard to force last request: committed or
uncommitted
        LogicalTransactionId ltxid
            = ((OracleConnection) jdbcConnection).getLogicalTransactionId();
        isJobDone = getTransactionOutcome(newJDBCCConnection, ltxid);
        jdbcConnection = newJDBCCConnection;
    }
}

void giveRaiseToAllEmployees(Connection conn, int percentage) throws
SQLException {
    Statement stmt = null;
    try {
        stmt = conn.createStatement();
        stmt.executeUpdate("UPDATE emp SET sal=sal+(sal*" + percentage + "/"
100)");
    } catch (SQLException sqle) {
        throw sqle;
    } finally {

```



```

        if (stmt != null)
            stmt.close();
    }
    // At the end of the request we commit our changes:
    conn.commit(); // commit can succeed but the commit outcome is lost
}

/**
 * GET_LTXID_OUTCOME_WRAPPER wraps DBMS_APP_CONT.GET_LTXID_OUTCOME
 */
private static final String GET_LTXID_OUTCOME_WRAPPER =
    "DECLARE PROCEDURE GET_LTXID_OUTCOME_WRAPPER(" +
    "  ltxid IN RAW," +
    "  is_committed OUT NUMBER ) " +
    "IS " +
    "  call_completed BOOLEAN; " +
    "  committed BOOLEAN; " +
    "BEGIN " +
    "  DBMS_APP_CONT.GET_LTXID_OUTCOME(ltxid, committed, call_completed); " +
    "  if committed then is_committed := 1; else is_committed := 0; end if;
" +
    "END; " +
    "BEGIN GET_LTXID_OUTCOME_WRAPPER(?,?); END;";

/**
 * getTransactionOutcome returns true if the LTXID committed or false
 * otherwise.
 * note that this particular version is not considering user call completion
 */
boolean getTransactionOutcome(Connection conn, LogicalTransactionId ltxid)
throws SQLException {
    boolean committed = false;
    CallableStatement cstmt = null;
    try {
        cstmt = conn.prepareCall(GET_LTXID_OUTCOME_WRAPPER);
        cstmt.setObject(1, ltxid); // use this starting in 12.1.0.2
        cstmt.registerOutParameter(2, OracleTypes.BIT);
        cstmt.execute();
        committed = cstmt.getBoolean(2);
    } catch (SQLException sqlexc) {
        throw sqlexc;
    } finally {
        if (cstmt != null)
            cstmt.close();
    }
    return committed;
}

```

Oracle Call Interface (OCI) applications; how to determine if an error is recoverable:

```

OCIError *errhp;
sb4 errcode = 0;
text errbuf[512];
boolean recoverableError;

```

```
(void) OCIErrorGet((dvoid *)errhp, (ub4) 1, (text *) NULL, &errcode,
                  errbuf, (ub4) sizeof(errbuf), OCI_HTYPE_ERROR);

/* check if error is recoverable */
OCIAttrGet((void *)errhp, OCI_HTYPE_ERROR,
           (dvoid *)&recoverableError, (ub4 *)0,
           (ub4)OCI_ATTR_ERROR_IS_RECOVERABLE, errhp);
```

Server-Side Planned Maintenance Command Examples

Note

- If you are using these commands in scripts, you may find it helpful to include `wait = yes`.
- The parameters, `-force -failover` cause the service to start on other available instances configured on each service.
- For more details see *Managing a Group of Services for Maintenance in Oracle Real Application Clusters Administration and Deployment Guide*.

Stop one instance (inst1) with all associated services' configured `-drain_timeout` and `-stopoption` parameters

```
srvctl stop instance -db myDB -instance inst1 -force -failover
-role primary
```

Stop all instances with explicit draining parameters that override the parameters configured for associated services.

```
srvctl stop instance -db db_name -node node_name
-stopoption IMMEDIATE -drain_timeout <#> -force -failover
```

Stop a service with explicit draining parameters.

```
srvctl stop service -db db_name -service service_name
-instance instance_name -drain_timeout <#> -stopoption IMMEDIATE
-force -failover
```

Stop a Data Guard instance with explicit draining parameters.

```
srvctl stop instance -db db_name -node node_name
-stopoption IMMEDIATE -drain_timeout <#> -force -failover
-role primary
```

Relocate all services by database, node, or PDB.

```
srvctl relocate service -database db_unique_name
-pdb pluggable_database
{-oldinst old_inst_name [-newinst new_inst_name] |
```

```

        -currentnode current_node
        [-targetnode target_node]}
    -drain_timeout timeout -stopoption stop_option -force

    srvctl relocate service -database db_unique_name
    -oldinst old_inst_name [-newinst new_inst_name]
    -drain_timeout timeout -stopoption stop_option
    -force

    srvctl relocate service -database db_unique_name
    -currentnode current_node [-targetnode target_node]
    -drain_timeout timeout -stopoption stop_option
    -force

```

Switch over to a Data Guard secondary site with a wait timeout of 60 seconds, using Data Guard Broker.

```
SWITCHOVER TO dg_south WAIT 60
```

Switch over to Data Guard secondary site with a wait timeout from the services, using Data Guard Broker.

```
SWITCHOVER TO dg_south WAIT
```

Configuring FAN for Specific Drivers and Pools

Specific requirements and recommendations for configuring FAN with various database client drivers.

JDBC FAN Requirements

Oracle Universal Connection Pool (UCP)

- Use the recommended connection URL/string for auto-configuration of ONS.
- Include JDBC JAR files `ojdbcXXX.jar`, `ons.jar`, and `simplefan.jar`, on the CLASSPATH.
 Include optional wallet JARs, if needed: `osdt_cert.jar` (prior to Oracle 23ai only), `osdt_core.jar` (prior to Oracle 23ai only), and `oraclepki.jar` for Oracle Cloud Autonomous Database (Serverless or Dedicated Exadata Infrastructure) with mutual TLS, or if you have set up client-side wallets for database connections.
 Ensure that all JAR files are from the same version (do not mix and match versions).
- Set the pool or driver property to enable Fast Connection Failover.
 For example, in UCP it is set for the `PoolDataSource` using `setFastConnectionFailoverEnabled(true)`.
 For UCP and JDBC 23ai or later, this is done by default when connected to an Oracle RAC database.
- If you are using a third party pool:
 - Java-based application server, the most effective method to achieve draining and failover is to replace the pooled data source with UCP.

This approach is supported by many application servers including Oracle WebLogic Server, IBM WebSphere, IBM Liberty (XA and non-XA usage), Apache Tomcat, Hikari CP, and others.

If your application is using J2EE or Container Managed Transactions (CMT) with Red Hat JBoss, request boundaries are provided with version Red Hat JBoss EAP 7.4.

This configuration supports draining with FAN (XA and non-XA usage).

```
<dependencies>

...
<dependency>
  <groupId>com.oracle.database.ha</groupId>
  <artifactId>ons</artifactId>
  <version>${oracle.jdbc.version}</version>
</dependency>
<dependency>
  <groupId>com.oracle.database.ha</groupId>
  <artifactId>simplefan</artifactId>
  <version>${oracle.jdbc.version}</version>
</dependency>

...
</dependencies>
```

Non-Oracle Connection Pools or Standalone JDBC Applications

For clients not using UCP, but using third-party connection pools or standalone JDBC you typically need to do the following:

- Configure the driver to handle FAN events by adding `simplefan` and `ons` jar files to your CLASSPATH or project's `pom.xml`.
- Call any Oracle JDBC draining API (`isValid()`, `isUsable()`, `pingDatabase()`, `endRequest()`), and set the following Java system property for lightweight connection validation:

```
-Doracle.jdbc.defaultConnectionValidation=SOCKET
```

OCI-Based Applications

- OCI-based clients embed FAN at the driver level. Pooling is not required to receive FAN notifications.
- The database service must have the attribute "-notification TRUE" set.
- If `oraaccess.xml` is in use, ensure that the `events` tag is `TRUE`:

```
<oraaccess> xmlns="http://xmlns.oracle.com/oci/oraaccess"
  xmlns:oci="http://xmlns.oracle.com/oci/oraaccess"
  schemaLocation="http://xmlns.oracle.com/oci/oraaccess
    http://xmlns.oracle.com/oci/oraaccess.xsd">
  <default_parameters>
    <events>true</events>
  </default_parameters>
</oraaccess>
```

- If you are not using `oraaccess.xml`, an alternative is to change the application code to use `OCI_EVENTS` mode. For example:

```
sword errcode = OCIEnvCreate(&envhp, OCI_EVENTS, 0, 0, 0, 0, 0, 0);  
if (errcode)
```

- For Python, use the OCI "thick" drivers and make connections with the "events" property set to true; for example:

```
conn = oracledb.connect(dsn="localhost:1521/DB23PDB1", user="my_user",  
password="my_pw", events=True)
```

ODP.NET FAN Requirements

All ODP.NET provider types (core, managed, and unmanaged) support FAN.

ODP.NET out-of-band FAN requires connection pooling and HA Events to be enabled, which they are by default. No code changes are necessary if these attributes are not changed.

The following is an example of a FAN-enabled ODP.NET connection string:

```
"user id=oracle; password=oracle; data source=HA; pooling=true; HA  
events=true;"
```

In-band FAN is enabled by default.

Read more about ODP.NET high availability in Real Application Clusters and Global Data Services.

JBoss

Use ONS FAN (recommended) if possible, because it also covers unplanned outages. If ONS FAN fails, in-band is used instead. Both can also work at the same time.

For testing – update data source definition in standalone/domain configuration XML

For production – use JBoss management console or CLI commands to update DS definition

JBoss configuration settings

- Validate connections on each check-out using "Validate on match"
- Do **not** use background validation
- Do **not** configure "purge pool disabled/abandon pool disable"

Follow up with Redhat documentation for draining configuration: <https://access.redhat.com/solutions/42463>

Also, see Redhat documentation for connection validation: https://docs.redhat.com/en/documentation/red_hat_jboss_enterprise_application_platform/7.4/html/configuration_guide/datasource_management#configure_database_connection_validation_settings

To specify additional JDBC driver connection properties:

```
<connection-property name="URL">  
    jdbc:oracle:thin:@CURRENT_URL?TNS_ADMIN=/absolute_path/ojdbc.properties  
</connection-property>
```

In `/absolute_path/ojdbc.properties`, include the following connection properties and values:

```
prop1=value1
prop2=value2
...
```

Other Pools and Drivers

- For Hikari connection pools, see the article [HikariCP Best Practices for Oracle Database and Spring Boot](#).

ONS FAN Application Requirements

Ensure your application server, driver version, and connection pool are configured properly to enable sessions to drain when using ONS FAN.

Oracle UCP

Connection Pool Version	Oracle Driver Version	Recommended Configuration Settings	Comments
Earlier than Oracle 23ai	Earlier than Oracle 23ai	In your app server/container/framework's data source configuration file: Set the UCP property: "fastConnectionFailoverEnabled(true)"	Include JDBC JAR files <code>ojdbcXXX.jar</code> , <code>ons.jar</code> on the CLASSPATH Optional wallet jars, if needed: <code>osdt_cert.jar</code> , <code>osdt_core.jar</code> , and <code>oraclepki.jar</code> for Oracle Cloud Autonomous Database, or if you have set up wallets for database connections. Ensure that all JAR files are from the same version (do not mix and match versions). ONS FAN is auto-configured when using the MAA recommended TNS/URL
23.5.0.24.7 or later	23.5.0.24.7 or later	Use the default settings	Include JDBC JAR files <code>ojdbcXXX.jar</code> , <code>ons.jar</code> on the CLASSPATH Optional wallet jars, if needed: <code>osdt_cert.jar</code> , <code>osdt_core.jar</code> , and <code>oraclepki.jar</code> for Oracle Cloud Autonomous Database, or if you have set up wallets for database connections. Ensure that all JAR files are from the same version (do not mix and match versions) ONS FAN is auto-configured when using the MAA recommended TNS/URL

JDBC

Applies to JDBC used in third-party pools or directly in custom code.

Oracle Driver Version	Recommended Configuration Settings	Comments
All supported driver versions	- Third-party pools should add pool property to validate connection at checkout (varies by pool; see examples for Hikari and Tomcat below), or applications directly calling the Oracle JDBC driver. <code>java.sql.Connection.isValid(int timeout)</code> <code>oracle.jdbc.OracleConnection.pingDatabase()</code> <code>oracle.jdbc.OracleConnection.pingDatabase(int timeout)</code> <code>oracle.jdbc.OracleConnection.endRequest()</code> - List of APIs: Safe Draining APIs for Planned Maintenance - Third-party pools or applications NOT calling Oracle JDBC driver: Execute SQL with <code>draining-hint</code> or in <code>DBA_CONNECTION_TESTS</code>	Include JDBC JAR files <code>ojdbcXXX.jar</code>, <code>ons.jar</code>, <code>simplefan.jar</code> on the CLASSPATH If you have set up wallets for database connections in Oracle Cloud Autonomous Database, include wallet jars: <code>osdt_cert.jar</code>, <code>osdt_core.jar</code>, and <code>oraclepki.jar</code> Ensure that all JAR files are from the same version (do not mix and match versions) ONS FAN is auto-configured when using the MAA recommended TNS/URL

WebLogic

Includes WebLogic Generic, WebLogic MDS, and WebLogic AGL.

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
14.1.1 or later	All supported versions Recommend upgrading JDBC driver to the latest version that WebLogic AGL supports	Upgrade to latest driver, or at a minimum driver version: 19.13 Enable FAN on AGL data source	By default, draining can happen at connection check-in with <code>endRequest</code> . With the recommended settings, draining can also happen at connection check-out with <code>beginRequest</code> .

Hikari

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
Earlier than 6.0	19.21 or later	Upgrade to Hikari 6.0	Not a recommended configuration No request boundaries - no draining by default

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
6.0	19.21 or later	With the following properties, draining occurs at endRequest (enabled by the following properties): - System.setProperty("com.zaxxer.hikari.aliveBypassWindowMs", "-1") Set the validation level using: - setProperty("oracle.jdbc.defaultConnectionValidation", "SOCKET"); - com.zaxxer.hikari.enableRequestBoundaries is set to true	See HikariCP Best Practices for Oracle Database and Spring Boot for configuration using ONS/simplefan

.NET

Application Server Version	Oracle Driver Version	Recommended Configuration Settings
Any currently supported version	ODP.NET Core - Any currently supported version	On by default in ODP.NET Pooling and HA Events connection string attributes must remain set to true

.NET Framework

Application Server Version	Oracle Driver Version	Recommended Configuration Settings
Any currently supported version	Managed ODP.NET - Any currently supported version Unmanaged ODP.NET - Any currently supported version	On by default in ODP.NET Pooling and HA Events connection string attributes must remain set to true

JBOSS

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
Versions earlier than EAP 7.3.9	All supported driver versions, latest version recommended	- Validate connections on each check-out using "Validate on match" - Do not use background validation - Do not configure "purge pool disabled/abandon pool disable" See also Configuring FAN for Specific Drivers and Pools	Without validation-SQL, a properly configured native pool may drain on each checkout (by validating connection) If the application is calling JDBC endRequest then it drains at check-in also, otherwise it will not. See the requirements of the JDBC driver above for details.

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
Versions higher than EAP 7.3.9	All supported driver versions, latest version recommended	• Validate connections on each check-out using "Validate on match" • Do not use background validation • Do not configure "purge pool disabled/abandon pool disable" See also Configuring FAN for Specific Drivers and Pools	Without validation-SQL, a properly configured native pool may drain on each check-out (by validating connection) and at check-in (automatically due to endRequest calls by JBOSS). See the requirements of the JDBC driver above for details.

Tomcat

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
9.0, 10.0, 11.0	Any version	If you are using Tomcat with UCP, follow the UCP recommendations above. If you are using Tomcat with the Oracle JDBC driver or Oracle DBCP: • Call any Oracle JDBC draining API (isValid(), isUsable(), pingDatabase(), endRequest()) at connection borrow and return. • Set the lightweight connection validation property to SOCKET: <code>setProperty("oracle.jdbc.defaultConnectionValidation", "SOCKET");</code>	With the recommended changes, draining occurs at connection check-out and check-in (endRequest)
9.0, 10.0, 11.0	Any version	Third-party pools or applications NOT calling Oracle JDBC driver: • Enable connection validation at borrow and return • Use isValid(), or execute SQL with draining-hint or in DBA_CONNECTION_TEST S	Draining occurs at connection check-in (endRequest) with the recommended changes

Long Running PL/SQL

Application Server Version	Recommended Configuration Settings	Comments
Oracle Database 19 or higher	Use the following SQL to test drain status between batches. If draining state is detected, then close the connection and obtain a new one. - When draining is in progress: SQL> select SYS_CONTEXT('USERENV', 'DRAIN_STATUS') from dual ; Returns: DRAINING - When not draining: SQL> select SYS_CONTEXT('USERENV', 'DRAIN_STATUS') from dual ; Returns: NONE	No draining by default; must use recommended SQL at batch intervals. This will work if the batch intervals are less than 1/2 of the drain timeout interval.

OCI-Based Applications

Including Pro*C/C++ , Pro*COBOL, and python-oracledb in Thick Mode

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
Oracle Database 19 or higher	Oracle Database 19 or higher	OCI clients embed FAN at the driver level so that all clients can use them regardless of the pooling solution. The database service must have the attribute "-notification TRUE" set If oraaccess.xml is in use, ensure that the events tag is TRUE: <pre><oraaccess> xmlns="http:// xmlns.oracle.com/oci/ oraaccess" xmlns:oci="http:// xmlns.oracle.com/oci/ oraaccess" schemaLocation="http :// xmlns.oracle.com/oci/ oraaccess http:// xmlns.oracle.com/oci/ oraaccess.xsd"> <default_parameters> <events>true</ events> </ default_parameters> </oraaccess></pre>	Python must use the "thick" driver and make connections with the events property set to TRUE

In-Band Draining Notification Application Requirements

Ensure your application server, driver version, and connection pool are configured properly to enable sessions to drain when using in-band notifications.

Oracle UCP

Connection Pool Version	Oracle Driver Version	Recommended Configuration Settings	Comments
23.5.0.24.7 or later	23.5.0.24.7 or later	Use the default settings	Preferred version

Connection Pool Version	Oracle Driver Version	Recommended Configuration Settings	Comments
19.12 or later	19.12 or later	Set the UCP property: <code>setValidateConnectionOnBorrow(true)</code> Note that there are several validation modes, and a mode can be set using driver settings.	Draining occurs at connection check-out (<code>beginRequest</code>) and check-in (<code>endRequest</code>) with the recommended changes UCP will use ensure that JDBC uses a lightweight connection check that doesn't require a network round trip when <code>setValidateConnectionOnBorrow(true)</code> is used.
19.11 and earlier	19.11 and earlier	Upgrade to latest driver, or at a minimum driver version 19.12	Not a recommended configuration No In-band notifications available; no draining possible unless ONS-based FAN is used

WebLogic

Includes WebLogic Generic, WebLogic MDS, and WebLogic AGL.

Application Server and Oracle Driver Version Combination	Recommended Configuration Settings	Comments
12.2.1.4 with 19.12 or later 12.1.1.6 with 19.12 or later 14.1.1 with 19.12 or later 14.1.2 with 23.5.0.24.7 or later	All WLS versions require fix for bug 35731335 The following parameters are needed: test-connections-on-reserve=true test-table-name=SQL ISVALID (default: null) seconds-to-trust-an-idle-pool-connection=0 (default: 10) test-frequency-seconds (default: 120 seconds, idle connections) count-of-refresh-failures-till-disable=0 (default: 2) count-of-test-failures-till-flush=0 (default: 2)	By default, draining can happen at connection check-in with <code>endRequest</code> . With the recommended settings, draining can also happen at connection check-out with <code>beginRequest</code> .
12.2.1.4 with Oracle 19.11 and earlier	Upgrade to latest Oracle driver, or at a minimum driver version: 19.12	Not a recommended configuration No In-band notifications available; no draining possible unless you have FAN

JDBC

Applies to JDBC used in third-party pools or directly in custom code.

Oracle Driver Version	Recommended Configuration Settings	Comments
19.12 or later	- Because JDBC driver does not have knowledge of pooling actions, the only places it can drain safely are when any of the "safe-draining APIs" gets called. Third-party pools should add pool property to validate connection at checkout (varies by pool, see examples for Hikari and Tomcat below), or applications directly calling the Oracle JDBC driver: <code>java.sql.Connection.isValid(int timeout)</code> <code>oracle.jdbc.OracleConnection.pingDatabase()</code> <code>oracle.jdbc.OracleConnection.pingDatabase(int timeout)</code> <code>oracle.jdbc.OracleConnection.endRequest()</code> List of APIs: Safe Draining APIs for Planned Maintenance - Alternatively, applications using <code>beginRequest</code> and <code>endRequest</code> will allow safe drain points to be used - Third-party pools or apps calling Oracle JDBC driver: Call any Oracle JDBC draining API (listed above) and set the lightweight connection validation property to <code>SOCKET</code> if applicable to the pool (see Configuring FAN for Specific Drivers and Pools) - Third-party pools or apps NOT calling Oracle JDBC driver: Execute SQL with <code>draining-hint</code> or in <code>DBA_CONNECTION_TESTS</code>	Draining occurs at connection check-out and check-in (if <code>endRequest</code> is used) with the recommended changes Oracle JDBC 19.15 is recommended if using AC or TAC so planned failover operates at <code>beginRequest</code>
19.11 and earlier	Upgrade to latest driver, or at a minimum driver version 19.12	Not a recommended configuration No In-band notifications available; no draining possible

Hikari

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
6.0	19.21 or later	With the following properties, draining occurs at endRequest (enabled by the following properties): <code>System.setProperty("com.zaxxer.hikari.aliveBytesWindowMs", "-1")</code> and set the validation level using : <code>setProperty("oracle.jdbc.defaultConnectionValidation", "SOCKET");</code> <code>com.zaxxer.hikari.enableRequestBoundaries</code> is set to true	Draining occurs at connection check-out (beginRequest) and check-in (endRequest) with the recommended changes
earlier than 6.0	19.21 or later	Upgrade to Hikari 6.0	Not a recommended configuration No request boundaries - no draining by default

.NET

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
Any currently supported version	ODP.NET Core 19.9 or later	Use the default settings	ODP.NET in-band FAN notifications support the following usage scenarios: • Pluggable database relocation • Database service relocation • Database rolling upgrade • Connection Manager planned maintenance Supported with Oracle Database 18c and higher.

.NET Framework

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
Any currently supported version	Managed ODP.NET 19.9 or later Unmanaged ODP.NET 19.10 or later	On by default in ODP.NET	ODP.NET in-band FAN notifications support the following usage scenarios: • Pluggable database relocation • Database service relocation • Database rolling upgrade • Connection Manager planned maintenance Supported with Oracle Database 18c and higher.

Tomcat

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
9.0, 10.0, 11.0	Any version	If you are using Tomcat with UCP, follow the UCP recommendations above. If you are using Tomcat with the Oracle JDBC driver or Oracle DBCP: • Call any Oracle JDBC draining API (<code>isValid()</code>, <code>isUsable()</code>, <code>pingDatabase()</code>, <code>endRequest()</code>) at connection borrow and return. • Set the lightweight connection validation property to <code>SOCKET</code>: <code>setProperty("oracle.jdbc.defaultConnectionValidation", "SOCKET");</code>	With the recommended changes, draining occurs at connection check-out (<code>beginRequest</code>) and check-in (<code>endRequest</code>) with the recommended changes UCP uses lightweight connection tests (<code>SOCKET</code>) by default.
9.0, 10.0, 11.0	Any version	Third-party pools or applications NOT calling Oracle JDBC driver: • Enable connection validation at borrow and return • Use <code>isValid()</code>, or execute SQL with <code>draining-hint</code> or in <code>DBA_CONNECTION_TEST</code>	Draining occurs at connection check-in (<code>endRequest</code>) with the recommended changes

JBOSS

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
EAP 7.3.9 and later	19.12 or later	- Validate connections on each check-out using "Validate on match" - Do not use background validation - Do not configure "purge pool disabled/abandon pool disable" See also Configuring FAN for Specific Drivers and Pools	Without validation-SQL, a properly configured native pool may drain on each checkout (by validating each connection) and at checkin (automatically due to endRequest calls by JBOSS).
Versions earlier than EAP 7.3.9	19.12 or later	- Validate connections on each check-out using "Validate on match" - Do not use background validation - Do not configure "purge pool disabled/abandon pool disable" See also Configuring FAN for Specific Drivers and Pools	Without validation-SQL, a properly configured native pool may drain on each checkout (by validating connection). If the application is calling JDBC endRequest, then it may drain at checkin also, otherwise it will not. See the requirements of the JDBC driver for detail.

Long Running PL/SQL

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
Oracle Database 19 or higher	NA	Use the following SQL to test drain status between batches. The idea is to check the draining state. If draining state is detected, then close the connection and obtain a new one. - When draining is in progress: <pre>SQL> select SYS_CONTEXT('USEREN V', 'DRAIN_STATUS') from dual ;</pre> Returns: DRAINING - When not draining: <pre>SQL> select SYS_CONTEXT('USEREN V', 'DRAIN_STATUS') from dual ;</pre> Returns: NONE	No draining by default; must use recommended SQL at batch intervals. This will work if the batch intervals are less than 1/2 of the drain timeout interval.

OCI-Based Applications

Pro*C/C++ , Pro*COBOL and python-oracledb in Thick mode.

Application Server Version	Oracle Driver Version	Recommended Configuration Settings	Comments
Oracle Database 19 or higher	Oracle Database 19 or higher	- No additional configuration is needed for OLTP-style applications - Long running applications should use the following Oracle Call Interface OCIAttrGet example to test drain status between batches and obtain a new session when draining is detected. If not connected state is detected, then close the connection and obtain a new one. <pre> ub4 serverStatus = 0 OCIAttrGet((dvoid *)srvhp, OCI_HTYPE_SERVER, (dvoid *)&serverStatus, (ub4 *)0, OCI_ATTR_SERVER_ST ATUS, errhp); if (serverStatus == OCI_SERVER_NORMAL) printf("Continue. Connection is up.\n"); else if (serverStatus == OCI_SERVER_NOT_CON NECTED) printf("close your session and get another .\n"); </pre> This will work if the batch intervals are less than 1/2 of the drain timeout interval.	Draining occurs at checkout from the OCI Session Pool for OCI and python-oracledb in Thick mode. Draining occurs at an explicit request boundary (OCIRequestBegin for OCI applications and EXEC SQL REQUEST BEGIN for Pro*/C++ and Pro*COBOL applications). Long running applications should test drain status as shown in the "Recommended Configuration Settings" here.

Long Running JDBC

Application Server Version	Oracle Driver Version	Recommended Configuration Settings
Oracle Database 19 or higher	Oracle Database 19 or higher	- Use the following connection test to test drain status between batches. If draining state is detected, then close the session and obtain a new one, then continue processing the next batch. Call any Oracle JDBC draining API (<code>isValid()</code>, <code>isUsable()</code>, <code>pingDatabase()</code>, <code>endRequest()</code>) and set the lightweight connection validation property to <code>SOCKET</code>: <pre>setProperty("oracle.jdbc.defaultConnectionValidation", "SOCKET");</pre> - Alternatively, configure Transparent Application Continuity to do planned failover during draining (requires Oracle Database 19.15 or higher)

Part IX

Oracle Multitenant Best Practices

- [Overview of Oracle Multitenant Best Practices](#)
- [PDB Switchover and Failover in a Multitenant Configuration](#)

Overview of Oracle Multitenant Best Practices

Oracle Multitenant is Oracle's strategic product for database consolidation.

The benefits of the Oracle Multitenant architecture include:

- Access isolation between individual Pluggable Databases (PDBs) stored in the same Container Database (CDB)
- Ability to manage many databases with the simplicity of managing just one CDB that contains many PDBs. By backing up your CDB, updating the CDB software or setting up a standby CDB for disaster recovery, you are essentially reducing the complexity and steps by administrating one CDB instead of applying the same administrative steps on many independent databases. You reduce administrative tasks, steps and errors.
- Sharing of system resources to reduce CAPEX with the flexibility to set resource limits for things like memory, I/O and on a per PDB level
- Flexibility to operate on an individual PDB, for example relocating a single PDB into another container and upgrading just that PDB
- Rapid cloning and provisioning
- Tight integration with Oracle RAC

The following table highlights various Oracle Multitenant configuration and operational best practices. All best practices should be used for on-premises environments. Many of them have been automated in Oracle Cloud Infrastructure (OCI).

Table 42-1 Oracle Multitenant configuration and operational best practices

Use Case	Best Practices	Oracle Cloud Infrastructure Support
Pluggable Database (PDB) configuration	For all Oracle RDBMS releases 12c Release 2 (12.2) to 21c, configure the CDB with local undo mode See Undo Modes in 12.2 Multitenant Databases - Local and Shared Modes (Doc ID 2169828.1)	Supported as part of CDB and PDB creation

Table 42-1 (Cont.) Oracle Multitenant configuration and operational best practices

Use Case	Best Practices	Oracle Cloud Infrastructure Support
PDB service management	Mandatory MAA best practices for any Oracle databases with Oracle Clusterware (for example, Oracle RAC and single instance databases with Oracle Clusterware installed) 1. Never use PDB default services, nor SAVED STATE (except during relocate operations), nor database triggers to manage role-based services. 2. Use clusterware-managed distinct services per PDB for your application service, and leverage that application service to connect to the database. 3. When defining a clusterware-managed application service, define which PDB and services will be started and in which RAC instance and database role. 4. For Data Guard, always use role-based services by assigning a role to each clusterware-managed service. If the above practices are applied, you will have predictable service management during PDB open and Data Guard role transitions. This will lead to higher application service availability and avoid application errors. For single instance databases without MAA-recommended Oracle clusterware setup, follow these practices: 1. Never use PDB default services. 2. Use distinct services per PDB for your application service, and leverage that application service to connect to the database. 3. For non-Data Guard, use only SAVED state to open PDBs and start up explicit application services OR for Data Guard, use only AFTER STARTUP database triggers to programmatically manage which application services should be started depending on primary, READ ONLY, or snapshot standby database role. See Best Practices for Pluggable Database End User and Application	OCI creates non-default services as part of PDB creation. Ensure that they are used.

Table 42-1 (Cont.) Oracle Multitenant configuration and operational best practices

Use Case	Best Practices	Oracle Cloud Infrastructure Support
	Connection and Open on Database Startup (Doc ID 2833029.1)	
Using Data Guard with Oracle Multitenant	The following My Oracle Support notes provide operational best practice recommendations when using Oracle Multitenant in an Oracle Data Guard configuration. • Data Guard Impact on Oracle Multitenant Environments (Doc ID 2049127.1) • Making Use of Deferred PDB Recovery and the STANDBYS=NONE Feature with Oracle Multitenant (Doc ID 1916648.1) during use cases: PDB creation, PDB migration, and PDB cloning • Reusing the Source Standby Database Files When Plugging a PDB into the Primary Database of a Data Guard Configuration (Doc ID 2273829.1) for PDB migration • Reusing the Source Standby Database Files When Plugging a non-CDB as a PDB into the Primary Database of a Data Guard Configuration (Doc ID 2273304.1) for PDB migration • Using standby_pdb_source_file_dblink and standby_pdb_source_file_directory to Maintain Standby Databases when Performing PDB Remote Clones or Plugins (Doc ID 2274735.1) for PDB remote clone or PDB plug-in • Parameter enabled_pdb_on_standby and STANDBYS Option With Data Guard Subset Standby (Doc ID 2417018.1) to support subset standby	Data Guard Impact on Oracle Multitenant Environments (Document 2049127.1) provides useful information and should be reviewed. OCI automates PDB instantiation at the standby but does not support the following additional options: • Making Use of the STANDBYS=NONE Feature with Oracle Multitenant Document 1916648.1 • Reusing the Source Standby Database Files When Plugging a PDB into the Primary Database of a Data Guard Configuration Document 2273829.1 • Reusing the Source Standby Database Files When Plugging a non-CDB as a PDB into the Primary Database of a Data Guard Configuration Document 2273304.1 • Using standby_pdb_source_file_dblink and standby_pdb_source_file_directory to Maintain Standby Databases when Performing PDB Remote Clones or Plugins Document 2274735.1 • Parameter enabled_pdb_on_standby and STANDBYS Option With Data Guard Subset Standby Document 2417018.1
Data Guard: PDB switchover and failover use cases	See PDB Switchover and Failover in a Multitenant Configuration	Oracle Cloud automation does not support PDB switchover and failover.

Table 42-1 (Cont.) Oracle Multitenant configuration and operational best practices

Use Case	Best Practices	Oracle Cloud Infrastructure Support
PDB migration	The following resources provide operational best practices for different types of PDB migration with minimal downtime: • Use Zero Downtime Migration if your target is Exadata platform or cloud. See https://www.oracle.com/database/technologies/rac/zdm.html • Step by Step Process of Migrating non-CDBs and PDBs Using ASM for File Storage (Doc ID 1576755.1) • Cloning a Pluggable Database from an RMAN Container Database Backup (Doc ID 2042607.1) • PDB Switchover and Failover in a Multitenant Configuration	Oracle Cloud Automation supports cloning a PDB from backups using its own automation.
PDB relocation	• Using PDB Relocation to Upgrade an Individual PDB (Doc ID 2771716.1) • Using PDB Relocation to Move a Single PDB to Another CDB Without Upgrade (Doc ID 2771737.1)	Oracle Cloud Automation supports both processes and automates end to end, including handling instantiation of the PDB at standby databases.
PDB resource management	The following My Oracle Support note provides operational use cases for Oracle Multitenant resource management: How to Control and Monitor the Memory Usage (Both SGA and PGA) Among the PDBs in Multitenant Database- 12.2 New Feature (Doc ID 2170772.1)	You can configure or modify resource plans and individual PDB settings for resource management tasks in Oracle cloud

With Oracle Multitenant MAA solutions, you can achieve administration and system resource savings while still benefiting from various MAA solutions. The following tables highlight zero and near-zero downtime and data loss for various unplanned outages and planned maintenance activities.

Table 42-2 Unplanned Outages

Unplanned Outages	Key Features for Solution	RTO	RPO
Recoverable node or instance failures	Real Application Cluster (RAC) Application Continuity (AC)	Seconds	Zero
Database, cluster, and site failures	Active Data Guard fast-start failover	<2 Minutes	Zero or Seconds
Data corruptions	Active Data Guard that includes auto block repair of physical corruptions	Zero	Zero

Table 42-2 (Cont.) Unplanned Outages

Unplanned Outages	Key Features for Solution	RTO	RPO
PDB unrecoverable failure or "sick" PDB	PDB failover using Data Guard migrate command Another target CDB on the same cluster is required See PDB Failover in a Data Guard environment: Using Data Guard Broker to Unplug a Single Failed PDB from a Standby Database and Plugging into a New Container or Migrate a Single PDB into a New Container (Doc ID 2088201.1)	<2 Minutes	Zero or Seconds
PDB failover to active replica	Option 1: Failover entire CDB with Primary and Standby CDB Data Guard architecture Option 2: Create PDB replica with Oracle GoldenGate. Perform PDB active failover using PDB replica in a different CDB. Use Global Data Services and practices from Application Checklist for Continuous Service for MAA Solutions help with the application failover	Potentially Zero	Zero or Seconds

Table 42-3 Planned Maintenance

Planned Downtime	Solution	RTO
Software and hardware updates	Real Application Cluster (RAC) Application Checklist for Continuous Service for MAA Solutions	Zero
Major database upgrade for entire CDB	Active Data Guard DBMS_ROLLING	Seconds
Major database upgrade for single PDB within CDB	PDB Relocate + Upgrade See Using PDB Relocation to Upgrade an Individual PDB (Doc ID 2771716.1)	Minutes
Migration to remote CDB	PDB Relocate See Using PDB Relocation to Move a Single PDB to Another CDB Without Upgrade (Doc ID 2771737.1)	Minutes
Migration to remote CDB (logical migration)	Data Pump and Oracle GoldenGate or Zero Downtime Migration	Potentially Zero

PDB Switchover and Failover in a Multitenant Configuration

The use cases documented here demonstrate how to set up single pluggable database (PDB) failover and switchover for an Oracle Data Guard configuration with a container database (CDB) with many PDBs.

With Oracle Multitenant and the ability to consolidate multiple pluggable databases (PDBs) in a container database (CDB), you can manage many databases that have similar SLAs and planned maintenance requirements with fewer system resources, and more importantly with less operational investment. Leveraging Oracle Multitenant and its CDBs/PDBs technologies with Oracle's resource management, it is an effective means to reduce overall hardware and operational costs.

Planning and sizing are key prerequisites in determining which databases to consolidate in the same CDB. For mission critical databases that require HA and DR protection and minimal downtime for planned maintenance, it's important that you

- Size and leverage resource management to ensure sufficient resources for each PDB to perform within response and throughput expectations
- Target PDB databases that have the same planned maintenance requirements and schedule
- Target PDB databases that can all fail over to same CDB standby in case of unplanned outages such as CDB, cluster, or site failures

Note that Data Guard failover and switchover times can increase as you add more PDBs and their associated application services. A good rule is to have fewer than 25 PDBs per CDB for mission critical "Gold" CDBs with Data Guard if you want to reduce Data Guard switchover and failover timings.

Separating mission critical databases and dev/test databases into different CDBs is important. For example a mission critical "Gold" CDB with a standby may have only 5 PDBs with identical HA/DR requirements and may be sized to have ample system resource headroom while an important CDB with standby can contain 100 PDBs for dev, UAT and application testing purposes and may set up some level of over subscription to reduce costs. Refer to [Overview of Oracle Multitenant Best Practices](#) for more information on Multitenant MAA and Multitenant best practices.

This use case provides an overview and step by step instructions for the exception cases where a complete CDB Data Guard switchover and failover operation is not possible. With PDB failover and switchover steps, you can isolate the Data Guard role transition to one PDB to achieve Recovery Time Objective (RTO) of less 5 minutes and zero or near zero Recovery Point Objective (RPO or data loss).

Starting with Oracle RDBMS 19c (19.15) you can use Data Guard broker command line interface (DGMGRL) to migrate PDBs from one Data Guard configuration to another. Using broker, you can initiate PDB disaster recovery (DR) and switchover operations in isolation without impacting other PDBs in the same CDB.

The following primary use cases are described below for Data Guard broker migration:

- PDB switchover use case - Planned maintenance DR validation which invokes a PDB switchover operation without impacting existing PDBs in a Data Guard CDB
- PDB Failover use case - Unplanned outage DR which invokes a PDB failover without impacting existing PDBs in a Data Guard CDB

① Note

To relocate a single PDB when upgrade is not required without impacting other PDBs in a CDB see [Using PDB Relocation to Move a Single PDB to Another CDB Without Upgrade \(Doc ID 2771737.1\)](#). To relocate a single PDB requiring upgrade without impacting other PDBs in a CDB see .

PDB Switchover Use Case

In this PDB switchover or "DR Test" use case, a PDB is migrated from one Oracle Data Guard protected CDB to another Data Guard protected CDB.

As part of this use case, the files for the PDB on both the primary and standby databases of the source CDB are used directly in the respective primary and standby databases of the destination CDB.

The source CDB contains multiple PDBs, but we perform role transition testing on only one PDB because the others are not able to accept the impact. Before starting the migration, a second CDB must be created and it must have the same database options as the source CDB. The destination CDB is also in a Data Guard configuration, but it contains no PDBs at the start. The two corresponding primary and standby databases share the same storage and no data file movement is performed.

Prerequisites

Make sure your environment meets these prerequisites for the use case.

The Oracle Data Guard broker CLI (DGMGRL) supports maintaining configurations with a single physical standby database.

Using the method described here, for the PDB being migrated (the source), the data files of both the primary and the standby databases physically remain in their existing directory structure at the source and are consumed by the destination CDB and its standby database.

- Oracle patches/versions required
 - Oracle RDBMS 19c (19.15) or later
 - Patch 33358233 installed on the source and destination CDB RDBMS Oracle Homes to provide the broker functionality to manage the switchover process. You don't need to apply the patch on Oracle RDBMS 19c (19.18) and later; it is included.
 - Patch 34904997 installed on the source and destination CDB RDBMS Oracle Homes to provide the functionality to migrate the PDB back to the original configuration after performing the PDB Failover Use Case.
- Configuration
 - `DB_CREATE_FILE_DEST = ASM_Disk_Group`
 - `DB_FILE_NAME_CONVERT=""`
 - `STANDBY_FILE_MANAGEMENT=AUTO`

- The source and destination standby CDBs must run on the same cluster
- The source and destination primary CDBs should run from the same Oracle Home, and the source and destination standby CDBs should run from the same Oracle Home
- The source and destination primary CDBs must run on the same host
- The source and destination primary databases must use the same ASM disk group, and the source and destination standby databases must use the same ASM disk group
- You must have access to the following
 - Password for the destination CDB sysdba user
 - Password for the standby site ASM sysasm user (to manage aliases)
 - Password for the destination CDB Transparent Data Encryption (TDE) keystore if TDE is enabled

Note

PDB snapshot clones and PDB snapshot clone parents are not supported for migration or failover.

For destination primary databases with multiple physical standby databases you must either use the manual steps in [Reusing the Source Standby Database Files When Plugging a PDB into the Primary Database of a Data Guard Configuration \(Doc ID 2273829.1\)](#), or use the `ENABLED_PDBS_ON_STANDBY` initialization parameter in the standby databases, to limit which standby will be managed by this process. See *Creating a Physical Standby of a CDB in Oracle Data Guard Concepts and Administration* for information about using `ENABLED_PDBS_ON_STANDBY`.

Existing ASM aliases for the source PDB migrated are managed by the broker during the migrate process. ASM only allows one alias per file, so existing aliases pointing to a different location must be removed and new ones in the correct location created.

Configuring PDB Switchover

You configure the "DR test" PDB switchover use case in the following steps.

The sample commands included in the steps below use the following CDB and PDB names.

- CDB100 (source CDB)
 - Contains PDB001, PDB002, PDB003; PDB001 will be configured for switchover
- CDB100_STBY (source standby CDB)
- CDB200 (destination CDB)
- CDB200_STBY (destination standby CDB)

Step 1: Extract PDB Clusterware managed services on the source database

Determine any application and end user services created for the source PDB that have been added to CRS.

Because there are certain service attributes such as database role not stored in the database, the detail attributes should be retrieved from CRS using `SRVCTL CONFIG SERVICE`.

1. Retrieve the service names from the primary PDB (PDB001 in our example).

```
PRIMARY_HOST $ sqlplus sys@cdb100 as sysdba
SQL> alter session set container=pdb001;
SQL> select name from dba_services;
```

2. For each service name returned, retrieve the configuration including DATABASE_ROLE.

```
PRIMARY_HOST $ srvctl config service -db cdb100 -s SERVICE_NAME
```

Step 2: Create an empty target database

Create an empty CDB (CDB200 in our example) on the same cluster as the source CDB (CDB100) which will be the destination for the PDB.

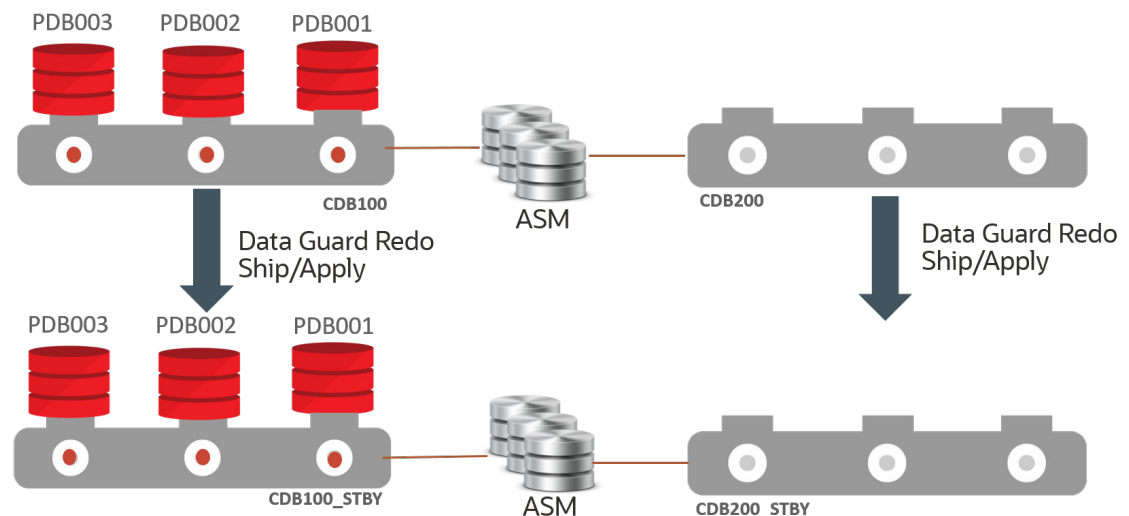
Allocate resources for this CDB to support the use of the PDB for the duration of the testing.

Step 3: Create a target standby database

Enable Oracle Data Guard on the target CDB to create a standby database (CDB200_STBY).

The standby database must reside on the same cluster as the source standby database.

The configuration should resemble the image below.



Step 4: Migrate the PDB

Migrate the PDB (PDB001) from the source CDB (CDB100) to the destination CDB (CDB200).

1. Start a connection to the source primary database using Oracle Data Guard broker command line (DGMGRL).

This session should run on a host that contains instances of both the source primary CDB and the destination primary CDB. The session should be started with a sysdba user.

The broker CLI should be run from the command line on the primary CDB environment and run while connected to the source primary CDB. If you are using a TNS alias to connect to the source primary, it should connect to the source primary instance running on the same host as the broker CLI session.

The host and environment settings when running the broker CLI must have access to SQL*Net aliases for:

- Destination primary CDB – This alias should connect to the destination primary instance that is on the same host as the broker CLI session/source primary database instance to ensure the plug-in operation can access the PDB unplug manifest file that will be created.
- Destination standby CDB, this can connect to any instance in the standby environment.
- Standby site ASM instance, this can connect to any instance in the standby environment.

```
PRIMARY_HOST1 $ dgmgrl sys@cdb100_prim_inst1 as sysdba
```

This session should run on a host that contains instances of both the source primary CDB and the destination primary CDB and connected to a sysdba user. Use specific host/instance combinations instead of SCAN to ensure connections are made to the desired instances.

2. Run the DGMGRL MIGRATE PLUGGABLE DATABASE command.

The STANDBY FILES keyword is required.

See [Full Example Commands with Output](#) for examples with complete output and MIGRATE PLUGGABLE DATABASE for additional information about the command line arguments.

- Sample command example without TDE:

```
DGMGRL> MIGRATE PLUGGABLE DATABASE PDB001 TO CONTAINER CDB200
USING '/tmp/PDB001.xml' CONNECT AS sys/password@cdb200_inst1
STANDBY FILES sys/standby_asm_sys_password@standby_asm_inst1
SOURCE STANDBY CDB100_STBY DESTINATION STANDBY CDB200_STBY ;
```

- Sample command example with TDE

```
DGMGRL> MIGRATE PLUGGABLE DATABASE PDB001 TO CONTAINER CDB200
USING '/tmp/pdb001.xml' CONNECT AS sys/password@cdb200_inst1
SECRET "some_value" KEYSTORE IDENTIFIED BY
"destination_TDE_keystore_passwd"
STANDBY FILES sys/standby_asm_sys_password@standby_asm_inst1
SOURCE STANDBY cdb100_stby DESTINATION STANDBY cdb200_stby;
```

Note

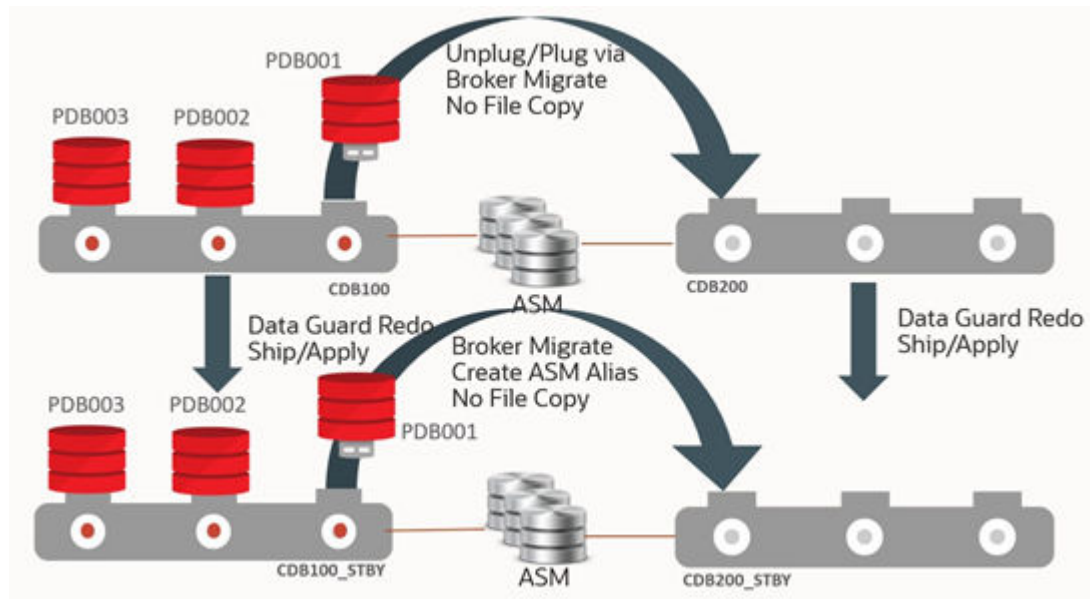
In TDE environments, if SECRET or KEYSTORE are not specified in the command line, the MIGRATE command fails. You can also use PROMPT SECRET, PROMPT KEYSTORE to provide a password using command line input during the migration.

When the command is executed, it:

1. Connects to the destination database and ASM instances to ensure credentials and connect strings are correct
2. Performs a variety of prechecks - If any precheck fails, the command stops processing and returns control to the user, an error is returned, and no changes are made on the target CDB
3. Creates a flashback guaranteed restore point in the destination standby CDB - This requires a short stop and start of redo apply
4. Closes the PDB on the source primary
5. Unplugs the PDB on the source primary - If TDE is in use, the keys are included in the manifest file generated as part of the unplug operation
6. Drops the PDB on the source primary database with the `KEEP DATAFILES` clause, ensuring that the source files are not dropped
7. Waits for the drop PDB redo to be applied to the source standby database - It must wait because the files are still owned by the source standby database until the drop redo is applied

The command waits a maximum of `TIMEOUT` minutes (default 10). If the redo hasn't been applied by then the command fails and you must manually complete the process.
8. Manages the ASM aliases for the PDB files at the standby, removing any existing aliases and creating new aliases as needed - If the standby files already exist in the correct location, all aliases for the standby copy of the PDB are removed
9. Plugs in the PDB into the destination primary CDB - If TDE is in use, the keys are imported into the destination primary keystore as part of the plug-in
10. Ships and applies redo for the plug-in operation to the destination CDB, which uses any created aliases (if necessary) to access the files and incorporate them into the standby database
11. Validates that the standby files are added to the destination standby using redo apply
12. Opens the PDB in the destination primary database
13. Stops redo apply
14. Drops the flashback guaranteed restore point from the destination standby database
15. If TDE is enabled, redo apply remains stopped, if TDE is not enabled, redo apply is restarted

The image below shows the process of PDB001 unplugging from CDB100/CDB100_STBY, and plugging into CDB200/CDB200_STBY.



Step 5: Post Migration - Optional TDE Configuration Step and Restart Apply

If TDE is in use, redo apply will have been stopped by the broker MIGRATE PLUGGABLE DATABASE operation on the destination standby (CDB200_STBY) to allow the new TDE keys to be managed. Copy the keystore for the destination primary (CDB200) to the destination standby keystore and start redo apply.

```
SOURCE_HOST $ scp DESTINATION_PRIMARY_WALLET_LOCATION/*>
DESTINATION_HOST:DESTINATION_STANDBY_WALLET_LOCATION/
```

```
$ dgmgrl sys/password@CDB200 as sysdba
DGMGRL> edit database cdb200_stby set state='APPLY-ON';
```

Step 6: Post Migration - Enable Services

Add any application services for the PDB to Cluster Ready Services (CRS), associating them with the PDB and correct database role in the destination CDB, and remove the corresponding service from the source CDB.

1. For each service on both the primary and standby environments, run the following:

```
PRIMARY_HOST $ srvctl add service -db cdb200 -s SERVICE_NAME
                -pdb pdb001 -role [PRIMARY|PHYSICAL_STANDBY]...
STANDBY_HOST $ srvctl add service -db cdb200_stby -s SERVICE_NAME
                -pdb pdb001 -role [PRIMARY|PHYSICAL_STANDBY]...

PRIMARY_HOST $ srvctl remove service -db cdb100 -s SERVICE_NAME
STANDBY_HOST $ srvctl remove service -db cdb100_stby -s SERVICE_NAME
```

2. Start the required services for the appropriate database role.

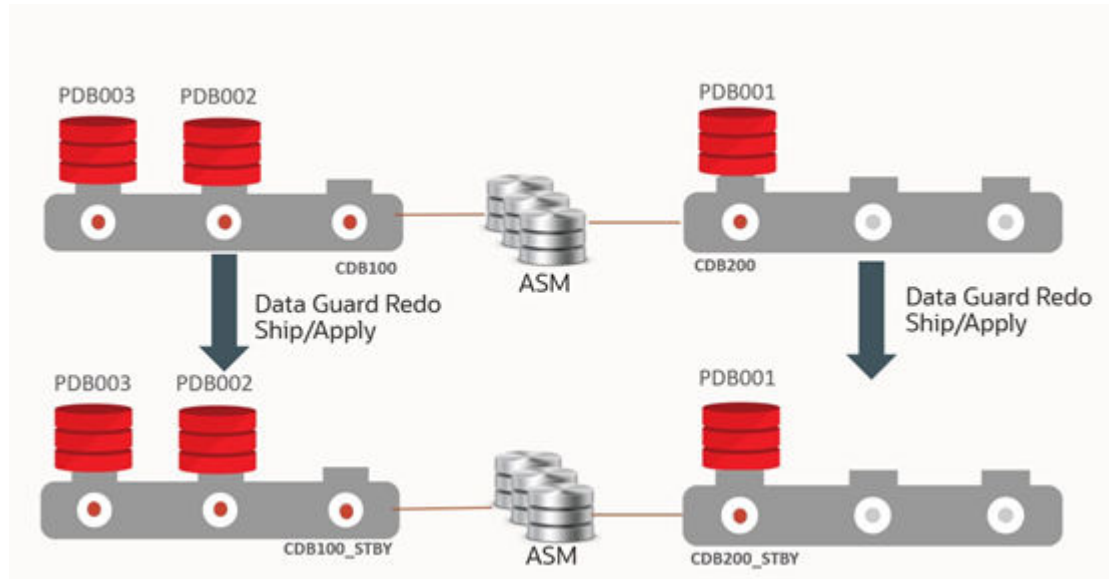
- a. Start each PRIMARY role database service

```
PRIMARY_HOST $ srvctl start service -db cdb200 -s SERVICE_NAME
```


- b. Start each PHYSICAL_STANDBY role database service:

```
STANDBY_HOST $ srvctl start service -db cdb200_stby -s SERVICE_NAME
```

The image below shows the configuration after migration.



Step 7: Perform PDB role transition testing

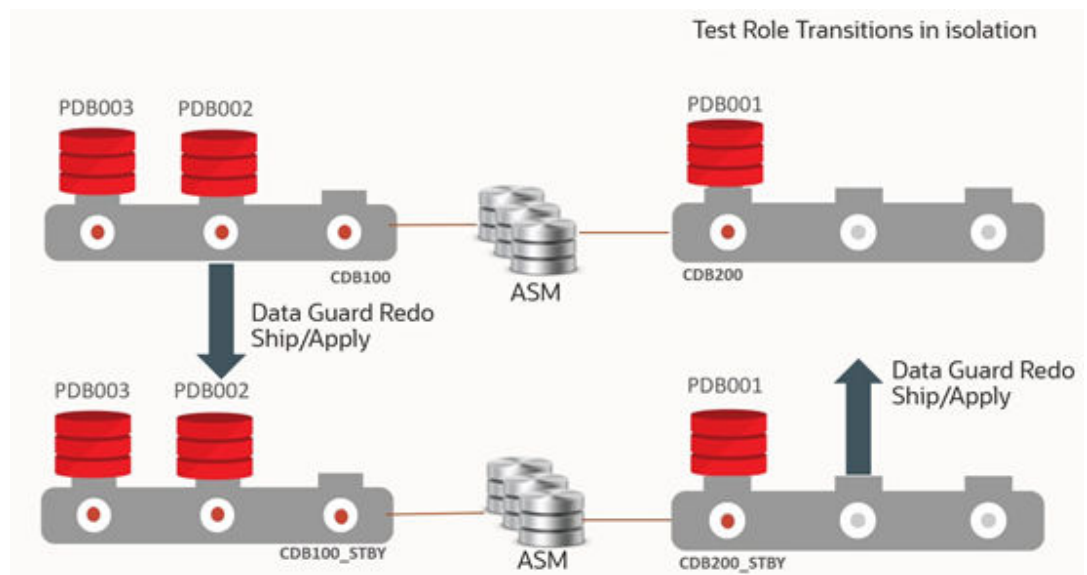
After completing the migration, you can now perform required Oracle Data Guard role transitions or DR test with the PDB in destination CDB (CDB200). No other PDBs in the source CDB (CDB100) are impacted.

In addition, you continue to maintain the Data Guard benefits for both the source and destination CDB, such as DR readiness, Automatic Block Media Recovery for data corruptions, Fast-Start Failover to bound recovery time, Lost Write detection for logical corruptions, offloading reads to the standby to reduce scale and reduce impact of the primary, and so on. etc.

- Connect to the destination CDB using DGMGRL and perform the switchover.

```
$ dgmgrl sys@cdb200 as sysdba
DGMGRL> switchover to CDB200_STBY;
```

The image below shows the configuration after Data Guard switchover of the PDB.



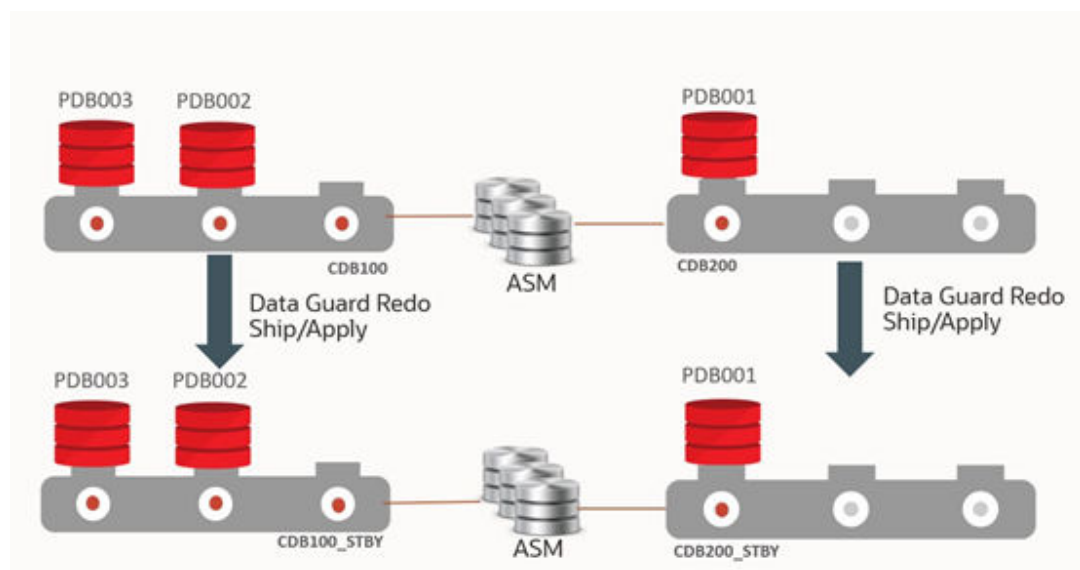
You can continue performing your DR testing for the PDB.

Once the DR testing for the PDB is complete, you can switch back and subsequently migrate the PDB back to the original source CDB.

- Connect to the destination CDB (CDB200) using DGMGRL and perform the switchback operation.

```
$ dgmgrl sys@cdb200 as sysdba
DGMGRL> switchover to CDB200;
```

The image below shows the configuration after the switchback operation.



Step 8: Return the PDB to the original CDB

After migration and role transition testing, switch back to the original configuration for this CDB and migrate the PDB back to the original Data Guard configuration, again automatically

maintaining the standby database files. Data Guard broker migration handles any aliases that need to be dropped or created as part of the migration process.

See [Full Example Commands with Output](#) for examples with complete output.

- Start a connection to the source primary using Data Guard Broker command line (DGMGRL)

```
$ dgmgrl
DGMGRL> connect sys/@cdb200_inst1 as sysdba
```

– Command without TDE

```
DGMGRL> MIGRATE PLUGGABLE DATABASE PDB001 TO CONTAINER CDB100
        USING '/tmp/PDB001_back.xml' CONNECT AS sys/password@cdb100_inst1
        STANDBY FILES sys/standby_asm_sys_password@standby_asm_inst
        SOURCE STANDBY CDB200_STBY DESTINATION STANDBY CDB100_STBY ;
```

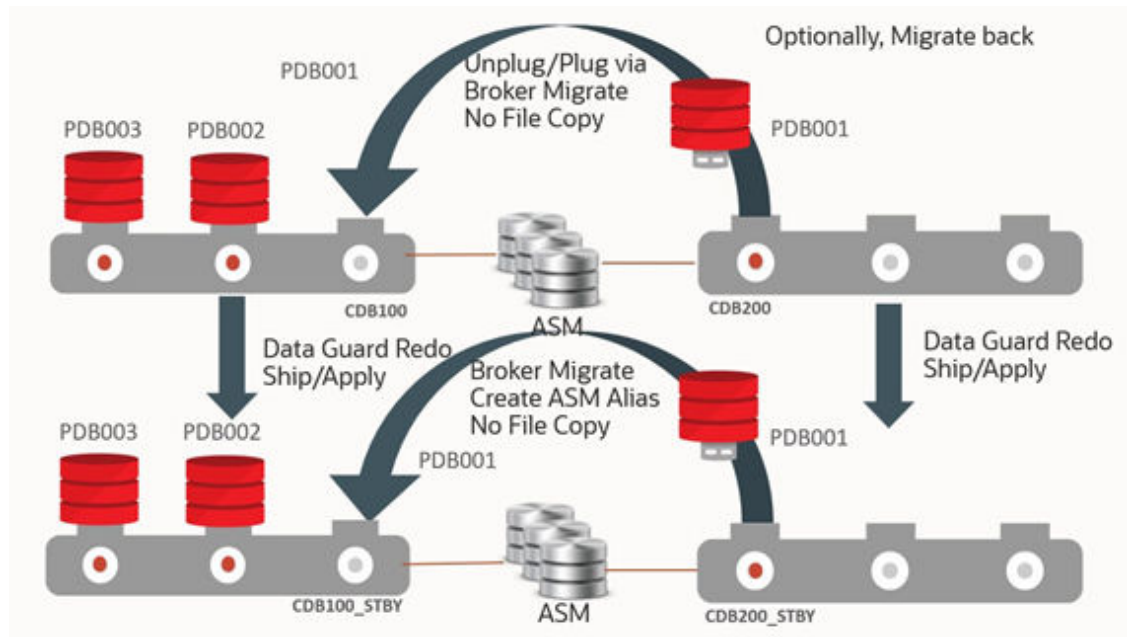
– Command with TDE

```
DGMGRL> MIGRATE PLUGGABLE DATABASE PDB001 TO CONTAINER CDB100
        USING '/tmp/PDB001_back.xml' CONNECT AS sys/password@cdb100_inst1
        SECRET "some_value" KEYSTORE
        IDENTIFIED BY "destination_TDE_keystore_passwd"
        STANDBY FILES sys/standby_asm_sys_password@standby_asm_inst
        SOURCE STANDBY CDB200_STBY DESTINATION STANDBY CDB100_STBY ;
```

Note

In TDE environments, if SECRET or KEYSTORE are not specified in the command line, the MIGRATE command fails. You can also use PROMPT SECRET, PROMPT KEYSTORE to provide a password using command line input during the migration.

The image below shows the configuration during reverse migration.



Step 9: Post Migration - Enable Services

Add any application services for the PDB to Cluster Ready Services (CRS), associating them with the PDB and correct database role in the destination CDB (CDB100), and remove the corresponding service from the source CDB (CDB200).

- For each service on both the primary and standby environments, run the following:

```
PRIMARY_HOST $ srvctl add service -db cdb100 -s SERVICE_NAME
                -pdb pdb001 -role [PRIMARY|PHYSICAL_STANDBY]...
STANDBY_HOST $ srvctl add service -db cdb100_stby -s SERVICE_NAME
                -pdb pdb001 -role [PRIMARY|PHYSICAL_STANDBY]...

<PRIMARY_HOST>PRIMARY_HOST $ srvctl remove service -db cdb200 -s
SERVICE_NAME
STANDBY_HOST $ srvctl remove service -db cdb200_stby -s SERVICE_NAME
```

- Start the required services for the appropriate database role.

1. Start each PRIMARY role database service.

```
PRIMARY_HOST $ srvctl start service -db cdb100 -s SERVICE_NAME
```

2. Start each PHYSICAL_STANDBY role database service.

```
STANDBY_HOST $ srvctl start service -db cdb100_stby -s SERVICE_NAME
```

PDB Failover Use Case

This is a very rare use case since a real disaster that encompasses CDB, cluster, or site failure should always leverage a complete CDB Data Guard failover operation to bound downtime, reduce potential data loss, and reduce administrative steps.

Even with widespread logical or data corruptions or inexplicable database hangs, it's more efficient to issue a CDB Data Guard role transition operation because the source environment may be suspect and root cause analysis may take a long time.

When does a PDB failover operation make sense? PDB failover may be viable if the application is getting fatal errors such as data integrity or corruption errors, or simply is not performing well (not due to system resources). If the source CDB and its corresponding PDBs are still running well, and the standby did not receive any errors for the target sick PDB, then you can fail over just the target sick PDB from the standby without impacting any other PDBs in the source primary CDB.

The process below describes how to set up a PDB failover of a sick PDB that migrates the standby's healthy PDB from the source CDB standby (CDB100_STBY) to an empty destination CDB (CDB200). Before starting the migration, the destination CDB must be created and it must have the same database options as the source standby CDB. The destination CDB will contain no PDBs. The source and destination CDBs share the same storage and no data file movement is performed.

Prerequisites

Make sure your environment meets these prerequisites for the use case.

In addition to the prerequisites listed in the PDB switchover use case, above, the following prerequisites exist for failing over.

- Oracle recommends that you shut down the services on both the primary and the standby that are accessing the PDB before starting the migration process.

If the PDB is not closed on the primary before running the `DGMGRL MIGRATE PLUGGABLE DATABASE` command, an error is returned stating that you will incur data loss. Closing the PDB on the primary resolves this issue. All existing connections to the PDB are terminated as part of the migration.

Assuming a destination CDB is already in place and patched correctly on the standby site, the entire process of moving the PDB can be completed in less than 15 minutes.

Additional Considerations

The following steps assume the source CDB database (either primary for migration or standby for failover) and the destination CDB database have access to the same storage, so copying data files is not required.

- Oracle Active Data Guard is required for the source CDB standby for failover operations.
- Create an empty CDB to be the destination for the PDB on the same cluster as the source CDB.
- Ensure that the TEMP file in the PDB has already been created in the source CDB standby before performing the migration.
- If the destination CDB is a later Oracle release the PDB will be plugged in but left closed to allow for manual upgrade as a post-migration task.
- After processing is completed, you may need to clean up leftover database files from the source databases.
- The plugin operation at the destination CDB is performed with `STANDBYS=NONE`, so you will need to manually enable recovery at any standby databases upon completion of the migration. See [Making Use Deferred PDB Recovery and the STANDBYS=NONE Feature with Oracle Multitenant \(Doc ID 1916648.1\)](#) for steps to enable recovery of a PDB.

Configuring PDB Failover

You configure the DR PDB failover use case in the following steps.

In this use case, the example topology has source primary CDB100 with 3 PDBs (PDB001, PDB002, PDB003). CDB100 also has a Data Guard physical standby (CDB100_STBY).

On the same environment as the standby CDB, we will create a new CDB (CDB200) which is a read-write database that becomes the new host for one of the source PDBs.

Step 1: Extract PDB Clusterware managed services on the source database

Determine any application and end user services created for the source PDB that have been added to CRS.

Because there are certain service attributes such as database role not stored in the database, the detail attributes should be retrieved from CRS using `SRVCTL CONFIG SERVICE`.

1. Retrieve the service names from the primary PDB (PDB002 in our example).

```
PRIMARY_HOST $ sqlplus sys@cdb100 as sysdba
SQL> alter session set container=pdb002;
SQL> select name from dba_services;
```

2. For each service name returned, retrieve the configuration including `DATABASE_ROLE`.

```
PRIMARY_HOST $ srvctl config service -db cdb100 -s SERVICE_NAME
```

Step 2: Create an empty target database

Create an empty CDB (CDB200 in our example) on the same cluster as the source standby CDB (CDB100_STBY) which will be the destination for the PDB (PDB002).

Allocate resources for this CDB to support the use of the PDB while it remains in this CDB.

Step 3: Create an Oracle Data Guard configuration for the empty target database

To allow Data Guard broker to access the new CDB (CDB200), it must be part of a Data Guard configuration. This configuration can consist of only a primary database.

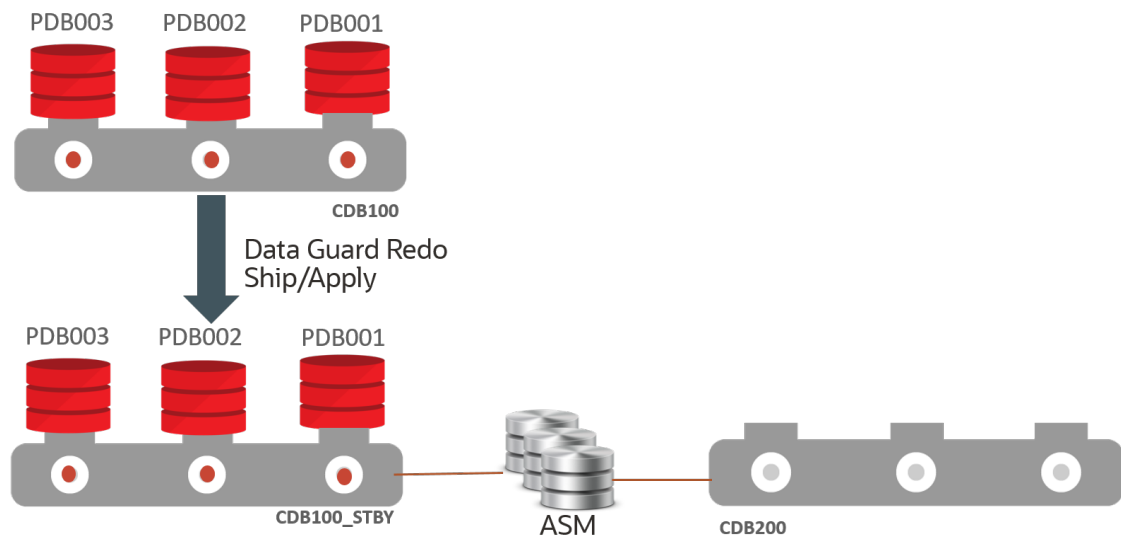
1. Configure the database for broker.

```
STANDBY_HOST $ sqlplus sys@cdb200 as sysdba
SQL> alter system set dg_broker_config_file1='+DATAC1/cdb200/
dg_broker_1.dat';
SQL> alter system set dg_broker_config_file2='+DATAC1/cdb200/
dg_broker_2.dat';
SQL> alter system set dg_broker_start=TRUE;
```

2. Create the configuration and add the database as the primary.

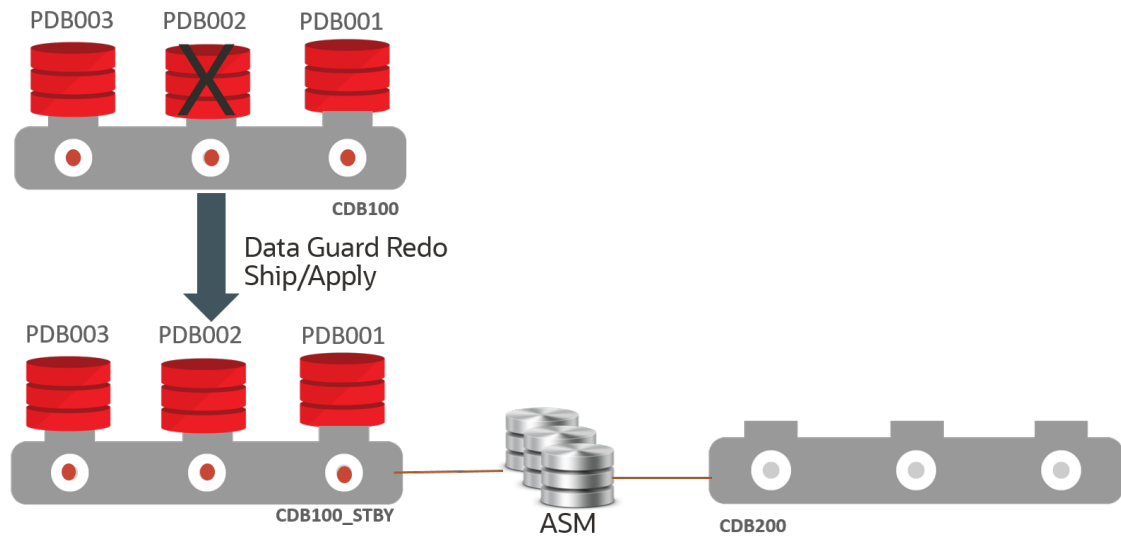
```
STANDBY_HOST $ dgmgrl
DGMGRL> connect sys@cdb200 as sysdba
DGMGRL> create configuration failover_dest as primary database is cdb200
connect identifier is 'cdb200';
DGMGRL> enable configuration;
```

The configuration should resemble the image below.



In this image, the source primary CDB (CDB100) and all PDBs are running normally. the source standby CDB (CDB100_STBY) must run in Active Data Guard mode to allow for the "unplug" operation to succeed without impacting other PDBs. The destination CDB (CDB200) is currently empty.

Assume that one of the source primary PDBs (PDB002) experiences a failure, as shown in the image below, which requires a long recovery period, but the failure does not impact the other PDBs (PDB001 and PDB003), and the standby for the source CDB continues to apply redo without error. The image below shows this outage scenario.



This configuration will use files from PDB002 at the standby site (CDB100_STBY) to plug into the destination CDB (CDB200) to restore read/write application access and then drop the sick PDB (PDB002) from the source primary CDB (CDB100). This will not be a native unplug operation because native unplug requires a read/write CDB and in this scenario we're extracting from the standby.

Step 4: Stop services for the failed PDB

Although not required, stop all services on both the source primary database and any standby database(s) pertaining to the PDB (PDB002) to be migrated.

The following commands stop all services defined in CRS but does not close the PDB.

```
SOURCE_PRIMARY $ srvctl stop service -d CDB100 -pdb PDB002
SOURCE_PRIMARY $ srvctl stop service -d CDB100_STBY -pdb PDB002
```

Step 5: Fail over the PDB from the standby

Fail over the sick PDB (PDB002) from the standby CDB (CDB100_STBY) to the destination CDB (CDB200).

1. Start a DGMGRL session connecting to the source configuration standby database (CDB100_STBY).

You must connect to the source standby database as SYSDBA using something similar to the following:

```
$ dgmgrl
DGMGRL> connect sys@cdb100_stby_inst1 as sysdba
```

2. Run the DGMGRL MIGRATE PLUGGABLE DATABASE command to perform the failover.

Note

The DGMGRL FAILOVER command has a similar format to the MIGRATE PLUGGABLE DATABASE command.

Do not use the STANDBY FILES keyword for the failover operation.

If data loss is detected (SCN in the header of the first SYSTEM tablespace standby data file is less than the corresponding SCN of the file in the primary) and IMMEDIATE has not been specified, the MIGRATE PLUGGABLE DATABASE command will fail. The most common reason is that the PDB in the primary CDB is still open, the PDB on the primary should be closed before attempting a failover.

You must resolve the SCN discrepancy or accept the data loss with the IMMEDIATE clause.

3. Fail over the PDB

See [Full Example Commands with Output](#) for examples with complete output.

The CONNECT alias should connect to the destination primary instance that is on the same host as the broker CLI session/source standby database instance to ensure that the plug-in operation can access the PDB unplug manifest file that will be created.

Note

In the following examples, you will be prompted for the SYSDBA password for the destination CDB (CDB200) when the broker attempts to connect to the CDB200_INST1 instance.

- For non-TDE enabled environments:

```
DGMGRL> migrate pluggable database PDB002 to container CDB200
using '/tmp/PDB002.xml>' connect as sys@"CDB200_INST1";
```

- For TDE enabled environments:

```
DGMGRL> migrate pluggable database PDB002 to container CDB200
using '/tmp/PDB002.xml>' connect as sys@"CDB200_INST1"
secret "some_value"
keystore identified by "destination_keystore_password"
keyfile '/tmp/pdb002_key.dat'
source keystore identified by "source_keystore_password";
```

Note

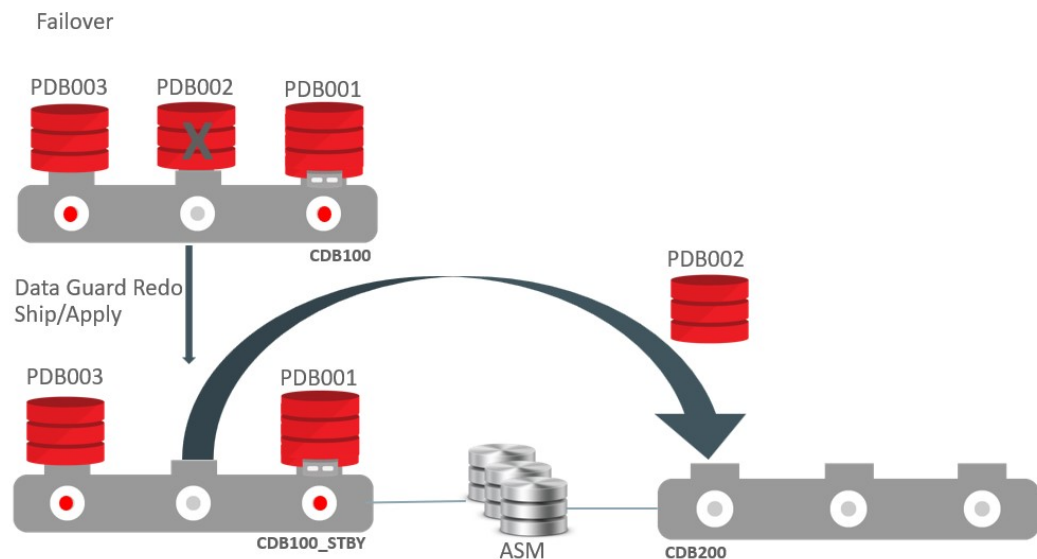
In TDE environments, if SECRET, KEYSTORE, KEYFILE, or SOURCE KEYSTORE are not specified in the command line, the MIGRATE command fails. You can also use PROMPT SECRET, PROMPT KEYSTORE, PROMPT SOURCE KEYSTORE to provide a password using command line input during the migration.

Once the connection to the destination is established the command will:

1. Perform all necessary validations for the failover operation

2. If TDE is enabled, export the TDE keys for the PDB from the source standby keystore
3. Stop redo apply on the source standby if it is running
4. Create the manifest on the standby at the location specified in the command using the `DBMS_PDB.DESCRIBE` command
5. Disable recovery of the PDB at the source standby
6. If TDE is enabled, import TDE keys into the destination CDB keystore to allow the plug-in to succeed
7. Plug in the PDB in the destination database using the standby's data files (`NOCOPY` clause) and with `STANDBYS=NONE`.
8. Open the PDB in all instances of the destination primary database
9. If TDE is enabled, issue `ADMINISTER KEY MANAGEMENT USE KEY` in the context of the PDB to associate the imported key and the PDB.
10. Unplug the PDB from the source primary. If errors occur on unplug messaging is provided to user to perform cleanup manually
11. If unplug succeeds, drop the PDB from the source primary with the `KEEP DATAFILES` clause. This will also drop the PDB in all of the source standby databases.

The image below shows the environment during failover.



Step 6: Post Migration - Enable Services

Add any application services for the PDB to Cluster Ready Services (CRS), associating them with the PDB and correct database role in the destination CDB, and remove the corresponding service from the source CDB.

1. For each service on both the primary and standby environments, run the following:

```
DESTINATION_PRIMARY_HOST $ srvctl add service -db cdb200 -s SERVICE_NAME
                          -pdb pdb002 -role [PRIMARY|PHYSICAL_STANDBY]...
```

```
SOURCE_PRIMARY_HOST $ srvctl remove service -db cdb100 -s SERVICE_NAME
```

```
SOURCE_STANDBY_HOST $ srvctl remove service -db cdb100_stby -s SERVICE_NAME
```

2. Start the required services for the appropriate database role.

Start each PRIMARY role database service

```
DESTINATION_PRIMARY_HOST $ srvctl start service -db cdb200 -s SERVICE_NAME
```

Step 7: Back up the PDB

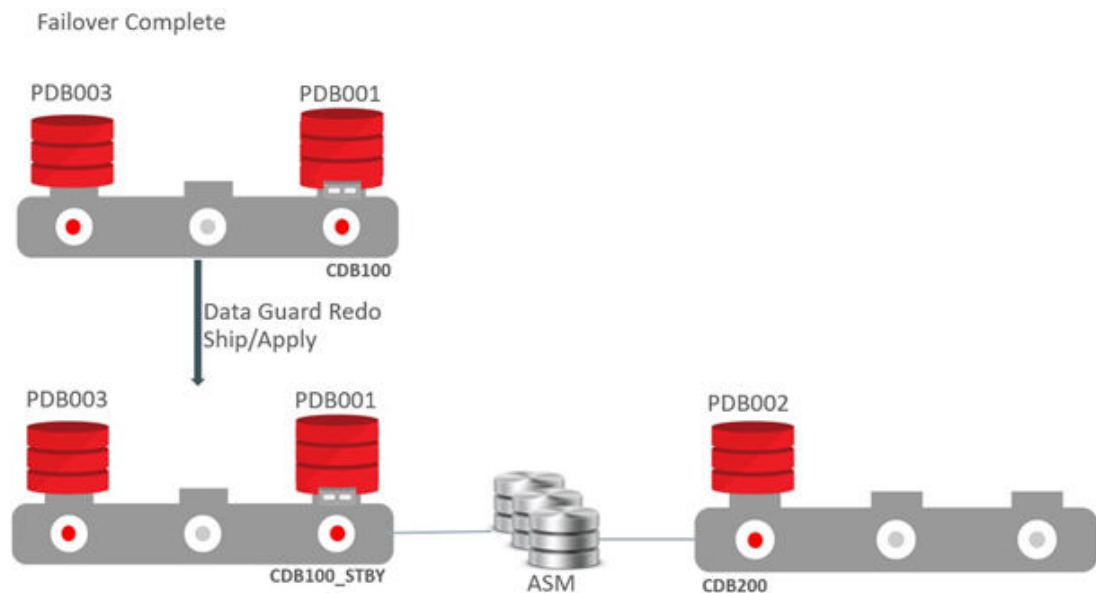
Back up the PDB in the destination CDB (CDB200) to allow for recovery going forward.

```
DESTINATION_PRIMARY_HOST $ rman
RMAN> connect target sys@cdb200
RMAN> backup pluggable database pdb002;
```

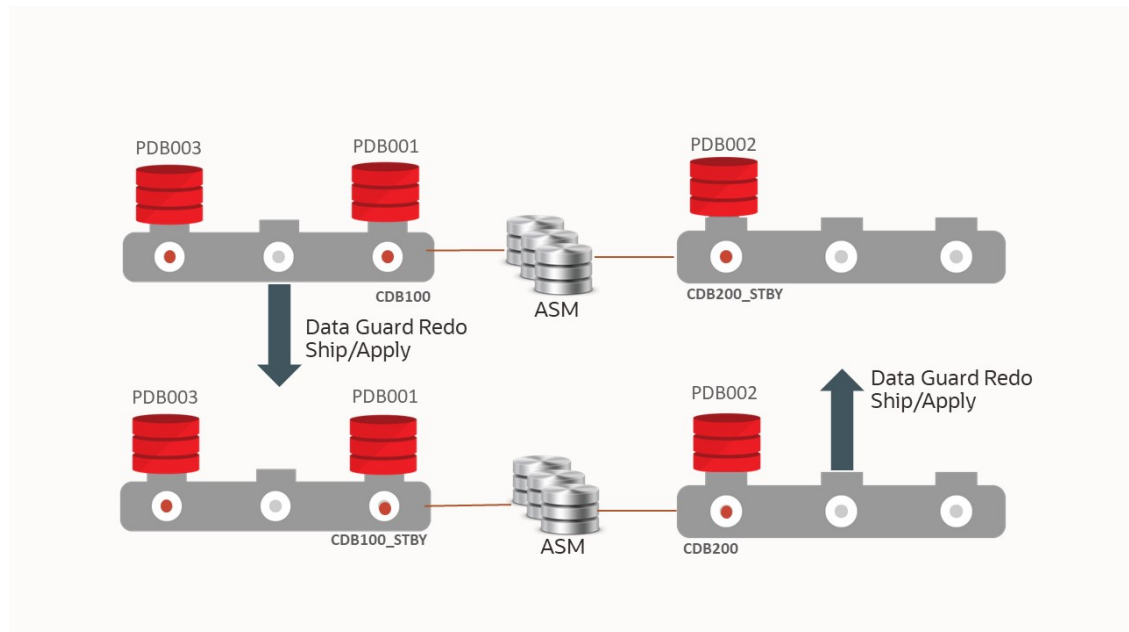
Step 8: Optionally enable recovery of the PDB

Follow the steps in [Making Use Deferred PDB Recovery and the STANDBYS=NONE Feature with Oracle Multitenant \(Doc ID 1916648.1\)](#) to enable recovery of the PDB at any standby databases to establish availability and disaster recovery requirements.

The image below shows the environment after failover.



And this image shows the environment after enabling recovery.



Step 9: Optionally to Migrate Back

See the migration steps in [Configuring PDB Switchover](#).

Reference

Note that the following examples may generate different output as part of the DGMGRL MIGRATE command than you will see while executing the command, based on the different states of PDBs and items found by DGMGRL running prechecks in your environment. In addition, Oracle does not ship message files with bug fixes, so instead of displaying full messages you may receive something similar to the following:

Message 17241 not found; product=rdbms; facility=DGM

This does not mean it's an error or a problem, it means that the text we want to display is missing from the message file. All messages are displayed in their entirety in the first release containing all of the fixes.

Full Example Commands with Output

The following are examples of the commands with output.

Example 43-1 Migrate without TDE

```
DGMGRL> MIGRATE PLUGGABLE DATABASE PDB001 TO CONTAINER CDB200
USING '/tmp/PDB001.xml' CONNECT AS sys/password@cdb200_inst1
STANDBY FILES sys/standby_asm_sys_passwd@standby_asm_inst1
SOURCE STANDBY CDB100_STBY DESTINATION STANDBY CDB200_STBY ;
```

```
Beginning migration of pluggable database PDB001.
Source multitenant container database is CDB100.
Destination multitenant container database is CDB200.
Connecting to "+ASM1".
Connected as SYSASM.
```

```

Stopping Redo Apply services on multitenant container database cdb200_stby.
The guaranteed restore point "<GRP name>" was created for multitenant
container
database "cdb2001_stby".
Restarting redo apply services on multitenant container database cdb200_stby.
Closing pluggable database PDB001 on all instances of multitenant container
database CDB100.
Unplugging pluggable database PDB001 from multitenant container database
cdb100.
Pluggable database description will be written to /tmp/pdb001.xml
Dropping pluggable database PDB001 from multitenant container database CDB100.
Waiting for the pluggable database PDB001 to be dropped from standby
multitenant
container database cdb100_stby.
Creating pluggable database PDB100 on multitenant container database CDB200.
Checking whether standby multitenant container database cdb200_stby has added
all
data files for pluggable database PDB001.
Opening pluggable database PDB001 on all instances of multitenant container
database CDB200.
The guaranteed restore point "<GRP_name>" was dropped for multitenant
container
database "cdb200_stby".
Migration of pluggable database PDB001 completed.

Succeeded.

```

Example 43-2 Migrate with TDE - JSON output

```

DGMGRL> MIGRATE PLUGGABLE DATABASE TBM1_PDB1 TO CONTAINER TB19MIG2
USING '/tmp/dg_tbm1pdb01.xml' CONNECT AS sys/<sys_pass>@TB19MIG2_INST1
SECRET "<secret_string>" KEYSTORE IDENTIFIED BY "<dest keystore password>"
STANDBY FILES sys/<sysasm_passwd>@ASM_STB_INST1
SOURCE STANDBY TB19MIG1_STB DESTINATION STANDBY TB19MIG2_STB;

```

```

{
  "Version" : "19.24.0.0.0",
  "MigratePDB" :
  {
    "StartTimestamp" : "2025-04-09T13:28:24.053+00:00",
    "EndTimestamp" : "2025-04-09T13:29:26.710+00:00",
    "PDBName" : "TBM1_PDB1",
    "VerifyOnly" : false,
    "UseTDE" : true,
    "ReuseStandbyDataFile" : true,
    "Status" : "Success",
    "MigrationSource" :
    {
      "DBUniqueName" : "tb19mig1",
      "ConnectedInstance" : "tb19mig11",
      "Version" : "19.24.0.0.0",
      "Role" : "Primary"
    },
    "MigrationDest" :
    {

```

```

        "DBUniqueName" : "tb19mig2",
        "ConnectedInstance" : "tb19mig21",
        "Version" : "19.24.0.0.0"
    },
    "ProcessMessage" :
    [
        "13:28:24 Connected as SYSDBA.",
        "13:28:24 Master keys of the pluggable database TBM1_PDB1 need to be
migrated.",
        "13:28:24 Keystore of pluggable database TBM1_PDB1 is open.",
        "13:28:24 Beginning migration of pluggable database TBM1_PDB1.",
        "13:28:24 Source multitenant container database is tb19mig1.",
        "13:28:24 Destination multitenant container database is tb19mig2.",
        "13:28:26 Connecting to \"ASM_STB_INST1\".",
        "13:28:26 Connected as SYSASM.",
        "13:28:27 Stopping Redo Apply services on multitenant container
database TB19MIG2_STB.",
        "13:28:33 The guaranteed restore point \"grp_20250409132833_1246692955\"
was created for multitenant container database \"TB19MIG2_STB\".",
        "13:28:33 Restarting redo apply services on multitenant container
database TB19MIG2_STB.",
        "13:28:41 Closing pluggable database TBM1_PDB1 on all instances of
multitenant container database tb19mig1.",
        "13:28:43 Unplugging pluggable database TBM1_PDB1 from multitenant
container database tb19mig1.",
        "13:28:43 Pluggable database description will be written to
/tmp/dg_tbm1pdb01.xml.",
        "13:28:49 Dropping pluggable database TBM1_PDB1 from multitenant
container
database tb19mig1.",
        "13:28:49 Waiting for the pluggable database TBM1_PDB1 to be dropped
from
standby multitenant container database TB19MIG1_STB.",
        "13:29:17 Creating pluggable database TBM1_PDB1 on multitenant container
database tb19mig2.",
        "13:29:19 Checking whether standby multitenant container database
TB19MIG2_STB has added all data files for pluggable database TBM1_PDB1.",
        "13:29:19 Stopping Redo Apply services on multitenant container database
TB19MIG2_STB.",
        "13:29:22 Opening pluggable database TBM1_PDB1 on all instances of
multitenant container database tb19mig2.",
        "13:29:26 The guaranteed restore point \"grp_20250409132833_1246692955\"
was dropped for multitenant container database \"TB19MIG2_STB\".",
        "13:29:26 Please complete the following steps to finish the operation:",
        "13:29:26 Copy keystore located in
/var/opt/oracle/dbaas_acfs/tb19mig2/wallet_root/tde/ for migration
destination
primary database to /var/opt/oracle/dbaas_acfs/tb19mig2/wallet_root/tde/ for
migration destination standby database.",
        "13:29:26 Start DGMGRL, connect to multitenant container database
tb19mig2,
and issue command \"EDIT DATABASE TB19MIG2_STB SET STATE=APPLY-ON\".",
        "13:29:26 If the Oracle Clusterware is configured on multitenant container
database tb19mig2, add all non-default services for the migrated pluggable
database in Cluster Ready Services.",
        "13:29:26 If the Oracle Clusterware is configured on multitenant

```

```

container
  database TB19MIG2_STB, add all non-default services for the migrated
pluggable
  database in Cluster Ready Services.",
    "13:29:26 Migration of pluggable database TBM1_PDB1 completed.",
    "13:29:26 Succeeded."
  ]
}
}

```

Example 43-3 Migrate with TDE

```

DGMGRL> MIGRATE PLUGGABLE DATABASE PDB001 TO CONTAINER CDB200 USING
'/tmp/pdb001.xml'
CONNECT AS sys/password@cdb200_inst1 SECRET "some_value"
KEYSTORE IDENTIFIED BY "destination_TDE_keystore_passwd"
STANDBY FILES sys/standby_ASM_sys_passwd@standby_asm_inst1
SOURCE STANDBY cdb100_stby DESTINATION STANDBY cdb200_stby;

```

Master keys of the pluggable database PDB001 to need to be migrated.
 Keystore of pluggable database PDB001 is open.
 Beginning migration of pluggable database PDB001.
 Source multitenant container database is cdb100.
 Destination multitenant container database is cdb200.
 Connecting to "+ASM1".
 Connected as SYSASM.
 Stopping Redo Apply services on multitenant container database cdb200_stby.
 The guaranteed restore point "... " was created for multitenant container database "cdb200_stby".
 Restarting redo apply services on multitenant container database cdb200_stby.
 Closing pluggable database PDB001 on all instances of multitenant container database cdb100.
 Unplugging pluggable database PDB001 from multitenant container database cdb100.
 Pluggable database description will be written to /tmp/pdb001.xml
 Dropping pluggable database PDBT001 from multitenant container database cdb100.
 Waiting for the pluggable database PDB001 to be dropped from standby multitenant container database cdb100_stby.
 Creating pluggable database PDB1001 on multitenant container database cdb200.
 Checking whether standby multitenant container database cdb200_stby has added all data files for pluggable database PDB001.
 Stopping Redo Apply services on multitenant container database cdb200_stby.
 Opening pluggable database PDB001 on all instances of multitenant container database cdb400.
 The guaranteed restore point "... " was dropped for multitenant container database "cdb200_stby".

Please complete the following steps to finish the operation:

1. Copy keystore located in <cdb200 primary keystore location> for migration destination primary database to <cdb200 standby keystore location> for migration destination standby database.
2. Start DGMGRL, connect to multitenant container database cdb200_stby, and

issue command "EDIT DATABASE cdb200_stby SET STATE=APPLY-ON".

3. If the clusterware is configured on multitenant container databases cdb200 or cdb200_stby, add all non-default services for the migrated pluggable database in cluster ready services.

Migration of pluggable database PDB001 completed.

Succeeded.

Example 43-4 Failover without TDE

```
DGMGRL> migrate pluggable database PDB002 immediate to container CDB200
using '/tmp/<pdb002.xml>';
Username: USERNAME@cdb200
Password:
Connected to "cdb200"
Connected as SYSDBA.
```

Beginning migration of pluggable database pdb002.
Source multitenant container database is cdb100_stby.
Destination multitenant container database is cdb200.

```
Connected to "cdb100"
Closing pluggable database pdb002 on all instances of multitenant container
database cdb100.
Continuing with migration of pluggable database pdb002 to multitenant
container
database cdb200.
Stopping Redo Apply services on source multitenant container database
cdb100_stby.
Succeeded.
Pluggable database description will be written to /tmp/pdb002.xml.
Closing pluggable database pdb002 on all instances of multitenant container
database cdb100_stby.
Disabling media recovery for pluggable database pdb002.
Restarting redo apply services on source multitenant container database
cdb100_stby.
Succeeded.
Creating pluggable database pdb002 on multitenant container database cdb200.
Opening pluggable database pdb002 on all instances of multitenant container
database cdb200.
Unplugging pluggable database pdb002 from multitenant container database
cdb100.
Pluggable database description will be written to /tmp/pdb002_temp.xml.
Dropping pluggable database pdb002 from multitenant container database cdb100.
Unresolved plug in violations found while migrating pluggable database pdb002
to multitenant container database cdb200.
Please examine the PDB_PLUG_IN_VIOLATIONS view to see the violations that
need to be
resolved.
Migration of pluggable database pdb002 completed.
Succeeded.
```

Example 43-5 Failover with TDE

NOTE: ORA-46655 errors in the output can be ignored.

```
DGMGRL> migrate pluggable database PDB002 to container CDB200
using '/tmp/PDB002.xml>' connect as sys@"CDB200" secret "some_value"
keystore identified by "destination_keystore_password"
keyfile '/tmp/pdb002_key.dat'
source keystore identified by "source_keystore_password";
Connected to "cdb200"
Connected as SYSDBA.
Master keys of the pluggable database PDB002 need to be migrated.
Keystore of pluggable database PDB002 is open.

Beginning migration of pluggable database PDB002.
Source multitenant container database is adg.
Destination multitenant container database is cdb200.

Connected to "cdb1001"
Exporting master keys of pluggable database PDB002.
Continuing with migration of pluggable database PDB002 to multitenant
container
database cdb200.
Stopping Redo Apply services on multitenant container database adg.
Pluggable database description will be written to /tmp/PDB002.xml.
Closing pluggable database PDB002 on all instances of multitenant container
database adg.
Disabling media recovery for pluggable database PDB002.
Restarting redo apply services on multitenant container database adg.
Unplugging pluggable database PDB002 from multitenant container database
cdb100.
Pluggable database description will be written to /tmp/ora_tfilSxnmva.xml.
Dropping pluggable database PDB002 from multitenant container database cdb100.
Importing master keys of pluggable database PDB002 to multitenant container
database cdb200.
Creating pluggable database PDB002 on multitenant container database cdb200.
Opening pluggable database PDB002 on all instances of multitenant container
database cdb200.
ORA-46655: no valid keys in the file from which keys are to be imported

Closing pluggable database PDB002 on all instances of multitenant container
database cdb200.
Opening pluggable database PDB002 on all instances of multitenant container
database cdb200.

Please complete the following steps to finish the operation:
If the Oracle Clusterware is configured on multitenant container database
CDB200,
add all non-default services for the migtded pluggable database in Cluster
Ready
Services.

Migration of pluggable database PDB002 completed.
Succeeded.
```


Keyword Definitions

The DGMGRL MIGRATE command keywords are explained below.

Syntax

```
DGMGRL> MIGRATE PLUGGABLE DATABASE [VERIFY] [IMMEDIATE] <pdb-name>
TO CONTAINER <destination-db-unique-name>
USING <XML-description-file>
[ CONNECT AS [<destination-username>]/[<destination-password>]
  @<destination-connect-identifier> ]
[ STANDBY FILES [<asm-instance-username>]/[<asm-instance-password>]
  @<asm-instance-connect-identifier> ]
[ SOURCE STANDBY <source-standby-db-unique-name> ]
[ DESTINATION STANDBY <destination-standby-db-unique-name> ]
[ TIMEOUT <seconds> ]
[ { PROMPT SECRET } | SECRET <secret> } ]
[ KEYFILE <keys-file> ]
[ { PROMPT KEYSTORE } |
  { KEYSTORE IDENTIFIED BY { EXTERNAL STORE | <keystore-password> } } ]
[ { PROMPT SOURCE KEYSTORE } |
  { SOURCE KEYSTORE IDENTIFIED BY { EXTERNAL STORE |
    <source-keystore-password> } } ];
```

These are the keyword definitions used on the MIGRATE PLUGGABLE DATABASE command

- **pdb-name** - The name of the PDB to be migrated.
- **destination-db-unique-name** - The database unique name of the CDB to receive the PDB to be migrated.
- **XML-description-file** - An XML file that contains the description of the PDB to be migrated. This file is automatically created by the SQL statements executed by the MIGRATE PLUGGABLE DATABASE command, and the location of the file must be directly accessible by both the source and destination primary database instances. The file cannot exist before command execution.
- **destination-username** - The user name of the user that has SYSDBA access to the destination CDB.
- **destination-password** - The password associated with the user name specified for dest-cdb-user.
- **destination-connect-identifier** - An Oracle Net connect identifier used to reach the destination CDB.
- **asm-instance-username** - A user having SYSASM privilege for the ASM instance.
- **asm-instance-password** - The password for sysasm-user.
- **asm-instance-connect-identifier** - The connect identifier to the ASM instance that has the source standby database file.
- **source-standby-db-unique-name** - DB_UNIUE_NAME of the migration source CDB's standby database.
- **destination-standby-db-unique-name** - DB_UNIUE_NAME of the migration destination CDB's standby database.

- **timeout** - The timeout value in seconds when waiting for the destination standby database to pick up the data files during migration. This is optional, the default is 5 minutes if the **TIMEOUT** clause is omitted.
- **secret** - A word used to encrypt the export file containing the exported encryption keys of the source PDB. This clause is only required for TDE enabled environments.
- **keys-file** - A data file that contains the exported encryption keys for the source PDB. This file is created by SQL statements executed by the **MIGRATE PLUGGABLE DATABASE** command in the failover use case, and the location of the file must be directly accessible by the source standby instance and the destination primary instance.
- **keystore-password** - The password of the destination CDB keystore containing the encryption keys. This is required if the source PDB was encrypted using a password keystore in TDE enabled environments.
- **source-keystore-password** - The password of the source CDB keystore containing the encryption keys.
- **PROMPT** - is an option to provide a password using command line input during the migration, rather than use them in plain text in command line input

Sample Oracle Database Net Services Connect Aliases

The following Net Services connect aliases must be accessible to DGMGRL when starting the broker session. This can be through default `tnsnames.ora` location or by setting `TNS_ADMIN` in the environment before starting DGMGRL.

PDB Switchover

The host names in the following examples reference Oracle Single Client Access Name (SCAN) host names. There is overlap in the host names between the source and destination databases as they must reside on the same hosts. In all cases the connect strings should connect to the `cdb$root` of the database.

Source primary database

```
CDB100 =
  (DESCRIPTION =
    (CONNECT_TIMEOUT=120)(TRANSPORT_CONNECT_TIMEOUT=90)(RETRY_COUNT=3)
    (ADDRESS =
      (PROTOCOL = TCP)
      (HOST = <source-primary-scan-name>)
      (PORT = <source-primary-listener-port>)
    )
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = <source-primary-service-name>)
      (FAILOVER_MODE =
        (TYPE = select)
        (METHOD = basic)
      )
    )
  )
```

Source primary database local instance

```

CDB100_INST1 =
  (DESCRIPTION =
    (CONNECT_TIMEOUT=120)(TRANSPORT_CONNECT_TIMEOUT=90)(RETRY_COUNT=3)
    (ADDRESS =
      (PROTOCOL = TCP)
      (HOST = <source-primary-scan-name>)
      (PORT = <source-primary-listener-port>)
    )
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = <source-primary-cdb$root-service-name>)
      (INSTANCE_NAME = <source-primary-local-instance-name>)
    )
  )

```

Destination primary database

```

CDB200=
  (DESCRIPTION=
    (CONNECT_TIMEOUT=120)(TRANSPORT_CONNECT_TIMEOUT=90)(RETRY_COUNT=3)
    (ADDRESS=
      (PROTOCOL= TCP)
      (HOST= <source-primary-scan-name>)
      (PORT= <source-primary-listener-port>))
    (CONNECT_DATA=
      (SERVER= DEDICATED)
      (SERVICE_NAME= <destination-primary-cdb$root-service-name>)))

```

Destination primary local instance

This must connect to an instance on the same host that dgmgrl is being executed

```

CDB200_INST1=
  (DESCRIPTION=
    (CONNECT_TIMEOUT=120)(TRANSPORT_CONNECT_TIMEOUT=90)(RETRY_COUNT=3)
    (ADDRESS=
      (PROTOCOL= TCP)
      (HOST= <source-primary-scan-name>)
      (PORT= <source-primary-listener-port>))
    (CONNECT_DATA=
      (SERVER= DEDICATED)
      (SERVICE_NAME= <destination-primary-cdb$root-service-name>)
      (INSTANCE_NAME = <destination-primary-local-instance-name>)
    )
  )

```

Source standby database

```

CDB100_STBY =
  (DESCRIPTION =
    (CONNECT_TIMEOUT=120)(TRANSPORT_CONNECT_TIMEOUT=90)(RETRY_COUNT=3)
    (ADDRESS =

```

```

        (PROTOCOL = TCP)
        (HOST = <source-standby-scan-name>)
        (PORT = <source-standby-listener-port>)
    )
    (CONNECT_DATA =
        (SERVER = DEDICATED)
        (SERVICE_NAME = <source-standby-cdb$root-service-name>)
        (FAILOVER_MODE =
            (TYPE = select)
            (METHOD = basic)
        )
    )
)

```

Destination standby database

```

CDB200_STBY =
    (DESCRIPTION =
        (CONNECT_TIMEOUT=120)(TRANSPORT_CONNECT_TIMEOUT=90)(RETRY_COUNT=3)
        (ADDRESS =
            (PROTOCOL = TCP)
            (HOST = <source-standby-scan-name>)
            (PORT = <source-standby-listener-port>)
        )
        (CONNECT_DATA =
            (SERVER = DEDICATED)
            (SERVICE_NAME = <destination-standby=cdb$root-service-name>)
            (FAILOVER_MODE =
                (TYPE = select)
                (METHOD = basic)
            )
        )
    )
)

```

Standby environment ASM

This must connect to an ASM instance running on the same host as one instance each of the source standby and destination standby

```

STANDBY_ASM_INST1=
    (DESCRIPTION=
        (CONNECT_TIMEOUT=120)(TRANSPORT_CONNECT_TIMEOUT=90)(RETRY_COUNT=3)
        (ADDRESS=
            (PROTOCOL= TCP)
            (HOST = <source-standby-scan-name>)
            (PORT= <source-standby-listener-port>))
        (CONNECT_DATA=
            (SERVER= DEDICATED)
            (SERVICE_NAME= +ASM)
            (INSTANCE_NAME=<ASM_instance_name>)
        )
    )
)

```

PDB Failover**Source primary database**

```

CDB100 =
  (DESCRIPTION =
    (CONNECT_TIMEOUT=120)(TRANSPORT_CONNECT_TIMEOUT=90)(RETRY_COUNT=3)
    (ADDRESS =
      (PROTOCOL = TCP)
      (HOST = <source-primary-scan-name>)
      (PORT = <source-primary-listener-port>)
    )
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = <source-primary-cdb$root-service-name>)
      (FAILOVER_MODE =
        (TYPE = select)
        (METHOD = basic)
      )
    )
  )

```

Source standby database

```

CDB100_STBY =
  (DESCRIPTION =
    (CONNECT_TIMEOUT=120)(TRANSPORT_CONNECT_TIMEOUT=90)(RETRY_COUNT=3)
    (ADDRESS =
      (PROTOCOL = TCP)
      (HOST = <source-standby-scan-name>)
      (PORT = <source-standby-listener-port>)
    )
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = <source-standby-cdb$root-service-name>)
      (FAILOVER_MODE =
        (TYPE = select)
        (METHOD = basic)
      )
    )
  )

```

Source standby database local instance

This must connect to an instance on the same host that dgmgrl is being executed

```

CDB100_STBY_INST1=
  (DESCRIPTION=
    (CONNECT_TIMEOUT=120)(TRANSPORT_CONNECT_TIMEOUT=90)(RETRY_COUNT=3)
    (ADDRESS=
      (PROTOCOL= TCP)
      (HOST= <source-standby-scan-name>)
    )
  )

```

```

        (PORT= <source-standby-listener-port>))
    (CONNECT_DATA=
        (SERVER= DEDICATED)
        (SERVICE_NAME= <source-standby-cdb$root-service-name>)
        (INSTANCE_NAME = <source-standby-local-instance-name>)
    )
)

```

Destination primary database

```

CDB200=
    (DESCRIPTION=
        (ADDRESS=
            (PROTOCOL= TCP)
            (HOST= <source-standby-scan-name>)
            (PORT= <source-standby-listener-port>))
        (CONNECT_DATA=
            (SERVER= DEDICATED)
            (SERVICE_NAME= <destination-primary-cdb$root-service-name>)))

```

Destination primary local instance

This must connect to an instance on the same host that dgmgrl is being executed

```

CDB200_INST1=
    (DESCRIPTION=
        (ADDRESS=
            (PROTOCOL= TCP)
            (HOST= <source-standby-scan-name>)
            (PORT= <source-standby-listener-port>))
        (CONNECT_DATA=
            (SERVER= DEDICATED)
            (SERVICE_NAME= <destination-primary-cdb$root-service-name>)
            (INSTANCE_NAME = <destination-primary-local-instance-name>)
        )
    )
)

```

Resolving Errors

For cases where the plugin to the destination primary CDB succeeds but there are issues such as file not found at the destination standby, you can use the GRP created on the destination CDB standby database to help in resolution.

If the broker detects an error at the standby it ends execution without removing the GRP, it can be used to help resolve errors. The GRP name is displayed in the output from the CLI command execution.

Before using this method, ensure that all patches from the prerequisites section have been applied.

1. Turn off redo apply in Data Guard Broker so it does not automatically start

```
DGMGRL> edit database CDB200_STBY set state='APPLY-OFF';
```

2. Restart the destination CDB standby in mount mode, ensuring in RAC environments only one instance is running.
 - For Oracle RAC


```
$ srvctl stop database -d cdb200_stby -o immediate
```

```
$ srvctl start instance -d cdb200_stby -i cdb200s1 -o mount
```
 - For SIDB


```
SQL> shutdown immediate
```

```
SQL> startup mount
```
3. Connect to the PDB in the destination CDB standby database and disable recovery of the PDB.


```
SQL> alter session set container=pdb001;
```

```
SQL> alter pluggable database disable recovery;
```
4. Connect to the CDB\$root of the destination CDB standby database and flashback the standby database.


```
SQL> alter session set container=cdb$root;
```

```
SQL> flashback database to restore point <GRP from execution>;
```
5. Repair any issues that caused redo apply to fail (e.g. missing ASM aliases).
6. Staying in mount mode on the CDB standby, start redo apply.


```
SQL> recover managed standby database disconnect;
```

Redo apply will now start applying all redo from the GRP forward, including rescanning for all the files for the newly plugged in PDB. The flashback GRP rolls back the destination CDB standby to the point where the PDB is unknown to the standby, so the disabling of recovery for the PDB is backed out as well.

Steps 1-6 can be repeated as many times as is required until all files are added to the standby and additional redo is being applied at which point you would:

1. Stop recovery


```
DGMGRL> edit database CDB200_STBY set state='APPLY-OFF';
```
2. Connect to the CDB\$root of the destination CDB standby database and drop the GRP from the destination standby database:


```
SQL> drop restore point <GRP from execution>;
```
3. Restart redo apply


```
DGMGRL> edit database CDB200_STBY set state='APPLY-ON';
```

If you continue to have issues and require that your CDB standby database maintain protection of additional PDBs in the standby during problem resolution:

- Disable recovery of the PDB as noted above
- Restart redo apply so that the other PDBs in the CDB standby are protected
- Follow the Enable Recovery steps in [Making Use Deferred PDB Recovery and the STANDBYS=NONE Feature with Oracle Multitenant \(Doc ID 1916648.1\)](#) to enable recovery of the failed PDB.
- Drop the GRP from the destination CDB standby.

During testing if there are repetitive errors on the standby that cannot be resolved:

1. Enable PDB operation debugging for redo apply on the standby.

```
SQL> alter system set "_pluggable_database_debug"=256 comment='set to help
debug PDB plugin issues for PDB100, reset when done' scope=both;
```

2. Follow the steps above to flashback the destination CDB standby database.
3. Restart redo apply.

After the new failure, gather the redo apply trace files from the standby host that was running redo apply (..../trace/<SID>_pr*.trc) and open a bug.

Once debugging is done:

1. Reset the parameter to turn off debugging.

```
SQL> alter system reset "_pluggable_database_debug" scope=spfile;
```

2. Bounce the CDB standby database.

Part X

Full Site Switch in Oracle Cloud or On-Premises

- [Full Site Switch in Oracle Cloud or On-Premise](#)

Full Site Switch in Oracle Cloud or On-Premise

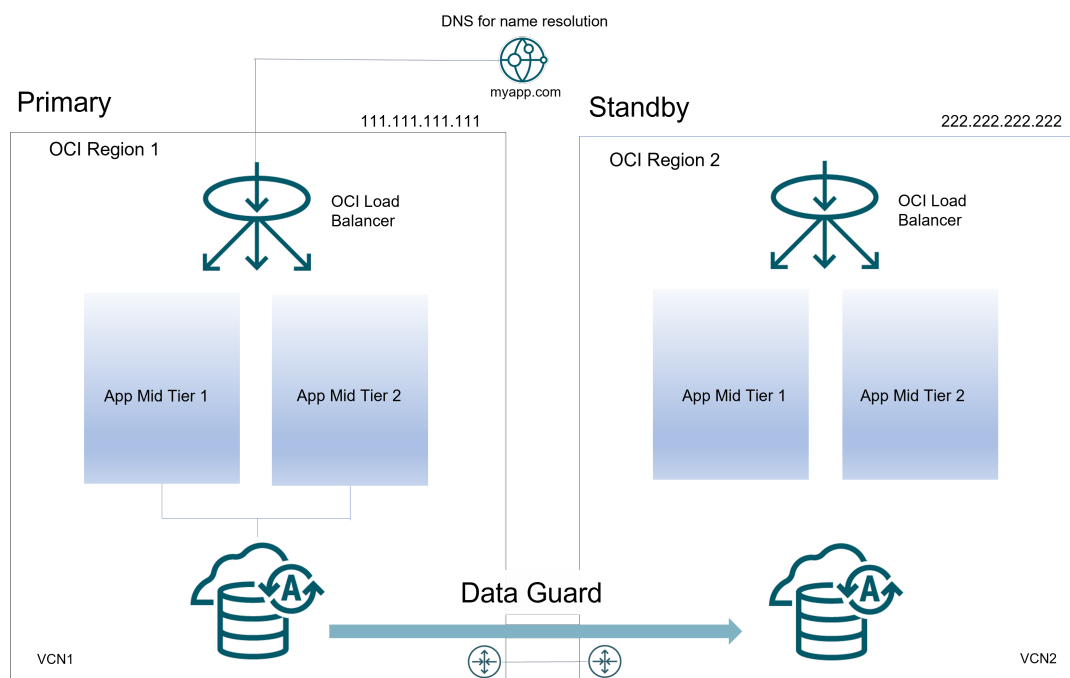
A complete-site or full site failure results in both the application and database tiers being unavailable. To maintain availability users must be redirected to a secondary site that hosts a redundant application tier and a synchronized copy of the production database. MAA best practice is to use Data Guard to maintain the synchronized copy of the production database. Upon site failure a WAN traffic manager or load balancer is used to perform a DNS failover (either manually or automatically) to redirect all users to the application tier at standby site while a Data Guard failover transitions the standby database to the primary production role.

During normal runtime operations the following occurs:

1. Client requests enter the client tier of the primary site and travel by the WAN traffic manager.
2. Client requests are sent to the application server tier.
3. Requests are forwarded through the active load balancer to the application servers.
4. Requests are sent into the database server tier.
5. The application requests, if required, are routed to an Oracle RAC instance.
6. Responses are sent back to the application and clients by a similar path.

The following illustrates the possible network routes before site switchover:

Figure 44-1 Sites before switchover

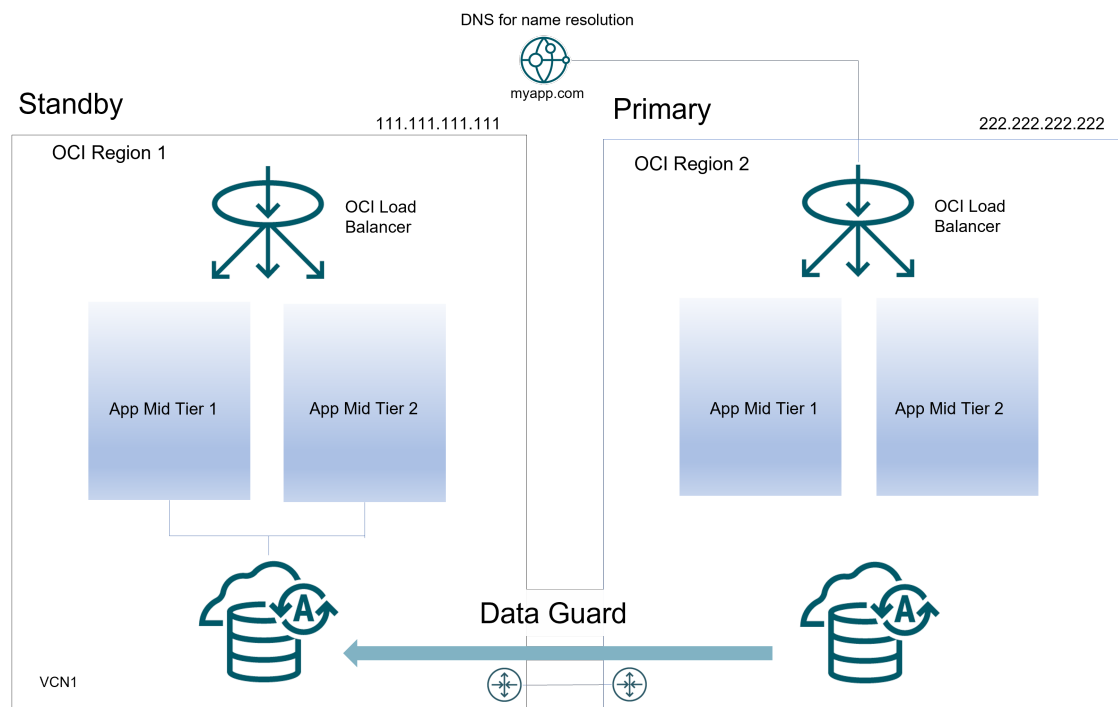


The following steps describe the effect of a site switchover:

1. The administrator has failed over or switched over the primary database to the secondary site. This is automatic if you are using Data Guard Fast-Start Failover. Autonomous Database on Dedicated Hardware supports Data Guard Fast-Start Failover.
2. The administrator starts the middle-tier application servers on the secondary site, if they are not running. In some cases the same middle-tier application servers can be leveraged if they do not reside in the failed site.
3. The wide-area traffic manager selection of the secondary site can be automatic for an entire site failure.
4. The wide-area traffic manager at the secondary site returns the virtual IP address of a load balancer at the secondary site and clients are directed automatically on the subsequent reconnect. In this scenario, the site failover is accomplished by an automatic domain name system (DNS) failover.

The following figure illustrates the network routes after site failover. Client or application requests enter the secondary site at the client tier and follow the same path on the secondary site that they followed on the primary site.

Figure 44-2 Sites after switchover



Failover also depends on the client's web browser. Most browser applications cache the DNS entry for a period. Consequently, sessions in progress during an outage might not fail over until the cache timeout expires. To resume service to such clients, close the browser and restart it.

Performing Role Transitions Between Regions

Examples below leverage Oracle Public Cloud. However similar steps can be done on-premise or hybrid cloud scenarios.

Failover to Another Region

A failover operation is performed when the primary site becomes unavailable, and it is commonly an unplanned operation. You can role-transition a standby database to a primary database when the original primary database fails and there is no possibility of recovering the primary database in a timely manner. There may or may not be data loss depending upon whether your primary and target standby databases were consistent at the time of the primary database failure.

To perform a manual failover in a DR configuration follow these steps:

1. Switchover DNS name.

Perform the required DNS push in the DNS server hosting the names used by the system or alter the file host resolution in clients to point the front-end address of the system to the public IP used by load balancer in site2. For scenarios where DNS is used for the external front-end resolution (OCI DNS, commercial DNS, etc.), appropriate API can be used to push the change. An example that push this change in an OCI DNS:

The following is an OCI client script that updates a front end DNS entry, such as `ordscsdroci.domainexample.com`, to the site 1 load balancer's public IP address (for example: `111.111.111.123`).

```
oci dns record rrset update
--config-file /home/opc/scripts/.oci_ordscsdr/config
--zone-name-or-id "domainexample.com"
--domain "ordscsdroci.domainexample.com"
--rtype "A"
--items
'["domain":"ordscsdroci.domainexample.com","rdata":"111.111.111.123","rtype":"A","ttl":60]'
```

2. Failover database.

On Oracle Cloud:

Use Oracle Control Plane and issue a Data Guard switchover or failover operation.

On-Premises:

Use Data Guard broker in secondary database host to perform the failover. As user oracle:

```
[oracle@drdbwlp1b ~]$ dgmgrl sys/your_sys_password@secondary_db_unqname
DGMGR> failover to "secondary_db_unqname"
```

3. Start the servers in the secondary site.

Restart the secondary application servers.

Switchover

A switchover is a planned operation where an administrator reverts the roles of the two sites. The roles change from the primary to the standby as well as from standby to primary. This is known as a manual switchover. To perform a manual switchover follow these steps:

1. Propagate any pending configuration changes.

For non-database files, you can use `rsync` or Object Storage Service (OSS) to replicate to your secondary site.

2. Stop servers in the primary site.

Use scripts to stop managed servers / mid tiers in primary Site.

3. Switchover DNS name

Perform the required DNS push in the DNS server hosting the names used by the system or alter the file host resolution in clients to point the front-end address of the system to the public IP used by load balancer in site 2. For scenarios where DNS is used for the external front-end resolution (OCI DNS, commercial DNS, etc.), appropriate API can be used to push the change.

The following example pushes this change in an OCI DNS.

The OCI client script updates the front end DNS entry, for example `ordscsdroci.domainexample.com`, to the site1 load balancer's public IP address (for example: `111.111.111.123`).

```
oci dns record rrset update
--config-file /home/opc/scripts/.oci_ordscsdr/config
--zone-name-or-id "domainexample.com"
--domain "ordscsdroci.domainexample.com"
--rtype "A"
--items
'["domain":"ordscsdroci.domainexample.com","rdata":"111.111.111.123","rtype":"A","ttl":60]'
```

Note that the TTL value of the DNS entry will affect to the effective RTO of the switchover: if the TTL is high (example, 20 mins), the DNS change will take that time to be effective in the clients. Using lower TTL values will make this to be faster, however, this can cause an overhead because the clients check the DNS more frequently. A good approach is to set the TTL to a low value temporarily (example, 1 min), before the change in the DNS. Then, perform the change, and once the switchover procedure is completed, set the TTL to the normal value again.

4. Perform database switchover.

On Oracle Cloud:

Use Oracle Control Plane and issue a Data Guard switchover operation.

On-Premises:

Use Data Guard broker on the primary database host to perform the switchover.

As user oracle:

```
$ dgmgrl sys/your_sys_password@primary_db_unqname
DGMGRL> switchover to "secondary_db_unqname"
```

5. Start the servers in secondary site (new primary).

Restart the secondary managed servers and mid tiers.

Best Practices for Full Site Switchover

Oracle recommends the following best practices:

- Maintain the same configuration in primary and standby sites: any changes applied to the primary system must be performed in the secondary system too, so both primary and

secondary systems have the same configuration. For example: a modification in the primary load balancer, any modifications to the operating system, and so on.

- Perform regular switchovers to verify the health of the secondary site.
- Perform any switchover related activity that does not require downtime before you stop the primary servers. For example, the WLS configuration replication based on `config_replica.sh` script does not require downtime, you can perform it while the primary system is up and running. Other example is to start any shutdown host in the standby site.
- If required to restart the application servers, stop and start the managed servers / mid tiers in parallel.
- The front-end update in DNS is customer dependent. Use a low TTL value in the appropriate DNS entry (at least during the switchover operation) to reduce the time for update. Once the switchover finished, the TTL can be reverted to its original value.
- The OCI load balancer takes some time also to realize that the servers are up and to start sending requests to them. It is usually some seconds, depending on the frequency of the OCI load balancer health checks. Lower the interval used for the checks is, faster it realizes that the servers are up. However, be cautious when you use too low intervals: if the health check is a heavy check, it could overload the back end.

More Information About Full Site Switchover

The previous topics describe full site failover in a generic fashion. For detailed information for full site failover for specific applications refer to the following sources:

- [SOA Suite on Oracle Cloud Infrastructure Marketplace Disaster Recovery](#)
- [Oracle WebLogic Server for Oracle Cloud Infrastructure Disaster Recovery](#)
- [Full Stack Disaster Recovery](#)
- [Oracle Cloud Infrastructure Full Stack Disaster Recovery](#)

Part XI

Planned Maintenance

.

Software Maintenance Best Practices

Oracle Maximum Availability Architecture (MAA) provides a comprehensive, proven framework to maintain Oracle software, designed to minimize disruption while safeguarding business operations.

Software Security Recommendations

Software Security Recommendations

Oracle environments can be high-value targets because they often support mission-critical business processes and may include broad database, middleware, application, and infrastructure footprints. In addition to maintaining strong security practices, such as network isolation, least-privilege access controls, secure configuration, and regular validation, timely software maintenance is a core security control and an MAA best practice.

Patch management is critical because many incidents exploit vulnerabilities for which fixes are already available. To help protect Oracle Database environments, Oracle recommends that you:

- **Oracle Database and Grid Infrastructure Software**
 - **Upgrade to a Long Term Support Database release**, such as Oracle Database 19c or Oracle AI Database 26ai.
 - **Apply the latest quarterly Database Release Updates (RUs)** for Oracle Database, Oracle Grid Infrastructure, and related software components. RUs contain the latest critical fixes, security fixes, and reliability improvements.
 - **Apply the latest monthly Database Critical Security Patch Update (CSPU)** when available and applicable to your platform and release. Monthly Critical Security Patch Updates (CSPUs) begin May 2026 and provide targeted fixes for critical vulnerabilities in a smaller, focused format, enabling customers to address high-priority issues without waiting for the next quarterly release update. Database and Grid Infrastructure on Linux x86-64 receive CSPU content in the existing Monthly Recommended Patch (MRP) update.
 - **Update client libraries and drivers** to supported Long Term Support release versions, such as 19c or 26ai.
- **Exadata System Software**
 - **Update to an Exadata Release still receiving monthly maintenance releases.**
 - **Apply the latest monthly Exadata System Software maintenance release** when available.

For more information see My Oracle Support article [PNEWS3015 Recommendations to Help Protect Oracle Databases from Emerging AI-enabled Security Threats](#).

For the latest software recommendations, use Oracle Update Advisor (OUA) or reference the Oracle Support knowledge articles applicable to your environment.

OUA evaluates and reports Oracle Database and Grid Infrastructure software health, and provides recommendations aligned with Oracle update policies and software images matching the recommendation. Many Oracle software maintenance tools integrate with OUA to deliver software health and recommendation. See Software Maintenance Tools and Capabilities section below. In addition, OUA can be integrated into existing custom software maintenance orchestration. See Oracle Update Advisor (OUA) API Reference and Integration Guide [KB886700](#).

The following table summarizes Oracle's general recommendation:

Software Component	General Recommendation	Documentation References
Oracle Database software	• Use release 19c or 26ai	Primary Note for Database Quarterly Release Updates KB106822
Oracle Grid Infrastructure software	• Apply the latest RU • Apply the latest MRP or CSPU • Use OUA to evaluate software health and provide recommendations	Exadata Database Machine and Exadata Storage Server Supported Versions KB153930 Exadata Database Service Software Versions KB114881
Exadata System Software	• Use an Exadata release receiving monthly updates • Apply the latest monthly Exadata maintenance release	Exadata Database Machine and Exadata Storage Server Supported Versions KB153930 Exadata Database Service Software Versions KB114881

Staying current with upgrades and patches reduces risk and helps maintain a secure, supportable environment over time. Oracle provides different levels of automation and assistance depending on the deployment model:

- **Oracle-managed services:** Oracle Autonomous Database services are automatically updated to current software levels by Oracle.
- **Co-managed cloud services:** Exadata Database Service and Oracle Database services in supported multicloud environments provide automation to help customers apply updates across their database fleets.
- **Customer-managed environments:** For on-premises and highly integrated environments, Oracle provides resources through [Oracle Platinum Services](#) for engineered systems, [My Oracle Support](#), and [Oracle Customer Success Services](#) to assist with planning, testing, and execution.

Software Maintenance Tools and Capabilities

Oracle continues to evolve software update tools and capabilities based on customer requirements. The primary tools used to evaluate, obtain, and apply Oracle software updates are summarized here.

Software Maintenance Capabilities

Definitions

Rolling orchestration to maximize database uptime— Coordinating software maintenance, typically offline patching, across one node or server at a time. For Database, Grid, and Exadata database servers rolling orchestration includes connection draining before each node is taken offline, so clustered database services remain available for the duration of the orchestration. For Exadata storage servers rolling orchestration includes handling storage offline and online actions while database availability remains unaffected.

Non-rolling orchestration to minimize maintenance time- Coordinating software maintenance, typically offline patching, across all nodes or servers at the same time to minimize maintenance time, requiring clustered database and/or storage service downtime for the duration of the orchestration.

Data Guard orchestration — Coordinating maintenance across primary and standby databases in a Data Guard configuration, often using standby-first patching.

Fleet orchestration — Managing and sequencing software maintenance actions across many Oracle databases, homes, clusters, or servers as a controlled fleet-level operation.

Database online hot patching — Applying online-eligible patches to an Oracle database instance while it remains running. See MOS KB140524.

Grid Zero-Downtime patching - Applying Grid home patches without bringing down Oracle RAC database instances.

Exadata Live Update - An Exadata update method that applies supported database server software updates online, using technologies such as RPM and Ksplice, to reduce or avoid database instance interruption and server reboots.

Upgrade — Moving from one major release to a newer major release.

Oracle Update Advisor integration — Integration with Oracle Update Advisor capabilities to provide software health, recommendation, and gold image based on Oracle's official recommendation.

Vertical stack update — Coordinated updating of multiple dependent layers in the Oracle technology stack, such as Grid Infrastructure software, Database software, and Exadata System Software, one node at a time across a cluster to minimize application disruption.

Database Home Software

Product	Database patching	Database online hot patching	Database rolling orchestration	Database non-rolling orchestration	Data Guard orchestration	Fleet orchestration	Database upgrade	Oracle Update Advisor integration	Vertical stack update
OPatch/ OPatchAuto (in place)	Yes	Yes	No	No	No	No	No	No	No
Database Configuration Assistant (DBCA/DBCACTL)	Yes	No	Yes	Yes	No	No	No	Yes	No
Fleet Patching and Provisioning (FPP)	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
AutoUpgrade	Yes	No	Yes	Yes	No	No	Yes	Yes	No

Product	Database patching	Database online hot patching	Database rolling orchestration	Database non-rolling orchestration	Data Guard orchestration	Fleet orchestration	Database upgrade	Oracle Update Advisor integration	Vertical stack update
Enterprise Manager (EM) with Database Lifecycle Management (DBLM)	Yes	No	Yes	Yes	Yes	Yes	Yes	No	No
Cloud Console Automation	Yes	No	Yes	No	No	No	Yes	No	No
Cloud Exadata Fleet Update	Yes	No	Yes	Yes	No	Yes	Yes	No	No

Grid Infrastructure Home Software

Product	Grid patching	Grid Zero-Downtime patching	Grid rolling orchestration	Grid non-rolling orchestration	Fleet orchestration	Grid upgrade	Oracle Update Advisor Integration	Vertical Stack Update
OPatch/OPatchAuto (in place)	Yes	No	No	No	No	No	No	No
gridSetup.sh	Yes	Yes	Yes	No	No	Yes	No	No
Fleet Patching and Provisioning (FPP)	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Enterprise Manager (EM) with Database Lifecycle Management (DBLM)	Yes	Yes	Yes	Yes	Yes	Yes	No	No
Cloud Console Automation	Yes	No	Yes	No	No	Yes	No	No
Cloud Exadata Fleet Update	Yes	No	Yes	No	Yes	Yes	No	Yes

Exadata Software

Product	Exadata patching	Exadata Live Update	Exadata rolling orchestration	Exadata non-rolling orchestration	Fleet orchestration	Vertical Stack Update
Patchmgr	Yes	Yes	Yes	Yes	No	No
Fleet Patching and Provisioning (FPP)	Yes	Yes	Yes	Yes	Yes	Yes
Enterprise Manager (EM) with Exadata Management pack	Yes	No	Yes	Yes	Yes	No
Cloud Console Automation	Yes	No	Yes	No	No	No
Cloud Exadata Fleet Update	Yes	No	No	Yes	Yes	Yes

Software Maintenance Tool Recommendations

Oracle Software Update Solution	Primary Use	Documentation/References	Licensing
Fleet Patching and Provisioning (FPP)	Fleet-scale out-of-place, gold image-based provisioning, patching, and upgrade orchestration.	Fleet Patching and Provisioning Documentation Fleet Patching and Provisioning (FPP) Fleet Patching and Provisioning (FPP) – Local Mode	Included with supported Oracle Database/Oracle RAC environments
Enterprise Manager with Database Lifecycle Management	Fleet-wide lifecycle orchestration across hybrid and multicloud environments with audit reporting	Enterprise Manager Lifecycle Management Guide Database Lifecycle Management Exadata Management Real Application Testing	Requires Database Lifecycle Management (DBLM) Pack
AutoUpgrade	Database upgrade and single-instance database patching	Oracle AutoUpgrade Patching	Included with Oracle Database
Database Configuration Assistant (DBCA/DBCACTL)	Single-instance database patching	Using Oracle Update Advisor with DBCA	Included with Oracle Database
OPatch	Database online hot patching	- OPatch RDBMS Online Patching ("Hot Patching") – KB140524	Included with Oracle Database
Patchmgr	Exadata System Software maintenance	Exadata Database Machine Maintenance Guide	Included with Oracle Exadata Systems
Cloud Console Automation	Database, Grid Infrastructure, and Exadata software maintenance for Oracle Cloud Database services.	Refer to the applicable Oracle Cloud Database Service documentation	Included with Oracle Cloud Services

Oracle Software Update Solution	Primary Use	Documentation/References	Licensing
Cloud Exadata Fleet Update	Fleet-scale Database, Grid Infrastructure, and Exadata software maintenance for Oracle Cloud Database services.	Refer to the applicable Oracle Cloud Database Service documentation	Included with Oracle Cloud Services

Patching and Upgrade Guidance and Recommendations

For patching lifecycle automation or Oracle-assisted patching services, Oracle recommends the following approaches.

Oracle Cloud Database Patching and Lifecycle Maintenance

- Oracle Autonomous Database is fully managed by Oracle.
- For co-managed Oracle Database services in Oracle Cloud Infrastructure or supported multicloud environments, Oracle manages the underlying infrastructure, and customers manage the VM guest and customer-controlled software with the automation provided by the service. Exadata Fleet Update provides fleet-level orchestration for Exadata Database Service updates.

Oracle Platinum Services

Oracle Platinum Services provides patching assistance for eligible on-premises engineered systems. It supports Exadata, Zero Data Loss Recovery Appliance, Oracle Grid Infrastructure, and Oracle Database software for certified configurations under applicable support terms. See [Oracle Platinum Services](#)

Oracle Customer Success Services

Oracle Customer Success Services can help customers manage and maintain Oracle databases, applications, and cloud environments through services for patching, upgrades, monitoring, lifecycle management, and operational execution. See [Oracle Customer Success Services](#)

Customer-Managed Patching with Oracle Tools

For customer-managed patching with Oracle tools, Oracle recommends:

- **Single-instance database deployments:** Use out-of-place, gold image-based updates with DBCA, AutoUpgrade patching, or Enterprise Manager Fleet Maintenance.
- **Advanced Oracle Database deployments:** Use FPP or Enterprise Manager Fleet Maintenance with out-of-place, gold image-based updates for Oracle RAC, Oracle Data Guard, Exadata, and large database fleets.
- **Database online hot patching:** Use OPatch, or OPatch integrated with FPP, only for supported online patch types and scenarios.
- **Database upgrades:** Use Oracle AutoUpgrade for database upgrade.

In all cases, prefer repeatable, automated, image-based workflows over manual, one-off procedures. This aligns with MAA principles by reducing configuration drift, enabling consistent validation, and preserving a clear fallback path.

Software Maintenance Lifecycle

Regular software maintenance is essential to:

- Strengthen software security and address newly discovered vulnerabilities
- Improve stability and reliability through tested and supported releases
- Maintain compatibility with newer database, infrastructure, and operating system components
- Ensure access to the most effective Oracle Support diagnostic and resolution tools
- Remain eligible for the latest fixes and enhancements

The rest of this chapter describes MAA software maintenance best practices for:

- Oracle Database
- Oracle Grid Infrastructure
- Oracle Exadata Database Machine

The guidelines cover the maintenance lifecycle, including detecting when updates are needed, selecting the recommended target version, obtaining the required software, and applying updates with minimal service disruption.

Software maintenance consists of three high-level phases:

1. [Detect When Software Needs To Be Updated](#)
2. [Obtain Recommended Software](#)
3. [Apply Software Updates](#)

By following these MAA software maintenance best practices, you can keep database environments secure, resilient, highly available, and supportable while minimizing disruption to business operations.

Detect When Software Needs To Be Updated

Defining a software maintenance policy and proactively detecting outdated software are critical for security, stability, availability, and compliance. Oracle recommends automating detection and alerting by using Oracle tools and selected My Oracle Support resources.

Key Practices

- Define a software maintenance policy that aligns with business requirements for compliance, risk tolerance, and update frequency.
- Use Oracle tools to evaluate software health and determine whether maintenance is required or recommended.
- Automate health checks with DBCA, DBCACTL, FPP, or other supported orchestration integrated with Oracle Update Advisor to produce consistent, policy-aware recommendations.
- Establish notification thresholds for critical and important advisories that require interim action.
- Monitor feature-specific recommendations for components that are critical to the business, such as Oracle Data Guard.

Define a Software Maintenance Policy

Establish a documented software maintenance policy that reflects compliance requirements, risk tolerance, service level agreements, and maintenance-window constraints. Review the policy whenever business or operational requirements change.

A comprehensive policy should define the following directives:

- **Apply frequency:** How often proactive software updates are applied, such as monthly, quarterly, or semiannually. Oracle typically releases proactive updates monthly or quarterly. Oracle recommends updating to the latest RU quarterly and applying monthly security or recommended updates when applicable.
- **Update lag:** Whether to apply the most recent proactive update (N) or a previous update, such as N-1. Oracle recommends using the latest RU as a security best practice. If N-1 is used to reduce early-adoption risk, combine it with MRPs or CSPUs.
- **Notification Level:** The severity of newly published issues that trigger applying interim fixes before the next proactive update. Levels include Critical and Important. The choice depends on business risk tolerance and operational priority.
 - **Critical:** Address only critical issues immediately. Examples are listed in [Exadata Critical Issues \(KB623062\)](#)
 - **Important:** Address both critical and important issues immediately. Examples for Oracle Database are listed in [Oracle Database 19c and Oracle AI Database 26ai Important Recommended One-off Patches \(KB188772\)](#)
- **Recommendation Area:** The specific Oracle product components or features for which software recommendations will be monitored. Identify recommendation areas that are critical to business success.

A maintenance policy should consider the CVSS level of newly announced security vulnerabilities when determining how security requirements affect software apply frequency and target versions. Higher CVSS levels, especially Critical and High vulnerabilities, may require accelerated adoption of a newer RU, MRP, CSPU, or interim fix instead of waiting for the next scheduled maintenance cycle.

Security Target	CVSS Range
SecurityCritical	CVSS 9.0 - 10.0
SecurityHigh+	CVSS 7.0 - 10.0 (Default)
SecurityMedium+	CVSS 4.0 - 10.0
SecurityLow+	CVSS 0.1 - 10.0

Oracle Database and Grid Infrastructure Software

Oracle Database and Oracle Grid Infrastructure require a defined, repeatable software maintenance policy that balances security, stability, availability, and change rate. The policy should specify how often proactive updates are applied, how far behind the latest recommended update the environment is allowed to operate, and what severity of missing fixes triggers interim action.

Oracle recommends using out-of-place, gold image-based updates orchestrated by DBCA for single-instance systems and FPP for clustered environments, with Oracle Update Advisor supplying policy-aware recommendations and ready-to-deploy images. This approach standardizes software baselines, reduces risk and downtime, and simplifies rollback, and aligns with MAA high availability principles.

Policy Scenarios

The following tables show examples of software maintenance goals and the policy directives used to achieve them.

Apply Frequency	Description
Quarterly	Update quarterly
Semiannually	Update twice per year
Monthly	Update monthly
Monthly Long Term RU	Update monthly staying on a Long Term RU

Update Lag	Description
N	Install latest RU
N-1	Install prior (N-1) RU
N-2	Install prior (N-2) RU

Apply Frequency	Update Lag N	Update Lag N-1	Update Lag N-2
Quarterly	Supported	Supported	Supported
Semiannually	Supported	Supported	Supported
Monthly	Supported	Supported	Not supported ¹
Monthly Long Term RU	Supported	Supported	Not supported ²

¹Not supported because monthly updates extend to only one prior RU.

²Not supported because monthly updates extend to only one prior Long Term RU.

Policy Examples

The following policy examples cover the most common customer goals.

- Policy Example 1 : Apply Frequency = Quarterly, Update Lag = N
- Policy Example 2 : Apply Frequency = Quarterly, Update Lag = N-1
- Policy Example 3 : Apply Frequency = Monthly, Update Lag = N-1
- Policy Example 4 : Apply Frequency = Monthly Long Term RU, Update Lag = N-1

Policy Example 1 : Apply Frequency = Quarterly, Update Lag = N

This is the default and most common policy. Adopt the latest RU each quarter as it is released, operate on that RU for approximately three months, and advance to the next RU at the next quarterly release. This keeps security and stability fixes current with no intentional lag.

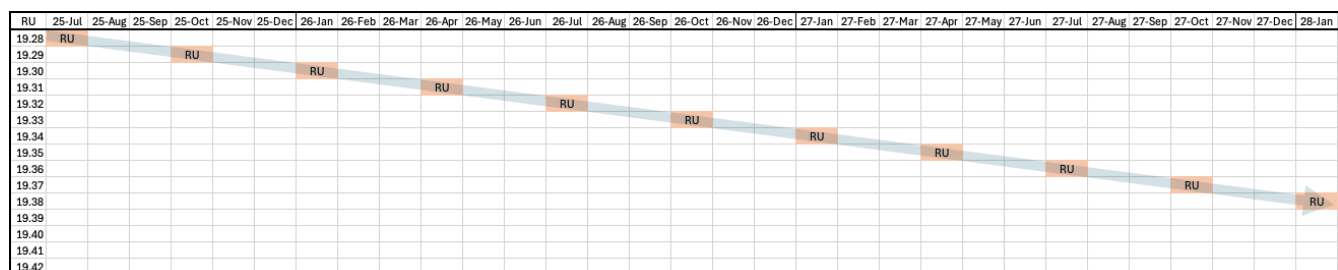
Policy summary:

- Stay on the latest RU for three months.
- When a new RU is released every quarter (January, April, July, and October), install the new RU.

Date	Released RU	Installed RU
25-Jul	19.28	19.28
25-Aug		19.28
25-Sep		19.28

Date	Released RU	Installed RU
25-Oct	19.29	19.28 19.29
25-Nov		19.29
25-Dec		19.29
26-Jan	19.30	19.29 19.30
26-Feb		19.30
26-Mar		19.30
26-Apr	19.31	19.30 19.31
26-May		19.31
26-Jun		19.31
26-Jul	19.32	19.31 19.32

The following graph for policy example 1 shows Oracle Database 19c quarterly RU releases and their release month over 28 months. It depicts the quarterly, no-lag policy, where you adopt each newly released RU that quarter, operate on it for about three months, and then advance to the next RU on release.



Policy Example 2 : Apply Frequency = Quarterly, Update Lag = N-1

Choose this policy when a conservative cadence is required. Stay one RU behind the latest by installing the previous quarter's RU each cycle, operating on that RU for approximately three months before advancing to the next previous RU. This reduces early-adoption risk while maintaining a predictable quarterly cadence.

Policy summary::

- Stay on the N-1 RU for three months.
- When a new RU is released every quarter (January, April, July, and October), install the next N-1 RU.
- Monitor critical and important fixes continuously; accelerate patching if Oracle publishes a fix that materially affects your risk profile.

Date	Released RU	Installed RU
25-Jul	19.28	19.27
25-Aug		19.27
25-Sep		19.27
25-Oct	19.29	19.27 19.28
25-Nov		19.28
25-Dec		19.28

Date	Released RU	Installed RU
26-Jan	19.30	19.28 19.29
26-Feb		19.29
26-Mar		19.29
26-Apr	19.31	19.29 19.30
26-May		19.30
26-Jun		19.30
26-Jul	19.32	19.30 19.31

Policy Example 3 : Apply Frequency = Monthly, Update Lag = N-1

This policy reduces early-adoption risk by staying one RU behind while still applying the latest monthly recommended patches (MRPs) for that RU. Operate on a given RU for approximately one quarter, and apply the current MRP for that RU each month. When a new quarterly RU is released, move to the prior RU for the next quarter and continue monthly patching on that RU.

Policy summary:

- Stay on an RU for three months.
- Install the latest MRP for the prior (N-1) RU every month.
- When a new RU is released each quarter (January, April, July, and October), install the prior (N-1) RU and the latest MRP for that RU.

MRP details:

- MRPs are cumulative monthly bundles applied on top of a specific RU and do not change the RU version. MRPs apply only to Linux x86-64.
- MRP labels in the tables (for example, MRP3, MRP4) denote the third, fourth, and later monthly cumulative bundles after the RU release.

Date	Released RU	Installed RU
25-Oct	19.29	19.28 + MRP3
25-Nov		19.28 + MRP3 19.28 + MRP4
25-Dec		19.28 + MRP4 19.28 + MRP5
26-Jan	19.30	19.28 + MRP5 19.29 + MRP3
26-Feb		19.29 + MRP3 19.29 + MRP4
26-Mar		19.29 + MRP4 19.29 + MRP5
26-Apr	19.31	19.29 + MRP5 19.30 + MRP3
26-May		19.30 + MRP3 19.30 + MRP4
26-Jun		19.30 + MRP4 19.30 + MRP5
26-Jul	19.32	19.30 + MRP5 19.31 + MRP3

The following graph for policy example 3 shows Oracle Database 19c quarterly RU and monthly MRP releases and their release month over 28 months. It depicts the monthly, N-1 policy, where you stay one RU behind, apply the latest monthly MRP to that RU each month, and shift to the next N-1 RU at the following quarterly release.

RU	25-Oct	25-Nov	25-Dec	26-Jan	26-Feb	26-Mar	26-Apr	26-May	26-Jun	26-Jul	26-Aug	26-Sep	26-Oct	26-Nov	26-Dec	27-Jan	27-Feb	27-Mar	27-Apr	27-May	27-Jun	27-Jul	27-Aug	27-Sep	27-Oct	27-Nov	27-Dec	28-Jan
19.28	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18												
19.29	RU	1	2	3	4	5	6																					
19.30				RU	1	2	3	4	5	6																		
19.31							RU	1	2	3	4	5	6															
19.32										RU	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
19.33													RU	1	2	3	4	5	6									
19.34																RU	1	2	3	4	5	6						
19.35																			RU	1	2	3	4	5	6			
19.36																						RU	1	2	3	4	5	6
19.37																									RU	1	2	3
19.38																												RU
19.39																												
19.40																												
19.41																												
19.42																												

Policy Example 4 : Apply Frequency = Monthly Long Term RU, Update Lag = N-1

This policy targets customers who prefer a long-lived RU foundation (Long Term RU) while still consuming monthly MRPs. Stay on a designated Long Term RU for approximately 12 months and apply the latest MRP each month. When the next Long Term RU becomes available (annually), move forward to the prior (N-1) Long Term RU during your annual change window, then continue monthly MRPs on that new Long Term RU.

For additional details see [Long Term Support Model for 19c Database Release Updates \(RU\) \(KB625385\)](#)

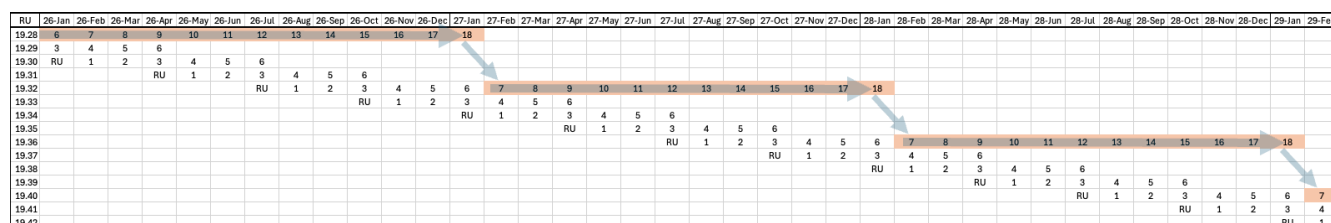
Policy summary:

- Stay on an Long Term RU for 12 months.
- Install the latest MRP for the prior (N-1) Long Term RU every month.
- Install the prior (N-1) Long Term RU and the latest MRP for that RU every year during your annual change window.

Date	Released RU	Installed RU
26-Jan	19.30	19.28 + MRP5 19.28 + MRP6
26-Feb		19.28 + MRP6 19.28 + MRP7
26-Mar		19.28 + MRP7 19.28 + MRP8
26-Apr	19.31	19.28 + MRP8 19.28 + MRP9
26-May		19.28 + MRP9 19.28 + MRP10
26-Jun		19.28 + MRP10 19.28 + MRP11
26-Jul	19.32 (Long Term RU)	19.28 + MRP11 19.28 + MRP12
26-Aug		19.28 + MRP12 19.28 + MRP13
26-Sep		19.28 + MRP13 19.28 + MRP14
26-Oct	19.33	19.28 + MRP14 19.28 + MRP15
26-Nov		19.28 + MRP15 19.28 + MRP16
26-Dec		19.28 + MRP16 19.28 + MRP17
27-Jan	19.34	19.28 + MRP17 19.28 + MRP18
27-Feb		19.28 + MRP18 19.32 + MRP7
27-Mar		19.32 + MRP7 19.32 + MRP8
27-Apr	19.35	19.32 + MRP8 19.32 + MRP9
27-May		19.32 + MRP9 19.32 + MRP10
27-Jun		19.32 + MRP10 19.32 + MRP11
27-Jul	19.36 (Long Term RU)	19.32 + MRP11 19.32 + MRP12
27-Aug		19.32 + MRP12 19.32 + MRP13
27-Sep		19.32 + MRP13 19.32 + MRP14

Date	Released RU	Installed RU
27-Oct	19.37	19.32 + MRP14 19.32 + MRP15
27-Nov		19.32 + MRP15 19.32 + MRP16
27-Dec		19.32 + MRP16 19.32 + MRP17
28-Jan	19.38	19.32 + MRP17 19.32 + MRP18
28-Feb		19.32 + MRP18 19.36 + MRP7

The following graph for policy example 4 shows Oracle Database 19c quarterly RU and monthly MRP releases and their release month over 38 months. It depicts the monthly Long Term RU, N-1 policy, where you run on a designated Long Term RU for about a year with monthly MRPs, then move annually to the prior Long Term RU and continue monthly MRPs.



Determine Software Health

After a software maintenance policy is established, evaluate installed software against that policy and Oracle's current recommendations to determine software health.

Evaluate whether:

- Installed software is behind Oracle's recommended proactive updates.
- Critical or important fixes announced by Oracle are missing for the base product software.
- Critical or important fixes announced by Oracle are missing for specific product components or features, such as Oracle Data Guard.

Evaluate software health frequently, such as weekly, and more frequently than the planned apply frequency. This helps identify newly recommended critical or important fixes shortly after they are published.

Oracle Database and Grid Infrastructure Software

Oracle recommends using automated tools integrated with Oracle Update Advisor (OUA) to assess software health for Oracle Database and Oracle Grid Infrastructure. This approach reduces manual tracking and improves consistency.

Automated tools such as DBCA/DBCACTL, and FPP can integrate with Oracle Update Advisor to evaluate installed software against the defined policy and Oracle's current recommendation to determine software health status. Customers can also integrate Oracle Update Advisor with their own custom software maintenance orchestration by using the Oracle Update Advisor API, as described in My Oracle Support article [KB886700](#). Using automation simplifies the identification of missing critical or important fixes and helps drive consistent, policy-aware maintenance decisions.

Software Health Status

Status	Action
GREEN	No action needed Installed software meets Oracle's recommendation based on your policy.
YELLOW	Update is recommended Installed software is missing important fixes or is behind Oracle's recommendation by one cycle.
RED	Update is required Installed software is missing critical fixes or is behind Oracle's recommendation by more than one cycle.

Examples

In the following examples:

- Installed software is Database Release Update 23.8. Database Release Update 23.9 is the latest version released and recommended. Therefore, the installed RU is one behind the latest RU.
- When Policy is not specified the default is used (Apply frequency = Quarterly, Update lag = No lag).

DBCA Example using default policy

```
$ dbca -managePatches -checkPatchStatus -silent
  "softwareHealth" : "YELLOW",
  "comment" : "One cycle behind your policy.",
  "recommendedVersion" : "23.9"
```

FPP Example (using FPP Server Mode) using default policy

```
$ rhpctl evaluate patch -image DB238000
Software health status: "YELLOW"
Software health comment: "One cycle behind your policy."
Software Type: "DB"
Current version: "23.8.0.25.4"
Recommended version: "23.9"
```

DBCACTL Example using semiannual apply frequency

```
$ dbcactl -managePatches -checkPatchStatus -applyFrequency SEMI_ANNUAL
  "softwareHealth" : "GREEN"
```

For additional details see Streamlining Your Update Experience with Oracle Update Advisor.

Alternative/Complementary Methods

If automated evaluation tools are not in use, administrators can track recommendations and health status manually by using:

- Exachk – Run Exachk regularly and review recommended software checks.

- My Oracle Support knowledge base articles - Proactively monitor prioritized KB articles for proactive software update release announcements, critical fixes and recommended fixes, and replacement updates.

For additional details see [Obtain Recommended Software](#).

Exadata Software

For Exadata environments, Oracle recommends:

- Exachk – Run Exachk and review recommended software checks.
- My Oracle Support knowledge base articles - Proactively monitor prioritized articles for proactive software update release announcements, critical and recommended fixes, or replacement fixes.

For additional details see [Obtain Recommended Software](#).

Obtain Recommended Software

Selecting and deploying the correct Oracle software versions and patches is essential for availability, security, and support. Use Oracle's automated tooling or validated My Oracle Support resources to determine the right targets and to obtain secure, policy-compliant software.

Key Practices

- Use automated tools (FPP, AutoUpgrade, or DBCA/DBCACTL with Oracle Update Advisor integration) wherever possible to minimize manual errors.
- For environments where automation is not feasible, carefully reference Exachk output and relevant KB articles.
- Always verify downloaded artifacts using Oracle's provided digital signatures and checksums.

Database and Grid Infrastructure, On-Premises

Recommended Approach – DBCA/DBCACTL, AutoUpgrade, or FPP with Oracle Update Advisor

Obtain recommended software as a Gold Image with Oracle Update Advisor functionality. Use Gold Images to perform updates in the recommended out-of-place, image-based manner.

Recommended workflow:

1. Generate a gold image that contains the recommended software matching your policy and required patches.
2. Download the recommended gold image.
3. Validate the image in a non-production environment.
4. Promote the validated image through the maintenance lifecycle.

Gold images are automatically tailored:

- Include Oracle's recommended RU and any required interim or critical patches.
- Resolve patch conflicts.

- Retain or add customer-specific patches when specified and supported.

DBCA Example

```
$ dbca -managePatches -generateGoldImage -silent
$ dbca -managePatches -downloadGoldImage -goldImageDownloadLocation /u01/
swdepot -silent
```

FPP Example using FPP Server Mode

```
$ rhpctl evaluate patch -image DB238000 -generateimage -importtoimage DB239000
```

Feature-Specific Database Recommendations (Manual Process Only)

Oracle Update Advisor may not provide all feature-specific recommendations for every database feature. For additional features such as Data Pump, review the applicable My Oracle Support knowledge base articles to identify recommended interim patches or feature-specific fixes.

If you are using manual methods for core database software:

An alternative approach is to review relevant KB articles, run Exachk, and download either gold images or recommended interim patches.

- Review relevant Knowledge Base notes and identify recommended software.
 - [Primary Note for Database Quarterly Release Updates \(KB888\)](#)
 - [Exadata Critical Issues \(KB623062\)](#)
 - [Oracle Database 19c and Oracle AI Database 26ai Important Recommended One-off Patches \(KB188772\)](#)
 - [Oracle Database End of Premier Support For Major Releases - Frequently Asked Questions \(FAQ2181\)](#)
- Run Exachk and review recommended software checks.
 - Review MAA Scorecard section for Recommended Release and Release Update (RU)
 - Review Critical issues section for Recommended critical issue updates
 - Review Important bug fixes check for Recommended important issue updates
- Adjust recommendation to adhere to your software maintenance policy.
- Follow [Platinum Process for Gold Image based In Place and Out of Place Grid - Oracle home Patching \(KB324119\)](#) to request and download gold images for Database and Grid Infrastructure software homes, supplying your inventory and the recommended software manually identified above as candidate patches.

Exadata (On-Premises)

Review relevant My Oracle Support knowledge base articles, run Exachk, and download recommended updates.

- Review relevant KB articles and identify recommended software.
 - Review [Oracle Exadata Database Machine Operations Best Practices \(KB214168\)](#) for Recommended Exadata software
 - [Exadata Critical Issues \(KB623062\)](#)

- Review [Oracle Exadata: Exadata and Linux Important Recommended Fixes \(KB403624\)](#) for Recommended important issue updates
- Run Exachk and review recommended software checks.
 - Review MAA Scorecard section for Recommended Release and Release Update (RU)
 - Review Critical issues section for Recommended critical issue patches
 - Review Important bug fixes check for Recommended important issue patches
- Adjust recommendation to adhere to your software maintenance policy.
- Follow [Oracle Exadata Database Machine Operations Best Practices \(KB214168\)](#) to download recommended updates for Exadata compute nodes, storage servers, and switches directly.

Apply Software Updates

Well-planned, tested, and properly executed software updates are crucial to maximizing uptime and minimizing business disruption. Wherever possible, Oracle recommends out-of-place, gold image-based patching—especially for clustered systems—to ensure safe rollbacks and rapid issue resolution.

Key Practices

- Before updating software run Exachk/Orachk and correct configuration that does not follow recommended best practices.
- Follow client failover best practices to reduce application-level impact during rolling updates that disrupt database instance connectivity. See [Continuous Availability for Applications](#) for details.
- Test the update and rollback process in a non-production environment with production-like workload and configuration.
- Use Data Guard Standby-First Patch Apply approach to minimize risk and primary database impact when updating mission critical systems. See [Oracle Patch Assurance - Data Guard Standby-First Patch Apply \(KB137118\)](#) for details.
- Take system and database backups immediately before software changes.
- Always validate, backup, and have a proven rollback plan before making software changes.
- Use repeatable, automated, image-based workflows provided by Oracle software maintenance tools to ensure consistency and reduce manual errors.
- Use DBCA/DBCACTL or AutoUpgrade for out-of-place, gold image-based Database updates on standalone/non-clustered systems.
- Use FPP or EM for coordinated, out-of-place, rolling, gold image-based Database and Grid Infrastructure updates on clustered environments such as RAC and Exadata.
- Apply online/in-place updates only for specific patch types or situations where minimal disruption is required. See MOS note [RDBMS Online Patching Aka Hot Patching \(KB140524\)](#) for details.

Software Update Method Selection: Benefits and Tradeoffs

Table 45-1 Database Software on Non-Clustered Systems (Standalone Hosts)

Method	Benefits/Tradeoffs
Out-of-place (gold image) with DBCA/DBCACTL or AutoUpgrade (Recommended)	- Fast, simple, easy rollback - Requires extra disk space <pre>\$ dbca -moveDatabase -sourceDB <dbname> -silent</pre>
Online hot patch with OPatch	- No database downtime or database connection disruption - Patching is done in-place in the existing database home - Limited to supported patches (MRP/interim)

Table 45-2 Database Software on Clustered Systems (Oracle RAC/Exadata)

Method	Benefits/Tradeoffs
Out-of-place (gold image), RAC rolling with FPP or EM (Recommended)	- Fast, simple, full cluster orchestration, easy rollback - Requires extra disk space <pre>\$ rhpctl move database -sourcewc <src> -patchedwc <dest></pre>
Online hot patch with FPP	- No database downtime or database connection disruption - Patching is done in-place in the existing database home - Limited to supported patches (MRP/interim)
Non-rolling with FPP	- All updates applied at once, reduced planned maintenance time - Full database downtime

Table 45-3 Grid Infrastructure Software on Clustered Systems (Oracle RAC/Exadata)

Method	Benefits/Tradeoffs
Out-of-place (gold image), Oracle RAC rolling with FPP (Recommended)	- Fast, simple, full cluster orchestration, easy rollback - Requires extra disk space <pre>\$ rhpctl move gihome -sourcewc <src> -destwc <dest></pre>
Zero-downtime out-of-place (gold image), Oracle RAC rolling with FPP	- No database connection disruption - Requires extra disk space <pre>\$ rhpctl move gihome -sourcewc <src> -destwc <dest> -tgip</pre>

Table 45-4 Exadata System Software

Method	Benefits/Tradeoffs
Rolling update with patchmgr or FPP (Recommended) - Storage servers - Database servers - Network fabric switches	No database downtime, simple, full system orchestration <pre>\$ patchmgr --dbnodes <group_file> --repo <repo> --upgrade --target_version <version> -- rolling</pre>
Live update with patchmgr or FPP Database servers	- No database connection disruption, supports partial updates to address security issues - Outstanding work update items cannot be completed online and require a system reboot <pre>\$ patchmgr --dbnodes <group_file> --repo <repo> --upgrade --target_version <version> -- rolling --live-update-target full</pre>
Non-rolling with patchmgr or FPP - Storage servers - Database servers - Network fabric switches	- All updates applied at once, reduced planned maintenance time - Full system downtime